

# ***TMS320x2806x Microcontrollers***

## ***Technical Reference Manual***



Literature Number: SPRUH18I  
JANUARY 2011 – REVISED JUNE 2022



|                                                                      |     |
|----------------------------------------------------------------------|-----|
| <b>Read This First</b>                                               | 37  |
| About This Manual                                                    | 37  |
| Notational Conventions                                               | 37  |
| Glossary                                                             | 37  |
| Related Documentation From Texas Instruments                         | 37  |
| Support Resources                                                    | 37  |
| Trademarks                                                           | 37  |
| <b>1 System Control and Interrupts</b>                               | 39  |
| 1.1 Flash and OTP Memory Blocks                                      | 40  |
| 1.1.1 Flash Memory                                                   | 40  |
| 1.1.2 OTP Memory                                                     | 40  |
| 1.1.3 Flash and OTP Power Modes                                      | 41  |
| 1.1.4 Flash and OTP Registers                                        | 46  |
| 1.2 Code Security Module (CSM)                                       | 52  |
| 1.2.1 Functional Description                                         | 52  |
| 1.2.2 CSM Impact on Other On-Chip Resources                          | 55  |
| 1.2.3 Incorporating Code Security in User Applications               | 55  |
| 1.2.4 Do's and Don'ts to Protect Security Logic                      | 60  |
| 1.2.5 CSM Features - Summary                                         | 60  |
| 1.2.6 CSM Status and Control Register                                | 61  |
| 1.3 Clocking                                                         | 62  |
| 1.3.1 Clocking and System Control                                    | 62  |
| 1.3.2 OSC and PLL Block                                              | 69  |
| 1.3.3 Low-Power Modes Block                                          | 93  |
| 1.3.4 CPU Watchdog Block                                             | 96  |
| 1.3.5 32-Bit CPU Timers 0/1/2                                        | 102 |
| 1.4 General-Purpose Input/Output (GPIO)                              | 107 |
| 1.4.1 GPIO Module Overview                                           | 107 |
| 1.4.2 Configuration Overview                                         | 113 |
| 1.4.3 Digital General Purpose I/O Control                            | 115 |
| 1.4.4 Input Qualification                                            | 116 |
| 1.4.5 GPIO and Peripheral Multiplexing (MUX)                         | 120 |
| 1.4.6 Register Bit Definitions                                       | 124 |
| 1.5 Peripheral Frames                                                | 157 |
| 1.5.1 Peripheral Frame Registers                                     | 157 |
| 1.5.2 EALLOW-Protected Registers                                     | 159 |
| 1.5.3 Device Emulation Registers                                     | 163 |
| 1.5.4 Write-Followed-by-Read Protection                              | 166 |
| 1.6 Peripheral Interrupt Expansion (PIE)                             | 167 |
| 1.6.1 Overview of the PIE Controller                                 | 167 |
| 1.6.2 Vector Table Mapping                                           | 170 |
| 1.6.3 Interrupt Sources                                              | 172 |
| 1.6.4 PIE Configuration Registers                                    | 181 |
| 1.6.5 PIE Interrupt Registers                                        | 182 |
| 1.6.6 External Interrupt Control Registers                           | 192 |
| 1.7 VREG/BOR/POR                                                     | 193 |
| 1.7.1 On-chip Voltage Regulator (VREG)                               | 193 |
| 1.7.2 On-chip Power-On Reset (POR) and Brown-Out Reset (BOR) Circuit | 194 |
| <b>2 Boot ROM</b>                                                    | 195 |
| 2.1 Boot ROM Memory Map                                              | 196 |

|                                                                              |            |
|------------------------------------------------------------------------------|------------|
| 2.1.1 On-Chip Boot ROM Math Tables.....                                      | 198        |
| 2.1.2 On-Chip Boot ROM IQmath Functions.....                                 | 200        |
| 2.1.3 On-Chip Flash API.....                                                 | 200        |
| 2.1.4 CPU Vector Table.....                                                  | 200        |
| 2.2 Bootloader Features.....                                                 | 202        |
| 2.2.1 Bootloader Functional Operation.....                                   | 202        |
| 2.2.2 Bootloader Device Configuration.....                                   | 203        |
| 2.2.3 PLL Multiplier and DIVSEL Selection.....                               | 203        |
| 2.2.4 Watchdog Module.....                                                   | 203        |
| 2.2.5 Taking an ITRAP Interrupt.....                                         | 204        |
| 2.2.6 Internal Pullup Resistors.....                                         | 204        |
| 2.2.7 PIE Configuration.....                                                 | 204        |
| 2.2.8 Reserved Memory.....                                                   | 204        |
| 2.2.9 Bootloader Modes.....                                                  | 205        |
| 2.2.10 Device_Cal.....                                                       | 211        |
| 2.2.11 Bootloader Data Stream Structure.....                                 | 212        |
| 2.2.12 Basic Transfer Procedure.....                                         | 217        |
| 2.2.13 InitBoot Assembly Routine.....                                        | 218        |
| 2.2.14 SelectBootMode Function.....                                          | 218        |
| 2.2.15 CopyData Function.....                                                | 220        |
| 2.2.16 SCI_Boot Function.....                                                | 221        |
| 2.2.17 Parallel_Boot Function (GPIO).....                                    | 223        |
| 2.2.18 SPI_Boot Function.....                                                | 227        |
| 2.2.19 I2C Boot Function.....                                                | 230        |
| 2.2.20 eCAN Boot Function.....                                               | 233        |
| 2.2.21 ExitBoot Assembly Routine.....                                        | 235        |
| 2.3 Building the Boot Table.....                                             | 236        |
| 2.3.1 The C2000 Hex Utility.....                                             | 236        |
| 2.3.2 Example: Preparing a COFF File For eCAN Bootloading.....               | 238        |
| 2.4 Bootloader Code Overview.....                                            | 240        |
| 2.4.1 Boot ROM Version and Checksum Information.....                         | 240        |
| <b>3 Enhanced Pulse Width Modulator (ePWM) Module.....</b>                   | <b>241</b> |
| 3.1 Introduction.....                                                        | 242        |
| 3.1.1 EPWM Related Collateral.....                                           | 242        |
| 3.1.2 Submodule Overview.....                                                | 242        |
| 3.1.3 Register Mapping.....                                                  | 247        |
| 3.2 ePWM Submodules.....                                                     | 249        |
| 3.2.1 Overview.....                                                          | 249        |
| 3.2.2 Time-Base (TB) Submodule.....                                          | 251        |
| 3.2.3 Counter-Compare (CC) Submodule.....                                    | 260        |
| 3.2.4 Action-Qualifier (AQ) Submodule.....                                   | 265        |
| 3.2.5 Dead-Band Generator (DB) Submodule.....                                | 278        |
| 3.2.6 PWM-Chopper (PC) Submodule.....                                        | 282        |
| 3.2.7 Trip-Zone (TZ) Submodule.....                                          | 286        |
| 3.2.8 Event-Trigger (ET) Submodule.....                                      | 292        |
| 3.2.9 Digital Compare (DC) Submodule.....                                    | 297        |
| 3.3 Applications to Power Topologies.....                                    | 302        |
| 3.3.1 Overview of Multiple Modules.....                                      | 302        |
| 3.3.2 Key Configuration Capabilities.....                                    | 303        |
| 3.3.3 Controlling Multiple Buck Converters With Independent Frequencies..... | 304        |
| 3.3.4 Controlling Multiple Buck Converters With Same Frequencies.....        | 307        |
| 3.3.5 Controlling Multiple Half H-Bridge (HHB) Converters.....               | 310        |
| 3.3.6 Controlling Dual 3-Phase Inverters for Motors (ACI and PMSM).....      | 312        |
| 3.3.7 Practical Applications Using Phase Control Between PWM Modules.....    | 316        |
| 3.3.8 Controlling a 3-Phase Interleaved DC/DC Converter.....                 | 318        |
| 3.3.9 Controlling Zero Voltage Switched Full Bridge (ZVSFB) Converter.....   | 321        |
| 3.3.10 Controlling a Peak Current Mode Controlled Buck Module.....           | 324        |
| 3.3.11 Controlling H-Bridge LLC Resonant Converter.....                      | 326        |
| 3.4 Registers.....                                                           | 328        |
| 3.4.1 Time-Base Submodule Registers.....                                     | 328        |
| 3.4.2 Counter-Compare Submodule Registers.....                               | 336        |



|                                                                             |            |
|-----------------------------------------------------------------------------|------------|
| 3.4.3 Action-Qualifier Submodule Registers.....                             | 342        |
| 3.4.4 Dead-Band Submodule Registers.....                                    | 347        |
| 3.4.5 Trip-Zone Submodule Control and Status Registers.....                 | 350        |
| 3.4.6 Event-Trigger Submodule Registers.....                                | 359        |
| 3.4.7 PWM-Chopper Submodule Control Register.....                           | 365        |
| 3.4.8 Digital Compare Submodule Registers.....                              | 367        |
| 3.4.9 Proper Interrupt Initialization Procedure.....                        | 375        |
| <b>4 High-Resolution Pulse Width Modulator (HRPWM).....</b>                 | <b>377</b> |
| 4.1 Introduction.....                                                       | 378        |
| 4.2 Operational Description of HRPWM.....                                   | 379        |
| 4.2.1 Controlling the HRPWM Capabilities.....                               | 380        |
| 4.2.2 Configuring the HRPWM.....                                            | 383        |
| 4.2.3 Principle of Operation.....                                           | 384        |
| 4.2.4 Scale Factor Optimizing Software (SFO).....                           | 392        |
| 4.2.5 HRPWM Examples Using Optimized Assembly Code.....                     | 392        |
| 4.3 SFO Library Software - SFO_TI_Build_V6.lib.....                         | 397        |
| 4.3.1 Scale Factor Optimizer Function - int SFO().....                      | 397        |
| 4.3.2 Software Usage.....                                                   | 398        |
| 4.3.3 SFO Library Version Software Differences.....                         | 400        |
| 4.4 HRPWM Registers.....                                                    | 401        |
| 4.4.1 HRPWM Configuration (HRCNFG) Register.....                            | 401        |
| 4.4.2 High Resolution Micro Step (HRMSTEP) Register.....                    | 402        |
| 4.4.3 High Resolution Period Control (HRPCTL) Register.....                 | 404        |
| <b>5 High-Resolution Capture (HRCAP) Module.....</b>                        | <b>405</b> |
| 5.1 Introduction.....                                                       | 406        |
| 5.1.1 Features.....                                                         | 406        |
| 5.1.2 HRCAP Related Collateral.....                                         | 406        |
| 5.2 Description.....                                                        | 406        |
| 5.3 Operational Details.....                                                | 407        |
| 5.3.1 HRCAP Clocking.....                                                   | 407        |
| 5.3.2 HRCAP Modes of Operation.....                                         | 408        |
| 5.3.3 HRCAP Interrupts.....                                                 | 412        |
| 5.4 HRCAP Calibration Library.....                                          | 413        |
| 5.4.1 HRCAP Calibration Library Functions.....                              | 414        |
| 5.4.2 HRCAP Calibration Library Software Usage.....                         | 419        |
| 5.5 HRCAP Register Descriptions.....                                        | 421        |
| 5.5.1 HRCAP Control Register (HCCTL).....                                   | 422        |
| 5.5.2 HRCAP Interrupt Flag Register (HCIFR).....                            | 423        |
| 5.5.3 HRCAP Interrupt Clear Register (HCICLR).....                          | 424        |
| 5.5.4 HRCAP Interrupt Force Register (HCIFRC).....                          | 425        |
| 5.5.5 HRCAP Counter Register (HCCOUNTER).....                               | 426        |
| 5.5.6 HRCAP Capture Counter On Rising Edge 0 Register (HCCAPCNTRISE0).....  | 427        |
| 5.5.7 HRCAP Capture Counter On Falling Edge 0 Register (HCCAPCNTFALL0)..... | 427        |
| 5.5.8 HRCAP Capture Counter On Rising Edge 1 Register (HCCAPCNTRISE1).....  | 428        |
| 5.5.9 HRCAP Capture Counter On Falling Edge 1 Register (HCCAPCNTFALL1)..... | 428        |
| <b>6 Enhanced Capture (eCAP).....</b>                                       | <b>429</b> |
| 6.1 Introduction.....                                                       | 430        |
| 6.1.1 Features.....                                                         | 430        |
| 6.1.2 ECAP Related Collateral.....                                          | 430        |
| 6.2 Description.....                                                        | 430        |
| 6.3 Capture and APWM Operating Mode.....                                    | 432        |
| 6.4 Capture Mode Description.....                                           | 434        |
| 6.4.1 Event Prescaler.....                                                  | 435        |
| 6.4.2 Edge Polarity Select and Qualifier.....                               | 436        |
| 6.4.3 Continuous/One-Shot Control.....                                      | 436        |
| 6.4.4 32-Bit Counter and Phase Control.....                                 | 437        |
| 6.4.5 CAP1-CAP4 Registers.....                                              | 437        |
| 6.4.6 Interrupt Control.....                                                | 438        |
| 6.4.7 DMA Interrupt.....                                                    | 439        |
| 6.4.8 Shadow Load and Lockout Control.....                                  | 440        |
| 6.4.9 APWM Mode Operation.....                                              | 440        |

|                                                                                          |            |
|------------------------------------------------------------------------------------------|------------|
| 6.5 Application of the eCAP Module.....                                                  | 442        |
| 6.5.1 Example 1 - Absolute Time-Stamp Operation Rising Edge Trigger.....                 | 442        |
| 6.5.2 Example 2 - Absolute Time-Stamp Operation Rising and Falling Edge Trigger.....     | 443        |
| 6.5.3 Example 3 - Time Difference (Delta) Operation Rising Edge Trigger.....             | 444        |
| 6.5.4 Example 4 - Time Difference (Delta) Operation Rising and Falling Edge Trigger..... | 445        |
| 6.6 Application of the APWM Mode.....                                                    | 446        |
| 6.6.1 Example 1 - Simple PWM Generation (Independent Channel/s).....                     | 446        |
| 6.7 eCAP Registers.....                                                                  | 446        |
| 6.7.1 ECAP Base Address Table.....                                                       | 446        |
| 6.7.2 ECAP_REGS Registers.....                                                           | 447        |
| <b>7 Enhanced Quadrature Encoder Pulse (eQEP).....</b>                                   | <b>461</b> |
| 7.1 Introduction.....                                                                    | 462        |
| 7.1.1 EQEP Related Collateral.....                                                       | 464        |
| 7.2 Configuring Device Pins.....                                                         | 464        |
| 7.3 Description.....                                                                     | 465        |
| 7.3.1 EQEP Inputs.....                                                                   | 465        |
| 7.3.2 Functional Description.....                                                        | 466        |
| 7.3.3 eQEP Memory Map.....                                                               | 467        |
| 7.4 Quadrature Decoder Unit (QDU).....                                                   | 468        |
| 7.4.1 Position Counter Input Modes.....                                                  | 468        |
| 7.4.2 eQEP Input Polarity Selection.....                                                 | 471        |
| 7.4.3 Position-Compare Sync Output.....                                                  | 471        |
| 7.5 Position Counter and Control Unit (PCCU).....                                        | 471        |
| 7.5.1 Position Counter Operating Modes.....                                              | 471        |
| 7.5.2 Position Counter Latch.....                                                        | 474        |
| 7.5.3 Position Counter Initialization.....                                               | 476        |
| 7.5.4 eQEP Position-compare Unit.....                                                    | 476        |
| 7.6 eQEP Edge Capture Unit.....                                                          | 478        |
| 7.7 eQEP Watchdog.....                                                                   | 482        |
| 7.8 eQEP Unit Timer Base.....                                                            | 482        |
| 7.9 eQEP Interrupt Structure.....                                                        | 483        |
| 7.10 eQEP Registers.....                                                                 | 483        |
| 7.10.1 eQEP Base Addresses.....                                                          | 483        |
| 7.10.2 EQEP_REGS Registers.....                                                          | 484        |
| <b>8 Analog-to-Digital Converter (ADC).....</b>                                          | <b>509</b> |
| 8.1 Introduction.....                                                                    | 510        |
| 8.1.1 Features.....                                                                      | 510        |
| 8.1.2 ADC Related Collateral.....                                                        | 510        |
| 8.1.3 Block Diagram.....                                                                 | 512        |
| 8.2 SOC Principle of Operation.....                                                      | 513        |
| 8.2.1 ADC Acquisition (Sample and Hold) Window.....                                      | 514        |
| 8.2.2 Trigger Operation.....                                                             | 519        |
| 8.2.3 Channel Selection.....                                                             | 519        |
| 8.3 ONESHOT Single Conversion Support.....                                               | 519        |
| 8.4 ADC Conversion Priority.....                                                         | 520        |
| 8.5 Sequential Sampling Mode.....                                                        | 523        |
| 8.6 Simultaneous Sampling Mode.....                                                      | 523        |
| 8.7 EOC and Interrupt Operation.....                                                     | 524        |
| 8.8 Power-Up Sequence.....                                                               | 525        |
| 8.9 ADC Calibration.....                                                                 | 525        |
| 8.9.1 Factory Settings and Calibration Function.....                                     | 525        |
| 8.9.2 ADC Zero Offset Calibration.....                                                   | 525        |
| 8.9.3 ADC Full Scale Gain Calibration.....                                               | 526        |
| 8.9.4 ADC Bias Current Calibration.....                                                  | 526        |
| 8.10 Internal/External Reference Voltage Selection.....                                  | 526        |
| 8.10.1 Internal Reference Voltage.....                                                   | 526        |
| 8.10.2 External Reference Voltage.....                                                   | 527        |
| 8.11 ADC Timings.....                                                                    | 527        |
| 8.12 Internal Temperature Sensor.....                                                    | 532        |
| 8.12.1 Transfer Function.....                                                            | 532        |
| 8.13 ADC Registers.....                                                                  | 534        |

|                                                                                 |            |
|---------------------------------------------------------------------------------|------------|
| 8.13.1 ADC Control Register 1 (ADCCTL1).....                                    | 535        |
| 8.13.2 ADC Control Register 2 (ADCCTL2).....                                    | 537        |
| 8.13.3 ADC Interrupt Registers.....                                             | 538        |
| 8.13.4 ADC Start of Conversion Priority Control Register (SOCPRCTL).....        | 545        |
| 8.13.5 ADC SOC Registers.....                                                   | 547        |
| 8.13.6 ADC Calibration Registers.....                                           | 556        |
| 8.13.7 Comparator Hysteresis Control Register (COMPHYSTCTL).....                | 557        |
| 8.13.8 ADC Revision Register (ADCREV).....                                      | 558        |
| 8.13.9 ADC RESULT0-RESULT15 Registers (ADCRESULTx).....                         | 558        |
| <b>9 Comparator (COMP).....</b>                                                 | <b>559</b> |
| 9.1 Introduction.....                                                           | 560        |
| 9.1.1 Features.....                                                             | 560        |
| 9.1.2 Block Diagram.....                                                        | 560        |
| 9.2 Comparator Function.....                                                    | 561        |
| 9.3 DAC Reference.....                                                          | 561        |
| 9.4 Ramp Generator Input.....                                                   | 562        |
| 9.5 Initialization.....                                                         | 563        |
| 9.6 Digital Domain Manipulation.....                                            | 563        |
| 9.7 Comparator Registers.....                                                   | 564        |
| 9.7.1 Comparator Control (COMPCTL) Register.....                                | 565        |
| 9.7.2 Compare Output Status (COMPSTS) Register.....                             | 566        |
| 9.7.3 DAC Control (DACCTL) Register.....                                        | 566        |
| 9.7.4 DAC Value (DACVAL) Register.....                                          | 567        |
| 9.7.5 Ramp Generator Maximum Reference Active (RAMPMAXREF_ACTIVE) Register..... | 567        |
| 9.7.6 Ramp Generator Maximum Reference Shadow (RAMPMAXREF_SHDW) Register.....   | 567        |
| 9.7.7 Ramp Generator Decrement Value Active (RAMPDECVAL_ACTIVE) Register.....   | 568        |
| 9.7.8 Ramp Generator Decrement Value Shadow (RAMPDECVAL_SHDW) Register.....     | 568        |
| 9.7.9 Ramp Generator Status (RAMPSTS) Register.....                             | 568        |
| <b>10 Control Law Accelerator (CLA).....</b>                                    | <b>569</b> |
| 10.1 Introduction.....                                                          | 570        |
| 10.1.1 Features.....                                                            | 570        |
| 10.1.2 CLA Related Collateral.....                                              | 570        |
| 10.1.3 Block Diagram.....                                                       | 571        |
| 10.2 CLA Interface.....                                                         | 572        |
| 10.2.1 CLA Memory.....                                                          | 572        |
| 10.2.2 CLA Memory Bus.....                                                      | 573        |
| 10.2.3 Shared Peripherals and EALLOW Protection.....                            | 573        |
| 10.2.4 CLA Tasks and Interrupt Vectors.....                                     | 574        |
| 10.3 CLA and CPU Arbitration.....                                               | 575        |
| 10.3.1 CLA Message RAM.....                                                     | 575        |
| 10.3.2 CLA Program Memory.....                                                  | 576        |
| 10.3.3 CLA Data Memory.....                                                     | 576        |
| 10.3.4 Peripheral Registers (ePWM, HRPWM, Comparator, eCAP, eQEP).....          | 577        |
| 10.4 CLA Configuration and Debug.....                                           | 578        |
| 10.4.1 Building a CLA Application.....                                          | 578        |
| 10.4.2 Typical CLA Initialization Sequence.....                                 | 579        |
| 10.4.3 Debugging CLA Code.....                                                  | 580        |
| 10.4.4 CLA Illegal Opcode Behavior.....                                         | 581        |
| 10.4.5 Resetting the CLA.....                                                   | 582        |
| 10.5 Pipeline.....                                                              | 582        |
| 10.5.1 Pipeline Overview.....                                                   | 582        |
| 10.5.2 CLA Pipeline Alignment.....                                              | 583        |
| 10.5.3 Parallel Instructions.....                                               | 587        |
| 10.6 Instruction Set.....                                                       | 588        |
| 10.6.1 Instruction Descriptions.....                                            | 588        |
| 10.6.2 Addressing Modes and Encoding.....                                       | 589        |
| 10.6.3 Instructions.....                                                        | 591        |
| 10.7 CLA Registers.....                                                         | 709        |
| 10.7.1 Register Memory Mapping.....                                             | 709        |
| 10.7.2 Task Interrupt Vector Registers.....                                     | 710        |
| 10.7.3 Configuration Registers.....                                             | 711        |

|                                                                                                                                             |            |
|---------------------------------------------------------------------------------------------------------------------------------------------|------------|
| 10.7.4 Execution Registers.....                                                                                                             | 724        |
| <b>11 Direct Memory Access (DMA) Module.....</b>                                                                                            | <b>727</b> |
| 11.1 Introduction.....                                                                                                                      | 728        |
| 11.2 DMA Overview.....                                                                                                                      | 728        |
| 11.3 Architecture.....                                                                                                                      | 729        |
| 11.3.1 Block Diagram.....                                                                                                                   | 729        |
| 11.3.2 Peripheral Interrupt Event Trigger Sources.....                                                                                      | 730        |
| 11.3.3 DMA Bus.....                                                                                                                         | 731        |
| 11.4 Pipeline Timing and Throughput.....                                                                                                    | 732        |
| 11.5 CPU Arbitration.....                                                                                                                   | 733        |
| 11.6 Channel Priority.....                                                                                                                  | 733        |
| 11.6.1 Round-Robin Mode.....                                                                                                                | 733        |
| 11.6.2 Channel 1 High Priority Mode.....                                                                                                    | 734        |
| 11.7 Address Pointer and Transfer Control.....                                                                                              | 735        |
| 11.8 Overrun Detection Feature.....                                                                                                         | 739        |
| 11.9 DMA Registers.....                                                                                                                     | 740        |
| 11.9.1 DMA Control Register (DMACR) — EALLOW Protected.....                                                                                 | 741        |
| 11.9.2 Debug Control Register (DEBUGCR) — EALLOW Protected.....                                                                             | 743        |
| 11.9.3 Revision Register (REVISION).....                                                                                                    | 743        |
| 11.9.4 Priority Control Register 1 (PRIORITYCTRL1) — EALLOW Protected.....                                                                  | 744        |
| 11.9.5 Priority Status Register (PRIORITYSTAT).....                                                                                         | 745        |
| 11.9.6 Mode Register (MODE) — EALLOW Protected.....                                                                                         | 746        |
| 11.9.7 Control Register (CONTROL) — EALLOW Protected.....                                                                                   | 748        |
| 11.9.8 Burst Size Register (BURST_SIZE) — EALLOW Protected.....                                                                             | 750        |
| 11.9.9 BURST_COUNT Register.....                                                                                                            | 750        |
| 11.9.10 Source Burst Step Register Size (SRC_BURST_STEP) — EALLOW Protected.....                                                            | 751        |
| 11.9.11 Destination Burst Step Register Size (DST_BURST_STEP) — EALLOW Protected.....                                                       | 752        |
| 11.9.12 Transfer Size Register (TRANSFER_SIZE) — EALLOW Protected.....                                                                      | 752        |
| 11.9.13 Transfer Count Register (TRANSFER_COUNT).....                                                                                       | 753        |
| 11.9.14 Source Transfer Step Size Register (SRC_TRANSFER_STEP) — EALLOW Protected.....                                                      | 753        |
| 11.9.15 Destination Transfer Step Size Register (DST_TRANSFER_STEP) — EALLOW Protected.....                                                 | 754        |
| 11.9.16 Source/Destination Wrap Size Register (SRC/DST_WRAP_SIZE) — EALLOW protected.....                                                   | 755        |
| 11.9.17 Source/Destination Wrap Count Register (SRC/DST_WRAP_COUNT).....                                                                    | 755        |
| 11.9.18 Source/Destination Wrap Step Size Registers (SRC/DST_WRAP_STEP) — EALLOW Protected.....                                             | 756        |
| 11.9.19 Shadow Source Begin and Current Address Pointer Registers (SRC_BEG_ADDR_SHADOW/<br>DST_BEG_ADDR_SHADOW) — All EALLOW Protected..... | 757        |
| 11.9.20 Active Source Begin and Current Address Pointer Registers (SRC_BEG_ADDR/DST_BEG_ADDR).....                                          | 757        |
| 11.9.21 Shadow Destination Begin and Current Address Pointer Registers (SRC_ADDR_SHADOW/<br>DST_ADDR_SHADOW) — All EALLOW Protected.....    | 758        |
| 11.9.22 Active Destination Begin and Current Address Pointer Registers (SRC_ADDR/DST_ADDR).....                                             | 758        |
| <b>12 Serial Peripheral Interface (SPI).....</b>                                                                                            | <b>759</b> |
| 12.1 Introduction.....                                                                                                                      | 760        |
| 12.1.1 Features.....                                                                                                                        | 760        |
| 12.1.2 Block Diagram.....                                                                                                                   | 761        |
| 12.2 System-Level Integration.....                                                                                                          | 762        |
| 12.2.1 SPI Module Signals.....                                                                                                              | 762        |
| 12.2.2 Configuring Device Pins.....                                                                                                         | 762        |
| 12.2.3 SPI Interrupts.....                                                                                                                  | 763        |
| 12.2.4 DMA Support.....                                                                                                                     | 764        |
| 12.3 SPI Operation.....                                                                                                                     | 766        |
| 12.3.1 Introduction to Operation.....                                                                                                       | 766        |
| 12.3.2 Master Mode.....                                                                                                                     | 768        |
| 12.3.3 Slave Mode.....                                                                                                                      | 768        |
| 12.3.4 Data Format.....                                                                                                                     | 769        |
| 12.3.5 Baud Rate Selection.....                                                                                                             | 770        |
| 12.3.6 SPI Clocking Schemes.....                                                                                                            | 771        |
| 12.3.7 SPI FIFO Description.....                                                                                                            | 772        |
| 12.3.8 SPI 3-Wire Mode Description.....                                                                                                     | 773        |
| 12.4 Programming Procedure.....                                                                                                             | 775        |
| 12.4.1 Initialization Upon Reset.....                                                                                                       | 775        |
| 12.4.2 Configuring the SPI.....                                                                                                             | 775        |

|                                                               |            |
|---------------------------------------------------------------|------------|
| 12.4.3 Data Transfer Example.....                             | 776        |
| 12.4.4 SPI 3-Wire Mode Code Examples.....                     | 777        |
| 12.4.5 SPI STEINV Bit in Digital Audio Transfers.....         | 778        |
| 12.5 SPI Registers.....                                       | 779        |
| 12.5.1 SPI Base Addresses.....                                | 779        |
| 12.5.2 SPI_REGS Registers.....                                | 779        |
| <b>13 Serial Communications Interface (SCI).....</b>          | <b>799</b> |
| 13.1 Introduction.....                                        | 800        |
| 13.1.1 Features.....                                          | 800        |
| 13.1.2 SCI Related Collateral.....                            | 801        |
| 13.1.3 Block Diagram.....                                     | 801        |
| 13.2 Architecture.....                                        | 801        |
| 13.3 SCI Module Signal Summary.....                           | 801        |
| 13.4 Configuring Device Pins.....                             | 802        |
| 13.5 Multiprocessor and Asynchronous Communication Modes..... | 803        |
| 13.6 SCI Programmable Data Format.....                        | 803        |
| 13.7 SCI Multiprocessor Communication.....                    | 804        |
| 13.7.1 Recognizing the Address Byte.....                      | 804        |
| 13.7.2 Controlling the SCI TX and RX Features.....            | 804        |
| 13.7.3 Receipt Sequence.....                                  | 804        |
| 13.8 Idle-Line Multiprocessor Mode.....                       | 805        |
| 13.8.1 Idle-Line Mode Steps.....                              | 805        |
| 13.8.2 Block Start Signal.....                                | 805        |
| 13.8.3 Wake-UP Temporary (WUT) Flag.....                      | 806        |
| 13.8.4 Receiver Operation.....                                | 806        |
| 13.9 Address-Bit Multiprocessor Mode.....                     | 807        |
| 13.9.1 Sending an Address.....                                | 807        |
| 13.10 SCI Communication Format.....                           | 808        |
| 13.10.1 Receiver Signals in Communication Modes.....          | 808        |
| 13.10.2 Transmitter Signals in Communication Modes.....       | 809        |
| 13.11 SCI Port Interrupts.....                                | 810        |
| 13.12 SCI Baud Rate Calculations.....                         | 810        |
| 13.13 SCI Enhanced Features.....                              | 811        |
| 13.13.1 SCI FIFO Description.....                             | 811        |
| 13.13.2 SCI Auto-Baud.....                                    | 813        |
| 13.13.3 Autobaud-Detect Sequence.....                         | 813        |
| 13.14 SCI Registers.....                                      | 814        |
| 13.14.1 SCI Base Addresses.....                               | 814        |
| 13.14.2 SCI_REGS Registers.....                               | 814        |
| <b>14 Inter-Integrated Circuit Module (I2C).....</b>          | <b>833</b> |
| 14.1 Introduction.....                                        | 834        |
| 14.1.1 I2C Related Collateral.....                            | 834        |
| 14.1.2 Features.....                                          | 835        |
| 14.1.3 Features Not Supported.....                            | 835        |
| 14.1.4 Functional Overview.....                               | 836        |
| 14.1.5 Clock Generation.....                                  | 837        |
| 14.1.6 I2C Clock Divider Registers (I2CCLKL and I2CCLKH)..... | 838        |
| 14.2 Configuring Device Pins.....                             | 839        |
| 14.3 I2C Module Operational Details.....                      | 839        |
| 14.3.1 Input and Output Voltage Levels.....                   | 839        |
| 14.3.2 Data Validity.....                                     | 839        |
| 14.3.3 Operating Modes.....                                   | 839        |
| 14.3.4 I2C Module START and STOP Conditions.....              | 843        |
| 14.3.5 Non-repeat Mode versus Repeat Mode.....                | 844        |
| 14.3.6 Serial Data Formats.....                               | 845        |
| 14.3.7 Clock Synchronization.....                             | 847        |
| 14.3.8 Arbitration.....                                       | 848        |
| 14.3.9 Digital Loopback Mode.....                             | 849        |
| 14.3.10 NACK Bit Generation.....                              | 850        |
| 14.4 Interrupt Requests Generated by the I2C Module.....      | 851        |
| 14.4.1 Basic I2C Interrupt Requests.....                      | 851        |

|                                                                                |            |
|--------------------------------------------------------------------------------|------------|
| 14.4.2 I2C FIFO Interrupts.....                                                | 853        |
| 14.5 Resetting or Disabling the I2C Module.....                                | 853        |
| 14.6 I2C Registers.....                                                        | 853        |
| 14.6.1 I2C Base Address Table.....                                             | 853        |
| 14.6.2 I2C_REGS Registers.....                                                 | 854        |
| <b>15 Multichannel Buffered Serial Port (McBSP).....</b>                       | <b>877</b> |
| 15.1 Introduction.....                                                         | 878        |
| 15.1.1 Features of the McBSP.....                                              | 878        |
| 15.1.2 McBSP Pins/Signals.....                                                 | 879        |
| 15.2 McBSP Operation.....                                                      | 880        |
| 15.2.1 Data Transfer Process of McBSP.....                                     | 880        |
| 15.2.2 Companding (Compressing and Expanding) Data.....                        | 881        |
| 15.2.3 Clocking and Framing Data.....                                          | 883        |
| 15.2.4 Frame Phases.....                                                       | 885        |
| 15.3 McBSP Sample Rate Generator.....                                          | 890        |
| 15.3.1 Block Diagram.....                                                      | 890        |
| 15.3.2 Frame Synchronization Generation in the Sample Rate Generator.....      | 893        |
| 15.3.3 Synchronizing Sample Rate Generator Outputs to an External Clock.....   | 893        |
| 15.3.4 Reset and Initialization Procedure for the Sample Rate Generator.....   | 895        |
| 15.4 McBSP Exception/Error Conditions.....                                     | 896        |
| 15.4.1 Types of Errors.....                                                    | 896        |
| 15.4.2 Overrun in the Receiver.....                                            | 897        |
| 15.4.3 Unexpected Receive Frame-Synchronization Pulse.....                     | 898        |
| 15.4.4 Overwrite in the Transmitter.....                                       | 900        |
| 15.4.5 Unexpected Transmit Frame-Synchronization Pulse.....                    | 902        |
| 15.5 Multichannel Selection Modes.....                                         | 904        |
| 15.5.1 Channels, Blocks, and Partitions.....                                   | 904        |
| 15.5.2 Multichannel Selection.....                                             | 905        |
| 15.5.3 Configuring a Frame for Multichannel Selection.....                     | 905        |
| 15.5.4 Using Two Partitions.....                                               | 905        |
| 15.5.5 Using Eight Partitions.....                                             | 907        |
| 15.5.6 Receive Multichannel Selection Mode.....                                | 908        |
| 15.5.7 Transmit Multichannel Selection Modes.....                              | 909        |
| 15.6 SPI Operation Using the Clock Stop Mode.....                              | 912        |
| 15.6.1 SPI Protocol.....                                                       | 912        |
| 15.6.2 Clock Stop Mode.....                                                    | 912        |
| 15.6.3 Bits Used to Enable and Configure the Clock Stop Mode.....              | 913        |
| 15.6.4 Clock Stop Mode Timing Diagrams.....                                    | 914        |
| 15.6.5 Procedure for Configuring a McBSP for SPI Operation.....                | 916        |
| 15.6.6 McBSP as the SPI Master.....                                            | 917        |
| 15.6.7 McBSP as an SPI Slave.....                                              | 918        |
| 15.7 Receiver Configuration.....                                               | 919        |
| 15.7.1 Programming the McBSP Registers for the Desired Receiver Operation..... | 919        |
| 15.7.2 Resetting and Enabling the Receiver.....                                | 920        |
| 15.7.3 Set the Receiver Pins to Operate as McBSP Pins.....                     | 920        |
| 15.7.4 Enable/Disable the Digital Loopback Mode.....                           | 921        |
| 15.7.5 Enable/Disable the Clock Stop Mode.....                                 | 921        |
| 15.7.6 Enable/Disable the Receive Multichannel Selection Mode.....             | 922        |
| 15.7.7 Choose One or Two Phases for the Receive Frame.....                     | 922        |
| 15.7.8 Set the Receive Word Length(s).....                                     | 923        |
| 15.7.9 Set the Receive Frame Length.....                                       | 923        |
| 15.7.10 Enable/Disable the Receive Frame-Synchronization Ignore Function.....  | 924        |
| 15.7.11 Set the Receive Companding Mode.....                                   | 925        |
| 15.7.12 Set the Receive Data Delay.....                                        | 926        |
| 15.7.13 Set the Receive Sign-Extension and Justification Mode.....             | 928        |
| 15.7.14 Set the Receive Interrupt Mode.....                                    | 929        |
| 15.7.15 Set the Receive Frame-Synchronization Mode.....                        | 930        |
| 15.7.16 Set the Receive Frame-Synchronization Polarity.....                    | 932        |
| 15.7.17 Set the Receive Clock Mode.....                                        | 934        |
| 15.7.18 Set the Receive Clock Polarity.....                                    | 935        |
| 15.7.19 Set the SRG Clock Divide-Down Value.....                               | 936        |



|                                                                                           |            |
|-------------------------------------------------------------------------------------------|------------|
| 15.7.20 Set the SRG Clock Synchronization Mode.....                                       | 937        |
| 15.7.21 Set the SRG Clock Mode (Choose an Input Clock).....                               | 937        |
| 15.7.22 Set the SRG Input Clock Polarity.....                                             | 938        |
| 15.8 Transmitter Configuration.....                                                       | 938        |
| 15.8.1 Programming the McBSP Registers for the Desired Transmitter Operation.....         | 938        |
| 15.8.2 Resetting and Enabling the Transmitter.....                                        | 939        |
| 15.8.3 Set the Transmitter Pins to Operate as McBSP Pins.....                             | 940        |
| 15.8.4 Enable/Disable the Digital Loopback Mode.....                                      | 940        |
| 15.8.5 Enable/Disable the Clock Stop Mode.....                                            | 940        |
| 15.8.6 Enable/Disable Transmit Multichannel Selection.....                                | 941        |
| 15.8.7 Choose One or Two Phases for the Transmit Frame.....                               | 942        |
| 15.8.8 Set the Transmit Word Length(s).....                                               | 942        |
| 15.8.9 Set the Transmit Frame Length.....                                                 | 943        |
| 15.8.10 Enable/Disable the Transmit Frame-Synchronization Ignore Function.....            | 944        |
| 15.8.11 Set the Transmit Companding Mode.....                                             | 945        |
| 15.8.12 Set the Transmit Data Delay.....                                                  | 946        |
| 15.8.13 Set the Transmit DXENA Mode.....                                                  | 948        |
| 15.8.14 Set the Transmit Interrupt Mode.....                                              | 948        |
| 15.8.15 Set the Transmit Frame-Synchronization Mode.....                                  | 949        |
| 15.8.16 Set the Transmit Frame-Synchronization Polarity.....                              | 950        |
| 15.8.17 Set the SRG Frame-Synchronization Period and Pulse Width.....                     | 951        |
| 15.8.18 Set the Transmit Clock Mode.....                                                  | 952        |
| 15.8.19 Set the Transmit Clock Polarity.....                                              | 952        |
| 15.9 Emulation and Reset Considerations.....                                              | 954        |
| 15.9.1 McBSP Emulation Mode.....                                                          | 954        |
| 15.9.2 Resetting and Initializing McBSP.....                                              | 954        |
| 15.10 Data Packing Examples.....                                                          | 956        |
| 15.10.1 Data Packing Using Frame Length and Word Length.....                              | 956        |
| 15.10.2 Data Packing Using Word Length and the Frame-Synchronization Ignore Function..... | 958        |
| 15.11 McBSP_REGS Registers.....                                                           | 959        |
| <b>16 Controller Area Network (CAN).....</b>                                              | <b>999</b> |
| 16.1 Introduction.....                                                                    | 1000       |
| 16.1.1 Features.....                                                                      | 1000       |
| 16.1.2 ECAN Related Collateral.....                                                       | 1000       |
| 16.1.3 Block Diagram.....                                                                 | 1001       |
| 16.2 eCAN Compatibility With Other TI CAN Modules.....                                    | 1001       |
| 16.3 CAN Network and Module.....                                                          | 1002       |
| 16.3.1 CAN Protocol Overview.....                                                         | 1002       |
| 16.4 eCAN Controller Overview.....                                                        | 1004       |
| 16.4.1 Standard CAN Controller (SCC) Mode.....                                            | 1004       |
| 16.4.2 Memory Map.....                                                                    | 1005       |
| 16.5 Message Objects.....                                                                 | 1007       |
| 16.6 Message Mailbox.....                                                                 | 1007       |
| 16.6.1 Transmit Mailbox.....                                                              | 1009       |
| 16.6.2 Receive Mailbox.....                                                               | 1010       |
| 16.6.3 CAN Module Operation in Normal Configuration.....                                  | 1010       |
| 16.7 eCAN Configuration.....                                                              | 1010       |
| 16.7.1 CAN Module Initialization.....                                                     | 1010       |
| 16.7.2 Steps to Configure eCAN.....                                                       | 1014       |
| 16.7.3 Handling of Remote Frame Mailboxes.....                                            | 1016       |
| 16.7.4 Interrupts.....                                                                    | 1017       |
| 16.7.5 CAN Power-Down Mode.....                                                           | 1022       |
| 16.8 Acceptance Filter.....                                                               | 1023       |
| 16.8.1 Local-Acceptance Masks (CANLAM).....                                               | 1024       |
| 16.9 eCAN Registers.....                                                                  | 1025       |
| 16.9.1 Mailbox Enable Register (CANME).....                                               | 1025       |
| 16.9.2 Mailbox-Direction Register (CANMD).....                                            | 1025       |
| 16.9.3 Transmission-Request Set Register (CANTRS).....                                    | 1026       |
| 16.9.4 Transmission-Request-Reset Register (CANTRR).....                                  | 1027       |
| 16.9.5 Transmission-Acknowledge Register (CANTA).....                                     | 1027       |
| 16.9.6 Abort-Acknowledge Register (CANAA).....                                            | 1028       |

|                                                                                                       |             |
|-------------------------------------------------------------------------------------------------------|-------------|
| 16.9.7 Received-Message-Pending Register (CANRMP).....                                                | 1028        |
| 16.9.8 Received-Message-Lost Register (CANRML).....                                                   | 1029        |
| 16.9.9 Remote-Frame-Pending Register (CANRFP).....                                                    | 1029        |
| 16.9.10 Global Acceptance Mask Register (CANGAM).....                                                 | 1031        |
| 16.9.11 Master Control Register (CANMC).....                                                          | 1032        |
| 16.9.12 Bit-Timing Configuration Register (CANBTC).....                                               | 1035        |
| 16.9.13 Error and Status Register (CANES).....                                                        | 1037        |
| 16.9.14 CAN Error Counter Registers (CANTEC/CANREC).....                                              | 1039        |
| 16.9.15 Interrupt Registers.....                                                                      | 1040        |
| 16.9.16 Overwrite Protection Control Register (CANOPC).....                                           | 1045        |
| 16.9.17 eCAN I/O Control Registers (CANTIOC, CANRIOC).....                                            | 1046        |
| 16.9.18 Timer Management Unit.....                                                                    | 1047        |
| 16.9.19 Mailbox Layout.....                                                                           | 1050        |
| 16.10 Message Data Registers (CANMDL, CANMDH).....                                                    | 1053        |
| <b>17 Universal Serial Bus (USB) Controller.....</b>                                                  | <b>1055</b> |
| 17.1 Introduction.....                                                                                | 1056        |
| 17.1.1 Features.....                                                                                  | 1056        |
| 17.1.2 USB Related Collateral.....                                                                    | 1058        |
| 17.2 Functional Description.....                                                                      | 1058        |
| 17.2.1 Operation as a Device.....                                                                     | 1058        |
| 17.2.2 Operation as a Host.....                                                                       | 1062        |
| 17.2.3 DMA Operation.....                                                                             | 1066        |
| 17.2.4 Address/Data Bus Bridge.....                                                                   | 1066        |
| 17.3 Initialization and Configuration.....                                                            | 1068        |
| 17.3.1 Pin Configuration.....                                                                         | 1068        |
| 17.3.2 Endpoint Configuration.....                                                                    | 1069        |
| 17.4 USB Global Interrupts.....                                                                       | 1069        |
| 17.5 USB Registers.....                                                                               | 1069        |
| 17.5.1 USB Device Functional Address Register (USBFADDR), offset 0x000.....                           | 1072        |
| 17.5.2 USB Power Management Register (USBPOWER), offset 0x001.....                                    | 1073        |
| 17.5.3 USB Transmit Interrupt Status Register (USBTXIS), offset 0x002.....                            | 1075        |
| 17.5.4 USB Receive Interrupt Status Register (USBRXIS), offset 0x004.....                             | 1076        |
| 17.5.5 USB Transmit Interrupt Enable Register (USBTXIE), offset 0x006.....                            | 1077        |
| 17.5.6 USB Receive Interrupt Enable Register (USBRXIE), offset 0x008.....                             | 1078        |
| 17.5.7 USB General Interrupt Status Register (USBIS), offset 0x00A.....                               | 1079        |
| 17.5.8 USB Interrupt Enable Register (USBIE), offset 0x00B.....                                       | 1081        |
| 17.5.9 USB Frame Value Register (USBFRAME), offset 0x00C.....                                         | 1083        |
| 17.5.10 USB Endpoint Index Register (USBEPIDX), offset 0x00E.....                                     | 1083        |
| 17.5.11 USB Test Mode Register (USBTEST), offset 0x00F.....                                           | 1084        |
| 17.5.12 USB FIFO Endpoint n Register (USBFIFO[0]-USBFIFO[3]).....                                     | 1086        |
| 17.5.13 USB Device Control Register (USBDEVCTL), offset 0x060.....                                    | 1087        |
| 17.5.14 USB Transmit Dynamic FIFO Sizing Register (USBTXFIFOSZ), offset 0x062.....                    | 1089        |
| 17.5.15 USB Receive Dynamic FIFO Sizing Register (USBRXFIFOSZ), offset 0x063.....                     | 1090        |
| 17.5.16 USB Transmit FIFO Start Address Register (USBTXFIFOADD), offset 0x064.....                    | 1091        |
| 17.5.17 USB Receive FIFO Start Address Register (USBRXFIFOADD), offset 0x066.....                     | 1092        |
| 17.5.18 USB Connect Timing Register (USBCONTIM), offset 0x07A.....                                    | 1093        |
| 17.5.19 USB Full-Speed Last Transaction to End of Frame Timing Register (USBFSEOF), offset 0x07D..... | 1094        |
| 17.5.20 USB Low-Speed Last Transaction to End of Frame Timing Register (USBLSEOF), offset 0x07E.....  | 1094        |
| 17.5.21 USB Transmit Functional Address Endpoint n Registers (USBTXFUNCADDR[0]-USBTXFUNCADDR[3])..... | 1095        |
| 17.5.22 USB Transmit Hub Address Endpoint n Registers (USBTXHUBADDR[0]-USBTXHUBADDR[3]).....          | 1096        |
| 17.5.23 USB Transmit Hub Port Endpoint n Registers (USBTXHUBPORT[0]-USBTXHUBPORT[3]).....             | 1097        |
| 17.5.24 USB Receive Functional Address Endpoint n Registers (USBRXFUNCADDR[1]-USBRXFUNCADDR[3]).....  | 1098        |
| 17.5.25 USB Receive Hub Address Endpoint n Registers (USBRXHUBADDR[1]-USBRXHUBADDR[3]).....           | 1099        |
| 17.5.26 USB Receive Hub Port Endpoint n Registers (USBRXHUBPORT[1]-USBRXHUBPORT[3]).....              | 1100        |
| 17.5.27 USB Maximum Transmit Data Endpoint n Registers (USBTXMAXP[1]-USBTXMAXP[3]).....               | 1101        |
| 17.5.28 USB Control and Status Endpoint 0 Low Register (USBCSRL0), offset 0x102.....                  | 1102        |
| 17.5.29 USB Control and Status Endpoint 0 High Register (USBCSRH0), offset 0x103.....                 | 1105        |
| 17.5.30 USB Receive Byte Count Endpoint 0 Register (USBCOUNT0), offset 0x108.....                     | 1107        |
| 17.5.31 USB Type Endpoint 0 Register (USBTYP0), offset 0x10A.....                                     | 1107        |
| 17.5.32 USB NAK Limit Register (USBNAKLMT), offset 0x10B.....                                         | 1108        |
| 17.5.33 USB Transmit Control and Status Endpoint n Low Register (USBTXCSRL[1]-USBTXCSRL[3]).....      | 1109        |



|                                                                                                                      |             |
|----------------------------------------------------------------------------------------------------------------------|-------------|
| 17.5.34 USB Transmit Control and Status Endpoint n High Register (USBTXCSRH[1]-USBTXCSRH[3]).....                    | 1112        |
| 17.5.35 USB Maximum Receive Data Endpoint n Registers (USBRXMAXP[1]-USBRXMAXP[3]).....                               | 1114        |
| 17.5.36 USB Receive Control and Status Endpoint n Low Register (USBRXCSSL[1]-USBRXCSSL[3]).....                      | 1115        |
| 17.5.37 USB Receive Control and Status Endpoint n High Register (USBRXCSRH[1]-USBRXCSRH[3]).....                     | 1118        |
| 17.5.38 USB Receive Byte Count Endpoint n Registers (USBRXCOUNT[1]-USBRXCOUNT[3]).....                               | 1120        |
| 17.5.39 USB Host Transmit Configure Type Endpoint n Register (USBTXTYPE[1]-USBTXTYPE[3]).....                        | 1121        |
| 17.5.40 USB Host Transmit Interval Endpoint n Register (USBTXINTERVAL[1]-USBTXINTERVAL[3]).....                      | 1122        |
| 17.5.41 USB Host Configure Receive Type Endpoint n Register (USBRXTYPE[1]-USBRXTYPE[3]).....                         | 1123        |
| 17.5.42 USB Host Receive Polling Interval Endpoint n Register (USBRXINTERVAL[1]-USBRXINTERVAL[3]).....               | 1124        |
| 17.5.43 USB Request Packet Count in Block Transfer Endpoint n Registers (USBRQPKTCOUNT[1]-<br>USBRQPKTCOUNT[3])..... | 1125        |
| 17.5.44 USB Receive Double Packet Buffer Disable Register (USBRXDPKTBUFFDIS), offset 0x340.....                      | 1126        |
| 17.5.45 USB Transmit Double Packet Buffer Disable Register (USBTXDPKTBUFFDIS), offset 0x342.....                     | 1127        |
| 17.5.46 USB External Power Control Register (USBEPCC), offset 0x400.....                                             | 1128        |
| 17.5.47 USB External Power Control Raw Interrupt Status Register (USBEPCCRIS), offset 0x404.....                     | 1130        |
| 17.5.48 USB External Power Control Interrupt Mask Register (USBEPCCIM), offset 0x408.....                            | 1131        |
| 17.5.49 USB External Power Control Interrupt Status and Clear Register (USBEPCCISC), offset 0x40C.....               | 1132        |
| 17.5.50 USB Device RESUME Raw Interrupt Status Register (USBDRRIS), offset 0x410.....                                | 1133        |
| 17.5.51 USB Device RESUME Raw Interrupt Mask Register (USBDRIM), offset 0x414.....                                   | 1134        |
| 17.5.52 USB Device RESUME Interrupt Status and Clear Register (USBDRISC), offset 0x418.....                          | 1135        |
| 17.5.53 USB General-Purpose Control and Status Register (USBGPCS), offset 0x41C.....                                 | 1136        |
| 17.5.54 USB DMA Select Register (USBDMASEL), offset 0x450.....                                                       | 1137        |
| <b>18 Revision History.....</b>                                                                                      | <b>1139</b> |

## List of Figures

|                                                                              |    |
|------------------------------------------------------------------------------|----|
| Figure 1-1. Flash Power Mode State Diagram.....                              | 42 |
| Figure 1-2. Flash Pipeline.....                                              | 44 |
| Figure 1-3. Flash Configuration Access Flow Diagram.....                     | 45 |
| Figure 1-4. Flash Options Register (FOPT).....                               | 47 |
| Figure 1-5. Flash Power Register (FPWR).....                                 | 47 |
| Figure 1-6. Flash Status Register (FSTATUS).....                             | 48 |
| Figure 1-7. Flash Standby Wait Register (FSTDBYWAIT).....                    | 49 |
| Figure 1-8. Flash Standby to Active Wait Counter Register (FACTIVEWAIT)..... | 49 |
| Figure 1-9. Flash Wait-State Register (FBANKWAIT).....                       | 50 |
| Figure 1-10. OTP Wait-State Register (FOTPWAIT).....                         | 51 |
| Figure 1-11. Password Match Flow (PMF).....                                  | 57 |
| Figure 1-12. CSM Status and Control Register (CSMSCR).....                   | 61 |
| Figure 1-13. Clock and Reset Domains.....                                    | 62 |
| Figure 1-14. Peripheral Clock Control 0 Register (PCLKCR0).....              | 63 |
| Figure 1-15. Peripheral Clock Control 1 Register (PCLKCR1).....              | 65 |
| Figure 1-16. Peripheral Clock Control 2 Register (PCLKCR2).....              | 67 |
| Figure 1-17. Peripheral Clock Control 3 Register (PCLKCR3).....              | 68 |
| Figure 1-18. Low-Speed Peripheral Clock Prescaler Register (LOSPCP).....     | 69 |
| Figure 1-19. Clocking Options.....                                           | 70 |
| Figure 1-20. Internal Oscillator Trim n (INTOSCnTRIM) Register.....          | 71 |
| Figure 1-21. Clocking (XCCLK) Register.....                                  | 72 |
| Figure 1-22. Clock Control (CLKCTL) Register.....                            | 73 |
| Figure 1-23. OSC and PLL Block.....                                          | 76 |
| Figure 1-24. PLLCR Change Procedure Flow Chart.....                          | 77 |
| Figure 1-25. PLLCR Register Layout.....                                      | 78 |
| Figure 1-26. PLL Status Register (PLLSTS).....                               | 79 |
| Figure 1-27. PLL Lock Period (PLLLOCKPRD) Register.....                      | 81 |
| Figure 1-28. PLL2 Input and Output Configurations.....                       | 82 |
| Figure 1-29. PLL2 Configuration (PLL2CTL) Register.....                      | 82 |
| Figure 1-30. PLL2 Multiplier (PLL2MULT) Register.....                        | 83 |
| Figure 1-31. PLL2 Lock Status (PLL2STS) Register.....                        | 83 |
| Figure 1-32. SYSCLK2 Clock Counter (SYSCLK2CNTR) Register.....               | 84 |
| Figure 1-33. EPWM DMA/CLA Configuration (EPWMCFG) Register.....              | 84 |
| Figure 1-34. Clocking and Reset Logic.....                                   | 85 |
| Figure 1-35. Clock Fail Interrupt.....                                       | 88 |
| Figure 1-36. NMI Configuration (NMICFG) Register.....                        | 89 |

|                                                                                             |     |
|---------------------------------------------------------------------------------------------|-----|
| Figure 1-37. NMI Flag (NMIFLG) Register.....                                                | 90  |
| Figure 1-38. NMI Flag Clear (NMIFLGCLR) Register.....                                       | 90  |
| Figure 1-39. NMI Flag Force (NMIFLGFRC) Register.....                                       | 91  |
| Figure 1-40. NMI Watchdog Counter (NMIWDCNT) Register.....                                  | 91  |
| Figure 1-41. NMI Watchdog Period (NMIWDPRD) Register.....                                   | 92  |
| Figure 1-42. XCLKOUT Generation.....                                                        | 93  |
| Figure 1-43. Low-Power Mode Control 0 Register (LPMCR0).....                                | 95  |
| Figure 1-44. CPU Watchdog Module.....                                                       | 96  |
| Figure 1-45. System Control and Status Register (SCSR).....                                 | 99  |
| Figure 1-46. Watchdog Counter Register (WDCNTR).....                                        | 100 |
| Figure 1-47. Watchdog Reset Key Register (WDKEY).....                                       | 100 |
| Figure 1-48. Watchdog Control Register (WDCR).....                                          | 101 |
| Figure 1-49. CPU-Timers.....                                                                | 102 |
| Figure 1-50. CPU-Timer Interrupts Signals and Output Signal.....                            | 102 |
| Figure 1-51. TIMERxTIM Register (x = 0, 1, 2).....                                          | 104 |
| Figure 1-52. TIMERxTIMH Register (x = 0, 1, 2).....                                         | 104 |
| Figure 1-53. TIMERxPRD Register (x = 0, 1, 2).....                                          | 105 |
| Figure 1-54. TIMERxPRDH Register (x = 0, 1, 2).....                                         | 105 |
| Figure 1-55. TIMERxTCR Register (x = 0, 1, 2).....                                          | 105 |
| Figure 1-56. TIMERxTPR Register (x = 0, 1, 2).....                                          | 107 |
| Figure 1-57. TIMERxTPRH Register (x = 0, 1, 2).....                                         | 107 |
| Figure 1-58. General GPIO Multiplexing Diagram.....                                         | 108 |
| Figure 1-59. GPIO32, GPIO33 Multiplexing Diagram.....                                       | 109 |
| Figure 1-60. JTAG Port/GPIO Multiplexing.....                                               | 110 |
| Figure 1-61. JTAGDEBUG Register (Address 0x702A, EALLOW protected).....                     | 111 |
| Figure 1-62. Analog/GPIO Multiplexing.....                                                  | 112 |
| Figure 1-63. Input Qualification Using a Sampling Window.....                               | 117 |
| Figure 1-64. Input Qualifier Clock Cycles.....                                              | 119 |
| Figure 1-65. GPIO Port A MUX 1 (GPAMUX1) Register.....                                      | 124 |
| Figure 1-66. GPIO Port A MUX 2 (GPAMUX2) Register.....                                      | 127 |
| Figure 1-67. GPIO Port B MUX 1 (GPBMUX1) Register.....                                      | 130 |
| Figure 1-68. GPIO Port B MUX 2 (GPBMUX2) Register.....                                      | 132 |
| Figure 1-69. Analog I/O MUX (AIOMUX1) Register.....                                         | 134 |
| Figure 1-70. GPIO Port A Qualification Control (GPACTRL) Register.....                      | 135 |
| Figure 1-71. GPIO Port B Qualification Control (GPBCTRL) Register.....                      | 136 |
| Figure 1-72. GPIO A Control Register 2 Register (GPACTRL2).....                             | 137 |
| Figure 1-73. GPIO Port A Qualification Select 1 (GPAQSEL1) Register.....                    | 138 |
| Figure 1-74. GPIO Port A Qualification Select 2 (GPAQSEL2) Register.....                    | 139 |
| Figure 1-75. GPIO Port B Qualification Select 1 (GPBQSEL1) Register.....                    | 140 |
| Figure 1-76. GPIO Port B Qualification Select 2 (GPBQSEL2) Register.....                    | 141 |
| Figure 1-77. GPIO Port A Direction (GPADIR) Register.....                                   | 142 |
| Figure 1-78. GPIO Port B Direction (GPBDIR) Register.....                                   | 143 |
| Figure 1-79. Analog I/O DIR (AIODIR) Register.....                                          | 144 |
| Figure 1-80. GPIO Port A Pullup Disable (GPAPUD) Registers.....                             | 145 |
| Figure 1-81. GPIO Port B Pullup Disable (GPBPUD) Registers.....                             | 146 |
| Figure 1-82. GPIO Port A Data (GPADAT) Register.....                                        | 147 |
| Figure 1-83. GPIO Port B Data (GPBDAT) Register.....                                        | 148 |
| Figure 1-84. Analog I/O DAT (AIODAT) Register.....                                          | 149 |
| Figure 1-85. GPIO Port A Set, Clear and Toggle (GPASET, GPACLEAR, GPATOGGLE) Registers..... | 150 |
| Figure 1-86. GPIO Port B Set, Clear and Toggle (GPBSET, GPBCLEAR, GPBTOGGLE) Registers..... | 152 |
| Figure 1-87. Analog I/O Toggle (AIOSET, AIOCLEAR, AIOTOGGLE) Register.....                  | 154 |
| Figure 1-88. GPIO XINTn Interrupt Select (GPIOXINTnSEL) Registers.....                      | 155 |
| Figure 1-89. GPIO Low Power Mode Wakeup Select (GPIO_LPMSEL) Register.....                  | 156 |
| Figure 1-90. Device Configuration (DEVICECNF) Register.....                                 | 163 |
| Figure 1-91. Part ID Register.....                                                          | 164 |
| Figure 1-92. REVID Register.....                                                            | 165 |
| Figure 1-93. Overview: Multiplexing of Interrupts Using the PIE Block.....                  | 167 |
| Figure 1-94. Typical PIE/CPU Interrupt Response - INTx.y.....                               | 169 |
| Figure 1-95. Reset Flow Diagram.....                                                        | 171 |
| Figure 1-96. PIE Interrupt Sources and External Interrupts XINT1/XINT2/XINT3.....           | 172 |
| Figure 1-97. Multiplexed Interrupt Request Flow Diagram.....                                | 175 |

|                                                                                                                       |     |
|-----------------------------------------------------------------------------------------------------------------------|-----|
| Figure 1-98. PIECTRL Register (Address 0xCE0).....                                                                    | 182 |
| Figure 1-99. PIE Interrupt Acknowledge Register (PIEACK) Register (Address 0xCE1).....                                | 182 |
| Figure 1-100. PIEIFRx Register (x = 1 to 12).....                                                                     | 183 |
| Figure 1-101. PIEIERx Register (x = 1 to 12).....                                                                     | 184 |
| Figure 1-102. Interrupt Flag Register (IFR) — CPU Register.....                                                       | 185 |
| Figure 1-103. Interrupt Enable Register (IER) — CPU Register.....                                                     | 188 |
| Figure 1-104. Debug Interrupt Enable Register (DBGIER) — CPU Register.....                                            | 190 |
| Figure 1-105. External Interrupt <i>n</i> Control Register (XINT <sub>n</sub> CR).....                                | 192 |
| Figure 1-106. External Interrupt <i>n</i> Counter (XINT <sub>n</sub> CTR) (Address 7078h).....                        | 193 |
| Figure 1-107. BOR Configuration (BORCFG) Register.....                                                                | 194 |
| Figure 2-1. F2806x Memory Map of On-Chip ROM.....                                                                     | 196 |
| Figure 2-2. F2806xM/2806xF Memory Map of On-Chip ROM.....                                                             | 197 |
| Figure 2-3. Vector Table Map.....                                                                                     | 200 |
| Figure 2-4. Bootloader Flow Diagram.....                                                                              | 202 |
| Figure 2-5. Boot ROM Stack.....                                                                                       | 204 |
| Figure 2-6. Boot ROM Function Overview.....                                                                           | 206 |
| Figure 2-7. Bootloader Basic Transfer Procedure.....                                                                  | 217 |
| Figure 2-8. Overview of InitBoot Assembly Function.....                                                               | 218 |
| Figure 2-9. Overview of the SelectBootMode Function.....                                                              | 219 |
| Figure 2-10. Overview of Get_mode() Function.....                                                                     | 220 |
| Figure 2-11. Overview of CopyData Function.....                                                                       | 221 |
| Figure 2-12. Overview of SCI Bootloader Operation.....                                                                | 221 |
| Figure 2-13. Overview of SCI_Boot Function.....                                                                       | 222 |
| Figure 2-14. Overview of SCI_GetWordData Function.....                                                                | 223 |
| Figure 2-15. Overview of Parallel GPIO Bootloader Operation.....                                                      | 223 |
| Figure 2-16. Parallel GPIO Boot Loader Handshake Protocol.....                                                        | 224 |
| Figure 2-17. Parallel GPIO Mode Overview.....                                                                         | 225 |
| Figure 2-18. Parallel GPIO Mode - Host Transfer Flow.....                                                             | 225 |
| Figure 2-19. 8-Bit Parallel GetWord Function.....                                                                     | 226 |
| Figure 2-20. SPI Loader.....                                                                                          | 227 |
| Figure 2-21. Data Transfer From EEPROM Flow.....                                                                      | 229 |
| Figure 2-22. Overview of SPIA_GetWordData Function.....                                                               | 229 |
| Figure 2-23. EEPROM Device at Address 0x50.....                                                                       | 230 |
| Figure 2-24. Overview of I2C_Boot Function.....                                                                       | 231 |
| Figure 2-25. Random Read.....                                                                                         | 232 |
| Figure 2-26. Sequential Read.....                                                                                     | 232 |
| Figure 2-27. Overview of eCAN-A bootloader Operation.....                                                             | 233 |
| Figure 2-28. ExitBoot Procedure Flow.....                                                                             | 235 |
| Figure 3-1. Multiple ePWM Modules.....                                                                                | 244 |
| Figure 3-2. Submodules and Signal Connections for an ePWM Module.....                                                 | 245 |
| Figure 3-3. ePWM Submodules and Critical Internal Signal Interconnects.....                                           | 246 |
| Figure 3-4. Time-Base Submodule Block Diagram.....                                                                    | 251 |
| Figure 3-5. Time-Base Submodule Signals and Registers.....                                                            | 252 |
| Figure 3-6. Time-Base Frequency and Period.....                                                                       | 254 |
| Figure 3-7. Time-Base Counter Synchronization Scheme 1.....                                                           | 255 |
| Figure 3-8. Time-Base Up-Count Mode Waveforms.....                                                                    | 257 |
| Figure 3-9. Time-Base Down-Count Mode Waveforms.....                                                                  | 258 |
| Figure 3-10. Time-Base Up-Down-Count Waveforms, TBCTL[PHSDIR = 0] Count Down On Synchronization Event.....            | 258 |
| Figure 3-11. Time-Base Up-Down Count Waveforms, TBCTL[PHSDIR = 1] Count Up On Synchronization Event.....              | 259 |
| Figure 3-12. Counter-Compare Submodule.....                                                                           | 260 |
| Figure 3-13. Detailed View of the Counter-Compare Submodule.....                                                      | 260 |
| Figure 3-14. Counter-Compare Event Waveforms in Up-Count Mode.....                                                    | 263 |
| Figure 3-15. Counter-Compare Events in Down-Count Mode.....                                                           | 263 |
| Figure 3-16. Counter-Compare Events In Up-Down-Count Mode, TBCTL[PHSDIR = 0] Count Down On Synchronization Event..... | 264 |
| Figure 3-17. Counter-Compare Events In Up-Down-Count Mode, TBCTL[PHSDIR = 1] Count Up On Synchronization Event.....   | 264 |
| Figure 3-18. Action-Qualifier Submodule.....                                                                          | 265 |
| Figure 3-19. Action-Qualifier Submodule Inputs and Outputs.....                                                       | 266 |
| Figure 3-20. Possible Action-Qualifier Actions for EPWMxA and EPWMxB Outputs.....                                     | 267 |
| Figure 3-21. Up-Down-Count Mode Symmetrical Waveform.....                                                             | 270 |

|                                                                                                                                 |     |
|---------------------------------------------------------------------------------------------------------------------------------|-----|
| Figure 3-22. Up, Single Edge Asymmetric Waveform, With Independent Modulation on EPWMxA and EPWMxB—Active High.....             | 271 |
| Figure 3-23. Up, Single Edge Asymmetric Waveform With Independent Modulation on EPWMxA and EPWMxB—Active Low.....               | 272 |
| Figure 3-24. Up-Count, Pulse Placement Asymmetric Waveform With Independent Modulation on EPWMxA.....                           | 274 |
| Figure 3-25. Up-Down-Count, Dual Edge Symmetric Waveform, With Independent Modulation on EPWMxA and EPWMxB — Active Low.....    | 275 |
| Figure 3-26. Up-Down-Count, Dual Edge Symmetric Waveform, With Independent Modulation on EPWMxA and EPWMxB — Complementary..... | 276 |
| Figure 3-27. Up-Down-Count, Dual Edge Asymmetric Waveform, With Independent Modulation on EPWMxA—Active Low.....                | 277 |
| Figure 3-28. Dead-Band Submodule.....                                                                                           | 278 |
| Figure 3-29. Configuration Options for the Dead-Band Submodule.....                                                             | 279 |
| Figure 3-30. Dead-Band Waveforms for Typical Cases ( $0\% < \text{Duty} < 100\%$ ).....                                         | 281 |
| Figure 3-31. PWM-Chopper Submodule.....                                                                                         | 282 |
| Figure 3-32. PWM-Chopper Submodule Operational Details.....                                                                     | 283 |
| Figure 3-33. Simple PWM-Chopper Submodule Waveforms Showing Chopping Action Only.....                                           | 283 |
| Figure 3-34. PWM-Chopper Submodule Waveforms Showing the First Pulse and Subsequent Sustaining Pulses.....                      | 284 |
| Figure 3-35. PWM-Chopper Submodule Waveforms Showing the Pulse Width (Duty Cycle) Control of Sustaining Pulses..                | 285 |
| Figure 3-36. Trip-Zone Submodule.....                                                                                           | 286 |
| Figure 3-37. Trip-Zone Submodule Mode Control Logic.....                                                                        | 290 |
| Figure 3-38. Trip-Zone Submodule Interrupt Logic.....                                                                           | 291 |
| Figure 3-39. Event-Trigger Submodule.....                                                                                       | 292 |
| Figure 3-40. Event-Trigger Submodule Inter-Connectivity of ADC Start of Conversion.....                                         | 293 |
| Figure 3-41. Event-Trigger Submodule Showing Event Inputs and Prescaled Outputs.....                                            | 293 |
| Figure 3-42. Event-Trigger Interrupt Generator.....                                                                             | 295 |
| Figure 3-43. Event-Trigger SOCA Pulse Generator.....                                                                            | 296 |
| Figure 3-44. Event-Trigger SOCB Pulse Generator.....                                                                            | 296 |
| Figure 3-45. Digital-Compare Submodule High-Level Block Diagram.....                                                            | 297 |
| Figure 3-46. DCAEVT1 Event Triggering.....                                                                                      | 299 |
| Figure 3-47. DCAEVT2 Event Triggering.....                                                                                      | 299 |
| Figure 3-48. DCBEVT1 Event Triggering.....                                                                                      | 300 |
| Figure 3-49. DCBEVT2 Event Triggering.....                                                                                      | 300 |
| Figure 3-50. Event Filtering.....                                                                                               | 301 |
| Figure 3-51. Blanking Window Timing Diagram.....                                                                                | 302 |
| Figure 3-52. Simplified ePWM Module.....                                                                                        | 302 |
| Figure 3-53. EPWM1 Configured as a Typical Master, EPWM2 Configured as a Slave.....                                             | 303 |
| Figure 3-54. Control of Four Buck Stages. Here $F_{PWM1} \neq F_{PWM2} \neq F_{PWM3} \neq F_{PWM4}$ .....                       | 304 |
| Figure 3-55. Buck Waveforms for Figure 3-54 (Note: Only three bucks shown here).....                                            | 305 |
| Figure 3-56. Control of Four Buck Stages. (Note: $F_{PWM2} = N \times F_{PWM1}$ ).....                                          | 307 |
| Figure 3-57. Buck Waveforms for Figure 3-56 (Note: $F_{PWM2} = F_{PWM1}$ ).....                                                 | 308 |
| Figure 3-58. Control of Two Half-H Bridge Stages ( $F_{PWM2} = N \times F_{PWM1}$ ).....                                        | 310 |
| Figure 3-59. Half-H Bridge Waveforms for Figure 3-58 (Note: Here $F_{PWM2} = F_{PWM1}$ ).....                                   | 311 |
| Figure 3-60. Control of Dual 3-Phase Inverter Stages as Is Commonly Used in Motor Control.....                                  | 313 |
| Figure 3-61. 3-Phase Inverter Waveforms for Figure 3-60 (Only One Inverter Shown).....                                          | 314 |
| Figure 3-62. Configuring Two PWM Modules for Phase Control.....                                                                 | 316 |
| Figure 3-63. Timing Waveforms Associated With Phase Control Between 2 Modules.....                                              | 317 |
| Figure 3-64. Control of a 3-Phase Interleaved DC/DC Converter.....                                                              | 318 |
| Figure 3-65. 3-Phase Interleaved DC/DC Converter Waveforms for Figure 3-64.....                                                 | 319 |
| Figure 3-66. Controlling a Full-H Bridge Stage ( $F_{PWM2} = F_{PWM1}$ ).....                                                   | 321 |
| Figure 3-67. ZVS Full-H Bridge Waveforms.....                                                                                   | 322 |
| Figure 3-68. Peak Current Mode Control of a Buck Converter.....                                                                 | 324 |
| Figure 3-69. Peak Current Mode Control Waveforms for Figure 3-68.....                                                           | 324 |
| Figure 3-70. Control of Two Resonant Converter Stages.....                                                                      | 326 |
| Figure 3-71. H-Bridge LLC Resonant Converter PWM Waveforms.....                                                                 | 326 |
| Figure 3-72. Time-Base Control Register (TBCTL).....                                                                            | 328 |
| Figure 3-73. Time-Base Status Register (TBSTS).....                                                                             | 330 |
| Figure 3-74. Time-Base Phase High Resolution Register (TBPHSHR).....                                                            | 331 |
| Figure 3-75. Time-Base Phase Register (TBPHS).....                                                                              | 331 |
| Figure 3-76. Time-Base Counter Register (TBCTR).....                                                                            | 332 |
| Figure 3-77. Time-Base Period Register (TBPRD).....                                                                             | 332 |
| Figure 3-78. Time Base Period High Resolution Register (TBPRDHR).....                                                           | 333 |

|                                                                                      |     |
|--------------------------------------------------------------------------------------|-----|
| Figure 3-79. Time-Base Period High Resolution Mirror Register (TBPRDHRM).....        | 334 |
| Figure 3-80. Time-Base Period Mirror Register (TBPRDM).....                          | 335 |
| Figure 3-81. Counter-Compare Control (CMPCTL) Register.....                          | 336 |
| Figure 3-82. Compare A High Resolution (CMPAHR) Register.....                        | 337 |
| Figure 3-83. Counter-Compare A (CMPA) Register.....                                  | 338 |
| Figure 3-84. Counter-Compare B (CMPB) Register.....                                  | 339 |
| Figure 3-85. Compare A High-Resolution Mirror (CMPAHRM) Register.....                | 340 |
| Figure 3-86. Counter-Compare A Mirror (CMPAM) Register.....                          | 341 |
| Figure 3-87. Action-Qualifier Output A Control Register (AQCTLA).....                | 342 |
| Figure 3-88. Action-Qualifier Output B Control Register (AQCTLB).....                | 343 |
| Figure 3-89. Action-Qualifier Software Force Register (AQSFRC).....                  | 345 |
| Figure 3-90. Action-Qualifier Continuous Software Force Register (AQCSFRC).....      | 346 |
| Figure 3-91. Dead-Band Generator Control (DBCTL) Register.....                       | 347 |
| Figure 3-92. Dead-Band Generator Rising Edge Delay (DBRED) Register.....             | 349 |
| Figure 3-93. Dead-Band Generator Falling Edge Delay (DBFED) Register.....            | 349 |
| Figure 3-94. Trip-Zone Select Register (TZSEL).....                                  | 350 |
| Figure 3-95. Trip-Zone Digital Compare Event Select Register (TZDCSEL).....          | 351 |
| Figure 3-96. Trip-Zone Control Register (TZCTL).....                                 | 353 |
| Figure 3-97. Trip-Zone Enable Interrupt Register (TZEINT).....                       | 354 |
| Figure 3-98. Trip-Zone Flag Register (TZFLG).....                                    | 355 |
| Figure 3-99. Trip-Zone Clear Register (TZCLR).....                                   | 357 |
| Figure 3-100. Trip-Zone Force Register (TZFRC).....                                  | 358 |
| Figure 3-101. Event-Trigger Selection Register (ETSEL).....                          | 359 |
| Figure 3-102. Event-Trigger Prescale Register (ETPS).....                            | 360 |
| Figure 3-103. Event-Trigger Flag Register (ETFLG).....                               | 362 |
| Figure 3-104. Event-Trigger Clear Register (ETCLR).....                              | 363 |
| Figure 3-105. Event-Trigger Force Register (ETFRC).....                              | 364 |
| Figure 3-106. PWM-Chopper Control (PCCTL) Register.....                              | 365 |
| Figure 3-107. Digital Compare Trip Select (DCTRIPSEL) Register.....                  | 367 |
| Figure 3-108. Digital Compare A Control (DCACTL) Register.....                       | 369 |
| Figure 3-109. Digital Compare B Control (DCBCTL) Register.....                       | 370 |
| Figure 3-110. Digital Compare Filter Control (DCFCTL) Register.....                  | 371 |
| Figure 3-111. Digital Compare Capture Control (DCCAPCTL) Register.....               | 372 |
| Figure 3-112. Digital Compare Filter Offset (DCFOFFSET) Register.....                | 373 |
| Figure 3-113. Digital Compare Filter Offset Counter (DCFOFFSETCNT) Register.....     | 373 |
| Figure 3-114. Digital Compare Filter Window (DCFWINDOW) Register.....                | 374 |
| Figure 3-115. Digital Compare Filter Window Counter (DCFWINDOWCNT) Register.....     | 374 |
| Figure 3-116. Digital Compare Counter Capture (DCCAP) Register.....                  | 375 |
| Figure 4-1. Resolution Calculations for Conventionally Generated PWM.....            | 378 |
| Figure 4-2. Operating Logic Using MEP.....                                           | 379 |
| Figure 4-3. HRPWM Extension Registers and Memory Configuration.....                  | 380 |
| Figure 4-4. HRPWM System Interface.....                                              | 381 |
| Figure 4-5. HRPWM Block Diagram.....                                                 | 382 |
| Figure 4-6. Required PWM Waveform for a Requested Duty = 30.0%.....                  | 384 |
| Figure 4-7. Low % Duty Cycle Range Limitation Example (HRPCTL[HRPE] = 0).....        | 387 |
| Figure 4-8. High % Duty Cycle Range Limitation Example (HRPCTL[HRPE] = 0).....       | 388 |
| Figure 4-9. Up-Count Duty Cycle Range Limitation Example (HRPCTL[HRPE]=1).....       | 388 |
| Figure 4-10. Up-Down Count Duty Cycle Range Limitation Example (HRPCTL[HRPE]=1)..... | 389 |
| Figure 4-11. Simple Buck Controlled Converter Using a Single PWM.....                | 393 |
| Figure 4-12. PWM Waveform Generated for Simple Buck Controlled Converter.....        | 393 |
| Figure 4-13. Simple Reconstruction Filter for a PWM Based DAC.....                   | 395 |
| Figure 4-14. PWM Waveform Generated for the PWM DAC Function.....                    | 395 |
| Figure 4-15. HRPWM Configuration (HRCNFG) Register.....                              | 401 |
| Figure 4-16. High Resolution Micro Step (HRMSTEP) Register.....                      | 402 |
| Figure 4-17. High Resolution Period Control (HRPCTL) Register.....                   | 404 |
| Figure 5-1. HRCAP Module System Block Diagram.....                                   | 406 |
| Figure 5-2. HRCAP Block Diagram.....                                                 | 407 |
| Figure 5-3. HCCAPCLK Generation.....                                                 | 408 |
| Figure 5-4. HCCOUNTER Behavior During High Pulse Width Capture.....                  | 408 |
| Figure 5-5. Rise versus Fall Capture Events.....                                     | 409 |
| Figure 5-6. High Pulse Width Normal Mode Capture.....                                | 410 |



|                                                                                                         |     |
|---------------------------------------------------------------------------------------------------------|-----|
| Figure 5-7. Low Pulse Width Normal Mode Capture.....                                                    | 410 |
| Figure 5-8. HRCAP High-Resolution Mode Operating Logic.....                                             | 411 |
| Figure 5-9. Interrupts in HRCAP Module.....                                                             | 412 |
| Figure 5-10. LowPulseWidth0 Capture on RISE and FALL Events.....                                        | 415 |
| Figure 5-11. HighPulseWidth0/1 Capture on RISE and FALL Events.....                                     | 416 |
| Figure 5-12. PeriodWidthRise0 and PeriodWidthFall0 Capture on RISE and FALL Events.....                 | 418 |
| Figure 5-13. HRCAP Control Register (HCCTL).....                                                        | 422 |
| Figure 5-14. HRCAP Interrupt Flag Register (HCIFR).....                                                 | 423 |
| Figure 5-15. HRCAP Interrupt Clear Register (HCICLR).....                                               | 424 |
| Figure 5-16. HRCAP Interrupt Force Register (HCIFRC).....                                               | 425 |
| Figure 5-17. HRCAP Counter Register (HCCOUNTER).....                                                    | 426 |
| Figure 5-18. HRCAP Capture Counter On Rising Edge 0 Register (HCCAPCNTRISE0).....                       | 427 |
| Figure 5-19. HRCAP Capture Counter On Falling Edge 0 Register (HCCAPCNTFALL0).....                      | 427 |
| Figure 5-20. HRCAP Capture Counter On Rising Edge 1 Register (HCCAPCNTRISE1).....                       | 428 |
| Figure 5-21. HRCAP Capture Counter On Falling Edge 1 Register (HCCAPCNTFALL1).....                      | 428 |
| Figure 6-1. Multiple eCAP Modules In A C28x System.....                                                 | 431 |
| Figure 6-2. Capture and APWM Modes of Operation.....                                                    | 432 |
| Figure 6-3. Counter Compare and PRD Effects on the eCAP Output in APWM Mode.....                        | 433 |
| Figure 6-4. eCAP Block Diagram.....                                                                     | 434 |
| Figure 6-5. Event Prescale Control.....                                                                 | 435 |
| Figure 6-6. Prescale Function Waveforms.....                                                            | 435 |
| Figure 6-7. Details of the Continuous/One-shot Block.....                                               | 436 |
| Figure 6-8. Details of the Counter and Synchronization Block.....                                       | 437 |
| Figure 6-9. Interrupts in eCAP Module.....                                                              | 439 |
| Figure 6-10. PWM Waveform Details Of APWM Mode Operation.....                                           | 440 |
| Figure 6-11. Time-Base Frequency and Period Calculation.....                                            | 441 |
| Figure 6-12. Capture Sequence for Absolute Time-stamp and Rising Edge Detect.....                       | 442 |
| Figure 6-13. Capture Sequence for Absolute Time-stamp With Rising and Falling Edge Detect.....          | 443 |
| Figure 6-14. Capture Sequence for Delta Mode Time-stamp and Rising Edge Detect.....                     | 444 |
| Figure 6-15. Capture Sequence for Delta Mode Time-stamp With Rising and Falling Edge Detect.....        | 445 |
| Figure 6-16. PWM Waveform Details of APWM Mode Operation.....                                           | 446 |
| Figure 6-17. TSCTR Register.....                                                                        | 448 |
| Figure 6-18. CTRPHS Register.....                                                                       | 448 |
| Figure 6-19. CAP1 Register.....                                                                         | 449 |
| Figure 6-20. CAP2 Register.....                                                                         | 449 |
| Figure 6-21. CAP3 Register.....                                                                         | 450 |
| Figure 6-22. CAP4 Register.....                                                                         | 450 |
| Figure 6-23. ECCTL1 Register.....                                                                       | 451 |
| Figure 6-24. ECCTL2 Register.....                                                                       | 453 |
| Figure 6-25. ECEINT Register.....                                                                       | 455 |
| Figure 6-26. ECFLG Register.....                                                                        | 457 |
| Figure 6-27. ECCLR Register.....                                                                        | 459 |
| Figure 6-28. ECFRC Register.....                                                                        | 460 |
| Figure 7-1. Optical Encoder Disk.....                                                                   | 462 |
| Figure 7-2. QEP Encoder Output Signal for Forward/Reverse Movement.....                                 | 462 |
| Figure 7-3. Index Pulse Example.....                                                                    | 463 |
| Figure 7-4. Functional Block Diagram of the eQEP Peripheral.....                                        | 466 |
| Figure 7-5. Functional Block Diagram of Decoder Unit.....                                               | 468 |
| Figure 7-6. Quadrature Decoder State Machine.....                                                       | 469 |
| Figure 7-7. Quadrature-clock and Direction Decoding.....                                                | 470 |
| Figure 7-8. Position Counter Reset by Index Pulse for 1000 Line Encoder (QPOS MAX = 3999 or 0xF9F)..... | 472 |
| Figure 7-9. Position Counter Underflow/Overflow (QPOS MAX = 4).....                                     | 473 |
| Figure 7-10. Software Index Marker for 1000-line Encoder (QEPCTL[IEL] = 1).....                         | 475 |
| Figure 7-11. Strobe Event Latch (QEPCTL[SEL] = 1).....                                                  | 475 |
| Figure 7-12. eQEP Position-compare Unit.....                                                            | 476 |
| Figure 7-13. eQEP Position-compare Event Generation Points.....                                         | 477 |
| Figure 7-14. eQEP Position-compare Sync Output Pulse Stretcher.....                                     | 477 |
| Figure 7-15. eQEP Edge Capture Unit.....                                                                | 479 |
| Figure 7-16. Unit Position Event for Low Speed Measurement (QCAPCTL[UUPS] = 0010).....                  | 480 |
| Figure 7-17. eQEP Edge Capture Unit - Timing Details.....                                               | 480 |
| Figure 7-18. eQEP Watchdog Timer.....                                                                   | 482 |

|                                                                                |     |
|--------------------------------------------------------------------------------|-----|
| Figure 7-19. eQEP Unit Timer Base.....                                         | 482 |
| Figure 7-20. eQEP Interrupt Generation.....                                    | 483 |
| Figure 7-21. QPOSCNT Register.....                                             | 486 |
| Figure 7-22. QPOSINIT Register.....                                            | 486 |
| Figure 7-23. QPOSMAX Register.....                                             | 487 |
| Figure 7-24. QPOSCMP Register.....                                             | 487 |
| Figure 7-25. QPOSILAT Register.....                                            | 487 |
| Figure 7-26. QPOSSLAT Register.....                                            | 488 |
| Figure 7-27. QPOSLAT Register.....                                             | 488 |
| Figure 7-28. QUTMR Register.....                                               | 488 |
| Figure 7-29. QUPRD Register.....                                               | 489 |
| Figure 7-30. QWDTMR Register.....                                              | 489 |
| Figure 7-31. QWDPRD Register.....                                              | 490 |
| Figure 7-32. QDECCTL Register.....                                             | 491 |
| Figure 7-33. QEPCTL Register.....                                              | 493 |
| Figure 7-34. QCAPCTL Register.....                                             | 496 |
| Figure 7-35. QPOSCTL Register.....                                             | 497 |
| Figure 7-36. QEINT Register.....                                               | 498 |
| Figure 7-37. QFLG Register.....                                                | 500 |
| Figure 7-38. QCLR Register.....                                                | 502 |
| Figure 7-39. QFRC Register.....                                                | 504 |
| Figure 7-40. QEPSTS Register.....                                              | 506 |
| Figure 7-41. QCTMR Register.....                                               | 507 |
| Figure 7-42. QCPRD Register.....                                               | 507 |
| Figure 7-43. QCTMRLAT Register.....                                            | 508 |
| Figure 7-44. QCPRDLAT Register.....                                            | 508 |
| Figure 8-1. ADC Block Diagram.....                                             | 512 |
| Figure 8-2. SOC Block Diagram.....                                             | 513 |
| Figure 8-3. ADCINx Input Model.....                                            | 515 |
| Figure 8-4. ONESHOT Single Conversion.....                                     | 519 |
| Figure 8-5. Round Robin Priority Example.....                                  | 521 |
| Figure 8-6. High Priority Example.....                                         | 522 |
| Figure 8-7. Interrupt Structure.....                                           | 524 |
| Figure 8-8. Timing Example For Sequential Mode / Late Interrupt Pulse.....     | 527 |
| Figure 8-9. Timing Example For Sequential Mode / Early Interrupt Pulse.....    | 528 |
| Figure 8-10. Timing Example For Simultaneous Mode / Late Interrupt Pulse.....  | 529 |
| Figure 8-11. Timing Example For Simultaneous Mode / Early Interrupt Pulse..... | 530 |
| Figure 8-12. Timing Example for NONOVERLAP Mode.....                           | 531 |
| Figure 8-13. Temperature Sensor Transfer Function.....                         | 532 |
| Figure 8-14. ADC Control Register 1 (ADCCTL1).....                             | 535 |
| Figure 8-15. ADC Control Register 2 (ADCCTL2).....                             | 537 |
| Figure 8-16. ADC Interrupt Flag Register (ADCINTFLG).....                      | 538 |
| Figure 8-17. ADC Interrupt Flag Clear Register (ADCINTFLGCLR).....             | 539 |
| Figure 8-18. ADC Interrupt Overflow Register (ADCINTOVF).....                  | 540 |
| Figure 8-19. ADC Interrupt Overflow Clear Register (ADCINTOVFCLR).....         | 541 |
| Figure 8-20. Interrupt Select 1 and 2 Register (INTSEL1N2).....                | 542 |
| Figure 8-21. Interrupt Select 3 and 4 Register (INTSEL3N4).....                | 542 |
| Figure 8-22. Interrupt Select 5 and 6 Register (INTSEL5N6).....                | 542 |
| Figure 8-23. Interrupt Select 7 and 8 Register (INTSEL7N8).....                | 543 |
| Figure 8-24. Interrupt Select 9 and 10 Register (INTSEL9N10).....              | 543 |
| Figure 8-25. ADC Start of Conversion Priority Control Register (SOCPRCTL)..... | 545 |
| Figure 8-26. ADC Sample Mode Register (ADCSAMPLEMODE).....                     | 547 |
| Figure 8-27. ADC Interrupt Trigger SOC Select 1 Register (ADCINTSOCSEL1).....  | 549 |
| Figure 8-28. ADC Interrupt Trigger SOC Select 2 Register (ADCINTSOCSEL2).....  | 549 |
| Figure 8-29. ADC SOC Flag 1 Register (ADCSOCFLG1).....                         | 550 |
| Figure 8-30. ADC SOC Force 1 Register (ADCSOCFRC1).....                        | 551 |
| Figure 8-31. ADC SOC Overflow 1 Register (ADCSOCOVF1).....                     | 552 |
| Figure 8-32. ADC SOC Overflow Clear 1 Register (ADCSOCOVFCLR1).....            | 552 |
| Figure 8-33. ADC SOC0-SOC15 Control Registers (ADCSOCxCTL).....                | 553 |
| Figure 8-34. ADC Reference/Gain Trim Register (ADCREFTRIM).....                | 556 |
| Figure 8-35. ADC Offset Trim Register (ADCOFFTRIM).....                        | 556 |



|                                                                                                                            |     |
|----------------------------------------------------------------------------------------------------------------------------|-----|
| Figure 8-36. Comparator Hysteresis Control Register (COMPHYSTCTL).....                                                     | 557 |
| Figure 8-37. ADC Revision Register (ADCREV).....                                                                           | 558 |
| Figure 8-38. ADC RESULT0-RESULT15 Registers (ADCRESULTx).....                                                              | 558 |
| Figure 9-1. Comparator Block Diagram.....                                                                                  | 560 |
| Figure 9-2. Comparator.....                                                                                                | 561 |
| Figure 9-3. Ramp Generator Block Diagram.....                                                                              | 562 |
| Figure 9-4. Ramp Generator Behavior.....                                                                                   | 563 |
| Figure 9-5. Comparator Control (COMPCTL) Register.....                                                                     | 565 |
| Figure 9-6. Compare Output Status (COMPSTS) Register.....                                                                  | 566 |
| Figure 9-7. DAC Control (DACCTL) Register.....                                                                             | 566 |
| Figure 9-8. DAC Value (DACVAL) Register.....                                                                               | 567 |
| Figure 9-9. Ramp Generator Maximum Reference Active (RAMPMAXREF_ACTIVE) Register.....                                      | 567 |
| Figure 9-10. Ramp Generator Maximum Reference Shadow (RAMPMAXREF_SHDW) Register.....                                       | 567 |
| Figure 9-11. Ramp Generator Decrement Value Active (RAMPDECVAL_ACTIVE) Register.....                                       | 568 |
| Figure 9-12. Ramp Generator Decrement Value Shadow (RAMPDECVAL_SHDW) Register.....                                         | 568 |
| Figure 9-13. Ramp Generator Status (RAMPSTS) Register.....                                                                 | 568 |
| Figure 10-1. CLA (Type 0) Block Diagram.....                                                                               | 571 |
| Figure 10-2. Task Interrupt Vector (MVECT1/2/3/4/5/6/7/8) Register.....                                                    | 710 |
| Figure 10-3. Control Register (MCTL).....                                                                                  | 711 |
| Figure 10-4. Memory Configuration Register (MMEMCFG).....                                                                  | 712 |
| Figure 10-5. CLA Peripheral Interrupt Source Select 1 Register (MPISRCSEL1).....                                           | 714 |
| Figure 10-6. Interrupt Flag Register (MIFR).....                                                                           | 716 |
| Figure 10-7. Interrupt Overflow Flag Register (MIOVF).....                                                                 | 717 |
| Figure 10-8. Interrupt Force Register (MIFRC).....                                                                         | 718 |
| Figure 10-9. Interrupt Flag Clear Register (MICLR).....                                                                    | 719 |
| Figure 10-10. Interrupt Overflow Flag Clear Register (MICLROVF).....                                                       | 720 |
| Figure 10-11. Interrupt Enable Register (MIER).....                                                                        | 721 |
| Figure 10-12. Interrupt Run Status Register (MIRUN).....                                                                   | 723 |
| Figure 10-13. Program Counter (MPC).....                                                                                   | 724 |
| Figure 10-14. CLA Status Register (MSTF).....                                                                              | 724 |
| Figure 11-1. DMA Block Diagram.....                                                                                        | 729 |
| Figure 11-2. Peripheral Interrupt Trigger Input Diagram.....                                                               | 730 |
| Figure 11-3. 4-Stage Pipeline DMA Transfer.....                                                                            | 732 |
| Figure 11-4. 4-Stage Pipeline With One Read Stall (McBSP as source).....                                                   | 732 |
| Figure 11-5. DMA State Diagram.....                                                                                        | 738 |
| Figure 11-6. Overrun Detection Logic.....                                                                                  | 739 |
| Figure 11-7. DMA Control Register (DMACTRL).....                                                                           | 741 |
| Figure 11-8. Debug Control Register (DEBUGCTRL).....                                                                       | 743 |
| Figure 11-9. Revision Register (REVISION).....                                                                             | 743 |
| Figure 11-10. Priority Control Register 1 (PRIORITYCTRL1).....                                                             | 744 |
| Figure 11-11. Priority Status Register (PRIORITYSTAT).....                                                                 | 745 |
| Figure 11-12. Mode Register (MODE).....                                                                                    | 746 |
| Figure 11-13. Control Register (CONTROL).....                                                                              | 748 |
| Figure 11-14. Burst Size Register (BURST_SIZE).....                                                                        | 750 |
| Figure 11-15. Burst Count Register (BURST_COUNT).....                                                                      | 750 |
| Figure 11-16. Source Burst Step Size Register (SRC_BURST_STEP).....                                                        | 751 |
| Figure 11-17. Destination Burst Step Register Size (DST_BURST_STEP).....                                                   | 752 |
| Figure 11-18. Transfer Size Register (TRANSFER_SIZE).....                                                                  | 752 |
| Figure 11-19. Transfer Count Register (TRANSFER_COUNT).....                                                                | 753 |
| Figure 11-20. Source Transfer Step Size Register (SRC_TRANSFER_STEP).....                                                  | 753 |
| Figure 11-21. Destination Transfer Step Size Register (DST_TRANSFER_STEP).....                                             | 754 |
| Figure 11-22. Source/Destination Wrap Size Register (SRC/DST_WRAP_SIZE).....                                               | 755 |
| Figure 11-23. Source/Destination Wrap Count Register (SRC/DST_WRAP_COUNT).....                                             | 755 |
| Figure 11-24. Source/Destination Wrap Step Size Registers (SRC/DST_WRAP_STEP).....                                         | 756 |
| Figure 11-25. Shadow Source Begin and Current Address Pointer Registers (SRC_BEG_ADDR_SHADOW/<br>DST_BEG_ADDR_SHADOW)..... | 757 |
| Figure 11-26. Active Source Begin and Current Address Pointer Registers (SRC_BEG_ADDR/DST_BEG_ADDR).....                   | 757 |
| Figure 11-27. Shadow Destination Begin and Current Address Pointer Registers (SRC_ADDR_SHADOW/<br>DST_ADDR_SHADOW).....    | 758 |
| Figure 11-28. Active Destination Begin and Current Address Pointer Registers (SRC_ADDR/DST_ADDR).....                      | 758 |
| Figure 12-1. SPI CPU Interface.....                                                                                        | 761 |

|                                                                                                         |     |
|---------------------------------------------------------------------------------------------------------|-----|
| Figure 12-2. SPI Interrupt Flags and Enable Logic Generation.....                                       | 764 |
| Figure 12-3. SPI DMA Trigger Diagram.....                                                               | 765 |
| Figure 12-4. SPI Master/Slave Connection.....                                                           | 766 |
| Figure 12-5. Serial Peripheral Interface Block Diagram.....                                             | 767 |
| Figure 12-6. SPICLK Signal Options.....                                                                 | 771 |
| Figure 12-7. SPI: SPICLK-LSPCLK Characteristic When (BRR + 1) is Odd, BRR > 3, and CLKPOLARITY = 1..... | 772 |
| Figure 12-8. SPI 3-wire Master Mode.....                                                                | 773 |
| Figure 12-9. SPI 3-wire Slave Mode.....                                                                 | 774 |
| Figure 12-10. Five Bits per Character.....                                                              | 776 |
| Figure 12-11. SPI Digital Audio Receiver Configuration Using Two SPIs.....                              | 778 |
| Figure 12-12. Standard Right-Justified Digital Audio Data Format.....                                   | 779 |
| Figure 12-13. SPI Configuration Control Register (SPICCR).....                                          | 781 |
| Figure 12-14. SPI Operation Control (SPICTL) Register.....                                              | 784 |
| Figure 12-15. SPI Status (SPISTS) Register.....                                                         | 786 |
| Figure 12-16. SPI Baud Rate Register (SPIBRR).....                                                      | 788 |
| Figure 12-17. SPI Emulation Buffer (SPIRXEMU) Register.....                                             | 789 |
| Figure 12-18. SPI Serial Input Buffer (SPIRXBUF) Register.....                                          | 790 |
| Figure 12-19. SPI Serial Output Buffer (SPITXBUF) Register.....                                         | 790 |
| Figure 12-20. SPI Serial Data (SPIDAT) Register.....                                                    | 791 |
| Figure 12-21. SPI FIFO Transmit (SPIFFTX) Register.....                                                 | 792 |
| Figure 12-22. SPI FIFO Receive (SPIFFRX) Register.....                                                  | 794 |
| Figure 12-23. SPI FIFO Control (SPIFFCT) Register.....                                                  | 796 |
| Figure 12-24. SPI Priority Control (SPIPRI) Register.....                                               | 797 |
| Figure 13-1. SCI CPU Interface.....                                                                     | 800 |
| Figure 13-2. Serial Communications Interface (SCI) Module Block Diagram.....                            | 802 |
| Figure 13-3. Typical SCI Data Frame Formats.....                                                        | 803 |
| Figure 13-4. Idle-Line Multiprocessor Communication Format.....                                         | 805 |
| Figure 13-5. Double-Buffered WUT and TXSHF.....                                                         | 806 |
| Figure 13-6. Address-Bit Multiprocessor Communication Format.....                                       | 807 |
| Figure 13-7. SCI Asynchronous Communications Format.....                                                | 808 |
| Figure 13-8. SCI RX Signals in Communication Modes.....                                                 | 808 |
| Figure 13-9. SCI TX Signals in Communications Mode.....                                                 | 809 |
| Figure 13-10. SCI FIFO Interrupt Flags and Enable Logic.....                                            | 812 |
| Figure 13-11. SCI Communications Control Register (SCICCR).....                                         | 816 |
| Figure 13-12. SCI Control Register 1 (SCICTL1).....                                                     | 818 |
| Figure 13-13. SCI Baud Rate (high) (SCIHBAUD) Register.....                                             | 820 |
| Figure 13-14. SCI Baud Rate (low) (SCILBAUD) Register.....                                              | 821 |
| Figure 13-15. SCI Control Register 2 (SCICTL2).....                                                     | 821 |
| Figure 13-16. SCI Receive Status (SCIRXST) Register.....                                                | 823 |
| Figure 13-17. SCI Receive Emulation Buffer (SCIRXEMU) Register.....                                     | 825 |
| Figure 13-18. SCI Receive Data Buffer (SCIRXBUF) Register.....                                          | 826 |
| Figure 13-19. SCI Transmit Data Buffer (SCITXBUF) Register.....                                         | 827 |
| Figure 13-20. SCI FIFO Transmit (SCIFFTX) Register.....                                                 | 827 |
| Figure 13-21. SCI FIFO Receive (SCIFFRX) Register.....                                                  | 829 |
| Figure 13-22. SCI FIFO Control (SCIFFCT) Register.....                                                  | 831 |
| Figure 13-23. SCI Priority Control (SCIPRI) Register.....                                               | 832 |
| Figure 14-1. Multiple I2C Modules Connected.....                                                        | 834 |
| Figure 14-2. I2C Module Conceptual Block Diagram.....                                                   | 837 |
| Figure 14-3. Clocking Diagram for the I2C Module.....                                                   | 837 |
| Figure 14-4. Roles of the Clock Divide-Down Values (ICCL and ICCH).....                                 | 838 |
| Figure 14-5. Bit Transfer on the I2C bus.....                                                           | 839 |
| Figure 14-6. I2C Slave TX / RX Flowchart.....                                                           | 841 |
| Figure 14-7. I2C Master TX / RX Flowchart.....                                                          | 842 |
| Figure 14-8. I2C Module START and STOP Conditions.....                                                  | 843 |
| Figure 14-9. I2C Module Data Transfer (7-Bit Addressing with 8-bit Data Configuration Shown).....       | 845 |
| Figure 14-10. I2C Module 7-Bit Addressing Format (FDF = 0, XA = 0 in I2CMDR).....                       | 845 |
| Figure 14-11. I2C Module 10-Bit Addressing Format (FDF = 0, XA = 1 in I2CMDR).....                      | 846 |
| Figure 14-12. I2C Module Free Data Format (FDF = 1 in I2CMDR).....                                      | 846 |
| Figure 14-13. Repeated START Condition (in This Case, 7-Bit Addressing Format).....                     | 847 |
| Figure 14-14. Synchronization of Two I2C Clock Generators During Arbitration.....                       | 847 |
| Figure 14-15. Arbitration Procedure Between Two Master-Transmitters.....                                | 848 |

|                                                                                                    |     |
|----------------------------------------------------------------------------------------------------|-----|
| Figure 14-16. Pin Diagram Showing the Effects of the Digital Loopback Mode (DLB) Bit.....          | 849 |
| Figure 14-17. Enable Paths of the I2C Interrupt Requests.....                                      | 852 |
| Figure 14-18. I2C FIFO Interrupt.....                                                              | 853 |
| Figure 14-19. I2C Own Address Register (I2COAR).....                                               | 856 |
| Figure 14-20. I2C Interrupt Enable Register (I2CIER).....                                          | 857 |
| Figure 14-21. I2C Status Register (I2CSTR).....                                                    | 858 |
| Figure 14-22. I2C Clock Low-time Divider (I2CCLKL) Register.....                                   | 862 |
| Figure 14-23. I2C Clock High-time Divider (I2CCLKH) Register.....                                  | 862 |
| Figure 14-24. I2C Data Count (I2CCNT) Register.....                                                | 863 |
| Figure 14-25. I2C Data Receive Register (I2CDRR).....                                              | 864 |
| Figure 14-26. I2C Slave Address Register (I2CSAR).....                                             | 865 |
| Figure 14-27. I2C Data Transmit Register (I2CDXR).....                                             | 866 |
| Figure 14-28. I2C Mode Register (I2CMDR).....                                                      | 867 |
| Figure 14-29. I2C Interrupt Source (I2CISRC) Register.....                                         | 871 |
| Figure 14-30. I2C Extended Mode Register (I2CEMDR).....                                            | 872 |
| Figure 14-31. I2C Prescaler (I2CPSC) Register.....                                                 | 873 |
| Figure 14-32. I2C FIFO Transmit (I2CFFTX) Register.....                                            | 874 |
| Figure 14-33. I2C FIFO Receive (I2CFFRX) Register.....                                             | 876 |
| Figure 15-1. Conceptual Block Diagram of the McBSP.....                                            | 880 |
| Figure 15-2. McBSP Data Transfer Paths.....                                                        | 881 |
| Figure 15-3. Companding Processes.....                                                             | 882 |
| Figure 15-4. $\mu$ -Law Transmit Data Companding Format.....                                       | 882 |
| Figure 15-5. A-Law Transmit Data Companding Format.....                                            | 882 |
| Figure 15-6. Two Methods by Which the McBSP Can Compand Internal Data.....                         | 883 |
| Figure 15-7. Example - Clock Signal Control of Bit Transfer Timing.....                            | 883 |
| Figure 15-8. McBSP Operating at Maximum Packet Frequency.....                                      | 885 |
| Figure 15-9. Single-Phase Frame for a McBSP Data Transfer.....                                     | 886 |
| Figure 15-10. Dual-Phase Frame for a McBSP Data Transfer.....                                      | 886 |
| Figure 15-11. Implementing the AC97 Standard With a Dual-Phase Frame.....                          | 887 |
| Figure 15-12. Timing of an AC97-Standard Data Transfer Near Frame Synchronization.....             | 887 |
| Figure 15-13. McBSP Reception Physical Data Path.....                                              | 887 |
| Figure 15-14. McBSP Reception Signal Activity.....                                                 | 888 |
| Figure 15-15. McBSP Transmission Physical Data Path.....                                           | 889 |
| Figure 15-16. McBSP Transmission Signal Activity.....                                              | 889 |
| Figure 15-17. Conceptual Block Diagram of the Sample Rate Generator.....                           | 890 |
| Figure 15-18. Possible Inputs to the Sample Rate Generator and the Polarity Bits.....              | 892 |
| Figure 15-19. CLKG Synchronization and FSG Generation When GSYNC = 1 and CLKGDV = 1.....           | 894 |
| Figure 15-20. CLKG Synchronization and FSG Generation When GSYNC = 1 and CLKGDV = 3.....           | 895 |
| Figure 15-21. Overrun in the McBSP Receiver.....                                                   | 897 |
| Figure 15-22. Overrun Prevented in the McBSP Receiver.....                                         | 898 |
| Figure 15-23. Possible Responses to Receive Frame-Synchronization Pulses.....                      | 898 |
| Figure 15-24. An Unexpected Frame-Synchronization Pulse During a McBSP Reception.....              | 899 |
| Figure 15-25. Proper Positioning of Frame-Synchronization Pulses.....                              | 900 |
| Figure 15-26. Data in the McBSP Transmitter Overwritten and Thus Not Transmitted.....              | 900 |
| Figure 15-27. Underflow During McBSP Transmission.....                                             | 901 |
| Figure 15-28. Underflow Prevented in the McBSP Transmitter.....                                    | 902 |
| Figure 15-29. Possible Responses to Transmit Frame-Synchronization Pulses.....                     | 902 |
| Figure 15-30. An Unexpected Frame-Synchronization Pulse During a McBSP Transmission.....           | 903 |
| Figure 15-31. Proper Positioning of Frame-Synchronization Pulses.....                              | 904 |
| Figure 15-32. Alternating Between the Channels of Partition A and the Channels of Partition B..... | 906 |
| Figure 15-33. Reassigning Channel Blocks Throughout a McBSP Data Transfer.....                     | 907 |
| Figure 15-34. McBSP Data Transfer in the 8-Partition Mode.....                                     | 908 |
| Figure 15-35. Activity on McBSP Pins for the Possible Values of XMCM.....                          | 911 |
| Figure 15-36. Typical SPI Interface.....                                                           | 912 |
| Figure 15-37. SPI Transfer With CLKSTP = 10b (No Clock Delay), CLKXP = 0, and CLKRP = 0.....       | 914 |
| Figure 15-38. SPI Transfer With CLKSTP = 11b (Clock Delay), CLKXP = 0, CLKRP = 1.....              | 914 |
| Figure 15-39. SPI Transfer With CLKSTP = 10b (No Clock Delay), CLKXP = 1, and CLKRP = 0.....       | 915 |
| Figure 15-40. SPI Transfer With CLKSTP = 11b (Clock Delay), CLKXP = 1, CLKRP = 1.....              | 915 |
| Figure 15-41. SPI Interface with McBSP Used as Master.....                                         | 917 |
| Figure 15-42. SPI Interface With McBSP Used as Slave.....                                          | 918 |
| Figure 15-43. Unexpected Frame-Synchronization Pulse With (R/X)FIG = 0.....                        | 925 |

|                                                                                                                    |      |
|--------------------------------------------------------------------------------------------------------------------|------|
| Figure 15-44. Unexpected Frame-Synchronization Pulse With (R/X)FIG = 1.....                                        | 925  |
| Figure 15-45. Companding Processes for Reception and for Transmission.....                                         | 926  |
| Figure 15-46. Range of Programmable Data Delay.....                                                                | 927  |
| Figure 15-47. 2-Bit Data Delay Used to Skip a Framing Bit.....                                                     | 928  |
| Figure 15-48. Data Clocked Externally Using a Rising Edge and Sampled by the McBSP Receiver on a Falling Edge..... | 933  |
| Figure 15-49. Frame of Period 16 CLKG Periods and Active Width of 2 CLKG Periods.....                              | 933  |
| Figure 15-50. Data Clocked Externally Using a Rising Edge and Sampled by the McBSP Receiver on a Falling Edge..... | 936  |
| Figure 15-51. Unexpected Frame-Synchronization Pulse With (R/X) FIG = 0.....                                       | 944  |
| Figure 15-52. Unexpected Frame-Synchronization Pulse With (R/X) FIG = 1.....                                       | 944  |
| Figure 15-53. Companding Processes for Reception and for Transmission.....                                         | 945  |
| Figure 15-54. $\mu$ -Law Transmit Data Companding Format.....                                                      | 945  |
| Figure 15-55. A-Law Transmit Data Companding Format.....                                                           | 946  |
| Figure 15-56. Range of Programmable Data Delay.....                                                                | 946  |
| Figure 15-57. 2-Bit Data Delay Used to Skip a Framing Bit.....                                                     | 947  |
| Figure 15-58. Data Clocked Externally Using a Rising Edge and Sampled by the McBSP Receiver on a Falling Edge..... | 951  |
| Figure 15-59. Frame of Period 16 CLKG Periods and Active Width of 2 CLKG Periods.....                              | 951  |
| Figure 15-60. Data Clocked Externally Using a Rising Edge and Sampled by the McBSP Receiver on a Falling Edge..... | 953  |
| Figure 15-61. Four 8-Bit Data Words Transferred To/From the McBSP.....                                             | 956  |
| Figure 15-62. One 32-Bit Data Word Transferred To/From the McBSP.....                                              | 957  |
| Figure 15-63. 8-Bit Data Words Transferred at Maximum Packet Frequency.....                                        | 958  |
| Figure 15-64. Configuring the Data Stream of <a href="#">Figure 15-63</a> as a Continuous 32-Bit Word.....         | 958  |
| Figure 15-65. DRR2 Register.....                                                                                   | 961  |
| Figure 15-66. DRR1 Register.....                                                                                   | 961  |
| Figure 15-67. DXR2 Register.....                                                                                   | 962  |
| Figure 15-68. DXR1 Register.....                                                                                   | 962  |
| Figure 15-69. SPCR2 Register.....                                                                                  | 963  |
| Figure 15-70. SPCR1 Register.....                                                                                  | 966  |
| Figure 15-71. RCR2 Register.....                                                                                   | 969  |
| Figure 15-72. RCR1 Register.....                                                                                   | 971  |
| Figure 15-73. XCR2 Register.....                                                                                   | 972  |
| Figure 15-74. XCR1 Register.....                                                                                   | 974  |
| Figure 15-75. SRGR2 Register.....                                                                                  | 975  |
| Figure 15-76. SRGR1 Register.....                                                                                  | 977  |
| Figure 15-77. MCR2 Register.....                                                                                   | 978  |
| Figure 15-78. MCR1 Register.....                                                                                   | 980  |
| Figure 15-79. RCERA Register.....                                                                                  | 983  |
| Figure 15-80. RCERB Register.....                                                                                  | 983  |
| Figure 15-81. XCERA Register.....                                                                                  | 984  |
| Figure 15-82. XCERB Register.....                                                                                  | 985  |
| Figure 15-83. PCR Register.....                                                                                    | 986  |
| Figure 15-84. RCERC Register.....                                                                                  | 989  |
| Figure 15-85. RCERD Register.....                                                                                  | 989  |
| Figure 15-86. XCERC Register.....                                                                                  | 990  |
| Figure 15-87. XCERD Register.....                                                                                  | 991  |
| Figure 15-88. RCERE Register.....                                                                                  | 992  |
| Figure 15-89. RCERF Register.....                                                                                  | 992  |
| Figure 15-90. XCERE Register.....                                                                                  | 993  |
| Figure 15-91. XCERF Register.....                                                                                  | 994  |
| Figure 15-92. RCERG Register.....                                                                                  | 995  |
| Figure 15-93. RCERH Register.....                                                                                  | 995  |
| Figure 15-94. XCERG Register.....                                                                                  | 996  |
| Figure 15-95. XCERH Register.....                                                                                  | 997  |
| Figure 15-96. MFFINT Register.....                                                                                 | 998  |
| Figure 16-1. eCAN Block Diagram and Interface Circuit.....                                                         | 1001 |
| Figure 16-2. CAN Data Frame.....                                                                                   | 1002 |
| Figure 16-3. Architecture of the eCAN Module.....                                                                  | 1003 |
| Figure 16-4. eCAN-A Memory Map.....                                                                                | 1006 |
| Figure 16-5. Initialization Sequence.....                                                                          | 1011 |
| Figure 16-6. CAN Bit Timing.....                                                                                   | 1012 |
| Figure 16-7. Interrupts Scheme.....                                                                                | 1018 |
| Figure 16-8. Local-Acceptance-Mask Register (LAMn).....                                                            | 1024 |

|                                                                                                   |      |
|---------------------------------------------------------------------------------------------------|------|
| Figure 16-9. Mailbox-Enable Register (CANME).....                                                 | 1025 |
| Figure 16-10. Mailbox-Direction Register (CANMD).....                                             | 1025 |
| Figure 16-11. Transmission-Request Set Register (CANTRS).....                                     | 1026 |
| Figure 16-12. Transmission-Request-Reset Register (CANTRR).....                                   | 1027 |
| Figure 16-13. Transmission-Acknowledge Register (CANTA).....                                      | 1027 |
| Figure 16-14. Abort-Acknowledge Register (CANAA).....                                             | 1028 |
| Figure 16-15. Received-Message-Pending Register (CANRMP).....                                     | 1028 |
| Figure 16-16. Received-Message-Lost Register (CANRML).....                                        | 1029 |
| Figure 16-17. Remote-Frame-Pending Register (CANRFP).....                                         | 1029 |
| Figure 16-18. Global Acceptance Mask Register (CANGAM).....                                       | 1031 |
| Figure 16-19. Master Control Register (CANMC).....                                                | 1032 |
| Figure 16-20. Bit-Timing Configuration Register (CANBTC).....                                     | 1035 |
| Figure 16-21. Error and Status Register (CANES).....                                              | 1037 |
| Figure 16-22. Transmit-Error-Counter Register (CANTEC).....                                       | 1039 |
| Figure 16-23. Receive-Error-Counter Register (CANREC).....                                        | 1039 |
| Figure 16-24. Global Interrupt Flag 0 Register (CANGIF0).....                                     | 1041 |
| Figure 16-25. Global Interrupt Flag 1 Register (CANGIF1).....                                     | 1041 |
| Figure 16-26. Global Interrupt Mask Register (CANGIM).....                                        | 1043 |
| Figure 16-27. Mailbox Interrupt Mask Register (CANMIM).....                                       | 1044 |
| Figure 16-28. Mailbox Interrupt Level Register (CANMIL).....                                      | 1045 |
| Figure 16-29. Overwrite Protection Control Register (CANOPC).....                                 | 1045 |
| Figure 16-30. TX I/O Control Register (CANTIOC).....                                              | 1046 |
| Figure 16-31. RX I/O Control Register (CANRIOC).....                                              | 1046 |
| Figure 16-32. Time-Stamp Counter Register (CANTSC).....                                           | 1047 |
| Figure 16-33. Message-Object Time-Out Registers (MOTO).....                                       | 1048 |
| Figure 16-34. Message-Object Time Stamp Registers (MOTS).....                                     | 1048 |
| Figure 16-35. Time-Out Control Register (CANTOC).....                                             | 1049 |
| Figure 16-36. Time-Out Status Register (CANTOS).....                                              | 1049 |
| Figure 16-37. Message Identifier Register (MSGID) Register.....                                   | 1050 |
| Figure 16-38. Message-Control Register (MSGCTRL).....                                             | 1052 |
| Figure 16-39. Message-Data-Low Register With DBO = 0 (CANMDL).....                                | 1053 |
| Figure 16-40. Message-Data-High Register With DBO = 0 (CANMDH).....                               | 1053 |
| Figure 16-41. Message-Data-Low Register With DBO = 1 (CANMDL).....                                | 1053 |
| Figure 16-42. Message-Data-High Register With DBO = 1 (CANMDH).....                               | 1053 |
| Figure 17-1. USB Block Diagram.....                                                               | 1056 |
| Figure 17-2. USB Scheme.....                                                                      | 1057 |
| Figure 17-3. Function Address Register (USBFADDR).....                                            | 1072 |
| Figure 17-4. Power Management Register (USBPOWER) in Host Mode.....                               | 1073 |
| Figure 17-5. Power Management Register (USBPOWER) in Device Mode.....                             | 1074 |
| Figure 17-6. USB Transmit Interrupt Status Register (USBTXIS).....                                | 1075 |
| Figure 17-7. USB Receive Interrupt Status Register (USBRXIS).....                                 | 1076 |
| Figure 17-8. USB Transmit Interrupt Status Enable Register (USBTXIE).....                         | 1077 |
| Figure 17-9. USB Receive Interrupt Enable Register (USBRXIE).....                                 | 1078 |
| Figure 17-10. USB General Interrupt Status Register (USBIS) in Host Mode.....                     | 1079 |
| Figure 17-11. USB General Interrupt Status Register (USBIS) in Device Mode.....                   | 1080 |
| Figure 17-12. USB Interrupt Enable Register (USBIE) in Host Mode.....                             | 1081 |
| Figure 17-13. USB Interrupt Enable Register (USBIE) in Device Mode.....                           | 1082 |
| Figure 17-14. Frame Number Register (FRAME).....                                                  | 1083 |
| Figure 17-15. USB Endpoint Index Register (USBEPIDX).....                                         | 1083 |
| Figure 17-16. USB Test Mode Register (USBTEST) in Host Mode.....                                  | 1084 |
| Figure 17-17. USB Test Mode Register (USBTEST) in Device Mode.....                                | 1085 |
| Figure 17-18. USB FIFO Endpoint <i>n</i> Register (USBFIFO[n]).....                               | 1086 |
| Figure 17-19. USB Device Control Register (USBDEVCTL).....                                        | 1087 |
| Figure 17-20. USB Transmit Dynamic FIFO Sizing Register (USBTXFIFOSZ).....                        | 1089 |
| Figure 17-21. USB Receive Dynamic FIFO Sizing Register (USBRXFIFOSZ).....                         | 1090 |
| Figure 17-22. USB Transmit FIFO Start Address Register (USBTXFIFOADDR).....                       | 1091 |
| Figure 17-23. USB Receive FIFO Start Address Register (USBRXFIFOADDR).....                        | 1092 |
| Figure 17-24. USB Connect Timing Register (USBCONTIM).....                                        | 1093 |
| Figure 17-25. USB Full-Speed Last Transaction to End of Frame Timing Register (USBFSEOF).....     | 1094 |
| Figure 17-26. USB Low-Speed Last Transaction to End of Frame Timing Register (USBLSEOF).....      | 1094 |
| Figure 17-27. USB Transmit Functional Address Endpoint <i>n</i> Registers (USBTXFUNCADDR[n])..... | 1095 |



|                                                                                                                 |      |
|-----------------------------------------------------------------------------------------------------------------|------|
| Figure 17-28. USB Transmit Hub Address Endpoint $n$ Registers (USBTXHUBADDR[ $n$ ]).....                        | 1096 |
| Figure 17-29. USB Transmit Hub Port Endpoint $n$ Registers (USBTXHUBPORT[ $n$ ]).....                           | 1097 |
| Figure 17-30. USB Receive Functional Address Endpoint $n$ Registers (USBFIFO[ $n$ ]).....                       | 1098 |
| Figure 17-31. USB Receive Hub Address Endpoint $n$ Registers (USBRXHUBADDR[ $n$ ]).....                         | 1099 |
| Figure 17-32. USB Transmit Hub Port Endpoint $n$ Registers (USBRXHUBPORT[ $n$ ]).....                           | 1100 |
| Figure 17-33. USB Maximum Transmit Data Endpoint $n$ Registers (USBTXMAXP[ $n$ ]).....                          | 1101 |
| Figure 17-34. USB Control and Status Endpoint 0 Low Register (USBCSRL0) in Host Mode.....                       | 1102 |
| Figure 17-35. USB Control and Status Endpoint 0 Low Register (USBCSRL0) in Device Mode.....                     | 1103 |
| Figure 17-36. USB Control and Status Endpoint 0 High Register (USBCSRH0) in Host Mode.....                      | 1105 |
| Figure 17-37. USB Control and Status Endpoint 0 High Register (USBCSRH0) in Device Mode.....                    | 1106 |
| Figure 17-38. USB Receive Byte Count Endpoint 0 Register (USBCOUNT0).....                                       | 1107 |
| Figure 17-39. USB Type Endpoint 0 Register (USBTYPE0).....                                                      | 1107 |
| Figure 17-40. USB NAK Limit Register (USBNAKLMT).....                                                           | 1108 |
| Figure 17-41. USB Transmit Control and Status Endpoint $n$ Low Register (USBTXCSRL[ $n$ ]) in Host Mode.....    | 1109 |
| Figure 17-42. USB Transmit Control and Status Endpoint $n$ Low Register (USBTXCSRL[ $n$ ]) in Device Mode.....  | 1110 |
| Figure 17-43. USB Transmit Control and Status Endpoint $n$ High Register (USBTXCSRH[ $n$ ]) in Host Mode.....   | 1112 |
| Figure 17-44. USB Transmit Control and Status Endpoint $n$ High Register (USBTXCSRH[ $n$ ]) in Device Mode..... | 1113 |
| Figure 17-45. USB Maximum Receive Data Endpoint $n$ Registers (USBRXMAXP[ $n$ ]).....                           | 1114 |
| Figure 17-46. USB Receive Control and Status Endpoint $n$ Low Register (USBCSRL[ $n$ ]) in Host Mode.....       | 1115 |
| Figure 17-47. USB Control and Status Endpoint $n$ Low Register (USBCSRL[ $n$ ]) in Device Mode.....             | 1116 |
| Figure 17-48. USB Receive Control and Status Endpoint $n$ High Register (USBCSRH[ $n$ ]) in Host Mode.....      | 1118 |
| Figure 17-49. USB Control and Status Endpoint $n$ High Register (USBCSRH[ $n$ ]) in Device Mode.....            | 1119 |
| Figure 17-50. USB Maximum Receive Data Endpoint $n$ Registers (USBRXCOUNT[ $n$ ]).....                          | 1120 |
| Figure 17-51. USB Host Transmit Configure Type Endpoint $n$ Register (USBTXTYPE[ $n$ ]).....                    | 1121 |
| Figure 17-52. USB Host Transmit Interval Endpoint $n$ Register (USBTXINTERVAL[ $n$ ]).....                      | 1122 |
| Figure 17-53. USB Host Configure Receive Type Endpoint $n$ Register (USBRXTYPE[ $n$ ]).....                     | 1123 |
| Figure 17-54. USB Host Receive Polling Interval Endpoint $n$ Register (USBRXINTERVAL[ $n$ ]).....               | 1124 |
| Figure 17-55. USB Request Packet Count in Block Transfer Endpoint $n$ Registers (USBQPKTCOUNT[ $n$ ]).....      | 1125 |
| Figure 17-56. USB Receive Double Packet Buffer Disable Register (USBXDPKTBUFFDIS).....                          | 1126 |
| Figure 17-57. USB Transmit Double Packet Buffer Disable Register (USBTXDPKTBUFFDIS).....                        | 1127 |
| Figure 17-58. USB External Power Control Register (USBEPCC).....                                                | 1128 |
| Figure 17-59. USB External Power Control Raw Interrupt Status Register (USBEPCCRIS).....                        | 1130 |
| Figure 17-60. USB External Power Control Interrupt Mask Register (USBEPCCIM).....                               | 1131 |
| Figure 17-61. USB External Power Control Interrupt Status and Clear Register (USBEPCCISC).....                  | 1132 |
| Figure 17-62. USB Device RESUME Raw Interrupt Status Register (USBDRRIS).....                                   | 1133 |
| Figure 17-63. USB Device RESUME Raw Interrupt Status Register (USBDRRIS).....                                   | 1134 |
| Figure 17-64. USB Device RESUME Interrupt Status and Clear Register (USBDRISC).....                             | 1135 |
| Figure 17-65. USB General-Purpose Control and Status Register (USBGPCS).....                                    | 1136 |
| Figure 17-66. USB DMA Select Register (USBDMASEL).....                                                          | 1137 |

## List of Tables

|                                                                                                |    |
|------------------------------------------------------------------------------------------------|----|
| Table 1-1. Flash/OTP Configuration Registers.....                                              | 46 |
| Table 1-2. Flash Options Register (FOPT) Field Descriptions.....                               | 47 |
| Table 1-3. Flash Power Register (FPWR) Field Descriptions.....                                 | 47 |
| Table 1-4. Flash Status Register (FSTATUS) Field Descriptions.....                             | 48 |
| Table 1-5. Flash Standby Wait Register (FSTDBYWAIT) Field Descriptions.....                    | 49 |
| Table 1-6. Flash Standby to Active Wait Counter Register (FACTIVEWAIT) Field Descriptions..... | 49 |
| Table 1-7. Flash Wait-State Register (FBANKWAIT) Field Descriptions.....                       | 50 |
| Table 1-8. OTP Wait-State Register (FOTPWAIT) Field Descriptions.....                          | 51 |
| Table 1-9. Security Levels.....                                                                | 52 |
| Table 1-10. Resources Affected by the CSM.....                                                 | 55 |
| Table 1-11. Resources Not Affected by the CSM.....                                             | 55 |
| Table 1-12. Code Security Module (CSM) Registers.....                                          | 61 |
| Table 1-13. CSM Status and Control Register (CSMSCR) Field Descriptions.....                   | 61 |
| Table 1-14. PLL, Clocking, Watchdog, and Low-Power Mode Registers.....                         | 63 |
| Table 1-15. Peripheral Clock Control 0 Register (PCLKCR0) Field Descriptions.....              | 64 |
| Table 1-16. Peripheral Clock Control 1 Register (PCLKCR1) Field Descriptions.....              | 65 |
| Table 1-17. Peripheral Clock Control 2 Register (PCLKCR2) Field Descriptions.....              | 67 |
| Table 1-18. Peripheral Clock Control 3 Register (PCLKCR3) Field Descriptions.....              | 68 |
| Table 1-19. Low-Speed Peripheral Clock Prescaler Register (LOSPCP) Field Descriptions.....     | 69 |
| Table 1-20. Internal Oscillator $n$ Trim (INTOSC $n$ TRIM) Register Field Descriptions.....    | 71 |



|                                                                                             |     |
|---------------------------------------------------------------------------------------------|-----|
| Table 1-21. Clocking (XCLK) Field Descriptions.....                                         | 72  |
| Table 1-22. Clock Control (CLKCTL) Register Field Descriptions.....                         | 73  |
| Table 1-23. Possible PLL Configuration Modes.....                                           | 76  |
| Table 1-24. PLL Settings.....                                                               | 78  |
| Table 1-25. PLL Status Register (PLLSTS) Field Descriptions.....                            | 79  |
| Table 1-26. PLL Lock Period (PLLLOCKPRD) Register Field Descriptions.....                   | 81  |
| Table 1-27. PLL2 Configuration (PLL2CTL) Register Field Descriptions.....                   | 82  |
| Table 1-28. PLL2 Multiplier (PLL2MULT) Register Field Descriptions.....                     | 83  |
| Table 1-29. PLL2 Lock Status (PLL2STS) Register Field Descriptions.....                     | 83  |
| Table 1-30. SYSCLK2 Clock Counter (SYSCLK2CNTR) Register Field Descriptions.....            | 84  |
| Table 1-31. EPWM DMA/CLA Configuration (EPWMCFG) Register Field Descriptions.....           | 84  |
| Table 1-32. NMI Interrupt Registers.....                                                    | 89  |
| Table 1-33. NMI Configuration (NMICFG) Register Bit Definitions (EALLOW).....               | 89  |
| Table 1-34. NMI Flag (NMIFLG) Register Bit Definitions (EALLOW Protected).....              | 90  |
| Table 1-35. NMI Flag Clear (NMIFLGCLR) Register Bit Definitions (EALLOW Protected).....     | 90  |
| Table 1-36. NMI Flag Force (NMIFLGFR) Register Bit Definitions (EALLOW Protected).....      | 91  |
| Table 1-37. NMI Watchdog Counter (NMIWDCNT) Register Bit Definitions.....                   | 91  |
| Table 1-38. NMI Watchdog Period (NMIWDPRD) Register Bit Definitions (EALLOW Protected)..... | 92  |
| Table 1-39. Low-Power Mode Summary.....                                                     | 93  |
| Table 1-40. Low Power Modes.....                                                            | 94  |
| Table 1-41. Low-Power Mode Control 0 Register (LPMCR0) Field Descriptions.....              | 95  |
| Table 1-42. Example Watchdog Key Sequences.....                                             | 97  |
| Table 1-43. System Control and Status Register (SCSR) Field Descriptions.....               | 99  |
| Table 1-44. Watchdog Counter Register (WDCNTR) Field Descriptions.....                      | 100 |
| Table 1-45. Watchdog Reset Key Register (WDKEY) Field Descriptions.....                     | 100 |
| Table 1-46. Watchdog Control Register (WDCR) Field Descriptions.....                        | 101 |
| Table 1-47. CPU-Timers 0, 1, 2 Configuration and Control Registers.....                     | 103 |
| Table 1-48. TIMERxTIM Register Field Descriptions.....                                      | 104 |
| Table 1-49. TIMERxTIMH Register Field Descriptions.....                                     | 104 |
| Table 1-50. TIMERxPRD Register Field Descriptions.....                                      | 105 |
| Table 1-51. TIMERxPRDH Register Field Descriptions.....                                     | 105 |
| Table 1-52. TIMERxTCR Register Field Descriptions.....                                      | 106 |
| Table 1-53. TIMERxTPR Register Field Descriptions.....                                      | 107 |
| Table 1-54. TIMERxTPRH Register Field Descriptions.....                                     | 107 |
| Table 1-55. JTAGDEBUG Register Field Descriptions.....                                      | 111 |
| Table 1-56. GPIO Control Registers.....                                                     | 113 |
| Table 1-57. GPIO Interrupt and Low Power Mode Select Registers.....                         | 113 |
| Table 1-58. GPIO Data Registers.....                                                        | 115 |
| Table 1-59. Sampling Period.....                                                            | 117 |
| Table 1-60. Sampling Frequency.....                                                         | 117 |
| Table 1-61. Case 1: Three-Sample Sampling Window Width.....                                 | 118 |
| Table 1-62. Case 2: Six-Sample Sampling Window Width.....                                   | 118 |
| Table 1-63. Default State of Peripheral Input.....                                          | 121 |
| Table 1-64. GPIOA MUX <sup>(1) (2)</sup> .....                                              | 122 |
| Table 1-65. GPIOB MUX <sup>(1) (2)</sup> .....                                              | 123 |
| Table 1-66. Analog MUX.....                                                                 | 124 |
| Table 1-67. GPIO Port A Multiplexing 1 (GPAMUX1) Register Field Descriptions.....           | 124 |
| Table 1-68. GPIO Port A MUX 2 (GPAMUX2) Register Field Descriptions.....                    | 127 |
| Table 1-69. GPIO Port B MUX 1 (GPBMUX1) Register Field Descriptions.....                    | 130 |
| Table 1-70. GPIO Port B MUX 2 (GPBMUX2) Register Field Descriptions.....                    | 132 |
| Table 1-71. Analog I/O MUX (AIOMUX1) Register Field Descriptions.....                       | 134 |
| Table 1-72. GPIO Port A Qualification Control (GPACTRL) Register Field Descriptions.....    | 135 |
| Table 1-73. GPIO Port B Qualification Control (GPBCTRL) Register Field Descriptions.....    | 136 |
| Table 1-74. GPIO A Control Register 2 Register (GPACTRL2) Field Descriptions.....           | 137 |
| Table 1-75. GPIO Port A Qualification Select 1 (GPAQSEL1) Register Field Descriptions.....  | 138 |
| Table 1-76. GPIO Port A Qualification Select 2 (GPAQSEL2) Register Field Descriptions.....  | 139 |
| Table 1-77. GPIO Port B Qualification Select 1 (GPBQSEL1) Register Field Descriptions.....  | 140 |
| Table 1-78. GPIO Port B Qualification Select 2 (GPBQSEL2) Register Field Descriptions.....  | 141 |
| Table 1-79. GPIO Port A Direction (GPADIR) Register Field Descriptions.....                 | 142 |
| Table 1-80. GPIO Port B Direction (GPBDIR) Register Field Descriptions.....                 | 143 |
| Table 1-81. Analog I/O DIR (AIODIR) Register Field Descriptions.....                        | 144 |

|                                                                                              |     |
|----------------------------------------------------------------------------------------------|-----|
| Table 1-82. GPIO Port A Internal Pullup Disable (GPAPUD) Register Field Descriptions.....    | 145 |
| Table 1-83. GPIO Port B Internal Pullup Disable (GPBPUD) Register Field Descriptions.....    | 146 |
| Table 1-84. GPIO Port A Data (GPADAT) Register Field Descriptions.....                       | 147 |
| Table 1-85. GPIO Port B Data (GPBDAT) Register Field Descriptions.....                       | 148 |
| Table 1-86. Analog I/O DAT (AIODAT) Register Field Descriptions.....                         | 149 |
| Table 1-87. GPIO Port A Set (GPASET) Register Field Descriptions.....                        | 150 |
| Table 1-88. GPIO Port A Clear (GPACLEAR) Register Field Descriptions.....                    | 150 |
| Table 1-89. GPIO Port A Toggle (GPATOGGLE) Register Field Descriptions.....                  | 151 |
| Table 1-90. GPIO Port B Set (GPBSET) Register Field Descriptions.....                        | 152 |
| Table 1-91. GPIO Port B Clear (GPBCLEAR) Register Field Descriptions.....                    | 153 |
| Table 1-92. GPIO Port B Toggle (GPBTOGGLE) Register Field Descriptions.....                  | 153 |
| Table 1-93. Analog I/O Set (AIOSET) Register Field Descriptions.....                         | 154 |
| Table 1-94. Analog I/O Clear (AIOCLEAR) Register Field Descriptions.....                     | 154 |
| Table 1-95. Analog I/O Toggle (AIOTOGGLE) Register Field Descriptions.....                   | 154 |
| Table 1-96. GPIO XINTn Interrupt Select (GPIOXINTnSEL) Register Field Descriptions.....      | 155 |
| Table 1-97. XINT1/XINT2/XINT3 Interrupt Select and Configuration Registers.....              | 155 |
| Table 1-98. GPIO Low Power Mode Wakeup Select (GPIOLPMSEL) Register Field Descriptions.....  | 156 |
| Table 1-99. Peripheral Frame 0 Registers.....                                                | 157 |
| Table 1-100. Peripheral Frame 1 Registers.....                                               | 158 |
| Table 1-101. Peripheral Frame 2 Registers.....                                               | 158 |
| Table 1-102. Peripheral Frame 3 Registers.....                                               | 158 |
| Table 1-103. Access to EALLOW-Protected Registers.....                                       | 159 |
| Table 1-104. EALLOW-Protected Device Emulation Registers.....                                | 159 |
| Table 1-105. EALLOW-Protected Flash/OTP Configuration Registers.....                         | 159 |
| Table 1-106. EALLOW-Protected Code Security Module (CSM) Registers.....                      | 160 |
| Table 1-107. EALLOW-Protected PLL, Clocking, Watchdog, and Low-Power Mode Registers.....     | 160 |
| Table 1-108. EALLOW-Protected GPIO Registers.....                                            | 161 |
| Table 1-109. EALLOW-Protected PIE Vector Table.....                                          | 162 |
| Table 1-110. EALLOW-Protected ePWM1 - ePWM 7 Registers.....                                  | 162 |
| Table 1-111. Device Emulation Registers.....                                                 | 163 |
| Table 1-112. DEVICECNF Register Field Descriptions.....                                      | 163 |
| Table 1-113. PARTID Register Field Descriptions.....                                         | 164 |
| Table 1-114. CLASSID Register Field Descriptions.....                                        | 165 |
| Table 1-115. REVID Register Field Descriptions.....                                          | 165 |
| Table 1-116. Enabling Interrupt.....                                                         | 169 |
| Table 1-117. Interrupt Vector Table Mapping.....                                             | 170 |
| Table 1-118. Vector Table Mapping After Reset Operation.....                                 | 170 |
| Table 1-119. PIE MUXed Peripheral Interrupt Vector Table.....                                | 177 |
| Table 1-120. PIE Vector Table.....                                                           | 178 |
| Table 1-121. PIE Configuration and Control Registers.....                                    | 181 |
| Table 1-122. PIECTRL Register Address Field Descriptions.....                                | 182 |
| Table 1-123. PIE Interrupt Acknowledge Register (PIEACK) Field Descriptions.....             | 182 |
| Table 1-124. PIEIFRx Register Field Descriptions.....                                        | 183 |
| Table 1-125. PIEIERx Register (x = 1 to 12) Field Descriptions.....                          | 184 |
| Table 1-126. Interrupt Flag Register (IFR) — CPU Register Field Descriptions.....            | 186 |
| Table 1-127. Interrupt Enable Register (IER) — CPU Register Field Descriptions.....          | 188 |
| Table 1-128. Debug Interrupt Enable Register (DBGIER) — CPU Register Field Descriptions..... | 190 |
| Table 1-129. Interrupt Control and Counter Registers (not EALLOW Protected).....             | 192 |
| Table 1-130. External Interrupt n Control Register (XINTnCR) Field Descriptions.....         | 192 |
| Table 1-131. External Interrupt n Counter (XINTnCTR) Field Descriptions.....                 | 193 |
| Table 1-132. BOR Configuration (BORCFG) Field Descriptions.....                              | 194 |
| Table 2-1. Vector Locations.....                                                             | 201 |
| Table 2-2. Configuration for Device Modes.....                                               | 203 |
| Table 2-3. PIE Vector SARAM Locations Used by the Boot ROM.....                              | 205 |
| Table 2-4. Boot Mode Selection.....                                                          | 205 |
| Table 2-5. Valid EMU_KEY and EMU_BMODE Values.....                                           | 207 |
| Table 2-6. OTP Values for GetMode.....                                                       | 209 |
| Table 2-7. Emulation Boot Modes (TRST = 1).....                                              | 210 |
| Table 2-8. Stand-Alone Boot Modes with (TRST = 0).....                                       | 211 |
| Table 2-9. General Structure Of Source Program Data Stream In 16-Bit Mode.....               | 213 |
| Table 2-10. LSB/MSB Loading Sequence in 8-Bit Data Stream.....                               | 215 |

|                                                                                                   |     |
|---------------------------------------------------------------------------------------------------|-----|
| Table 2-11. Parallel GPIO Boot 8-Bit Data Stream.....                                             | 224 |
| Table 2-12. SPI 8-Bit Data Stream.....                                                            | 227 |
| Table 2-13. I2C 8-Bit Data Stream.....                                                            | 232 |
| Table 2-14. Bit-Rate Value for Internal Oscillators.....                                          | 233 |
| Table 2-15. eCAN 8-Bit Data Stream.....                                                           | 234 |
| Table 2-16. CPU Register Restored Values.....                                                     | 236 |
| Table 2-17. Boot Loader Options.....                                                              | 237 |
| Table 2-18. Bootloader Revision and Checksum Information.....                                     | 240 |
| Table 3-1. ePWM Module Control and Status Register Set Grouped by Submodule.....                  | 247 |
| Table 3-2. Submodule Configuration Parameters.....                                                | 249 |
| Table 3-3. Time-Base Submodule Registers.....                                                     | 252 |
| Table 3-4. Key Time-Base Signals.....                                                             | 253 |
| Table 3-5. Counter-Compare Submodule Registers.....                                               | 261 |
| Table 3-6. Counter-Compare Submodule Key Signals.....                                             | 261 |
| Table 3-7. Action-Qualifier Submodule Registers.....                                              | 266 |
| Table 3-8. Action-Qualifier Submodule Possible Input Events.....                                  | 266 |
| Table 3-9. Action-Qualifier Event Priority for Up-Down-Count Mode.....                            | 268 |
| Table 3-10. Action-Qualifier Event Priority for Up-Count Mode.....                                | 268 |
| Table 3-11. Action-Qualifier Event Priority for Down-Count Mode.....                              | 268 |
| Table 3-12. Behavior if CMPA/CMPB is Greater than the Period.....                                 | 268 |
| Table 3-13. Dead-Band Generator Submodule Registers.....                                          | 278 |
| Table 3-14. Classical Dead-Band Operating Modes.....                                              | 280 |
| Table 3-15. PWM-Chopper Submodule Registers.....                                                  | 282 |
| Table 3-16. Possible Pulse Width Values for SYSCLKOUT = 90 MHz .....                              | 284 |
| Table 3-17. Trip-Zone Submodule Registers.....                                                    | 287 |
| Table 3-18. Possible Actions On a Trip Event.....                                                 | 288 |
| Table 3-19. Event-Trigger Submodule Registers.....                                                | 294 |
| Table 3-20. Digital Compare Submodule Registers.....                                              | 298 |
| Table 3-21. Time-Base Control Register (TBCTL) Field Descriptions.....                            | 328 |
| Table 3-22. Time-Base Status Register (TBSTS) Field Descriptions.....                             | 330 |
| Table 3-23. Time-Base Phase High Resolution Register (TBPHSHR) Field Descriptions.....            | 331 |
| Table 3-24. Time-Base Phase Register (TBPHS) Field Descriptions.....                              | 331 |
| Table 3-25. Time-Base Counter Register (TBCTR) Field Descriptions.....                            | 332 |
| Table 3-26. Time-Base Period Register (TBPRD) Field Descriptions.....                             | 332 |
| Table 3-27. Time Base Period High Resolution Register (TBPRDHR) Field Descriptions.....           | 333 |
| Table 3-28. Time-Base Period High Resolution Mirror Register (TBPRDHRM) Field Descriptions.....   | 334 |
| Table 3-29. Time-Base Period Mirror Register (TBPRDM) Field Descriptions.....                     | 335 |
| Table 3-30. Counter-Compare Control (CMPCTL) Register Field Descriptions.....                     | 336 |
| Table 3-31. Compare A High Resolution (CMPAHR) Register Field Descriptions.....                   | 337 |
| Table 3-32. Counter-Compare A (CMPA) Register Field Descriptions.....                             | 338 |
| Table 3-33. Counter-Compare B (CMPB) Register Field Descriptions.....                             | 339 |
| Table 3-34. Compare A High-Resolution Mirror (CMPAHRM) Register Field Descriptions.....           | 340 |
| Table 3-35. Counter-Compare A Mirror (CMPAM) Register Field Descriptions.....                     | 341 |
| Table 3-36. Action-Qualifier Output A Control Register (AQCTLA) Field Descriptions.....           | 342 |
| Table 3-37. Action-Qualifier Output B Control Register (AQCTLB) Field Descriptions.....           | 343 |
| Table 3-38. Action-Qualifier Software Force Register (AQSFRC) Field Descriptions.....             | 345 |
| Table 3-39. Action-Qualifier Continuous Software Force Register (AQCSFRC) Field Descriptions..... | 346 |
| Table 3-40. Dead-Band Generator Control (DBCTL) Register Field Descriptions.....                  | 347 |
| Table 3-41. Dead-Band Generator Rising Edge Delay (DBRED) Register Field Descriptions.....        | 349 |
| Table 3-42. Dead-Band Generator Falling Edge Delay (DBFED) Register Field Descriptions.....       | 349 |
| Table 3-43. Trip-Zone Submodule Select Register (TZSEL) Field Descriptions.....                   | 350 |
| Table 3-44. Trip Zone Digital Compare Event Select Register (TZDCSEL) Field Descriptions.....     | 351 |
| Table 3-45. Trip-Zone Control Register (TZCTL) Field Descriptions.....                            | 353 |
| Table 3-46. Trip-Zone Enable Interrupt Register (TZEINT) Field Descriptions.....                  | 354 |
| Table 3-47. Trip-Zone Flag Register (TZFLG) Field Descriptions.....                               | 355 |
| Table 3-48. Trip-Zone Clear Register (TZCLR) Field Descriptions.....                              | 357 |
| Table 3-49. Trip-Zone Force Register (TZFRC) Field Descriptions.....                              | 358 |
| Table 3-50. Event-Trigger Selection Register (ETSEL) Field Descriptions.....                      | 359 |
| Table 3-51. Event-Trigger Prescale Register (ETPS) Field Descriptions.....                        | 360 |
| Table 3-52. Event-Trigger Flag Register (ETFLG) Field Descriptions.....                           | 362 |
| Table 3-53. Event-Trigger Clear Register (ETCLR) Field Descriptions.....                          | 363 |

|                                                                                                      |     |
|------------------------------------------------------------------------------------------------------|-----|
| Table 3-54. Event-Trigger Force Register (ETFRC) Field Descriptions.....                             | 364 |
| Table 3-55. PWM-Chopper Control (PCCTL) Register Field Descriptions.....                             | 365 |
| Table 3-56. Digital Compare Trip Select (DCTRIPSEL) Register Field Descriptions.....                 | 367 |
| Table 3-57. Digital Compare A Control (DCACTL) Register Field Descriptions.....                      | 369 |
| Table 3-58. Digital Compare B Control (DCBCTL) Register Field Descriptions.....                      | 370 |
| Table 3-59. Digital Compare Filter Control (DCFCTL) Register Field Descriptions.....                 | 371 |
| Table 3-60. Digital Compare Capture Control (DCCAPCTL) Register Field Descriptions.....              | 372 |
| Table 3-61. Digital Compare Filter Offset (DCFOFFSET) Register Field Descriptions.....               | 373 |
| Table 3-62. Digital Compare Filter Offset Counter (DCFOFFSETCNT) Register Field Descriptions.....    | 373 |
| Table 3-63. Digital Compare Filter Window (DCFWINDOW) Register Field Descriptions.....               | 374 |
| Table 3-64. Digital Compare Filter Window Counter (DCFWINDOWCNT) Register Field Descriptions.....    | 374 |
| Table 3-65. Digital Compare Counter Capture (DCCAP) Register Field Descriptions.....                 | 375 |
| Table 4-1. Resolution for PWM and HRPWM.....                                                         | 378 |
| Table 4-2. HRPWM Registers.....                                                                      | 379 |
| Table 4-3. Relationship Between MEP Steps, PWM Frequency, and Resolution.....                        | 384 |
| Table 4-4. CMPA versus Duty (left), and [CMPA:CMPAHR] versus Duty (right).....                       | 385 |
| Table 4-5. Duty Cycle Range Limitation for 3 SYSCLK/TBCLK Cycles.....                                | 387 |
| Table 4-6. SFO Library Features.....                                                                 | 397 |
| Table 4-7. Factor Values.....                                                                        | 398 |
| Table 4-8. HRPWM Registers.....                                                                      | 401 |
| Table 4-9. HRPWM Configuration (HRCNFG) Register Field Descriptions.....                             | 402 |
| Table 4-10. High Resolution Micro Step (HRMSTEP) Register Field Descriptions.....                    | 403 |
| Table 4-11. High Resolution Period Control (HRPCTL) Register Field Descriptions.....                 | 404 |
| Table 5-1. HRCAP Register Summary.....                                                               | 421 |
| Table 5-2. HRCAP Control Register (HCCTL) Field Descriptions.....                                    | 422 |
| Table 5-3. HRCAP Interrupt Flag Register (HCIFR) Field Descriptions.....                             | 423 |
| Table 5-4. HRCAP Interrupt Clear Register (HCICLR) Field Descriptions.....                           | 424 |
| Table 5-5. HRCAP Interrupt Force Register (HCIFRC) Field Descriptions.....                           | 425 |
| Table 5-6. HRCAP Counter Register (HCCOUNTER) Field Descriptions.....                                | 426 |
| Table 5-7. HRCAP Capture Counter On Rising Edge 0 Register (HCCAPCNTRISE0) Field Descriptions.....   | 427 |
| Table 5-8. HRCAP Capture Counter On Falling Edge 0 Register (HCCAPCNTFALL0) Field Descriptions.....  | 427 |
| Table 5-9. HRCAP Capture Counter On Rising Edge 1 Register (HCCAPCNTRISE1) Field Descriptions.....   | 428 |
| Table 5-10. HRCAP Capture Counter On Falling Edge 1 Register (HCCAPCNTFALL1) Field Descriptions..... | 428 |
| Table 6-1. ECAP Base Address Table.....                                                              | 446 |
| Table 6-2. ECAP_REGS Registers.....                                                                  | 447 |
| Table 6-3. ECAP_REGS Access Type Codes.....                                                          | 447 |
| Table 6-4. TSCTR Register Field Descriptions.....                                                    | 448 |
| Table 6-5. CTRPHS Register Field Descriptions.....                                                   | 448 |
| Table 6-6. CAP1 Register Field Descriptions.....                                                     | 449 |
| Table 6-7. CAP2 Register Field Descriptions.....                                                     | 449 |
| Table 6-8. CAP3 Register Field Descriptions.....                                                     | 450 |
| Table 6-9. CAP4 Register Field Descriptions.....                                                     | 450 |
| Table 6-10. ECCTL1 Register Field Descriptions.....                                                  | 451 |
| Table 6-11. ECCTL2 Register Field Descriptions.....                                                  | 453 |
| Table 6-12. ECEINT Register Field Descriptions.....                                                  | 455 |
| Table 6-13. ECFLG Register Field Descriptions.....                                                   | 457 |
| Table 6-14. ECCLR Register Field Descriptions.....                                                   | 459 |
| Table 6-15. ECFRC Register Field Descriptions.....                                                   | 460 |
| Table 7-1. EQEP Memory Map.....                                                                      | 467 |
| Table 7-2. Quadrature Decoder Truth Table.....                                                       | 469 |
| Table 7-3. eQEP Base Address Table.....                                                              | 483 |
| Table 7-4. EQEP_REGS Registers.....                                                                  | 484 |
| Table 7-5. EQEP_REGS Access Type Codes.....                                                          | 485 |
| Table 7-6. QPOSCNT Register Field Descriptions.....                                                  | 486 |
| Table 7-7. QPOSINIT Register Field Descriptions.....                                                 | 486 |
| Table 7-8. QPOSMAX Register Field Descriptions.....                                                  | 487 |
| Table 7-9. QPOSCMP Register Field Descriptions.....                                                  | 487 |
| Table 7-10. QPOSILAT Register Field Descriptions.....                                                | 487 |
| Table 7-11. QPOSSLAT Register Field Descriptions.....                                                | 488 |
| Table 7-12. QPOSLAT Register Field Descriptions.....                                                 | 488 |
| Table 7-13. QUTMR Register Field Descriptions.....                                                   | 488 |



|                                                                                                          |     |
|----------------------------------------------------------------------------------------------------------|-----|
| Table 7-14. QUPRD Register Field Descriptions.....                                                       | 489 |
| Table 7-15. QWDTMR Register Field Descriptions.....                                                      | 489 |
| Table 7-16. QWDPRD Register Field Descriptions.....                                                      | 490 |
| Table 7-17. QDECCTL Register Field Descriptions.....                                                     | 491 |
| Table 7-18. QEPCTL Register Field Descriptions.....                                                      | 493 |
| Table 7-19. QCAPCTL Register Field Descriptions.....                                                     | 496 |
| Table 7-20. QPOSCTL Register Field Descriptions.....                                                     | 497 |
| Table 7-21. QEINT Register Field Descriptions.....                                                       | 498 |
| Table 7-22. QFLG Register Field Descriptions.....                                                        | 500 |
| Table 7-23. QCLR Register Field Descriptions.....                                                        | 502 |
| Table 7-24. QFRC Register Field Descriptions.....                                                        | 504 |
| Table 7-25. QEPSTS Register Field Descriptions.....                                                      | 506 |
| Table 7-26. QCTMR Register Field Descriptions.....                                                       | 507 |
| Table 7-27. QCPRD Register Field Descriptions.....                                                       | 507 |
| Table 7-28. QCTMRLAT Register Field Descriptions.....                                                    | 508 |
| Table 7-29. QCPRDLAT Register Field Descriptions.....                                                    | 508 |
| Table 8-1. Sample Timings with Different Values of ACQPS.....                                            | 514 |
| Table 8-2. Estimated Droop Error from $n_T$ Value.....                                                   | 518 |
| Table 8-3. ADC Configuration and Control Registers (AdcRegs and AdcResult).....                          | 534 |
| Table 8-4. ADC Control Register 1 (ADCCTL1) Field Descriptions.....                                      | 535 |
| Table 8-5. ADC Control Register 2 (ADCCTL2) Field Descriptions.....                                      | 537 |
| Table 8-6. CLKDIV2EN and CLKDIV4EN Usage.....                                                            | 537 |
| Table 8-7. ADC Interrupt Flag Register (ADCINTFLG) Field Descriptions.....                               | 538 |
| Table 8-8. ADC Interrupt Flag Clear Register (ADCINTFLGCLR) Field Descriptions.....                      | 539 |
| Table 8-9. ADC Interrupt Overflow Register (ADCINTOVF) Field Descriptions.....                           | 540 |
| Table 8-10. ADC Interrupt Overflow Clear Register (ADCINTOVFCLR) Field Descriptions.....                 | 541 |
| Table 8-11. Interrupt Select Register (INTSELxNy) Field Descriptions.....                                | 543 |
| Table 8-12. SOCPRCTL Register Field Descriptions.....                                                    | 545 |
| Table 8-13. ADC Sample Mode Register (ADCSAMPLEMODE) Field Descriptions.....                             | 547 |
| Table 8-14. ADC Interrupt Trigger SOC Select 1 Register (ADCINTSOCSEL1) Register Field Descriptions..... | 549 |
| Table 8-15. ADC Interrupt Trigger SOC Select 2 Register (ADCINTSOCSEL2) Field Descriptions.....          | 549 |
| Table 8-16. ADC SOC Flag 1 Register (ADCSOCFLG1) Field Descriptions.....                                 | 550 |
| Table 8-17. ADC SOC Force 1 Register (ADCSOCFRC1) Field Descriptions.....                                | 551 |
| Table 8-18. ADC SOC Overflow 1 Register (ADCSOCOVF1) Field Descriptions.....                             | 552 |
| Table 8-19. ADC SOC Overflow Clear 1 Register (ADCSOCOVFCLR1) Field Descriptions.....                    | 552 |
| Table 8-20. ADC SOC0-SOC15 Control Registers (ADCSOCxCTL) Register Field Descriptions.....               | 553 |
| Table 8-21. ADC Reference/Gain Trim Register (ADCREFTRIM) Field Descriptions.....                        | 556 |
| Table 8-22. ADC Offset Trim Register (ADCOFFTRIM) Field Descriptions.....                                | 556 |
| Table 8-23. Comparator Hysteresis Control Register (COMPHYSTCTL) Field Descriptions.....                 | 557 |
| Table 8-24. ADC Revision Register (ADCREV) Field Descriptions.....                                       | 558 |
| Table 8-25. ADC RESULT0-ADCRESULT15 Registers (ADCRESULTx) Field Descriptions.....                       | 558 |
| Table 9-1. Comparator Truth Table.....                                                                   | 561 |
| Table 9-2. Comparator Modules.....                                                                       | 564 |
| Table 9-3. Comparator Module Registers.....                                                              | 564 |
| Table 9-4. Comparator Control (COMPCTL) Register Field Descriptions.....                                 | 565 |
| Table 9-5. Compare Output Status (COMPSTS) Register Field Descriptions.....                              | 566 |
| Table 9-6. DAC Control (DACCTL) Register Field Descriptions.....                                         | 566 |
| Table 9-7. DAC Value (DACVAL) Register Field Descriptions.....                                           | 567 |
| Table 9-8. Ramp Generator Maximum Reference Active (RAMPMAXREF_ACTIVE) Register Field Descriptions.....  | 567 |
| Table 9-9. Ramp Generator Maximum Reference Shadow (RAMPMAXREF_SHDW) Register Field Descriptions.....    | 567 |
| Table 9-10. Ramp Generator Decrement Value Active (RAMPDECVAL_ACTIVE) Register Field Descriptions.....   | 568 |
| Table 9-11. Ramp Generator Decrement Value Shadow (RAMPDECVAL_SHDW) Register Field Descriptions.....     | 568 |
| Table 9-12. Ramp Generator Status (RAMPSTS) Register Field Descriptions.....                             | 568 |
| Table 10-1. Write Followed by Read - Read Occurs First.....                                              | 583 |
| Table 10-2. Write Followed by Read - Write Occurs First.....                                             | 583 |
| Table 10-3. ADC to CLA Early Interrupt Response.....                                                     | 586 |
| Table 10-4. Operand Nomenclature.....                                                                    | 588 |
| Table 10-5. INSTRUCTION dest, source1, source2 Short Description.....                                    | 589 |
| Table 10-6. Addressing Modes.....                                                                        | 590 |
| Table 10-7. Shift Field Encoding.....                                                                    | 590 |
| Table 10-8. Operand Encoding.....                                                                        | 591 |

|                                                                                                                                              |     |
|----------------------------------------------------------------------------------------------------------------------------------------------|-----|
| Table 10-9. Condition Field Encoding.....                                                                                                    | 591 |
| Table 10-10. Pipeline Activity For MBCNDD, Branch Not Taken.....                                                                             | 607 |
| Table 10-11. Pipeline Activity For MBCNDD, Branch Taken.....                                                                                 | 607 |
| Table 10-12. Pipeline Activity For MCCNDD, Call Not Taken.....                                                                               | 612 |
| Table 10-13. Pipeline Activity For MCCNDD, Call Taken.....                                                                                   | 612 |
| Table 10-14. Pipeline Activity For MMOV16 MARx, MRa , #16l.....                                                                              | 648 |
| Table 10-15. Pipeline Activity For MMOV16 MAR0/MAR1, mem16.....                                                                              | 651 |
| Table 10-16. Pipeline Activity For MMOVI16 MAR0/MAR1, #16l.....                                                                              | 667 |
| Table 10-17. Pipeline Activity For MRCNDD, Return Not Taken.....                                                                             | 690 |
| Table 10-18. Pipeline Activity For MRCNDD, Return Taken.....                                                                                 | 690 |
| Table 10-19. Pipeline Activity For MSTOP.....                                                                                                | 693 |
| Table 10-20. CLA Module Control and Status Register Set.....                                                                                 | 709 |
| Table 10-21. Task Interrupt Vector (MVECT1/2/3/4/5/6/7/8) Field Descriptions.....                                                            | 710 |
| Table 10-22. Control Register (MCTL) Field Descriptions.....                                                                                 | 711 |
| Table 10-23. Memory Configuration Register (MMECFG) Field Descriptions.....                                                                  | 712 |
| Table 10-24. Peripheral Interrupt Source Select 1 (MPISRCSEL1) Register Field Descriptions.....                                              | 714 |
| Table 10-25. Interrupt Flag Register (MIFR) Field Descriptions.....                                                                          | 716 |
| Table 10-26. Interrupt Overflow Flag Register (MIOVF) Field Descriptions.....                                                                | 717 |
| Table 10-27. Interrupt Force Register (MIFRC) Field Descriptions.....                                                                        | 718 |
| Table 10-28. Interrupt Flag Clear Register (MICLR) Field Descriptions.....                                                                   | 719 |
| Table 10-29. Interrupt Overflow Flag Clear Register (MICLROVF) Field Descriptions.....                                                       | 720 |
| Table 10-30. Interrupt Enable Register (MIER) Field Descriptions.....                                                                        | 721 |
| Table 10-31. Interrupt Run Status Register (MIRUN) Field Descriptions.....                                                                   | 723 |
| Table 10-32. Program Counter (MPC) Field Descriptions.....                                                                                   | 724 |
| Table 10-33. CLA Status (MSTF) Register Field Descriptions.....                                                                              | 725 |
| Table 11-1. Peripheral Interrupt Trigger Source Options.....                                                                                 | 731 |
| Table 11-2. DMA Register Summary.....                                                                                                        | 740 |
| Table 11-3. DMA Control Register (DMACTRL) Field Descriptions.....                                                                           | 741 |
| Table 11-4. Debug Control Register (DEBUGCTRL) Field Descriptions.....                                                                       | 743 |
| Table 11-5. Revision Register (REVISION) Field Descriptions.....                                                                             | 743 |
| Table 11-6. Priority Control Register 1 (PRIORITYCTRL1) Field Descriptions.....                                                              | 744 |
| Table 11-7. Priority Status Register (PRIORITYSTAT) Field Descriptions.....                                                                  | 745 |
| Table 11-8. Mode Register (MODE) Field Descriptions.....                                                                                     | 746 |
| Table 11-9. Control Register (CONTROL) Field Descriptions.....                                                                               | 748 |
| Table 11-10. Burst Size Register (BURST_SIZE) Field Descriptions.....                                                                        | 750 |
| Table 11-11. Burst Count Register (BURST_COUNT) Field Descriptions.....                                                                      | 750 |
| Table 11-12. Source Burst Step Size Register (SRC_BURST_STEP) Field Descriptions.....                                                        | 751 |
| Table 11-13. Destination Burst Step Register Size (DST_BURST_STEP) Field Descriptions.....                                                   | 752 |
| Table 11-14. Transfer Size Register (TRANSFER_SIZE) Field Descriptions.....                                                                  | 752 |
| Table 11-15. Transfer Count Register (TRANSFER_COUNT) Field Descriptions.....                                                                | 753 |
| Table 11-16. Source Transfer Step Size Register (SRC_TRANSFER_STEP) Field Descriptions.....                                                  | 753 |
| Table 11-17. Destination Transfer Step Size Register (DST_TRANSFER_STEP) Field Descriptions.....                                             | 754 |
| Table 11-18. Source/Destination Wrap Size Register (SRC/DST_WRAP_SIZE) Field Descriptions.....                                               | 755 |
| Table 11-19. Source/Destination Wrap Count Register (SRC/DST_WRAP_COUNT) Field Descriptions.....                                             | 755 |
| Table 11-20. Source/Destination Wrap Step Size Registers (SRC/DST_WRAP_STEP) Field Descriptions.....                                         | 756 |
| Table 11-21. Shadow Source Begin and Current Address Pointer Registers (SRC_BEG_ADDR_SHADOW/<br>DST_BEG_ADDR_SHADOW) Field Descriptions..... | 757 |
| Table 11-22. Active Source Begin and Current Address Pointer Registers (SRC_BEG_ADDR/DST_BEG_ADDR) Field<br>Descriptions.....                | 757 |
| Table 11-23. Shadow Destination Begin and Current Address Pointer Registers (SRC_ADDR_SHADOW/<br>DST_ADDR_SHADOW) Field Descriptions.....    | 758 |
| Table 11-24. Active Destination Begin and Current Address Pointer Registers (SRC_ADDR/DST_ADDR) Field<br>Descriptions.....                   | 758 |
| Table 12-1. SPI Module Signal Summary.....                                                                                                   | 762 |
| Table 12-2. SPI Interrupt Flag Modes.....                                                                                                    | 764 |
| Table 12-3. SPI Clocking Scheme Selection Guide.....                                                                                         | 771 |
| Table 12-4. 4-wire vs. 3-wire SPI Pin Functions.....                                                                                         | 773 |
| Table 12-5. 3-Wire SPI Pin Configuration.....                                                                                                | 774 |
| Table 12-6. SPI Base Address Table.....                                                                                                      | 779 |
| Table 12-7. SPI_REGS Registers.....                                                                                                          | 779 |
| Table 12-8. SPI_REGS Access Type Codes.....                                                                                                  | 780 |



|                                                                                                         |     |
|---------------------------------------------------------------------------------------------------------|-----|
| Table 12-9. SPI Configuration Control Register (SPICCR) Field Descriptions.....                         | 781 |
| Table 12-10. SPI Operation Control (SPICCTL) Register Field Descriptions.....                           | 784 |
| Table 12-11. SPI Status (SPISTS) Register Field Descriptions.....                                       | 786 |
| Table 12-12. SPI Baud Rate Register (SPIBRR) Field Descriptions.....                                    | 788 |
| Table 12-13. SPI Emulation Buffer (SPIRXEMU) Register Field Descriptions.....                           | 789 |
| Table 12-14. SPI Serial Input Buffer (SPIRXBUF) Register Field Descriptions.....                        | 790 |
| Table 12-15. SPI Serial Output Buffer (SPITXBUF) Register Field Descriptions.....                       | 790 |
| Table 12-16. SPI Serial Data (SPIDAT) Register Field Descriptions.....                                  | 791 |
| Table 12-17. SPI FIFO Transmit (SPIFFTX) Register Field Descriptions.....                               | 792 |
| Table 12-18. SPI FIFO Receive (SPIFFRX) Register Field Descriptions.....                                | 794 |
| Table 12-19. SPI FIFO Control (SPIFFCT) Register Field Descriptions.....                                | 796 |
| Table 12-20. SPI Priority Control (SPIPRI) Register Field Descriptions.....                             | 797 |
| Table 13-1. SCI Module Signal Summary.....                                                              | 801 |
| Table 13-2. Programming the Data Format Using SCICCR.....                                               | 803 |
| Table 13-3. Asynchronous Baud Register Values for Common SCI Bit Rates.....                             | 810 |
| Table 13-4. SCI Interrupt Flags.....                                                                    | 812 |
| Table 13-5. SCI Base Address Table.....                                                                 | 814 |
| Table 13-6. SCI_REGS Registers.....                                                                     | 814 |
| Table 13-7. SCI_REGS Access Type Codes.....                                                             | 815 |
| Table 13-8. SCI Communications Control Register (SCICCR) Field Descriptions.....                        | 816 |
| Table 13-9. SCI Control Register 1 (SCICTL1) Field Descriptions.....                                    | 818 |
| Table 13-10. SCI Baud Rate (high) (SCIHBAUD) Register Field Descriptions.....                           | 820 |
| Table 13-11. SCI Baud Rate (low) (SCILBAUD) Register Field Descriptions.....                            | 821 |
| Table 13-12. SCI Control Register 2 (SCICTL2) Field Descriptions.....                                   | 822 |
| Table 13-13. SCI Receive Status (SCIRXST) Register Field Descriptions.....                              | 823 |
| Table 13-14. SCI Receive Emulation Buffer (SCIRXEMU) Register Field Descriptions.....                   | 825 |
| Table 13-15. SCI Receive Data Buffer (SCIRXBUF) Register Field Descriptions.....                        | 826 |
| Table 13-16. SCI Transmit Data Buffer (SCITXBUF) Register Field Descriptions.....                       | 827 |
| Table 13-17. SCI FIFO Transmit (SCIFFTX) Register Field Descriptions.....                               | 828 |
| Table 13-18. SCI FIFO Receive (SCIFFRX) Register Field Descriptions.....                                | 829 |
| Table 13-19. SCI FIFO Control (SCIFFCT) Register Field Descriptions.....                                | 831 |
| Table 13-20. SCI Priority Control (SCIPRI) Register Field Descriptions.....                             | 832 |
| Table 14-1. Dependency of Delay d on the Divide-Down Value IPSC.....                                    | 838 |
| Table 14-2. Operating Modes of the I2C Module.....                                                      | 840 |
| Table 14-3. Master-Transmitter/Receiver Bus Activity Defined by the RM, STT, and STP Bits of I2CMR..... | 840 |
| Table 14-4. How the MST and FDF Bits of I2CMR Affect the Role of the TRX Bit of I2CMR.....              | 846 |
| Table 14-5. Ways to Generate a NACK Bit.....                                                            | 850 |
| Table 14-6. Descriptions of the Basic I2C Interrupt Requests.....                                       | 851 |
| Table 14-7. I2C Base Address Table.....                                                                 | 853 |
| Table 14-8. I2C_REGS Registers.....                                                                     | 854 |
| Table 14-9. I2C_REGS Access Type Codes.....                                                             | 855 |
| Table 14-10. I2C Own Address Register (I2COAR) Field Descriptions.....                                  | 856 |
| Table 14-11. I2C Interrupt Enable Register (I2CIER) Field Descriptions.....                             | 857 |
| Table 14-12. I2C Status Register (I2CSTR) Field Descriptions.....                                       | 858 |
| Table 14-13. I2C Clock Low-time Divider (I2CCLKL) Register Field Descriptions.....                      | 862 |
| Table 14-14. I2C Clock High-time Divider (I2CCLKH) Register Field Descriptions.....                     | 862 |
| Table 14-15. I2C Data Count (I2CCNT) Register Field Descriptions.....                                   | 863 |
| Table 14-16. I2C Data Receive Register (I2CDRR) Field Descriptions.....                                 | 864 |
| Table 14-17. I2C Slave Address Register (I2CSAR) Field Descriptions.....                                | 865 |
| Table 14-18. I2C Data Transmit Register (I2CDXR) Field Descriptions.....                                | 866 |
| Table 14-19. I2C Mode Register (I2CMR) Field Descriptions.....                                          | 867 |
| Table 14-20. I2C Interrupt Source (I2CISRC) Register Field Descriptions.....                            | 871 |
| Table 14-21. I2C Extended Mode Register (I2CEMR) Field Descriptions.....                                | 872 |
| Table 14-22. I2C Prescaler (I2CPSC) Register Field Descriptions.....                                    | 873 |
| Table 14-23. I2C FIFO Transmit (I2CFFTX) Register Field Descriptions.....                               | 874 |
| Table 14-24. I2C FIFO Receive (I2CFFRX) Register Field Descriptions.....                                | 876 |
| Table 15-1. McBSP Interface Pins/Signals.....                                                           | 879 |
| Table 15-2. Register Bits That Determine the Number of Phases, Words, and Bits.....                     | 886 |
| Table 15-3. Interrupts and DMA Events Generated by a McBSP.....                                         | 890 |
| Table 15-4. Effects of DLB and CLKSTP on Clock Modes.....                                               | 891 |
| Table 15-5. Choosing an Input Clock for the Sample Rate Generator with the SCLKME and CLKSM Bits.....   | 892 |

|                                                                                                                    |     |
|--------------------------------------------------------------------------------------------------------------------|-----|
| Table 15-6. Polarity Options for the Input to the Sample Rate Generator.....                                       | 892 |
| Table 15-7. Input Clock Selection for Sample Rate Generator.....                                                   | 896 |
| Table 15-8. Block - Channel Assignment.....                                                                        | 904 |
| Table 15-9. 2-Partition Mode.....                                                                                  | 905 |
| Table 15-10. 8-Partition Mode.....                                                                                 | 905 |
| Table 15-11. Receive Channel Assignment and Control With Eight Receive Partitions.....                             | 907 |
| Table 15-12. Transmit Channel Assignment and Control When Eight Transmit Partitions Are Used.....                  | 908 |
| Table 15-13. Selecting a Transmit Multichannel Selection Mode With the XMCM Bits.....                              | 909 |
| Table 15-14. Bits Used to Enable and Configure the Clock Stop Mode.....                                            | 913 |
| Table 15-15. Effects of CLKSTP, CLKXP, and CLKRP on the Clock Scheme.....                                          | 913 |
| Table 15-16. Bit Values Required to Configure the McBSP as an SPI Master.....                                      | 917 |
| Table 15-17. Bit Values Required to Configure the McBSP as an SPI Slave.....                                       | 918 |
| Table 15-18. Register Bits Used to Reset or Enable the McBSP Receiver Field Descriptions.....                      | 920 |
| Table 15-19. Reset State of Each McBSP Pin.....                                                                    | 920 |
| Table 15-20. Register Bit Used to Enable/Disable the Digital Loopback Mode.....                                    | 921 |
| Table 15-21. Receive Signals Connected to Transmit Signals in Digital Loopback Mode.....                           | 921 |
| Table 15-22. Register Bits Used to Enable/Disable the Clock Stop Mode.....                                         | 921 |
| Table 15-23. Effects of CLKSTP, CLKXP, and CLKRP on the Clock Scheme.....                                          | 922 |
| Table 15-24. Register Bit Used to Enable/Disable the Receive Multichannel Selection Mode.....                      | 922 |
| Table 15-25. Register Bit Used to Choose One or Two Phases for the Receive Frame.....                              | 922 |
| Table 15-26. Register Bits Used to Set the Receive Word Length(s).....                                             | 923 |
| Table 15-27. Register Bits Used to Set the Receive Frame Length.....                                               | 923 |
| Table 15-28. How to Calculate the Length of the Receive Frame.....                                                 | 924 |
| Table 15-29. Register Bit Used to Enable/Disable the Receive Frame-Synchronization Ignore Function.....            | 924 |
| Table 15-30. Register Bits Used to Set the Receive Companding Mode.....                                            | 925 |
| Table 15-31. Register Bits Used to Set the Receive Data Delay.....                                                 | 926 |
| Table 15-32. Register Bits Used to Set the Receive Sign-Extension and Justification Mode.....                      | 928 |
| Table 15-33. Example: Use of RJUST Field With 12-Bit Data Value ABCh.....                                          | 928 |
| Table 15-34. Example: Use of RJUST Field With 20-Bit Data Value ABCDEh.....                                        | 928 |
| Table 15-35. Register Bits Used to Set the Receive Interrupt Mode.....                                             | 929 |
| Table 15-36. Register Bits Used to Set the Receive Frame Synchronization Mode.....                                 | 930 |
| Table 15-37. Select Sources to Provide the Receive Frame-Synchronization Signal and the Effect on the FSR Pin..... | 931 |
| Table 15-38. Register Bit Used to Set Receive Frame-Synchronization Polarity.....                                  | 932 |
| Table 15-39. Register Bits Used to Set the SRG Frame-Synchronization Period and Pulse Width.....                   | 933 |
| Table 15-40. Register Bits Used to Set the Receive Clock Mode.....                                                 | 934 |
| Table 15-41. Receive Clock Signal Source Selection.....                                                            | 935 |
| Table 15-42. Register Bit Used to Set Receive Clock Polarity.....                                                  | 935 |
| Table 15-43. Register Bits Used to Set the Sample Rate Generator (SRG) Clock Divide-Down Value.....                | 936 |
| Table 15-44. Register Bit Used to Set the SRG Clock Synchronization Mode.....                                      | 937 |
| Table 15-45. Register Bits Used to Set the SRG Clock Mode (Choose an Input Clock).....                             | 937 |
| Table 15-46. Register Bits Used to Set the SRG Input Clock Polarity.....                                           | 938 |
| Table 15-47. Register Bits Used to Place Transmitter in Reset Field Descriptions.....                              | 939 |
| Table 15-48. Register Bit Used to Enable/Disable the Digital Loopback Mode.....                                    | 940 |
| Table 15-49. Receive Signals Connected to Transmit Signals in Digital Loopback Mode.....                           | 940 |
| Table 15-50. Register Bits Used to Enable/Disable the Clock Stop Mode.....                                         | 940 |
| Table 15-51. Effects of CLKSTP, CLKXP, and CLKRP on the Clock Scheme.....                                          | 941 |
| Table 15-52. Register Bits Used to Enable/Disable Transmit Multichannel Selection.....                             | 941 |
| Table 15-53. Register Bit Used to Choose 1 or 2 Phases for the Transmit Frame.....                                 | 942 |
| Table 15-54. Register Bits Used to Set the Transmit Word Length(s).....                                            | 942 |
| Table 15-55. Register Bits Used to Set the Transmit Frame Length.....                                              | 943 |
| Table 15-56. How to Calculate Frame Length.....                                                                    | 943 |
| Table 15-57. Register Bit Used to Enable/Disable the Transmit Frame-Synchronization Ignore Function.....           | 944 |
| Table 15-58. Register Bits Used to Set the Transmit Companding Mode.....                                           | 945 |
| Table 15-59. Register Bits Used to Set the Transmit Data Delay.....                                                | 946 |
| Table 15-60. Register Bit Used to Set the Transmit DXENA (DX Delay Enabler) Mode.....                              | 948 |
| Table 15-61. Register Bits Used to Set the Transmit Interrupt Mode.....                                            | 948 |
| Table 15-62. Register Bits Used to Set the Transmit Frame-Synchronization Mode.....                                | 949 |
| Table 15-63. How FSXM and FSGM Select the Source of Transmit Frame-Synchronization Pulses.....                     | 949 |
| Table 15-64. Register Bit Used to Set Transmit Frame-Synchronization Polarity.....                                 | 950 |
| Table 15-65. Register Bits Used to Set SRG Frame-Synchronization Period and Pulse Width.....                       | 951 |
| Table 15-66. Register Bit Used to Set the Transmit Clock Mode.....                                                 | 952 |

|                                                                                                                               |      |
|-------------------------------------------------------------------------------------------------------------------------------|------|
| Table 15-67. How the CLKXM Bit Selects the Transmit Clock and the Corresponding Status of the MCLKX pin.....                  | 952  |
| Table 15-68. Register Bit Used to Set Transmit Clock Polarity.....                                                            | 952  |
| Table 15-69. McBSP Emulation Modes Selectable with FREE and SOFT Bits of SPCR2.....                                           | 954  |
| Table 15-70. Reset State of Each McBSP Pin.....                                                                               | 954  |
| Table 15-71. MCBSP_REGS Registers.....                                                                                        | 959  |
| Table 15-72. McBSP_REGS Access Type Codes.....                                                                                | 960  |
| Table 15-73. DRR2 Register Field Descriptions.....                                                                            | 961  |
| Table 15-74. DRR1 Register Field Descriptions.....                                                                            | 961  |
| Table 15-75. DXR2 Register Field Descriptions.....                                                                            | 962  |
| Table 15-76. DXR1 Register Field Descriptions.....                                                                            | 962  |
| Table 15-77. SPCR2 Register Field Descriptions.....                                                                           | 963  |
| Table 15-78. SPCR1 Register Field Descriptions.....                                                                           | 966  |
| Table 15-79. RCR2 Register Field Descriptions.....                                                                            | 969  |
| Table 15-80. RCR1 Register Field Descriptions.....                                                                            | 971  |
| Table 15-81. XCR2 Register Field Descriptions.....                                                                            | 972  |
| Table 15-82. XCR1 Register Field Descriptions.....                                                                            | 974  |
| Table 15-83. SRGR2 Register Field Descriptions.....                                                                           | 975  |
| Table 15-84. SRGR1 Register Field Descriptions.....                                                                           | 977  |
| Table 15-85. MCR2 Register Field Descriptions.....                                                                            | 978  |
| Table 15-86. MCR1 Register Field Descriptions.....                                                                            | 980  |
| Table 15-87. RCERA Register Field Descriptions.....                                                                           | 983  |
| Table 15-88. RCERB Register Field Descriptions.....                                                                           | 983  |
| Table 15-89. XCERA Register Field Descriptions.....                                                                           | 984  |
| Table 15-90. XCERB Register Field Descriptions.....                                                                           | 985  |
| Table 15-91. PCR Register Field Descriptions.....                                                                             | 986  |
| Table 15-92. RCERC Register Field Descriptions.....                                                                           | 989  |
| Table 15-93. RCERD Register Field Descriptions.....                                                                           | 989  |
| Table 15-94. XCERC Register Field Descriptions.....                                                                           | 990  |
| Table 15-95. XCERD Register Field Descriptions.....                                                                           | 991  |
| Table 15-96. RCERE Register Field Descriptions.....                                                                           | 992  |
| Table 15-97. RCERF Register Field Descriptions.....                                                                           | 992  |
| Table 15-98. XCERE Register Field Descriptions.....                                                                           | 993  |
| Table 15-99. XCERF Register Field Descriptions.....                                                                           | 994  |
| Table 15-100. RCERG Register Field Descriptions.....                                                                          | 995  |
| Table 15-101. RCERH Register Field Descriptions.....                                                                          | 995  |
| Table 15-102. XCERG Register Field Descriptions.....                                                                          | 996  |
| Table 15-103. XCERH Register Field Descriptions.....                                                                          | 997  |
| Table 15-104. MFFINT Register Field Descriptions.....                                                                         | 998  |
| Table 16-1. Message Object Behavior Configuration.....                                                                        | 1007 |
| Table 16-2. eCAN-A Mailbox RAM Layout.....                                                                                    | 1008 |
| Table 16-3. Addresses of LAM, MOTS, and MOTO Registers for Mailboxes (eCAN-A).....                                            | 1009 |
| Table 16-4. BRP Field for Bit Rates (BT = 15, TSEG1 <sub>reg</sub> = 10, TSEG2 <sub>reg</sub> = 2, Sampling Point = 80%)..... | 1013 |
| Table 16-5. Achieving Different Sampling Points With a BT of 15.....                                                          | 1013 |
| Table 16-6. eCAN Interrupt Assertion/Clearing.....                                                                            | 1020 |
| Table 16-7. Local-Acceptance-Mask Register (LAM <sub>n</sub> ) Field Descriptions.....                                        | 1024 |
| Table 16-8. Mailbox-Enable Register (CANME) Field Descriptions.....                                                           | 1025 |
| Table 16-9. Mailbox-Direction Register (CANMD) Field Descriptions.....                                                        | 1025 |
| Table 16-10. Transmission-Request Set Register (CANTRS) Field Descriptions.....                                               | 1026 |
| Table 16-11. Transmission-Request-Reset Register (CANTRR) Field Descriptions.....                                             | 1027 |
| Table 16-12. Transmission-Acknowledge Register (CANTA) Field Descriptions.....                                                | 1027 |
| Table 16-13. Abort-Acknowledge Register (CANAA) Field Descriptions.....                                                       | 1028 |
| Table 16-14. Received-Message-Pending Register (CANRMP) Field Descriptions.....                                               | 1028 |
| Table 16-15. Received-Message-Lost Register (CANRML) Field Descriptions.....                                                  | 1029 |
| Table 16-16. Remote-Frame-Pending Register (CANRFP) Field Descriptions.....                                                   | 1030 |
| Table 16-17. Global Acceptance Mask Register (CANGAM) Field Descriptions.....                                                 | 1031 |
| Table 16-18. Master Control Register (CANMC) Field Descriptions.....                                                          | 1032 |
| Table 16-19. Bit-Timing Configuration Register (CANBTC) Field Descriptions.....                                               | 1035 |
| Table 16-20. Error and Status Register (CANES) Field Descriptions.....                                                        | 1037 |
| Table 16-21. Global Interrupt Flag Registers (CANGIF0/CANGIF1) Field Descriptions.....                                        | 1042 |
| Table 16-22. Global Interrupt Mask Register (CANGIM) Field Descriptions.....                                                  | 1043 |
| Table 16-23. Mailbox Interrupt Mask Register (CANMIM) Field Descriptions.....                                                 | 1044 |

|                                                                                                                                             |      |
|---------------------------------------------------------------------------------------------------------------------------------------------|------|
| Table 16-24. Mailbox Interrupt Level Register (CANMIL) Field Descriptions.....                                                              | 1045 |
| Table 16-25. Overwrite Protection Control Register (CANOPC) Field Descriptions.....                                                         | 1045 |
| Table 16-26. TX I/O Control Register (CANTIOC) Field Descriptions.....                                                                      | 1046 |
| Table 16-27. RX I/O Control Register (CANRIOC) Field Descriptions.....                                                                      | 1046 |
| Table 16-28. Time-Stamp Counter Register (CANTSC) Field Descriptions.....                                                                   | 1047 |
| Table 16-29. Message-Object Time-Out Registers (MOTO) Field Descriptions.....                                                               | 1048 |
| Table 16-30. Message-Object Time Stamp Registers (MOTS) Field Descriptions.....                                                             | 1048 |
| Table 16-31. Time-Out Control Register (CANTOC) Field Descriptions.....                                                                     | 1049 |
| Table 16-32. Time-Out Status Register (CANTOS) Field Descriptions.....                                                                      | 1049 |
| Table 16-33. Message Identifier Register (MSGID) Field Descriptions.....                                                                    | 1051 |
| Table 16-34. Message-Control Register (MSGCTRL) Field Descriptions.....                                                                     | 1052 |
| Table 17-1. USB Memory Access From Software.....                                                                                            | 1067 |
| Table 17-2. USB Memory Access From CCS.....                                                                                                 | 1068 |
| Table 17-3. Universal Serial Bus (USB) Controller Register Map.....                                                                         | 1069 |
| Table 17-4. Function Address Register (USBFADDR) Field Descriptions.....                                                                    | 1072 |
| Table 17-5. Power Management Register (USBPOWER) in Host Mode Field Descriptions.....                                                       | 1073 |
| Table 17-6. Power Management Register (USBPOWER) in Device Mode Field Descriptions.....                                                     | 1074 |
| Table 17-7. USB Transmit Interrupt Status Register (USBTXIS) Field Descriptions.....                                                        | 1075 |
| Table 17-8. USB Receive Interrupt Status Register (USBRXIS) Field Descriptions.....                                                         | 1076 |
| Table 17-9. USB Transmit Interrupt Status Register (USBTXIE) Field Descriptions.....                                                        | 1077 |
| Table 17-10. USB Receive Interrupt Register (USBRXIE) Field Descriptions.....                                                               | 1078 |
| Table 17-11. USB General Interrupt Status Register (USBIS) in Host Mode Field Descriptions.....                                             | 1079 |
| Table 17-12. USB General Interrupt Status Register (USBIS) in Device Mode Field Descriptions.....                                           | 1080 |
| Table 17-13. USB Interrupt Enable Register (USBIE) in Host Mode Field Descriptions.....                                                     | 1081 |
| Table 17-14. USB Interrupt Enable Register (USBIE) in Device Mode Field Descriptions.....                                                   | 1082 |
| Table 17-15. Frame Number Register (FRAME) Field Descriptions.....                                                                          | 1083 |
| Table 17-16. USB Endpoint Index Register (USBEPIDX) Field Descriptions.....                                                                 | 1083 |
| Table 17-17. USB Test Mode Register (USBTEST) in Host Mode Field Descriptions.....                                                          | 1084 |
| Table 17-18. USB Test Mode Register (USBTEST) in Device Mode Field Descriptions.....                                                        | 1085 |
| Table 17-19. USB FIFO Endpoint <i>n</i> Register (USBFIFO[ <i>n</i> ]) Field Descriptions.....                                              | 1086 |
| Table 17-20. USB Device Control Register (USBDEVCTL) Field Descriptions.....                                                                | 1087 |
| Table 17-21. USB Transmit Dynamic FIFO Sizing Register (USBTXFIFOSZ) Field Descriptions.....                                                | 1089 |
| Table 17-22. USB Receive Dynamic FIFO Sizing Register (USBRXFIFOSZ) Field Descriptions.....                                                 | 1090 |
| Table 17-23. USB Transmit FIFO Start Address Register (USBTXFIFOADDR) Field Descriptions.....                                               | 1091 |
| Table 17-24. USB Receive FIFO Start Address Register (USBRXFIFOADDR) Field Descriptions.....                                                | 1092 |
| Table 17-25. USB Connect Timing Register (USBCONTIM) Field Descriptions.....                                                                | 1093 |
| Table 17-26. USB Full-Speed Last Transaction to End of Frame Timing Register (USBFSEOF) Field Descriptions.....                             | 1094 |
| Table 17-27. USB Low-Speed Last Transaction to End of Frame Timing Register (USBLSEOF) Field Descriptions.....                              | 1094 |
| Table 17-28. USB Transmit Functional Address Endpoint <i>n</i> Registers (USBTXFUNCADDR[ <i>n</i> ]) Field Descriptions.....                | 1095 |
| Table 17-29. USB Transmit Hub Address Endpoint <i>n</i> Registers(USBTXHUBADDR[ <i>n</i> ]) Field Descriptions.....                         | 1096 |
| Table 17-30. USB Transmit Hub Port Endpoint <i>n</i> Registers(USBTXHUBPORT[ <i>n</i> ]) Field Descriptions.....                            | 1097 |
| Table 17-31. USB Receive Functional Address Endpoint <i>n</i> Registers(USBFIFO[ <i>n</i> ]) Field Descriptions.....                        | 1098 |
| Table 17-32. USB Receive Hub Address Endpoint <i>n</i> Registers(USBRXHUBADDR[ <i>n</i> ]) Field Descriptions.....                          | 1099 |
| Table 17-33. USB Transmit Hub Port Endpoint <i>n</i> Registers(USBRXHUBPORT[ <i>n</i> ]) Field Descriptions.....                            | 1100 |
| Table 17-34. USB Maximum Transmit Data Endpoint <i>n</i> Registers(USBTXMAXP[ <i>n</i> ]) Field Descriptions.....                           | 1101 |
| Table 17-35. USB Control and Status Endpoint 0 Low Register(USBCSRL0) in Host Mode Field Descriptions.....                                  | 1102 |
| Table 17-36. USB Control and Status Endpoint 0 Low Register (USBCSRL0) in Device Mode Field Descriptions.....                               | 1103 |
| Table 17-37. USB Control and Status Endpoint 0 High Register (USBCSRH0) in Host Mode Field Descriptions.....                                | 1105 |
| Table 17-38. USB Control and Status Endpoint 0 High Register (USBCSRH0) in Device Mode Field Descriptions.....                              | 1106 |
| Table 17-39. USB Receive Byte Count Endpoint 0 Register (USBCOUNT0) Field Descriptions.....                                                 | 1107 |
| Table 17-40. USB Type Endpoint 0 Register (USBTYP0) Field Descriptions.....                                                                 | 1107 |
| Table 17-41. USB NAK Limit Register (USBNAKLMT) Field Descriptions.....                                                                     | 1108 |
| Table 17-42. USB Transmit Control and Status Endpoint <i>n</i> Low Register (USBTXCSRL[ <i>n</i> ]) in Host Mode Field Descriptions.....    | 1109 |
| Table 17-43. USB Transmit Control and Status Endpoint <i>n</i> Low Register (USBTXCSRL[ <i>n</i> ]) in Device Mode Field Descriptions.....  | 1110 |
| Table 17-44. USB Transmit Control and Status Endpoint <i>n</i> High Register (USBTXCSRH[ <i>n</i> ]) in Host Mode Field Descriptions.....   | 1112 |
| Table 17-45. USB Transmit Control and Status Endpoint <i>n</i> High Register (USBTXCSRH[ <i>n</i> ]) in Device Mode Field Descriptions..... | 1113 |
| Table 17-46. USB Maximum Receive Data Endpoint <i>n</i> Registers (USBTXMAXP[ <i>n</i> ]) Field Descriptions.....                           | 1114 |



|                                                                                                                               |      |
|-------------------------------------------------------------------------------------------------------------------------------|------|
| Table 17-47. USB Control and Status Endpoint $n$ Low Register(USBCSRL[ $n$ ]) in Host Mode Field Descriptions.....            | 1115 |
| Table 17-48. USB Control and Status Endpoint 0 Low Register(USBCSRL[ $n$ ]) in Device Mode Field Descriptions.....            | 1116 |
| Table 17-49. USB Control and Status Endpoint $n$ High Register (USBCSRH[ $n$ ]) in Host Mode Field Descriptions.....          | 1118 |
| Table 17-50. USB Control and Status Endpoint 0 High Register(USBCSRH[ $n$ ]) in Device Mode Field Descriptions.....           | 1119 |
| Table 17-51. USB Maximum Receive Data Endpoint $n$ Registers (USBRXCOUNT[ $n$ ]) Field Descriptions.....                      | 1120 |
| Table 17-52. USB Host Transmit Configure Type Endpoint $n$ Register(USBTXTYPE[ $n$ ]) Field Descriptions.....                 | 1121 |
| Table 17-53. USBTXINTERVAL[ $n$ ] Frame Numbers.....                                                                          | 1122 |
| Table 17-54. USB Host Transmit Interval Endpoint $n$ Register(USBTXINTERVAL[ $n$ ]) Field Descriptions.....                   | 1122 |
| Table 17-55. USB Host Configure Receive Type Endpoint $n$ Register (USBRXTYPE[ $n$ ]) Field Descriptions.....                 | 1123 |
| Table 17-56. USBRXINTERVAL[ $n$ ] Frame Numbers.....                                                                          | 1124 |
| Table 17-57. USB Host Receive Polling Interval Endpoint $n$ Register (USBRXINTERVAL[ $n$ ]) Field Descriptions.....           | 1124 |
| Table 17-58. USB Request Packet Count in Block Transfer Endpoint $n$ Registers (USBRQPKTCOUNT[ $n$ ]) Field Descriptions..... | 1125 |
| Table 17-59. USB Receive Double Packet Buffer Disable Register (USBRXDPKTBUFFDIS) Field Descriptions.....                     | 1126 |
| Table 17-60. USB Transmit Double Packet Buffer Disable Register (USBTXDPKTBUFFDIS) Field Descriptions.....                    | 1127 |
| Table 17-61. USB External Power Control Register (USBEPCC) Field Descriptions.....                                            | 1128 |
| Table 17-62. USB External Power Control Raw Interrupt Status Register (USBEPCCRIS) Field Descriptions.....                    | 1130 |
| Table 17-63. USB External Power Control Interrupt Mask Register (USBEPCCIM) Field Descriptions.....                           | 1131 |
| Table 17-64. USB External Power Control Interrupt Status and Clear Register (USBEPCCISC) Field Descriptions.....              | 1132 |
| Table 17-65. USB Device RESUME Raw Interrupt Status Register (USBDRRIS) Field Descriptions.....                               | 1133 |
| Table 17-66. USB Device RESUME Raw Interrupt Status Register (USBDRRIS) Field Descriptions.....                               | 1134 |
| Table 17-67. USB Device RESUME Interrupt Status and Clear Register (USBDRISC) Field Descriptions.....                         | 1135 |
| Table 17-68. USB General-Purpose Control and Status Register (USBGPCS) Field Descriptions.....                                | 1136 |
| Table 17-69. USB DMA Select Register (USBDMASEL) Field Descriptions.....                                                      | 1137 |



## About This Manual

This Technical Reference Manual (TRM) details the integration, the environment, the functional description, and the programming models for each peripheral and subsystem in the device.

The TRM should not be considered a substitute for the data manual, rather a companion guide that should be used alongside the device-specific data manual to understand the details to program the device. The primary purpose of the TRM is to abstract the programming details of the device from the data manual. This allows the data manual to outline the high-level features of the device without unnecessary information about register descriptions or programming models.

## Notational Conventions

This document uses the following conventions.

- Hexadecimal numbers can be shown with the suffix h or the prefix 0x. For example, the following number is 40 hexadecimal (decimal 64): 40h or 0x40.
- Registers in this document are shown in figures and described in tables.
  - Each register figure shows a rectangle divided into fields that represent the fields of the register. Each field is labeled with its bit name, its beginning and ending bit numbers above, and its read/write properties with default reset value below. A legend explains the notation used for the properties.
  - Reserved bits in a register figure can have one of multiple meanings:
    - Not implemented on the device
    - Reserved for future device expansion
    - Reserved for TI testing
    - Reserved configurations of the device that are not supported
  - Writing nondefault values to the Reserved bits could cause unexpected behavior and should be avoided.

## Glossary

[TI Glossary](#) This glossary lists and explains terms, acronyms, and definitions.

## Related Documentation From Texas Instruments

For a complete listing of related documentation and development-support tools for these devices, visit the Texas Instruments website at <http://www.ti.com>. Additionally, the [TMS320C28x DSP CPU and Instruction Set Reference Guide](#) and the [TMS320C28x Floating Point Unit and Instruction Set Reference Guide](#) must be used in conjunction with this TRM.

## Support Resources

[TI E2E™ support forums](#) are an engineer's go-to source for fast, verified answers and design help — straight from the experts. Search existing answers or ask your own question to get the quick design help you need.

Linked content is provided "AS IS" by the respective contributors. They do not constitute TI specifications and do not necessarily reflect TI's views; see TI's [Terms of Use](#).

## Trademarks

TI E2E™ and Code Composer Studio™ are trademarks of Texas Instruments.  
USB Specification Revision 2.0™ is a trademark of Compaq Computer Corp.

All trademarks are the property of their respective owners.

This chapter describes how various system controls and interrupts work and provides information on the:

- Flash and one-time programmable (OTP) memories
- Code security module (CSM), which is a security feature.
- Clocking mechanisms including the oscillator, PLL, XCLKOUT, watchdog module, and the low-power modes. In addition, the 32-bit CPU-Timers are also described.
- GPIO multiplexing (MUX) registers used to select the operation of shared pins on the device.
- Accessing the peripheral frames to write to and read from various peripheral registers on the device.
- Interrupt sources both external and the peripheral interrupt expansion (PIE) block that multiplexes numerous interrupt sources into a smaller set of interrupt inputs.

For more information on the Viterbi, Complex Math, CRC Unit (VCU), refer to the [TMS320C28x Extended Instruction Sets Technical Reference Manual](#).

|                                                      |            |
|------------------------------------------------------|------------|
| <b>1.1 Flash and OTP Memory Blocks.....</b>          | <b>40</b>  |
| <b>1.2 Code Security Module (CSM).....</b>           | <b>52</b>  |
| <b>1.3 Clocking.....</b>                             | <b>62</b>  |
| <b>1.4 General-Purpose Input/Output (GPIO).....</b>  | <b>107</b> |
| <b>1.5 Peripheral Frames.....</b>                    | <b>157</b> |
| <b>1.6 Peripheral Interrupt Expansion (PIE).....</b> | <b>167</b> |
| <b>1.7 VREG/BOR/POR.....</b>                         | <b>193</b> |

## 1.1 Flash and OTP Memory Blocks

This section describes the proper sequence to configure the wait states and operating mode of flash and one-time programmable (OTP) memories. It also includes information on flash and OTP power modes and how to improve flash performance by enabling the flash pipeline mode.

### 1.1.1 Flash Memory

The on-chip flash is uniformly mapped in both program and data memory space. This flash memory is always enabled and features:

- **Multiple sectors**

The minimum amount of flash memory that can be erased is a sector. Having multiple sectors provides the option of leaving some sectors programmed and only erasing specific sectors.

- **Code security**

The flash is protected by the Code Security Module (CSM). By programming a password into the flash, the user can prevent access to the flash by unauthorized persons. See [Section 1.2](#) for information on using the Code Security Module.

- **Low power modes**

To save power when the flash is not in use, two levels of low power modes are available. See [Section 1.1.3](#) for more information on the available flash power modes.

- **Configurable wait states**

Configurable wait states can be adjusted based on CPU frequency to give the best performance for a given execution speed.

- **Enhanced performance**

A flash pipeline mode is provided to improve performance of linear code execution.

### 1.1.2 OTP Memory

The 1K x 16 block of one-time programmable (OTP) memory is uniformly mapped in both program and data memory space. Thus, the OTP can be used to program data or code. This block, unlike flash, can be programmed only one time and cannot be erased.

### 1.1.3 Flash and OTP Power Modes

The following operating states apply to the flash and OTP memory:

- **Reset or Sleep State**

This is the state after a device reset. In this state, the bank and pump are in a sleep state (lowest power). When the flash is in the sleep state, a CPU data read or opcode fetch to the flash or OTP memory map area will automatically initiate a change in power modes to the standby state and then to the active state. During this transition time to the active state, the CPU will automatically be stalled. Once the transition to the active state is completed, the CPU access will complete as normal.

- **Standby State**

In this state, the bank and pump are in standby power mode state. This state uses more power than the sleep state, but takes a shorter time to transition to the active or read state. When the flash is in the standby state, a CPU data read or opcode fetch to the flash or OTP memory map area will automatically initiate a change in power modes to the active state. During this transition time to the active state, the CPU will automatically be stalled. Once the flash/OTP has reached the active state, the CPU access will complete as normal.

- **Active or Read State**

In this state, the bank and pump are in active power mode state (highest power). The CPU read or fetch access wait states to the flash/OTP memory map area is controlled by the FBANKWAIT and FOTPWAIT registers. A prefetch mechanism called flash pipeline can also be enabled to improve fetch performance for linear code execution.

---

#### Note

During the boot process, the Boot ROM performs a dummy read of the Code Security Module (CSM) password locations located in the flash. This read is performed to unlock a new or erased device that has no password stored in it so that flash programming or loading of code into CSM protected SARAM can be performed. On devices with a password stored, this read has no effect and the CSM remains locked (see [Section 1.2](#) for information on the CSM). One effect of this read is that the flash will transition from the sleep (reset) state to the active state.

---

The flash/OTP bank and pump are always in the same power mode. See [Figure 1-1](#) for a graphic depiction of the available power states. You can change the current flash/OTP memory power state as follows:

- **To move to a lower power state**

Change the PWR mode bits from a higher power mode to a lower power mode. This change instantaneously moves the flash/OTP bank to the lower power state. This register should be accessed only by code running outside the flash/OTP memory.

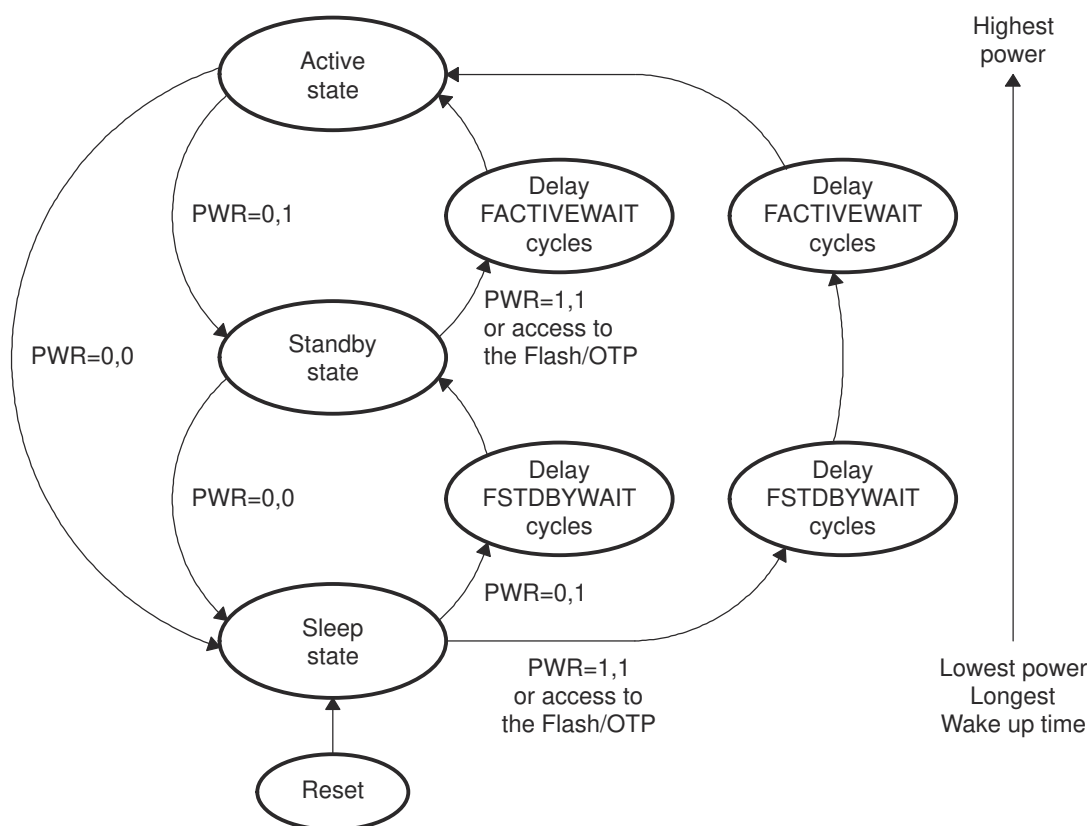
- **To move to a higher power state**

To move from a lower power state to a higher power state, there are two options.

1. Change the FPWR register from a lower state to a higher state. This access brings the flash/OTP memory to the higher state.
2. Access the flash or OTP memory by a read access or program opcode fetch access. This access automatically brings the flash/OTP memory to the active state.

There is a delay when moving from a lower power state to a higher one. See [Figure 1-1](#). This delay is required to allow the flash to stabilize at the higher power mode. If any access to the flash/OTP memory occurs during this delay the CPU automatically stalls until the delay is complete.





**Figure 1-1. Flash Power Mode State Diagram**

The duration of the delay is determined by the FSTDBYWAIT and FACTIVEWAIT registers. Moving from the sleep state to a standby state is delayed by a count determined by the FSTDBYWAIT register. Moving from the standby state to the active state is delayed by a count determined by the FACTIVEWAIT register. Moving from the sleep mode (lowest power) to the active mode (highest power) is delayed by FSTDBYWAIT + FACTIVEWAIT. These registers should be left in their default state.

### 1.1.3.1 Flash and OTP Performance

CPU read or data fetch operations to the flash/OTP can take one of the following forms:

- 32-bit instruction fetch
- 16-bit or 32-bit data space read
- 16-bit program space read

Once flash is in the active power state, then a read or fetch access to the bank memory map area can be classified as a flash access or an OTP access.

The main flash array is organized into rows and columns. The rows contain 2048 bits of information. Accesses to flash and OTP are one of three types:

#### 1. Flash Memory Random Access

The first access to a 2048 bit row is considered a random access.

#### 2. Flash Memory Paged Access

While the first access to a row is considered a random access, subsequent accesses within the same row are termed paged accesses.

The number of wait states for both a random and a paged access can be configured by programming the FBANKWAIT register. The number of wait states used by a random access is controlled by the RANDWAIT bits and the number of wait states used by a paged access is controlled by the PAGEWAIT bits. The

FBANKWAIT register defaults to a worst-case wait state count and, thus, needs to be initialized for the appropriate number of wait states to improve performance based on the CPU clock rate and the access time of the flash. The flash supports 0-wait accesses when the PAGEWAIT bits are set to zero. This assumes that the CPU speed is low enough to accommodate the access time. To determine the random and paged access time requirements, refer to the datasheet for your particular device.

### 3. OTP Access

Read or fetch accesses to the OTP are controlled by the OTPWAIT bits in the FOTPWAIT register. Accesses to the OTP take longer than the flash and there is no paged mode. To determine OTP access time requirements, see the datasheet for your particular device.

Some other points to keep in mind when working with flash:

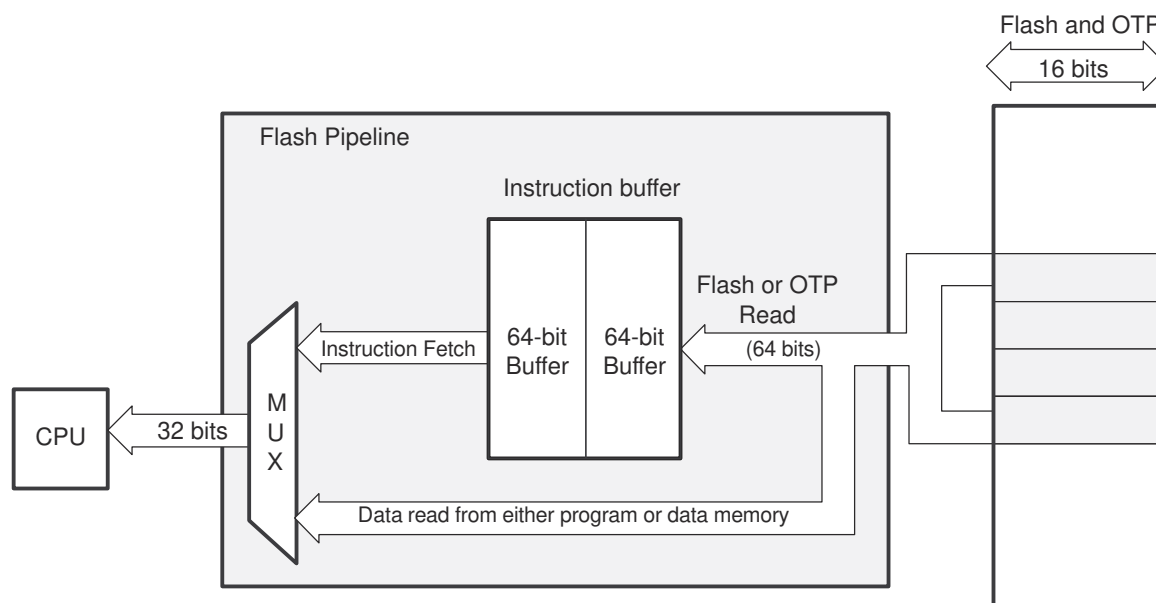
- CPU writes to the flash or OTP memory map area are ignored. They complete in a single cycle.
- When the Code Security Module (CSM) is secured, reads to the flash/OTP memory map area from outside the secure zone take the same number of cycles as a normal access. However, the read operation returns a zero.
- Reads of the CSM password locations are hardwired for 16 wait-states. The PAGEWAIT and RANDOMWAIT bits have no effect on these locations. See [Section 1.2](#) for more information on the CSM.

#### 1.1.3.2 Flash Pipeline Mode

Flash memory is typically used to store application code. During code execution, instructions are fetched from sequential memory addresses, except when a discontinuity occurs. Usually the portion of the code that resides in sequential addresses makes up the majority of the application code and is referred to as linear code. To improve the performance of linear code execution, a flash pipeline mode has been implemented. The flash pipeline feature is disabled by default. Setting the ENPIPE bit in the FOPT register enables this mode. The flash pipeline mode is independent of the CPU pipeline.

An instruction fetch from the flash or OTP reads out 64 bits per access. The starting address of the access from flash is automatically aligned to a 64-bit boundary such that the instruction location is within the 64 bits to be fetched. With flash pipeline mode enabled (see [Figure 1-2](#)), the 64 bits read from the instruction fetch are stored in a 64-bit wide by 2-level deep instruction pre-fetch buffer. The contents of this pre-fetch buffer are then sent to the CPU for processing as required.

Up to two 32-bit instructions or up to four 16-bit instructions can reside within a single 64-bit access. The majority of C28x instructions are 16 bits, so for every 64-bit instruction fetch from the flash bank it is likely that there are up to four instructions in the pre-fetch buffer ready to process through the CPU. During the time it takes to process these instructions, the flash pipeline automatically initiates another access to the flash bank to pre-fetch the next 64 bits. In this manner, the flash pipeline mode works in the background to keep the instruction pre-fetch buffers as full as possible. Using this technique, the overall efficiency of sequential code execution from flash or OTP is improved significantly.



**Figure 1-2. Flash Pipeline**

The flash pipeline pre-fetch is aborted only on a PC discontinuity caused by executing an instruction such as a branch, BANTZ, call, or loop. When this occurs, the pre-fetch is aborted and the contents of the pre-fetch buffer are flushed. There are two possible scenarios when this occurs:

1. If the destination address is within the flash or OTP, the pre-fetch aborts and then resumes at the destination address.
2. If the destination address is outside of the flash and OTP, the pre-fetch is aborted and begins again only when a branch is made back into the flash or OTP. The flash pipeline pre-fetch mechanism only applies to instruction fetches from program space. Data reads from data memory and from program memory do not utilize the pre-fetch buffer capability and thus bypass the pre-fetch buffer. For example, instructions such as MAC, DMAC, and PREAD read a data value from program memory. When this read happens, the pre-fetch buffer is bypassed but the buffer is not flushed. If an instruction pre-fetch is already in progress when a data read operation is initiated, then the data read will be stalled until the pre-fetch completes.

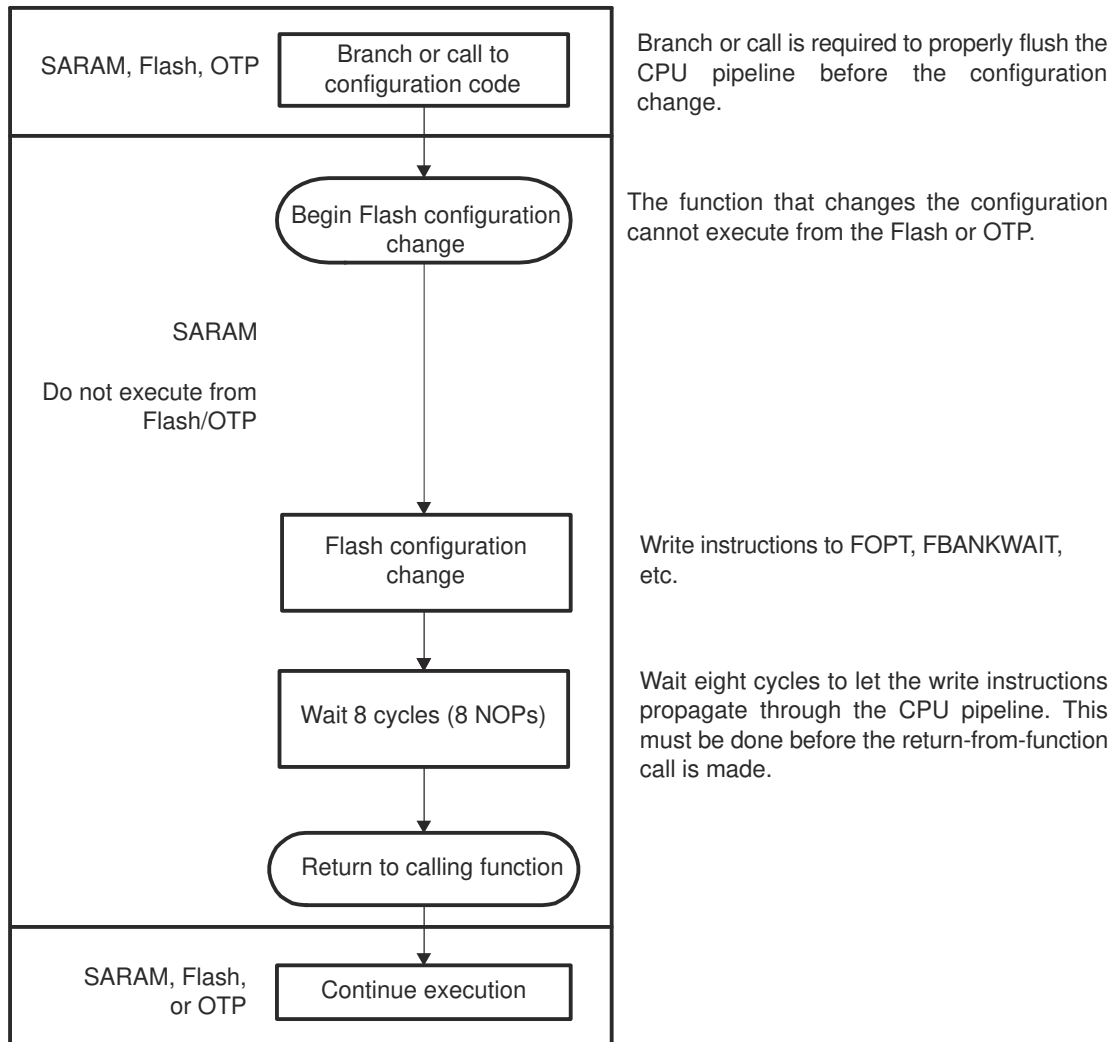
### 1.1.3.3 Reserved Locations Within Flash and OTP

When allocating code and data to flash and OTP memory, keep the following in mind:

1. Address locations 0x3F 7FF6 and 0x3F 7FF7 are reserved for an entry into flash branch instruction. When the boot to flash boot option is used, the boot ROM will jump to address 0x3F 7FF6. If you program a branch instruction here that will then re-direct code execution to the entry point of the application.
2. Addresses from 0x3F 7FF0 to 0x3F 7FF5 are reserved for data variables and should not contain program code.

### 1.1.3.4 Procedure to Change the Flash Configuration Registers

During flash configuration, no accesses to the flash or OTP can be in progress. This includes instructions still in the CPU pipeline, data reads, and instruction pre-fetch operations. To be sure that no access takes place during the configuration change, you should follow the procedure shown in [Figure 1-3](#) for any code that modifies the FOPT, FPWR, FBANKWAIT, or FOTPWAIT registers.



**Figure 1-3. Flash Configuration Access Flow Diagram**

### 1.1.4 Flash and OTP Registers

The Flash and OTP memory can be configured by the registers shown in [Table 1-1](#). The configuration registers are all EALLOW protected. The bit descriptions are in [Figure 1-4](#) through [Figure 1-10](#).

**Table 1-1. Flash/OTP Configuration Registers**

| Name <sup>(1)</sup> <sup>(2)</sup> | Address | Size (x16) | Description                           | Bit Description             |
|------------------------------------|---------|------------|---------------------------------------|-----------------------------|
| FOPT                               | 0x0A80  | 1          | Flash Option Register                 | <a href="#">Figure 1-4</a>  |
| Reserved                           | 0x0A81  | 1          | Reserved                              | -                           |
| FPWR                               | 0x0A82  | 1          | Flash Power Modes Register            | <a href="#">Figure 1-5</a>  |
| FSTATUS                            | 0x0A83  | 1          | Status Register                       | <a href="#">Figure 1-6</a>  |
| FSTDBYWAIT <sup>(3)</sup>          | 0x0A84  | 1          | Flash Sleep To Standby Wait Register  | <a href="#">Figure 1-7</a>  |
| FACTIVEWAIT <sup>(3)</sup>         | 0x0A85  | 1          | Flash Standby To Active Wait Register | <a href="#">Figure 1-8</a>  |
| FBANKWAIT                          | 0x0A86  | 1          | Flash Read Access Wait State Register | <a href="#">Figure 1-9</a>  |
| FOTPWAIT                           | 0x0A87  | 1          | OTP Read Access Wait State Register   | <a href="#">Figure 1-10</a> |

(1) These registers are EALLOW protected. See [Section 1.5.2](#) for information.

(2) These registers are protected by the Code Security Module (CSM). See [Section 1.2](#) for more information.

(3) These registers should be left in their default state.

#### Note

The Flash configuration registers should not be written to by code that is running from OTP or Flash memory or while an access to Flash or OTP may be in progress. All register accesses to the Flash registers should be made from code executing outside of Flash/OTP memory and an access should not be attempted until all activity on the Flash/OTP has completed. No hardware is included to protect against this.

To summarize, you can read the Flash registers from code executing in Flash/OTP; however, do not write to the registers.

CPU write access to the Flash configuration registers can be enabled only by executing the EALLOW instruction. Write access is disabled when the EDIS instruction is executed. This protects the registers from spurious accesses. Read access is always available. The registers can be accessed through the JTAG port without the need to execute EALLOW. See [Section 1.5.2](#) for information on EALLOW protection. These registers support both 16-bit and 32-bit accesses.



**Figure 1-4. Flash Options Register (FOPT)**

|          |  |   |        |
|----------|--|---|--------|
| 15       |  | 1 | 0      |
| Reserved |  |   | ENPIPE |
| R-0      |  |   | R/W-0  |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 1-2. Flash Options Register (FOPT) Field Descriptions**

| Bit  | Field    | Value | Description <sup>(1)</sup> <sup>(2)</sup> <sup>(3)</sup>                                                                                                                                                                                                                                                                                                                                                                |
|------|----------|-------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-1 | Reserved |       | Any writes to these bits must always have a value of 0.                                                                                                                                                                                                                                                                                                                                                                 |
| 0    | ENPIPE   |       | Enable Flash Pipeline Mode Bit. Flash pipeline mode is active when this bit is set. The pipeline mode improves performance of instruction fetches by pre-fetching instructions. See <a href="#">Section 1.1.3.2</a> for more information.<br><br>When pipeline mode is enabled, the Flash wait states (paged and random) must be greater than zero.<br><br>On Flash devices, ENPIPE affects fetches from Flash and OTP. |
|      |          | 0     | Flash Pipeline mode is not active. (default)                                                                                                                                                                                                                                                                                                                                                                            |
|      |          | 1     | Flash Pipeline mode is active.                                                                                                                                                                                                                                                                                                                                                                                          |

(1) This register is EALLOW protected. See [Section 1.5.2](#) for more information.

(2) This register is protected by the Code Security Module (CSM). See [Section 1.2](#) for more information.

(3) When writing to this register, follow the procedure described in [Section 1.1.3.4](#).

**Figure 1-5. Flash Power Register (FPWR)**

|          |  |   |   |       |
|----------|--|---|---|-------|
| 15       |  | 2 | 1 | 0     |
| Reserved |  |   |   | PWR   |
| R-0      |  |   |   | R/W-0 |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 1-3. Flash Power Register (FPWR) Field Descriptions**

| Bit  | Field    | Value | Description <sup>(1)</sup> <sup>(2)</sup>                                                                                                                                                             |
|------|----------|-------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-2 | Reserved |       | Any writes to these bits must always have a value of 0.                                                                                                                                               |
| 1-0  | PWR      |       | Flash Power Mode Bits. Writing to these bits changes the current power mode of the Flash bank and pump. See <a href="#">Section 1.1.3</a> for more information on changing the Flash bank power mode. |
|      |          | 00    | Pump and bank sleep (lowest power)                                                                                                                                                                    |
|      |          | 01    | Pump and bank standby                                                                                                                                                                                 |
|      |          | 10    | Reserved (no effect)                                                                                                                                                                                  |
|      |          | 11    | Pump and bank active (highest power)                                                                                                                                                                  |

(1) This register is EALLOW protected. See [Section 1.5.2](#) for more information.

(2) This register is protected by the Code Security Module (CSM). See [Section 1.2](#) for more information.

**Figure 1-6. Flash Status Register (FSTATUS)**

|          |  |  |   |             |   |            |   |      |         |   |
|----------|--|--|---|-------------|---|------------|---|------|---------|---|
| 15       |  |  |   |             |   |            |   | 9    |         | 8 |
| Reserved |  |  |   |             |   |            |   |      | 3VSTAT  |   |
| R-0      |  |  |   |             |   |            |   |      | R/W1C-0 |   |
| 7        |  |  | 4 |             | 3 |            | 2 |      | 1       | 0 |
| Reserved |  |  |   | ACTIVEWAITS |   | STDBYWAITS |   | PWRS |         |   |
| R-0      |  |  |   | R-0         |   | R-0        |   | R-0  |         |   |

LEGEND: R/W = Read/Write; R = Read only; W1C = Write 1 to clear; -n = value after reset

**Table 1-4. Flash Status Register (FSTATUS) Field Descriptions**

| Bit  | Field       | Value                | Description <sup>(1)</sup> <sup>(2)</sup>                                                                                                                                                                                                                                                                                                                                                         |
|------|-------------|----------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-9 | Reserved    |                      | Any writes to these bits must always have a value of 0.                                                                                                                                                                                                                                                                                                                                           |
| 8    | 3VSTAT      | 0<br>1               | Flash Voltage ( $V_{DD3VFL}$ ) Status Latch Bit. When set, this bit indicates that the 3VSTAT signal from the pump module went to a high level. This signal indicates that the Flash 3.3-V supply went out of the allowable range.<br>Writes of 0 are ignored.<br>When this bit reads 1, it indicates that the Flash 3.3-V supply went out of the allowable range. Clear this bit by writing a 1. |
| 7-4  | Reserved    |                      | Any writes to these bits must always have a value of 0.                                                                                                                                                                                                                                                                                                                                           |
| 3    | ACTIVEWAITS | 0<br>1               | Bank and Pump Standby To Active Wait Counter Status Bit. This bit indicates whether the respective wait counter is timing out an access.<br>The counter is not counting.<br>The counter is counting.                                                                                                                                                                                              |
| 2    | STDBYWAITS  | 0<br>1               | Bank and Pump Sleep To Standby Wait Counter Status Bit. This bit indicates whether the respective wait counter is timing out an access.<br>The counter is not counting.<br>The counter is counting.                                                                                                                                                                                               |
| 1-0  | PWRS        | 00<br>01<br>10<br>11 | Power Modes Status Bits. These bits indicate which power mode the Flash/OTP is currently in. The PWRS bits are set to the new power mode only after the appropriate timing delays have expired.<br>Pump and bank in sleep mode (lowest power)<br>Pump and bank in standby mode<br>Reserved<br>Pump and bank active and in read mode (highest power)                                               |

(1) This register is EALLOW protected. See [Section 1.5.2](#) for more information.

(2) This register is protected by the Code Security Module (CSM). See [Section 1.2](#) for more information.

**Figure 1-7. Flash Standby Wait Register (FSTDBYWAIT)**

|          |   |           |   |
|----------|---|-----------|---|
| 15       | 9 | 8         | 0 |
| Reserved |   | STDBYWAIT |   |
| R-0      |   | R/W-0x1FF |   |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 1-5. Flash Standby Wait Register (FSTDBYWAIT) Field Descriptions**

| Bit  | Field     | Value    | Description <sup>(1)</sup> <sup>(2)</sup>                                                                                                 |
|------|-----------|----------|-------------------------------------------------------------------------------------------------------------------------------------------|
| 15-9 | Reserved  |          | Any writes to these bits must always have a value of 0.                                                                                   |
| 8-0  | STDBYWAIT | 11111111 | <b>This register should be left in its default state.</b><br>Bank and Pump Sleep To Standby Wait Count.<br>511 SYSCLKOUT cycles (default) |

(1) This register is EALLOW protected. See [Section 1.5.2](#) for more information.

(2) This register is protected by the Code Security Module (CSM). See [Section 1.2](#) for more information.

**Figure 1-8. Flash Standby to Active Wait Counter Register (FACTIVEWAIT)**

|          |   |            |   |
|----------|---|------------|---|
| 7        | 9 | 8          | 0 |
| Reserved |   | ACTIVEWAIT |   |
| R-0      |   | R/W-0x1FF  |   |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 1-6. Flash Standby to Active Wait Counter Register (FACTIVEWAIT) Field Descriptions**

| Bits | Field      | Value    | Description <sup>(1)</sup> <sup>(2)</sup>                                                                                                  |
|------|------------|----------|--------------------------------------------------------------------------------------------------------------------------------------------|
| 15-9 | Reserved   |          | Any writes to these bits must always have a value of 0.                                                                                    |
| 8-0  | ACTIVEWAIT | 11111111 | <b>This register should be left in its default state.</b><br>Bank and Pump Standby To Active Wait Count:<br>511 SYSCLKOUT cycles (default) |

(1) This register is EALLOW protected. See [Section 1.5.2](#) for more information.

(2) This register is protected by the Code Security Module (CSM). See [Section 1.2](#) for more information.

**Figure 1-9. Flash Wait-State Register (FBANKWAIT)**

|          |    |          |   |          |   |          |   |
|----------|----|----------|---|----------|---|----------|---|
| 15       | 12 | 11       | 8 | 7        | 4 | 3        | 0 |
| Reserved |    | PAGEWAIT |   | Reserved |   | RANDWAIT |   |
| R-0      |    | R/W-0xF  |   | R-0      |   | R/W-0xF  |   |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 1-7. Flash Wait-State Register (FBANKWAIT) Field Descriptions**

| Bits  | Field    | Value                                       | Description <sup>(1)</sup> <sup>(2)</sup> <sup>(3)</sup>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|-------|----------|---------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-12 | Reserved |                                             | Any writes to these bits must always have a value of 0.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| 11-8  | PAGEWAIT | 0000<br>0001<br>0010<br>0011<br>...<br>1111 | Flash Paged Read Wait States. These register bits specify the number of wait states for a paged read operation in CPU clock cycles (0..15 SYSCLKOUT cycles) to the Flash bank. See <a href="#">Section 1.1.3.1</a> for more information.<br><br>See the device-specific data sheet for the minimum time required for a PAGED Flash access.<br><br>You must set RANDWAIT to a value greater than or equal to the PAGEWAIT setting. No hardware is provided to detect a PAGEWAIT value that is greater than RANDWAIT.<br><br>Zero wait-state per paged Flash access or one SYSCLKOUT cycle per access<br>One wait state per paged Flash access or a total of two SYSCLKOUT cycles per access<br>Two wait states per paged Flash access or a total of three SYSCLKOUT cycles per access<br>Three wait states per paged Flash access or a total of four SYSCLKOUT cycles per access<br>...<br>15 wait states per paged Flash access or a total of 16 SYSCLKOUT cycles per access. (default)                                                                                                 |
| 7-4   | Reserved |                                             | Any writes to these bits must always have a value of 0.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| 3-0   | RANDWAIT | 0000<br>0001<br>0010<br>0011<br>...<br>1111 | Flash Random Read Wait States. These register bits specify the number of wait states for a random read operation in CPU clock cycles (1..15 SYSCLKOUT cycles) to the Flash bank. See <a href="#">Section 1.1.3.1</a> for more information.<br><br>See the device-specific data sheet for the minimum time required for a RANDOM Flash access.<br><br>RANDWAIT must be set greater than 0. That is, at least 1 random wait state must be used. In addition, you must set RANDWAIT to a value greater than or equal to the PAGEWAIT setting. The device will not detect and correct a PAGEWAIT value that is greater than RANDWAIT.<br><br>Illegal value. RANDWAIT must be set greater than 0.<br>One wait state per random Flash access or a total of two SYSCLKOUT cycles per access.<br>Two wait states per random Flash access or a total of three SYSCLKOUT cycles per access.<br>Three wait states per random Flash access or a total of four SYSCLKOUT cycles per access.<br>...<br>15 wait states per random Flash access or a total of 16 SYSCLKOUT cycles per access. (default) |

(1) This register is EALLOW protected. See [Section 1.5.2](#) for more information.

(2) This register is protected by the Code Security Module (CSM). See [Section 1.2](#) for more information.

(3) When writing to this register, follow the procedure described in [Section 1.1.3.4](#).

**Figure 1-10. OTP Wait-State Register (FOTPWAIT)**

|          |  |   |   |          |
|----------|--|---|---|----------|
| 15       |  | 5 | 4 | 0        |
| Reserved |  |   |   | OTPWAIT  |
| R-0      |  |   |   | R/W-0x1F |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 1-8. OTP Wait-State Register (FOTPWAIT) Field Descriptions**

| Bits | Field    | Value | Description <sup>(1)</sup> <sup>(2)</sup> <sup>(3)</sup>                                                                                                                                                                                                                                                                                                                                                                                     |
|------|----------|-------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-5 | Reserved |       | Any writes to these bits must always have a value of 0.                                                                                                                                                                                                                                                                                                                                                                                      |
| 4-0  | OTPWAIT  |       | OTP Read Wait States. These register bits specify the number of wait states for a read operation in CPU clock cycles (1..31 SYSCLKOUT cycles) to the OTP. See CPU Read Or Fetch Access From Flash/OTP section for details. There is no PAGE mode in the OTP.<br><br>OTPWAIT must be set greater than 0. That is, a minimum of 1 wait state must be used. See the device-specific data sheet for the minimum time required for an OTP access. |
|      |          | 00000 | Illegal value. OTPWAIT must be set to 1 or greater.                                                                                                                                                                                                                                                                                                                                                                                          |
|      |          | 00001 | One wait state will be used each OTP access for a total of two SYSCLKOUT cycles per access.                                                                                                                                                                                                                                                                                                                                                  |
|      |          | 00010 | Two wait states will be used for each OTP access for a total of three SYSCLKOUT cycles per access.                                                                                                                                                                                                                                                                                                                                           |
|      |          | 00011 | Three wait states will be used for each OTP access for a total of four SYSCLKOUT cycles per access.                                                                                                                                                                                                                                                                                                                                          |
|      |          | ...   | ...                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|      |          | 11111 | 31 wait states will be used for an OTP access for a total of 32 SYSCLKOUT cycles per access.                                                                                                                                                                                                                                                                                                                                                 |

- (1) This register is EALLOW protected. See [Section 1.5.2](#) for more information.  
(2) This register is protected by the Code Security Module (CSM). See [Section 1.2](#) for more information.  
(3) When writing to this register, follow the procedure described in [Section 1.1.3.4](#).

## 1.2 Code Security Module (CSM)

The code security module (CSM) is a security feature incorporated in the C28x devices. It prevents access/visibility to on-chip memory to unauthorized persons — that is, it prevents duplication/reverse engineering of proprietary code.

The word "secure" means access to on-chip secure memory is protected. The word "unsecure" means access to on-chip secure memory is not protected — that is, the contents of the memory could be read by any means (through a debugging tool such as the Code Composer Studio™ integrated development environment (IDE), for example).

### 1.2.1 Functional Description

The security module restricts the CPU access to certain on-chip memory without interrupting or stalling CPU execution. When a read occurs to a protected memory location, the read returns a zero value and CPU execution continues with the next instruction. This, in effect, blocks read and write access to various memories through the JTAG port or external peripherals. Security is defined with respect to the access of on-chip memory and prevents unauthorized copying of proprietary code or data.

The device is secure when CPU access to the on-chip secure memory locations is restricted. When secure, two levels of protection are possible, depending on where the program counter is currently pointing. If code is currently running from inside secure memory, only an access through JTAG is blocked (that is, through the debug probe). This allows secure code to access secure data. Conversely, if code is running from nonsecure memory, all accesses to secure memories are blocked. User code can dynamically jump in and out of secure memory, thereby allowing secure function calls from nonsecure memory. Similarly, interrupt service routines can be placed in secure memory, even if the main program loop is run from nonsecure memory.

Security is protected by a password of 128-bits of data (eight 16-bit words) that is used to secure or unsecure the device. This password is stored at the end of flash in 8 words referred to as the password locations.

The device is unsecured by executing the password match flow (PMF), described in [Section 1.2.3.2](#). The levels of security are shown in [Table 1-9](#).

**Table 1-9. Security Levels**

| PMF Executed With Correct Password? | Operating Mode | Program Fetch Location | Security Description                                                                                                       |
|-------------------------------------|----------------|------------------------|----------------------------------------------------------------------------------------------------------------------------|
| No                                  | Secure         | Outside secure memory  | Only instruction fetches by the CPU are allowed to secure memory. In other words, code can still be executed, but not read |
| No                                  | Secure         | Inside secure memory   | CPU has full access.<br>JTAG port cannot read the secured memory contents.                                                 |
| Yes                                 | Not Secure     | Anywhere               | Full access for CPU and JTAG port to secure memory                                                                         |

The password is stored in code security password locations (PWL) in flash memory (0x3F 7FF8 - 0x3F 7FFF). These locations store the password predetermined by the system designer.

If the password locations have all 128 bits as ones, the device is labeled unsecure. Since new flash devices have erased flash (all ones), only a read of the password locations is required to bring the device into unsecure mode. If the password locations have all 128 bits as zeros, the device is secure, regardless of the contents of the KEY registers. Do not use all zeros as a password or reset the device during an erase of the flash. Resetting the device during an erase routine can result in either an all zero or unknown password. If a device is reset when the password locations are all zeros, the device cannot be unlocked by the password match flow described in [Section 1.2.3.2](#). Using a password of all zeros will seriously limit your ability to debug secure code or reprogram the flash.



---

**Note**

If a device is reset while the password locations are all zero or an unknown value, the device will be permanently locked unless a method to run the flash erase routine from secure SARAM is embedded into the flash or OTP. Care must be taken when implementing this procedure to avoid introducing a security hole.

---

User accessible registers (eight 16-bit words) that are used to unsecure the device are referred to as KEY registers. These registers are mapped in the memory space at addresses 0x00 0AE0 - 0x00 0AE7 and are EALLOW protected.

In addition to the CSM, the emulation code security logic (ECSL) has been implemented to prevent unauthorized users from stepping through secure code. Any code or data access to flash, user OTP, L0 memory while the debug probe is connected will trip the ECSL and break the debug probe connection. To allow emulation of secure code, while maintaining the CSM protection against secure memory reads, you must write the correct value into the lower 64 bits of the KEY register, which matches the value stored in the lower 64 bits of the password locations within the flash. Note that dummy reads of all 128 bits of the password in the flash must still be performed. If the lower 64 bits of the password locations are all ones (unprogrammed), then the KEY value does not need to match.

When initially debugging a device with the password locations in flash programmed (that is, secured), the debug probe takes some time to take control of the CPU. During this time, the CPU will start running and may execute an instruction that performs an access to a protected ECSL area. If this happens, the ECSL will trip and cause the debug probe connection to be broken. Two solutions to this problem exist:

1. The first is to use the Wait-In-Reset emulation mode, which will hold the device in reset until the debug probe takes control. The debug probe must support this mode for this option.
2. The second option is to use the “Branch to check boot mode” boot option. This will sit in a loop and continuously poll the boot mode select pins. You can select this boot mode and then exit this mode once the debug probe is connected by re-mapping the PC to another address or by changing the boot mode selection pin to the desired boot mode.

---

**Note****Reserved Flash Locations When Using Code Security**

For code security operation, **all addresses between 0x3F 7F80 and 0x3F 7FF5 cannot be used as program code or data, but must be programmed to 0x0000** when the Code Security Password is programmed. If security is not a concern, then these addresses may be used for code or data. The 128-bit password (at 0x3F 7FF8 - 0x3F 7FFF) must not be programmed to zeros. Doing so would permanently lock the device.

Addresses 0x3F 7FF0 through 0x3F 7FF5 are reserved for data variables and should not contain program code.

---

---

**Note****Code Security Module Disclaimer**

The Code Security Module ( CSM ) included on this device was designed to password protect the data stored in the associated memory and is warranted by Texas Instruments (TI), in accordance with its standard terms and conditions, to conform to TI's published specifications for the warranty period applicable for this device.

TI DOES NOT, HOWEVER, WARRANT OR REPRESENT THAT THE CSM CANNOT BE COMPROMISED OR BREACHED OR THAT THE DATA STORED IN THE ASSOCIATED MEMORY CANNOT BE ACCESSED THROUGH OTHER MEANS. MOREOVER, EXCEPT AS SET FORTH ABOVE, TI MAKES NO WARRANTIES OR REPRESENTATIONS CONCERNING THE CSM OR OPERATION OF THIS DEVICE, INCLUDING ANY IMPLIED WARRANTIES OF MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE.

IN NO EVENT SHALL TI BE LIABLE FOR ANY CONSEQUENTIAL, SPECIAL, INDIRECT, INCIDENTAL, OR PUNITIVE DAMAGES, HOWEVER CAUSED, ARISING IN ANY WAY OUT OF YOUR USE OF THE CSM OR THIS DEVICE, WHETHER OR NOT TI HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES. EXCLUDED DAMAGES INCLUDE, BUT ARE NOT LIMITED TO LOSS OF DATA, LOSS OF GOODWILL, LOSS OF USE OR INTERRUPTION OF BUSINESS OR OTHER ECONOMIC LOSS.

---

### 1.2.2 CSM Impact on Other On-Chip Resources

The CSM affects access to the on-chip resources listed in [Table 1-10](#).

**Table 1-10. Resources Affected by the CSM**

| Address                                           | Block                                                   |
|---------------------------------------------------|---------------------------------------------------------|
| 0x00 0A80 - 0x00 0A87                             | Flash Configuration Registers                           |
| 0x00 8000 - 0x00 87FF                             | L0 SARAM (2K x 16)                                      |
| 0x00 8800 - 0x00 8BFF                             | L1 DPSARAM - CLA Data RAM 0 (1K x 16)                   |
| 0x00 8C00 - 0x00 8FFF                             | L2 DPSARAM - CLA Data RAM 1 (1K x 16)                   |
| 0x00 9000 - 0x00 9FFF                             | L3 DPSARAM - CLA Program RAM (4K x 16)                  |
| 0x3D 8000 - 0x3F 7FFF or<br>0x3E 8000 - 0x3F 7FFF | Flash (128K x 16)<br>Flash (64K x 16)                   |
| 0x3D 7800 - 0x3D 7BFF                             | User One-Time Programmable (OTP) (1K x 16)              |
| 0x3D 7C00 - 0x3D 7FFF                             | TI One-Time Programmable (OTP) <sup>(1)</sup> (1K x 16) |
| 0x00 A000 - 0x00 BFFF                             | L4 SARAM                                                |

(1) Not affected by ECSL

The Code Security Module has no impact whatsoever on the following on-chip resources:

- Single-access RAM (SARAM) blocks not designated as secure - These memory blocks can be freely accessed and code run from them, whether the device is in secure or unsecure mode.
- Boot ROM contents - Visibility to the boot ROM contents is not impacted by the CSM.
- On-chip peripheral registers - The peripheral registers can be initialized by code running from on-chip or off-chip memory, whether the device is in secure or unsecure mode.
- PIE Vector Table - Vector tables can be read and written regardless of whether the device is in secure or unsecure mode. [Table 1-10](#) and [Table 1-11](#) show which on-chip resources are affected (or are not affected) by the CSM.

**Table 1-11. Resources Not Affected by the CSM**

| Address               | Block                        |
|-----------------------|------------------------------|
| 0x00 0000 - 0x00 03FF | M0 SARAM (1K x 16)           |
| 0x00 0400 - 0x00 07FF | M1 SARAM (1K x 16)           |
| 0x00 0800 - 0x00 0CFF | Peripheral Frame 0 (2K x 16) |
| 0x00 0D00 - 0x00 0FFF | PIE Vector RAM (256 x 16)    |
| 0x00 6000 - 0x00 6FFF | Peripheral Frame 1 (4K x 16) |
| 0x00 7000 - 0x00 7FFF | Peripheral Frame 2 (4K x 16) |

To summarize, it is possible to load code onto the unprotected on-chip program SARAM using the JTAG connector without any impact from the Code Security Module. The code can be debugged and the peripheral registers initialized, independent of whether the device is in secure or unsecure mode.

### 1.2.3 Incorporating Code Security in User Applications

Code security is typically not employed in the development phase of a project; however, security may be desired once the application code is finalized. Before such a code is programmed in the Flash memory, a password should be chosen to secure the device. Once a password is in place, the device is secured (that is, programming a password at the appropriate locations and either performing a device reset or setting the FORCESEC bit (CSMSCR.15) is the action that secures the device). From that time on, access to debug the contents of secure memory by any means (by way of JTAG, code running off external/on-chip memory, and so on) requires the supply of a valid password. A password is not needed to run the code out of secure memory (such as in end-customer usage); however, access to secure memory contents for debug purpose requires a password.

If the code-security feature is used, any one of the following directives must be used when a function residing in secure memory calls another function which belongs to a different secure zone or to unsecure memory:

- Use unsecure memory as stack
- Switch stack to unsecure memory before calling the function
- Unlock security before calling the function

Note that the above directives apply for any address-based-parameters passed on to the called function, basically making sure that the called function can read/write to these address-based parameters.

### 1.2.3.1 Environments That Require Security Unlocking

Following are the typical situations under which unsecuring may be required:

- Code development using debuggers (such as Code Composer Studio™ IDE).  
This is the most common environment during the design phase of a product.
- Flash programming using TI's flash utilities such as Code Composer Studio™ Flash Programmer plug-in or Uniflash.

Flash programming is common during code development and testing. Once the user supplies the necessary password, the flash utilities disable the security logic before attempting to program the flash. The flash utilities can disable the code security logic in new devices without any authorization, since new devices come with an erased flash. However, reprogramming devices (that already contain a custom password) require the password to be supplied to the flash utilities in order to unlock the device to enable programming. In custom programming solutions that use the flash API supplied by TI unlocking the CSM can be avoided by executing the flash programming algorithms from secure memory.

- Custom environment defined by the application

In addition to the above, access to secure memory contents can be required in situations such as:

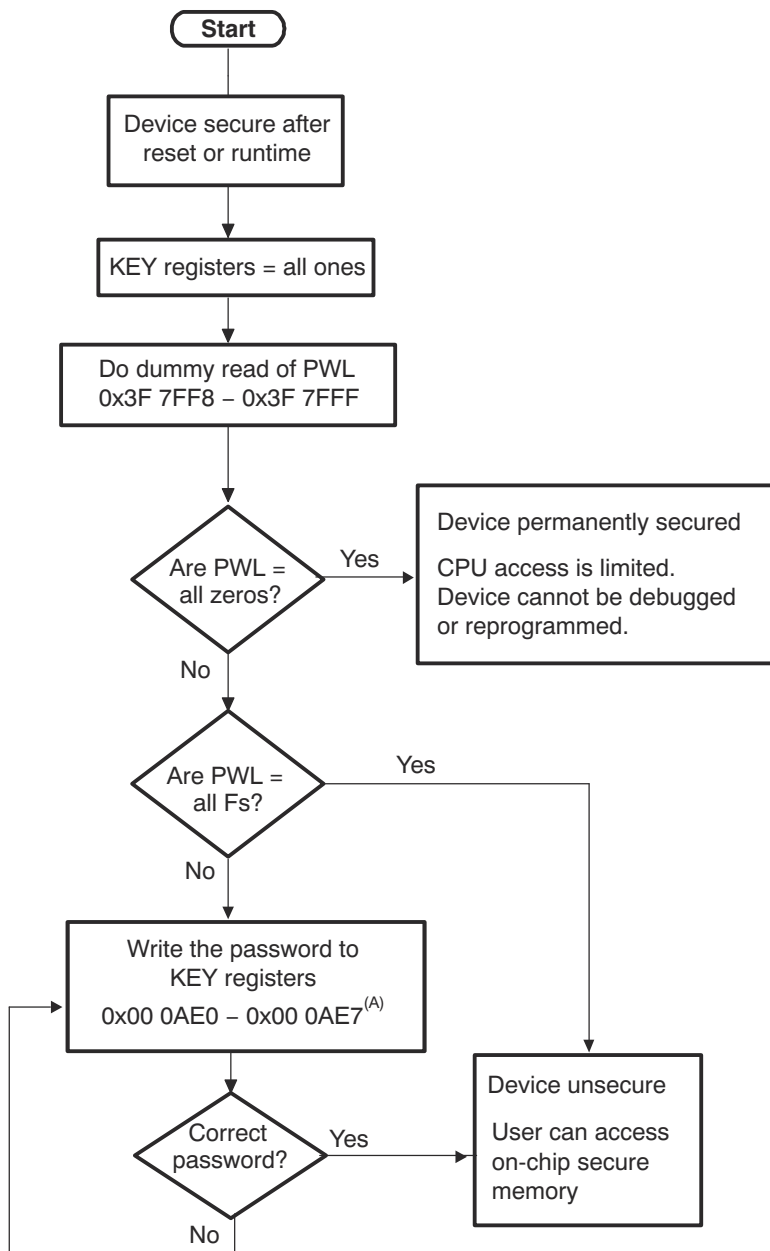
- Using the on-chip bootloader to load code or data into secure SARAM or to erase/program the flash.
- Executing code from on-chip unsecure memory and requiring access to secure memory for lookup table. This is not a suggested operating condition as supplying the password from external code could compromise code security.

The unsecuring sequence is identical in all the above situations. This sequence is referred to as the *password match flow (PMF)* for simplicity. [Figure 1-11](#) explains the sequence of operation that is required every time the user attempts to unsecure a device. A code example is listed for clarity.

### 1.2.3.2 Password Match Flow

Password match flow (PMF) is essentially a sequence of eight dummy reads from password locations (PWL) followed by eight writes to KEY registers.

Figure 1-11 shows how the PMF helps to initialize the security logic registers and disable security logic.



A. The KEY registers are EALLOW protected.

**Figure 1-11. Password Match Flow (PMF)**

---

**Note**

**NOTE:** Any read of the CSM password would yield 0x0000 until the device is unlocked. These reads are labeled "dummy read" or a "fake read." The application reads the password locations, but will always get 0's no matter what the actual value is. What is important is the actual value of the password. If the actual value is all 0xFFFF, then doing this "dummy read" will unlock the device. If the actual value is all 0x0000, then no matter what the application code does, one will never be able to unlock the device. If the actual value is something other than all 0xFFFF or 0x0000, then when the dummy read is performed, the actual value must match the password the user provided.

---

**1.2.3.3 Unsecuring Considerations for Devices With/Without Code Security**

Case 1 and Case 2 provide unsecuring considerations for devices with and without code security.

**Case 1: Device With Code Security**

A device with code security should have a predetermined password stored in the password locations (0x3F 7FF8 - 0x3F 7FFF in memory). In addition, locations 0x3F 7F80 - 0x3F 7FF5 should be programmed with all 0x0000 and not used for program and/or data storage. The following are steps to unsecure this device:

1. Perform a dummy read of the password locations. The CSM blocks the OTP reads to password location. Hence the dummy reads to password location can be done only from RAM (secure/unsecure) or Flash.
2. Write the password into the KEY registers (locations 0x00 0AE0 - 0x00 0AE7 in memory).
3. If the password is correct, the device becomes unsecure; otherwise, it stays secure.

**Case 2: Device Without Code Security**

A device without code security should have 0x FFFF FFFF FFFF FFFF FFFF FFFF FFFF FFFF (128 bits of all ones) stored in the password locations. The following are steps to use this device:

1. At reset, the CSM will lock memory regions protected by the CSM.
2. Perform a dummy read of the password locations. The CSM blocks the OTP reads to password location. Hence the dummy reads to password location can be done only from RAM (secure/unsecure) or Flash.
3. Since the password is all ones, this alone will unlock all memory regions. Secure memory is fully accessible immediately after this operation is completed.

---

**Note**

Even if a device is not protected with a password (all password locations all ones), the CSM will lock at reset. Thus, a dummy read operation must still be performed on these devices prior to reading, writing, or programming secure memory if the code performing the access is executing from outside of the CSM protected memory region. The Boot ROM code does this dummy read for convenience.

---



### 1.2.3.3.1 C Code Example to Unsecure

```
volatile int *CSM = (volatile int *)0x000AE0; //CSM register file
volatile int *PWL = (volatile int *)0x003F7FF8; //Password location
volatile int tmp;
int I;
// Read the 128-bits of the password locations (PWL)
// in flash at address 0x3F 7FF8 - 0x3F 7FFF
// If the device is secure, then the values read will
// not actually be loaded into the temp variable, so
// this is called a dummy read.
for (I=0; I<8; I++) tmp = *PWL++;
// If the password locations (PWL) are all = ones (0xFFFF),
// then the device will now be unsecure. If the password
// is not all ones (0xFFFF), then the code below is required
// to unsecure the CSM.
// Write the 128-bit password to the KEY registers
// If this password matches that stored in the
// PWL then the CSM will become unsecure. If it does not
// match, then the device will remain secure.
// An example password of:
// 0x11112222333344445555666677778888 is used.
asm(" EALLOW"); // Key registers are EALLOW protected
*CSM++ = 0x1111; // Register KEY0 at 0xAE0
*CSM++ = 0x2222; // Register KEY1 at 0xAE1
*CSM++ = 0x3333; // Register KEY2 at 0xAE2
*CSM++ = 0x4444; // Register KEY3 at 0xAE3
*CSM++ = 0x5555; // Register KEY4 at 0xAE4
*CSM++ = 0x6666; // Register KEY5 at 0xAE5
*CSM++ = 0x7777; // Register KEY6 at 0xAE6
*CSM++ = 0x8888; // Register KEY7 at 0xAE7
asm(" EDIS");
```

### 1.2.3.3.2 C Code Example to Resecure

```
volatile int *CSMSCR = 0x00AEF; //CSMSCR register
//Set FORCESEC bit
asm(" EALLOW"); //CSMSCR register is EALLOW protected.
*CSMSCR = 0x8000;
asm("EDIS");
```

## 1.2.4 Do's and Don'ts to Protect Security Logic

### 1.2.4.1 Do's

- To keep the debug and code development phase simple, use the device in the unsecure mode; that is, use all 128 bits as ones in the password locations (or use a password that is easy to remember). Use a password after the development phase when the code is frozen.
- Recheck the password stored in the password locations before programming the COFF file using flash utilities.
- The flow of code execution can freely toggle back and forth between secure memory and unsecure memory without compromising security. To access data variables located in secure memory when the device is secured, code execution must currently be running from secure memory.
- Program locations 0x3F 7F80 - 0x3F 7FF5 with 0x0000 when using the CSM.

### 1.2.4.2 Don'ts

- If code security is desired, do not embed the password in your application anywhere other than in the password locations or security can be compromised.
- Do not use 128 bits of all zeros as the password. This automatically secures the device, regardless of the contents of the KEY register. The device is not debuggable nor reprogrammable.
- Do not pull a reset during an erase operation on the flash array. This can leave either zeros or an unknown value in the password locations. If the password locations are all zero during a reset, the device will always be secure, regardless of the contents of the KEY register.
- Do not use locations 0x3F 7F80 - 0x3F 7FF5 to store program and/or data. These locations should be programmed to 0x0000 when using the CSM.

## 1.2.5 CSM Features - Summary

1. The flash is secured after a reset until the password match flow described in [Section 1.2.3.2](#) is executed.
2. The standard way of running code out of the flash is to program the flash with the code and power up the device. Since instruction fetches are always allowed from secure memory, regardless of the state of the CSM, the code functions correctly even without executing the password match flow.
3. Secure memory cannot be modified by code executing from unsecure memory while the device is secured.
4. Secure memory cannot be read from any code running from unsecure memory while the device is secured.
5. Secure memory cannot be read or written to by the debugger (Code Composer Studio™ IDE) at any time that the device is secured.
6. Complete access to secure memory from both the CPU code and the debugger is granted while the device is unsecured.

## 1.2.6 CSM Status and Control Register

**Table 1-12. Code Security Module (CSM) Registers**

| Memory Address                                                                       | Register Name         | Reset Values | Register Description                    |
|--------------------------------------------------------------------------------------|-----------------------|--------------|-----------------------------------------|
| <b>KEY Registers</b>                                                                 |                       |              |                                         |
| 0x00 - 0AE0                                                                          | KEY0 <sup>(1)</sup>   | 0xFFFF       | Low word of the 128-bit KEY register    |
| 0x00 - 0AE1                                                                          | KEY1 <sup>(1)</sup>   | 0xFFFF       | Second word of the 128-bit KEY register |
| 0x00 - 0AE2                                                                          | KEY2 <sup>(1)</sup>   | 0xFFFF       | Third word of the 128-bit KEY register  |
| 0x00 - 0AE3                                                                          | KEY3 <sup>(1)</sup>   | 0xFFFF       | Fourth word of the 128-bit key          |
| 0x00 - 0AE4                                                                          | KEY4 <sup>(1)</sup>   | 0xFFFF       | Fifth word of the 128-bit key           |
| 0x00 - 0AE5                                                                          | KEY5 <sup>(1)</sup>   | 0xFFFF       | Sixth word of the 128-bit key           |
| 0x00 - 0AE6                                                                          | KEY6 <sup>(1)</sup>   | 0xFFFF       | Seventh word of the 128-bit key         |
| 0x00 - 0AE7                                                                          | KEY7 <sup>(1)</sup>   | 0xFFFF       | High word of the 128-bit KEY register   |
| 0x00 - 0AEF                                                                          | CSMSCR <sup>(1)</sup> | 0x002F       | CSM status and control register         |
| <b>Password Locations (PWL) in Flash Memory - Reserved for the CSM password only</b> |                       |              |                                         |
| 0x3F - 7FF8                                                                          | PWL0                  | User defined | Low word of the 128-bit password        |
| 0x3F - 7FF9                                                                          | PWL1                  | User defined | Second word of the 128-bit password     |
| 0x3F - 7FFA                                                                          | PWL2                  | User defined | Third word of the 128-bit password      |
| 0x3F - 7FFB                                                                          | PWL3                  | User defined | Fourth word of the 128-bit password     |
| 0x3F - 7FFC                                                                          | PWL4                  | User defined | Fifth word of the 128-bit password      |
| 0x3F - 7FFD                                                                          | PWL5                  | User defined | Sixth word of the 128-bit password      |
| 0x3F - 7FFE                                                                          | PWL6                  | User defined | Seventh word of the 128-bit password    |
| 0x3F - 7FFF                                                                          | PWL7                  | User defined | High word of the 128-bit password       |

(1) These registers are EALLOW protected. Refer to [Section 1.5.2](#) for more information.

### 1.2.6.1 CSM Status and Control Register (CSMSCR)

**Figure 1-12. CSM Status and Control Register (CSMSCR)**

|          |          |   |        |
|----------|----------|---|--------|
| 15       | 14       | 1 | 0      |
| FORCESEC | Reserved |   | SECURE |
| W-0      | R-0x002E |   | R-1    |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 1-13. CSM Status and Control Register (CSMSCR) Field Descriptions**

| Bits | Field    | Value | Description <sup>(1)</sup>                                                                                                                                           |
|------|----------|-------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15   | FORCESEC | 0     | Writing a 1 clears the KEY registers and secures the device.<br>A read always returns a zero.                                                                        |
|      |          | 1     | Clears the KEY registers and secures the device. The password match flow described in <a href="#">Section 1.2.3.2</a> must be followed to unsecure the device again. |
| 14-1 | Reserved |       | Reserved                                                                                                                                                             |
| 0    | SECURE   | 0     | Read-only bit that reflects the security state of the device.<br>Device is unsecure (CSM unlocked).                                                                  |
|      |          | 1     | Device is secure (CSM locked).                                                                                                                                       |

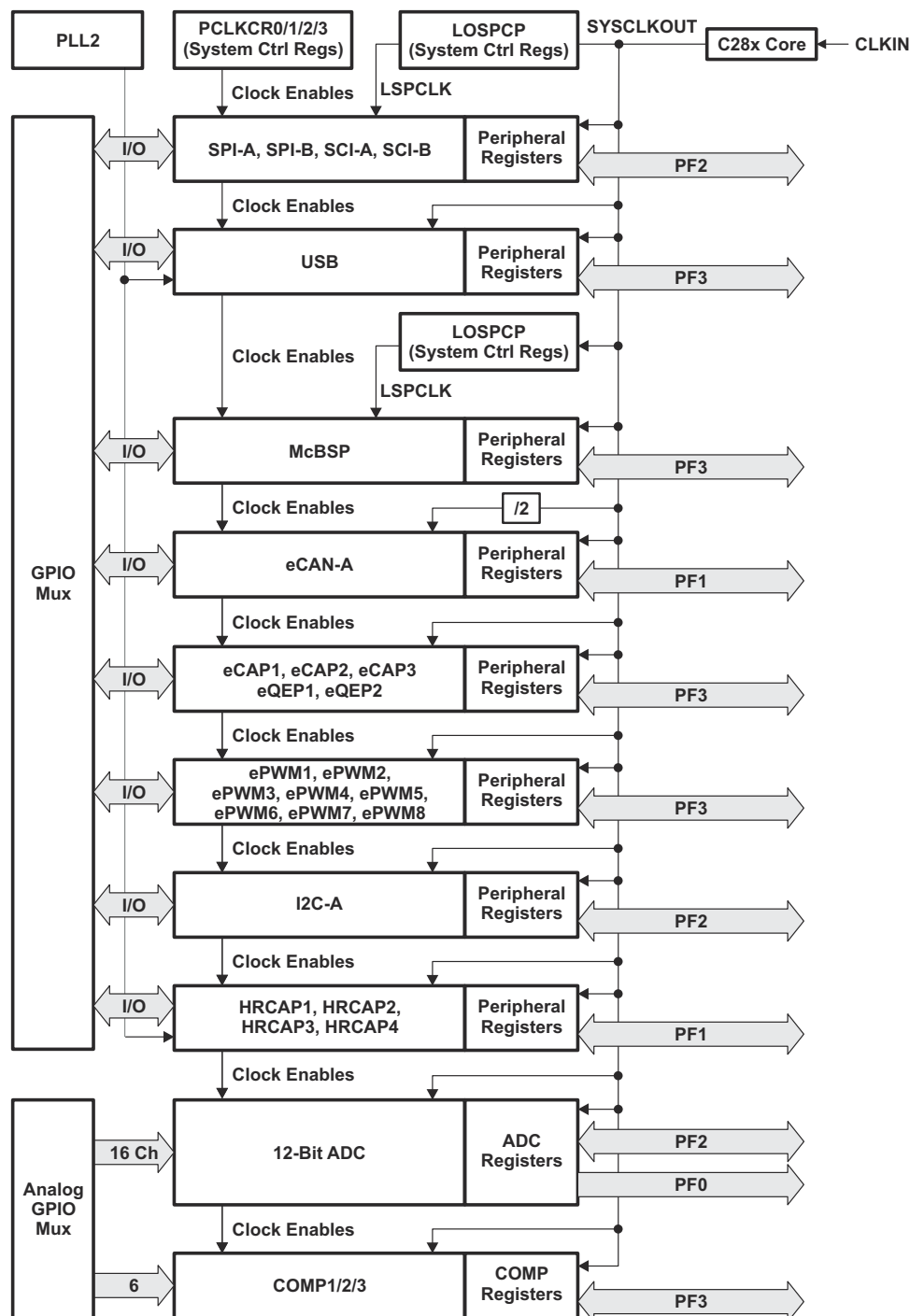
(1) This register is EALLOW protected. Refer to [Section 1.5.2](#) for more information.

## 1.3 Clocking

This section describes the oscillator, PLL and clocking mechanisms, the watchdog function, and the low-power modes.

### 1.3.1 Clocking and System Control

Figure 1-13 shows the various clock and reset domains.



**Figure 1-13. Clock and Reset Domains**

The PLL, clocking, watchdog, and low-power modes, are controlled by the registers listed in [Table 1-14](#).

**Table 1-14. PLL, Clocking, Watchdog, and Low-Power Mode Registers**

| Name         | Address   | Size (x16) | Description                                   | Section                           |
|--------------|-----------|------------|-----------------------------------------------|-----------------------------------|
| BORCFG       | 0x00 0985 | 1          | BOR Configuration Register                    |                                   |
| XCLK         | 0x00 7010 | 1          | XCLKOUT Control                               | <a href="#">Section 1.3.2.2</a>   |
| PLLSTS       | 0x00 7011 | 1          | PLL Status Register                           |                                   |
| CLKCTL       | 0x00 7012 | 1          | Clock Control Register                        | <a href="#">Section 1.3.2.3</a>   |
| PLLLOCKPRD   | 0x00 7013 | 1          | PLL Lock Period                               |                                   |
| INTOSC1TRIM  | 0x00 7014 | 1          | Internal Oscillator 1 Trim Register           | <a href="#">Section 1.3.2.1.1</a> |
| INTOSC2TRIM  | 0x00 7016 | 1          | Internal Oscillator 2 Trim Register           | <a href="#">Section 1.3.2.1.1</a> |
| PCLKCR2      | 0x00 7019 | 1          | Peripheral Clock Control Register 2           | <a href="#">Section 1.3.1.1</a>   |
| LOSPCP       | 0x00 701B | 1          | Low-Speed Peripheral Clock Prescaler Register | <a href="#">Section 1.3.1.2</a>   |
| PCLKCR0      | 0x00 701C | 1          | Peripheral Clock Control Register 0           | <a href="#">Section 1.3.1.1</a>   |
| PCLKCR1      | 0x00 701D | 1          | Peripheral Clock Control Register 1           | <a href="#">Section 1.3.1.1</a>   |
| LPMCR0       | 0x00 701E | 1          | Low Power Mode Control Register 0             |                                   |
| PCLKCR3      | 0x00 7020 | 1          | Peripheral Clock Control Register 3           | <a href="#">Section 1.3.1.1</a>   |
| PLLCR        | 0x00 7021 | 1          | PLL Control Register                          |                                   |
| SCSR         | 0x00 7022 | 1          | System Control and Status Register            |                                   |
| WDCNTR       | 0x00 7023 | 1          | Watchdog Counter Register                     |                                   |
| WDKEY        | 0x00 7025 | 1          | Watchdog Reset Key Register                   |                                   |
| WDCR         | 0x00 7029 | 1          | Watchdog Control Register                     |                                   |
| PLL2CTL      | 0x00 7030 | 1          | PLL2 Configuration Register                   |                                   |
| PLL2MULT     | 0x00 7032 | 1          | PLL2 Multiplier Register                      |                                   |
| PLL2STS      | 0x00 7034 | 1          | PLL2 Lock Status Register                     |                                   |
| SYSCCLK2CNTR | 0x00 7036 | 1          | SYSCCLK2 Clock Counter Register               |                                   |
| EPWMCFG      | 0x00 703A | 1          | ePWM DMA/CLA Configuration Register           |                                   |

### 1.3.1.1 Enabling/Disabling Clocks to the Peripheral Modules (PCLKCR0/1/2/3)

The PCLKCR0/1/2/3 registers enable/disable clocks to the various peripheral modules. There is a 2-SYSCCLKOUT cycle delay from when a write to the PCLKCR0/1/2/3 registers occurs to when the action is valid. This delay must be taken into account before attempting to access the peripheral configuration registers. Due to the peripheral-GPIO multiplexing at the pin level, all peripherals cannot be used at the same time. While it is possible to turn on the clocks to all the peripherals at the same time, such a configuration may not be useful. If this is done, the current drawn will be more than required. To avoid this, only enable the clocks required by the application.

**Figure 1-14. Peripheral Clock Control 0 Register (PCLKCR0)**

|          |             |          |              |            |            |            |             |
|----------|-------------|----------|--------------|------------|------------|------------|-------------|
| 15       | 14          | 13       | 12           | 11         | 10         | 9          | 8           |
| Reserved | ECANA ENCLK | Reserved | MCBSPA ENCLK | SCIB ENCLK | SCIA ENCLK | SPIB ENCLK | SPIA ENCLK  |
| R-0      | R/W-0       | R-0      | R/W-0        | R/W-0      | R/W-0      | R/W-0      | R/W-0       |
| 7        | 5           | 4        | 3            | 2          | 1          | 0          |             |
| Reserved |             |          | I2CA ENCLK   | ADC ENCLK  | TBCLKSYNC  | Reserved   | HRPWM ENCLK |
| R-0      |             |          | R/W-0        | R/W-0      | R/W-0      | R-0        | R/W-0       |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 1-15. Peripheral Clock Control 0 Register (PCLKCR0) Field Descriptions**

| Bit | Field       | Value  | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|-----|-------------|--------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15  | Reserved    |        | Any writes to these bits must always have a value of 0.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| 14  | ECANAENCLK  | 0<br>1 | ECAN-A clock enable<br>The eCAN-A module is not clocked. (default) <sup>(1)</sup><br>The eCAN-A module is clocked (SYSCLKOUT/2).                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| 13  | Reserved    |        | Any writes to these bits must always have a value of 0.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| 12  | MCBSPAENCLK | 0<br>1 | MCBSP clock enable<br>The McBSP module is not clocked.<br>The McBSP module is clocked.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| 11  | SCIBENCLK   | 0<br>1 | SCI-B clock enable<br>The SCI-B module is not clocked.<br>The SCI-B module is clocked.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| 10  | SCIAENCLK   | 0<br>1 | SCI-A clock enable<br>The SCI-A module is not clocked. (default) <sup>(1)</sup><br>The SCI-A module is clocked by the low-speed clock (LSPCLK).                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| 9   | SPIBENCLK   | 0<br>1 | SPI-B clock enable<br>The SPI-B module is not clocked. (default) <sup>(1)</sup><br>The SPI-B module is clocked by the low-speed clock (LSPCLK).                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| 8   | SPIAENCLK   | 0<br>1 | SPI-A clock enable<br>The SPI-A module is not clocked. (default) <sup>(1)</sup><br>The SPI-A module is clocked by the low-speed clock (LSPCLK).                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| 7-5 | Reserved    |        | Any writes to these bits must always have a value of 0.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| 4   | I2CAENCLK   | 0<br>1 | I <sup>2</sup> C clock enable<br>The I <sup>2</sup> C module is not clocked. (default) <sup>(1)</sup><br>The I <sup>2</sup> C module is clocked.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| 3   | ADCENCLK    | 0<br>1 | ADC clock enable<br>The ADC is not clocked. (default) <sup>(1)</sup><br>The ADC module is clocked                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| 2   | TBCLKSYNC   | 0<br>1 | ePWM Module Time Base Clock (TBCLK) Sync: Allows the user to globally synchronize all enabled ePWM modules to the time base clock (TBCLK):<br>0 The TBCLK (Time Base Clock) within each enabled ePWM module is stopped. (default). If, however, the ePWM clock enable bit is set in the PCLKCR1 register, then the ePWM module will still be clocked by SYSCLKOUT even if TBCLKSYNC is 0.<br>1 All enabled ePWM module clocks are started with the first rising edge of TBCLK aligned. For perfectly synchronized TBCLKs, the prescaler bits in the TBCTL register of each ePWM module must be set identically. The proper procedure for enabling ePWM clocks is as follows: <ul style="list-style-type: none"> <li>• Enable ePWM module clocks in the PCLKCR1 register.</li> <li>• Set TBCLKSYNC to 0.</li> <li>• Configure prescaler values and ePWM modes.</li> <li>• Set TBCLKSYNC to 1.</li> </ul> |
| 1   | Reserved    |        | Any writes to these bits must always have a value of 0.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| 0   | HRPWMENCLK  | 0<br>1 | HRPWM clock enable<br>HRPWM is not enabled.<br>HRPWM is enabled.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |

(1) If a peripheral block is not used, the clock to that peripheral can be turned off to minimize power consumption.



**Figure 1-15. Peripheral Clock Control 1 Register (PCLKCR1)**

| 15          |  | 14          |  | 13          |  | 11          |  | 10          |  | 9           |  | 8           |  |             |  |
|-------------|--|-------------|--|-------------|--|-------------|--|-------------|--|-------------|--|-------------|--|-------------|--|
| EQEP2 ENCLK |  | EQEP1 ENCLK |  | Reserved    |  |             |  | ECAP3 ENCLK |  | ECAP2 ENCLK |  | ECAP1 ENCLK |  |             |  |
| R/W-0       |  | R/W-0       |  | R-0         |  |             |  | R/W-0       |  | R/W-0       |  | R/W-0       |  |             |  |
| 7           |  | 5           |  | 4           |  | 3           |  | 2           |  | 1           |  | 0           |  |             |  |
| EPWM8 ENCLK |  | EPWM7 ENCLK |  | EPWM6 ENCLK |  | EPWM5 ENCLK |  | EPWM4 ENCLK |  | EPWM3 ENCLK |  | EPWM2 ENCLK |  | EPWM1 ENCLK |  |
| R/W-0       |  | R/W-0       |  | R/W-0       |  | R/W-0       |  | R/W-0       |  | R/W-0       |  | R/W-0       |  | R/W-0       |  |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 1-16. Peripheral Clock Control 1 Register (PCLKCR1) Field Descriptions**

| Bits  | Field      | Value  | Description <sup>(1)</sup>                                                                                                                                      |
|-------|------------|--------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15    | EQEP2ENCLK | 0<br>1 | eQEP2 clock enable<br>The eQEP2 module is not clocked. (default) <sup>(2)</sup><br>The eQEP2 module is clocked by the system clock (SYSCLKOUT).                 |
| 14    | EQEP1ENCLK | 0<br>1 | eQEP1 clock enable<br>The eQEP1 module is not clocked. (default) <sup>(2)</sup><br>The eQEP1 module is clocked by the system clock (SYSCLKOUT).                 |
| 13-11 | Reserved   |        | Any writes to these bits must always have a value of 0.                                                                                                         |
| 10    | ECAP3ENCLK | 0<br>1 | eCAP3 clock enable<br>The eCAP3 module is not clocked. (default) <sup>(2)</sup><br>The eCAP3 module is clocked by the system clock (SYSCLKOUT).                 |
| 9     | ECAP2ENCLK | 0<br>1 | eCAP2 clock enable<br>The eCAP2 module is not clocked. (default) <sup>(2)</sup><br>The eCAP2 module is clocked by the system clock (SYSCLKOUT).                 |
| 8     | ECAP1ENCLK | 0<br>1 | eCAP1 clock enable<br>The eCAP1 module is not clocked. (default) <sup>(2)</sup><br>The eCAP1 module is clocked by the system clock (SYSCLKOUT).                 |
| 7     | EPWM8ENCLK | 0<br>1 | ePWM8 clock enable. <sup>(3)</sup><br>The ePWM8 module is not clocked. (default) <sup>(2)</sup><br>The ePWM8 module is clocked by the system clock (SYSCLKOUT). |
| 6     | EPWM7ENCLK | 0<br>1 | ePWM7 clock enable. <sup>(3)</sup><br>The ePWM7 module is not clocked. (default) <sup>(2)</sup><br>The ePWM7 module is clocked by the system clock (SYSCLKOUT). |
| 5     | EPWM6ENCLK | 0<br>1 | ePWM6 clock enable. <sup>(3)</sup><br>The ePWM6 module is not clocked. (default) <sup>(2)</sup><br>The ePWM6 module is clocked by the system clock (SYSCLKOUT). |
| 4     | EPWM5ENCLK | 0<br>1 | ePWM5 clock enable <sup>(3)</sup><br>The ePWM5 module is not clocked. (default) <sup>(2)</sup><br>The ePWM5 module is clocked by the system clock (SYSCLKOUT).  |
| 3     | EPWM4ENCLK | 0<br>1 | ePWM4 clock enable. <sup>(3)</sup><br>The ePWM4 module is not clocked. (default) <sup>(2)</sup><br>The ePWM4 module is clocked by the system clock (SYSCLKOUT). |

**Table 1-16. Peripheral Clock Control 1 Register (PCLKCR1) Field Descriptions (continued)**

| Bits | Field      | Value | Description <sup>(1)</sup>                                                                      |
|------|------------|-------|-------------------------------------------------------------------------------------------------|
| 2    | EPWM3ENCLK | 0     | ePWM3 clock enable. <sup>(3)</sup><br>The ePWM3 module is not clocked. (default) <sup>(2)</sup> |
|      |            | 1     | The ePWM3 module is clocked by the system clock (SYSCLKOUT).                                    |
| 1    | EPWM2ENCLK | 0     | ePWM2 clock enable. <sup>(3)</sup><br>The ePWM2 module is not clocked. (default) <sup>(2)</sup> |
|      |            | 1     | The ePWM2 module is clocked by the system clock (SYSCLKOUT).                                    |
| 0    | EPWM1ENCLK | 0     | ePWM1 clock enable. <sup>(3)</sup><br>The ePWM1 module is not clocked. (default) <sup>(2)</sup> |
|      |            | 1     | The ePWM1 module is clocked by the system clock (SYSCLKOUT).                                    |

(1) This register is EALLOW protected. See [Section 1.5.2](#) for more information.

(2) If a peripheral block is not used, the clock to that peripheral can be turned off to minimize power consumption.

(3) To start the ePWM Time-base clock (TBCLK) within the ePWM modules, the TBCLKSYNC bit in PCLKCR0 must also be set.

**Figure 1-16. Peripheral Clock Control 2 Register (PCLKCR2)**

|          |    |                 |                 |                 |                 |
|----------|----|-----------------|-----------------|-----------------|-----------------|
| 15       | 12 | 11              | 10              | 9               | 8               |
| Reserved |    | HRCAP4<br>ENCLK | HRCAP3<br>ENCLK | HRCAP2<br>ENCLK | HRCAP1<br>ENCLK |
| R-0      |    | R/W-0           | R/W-0           | R/W-0           | R/W-0           |
| 7        |    |                 |                 |                 | 0               |
| Reserved |    |                 |                 |                 |                 |
| R-0      |    |                 |                 |                 |                 |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 1-17. Peripheral Clock Control 2 Register (PCLKCR2) Field Descriptions**

| Bit   | Field       | Value  | Description                                                                                 |
|-------|-------------|--------|---------------------------------------------------------------------------------------------|
| 15-12 | Reserved    | 0      | Any writes to these bits must always have a value of 0.                                     |
| 11    | HRCAP4ENCLK | 0<br>1 | The HRCAP4 module is not clocked. (default) <sup>(1)</sup><br>The HRCAP4 module is clocked. |
| 10    | HRCAP3ENCLK | 0<br>1 | The HRCAP3 module is not clocked. (default) <sup>(1)</sup><br>The HRCAP3 module is clocked. |
| 9     | HRCAP2ENCLK | 0<br>1 | The HRCAP2 module is not clocked. (default) <sup>(1)</sup><br>The HRCAP2 module is clocked. |
| 8     | HRCAP1ENCLK | 0<br>1 | The HRCAP1 module is not clocked. (default) <sup>(1)</sup><br>The HRCAP1 module is clocked. |
| 0-7   | Reserved    |        | Any writes to these bits must always have a value of 0.                                     |

(1) If a peripheral block is not used, the clock to that peripheral can be turned off to minimize power consumption.

**Figure 1-17. Peripheral Clock Control 3 Register (PCLKCR3)**

|               |               |          |          |              |                    |                    |                    |
|---------------|---------------|----------|----------|--------------|--------------------|--------------------|--------------------|
| 15            | 14            | 13       | 12       | 11           | 10                 | 9                  | 8                  |
| USB0<br>ENCLK | CLA1<br>ENCLK | Reserved | Reserved | DMA<br>ENCLK | CPUTIMER2<br>ENCLK | CPUTIMER1<br>ENCLK | CPUTIMER0<br>ENCLK |
| R-0           | R/W-0         | R-1      | R-0      | R/W-0        | R/W-1              | R/W-1              | R/W-1              |
| 7             |               |          |          | 3            | 2                  | 1                  | 0                  |
| Reserved      |               |          |          |              | COMP3<br>ENCLK     | COMP2<br>ENCLK     | COMP1<br>ENCLK     |
| R-0           |               |          |          |              | R/W-0              | R/W-0              | R/W-0              |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 1-18. Peripheral Clock Control 3 Register (PCLKCR3) Field Descriptions**

| Bit | Field          | Value  | Description                                                                                |
|-----|----------------|--------|--------------------------------------------------------------------------------------------|
| 15  | USB0ENCLK      | 0<br>1 | USB module clock enable<br>Clock is disabled<br>Clock is enabled                           |
| 14  | CLA1ENCLK      | 0<br>1 | CLA module clock enable<br>CLA is not clocked<br>CLA is clocked                            |
| 13  | Reserved       |        | Reserved                                                                                   |
| 12  | Reserved       |        | Any writes to these bits must always have a value of 0.                                    |
| 11  | DMAENCLK       | 0<br>1 | DMA module clock enable<br>DMA is not clocked<br>DMA is clocked                            |
| 10  | CPUTIMER2ENCLK | 0<br>1 | CPU Timer 2 Clock Enable<br>The CPU Timer 2 is not clocked.<br>The CPU Timer 2 is clocked. |
| 9   | CPUTIMER1ENCLK | 0<br>1 | CPU Timer 1 Clock Enable<br>The CPU Timer 1 is not clocked.<br>The CPU Timer 1 is clocked. |
| 8   | CPUTIMER0ENCLK | 0<br>1 | CPU Timer 0 Clock Enable<br>The CPU Timer 0 is not clocked.<br>The CPU Timer 0 is clocked. |
| 7-3 | Reserved       |        | Any writes to these bits must always have a value of 0.                                    |
| 2   | COMP3ENCLK     | 0<br>1 | Comparator3 clock enable<br>Comparator3 is not clocked<br>Comparator3 is clocked           |
| 1   | COMP2ENCLK     | 0<br>1 | Comparator2 clock enable<br>Comparator2 is not clocked<br>Comparator2 is clocked           |
| 0   | COMP1ENCLK     | 0<br>1 | Comparator1 clock enable<br>Comparator1 is not clocked<br>Comparator1 is clocked           |

### 1.3.1.2 Configuring the Low-Speed Peripheral Clock Prescaler (LOSPCP)

The low-speed peripheral clock prescale (LOSPCP) registers are used to configure the low-speed peripheral clocks. See [Figure 1-18](#) for the LOSPCP layout.

**Figure 1-18. Low-Speed Peripheral Clock Prescaler Register (LOSPCP)**

|          |  |   |   |         |
|----------|--|---|---|---------|
| 15       |  | 3 | 2 | 0       |
| Reserved |  |   |   | LSPCLK  |
| R-0      |  |   |   | R/W-010 |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 1-19. Low-Speed Peripheral Clock Prescaler Register (LOSPCP) Field Descriptions**

| Bits | Field    | Value | Description <sup>(1)</sup>                                                                                                                                                                               |
|------|----------|-------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-3 | Reserved |       | Reserved                                                                                                                                                                                                 |
| 2-0  | LSPCLK   |       | These bits configure the low-speed peripheral clock (LSPCLK) rate relative to SYSCLKOUT:<br>If LOSPCP <sup>(2)</sup> ≠ 0, then LSPCLK = SYSCLKOUT/(LOSPCP x 2)<br>If LOSPCP = 0, then LSPCLK = SYSCLKOUT |
|      |          | 000   | Low speed clock = SYSCLKOUT/1                                                                                                                                                                            |
|      |          | 001   | Low speed clock= SYSCLKOUT/2                                                                                                                                                                             |
|      |          | 010   | Low speed clock= SYSCLKOUT/4 (reset default)                                                                                                                                                             |
|      |          | 011   | Low speed clock= SYSCLKOUT/6                                                                                                                                                                             |
|      |          | 100   | Low speed clock= SYSCLKOUT/8                                                                                                                                                                             |
|      |          | 101   | Low speed clock= SYSCLKOUT/10                                                                                                                                                                            |
|      |          | 110   | Low speed clock= SYSCLKOUT/12                                                                                                                                                                            |
|      |          | 111   | Low speed clock= SYSCLKOUT/14                                                                                                                                                                            |

(1) This register is EALLOW protected. See [Section 1.5.2](#) for more information.

(2) LOSPCP in this equation denotes the value of bits 2:0 in the LOSPCP register.

### 1.3.2 OSC and PLL Block

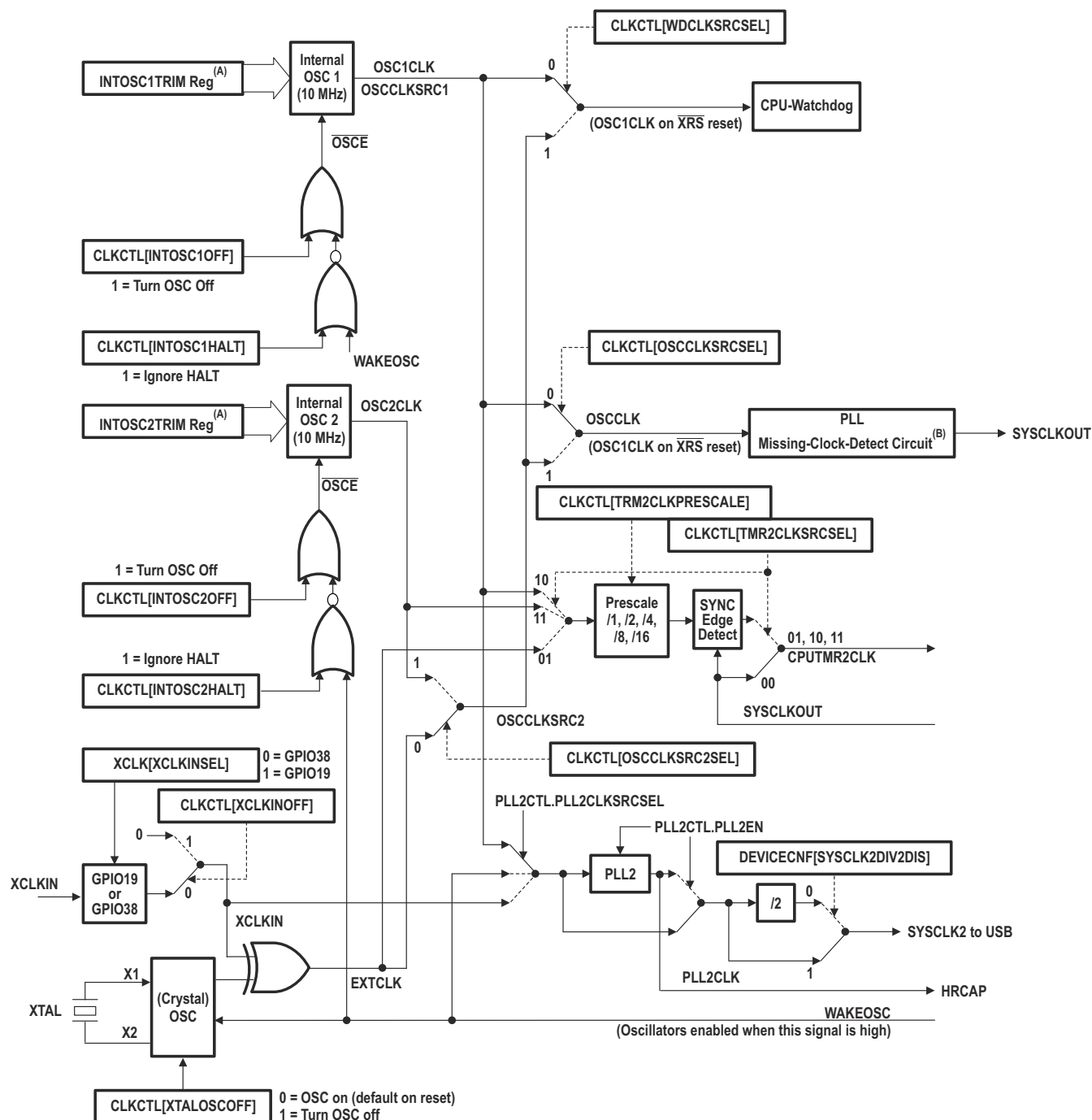
The on-chip oscillators and phase-locked loop (PLL) block provide the clocking signals for the device, as well as control for low-power mode (LPM) entry or exit.

#### 1.3.2.1 Input Clock Options

The device has two internal oscillators (INTOSC1 and INTOSC2) that need no external components. It also has an on-chip, PLL-based clock module. [Figure 1-19](#) shows the different options that are available to clock the device. Following are the input clock options available:

- **INTOSC1 (Internal zero-pin Oscillator 1):** This is the on-chip internal oscillator 1. It can provide the clock for the Watchdog block, CPU-core and CPU-Timer 2. This is the default clock source upon reset.
- **INTOSC2 (Internal zero-pin Oscillator 2):** This is the on-chip internal oscillator 2. It can provide the clock for the Watchdog block, CPU-core and CPU-Timer 2. Both INTOSC1 and INTOSC2 can be independently chosen for the Watchdog block, CPU-core, and CPU-Timer 2. If using INTOSC2 as a clock source, refer to the *Advisory Oscillator: CPU clock switching to INTOSC2 may result in missing clock condition after reset* in the device errata.
- **XTAL OSC (Crystal or Resonator):** The on-chip crystal oscillator enables the use of an external quartz crystal or ceramic resonator. The crystal or resonator is connected to the X1/X2 pins.
- **XCLKIN (External clock source):** If the on-chip crystal oscillator is not used, this mode allows it to be bypassed. The device clock is generated from an external clock source input on the XCLKIN pin. Note that the XCLKIN is multiplexed with GPIO19 or GPIO38 pin. The XCLKIN input can be selected as GPIO19 or GPIO38 via the XCLKINSEL bit in XCLK register. The CLKCTL[XCLKINOFF] bit disables this clock input

(forced low). If the clock source is not used or the respective pins are used as GPIOs, the user should disable it at boot time.



A. Register loaded from TI OTP-based calibration function.

**Figure 1-19. Clocking Options**



### 1.3.2.1.1 Trimming INTOSCn

The nominal frequency of both INTOSC1 and INTOSC2 is 10 MHz. Two 16-bit registers are provided for trimming each oscillator at manufacturing time (called coarse trim) and also provide a way to trim the oscillator using software (called fine trim). The bit layout for both registers is the same, so only one is shown with "n" in place of the numbers 1 or 2.

**Figure 1-20. Internal Oscillator Trim n (INTOSCnTRIM) Register**

|          |          |          |            |   |   |
|----------|----------|----------|------------|---|---|
| 15       | 14       | 9        | 8          | 7 | 0 |
| Reserved | FINETRIM | Reserved | COARSETRIM |   |   |
| R-0      | R/W-0    | R-0      | R/W-0      |   |   |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 1-20. Internal Oscillator n Trim (INTOSCnTRIM) Register Field Descriptions**

| Bit  | Field      | Value | Description <sup>(1) (2)</sup>                                   |
|------|------------|-------|------------------------------------------------------------------|
| 15   | Reserved   |       | Any writes to these bits must always have a value of 0.          |
| 14-9 | FINETRIM   |       | 6-bit Fine Trim Value: Signed magnitude value (- 31 to + 31)     |
| 8    | Reserved   |       | Any writes to these bits must always have a value of 0.          |
| 7-0  | COARSETRIM |       | 8-bit Coarse Trim Value: Signed magnitude value (- 127 to + 127) |

(1) These registers are EALLOW protected.

(2) The internal oscillators are software trimmed with parameters stored in OTP. During boot time, the boot-ROM copies this value to the above registers.

### 1.3.2.1.2 Device\_Cal

The device calibration routine, *Device\_cal()*, is programmed into TI reserved memory by the factory. The boot ROM automatically calls the *Device\_cal()* routine to calibrate the internal oscillators and ADC with device-specific calibration data. During normal operation, this process occurs automatically and no action is required by the user.

If the boot ROM is bypassed by the Code Composer Studio™ IDE during the development process, then the calibration process must be initiated by the application. For working examples, see the system initialization routines in *C2000Ware*.

#### Note

Failure to initialize these registers will cause the oscillators and ADC to function out of specification. The following three steps describe how to call the *Device\_cal* routine from an application.

**Step 1:** Create a pointer to the *Device\_cal* function as shown in [Example 1-1](#). This #define is included in *C2000Ware*.

**Step 2:** Call the function pointed to by *Device\_cal()* as shown in [Example 1-1](#). The ADC clocks must be enabled before making this call.

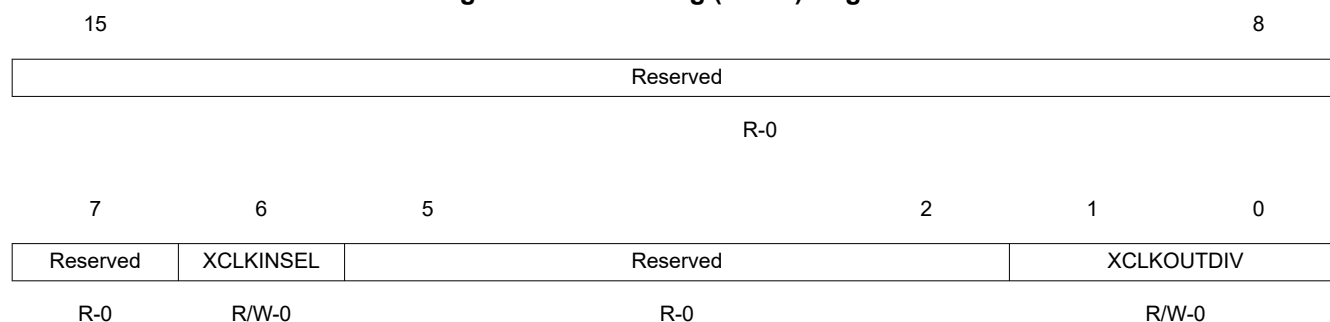
### Example 1-1. Calling the Device\_cal() function

```
//Device_cal is a pointer to a function
//that begins at the address shown
# define Device_cal (void(*) (void)) 0x3D7C80
...
EALLOW;
SysCtrlRegs.PCLKCR0.bit.ADCENCLK = 1;
(*Device_cal)();
SysCtrlRegs.PCLKCR0.bit.ADCENCLK = 0;
EDIS;
...
```

### 1.3.2.2 Configuring Input Clock Source and XCLKOUT Options (XCLK)

The XCLK register is used to choose the GPIO pin for XCLKIN input and to configure the XCLKOUT pin frequency.

**Figure 1-21. Clocking (XCLK) Register**



LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 1-21. Clocking (XCLK) Field Descriptions**

| Bit  | Field                     | Value                | Description <sup>(1)</sup>                                                                                                                                                                               |
|------|---------------------------|----------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-7 | Reserved                  |                      | Any writes to these bits must always have a value of 0.                                                                                                                                                  |
| 6    | XCLKINSEL                 | 0<br>1               | XCLKIN Source Select Bit: This bit selects the source<br>GPIO38 is XCLKIN input source (this is also the JTAG port TCK source)<br>GPIO19 is XCLKIN input source                                          |
| 5-2  | Reserved                  |                      | Any writes to these bits must always have a value of 0.                                                                                                                                                  |
| 1-0  | XCLKOUTDIV <sup>(2)</sup> | 00<br>01<br>10<br>11 | XCLKOUT Divide Ratio: These two bits select the XCLKOUT frequency ratio relative to SYSCLKOUT. The ratios are:<br>XCLKOUT = SYSCLKOUT/4<br>XCLKOUT = SYSCLKOUT/2<br>XCLKOUT = SYSCLKOUT<br>XCLKOUT = Off |

(1) The XCLKINSEL bit in the XCLK register is reset by  $\overline{\text{XRS}}$  input signal.

(2) Refer to the device data sheet for the maximum permissible XCLKOUT frequency.

### 1.3.2.3 Configuring Device Clock Domains (CLKCTL)

The CLKCTL register is used to choose between the available clock sources and also configure device behavior during clock failure.

**Figure 1-22. Clock Control (CLKCTL) Register**

|                 |            |           |               |              |              |                |               |
|-----------------|------------|-----------|---------------|--------------|--------------|----------------|---------------|
| 15              | 14         | 13        | 12            | 11           | 10           | 9              | 8             |
| NMIRESETSEL     | XTALOSCOFF | XCLKINOFF | WDHALTI       | INTOSC2HALTI | INTOSC2OFF   | INTOSC1HALTI   | INTOSC1OFF    |
| R/W-0           | R/W-0      | R/W-0     | R/W-0         | R/W-0        | R/W-0        | R/W-0          | R/W-0         |
| 7               | 5          | 4         | 3             | 2            | 1            | 0              |               |
| TMR2CLKPRESCALE |            |           | TMR2CLKSRCSEL |              | WDCLK SRCSEL | OSCCLK SRC2SEL | OSCCLK SRCSEL |
| R/W-0           |            |           | R/W-0         |              | R/W-0        | R/W-0          | R/W-0         |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 1-22. Clock Control (CLKCTL) Register Field Descriptions**

| Bit | Field        | Value  | Description <sup>(1)</sup>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|-----|--------------|--------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15  | NMIRESETSEL  | 0<br>1 | NMI Reset Select bit. This bit selects the action when the VCOCLK counter overflows due to a missing clock condition.<br><br>0 $\overline{\text{MCLKRS}}$ is driven without any delay (default on reset)<br>1 NMI Watchdog Reset ( $\overline{\text{NMIRS}}$ ) initiates $\overline{\text{MCLKRS}}$<br><b>Note:</b> The $\overline{\text{CLOCKFAIL}}$ signal is generated regardless of this mode selection.                                                                                                                                                                                                     |
| 14  | XTALOSCOFF   | 0<br>1 | Crystal Oscillator Off bit. This bit could be used to turn off the crystal oscillator if it is not used.<br><br>0 Crystal oscillator on (default on reset)<br>1 Crystal oscillator off                                                                                                                                                                                                                                                                                                                                                                                                                           |
| 13  | XCLKINOFF    | 0<br>1 | XCLKIN Off Bit: This bit turns external XCLKIN oscillator input off:<br><br>0 XCLKIN oscillator input on (default on reset)<br>1 XCLKIN oscillator input off<br><b>Note:</b> You need to select XCLKIN GPIO pin source via the XCLKINSEL bit in the XCLK register. See the XCLK register description ( ) for more details. XTALOSCOFF must be set to 1, if XCLKIN is used.                                                                                                                                                                                                                                       |
| 12  | WDHALTI      | 0<br>1 | Watchdog HALT Mode Ignore bit. This bit selects if the watchdog is automatically turned off by the HALT mode or not. This feature can be used to allow the selected watchdog clock source to continue clocking the watchdog when HALT mode is active. This would enable the watchdog to periodically wake up the device.<br><br>0 Watchdog automatically turned off by HALT (default on reset)<br>1 Watchdog continues to function in HALT mode.                                                                                                                                                                 |
| 11  | INTOSC2HALTI | 0<br>1 | Internal Oscillator 2 HALT Mode Ignore bit. This bit selects if the internal oscillator 2 is automatically turned off by the HALT mode or not. This feature can be used to allow the internal oscillator to continue clocking when HALT mode is active. This would enable a quicker wake-up from HALT.<br><br>0 Internal oscillator 2 automatically turned Off by HALT (default on reset)<br>1 Internal oscillator 2 continues to function in HALT mode. This feature can be used to allow the internal oscillator to continue clocking when HALT mode is active. This would enable a quicker wake-up from HALT. |
| 10  | INTOSC2OFF   | 0<br>1 | Internal Oscillator 2 Off bit. This bit turns oscillator 2 off.<br><br>0 Internal oscillator 2 On (default on reset)<br>1 Internal oscillator 2 Off. This bit could be used by the user to turn off the internal oscillator 2 if it is not used. This selection is not affected by the missing clock detect circuit.                                                                                                                                                                                                                                                                                             |

**Table 1-22. Clock Control (CLKCTL) Register Field Descriptions (continued)**

| Bit | Field           | Value                                                | Description <sup>(1)</sup>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|-----|-----------------|------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 9   | INTOSC1HALTI    | 0<br>1                                               | Internal Oscillator 1 HALT Mode Ignore bit. This bit selects if the internal oscillator 1 is automatically turned off by the HALT mode or not:<br>Internal oscillator 1 automatically turned Off by HALT (default on reset)<br>Internal oscillator 1 continues to function in HALT mode. This feature can be used to allow the internal oscillator to continue clocking when HALT mode is active. This would enable a quicker wake-up from HALT.                                                                                                                                    |
| 8   | INTOSC1OFF      | 0<br>1                                               | Internal Oscillator 1 Off bit. This bit turns oscillator 1 off:<br>Internal oscillator 1 On (default on reset)<br>Internal oscillator 1 Off. This bit could be used by the user to turn off the internal oscillator 1 if it is not used. This selection is not affected by the missing clock detect circuit.                                                                                                                                                                                                                                                                        |
| 7-5 | TMR2CLKPRESCALE | 000<br>001<br>010<br>011<br>100<br>101<br>110<br>111 | CPU Timer 2 Clock Pre-Scale Value. These bits select the pre-scale value for the selected clock source for CPU Timer 2. This selection is not affected by the missing clock detect circuit.<br>/1 (default on reset)<br>/2<br>/4<br>/8<br>/16<br>Reserved<br>Reserved<br>Reserved                                                                                                                                                                                                                                                                                                   |
| 4-3 | TMR2CLKSRCSEL   | 00<br>01<br>10<br>11                                 | CPU Timer 2 Clock Source Select bit. This bit selects the source for CPU Timer 2.<br>SYSCLKOUT selected (default on reset, pre-scaler is bypassed)<br>External oscillator selected (at XOR output)<br>Internal oscillator 1 selected<br>Internal oscillator 2 selected. This selection is not affected by the missing clock detect circuit.                                                                                                                                                                                                                                         |
| 2   | WDCLKSRCSEL     | 0<br>1                                               | Watchdog Clock Source Select bit. This bit selects the source for the watchdog clock. On XRS low and after XRS goes high, internal oscillator 1 is selected by default. You need to select external oscillator or internal oscillator 2 during their initialization process. If missing clock detect circuit detects a missing clock, then this bit is forced to 0 and internal oscillator 1 is selected. The user changing this bit does not affect the PLLCR value.<br>Internal oscillator 1 selected (default on reset)<br>External oscillator or Internal oscillator 2 selected |
| 1   | OSCCLKSRC2SEL   | 0<br>1                                               | Oscillator 2 Clock Source Select bit. This bit selects between internal oscillator 2 or external oscillator. This selection is not affected by the missing clock detect circuit.<br>External oscillator selected (default on reset)<br>Internal oscillator 2 selected                                                                                                                                                                                                                                                                                                               |

**Table 1-22. Clock Control (CLKCTL) Register Field Descriptions (continued)**

| Bit | Field        | Value | Description <sup>(1)</sup>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|-----|--------------|-------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 0   | OSCCLKSRCSEL |       | Oscillator Clock Source Select bit. This bit selects the source for OSCCLK. On $\overline{XRS}$ low and after $\overline{XRS}$ goes high, internal oscillator 1 is selected by default. The user would need to select external oscillator or internal oscillator 2 during their initialization process. Whenever the user changes the clock source using these bits, the PLLCR register will be automatically forced to zero. This prevents potential PLL overshoot. The user will then have to write to the PLLCR register to configure the appropriate PLL multiplier value. The user can also configure the PLL lock period using the PLLLOCKPRD register to reduce the lock time if necessary. If the missing clock detect circuit detects a missing clock, then this bit is automatically forced to 0 and internal oscillator 1 is selected. The PLLCR register will also be automatically forced to zero to prevent any potential overshoot. |
|     |              | 0     | Internal oscillator 1 selected (default on reset)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|     |              | 1     | External oscillator or Internal oscillator 2 selected. Note: If users wish to use oscillator 2 or external oscillator to clock the CPU, they should configure the OSCCLKSRC2SEL bit first, and then write to the OSCCLKSRCSEL bit next.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |

(1) The internal oscillators are software trimmed with parameters stored in OTP. During boot time, the boot-ROM copies this value to the above registers.

### 1.3.2.3.1 Switching the Input Clock Source

The following procedure may be used to switch clock sources:

1. Use CPU Timer 2 to detect if clock sources are functional.
2. If any of the clock sources is not functional, turn off the respective clock source (using the respective CLKCTL bit).
3. Switch over to a new clock source.
4. If clock source switching occurred while in Limp Mode, then write a 1 to MCLKCLR will be issued to exit Limp Mode.

If OSCCLKSRC2 (an external Crystal [XTAL] or oscillator [XCLKIN input] or Internal Oscillator 2 [INTOSC2]) is selected as the clock source and a missing clock is detected, the missing clock detect circuit will automatically switch to Internal Oscillator 1 (OSCCLKSRC1) and generate a  $\overline{CLOCKFAIL}$  signal. In addition, the PLLCR register is forced to zero (PLL is bypassed) to prevent any potential overshoot. The user can then write to the PLLCR register to re-lock the PLL. Under this situation, the missing clock detect circuit will be automatically re-enabled (PLLSTS[MCLKSTS] bit will be automatically cleared). If Internal Oscillator 1 (OSCCLKSRC1) should also fail, then under this situation, the missing clock detect circuit will remain in limp mode. The user will have to re-enable the logic via the PLLSTS[MCLKCLR] bit.

### 1.3.2.3.2 Switching to INTOSC2 in the Absence of External Clocks

For the device to work properly upon a switch from INTOSC1 to INTOSC2 in the absence of any external clock, the application code needs to write a 1 to the CLKCTL.XTALOSCOFF and CLKCTL.XCLKINOFF bits first. This is to indicate to the clock switching circuitry that external clocks are not present. Only after this should the OSCCLKSRCSEL and OSCCLKSRC2SEL bits be written to. Note that this sequence should be separated into two writes as follows:

First write → CLKCTL.XTALOSCOFF=1 and CLKCTL.XCLKINOFF=1

Second write → CLKCTL.OSCCLKSRCSEL=1 and CLKCTL.OSCCLKSRC2SEL=1

The second write should not alter the values of XTALOSCOFF and XCLKINOFF bits. If *C2000Ware*, supplied by Texas Instruments is used, clock switching can be achieved with the following code snip:

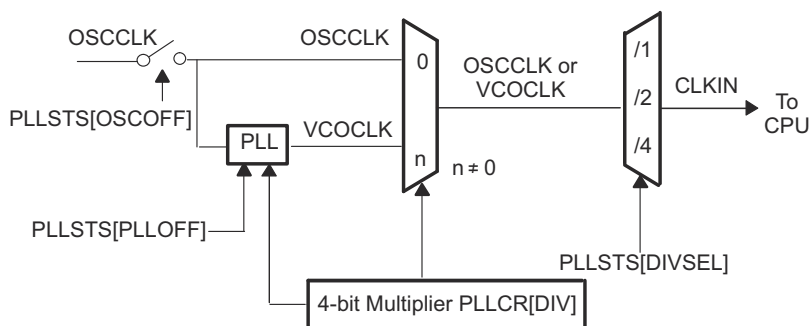
```
SysCtrlRegs.CLKCTL.all = 0x6000; // Set XTALOSCOFF=1 & XCLKINOFF=1
SysCtrlRegs.CLKCTL.all = 0x6003; // Set OSCCLKSRCSEL=1 & OSCCLKSRC2SEL=1
```

The system initialization file DSP2806x\_SysCtrl.c, provided as part of *C2000Ware* also contain functions to switch to different clock sources. If an attempt is made to switch from INTOSC1 to INTOSC2 without the write to the XTALOSCOFF and XCLKINOFF bits, a missing clock will be detected due to the absence of external clock

source (even after the proper source selection). The PLLCR will be zeroed out and the device will automatically clear the MCLKSTS bit and switch back INTOSC1.

### 1.3.2.4 PLL-based Clock Module

This device has two PLL modules. “PLL” refers to the main PLL that generates the clock for the core and all peripherals. PLL2 refers to the PLL that generates the clock for the USB and HRCAP modules. [Figure 1-23](#) shows the OSC and PLL block diagram.



**Figure 1-23. OSC and PLL Block**

The following is applicable for devices that have X1/X2 pins:

When using XCLKIN as the external clock source, you must tie X1 low and leave X2 disconnected.

**Table 1-23. Possible PLL Configuration Modes**

| PLL Mode    | Remarks                                                                                                                                                                                                                                                                                                                                                                                                         | PLLSTS[DIVSEL] <sup>(1)</sup> | CLKIN and SYSCLKOUT <sup>(2)</sup>     |
|-------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------|----------------------------------------|
| PLL Off     | Invoked by the user setting the PLLOFF bit in the PLLSTS register. The PLL block is disabled in this mode. The CPU clock (CLKIN) can then be derived directly from any one of the following sources: INTOSC1, INTOSC2, XCLKIN pin, or X1/X2 pins. This can be useful to reduce system noise and for low power operation. The PLLCR register must first be set to 0x0000 (PLL Bypass) before entering this mode. | 0, 1<br>2<br>3                | OSCCLK/4<br>OSCCLK/2<br>OSCCLK/1       |
| PLL Bypass  | PLL Bypass is the default PLL configuration upon power-up or after an external reset (XRS). This mode is selected when the PLLCR register is set to 0x0000 or while the PLL locks to a new frequency after the PLLCR register has been modified. In this mode, the PLL itself is bypassed but the PLL is not turned off.                                                                                        | 0, 1<br>2<br>3                | OSCCLK/4<br>OSCCLK/2<br>OSCCLK/1       |
| PLL Enabled | Achieved by writing a non-zero value n into the PLLCR register. Upon writing to the PLLCR, the device will switch to PLL Bypass mode until the PLL locks.                                                                                                                                                                                                                                                       | 0, 1<br>2<br>3                | OSCCLK*n/4<br>OSCCLK*n/2<br>OSCCLK*n/1 |

(1) PLLSTS[DIVSEL] must be 0 before writing to the PLLCR and should be changed only after PLLSTS[PLLLOCKS] = 1. See [Figure 1-24](#).

(2) The input clock and PLLCR[DIV] bits should be chosen in such a way that the output frequency of the PLL (VCOCLK) is a minimum of 50 MHz.

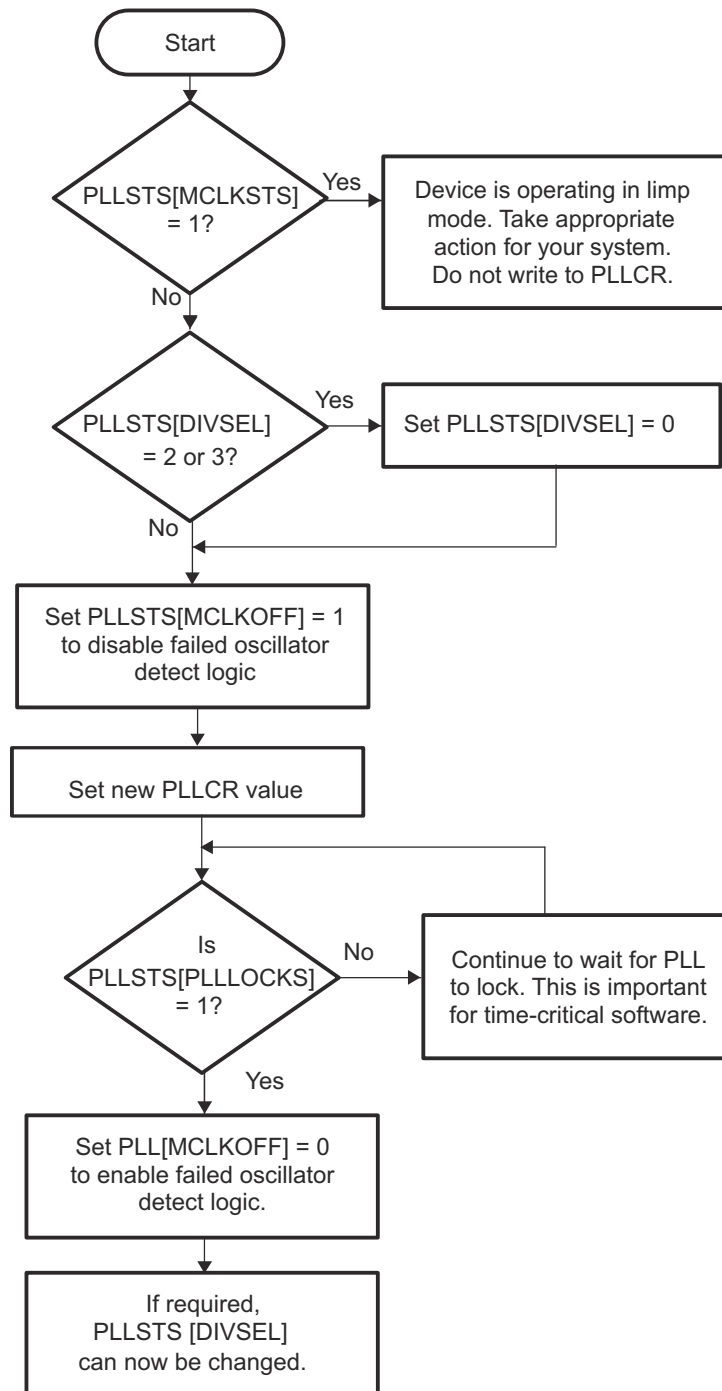
#### 1.3.2.4.1 PLL Control Registers

The PLLCR register is used to change the PLL multiplier of the device. Before writing to the PLLCR register, the following requirements must be met:

- The PLLSTS[DIVSEL] bit must be 0 (CLKIN divide by 4 enabled). Change PLLSTS[DIVSEL] only after the PLL has completed locking, that is, after PLLSTS[PLLLOCKS] = 1.

Once the PLL is stable and has locked at the new specified frequency, the PLL switches CLKIN to the new value as shown in [Table 1-24](#). When this happens, the PLLLOCKS bit in the PLLSTS register is set, indicating that the PLL has finished locking and the device is now running at the new frequency. User software can monitor the PLLLOCKS bit to determine when the PLL has completed locking. Once PLLSTS[PLLLOCKS] = 1, DIVSEL can be changed.

Follow the procedure in [Figure 1-24](#) any time you are writing to the PLLCR register.



**Figure 1-24. PLLCR Change Procedure Flow Chart**



### 1.3.2.4.1.1 PLL Control and Status Register Descriptions

The DIV field in the PLLCR register controls whether the PLL is bypassed or not and sets the PLL clocking ratio when it is not bypassed. PLL bypass is the default mode after reset. Do not write to the DIV field if the PLLSTS[DIVSEL] bit is 10 or 11, or if the PLL is operating in limp mode as indicated by the PLLSTS[MCLKSTS] bit being set. See the procedure for changing the PLLCR described in [Figure 1-24](#).

**Figure 1-25. PLLCR Register Layout**

|          |   |   |       |
|----------|---|---|-------|
| 15       | 5 | 4 | 0     |
| Reserved |   |   | DIV   |
| R-0      |   |   | R/W-0 |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 1-24. PLL Settings**

| SYSCLKOUT (CLKIN) <sup>(1) (2)</sup> |                         |                    |                    |
|--------------------------------------|-------------------------|--------------------|--------------------|
| PLLCR[DIV] Value <sup>(3)</sup>      | PLLSTS[DIVSEL] = 0 or 1 | PLLSTS[DIVSEL] = 2 | PLLSTS[DIVSEL] = 3 |
| 0000 (PLL bypass)                    | OSCCLK/4 (Default)      | OSCCLK/2           | OSCCLK/1           |
| 00001                                | (OSCCLK * 1)/4          | (OSCCLK * 1)/2     | (OSCCLK * 1)/1     |
| 00010                                | (OSCCLK * 2)/4          | (OSCCLK * 2)/2     | (OSCCLK * 2)/1     |
| 00011                                | (OSCCLK * 3)/4          | (OSCCLK * 3)/2     | (OSCCLK * 3)/1     |
| 00100                                | (OSCCLK * 4)/4          | (OSCCLK * 4)/2     | (OSCCLK * 4)/1     |
| 00101                                | (OSCCLK * 5)/4          | (OSCCLK * 5)/2     | (OSCCLK * 5)/1     |
| 00110                                | (OSCCLK * 6)/4          | (OSCCLK * 6)/2     | (OSCCLK * 6)/1     |
| 00111                                | (OSCCLK * 7)/4          | (OSCCLK * 7)/2     | (OSCCLK * 7)/1     |
| 01000                                | (OSCCLK * 8)/4          | (OSCCLK * 8)/2     | (OSCCLK * 8)/1     |
| 01001                                | (OSCCLK * 9)/4          | (OSCCLK * 9)/2     | (OSCCLK * 9)/1     |
| 01010                                | (OSCCLK * 10)/4         | (OSCCLK * 10)/2    | (OSCCLK * 10)/1    |
| 01011                                | (OSCCLK * 11)/4         | (OSCCLK * 11)/2    | (OSCCLK * 11)/1    |
| 01100                                | (OSCCLK * 12)/4         | (OSCCLK * 12)/2    | (OSCCLK * 12)/1    |
| 01101                                | (OSCCLK * 13)/4         | (OSCCLK * 13)/2    | (OSCCLK * 13)/1    |
| 01110                                | (OSCCLK * 14)/4         | (OSCCLK * 14)/2    | (OSCCLK * 14)/1    |
| 01111                                | (OSCCLK * 15)/4         | (OSCCLK * 15)/2    | (OSCCLK * 15)/1    |
| 10000                                | (OSCCLK * 16)/4         | (OSCCLK * 16)/2    | (OSCCLK * 16)/1    |
| 10001                                | (OSCCLK * 17)/4         | (OSCCLK * 17)/2    | (OSCCLK * 17)/1    |
| 10010                                | (OSCCLK * 18)/4         | (OSCCLK * 18)/2    | (OSCCLK * 18)/1    |
| 10011-11111                          | Reserved                | Reserved           | Reserved           |

(1) This register is EALLOW protected. See [Section 1.5.2](#) for more information.

(2) PLLSTS[DIVSEL] must be 0 or 1 before writing to the PLLCR and should be changed only after PLLSTS[PLLLOCKS] = 1. See [Figure 1-24](#).

(3) The PLL control register (PLLCR) and PLL Status Register (PLLSTS) are reset to their default state by the  $\overline{\text{XRS}}$  signal or a watchdog reset only. A reset issued by the debugger or the missing clock detect logic have no effect.

### 1.3.2.4.1.2 PLL Status Register (PLLSTS)

**Figure 1-26. PLL Status Register (PLLSTS)**

|          |          |        |         |         |        |          |          |
|----------|----------|--------|---------|---------|--------|----------|----------|
| 15       | 14       |        |         |         |        | 9        | 8        |
| NORMRDYE | Reserved |        |         |         |        |          | DIVSEL   |
| R/W-0    | R-0      |        |         |         |        |          | R/W-0    |
| 7        | 6        | 5      | 4       | 3       | 2      | 1        | 0        |
| DIVSEL   | MCLKOFF  | OSCOFF | MCLKCLR | MCLKSTS | PLLOFF | Reserved | PLLLOCKS |
| R/W-0    | R/W-0    | R/W-0  | W-0     | R-0     | R/W-0  | R-0      | R-1      |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 1-25. PLL Status Register (PLLSTS) Field Descriptions**

| Bits | Field    | Value              | Description <sup>(1) (2)</sup>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
|------|----------|--------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15   | NORMRDYE | 0<br>1             | <p>NORMRDY Enable Bit: This bit selects if NORMRDY signal from VREG gates the PLL from turning on when the VREG is out of regulation. It may be required to keep the PLL off while coming in and out of HALT mode and this signal can be used for that purpose:</p> <p>0 NORMRDY signal from VREG does not gate PLL (PLL ignores NORMRDY)</p> <p>1 NORMRDY signal from VREG will gate PLL (PLL off when NORMRDY low)</p> <p>The NORMRDY signal from the VREG is low when the VREG is out of regulation and this signal will go high if the VREG is within regulation.</p>                                                                                                                                                                                                                                           |
| 14-9 | Reserved |                    | Any writes to these bits must always have a value of 0.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| 8:7  | DIVSEL   | 00, 01<br>10<br>11 | <p>Divide Select: This bit selects between /4, /2, and /1 for CLKIN to the CPU. The configuration of the DIVSEL bit is as follows:</p> <p>Select Divide By 4 for CLKIN</p> <p>Select Divide By 2 for CLKIN</p> <p>Select Divide By 1 for CLKIN</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| 6    | MCLKOFF  | 0<br>1             | <p>Missing clock-detect off bit</p> <p>0 Main oscillator fail-detect logic is enabled. (default)</p> <p>1 Main oscillator fail-detect logic is disabled and the PLL will not issue a limp-mode clock. Use this mode when code must not be affected by the detection circuit. For example, if external clocks are turned off.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| 5    | OSCOFF   | 0<br>1             | <p>Oscillator Clock Off Bit</p> <p>0 The OSCCLK signal from X1/X2 or XCLKIN is fed to the PLL block. (default)</p> <p>1 The OSCCLK signal from X1/X2 or XCLKIN is not fed to the PLL block. This does not shut down the internal oscillator. The OSCOFF bit is used for testing the missing clock detection logic.</p> <p>When the OSCOFF bit is set, do not enter HALT or STANDBY modes or write to PLLCR as these operations can result in unpredictable behavior.</p> <p>When the OSCOFF bit is set, the behavior of the watchdog is different depending on which input clock source (X1, X1/X2 or XCLKIN) is being used:</p> <ul style="list-style-type: none"> <li>X1/X2: The watchdog is not functional.</li> <li>XCLKIN: The watchdog is functional and should be disabled before setting OSCOFF.</li> </ul> |
| 4    | MCLKCLR  | 0<br>1             | <p>Missing Clock Clear Bit.</p> <p>0 Writing a 0 has no effect. This bit always reads 0.</p> <p>1 Forces the missing clock detection circuits to be cleared and reset. If OSCCLK is still missing, the detection circuit will again generate a reset to the system, set the missing clock status bit (MCLKSTS), and the CPU will be clocked by the PLL operating at a limp mode frequency.</p>                                                                                                                                                                                                                                                                                                                                                                                                                      |

**Table 1-25. PLL Status Register (PLLSTS) Field Descriptions (continued)**

| Bits | Field    | Value             | Description <sup>(1)</sup> <sup>(2)</sup>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
|------|----------|-------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 3    | MCLKSTS  | <p>0</p> <p>1</p> | <p>Missing Clock Status Bit. Check the status of this bit after a reset to determine whether a missing oscillator condition was detected. Under normal conditions, this bit should be 0. Writes to this bit are ignored. This bit will be cleared by writing to the MCLKCLR bit or by forcing an external reset.</p> <p>Indicates normal operation. A missing clock condition has not been detected.</p> <p>Indicates that OSCCLK was detected as missing. The main oscillator fail detect logic has reset the device and the CPU is now clocked by the PLL operating at the limp mode frequency.</p> <p>When the missing clock detection circuit automatically switches between OSCCLKSRC2 to OSCCLKSRC1 (upon detecting OSCCLKSRC2 failure), this bit will be automatically cleared and the missing clock detection circuit will be re-enabled. For all other cases, the user needs to re-enable this mode by writing a 1 to the MCLKCLR bit.</p> |
| 2    | PLLOFF   | <p>0</p> <p>1</p> | <p>PLL Off Bit. This bit turns off the PLL. This is useful for system noise testing. This mode must only be used when the PLLCR register is set to 0x0000.</p> <p>PLL On (default)</p> <p>PLL Off. While the PLLOFF bit is set the PLL module will be kept powered down.</p> <p>The device must be in PLL bypass mode (PLLCR = 0x0000) before writing a 1 to PLLOFF. While the PLL is turned off (PLLOFF = 1), do not write a non-zero value to the PLLCR.</p> <p>The STANDBY and HALT low power modes will work as expected when PLLOFF = 1. After waking up from HALT or STANDBY the PLL module will remain powered down.</p>                                                                                                                                                                                                                                                                                                                     |
| 1    | Reserved |                   | Any writes to these bits must always have a value of 0.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| 0    | PLLLOCKS | <p>0</p> <p>1</p> | <p>PLL Lock Status Bit.</p> <p>Indicates that the PLLCR register has been written to and the PLL is currently locking. The CPU is clocked by OSCCLK/2 until the PLL locks.</p> <p>Indicates that the PLL has finished locking and is now stable.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |

(1) This register is reset to its default state only by the  $\overline{XRS}$  signal or a watchdog reset. It is not reset by a missing clock or debugger reset.

(2) This register is EALLOW protected. See [Section 1.5.2](#) for more information.

### 1.3.2.4.1.3 PLL Lock Period (PLLLOCKPRD) Register

**Figure 1-27. PLL Lock Period (PLLLOCKPRD) Register**

15

0



LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 1-26. PLL Lock Period (PLLLOCKPRD) Register Field Descriptions**

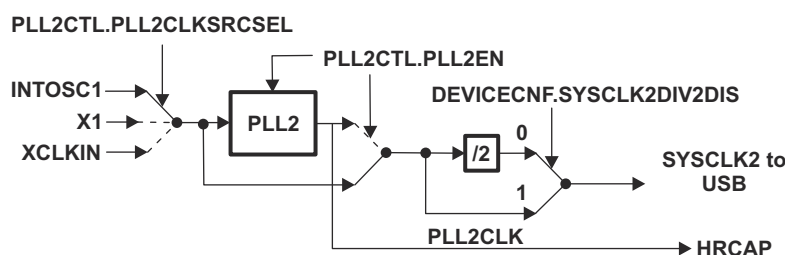
| Bit  | Field      | Value | Description <sup>(1)</sup> <sup>(2)</sup>                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|------|------------|-------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-0 | PLLLOCKPRD |       | PLL Lock Period Counter Value<br><br>These 16 bits configure the PLL lock period. This value is programmable, so shorter PLL lock-time can be programmed by user. The user needs to compute the number of OSCCLK cycles (based on the OSCCLK value used in the design) and update this register.<br><br>PLL Lock Period<br>FFFFh 65535 OSCCLK Cycles (default on reset)<br>FFFEh 65534 OSCCLK Cycles<br>...<br>0001h 1 OSCCLK Cycle<br>0000h 0 OSCCLK Cycles (no PLL lock period) |

(1) PLLLOCKPRD is affected by the  $\overline{XRSn}$  signal only.

(2) This register is EALLOW protected. See [Section 1.5.2](#) for more information.

### 1.3.2.4.1.4 PLL2 Registers

In addition to the main PLL that clocks the CPU, a second PLL (PLL2) exists for clocking the USB and HRCAP modules. Figure 1-28 shows the possible input and output configurations for PLL2.



**Figure 1-28. PLL2 Input and Output Configurations**

#### 1.3.2.4.1.4.1 PLL2 Configuration (PLL2CTL) Register

#### Note

PLL2CTL is affected by the  $\overline{\text{XRS}}$  signal only.

**Figure 1-29. PLL2 Configuration (PLL2CTL) Register**

|          |  |   |        |               |   |
|----------|--|---|--------|---------------|---|
| 15       |  | 3 | 2      | 1             | 0 |
| Reserved |  |   | PLL2EN | PLL2CLKSRCSEL |   |
| R-0      |  |   | R/W-1  | R/W-00        |   |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 1-27. PLL2 Configuration (PLL2CTL) Register Field Descriptions**

| Bit  | Field         | Value                | Description <sup>(1)</sup>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|------|---------------|----------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-3 | Reserved      |                      | Any writes to these bits must always have a value of 0.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| 2    | PLL2EN        | 0<br>1               | PLL enabled or disabled: This bit decides if PLL2 is enabled or not<br>PLL2 is powered off – clock to SYSCLK2 is a direct feed from input clock source as decided by the PLL2CLKSRCSEL bit<br>PLL2 is enabled and clock to SYSCLK2 will depend on the DEVICECNF[SYSCLK2DIV2DIS] bit.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| 1-0  | PLL2CLKSRCSEL | 00<br>01<br>10<br>11 | PLL2 Clock Source Select Bits: These bit select the source for the PLL2 input clock:<br>00 Internal oscillator 1 is selected as clock to PLL2<br>01 Internal oscillator 1 is selected as clock to PLL2<br>10 X1 clock source is selected as clock to PLL2<br>11 GPIO_XCLKIN is selected as clock to PLL2<br>On $\overline{\text{XRS}}$ low and after $\overline{\text{XRS}}$ goes high, X1 is selected as clock source to the USB PLL by default. The user would need to select X1 or GPIO_XCLKIN as clock source during their initialization process.<br>Whenever the user changes the clock source using these bits, the DEVICECNF[SYSCLK2DIV2DIS] bit will be automatically forced to zero. This prevents potential PLL overshoot. The user will then have to write to the DEVICECNF[SYSCLK2DIV2DIS] bit to configure the appropriate divisor ratio. |

(1) This register is EALLOW protected. See Section 1.5.2 for more information.

#### 1.3.2.4.1.4.2 PLL2 Multiplier (PLL2MULT) Register

##### Note

PLL2MULT is affected by  $\overline{\text{XRSn}}$  signal only.

**Figure 1-30. PLL2 Multiplier (PLL2MULT) Register**

|          |  |   |   |          |
|----------|--|---|---|----------|
| 15       |  | 4 | 3 | 0        |
| Reserved |  |   |   | PLL2MULT |
| R-0      |  |   |   | R/W-0x0  |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 1-28. PLL2 Multiplier (PLL2MULT) Register Field Descriptions**

| Bit  | Field    | Value | Description <sup>(1)</sup>                                                                                                                                                                                                               |
|------|----------|-------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-4 | Reserved |       | Any writes to these bits must always have a value of 0.                                                                                                                                                                                  |
| 3-0  | PLL2MULT |       | PLL2 Multiplier. This bit field determines the output frequency of PLL2 ( $\text{PLL2F}_{\text{out}}$ ) for a given input ( $\text{PLL2F}_{\text{in}}$ ).<br>PLL2 should be enabled ( $\text{PLL2EN} = 1$ ) prior to setting these bits. |
|      |          | 0000  | $\text{PLL2F}_{\text{out}} = \text{PLL2F}_{\text{in}}$ (PLL2BYPASS)                                                                                                                                                                      |
|      |          | 0001  | $\text{PLL2F}_{\text{out}} = \text{PLL2F}_{\text{in}} * 1$                                                                                                                                                                               |
|      |          | 0010  | $\text{PLL2F}_{\text{out}} = \text{PLL2F}_{\text{in}} * 2$                                                                                                                                                                               |
|      |          | 0011  | $\text{PLL2F}_{\text{out}} = \text{PLL2F}_{\text{in}} * 3$                                                                                                                                                                               |
|      |          | ...   |                                                                                                                                                                                                                                          |
|      |          | 1111  | $\text{PLL2F}_{\text{out}} = \text{PLL2F}_{\text{in}} * 15$                                                                                                                                                                              |

(1) This register is EALLOW protected. See [Section 1.5.2](#) for more information.

#### 1.3.2.4.1.4.3 PLL2 Lock Status (PLL2STS) Register

##### Note

PLL2STS is affected by  $\overline{\text{XRSn}}$  signal only.

**Figure 1-31. PLL2 Lock Status (PLL2STS) Register**

|          |  |   |           |
|----------|--|---|-----------|
| 15       |  | 1 | 0         |
| Reserved |  |   | PLL2LOCKS |
| R-0      |  |   | R-0x0     |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 1-29. PLL2 Lock Status (PLL2STS) Register Field Descriptions**

| Bit  | Field    | Value | Description                                                                                                                                                                                                                                                                                                                                                    |
|------|----------|-------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-4 | Reserved |       | Any writes to these bits must always have a value of 0.                                                                                                                                                                                                                                                                                                        |
| 3-0  | PLL2MULT |       | PLL2 Lock Status Bit: This bit indicates whether PLL2 is locked or not. When the PLL2MULT setting is modified, a counter is loaded with the value from the main PLL PLLLOCKPRD bit field and the PLL2LOCKS bit is cleared. Once set to a non-zero value, the counter begins down-counting. Upon reaching zero, the counter stops and the PLL2LOCKS bit is set. |
|      |          | 0     | PLL2 is not yet locked                                                                                                                                                                                                                                                                                                                                         |
|      |          | 1     | PLL2 is locked                                                                                                                                                                                                                                                                                                                                                 |

#### 1.3.2.4.1.4.4 SYSCCLK2 Clock Counter (SYSCCLK2CNTR) Register

Since COUNT will begin updating as soon as reset is released, the value of this register after reset will be non-zero before software can read it. SYSCCLK2CNTR will tap off the clock after the /2 from PLL2OUT, if it is enabled. If not enabled, it will tap off PLL2OUT.

**Figure 1-32. SYSCCLK2 Clock Counter (SYSCCLK2CNTR) Register**

|         |   |
|---------|---|
| 15      | 0 |
| COUNT   |   |
| R/C-0x0 |   |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 1-30. SYSCCLK2 Clock Counter (SYSCCLK2CNTR) Register Field Descriptions**

| Bit  | Field | Value | Description                                                                                                                                                                                                                                                                      |
|------|-------|-------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-0 | COUNT |       | SYSCCLK2 Counter Bit Field: This bit field is a free-running counter based off the SYSCCLK2 clock. Software can compare the rate of update of this bit field against the rate of update of a CPU Timer to determine at what approximate frequency the SYSCCLK2 clock is running. |

#### 1.3.2.4.1.4.5 EPWM DMA/CLA Configuration (EPWMCFG) Register

**Figure 1-33. EPWM DMA/CLA Configuration (EPWMCFG) Register**

|          |   |         |
|----------|---|---------|
| 15       | 1 | 0       |
| Reserved |   | CONFIG  |
| R-0      |   | R/W-0x0 |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 1-31. EPWM DMA/CLA Configuration (EPWMCFG) Register Field Descriptions**

| Bit  | Field    | Value  | Description                                                                                                                                                                                  |
|------|----------|--------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-1 | Reserved |        | Any writes to these bits must always have a value of 0.                                                                                                                                      |
| 0    | CONFIG   | 0<br>1 | EPWM DMA Enable Bit:<br>The EPWM blocks are connected to the CLA bus and are inaccessible to the DMA bus<br>The EPWM blocks are connected to the DMA bus and are inaccessible to the CLA bus |

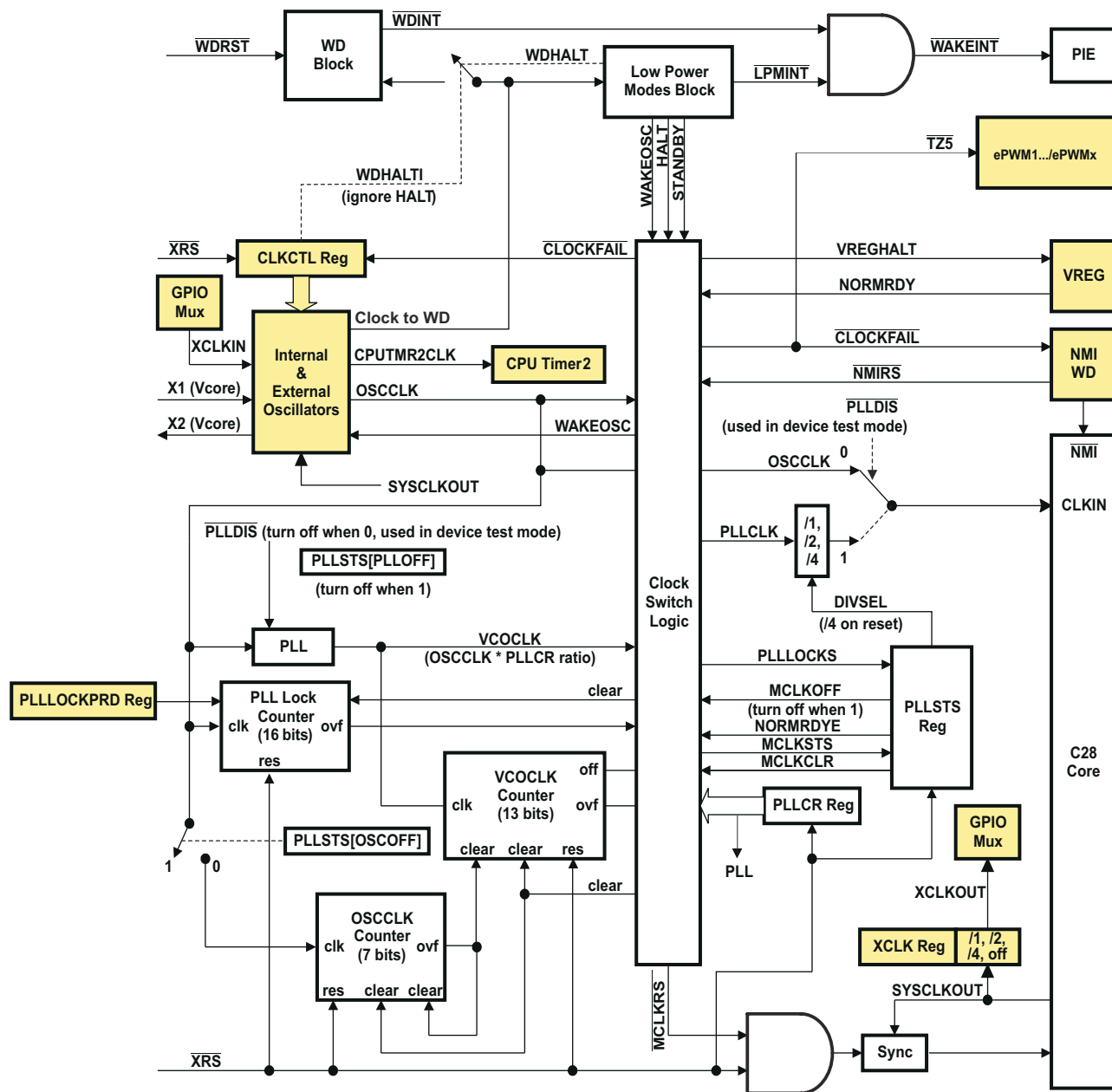
#### 1.3.2.5 Input Clock Fail Detection

It is possible for the clock source of the device to fail. When the PLL is not disabled, the main oscillator fail logic allows the device to detect this condition and handle it as described in this section.

Two counters are used to monitor the presence of the OSCCLK signal as shown in [Figure 1-34](#). The first counter is incremented by the OSCCLK signal itself. When the PLL is not turned off, the second counter is incremented by the VCOCLK coming out of the PLL block. These counters are configured such that when the 7-bit OSCCLK counter overflows, it clears the 13-bit VCOCLK counter. In normal operating mode, as long as OSCCLK is present, the VCOCLK counter will never overflow.

If the OSCCLK input signal is missing, then the PLL will output a default limp mode frequency and the VCOCLK counter will continue to increment. Since the OSCCLK signal is missing, the OSCCLK counter will not increment, and therefore, the VCOCLK counter is not periodically cleared. Eventually, the VCOCLK counter overflows. This signals a missing clock condition to the missing-clock-detection logic. What happens next is based on which clock source has been chosen for the PLL and the value of NMIRESETSEL.





### Figure 1-34. Clocking and Reset Logic

### Case A:

#### **INTOSC1 is used as the clock source. NMIWD is disabled (NMIRESETSEL = 0)**

Failure of INTOSC1 causes PLL to issue a limp mode clock. The system continues to function with the limp clock and so does the VCOCLK counter. Eventually, VCOCLK counter overflows and issues a `CLOCKFAIL` signal (`MCLKSTS` bit is set) and the missing clock detection logic resets the CPU, peripherals, and other device logic by way of `MCLKRS`. The exact delay (from the time the clock was stopped to the time a reset is asserted) depends on the VCOCLK counter value when the INTOSC1 clock vanished. The `MCLKSTS` bit is only affected by `XRS`, not by a missing clock reset. So, after a reset, code can examine this bit to determine if the reset was due to a missing clock and take appropriate action. Note that even though the `CLOCKFAIL` signal is generated, the `NMIWDCNTR` will not count.

### Case B:

#### **INTOSC1 is used as the clock source. NMIWD is enabled (NMIRESETSEL = 1)**

Failure of INTOSC1 causes PLL to issue a limp mode clock. The system continues to function with the limp clock and so does the VCOCLK counter. Eventually, the VCOCLK counter overflows and issues `CLOCKFAIL` (`MCLKSTS` bit is set), which asserts the NMI and starts the `NMIWDCNTR`. If `NMIWDCNTR` is allowed to reach the `NMIWDPRD` value, a reset (`MCLKRS`) is asserted. In the interim period, the application could choose to gracefully shut down the system before a reset is generated. Inside the `NMI_ISR`, the flags in `NMIFLG` register may be cleared, which prevents a reset.

In case A, reset is inevitable and cannot be delayed. In case B, the software can

- Choose to clear the flags to prevent a reset.
- Perform a graceful shutdown of the system.
- Switch to `OSCCLKSRC2`, if need be.

### Case C:

#### **OSCCLKSRC2 (INTOSC2 or X1/X2 or XCLKIN) is used as the clock source. NMIWD is disabled (NMIRESETSEL = 0)**

When the VCOCLK counter overflows (due to loss of `OSCCLKSRC2`), the Missing-Clock-Detect circuit recognizes the missing clock condition. `CLOCKFAIL` will be generated (but it is of no consequence). Since `NMIRESETSEL=0`, the device will be reset. No switching of clock source happens, since the device is reset. This is similar to Case A.

### Case D:

#### **OSCCLKSRC2 (INTOSC2 or X1/X2 or XCLKIN) is used as the clock source. NMIWD is enabled (NMIRESETSEL = 1)**

When the VCOCLK counter overflows (due to loss of `OSCCLKSRC2`), the Missing-Clock-Detect circuit recognizes the missing clock condition. `CLOCKFAIL` is generated and `OSCCLK` is switched to INTOSC1. For this reason, INTOSC1 should not be disabled in user code. The `MCLKSTS` bit is set, but cleared automatically after the clock switch. `PLLCR` is zeroed. The user must reconfigure `PLLCR`. Since `NMIRESETSEL=1`, NMI interrupt will be triggered and PLL could be reconfigured there. Inside the `NMI_ISR`, the flags in the `NMIFLG` register may be cleared, which prevents a reset. If INTOSC1 also fails, this becomes similar to Case B. The advantage of using `OSCCLKSRC2` as the source for the PLL is that the clock source is automatically switched to INTOSC1 upon loss of `OSCCLKSRC2`.

### 1.3.2.6 Missing Clock Reset and Missing Clock Status

The  $\overline{\text{MCLKRS}}$  is an internal reset only. The external  $\overline{\text{XRS}}$  pin of the device is not pulled low by  $\overline{\text{MCLKRS}}$ , and the PLLCR and PLLSTS registers are not reset. In addition to resetting the device, the missing clock detect logic sets the PLLSTS[MCLKSTS] register bit. When the MCLKSTS bit is 1, this indicates that the missing oscillator detect logic has reset the part and that the CPU is now running at the limp mode frequency.

Software should check the PLLSTS[MCLKSTS] bit after a reset to determine if the device was reset by  $\overline{\text{MCLKRS}}$  due to a missing clock condition. If MCLKSTS is set, then the firmware should take the action appropriate for the system such as a system shutdown. The missing clock status can be cleared by writing a 1 to the PLLSTS[MCLKCLR] bit. This will reset the missing clock detection circuits and counters. If OSCCLK is still missing after writing to the MCLKCLR bit, then the VCOCLK counter again overflows and the process will repeat.

#### Note

Applications in which the correct CPU operating frequency is absolutely critical should implement a mechanism by which the DSP will be held in reset should the input clocks ever fail. For example, an R-C circuit may be used to trigger the  $\overline{\text{XRS}}$  pin of the DSP should the capacitor ever get fully charged. An I/O pin may be used to discharge the capacitor on a periodic basis to prevent it from getting fully charged. Such a circuit would also help in detecting failure of the flash memory.

The following precautions and limitations should be kept in mind:

- **Use the proper procedure when changing the PLL Control Register.** Always follow the procedure outlined in [Figure 1-24](#) when modifying the PLLCR register.
- **Do not write to the PLLCR register when the device is operating in limp mode.** When writing to the PLLCR register, the device switches to the CPU's CLKIN input to OSCCLK/2. When operating after limp mode has been detected, OSCCLK may not be present and the clocks to the system will stop. Always check that the PLLSTS[MCLKSTS] bit = 0 before writing to the PLLCR register as described in [Figure 1-24](#).
- **Do not enter HALT low power mode when the device is operating in limp mode.** If you try to enter HALT mode when the device is already operating in limp mode then the device may not properly enter HALT. The device may instead enter STANDBY mode or may hang and you may not be able to exit HALT mode. For this reason, always check that the PLLSTS[MCLKSTS] bit = 0 before entering HALT mode.

The following list describes the behavior of the missing clock detect logic in various operating modes:

- **PLL by-pass mode**

When the PLL control register is set to 0x0000, the PLL is bypassed. Depending on the state of the PLLSTS[DIVSEL] bit, OSCCLK, OSCCLK/2, or OSCCLK/4 is connected directly to the CPU's input clock, CLKIN. If the OSCCLK is detected as missing, the device will automatically switch to the PLL's limp mode clock. Further behavior is determined by the clock source used for OSCCLK and the value of NMIRESETSEL bit as explained before.

- **STANDBY low power mode**

In this mode, the CLKIN to the CPU is stopped. If a missing input clock is detected, the missing clock status bit will be set and the device will generate a missing clock reset. If the PLL is in by-pass mode when this occurs, then one-half of the PLL limp frequency will automatically be routed to the CPU. The device will now run at the PLL limp mode frequency or at one-half or one-fourth of the PLL limp mode frequency, depending on the state of the PLLSTS[DIVSEL] bit.

- **HALT low power mode**

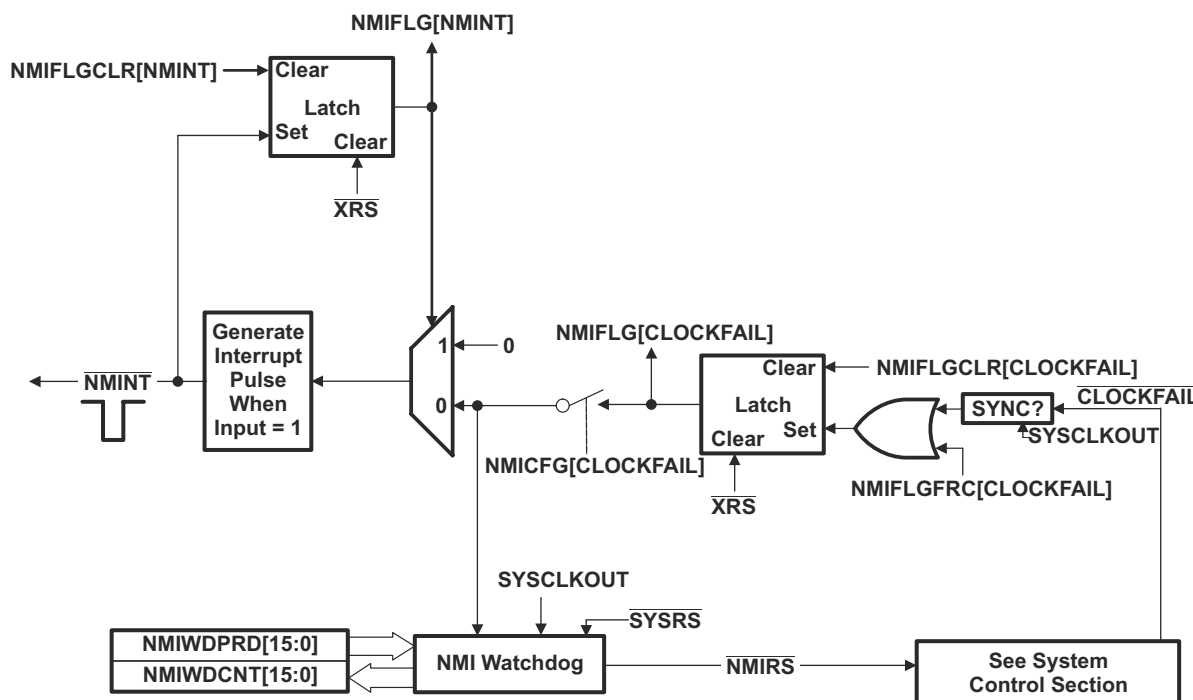
In HALT low power mode, all of the clocks to the device are turned off. When the device comes out of HALT mode, the oscillator and PLL will power up. The counters that are used to detect a missing input clock (VCOCLK and OSCCLK) will be enabled only after this power-up has completed. If VCOCLK counter overflows, the missing clock detect status bit will be set and the device will generate a missing clock reset. If the PLL is in by-pass mode when the overflow occurs, then one-half of the PLL limp frequency will automatically be routed to the CPU. The device will now run at the PLL limp mode frequency or at one-half or one-fourth of the PLL limp mode frequency depending on the state of the PLLSTS[DIVSEL] bit.

### 1.3.2.7 NMI Interrupt and Watchdog

The NMI watchdog (NMIWD) is used to detect and aid in recovery from a clock failure condition. The NMI interrupt enables the monitoring of a clock failure. In 280x/2833x/2823x devices, when the VCOCLK counter overflows (due to loss of input clock), a missing clock condition is detected and a missing-clock-reset ( $\overline{\text{MCLKRS}}$ ) is generated immediately. In this device, a  $\overline{\text{CLOCKFAIL}}$  signal is generated first, which is then applied to the NMI watchdog circuit and a reset can be generated after a preprogrammed delay. Alternatively, an interrupt can be asserted. This feature is not enabled upon power-up, however. That is, when this device first powers up, the  $\overline{\text{MCLKRS}}$  signal is generated immediately upon a clock failure like on 280x/2833x/2823x devices. The user must enable the generation of the NMI signal with the CLKCTL[NMIRESETSEL] bit. Note that the NMI watchdog is different from the watchdog described in [Section 1.3.4](#).

When the OSCCLK goes missing, the  $\overline{\text{CLOCKFAIL}}$  signal triggers the NMI and gets the NMIWD counter running. In the NMI ISR, the application is expected to take corrective action (such as gracefully shut down the system before a reset is generated or clear the  $\overline{\text{CLOCKFAIL}}$  and NMIINT flags and switch to an alternate clock source, if applicable). If this is not done, the NMIWDCTR overflows and generates an NMI reset (NMIRS) after a preprogrammed number of SYSCLKOUT cycles.  $\overline{\text{NMIRS}}$  is fed to  $\overline{\text{MCLKRS}}$  to generate a system reset back into the core. Note that NMI reset is internal to the device and will not be reflected on the  $\overline{\text{XRS}}$  pin.

The  $\overline{\text{CLOCKFAIL}}$  signal could also be used to activate the  $\overline{\text{TZ5}}$  signal to drive the PWM pins into a high impedance state. This allows the PWM outputs to be tripped in case of clock failure. [Figure 1-35](#) shows the  $\overline{\text{CLOCKFAIL}}$  interrupt mechanism. Likewise,  $\overline{\text{TZ6}}$  is connected to EMUSTOP output from the CPU. This allows the user to configure trip action during a CPU halt, such as during emulation or debug sessions.



- A. The NMI watchdog module is clocked by SYSCLKOUT. Due to the limp mode function of the PLL, SYSCLKOUT is present even if the source clock for OSCCLK fails.

**Figure 1-35. Clock Fail Interrupt**

### 1.3.2.7.1 NMI Interrupt Registers

The NMI Interrupt support registers are listed in [Table 1-32](#).

**Table 1-32. NMI Interrupt Registers**

| Name      | Address Range | Size (x16) | EALLOW | Description                   | Section                             |
|-----------|---------------|------------|--------|-------------------------------|-------------------------------------|
| NMICFG    | 0x7060        | 1          | yes    | NMI Configuration Register    | <a href="#">Section 1.3.2.7.1.1</a> |
| NMIFLG    | 0x7061        | 1          | yes    | NMI Flag Register             | <a href="#">Section 1.3.2.7.1.2</a> |
| NMIFLGCLR | 0x7062        | 1          | yes    | NMI Flag Clear Register       | <a href="#">Section 1.3.2.7.1.3</a> |
| NMIFLGFRC | 0x7063        | 1          | yes    | NMI Flag Force Register       | <a href="#">Section 1.3.2.7.1.4</a> |
| NMIWDCNT  | 0x7064        | 1          | -      | NMI Watchdog Counter Register | <a href="#">Section 1.3.2.7.1.5</a> |
| NMIWDPRD  | 0x7065        | 1          | yes    | NMI Watchdog Period Register  | <a href="#">Section 1.3.2.7.1.6</a> |

#### 1.3.2.7.1.1 NMI Configuration (NMICFG) Register

**Figure 1-36. NMI Configuration (NMICFG) Register**

|          |  |   |           |          |
|----------|--|---|-----------|----------|
| 15       |  | 2 | 1         | 0        |
| Reserved |  |   | CLOCKFAIL | Reserved |
| R-0      |  |   | R/W-0     | R-0      |

LEGEND: R = Read only; W = Write only; -n = value after reset

**Table 1-33. NMI Configuration (NMICFG) Register Bit Definitions (EALLOW)**

| Bits | Name      | Type   | Description                                                                                                                                                                                                                                                                                                                                                                |
|------|-----------|--------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-2 | Reserved  |        | Any writes to these bits must always have a value of 0.                                                                                                                                                                                                                                                                                                                    |
| 1    | CLOCKFAIL | 0<br>1 | CLOCKFAIL-interrupt Enable Bit: This bit, when set to 1 enables the CLOCKFAIL condition to generate an NMI interrupt. Once enabled, the flag cannot be cleared by the user. Only a device reset clears the flag. Writes of 0 are ignored. Reading the bit will indicate if the flag is enabled or disabled:<br>CLOCKFAIL Interrupt Disabled<br>CLOCKFAIL Interrupt Enabled |
| 0    | Reserved  |        | Any writes to these bits must always have a value of 0.                                                                                                                                                                                                                                                                                                                    |

### 1.3.2.7.1.2 NMI Flag (NMIFLG) Register

**Figure 1-37. NMI Flag (NMIFLG) Register**

|          |  |   |           |        |
|----------|--|---|-----------|--------|
| 15       |  | 2 | 1         | 0      |
| Reserved |  |   | CLOCKFAIL | NMIINT |
| R-0      |  |   | R-0       | R-0    |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 1-34. NMI Flag (NMIFLG) Register Bit Definitions (EALLOW Protected)**

| Bits | Name      | Type   | Description <sup>(1)</sup>                                                                                                                                                                                                                                                                                                                                   |
|------|-----------|--------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-2 | Reserved  |        | Any writes to these bits must always have a value of 0.                                                                                                                                                                                                                                                                                                      |
| 1    | CLOCKFAIL | 0<br>1 | CLOCKFAIL Interrupt Flag: This bit indicates if the CLOCKFAIL condition is latched. This bit can be cleared only by writing to the respective bit in the NMIFLGCLR register or by a device reset ( $\overline{\text{XRS}}$ ):<br>0 No CLOCKFAIL condition pending<br>1 CLOCKFAIL condition detected. This bit will be set in the event of any clock failure. |
| 0    | NMIINT    | 0<br>1 | NMI Interrupt Flag: This bit indicates if an NMI interrupt was generated. This bit can only be cleared by writing to the respective bit in the NMIFLGCLR register or by an $\overline{\text{XRS}}$ reset:<br>0 No NMI interrupt generated<br>1 NMI interrupt generated<br>No further NMI interrupts are generated until you clear this flag.                 |

(1) The NMIFLG register is only reset by the  $\overline{\text{XRS}}$  signal, not  $\overline{\text{SYSRS}}$

### 1.3.2.7.1.3 NMI Flag Clear (NMIFLGCLR) Register

**Figure 1-38. NMI Flag Clear (NMIFLGCLR) Register**

|          |  |   |           |        |
|----------|--|---|-----------|--------|
| 15       |  | 2 | 1         | 0      |
| Reserved |  |   | CLOCKFAIL | NMIINT |
| R-0      |  |   | W-0       | W-0    |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

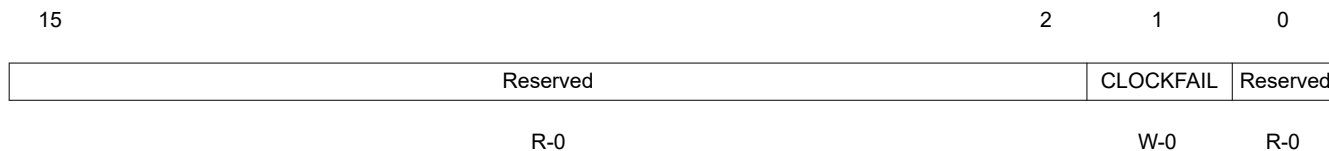
**Table 1-35. NMI Flag Clear (NMIFLGCLR) Register Bit Definitions (EALLOW Protected)**

| Bits | Name                     | Type   | Description                                                                                                                                                              |
|------|--------------------------|--------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-2 | Reserved                 |        | Any writes to these bits must always have a value of 0.                                                                                                                  |
| 1    | CLOCKFAIL <sup>(1)</sup> | 0<br>1 | CLOCKFAIL Flag Clear<br>0 Writes of 0 are ignored. Always reads back 0.<br>1 Writing a 1 to the respective bit clears the corresponding flag bit in the NMIFLG register. |
| 0    | NMIINT <sup>(1)</sup>    | 0<br>1 | NMI Flag Clear<br>0 Writes of 0 are ignored. Always reads back 0.<br>1 Writing a 1 to the respective bit clears the corresponding flag bit in the NMIFLG register.       |

(1) If hardware is trying to set a bit to 1 while software is trying to clear a bit to 0 on the same cycle, hardware has priority. You should clear the pending CLOCKFAIL flag first and then clear the NMIINT flag.

#### 1.3.2.7.1.4 NMI Flag Force (NMIFLGFR) Register

**Figure 1-39. NMI Flag Force (NMIFLGFR) Register**



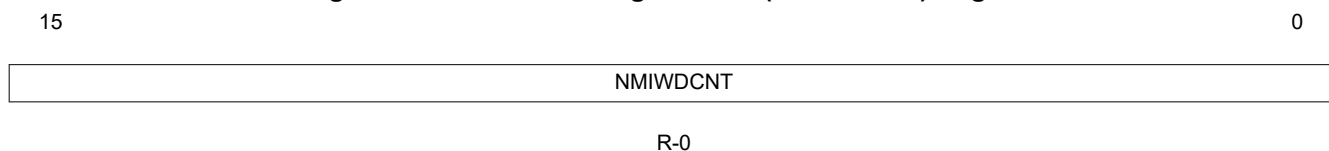
LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 1-36. NMI Flag Force (NMIFLGFR) Register Bit Definitions (EALLOW Protected)**

| Bits | Name      | Value | Description                                             |
|------|-----------|-------|---------------------------------------------------------|
| 15-2 | Reserved  |       | Any writes to these bits must always have a value of 0. |
| 1    | CLOCKFAIL | 0     | Writes of 0 are ignored. Always reads back 0.           |
|      |           | 1     | Writing a 1 sets the CLOCKFAIL flag.                    |
| 0    | Reserved  |       | Any writes to these bits must always have a value of 0. |

#### 1.3.2.7.1.5 NMI Watchdog Counter (NMIWDCNT) Register

**Figure 1-40. NMI Watchdog Counter (NMIWDCNT) Register**



LEGEND: R = Read only; W = Write only; -n = value after reset

**Table 1-37. NMI Watchdog Counter (NMIWDCNT) Register Bit Definitions**

| Bits | Name     | Type | Description                                                                                                                                                                                                                                                                                                                                                         |
|------|----------|------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-0 | NMIWDCNT |      | NMI Watchdog Counter: This 16-bit incremental counter will start incrementing whenever any one of the enabled FAIL flags are set. If the counter reaches the period value, an NMIRS signal is fired, which then resets the system. The counter resets to zero when it reaches the period value and then restarts counting if any of the enabled FAIL flags are set. |
|      |          | 0    | If no enabled FAIL flag is set, then the counter resets to zero and remains at zero until an enabled FAIL flag is set.                                                                                                                                                                                                                                              |
|      |          | 1    | Normally, the software would respond to the NMI interrupt generated and clear the offending FLAGS before the NMI watchdog triggers a reset. In some situations, the software may decide to allow the watchdog to reset the device anyway.                                                                                                                           |
|      |          |      | The counter is clocked at the SYSCLKOUT rate. Reset value of this counter is zero.                                                                                                                                                                                                                                                                                  |



### 1.3.2.7.1.6 NMI Watchdog Period (NMIWDPRD) Register

**Figure 1-41. NMI Watchdog Period (NMIWDPRD) Register**

15

0

|          |
|----------|
| NMIWDPRD |
|----------|

R/W-0xFFFF

LEGEND: R = Read only; W = Write only; -n = value after reset

**Table 1-38. NMI Watchdog Period (NMIWDPRD) Register Bit Definitions (EALLOW Protected)**

| Bits | Name     | Type | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|------|----------|------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-0 | NMIWDPRD | R/W  | <p>NMI Watchdog Period: This 16-bit value contains the period value at which a reset is generated when the watchdog counter matches. At reset this value is set at the maximum. The software can decrease the period value at initialization time.</p> <p>Writing a PERIOD value that is equal to the current counter value automatically forces an NMIRS and resets the watchdog counter. If a PERIOD value is written that is smaller than the current counter value, the counter will continue counting until it overflows and starts counting up again from 0. After the overflow, once the COUNTER value equals the new PERIOD value, an NMIRS is forced which resets the watchdog counter.</p> |

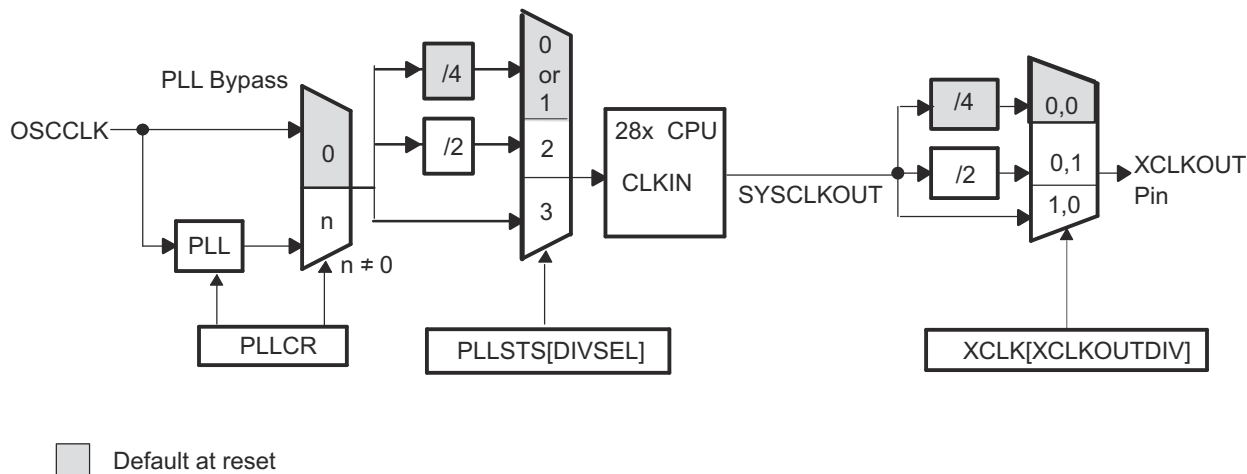
### 1.3.2.7.2 NMI Watchdog Emulation Considerations

The NMI watchdog module does not operate when trying to debug the target device (emulation suspend such as breakpoint). The NMI watchdog module behaves as follows under various debug conditions:

|                                    |                                                                                                                                                       |
|------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>CPU Suspended:</i>              | When the CPU is suspended, the NMI watchdog counter is suspended.                                                                                     |
| <i>Run-Free Mode:</i>              | When the CPU is placed in run-free mode, the NMI watchdog counter resumes operation as normal.                                                        |
| <i>Real-Time Single-Step Mode:</i> | When the CPU is in real-time single-step mode, the NMI watchdog counter is suspended. The counter remains suspended even within real-time interrupts. |
| <i>Real-Time Run-Free Mode:</i>    | When the CPU is in real-time run-free mode, the NMI watchdog counter operates as normal.                                                              |

### 1.3.2.8 XCLKOUT Generation

The XCLKOUT signal is directly derived from the system clock SYSCLKOUT as shown in [Figure 1-42](#). XCLKOUT can be either equal to, one-half, or one-fourth of SYSCLKOUT. By default, at power-up, XCLKOUT = SYSCLKOUT/4 or XCLKOUT = OSCCLK/16.



**Figure 1-42. XCLKOUT Generation**

If XCLKOUT is not being used, it can be turned off by setting the XCLKOUTDIV bit to 3 in the XCLK register.

### 1.3.2.9 External Reference Oscillator Clock Option

TI recommends that customers have the resonator/crystal vendor characterize the operation of their device with the DSP chip. The resonator/crystal vendor has the equipment and expertise to tune the tank circuit. The vendor can also advise the customer regarding the proper tank component values to provide proper start-up and stability over the entire operating range.

### 1.3.3 Low-Power Modes Block

[Table 1-39](#) summarizes the various modes.

The various low-power modes operate as shown in [Table 1-40](#).

Refer to the device data sheet for exact timing for entering and exiting the low power modes.

**Table 1-39. Low-Power Mode Summary**

| Mode    | LPMCR0[1:0] | OSCCLK                                                             | CLKIN | SYSCLKOUT | Exit <sup>(1)</sup>                                                                         |
|---------|-------------|--------------------------------------------------------------------|-------|-----------|---------------------------------------------------------------------------------------------|
| IDLE    | 00          | On                                                                 | On    | On        | $\overline{XRS}$ ,<br>Watchdog interrupt,<br>Any enabled interrupt                          |
| STANDBY | 01          | On<br>(watchdog still running)                                     | Off   | Off       | $\overline{XRS}$ ,<br>Watchdog interrupt,<br>GPIO Port A signal,<br>Debugger <sup>(2)</sup> |
| HALT    | 1X          | Off<br>(oscillator and PLL turned off,<br>watchdog not functional) | Off   | Off       | $\overline{XRS}$ ,<br>GPIO Port A Signal,<br>Debugger <sup>(2)</sup>                        |

(1) The Exit column lists which signals or under what conditions the low power mode is exited. This signal must be kept low long enough for an interrupt to be recognized by the device. Otherwise the IDLE mode is not exited and the device goes back into the indicated low power mode.

(2) On the 28x, the JTAG port can still function even if the clock to the CPU (CLKIN) is turned off.

**Table 1-40. Low Power Modes**

| Mode          | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|---------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| IDLE Mode:    | This mode is exited by any enabled interrupt. The LPM block itself performs no tasks during this mode.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| STANDBY Mode: | <p>If the LPM bits in the LPMCR0 register are set to 01, the device enters STANDBY mode when the IDLE instruction is executed. In STANDBY mode the clock input to the CPU (CLKIN) is disabled, which disables all clocks derived from SYSCLKOUT. The oscillator and PLL and watchdog will still function. Before entering the STANDBY mode, you should perform the following tasks:</p> <ul style="list-style-type: none"> <li>• Enable the WAKEINT interrupt in the PIE module. This interrupt is connected to both the watchdog and the low power mode module interrupt.</li> <li>• If desired, specify one of the GPIO port A signals to wake the device in the GPIO_LPMSEL register. The GPIO_LPMSEL register is part of the GPIO module. In addition to the selected GPIO signal, the <math>\overline{XRS}</math> input and the watchdog interrupt, if enabled in the LPMCR0 register, can wake the device from the STANDBY mode.</li> <li>• Select the input qualification in the LPMCR0 register for the signal that will wake the device.</li> </ul> <p>When the selected external signal goes low, it must remain low a number of OSCCLK cycles as specified by the qualification period in the LPMCR0 register. If the signal should be sampled high during this time, the qualification will restart. At the end of the qualification period, the PLL enables the CLKIN to the CPU and the WAKEINT interrupt is latched in the PIE block. The CPU then responds to the WAKEINT interrupt if it is enabled.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| HALT Mode:    | <p>If the LPM bits in the LPMCR0 register are set to 1x, the device enters the HALT mode when the IDLE instruction is executed. In HALT mode all of the device clocks, including the PLL and oscillator, are shut down. Before entering the HALT mode, you should perform the following tasks:</p> <ul style="list-style-type: none"> <li>• Enable the WAKEINT interrupt in the PIE module (PIEIER1.8 = 1). This interrupt is connected to both the watchdog and the Low-Power-Mode module interrupt.</li> <li>• Specify one of the GPIO port A signals to wake the device in the GPIO_LPMSEL register. The GPIO_LPMSEL register is part of the GPIO module. In addition to the selected GPIO signal, the <math>\overline{XRS}</math> input can also wake the device from the HALT mode.</li> <li>• Disable all interrupts with the possible exception of the HALT mode wakeup interrupt. The interrupts can be re-enabled after the device is brought out of HALT mode.</li> </ul> <ol style="list-style-type: none"> <li>1. For device to exit HALT mode properly, the following conditions must be met: <ul style="list-style-type: none"> <li>Bit 7 (INT1.8) of PIEIER1 register should be 1.</li> <li>Bit 0 (INT1) of IER register must be 1.</li> </ul> </li> <li>2. If the above conditions are met, <ol style="list-style-type: none"> <li>a. WAKE_INT ISR will be executed first, followed by the instructions after IDLE, if INTM = 0.</li> <li>b. WAKE_INT ISR will not be executed and instructions after IDLE will be executed, if INTM = 1.</li> </ol> </li> </ol> <p><b>Do not enter HALT low power mode when the device is operating in limp mode (PLLSTS[MCLKSTS] = 1).</b><br/>If you try to enter HALT mode when the device is already operating in limp mode then the device may not properly enter HALT. The device may instead enter STANDBY mode or may hang and you may not be able to exit HALT mode. For this reason, always check that the PLLSTS[MCLKSTS] bit = 0 before entering HALT mode.</p> <p>When the selected external signal goes low, it is fed asynchronously to the LPM block. The oscillator is turned on and begins to power up. You must hold the signal low long enough for the oscillator to complete power up. When the signal is held low for enough time and driven high, this will asynchronously release the PLL and it will begin to lock. Once the PLL has locked, it feeds the CLKIN to the CPU at which time the CPU responds to the WAKEINT interrupt if enabled.</p> |

The low-power modes are controlled by the LPMCR0 register ([Figure 1-43](#)).

**Figure 1-43. Low-Power Mode Control 0 Register (LPMCR0)**

|        |          |           |       |   |   |   |
|--------|----------|-----------|-------|---|---|---|
| 15     | 14       | 8         | 7     | 2 | 1 | 0 |
| WDINTE | Reserved | QUALSTDBY | LPM   |   |   |   |
| R/W-0  | R-0      | R/W-0x3F  | R/W-0 |   |   |   |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 1-41. Low-Power Mode Control 0 Register (LPMCR0) Field Descriptions**

| Bits | Field              | Value                             | Description <sup>(2)</sup>                                                                                                                                                                                                                                                                                                       |
|------|--------------------|-----------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15   | WDINTE             | 0<br>1                            | Watchdog interrupt enable<br>The watchdog interrupt is not allowed to wake the device from STANDBY. (default)<br>The watchdog is allowed to wake the device from STANDBY. The watchdog interrupt must also be enabled in the SCSR register.                                                                                      |
| 14-8 | Reserved           |                                   | Any writes to these bits must always have a value of 0.                                                                                                                                                                                                                                                                          |
| 7-2  | QUALSTDBY          | 000000<br>000001<br>...<br>111111 | Select number of OSCCLK clock cycles to qualify the selected GPIO inputs that wake the device from STANDBY mode. This qualification is only used when in STANDBY mode. The GPIO signals that can wake the device from STANDBY are specified in the GPIOLPMSEL register.<br>2 OSCCLKs<br>3 OSCCLKs<br>...<br>65 OSCCLKs (default) |
| 1-0  | LPM <sup>(1)</sup> | 00<br>01<br>10<br>11              | These bits set the low power mode for the device.<br>Set the low power mode to IDLE (default)<br>Set the low power mode to STANDBY<br>Set the low power mode to HALT<br>Set the low power mode to HALT                                                                                                                           |

(1) The low power mode bits (LPM) only take effect when the IDLE instruction is executed. Therefore, you must set the LPM bits to the appropriate mode before executing the IDLE instruction.

(2) This register is EALLOW protected. See [Section 1.5.2](#) for more information.

### 1.3.3.1 Options for Automatic Wakeup in Low-power Modes

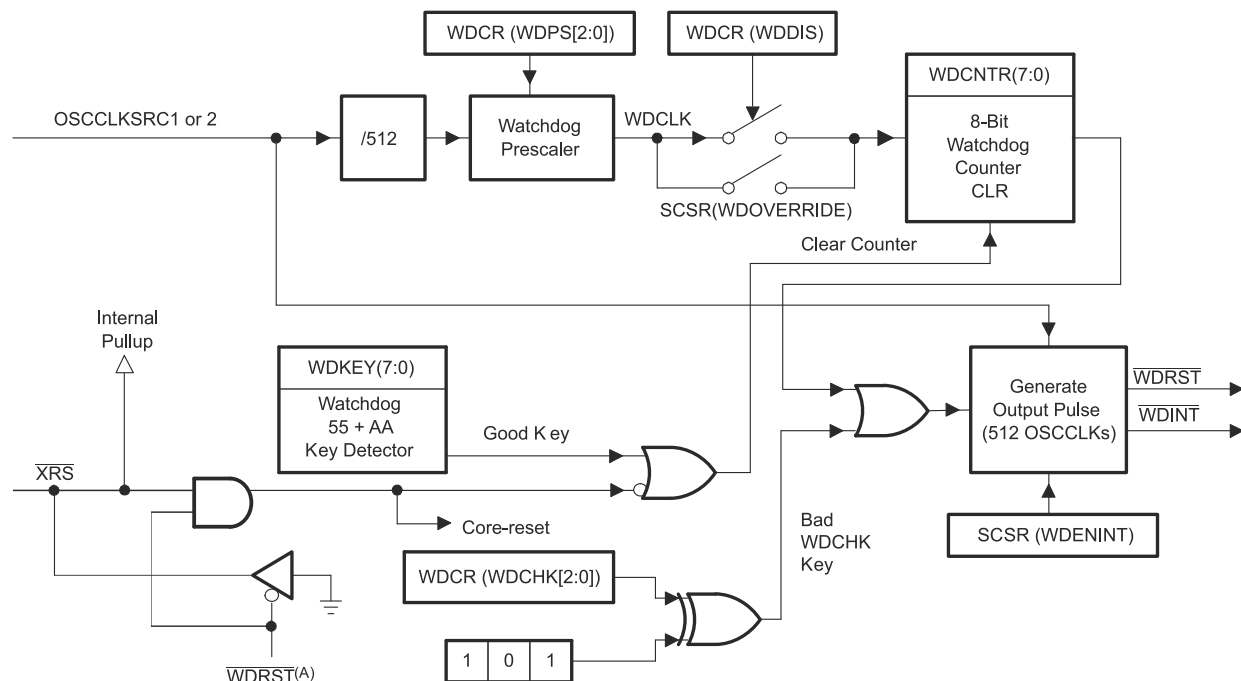
The device provides two options to automatically wake up from HALT and STANDBY modes, without the need for an external stimulus:

**Wakeup from HALT:** Set WDHALTI bit in CLKCTL register to 1. When the device wakes up from HALT, it will be through a CPU-watchdog reset. The WDFLAG bit in the WDCR register can be used to differentiate between a CPU-watchdog-reset and a device reset.

**Wakeup from STANDBY:** Set WDINTE bit in LPMCR0 register to 1. When the device wakes up from STANDBY, it will be through the WAKEINT interrupt (Interrupt 1.8 in the PIE).

### 1.3.4 CPU Watchdog Block

The watchdog module generates an output pulse, 512 oscillator-clocks (OSCCLK) wide whenever the 8-bit watchdog up counter has reached its maximum value. To prevent this, the user can either disable the counter or the software must periodically write a 0x55 + 0xAA sequence into the watchdog key register which resets the watchdog counter. Figure 1-44 shows the various functional blocks within the watchdog module.



- A. The  $\overline{\text{WDRST}}$  and  $\overline{\text{XRS}}$  signals are driven low for 512 OSCCLK cycles when a watchdog reset occurs. Likewise, if the watchdog interrupt is enabled, the  $\overline{\text{WDINT}}$  signal will be driven low for 512 OSCCLK cycles when an interrupt occurs.

**Figure 1-44. CPU Watchdog Module**

### 1.3.4.1 Servicing the Watchdog Timer

The WDCNTR is reset when the proper sequence is written to the WDKEY register before the 8-bit watchdog counter (WDCNTR) overflows. The WDCNTR is reset-enabled when a value of 0x55 is written to the WDKEY. When the next value written to the WDKEY register is 0xAA then the WDCNTR is reset. Any value written to the WDKEY other than 0x55 or 0xAA causes no action. Any sequence of 0x55 and 0xAA values can be written to the WDKEY without causing a system reset; only a write of 0x55 followed by a write of 0xAA to the WDKEY resets the WDCNTR.

**Table 1-42. Example Watchdog Key Sequences**

| Step | Value Written to WDKEY | Result                                                                                            |
|------|------------------------|---------------------------------------------------------------------------------------------------|
| 1    | 0xAA                   | No action                                                                                         |
| 2    | 0xAA                   | No action                                                                                         |
| 3    | 0x55                   | WDCNTR is enabled to be reset if next value is 0xAA.                                              |
| 4    | 0x55                   | WDCNTR is enabled to be reset if next value is 0xAA.                                              |
| 5    | 0x55                   | WDCNTR is enabled to be reset if next value is 0xAA.                                              |
| 6    | 0xAA                   | WDCNTR is reset.                                                                                  |
| 7    | 0xAA                   | No action                                                                                         |
| 8    | 0x55                   | WDCNTR is enabled to be reset if next value is 0xAA.                                              |
| 9    | 0xAA                   | WDCNTR is reset.                                                                                  |
| 10   | 0x55                   | WDCNTR is enabled to be reset if next value is 0xAA.                                              |
| 11   | 0x32                   | Improper value written to WDKEY.<br>No action, WDCNTR no longer enabled to be reset by next 0xAA. |
| 12   | 0xAA                   | No action due to previous invalid value.                                                          |
| 13   | 0x55                   | WDCNTR is enabled to be reset if next value is 0xAA.                                              |
| 14   | 0xAA                   | WDCNTR is reset.                                                                                  |

Step 3 in [Table 1-42](#) is the first action that enables the WDCNTR to be reset. The WDCNTR is not actually reset until step 6. Step 8 again re-enables the WDCNTR to be reset and step 9 resets the WDCNTR. Step 10 again re-enables the WDCNTR to be reset. Writing the wrong key value to the WDKEY in step 11 causes no action, however the WDCNTR is no longer enabled to be reset and the 0xAA in step 12 now has no effect.

If the watchdog is configured to reset the device, then a WDCR overflow or writing the incorrect value to the WDCR[WDCHK] bits will reset the device and set the watchdog flag (WDFLAG) in the WDCR register. After a reset, the program can read the state of this flag to determine the source of the reset. After reset, the WDFLAG should be cleared by software to allow the source of subsequent resets to be determined. Watchdog resets are not prevented when the flag is set.

### 1.3.4.2 Watchdog Reset or Watchdog Interrupt Mode

The watchdog can be configured in the SCSR register to either reset the device ( $\overline{\text{WDRST}}$ ) or assert an interrupt ( $\overline{\text{WDINT}}$ ) if the watchdog counter reaches its maximum value. The behavior of each condition is described below:

- **Reset mode:**

If the watchdog is configured to reset the device, then the  $\overline{\text{WDRST}}$  signal will pull the device reset ( $\overline{\text{XRS}}$ ) pin low for 512 OSCCLK cycles when the watchdog counter reaches its maximum value.

- **Interrupt mode:**

If the watchdog is configured to assert an interrupt, then the  $\overline{\text{WDINT}}$  signal will be driven low for 512 OSCCLK cycles, causing the WAKEINT interrupt in the PIE to be taken if it is enabled in the PIE module. The watchdog interrupt is edge triggered on the falling edge of  $\overline{\text{WDINT}}$ . Thus, if the WAKEINT interrupt is re-enabled before  $\overline{\text{WDINT}}$  goes inactive, you will not immediately get another interrupt. The next WAKEINT interrupt will occur at the next watchdog timeout. If the watchdog is disabled before  $\overline{\text{WDINT}}$  goes inactive, the 512-cycle count will halt and  $\overline{\text{WDINT}}$  will remain active. The count will resume when the watchdog is enabled again.

If the watchdog is reconfigured from interrupt mode to reset mode while  $\overline{\text{WDINT}}$  is still active low, then the device will reset immediately. The WDINTS bit in the SCSR register can be read to determine the current state of the  $\overline{\text{WDINT}}$  signal before reconfiguring the watchdog to reset mode.

### 1.3.4.3 Watchdog Operation in Low-power Modes

In STANDBY mode, all of the clocks to the peripherals are turned off on the device. The only peripheral that remains functional is the watchdog since the watchdog module runs off the oscillator clock (OSCCLK). The  $\overline{\text{WDINT}}$  signal is fed to the Low Power Modes (LPM) block so that it can be used to wake the device from STANDBY low power mode (if enabled). See the Low Power Modes Block section of the device datasheet for details.

In IDLE mode, the watchdog interrupt ( $\overline{\text{WDINT}}$ ) signal can generate an interrupt to the CPU to take the CPU out of IDLE mode. The watchdog is connected to the WAKEINT interrupt in the PIE.

#### Note

If the watchdog interrupt is used to wake-up from an IDLE or STANDBY low power mode condition, then make sure that the  $\overline{\text{WDINT}}$  signal goes back high again before attempting to go back into the IDLE or STANDBY mode. The  $\overline{\text{WDINT}}$  signal will be held low for 512 OSCCLK cycles when the watchdog interrupt is generated. You can determine the current state of  $\overline{\text{WDINT}}$  by reading the watchdog interrupt status bit (WDINTS) bit in the SCSR register. WDINTS follows the state of  $\overline{\text{WDINT}}$  by two SYSCLKOUT cycles.

By using WDHalti and INTOSC1HALTI bits, INTOSC1 and the watchdog module could be kept alive in HALT mode. The device can then wake-up from HALT mode through the watchdog, but it is through the watchdog reset, not the interrupt. When this happens, the RAM contents are not disturbed, but the peripherals will have to be re-initialized.

### 1.3.4.4 Emulation Considerations

The watchdog module behaves as follows under various debug conditions:

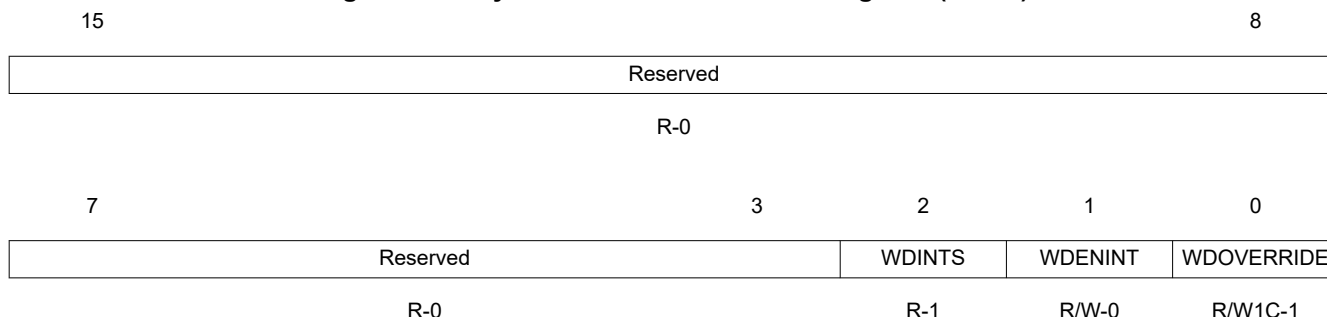
|                             |                                                                                                                                                          |
|-----------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| CPU Suspended:              | When the CPU is suspended, the watchdog clock (WDCLK) is suspended                                                                                       |
| Run-Free Mode:              | When the CPU is placed in run-free mode, then the watchdog module resumes operation as normal.                                                           |
| Real-Time Single-Step Mode: | When the CPU is in real-time single-step mode, the watchdog clock (WDCLK) is suspended. The watchdog remains suspended even within real-time interrupts. |
| Real-Time Run-Free Mode:    | When the CPU is in real-time run-free mode, the watchdog operates as normal.                                                                             |



### 1.3.4.5 Watchdog Registers

The system control and status register (SCSR) contains the watchdog override bit and the watchdog interrupt enable/disable bit. [Figure 1-45](#) describes the bit functions of the SCSR register.

**Figure 1-45. System Control and Status Register (SCSR)**



LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 1-43. System Control and Status Register (SCSR) Field Descriptions**

| Bit  | Field      | Value  | Description <sup>(1)</sup>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|------|------------|--------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-3 | Reserved   |        | Any writes to these bits must always have a value of 0.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| 2    | WDINTS     | 0<br>1 | <p>Watchdog interrupt status bit. WDINTS reflects the current state of the <math>\overline{\text{WDINT}}</math> signal from the watchdog block. WDINTS follows the state of <math>\overline{\text{WDINT}}</math> by two SYSCLKOUT cycles.</p> <p>If the watchdog interrupt is used to wake the device from IDLE or STANDBY low power mode, use this bit to make sure <math>\overline{\text{WDINT}}</math> is not active before attempting to go back into IDLE or STANDBY mode.</p> <p>0 Watchdog interrupt signal ( <math>\overline{\text{WDINT}}</math> ) is active.</p> <p>1 Watchdog interrupt signal ( <math>\overline{\text{WDINT}}</math> ) is not active.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| 1    | WDENINT    | 0<br>1 | <p>Watchdog interrupt enable.</p> <p>0 The watchdog reset ( <math>\overline{\text{WDRST}}</math> ) output signal is enabled and the watchdog interrupt ( <math>\overline{\text{WDINT}}</math> ) output signal is disabled. This is the default state on reset ( <math>\overline{\text{XRS}}</math> ). When the watchdog interrupt occurs the <math>\overline{\text{WDRST}}</math> signal will stay low for 512 OSCCLK cycles.</p> <p>If the WDENINT bit is cleared while <math>\overline{\text{WDINT}}</math> is low, a reset will immediately occur. The WDINTS bit can be read to determine the state of the <math>\overline{\text{WDINT}}</math> signal.</p> <p>1 The <math>\overline{\text{WDRST}}</math> output signal is disabled and the <math>\overline{\text{WDINT}}</math> output signal is enabled. When the watchdog interrupt occurs, the <math>\overline{\text{WDINT}}</math> signal will stay low for 512 OSCCLK cycles.</p> <p>If the watchdog interrupt is used to wake the device from IDLE or STANDBY low power mode, use the WDINTS bit to make sure <math>\overline{\text{WDINT}}</math> is not active before attempting to go back into IDLE or STANDBY mode.</p> |
| 0    | WDOVERRIDE | 0<br>1 | <p>Watchdog override - Protects the watchdog from being disabled by the watchdog disable (WDDIS) bit in the watchdog control (WDCR) register.</p> <p>0 WDDIS in WDCR has no effect and WD cannot be disabled. If this bit is cleared, it remains in this state until a reset occurs.</p> <p>1 WDDIS bit in WDCR can disable the watchdog. This bit is a clear-only bit (write 1 to clear). The reset default of this bit is a 1, and writing a 1 clears this bit (bit becomes 0) and the WD cannot be disabled. Writing a 0 has no effect.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |

(1) This register is EALLOW protected. See [Section 1.5.2](#) for more information.

**Figure 1-46. Watchdog Counter Register (WDCNTR)**

|          |   |        |   |
|----------|---|--------|---|
| 15       | 8 | 7      | 0 |
| Reserved |   | WDCNTR |   |
| R-0      |   | R-0    |   |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 1-44. Watchdog Counter Register (WDCNTR) Field Descriptions**

| Bits | Field    | Description                                                                                                                                                                                                                                                                                                                                                    |
|------|----------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-8 | Reserved | Any writes to these bits must always have a value of 0.                                                                                                                                                                                                                                                                                                        |
| 7-0  | WDCNTR   | These bits contain the current value of the WD counter. The 8-bit counter continually increments at the watchdog clock (WDCLK), rate. If the counter overflows, then the watchdog initiates a reset. If the WDKEY register is written with a valid combination, then the counter is reset to zero. The watchdog clock rate is configured in the WDCR register. |

**Figure 1-47. Watchdog Reset Key Register (WDKEY)**

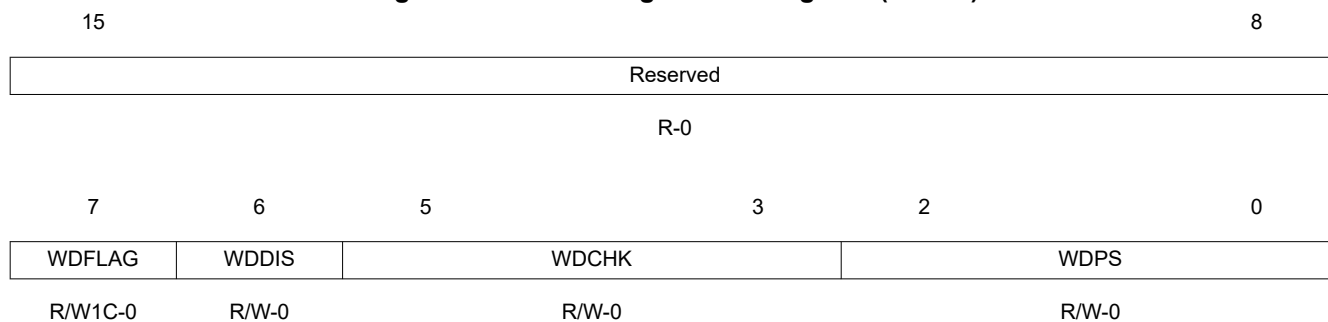
|          |   |       |   |
|----------|---|-------|---|
| 15       | 8 | 7     | 0 |
| Reserved |   | WDKEY |   |
| R-0      |   | R/W-0 |   |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 1-45. Watchdog Reset Key Register (WDKEY) Field Descriptions**

| Bits | Field    | Value                      | Description <sup>(1)</sup>                                                                                                                                                                                                                                                                                                                                                                                   |
|------|----------|----------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-8 | Reserved |                            | Any writes to these bits must always have a value of 0.                                                                                                                                                                                                                                                                                                                                                      |
| 7-0  | WDKEY    | 0x55 + 0xAA<br>Other value | Refer to <a href="#">Table 1-42</a> for examples of different WDKEY write sequences.<br>Writing 0x55 followed by 0xAA to WDKEY causes the WDCNTR bits to be cleared.<br>Writing any value other than 0x55 or 0xAA causes no action to be generated. If any value other than 0xAA is written after 0x55, then the sequence must restart with 0x55.<br>Reads from WDKEY return the value of the WDCR register. |

(1) This register is EALLOW protected. See [Section 1.5.2](#) for more information.

**Figure 1-48. Watchdog Control Register (WDCR)**


LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 1-46. Watchdog Control Register (WDCR) Field Descriptions**

| Bits | Field    | Value                                                | Description <sup>(1)</sup>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
|------|----------|------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-8 | Reserved |                                                      | Any writes to these bits must always have a value of 0.                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| 7    | WDFLAG   | 0<br>1                                               | Watchdog reset status flag bit<br>The reset was caused either by the $\overline{\text{XRS}}$ pin or because of power-up. The bit remains latched until you write a 1 to clear the condition. Writes of 0 are ignored.<br>Indicates a watchdog reset ( $\overline{\text{WDRST}}$ ) generated the reset condition. .                                                                                                                                                                                                       |
| 6    | WDDIS    | 0<br>1                                               | Watchdog disable bit. This bit can be modified only if the WDOVERRIDE bit in the SCSR register is set to 1 (default behavior upon reset).<br>Enables the watchdog module. Upon reset, the watchdog module is enabled.<br>Disables the watchdog module.                                                                                                                                                                                                                                                                   |
| 5-3  | WDCHK    | 0,0,0<br>other                                       | Watchdog check.<br>You must ALWAYS write 1,0,1 to these bits whenever a write to this register is performed unless the intent is to reset the device via software.<br>Writing any other value causes an immediate device reset or watchdog interrupt to be taken. Note that this happens even when watchdog module is disabled. Do not write to WDCHK bits when the watchdog module is disabled. These bits can be used to generate a software reset of the device. These three bits always read back as zero (0, 0, 0). |
| 2-0  | WDPS     | 000<br>001<br>010<br>011<br>100<br>101<br>110<br>111 | Watchdog pre-scale. These bits configure the watchdog counter clock (WDCLK) rate relative to (OSCCLKSRC1 or 2)/512:<br>WDCLK = (OSCCLKSRC1 or 2)/512/1 (default)<br>WDCLK = (OSCCLKSRC1 or 2)/512/1<br>WDCLK = (OSCCLKSRC1 or 2)/512/2<br>WDCLK = (OSCCLKSRC1 or 2)/512/4<br>WDCLK = (OSCCLKSRC1 or 2)/512/8<br>WDCLK = (OSCCLKSRC1 or 2)/512/16<br>WDCLK = (OSCCLKSRC1 or 2)/512/32<br>WDCLK = (OSCCLKSRC1 or 2)/512/64                                                                                                 |

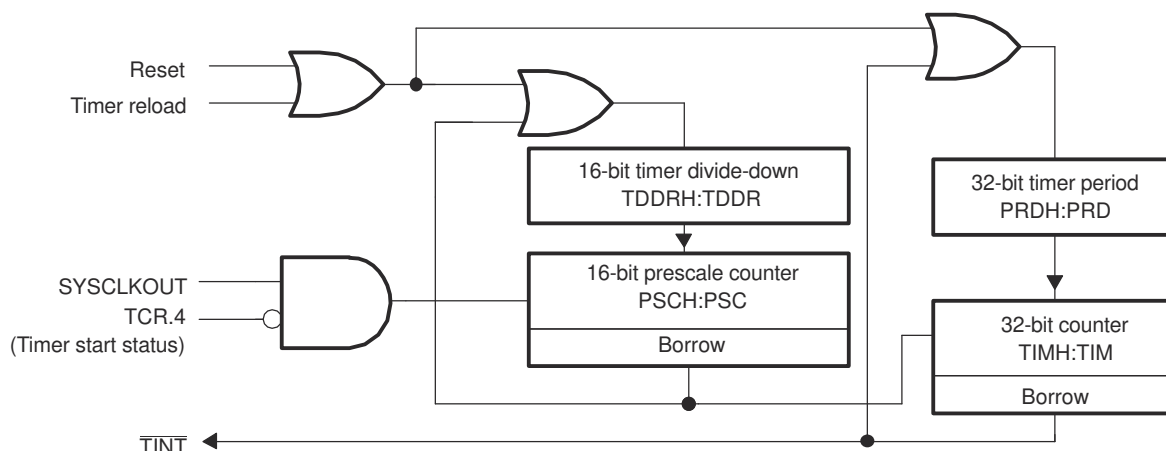
(1) This register is EALLOW protected. See [Section 1.5.2](#) for more information.

When the  $\overline{\text{XRS}}$  line is low, the WDFLAG bit is forced low. The WDFLAG bit is only set if a rising edge on  $\overline{\text{WDRST}}$  signal is detected (after synch and an 8192 SYSCLKOUT cycle delay) and the  $\overline{\text{XRS}}$  signal is high. If the  $\overline{\text{XRS}}$  signal is low when  $\overline{\text{WDRST}}$  goes high, then the WDFLAG bit remains at 0. In a typical application, the  $\overline{\text{WDRST}}$  signal connects to the  $\overline{\text{XRS}}$  input. Hence to distinguish between a watchdog reset and an external device reset, an external reset must be longer in duration than the watchdog pulse.

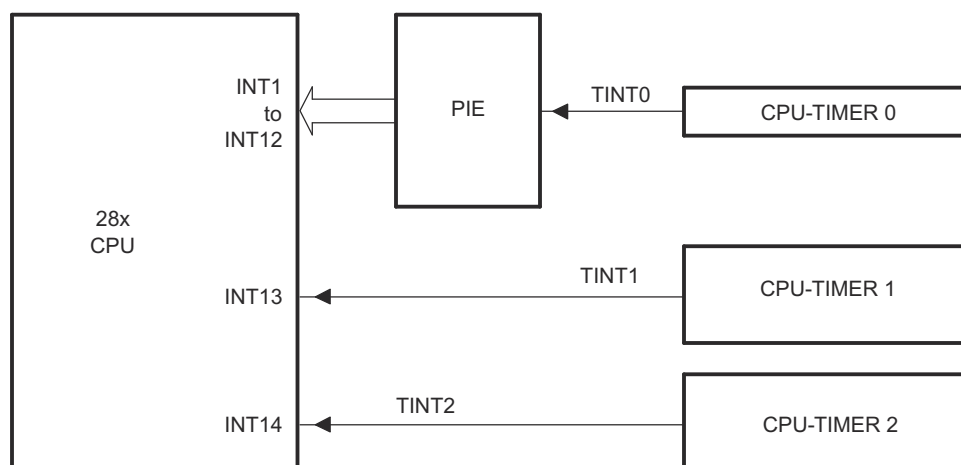
### 1.3.5 32-Bit CPU Timers 0/1/2

This section describes the three 32-bit CPU-timers (TIMER0/1/2) shown in (Figure 1-49).

The CPU Timer-0 and CPU-Timer 1 can be used in user applications. Timer 2 is reserved for DSP/BIOS. If the application is not using DSP/BIOS, then Timer 2 can be used in the application. The CPU-timer interrupt signals (TINT0, TINT1, TINT2) are connected as shown in Figure 1-50.



**Figure 1-49. CPU-Timers**



- A. The timer registers are connected to the Memory Bus of the 28x processor.
- B. The timing of the timers is synchronized to SYSCLKOUT of the processor clock.

**Figure 1-50. CPU-Timer Interrupts Signals and Output Signal**

The general operation of the CPU-timer is as follows: The 32-bit counter register TIMH:TIM is loaded with the value in the period register PRDH:PRD. The counter decrements once every (TPR[TDDRH:TDDR]+1) SYSCLOCKOUT cycles, where TDDRH:TDDR is the timer divider. When the counter reaches 0, a timer interrupt output signal generates an interrupt pulse. The registers listed in [Table 1-47](#) are used to configure the timers.

**Table 1-47. CPU-Timers 0, 1, 2 Configuration and Control Registers**

| Name       | Address | Size (x16) | Description                         | Bit Description             |
|------------|---------|------------|-------------------------------------|-----------------------------|
| TIMER0TIM  | 0x0C00  | 1          | CPU-Timer 0, Counter Register       | <a href="#">Figure 1-51</a> |
| TIMER0TIMH | 0x0C01  | 1          | CPU-Timer 0, Counter Register High  | <a href="#">Figure 1-52</a> |
| TIMER0PRD  | 0x0C02  | 1          | CPU-Timer 0, Period Register        | <a href="#">Figure 1-53</a> |
| TIMER0PRDH | 0x0C03  | 1          | CPU-Timer 0, Period Register High   | <a href="#">Figure 1-54</a> |
| TIMER0TCR  | 0x0C04  | 1          | CPU-Timer 0, Control Register       | <a href="#">Figure 1-55</a> |
| TIMER0TPR  | 0x0C06  | 1          | CPU-Timer 0, Prescale Register      | <a href="#">Figure 1-56</a> |
| TIMER0TPRH | 0x0C07  | 1          | CPU-Timer 0, Prescale Register High | <a href="#">Figure 1-57</a> |
| TIMER1TIM  | 0x0C08  | 1          | CPU-Timer 1, Counter Register       | <a href="#">Figure 1-51</a> |
| TIMER1TIMH | 0x0C09  | 1          | CPU-Timer 1, Counter Register High  | <a href="#">Figure 1-52</a> |
| TIMER1PRD  | 0x0C0A  | 1          | CPU-Timer 1, Period Register        | <a href="#">Figure 1-53</a> |
| TIMER1PRDH | 0x0C0B  | 1          | CPU-Timer 1, Period Register High   | <a href="#">Figure 1-54</a> |
| TIMER1TCR  | 0x0C0C  | 1          | CPU-Timer 1, Control Register       | <a href="#">Figure 1-55</a> |
| TIMER1TPR  | 0x0C0E  | 1          | CPU-Timer 1, Prescale Register      | <a href="#">Figure 1-56</a> |
| TIMER1TPRH | 0x0C0F  | 1          | CPU-Timer 1, Prescale Register High | <a href="#">Figure 1-57</a> |
| TIMER2TIM  | 0x0C10  | 1          | CPU-Timer 2, Counter Register       | <a href="#">Figure 1-51</a> |
| TIMER2TIMH | 0x0C11  | 1          | CPU-Timer 2, Counter Register High  | <a href="#">Figure 1-52</a> |
| TIMER2PRD  | 0x0C12  | 1          | CPU-Timer 2, Period Register        | <a href="#">Figure 1-53</a> |
| TIMER2PRDH | 0x0C13  | 1          | CPU-Timer 2, Period Register High   | <a href="#">Figure 1-54</a> |
| TIMER2TCR  | 0x0C14  | 1          | CPU-Timer 2, Control Register       | <a href="#">Figure 1-55</a> |
| TIMER2TPR  | 0x0C16  | 1          | CPU-Timer 2, Prescale Register      | <a href="#">Figure 1-56</a> |
| TIMER2TPRH | 0x0C17  | 1          | CPU-Timer 2, Prescale Register High | <a href="#">Figure 1-57</a> |

**Figure 1-51. TIMERxTIM Register (x = 0, 1, 2)**

15

0

|     |  |
|-----|--|
| TIM |  |
|-----|--|

R/W-0

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 1-48. TIMERxTIM Register Field Descriptions**

| Bits | Field | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|------|-------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-0 | TIM   | CPU-Timer Counter Registers (TIMH:TIM): The TIM register holds the low 16 bits of the current 32-bit count of the timer. The TIMH register holds the high 16 bits of the current 32-bit count of the timer. The TIMH:TIM decrements by one every (TDDRH:TDDR+1) clock cycles, where TDDRH:TDDR is the timer prescale divide-down value. When the TIMH:TIM decrements to zero, the TIMH:TIM register is reloaded with the period value contained in the PRDH:PRD registers. The timer interrupt (TINT) signal is generated. |

**Figure 1-52. TIMERxTIMH Register (x = 0, 1, 2)**

15

0

|      |  |
|------|--|
| TIMH |  |
|------|--|

R/W-0

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 1-49. TIMERxTIMH Register Field Descriptions**

| Bits | Field | Description                    |
|------|-------|--------------------------------|
| 15-0 | TIMH  | See description for TIMERxTIM. |

**Figure 1-53. TIMERxPRD Register (x = 0, 1, 2)**

15

0

|     |
|-----|
| PRD |
|-----|

R/W-1

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 1-50. TIMERxPRD Register Field Descriptions**

| Bits | Field | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|------|-------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-0 | PRD   | CPU-Timer Period Registers (PRDH:PRD): The PRD register holds the low 16 bits of the 32-bit period. The PRDH register holds the high 16 bits of the 32-bit period. When the TIMH:TIM decrements to zero, the TIMH:TIM register is reloaded with the period value contained in the PRDH:PRD registers, at the start of the next timer input clock cycle (the output of the prescaler). The PRDH:PRD contents are also loaded into the TIMH:TIM when you set the timer reload bit (TRB) in the Timer Control Register (TCR). |

**Figure 1-54. TIMERxPRDH Register (x = 0, 1, 2)**

15

0

|      |
|------|
| PRDH |
|------|

R/W-0

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 1-51. TIMERxPRDH Register Field Descriptions**

| Bits | Field | Description                   |
|------|-------|-------------------------------|
| 15-0 | PRDH  | See description for TIMERxPRD |

**Figure 1-55. TIMERxTCR Register (x = 0, 1, 2)**

15

14

13

12

11

10

9

8

|     |     |          |      |      |          |
|-----|-----|----------|------|------|----------|
| TIF | TIE | Reserved | FREE | SOFT | Reserved |
|-----|-----|----------|------|------|----------|

R/W-0

R/W-0

R-0

R/W-0

R/W-0

R-0

7

6

5

4

3

0

|          |     |     |          |
|----------|-----|-----|----------|
| Reserved | TRB | TSS | Reserved |
|----------|-----|-----|----------|

R-0

R/W-0

R/W-0

R-0

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset



**Table 1-52. TIMERxTCR Register Field Descriptions**

| Bits  | Field        | Value                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            | Description                                                                                                                                                                                                                                                                                                                                               |
|-------|--------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15    | TIF          | 0<br>1                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           | CPU-Timer Interrupt Flag.<br>The CPU-Timer has not decremented to zero.<br>Writes of 0 are ignored.<br>This flag gets set when the CPU-timer decrements to zero.<br>Writing a 1 to this bit clears the flag.                                                                                                                                              |
| 14    | TIE          | 0<br>1                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           | CPU-Timer Interrupt Enable.<br>The CPU-Timer interrupt is disabled.<br>The CPU-Timer interrupt is enabled. If the timer decrements to zero, and TIE is set, the timer asserts its interrupt request.                                                                                                                                                      |
| 13-12 | Reserved     |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  | Any writes to these bits must always have a value of 0.                                                                                                                                                                                                                                                                                                   |
| 11-10 | FREE<br>SOFT | <div> <div>CPU-Timer Emulation Modes: These bits are special emulation bits that determine the state of the timer when a breakpoint is encountered in the high-level language debugger. If the FREE bit is set to 1, then, upon a software breakpoint, the timer continues to run (that is, free runs). In this case, SOFT is a <i>don't care</i>. But if FREE is 0, then SOFT takes effect. In this case, if SOFT = 0, the timer halts the next time the TIMH:TIM decrements. If the SOFT bit is 1, then the timer halts when the TIMH:TIM has decremented to zero.</div> <div> <div>CPU-Timer Emulation Mode</div> <div> <div>00</div> <div>01</div> <div>10</div> <div>11</div> </div> <div> <div>Stop after the next decrement of the TIMH:TIM (hard stop)</div> <div>Stop after the TIMH:TIM decrements to 0 (soft stop)</div> <div>Free run</div> <div>Free run</div> </div> </div> </div> | <div> <div>In the SOFT STOP mode, the timer generates an interrupt before shutting down (since reaching 0 is the interrupt causing condition).</div> </div>                                                                                                                                                                                               |
| 9-6   | Reserved     |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  | Any writes to these bits must always have a value of 0.                                                                                                                                                                                                                                                                                                   |
| 5     | TRB          | 0<br>1                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           | CPU-Timer Reload bit.<br>The TRB bit is always read as zero. Writes of 0 are ignored.<br>When you write a 1 to TRB, the TIMH:TIM is loaded with the value in the PRDH:PRD, and the prescaler counter (PSCH:PSC) is loaded with the value in the timer divide-down register (TDDR:TDDR).                                                                   |
| 4     | TSS          | 0<br>1                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           | CPU-Timer stop status bit. TSS is a 1-bit flag that stops or starts the CPU-timer.<br>Reads of 0 indicate the CPU-timer is running.<br>To start or restart the CPU-timer, set TSS to 0. At reset, TSS is cleared to 0 and the CPU-timer immediately starts.<br>Reads of 1 indicate that the CPU-timer is stopped.<br>To stop the CPU-timer, set TSS to 1. |
| 3-0   | Reserved     |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  | Any writes to these bits must always have a value of 0.                                                                                                                                                                                                                                                                                                   |

**Figure 1-56. TIMERxTPR Register (x = 0, 1, 2)**

|     |   |       |   |
|-----|---|-------|---|
| 15  | 8 | 7     | 0 |
| PSC |   | TDDR  |   |
| R-0 |   | R/W-0 |   |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 1-53. TIMERxTPR Register Field Descriptions**

| Bits | Field | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|------|-------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-8 | PSC   | CPU-Timer Prescale Counter. These bits hold the current prescale count for the timer. For every timer clock source cycle that the PSCH:PSC value is greater than 0, the PSCH:PSC decrements by one. One timer clock (output of the timer prescaler) cycle after the PSCH:PSC reaches 0, the PSCH:PSC is loaded with the contents of the TDDRH:TDDR, and the timer counter register (TIMH:TIM) decrements by one. The PSCH:PSC is also reloaded whenever the timer reload bit (TRB) is set by software. The PSCH:PSC can be checked by reading the register, but it cannot be set directly. It must get its value from the timer divide-down register (TDDRH:TDDR). At reset, the PSCH:PSC is set to 0. |
| 7-0  | TDDR  | CPU-Timer Divide-Down. Every (TDDRH:TDDR + 1) timer clock source cycles, the timer counter register (TIMH:TIM) decrements by one. At reset, the TDDRH:TDDR bits are cleared to 0. To increase the overall timer count by an integer factor, write this factor minus one to the TDDRH:TDDR bits. When the prescaler counter (PSCH:PSC) value is 0, one timer clock source cycle later, the contents of the TDDRH:TDDR reload the PSCH:PSC, and the TIMH:TIM decrements by one. TDDRH:TDDR also reloads the PSCH:PSC whenever the timer reload bit (TRB) is set by software.                                                                                                                             |

**Figure 1-57. TIMERxTPRH Register (x = 0, 1, 2)**

|      |   |       |   |
|------|---|-------|---|
| 15   | 8 | 7     | 0 |
| PSCH |   | TDDRH |   |
| R-0  |   | R/W-0 |   |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 1-54. TIMERxTPRH Register Field Descriptions**

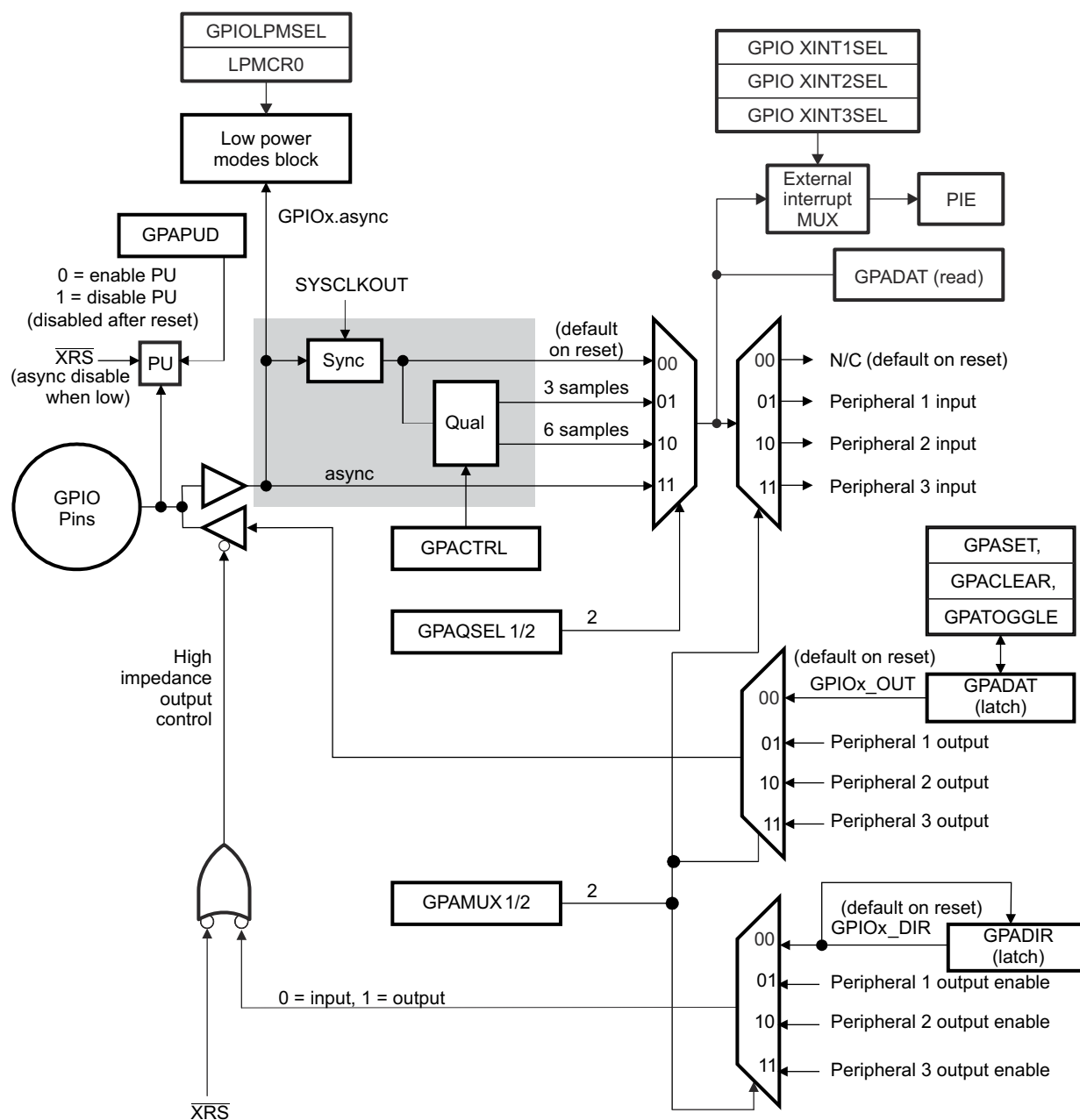
| Bits | Field | Description                   |
|------|-------|-------------------------------|
| 15-8 | PSCH  | See description of TIMERxTPR. |
| 7-0  | TDDRH | See description of TIMERxTPR. |

## 1.4 General-Purpose Input/Output (GPIO)

The GPIO multiplexing (MUX) registers are used to select the operation of shared pins. The pins are named by their general purpose I/O name (GPIO0-GPIO58). These pins can be individually selected to operate as digital I/O, referred to as GPIO, or connected to one of up to three peripheral I/O signals (via the GPxMUXn registers). If selected for digital I/O mode, registers are provided to configure the pin direction (via the GPxDIR registers). You can also qualify the input signals to remove unwanted noise (via the GPxQSELn, GPACTRL, and GPBCTRL registers).

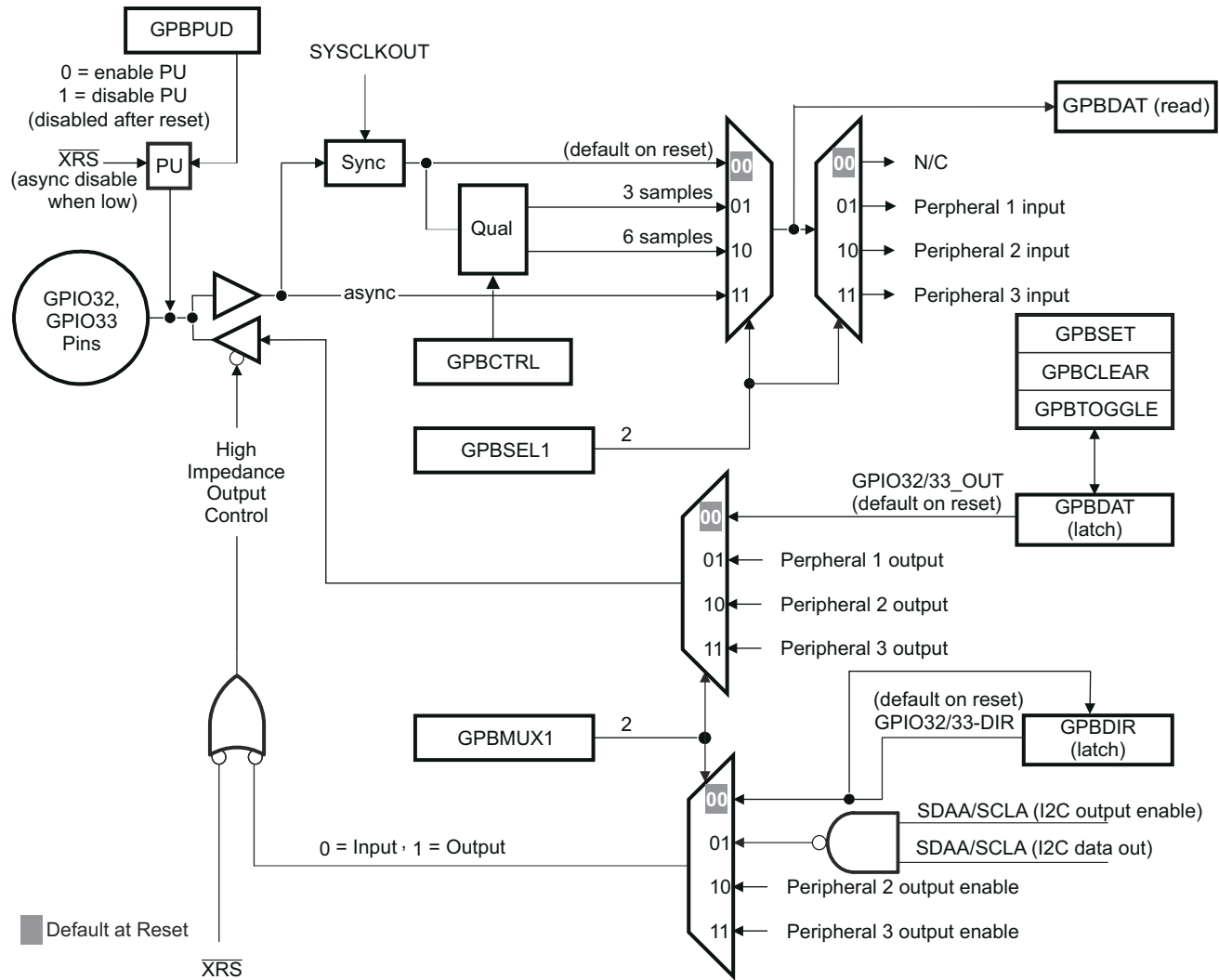
### 1.4.1 GPIO Module Overview

Up to three independent peripheral signals are multiplexed on a single GPIO-enabled pin in addition to individual pin bit-I/O capability. There are three I/O ports. Port A consists of GPIO0 - GPIO31, port B consists of GPIO32 - GPIO58. The analog port consists of AIO0-AIO15. [Figure 1-58](#) shows the basic modes of operation for the GPIO module. Note that GPIO functionality is provided on JTAG pins as well.



A. GPxDAT latch/read are accessed at the same memory location.

**Figure 1-58. General GPIO Multiplexing Diagram**



- A. The input qualification circuit is not reset when modes are changed (such as changing from output to input mode). Any state will get flushed by the circuit eventually.

**Figure 1-59. GPIO32, GPIO33 Multiplexing Diagram**

### 1.4.1.1 JTAG Port

The JTAG port is reduced to 5 pins ( $\overline{\text{TRST}}$ , TCK, TDI, TMS, TDO). TCK, TDI, TMS and TDO pins are also GPIO pins. The  $\overline{\text{TRST}}$  signal selects either JTAG or GPIO operating mode for the pins in [Figure 1-60](#).

#### Note

The JTAG pins may also be used as GPIO pins. Care should be taken in the board design to ensure that the circuitry connected to these pins do not affect the emulation capabilities of the JTAG pin function. Any circuitry connected to these pins should not prevent the debug probe from driving (or being driven by) the JTAG pins for successful debug.

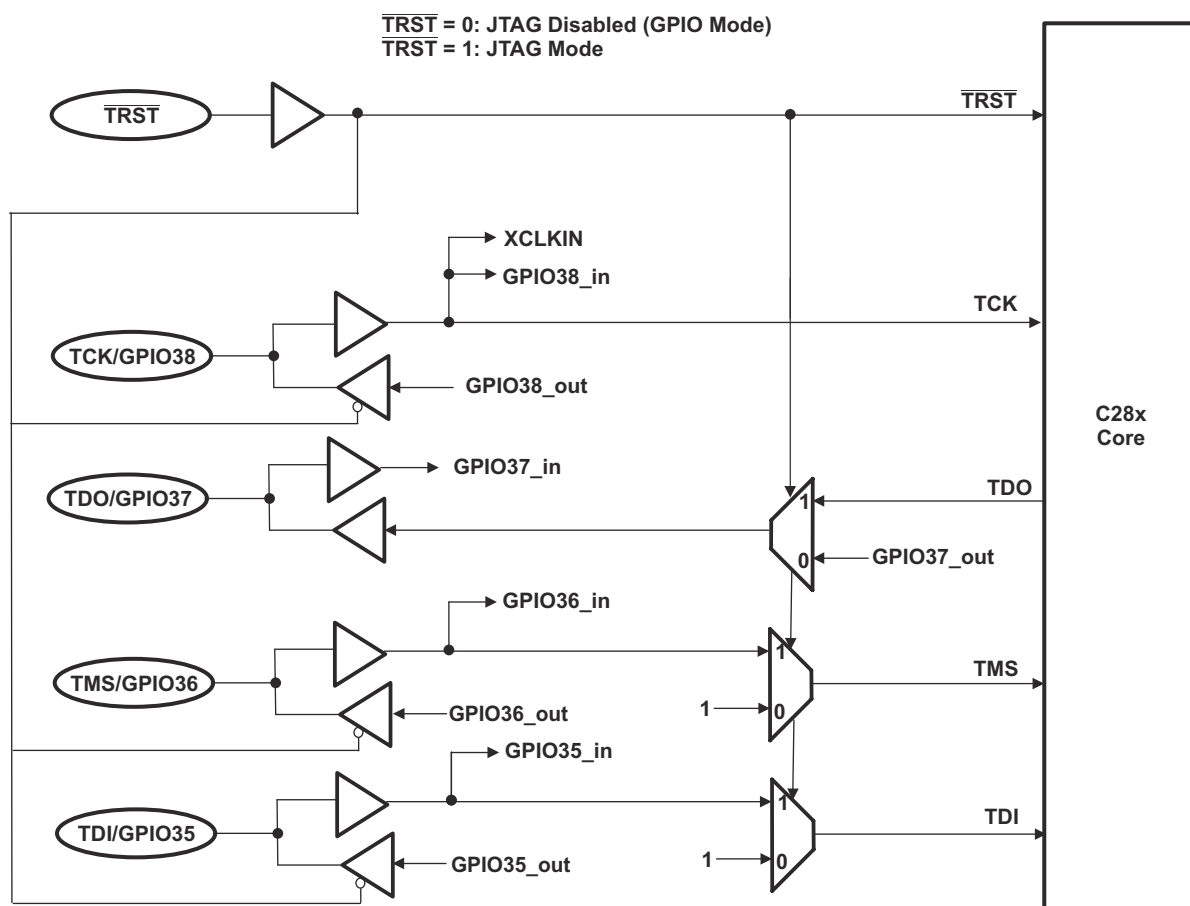


Figure 1-60. JTAG Port/GPIO Multiplexing

### 1.4.1.2 Choosing JTAG or GPIO Functionality

The  $\overline{\text{TRST}}$  signal selects the functionality of the JTAG signals, in combination with the JTAGDIS bit in the JTAGDEBUG register as follows.

| $\overline{\text{TRST}}$ | JTAGDIS bit | JTAG Port Mode                                           |
|--------------------------|-------------|----------------------------------------------------------|
| 0                        | X           | GPIO mode enabled, JTAG port disabled                    |
| 1                        | 0           | JTAG port enabled (GPIOs should be configured as inputs) |
| 1                        | 1           | GPIO mode enabled, JTAG port disabled                    |

The JTAGDEBUG register is shown and described below.

**Figure 1-61. JTAGDEBUG Register (Address 0x702A, EALLOW protected)**

|          |  |   |         |
|----------|--|---|---------|
| 15       |  | 1 | 0       |
| Reserved |  |   | JTAGDIS |
| R-0      |  |   | R/W-0   |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 1-55. JTAGDEBUG Register Field Descriptions**

| Bits | Field    | Value  | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|------|----------|--------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-1 | Reserved |        | Any writes to these bits must always have a value of 0.                                                                                                                                                                                                                                                                                                                                                                                                      |
| 0    | JTAGDIS  | 0<br>1 | JTAG Port Disable Bit: This bit enables/disables the JTAG port. When disabled, the JTAG pins can be used as GPIOs:<br>JTAG Port Enabled<br>JTAG Port Disabled (GPIO Mode)<br><br>This bit is reset by $\overline{\text{TRST}}$ . The bit is forced to "0" when $\overline{\text{TRST}}$ is "0". When $\overline{\text{TRST}}$ is "1", then JTAGDIS bit can be modified by CPU. <b>Note:</b> Ensure no contention with the debug probe signals when JTAGDIS=1 |





## 1.4.2 Configuration Overview

The pin function assignments, input qualification, and the external interrupt sources are all controlled by the GPIO configuration control registers. In addition, you can assign pins to wake the device from the HALT and STANDBY low power modes and enable/disable internal pullup resistors. [Table 1-56](#) and [Table 1-57](#) list the registers that are used to configure the GPIO pins to match the system requirements.

**Table 1-56. GPIO Control Registers**

| Name <sup>(1)</sup> | Address | Size (x16) | Register Description                               | Bit Description             |
|---------------------|---------|------------|----------------------------------------------------|-----------------------------|
| GPACTRL             | 0x6F80  | 2          | GPIO A Control Register (GPIO0-GPIO31)             | <a href="#">Figure 1-70</a> |
| GPAQSEL1            | 0x6F82  | 2          | GPIO A Qualifier Select 1 Register (GPIO0-GPIO15)  | <a href="#">Figure 1-73</a> |
| GPAQSEL2            | 0x6F84  | 2          | GPIO A Qualifier Select 2 Register (GPIO16-GPIO31) | <a href="#">Figure 1-74</a> |
| GPAMUX1             | 0x6F86  | 2          | GPIO A MUX 1 Register (GPIO0-GPIO15)               | <a href="#">Figure 1-65</a> |
| GPAMUX2             | 0x6F88  | 2          | GPIO A MUX 2 Register (GPIO16-GPIO31)              | <a href="#">Figure 1-66</a> |
| GPADIR              | 0x6F8A  | 2          | GPIO A Direction Register (GPIO0-GPIO31)           | <a href="#">Figure 1-77</a> |
| GPAPUD              | 0x6F8C  | 2          | GPIO A Pull Up Disable Register (GPIO0-GPIO31)     | <a href="#">Figure 1-80</a> |
| GPACTRL2            | 0x6F8E  | 2          | USB I/O Control                                    | <a href="#">Figure 1-72</a> |
| GPBCTRL             | 0x6F90  | 2          | GPIO B Control Register (GPIO32-GPIO58)            | <a href="#">Figure 1-71</a> |
| GPBQSEL1            | 0x6F92  | 2          | GPIO B Qualifier Select 1 Register (GPIO32-GPIO44) | <a href="#">Figure 1-75</a> |
| GPBQSEL2            | 0x6F94  | 2          | GPIO B Qualifier Select 2 Register (GPIO50-GPIO58) | <a href="#">Figure 1-76</a> |
| GPBMUX1             | 0x6F96  | 2          | GPIO B MUX 1 Register (GPIO32-GPIO44)              | <a href="#">Figure 1-67</a> |
| GPBMUX2             | 0x6F98  | 2          | GPIO B MUX2 Register (GPIO50-GPIO58)               | <a href="#">Figure 1-68</a> |
| GPBDIR              | 0x6F9A  | 2          | GPIO B Direction Register (GPIO32-GPIO58)          | <a href="#">Figure 1-78</a> |
| GPBPUD              | 0x6F9C  | 2          | GPIO B Pull Up Disable Register (GPIO32-GPIO58)    | <a href="#">Figure 1-81</a> |
| AIOMUX1             | 0x6FB6  | 2          | Analog, I/O MUX 1 register (AIO0 - AIO15)          | <a href="#">Figure 1-69</a> |
| AIDIR               | 0x6FBA  | 2          | Analog, I/O Direction Register (AIO0 - AIO15)      | <a href="#">Figure 1-79</a> |

(1) The registers in this table are EALLOW protected. See [Section 1.5.2](#) for more information.

**Table 1-57. GPIO Interrupt and Low Power Mode Select Registers**

| Name <sup>(1)</sup> | Address | Size (x16) | Register Description                             | Bit Description             |
|---------------------|---------|------------|--------------------------------------------------|-----------------------------|
| GPIOXINT1SEL        | 0x6FE0  | 1          | XINT1 Source Select Register (GPIO0-GPIO31)      | <a href="#">Figure 1-88</a> |
| GPIOXINT2SEL        | 0x6FE1  | 1          | XINT2 Source Select Register (GPIO0-GPIO31)      | <a href="#">Figure 1-88</a> |
| GPIOXINT3SEL        | 0x6FE2  | 1          | XINT3 Source Select Register (GPIO0 - GPIO31)    | <a href="#">Figure 1-88</a> |
| GPIOLPMSSEL         | 0x6FE8  | 1          | LPM wakeup Source Select Register (GPIO0-GPIO31) | <a href="#">Figure 1-89</a> |

(1) The registers in this table are EALLOW protected. See [Section 1.5.2](#) for more information.

To plan configuration of the GPIO module, consider the following steps:

### 1. Plan the device pin-out:

Through a pin multiplexing scheme, a lot of flexibility is provided for assigning functionality to the GPIO-capable pins. Before getting started, look at the peripheral options available for each pin, and plan pin-out for your specific system. Will the pin be used as a general purpose input or output (GPIO) or as one of up to three available peripheral functions? Knowing this information will help determine how to further configure the pin.

### 2. Enable or disable internal pull-up resistors:

To enable or disable the internal pullup resistors, write to the respective bits in the GPIO pullup disable (GPAPUD and GPBPUD) registers. For pins that can function as ePWM output pins, the internal pullup resistors are disabled by default. All other GPIO-capable pins have the pullup enabled by default. The AIOx pins do not have internal pull-up resistors.

**3. Select input qualification:**

If the pin will be used as an input, specify the required input qualification, if any. The input qualification is specified in the GPACTRL, GPBCTRL, GPAQSEL1, GPAQSEL2, GPBQSEL1, and GPBQSEL2 registers. By default, all of the input signals are synchronized to SYSCLKOUT only.

**4. Select the pin function:**

Configure the GPxMUXn or AIOMUXn registers such that the pin is a GPIO or one of three available peripheral functions. By default, all GPIO-capable pins are configured at reset as general purpose input pins.

**5. For digital general purpose I/O, select the direction of the pin:**

If the pin is configured as an GPIO, specify the direction of the pin as either input or output in the GPADIR, GPBDIR, or AIODIR registers. By default, all GPIO pins are inputs. To change the pin from input to output, first load the output latch with the value to be driven by writing the appropriate value to the GPxCLEAR, GPxSET, or GPxTOGGLE (or AIOCLEAR, AIOSET, or AIOTOGGLE) registers. Once the output latch is loaded, change the pin direction from input to output via the GPxDIR registers. The output latch for all pins is cleared at reset.

**6. Select low power mode wake-up sources:**

Specify which pins, if any, will be able to wake the device from HALT and STANDBY low power modes. The pins are specified in the GPIOLPMSEL register.

**7. Select external interrupt sources:**

Specify the source for the XINT1 - XINT3 interrupts. For each interrupt you can specify one of the port A signals as the source. This is done by specifying the source in the GPIOXINTnSEL register. The polarity of the interrupts can be configured in the XINTnCR register as described in [Section 1.6.6](#).

---

**Note**

There is a 2-SYSCLKOUT cycle delay from when a write to configuration registers such as GPxMUXn and GPxQSELn occurs to when the action is valid

---

### 1.4.3 Digital General Purpose I/O Control

For pins that are configured as GPIO you can change the values on the pins by using the registers in [Table 1-58](#).

**Table 1-58. GPIO Data Registers**

| Name <sup>(1)</sup> | Address | Size (x16) | Register Description                        | Bit Description             |
|---------------------|---------|------------|---------------------------------------------|-----------------------------|
| GPADAT              | 0x6FC0  | 2          | GPIO A Data Register (GPIO0-GPIO31)         | <a href="#">Figure 1-82</a> |
| GPASET              | 0x6FC2  | 2          | GPIO A Set Register (GPIO0-GPIO31)          | <a href="#">Figure 1-85</a> |
| GPACLEAR            | 0x6FC4  | 2          | GPIO A Clear Register (GPIO0-GPIO31)        | <a href="#">Figure 1-85</a> |
| GPATOGGLE           | 0x6FC6  | 2          | GPIO A Toggle Register (GPIO0-GPIO31)       | <a href="#">Figure 1-85</a> |
| GPBDAT              | 0x6FC8  | 2          | GPIO B Data Register (GPIO32-GPIO58)        | <a href="#">Figure 1-83</a> |
| GPBSET              | 0x6FCA  | 2          | GPIO B Set Register (GPIO32-GPIO58)         | <a href="#">Figure 1-86</a> |
| GPBCLEAR            | 0x6FCC  | 2          | GPIO B Clear Register (GPIO32-GPIO58)       | <a href="#">Figure 1-86</a> |
| GPBTOGGLE           | 0x6FCE  | 2          | GPIO B Toggle Register (GPIO32-GPIO58)      | <a href="#">Figure 1-86</a> |
| AIODAT              | 0x6FD8  | 2          | Analog I/O Data Register (AIO0 - AIO15)     | <a href="#">Figure 1-84</a> |
| AIOSET              | 0x6FDA  | 2          | Analog I/O Data Set Register (AIO0 - AIO15) | <a href="#">Figure 1-87</a> |
| AIOCLEAR            | 0x6FDC  | 2          | Analog I/O Clear Register (AIO0 - AIO15)    | <a href="#">Figure 1-87</a> |
| AIOGGLE             | 0x6FDE  | 2          | Analog I/O Toggle Register (AIO0 - AIO15)   | <a href="#">Figure 1-87</a> |

(1) The registers in this table are NOT EALLOW protected.

#### • GPxDAT/AIODAT Registers

Each I/O port has one data register. Each bit in the data register corresponds to one GPIO pin. No matter how the pin is configured (GPIO or peripheral function), the corresponding bit in the data register reflects the current state of the pin after qualification (This does not apply to AIOx pins). Writing to the GPxDAT/AIODAT register clears or sets the corresponding output latch and if the pin is enabled as a general purpose output (GPIO output) the pin will also be driven either low or high. If the pin is not configured as a GPIO output then the value will be latched, but the pin will not be driven. Only if the pin is later configured as a GPIO output, will the latched value be driven onto the pin.

When using the GPxDAT register to change the level of an output pin, you should be cautious not to accidentally change the level of another pin. For example, if you mean to change the output latch level of GPIOA1 by writing to the GPADAT register bit 0 using a read-modify-write instruction, a problem can occur if another I/O port A signal changes level between the read and the write stage of the instruction. Following is an analysis of why this happens:

The GPxDAT registers reflect the state of the pin, not the latch. This means the register reflects the actual pin value. However, there is a lag between when the register is written to when the new pin value is reflected back in the register. This may pose a problem when this register is used in subsequent program statements to alter the state of GPIO pins. An example is shown below where two program statements attempt to drive two different GPIO pins that are currently low to a high state.

If Read-Modify-Write operations are used on the GPxDAT registers, because of the delay between the output and the input of the first instruction (I1), the second instruction (I2) will read the old value and write it back.

```
GpioDataRegs.GPADAT.bit.GPIO1 = 1 ; I1 performs read-modify-write of GPADAT
GpioDataRegs.GPADAT.bit.GPIO2 = 1 ; I2 also a read-modify-write of GPADAT.
; It gets the old value of GPIO1 due to the delay
```

The second instruction will wait for the first to finish its write due to the write-followed-by-read protection on this peripheral frame. There will be some lag, however, between the write of (I1) and the GPxDAT bit reflecting the new value (1) on the pin. During this lag, the second instruction will read the old value of GPIO1 (0) and write it back along with the new value of GPIO2 (1). Therefore, GPIO1 pin stays low.

One solution is to put some NOP's between instructions. A better solution is to use the GPxSET/GPxCLEAR/GPxTOGGLE registers instead of the GPxDAT registers. These registers always read back a 0 and writes of 0 have no effect. Only bits that need to be changed can be specified without disturbing any other bits that are currently in the process of changing.

- **GPxSET/AIOSET Registers**

The set registers are used to drive specified GPIO pins high without disturbing other pins. Each I/O port has one set register and each bit corresponds to one GPIO pin. The set registers always read back 0. If the corresponding pin is configured as an output, then writing a 1 to that bit in the set register will set the output latch high and the corresponding pin will be driven high. If the pin is not configured as a GPIO output, then the value will be latched but the pin will not be driven. Only if the pin is later configured as a GPIO output will the latched value will be driven onto the pin. Writing a 0 to any bit in the set registers has no effect.

- **GPxCLEAR/AIOCLEAR Registers**

The clear registers are used to drive specified GPIO pins low without disturbing other pins. Each I/O port has one clear register. The clear registers always read back 0. If the corresponding pin is configured as a general purpose output, then writing a 1 to the corresponding bit in the clear register will clear the output latch and the pin will be driven low. If the pin is not configured as a GPIO output, then the value will be latched but the pin will not be driven. Only if the pin is later configured as a GPIO output will the latched value will be driven onto the pin. Writing a 0 to any bit in the clear registers has no effect.

- **GPxTOGGLE/AIOTOGGLE Registers**

The toggle registers are used to drive specified GPIO pins to the opposite level without disturbing other pins. Each I/O port has one toggle register. The toggle registers always read back 0. If the corresponding pin is configured as an output, then writing a 1 to that bit in the toggle register flips the output latch and pulls the corresponding pin in the opposite direction. That is, if the output pin is driven low, then writing a 1 to the corresponding bit in the toggle register will pull the pin high. Likewise, if the output pin is high, then writing a 1 to the corresponding bit in the toggle register will pull the pin low. If the pin is not configured as a GPIO output, then the value will be latched but the pin will not be driven. Only if the pin is later configured as a GPIO output will the latched value will be driven onto the pin. Writing a 0 to any bit in the toggle registers has no effect.

#### 1.4.4 Input Qualification

The input qualification scheme has been designed to be very flexible. You can select the type of input qualification for each GPIO pin by configuring the GPAQSEL1, GPAQSEL2, GPBQSEL1 and GPBQSEL2 registers. In the case of a GPIO input pin, the qualification can be specified as only synchronize to SYSCLKOUT or qualification by a sampling window. For pins that are configured as peripheral inputs, the input can also be asynchronous in addition to synchronized to SYSCLKOUT or qualified by a sampling window. The remainder of this section describes the options available.

##### 1.4.4.1 No Synchronization (asynchronous input)

This mode is used for peripherals where input synchronization is not required or the peripheral itself performs the synchronization. Examples include communication ports SCI, SPI, eCAN and I<sup>2</sup>C. In addition, it may be desirable to have the ePWM trip zone ( $\overline{TZn}$ ) signals function independent of the presence of SYSCLKOUT.

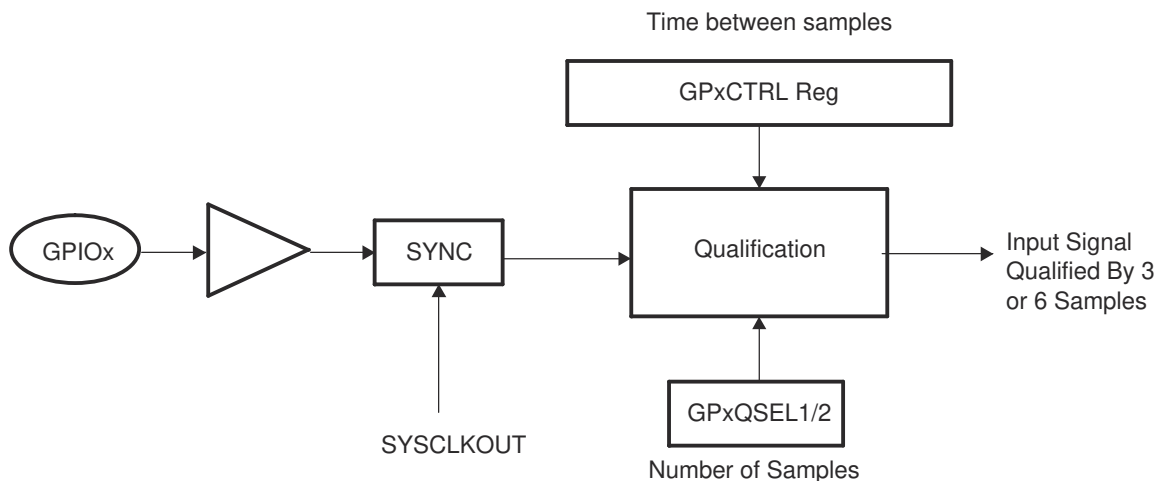
The asynchronous option is not valid if the pin is used as a general purpose digital input pin (GPIO). If the pin is configured as a GPIO input and the asynchronous option is selected then the qualification defaults to synchronization to SYSCLKOUT as described in [Section 1.4.4.2](#).

##### 1.4.4.2 Synchronization to SYSCLKOUT Only

This is the default qualification mode of all the pins at reset. In this mode, the input signal is only synchronized to the system clock (SYSCLKOUT). Because the incoming signal is asynchronous, it can take up to a SYSCLKOUT period of delay in order for the input to the device to be changed. No further qualification is performed on the signal.

### 1.4.4.3 Qualification Using a Sampling Window

In this mode, the signal is first synchronized to the system clock (SYSCLKOUT) and then qualified by a specified number of cycles before the input is allowed to change. [Figure 1-63](#) and [Figure 1-64](#) show how the input qualification is performed to eliminate unwanted noise. Two parameters are specified by the user for this type of qualification: 1) the sampling period, or how often the signal is sampled, and 2) the number of samples to be taken.



**Figure 1-63. Input Qualification Using a Sampling Window**

#### Time between samples (sampling period):

To qualify the signal, the input signal is sampled at a regular period. The sampling period is specified by the user and determines the time duration between samples, or how often the signal will be sampled, relative to the CPU clock (SYSCLKOUT).

The sampling period is specified by the qualification period (QUALPRDn) bits in the GPxCTRL register. The sampling period is configurable in groups of 8 input signals. For example, GPIO0 to GPIO7 use GPxCTRL[QUALPRD0] setting and GPIO8 to GPIO15 use GPxCTRL[QUALPRD1]. [Table 1-59](#) and [Table 1-60](#) show the relationship between the sampling period or sampling frequency and the GPxCTRL[QUALPRDn] setting.

**Table 1-59. Sampling Period**

| Sampling Period                                                 |                                                                 |
|-----------------------------------------------------------------|-----------------------------------------------------------------|
| If GPxCTRL[QUALPRDn] = 0                                        | $1 \times T_{\text{SYSCLKOUT}}$                                 |
| If GPxCTRL[QUALPRDn] ≠ 0                                        | $2 \times \text{GPxCTRL[QUALPRDn]} \times T_{\text{SYSCLKOUT}}$ |
| Where $T_{\text{SYSCLKOUT}}$ is the period in time of SYSCLKOUT |                                                                 |

**Table 1-60. Sampling Frequency**

| Sampling Frequency                                         |                                                                          |
|------------------------------------------------------------|--------------------------------------------------------------------------|
| If GPxCTRL[QUALPRDn] = 0                                   | $f_{\text{SYSCLKOUT}}$                                                   |
| If GPxCTRL[QUALPRDn] ≠ 0                                   | $f_{\text{SYSCLKOUT}} \times 1 \div (2 \times \text{GPxCTRL[QUALPRDn]})$ |
| Where $f_{\text{SYSCLKOUT}}$ is the frequency of SYSCLKOUT |                                                                          |

From these equations, the minimum and maximum time between samples can be calculated for a given SYSCLKOUT frequency:

**Example: Maximum Sampling Frequency:**

If GPxCTRL[QUALPRDn] = 0

then the sampling frequency is  $f_{\text{SYSCLKOUT}}$

If, for example,  $f_{\text{SYSCLKOUT}} = 60 \text{ MHz}$

then the signal will be sampled at 60 MHz or one sample every 16.67 ns.

**Example: Minimum Sampling Frequency:**

If GPxCTRL[QUALPRDn] = 0xFF (255)

then the sampling frequency is  $f_{\text{SYSCLKOUT}} \times 1 \div (2 \times \text{GPxCTRL[QUALPRDn]})$

If, for example,  $f_{\text{SYSCLKOUT}} = 60 \text{ MHz}$

then the signal will be sampled at  $60 \text{ MHz} \times 1 \div (2 \times 255)$  or one sample every 8.5  $\mu\text{s}$ .

**Number of samples:**

The number of times the signal is sampled is either 3 samples or 6 samples as specified in the qualification selection (GPAQSEL1, GPAQSEL2, GPBQSEL1, and GPBQSEL2) registers. When 3 or 6 consecutive cycles are the same, then the input change will be passed through to the DSP.

**Total Sampling Window Width:**

The sampling window is the time during which the input signal will be sampled as shown in [Figure 1-64](#). By using the equation for the sampling period along with the number of samples to be taken, the total width of the window can be determined.

For the input qualifier to detect a change in the input, the level of the signal must be stable for the duration of the sampling window width or longer.

The number of sampling periods within the window is always one less than the number of samples taken. For a three-sample window, the sampling window width is 2 sampling periods wide where the sampling period is defined in [Table 1-59](#). Likewise, for a six-sample window, the sampling window width is 5 sampling periods wide. [Table 1-61](#) and [Table 1-62](#) show the calculations that can be used to determine the total sampling window width based on GPxCTRL[QUALPRDn] and the number of samples taken.

**Table 1-61. Case 1: Three-Sample Sampling Window Width**

|                               | Total Sampling Window Width                                              |
|-------------------------------|--------------------------------------------------------------------------|
| If GPxCTRL[QUALPRDn] = 0      | $2 \times T_{\text{SYSCLKOUT}}$                                          |
| If GPxCTRL[QUALPRDn] $\neq$ 0 | $2 \times 2 \times \text{GPxCTRL[QUALPRDn]} \times T_{\text{SYSCLKOUT}}$ |
|                               | Where $T_{\text{SYSCLKOUT}}$ is the period in time of SYSCLKOUT          |

**Table 1-62. Case 2: Six-Sample Sampling Window Width**

|                               | Total Sampling Window Width                                              |
|-------------------------------|--------------------------------------------------------------------------|
| If GPxCTRL[QUALPRDn] = 0      | $5 \times T_{\text{SYSCLKOUT}}$                                          |
| If GPxCTRL[QUALPRDn] $\neq$ 0 | $5 \times 2 \times \text{GPxCTRL[QUALPRDn]} \times T_{\text{SYSCLKOUT}}$ |
|                               | Where $T_{\text{SYSCLKOUT}}$ is the period in time of SYSCLKOUT          |

### Note

The external signal change is asynchronous with respect to both the sampling period and SYSCLKOUT. Due to the asynchronous nature of the external signal, the input should be held stable for a time greater than the sampling window width to make sure the logic detects a change in the signal. The extra time required can be up to an additional sampling period +  $T_{\text{SYSCLKOUT}}$ .

The required duration for an input signal to be stable for the qualification logic to detect a change is described in the device specific data sheet.

### Example Qualification Window:

For the example shown in Figure 1-64, the input qualification has been configured as follows:

- GPxQSEL1/2 = 1,0. This indicates a six-sample qualification.
- GPxCTRL[QUALPRDn] = 1. The sampling period is  $t_w(\text{SP}) = 2 \times \text{GPxCTRL}[\text{QUALPRDn}] \times T_{\text{SYSCLKOUT}}$ .

This configuration results in the following:

- The width of the sampling window is:

$$t_w(\text{IQSW}) = 5 \times t_w(\text{SP}) = 5 \times 2 \times \text{GPxCTRL}[\text{QUALPRDn}] \times T_{\text{SYSCLKOUT}} \text{ or } 5 \times 2 \times T_{\text{SYSCLKOUT}}$$

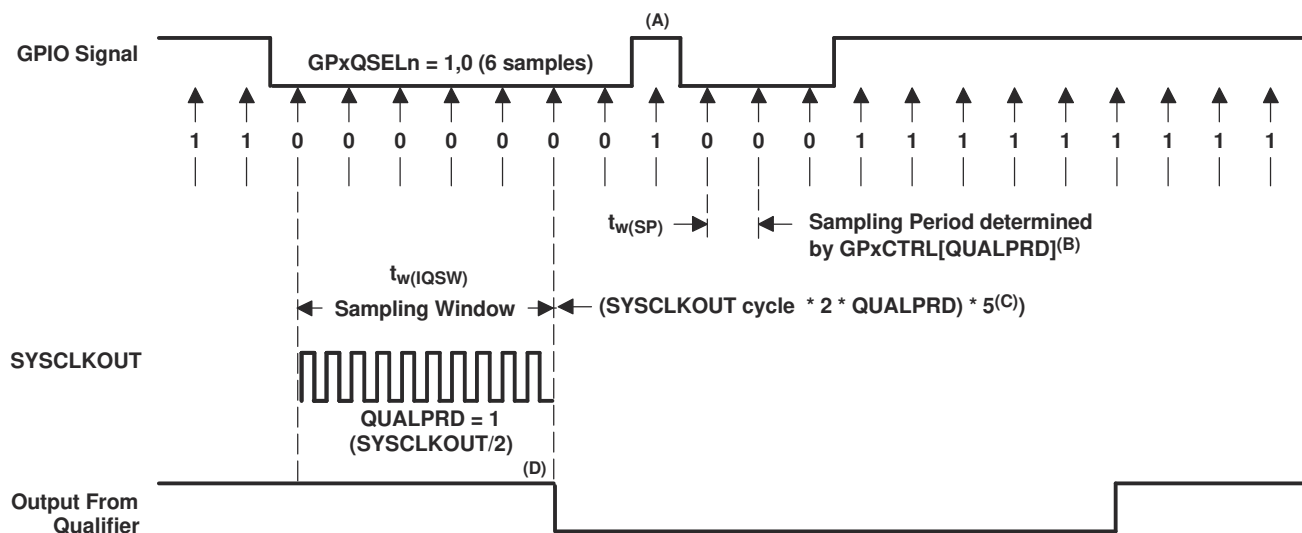
- If, for example,  $T_{\text{SYSCLKOUT}} = 16.67 \text{ ns}$ , then the duration of the sampling window is:

$$t_w(\text{IQSW}) = 5 \times 2 \times 16.67 \text{ ns} = 166.7 \text{ ns}.$$

- To account for the asynchronous nature of the input relative to the sampling period and SYSCLKOUT, up to an additional sampling period,  $t_w(\text{SP})$ , +  $T_{\text{SYSCLKOUT}}$  may be required to detect a change in the input signal. For this example:

$$t_w(\text{SP}) + T_{\text{SYSCLKOUT}} = 333.4 \text{ ns} + 166.67 \text{ ns} = 500.1 \text{ ns}$$

- In Figure 1-64, the glitch (A) is shorter than the qualification window and will be ignored by the input qualifier.



- This glitch will be ignored by the input qualifier. The QUALPRD bit field specifies the qualification sampling period. It can vary from 00 to 0xFF. If QUALPRD = 00, then the sampling period is 1 SYSCLKOUT cycle. For any other value "n", the qualification sampling period in 2n SYSCLKOUT cycles (i.e., at every 2n SYSCLKOUT cycles, the GPIO pin will be sampled).
- The qualification period selected via the GPxCTRL register applies to groups of 8 GPIO pins.
- The qualification block can take either three or six samples. The GPxQSELn Register selects which sample mode is used.
- In the example shown, for the qualifier to detect the change, the input should be stable for 10 SYSCLKOUT cycles or greater. In other words, the inputs should be stable for  $(5 \times \text{QUALPRD} \times 2)$  SYSCLKOUT cycles. That would ensure 5 sampling periods for detection to occur. Since external signals are driven asynchronously, a 13-SYSCLKOUT-wide pulse ensures reliable recognition.

**Figure 1-64. Input Qualifier Clock Cycles**



## 1.4.5 GPIO and Peripheral Multiplexing (MUX)

Up to three different peripheral functions are multiplexed along with a general input/output (GPIO) function per pin. This allows you to pick and choose a peripheral mix that will work best for the particular application.

[Table 1-64](#) and [Table 1-65](#) show an overview of the possible multiplexing combinations sorted by GPIO pin. The second column indicates the I/O name of the pin on the device. Since the I/O name is unique, it is the best way to identify a particular pin. Therefore, the register descriptions in this section only refer to the GPIO name of a particular pin. The MUX register and particular bits that control the selection for each pin are indicated in the first column.

For example, the multiplexing for the GPIO6 pin is controlled by writing to GPAMUX1[13:12]. By writing to these bits, the pin is configured as either GPIO6, or one of up to three peripheral functions. The GPIO6 pin can be configured as follows:

| GPAMUX1[13:12] Bit Setting | Pin Functionality Selected      |
|----------------------------|---------------------------------|
| If GPAMUX1[13:12] = 0,0    | Pin configured as GPIO6         |
| If GPAMUX1[13:12] = 0,1    | Pin configured as EPWM4A (O)    |
| If GPAMUX1[13:12] = 1,0    | Pin configured as EPWMSYNCl (I) |
| If GPAMUX1[13:12] = 1,1    | Pin configured as EPWMSYNCO (O) |

The devices have different multiplexing schemes. If a peripheral is not available on a particular device, that MUX selection is reserved on that device and should not be used.

### Note

If you should select a reserved GPIO MUX configuration that is not mapped to a peripheral, the state of the pin will be undefined and the pin may be driven. Reserved configurations are for future expansion and should not be selected. In the device MUX tables ([Table 1-64](#) and [Table 1-65](#)) these options are indicated as Reserved.

Some peripherals can be assigned to more than one pin via the MUX registers. For example, the SPISIMOB can be assigned to either the GPIO12 or GPIO24 pin, depending on individual system requirements as shown.

| Pin Assigned to SPISIMOB | MUX Configuration   |
|--------------------------|---------------------|
| Choice 1 GPIO12          | GPAMUX1[25:24] = 11 |
| or Choice 2 GPIO24       | GPAMUX2[17:16] = 11 |

If no pin is configured as an input to a peripheral, or if more than one pin is configured as an input for the same peripheral, then the input to the peripheral will either default to a 0 or a 1 as shown in [Table 1-63](#). For example, if SPISIMOB were assigned to both GPIO12 and GPIO24, the input to the SPI peripheral would default to a high state as shown in [Table 1-63](#) and the input would not be connected to GPIO12 or GPIO24.

**Table 1-63. Default State of Peripheral Input**

| Peripheral Input                          | Description                | Default Input <sup>(1)</sup> |
|-------------------------------------------|----------------------------|------------------------------|
| TZ1- TZ3                                  | Trip zone 1-3              | 1                            |
| EPWMSYNCl                                 | ePWM Synch Input           | 0                            |
| ECAP1                                     | eCAP1 input                | 1                            |
| EQEP1A                                    | eQEP input                 | 1                            |
| EQEP1I                                    | eQEP index                 | 1                            |
| EQEP1S                                    | eQEP strobe                | 1                            |
| SPICLK <sub>A</sub> /SPICLK <sub>B</sub>  | SPI-A clock                | 1                            |
| SPISTE <sub>A</sub> / SPISTE <sub>B</sub> | SPI-A transmit enable      | 0                            |
| SPISIMOA/SPISIMOB                         | SPI-A Slave-in, master-out | 1                            |
| SPISOMIA/SPISOMIB                         | SPI-A Slave-out, master-in | 1                            |
| SCIRXDA - SCIRXDB                         | SCI-A - SCI-B receive      | 1                            |
| CANRXA                                    | eCAN-A receive             | 1                            |
| SDAA                                      | I <sup>2</sup> C data      | 1                            |
| SCLA1                                     | I <sup>2</sup> C clock     | 1                            |

(1) This value will be assigned to the peripheral input if more then one pin has been assigned to the peripheral function in the GPxMUX1/2 registers or if no pin has been assigned.

**Table 1-64. GPIOA MUX<sup>(1) (2)</sup>**

|                          | Default at Reset<br>Primary I/O Function | PERIPHERAL<br>SELECTION 1 | PERIPHERAL<br>SELECTION 2   | PERIPHERAL<br>SELECTION 3  |
|--------------------------|------------------------------------------|---------------------------|-----------------------------|----------------------------|
| GPAMUX1 REGISTER<br>BITS | (GPAMUX1 BITS = 00)                      | (GPAMUX1 BITS = 01)       | (GPAMUX1 BITS = 10)         | (GPAMUX1 BITS = 11)        |
| 1-0                      | GPIO0                                    | EPWM1A (O)                | Reserved                    | Reserved                   |
| 3-2                      | GPIO1                                    | EPWM1B (O)                | Reserved                    | COMP1OUT (O)               |
| 5-4                      | GPIO2                                    | EPWM2A (O)                | Reserved                    | Reserved                   |
| 7-6                      | GPIO3                                    | EPWM2B (O)                | SPISOMIA (I/O)              | COMP2OUT (O)               |
| 9-8                      | GPIO4                                    | EPWM3A (O)                | Reserved                    | Reserved                   |
| 11-10                    | GPIO5                                    | EPWM3B (O)                | SPISIMOA (I/O)              | ECAP1 (I/O)                |
| 13-12                    | GPIO6                                    | EPWM4A (O)                | EPWMSYNCl (I)               | EPWMSYNCO (O)              |
| 15-14                    | GPIO7                                    | EPWM4B (O)                | SCIRXDA (I)                 | ECAP2 (I/O)                |
| 17-16                    | GPIO8                                    | EPWM5A (O)                | Reserved                    | ADCSOCl (O)                |
| 19-18                    | GPIO9                                    | EPWM5B (O)                | SCITXDB <sup>(3)</sup> (O)  | ECAP3 (I/O)                |
| 21-20                    | GPIO10                                   | EPWM6A (O)                | Reserved                    | ADCSOCBO (O)               |
| 23-22                    | GPIO11                                   | EPWM6B (O)                | SCIRXDB <sup>(3)</sup> (I)  | ECAP1 (I/O)                |
| 25-24                    | GPIO12                                   | TZ1 (I)                   | SCITXDA (O)                 | SPISIMOB (I/O)             |
| 27-26                    | GPIO13                                   | TZ2 (I)                   | Reserved                    | SPISOMIB (I/O)             |
| 29-28                    | GPIO14                                   | TZ3 (I)                   | SCITXDB <sup>(3)</sup> (O)  | SPICLKB (I/O)              |
| 31-30                    | GPIO15                                   | ECAP2 (I/O)               | SCIRXDB <sup>(3)</sup> (I)  | SPISTEB (I/O)              |
| GPAMUX2 REGISTER<br>BITS | (GPAMUX2 BITS = 00)                      | (GPAMUX2 BITS = 01)       | (GPAMUX2 BITS = 10)         | (GPAMUX2 BITS = 11)        |
| 1-0                      | GPIO16                                   | SPISIMOA (I/O)            | Reserved                    | TZ2 (I)                    |
| 3-2                      | GPIO17                                   | SPISOMIA (I/O)            | Reserved                    | TZ3 (I)                    |
| 5-4                      | GPIO18                                   | SPICLKA (I/O)             | SCITXDB <sup>(3)</sup> (O)  | XCLKOUT (O)                |
| 7-6                      | GPIO19/XCLKIN                            | SPISTEA (I/O)             | SCIRXDB <sup>(3)</sup> (I)  | ECAP1 (I/O)                |
| 9-8                      | GPIO20                                   | EQEP1A (I)                | MDXA (O)                    | COMP1OUT (O)               |
| 11-10                    | GPIO21                                   | EQEP1B (I)                | MDRA (I)                    | COMP2OUT (O)               |
| 13-12                    | GPIO22                                   | EQEP1S (I/O)              | MCLKXA (I/O)                | SCITXDB <sup>(3)</sup> (O) |
| 15-14                    | GPIO23                                   | EQEP1I (I/O)              | MFSXA (I/O)                 | SCIRXDB <sup>(3)</sup> (I) |
| 17-16                    | GPIO24                                   | ECAP1 (I/O)               | EQEP2A <sup>(3)</sup> (I)   | SPISIMOB (I/O)             |
| 19-18                    | GPIO25                                   | ECAP2 (I/O)               | EQEP2B <sup>(3)</sup> (I)   | SPISOMIB (I/O)             |
| 21-20                    | GPIO26                                   | ECAP3 (I/O)               | EQEP2I <sup>(3)</sup> (I/O) | SPICLKB (I/O)              |
| 23-22                    | GPIO27                                   | HRCAP2 (I)                | EQEP2S <sup>(3)</sup> (I/O) | SPISTEB (I/O)              |
| 25-24                    | GPIO28                                   | SCIRXDA (I)               | SDAA (I/OD)                 | TZ2 (I)                    |
| 27-26                    | GPIO29                                   | SCITXDA (O)               | SCLA (I/OD)                 | TZ3 (I)                    |
| 29-28                    | GPIO30                                   | CANRXA (I)                | EQEP2I <sup>(3)</sup> (I/O) | EPWM7A (O)                 |
| 31-30                    | GPIO31                                   | CANTXA (O)                | EQEP2S <sup>(3)</sup> (I/O) | EPWM8A (O)                 |

(1) The word "Reserved" means that there is no peripheral assigned to this GPxMUX1/2 register setting. Should it be selected, the state of the pin will be undefined and the pin may be driven. This selection is a reserved configuration for future expansion.

(2) I = Input, O = Output, OD = Open Drain

(3) eQEP2 is not available on the 80-pin PN/PFP package.

**Table 1-65. GPIOB MUX<sup>(1) (2)</sup>**

|                                  | Default at Reset<br>Primary I/O Function | PERIPHERAL<br>SELECTION 1  | PERIPHERAL<br>SELECTION 2  | PERIPHERAL<br>SELECTION 3  |
|----------------------------------|------------------------------------------|----------------------------|----------------------------|----------------------------|
| <b>GPBMUX1 REGISTER<br/>BITS</b> | <b>(GPBMUX1 BITS = 00)</b>               | <b>(GPBMUX1 BITS = 01)</b> | <b>(GPBMUX1 BITS = 10)</b> | <b>(GPBMUX1 BITS = 11)</b> |
| 1-0                              | GPIO32                                   | SDAA (I/OD)                | EPWMSYNCI (I)              | ADCSOAO (O)                |
| 3-2                              | GPIO33                                   | SCLA (I/OD)                | EPWMSYNCO (O)              | ADCSOCBO (O)               |
| 5-4                              | GPIO34                                   | COMP2OUT (O)               | Reserved                   | COMP3OUT (O)               |
| 7-6                              | GPIO35 (TDI)                             | Reserved                   | Reserved                   | Reserved                   |
| 9-8                              | GPIO36 (TMS)                             | Reserved                   | Reserved                   | Reserved                   |
| 11-10                            | GPIO37 (TDO)                             | Reserved                   | Reserved                   | Reserved                   |
| 13-12                            | GPIO38/XCLKIN (TCK)                      | Reserved                   | Reserved                   | Reserved                   |
| 15-14                            | GPIO39                                   | Reserved                   | Reserved                   | Reserved                   |
| 17-16                            | GPIO40 <sup>(3)</sup>                    | EPWM7A (O)                 | SCITXDB (O)                | Reserved                   |
| 19-18                            | GPIO41 <sup>(3)</sup>                    | EPWM7B (O)                 | SCIRXDB (I)                | Reserved                   |
| 21-20                            | GPIO42 <sup>(3)</sup>                    | EPWM8A (O)                 | TZ1 (I)                    | COMP1OUT (O)               |
| 23-22                            | GPIO43 <sup>(3)</sup>                    | EPWM8B (O)                 | TZ2 (I)                    | COMP2OUT (O)               |
| 25-24                            | GPIO44 <sup>(3)</sup>                    | MFSRA (I/O)                | SCIRXDB (I)                | EPWM7B (O)                 |
| 27-26                            | Reserved                                 | Reserved                   | Reserved                   | Reserved                   |
| 29-28                            | Reserved                                 | Reserved                   | Reserved                   | Reserved                   |
| 31-30                            | Reserved                                 | Reserved                   | Reserved                   | Reserved                   |
| <b>GPBMUX2 REGISTER<br/>BITS</b> | <b>(GPBMUX2 BITS = 00)</b>               | <b>(GPBMUX2 BITS = 01)</b> | <b>(GPBMUX2 BITS = 10)</b> | <b>(GPBMUX2 BITS = 11)</b> |
| 1-0                              | Reserved                                 | Reserved                   | Reserved                   | Reserved                   |
| 3-2                              | Reserved                                 | Reserved                   | Reserved                   | Reserved                   |
| 5-4                              | GPIO50 <sup>(3)</sup>                    | EQEP1A (I)                 | MDXA (O)                   | TZ1 (I)                    |
| 7-6                              | GPIO51 <sup>(3)</sup>                    | EQEP1B (I)                 | MDRA (I)                   | TZ2 (I)                    |
| 9-8                              | GPIO52 <sup>(3)</sup>                    | EQEP1S (I/O)               | MCLKXA (I/O)               | TZ3 (I)                    |
| 11-10                            | GPIO53 <sup>(3)</sup>                    | EQEP1I (I/O)               | MFSXA (I/O)                | Reserved                   |
| 13-12                            | GPIO54 <sup>(3)</sup>                    | SPISIMOA (I/O)             | EQEP2A (I)                 | HRCAP1 (I)                 |
| 15-14                            | GPIO55 <sup>(3)</sup>                    | SPISOMIA (I/O)             | EQEP2B (I)                 | HRCAP2 (I)                 |
| 17-16                            | GPIO56 <sup>(3)</sup>                    | SPICLKA (I/O)              | EQEP2I (I/O)               | HRCAP3 (I)                 |
| 19-18                            | GPIO57 <sup>(3)</sup>                    | SPISTEA (I/O)              | EQEP2S (I/O)               | HRCAP4 (I)                 |
| 21-20                            | GPIO58 <sup>(3)</sup>                    | MCLKRA (I/O)               | SCITXDB (O)                | EPWM7A (O)                 |
| 23-22                            | Reserved                                 | Reserved                   | Reserved                   | Reserved                   |
| 25-24                            | Reserved                                 | Reserved                   | Reserved                   | Reserved                   |
| 27-26                            | Reserved                                 | Reserved                   | Reserved                   | Reserved                   |
| 29-28                            | Reserved                                 | Reserved                   | Reserved                   | Reserved                   |
| 31-30                            | Reserved                                 | Reserved                   | Reserved                   | Reserved                   |

- (1) The word "Reserved" means that there is no peripheral assigned to this GPxMUX1/2 register setting. Should it be selected, the state of the pin will be undefined and the pin may be driven. This selection is a reserved configuration for future expansion.
- (2) I = Input, O = Output, OD = Open Drain
- (3) This pin is not available in the 80-pin PN/PFP package.

**Table 1-66. Analog MUX**

|                       |                    | Default at Reset                |                                                   |
|-----------------------|--------------------|---------------------------------|---------------------------------------------------|
|                       |                    | AIOx and Peripheral Selection 1 | Peripheral Selection 2 and Peripheral Selection 3 |
| AIOMUX1 Register bits | AIOMUX1 bits = 0,x | AIOMUX1 bits = 1,x              |                                                   |
| 1-0                   | ADCINA0 (I)        | ADCINA0 (I)                     |                                                   |
| 3-2                   | ADCINA1 (I)        | ADCINA1 (I)                     |                                                   |
| 5-4                   | AIO2 (I/O)         | ADCINA2 (I), COMP1A (I)         |                                                   |
| 7-6                   | ADCINA3 (I)        | ADCINA3 (I)                     |                                                   |
| 9-8                   | AIO4 (I/O)         | ADCINA4 (I), COMP2A (I)         |                                                   |
| 11-10                 | ADCINA5 (I)        | ADCINA5 (I)                     |                                                   |
| 13-12                 | AIO6 (I/O)         | ADCINA6 (I), COMP3A (1)         |                                                   |
| 15-14                 | ADCINA7 (I)        | ADCINA7 (I)                     |                                                   |
| 17-16                 | ADCINB0 (I)        | ADCINB0 (I)                     |                                                   |
| 19-18                 | ADCINB1 (I)        | ADCINB1 (I)                     |                                                   |
| 21-20                 | AIO10 (I/O)        | ADCINB2 (I), COMP1B (I)         |                                                   |
| 23-22                 | ADCINB3 (I)        | ADCINB3 (I)                     |                                                   |
| 25-24                 | AIO12 (I/O)        | ADCINB4 (I), COMP2B (I)         |                                                   |
| 27-26                 | ADCINB5 (I)        | ADCINB5 (I)                     |                                                   |
| 29-28                 | AIO14 (I/O)        | ADCINB6 (I), COMP3B (1)         |                                                   |
| 31-30                 | ADCINB7 (I)        | ADCINB7 (I)                     |                                                   |

### 1.4.6 Register Bit Definitions

**Figure 1-65. GPIO Port A MUX 1 (GPAMUX1) Register**

|        |        |        |        |        |        |       |       |    |    |    |    |    |    |    |    |
|--------|--------|--------|--------|--------|--------|-------|-------|----|----|----|----|----|----|----|----|
| 31     | 30     | 29     | 28     | 27     | 26     | 25    | 24    | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| GPIO15 | GPIO14 | GPIO13 | GPIO12 | GPIO11 | GPIO10 | GPIO9 | GPIO8 |    |    |    |    |    |    |    |    |
| R/W-0  | R/W-0  | R/W-0  | R/W-0  | R/W-0  | R/W-0  | R/W-0 | R/W-0 |    |    |    |    |    |    |    |    |
| 15     | 14     | 13     | 12     | 11     | 10     | 9     | 8     | 7  | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
| GPIO7  | GPIO6  | GPIO5  | GPIO4  | GPIO3  | GPIO2  | GPIO1 | GPIO0 |    |    |    |    |    |    |    |    |
| R/W-0  | R/W-0  | R/W-0  | R/W-0  | R/W-0  | R/W-0  | R/W-0 | R/W-0 |    |    |    |    |    |    |    |    |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 1-67. GPIO Port A Multiplexing 1 (GPAMUX1) Register Field Descriptions**

| Bits  | Field  | Value | Description                                                                                      |
|-------|--------|-------|--------------------------------------------------------------------------------------------------|
| 31-30 | GPIO15 | 00    | Configure the GPIO15 pin as:<br>GPIO15 - General purpose input/output 15 (default) (I/O)         |
|       |        | 01    | ECAP2 (I/O)                                                                                      |
|       |        | 10    | SCIRXDB (I)                                                                                      |
|       |        | 11    | SPISTEB (I/O)                                                                                    |
| 29-28 | GPIO14 | 00    | Configure the GPIO14 pin as:<br>GPIO14 - General purpose I/O 14 (default) (I/O)                  |
|       |        | 01    | TZ3 - Trip zone 3 (I)                                                                            |
|       |        | 10    | SCITXDB (O)                                                                                      |
|       |        | 11    | SPICLKB (IO) - SPI-B clock<br>This option is reserved on devices that do not have an SPI-B port. |

**Table 1-67. GPIO Port A Multiplexing 1 (GPAMUX1) Register Field Descriptions (continued)**

| Bits  | Field  | Value | Description                                                                                                            |
|-------|--------|-------|------------------------------------------------------------------------------------------------------------------------|
| 27-26 | GPIO13 |       | Configure the GPIO13 pin as:                                                                                           |
|       |        | 00    | GPIO13 - General purpose I/O 13 (default) (I/O)                                                                        |
|       |        | 01    | $\overline{TZ2}$ - Trip zone 2 (I)                                                                                     |
|       |        | 10    | Reserved                                                                                                               |
|       |        | 11    | SPISOMIB (I/O) - SPI-B Slave Output/Master input<br>This option is reserved on devices that do not have an SPI-B port. |
| 25-24 | GPIO12 |       | Configure the GPIO12 pin as:                                                                                           |
|       |        | 00    | GPIO12 - General purpose I/O 12 (default) (I/O)                                                                        |
|       |        | 01    | $\overline{TZ1}$ - Trip zone 1 (I)                                                                                     |
|       |        | 10    | SCITXDA - SCI-A Transmit (O)                                                                                           |
|       |        | 11    | SPISIMOB (I/O) - SPI-B Slave input/Master output<br>This option is reserved on devices that do not have an SPI-B port. |
| 23-22 | GPIO11 |       | Configure the GPIO11 pin as:                                                                                           |
|       |        | 00    | GPIO11 - General purpose I/O 11 (default) (I/O)                                                                        |
|       |        | 01    | EPWM6B - ePWM 6 output B (O)                                                                                           |
|       |        | 10    | SCIRXDB (I)                                                                                                            |
|       |        | 11    | ECAP1 (I/O)                                                                                                            |
| 21-20 | GPIO10 |       | Configure the GPIO10 pin as:                                                                                           |
|       |        | 00    | GPIO10 - General purpose I/O 10 (default) (I/O)                                                                        |
|       |        | 01    | EPWM6A - ePWM6 output A (O)                                                                                            |
|       |        | 10    | Reserved                                                                                                               |
|       |        | 11    | $\overline{ADCSOCBO}$ - ADC Start of conversion B (O)                                                                  |
| 19-18 | GPIO9  |       | Configure the GPIO9 pin as:                                                                                            |
|       |        | 00    | GPIO9 - General purpose I/O 9 (default) (I/O)                                                                          |
|       |        | 01    | EPWM5B - ePWM5 output B                                                                                                |
|       |        | 10    | SCITXDB (O)                                                                                                            |
|       |        | 11    | ECAP3 (I/O)                                                                                                            |
| 17-16 | GPIO8  |       | Configure the GPIO8 pin as:                                                                                            |
|       |        | 00    | GPIO8 - General purpose I/O 8 (default) (I/O)                                                                          |
|       |        | 01    | EPWM5A - ePWM5 output A (O)                                                                                            |
|       |        | 10    | Reserved                                                                                                               |
|       |        | 11    | $\overline{ADCSOCAO}$ - ADC Start of conversion A                                                                      |
| 15-14 | GPIO7  |       | Configure the GPIO7 pin as:                                                                                            |
|       |        | 00    | GPIO7 - General purpose I/O 7 (default) (I/O)                                                                          |
|       |        | 01    | EPWM4B - ePWM4 output B (O)                                                                                            |
|       |        | 10    | SCIRXDA (I) - SCI-A receive (I)                                                                                        |
|       |        | 11    | ECAP2 (I/O)                                                                                                            |
| 13-12 | GPIO6  |       | Configure the GPIO6 pin as:                                                                                            |
|       |        | 00    | GPIO6 - General purpose I/O 6 (default)                                                                                |
|       |        | 01    | EPWM4A - ePWM4 output A (O)                                                                                            |
|       |        | 10    | EPWMSYNCl - ePWM Synch-in (I)                                                                                          |
|       |        | 11    | EPWMSYNCO - ePWM Synch-out (O)                                                                                         |
| 11-10 | GPIO5  |       | Configure the GPIO5 pin as:                                                                                            |
|       |        | 00    | GPIO5 - General purpose I/O 5 (default) (I/O)                                                                          |
|       |        | 01    | EPWM3B - ePWM3 output B                                                                                                |
|       |        | 10    | SPISIMOA (I/O) - SPI-A Slave input/Master output                                                                       |
|       |        | 11    | ECAP1 - eCAP1 (I/O)                                                                                                    |

**Table 1-67. GPIO Port A Multiplexing 1 (GPAMUX1) Register Field Descriptions (continued)**

| Bits | Field | Value | Description                                       |
|------|-------|-------|---------------------------------------------------|
| 9-8  | GPIO4 |       | Configure the GPIO4 pin as:                       |
|      |       | 00    | GPIO4 - General purpose I/O 4 (default) (I/O)     |
|      |       | 01    | EPWM3A - ePWM3 output A (O)                       |
|      |       | 10    | Reserved                                          |
|      |       | 11    | Reserved                                          |
| 7-6  | GPIO3 |       | Configure the GPIO3 pin as:                       |
|      |       | 00    | GPIO3 - General purpose I/O 3 (default) (I/O)     |
|      |       | 01    | EPWM2B - ePWM2 output B (O)                       |
|      |       | 10    | SPISOMIA (I/O) - SPI-A Slave output/Master input  |
|      |       | 11    | COMP2OUT (O) - Comparator 2 output                |
| 5-4  | GPIO2 |       | Configure the GPIO2 pin as:                       |
|      |       | 00    | GPIO2 (I/O) General purpose I/O 2 (default) (I/O) |
|      |       | 01    | EPWM2A - ePWM2 output A (O)                       |
|      |       | 10    | Reserved                                          |
|      |       | 11    | Reserved                                          |
| 3-2  | GPIO1 |       | Configure the GPIO1 pin as:                       |
|      |       | 00    | GPIO1 - General purpose I/O 1 (default) (I/O)     |
|      |       | 01    | EPWM1B - ePWM1 output B (O)                       |
|      |       | 10    | Reserved                                          |
|      |       | 11    | COMP1OUT (O) - Comparator 1 output                |
| 1-0  | GPIO0 |       | Configure the GPIO0 pin as:                       |
|      |       | 00    | GPIO0 - General purpose I/O 0 (default) (I/O)     |
|      |       | 01    | EPWM1A - ePWM1 output A (O)                       |
|      |       | 10    | Reserved                                          |
|      |       | 11    | Reserved                                          |



**Figure 1-66. GPIO Port A MUX 2 (GPAMUX2) Register**

|        |        |        |        |                   |        |        |        |    |    |    |    |    |    |    |    |
|--------|--------|--------|--------|-------------------|--------|--------|--------|----|----|----|----|----|----|----|----|
| 31     | 30     | 29     | 28     | 27                | 26     | 25     | 24     | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| GPIO31 | GPIO30 | GPIO29 | GPIO28 | GPIO27            | GPIO26 | GPIO25 | GPIO24 |    |    |    |    |    |    |    |    |
| R/W-0  | R/W-0  | R/W-0  | R/W-0  | R/W-0             | R/W-0  | R/W-0  | R/W-0  |    |    |    |    |    |    |    |    |
| 15     | 14     | 13     | 12     | 11                | 10     | 9      | 8      | 7  | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
| GPIO23 | GPIO22 | GPIO21 | GPIO20 | GPIO19/<br>XCLKIN | GPIO18 | GPIO17 | GPIO16 |    |    |    |    |    |    |    |    |
| R/W-0  | R/W-0  | R/W-0  | R/W-0  | R/W-0             | R/W-0  | R/W-0  | R/W-0  |    |    |    |    |    |    |    |    |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 1-68. GPIO Port A MUX 2 (GPAMUX2) Register Field Descriptions**

| Bits  | Field  | Value                | Description                                                                                                                                                                                                                    |
|-------|--------|----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 31-30 | GPIO31 | 00<br>01<br>10<br>11 | Configure the GPIO31 pin as:<br>GPIO31 - General purpose I/O 31 (default) (I/O)<br>CANTXA - eCAN-A transmit (O)<br>EQEP2S (I/O)<br>EPWM8A (O)                                                                                  |
| 29-28 | GPIO30 | 00<br>01<br>10<br>11 | Configure the GPIO30 pin as:<br>GPIO30 (I/O) General purpose I/O 30 (default) (I/O)<br>CANRXA - eCAN-A receive (I)<br>EQEP2I (I/O)<br>EPWM7A (O)                                                                               |
| 27-26 | GPIO29 | 00<br>01<br>10<br>11 | Configure the GPIO29 pin as:<br>GPIO29 (I/O) General purpose I/O 29 (default) (I/O)<br>SCITXDA - SCI-A transmit. (O)<br>SCLA - I <sup>2</sup> C clock open drain bidirectional port (I/O)<br>$\overline{TZ3}$ - Trip zone 3(I) |
| 25-24 | GPIO28 | 00<br>01<br>10<br>11 | Configure the GPIO28 pin as:<br>GPIO28 (I/O) General purpose I/O 28 (default) (I/O)<br>SCIRXDA - SCI-A receive (I)<br>SDAA - I <sup>2</sup> C data open drain bidirectional port (I/O)<br>$\overline{TZ2}$ - Trip zone 2 (I)   |
| 23-22 | GPIO27 | 00<br>01<br>10<br>11 | Configure the GPIO27 pin as:<br>GPIO27 - General purpose I/O 27 (default) (I/O)<br>HRCAP2 (I)<br>EQEP2S (I/O)<br>$\overline{SPISTEB}$ (I/O) - SPI-B Slave transmit enable                                                      |
| 21-20 | GPIO26 | 00<br>01<br>10<br>11 | Configure the GPIO26 pin as:<br>GPIO26 - General purpose I/O 26 (default) (I/O)<br>ECAP3 (I/O)<br>EQEP2I (I/O)<br>SPICLKB (I/O) - SPI-B clock                                                                                  |

**Table 1-68. GPIO Port A MUX 2 (GPAMUX2) Register Field Descriptions (continued)**

| Bits  | Field         | Value | Description                                                                               |
|-------|---------------|-------|-------------------------------------------------------------------------------------------|
| 19-18 | GPIO25        | 00    | Configure the GPIO25 pin as:<br>GPIO25 - General purpose I/O 25 (default) (I/O)           |
|       |               | 01    | ECAP2 (I/O)                                                                               |
|       |               | 10    | EQEP2B (I)                                                                                |
|       |               | 11    | SPISOMIB (I/O) - SPI-B Slave Output/Master input                                          |
| 17-16 | GPIO24        | 00    | Configure the GPIO24 pin as:<br>GPIO24 - General purpose I/O 24 (default) (I/O)           |
|       |               | 01    | ECAP1 - eCAP1 (I/O)                                                                       |
|       |               | 10    | EQEP2A (I)                                                                                |
|       |               | 11    | SPISIMOB (I/O) - SPI-B Slave input/Master output                                          |
| 15-14 | GPIO23        | 00    | Configure the GPIO23 pin as:<br>GPIO23 - General purpose I/O 23 (default) (I/O)           |
|       |               | 01    | EQEP1I - eQEP1 index (I/O)                                                                |
|       |               | 10    | MFSXA (I/O)                                                                               |
|       |               | 11    | SCIRXDB (I)                                                                               |
| 13-12 | GPIO22        | 00    | Configure the GPIO22 pin as:<br>GPIO22 - General purpose I/O 22 (default) (I/O)           |
|       |               | 01    | EQEP1S - eQEP1 strobe (I/O)                                                               |
|       |               | 10    | MCLKXA (I/O)                                                                              |
|       |               | 11    | SCITXDB (O)                                                                               |
| 11-10 | GPIO21        | 00    | Configure the GPIO21 pin as:<br>GPIO21 - General purpose I/O 21 (default) (I/O)           |
|       |               | 01    | EQEP1B - eQEP1 input B (I)                                                                |
|       |               | 10    | MDRA (I)                                                                                  |
|       |               | 11    | COMP2OUT (O) - Comparator 2 output                                                        |
| 9-8   | GPIO20        | 00    | Configure the GPIO20 pin as:<br>GPIO20 - General purpose I/O 20 (default) (I/O)           |
|       |               | 01    | EQEP1A - eQEP1 input A (I)                                                                |
|       |               | 10    | MDXA (O)                                                                                  |
|       |               | 11    | COMP1OUT (O) - Comparator 1 output                                                        |
| 7-6   | GPIO19/XCLKIN | 00    | Configure the GPIO19 pin as:<br>GPIO19 - General purpose I/O 19 (default) (I/O) or XCLKIN |
|       |               | 01    | $\overline{\text{SPISTEA}}$ - SPI-A slave transmit enable (I/O)                           |
|       |               | 10    | SCIRXDB (I)                                                                               |
|       |               | 11    | ECAP1 - eCAP1 (I/O)                                                                       |
| 5-4   | GPIO18        | 00    | Configure the GPIO18 pin as:<br>GPIO18 - General purpose I/O 18 (default) (I/O)           |
|       |               | 01    | SPICLKA - SPI-A clock (I/O)                                                               |
|       |               | 10    | SCITXDB (O)                                                                               |
|       |               | 11    | XCLKOUT (O) - External clock output                                                       |
| 3-2   | GPIO17        | 00    | Configure the GPIO17 pin as:<br>GPIO17 - General purpose I/O 17 (default) (I/O)           |
|       |               | 01    | SPISOMIA - SPI-A Slave output/Master input (I/O)                                          |
|       |               | 10    | Reserved                                                                                  |
|       |               | 11    | $\overline{\text{TZ3}}$ - Trip zone 3 (I)                                                 |

**Table 1-68. GPIO Port A MUX 2 (GPAMUX2) Register Field Descriptions (continued)**

| Bits | Field  | Value | Description                                     |
|------|--------|-------|-------------------------------------------------|
| 1-0  | GPIO16 |       | Configure the GPIO16 pin as:                    |
|      |        | 00    | GPIO16 - General purpose I/O 16 (default) (I/O) |
|      |        | 01    | SPISIMOA - SPI-A slave-in, master-out (I/O),    |
|      |        | 10    | Reserved                                        |
|      |        | 11    | TZ2 - Trip zone 2 (I)                           |

31 26 25 24 23 22 21 20 19 18 17 16

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

| Bit   | Field    | Value                | Description                                                                                                                                     |
|-------|----------|----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| 31:26 | Reserved | Reserved             | Reserved                                                                                                                                        |
| 25:24 | GPIO44   | 00<br>01<br>10<br>11 | Configure this pin as:<br>GPIO44 - general purpose I/O 44 (default)<br>MFSRA (I/O)<br>SCIRXDB (I)<br>EPWM7B (O)                                 |
| 23:22 | GPIO43   | 00<br>01<br>10<br>11 | Configure this pin as:<br>GPIO43 - general purpose I/O 43 (default)<br>EPWM8B (O)<br>$\overline{TZ2}$ (I)<br>COMP2OUT (O) - Comparator 2 output |
| 21:20 | GPIO42   | 00<br>01<br>10<br>11 | Configure this pin as:<br>GPIO42 - general purpose I/O 42 (default)<br>EPWM8A (O)<br>$\overline{TZ1}$ (I)<br>COMP1OUT (O) - Comparator 1 output |
| 19:18 | GPIO41   | 00<br>01<br>10<br>11 | Configure this pin as:<br>GPIO41 - general purpose I/O 41 (default)<br>EPWM7B (O) ePWM7 output B (O)<br>SCIRXDB (I)<br>Reserved                 |
| 17:16 | GPIO40   | 00<br>01<br>10<br>11 | Configure this pin as:<br>GPIO40 - general purpose I/O 40 (default)<br>EPWM7A (O) - ePWM7 output A (O)<br>SCITXDB (O)<br>Reserved               |
| 15:14 | GPIO39   | 00<br>01<br>10 or 11 | Configure this pin as:<br>GPIO39 - general purpose I/O 39 (default)<br>Reserved<br>Reserved                                                     |

**Table 1-69. GPIO Port B MUX 1 (GPBMUX1) Register Field Descriptions (continued)**

| Bit   | Field               | Value                | Description                                                                                                                                                                                                                                                    |
|-------|---------------------|----------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 13:12 | GPIO38/XCLKIN (TCK) | 00<br>01<br>10 or 11 | Configure this pin as:<br>GPIO38 - general purpose I/O 38 (default). If $\overline{\text{TRST}} = 1$ , JTAG TCK function is chosen for this pin. This pin can also be used to provide a clock from an external oscillator to the core.<br>Reserved<br>Reserved |
| 11:10 | GPIO37(TDO)         | 00<br>01<br>10 or 11 | Configure this pin as:<br>GPIO37 - general purpose I/O 37 (default). If $\overline{\text{TRST}} = 1$ , JTAG TDO function is chosen for this pin.<br>Reserved<br>Reserved                                                                                       |
| 9:8   | GPIO36(TMS)         | 00<br>01<br>10 or 11 | Configure this pin as:<br>GPIO36 - general purpose I/O 36 (default). If $\overline{\text{TRST}} = 1$ , JTAG TMS function is chosen for this pin.<br>Reserved<br>Reserved                                                                                       |
| 7:6   | GPIO35(TDI)         | 00<br>01<br>10 or 11 | Configure this pin as:<br>GPIO35 - general purpose I/O 35 (default). If $\overline{\text{TRST}} = 1$ , JTAG TDI function is chosen for this pin.<br>Reserved<br>Reserved                                                                                       |
| 5:4   | GPIO34              | 00<br>01<br>10<br>11 | Configure this pin as:<br>GPIO34 - general purpose I/O 34 (default)<br>COMP2OUT (O) - Comparator 2 output<br>Reserved<br>COMP3OUT (O) - Comparator 3 output                                                                                                    |
| 3:2   | GPIO33              | 00<br>01<br>10<br>11 | Configure this pin as:<br>GPIO33 - general purpose I/O 33 (default)<br>SCLA - I <sup>2</sup> C clock open drain bidirectional port (I/O)<br>EPWMSYNCO - External ePWM sync pulse output (O)<br>$\overline{\text{ADCSOCBO}}$ - ADC start-of-conversion B (O)    |
| 1:0   | GPIO32              | 00<br>01<br>10<br>11 | Configure this pin as:<br>GPIO32 - general purpose I/O 32 (default)<br>SDAA - I <sup>2</sup> C data open drain bidirectional port (I/O)<br>EPWMSYNCI - External ePWM sync pulse input (I)<br>$\overline{\text{ADCSOCAO}}$ - ADC start-of-conversion A (O)      |

|          |  |        |  |        |  |        |  |        |  |        |  |          |  |        |  |     |  |    |  |    |  |    |  |   |  |   |  |
|----------|--|--------|--|--------|--|--------|--|--------|--|--------|--|----------|--|--------|--|-----|--|----|--|----|--|----|--|---|--|---|--|
| 31       |  |        |  |        |  |        |  | 22     |  |        |  | 21       |  | 20     |  | 19  |  | 18 |  | 17 |  | 16 |  |   |  |   |  |
| Reserved |  |        |  |        |  |        |  |        |  | GPIO58 |  | GPIO57   |  | GPIO56 |  |     |  |    |  |    |  |    |  |   |  |   |  |
| R-0      |  |        |  |        |  |        |  |        |  | R/W-0  |  | R/W-0    |  | R/W-0  |  |     |  |    |  |    |  |    |  |   |  |   |  |
| 15       |  | 14     |  | 13     |  | 12     |  | 11     |  | 10     |  | 9        |  | 8      |  | 7   |  | 6  |  | 5  |  | 4  |  | 3 |  | 0 |  |
| GPIO55   |  | GPIO54 |  | GPIO53 |  | GPIO52 |  | GPIO51 |  | GPIO50 |  | Reserved |  |        |  |     |  |    |  |    |  |    |  |   |  |   |  |
| R/W-0    |  | R/W-0  |  | R/W-0  |  | R/W-0  |  | R/W-0  |  | R/W-0  |  | R/W-0    |  |        |  | R-0 |  |    |  |    |  |    |  |   |  |   |  |

### Table 1-70. GPIO Port B MUX 2 (GPBMUX2) Register Field Descriptions

Copyright © 2022 Texas Instruments Incorporated

**Table 1-70. GPIO Port B MUX 2 (GPBMUX2) Register Field Descriptions (continued)**

| Bit | Field    | Value | Description                                                          |
|-----|----------|-------|----------------------------------------------------------------------|
| 9:8 | GPIO52   | 00    | Configure this pin as:<br>GPIO52 - general purpose I/O 52 (default). |
|     |          | 01    | EQEP1S (I/O)                                                         |
|     |          | 10    | MCLKXA (I/O)                                                         |
|     |          | 11    | $\overline{TZ3}$ (I)                                                 |
| 7:6 | GPIO51   | 00    | Configure this pin as:<br>GPIO51 - general purpose I/O 51 (default). |
|     |          | 01    | EQEP1B (I)                                                           |
|     |          | 10    | MDRA (I)                                                             |
|     |          | 11    | $\overline{TZ2}$ (I)                                                 |
| 5:4 | GPIO50   | 00    | Configure this pin as:<br>GPIO50 - general purpose I/O 50 (default)  |
|     |          | 01    | EQEP1A (I)                                                           |
|     |          | 10    | MDXA (O)                                                             |
|     |          | 11    | $\overline{TZ1}$ (I)                                                 |
| 3:0 | Reserved |       | Any writes to these bits must always have a value of 0.              |

**Figure 1-69. Analog I/O MUX (AIOMUX1) Register**

|          |    |    |    |         |    |          |    |         |    |          |    |         |  |          |  |    |
|----------|----|----|----|---------|----|----------|----|---------|----|----------|----|---------|--|----------|--|----|
| 31       | 30 | 29 | 28 | 27      | 26 | 25       | 24 | 23      | 22 | 21       | 20 | 19      |  |          |  | 16 |
| Reserved |    |    |    | AIO14   |    | Reserved |    | AIO12   |    | Reserved |    | AIO10   |  | Reserved |  |    |
| R-0      |    |    |    | R/W-1,x |    | R-0      |    | R/W-1,x |    | R-0      |    | R/W-1,x |  | R-0      |  |    |
| 15       | 14 | 13 | 12 | 11      | 10 | 9        | 8  | 7       | 6  | 5        | 4  | 3       |  |          |  | 0  |
| Reserved |    |    |    | AIO6    |    | Reserved |    | AIO4    |    | Reserved |    | AIO2    |  | Reserved |  |    |
| R-0      |    |    |    | R/W-1,x |    | R-0      |    | R/W-1,x |    | R-0      |    | R/W-1,x |  | R-0      |  |    |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 1-71. Analog I/O MUX (AIOMUX1) Register Field Descriptions**

| Bit   | Field    | Value                | Description                                             |
|-------|----------|----------------------|---------------------------------------------------------|
| 31:30 | Reserved |                      | Any writes to these bits must always have a value of 0. |
| 29:28 | AIO14    | 00 or 01<br>10 or 11 | AIO14 enabled<br>AIO14 disabled (default)               |
| 27:26 | Reserved |                      | Any writes to these bits must always have a value of 0. |
| 25:24 | AIO12    | 00 or 01<br>10 or 11 | AIO12 enabled<br>AIO12 disabled (default)               |
| 23:22 | Reserved |                      | Any writes to these bits must always have a value of 0. |
| 21:20 | AIO10    | 00 or 01<br>10 or 11 | AIO10 enabled<br>AIO10 disabled (default)               |
| 19:14 | Reserved |                      | Any writes to these bits must always have a value of 0. |
| 13:12 | AIO6     | 00 or 01<br>10 or 11 | AIO6 enabled<br>AIO6 disabled (default)                 |
| 11:10 | Reserved |                      | Any writes to these bits must always have a value of 0. |
| 9:8   | AIO4     | 00 or 01<br>10 or 11 | AIO4 enabled<br>AIO4 disabled (default)                 |
| 7:6   | Reserved |                      | Any writes to these bits must always have a value of 0. |
| 5:4   | AIO2     | 00 or 01<br>10 or 11 | AIO2 enabled<br>AIO2 disabled (default)                 |
| 3:0   | Reserved |                      | Any writes to these bits must always have a value of 0. |



The GPxCTRL registers specify the sampling period for input pins when configured for input qualification using a window of three or six samples. The sampling period is the amount of time between qualification samples relative to the period of SYSCLKOUT. The number of samples is specified in the GPxQSELn registers.

**Figure 1-70. GPIO Port A Qualification Control (GPACTRL) Register**

|          |    |          |    |
|----------|----|----------|----|
| 31       | 24 | 23       | 16 |
| QUALPRD3 |    | QUALPRD2 |    |
| R/W-0    |    | R/W-0    |    |
| 15       | 8  | 7        | 0  |
| QUALPRD1 |    | QUALPRD0 |    |
| R/W-0    |    | R/W-0    |    |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 1-72. GPIO Port A Qualification Control (GPACTRL) Register Field Descriptions**

| Bits  | Field    | Value | Description                                              |
|-------|----------|-------|----------------------------------------------------------|
| 31-24 | QUALPRD3 |       | Specifies the sampling period for pins GPIO24 to GPIO31. |
|       |          | 0x00  | Sampling Period = $T_{\text{SYSCLKOUT}}$ <sup>(1)</sup>  |
|       |          | 0x01  | Sampling Period = $2 \times T_{\text{SYSCLKOUT}}$        |
|       |          | 0x02  | Sampling Period = $4 \times T_{\text{SYSCLKOUT}}$        |
|       |          | ...   | ...                                                      |
|       |          | 0xFF  | Sampling Period = $510 \times T_{\text{SYSCLKOUT}}$      |
| 23-16 | QUALPRD2 |       | Specifies the sampling period for pins GPIO16 to GPIO23. |
|       |          | 0x00  | Sampling Period = $T_{\text{SYSCLKOUT}}$ <sup>(1)</sup>  |
|       |          | 0x01  | Sampling Period = $2 \times T_{\text{SYSCLKOUT}}$        |
|       |          | 0x02  | Sampling Period = $4 \times T_{\text{SYSCLKOUT}}$        |
|       |          | ...   | ...                                                      |
|       |          | 0xFF  | Sampling Period = $510 \times T_{\text{SYSCLKOUT}}$      |
| 15-8  | QUALPRD1 |       | Specifies the sampling period for pins GPIO8 to GPIO15.  |
|       |          | 0x00  | Sampling Period = $T_{\text{SYSCLKOUT}}$ <sup>(1)</sup>  |
|       |          | 0x01  | Sampling Period = $2 \times T_{\text{SYSCLKOUT}}$        |
|       |          | 0x02  | Sampling Period = $4 \times T_{\text{SYSCLKOUT}}$        |
|       |          | ...   | ...                                                      |
|       |          | 0xFF  | Sampling Period = $510 \times T_{\text{SYSCLKOUT}}$      |
| 7-0   | QUALPRD0 |       | Specifies the sampling period for pins GPIO0 to GPIO7.   |
|       |          | 0x00  | Sampling Period = $T_{\text{SYSCLKOUT}}$ <sup>(1)</sup>  |
|       |          | 0x01  | Sampling Period = $2 \times T_{\text{SYSCLKOUT}}$        |
|       |          | 0x02  | Sampling Period = $4 \times T_{\text{SYSCLKOUT}}$        |
|       |          | ...   | ...                                                      |
|       |          | 0xFF  | Sampling Period = $510 \times T_{\text{SYSCLKOUT}}$      |

(1)  $T_{\text{SYSCLKOUT}}$  indicates the period of SYSCLKOUT.

**Figure 1-71. GPIO Port B Qualification Control (GPBCTRL) Register**

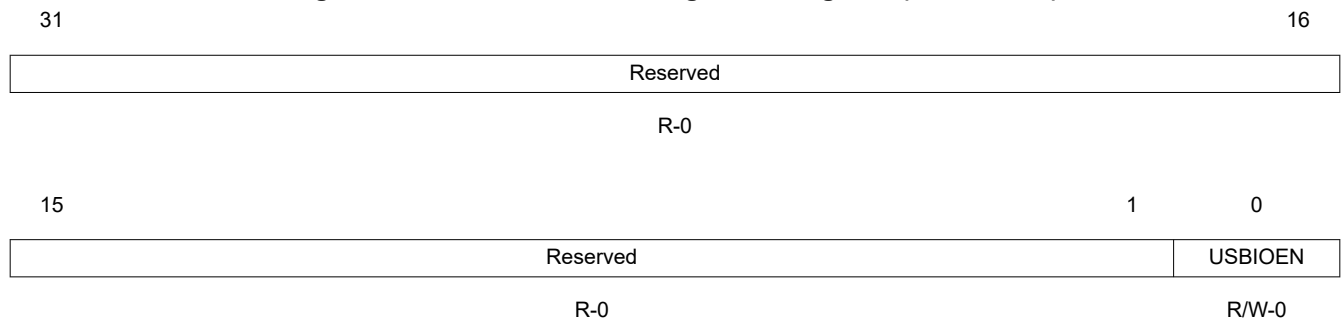
|          |    |          |    |
|----------|----|----------|----|
| 31       | 24 | 23       | 16 |
| QUALPRD3 |    | QUALPRD2 |    |
| R/W-0    |    | R/W-0    |    |
| 15       | 8  | 7        | 0  |
| QUALPRD1 |    | QUALPRD0 |    |
| R/W-0    |    | R/W-0    |    |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 1-73. GPIO Port B Qualification Control (GPBCTRL) Register Field Descriptions**

| Bits  | Field    | Value | Description                                             |
|-------|----------|-------|---------------------------------------------------------|
| 31-24 | QUALPRD3 |       | Specifies the sampling period for pins GPIO56 to GPIO58 |
|       |          | 0x00  | Sampling Period = $T_{\text{SYSCLKOUT}}$ <sup>(1)</sup> |
|       |          | 0x01  | Sampling Period = $2 \times T_{\text{SYSCLKOUT}}$       |
|       |          | 0x02  | Sampling Period = $4 \times T_{\text{SYSCLKOUT}}$       |
|       |          | ...   | ...                                                     |
|       |          | 0xFF  | Sampling Period = $510 \times T_{\text{SYSCLKOUT}}$     |
| 23-16 | QUALPRD2 |       | Specifies the sampling period for pins GPIO50 to GPIO55 |
|       |          | 0xFF  | Sampling Period = $510 \times T_{\text{SYSCLKOUT}}$     |
|       |          | 0x00  | Sampling Period = $T_{\text{SYSCLKOUT}}$ <sup>(1)</sup> |
|       |          | 0x01  | Sampling Period = $2 \times T_{\text{SYSCLKOUT}}$       |
|       |          | 0x02  | Sampling Period = $4 \times T_{\text{SYSCLKOUT}}$       |
|       |          | ...   | ...                                                     |
| 15-8  | QUALPRD1 |       | Specifies the sampling period for pins GPIO40 to GPIO44 |
|       |          | 0xFF  | Sampling Period = $510 \times T_{\text{SYSCLKOUT}}$     |
|       |          | 0x00  | Sampling Period = $T_{\text{SYSCLKOUT}}$ <sup>(1)</sup> |
|       |          | 0x01  | Sampling Period = $2 \times T_{\text{SYSCLKOUT}}$       |
|       |          | 0x02  | Sampling Period = $4 \times T_{\text{SYSCLKOUT}}$       |
|       |          | ...   | ...                                                     |
| 7-0   | QUALPRD0 |       | Specifies the sampling period for pins GPIO32 to GPIO39 |
|       |          | 0xFF  | Sampling Period = $510 \times T_{\text{SYSCLKOUT}}$     |
|       |          | 0x00  | Sampling Period = $T_{\text{SYSCLKOUT}}$ <sup>(1)</sup> |
|       |          | 0x01  | Sampling Period = $2 \times T_{\text{SYSCLKOUT}}$       |
|       |          | 0x02  | Sampling Period = $4 \times T_{\text{SYSCLKOUT}}$       |
|       |          | ...   | ...                                                     |
|       |          | 0xFF  | Sampling Period = $510 \times T_{\text{SYSCLKOUT}}$     |

(1)  $T_{\text{SYSCLKOUT}}$  indicates the period of SYSCLKOUT.

**Figure 1-72. GPIO A Control Register 2 Register (GPACTRL2)**


LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 1-74. GPIO A Control Register 2 Register (GPACTRL2) Field Descriptions**

| Bits | Field    | Value | Description                                                                                  |
|------|----------|-------|----------------------------------------------------------------------------------------------|
| 31-1 | Reserved |       | Any writes to these bits must always have a value of 0.                                      |
| 0    | USBIOEN  |       | USB I/O Enable Bit                                                                           |
|      |          | 0     | USB0DP and USB0DM pins are controlled by GPIO Mux register settings. USBPHY is powered down. |
|      |          | 1     | USB0DP and USB0DM pins configured as USB function. GPIO function is disabled.                |

**Figure 1-73. GPIO Port A Qualification Select 1 (GPAQSEL1) Register**

|        |    |        |    |        |    |        |    |        |    |        |    |       |    |       |    |
|--------|----|--------|----|--------|----|--------|----|--------|----|--------|----|-------|----|-------|----|
| 31     | 30 | 29     | 28 | 27     | 26 | 25     | 24 | 23     | 22 | 21     | 20 | 19    | 18 | 17    | 16 |
| GPIO15 |    | GPIO14 |    | GPIO13 |    | GPIO12 |    | GPIO11 |    | GPIO10 |    | GPIO9 |    | GPIO8 |    |
| R/W-0  |    | R/W-0  |    | R/W-0  |    | R/W-0  |    | R/W-0  |    | R/W-0  |    | R/W-0 |    | R/W-0 |    |
| 15     | 14 | 13     | 12 | 11     | 10 | 9      | 8  | 7      | 6  | 5      | 4  | 3     | 2  | 1     | 0  |
| GPIO7  |    | GPIO6  |    | GPIO5  |    | GPIO4  |    | GPIO3  |    | GPIO2  |    | GPIO1 |    | GPIO0 |    |
| R/W-0  |    | R/W-0  |    | R/W-0  |    | R/W-0  |    | R/W-0  |    | R/W-0  |    | R/W-0 |    | R/W-0 |    |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 1-75. GPIO Port A Qualification Select 1 (GPAQSEL1) Register Field Descriptions**

| Bits | Field        | Value                | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|------|--------------|----------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 31-0 | GPIO15-GPIO0 | 00<br>01<br>10<br>11 | Select input qualification type for GPIO0 to GPIO15. The input qualification of each GPIO input is controlled by two bits as shown in <a href="#">Figure 1-73</a> .<br>Synchronize to SYSCLKOUT only. Valid for both peripheral and GPIO pins.<br>Qualification using 3 samples. Valid for pins configured as GPIO or a peripheral function. The time between samples is specified in the GPACTRL register.<br>Qualification using 6 samples. Valid for pins configured as GPIO or a peripheral function. The time between samples is specified in the GPACTRL register.<br>Asynchronous. (no synchronization or qualification). This option applies to pins configured as peripherals only. If the pin is configured as a GPIO input, then this option is the same as 0,0 or synchronize to SYSCLKOUT. |

**Figure 1-74. GPIO Port A Qualification Select 2 (GPAQSEL2) Register**

|        |        |        |        |        |        |        |        |    |    |    |    |    |    |    |    |
|--------|--------|--------|--------|--------|--------|--------|--------|----|----|----|----|----|----|----|----|
| 31     | 30     | 29     | 28     | 27     | 26     | 25     | 24     | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| GPIO31 | GPIO30 | GPIO29 | GPIO28 | GPIO27 | GPIO26 | GPIO25 | GPIO24 |    |    |    |    |    |    |    |    |
| R/W-0  | R/W-0  | R/W-0  | R/W-0  | R/W-0  | R/W-0  | R/W-0  | R/W-0  |    |    |    |    |    |    |    |    |
| 15     | 14     | 13     | 12     | 11     | 10     | 9      | 8      | 7  | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
| GPIO23 | GPIO22 | GPIO21 | GPIO20 | GPIO19 | GPIO18 | GPIO17 | GPIO16 |    |    |    |    |    |    |    |    |
| R/W-0  | R/W-0  | R/W-0  | R/W-0  | R/W-0  | R/W-0  | R/W-0  | R/W-0  |    |    |    |    |    |    |    |    |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 1-76. GPIO Port A Qualification Select 2 (GPAQSEL2) Register Field Descriptions**

| Bits | Field         | Value | Description                                                                                                                                                                                                                 |
|------|---------------|-------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 31-0 | GPIO31-GPIO16 |       | Select input qualification type for GPIO16 to GPIO31. The input qualification of each GPIO input is controlled by two bits as shown in <a href="#">Figure 1-74</a> .                                                        |
|      |               | 00    | Synchronize to SYSCLKOUT only. Valid for both peripheral and GPIO pins.                                                                                                                                                     |
|      |               | 01    | Qualification using 3 samples. Valid for pins configured as GPIO or a peripheral function. The time between samples is specified in the GPACTRL register.                                                                   |
|      |               | 10    | Qualification using 6 samples. Valid for pins configured as GPIO or a peripheral function. The time between samples is specified in the GPACTRL register.                                                                   |
|      |               | 11    | Asynchronous. (no synchronization or qualification). This option applies to pins configured as peripherals only. If the pin is configured as a GPIO input, then this option is the same as 0,0 or synchronize to SYSCLKOUT. |

31 26 25 24 23 22 21 20 19 18 17 16

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

| Bits   | Field          | Value | Description                                                                                                                                                                                                                 |
|--------|----------------|-------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 31- 26 | Reserved       |       | Any writes to these bits must always have a value of 0.                                                                                                                                                                     |
| 25-0   | GPIO 44-GPIO32 |       | Select input qualification type for GPIO32 to GPIO44. The input qualification of each GPIO input is controlled by two bits as shown in <a href="#">Figure 1-75</a> .                                                        |
|        |                | 00    | Synchronize to SYSCLKOUT only. Valid for both peripheral and GPIO pins.                                                                                                                                                     |
|        |                | 01    | Qualification using 3 samples. Valid for pins configured as GPIO or a peripheral function. The time between samples is specified in the GPACTRL register.                                                                   |
|        |                | 10    | Qualification using 6 samples. Valid for pins configured as GPIO or a peripheral function. The time between samples is specified in the GPACTRL register.                                                                   |
|        |                | 11    | Asynchronous. (no synchronization or qualification). This option applies to pins configured as peripherals only. If the pin is configured as a GPIO input, then this option is the same as 0,0 or synchronize to SYSCLKOUT. |

**Figure 1-76. GPIO Port B Qualification Select 2 (GPBQSEL2) Register**

[illegible]

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 1-78. GPIO Port B Qualification Select 2 (GPBQSEL2) Register Field Descriptions**

| Bits   | Field         | Value | Description                                                                                                                                                                                                                 |
|--------|---------------|-------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 31- 22 | Reserved      |       | Any writes to these bits must always have a value of 0.                                                                                                                                                                     |
| 21-0   | GPIO58-GPIO50 |       | Select input qualification type for GPIO58 to GPIO50. The input qualification of each GPIO input is controlled by two bits as shown in <a href="#">Figure 1-76</a> .                                                        |
|        |               | 00    | Synchronize to SYSCLKOUT only. Valid for both peripheral and GPIO pins.                                                                                                                                                     |
|        |               | 01    | Qualification using 3 samples. Valid for pins configured as GPIO or a peripheral function. The time between samples is specified in the GPACTRL register.                                                                   |
|        |               | 10    | Qualification using 6 samples. Valid for pins configured as GPIO or a peripheral function. The time between samples is specified in the GPACTRL register.                                                                   |
|        |               | 11    | Asynchronous. (no synchronization or qualification). This option applies to pins configured as peripherals only. If the pin is configured as a GPIO input, then this option is the same as 0,0 or synchronize to SYSCLKOUT. |

The GPADIR and GPBDIR registers control the direction of the pins when they are configured as a GPIO in the appropriate MUX register. The direction register has no effect on pins configured as peripheral functions.

**Figure 1-77. GPIO Port A Direction (GPADIR) Register**

|        |        |        |        |        |        |        |        |
|--------|--------|--------|--------|--------|--------|--------|--------|
| 31     | 30     | 29     | 28     | 27     | 26     | 25     | 24     |
| GPIO31 | GPIO30 | GPIO29 | GPIO28 | GPIO27 | GPIO26 | GPIO25 | GPIO24 |
| R/W-0  | R/W-0  | R/W-0  | R/W-0  | R/W-0  | R/W-0  | R/W-0  | R/W-0  |
| 23     | 22     | 21     | 20     | 19     | 18     | 17     | 16     |
| GPIO23 | GPIO22 | GPIO21 | GPIO20 | GPIO19 | GPIO18 | GPIO17 | GPIO16 |
| R/W-0  | R/W-0  | R/W-0  | R/W-0  | R/W-0  | R/W-0  | R/W-0  | R/W-0  |
| 15     | 14     | 13     | 12     | 11     | 10     | 9      | 8      |
| GPIO15 | GPIO14 | GPIO13 | GPIO12 | GPIO11 | GPIO10 | GPIO9  | GPIO8  |
| R/W-0  | R/W-0  | R/W-0  | R/W-0  | R/W-0  | R/W-0  | R/W-0  | R/W-0  |
| 7      | 6      | 5      | 4      | 3      | 2      | 1      | 0      |
| GPIO7  | GPIO6  | GPIO5  | GPIO4  | GPIO3  | GPIO2  | GPIO1  | GPIO0  |
| R/W-0  | R/W-0  | R/W-0  | R/W-0  | R/W-0  | R/W-0  | R/W-0  | R/W-0  |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 1-79. GPIO Port A Direction (GPADIR) Register Field Descriptions**

| Bits | Field        | Value | Description                                                                                                                                                                                                                                          |
|------|--------------|-------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 31-0 | GPIO31-GPIO0 | 0     | Controls direction of GPIO Port A pins when the specified pin is configured as a GPIO in the appropriate GPAMUX1 or GPAMUX2 register.<br>Configures the GPIO pin as an input. (default)                                                              |
|      |              | 1     | Configures the GPIO pin as an output<br>The value currently in the GPADAT output latch is driven on the pin. To initialize the GPADAT latch prior to changing the pin from an input to an output, use the GPASET, GPACLEAR, and GPATOGGLE registers. |



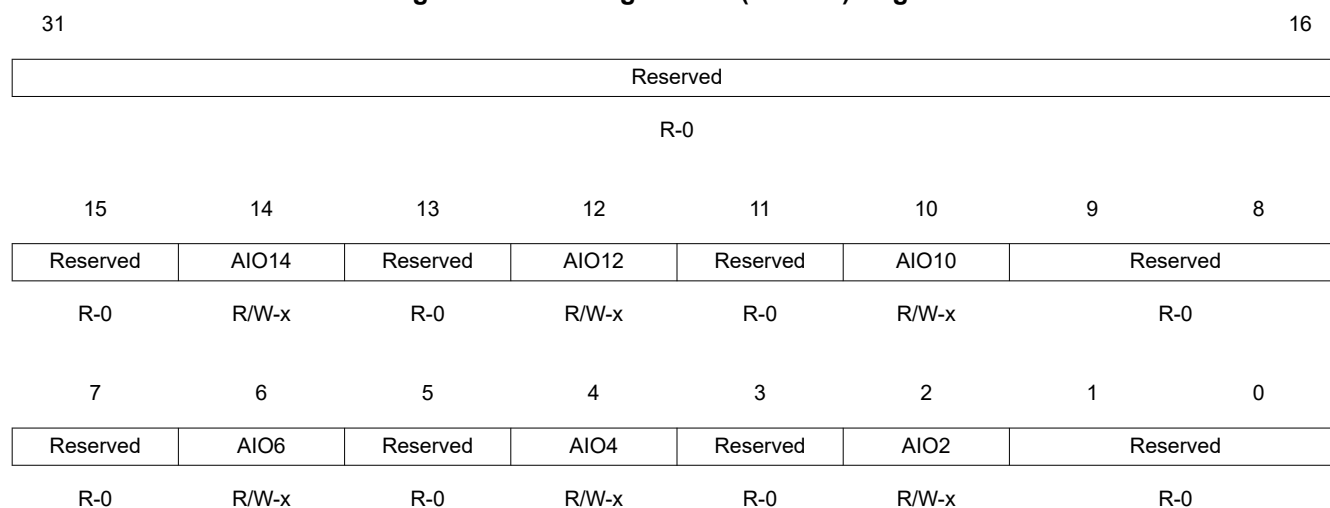
**Figure 1-78. GPIO Port B Direction (GPBDIR) Register**

|          |        |        |        |        |        |          |        |
|----------|--------|--------|--------|--------|--------|----------|--------|
| 31       |        |        |        | 27     | 26     | 25       | 24     |
| Reserved |        |        |        |        | GPIO58 | GPIO57   | GPIO56 |
| R-0      |        |        |        |        | R/W-0  | R/W-0    | R/W-0  |
| 23       | 22     | 21     | 20     | 19     | 18     | 17       | 16     |
| GPIO55   | GPIO54 | GPIO53 | GPIO52 | GPIO51 | GPIO50 | Reserved |        |
| R/W-0    | R/W-0  | R/W-0  | R/W-0  | R/W-0  | R/W-0  | R-0      |        |
| 15       |        | 13     | 12     | 11     | 10     | 9        | 8      |
| Reserved |        |        | GPIO44 | GPIO43 | GPIO42 | GPIO41   | GPIO40 |
| R-0      |        |        | R/W-0  | R/W-0  | R/W-0  | R/W-0    | R/W-0  |
| 7        | 6      | 5      | 4      | 3      | 2      | 1        | 0      |
| GPIO39   | GPIO38 | GPIO37 | GPIO36 | GPIO35 | GPIO34 | GPIO33   | GPIO32 |
| R/W-0    | R/W-0  | R/W-0  | R/W-0  | R/W-0  | R/W-0  | R/W-0    | R/W-0  |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 1-80. GPIO Port B Direction (GPBDIR) Register Field Descriptions**

| Bits  | Field         | Value | Description                                                                                                                        |
|-------|---------------|-------|------------------------------------------------------------------------------------------------------------------------------------|
| 31-27 | Reserved      |       | Any writes to these bits must always have a value of 0.                                                                            |
| 26-18 | GPIO58-GPIO50 |       | Controls direction of GPIO pin when GPIO mode is selected. Reading the register returns the current value of the register setting. |
|       |               | 0     | Configures the GPIO pin as an input. (default)                                                                                     |
|       |               | 1     | Configures the GPIO pin as an output                                                                                               |
| 17-13 | Reserved      |       | Any writes to these bits must always have a value of 0.                                                                            |
| 12-0  | GPIO44-GPIO32 |       | Controls direction of GPIO pin when GPIO mode is selected. Reading the register returns the current value of the register setting. |
|       |               | 0     | Configures the GPIO pin as an input. (default)                                                                                     |
|       |               | 1     | Configures the GPIO pin as an output                                                                                               |

**Figure 1-79. Analog I/O DIR (AIODIR) Register**


LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 1-81. Analog I/O DIR (AIODIR) Register Field Descriptions**

| Bit   | Field            | Value | Description                                                                                                                                   |
|-------|------------------|-------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| 31:15 | Reserved         |       | Any writes to these bits must always have a value of 0.                                                                                       |
| 14:0  | AIO <sub>n</sub> |       | Controls direction of the available AIO pin when AIO mode is selected. Reading the register returns the current value of the register setting |
|       |                  | 0     | Configures the AIO pin as an input. (default)                                                                                                 |
|       |                  | 1     | Configures the AIO pin as an output                                                                                                           |

The pullup disable (GPxPUD) registers allow you to specify which pins should have an internal pullup resister enabled. The internal pullups on the pins that can be configured as ePWM outputs (GPIO0-GPIO11) are all disabled asynchronously when the external reset signal (XRS) is low. The internal pullups on all other pins are enabled on reset. When coming out of reset, the pullups remain in their default state until you enable or disable them selectively in software by writing to this register. The pullup configuration applies both to pins configured as I/O and those configured as peripheral functions.

**Figure 1-80. GPIO Port A Pullup Disable (GPAPUD) Registers**

|        |        |        |        |        |        |        |        |
|--------|--------|--------|--------|--------|--------|--------|--------|
| 31     | 30     | 29     | 28     | 27     | 26     | 25     | 24     |
| GPIO31 | GPIO30 | GPIO29 | GPIO28 | GPIO27 | GPIO26 | GPIO25 | GPIO24 |
| R/W-0  | R/W-0  | R/W-0  | R/W-0  | R/W-0  | R/W-0  | R/W-0  | R/W-0  |
| 23     | 22     | 21     | 20     | 19     | 18     | 17     | 16     |
| GPIO23 | GPIO22 | GPIO21 | GPIO20 | GPIO19 | GPIO18 | GPIO17 | GPIO16 |
| R/W-0  | R/W-0  | R/W-0  | R/W-0  | R/W-0  | R/W-0  | R/W-0  | R/W-0  |
| 15     | 14     | 13     | 12     | 11     | 10     | 9      | 8      |
| GPIO15 | GPIO14 | GPIO13 | GPIO12 | GPIO11 | GPIO10 | GPIO9  | GPIO8  |
| R/W-0  | R/W-0  | R/W-0  | R/W-0  | R/W-1  | R/W-1  | R/W-1  | R/W-1  |
| 7      | 6      | 5      | 4      | 3      | 2      | 1      | 0      |
| GPIO7  | GPIO6  | GPIO5  | GPIO4  | GPIO3  | GPIO2  | GPIO1  | GPIO0  |
| R/W-1  | R/W-1  | R/W-1  | R/W-1  | R/W-1  | R/W-1  | R/W-1  | R/W-1  |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 1-82. GPIO Port A Internal Pullup Disable (GPAPUD) Register Field Descriptions**

| Bits | Field        | Value | Description                                                                                                                    |
|------|--------------|-------|--------------------------------------------------------------------------------------------------------------------------------|
| 31-0 | GPIO31-GPIO0 |       | Configure the internal pullup resister on the selected GPIO Port A pin. Each GPIO pin corresponds to one bit in this register. |
|      |              | 0     | Enable the internal pullup on the specified pin. (default for GPIO12-GPIO31)                                                   |
|      |              | 1     | Disable the internal pullup on the specified pin. (default for GPIO0-GPIO11)                                                   |

**Figure 1-81. GPIO Port B Pullup Disable (GPBPUD) Registers**

|          |        |        |        |        |        |          |        |
|----------|--------|--------|--------|--------|--------|----------|--------|
| 31       |        |        |        | 27     | 26     | 25       | 24     |
| Reserved |        |        |        |        | GPIO58 | GPIO57   | GPIO56 |
| R-0      |        |        |        |        | R/W-0  | R/W-0    | R/W-0  |
| 23       | 22     | 21     | 20     | 19     | 18     | 17       | 16     |
| GPIO55   | GPIO54 | GPIO53 | GPIO52 | GPIO51 | GPIO50 | Reserved |        |
| R/W-0    | R/W-0  | R/W-0  | R/W-0  | R/W-0  | R/W-0  | R-0      |        |
| 15       |        | 13     | 12     | 11     | 10     | 9        | 8      |
| Reserved |        |        | GPIO44 | GPIO43 | GPIO42 | GPIO41   | GPIO40 |
| R-0      |        |        | R/W-0  | R/W-1  | R/W-1  | R/W-1    | R/W-1  |
| 7        | 6      | 5      | 4      | 3      | 2      | 1        | 0      |
| GPIO39   | GPIO38 | GPIO37 | GPIO36 | GPIO35 | GPIO34 | GPIO33   | GPIO32 |
| R/W-0    | R/W-0  | R/W-0  | R/W-0  | R/W-0  | R/W-0  | R/W-0    | R/W-0  |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 1-83. GPIO Port B Internal Pullup Disable (GPBPUD) Register Field Descriptions**

| Bits   | Field         | Value | Description                                                                                                                    |
|--------|---------------|-------|--------------------------------------------------------------------------------------------------------------------------------|
| 31- 27 | Reserved      |       | Any writes to these bits must always have a value of 0.                                                                        |
| 26-18  | GPIO58-GPIO50 |       | Configure the internal pullup resister on the selected GPIO Port B pin. Each GPIO pin corresponds to one bit in this register. |
|        |               | 0     | Enable the internal pullup on the specified pin. (default for GPIO58-GPIO50))                                                  |
|        |               | 1     | Disable the internal pullup on the specified pin.                                                                              |
| 17-13  | Reserved      |       | Any writes to these bits must always have a value of 0.                                                                        |
| 12-8   | GPIO44-GPIO40 |       | Configure the internal pullup resister on the selected GPIO Port B pin. Each GPIO pin corresponds to one bit in this register. |
|        |               | 0     | Enable the internal pullup on the specified pin. (default for GPIO44)                                                          |
|        |               | 1     | Disable the internal pullup on the specified pin. (default for GPIO43-GPIO40)                                                  |
| 7-0    | GPIO39-GPIO32 |       | Configure the internal pullup resister on the selected GPIO Port B pin. Each GPIO pin corresponds to one bit in this register. |
|        |               | 0     | Enable the internal pullup on the specified pin. (default for GPIO39-GPIO32)                                                   |
|        |               | 1     | Disable the internal pullup on the specified pin                                                                               |

The GPIO data registers indicate the current status of the GPIO pin, irrespective of which mode the pin is in. Writing to this register will set the respective GPIO pin high or low if the pin is enabled as a GPIO output, otherwise the value written is latched but ignored. The state of the output register latch will remain in its current state until the next write operation. A reset will clear all bits and latched values to zero. The value read from the GPxDAT registers reflect the state of the pin (after qualification), not the state of the output latch of the GPxDAT register.

Typically the DAT registers are used for reading the current state of the pins. To easily modify the output level of the pin refer to the SET, CLEAR and TOGGLE registers.

**Figure 1-82. GPIO Port A Data (GPADAT) Register**

|        |        |        |        |        |        |        |        |
|--------|--------|--------|--------|--------|--------|--------|--------|
| 31     | 30     | 29     | 28     | 27     | 26     | 25     | 24     |
| GPIO31 | GPIO30 | GPIO29 | GPIO28 | GPIO27 | GPIO26 | GPIO25 | GPIO24 |
| R/W-x  | R/W-x  | R/W-x  | R/W-x  | R/W-x  | R/W-x  | R/W-x  | R/W-x  |
| 23     | 22     | 21     | 20     | 19     | 18     | 17     | 16     |
| GPIO23 | GPIO22 | GPIO21 | GPIO20 | GPIO19 | GPIO18 | GPIO17 | GPIO16 |
| R/W-x  | R/W-x  | R/W-x  | R/W-x  | R/W-x  | R/W-x  | R/W-x  | R/W-x  |
| 15     | 14     | 13     | 12     | 11     | 10     | 9      | 8      |
| GPIO15 | GPIO14 | GPIO13 | GPIO12 | GPIO11 | GPIO10 | GPIO9  | GPIO8  |
| R/W-x  | R/W-x  | R/W-x  | R/W-x  | R/W-x  | R/W-x  | R/W-x  | R/W-x  |
| 7      | 6      | 5      | 4      | 3      | 2      | 1      | 0      |
| GPIO7  | GPIO6  | GPIO5  | GPIO4  | GPIO3  | GPIO2  | GPIO1  | GPIO0  |
| R/W-x  | R/W-x  | R/W-x  | R/W-x  | R/W-x  | R/W-x  | R/W-x  | R/W-x  |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset<sup>(1)</sup>

(1) x = The state of the GPADAT register is unknown after reset. It depends on the level of the pin after reset.

**Table 1-84. GPIO Port A Data (GPADAT) Register Field Descriptions**

| Bits | Field        | Value | Description                                                                                                                                                                                                                                                                                                                                                                                                                        |
|------|--------------|-------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 31-0 | GPIO31-GPIO0 | 0     | Each bit corresponds to one GPIO port A pin (GPIO0-GPIO31) as shown in <a href="#">Figure 1-82</a> .<br>Reading a 0 indicates that the state of the pin is currently low, irrespective of the mode the pin is configured for.<br>Writing a 0 will force an output of 0 if the pin is configured as a GPIO output in the appropriate GPAMUX1/2 and GPADIR registers; otherwise, the value is latched but not used to drive the pin. |
|      |              | 1     | Reading a 1 indicates that the state of the pin is currently high irrespective of the mode the pin is configured for.<br>Writing a 1 will force an output of 1 if the pin is configured as a GPIO output in the appropriate GPAMUX1/2 and GPADIR registers; otherwise, the value is latched but not used to drive the pin.                                                                                                         |

**Figure 1-83. GPIO Port B Data (GPBDAT) Register**

|          |        |        |        |        |        |          |        |
|----------|--------|--------|--------|--------|--------|----------|--------|
| 31       |        |        |        | 27     | 26     | 25       | 24     |
| Reserved |        |        |        |        | GPIO58 | GPIO57   | GPIO56 |
| R-0      |        |        |        |        | R/W-0  | R/W-0    | R/W-0  |
| 23       | 22     | 21     | 20     | 19     | 18     | 17       | 16     |
| GPIO55   | GPIO54 | GPIO53 | GPIO52 | GPIO51 | GPIO50 | Reserved |        |
| R/W-0    | R/W-0  | R/W-0  | R/W-0  | R/W-0  | R/W-0  | R-0      |        |
| 15       |        | 13     | 12     | 11     | 10     | 9        | 8      |
| Reserved |        |        | GPIO44 | GPIO43 | GPIO42 | GPIO41   | GPIO40 |
| R-0      |        |        | R/W-0  | R/W-0  | R/W-0  | R/W-0    | R/W-0  |
| 7        | 6      | 5      | 4      | 3      | 2      | 1        | 0      |
| GPIO39   | GPIO38 | GPIO37 | GPIO36 | GPIO35 | GPIO34 | GPIO33   | GPIO32 |
| R/W-0    | R/W-0  | R/W-0  | R/W-0  | R/W-0  | R/W-0  | R/W-0    | R/W-0  |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 1-85. GPIO Port B Data (GPBDAT) Register Field Descriptions**

| Bit   | Field          | Value | Description                                                                                                                                                                                                                                                                                                                                                                                                                       |
|-------|----------------|-------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 31-27 | Reserved       |       | Any writes to these bits must always have a value of 0.                                                                                                                                                                                                                                                                                                                                                                           |
| 26-18 | GPIO 58-GPIO50 | 0     | Each bit corresponds to one GPIO port B pin (GPIO58-GPIO50) as shown in <a href="#">Figure 1-83</a> .<br>Reading a 0 indicates that the state of the pin is currently low, irrespective of the mode the pin is configured for.<br>Writing a 0 will force an output of 0 if the pin is configured as a GPIO output in the appropriate GPBMUX1 and GPBDIR registers; otherwise, the value is latched but not used to drive the pin. |
|       |                | 1     | Reading a 1 indicates that the state of the pin is currently high irrespective of the mode the pin is configured for.<br>Writing a 1 will force an output of 1 if the pin is configured as a GPIO output in the GPBMUX1 and GPBDIR registers; otherwise, the value is latched but not used to drive the pin.                                                                                                                      |
| 17-13 | Reserved       |       | Any writes to these bits must always have a value of 0.                                                                                                                                                                                                                                                                                                                                                                           |
| 12-0  | GPIO44-GPIO32  | 0     | Each bit corresponds to one GPIO port B pin (GPIO44-GPIO32) as shown in <a href="#">Figure 1-83</a> .<br>Reading a 0 indicates that the state of the pin is currently low, irrespective of the mode the pin is configured for.<br>Writing a 0 will force an output of 0 if the pin is configured as a GPIO output in the appropriate GPBMUX1 and GPBDIR registers; otherwise, the value is latched but not used to drive the pin. |
|       |                | 1     | Reading a 1 indicates that the state of the pin is currently high irrespective of the mode the pin is configured for.<br>Writing a 1 will force an output of 1 if the pin is configured as a GPIO output in the GPBMUX1 and GPBDIR registers; otherwise, the value is latched but not used to drive the pin.                                                                                                                      |

**Figure 1-84. Analog I/O DAT (AIODAT) Register**

31

16

|          |       |          |       |          |       |          |   |
|----------|-------|----------|-------|----------|-------|----------|---|
| Reserved |       |          |       |          |       |          |   |
| R-0      |       |          |       |          |       |          |   |
| 15       | 14    | 13       | 12    | 11       | 10    | 9        | 8 |
| Reserved | AIO14 | Reserved | AIO12 | Reserved | AIO10 | Reserved |   |
| R-0      | R/W-x | R-0      | R/W-x | R-0      | R/W-x | R-0      |   |
| 7        | 6     | 5        | 4     | 3        | 2     | 1        | 0 |
| Reserved | AIO6  | Reserved | AIO4  | Reserved | AIO2  | Reserved |   |
| R-0      | R/W-x | R-0      | R/W-x | R-0      | R/W-x | R-0      |   |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 1-86. Analog I/O DAT (AIODAT) Register Field Descriptions**

| Bit   | Field            | Value | Description                                                                                                                                                                                                                                                                                                                                      |
|-------|------------------|-------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 31:15 | Reserved         |       | Any writes to these bits must always have a value of 0.                                                                                                                                                                                                                                                                                          |
| 14:0  | AIO <sub>n</sub> | 0     | Each bit corresponds to one AIO port pin<br>Reading a 0 indicates that the state of the pin is currently low, irrespective of the mode the pin is configured for.<br>Writing a 0 will force an output of 0 if the pin is configured as a AIO output in the appropriate registers; otherwise, the value is latched but not used to drive the pin. |
|       |                  | 1     | Reading a 1 indicates that the state of the pin is currently high irrespective of the mode the pin is configured for.<br>Writing a 1 will force an output of 1 if the pin is configured as a AIO output in the appropriate registers; otherwise, the value is latched but not used to drive the pin.                                             |

**Figure 1-85. GPIO Port A Set, Clear and Toggle (GPASET, GPACLEAR, GPATOGGLE) Registers**

|        |        |        |        |        |        |        |        |
|--------|--------|--------|--------|--------|--------|--------|--------|
| 31     | 30     | 29     | 28     | 27     | 26     | 25     | 24     |
| GPIO31 | GPIO30 | GPIO29 | GPIO28 | GPIO27 | GPIO26 | GPIO25 | GPIO24 |
| R/W-0  | R/W-0  | R/W-0  | R/W-0  | R/W-0  | R/W-0  | R/W-0  | R/W-0  |
| 23     | 22     | 21     | 20     | 19     | 18     | 17     | 16     |
| GPIO23 | GPIO22 | GPIO21 | GPIO20 | GPIO19 | GPIO18 | GPIO17 | GPIO16 |
| R/W-0  | R/W-0  | R/W-0  | R/W-0  | R/W-0  | R/W-0  | R/W-0  | R/W-0  |
| 15     | 14     | 13     | 12     | 11     | 10     | 9      | 8      |
| GPIO15 | GPIO14 | GPIO13 | GPIO12 | GPIO11 | GPIO10 | GPIO9  | GPIO8  |
| R/W-0  | R/W-0  | R/W-0  | R/W-0  | R/W-0  | R/W-0  | R/W-0  | R/W-0  |
| 7      | 6      | 5      | 4      | 3      | 2      | 1      | 0      |
| GPIO7  | GPIO6  | GPIO5  | GPIO4  | GPIO3  | GPIO2  | GPIO1  | GPIO0  |
| R/W-0  | R/W-0  | R/W-0  | R/W-0  | R/W-0  | R/W-0  | R/W-0  | R/W-0  |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 1-87. GPIO Port A Set (GPASET) Register Field Descriptions**

| Bits | Field        | Value | Description                                                                                                                                                                                                                             |
|------|--------------|-------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 31-0 | GPIO31-GPIO0 | 0     | Each GPIO port A pin (GPIO0-GPIO31) corresponds to one bit in this register as shown in <a href="#">Figure 1-85</a> .<br>Writes of 0 are ignored. This register always reads back a 0.                                                  |
|      |              | 1     | Writing a 1 forces the respective output data latch to high. If the pin is configured as a GPIO output then it will be driven high. If the pin is not configured as a GPIO output then the latch is set high but the pin is not driven. |

**Table 1-88. GPIO Port A Clear (GPACLEAR) Register Field Descriptions**

| Bits | Field          | Value | Description                                                                                                                                                                                                                          |
|------|----------------|-------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 31-0 | GPIO31 - GPIO0 | 0     | Each GPIO port A pin (GPIO0-GPIO31) corresponds to one bit in this register as shown in <a href="#">Figure 1-85</a> .<br>Writes of 0 are ignored. This register always reads back a 0.                                               |
|      |                | 1     | Writing a 1 forces the respective output data latch to low. If the pin is configured as a GPIO output then it will be driven low. If the pin is not configured as a GPIO output then the latch is cleared but the pin is not driven. |



**Table 1-89. GPIO Port A Toggle (GPATOGGLE) Register Field Descriptions**

| Bits | Field        | Value | Description                                                                                                                                                                                                                                                                                               |
|------|--------------|-------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 31-0 | GPIO31-GPIO0 |       | Each GPIO port A pin (GPIO0-GPIO31) corresponds to one bit in this register as shown in <a href="#">Figure 1-85</a> .                                                                                                                                                                                     |
|      |              | 0     | Writes of 0 are ignored. This register always reads back a 0.                                                                                                                                                                                                                                             |
|      |              | 1     | Writing a 1 forces the respective output data latch to toggle from its current state. If the pin is configured as a GPIO output then it will be driven in the opposite direction of its current state. If the pin is not configured as a GPIO output then the latch is toggled but the pin is not driven. |

**Figure 1-86. GPIO Port B Set, Clear and Toggle (GPBSET, GPBCLEAR, GPBTOGGLE) Registers**

|          |        |        |        |        |        |          |        |
|----------|--------|--------|--------|--------|--------|----------|--------|
| 31       |        |        |        | 27     | 26     | 25       | 24     |
| Reserved |        |        |        |        | GPIO58 | GPIO57   | GPIO56 |
| R-0      |        |        |        |        | R/W-0  | R/W-0    | R/W-0  |
| 23       | 22     | 21     | 20     | 19     | 18     | 17       | 16     |
| GPIO55   | GPIO54 | GPIO53 | GPIO52 | GPIO51 | GPIO50 | Reserved |        |
| R/W-0    | R/W-0  | R/W-0  | R/W-0  | R/W-0  | R/W-0  | R-0      |        |
| 15       |        | 13     | 12     | 11     | 10     | 9        | 8      |
| Reserved |        |        | GPIO44 | GPIO43 | GPIO42 | GPIO41   | GPIO40 |
| R-0      |        |        | R/W-0  | R/W-0  | R/W-0  | R/W-0    | R/W-0  |
| 7        | 6      | 5      | 4      | 3      | 2      | 1        | 0      |
| GPIO39   | GPIO38 | GPIO37 | GPIO36 | GPIO35 | GPIO34 | GPIO33   | GPIO32 |
| R/W-0    | R/W-0  | R/W-0  | R/W-0  | R/W-0  | R/W-0  | R/W-0    | R/W-0  |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 1-90. GPIO Port B Set (GPBSET) Register Field Descriptions**

| Bits   | Field          | Value  | Description                                                                                                                                                                                                                                                                                                                                                                                                                   |
|--------|----------------|--------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 31- 27 | Reserved       |        | Any writes to these bits must always have a value of 0.                                                                                                                                                                                                                                                                                                                                                                       |
| 26-18  | GPIO58 -GPIO50 | 0<br>1 | Each GPIO port B pin (GPIO58-GPIO50) corresponds to one bit in this register as shown in <a href="#">Figure 1-86</a> .<br>Writes of 0 are ignored. This register always reads back a 0.<br>Writing a 1 forces the respective output data latch to high. If the pin is configured as a GPIO output then it will be driven high. If the pin is not configured as a GPIO output then the latch is set but the pin is not driven. |
| 17-13  | Reserved       |        | Any writes to these bits must always have a value of 0.                                                                                                                                                                                                                                                                                                                                                                       |
| 12-0   | GPIO44-GPIO32  | 0<br>1 | Each GPIO port B pin (GPIO44-GPIO32) corresponds to one bit in this register as shown in <a href="#">Figure 1-86</a> .<br>Writes of 0 are ignored. This register always reads back a 0.<br>Writing a 1 forces the respective output data latch to high. If the pin is configured as a GPIO output then it will be driven high. If the pin is not configured as a GPIO output then the latch is set but the pin is not driven. |

**Table 1-91. GPIO Port B Clear (GPBCLEAR) Register Field Descriptions**

| Bits   | Field         | Value | Description                                                                                                                                                                                                                          |
|--------|---------------|-------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 31- 27 | Reserved      |       | Any writes to these bits must always have a value of 0.                                                                                                                                                                              |
| 26-18  | GPIO58-GPIO50 |       | Each GPIO port B pin (GPIO58-GPIO50) corresponds to one bit in this register as shown in <a href="#">Figure 1-86</a> .                                                                                                               |
|        |               | 0     | Writes of 0 are ignored. This register always reads back a 0.                                                                                                                                                                        |
|        |               | 1     | Writing a 1 forces the respective output data latch to low. If the pin is configured as a GPIO output then it will be driven low. If the pin is not configured as a GPIO output then the latch is cleared but the pin is not driven. |
| 17-13  | Reserved      |       | Any writes to these bits must always have a value of 0.                                                                                                                                                                              |
| 12-0   | GPIO44-GPIO32 |       | Each GPIO port B pin (GPIO32-GPIO 44) corresponds to one bit in this register as shown in <a href="#">Figure 1-86</a> .                                                                                                              |
|        |               | 0     | Writes of 0 are ignored. This register always reads back a 0.                                                                                                                                                                        |
|        |               | 1     | Writing a 1 forces the respective output data latch to low. If the pin is configured as a GPIO output then it will be driven low. If the pin is not configured as a GPIO output then the latch is cleared but the pin is not driven. |

**Table 1-92. GPIO Port B Toggle (GPBTOGGLE) Register Field Descriptions**

| Bits   | Field          | Value | Description                                                                                                                                                                                                                                                                                               |
|--------|----------------|-------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 31- 27 | Reserved       |       | Any writes to these bits must always have a value of 0.                                                                                                                                                                                                                                                   |
| 26-18  | GPIO58 -GPIO50 |       | Each GPIO port B pin (GPIO58-GPIO50) corresponds to one bit in this register as shown in <a href="#">Figure 1-86</a> .                                                                                                                                                                                    |
|        |                | 0     | Writes of 0 are ignored. This register always reads back a 0.                                                                                                                                                                                                                                             |
|        |                | 1     | Writing a 1 forces the respective output data latch to toggle from its current state. If the pin is configured as a GPIO output then it will be driven in the opposite direction of its current state. If the pin is not configured as a GPIO output then the latch is cleared but the pin is not driven. |
| 17-13  | Reserved       |       | Any writes to these bits must always have a value of 0.                                                                                                                                                                                                                                                   |
| 12-0   | GPIO44-GPIO32  |       | Each GPIO port B pin (GPIO44-GPIO32) corresponds to one bit in this register as shown in <a href="#">Figure 1-86</a> .                                                                                                                                                                                    |
|        |                | 0     | Writes of 0 are ignored. This register always reads back a 0.                                                                                                                                                                                                                                             |
|        |                | 1     | Writing a 1 forces the respective output data latch to toggle from its current state. If the pin is configured as a GPIO output then it will be driven in the opposite direction of its current state. If the pin is not configured as a GPIO output then the latch is cleared but the pin is not driven. |

**Figure 1-87. Analog I/O Toggle (AIOSET, AIOCLEAR, AIOTOGGLE) Register**

31

16

|          |       |          |       |          |       |          |   |
|----------|-------|----------|-------|----------|-------|----------|---|
| Reserved |       |          |       |          |       |          |   |
| R-0      |       |          |       |          |       |          |   |
| 15       | 14    | 13       | 12    | 11       | 10    | 9        | 8 |
| Reserved | AIO14 | Reserved | AIO12 | Reserved | AIO10 | Reserved |   |
| R-0      | R/W-x | R-0      | R/W-x | R-0      | R/W-x | R-0      |   |
| 7        | 6     | 5        | 4     | 3        | 2     | 1        | 0 |
| Reserved | AIO6  | Reserved | AIO4  | Reserved | AIO2  | Reserved |   |
| R-0      | R/W-x | R-0      | R/W-x | R-0      | R/W-x | R-0      |   |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 1-93. Analog I/O Set (AIOSET) Register Field Descriptions**

| Bits  | Field            | Value | Description                                                                                                                                                                                                                      |
|-------|------------------|-------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 31-15 | Reserved         |       | Any writes to these bits must always have a value of 0.                                                                                                                                                                          |
| 14-0  | AIO <sub>n</sub> | 0     | Each AIO pin corresponds to one bit in this register.<br>Writes of 0 are ignored. This register always reads back a 0.                                                                                                           |
|       |                  | 1     | Writing a 1 forces the respective output data latch to high. If the pin is configured as a AIO output then it will be driven high. If the pin is not configured as a AIO output then the latch is set but the pin is not driven. |

**Table 1-94. Analog I/O Clear (AIOCLEAR) Register Field Descriptions**

| Bits  | Field            | Value | Description                                                                                                                                                                                                                        |
|-------|------------------|-------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 31-15 | Reserved         |       | Any writes to these bits must always have a value of 0.                                                                                                                                                                            |
| 14-0  | AIO <sub>n</sub> | 0     | Each AIO pin corresponds to one bit in this register.<br>Writes of 0 are ignored. This register always reads back a 0.                                                                                                             |
|       |                  | 1     | Writing a 1 forces the respective output data latch to low. If the pin is configured as a AIO output then it will be driven low. If the pin is not configured as a AIO output then the latch is cleared but the pin is not driven. |

**Table 1-95. Analog I/O Toggle (AIOTOGGLE) Register Field Descriptions**

| Bits  | Field            | Value | Description                                                                                                                                                                                                                                                                                             |
|-------|------------------|-------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 31-15 | Reserved         |       | Any writes to these bits must always have a value of 0.                                                                                                                                                                                                                                                 |
| 14-0  | AIO <sub>n</sub> | 0     | Each AIO pin corresponds to one bit in this register.<br>Writes of 0 are ignored. This register always reads back a 0.                                                                                                                                                                                  |
|       |                  | 1     | Writing a 1 forces the respective output data latch to toggle from its current state. If the pin is configured as a AIO output then it will be driven in the opposite direction of its current state. If the pin is not configured as a AIO output then the latch is cleared but the pin is not driven. |

**Figure 1-88. GPIO XINTn Interrupt Select (GPIOXINTnSEL) Registers**

15

5

4

0

|          |  |              |  |
|----------|--|--------------|--|
| Reserved |  | GPIOXINTnSEL |  |
| R-0      |  | R/W-0        |  |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 1-96. GPIO XINTn Interrupt Select (GPIOXINTnSEL) Register Field Descriptions**

| Bits | Field        | Value | Description                                                                                                                                                                                                                                                                                                                                                                                                               |
|------|--------------|-------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-5 | Reserved     |       | Any writes to these bits must always have a value of 0.                                                                                                                                                                                                                                                                                                                                                                   |
| 4-0  | GPIOXINTnSEL |       | Select the port A GPIO signal (GPIO0 - GPIO31) that will be used as the XINT1, XINT2, or XINT3 interrupt source. In addition, you can configure the interrupt in the XINT1CR, XINT2CR, or XINT3CR registers described in <a href="#">Section 1.6.6</a> .<br>To use XINT2 as ADC start of conversion, enable it in the desired ADCSOCxCTL register. The $\overline{\text{ADCSOC}}$ signal is always rising edge sensitive. |
|      |              | 00000 | Select the GPIO0 pin as the XINTn interrupt source (default)                                                                                                                                                                                                                                                                                                                                                              |
|      |              | 00001 | Select the GPIO1 pin as the XINTn interrupt source                                                                                                                                                                                                                                                                                                                                                                        |
|      |              | ...   | ...                                                                                                                                                                                                                                                                                                                                                                                                                       |
|      |              | 11110 | Select the GPIO30 pin as the XINTn interrupt source                                                                                                                                                                                                                                                                                                                                                                       |
|      |              | 11111 | Select the GPIO31 pin as the XINTn interrupt source                                                                                                                                                                                                                                                                                                                                                                       |

**Table 1-97. XINT1/XINT2/XINT3 Interrupt Select and Configuration Registers**

| n | Interrupt | Interrupt Select Register | Configuration Register |
|---|-----------|---------------------------|------------------------|
| 1 | XINT1     | GPIOXINT1SEL              | XINT1CR                |
| 2 | XINT2     | GPIOXINT2SEL              | XINT2CR                |
| 3 | XINT3     | GPIOXINT3SEL              | XINT3CR                |

**Figure 1-89. GPIO Low Power Mode Wakeup Select (GPIOLPMSEL) Register**

|        |        |        |        |        |        |        |        |
|--------|--------|--------|--------|--------|--------|--------|--------|
| 31     | 30     | 29     | 28     | 27     | 26     | 25     | 24     |
| GPIO31 | GPIO30 | GPIO29 | GPIO28 | GPIO27 | GPIO26 | GPIO25 | GPIO24 |
| R/W-0  | R/W-0  | R/W-0  | R/W-0  | R/W-0  | R/W-0  | R/W-0  | R/W-0  |
| 23     | 22     | 21     | 20     | 19     | 18     | 17     | 16     |
| GPIO23 | GPIO22 | GPIO21 | GPIO20 | GPIO19 | GPIO18 | GPIO17 | GPIO16 |
| R/W-0  | R/W-0  | R/W-0  | R/W-0  | R/W-0  | R/W-0  | R/W-0  | R/W-0  |
| 15     | 14     | 13     | 12     | 11     | 10     | 9      | 8      |
| GPIO15 | GPIO14 | GPIO13 | GPIO12 | GPIO11 | GPIO10 | GPIO9  | GPIO8  |
| R/W-0  | R/W-0  | R/W-0  | R/W-0  | R/W-0  | R/W-0  | R/W-0  | R/W-0  |
| 7      | 6      | 5      | 4      | 3      | 2      | 1      | 0      |
| GPIO7  | GPIO6  | GPIO5  | GPIO4  | GPIO3  | GPIO2  | GPIO1  | GPIO0  |
| R/W-0  | R/W-0  | R/W-0  | R/W-0  | R/W-0  | R/W-0  | R/W-0  | R/W-0  |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 1-98. GPIO Low Power Mode Wakeup Select (GPIOLPMSEL) Register Field Descriptions**

| Bits | Field          | Value | Description                                                                                                                                                                                                                                                                         |
|------|----------------|-------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 31-0 | GPIO31 - GPIO0 | 0     | Low Power Mode Wakeup Selection. Each bit in this register corresponds to one GPIO port A pin (GPIO0 - GPIO31) as shown in <a href="#">Figure 1-89</a> .<br>If the bit is cleared, the signal on the corresponding pin will have no effect on the HALT and STANDBY low power modes. |
|      |                | 1     | If the respective bit is set to 1, the signal on the corresponding pin is able to wake the device from both HALT and STANDBY low power modes.                                                                                                                                       |

## 1.5 Peripheral Frames

This chapter describes the peripheral frames and device emulation registers.

### 1.5.1 Peripheral Frame Registers

This device contains four peripheral register spaces. The spaces are categorized as follows:

- Peripheral Frame 0: These are peripherals that are mapped directly to the CPU memory bus. See [Table 1-99](#).
- Peripheral Frame 1: These are peripherals that are mapped to the 32-bit peripheral bus. See [Table 1-100](#).
- Peripheral Frame 2: These are peripherals that are mapped to the 16-bit peripheral bus. See [Table 1-101](#).
- Peripheral Frame 3: These are peripherals that are mapped to the 16-bit peripheral bus. See [Table 1-102](#).

**Table 1-99. Peripheral Frame 0 Registers**

| NAME <sup>(1)</sup>                         | ADDRESS RANGE         | SIZE (×16) | EALLOW PROTECTED <sup>(2)</sup> |
|---------------------------------------------|-----------------------|------------|---------------------------------|
| Device Emulation Registers                  | 0x00 0880 – 0x00 0984 | 261        | Yes                             |
| System Power Control Registers              | 0x00 0985 – 0x00 0987 | 3          | Yes                             |
| FLASH Registers <sup>(3)</sup>              | 0x00 0A80 – 0x00 0ADF | 96         | Yes                             |
| Code Security Module Registers              | 0x00 0AE0 – 0x00 0AEF | 16         | Yes                             |
| ADC registers<br>(0 wait read only)         | 0x00 0B00 – 0x00 0B0F | 16         | No                              |
| CPU–TIMER0/1/2 Registers                    | 0x00 0C00 – 0x00 0C3F | 64         | No                              |
| PIE Registers                               | 0x00 0CE0 – 0x00 0CFF | 32         | No                              |
| PIE Vector Table                            | 0x00 0D00 – 0x00 0DFF | 256        | No                              |
| DMA Registers                               | 0x00 1000 – 0x00 11FF | 512        | Yes                             |
| CLA Registers                               | 0x00 1400 – 0x00 147F | 128        | Yes                             |
| CLA to CPU Message RAM (CPU writes ignored) | 0x00 1480 – 0x00 14FF | 128        | NA                              |
| CPU to CLA Message RAM (CLA writes ignored) | 0x00 1500 – 0x00 157F | 128        | NA                              |

(1) Registers in Frame 0 support 16-bit and 32-bit accesses.

(2) If registers are EALLOW protected, then writes cannot be performed until the EALLOW instruction is executed. The EDIS instruction disables writes to prevent stray code or pointers from corrupting register contents.

(3) The Flash Registers are also protected by the Code Security Module (CSM).

**Table 1-100. Peripheral Frame 1 Registers**

| NAME <sup>(1)</sup>      | ADDRESS RANGE         | SIZE (×16) | EALLOW PROTECTED <sup>(2)</sup> |
|--------------------------|-----------------------|------------|---------------------------------|
| eCAN-A registers         | 0x00 6000 – 0x00 61FF | 512        | (3)                             |
| Comparator 1 registers   | 0x00 6400 – 0x00 641F | 32         | (3)                             |
| Comparator 2 registers   | 0x00 6420 – 0x00 643F | 32         | (3)                             |
| Comparator 3 registers   | 0x00 6440 – 0x00 645F | 32         | (3)                             |
| ePWM1 + HRPWM1 registers | 0x00 6800 – 0x00 683F | 64         | (3)                             |
| ePWM2 + HRPWM2 registers | 0x00 6840 – 0x00 687F | 64         | (3)                             |
| ePWM3 + HRPWM3 registers | 0x00 6880 – 0x00 68BF | 64         | (3)                             |
| ePWM4 + HRPWM4 registers | 0x00 68C0 – 0x00 68FF | 64         | (3)                             |
| ePWM5 + HRPWM5 registers | 0x00 6900 – 0x00 693F | 64         | (3)                             |
| ePWM6 + HRPWM6 registers | 0x00 6940 – 0x00 697F | 64         | (3)                             |
| ePWM7 + HRPWM7 registers | 0x00 6980 – 0x00 69BF | 64         | (3)                             |
| ePWM8 + HRPWM8 registers | 0x00 69C0 – 0x00 69FF | 64         | (3)                             |
| eCAP1 registers          | 0x00 6A00 – 0x00 6A1F | 32         | No                              |
| eCAP2 registers          | 0x00 6A20 – 0x00 6A3F | 32         | No                              |
| eCAP3 registers          | 0x00 6A40 – 0x00 6A57 | 32         | No                              |
| HRCAP1 registers         | 0x00 6AC0 – 0x00 6ADF | 32         | (3)                             |
| HRCAP2 registers         | 0x00 6AE0 – 0x00 6AFF | 32         | (3)                             |
| eQEP1 registers          | 0x00 6B00 – 0x00 6B3F | 64         | (3)                             |
| eQEP2 registers          | 0x00 6B40 – 0x00 6B7F | 64         | (3)                             |
| HRCAP3 registers         | 0x00 6C80 – 0x00 6C9F | 32         | (3)                             |
| HRCAP4 registers         | 0x00 6CA0 – 0x00 6CBF | 32         | (3)                             |
| GPIO registers           | 0x00 6F80 – 0x00 6FFF | 128        | (3)                             |

(1) Back-to-back write operations to Peripheral Frame 1 registers will incur a 1-cycle stall (1 cycle delay).

(2) Peripheral Frame 1 allows 16-bit and 32-bit accesses. All 32-bit accesses are aligned to even address boundaries.

(3) Some registers are EALLOW protected. See the module reference guide for more information.

**Table 1-101. Peripheral Frame 2 Registers**

| NAME                             | ADDRESS RANGE         | SIZE (×16) | EALLOW PROTECTED |
|----------------------------------|-----------------------|------------|------------------|
| System Control Registers         | 0x00 7010 – 0x00 702F | 32         | Yes              |
| SPI-A Registers                  | 0x00 7040 – 0x00 704F | 16         | No               |
| SCI-A Registers                  | 0x00 7050 – 0x00 705F | 16         | No               |
| NMI Watchdog Interrupt Registers | 0x00 7060 – 0x00 706F | 16         | Yes              |
| External Interrupt Registers     | 0x00 7070 – 0x00 707F | 16         | Yes              |
| ADC Registers                    | 0x00 7100 – 0x00 717F | 128        | (1)              |
| SPI-B Registers                  | 0x00 7740 – 0x00 774F | 16         | No               |
| SCI-B Registers                  | 0x00 7750 – 0x00 775F | 16         | No               |
| I2C-A Registers                  | 0x00 7900 – 0x00 793F | 64         | (1)              |

(1) Some registers are EALLOW protected.

**Table 1-102. Peripheral Frame 3 Registers**

| NAME              | ADDRESS RANGE         | SIZE (×16) | EALLOW PROTECTED |
|-------------------|-----------------------|------------|------------------|
| USB0 Registers    | 0x00 4000 – 0x4FFF    | 4096       | No               |
| McBSP-A Registers | 0x00 5000 – 0x00 503F | 64         | No               |



## 1.5.2 EALLOW-Protected Registers

Several control registers are protected from spurious CPU writes by the EALLOW protection mechanism. The EALLOW bit in status register 1 (ST1) indicates if the state of protection as shown in [Table 1-103](#).

**Table 1-103. Access to EALLOW-Protected Registers**

| EALLOW Bit | CPU Writes | CPU Reads | JTAG Writes            | JTAG Reads |
|------------|------------|-----------|------------------------|------------|
| 0          | Ignored    | Allowed   | Allowed <sup>(1)</sup> | Allowed    |
| 1          | Allowed    | Allowed   | Allowed                | Allowed    |

(1) The EALLOW bit is overridden via the JTAG port, allowing full access of protected registers during debug from the Code Composer Studio™ IDE.

At reset the EALLOW bit is cleared enabling EALLOW protection. While protected, all writes to protected registers by the CPU are ignored and only CPU reads, JTAG reads, and JTAG writes are allowed. If this bit is set, by executing the EALLOW instruction, then the CPU is allowed to write freely to protected registers. After modifying registers, they can once again be protected by executing the EDI instruction to clear the EALLOW bit.

The following registers are EALLOW-protected:

- Device Emulation Registers
- Flash Registers
- CSM Registers
- PIE Vector Table
- System Control Registers
- GPIO MUX Registers

**Table 1-104. EALLOW-Protected Device Emulation Registers**

| Name      | Address          | Size (x16) | Description                   |
|-----------|------------------|------------|-------------------------------|
| DEVICECNF | 0x0880<br>0x0881 | 2          | Device Configuration Register |

**Table 1-105. EALLOW-Protected Flash/OTP Configuration Registers**

| Name        | Address | Size (x16) | Description                                 |
|-------------|---------|------------|---------------------------------------------|
| FOPT        | 0x0A80  | 1          | Flash Option Register                       |
| FPWR        | 0x0A82  | 1          | Flash Power Modes Register                  |
| FSTATUS     | 0x0A83  | 1          | Status Register                             |
| FSTDBYWAIT  | 0x0A84  | 1          | Flash Sleep To Standby Wait State Register  |
| FACTIVEWAIT | 0x0A85  | 1          | Flash Standby To Active Wait State Register |
| FBANKWAIT   | 0x0A86  | 1          | Flash Read Access Wait State Register       |
| FOTPWAIT    | 0x0A87  | 1          | OTP Read Access Wait State Register         |

**Table 1-106. EALLOW-Protected Code Security Module (CSM) Registers**

| Register Name | Address | Size (x16) | Register Description                     |
|---------------|---------|------------|------------------------------------------|
| KEY0          | 0x0AE0  | 1          | Low word of the 128-bit KEY register     |
| KEY1          | 0x0AE1  | 1          | Second word of the 128-bit KEY register  |
| KEY2          | 0x0AE2  | 1          | Third word of the 128-bit KEY register   |
| KEY3          | 0x0AE3  | 1          | Fourth word of the 128-bit KEY register  |
| KEY4          | 0x0AE4  | 1          | Fifth word of the 128-bit KEY register   |
| KEY5          | 0x0AE5  | 1          | Sixth word of the 128-bit KEY register   |
| KEY6          | 0x0AE6  | 1          | Seventh word of the 128-bit KEY register |
| KEY7          | 0x0AE7  | 1          | High word of the 128-bit KEY register    |
| CSMSCR        | 0x0AEF  | 1          | CSM status and control register          |

**Table 1-107. EALLOW-Protected PLL, Clocking, Watchdog, and Low-Power Mode Registers**

| NAME        | ADDRESS   | SIZE (x16) | DESCRIPTION                                   |
|-------------|-----------|------------|-----------------------------------------------|
| BORCFG      | 0x00 0985 | 1          | BOR Configuration Register                    |
| XCLK        | 0x00 7010 | 1          | XCLKOUT and XCLKIN Control Register           |
| PLLSTS      | 0x00 7011 | 1          | PLL Status Register                           |
| CLKCTL      | 0x00 7012 | 1          | Clock Control Register                        |
| PLLLOCKPRD  | 0x00 7013 | 1          | PLL Lock Period Register                      |
| INTOSC1TRIM | 0x00 7014 | 1          | Internal Oscillator 1 Trim Register           |
| INTOSC2TRIM | 0x00 7016 | 1          | Internal Oscillator 2 Trim Register           |
| PCLKCR2     | 0x00 7019 | 1          | Peripheral Clock Control Register 2           |
| LOSPCP      | 0x00 701B | 1          | Low-Speed Peripheral Clock Prescaler Register |
| PCLKCR0     | 0x00 701C | 1          | Peripheral Clock Control Register 0           |
| PCLKCR1     | 0x00 701D | 1          | Peripheral Clock Control Register 1           |
| LPMCR0      | 0x00 701E | 1          | Low Power Mode Control Register 0             |
| PCLKCR3     | 0x00 7020 | 1          | Peripheral Clock Control Register 3           |
| PLLCR       | 0x00 7021 | 1          | PLL Control Register                          |
| SCSR        | 0x00 7022 | 1          | System Control and Status Register            |
| WDCNTR      | 0x00 7023 | 1          | Watchdog Counter Register                     |
| WDKEY       | 0x00 7025 | 1          | Watchdog Reset Key Register                   |
| WDCR        | 0x00 7029 | 1          | Watchdog Control Register                     |
| PLL2CTL     | 0x00 7030 | 1          | PLL2 Configuration Register                   |
| PLL2MULT    | 0x00 7032 | 1          | PLL2 Multiplier Register                      |
| PLL2STS     | 0x00 7034 | 1          | PLL2 Lock Status Register                     |
| SYSCLK2CNTR | 0x00 7036 | 1          | SYSCLK2 Clock Counter Register                |
| EPWMCFG     | 0x00 703A | 1          | ePWM DMA/CLA Configuration Register           |

**Table 1-108. EALLOW-Protected GPIO Registers**

| Name <sup>(1)</sup> | Address | Size (x16) | Register Description                             |
|---------------------|---------|------------|--------------------------------------------------|
| GPACTRL             | 0x6F80  | 2          | GPIO A Control Register                          |
| GPAQSEL1            | 0x6F82  | 2          | GPIO A Qualifier Select 1 Register               |
| GPAQSEL2            | 0x6F84  | 2          | GPIO A Qualifier Select 2 Register               |
| GPAMUX1             | 0x6F86  | 2          | GPIO A MUX 1 Register                            |
| GPAMUX2             | 0x6F88  | 2          | GPIO A MUX 2 Register                            |
| GPADIR              | 0x6F8A  | 2          | GPIO A Direction Register                        |
| GPAPUD              | 0x6F8C  | 2          | GPIO A Pull Up Disable Register                  |
| GPBCTRL             | 0x6F90  | 2          | GPIO B Control Register                          |
| GPBQSEL1            | 0x6F92  | 2          | GPIO B Qualifier Select 1 Register               |
| GPBQSEL2            | 0x6F94  | 2          | GPIO B Qualifier Select 2 Register               |
| GPBMUX1             | 0x6F96  | 2          | GPIO B MUX 1 Register                            |
| GPBMUX2             | 0x6F98  | 2          | GPIO B MUX 2 Register                            |
| GPBDIR              | 0x6F9A  | 2          | GPIO B Direction Register                        |
| GPBPUD              | 0x6F9C  | 2          | GPIO B Pull Up Disable Register                  |
| AIOMUX1             | 0x6FB6  | 2          | Analog, I/O MUX 1 register                       |
| AIODIR              | 0x6FBA  | 2          | Analog, IO Direction Register                    |
| GPIOXINT1SEL        | 0x6FE0  | 1          | XINT1 Source Select Register (GPIO0-GPIO31)      |
| GPIOXINT2SEL        | 0x6FE1  | 1          | XINT2 Source Select Register (GPIO0-GPIO31)      |
| GPIOXINT3SEL        | 0x6FE2  | 1          | XINT3 Source Select Register (GPIO0 - GPIO31)    |
| GPIOLPMSEL          | 0x6FE8  | 1          | LPM wakeup Source Select Register (GPIO0-GPIO31) |

(1) The registers in this table are EALLOW protected. See [Section 1.5.2](#) for more information.

[Table 1-110](#) shows addresses for the following ePWM EALLOW-protected registers:

- Trip Zone Select Register (TZSEL)
- Trip Zone Control Register (TZCTL)
- Trip Zone Enable Interrupt Register (TZEINT)
- Trip Zone Clear Register (TZCLR)
- Trip Zone Force Register (TZFRC)
- HRPWM Configuration Register (HRCNFG)

**Table 1-109. EALLOW-Protected PIE Vector Table**

| Name     | Address | Size (x16) | Description                     |
|----------|---------|------------|---------------------------------|
| Not used | 0x0D00  | 2          | Reserved                        |
|          | 0x0D02  |            |                                 |
|          | 0x0D04  |            |                                 |
|          | 0x0D06  |            |                                 |
|          | 0x0D08  |            |                                 |
|          | 0x0D0A  |            |                                 |
|          | 0x0D0C  |            |                                 |
|          | 0x0D0E  |            |                                 |
|          | 0x0D10  |            |                                 |
|          | 0x0D12  |            |                                 |
|          | 0x0D14  |            |                                 |
|          | 0x0D16  |            |                                 |
|          | 0x0D18  |            |                                 |
| INT13    | 0x0D1A  | 2          | CPU-Timer 1                     |
| INT14    | 0x0D1C  | 2          | CPU-Timer 2                     |
| DATALOG  | 0x0D1E  | 2          | CPU Data Logging Interrupt      |
| RTOSINT  | 0x0D20  | 2          | CPU Real-Time OS Interrupt      |
| EMUINT   | 0x0D22  | 2          | CPU Emulation Interrupt         |
| NMI      | 0x0D24  | 2          | External Non-Maskable Interrupt |
| ILLEGAL  | 0x0D26  | 2          | Illegal Operation               |
| USER1    | 0x0D28  | 2          | User-Defined Trap               |
| .        | .       | .          | .                               |
| USER12   | 0x0D3E  | 2          | User-Defined Trap               |
| INT1.1   | 0x0D40  | 2          | Group 1 Interrupt Vectors       |
| .        | .       | .          | .                               |
| INT1.8   | 0x0D4E  | 2          |                                 |
| .        | .       | .          | Group 2 Interrupt Vectors       |
| .        | .       | .          | to Group 11 Interrupt Vectors   |
| .        | .       | .          | .                               |
| INT12.1  | 0x0DF0  | 2          | Group 12 Interrupt Vectors      |
| .        | .       | .          | .                               |
| INT12.8  | 0x0DFE  | 2          |                                 |

**Table 1-110. EALLOW-Protected ePWM1 - ePWM 7 Registers**

|       | TZSEL  | TZCTL  | TZEINT | TZCLR  | TZFRC  | HRCNFG | Size x16 |
|-------|--------|--------|--------|--------|--------|--------|----------|
| ePWM1 | 0x6812 | 0x6814 | 0x6815 | 0x6817 | 0x6818 | 0x6820 | 1        |
| ePWM2 | 0x6852 | 0x6854 | 0x6855 | 0x6857 | 0x6858 | 0x6860 | 1        |
| ePWM3 | 0x6892 | 0x6894 | 0x6895 | 0x6897 | 0x6898 | 0x68A0 | 1        |
| ePWM4 | 0x68D2 | 0x68D4 | 0x68D5 | 0x68D7 | 0x68D8 | 0x68E0 | 1        |
| ePWM5 | 0x6912 | 0x6914 | 0x6915 | 0x6917 | 0x6918 | 0x6920 | 1        |
| ePWM6 | 0x6952 | 0x6954 | 0x6955 | 0x6957 | 0x6958 | 0x6960 | 1        |
| ePWM7 | 0x6992 | 0x6994 | 0x6995 | 0x6997 | 0x6998 | 0x69A0 | 1        |

### 1.5.3 Device Emulation Registers

These registers are used to control the protection mode of the C28x CPU and to monitor some critical device signals. The registers are defined in [Table 1-111](#).

**Table 1-111. Device Emulation Registers**

| Name      | Address          | Size (x16) | Description                   |
|-----------|------------------|------------|-------------------------------|
| DEVICECNF | 0x0880<br>0x0881 | 2          | Device Configuration Register |
| PARTID    | 0x3D 7E80        | 1          | Part ID Register              |
| CLASSID   | 0x0882           | 1          | Class ID Register             |
| REVID     | 0x0883           | 1          | Revision ID Register          |

**Figure 1-90. Device Configuration (DEVICECNF) Register**

|          |                |          |      |          |    |     |        |          |          |   |
|----------|----------------|----------|------|----------|----|-----|--------|----------|----------|---|
| 31       | 30             | 29       | 28   | 27       | 26 | 20  | 19     | 18       | 16       |   |
| Rsvd     | SYSCLK2DIV2DIS | Reserved | TRST | Reserved |    |     | ENPROT | Reserved |          |   |
| R-0      | R/W-0          | R-0      | R-0  | R-0      |    |     | R/W-1  | R-111    |          |   |
| 15       |                |          |      |          |    | 5   | 4      | 3        | 2        | 0 |
| Reserved |                |          |      |          |    | XRS | Rsvd   | VMAPS    | Reserved |   |
| R-0      |                |          |      |          |    | R-P | R-0    | R-1      | R-011    |   |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 1-112. DEVICECNF Register Field Descriptions**

| Bits  | Field          | Value  | Description                                                                                                                                               |
|-------|----------------|--------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| 31    | Reserved       |        | Any writes to these bits must always have a value of 0.                                                                                                   |
| 30    | SYSCLK2DIV2DIS | 0<br>1 | SYSCLK2 Clock Divide by 2 Disable Bit<br>PLL2 Output/2<br>PLL2 Output/1                                                                                   |
| 29-28 | Reserved       |        |                                                                                                                                                           |
| 27    | TRST           | 0<br>1 | Read status of TRST signal. Reading this bit gives the current status of the TRST signal.<br>No debug probe is connected.<br>An debug probe is connected. |
| 26:20 | Reserved       |        | Any writes to these bits must always have a value of 0.                                                                                                   |
| 19    | ENPROT         | 0<br>1 | Enable Write-Read Protection Mode Bit.<br>Disables write-read protection mode<br>Enables write-read protection for the address range 0x4000-0x7FFF        |
| 18-6  | Reserved       |        | Any writes to these bits must always have a value of 0.                                                                                                   |
| 5     | XRS            |        | Reset Input Signal Status. This is connected directly to the XRS input pin.                                                                               |
| 4     | Reserved       |        | Reserved                                                                                                                                                  |
| 3     | VMAPS          |        | VMAP Configure Status. This indicates the status of VMAP.                                                                                                 |
| 2-0   | Reserved       |        | Any writes to these bits must always have a value of 0.                                                                                                   |

**Figure 1-91. Part ID Register**

|          |   |        |   |
|----------|---|--------|---|
| 15       | 8 | 7      | 0 |
| PARTTYPE |   | PARTID |   |
| R        |   | R      |   |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 1-113. PARTID Register Field Descriptions**

| Bit  | Field    | Value <sup>(1)</sup> | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|------|----------|----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15:8 | PARTTYPE | 0x00                 | Part type ID register. Specifies the type of device such as flash-based.<br>Flash-based device<br>All other values are reserved.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| 7:0  | PARTID   | 0x3D 7E80            | Part ID Register. These 8 bits specify the feature set of this device. For TMS320F28069U devices, the PARTID/CLASSID numbers (see <a href="#">Table 1-114</a> ) are also used for TMX devices. In the case of TMX320F28069UPFPA and TMX320F28069UPZPA devices, the temperature rating is "A" instead of "T".<br><br><div> <div>TMS320F28069PZP/PZ</div> <div>0x9E</div> </div> <div> <div>TMS320F28069UPZP/ PZ</div> <div>0x9F</div> </div> <div> <div>TMS320F28069PFP/PN</div> <div>0x9C</div> </div> <div> <div>TMS320F28069UPFP/PN</div> <div>0x9D</div> </div> <div> <div>TMS320F28068PZP/PZ</div> <div>0x8E</div> </div> <div> <div>TMS320F28068UPZP/PZ</div> <div>0x8F</div> </div> <div> <div>TMS320F28068PFP/PN</div> <div>0x8C</div> </div> <div> <div>TMS320F28068UPFP/PN</div> <div>0x8D</div> </div> <div> <div>TMS320F28067PZP/PZ</div> <div>0x8A</div> </div> <div> <div>TMS320F28067UPZP/PZ</div> <div>0x8B</div> </div> <div> <div>TMS320F28067PFP/PN</div> <div>0x88</div> </div> <div> <div>TMS320F28067UPFP/PN</div> <div>0x89</div> </div> <div> <div>TMS320F28066PZP/PZ</div> <div>0x86</div> </div> <div> <div>TMS320F28066UPZP/PZ</div> <div>0x87</div> </div> <div> <div>TMS320F28066PFP/PN</div> <div>0x84</div> </div> <div> <div>TMS320F28066UPFP/PN</div> <div>0x85</div> </div> <div> <div>TMS320F28065PZP/PZ</div> <div>0x7E</div> </div> <div> <div>TMS320F28065UPZP/PZ</div> <div>0x7F</div> </div> <div> <div>TMS320F28065PFP/PN</div> <div>0x7C</div> </div> <div> <div>TMS320F28065UPFP/PN</div> <div>0x7D</div> </div> <div> <div>TMS320F28064PZP/PZ</div> <div>0x6E</div> </div> <div> <div>TMS320F28064UPZP/PZ</div> <div>0x6F</div> </div> <div> <div>TMS320F28064PFP/PN</div> <div>0x6C</div> </div> <div> <div>TMS320F28064UPFP/PN</div> <div>0x6D</div> </div> <div> <div>TMS320F28063PZP/PZ</div> <div>0x6A</div> </div> <div> <div>TMS320F28063UPZP/PZ</div> <div>0x6B</div> </div> <div> <div>TMS320F28063PFP/PN</div> <div>0x68</div> </div> <div> <div>TMS320F28063UPFP/PN</div> <div>0x69</div> </div> <div> <div>TMS320F28062PZP/PZ</div> <div>0x66</div> </div> <div> <div>TMS320F28062UPZP/PZ</div> <div>0x67</div> </div> <div> <div>TMS320F28062PFP/PN</div> <div>0x65</div> </div> |

**Table 1-113. PARTID Register Field Descriptions (continued)**

| Bit | Field | Value <sup>(1)</sup> | Description              |
|-----|-------|----------------------|--------------------------|
|     |       |                      | TMS320F28062UPFP/PN 0x64 |

(1) The reset value depends on the device as indicated in the register description.

**Table 1-114. CLASSID Register Field Descriptions**

| Bit | Field   | Value <sup>(1)</sup> | Description                                                                                                                                                                                                                                                                                                                                     |
|-----|---------|----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 7:0 | CLASSID | 0x0882               | Class ID register. These 8 bits specify the feature set of this device:<br><div> <div>TMS320F28069 0x009F</div> <div>TMS320F28068 0x008F</div> <div>TMS320F28067 0x008F</div> <div>TMS320F28066 0x008F</div> <div>TMS320F28065 0x007F</div> <div>TMS320F28064 0x006F</div> <div>TMS320F28063 0x006F</div> <div>TMS320F28062 0x006F</div> </div> |

(1) The reset value depends on the device as indicated in the register description.

**Figure 1-92. REVID Register**

15

0

|       |
|-------|
| REVID |
|-------|

R-(1)

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 1-115. REVID Register Field Descriptions**

| Bits | Field | Value          | Description                                                                                                                                                                                                                                                                                                                                                                                                |
|------|-------|----------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-0 | REVID | <sup>(1)</sup> | These 16 bits specify the silicon revision number for the particular part. This number always starts with 0x0000. It is typically incremented when major changes are made to the silicon. The silicon revision information is also part of the package symbolization. It may be used to discern the revision information in cases where the REVID is not incremented from one silicon revision to another. |
|      |       | 0x0000         | Silicon Revision 0 - TMX                                                                                                                                                                                                                                                                                                                                                                                   |
|      |       | 0x0001         | Silicon Revision A - TMS                                                                                                                                                                                                                                                                                                                                                                                   |

(1) The reset value depends on the silicon revision as described in the register field description.

### 1.5.4 Write-Followed-by-Read Protection

The memory address range for which CPU write followed by read operations are protected is 0x4000 - 0x7FFF (operations occur in sequence rather than in their natural pipeline order). This is necessary protection for certain peripheral operations.

**Example:** The following lines of code perform a write to register 1 (REG1) location and then the next instruction performs a read from Register 2 (REG2) location. On the processor memory bus, with block protection disabled, the read operation is issued before the write as shown.

```
MOV @REG1,AL      -----+
TBIT @REG2,#BIT_X -----|-----> Read
                        +-----> Write
If block protection is enabled, then the read is stalled until the write occurs as shown:
MOV      @REG1,AL      -----+
TBIT      @REG2,#BIT_X -----|-----+
                        +-----|---> Write
                        +---> Read
```



## 1.6 Peripheral Interrupt Expansion (PIE)

The peripheral interrupt expansion (PIE) block multiplexes numerous interrupt sources into a smaller set of interrupt inputs. The PIE block can support 96 individual interrupts that are grouped into blocks of eight. Each group is fed into one of 12 core interrupt lines (INT1 to INT12). Each of the 96 interrupts is supported by its own vector stored in a dedicated RAM block that you can modify. The CPU, upon servicing the interrupt, automatically fetches the appropriate interrupt vector. It takes nine CPU clock cycles to fetch the vector and save critical CPU registers. Therefore, the CPU can respond quickly to interrupt events. Prioritization of interrupts is controlled in hardware and software. Each individual interrupt can be enabled/disabled within the PIE block.

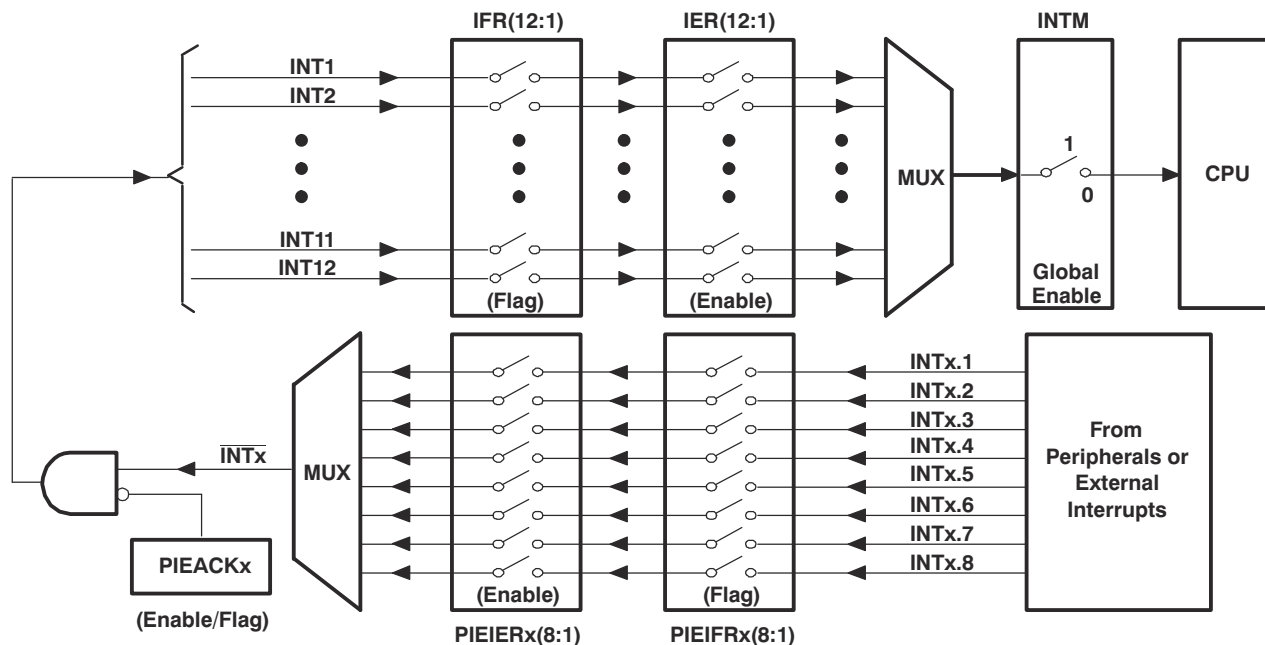
### 1.6.1 Overview of the PIE Controller

The 28x CPU supports one nonmaskable interrupt (NMI) and 16 maskable prioritized interrupt requests (INT1-INT14, RTOSINT, and DLOGINT) at the CPU level. The 28x devices have many peripherals and each peripheral is capable of generating one or more interrupts in response to many events at the peripheral level. Because the CPU does not have sufficient capacity to handle all peripheral interrupt requests at the CPU level, a centralized peripheral interrupt expansion (PIE) controller is required to arbitrate the interrupt requests from various sources such as peripherals and other external pins.

The PIE vector table is used to store the address (vector) of each interrupt service routine (ISR) within the system. There is one vector per interrupt source including all MUXed and nonMUXed interrupts. You populate the vector table during device initialization and you can update it during operation.

#### 1.6.1.1 Interrupt Operation Sequence

Figure 1-93 shows an overview of the interrupt operation sequence for all multiplexed PIE interrupts. Interrupt sources that are not multiplexed are fed directly to the CPU.



**Figure 1-93. Overview: Multiplexing of Interrupts Using the PIE Block**

- **Peripheral Level**

An interrupt-generating event occurs in a peripheral. The interrupt flag (IF) bit corresponding to that event is set in a register for that particular peripheral.

If the corresponding interrupt enable (IE) bit is set, the peripheral generates an interrupt request to the PIE controller. If the interrupt is not enabled at the peripheral level, then the IF remains set until cleared by software. If the interrupt is enabled at a later time, and the interrupt flag is still set, the interrupt request is asserted to the PIE.

Interrupt flags within the peripheral registers must be manually cleared. See the peripheral reference guide for a specific peripheral for more information.

- **PIE Level**

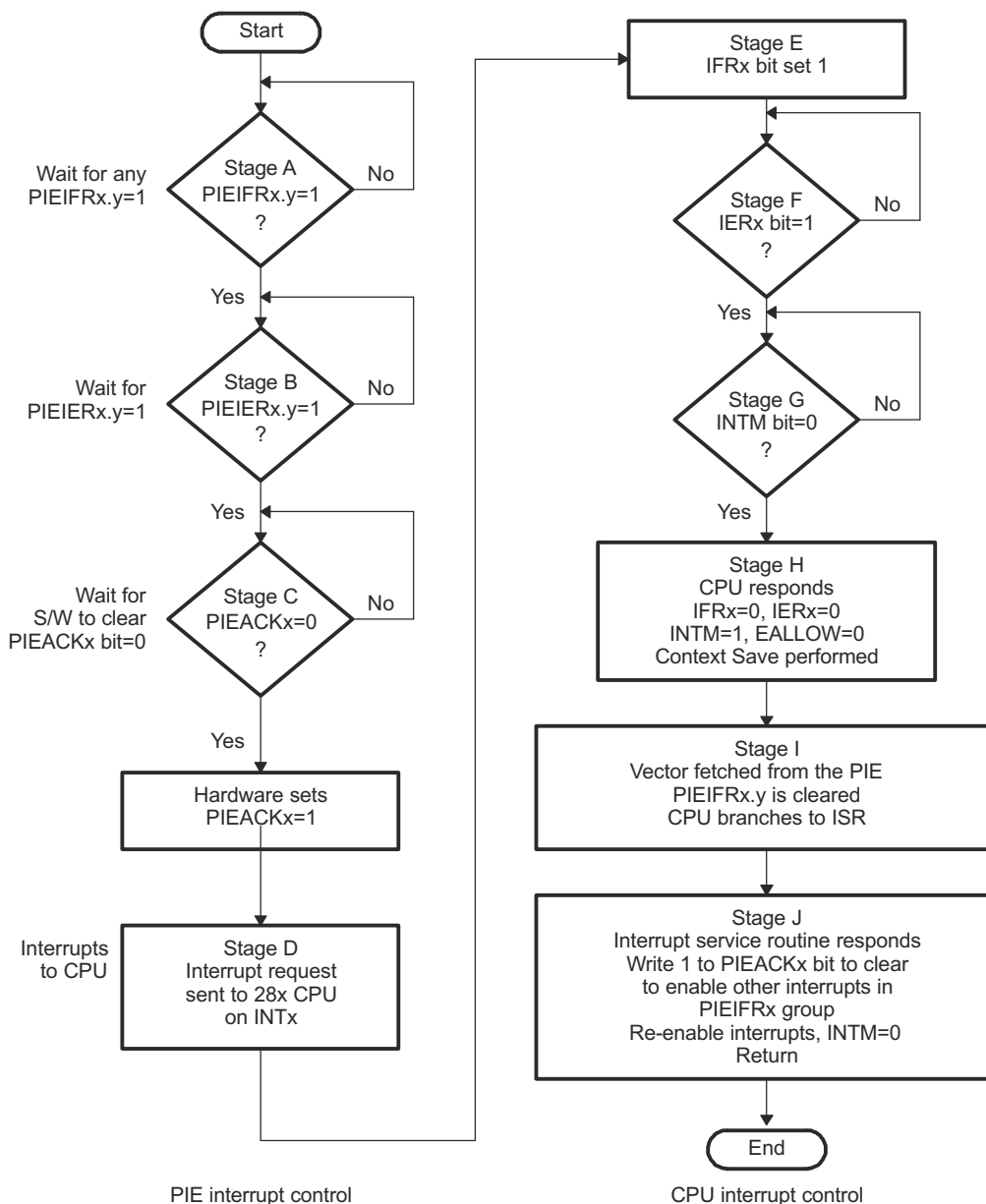
The PIE block multiplexes eight peripheral and external pin interrupts into one CPU interrupt. These interrupts are divided into 12 groups: PIE group 1 - PIE group 12. The interrupts within a group are multiplexed into one CPU interrupt. For example, PIE group 1 is multiplexed into CPU interrupt 1 (INT1) while PIE group 12 is multiplexed into CPU interrupt 12 (INT12). Interrupt sources connected to the remaining CPU interrupts are not multiplexed. For the non-multiplexed interrupts, the PIE passes the request directly to the CPU.

For multiplexed interrupt sources, each interrupt group in the PIE block has an associated flag register (PIEIFRx) and enable (PIEIERx) register (x = PIE group 1 - PIE group 12). Each bit, referred to as y, corresponds to one of the 8 MUXed interrupts within the group. Thus PIEIFRx.y and PIEIERx.y correspond to interrupt y (y = 1-8) in PIE group x (x = 1-12). In addition, there is one acknowledge bit (PIEACK) for every PIE interrupt group referred to as PIEACKx (x = 1-12). [Figure 1-94](#) illustrates the behavior of the PIE hardware under various PIEIFR and PIEIER register conditions.

Once the request is made to the PIE controller, the corresponding PIE interrupt flag (PIEIFRx.y) bit is set. If the PIE interrupt enable (PIEIERx.y) bit is also set for the given interrupt then the PIE checks the corresponding PIEACKx bit to determine if the CPU is ready for an interrupt from that group. If the PIEACKx bit is clear for that group, then the PIE sends the interrupt request to the CPU. If PIEACKx is set, then the PIE waits until it is cleared to send the request for INTx. [See Section 1.7.3 for details.](#)

- **CPU Level**

Once the request is sent to the CPU, the CPU level interrupt flag (IFR) bit corresponding to INTx is set. After a flag has been latched in the IFR, the corresponding interrupt is not serviced until it is appropriately enabled in the CPU interrupt enable (IER) register or the debug interrupt enable register (DBGIER) and the global interrupt mask (INTM) bit.



- A. For multiplexed interrupts, the PIE responds with the highest priority interrupt that is both flagged and enabled. If there is no interrupt both flagged and enabled, then the highest priority interrupt within the group (INTx.1 where x is the PIE group) is used. See [Section 1.6.3.3](#) for details.

**Figure 1-94. Typical PIE/CPU Interrupt Response - INTx.y**

As shown in [Table 1-116](#), the requirements for enabling the maskable interrupt at the CPU level depends on the interrupt handling process being used. In the standard process, which happens most of the time, the DBGIER register is not used. When the 28x is in real-time emulation mode and the CPU is halted, a different process is used. In this special case, the DBGIER is used and the INTM bit is ignored. If the DSP is in real-time mode and the CPU is running, the standard interrupt-handling process applies.

**Table 1-116. Enabling Interrupt**

| Interrupt Handling Process       | Interrupt Enabled If...         |
|----------------------------------|---------------------------------|
| Standard                         | INTM = 0 and bit in IER is 1    |
| DSP in real-time mode and halted | Bit in IER is 1 and DBGIER is 1 |

The CPU then prepares to service the interrupt. This preparation process is described in detail in [TMS320C28x DSP CPU and Instruction Set Reference Guide](#). In preparation, the corresponding CPU IFR and IER bits are cleared, EALLOW and LOOP are cleared, INTM and DBGM are set, the pipeline is flushed and the return address is stored, and the automatic context save is performed. The vector of the ISR is then fetched from the PIE module. If the interrupt request comes from a multiplexed interrupt, the PIE module uses the group PIEIERx and PIEIFRx registers to decode which interrupt needs to be serviced. This decode process is described in [Section 1.6.3.3](#).

The address for the interrupt service routine that is executed is fetched directly from the PIE interrupt vector table. There is one 32-bit vector for each of the possible 96 interrupts within the PIE. Interrupt flags within the PIE module (PIEIFRx.y) are automatically cleared when the interrupt vector is fetched. The PIE acknowledge bit for a given interrupt group, however, must be cleared manually when ready to receive more interrupts from the PIE group.

## 1.6.2 Vector Table Mapping

On 28xx devices, the interrupt vector table can be mapped to four distinct locations in memory. In practice only the PIE vector table mapping is used.

This vector mapping is controlled by the following mode bits/signals:

|          |                                                                                                                                                                                                                                                                                                     |
|----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| VMAP:    | VMAP is found in the Status Register 1 ST1 (bit 3). A device reset sets this bit to 1. The state of this bit can be modified by writing to ST1 or by SETC/CLRC VMAP instructions. For normal operation leave this bit set.                                                                          |
| M0M1MAP: | M0M1MAP is found in the Status Register 1 ST1 (bit 11). A device reset sets this bit to 1. The state of this bit can be modified by writing to ST1 or by SETC/CLRC M0M1MAP instructions. For normal 28xx device operation, this bit should remain set. M0M1MAP = 0 is reserved for TI testing only. |
| ENPIE:   | ENPIE is found in the PIECTRL Register (bit 0). The default value of this bit, on reset, is set to 0 (PIE disabled). The state of this bit can be modified after reset by writing to the PIECTRL register (address 0x0000 0CE0).                                                                    |

Using these bits and signals the possible vector table mappings are shown in [Table 1-117](#).

**Table 1-117. Interrupt Vector Table Mapping**

| Vector MAPS              | Vectors Fetched From | Address Range         | VMAP | M0M1MAP | ENPIE |
|--------------------------|----------------------|-----------------------|------|---------|-------|
| M1 Vector <sup>(1)</sup> | M1 SARAM Block       | 0x000000 - 0x00003F   | 0    | 0       | X     |
| M0 Vector <sup>(1)</sup> | M0 SARAM Block       | 0x000000 - 0x00003F   | 0    | 1       | X     |
| BROM Vector              | Boot ROM Block       | 0x3FFFC0 - 0x3FFFFFFF | 1    | X       | 0     |
| PIE Vector               | PIE Block            | 0x000D00 - 0x000DFF   | 1    | X       | 1     |

(1) Vector map M0 and M1 Vector is a reserved mode only. On the 28x devices these are used as SARAM.

The M1 and M0 vector table mapping are reserved for TI testing only. When using other vector mappings, the M0 and M1 memory blocks are treated as SARAM blocks and can be used freely without any restrictions.

After a device reset operation, the vector table is mapped as shown in [Table 1-118](#).

**Table 1-118. Vector Table Mapping After Reset Operation**

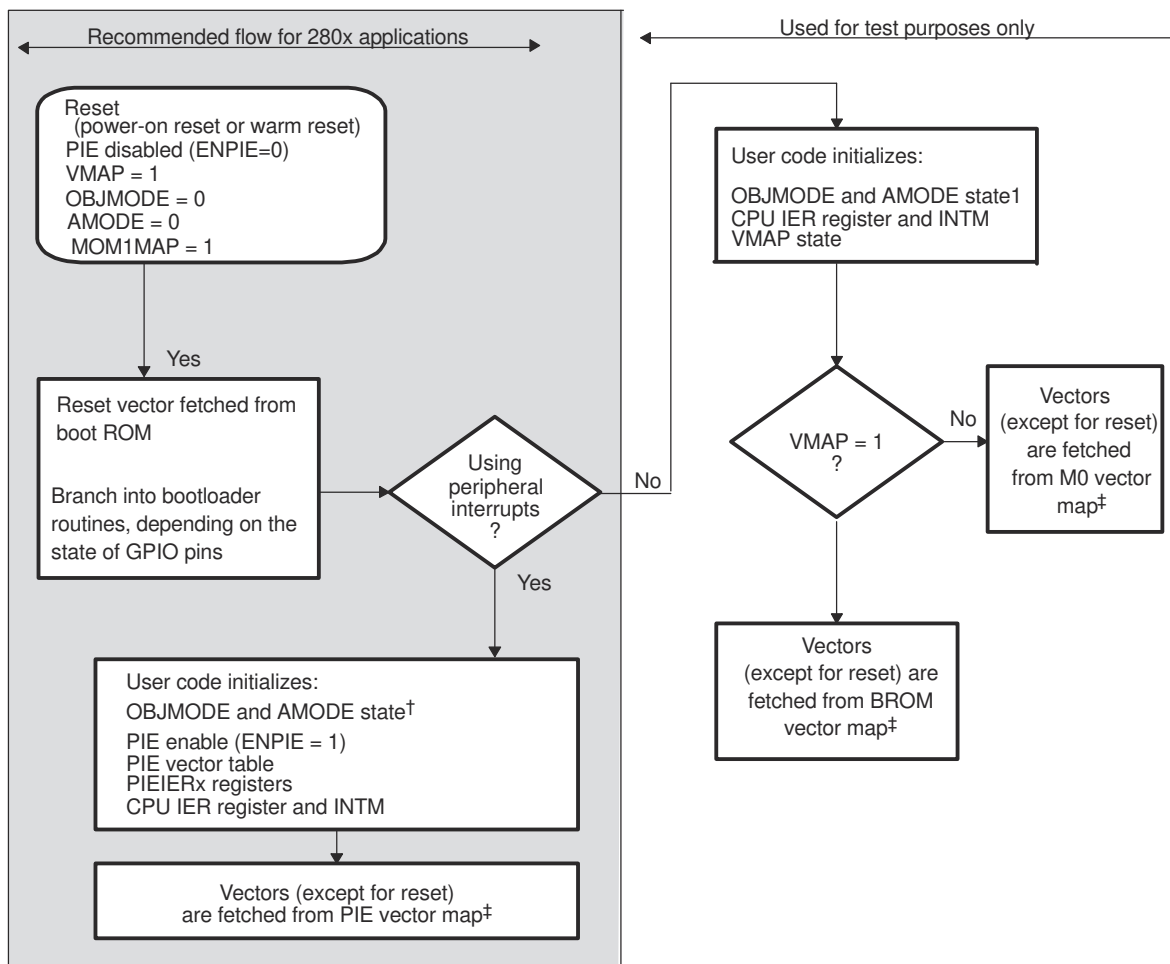
| Vector MAPS                | Reset Fetched From | Address Range         | VMAP <sup>(1)</sup> | M0M1MAP <sup>(1)</sup> | ENPIE <sup>(1)</sup> |
|----------------------------|--------------------|-----------------------|---------------------|------------------------|----------------------|
| BROM Vector <sup>(2)</sup> | Boot ROM Block     | 0x3FFFC0 - 0x3FFFFFFF | 1                   | 1                      | 0                    |

(1) On the 28x devices, the VMAP and M0M1MAP modes are set to 1 on reset. The ENPIE mode is forced to 0 on reset.

(2) The reset vector is always fetched from the boot ROM.

After the reset and boot is complete, the PIE vector table should be initialized by the user's code. Then the application enables the PIE vector table. From that point on the interrupt vectors are fetched from the PIE vector table. Note: when a reset occurs, the reset vector is always fetched from the vector table as shown in [Table 1-118](#). After a reset the PIE vector table is always disabled.

[Figure 1-95](#) illustrates the process by which the vector table mapping is selected.



- A. The compatibility operating mode of the 28x CPU is determined by a combination of the OBJMODE and AMODE bits in Status Register 1 (ST1):

| Operating Mode              | OBJMODE | AMODE |                    |
|-----------------------------|---------|-------|--------------------|
| C28x Mode                   | 1       | 0     |                    |
| 24x/240xA Source-Compatible | 1       | 1     |                    |
| C27x Object-Compatible      | 0       | 0     | (Default at reset) |

- B. The reset vector is always fetched from the boot ROM.

**Figure 1-95. Reset Flow Diagram**

### 1.6.3 Interrupt Sources

Figure 1-96 shows how the various interrupt sources are multiplexed within the devices. This multiplexing (MUX) scheme may not be exactly the same on all C28x devices. See the device data sheet for details.

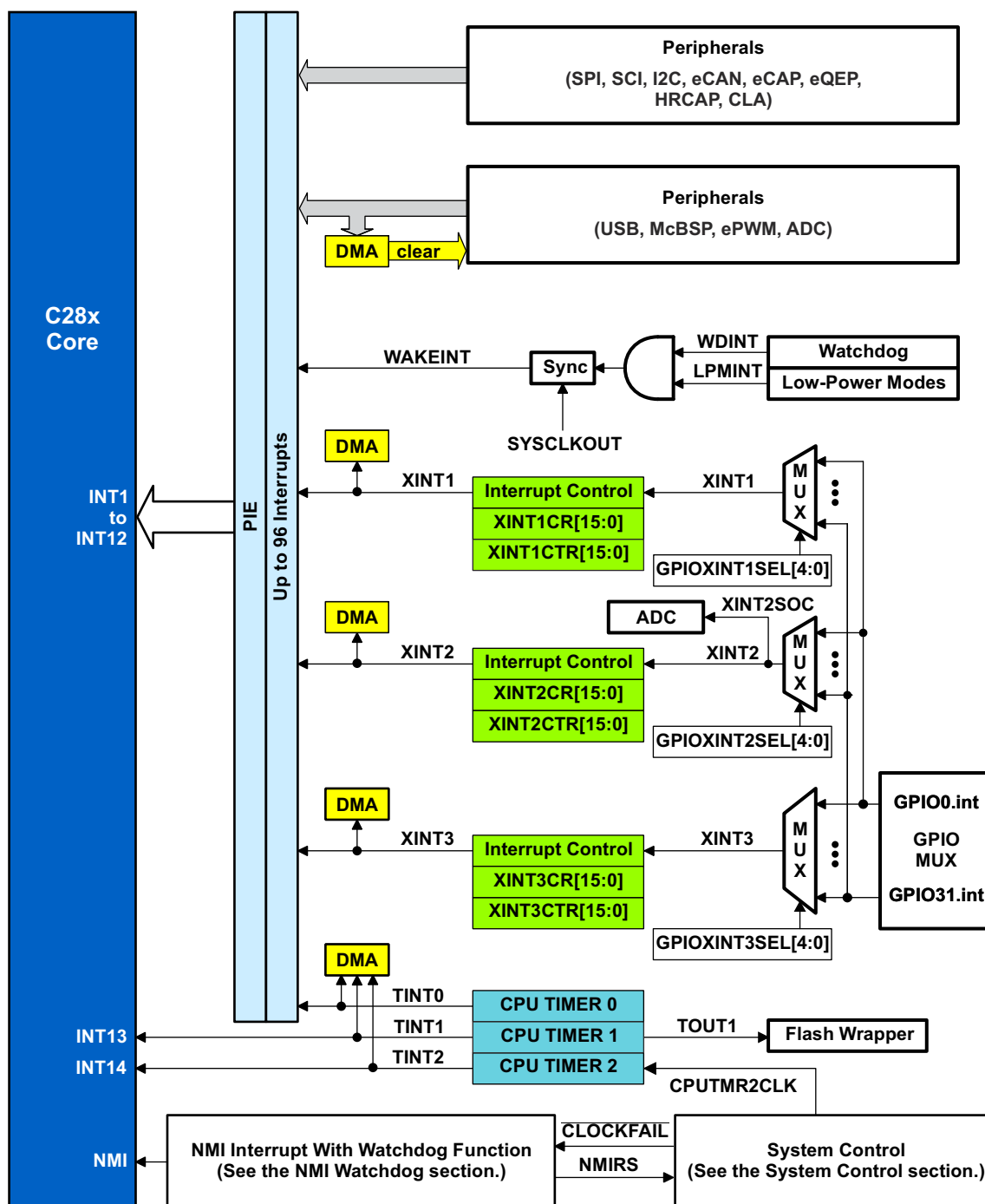


Figure 1-96. PIE Interrupt Sources and External Interrupts XINT1/XINT2/XINT3

### 1.6.3.1 Procedure for Handling Multiplexed Interrupts

The PIE module multiplexes eight peripheral and external pin interrupts into one CPU interrupt. These interrupts are divided into 12 groups: PIE group 1 - PIE group 12. Each group has an associated enable PIEIER and flag PIEIFR register. These registers are used to control the flow of interrupts to the CPU. The PIE module also uses the PIEIER and PIEIFR registers to decode to which interrupt service routine the CPU should branch.

There are three main rules that should be followed when clearing bits within the PIEIFR and the PIEIER registers:

#### **Rule 1: Never clear a PIEIFR bit by software**

An incoming interrupt may be lost while a write or a read-modify-write operation to the PIEIFR register takes place. To clear a PIEIFR bit, the pending interrupt must be serviced. If you want to clear the PIEIFR bit without executing the normal service routine, then use the following procedure:

1. Set the EALLOW bit to allow modification to the PIE vector table.
2. Modify the PIE vector table so that the vector for the peripheral's service routine points to a temporary ISR. This temporary ISR will only perform a return from interrupt (IRET) operation.
3. Enable the interrupt so that the interrupt will be serviced by the temporary ISR.
4. After the temporary interrupt routine is serviced, the PIEIFR bit will be clear
5. Modify the PIE vector table to re-map the peripheral's service routine to the proper service routine.
6. Clear the EALLOW bit.

#### **Rule 2: Procedure for software-prioritizing interrupts**

Refer to the method shown in the C2000Ware example for software prioritization of interrupts.

1. Use the CPU IER register as a global priority and the individual PIEIER registers for group priorities. In this case the PIEIER register is only modified within an interrupt. In addition, only the PIEIER for the same group as the interrupt being serviced is modified. This modification is done while the PIEACK bit holds additional interrupts back from the CPU.
2. Never disable a PIEIER bit for a group when servicing an interrupt from an unrelated group.

#### **Rule 3: Disabling interrupts using PIEIER**

If the PIEIER registers are used to enable and then later disable an interrupt then the procedure described in [Section 1.6.3.2](#) must be followed.

### 1.6.3.2 Procedures for Enabling And Disabling Multiplexed Peripheral Interrupts

The proper procedure for enabling or disabling an interrupt is by using the peripheral interrupt enable/disable flags. The primary purpose of the PIEIER and CPU IER registers is for software prioritization of interrupts within the same PIE interrupt group. Refer to the method shown in the C2000Ware example for software prioritization of interrupts.

Should bits within the PIEIER registers need to be cleared outside of this context, one of the following two procedures should be followed. The first method preserves the associated PIE flag register so that interrupts are not lost. The second method clears the associated PIE flag register.

#### **Method 1: Use the PIEIERx register to disable the interrupt and preserve the associated PIEIFRx flags.**

To clear bits within a PIEIERx register while preserving the associated flags in the PIEIFRx register, the following procedure should be followed:

1. Disable global interrupts (INTM = 1).
2. Clear the PIEIERx.y bit to disable the interrupt for a given peripheral. This can be done for one or more peripherals within the same group.
3. Wait 5 cycles. This delay is required to be sure that any interrupt that was incoming to the CPU has been flagged within the CPU IFR register.
4. Clear the CPU IFRx bit for the peripheral group. This is a safe operation on the CPU IFR register.
5. Clear the PIEACKx bit for the peripheral group.
6. Enable global interrupts (INTM = 0).

#### **Method 2: Use the PIEIERx register to disable the interrupt and clear the associated PIEIFRx flags.**

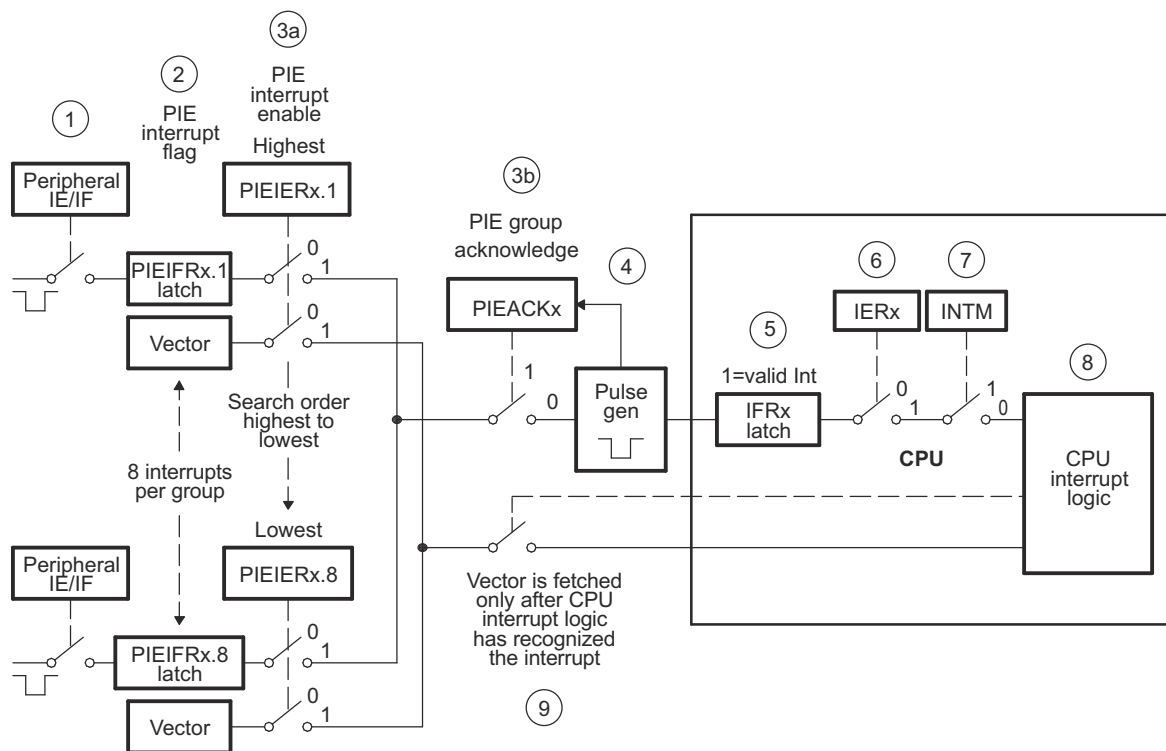
To perform a software reset of a peripheral interrupt and clear the associated flag in the PIEIFRx register and CPU IFR register, the following procedure should be followed:

1. Disable global interrupts (INTM = 1).
2. Set the EALLOW bit.
3. Modify the PIE vector table to temporarily map the vector of the specific peripheral interrupt to a empty interrupt service routine (ISR). This empty ISR will only perform a return from interrupt (IRET) instruction. This is the safe way to clear a single PIEIFRx.y bit without losing any interrupts from other peripherals within the group.
4. Disable the peripheral interrupt at the peripheral register.
5. Enable global interrupts (INTM = 0).
6. Wait for any pending interrupt from the peripheral to be serviced by the empty ISR routine.
7. Disable global interrupts (INTM = 1).
8. Modify the PIE vector table to map the peripheral vector back to its original ISR.
9. Clear the EALLOW bit.
10. Disable the PIEIER bit for given peripheral.
11. Clear the IFR bit for given peripheral group (this is safe operation on CPU IFR register).
12. Clear the PIEACK bit for the PIE group.
13. Enable global interrupts.



### 1.6.3.3 Flow of a Multiplexed Interrupt Request From a Peripheral to the CPU

Figure 1-97 shows the flow with the steps shown in circled numbers. Following the diagram, the steps are described.



**Figure 1-97. Multiplexed Interrupt Request Flow Diagram**

- Any peripheral or external interrupt within the PIE group generates an interrupt. If interrupts are enabled within the peripheral module then the interrupt request is sent to the PIE module.
- The PIE module recognizes that interrupt y within PIE group x (INTx.y) has asserted an interrupt and the appropriate PIE interrupt flag bit is latched: PIEIFRx.y = 1.
- For the interrupt request to be sent from the PIE to the CPU, both of the following conditions must be true:
  - The proper enable bit must be set (PIEIERx.y = 1) and
  - The PIEACKx bit for the group must be clear.
- If both conditions in 3a and 3b are true, then an interrupt request is sent to the CPU and the acknowledge bit is again set (PIEACKx = 1). The PIEACKx bit will remain set until you clear it to indicate that additional interrupts from the group can be sent from the PIE to the CPU.
- The CPU interrupt flag bit is set (CPU IFRx = 1) to indicate a pending interrupt x at the CPU level.
- If the CPU interrupt is enabled (CPU IER bit x = 1, or DBGIER bit x = 1) AND the global interrupt mask is clear (INTM = 0) then the CPU will service the INTx.
- The CPU recognizes the interrupt and performs the automatic context save, clears the IER bit, sets INTM, and clears EALLOW. All of the steps that the CPU takes in order to prepare to service the interrupt are documented in the *TM S320C28x DSP CPU and Instruction Set Reference Guide* (literature number SPRU430).
- The CPU will then request the appropriate vector from the PIE.
- For multiplexed interrupts, the PIE module uses the current value in the PIEIERx and PIEIFRx registers to decode which vector address should be used. There are two possible cases:
  - The vector for the highest priority interrupt within the group that is both enabled in the PIEIERx register, and flagged as pending in the PIEIFRx is fetched and used as the branch address. In this manner if an even higher priority enabled interrupt was flagged after Step 7, it will be serviced first.

- b. If no flagged interrupts within the group are enabled, then the PIE will respond with the vector for the highest priority interrupt within that group. That is the branch address used for INTx.1. This behavior corresponds to the 28x TRAP or INT instructions.

### Note

Because the PIEIERx register is used to determine which vector will be used for the branch, you must take care when clearing bits within the PIEIERx register. The proper procedure for clearing bits within a PIEIERx register is described in [Section 1.6.3.2](#). Failure to follow these steps can result in changes occurring to the PIEIERx register after an interrupt has been passed to the CPU at Step 5 in Figure 6-5. In this case, the PIE will respond as if a TRAP or INT instruction was executed unless there are other interrupts both pending and enabled.

At this point, the PIEIFRx.y bit is cleared and the CPU branches to the vector of the interrupt fetched from the PIE.

### 1.6.3.4 The PIE Vector Table

The PIE vector table (see [Table 1-120](#)) consists of a 256 x 16 SARAM block that can also be used as RAM (in data space only) if the PIE block is not in use. The PIE vector table contents are undefined on reset. The CPU fixes interrupt priority for INT1 to INT12. The PIE controls priority for each group of eight interrupts. For example, if INT1.1 should occur simultaneously with INT8.1, both interrupts are presented to the CPU simultaneously by the PIE block, and the CPU services INT1.1 first. If INT1.1 should occur simultaneously with INT1.8, then INT1.1 is sent to the CPU first and then INT1.8 follows. Interrupt prioritization is performed during the vector fetch portion of the interrupt processing.

When the PIE is enabled, a TRAP #1 through TRAP #12 or an INTR INT1 to INTR INT12 instruction transfers program control to the interrupt service routine corresponding to the first vector within the PIE group. For example: TRAP #1 fetches the vector from INT1.1, TRAP #2 fetches the vector from INT2.1 and so forth. Similarly an OR IFR, #16-bit operation causes the vector to be fetched from INTR1.1 to INTR12.1 locations, if the respective interrupt flag is set. All other TRAP, INTR, OR IFR, #16-bit operations fetch the vector from the respective table location. The vector table is EALLOW protected.

Out of the 96 possible MUXed interrupts in [Table 1-119](#), 43 interrupts are currently used. The remaining interrupts are reserved for future devices. These reserved interrupts can be used as software interrupts if they are enabled at the PIEIFRx level, provided none of the interrupts within the group is being used by a peripheral. Otherwise, interrupts coming from peripherals may be lost by accidentally clearing their flags when modifying the PIEIFR.

To summarize, there are two safe cases when the reserved interrupts can be used as software interrupts:

1. No peripheral within the group is asserting interrupts.
2. No peripheral interrupts are assigned to the group. For example, PIE group 11 and 12 do not have any peripherals attached to them.

The interrupt grouping for peripherals and external interrupts connected to the PIE module is shown in [Table 1-119](#). Each row in the table shows the 8 interrupts multiplexed into a particular CPU interrupt. The entire PIE vector table, including both MUXed and non-MUXed interrupts, is shown in [Table 1-120](#).

**Table 1-119. PIE MUXed Peripheral Interrupt Vector Table**

|                | INTx.8                               | INTx.7                          | INTx.6                          | INTx.5                          | INTx.4                          | INTx.3                          | INTx.2                          | INTx.1                          |
|----------------|--------------------------------------|---------------------------------|---------------------------------|---------------------------------|---------------------------------|---------------------------------|---------------------------------|---------------------------------|
| <b>INT1.y</b>  | WAKEINT<br>(LPM/WD)<br>0xD4E         | TINT0<br>(TIMER 0)<br>0xD4C     | ADCINT9<br>(ADC)<br>0xD4A       | XINT2<br>Ext. int. 2<br>0xD48   | XINT1<br>Ext. int. 1<br>0xD46   | Reserved<br>–<br>0xD44          | ADCINT2<br>(ADC)<br>0xD42       | ADCINT1<br>(ADC)<br>0xD40       |
| <b>INT2.y</b>  | EPWM8_TZINT<br>(ePWM8)<br>0xD5E      | EPWM7_TZINT<br>(ePWM7)<br>0xD5C | EPWM6_TZINT<br>(ePWM6)<br>0xD5A | EPWM5_TZINT<br>(ePWM5)<br>0xD58 | EPWM4_TZINT<br>(ePWM4)<br>0xD56 | EPWM3_TZINT<br>(ePWM3)<br>0xD54 | EPWM2_TZINT<br>(ePWM2)<br>0xD52 | EPWM1_TZINT<br>(ePWM1)<br>0xD50 |
| <b>INT3.y</b>  | EPWM8_INT<br>(ePWM8)<br>0xD6E        | EPWM7_INT<br>(ePWM7)<br>0xD6C   | EPWM6_INT<br>(ePWM6)<br>0xD6A   | EPWM5_INT<br>(ePWM5)<br>0xD68   | EPWM4_INT<br>(ePWM4)<br>0xD66   | EPWM3_INT<br>(ePWM3)<br>0xD64   | EPWM2_INT<br>(ePWM2)<br>0xD62   | EPWM1_INT<br>(ePWM1)<br>0xD60   |
| <b>INT4.y</b>  | HRCAP2_INT<br>(HRCAP2)<br>0xD7E      | HRCAP1_INT<br>(HRCAP1)<br>0xD7C | Reserved<br>–<br>0xD7A          | Reserved<br>–<br>0xD78          | Reserved<br>–<br>0xD76          | ECAP3_INT<br>(eCAP3)<br>0xD74   | ECAP2_INT<br>(eCAP2)<br>0xD72   | ECAP1_INT<br>(eCAP1)<br>0xD70   |
| <b>INT5.y</b>  | Reserved<br>USB0_INT (USB0)<br>0xD8E | Reserved<br>–<br>0xD8C          | Reserved<br>–<br>0xD8A          | HRCAP4_INT<br>(HRCAP4)<br>0xD88 | HRCAP3_INT<br>(HRCAP3)<br>0xD86 | Reserved<br>–<br>0xD84          | EQEP2_INT<br>(eQEP2)<br>0xD82   | EQEP1_INT<br>(eQEP1)<br>0xD80   |
| <b>INT6.y</b>  | Reserved<br>–<br>0xD9E               | Reserved<br>–<br>0xD9C          | MXINTA<br>(McBSP-A)<br>0xD9A    | MRINTA<br>(McBSP-A)<br>0xD98    | SPITXINTB<br>(SPI-B)<br>0xD96   | SPIRXINTB<br>(SPI-B)<br>0xD94   | SPITXINTA<br>(SPI-A)<br>0xD92   | SPIRXINTA<br>(SPI-A)<br>0xD90   |
| <b>INT7.y</b>  | Reserved<br>–<br>0xDAE               | Reserved<br>–<br>0xDAC          | DINTCH6<br>(DMA)<br>0xDAA       | DINTCH5<br>(DMA)<br>0xDA8       | DINTCH4<br>(DMA)<br>0xDA6       | DINTCH3<br>(DMA)<br>0xDA4       | DINTCH2<br>(DMA)<br>0xDA2       | DINTCH1<br>(DMA)<br>0xDA0       |
| <b>INT8.y</b>  | Reserved<br>–<br>0xDBE               | Reserved<br>–<br>0xDBC          | Reserved<br>–<br>0xDBA          | Reserved<br>–<br>0xDB8          | Reserved<br>–<br>0xDB6          | Reserved<br>–<br>0xDB4          | I2CINT2A<br>(I2C-A)<br>0xDB2    | I2CINT1A<br>(I2C-A)<br>0xDB0    |
| <b>INT9.y</b>  | Reserved<br>–<br>0xDCE               | Reserved<br>–<br>0xDCC          | ECAN1_INT1<br>(CAN-A)<br>0xDCA  | ECAN0_INT0<br>(CAN-A)<br>0xDC8  | SCITXINTB<br>(SCI-B)<br>0xDC6   | SCIRXINTB<br>(SCI-B)<br>0xDC4   | SCITXINTA<br>(SCI-A)<br>0xDC2   | SCIRXINTA<br>(SCI-A)<br>0xDC0   |
| <b>INT10.y</b> | ADCINT8<br>(ADC)<br>0xDDE            | ADCINT7<br>(ADC)<br>0xDDC       | ADCINT6<br>(ADC)<br>0xDDA       | ADCINT5<br>(ADC)<br>0xDD8       | ADCINT4<br>(ADC)<br>0xDD6       | ADCINT3<br>(ADC)<br>0xDD4       | ADCINT2<br>(ADC)<br>0xDD2       | ADCINT1<br>(ADC)<br>0xDD0       |
| <b>INT11.y</b> | CLA1_INT8<br>(CLA)<br>0xDEE          | CLA1_INT7<br>(CLA)<br>0xDEC     | CLA1_INT6<br>(CLA)<br>0xDEA     | CLA1_INT5<br>(CLA)<br>0xDE8     | CLA1_INT4<br>(CLA)<br>0xDE6     | CLA1_INT3<br>(CLA)<br>0xDE4     | CLA1_INT2<br>(CLA)<br>0xDE2     | CLA1_INT1<br>(CLA)<br>0xDE0     |
| <b>INT12.y</b> | LUF<br>(CLA, FPU32)<br>0xDFE         | LVF<br>(CLA, FPU32)<br>0xDFC    | Reserved<br>–<br>0xDFA          | Reserved<br>–<br>0xDF8          | Reserved<br>–<br>0xDF6          | Reserved<br>–<br>0xDF4          | Reserved<br>–<br>0xDF2          | XINT3<br>Ext. Int. 3<br>0xDF0   |

**Table 1-120. PIE Vector Table**

| Name                                             | VECTOR ID | Address <sup>(1)</sup> | Size (x16) | Description <sup>(2)</sup>                                     | CPU Priority | PIE Group Priority |
|--------------------------------------------------|-----------|------------------------|------------|----------------------------------------------------------------|--------------|--------------------|
| Reset                                            | 0         | 0x0000 0D00            | 2          | Reset is always fetched from location 0x003F FFC0 in Boot ROM. | 1 (highest)  | -                  |
| INT1                                             | 1         | 0x0000 0D02            | 2          | Not used. See PIE Group 1                                      | 5            | -                  |
| INT2                                             | 2         | 0x0000 0D04            | 2          | Not used. See PIE Group 2                                      | 6            | -                  |
| INT3                                             | 3         | 0x0000 0D06            | 2          | Not used. See PIE Group 3                                      | 7            | -                  |
| INT4                                             | 4         | 0x0000 0D08            | 2          | Not used. See PIE Group 4                                      | 8            | -                  |
| INT5                                             | 5         | 0x0000 0D0A            | 2          | Not used. See PIE Group 5                                      | 9            | -                  |
| INT6                                             | 6         | 0x0000 0D0C            | 2          | Not used. See PIE Group 6                                      | 10           | -                  |
| INT7                                             | 7         | 0x0000 0D0E            | 2          | Not used. See PIE Group 7                                      | 11           | -                  |
| INT8                                             | 8         | 0x0000 0D10            | 2          | Not used. See PIE Group 8                                      | 12           | -                  |
| INT9                                             | 9         | 0x0000 0D12            | 2          | Not used. See PIE Group 9                                      | 13           | -                  |
| INT10                                            | 10        | 0x0000 0D14            | 2          | Not used. See PIE Group 10                                     | 14           | -                  |
| INT11                                            | 11        | 0x0000 0D16            | 2          | Not used. See PIE Group 11                                     | 15           | -                  |
| INT12                                            | 12        | 0x0000 0D18            | 2          | Not used. See PIE Group 12                                     | 16           | -                  |
| INT13                                            | 13        | 0x0000 0D1A            | 2          | External Interrupt 13 (XINT13) or CPU-Timer1                   | 17           | -                  |
| INT14                                            | 14        | 0x0000 0D1C            | 2          | CPU-Timer2 (for TI/RTOS use)                                   | 18           | -                  |
| DATALOG                                          | 15        | 0x0000 0D1E            | 2          | CPU Data Logging Interrupt                                     | 19 (lowest)  | -                  |
| RTOSINT                                          | 16        | 0x0000 0D20            | 2          | CPU Real-Time OS Interrupt                                     | 4            | -                  |
| EMUINT                                           | 17        | 0x0000 0D22            | 2          | CPU Emulation Interrupt                                        | 2            | -                  |
| NMI                                              | 18        | 0x0000 0D24            | 2          | External Non-Maskable Interrupt                                | 3            | -                  |
| ILLEGAL                                          | 19        | 0x0000 0D26            | 2          | Illegal Operation                                              | -            | -                  |
| USER1                                            | 20        | 0x0000 0D28            | 2          | User-Defined Trap                                              | -            | -                  |
| USER2                                            | 21        | 0x0000 0D2A            | 2          | User Defined Trap                                              | -            | -                  |
| USER3                                            | 22        | 0x0000 0D2C            | 2          | User Defined Trap                                              | -            | -                  |
| USER4                                            | 23        | 0x0000 0D2E            | 2          | User Defined Trap                                              | -            | -                  |
| USER5                                            | 24        | 0x0000 0D30            | 2          | User Defined Trap                                              | -            | -                  |
| USER6                                            | 25        | 0x0000 0D32            | 2          | User Defined Trap                                              | -            | -                  |
| USER7                                            | 26        | 0x0000 0D34            | 2          | User Defined Trap                                              | -            | -                  |
| USER8                                            | 27        | 0x0000 0D36            | 2          | User Defined Trap                                              | -            | -                  |
| USER9                                            | 28        | 0x0000 0D38            | 2          | User Defined Trap                                              | -            | -                  |
| USER10                                           | 29        | 0x0000 0D3A            | 2          | User Defined Trap                                              | -            | -                  |
| USER11                                           | 30        | 0x0000 0D3C            | 2          | User Defined Trap                                              | -            | -                  |
| USER12                                           | 31        | 0x0000 0D3E            | 2          | User Defined Trap                                              | -            | -                  |
| <b>PIE Group 1 Vectors - MUXed into CPU INT1</b> |           |                        |            |                                                                |              |                    |
| INT1.1                                           | 32        | 0x0000 0D40            | 2          | ADCINT1 (ADC)                                                  | 5            | 1 (highest)        |
| INT1.2                                           | 33        | 0x0000 0D42            | 2          | ADCINT2 (ADC)                                                  | 5            | 2                  |
| INT1.3                                           | 34        | 0x0000 0D44            | 2          | Reserved                                                       | 5            | 3                  |
| INT1.4                                           | 35        | 0x0000 0D46            | 2          | XINT1                                                          | 5            | 4                  |
| INT1.5                                           | 36        | 0x0000 0D48            | 2          | XINT2                                                          | 5            | 5                  |
| INT1.6                                           | 37        | 0x0000 0D4A            | 2          | ADCINT9 (ADC)                                                  | 5            | 6                  |
| INT1.7                                           | 38        | 0x0000 0D4C            | 2          | TINT0 (CPU-Timer0)                                             | 5            | 7                  |
| INT1.8                                           | 39        | 0x0000 0D4E            | 2          | WAKEINT (LPM/WD)                                               | 5            | 8 (lowest)         |

**Table 1-120. PIE Vector Table (continued)**

| Name                                             | VECTOR ID | Address <sup>(1)</sup> | Size (x16) | Description <sup>(2)</sup> |           | CPU Priority | PIE Group Priority |
|--------------------------------------------------|-----------|------------------------|------------|----------------------------|-----------|--------------|--------------------|
| <b>PIE Group 2 Vectors - MUXed into CPU INT2</b> |           |                        |            |                            |           |              |                    |
| INT2.1                                           | 40        | 0x0000 0D50            | 2          | EPWM1_TZINT                | (EPWM1)   | 6            | 1 (highest)        |
| INT2.2                                           | 41        | 0x0000 0D52            | 2          | EPWM2_TZINT                | (EPWM2)   | 6            | 2                  |
| INT2.3                                           | 42        | 0x0000 0D54            | 2          | EPWM3_TZINT                | (EPWM3)   | 6            | 3                  |
| INT2.4                                           | 43        | 0x0000 0D56            | 2          | EPWM4_TZINT                | (EPWM4)   | 6            | 4                  |
| INT2.5                                           | 44        | 0x0000 0D58            | 2          | EPWM5_TZINT                | (EPWM5)   | 6            | 5                  |
| INT2.6                                           | 45        | 0x0000 0D5A            | 2          | EPWM6_TZINT                | (EPWM6)   | 6            | 6                  |
| INT2.7                                           | 46        | 0x0000 0D5C            | 2          | EPWM7_TZINT                | (EPWM7)   | 6            | 7                  |
| INT2.8                                           | 47        | 0x0000 0D5E            | 2          | Reserved                   |           | 6            | 8 (lowest)         |
| <b>PIE Group 3 Vectors - MUXed into CPU INT3</b> |           |                        |            |                            |           |              |                    |
| INT3.1                                           | 48        | 0x0000 0D60            | 2          | EPWM1_INT                  | (EPWM1)   | 7            | 1 (highest)        |
| INT3.2                                           | 49        | 0x0000 0D62            | 2          | EPWM2_INT                  | (EPWM2)   | 7            | 2                  |
| INT3.3                                           | 50        | 0x0000 0D64            | 2          | EPWM3_INT                  | (EPWM3)   | 7            | 3                  |
| INT3.4                                           | 51        | 0x0000 0D66            | 2          | EPWM4_INT                  | (EPWM4)   | 7            | 4                  |
| INT3.5                                           | 52        | 0x0000 0D68            | 2          | EPWM5_INT                  | (EPWM5)   | 7            | 5                  |
| INT3.6                                           | 53        | 0x0000 0D6A            | 2          | EPWM6_INT                  | (EPWM6)   | 7            | 6                  |
| INT3.7                                           | 54        | 0x0000 0D6C            | 2          | EPWM7_INT                  | (EPWM7)   | 7            | 7                  |
| INT3.8                                           | 55        | 0x0000 0D6E            | 2          | EPWM8_INT                  | (EPWM8)   | 7            | 8 (lowest)         |
| <b>PIE Group 4 Vectors - MUXed into CPU INT4</b> |           |                        |            |                            |           |              |                    |
| INT4.1                                           | 56        | 0x0000 0D70            | 2          | ECAP1_INT                  | (ECAP1)   | 8            | 1 (highest)        |
| INT4.2                                           | 57        | 0x0000 0D72            | 2          | ECAP2_INT                  | (ECAP2)   | 8            | 2                  |
| INT4.3                                           | 58        | 0x0000 0D74            | 2          | ECAP3_INT                  | (ECAP3)   | 8            | 3                  |
| INT4.4                                           | 59        | 0x0000 0D76            | 2          | Reserved                   | -         | 8            | 4                  |
| INT4.5                                           | 60        | 0x0000 0D78            | 2          | Reserved                   | -         | 8            | 5                  |
| INT4.6                                           | 61        | 0x0000 0D7A            | 2          | Reserved                   | -         | 8            | 6                  |
| INT4.7                                           | 62        | 0x0000 0D7C            | 2          | Reserved                   | -         | 8            | 7                  |
| INT4.8                                           | 63        | 0x0000 0D7E            | 2          | Reserved                   | -         | 8            | 8 (lowest)         |
| <b>PIE Group 5 Vectors - MUXed into CPU INT5</b> |           |                        |            |                            |           |              |                    |
| INT5.1                                           | 64        | 0x0000 0D80            | 2          | EQEP1_INT                  | (EQEP1)   | 9            | 1 (highest)        |
| INT5.2                                           | 65        | 0x0000 0D82            | 2          | EQEP2_INT                  | (EQEP2)   | 9            | 2                  |
| INT5.3                                           | 66        | 0x0000 0D84            | 2          | Reserved                   |           | 9            | 3                  |
| INT5.4                                           | 67        | 0x0000 0D86            | 2          | HRCAP3INT                  | HRCAP3    | 9            | 4                  |
| INT5.5                                           | 68        | 0x0000 0D88            | 2          | HRCAP4INT                  | HRCAP4    | 9            | 5                  |
| INT5.6                                           | 69        | 0x0000 0D8A            | 2          | Reserved                   | -         | 9            | 6                  |
| INT5.7                                           | 70        | 0x0000 0D8C            | 2          | Reserved                   | -         | 9            | 7                  |
| INT5.8                                           | 71        | 0x0000 0D8E            | 2          | USB0_INT (USB0)            | -         | 9            | 8 (lowest)         |
| <b>PIE Group 6 Vectors - MUXed into CPU INT6</b> |           |                        |            |                            |           |              |                    |
| INT6.1                                           | 72        | 0x0000 0D90            | 2          | SPIRXINTA                  | (SPI-A)   | 10           | 1 (highest)        |
| INT6.2                                           | 73        | 0x0000 0D92            | 2          | SPITXINTA                  | (SPI-A)   | 10           | 2                  |
| INT6.3                                           | 74        | 0x0000 0D94            | 2          | SPIRXINTB                  | (SPI-B)   | 10           | 3                  |
| INT6.4                                           | 75        | 0x0000 0D96            | 2          | SPITXINTB                  | (SPI-B)   | 10           | 4                  |
| INT6.5                                           | 76        | 0x0000 0D98            | 2          | MRINTA                     | (McBSP-A) | 10           | 5                  |
| INT6.6                                           | 77        | 0x0000 0D9A            | 2          | MXINTA                     | (McBSP-A) | 10           | 6                  |
| INT6.7                                           | 78        | 0x0000 0D9C            | 2          | Reserved                   | -         | 10           | 7                  |
| INT6.8                                           | 79        | 0x0000 0D9E            | 2          | Reserved                   | -         | 10           | 8 (lowest)         |

**Table 1-120. PIE Vector Table (continued)**

| Name                                               | VECTOR ID | Address <sup>(1)</sup> | Size (x16) | Description <sup>(2)</sup> |                      | CPU<br>Priority | PIE Group<br>Priority |
|----------------------------------------------------|-----------|------------------------|------------|----------------------------|----------------------|-----------------|-----------------------|
| <b>PIE Group 7 Vectors - MUXed into CPU INT7</b>   |           |                        |            |                            |                      |                 |                       |
| INT7.1                                             | 80        | 0x0000 0DA0            | 2          | DINTCH1                    | (DMA)                | 11              | 1 (highest)           |
| INT7.2                                             | 81        | 0x0000 0DA2            | 2          | DINTCH2                    | (DMA)                | 11              | 2                     |
| INT7.3                                             | 82        | 0x0000 0DA4            | 2          | DINTCH3                    | (DMA)                | 11              | 3                     |
| INT7.4                                             | 83        | 0x0000 0DA6            | 2          | DINTCH4                    | (DMA)                | 11              | 4                     |
| INT7.5                                             | 84        | 0x0000 0DA8            | 2          | DINTCH5                    | (DMA)                | 11              | 5                     |
| INT7.6                                             | 85        | 0x0000 0DAA            | 2          | DINTCH6                    | (DMA)                | 11              | 6                     |
| INT7.7                                             | 86        | 0x0000 0DAC            | 2          | Reserved                   | -                    | 11              | 7                     |
| INT7.8                                             | 87        | 0x0000 0DAE            | 2          | Reserved                   | -                    | 11              | 8 (lowest)            |
| <b>PIE Group 8 Vectors - MUXed into CPU INT8</b>   |           |                        |            |                            |                      |                 |                       |
| INT8.1                                             | 88        | 0x0000 0DB0            | 2          | I2CINT1A                   | (I <sup>2</sup> C-A) | 12              | 1 (highest)           |
| INT8.2                                             | 89        | 0x0000 0DB2            | 2          | I2CINT2A                   | (I <sup>2</sup> C-A) | 12              | 2                     |
| INT8.3                                             | 90        | 0x0000 0DB4            | 2          | Reserved                   | -                    | 12              | 3                     |
| INT8.4                                             | 91        | 0x0000 0DB6            | 2          | Reserved                   | -                    | 12              | 4                     |
| INT8.5                                             | 92        | 0x0000 0DB8            | 2          | Reserved                   | -                    | 12              | 5                     |
| INT8.6                                             | 93        | 0x0000 0DBA            | 2          | Reserved                   | -                    | 12              | 6                     |
| INT8.7                                             | 94        | 0x0000 0DBC            | 2          | Reserved                   | -                    | 12              | 7                     |
| INT8.8                                             | 95        | 0x0000 0DBE            | 2          | Reserved                   | -                    | 12              | 8 (lowest)            |
| <b>PIE Group 9 Vectors - MUXed into CPU INT9</b>   |           |                        |            |                            |                      |                 |                       |
| INT9.1                                             | 96        | 0x0000 0DC0            | 2          | SCIRXINTA                  | (SCI-A)              | 13              | 1 (highest)           |
| INT9.2                                             | 97        | 0x0000 0DC2            | 2          | SCITXINTA                  | (SCI-A)              | 13              | 2                     |
| INT9.3                                             | 98        | 0x0000 0DC4            | 2          | SCIRXINTB                  | (SCI-B)              | 13              | 3                     |
| INT9.4                                             | 99        | 0x0000 0DC6            | 2          | SCITXINTB                  | (SCI-B)              | 13              | 4                     |
| INT9.5                                             | 100       | 0x0000 0DC8            | 2          | ECANAINT0                  | (CAN-A)              | 13              | 5                     |
| INT9.6                                             | 101       | 0x0000 0DCA            | 2          | ECANAINT1                  | (CAN-A)              | 13              | 6                     |
| INT9.7                                             | 102       | 0x0000 0DCC            | 2          | Reserved                   | -                    | 13              | 7                     |
| INT9.8                                             | 103       | 0x0000 0DCE            | 2          | Reserved                   | -                    | 13              | 8 (lowest)            |
| <b>PIE Group 10 Vectors - MUXed into CPU INT10</b> |           |                        |            |                            |                      |                 |                       |
| INT10.1                                            | 104       | 0x0000 0DD0            | 2          | ADCINT1                    | (ADC)                | 14              | 1 (highest)           |
| INT10.2                                            | 105       | 0x0000 0DD2            | 2          | ADCINT2                    | (ADC)                | 14              | 2                     |
| INT10.3                                            | 106       | 0x0000 0DD4            | 2          | ADCINT3                    | (ADC)                | 14              | 3                     |
| INT10.4                                            | 107       | 0x0000 0DD6            | 2          | ADCINT4                    | (ADC)                | 14              | 4                     |
| INT10.5                                            | 108       | 0x0000 0DD8            | 2          | ADCINT5                    | (ADC)                | 14              | 5                     |
| INT10.6                                            | 109       | 0x0000 0DDA            | 2          | ADCINT6                    | (ADC)                | 14              | 6                     |
| INT10.7                                            | 110       | 0x0000 0DDC            | 2          | ADCINT7                    | (ADC)                | 14              | 7                     |
| INT10.8                                            | 111       | 0x0000 0DDE            | 2          | ADCINT8                    | (ADC)                | 14              | 8 (lowest)            |
| <b>PIE Group 11 Vectors - MUXed into CPU INT11</b> |           |                        |            |                            |                      |                 |                       |
| INT11.1                                            | 112       | 0x0000 0DE0            | 2          | CLA1_INT1                  | (CLA)                | 15              | 1 (highest)           |
| INT11.2                                            | 113       | 0x0000 0DE2            | 2          | CLA1_INT2                  | (CLA)                | 15              | 2                     |
| INT11.3                                            | 114       | 0x0000 0DE4            | 2          | CLA1_INT3                  | (CLA)                | 15              | 3                     |
| INT11.4                                            | 115       | 0x0000 0DE6            | 2          | CLA1_INT4                  | (CLA)                | 15              | 4                     |
| INT11.5                                            | 116       | 0x0000 0DE8            | 2          | CLA1_INT5                  | (CLA)                | 15              | 5                     |
| INT11.6                                            | 117       | 0x0000 0DEA            | 2          | CLA1_INT6                  | (CLA)                | 15              | 6                     |
| INT11.7                                            | 118       | 0x0000 0DEC            | 2          | CLA1_INT7                  | (CLA)                | 15              | 7                     |
| INT11.8                                            | 119       | 0x0000 0DEE            | 2          | CLA1_INT8                  | (CLA)                | 15              | 8 (lowest)            |

**Table 1-120. PIE Vector Table (continued)**

| Name                                               | VECTOR ID | Address <sup>(1)</sup> | Size (x16) | Description <sup>(2)</sup> |       | CPU Priority | PIE Group Priority |
|----------------------------------------------------|-----------|------------------------|------------|----------------------------|-------|--------------|--------------------|
| <b>PIE Group 12 Vectors - MUXed into CPU INT12</b> |           |                        |            |                            |       |              |                    |
| INT12.1                                            | 120       | 0x0000 0DF0            | 2          | XINT3                      | -     | 16           | 1 (highest)        |
| INT12.2                                            | 121       | 0x0000 0DF2            | 2          | Reserved                   | -     | 16           | 2                  |
| INT12.3                                            | 122       | 0x0000 0DF4            | 2          | Reserved                   | -     | 16           | 3                  |
| INT12.4                                            | 123       | 0x0000 0DF6            | 2          | Reserved                   | -     | 16           | 4                  |
| INT12.5                                            | 124       | 0x0000 0DF8            | 2          | Reserved                   | -     | 16           | 5                  |
| INT12.6                                            | 125       | 0x0000 0DFA            | 2          | Reserved                   | -     | 16           | 6                  |
| INT12.7                                            | 126       | 0x0000 0DFC            | 2          | LVF                        | (CLA) | 16           | 7                  |
| INT12.8                                            | 127       | 0x0000 0DFE            | 2          | LUF                        | (CLA) | 16           | 8 (lowest)         |

(1) Reset is always fetched from location 0x003F FFC0 in Boot ROM.

(2) All the locations within the PIE vector table are EALLOW protected.

### 1.6.4 PIE Configuration Registers

The registers controlling the functionality of the PIE block are shown in [Table 1-121](#).

**Table 1-121. PIE Configuration and Control Registers**

| Name     | Address       | Size (x16) | Description                      |
|----------|---------------|------------|----------------------------------|
| PIECTRL  | 0x0000 - 0CE0 | 1          | PIE, Control Register            |
| PIEACK   | 0x0000 - 0CE1 | 1          | PIE, Acknowledge Register        |
| PIEIER1  | 0x0000 - 0CE2 | 1          | PIE, INT1 Group Enable Register  |
| PIEIFR1  | 0x0000 - 0CE3 | 1          | PIE, INT1 Group Flag Register    |
| PIEIER2  | 0x0000 - 0CE4 | 1          | PIE, INT2 Group Enable Register  |
| PIEIFR2  | 0x0000 - 0CE5 | 1          | PIE, INT2 Group Flag Register    |
| PIEIER3  | 0x0000 - 0CE6 | 1          | PIE, INT3 Group Enable Register  |
| PIEIFR3  | 0x0000 - 0CE7 | 1          | PIE, INT3 Group Flag Register    |
| PIEIER4  | 0x0000 - 0CE8 | 1          | PIE, INT4 Group Enable Register  |
| PIEIFR4  | 0x0000 - 0CE9 | 1          | PIE, INT4 Group Flag Register    |
| PIEIER5  | 0x0000 - 0CEA | 1          | PIE, INT5 Group Enable Register  |
| PIEIFR5  | 0x0000 - 0CEB | 1          | PIE, INT5 Group Flag Register    |
| PIEIER6  | 0x0000 - 0CEC | 1          | PIE, INT6 Group Enable Register  |
| PIEIFR6  | 0x0000 - 0CED | 1          | PIE, INT6 Group Flag Register    |
| PIEIER7  | 0x0000 - 0CEE | 1          | PIE, INT7 Group Enable Register  |
| PIEIFR7  | 0x0000 - 0CEF | 1          | PIE, INT7 Group Flag Register    |
| PIEIER8  | 0x0000 - 0CF0 | 1          | PIE, INT8 Group Enable Register  |
| PIEIFR8  | 0x0000 - 0CF1 | 1          | PIE, INT8 Group Flag Register    |
| PIEIER9  | 0x0000 - 0CF2 | 1          | PIE, INT9 Group Enable Register  |
| PIEIFR9  | 0x0000 - 0CF3 | 1          | PIE, INT9 Group Flag Register    |
| PIEIER10 | 0x0000 - 0CF4 | 1          | PIE, INT10 Group Enable Register |
| PIEIFR10 | 0x0000 - 0CF5 | 1          | PIE, INT10 Group Flag Register   |
| PIEIER11 | 0x0000 - 0CF6 | 1          | PIE, INT11 Group Enable Register |
| PIEIFR11 | 0x0000 - 0CF7 | 1          | PIE, INT11 Group Flag Register   |
| PIEIER12 | 0x0000 - 0CF8 | 1          | PIE, INT12 Group Enable Register |
| PIEIFR12 | 0x0000 - 0CF9 | 1          | PIE, INT12 Group Flag Register   |





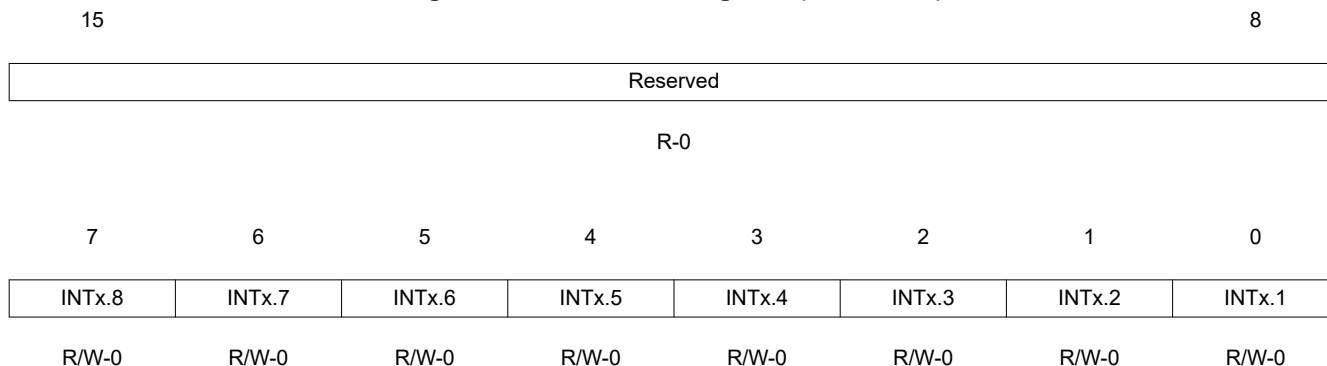
### 1.6.5.1 PIE Interrupt Flag Registers

There are twelve PIEIFR registers, one for each CPU interrupt used by the PIE module (INT1-INT12).

#### CAUTION

Never clear a PIEIFR bit. An interrupt may be lost during the read-modify-write operation. See [Section 1.6.3.1](#) for a method to clear flagged interrupts.

**Figure 1-100. PIEIFRx Register (x = 1 to 12)**



LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 1-124. PIEIFRx Register Field Descriptions**

| Bits | Field    | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|------|----------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-8 | Reserved | Reserved                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| 7    | INTx.8   | These register bits indicate whether an interrupt is currently active. They behave very much like the CPU interrupt flag register. When an interrupt is active, the respective register bit is set. The bit is cleared when the interrupt is serviced or by writing a 0 to the register bit. This register can also be read to determine which interrupts are active or pending. x = 1 to 12. INTx means CPU INT1 to INT12<br><br>The PIEIFR register bit is cleared during the interrupt vector fetch portion of the interrupt processing.<br><br>Hardware has priority over CPU accesses to the PIEIFR registers. |
| 6    | INTx.7   |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| 5    | INTx.6   |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| 4    | INTx.5   |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| 3    | INTx.4   |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| 2    | INTx.3   |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| 1    | INTx.2   |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| 0    | INTx.1   |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |

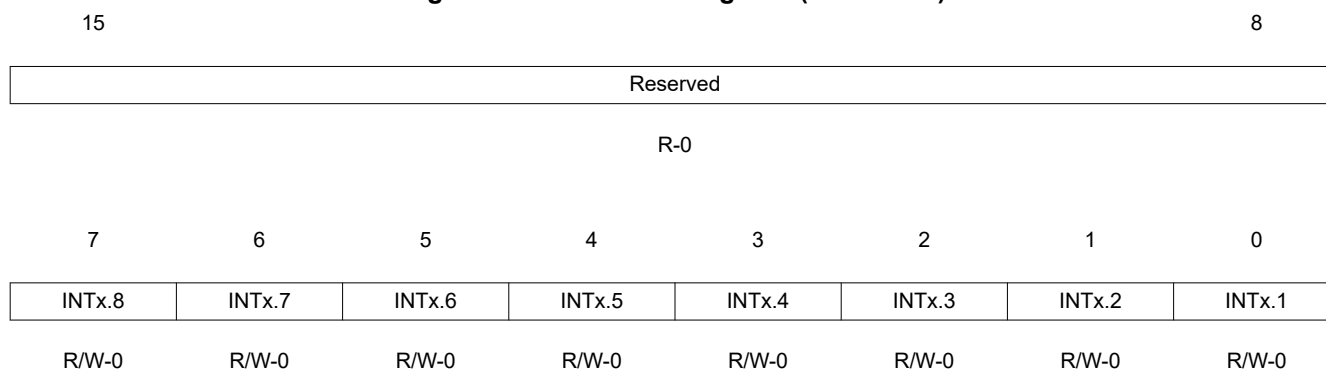
### 1.6.5.2 PIE Interrupt Enable Registers

There are twelve PIEIER registers, one for each CPU interrupt used by the PIE module (INT1-INT12).

#### Note

Care must be taken when clearing PIEIER bits during normal operation. See [Section 1.6.3.2](#) for the proper procedure for handling these bits.

**Figure 1-101. PIEIERx Register (x = 1 to 12)**



LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 1-125. PIEIERx Register (x = 1 to 12) Field Descriptions**

| Bits | Field    | Description                                                                                                                                                                                                                                                                                                   |
|------|----------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-8 | Reserved | Reserved                                                                                                                                                                                                                                                                                                      |
| 7    | INTx.8   | These register bits individually enable an interrupt within a group and behave very much like the core interrupt enable register. Setting a bit to 1 enables the servicing of the respective interrupt. Setting a bit to 0 disables the servicing of the interrupt. x = 1 to 12. INTx means CPU INT1 to INT12 |
| 6    | INTx.7   |                                                                                                                                                                                                                                                                                                               |
| 5    | INTx.6   |                                                                                                                                                                                                                                                                                                               |
| 4    | INTx.5   |                                                                                                                                                                                                                                                                                                               |
| 3    | INTx.4   |                                                                                                                                                                                                                                                                                                               |
| 2    | INTx.3   |                                                                                                                                                                                                                                                                                                               |
| 1    | INTx.2   |                                                                                                                                                                                                                                                                                                               |
| 0    | INTx.1   |                                                                                                                                                                                                                                                                                                               |

### 1.6.5.3 CPU Interrupt Flag Register (IFR)

The CPU interrupt flag register (IFR), is a 16-bit, CPU register and is used to identify and clear pending interrupts. The IFR contains flag bits for all the maskable interrupts at the CPU level (INT1-INT14, DLOGINT and RTOSINT). When the PIE is enabled, the PIE module multiplexes interrupt sources for INT1-INT12.

When a maskable interrupt is requested, the flag bit in the corresponding peripheral control register is set to 1. If the corresponding mask bit is also 1, the interrupt request is sent to the CPU, setting the corresponding flag in the IFR. This indicates that the interrupt is pending or waiting for acknowledgment.

To identify pending interrupts, use the PUSH IFR instruction and then test the value on the stack. Use the OR IFR instruction to set IFR bits and use the AND IFR instruction to manually clear pending interrupts. All pending interrupts are cleared with the AND IFR #0 instruction or by a hardware reset.

The following events also clear an IFR flag:

- The CPU acknowledges the interrupt.
- The 28x device is reset.

#### Note

1. To clear a CPU IFR bit, you must write a zero to it, not a one.
2. When a maskable interrupt is acknowledged, only the IFR bit is cleared automatically. The flag bit in the corresponding peripheral control register is not cleared. If an application requires that the control register flag be cleared, the bit must be cleared by software.
3. When an interrupt is requested by an INTR instruction and the corresponding IFR bit is set, the CPU does not clear the bit automatically. If an application requires that the IFR bit be cleared, the bit must be cleared by software.
4. IMR and IFR registers pertain to core-level interrupts. All peripherals have their own interrupt mask and flag bits in their respective control/configuration registers. Note that several peripheral interrupts are grouped under one core-level interrupt.

**Figure 1-102. Interrupt Flag Register (IFR) — CPU Register**

|         |         |       |       |       |       |       |       |
|---------|---------|-------|-------|-------|-------|-------|-------|
| 15      | 14      | 13    | 12    | 11    | 10    | 9     | 8     |
| RTOSINT | DLOGINT | INT14 | INT13 | INT12 | INT11 | INT10 | INT9  |
| R/W-0   | R/W-0   | R/W-0 | R/W-0 | R/W-0 | R/W-0 | R/W-0 | R/W-0 |
| 7       | 6       | 5     | 4     | 3     | 2     | 1     | 0     |
| INT8    | INT7    | INT6  | INT5  | INT4  | INT3  | INT2  | INT1  |
| R/W-0   | R/W-0   | R/W-0 | R/W-0 | R/W-0 | R/W-0 | R/W-0 | R/W-0 |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 1-126. Interrupt Flag Register (IFR) — CPU Register Field Descriptions**

| Bits | Field   | Value  | Description                                                                                                                                                                                                                                     |
|------|---------|--------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15   | RTOSINT | 0<br>1 | Real-time operating system flag. RTOSINT is the flag for RTOS interrupts.<br>No RTOS interrupt is pending<br>At least one RTOS interrupt is pending. Write a 0 to this bit to clear it to 0 and clear the interrupt request                     |
| 14   | DLOGINT | 0<br>1 | Data logging interrupt flag. DLOGINT is the flag for data logging interrupts.<br>No DLOGINT is pending<br>At least one DLOGINT interrupt is pending. Write a 0 to this bit to clear it to 0 and clear the interrupt request                     |
| 13   | INT14   | 0<br>1 | Interrupt 14 flag. INT14 is the flag for interrupts connected to CPU interrupt level INT14.<br>No INT14 interrupt is pending<br>At least one INT14 interrupt is pending. Write a 0 to this bit to clear it to 0 and clear the interrupt request |
| 12   | INT13   | 0<br>1 | Interrupt 13 flag. INT13 is the flag for interrupts connected to CPU interrupt level INT13.<br>No INT13 interrupt is pending<br>At least one INT13 interrupt is pending. Write a 0 to this bit to clear it to 0 and clear the interrupt request |
| 11   | INT12   | 0<br>1 | Interrupt 12 flag. INT12 is the flag for interrupts connected to CPU interrupt level INT12.<br>No INT12 interrupt is pending<br>At least one INT12 interrupt is pending. Write a 0 to this bit to clear it to 0 and clear the interrupt request |
| 10   | INT11   | 0<br>1 | Interrupt 11 flag. INT11 is the flag for interrupts connected to CPU interrupt level INT11.<br>No INT11 interrupt is pending<br>At least one INT11 interrupt is pending. Write a 0 to this bit to clear it to 0 and clear the interrupt request |
| 9    | INT10   | 0<br>1 | Interrupt 10 flag. INT10 is the flag for interrupts connected to CPU interrupt level INT10.<br>No INT10 interrupt is pending<br>At least one INT6 interrupt is pending. Write a 0 to this bit to clear it to 0 and clear the interrupt request  |
| 8    | INT9    | 0<br>1 | Interrupt 9 flag. INT9 is the flag for interrupts connected to CPU interrupt level INT6.<br>No INT9 interrupt is pending<br>At least one INT9 interrupt is pending. Write a 0 to this bit to clear it to 0 and clear the interrupt request      |
| 7    | INT8    | 0<br>1 | Interrupt 8 flag. INT8 is the flag for interrupts connected to CPU interrupt level INT6.<br>No INT8 interrupt is pending<br>At least one INT8 interrupt is pending. Write a 0 to this bit to clear it to 0 and clear the interrupt request      |
| 6    | INT7    | 0<br>1 | Interrupt 7 flag. INT7 is the flag for interrupts connected to CPU interrupt level INT7.<br>No INT7 interrupt is pending<br>At least one INT7 interrupt is pending. Write a 0 to this bit to clear it to 0 and clear the interrupt request      |
| 5    | INT6    | 0<br>1 | Interrupt 6 flag. INT6 is the flag for interrupts connected to CPU interrupt level INT6.<br>No INT6 interrupt is pending<br>At least one INT6 interrupt is pending. Write a 0 to this bit to clear it to 0 and clear the interrupt request      |
| 4    | INT5    | 0<br>1 | Interrupt 5 flag. INT5 is the flag for interrupts connected to CPU interrupt level INT5.<br>No INT5 interrupt is pending<br>At least one INT5 interrupt is pending. Write a 0 to this bit to clear it to 0 and clear the interrupt request      |

**Table 1-126. Interrupt Flag Register (IFR) — CPU Register Field Descriptions (continued)**

| Bits | Field | Value | Description                                                                                                              |
|------|-------|-------|--------------------------------------------------------------------------------------------------------------------------|
| 3    | INT4  | 0     | Interrupt 4 flag. INT4 is the flag for interrupts connected to CPU interrupt level INT4.<br>No INT4 interrupt is pending |
|      |       | 1     | At least one INT4 interrupt is pending. Write a 0 to this bit to clear it to 0 and clear the interrupt request           |
| 2    | INT3  | 0     | Interrupt 3 flag. INT3 is the flag for interrupts connected to CPU interrupt level INT3.<br>No INT3 interrupt is pending |
|      |       | 1     | At least one INT3 interrupt is pending. Write a 0 to this bit to clear it to 0 and clear the interrupt request           |
| 1    | INT2  | 0     | Interrupt 2 flag. INT2 is the flag for interrupts connected to CPU interrupt level INT2.<br>No INT2 interrupt is pending |
|      |       | 1     | At least one INT2 interrupt is pending. Write a 0 to this bit to clear it to 0 and clear the interrupt request           |
| 0    | INT1  | 0     | Interrupt 1 flag. INT1 is the flag for interrupts connected to CPU interrupt level INT1.<br>No INT1 interrupt is pending |
|      |       | 1     | At least one INT1 interrupt is pending. Write a 0 to this bit to clear it to 0 and clear the interrupt request           |

### 1.6.5.4 Interrupt Enable Register (IER) and Debug Interrupt Enable Register (DBGIER)

The IER is a 16-bit CPU register. The IER contains enable bits for all the maskable CPU interrupt levels (INT1-INT14, RTOSINT and DLOGINT). Neither NMI nor  $\overline{XRS}$  is included in the IER; thus, IER has no effect on these interrupts.

You can read the IER to identify enabled or disabled interrupt levels, and you can write to the IER to enable or disable interrupt levels. To enable an interrupt level, set its corresponding IER bit to one using the OR IER instruction. To disable an interrupt level, set its corresponding IER bit to zero using the AND IER instruction. When an interrupt is disabled, it is not acknowledged, regardless of the value of the INTM bit. When an interrupt is enabled, it is acknowledged if the corresponding IFR bit is one and the INTM bit is zero.

When using the OR IER and AND IER instructions to modify IER bits make sure they do not modify the state of bit 15 (RTOSINT) unless a real-time operating system is present.

When a hardware interrupt is serviced or an INTR instruction is executed, the corresponding IER bit is cleared automatically. When an interrupt is requested by the TRAP instruction the IER bit is not cleared automatically. In the case of the TRAP instruction if the bit needs to be cleared it must be done by the interrupt service routine.

At reset, all the IER bits are cleared to 0, disabling all maskable CPU level interrupts.

The IER register is shown in [Figure 1-103](#), and descriptions of the bits follow the figure.

**Figure 1-103. Interrupt Enable Register (IER) — CPU Register**

|         |         |       |       |       |       |       |       |
|---------|---------|-------|-------|-------|-------|-------|-------|
| 15      | 14      | 13    | 12    | 11    | 10    | 9     | 8     |
| RTOSINT | DLOGINT | INT14 | INT13 | INT12 | INT11 | INT10 | INT9  |
| R/W-0   | R/W-0   | R/W-0 | R/W-0 | R/W-0 | R/W-0 | R/W-0 | R/W-0 |
| 7       | 6       | 5     | 4     | 3     | 2     | 1     | 0     |
| INT8    | INT7    | INT6  | INT5  | INT4  | INT3  | INT2  | INT1  |
| R/W-0   | R/W-0   | R/W-0 | R/W-0 | R/W-0 | R/W-0 | R/W-0 | R/W-0 |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 1-127. Interrupt Enable Register (IER) — CPU Register Field Descriptions**

| Bits | Field   | Value  | Description                                                                                                                                   |
|------|---------|--------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| 15   | RTOSINT | 0<br>1 | Real-time operating system interrupt enable. RTOSINT enables or disables the CPU RTOS interrupt.<br>RTOSINT is disabled<br>RTOSINT is enabled |
| 14   | DLOGINT | 0<br>1 | Data logging interrupt enable. DLOGINT enables or disables the CPU data logging interrupt.<br>DLOGINT is disabled<br>DLOGINT is enabled       |
| 13   | INT14   | 0<br>1 | Interrupt 14 enable. INT14 enables or disables CPU interrupt level INT14.<br>Level INT14 is disabled<br>Level INT14 is enabled                |
| 12   | INT13   | 0<br>1 | Interrupt 13 enable. INT13 enables or disables CPU interrupt level INT13.<br>Level INT13 is disabled<br>Level INT13 is enabled                |
| 11   | INT12   | 0<br>1 | Interrupt 12 enable. INT12 enables or disables CPU interrupt level INT12.<br>Level INT12 is disabled<br>Level INT12 is enabled                |

**Table 1-127. Interrupt Enable Register (IER) — CPU Register Field Descriptions (continued)**

| Bits | Field | Value  | Description                                                                                                                    |
|------|-------|--------|--------------------------------------------------------------------------------------------------------------------------------|
| 10   | INT11 | 0<br>1 | Interrupt 11 enable. INT11 enables or disables CPU interrupt level INT11.<br>Level INT11 is disabled<br>Level INT11 is enabled |
| 9    | INT10 | 0<br>1 | Interrupt 10 enable. INT10 enables or disables CPU interrupt level INT10.<br>Level INT10 is disabled<br>Level INT10 is enabled |
| 8    | INT9  | 0<br>1 | Interrupt 9 enable. INT9 enables or disables CPU interrupt level INT9.<br>Level INT9 is disabled<br>Level INT9 is enabled      |
| 7    | INT8  | 0<br>1 | Interrupt 8 enable. INT8 enables or disables CPU interrupt level INT8.<br>Level INT8 is disabled<br>Level INT8 is enabled      |
| 6    | INT7  | 0<br>1 | Interrupt 7 enable. INT7 enables or disables CPU interrupt level INT7.<br>Level INT7 is disabled<br>Level INT7 is enabled      |
| 5    | INT6  | 0<br>1 | Interrupt 6 enable. INT6 enables or disables CPU interrupt level INT6.<br>Level INT6 is disabled<br>Level INT6 is enabled      |
| 4    | INT5  | 0<br>1 | Interrupt 5 enable. INT5 enables or disables CPU interrupt level INT5.<br>Level INT5 is disabled<br>Level INT5 is enabled      |
| 3    | INT4  | 0<br>1 | Interrupt 4 enable. INT4 enables or disables CPU interrupt level INT4.<br>Level INT4 is disabled<br>Level INT4 is enabled      |
| 2    | INT3  | 0<br>1 | Interrupt 3 enable. INT3 enables or disables CPU interrupt level INT3.<br>Level INT3 is disabled<br>Level INT3 is enabled      |
| 1    | INT2  | 0<br>1 | Interrupt 2 enable. INT2 enables or disables CPU interrupt level INT2.<br>Level INT2 is disabled<br>Level INT2 is enabled      |
| 0    | INT1  | 0<br>1 | Interrupt 1 enable. INT1 enables or disables CPU interrupt level INT1.<br>Level INT1 is disabled<br>Level INT1 is enabled      |

The Debug Interrupt Enable Register (DBGIER) is used only when the CPU is halted in real-time emulation mode. An interrupt enabled in the DBGIER is defined as a time-critical interrupt. When the CPU is halted in real-time mode, the only interrupts that are serviced are time-critical interrupts that are also enabled in the IER. If the CPU is running in real-time emulation mode, the standard interrupt-handling process is used and the DBGIER is ignored.

As with the IER, you can read the DBGIER to identify enabled or disabled interrupts and write to the DBGIER to enable or disable interrupts. To enable an interrupt, set its corresponding bit to 1. To disable an interrupt, set its corresponding bit to 0. Use the PUSH DBGIER instruction to read from the DBGIER and POP DBGIER to write to the DBGIER register. At reset, all the DBGIER bits are set to 0.

**Figure 1-104. Debug Interrupt Enable Register (DBGIER) — CPU Register**

|         |         |       |       |       |       |       |       |
|---------|---------|-------|-------|-------|-------|-------|-------|
| 15      | 14      | 13    | 12    | 11    | 10    | 9     | 8     |
| RTOSINT | DLOGINT | INT14 | INT13 | INT12 | INT11 | INT10 | INT9  |
| R/W-0   | R/W-0   | R/W-0 | R/W-0 | R/W-0 | R/W-0 | R/W-0 | R/W-0 |
| 7       | 6       | 5     | 4     | 3     | 2     | 1     | 0     |
| INT8    | INT7    | INT6  | INT5  | INT4  | INT3  | INT2  | INT1  |
| R/W-0   | R/W-0   | R/W-0 | R/W-0 | R/W-0 | R/W-0 | R/W-0 | R/W-0 |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 1-128. Debug Interrupt Enable Register (DBGIER) — CPU Register Field Descriptions**

| Bits | Field   | Value  | Description                                                                                                                                   |
|------|---------|--------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| 15   | RTOSINT | 0<br>1 | Real-time operating system interrupt enable. RTOSINT enables or disables the CPU RTOS interrupt.<br>RTOSINT is disabled<br>RTOSINT is enabled |
| 14   | DLOGINT | 0<br>1 | Data logging interrupt enable. DLOGINT enables or disables the CPU data logging interrupt<br>DLOGINT is disabled<br>DLOGINT is enabled        |
| 13   | INT14   | 0<br>1 | Interrupt 14 enable. INT14 enables or disables CPU interrupt level INT14<br>Level INT14 is disabled<br>Level INT14 is enabled                 |
| 12   | INT13   | 0<br>1 | Interrupt 13 enable. INT13 enables or disables CPU interrupt level INT13.<br>Level INT13 is disabled<br>Level INT13 is enabled                |
| 11   | INT12   | 0<br>1 | Interrupt 12 enable. INT12 enables or disables CPU interrupt level INT12.<br>Level INT12 is disabled<br>Level INT12 is enabled                |
| 10   | INT11   | 0<br>1 | Interrupt 11 enable. INT11 enables or disables CPU interrupt level INT11.<br>Level INT11 is disabled<br>Level INT11 is enabled                |
| 9    | INT10   | 0<br>1 | Interrupt 10 enable. INT10 enables or disables CPU interrupt level INT10.<br>Level INT10 is disabled<br>Level INT10 is enabled                |



**Table 1-128. Debug Interrupt Enable Register (DBGIER) — CPU Register Field Descriptions (continued)**

| Bits | Field | Value  | Description                                                                                                               |
|------|-------|--------|---------------------------------------------------------------------------------------------------------------------------|
| 8    | INT9  | 0<br>1 | Interrupt 9 enable. INT9 enables or disables CPU interrupt level INT9.<br>Level INT9 is disabled<br>Level INT9 is enabled |
| 7    | INT8  | 0<br>1 | Interrupt 8 enable. INT8 enables or disables CPU interrupt level INT8.<br>Level INT8 is disabled<br>Level INT8 is enabled |
| 6    | INT7  | 0<br>1 | Interrupt 7 enable. INT7 enables or disables CPU interrupt level INT7.<br>Level INT7 is disabled<br>Level INT7 is enabled |
| 5    | INT6  | 0<br>1 | Interrupt 6 enable. INT6 enables or disables CPU interrupt level INT6.<br>Level INT6 is disabled<br>Level INT6 is enabled |
| 4    | INT5  | 0<br>1 | Interrupt 5 enable. INT5 enables or disables CPU interrupt level INT5.<br>Level INT5 is disabled<br>Level INT5 is enabled |
| 3    | INT4  | 0<br>1 | Interrupt 4 enable. INT4 enables or disables CPU interrupt level INT4.<br>Level INT4 is disabled<br>Level INT4 is enabled |
| 2    | INT3  | 0<br>1 | Interrupt 3 enable. INT3 enables or disables CPU interrupt level INT3.<br>Level INT3 is disabled<br>Level INT3 is enabled |
| 1    | INT2  | 0<br>1 | Interrupt 2 enable. INT2 enables or disables CPU interrupt level INT2.<br>Level INT2 is disabled<br>Level INT2 is enabled |
| 0    | INT1  | 0<br>1 | Interrupt 1 enable. INT1 enables or disables CPU interrupt level INT1.<br>Level INT1 is disabled<br>Level INT1 is enabled |

## 1.6.6 External Interrupt Control Registers

Three external interrupts, XINT1–XINT3, are supported. Each of these external interrupts can be selected for negative or positive edge triggered and can also be enabled or disabled. The masked interrupts also contain a 16-bit free-running up-counter that is reset to zero when a valid interrupt edge is detected. This counter can be used to accurately time stamp the interrupt.

**Table 1-129. Interrupt Control and Counter Registers (not EALLOW Protected)**

| Name     | Address Range             | Size (x16) | Description                  |
|----------|---------------------------|------------|------------------------------|
| XINT1CR  | 0x0000 7070               | 1          | XINT1 configuration register |
| XINT2CR  | 0x0000 7071               | 1          | XINT2 configuration register |
| XINT3CR  | 0x0000 7072               | 1          | XINT3 configuration register |
| reserved | 0x0000 7073 - 0x0000 7077 | 5          |                              |
| XINT1CTR | 0x0000 7078               | 1          | XINT1 counter register       |
| XINT2CTR | 0x0000 7079               | 1          | XINT2 counter register       |
| XINT3CTR | 0x0000 707A               | 1          | XINT3 counter register       |
| reserved | 0x0000 707B - 0x0000 707E | 5          |                              |

XINT1CR through XINT3CR are identical except for the interrupt number; therefore, [Figure 1-105](#) and [Table 1-130](#) represent registers for external interrupts 1 through 3 as XINT $n$ CR where  $n$  = the interrupt number.

**Figure 1-105. External Interrupt  $n$  Control Register (XINT $n$ CR)**

|          |   |          |   |          |        |
|----------|---|----------|---|----------|--------|
| 15       | 4 | 3        | 2 | 1        | 0      |
| Reserved |   | Polarity |   | Reserved | Enable |
| R-0      |   | R/W-0    |   | R-0      | R/W-0  |

LEGEND: R/W = Read/Write; R = Read only; - $n$  = value after reset

**Table 1-130. External Interrupt  $n$  Control Register (XINT $n$ CR) Field Descriptions**

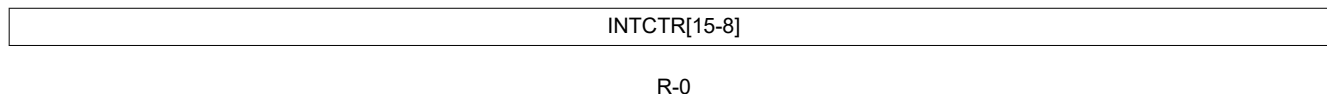
| Bits | Field    | Value                | Description                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|------|----------|----------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-4 | Reserved |                      | Reads return zero; writes have no effect.                                                                                                                                                                                                                                                                                                                                                                                                    |
| 3-2  | Polarity | 00<br>01<br>10<br>11 | This read/write bit determines whether interrupts are generated on the rising edge or the falling edge of a signal on the pin.<br>Interrupt generated on a falling edge (high-to-low transition)<br>Interrupt generated on a rising edge (low-to-high transition)<br>Interrupt generated on a falling edge (high-to-low transition)<br>Interrupt generated on both a falling edge and a rising edge (high-to-low and low-to-high transition) |
| 1    | Reserved |                      | Reads return zero; writes have no effect                                                                                                                                                                                                                                                                                                                                                                                                     |
| 0    | Enable   | 0<br>1               | This read/write bit enables or disables external interrupt XINT $n$ .<br>Disable interrupt<br>Enable interrupt                                                                                                                                                                                                                                                                                                                               |

For XINT1, XINT2, and XINT3, there is also a 16-bit counter that is reset to 0x000 whenever an interrupt edge is detected. These counters can be used to accurately time stamp an occurrence of the interrupt. XINT1CTR through XINT3CTR are identical except for the interrupt number; therefore, [Figure 1-106](#) and [Table 1-131](#) represent registers for the external interrupts as XINT $n$ CTR, where  $n$  = the interrupt number.

**Figure 1-106. External Interrupt n Counter (XINTnCTR) (Address 7078h)**

15

0



LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 1-131. External Interrupt n Counter (XINTnCTR) Field Descriptions**

| Bits | Field  | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|------|--------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-0 | INTCTR | This is a 16-bit free-running up-counter that is clocked at the SYSCLKOUT rate. The counter value is reset to 0x0000 when a valid interrupt edge is detected and then continues counting until the next valid interrupt edge is detected. When the interrupt is disabled, the counter stops. The counter is a free-running counter and wraps around to zero when the maximum value is reached. The counter is a read only register and can only be reset to zero by a valid interrupt edge or by reset. |

## 1.7 VREG/BOR/POR

Although the core and I/O circuitry operate on two different voltages, these devices have an on-chip voltage regulator (VREG) to generate the  $V_{DD}$  voltage from the  $V_{DDIO}$  supply. This eliminates the cost and area of a second external regulator on an application board. Additionally, internal power-on reset (POR) and brown-out reset (BOR) circuits monitor both the  $V_{DD}$  and  $V_{DDIO}$  rails during power-up and run mode, eliminating a need for any external voltage supervisory circuits.

The  $V_{DD}$  BOR is only valid when the VREG is enabled. If VREG is disabled, and external LDO is used for 1.8V, then there is no BOR function on  $V_{DD}$ .

### 1.7.1 On-chip Voltage Regulator (VREG)

An on-chip voltage regulator facilitates the powering of the device without adding the cost or board space of a second external regulator. This linear regulator generates the core  $V_{DD}$  voltage from the  $V_{DDIO}$  supply. Therefore, although capacitors are required on each  $V_{DD}$  pin to stabilize the generated voltage, power need not be supplied to these pins to operate the device. Conversely, the VREG can be bypassed or overdriven, should power or redundancy be the primary concern of the application.

#### 1.7.1.1 Using the on-chip VREG

To utilize the on-chip VREG, the VREGENZ pin should be pulled low and the appropriate recommended operating voltage should be supplied to the  $V_{DDIO}$  and  $V_{DDA}$  pins. In this case, the  $V_{DD}$  voltage needed by the core logic will be generated by the VREG. Each  $V_{DD}$  pin requires on the order of 1.2  $\mu$ F capacitance for proper regulation of the VREG. These capacitors should be located as close as possible to the device pins. Refer to the device datasheet for the acceptable range of capacitance.

#### 1.7.1.2 Bypassing the on-chip VREG

To conserve power, it is also possible to bypass the on-chip VREG and supply the core logic voltage to the  $V_{DD}$  pins with a more efficient external regulator. To enable this option, the VREGENZ pin must be pulled high. Refer to the device datasheet for the acceptable range of voltage that must be supplied to the  $V_{DD}$  pins.

## 1.7.2 On-chip Power-On Reset (POR) and Brown-Out Reset (BOR) Circuit

Two on-chip supervisory circuits, the power-on reset (POR) and the brown-out reset (BOR) remove the burden of monitoring the  $V_{DD}$  and  $V_{DDIO}$  supply rails from the application board. The purpose of the POR is to create a clean reset throughout the device during the entire power-up procedure. The trip point is a looser, lower trip point than the BOR, which watches for dips in the  $V_{DD}$  or  $V_{DDIO}$  rail during device operation. The POR function is present on both  $V_{DD}$  and  $V_{DDIO}$  rails at all times. After initial device power-up, the BOR function is present on  $V_{DDIO}$  at all times, and on  $V_{DD}$  when the internal VREG is enabled (VREGENZ pin is pulled low). Both functions pull the  $\overline{XRS}$  pin low when one of the voltages is below their respective trip point. Additionally, when monitoring the  $V_{DD}$  rail, the BOR pulls  $\overline{XRS}$  low when  $V_{DD}$  is above its overvoltage trip point. See the device datasheet for the various trip points as well as the delay time from the removal of the fault condition to enable the BOR function, to the release of the  $\overline{XRS}$  pin.

A bit is provided in the BORCFG register (address 0x985) to disable both the VDD and VDDIO BOR functions. The default state of this bit is to enable the BOR function. When the BOR functions are disabled, the POR functions will remain enabled. See [Table 1-132](#) for a description of the BORCFG register. The BORCFG register state can only be modified by software and the  $\overline{XRS}$  pin signal. A CPU reset from the debugger will not modify this register.

**Figure 1-107. BOR Configuration (BORCFG) Register**

|          |  |   |          |          |        |
|----------|--|---|----------|----------|--------|
| 15       |  | 3 | 2        | 1        | 0      |
| Reserved |  |   | Reserved | Reserved | BORENZ |
| R-0      |  |   | R-1      | R/W-0    | R/W-0  |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 1-132. BOR Configuration (BORCFG) Field Descriptions**

| Bits | Field    | Value | Description                                              |
|------|----------|-------|----------------------------------------------------------|
| 15-1 | Reserved |       | Reserved                                                 |
| 0    | BORENZ   | 0     | BOR enable active low bit.<br>BOR functions are enabled. |
|      |          | 1     | BOR functions are disabled.                              |

This chapter is applicable for the code and data stored in the on-chip boot ROM. The boot ROM is factory-programmed with bootloading software. Bootmode signals (-TRST and select general-purpose I/Os) indicate to the bootloader which mode to use on power-up. The boot ROM also contains standard math tables, such as SIN/COS waveforms, for use in IQ math related algorithms found in the [C28x IQmath Library - A Virtual Floating Point Engine](#).

This chapter describes the purpose and features of the bootloader, as well as other contents of the device on-chip boot ROM, and identifies where all of the information is located within that memory. This chapter also refers to associated code that can be downloaded by way of the latest version of C2000ware in [C2000Ware](#).

|                                          |            |
|------------------------------------------|------------|
| <b>2.1 Boot ROM Memory Map.....</b>      | <b>196</b> |
| <b>2.2 Bootloader Features.....</b>      | <b>202</b> |
| <b>2.3 Building the Boot Table.....</b>  | <b>236</b> |
| <b>2.4 Bootloader Code Overview.....</b> | <b>240</b> |

## 2.1 Boot ROM Memory Map

The boot ROM is an 32K x 16 block of read-only memory located at addresses 0x3F 8000 - 0x3F FFFF.

The on-chip boot ROM is factory programmed with boot-load routines and both fixed-point and floating-point math tables. These are for use with the [C28x IQMath Library - A Virtual Floating Point Engine](#) and the [C28x FPU Fast RTS Library](#). This document describes the following items:

- Bootloader functions
- Version number, release date and checksum
- Reset vector
- Illegal trap vector (ITRAP)
- CPU vector table (used for test purposes only)
- IQmath Tables
- Selected IQmath functions
- Floating Point unit (FPU) math tables
- Flash API library

[Figure 2-1](#) and [Figure 2-2](#) show the memory map of the on-chip boot ROM. This will vary between F2806x parts and F2806xF and F2806M parts. The memory block is 32Kx16 in size and is located at 0x3F 8000-0x3F FFFF in both program and data space.

| On Chip Boot ROM                 |            | Section Start Address |
|----------------------------------|------------|-----------------------|
| Data Space                       | Prog Space |                       |
| Reserved                         |            | 0x3F 8000             |
|                                  |            |                       |
| FPU Math Tables                  |            | 0x3F D860             |
| IQTABLES                         |            | 0x3F DF00             |
| IQTABLES2                        |            | 0x3F EA50             |
| IQTABLES3                        |            | 0x3F EADC             |
| IQ Math Functions                |            | 0x3F EB86             |
| Bootloader Functions             |            | 0x3F F3B0             |
| Flash API                        |            | 0x3F F7D2             |
| ROM API Table                    |            | 0x3FFEB9              |
| ROM Version<br>ROM Checksum      |            | 0x3FFFBA              |
| Reset Vector<br>CPU Vector Table |            | 0x3F FFC0             |
|                                  |            | 0x3F FFFF             |

**Figure 2-1. F2806x Memory Map of On-Chip ROM**

| On Chip Boot ROM                 |            | Section Start Address |
|----------------------------------|------------|-----------------------|
| Data Space                       | Prog Space |                       |
| FAST + SPIN libraries            |            | 0x3F 8000             |
| FPU Math Tables                  |            | 0x3F D590             |
| IQTABLES                         |            | 0x3F DC30             |
| IQTABLES2                        |            | 0x3F E780             |
| IQTABLES3                        |            | 0x3F E80C             |
| IQ Math Functions                |            | 0x3F E8B6             |
| Bootloader Functions             |            | 0x3F F3B0             |
| Flash API                        |            | 0x3F F7D2             |
| ROM API Table                    |            | 0x3FFEB9              |
| ROM Version<br>ROM Checksum      |            | 0x3FFFBA              |
| Reset Vector<br>CPU Vector Table |            | 0x3F FFC0             |
|                                  |            | 0x3F FFFF             |

**Figure 2-2. F2806xM/2806xF Memory Map of On-Chip ROM**

### 2.1.1 On-Chip Boot ROM Math Tables

Approximately 4K of the boot ROM is reserved for floating-point and IQ math tables. These tables are provided to help improve performance and save SARAM space.

The floating-point math tables included in the boot ROM are used by the Texas Instruments™ [C28x FPU Fast RTS Library](#). The *C28x Fast RTS Library* is a collection of optimized floating-point math functions for C programmers of the C28x with floating-point unit. Designers of computationally intensive real-time applications can achieve execution speeds considerably faster than what are currently available without having to rewrite existing code. The functions listed in the features section are specifically optimized for the C28x + FPU controllers. The *Fast RTS Library* accesses the floating-point tables through the FPUmathTables memory section. If you do not wish to load a copy of these tables into the device, use the boot ROM memory addresses and label the section as “NOLOAD” as shown in [Example 2-1](#). This facilitates referencing the look-up tables without actually loading the section to the target.

The following math tables are included in the Boot ROM:

- **Sine/Cosine Table, Single-precision Floating point**
  - Table size: 1282 words
  - Contents: 32-bit floating-point samples for one and a quarter period sine wave
- **Normalized Arctan Table, Single-precision Floating point**
  - Table size: 388 words
  - Contents 32-bit second order coefficients for line of best fit
- **Exp Coefficient Table, Single-precision Floating point**
  - Table size: 20 words
  - Contents: 32-bit coefficients for calculating exp (X) using a Taylor series

#### Example 2-1. Linker Command File to Access FPU Tables

```
MEMORY
{
    PAGE 0 :
    ...
    FPUTABLES : origin = 0x3FD860, length = 0x0006A0
    ...
}
SECTIONS
{
    ...
    FPUmathTables : > FPOTABLES, PAGE = 0, TYPE = NOLOAD,
    ...
}
```

The fixed-point math tables included in the boot ROM are used by the Texas Instruments™ [C28x IQMath Library - A Virtual Floating Point Engine](#). The 28x IQmath Library is a collection of highly optimized and high precision mathematical functions for C/C++ programmers to seamlessly port a floating-point algorithm into fixed-point code on TMS320C28x devices.

These routines are typically used in computational-intensive real-time applications where optimal execution speed and high accuracy is critical. By using these routines, you can achieve execution speeds that are considerably faster than equivalent code written in standard ANSI C language. In addition, by providing ready-to-use high precision functions, the TI IQmath Library can shorten significantly your DSP application development time.

The IQmath library accesses the tables through the IQmathTables and the IQmathTablesRam linker sections. The IQmathTables section is completely included in the boot ROM. From the IQmathTablesRam section, only the IQexp table is included and the remainder must be loaded into the device if used. If you do not wish to load a copy of these tables already included in the ROM into the device, use the boot ROM memory addresses and



label the sections as “NOLOAD” as shown in [Example 2-2](#). This facilitates referencing the look-up tables without actually loading the section to the target. Refer to the IQMath Library documentation for more information.

### Example 2-2. Linker Command File to Access IQ Tables

```
MEMORY
{
    PAGE 0 :
    ...
    IQTABLES      : origin = 0x3FDB52, length = 0x000b50
    IQTABLES2     : origin = 0x3FE6A2, length = 0x00008C
    IQTABLES3     : origin = 0x3FE72E, length = 0x0000AA
    ...
}
SECTIONS
{
    ...
    IQmathTables : load = IQTABLES, type = NOLOAD, PAGE = 0
    IQmathTables2 > IQTABLES2, type = NOLOAD, PAGE = 0
    {
        IQmath.lib<IQNexpTable.obj> (IQmathTablesRam)
    }
    IQmathTables3 : load = IQTABLES3, PAGE = 0
    {
        IQNasinTable.obj (IQmathTablesRam)
    }
    IQmathTablesRam : load = DRAML1, PAGE = 1
    ...
}
```

The following math tables are included in the Boot ROM:

- **Sine/Cosine Table, IQ Math Table**

- Table size: 1282 words
- Q format: Q30
- Contents: 32-bit samples for one and a quarter period sine wave

This is useful for accurate sine wave generation and 32-bit FFTs. This can also be used for 16-bit math, just skip over every second value.

- **Normalized Inverse Table, IQ Math Table**

- Table size: 528 words
- Q format: Q29
- Contents: 32-bit normalized inverse samples plus saturation limits

This table is used as an initial estimate in the Newton-Raphson inverse algorithm. By using a more accurate estimate the convergence is quicker and hence cycle time is faster.

- **Normalized Square Root Table, IQ Math Table**

- Table size: 274 words
- Q format: Q30
- Contents: 32-bit normalized inverse square root samples plus saturation

This table is used as an initial estimate in the Newton-Raphson square-root algorithm. By using a more accurate estimate the convergence is quicker and hence cycle time is faster.

- **Normalized Arctan Table, IQ Math Table**

- Table size: 452 words
- Q format: Q30
- Contents 32-bit second order coefficients for line of best fit plus normalization table

This table is used as an initial estimate in the Arctan iterative algorithm. By using a more accurate estimate the convergence is quicker and hence cycle time is faster.

- **Rounding and Saturation Table, IQ Math Table**

- Table size: 360 words

- Q format: Q30
- Contents: 32-bit rounding and saturation limits for various Q values
- **Exp Min/Max Table, IQMath Table**
  - Table size: 120 words
  - Q format: Q1 - Q30
  - Contents: 32-bit Min and Max values for each Q value
- **Exp Coefficient Table, IQMath Table**
  - Table size: 20 words
  - Q format: Q31
  - Contents: 32-bit coefficients for calculating exp (X) using a taylor series
- **Inverse Sin/Cos Table, IQ Math Table**
  - Table size: 85 x 16
  - Q format: Q29
  - Contents: Coefficient table to calculate the formula  $f(x) = c4*x^4 + c3*x^3 + c2*x^2 + c1*x + c0$ .

### 2.1.2 On-Chip Boot ROM IQmath Functions

The following IQmath functions are included in the Boot ROM:

- IQNatan2 N= 15, 20, 24, 29
- IQNcos N= 15, 20, 24, 29
- IQNdiv N= 15, 20, 24, 29
- IQisqrt N= 15, 20, 24, 29
- IQNmag N= 15, 20, 24, 29
- IQNsin N= 15, 20, 24, 29
- IQNsqr t N= 15, 20, 24, 29

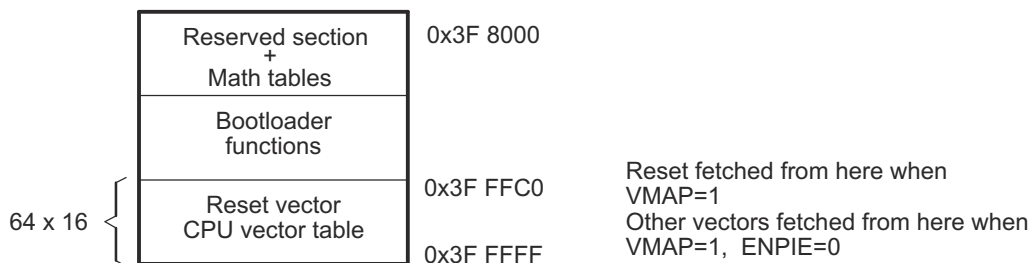
These functions can be accessed using the IQmath boot ROM symbol library included with the boot ROM source. If this library is linked in the project before the IQmath library, and the linker-priority option is used, then any math tables and IQmath functions within the boot ROM will be used first. Refer to the IQMath Library documentation for more information.

### 2.1.3 On-Chip Flash API

The boot ROM contains the API to program and erase the flash. This flash API can be accessed using the boot ROM flash API symbol library released with the boot ROM source. Refer to the 2806x Flash API Library documentation for information on how to use the symbol library.

### 2.1.4 CPU Vector Table

A CPU vector table resides in boot ROM memory from address 0x3F 8000-0x3F FFFF. This vector table is active after reset when VMAP = 1, ENPIE = 0 (PIE vector table disabled).



- A. The VMAP bit is located in Status Register 1 (ST1). VMAP is always 1 on reset. It can be changed after reset by software, however, the normal operating mode will be to leave VMAP=1.
- B. The ENPIE bit is located in the PIECTRL register. The default state of this bit at reset is 0, which disables the Peripheral Interrupt Expansion block (PIE).

**Figure 2-3. Vector Table Map**

The only vector that will normally be handled from the internal boot ROM memory is the reset vector located at 0x3F FFC0. The reset vector is factory programmed to point to the InitBoot function stored in the boot ROM. This function starts the boot load process. A series of checking operations is performed on TRST and General-Purpose I/O (GPIO I/O) pins to determine which boot mode to use. This boot mode selection is described in [Section 2.2.9](#).

The remaining vectors in the boot ROM are not used during normal operation. After the boot process is complete, you should initialize the Peripheral Interrupt Expansion (PIE) vector table and enable the PIE block. From that point on, all vectors, except reset, will be fetched from the PIE module and not the CPU vector table shown in [Table 2-1](#).

For TI silicon debug and test purposes the vectors located in the boot ROM memory point to locations in the M0 SARAM block as described in [Table 2-1](#). During silicon debug, you can program the specified locations in M0 with branch instructions to catch any vectors fetched from boot ROM. This is not required for normal device operation.

**Table 2-1. Vector Locations**

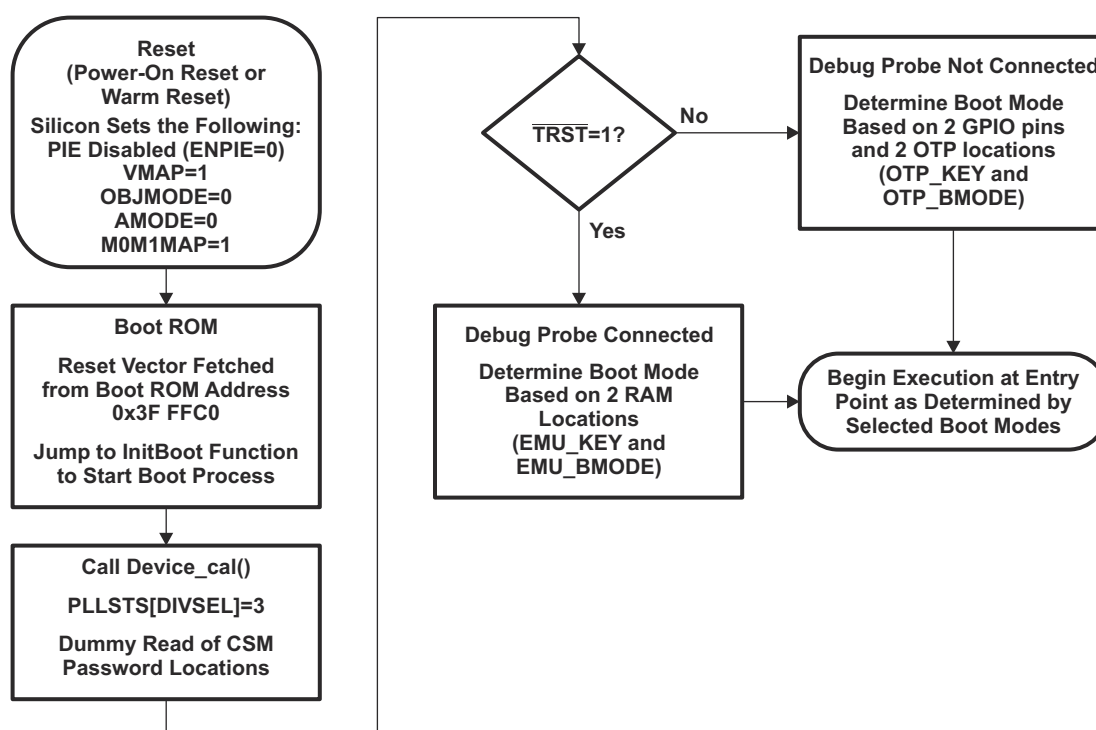
| Vector  | Location in Boot ROM | Contents (points to) | Vector   | Location in Boot ROM | Contents (points to) |
|---------|----------------------|----------------------|----------|----------------------|----------------------|
| RESET   | 0x3F FFC0            | InitBoot             | RTOSINT  | 0x3F FFE0            | 0x00 0060            |
| INT1    | 0x3F FFC2            | 0x00 0042            | Reserved | 0x3F FFE2            | 0x00 0062            |
| INT2    | 0x3F FFC4            | 0x00 0044            | NMI      | 0x3F FFE4            | 0x00 0064            |
| INT3    | 0x3F FFC6            | 0x00 0046            | ILLEGAL  | 0x3F FFE6            | ITRAPIsr             |
| INT4    | 0x3F FFC8            | 0x00 0048            | USER1    | 0x3F FFE8            | 0x00 0068            |
| INT5    | 0x3F FFCA            | 0x00 004A            | USER2    | 0x3F FFEA            | 0x00 006A            |
| INT6    | 0x3F FFCC            | 0x00 004C            | USER3    | 0x3F FFEC            | 0x00 006C            |
| INT7    | 0x3F FFCE            | 0x00 004E            | USER4    | 0x3F FFEE            | 0x00 006E            |
| INT8    | 0x3F FFD0            | 0x00 0050            | USER5    | 0x3F FFF0            | 0x00 0070            |
| INT9    | 0x3F FFD2            | 0x00 0052            | USER6    | 0x3F FFF2            | 0x00 0072            |
| INT10   | 0x3F FFD4            | 0x00 0054            | USER7    | 0x3F FFF4            | 0x00 0074            |
| INT11   | 0x3F FFD6            | 0x00 0056            | USER8    | 0x3F FFF6            | 0x00 0076            |
| INT12   | 0x3F FFD8            | 0x00 0058            | USER9    | 0x3F FFF8            | 0x00 0078            |
| INT13   | 0x3F FFDA            | 0x00 005A            | USER10   | 0x3F FFFA            | 0x00 007A            |
| INT14   | 0x3F FFDC            | 0x00 005C            | USER11   | 0x3F FFFC            | 0x00 007C            |
| DLOGINT | 0x3F FFDE            | 0x00 005E            | USER12   | 0x3F FFFE            | 0x00 007E            |

## 2.2 Bootloader Features

This section describes in detail the boot mode selection process, as well as the specifics of the bootloader operation.

### 2.2.1 Bootloader Functional Operation

The bootloader uses the state of  $\overline{\text{TRST}}$  and two GPIO signals to determine which boot mode to use. The boot mode selection process and the specifics of each bootloader are described in the remainder of this document. [Figure 2-4](#) shows the basic bootloader flow.



**Figure 2-4. Bootloader Flow Diagram**

The reset vector in boot ROM redirects program execution to the InitBoot function. After performing device initialization the bootloader will check the state of the  $\overline{\text{TRST}}$  pin to determine if a debugger is connected.

- **Debugger Boot (Debugger is connected and  $\overline{\text{TRST}} = 1$ )**

In debugger boot, the boot ROM checks two SARAM locations called EMU\_KEY and EMU\_BMODE for a boot mode. If the contents of either location are invalid, then the "wait" boot mode is used. All boot mode options can be accessed by modifying the value of EMU\_BMODE through the debugger when performing a debugger boot.

#### Stand-alone Boot ( $\overline{\text{TRST}} = 0$ )

If the device is in stand-alone boot mode, then the state of two GPIO pins are used to determine which boot mode execute. Options include: GetMode, wait, SCI, and parallel I/O. Each of the modes is described in detail in [Table 2-4](#). The GetMode option by default boots to flash but can be customized by programming two values into OTP to select another boot loader.

These boot modes mentioned here are discussed in detail in [Section 2.2.9](#).

After the selection process and if the required boot loading is complete, the processor will continue execution at an entry point determined by the boot mode selected. If a bootloader was called, then the input stream loaded by the peripheral determines this entry address. This data stream is described in [Section 2.2.11](#). If, instead, you choose to boot directly to Flash, OTP, or SARAM, the entry address is predefined for each of these memory blocks.

The following sections discuss in detail the different boot modes available and the process used for loading data code into the device.

### 2.2.2 Bootloader Device Configuration

At reset, any C28x CPU-based device is in C27x object-compatible mode. It is up to the application to place the device in the proper operating mode before execution proceeds.

On the C28x devices, when booting from the internal boot ROM, the device is configured for C28x operating mode by the boot ROM software. You are responsible for any additional configuration required.

For example, if your application includes C2xLP source, then you are responsible for configuring the device for C2xLP source compatibility prior to execution of code generated from C2xLP source.

The configuration required for each operating mode is summarized in [Table 2-2](#).

**Table 2-2. Configuration for Device Modes**

|                        | C27x Mode (Reset) <sup>(1)</sup> | C28x Mode | C2xLP Source Compatible Mode <sup>(1)</sup> |
|------------------------|----------------------------------|-----------|---------------------------------------------|
| OBJMODE                | 0                                | 1         | 1                                           |
| AMODE                  | 0                                | 0         | 1                                           |
| PAGE0                  | 0                                | 0         | 0                                           |
| M0M1MAP <sup>(2)</sup> | 1                                | 1         | 1                                           |
| Other Settings         | SXM = 1, C = 1, SPM = 0          |           |                                             |

(1) C27x refers to the TMS320C27x family of processors. C2xLP refers to the TMS320F24x/TMS320LF240xA family of devices that incorporate the C2xLP core. The information in [Table 2-2](#) is for reference only and is not applicable for the typical user development. For more information on the C2xLP core, refer to the [TMS320C28x DSP CPU and Instruction Set Reference Guide](#).

(2) Normally for C27x compatibility, the M0M1MAP would be 0. On these devices, however, it is tied off high internally; therefore, at reset, M0M1MAP is always configured for C28x mode.

### 2.2.3 PLL Multiplier and DIVSEL Selection

The Boot ROM changes the PLL multiplier (PLLCCR) and divider (PLLSTS[DIVSEL]) bits as follows:

- **All boot modes:** PLLCCR is not modified. PLLSTS[DIVSEL] is set to 3 for SYSCLKOUT = CLKIN/1. This increases the speed of the loaders.

#### Note

The PLL multiplier (PLLCCR) and divider (PLLSTS[DIVSEL]) are not affected by a reset from the debugger. Therefore, a boot that is initialized from a reset from the Code Composer Studio™ IDE may be at a different speed than booting by pulling the external reset line ( $\overline{\text{XRS}}$ ) low.

The reset value of PLLSTS[DIVSEL] is 0. This configures the device for SYSCLKOUT = CLKIN/4. The boot ROM will change this to SYSCLKOUT = CLKIN/1 to improve performance of the loaders. PLLSTS[DIVSEL] is left in this state when the boot ROM exits and it is up to the application to change it before configuring the PLLCCR register.

The boot ROM leaves PLLSTS[DIVSEL] in the CLKIN/1 state when the boot ROM exits. This is not a valid configuration if the PLL is used. Thus, the application must change it before configuring the PLLCCR register.

### 2.2.4 Watchdog Module

When branching directly to Flash, OTP, or M0 single-access RAM (SARAM) the watchdog is not touched. In the other boot modes, the watchdog is disabled before booting and then re-enabled and cleared before branching to the final destination address. In the case of an incorrect key value passed to the loader, the watchdog will be enabled and the device will boot to flash.

## 2.2.5 Taking an ITRAP Interrupt

If an illegal opcode is fetched, the 28x will take an ITRAP (illegal trap) interrupt. During the boot process, the interrupt vector used by the ITRAP is within the CPU vector table of the boot ROM. The ITRAP vector points to an interrupt service routine (ISR) within the boot ROM named ITRAPISR(). This interrupt service routine attempts to enable the watchdog and then loops forever until the processor is reset. This ISR will be used for any ITRAP until the user's application initializes and enables the peripheral interrupt expansion (PIE) block. Once the PIE is enabled, the ITRAP vector located within the PIE vector table will be used.

## 2.2.6 Internal Pullup Resistors

Each GPIO pin has an internal pullup resistor that can be enabled or disabled in software. The pins that are read by the boot mode selection code to determine the boot mode selection have pull-ups enabled after reset by default. In noisy conditions it is still recommended that you configure each of the boot mode selection pins externally.

The peripheral bootloaders all enable the pullup resistors for the pins that are used for control and data transfer. The bootloader leaves the resistors enabled for these pins when it exits. For example, the SCI-A bootloader enables the pullup resistors on the SCITXA and SCIRXA pins. It is your responsibility to disable them, if desired, after the bootloader exits.

## 2.2.7 PIE Configuration

The boot modes do not enable the PIE. It is left in its default state, which is disabled.

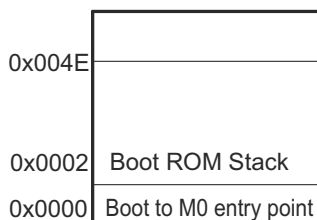
The boot ROM does, however, use the first six locations within the PIE vector table for emulation boot mode information and Flash API variables. These locations are not used by the PIE itself and not used by typical applications.

### Note

If you are porting code from another 28x processor, check to see if the code initializes the first six locations in the PIE vector table to some default value. If it does, then consider modifying the code to not write to these locations so the EMU boot mode will not be over written during debug. Refer to the 2806x C/C++ Header Files and Peripheral Examples.

## 2.2.8 Reserved Memory

The M0 memory block address range 0x0002-0x004E is reserved for the stack and .ebss code sections during the boot-load process. If code is bootloaded into this region there is no error checking to prevent it from corrupting the boot ROM stack. Address 0x0000-0x0001 is the boot to M0 entry point. This should be loaded with a branch instruction to the start of the main application when using "boot to SARAM" mode.



Boot ROM loaders on older C28x devices had the stack in M1 memory.

**Figure 2-5. Boot ROM Stack**

### Note

If code or data is bootloaded into the address range address range 0x0002-0x004E there is no error checking to prevent it from corrupting the boot ROM stack.

In addition, the first 6 locations of the PIE vector table are used by the boot ROM. These locations are not used by the PIE itself and are not used by typical applications. These locations are used as SARAM by the boot ROM and do not effect the behavior of the PIE. **Note:** Some example code from previous devices may initialize these locations. This will overwrite any boot mode you have populated. These locations are:

**Table 2-3. PIE Vector SARAM Locations Used by the Boot ROM**

| Location    | Name                 | Note                    |
|-------------|----------------------|-------------------------|
| 0x0D00 x 16 | EMU_KEY              | Used for emulation boot |
| 0x0D01 x 16 | EMU_BMODE            | Used for emulation boot |
| 0x0D02 x 32 | Flash_CallbackPtr    | Used by the flash API   |
| 0x0D04 x 32 | Flash_CPUScaleFactor | Used by the flash API   |

## 2.2.9 Bootloader Modes

To accommodate different system requirements, the boot ROM offers a variety of boot modes. This section describes the different boot modes and gives brief summary of their functional operation. The states of TRST and two GPIO pins are used to determine the desired boot mode as shown in [Table 2-4](#).

**Table 2-4. Boot Mode Selection**

|          | GPIO37 TDO | GPIO34 CMP2OUT | TRST |                |
|----------|------------|----------------|------|----------------|
| Mode EMU | x          | x              | 1    | Emulation Boot |
| Mode 0   | 0          | 0              | 0    | Parallel I/O   |
| Mode 1   | 0          | 1              | 0    | SCI            |
| Mode 2   | 1          | 0              | 0    | Wait           |
| Mode 3   | 1          | 1              | 0    | GetMode        |

### Note

The default behavior of the GetMode option on unprogrammed devices is to boot to flash. This behavior can be changed by programming two locations in the OTP as shown in [Table 2-6](#). In addition, if these locations are used by an application, then GetMode will jump to flash as long as OTP\_KEY ! =0x005A and/or OTP\_BMODE is not a valid value.

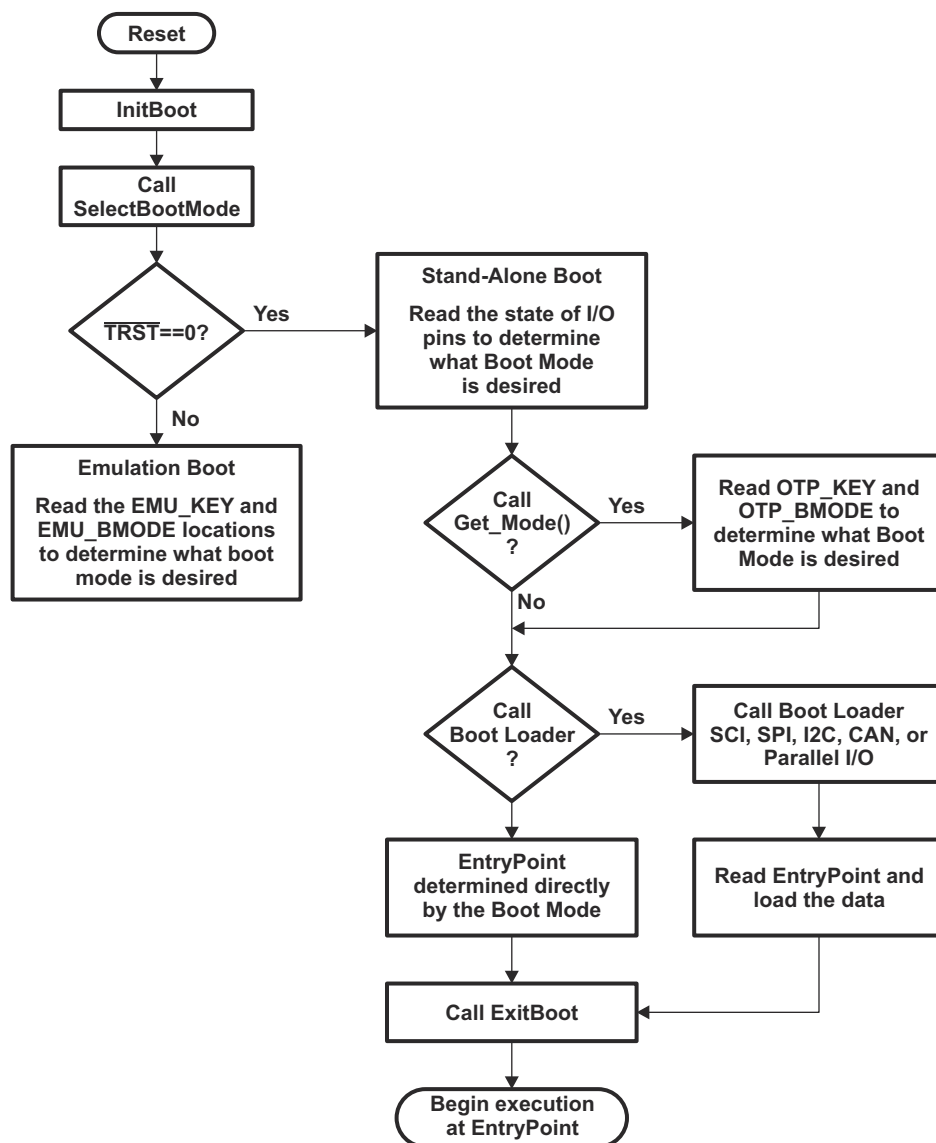
This device does not support the hardware wait-in-reset mode that is available on other C2000 parts. The "wait" boot mode can be used to emulate a wait-in-reset mode. The "wait" mode is very important for debugging devices with the CSM password programmed (that is, secured). When the device is powered up, the CPU will start running and may execute an instruction that performs an access to a protected emulation code security logic (ECSL) area. If this happens, the ECSL will trip and cause the debug connection to be cut. The "wait" mode keeps this from happening by looping within the boot ROM until the debug probe is connected.

[Figure 2-6](#) shows an overview of the boot process. Each step is described in greater detail in following sections.

While each sub-section gives details regarding the implementation of the native boot modes on the device, they do not address utilizing each boot mode for common system operations such as:

- Device Firmware Upgrade (DFU)
- Erasing the flash memory
- Verifying the flash memory
- Unlocking the security zones
- Running the embedded code from "main"
- Resetting the MCU

These operations and more are covered in the [Serial Flash Programming of C2000™ Microcontrollers](#).



**Figure 2-6. Boot ROM Function Overview**



**Table 2-5. Valid EMU\_KEY and EMU\_BMODE Values**

| Address | Name      | Value                                                                                                                               |
|---------|-----------|-------------------------------------------------------------------------------------------------------------------------------------|
| 0x0D00  | EMU_KEY   | if TRST == 1 and EMU_KEY == 0x55AA,<br>then check EMU_BMODE for the boot mode,<br>else { Invalid EMU_KEY<br>Boot mode = WAIT_BOOT } |
| 0x0D01  | EMU_BMODE | 0x0000      Boot mode = PARALLEL_BOOT                                                                                               |
|         |           | 0x0001      Boot mode = SCI_BOOT                                                                                                    |
|         |           | 0x0002      Boot mode = WAIT_BOOT                                                                                                   |
|         |           | 0x0003      Boot mode = GET_BOOT<br>(GetMode from OTP_KEY/OTP_BMODE)                                                                |
|         |           | 0x0004      Boot mode = SPI_BOOT                                                                                                    |
|         |           | 0x0005      Boot mode = I2C_BOOT                                                                                                    |
|         |           | 0x0006      Boot mode = OTP_BOOT                                                                                                    |
|         |           | 0x0007      Boot mode = CAN_BOOT                                                                                                    |
|         |           | 0x000A      Boot mode = RAM_BOOT                                                                                                    |
|         |           | 0x000B      Boot mode = FLASH_BOOT                                                                                                  |
|         |           | Other      Boot mode = WAIT_BOOT                                                                                                    |

Table 2-7 shows the expanded emulation boot mode table.

Following are two examples of an emulation boot.

**Example 2-3. Debug an application that loads through the SCI at boot.**

To debug an application that loads through the SCI at boot, follow these steps:

- Configure the pins for mode 1, SCI, and initiate a power-on-reset.
- The boot ROM will detect  $\overline{\text{TRST}} = 0$  and will use the two pins to determine SCI boot.
- The boot ROM populates EMU\_KEY with 0x55AA and EMU\_BMODE with SCI\_BOOT.
- The boot ROM sits in the SCI loader waiting for data.
- Connect the debugger.  $\overline{\text{TRST}}$  will go high.
- Perform a debugger reset and run. The boot loader will use the EMU\_BMODE and boot to SCI.

**Example 2-4. You want to connect your debug probe, but do not want application code to start executing before the debug probe connects.**

To connect your debug probe, but keep application code from executing before the debug probe connects:

- Configure GPIO37 and GPIO34 pins for mode 2, WAIT, and initiate a power-on-reset.
- The boot ROM will detect  $\overline{\text{TRST}} = 0$  and will use the two pins to determine wait boot.
- The boot ROM populates EMU\_KEY with 0x55AA and EMU\_BMODE with WAIT\_BOOT.
- The boot ROM sits in the wait routine.
- Connect the debugger;  $\overline{\text{TRST}}$  will go high.
- Modify the EMU\_BMODE via the debugger to boot to FLASH or other desired boot mode.
- Perform a debugger reset and run. The boot loader will use the EMU\_BMODE and boot to the desired loader or location.

### Note

**The behavior of debug probes with regards to  $\overline{\text{TRST}}$  differs.** Some debug probes pull  $\overline{\text{TRST}}$  high only when the Code Composer Studio™ IDE is in a connected state. For these emulators, if the CCS IDE is disconnected  $\overline{\text{TRST}}$  will return to a low state. With the CCS IDE disconnected, GPIO34 and GPIO37 will be used to determine the boot mode. For these debug probes, this is true even if the debug probe is physically connected.

Some debug probes pull  $\overline{\text{TRST}}$  high when the CCS IDE connects and leave it high as long as the power sense pin is active.  $\overline{\text{TRST}}$  will remain high even after the CCS IDE disconnects. For these debug probes, the EMU mode stored in RAM will be used unless the target is power cycled, causing the state of  $\overline{\text{TRST}}$  to reset back to a low state.

The following boot modes are invoked by the state of the boot mode pins if a debug probe is not connected:

#### • Wait

This device does not support the hardware wait-in-reset mode that is available on other C2000 parts. The "wait" boot mode can be used to emulate a wait-in-reset mode. The "wait" mode is very important for debugging devices with the CSM password programmed (that is, secured). When the device is powered up, the CPU will start running and may execute an instruction that performs an access to a emulation code security logic (ECSL) protected area. If this happens, the ECSL will trip and cause the debug connection to be cut. The "wait" mode keeps this from happening by looping within the boot ROM until a debug probe is connected.

This mode writes WAIT\_BOOT to EMU\_BMODE. Once the debug probe is connected you can then manually populate the EMU\_BMODE with the appropriate boot mode for the debug session.

#### • SCI

In this mode, the boot ROM will load code to be executed into on-chip memory via the SCI-A port. When invoked as a stand-alone mode, the boot ROM writes SCI\_BOOT to EMU\_BMODE.

#### • Parallel I/O 8-bit

The parallel I/O boot mode is typically used only by production flash programmers.

#### • GetMode

The GetMode option uses two locations within the USER OTP to determine the boot mode. On an un-programmed device, this mode will always boot to flash. On a programmed device, you can choose to program these locations to change the behavior. If either of these locations is not an expected value, then boot to flash will be used.

The last six words of user OTP region (0x3D7BFA to 0x3D7BFF) are reserved for the GetMode function usage.

The values used by the Get\_Mode() function are shown in [Table 2-6](#).

**Table 2-6. OTP Values for GetMode**

| Address  | Name      | Value                                                                                                                                                                                                                                                                                                                                                                                     |
|----------|-----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 0x3D7BFB | OTP_KEY   | GetMode will be entered if one of the two conditions is true:<br>Case 1: $\overline{\text{TRST}} == 0$ , GPIO34 == 1 and GPIO37 == 1<br>Case 2: $\overline{\text{TRST}} == 1$ , EMU_KEY == 0x55AA and EMU_BMODE == GET_BOOT<br>GetMode first checks the value of OTP_KEY:<br>if OTP_KEY == 0x005A, then check OTP_BMODE for the boot mode<br>else { Invalid key: Boot mode = FLASH_BOOT } |
| 0x3D7BFE | OTP_BMODE | 0x0001      Boot mode = SCI_BOOT<br>0x0004      Boot mode = SPI_BOOT<br>0x0005      Boot mode = I2C_BOOT<br>0x0006      Boot mode = OTP_BOOT<br>0x0007      Boot mode = CAN_BOOT<br>Other        Boot mode = FLASH_BOOT                                                                                                                                                                   |

The following boot modes are available through the emulation boot option. Some are also available as a programmed get mode option.

- **Jump to M0 SARAM**

This mode is only available in emulation boot. The boot ROM software configures the device for C28x operation and branches directly to address 0X000000. This is the first address in the M0 memory block.

- **Jump to branch instruction in flash memory.**

Jump to flash is the default behavior of the Get Mode boot option. Jump to flash is also available as an emulation boot option.

In this mode, the boot ROM software configures the device for C28x operation and branches directly to location 0x3F 7FF6. This location is just before the 128-bit code security module (CSM) password locations. You are required to have previously programmed a branch instruction at location 0x3F 7FF6 that will redirect code execution to either a custom boot-loader or the application code.

- **Jump to OTP Memory**

Jump to OTP is available only as an option programmed into OTP\_BMODE via the get mode function. With the debug probe connected, jump to OTP can also be achieved by manually writing the OTP\_BOOT value to EMU\_BMODE. The entry point location is 0x3D 7800.

- **SPI EEPROM or Flash boot mode (SPI-A)**

Jump to SPI is available in stand-alone mode as a programmed Get Mode option. That is, to configure a device for SPI boot in stand-alone mode, the OTP\_KEY and OTP\_BMODE locations must be programmed for SPI\_BOOT and the boot mode pins configured for the Get Mode boot option.

SCI boot is also available as an emulation boot option.

In this mode, the boot ROM will load code and data into on-chip memory from an external SPI EEPROM or SPI flash via the SPI-A port.

- **I2C-A boot mode (I2C-A)**

Jump to I2C is available in stand-alone mode as a programmed Get mode option. That is, to configure a device for I2C boot in stand-alone mode, the OTP\_KEY and OTP\_BMODE locations must be programmed for I2C\_BOOT and the boot mode pins configured for the Get Mode boot option.

I2C boot is also available as an emulation boot option.

In this mode, the boot ROM will load code and data into on-chip memory from an external serial EEPROM or flash at address 0x50 on the I2C-A bus.

- **eCAN-A boot mode (eCAN-A)**

Jump to eCAN is available in stand-alone mode as a programmed Get mode option. That is, to configure a device for eCAN boot in stand-alone mode, the OTP\_KEY and OTP\_BMODE locations must be programmed for CAN\_BOOT and the boot mode pins configured for the Get Mode boot option. eCAN boot is also available as an emulation boot option. In this mode, the eCAN-A peripheral is used to transfer data and code into the on-chip memory using eCAN-A mailbox 1. The transfer is an 8-bit data stream with two 8-bit values being transferred during each communication.

**Table 2-7. Emulation Boot Modes (TRST = 1)**

| TRST | GPIO37<br>TDO    | GPIO34 | EMU<br>KEY             | EMU<br>BMODE           | OTP<br>KEY               | OTP<br>BMODE             | Boot Mode<br>Selected <sup>(1)</sup> | EMU<br>KEY              | EMU<br>BMODE            |
|------|------------------|--------|------------------------|------------------------|--------------------------|--------------------------|--------------------------------------|-------------------------|-------------------------|
|      |                  |        | Read<br>from<br>0x0D00 | Read<br>from<br>0x0D01 | Read<br>from<br>0x3D7BFB | Read<br>from<br>0x3D7BFE |                                      | Written<br>to<br>0x0D00 | Written<br>to<br>0x0D01 |
| 1    | x <sup>(2)</sup> | x      | !=0x55AA               | x                      | x                        | x                        | Wait                                 | -                       | -                       |
|      |                  |        | 0x55AA                 | 0x0000                 | x                        | x                        | Parallel I/O                         | -                       | -                       |
|      |                  |        |                        | 0x0001                 | x                        | x                        | SCI                                  | -                       | -                       |
|      |                  |        |                        | 0x0002                 | x                        | x                        | Wait                                 | -                       | -                       |
|      |                  |        |                        | 0x0003                 | != 0x005A<br><br>0x005A  | x                        | GetMode: Flash                       | -                       | -                       |
|      |                  |        |                        |                        |                          | 0x0001                   | GetMode: SCI                         | -                       | -                       |
|      |                  |        |                        |                        |                          | 0x000B                   | GetMode: Flash                       | -                       | -                       |
|      |                  |        |                        |                        |                          | 0x0004                   | GetMode: SPI                         | -                       | -                       |
|      |                  |        |                        |                        |                          | 0x0005                   | GetMode: I2C                         | -                       | -                       |
|      |                  |        |                        |                        |                          | 0x0006                   | GetMode: OTP                         | -                       | -                       |
|      |                  |        |                        |                        |                          | 0x0007                   | GetMode: CAN                         | -                       | -                       |
|      |                  |        |                        |                        |                          | Other                    | GetMode: Flash                       | -                       | -                       |
|      |                  |        |                        | 0x0004                 | x                        | x                        | SPI                                  | -                       | -                       |
|      |                  |        |                        | 0x0005                 | x                        | x                        | I2C                                  | -                       | -                       |
|      |                  |        |                        | 0x0006                 | x                        | x                        | OTP                                  | -                       | -                       |
|      |                  |        |                        | 0x0007                 | x                        | x                        | CAN                                  | -                       | -                       |
|      |                  |        |                        | 0x000A                 | x                        | x                        | Boot to RAM                          | -                       | -                       |
|      |                  |        |                        | 0x000B                 | x                        | x                        | Boot to FLASH                        | -                       | -                       |
|      |                  |        |                        | Other                  | x                        | x                        | Wait                                 | -                       | -                       |

(1) Get Mode indicated the boot mode was derived from the values programmed in the OTP\_KEY and OTP\_BMODE locations.

(2) x = don't care.

**Table 2-8. Stand-Alone Boot Modes with (TRST = 0)**

| TRST | GPIO37<br>TDO | GPIO34 | EMU<br>KEY             | EMU<br>BMODE           | OTP<br>KEY               | OTP<br>BMODE             | Boot Mode<br>Selected <sup>(1)</sup> | <sup>(3)</sup><br>EMU<br>KEY | <sup>(3)</sup><br>EMU<br>BMODE |
|------|---------------|--------|------------------------|------------------------|--------------------------|--------------------------|--------------------------------------|------------------------------|--------------------------------|
|      |               |        | Read<br>from<br>0x0D00 | Read<br>from<br>0x0D01 | Read<br>from<br>0x3D7BFB | Read<br>from<br>0x3D7BFE |                                      | Written<br>to<br>0x0D00      | Written<br>to<br>0x0D01        |
| 0    | 0             | 0      | x <sup>(2)</sup>       | x                      | x                        | x                        | Parallel I/O                         | 0x55AA                       | 0x0000                         |
| 0    | 0             | 1      | x                      | x                      | x                        | x                        | SCI                                  | 0x55AA                       | 0x0001                         |
| 0    | 1             | 0      | x                      | x                      | x                        | x                        | Wait                                 | 0x55AA                       | 0x0002                         |
| 0    | 1             | 1      | x                      | x                      | !=0x005A                 | x                        | GetMode: Flash                       | 0x55AA                       | 0x0003                         |
|      |               |        |                        |                        | 0x005A                   | 0x0001                   | GetMode: SCI                         |                              |                                |
|      |               |        |                        |                        |                          | 0x000B                   | GetMode: Flash                       |                              |                                |
|      |               |        |                        |                        |                          | 0x0004                   | GetMode: SPI                         |                              |                                |
|      |               |        |                        |                        |                          | 0x0005                   | GetMode: I2C                         |                              |                                |
|      |               |        |                        |                        |                          | 0x0006                   | GetMode: OTP                         |                              |                                |
|      |               |        |                        |                        |                          | 0x0007                   | GetMode: CAN                         |                              |                                |
|      |               |        |                        |                        |                          | Other                    | GetMode: Flash                       |                              |                                |

(1) Get Mode indicates the boot mode was derived from the values programmed in the OTP\_KEY and OTP\_BMODE locations.

(2) x = don't care.

(3) The boot ROM will write this value to EMU\_KEY and EMU\_BMODE. This value can be used or overwritten by the user if a debugger is connected.

## 2.2.10 Device\_cal

The Device\_cal() routine is programmed into TI reserved memory by the factory. The boot ROM automatically calls the Device\_cal() routine to calibrate the internal oscillators and ADC with device specific calibration data. During normal operation, this process occurs automatically and no action is required by the user.

If the boot ROM is bypassed by the Code Composer Studio™ IDE during the development process, then the calibration must be initialized by application. For working examples, see the system initialization in the *C280 3x C/C++ Header Files and Peripheral Examples*.

### Note

Failure to initialize these registers will cause the oscillators and ADC to function out of specification. The following three steps describe how to call the Device\_cal routine from an application.

Step 1: Create a pointer to the Device\_cal function as shown in [Example 2-5](#). This #define is included in the Header Files and Peripheral Examples.

Step 2: Call the function pointed to by Device\_cal() as shown in [Example 2-5](#). The ADC clocks must be enabled before making this call.

### Example 2-5. Calling the Device\_cal() function

```
//Device call is a pointer to a function
//that begins at the address shown
# define Device_cal (void(*) (void)) 0x3D7C80
...
EALLOW;
SysCtrlRegs.PCLKCR0.bit.ADCENCLK = 1;
(*Device_cal)();
SysCtrlRegs.PCLKCR0.bit.ADCENCLK = 0;
EDIS;
...
```

### 2.2.11 Bootloader Data Stream Structure

The following two tables and associated examples show the structure of the data stream incoming to the bootloader. The basic structure is the same for all the bootloaders and is based on the C54x source data stream generated by the C54x hex utility. The C28x hex utility (hex2000.exe) has been updated to support this structure. The hex2000.exe utility is included with the C2000 code generation tools. All values in the data stream structure are in hex.

The first 16-bit word in the data stream is known as the key value. The key value indicates to the bootloader the width of the incoming stream: 8 or 16 bits. Note that not all bootloaders will accept both 8 and 16-bit streams. Please refer to the detailed information on each loader for the valid data stream width. For an 8-bit data stream, the key value is 0x08AA and for a 16-bit stream it is 0x10AA. If a bootloader receives an invalid key value, then the load is aborted.

The next eight words are used to initialize register values or otherwise enhance the bootloader by passing values to it. If a bootloader does not use these values then they are reserved for future use and the bootloader simply reads the value and then discards it. Currently only the SPI and I2C and parallel bootloaders use these words to initialize registers.

The tenth and eleventh words comprise the 22-bit entry point address. This address is used to initialize the PC after the boot load is complete. This address is most likely the entry point of the program downloaded by the bootloader.

The twelfth word in the data stream is the size of the first data block to be transferred. The size of the block is defined for both 8-bit and 16-bit data stream formats as the number of 16-bit words in the block. For example, to transfer a block of 20 8-bit data values from an 8-bit data stream, the block size would be 0x000A to indicate 10 16-bit words.

The next two words indicate to the loader the destination address of the block of data. Following the size and address will be the 16-bit words that makeup that block of data.

This pattern of block size/destination address repeats for each block of data to be transferred. Once all the blocks have been transferred, a block size of 0x0000 signals to the loader that the transfer is complete. At this point the loader will return the entry point address to the calling routine which in turn will cleanup and exit. Execution will then continue at the entry point address as determined by the input data stream contents.

**Table 2-9. General Structure Of Source Program Data Stream In 16-Bit Mode**

| Word | Contents                                                                                                                                                                  |
|------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1    | 10AA (KeyValue for memory width = 16 bits)                                                                                                                                |
| 2    | Register initialization value or reserved for future use                                                                                                                  |
| 3    | Register initialization value or reserved for future use                                                                                                                  |
| 4    | Register initialization value or reserved for future use                                                                                                                  |
| 5    | Register initialization value or reserved for future use                                                                                                                  |
| 6    | Register initialization value or reserved for future use                                                                                                                  |
| 7    | Register initialization value or reserved for future use                                                                                                                  |
| 8    | Register initialization value or reserved for future use                                                                                                                  |
| 9    | Register initialization value or reserved for future use                                                                                                                  |
| 10   | Entry point PC[22:16]                                                                                                                                                     |
| 11   | Entry point PC[15:0]                                                                                                                                                      |
| 12   | Block size (number of words) of the first block of data to load. If the block size is 0, this indicates the end of the source program. Otherwise another section follows. |
| 13   | Destination address of first block Address[31:16]                                                                                                                         |
| 14   | Destination address of first block Address[15:0]                                                                                                                          |
| 15   | First word of the first block in the source being loaded                                                                                                                  |
| ...  | ...                                                                                                                                                                       |
| ...  | ...                                                                                                                                                                       |
| .    | Last word of the first block of the source being loaded                                                                                                                   |
| .    | Block size of the 2nd block to load.                                                                                                                                      |
| .    | Destination address of second block Address[31:16]                                                                                                                        |
| .    | Destination address of second block Address[15:0]                                                                                                                         |
| .    | First word of the second block in the source being loaded                                                                                                                 |
| .    | ...                                                                                                                                                                       |
| .    | Last word of the second block of the source being loaded                                                                                                                  |
| .    | Block size of the last block to load                                                                                                                                      |
| .    | Destination address of last block Address[31:16]                                                                                                                          |
| .    | Destination address of last block Address[15:0]                                                                                                                           |
| .    | First word of the last block in the source being loaded                                                                                                                   |
| ...  | ...                                                                                                                                                                       |
| ...  | ...                                                                                                                                                                       |
| n    | Last word of the last block of the source being loaded                                                                                                                    |
| n+1  | Block size of 0000h - indicates end of the source program                                                                                                                 |

### Example 2-6. Data Stream Structure 16-bit

```

10AA          ; 0x10AA 16-bit key value
0000 0000 0000 0000 ; 8 reserved words
0000 0000 0000 0000
003F 8000     ; 0x003F8000 EntryAddr, starting point after boot load completes
0005          ; 0x0005 - First block consists of 5 16-bit words
003F 9010     ; 0x003F9010 - First block will be loaded starting at 0x3F9010
0001 0002 0003 0004 ; Data loaded = 0x0001 0x0002 0x0003 0x0004 0x0005
0005
0002          ; 0x0002 - 2nd block consists of 2 16-bit words
003F 8000     ; 0x003F8000 - 2nd block will be loaded starting at 0x3F8000
7700 7625     ; Data loaded = 0x7700 0x7625
0000          ; 0x0000 - Size of 0 indicates end of data stream
After load has completed the following memory values will have been initialized as follows:
Location      Value
0x3F9010      0x0001
0x3F9011      0x0002
0x3F9012      0x0003
0x3F9013      0x0004
0x3F9014      0x0005
0x3F8000      0x7700
0x3F8001      0x7625
PC Begins execution at 0x3F8000

```

In 8-bit mode, the least significant byte (LSB) of the word is sent first followed by the most significant byte (MSB). For 32-bit values, such as a destination address, the most significant word (MSW) is loaded first, followed by the least significant word (LSW). The bootloaders take this into account when loading an 8-bit data stream.



**Table 2-10. LSB/MSB Loading Sequence in 8-Bit Data Stream**

| Byte                  |     | Contents                                                                                                                                                                                                                                          |                                                        |
|-----------------------|-----|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------|
| LSB (First Byte of 2) |     | MSB (Second Byte of 2)                                                                                                                                                                                                                            |                                                        |
| 1                     | 2   | LSB: AA (KeyValue for memory width = 8 bits)                                                                                                                                                                                                      | MSB: 08h (KeyValue for memory width = 8 bits)          |
| 3                     | 4   | LSB: Register initialization value or reserved                                                                                                                                                                                                    | MSB: Register initialization value or reserved         |
| 5                     | 6   | LSB: Register initialization value or reserved                                                                                                                                                                                                    | MSB: Register initialization value or reserved         |
| 7                     | 8   | LSB: Register initialization value or reserved                                                                                                                                                                                                    | MSB: Register initialization value or reserved         |
| ...                   | ... | ...                                                                                                                                                                                                                                               | ...                                                    |
| 17                    | 18  | LSB: Register initialization value or reserved                                                                                                                                                                                                    | MSB: Register initialization value or reserved         |
| 19                    | 20  | LSB: Upper half of Entry point PC[23:16]                                                                                                                                                                                                          | MSB: Upper half of entry point PC[31:24] (Always 0x00) |
| 21                    | 22  | LSB: Lower half of Entry point PC[7:0]                                                                                                                                                                                                            | MSB: Lower half of Entry point PC[15:8]                |
| 23                    | 24  | LSB: Block size in words of the first block to load. If the block size is 0, this indicates the end of the source program. Otherwise another block follows. For example, a block size of 0x000A would indicate 10 words or 20 bytes in the block. | MSB: block size                                        |
| 25                    | 26  | LSB: MSW destination address, first block Addr[23:16]                                                                                                                                                                                             | MSB: MSW destination address, first block Addr[31:24]  |
| 27                    | 28  | LSB: LSW destination address, first block Addr[7:0]                                                                                                                                                                                               | MSB: LSW destination address, first block Addr[15:8]   |
| 29                    | 30  | LSB: First word of the first block being loaded                                                                                                                                                                                                   | MSB: First word of the first block being loaded        |
| ...                   | ... | ...                                                                                                                                                                                                                                               | ...                                                    |
| ...                   | ... | ...                                                                                                                                                                                                                                               | ...                                                    |
| .                     | .   | LSB: Last word of the first block to load                                                                                                                                                                                                         | MSB: Last word of the first block to load              |
| .                     | .   | LSB: Block size of the second block                                                                                                                                                                                                               | MSB: Block size of the second block                    |
| .                     | .   | LSB: MSW destination address, second block Addr[23:16]                                                                                                                                                                                            | MSB: MSW destination address, second block Addr[31:24] |
| .                     | .   | LSB: LSW destination address, second block Addr[7:0]                                                                                                                                                                                              | MSB: LSW destination address, second block Addr[15:8]  |
| .                     | .   | LSB: First word of the second block being loaded                                                                                                                                                                                                  | MSB: First word of the second block being loaded       |
| ...                   | ... | ...                                                                                                                                                                                                                                               | ...                                                    |
| ...                   | ... | ...                                                                                                                                                                                                                                               | ...                                                    |
| .                     | .   | LSB: Last word of the second block                                                                                                                                                                                                                | MSB: Last word of the second block                     |
| .                     | .   | LSB: Block size of the last block                                                                                                                                                                                                                 | MSB: Block size of the last block                      |
| .                     | .   | LSB: MSW of destination address of last block Addr[23:16]                                                                                                                                                                                         | MSB: MSW destination address, last block Addr[31:24]   |
| .                     | .   | LSB: LSW destination address, last block Addr[7:0]                                                                                                                                                                                                | MSB: LSW destination address, last block Addr[15:8]    |
| .                     | .   | LSB: First word of the last block being loaded                                                                                                                                                                                                    | MSB: First word of the last block being loaded         |
| ...                   | ... | ...                                                                                                                                                                                                                                               | ...                                                    |
| ...                   | ... | ...                                                                                                                                                                                                                                               | ...                                                    |
| .                     | .   | LSB: Last word of the last block                                                                                                                                                                                                                  | MSB: Last word of the last block                       |
| n                     | n+1 | LSB: 00h                                                                                                                                                                                                                                          | MSB: 00h - indicates the end of the source             |

### Example 2-7. Data Stream Structure 8-bit

```

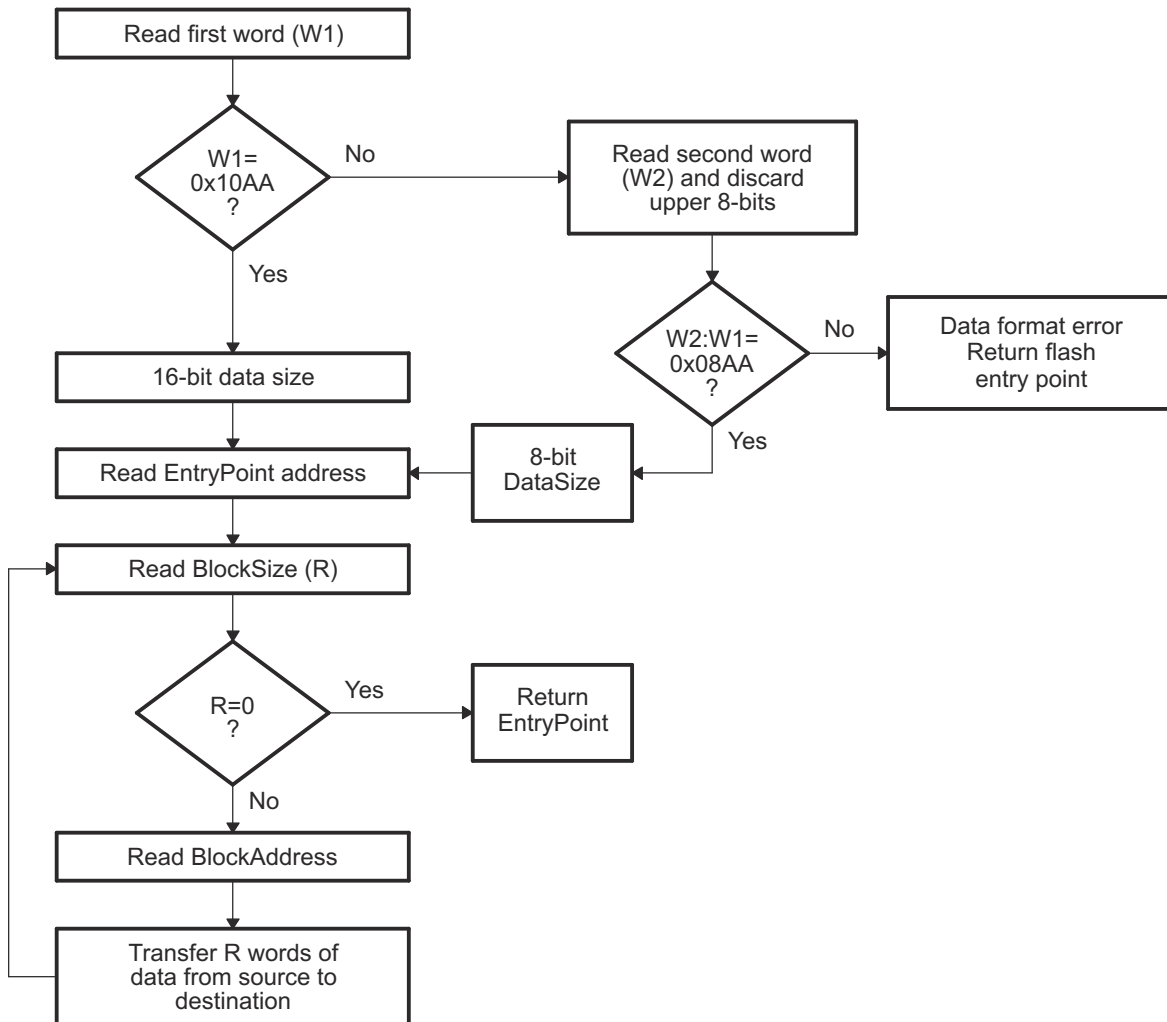
AA 08          ; 0x08AA 8-bit key value
00 00 00 00    ; 8 reserved words
00 00 00 00
00 00 00 00
00 00 00 00
3F 00 00 80    ; 0x003F8000 EntryAddr, starting point after boot load completes
05 00          ; 0x0005 - First block consists of 5 16-bit words
3F 00 10 90    ; 0x003F9010 - First block will be loaded starting at 0x3F9010
01 00          ; Data loaded = 0x0001 0x0002 0x0003 0x0004 0x0005
02 00
03 00
04 00
05 00
02 00          ; 0x0002 - 2nd block consists of 2 16-bit words
3F 00 00 80    ; 0x003F8000 - 2nd block will be loaded starting at 0x3F8000
00 77          ; Data loaded = 0x7700 0x7625
25 76
00 00          ; 0x0000 - Size of 0 indicates end of data stream
After load has completed the following memory values will have been initialized as follows:
Location      Value
0x3F9010      0x0001
0x3F9011      0x0002
0x3F9012      0x0003
0x3F9013      0x0004
0x3F9014      0x0005
0x3F8000      0x7700
0x3F8001      0x7625
PC Begins execution at 0x3F8000

```

## 2.2.12 Basic Transfer Procedure

Figure 2-7 illustrates the basic process a bootloader uses to transfer data and start program execution. This process occurs after the bootloader determines the valid boot mode selected by the state of  $\overline{\text{TRST}}$  and GPIO pins.

The loader first compares the first value sent by the host against the 16-bit key value of 0x10AA. If the first word matches the key value, the loader continuously fetches data until it transfers all the hex words and then starts program execution. But, if the key does not match the key value, then the loader aborts and the CPU jumps to the default flash entry point.



8-bit and 16-bit transfers are not valid for all boot modes. If only one mode is valid, then this decision tree is skipped and the key value is only checked for correctness. See the info specific to a particular bootloader for any limitations.

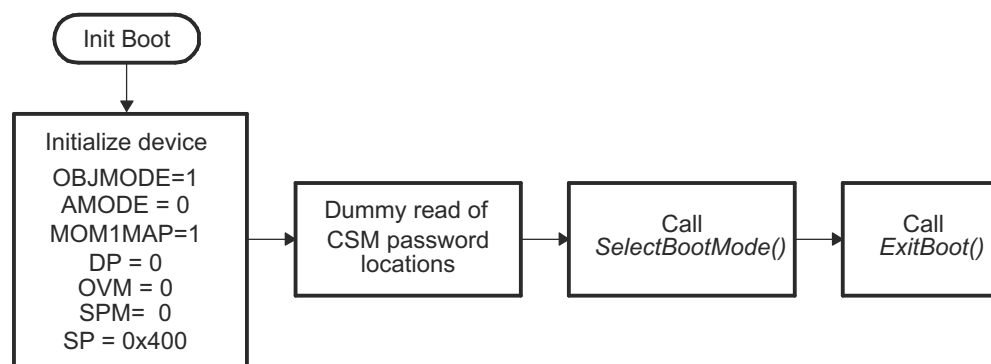
In 8-bit mode, the LSB of the 16-bit word is read first followed by the MSB.

**Figure 2-7. Bootloader Basic Transfer Procedure**

## 2.2.13 InitBoot Assembly Routine

The first routine called after reset is the InitBoot assembly routine. This routine initializes the device for operation in C28x object mode. InitBoot also performs a dummy read of the Code Security Module (CSM) password locations. If the CSM passwords are erased (all 0xFFFFs) then this has the effect of unlocking the CSM. Otherwise the CSM will remain locked and this dummy read of the password locations will have no effect. This can be useful if you have a new device that you want to boot load.

After the dummy read of the CSM password locations, the InitBoot routine calls the SelectBootMode function. This function determines the type of boot mode desired by the state of  $\overline{\text{TRST}}$  and certain GPIO pins. This process is described in [Section 2.2.14](#). Once the boot is complete, the SelectBootMode function passes back the entry point address (EntryAddr) to the InitBoot function. EntryAddr is the location where code execution will begin after the bootloader exits. InitBoot then calls the ExitBoot routine that then restores CPU registers to their reset state and exits to the EntryAddr that was determined by the boot mode.



**Figure 2-8. Overview of InitBoot Assembly Function**

## 2.2.14 SelectBootMode Function

To determine the desired boot mode, the SelectBootMode function examines the state of  $\overline{\text{TRST}}$  and 2 GPIO pins as shown in [Table 2-4](#).

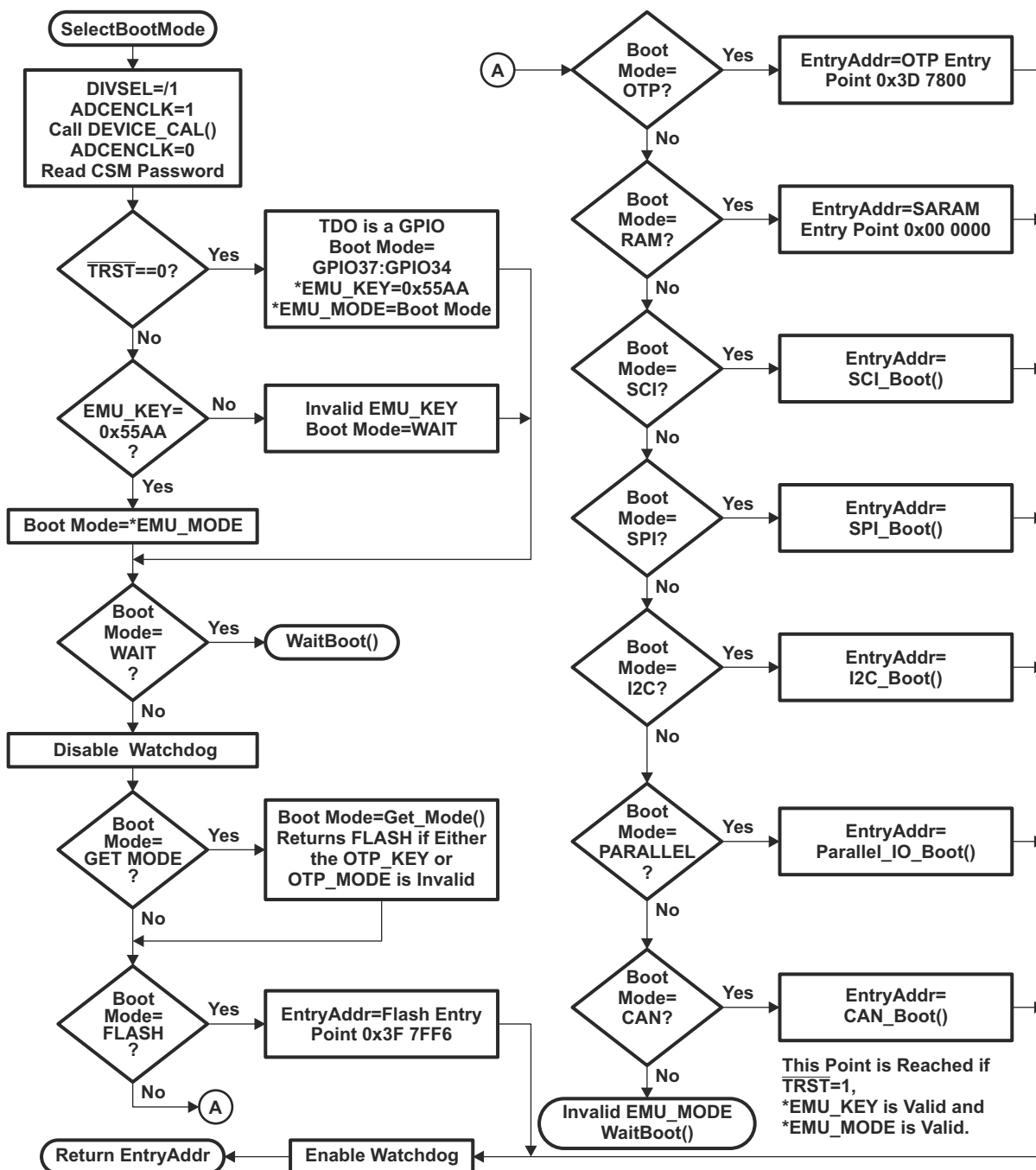
For a boot mode to be selected, the pins corresponding to the desired boot mode have to be pulled low or high until the selection process completes. Note that the state of the selection pins is not latched at reset; they are sampled some cycles later in the SelectBootMode function. The internal pullup resistors are enabled at reset for the boot mode selection pins. It is still suggested that the boot mode configuration be made externally to avoid the effect of any noise on these pins.

### Note

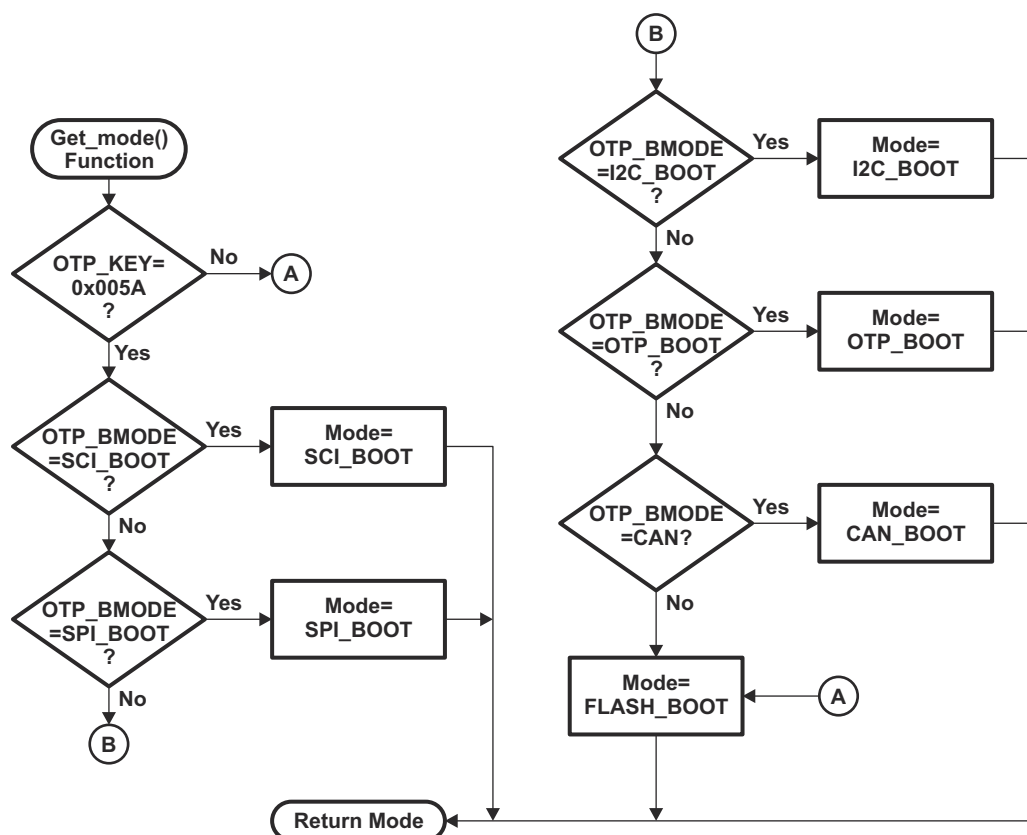
The SelectBootMode routine disables the watchdog before calling the SCI, I2C, SPI, or parallel bootloaders. The bootloaders do not service the watchdog and assume that it is disabled. Before exiting, the SelectBootMode routine will re-enable the watchdog and reset its timer.

If a bootloader is not going to be called, then the watchdog is left untouched.

When selecting a boot mode, the pins should be pulled high or low through a weak pulldown or weak pull-up such that the device can drive them to a new state when required.



**Figure 2-9. Overview of the SelectBootMode Function**



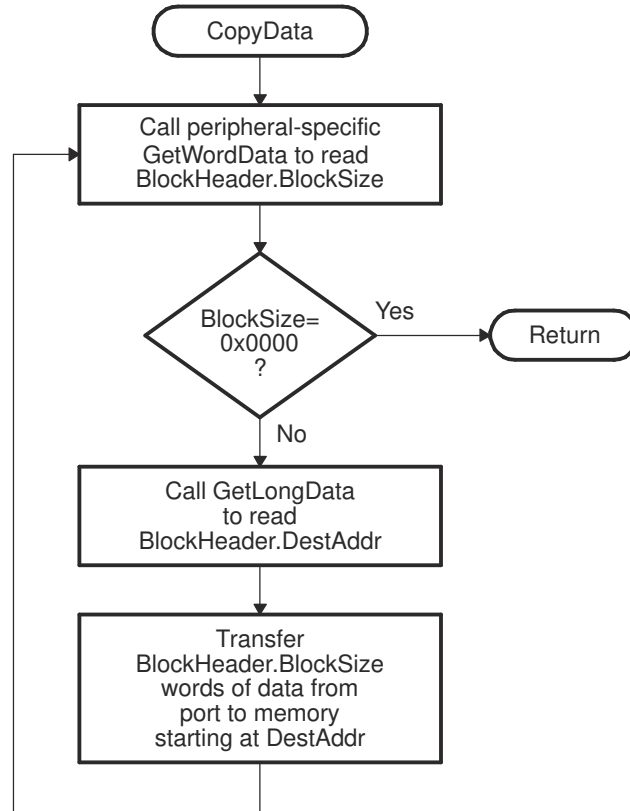
**Figure 2-10. Overview of Get\_mode() Function**

## 2.2.15 CopyData Function

Each of the bootloaders uses the same function to copy data from the port to the device's SARAM. This function is the CopyData() function. This function uses a pointer to a GetWordData function that is initialized by each of the loaders to properly read data from that port. For example, when the SPI loader is evoked, the GetWordData function pointer is initialized to point to the SPI-specific SPI\_GetWordData function. Thus when the CopyData() function is called, the correct port is accessed. The flow of the CopyData function is shown in [Figure 2-11](#).

### Note

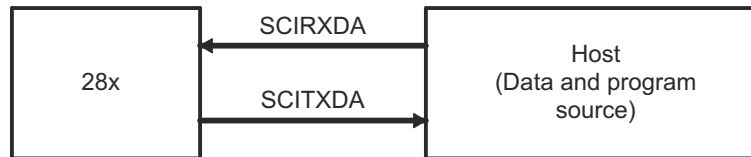
BlockSize must be less than 0xFFFF for correct operation of the CopyData function. This means the max possible value of BlockSize is 0xFFFE, not 0xFFFF.



**Figure 2-11. Overview of CopyData Function**

### 2.2.16 SCI\_Boot Function

The SCI boot mode asynchronously transfers code from SCI-A to internal memory. This boot mode only supports an incoming 8-bit data stream and follows the same data flow as outlined in [Example 2-7](#).



**Figure 2-12. Overview of SCI Bootloader Operation**

The SCI-A loader uses following pins:

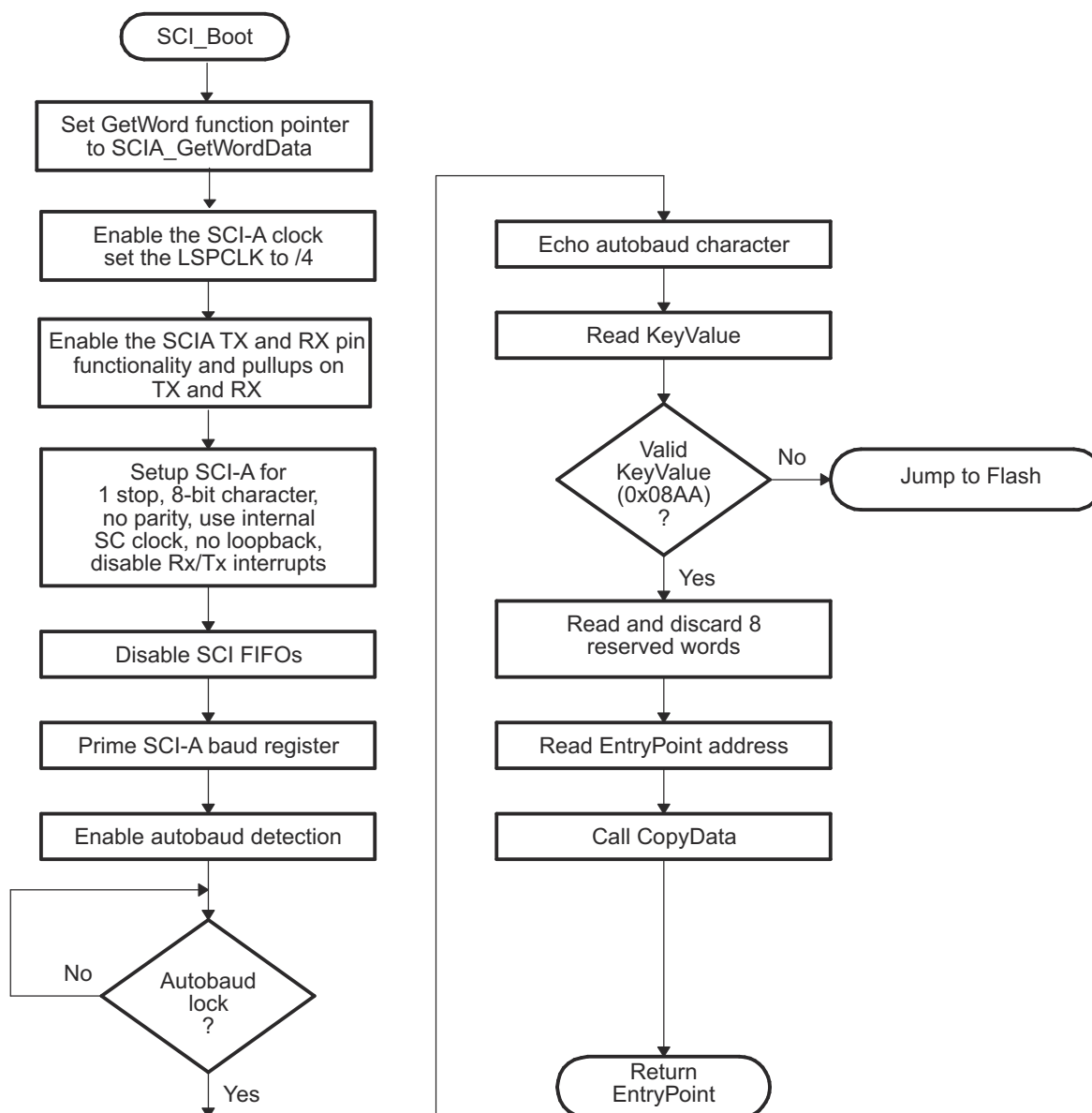
- SCIRXDA on GPIO28
- SCITXDA on GPIO29

The 28x device communicates with the external host device by communication through the SCI-A peripheral. The autobaud feature of the SCI port is used to lock baud rates with the host. For this reason the SCI loader is very flexible and you can use a number of different baud rates to communicate with the device.

After each data transfer, the 28x will echo back the 8-bit character received to the host. In this manner, the host can perform checks that each character was received by the 28x.

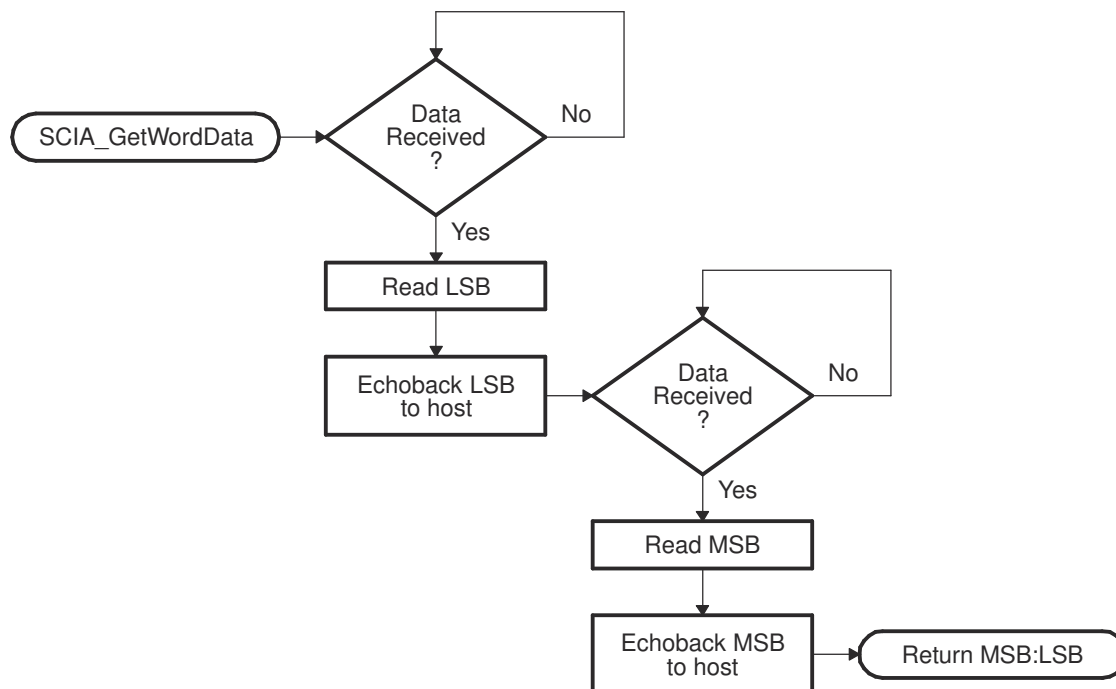
At higher baud rates, the slew rate of the incoming data bits can be effected by transceiver and connector performance. While normal serial communications may work well, this slew rate may limit reliable auto-baud detection at higher baud rates (typically beyond 100 kbaud) and cause the auto-baud lock feature to fail. To avoid this, the following is recommended:

1. Achieve a baud-lock between the host and 28x SCI bootloader using a lower baud rate.
2. Load the incoming 28x application or custom loader at this lower baud rate.
3. The host may then handshake with the loaded 28x application to set the SCI baud rate register to the desired high baud rate.



**Figure 2-13. Overview of SCI\_Boot Function**

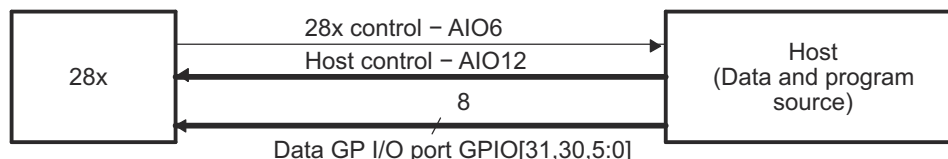




**Figure 2-14. Overview of SCI\_GetWordData Function**

### 2.2.17 Parallel\_Boot Function (GPIO)

The parallel general purpose I/O (GPIO) boot mode asynchronously transfers code from GPIO0 -GPIO5, GPIO30-GPIO31 to internal memory. Each value is 8-bits long and follows the same data flow as outlined in [Section 2.2.11](#).



**Figure 2-15. Overview of Parallel GPIO Bootloader Operation**

The parallel GPIO loader uses following pins:

- Data on GPIO[31,30,5:0]
- 28x Control on AIO6 (external pull-up resistor may be required)
- Host Control on AIO12 (external pull-up resistor required)

The 28x communicates with the external host device by polling and driving the AIO12 and AIO6 lines. An external pull-up resistor is required for AIO12 because AIO pins lack internal pull-up circuitry required to prevent the 28x from reading data prematurely. Depending on your system an external pull-up may also be required on AIO6. The handshake protocol shown in [Figure 2-16](#) must be used to successfully transfer each word with GPIO [31,30,5:0]. This protocol is very robust and allows for a slower or faster host to communicate with the 28x.

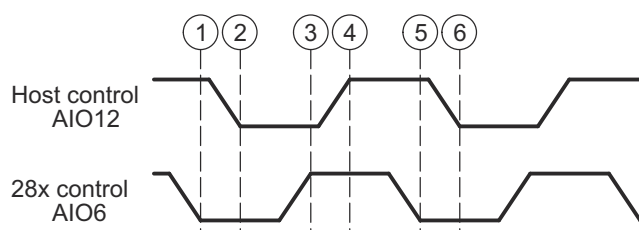
Two consecutive 8-bit words are read to form a single 16-bit word. The most-significant byte (MSB) is read first followed by the least-significant byte (LSB). In this case, data is read from GPIO[31,30,5:0].

The 8-bit data stream is shown in [Table 2-11](#).

**Table 2-11. Parallel GPIO Boot 8-Bit Data Stream**

| Bytes   | GPIO[ 31,30,5:0]<br>(Byte 1 of 2) | GPIO[ 31,30,5:0]<br>(Byte 2 of 2) | Description                                                                           |
|---------|-----------------------------------|-----------------------------------|---------------------------------------------------------------------------------------|
| 1 2     | AA                                | 08                                | 0x08AA (KeyValue for memory width = 16 bits)                                          |
| 3 4     | 00                                | 00                                | 8 reserved words (words 2-9)                                                          |
| ...     | ...                               | ...                               | ...                                                                                   |
| 17 18   | 00                                | 00                                | Last reserved word                                                                    |
| 19 20   | BB                                | 00                                | Entry point PC[22:16]                                                                 |
| 21 22   | DD                                | CC                                | Entry point PC[15:0] (PC = 0x00BBCCDD)                                                |
| 23 24   | NN                                | MM                                | Block size of the first block of data to load = 0xMMNN words                          |
| 25 26   | BB                                | AA                                | Destination address of first block Address[31:16]                                     |
| 27 28   | DD                                | CC                                | Destination address of first block Address[15:0] (Address = 0xAABBCCDD)               |
| 29 30   | BB                                | AA                                | First word of the first block in the source being loaded = 0xAABB                     |
| ...     | ...                               | ...                               | ...                                                                                   |
| ...     | ...                               | ...                               | Data for this section.                                                                |
| ...     | ...                               | ...                               | ...                                                                                   |
| .       | BB                                | AA                                | Last word of the first block of the source being loaded = 0xAABB                      |
| .       | NN                                | MM                                | Block size of the 2nd block to load = 0xMMNN words                                    |
| .       | BB                                | AA                                | Destination address of second block Address[31:16]                                    |
| .       | DD                                | CC                                | Destination address of second block Address[15:0]                                     |
| .       | BB                                | AA                                | First word of the second block in the source being loaded                             |
| .       | ...                               | ...                               | ...                                                                                   |
| n n+1   | BB                                | AA                                | Last word of the last block of the source being loaded<br>(More sections if required) |
| n+2 n+3 | 00                                | 00                                | Block size of 0000h - indicates end of the source program                             |

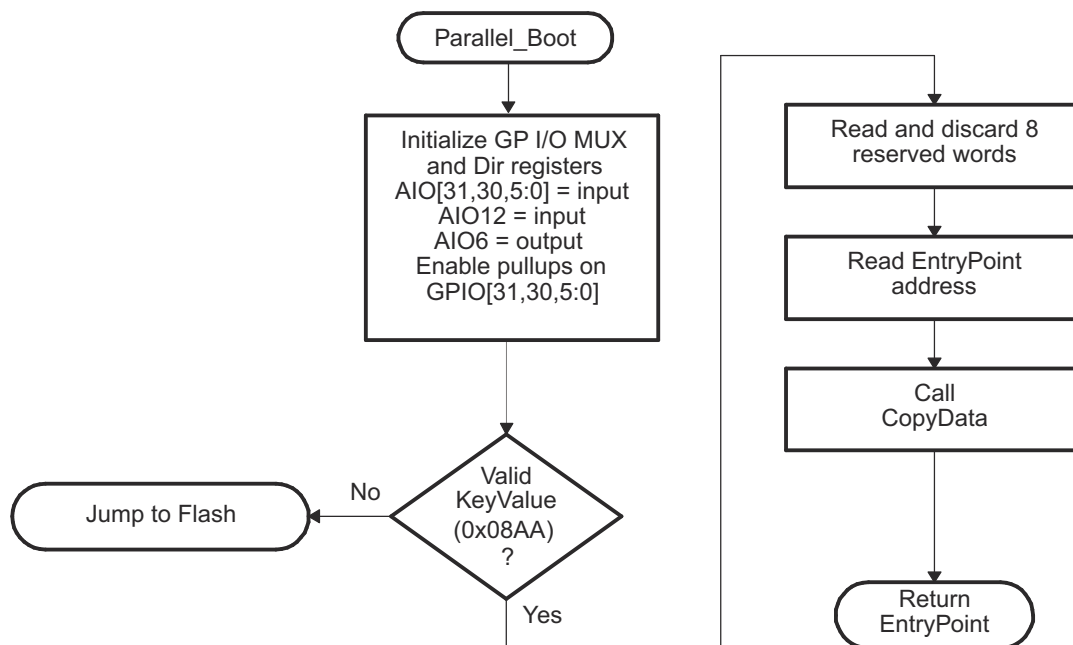
The 28x device first signals the host that it is ready to begin data transfer by pulling the AIO6 pin low. The host load then initiates the data transfer by pulling the AIO12 pin low. The complete protocol is shown in the diagram below:


**Figure 2-16. Parallel GPIO Boot Loader Handshake Protocol**

1. The 28x device indicates it is ready to start receiving data by pulling the AIO6 pin low.
2. The bootloader waits until the host puts data on GPIO [31,30,5:0]. The host signals to the 28x device that data is ready by pulling the AIO12 pin low.
3. The 28x device reads the data and signals the host that the read is complete by pulling AIO6 pin high.
4. The bootloader waits until the host acknowledges the 28x by pulling AIO12 pin high.
5. The 28x device again indicates it is ready for more data by pulling the AIO6 pin low.

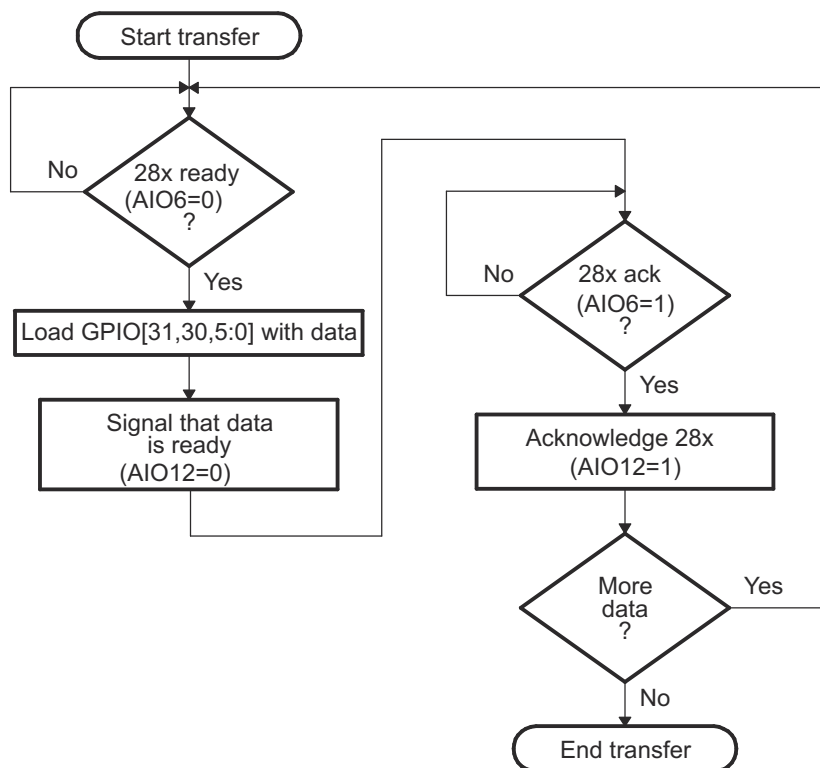
This process is repeated for each data value to be sent.

Figure 2-17 shows an overview of the Parallel GPIO bootloader flow.



**Figure 2-17. Parallel GPIO Mode Overview**

Figure 2-18 shows the transfer flow from the host side. The operating speed of the CPU and host are not critical in this mode as the host will wait for the 28x and the 28x will in turn wait for the host. In this manner the protocol will work with both a host running faster and a host running slower than the 28x.

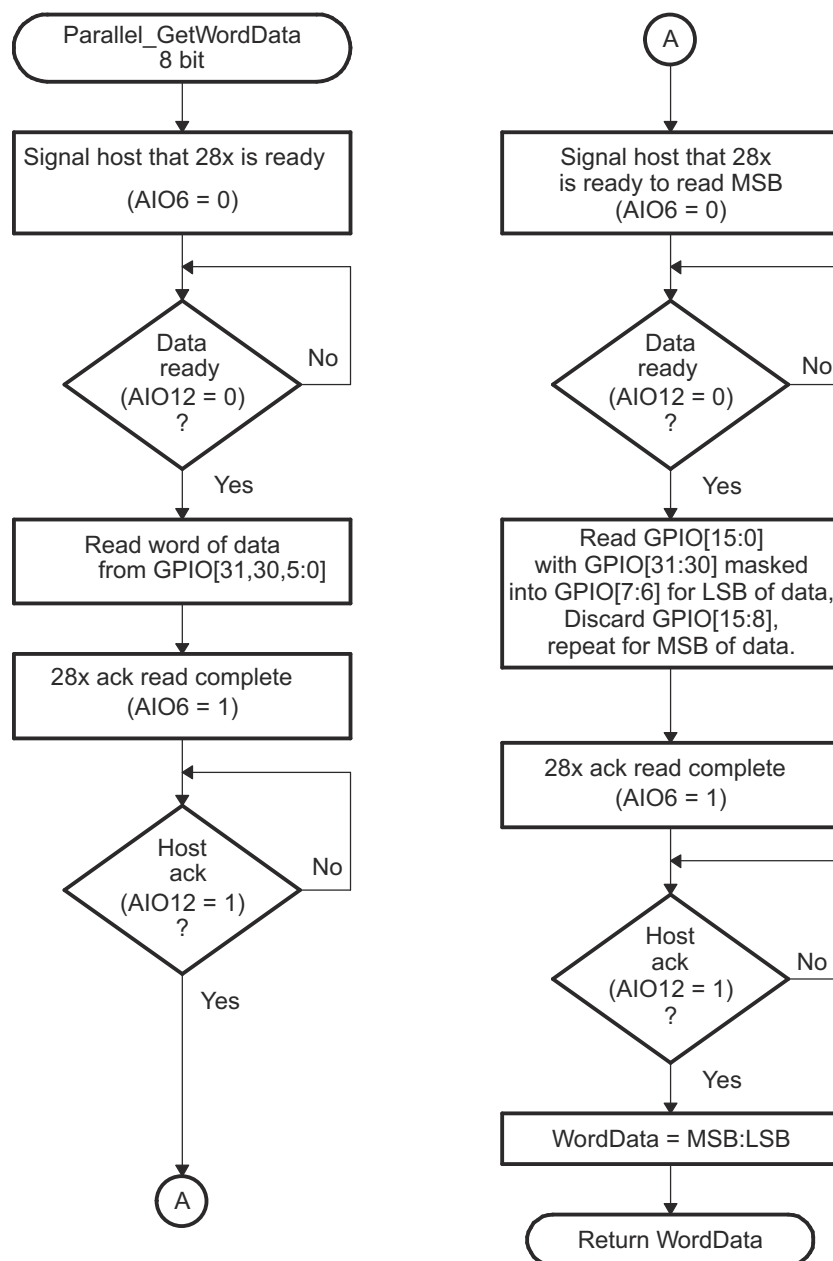


**Figure 2-18. Parallel GPIO Mode - Host Transfer Flow**

Figure 2-19 shows the flow used to read a single word of data from the parallel port.

#### • 8-bit data stream

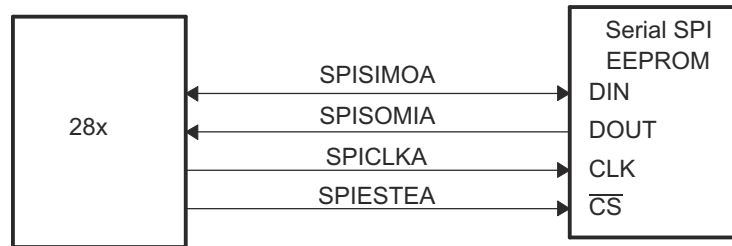
The 8-bit routine, shown in Figure 2-19, discards the upper 8-bits of the first read from the port and treats the lower 8-bits masked with GPIO31 in bit position 7 and GPIO30 in bit position 6 as the least-significant byte (LSB) of the word to be fetched. The routine will then perform a second read to fetch the most-significant byte (MSB). It then combines the MSB and LSB into a single 16-bit value to be passed back to the calling routine.



**Figure 2-19. 8-Bit Parallel GetWord Function**

## 2.2.18 SPI\_Boot Function

The SPI boot ROM loader initializes the SPI module to interface to a SPI-compatible, 16-bit or 24-bit addressable serial EEPROM or flash device. It expects such a device to be present on the SPI-A pins as indicated in [Figure 2-20](#). The SPI bootloader supports an 8-bit data stream. It does not support a 16-bit data stream.



**Figure 2-20. SPI Loader**

The SPI-A loader uses following pins:

- SPISIMOA on GPIO16
- SPISOMIA on GPIO17
- SPICLK on GPIO18
- SPISTE A on GPIO19

The SPI boot ROM loader initializes the SPI with the following settings: FIFO enabled, 8-bit character, internal SPICLK master mode and talk mode, clock phase = 1, polarity = 0, using the slowest baud rate.

If the download is to be performed from an SPI port on another device, then that device must be setup to operate in the slave mode and mimic a serial SPI EEPROM. Immediately after entering the SPI\_Boot function, the pin functions for the SPI pins are set to primary and the SPI is initialized. The initialization is done at the slowest speed possible. Once the SPI is initialized and the key value read, you could specify a change in baud rate or low speed peripheral clock.

**Table 2-12. SPI 8-Bit Data Stream**

| Byte | Contents                                                                                                   |
|------|------------------------------------------------------------------------------------------------------------|
| 1    | LSB: AA (KeyValue for memory width = 8-bits)                                                               |
| 2    | MSB: 08h (KeyValue for memory width = 8-bits)                                                              |
| 3    | LSB: LOSPCP                                                                                                |
| 4    | MSB: SPIBRR                                                                                                |
| 5    | LSB: reserved for future use                                                                               |
| 6    | MSB: reserved for future use                                                                               |
| ...  | ...                                                                                                        |
| ...  | Data for this section.                                                                                     |
| ...  | ...                                                                                                        |
| 17   | LSB: reserved for future use                                                                               |
| 18   | MSB: reserved for future use                                                                               |
| 19   | LSB: Upper half (MSW) of Entry point PC[23:16]                                                             |
| 20   | MSB: Upper half (MSW) of Entry point PC[31:24] (Note: Always 0x00)                                         |
| 21   | LSB: Lower half (LSW) of Entry point PC[7:0]                                                               |
| 22   | MSB: Lower half (LSW) of Entry point PC[15:8]                                                              |
| ...  | ....                                                                                                       |
| ...  | Data for this section.                                                                                     |
| ...  | ...                                                                                                        |
| ...  | Blocks of data in the format size/destination address/data as shown in the generic data stream description |

**Table 2-12. SPI 8-Bit Data Stream (continued)**

| Byte | Contents                                   |
|------|--------------------------------------------|
| ...  | ...                                        |
| ...  | Data for this section.                     |
| ...  | ...                                        |
| n    | LSB: 00h                                   |
| n+1  | MSB: 00h - indicates the end of the source |

The data transfer is done in "burst" mode from the serial SPI EEPROM. The transfer is carried out entirely in byte mode (SPI at 8 bits/character). A step-by-step description of the sequence follows:

1. The SPI-A port is initialized
2. The GPIO19 (SPISTE) pin is used as a chip-select for the serial SPI EEPROM or flash
3. The SPI-A outputs a read command for the serial SPI EEPROM or flash
4. The SPI-A sends the serial SPI EEPROM an address 0x0000; that is, the host requires that the EEPROM or flash must have the downloadable packet starting at address 0x0000 in the EEPROM or flash. The loader is compatible with both 16-bit addresses and 24-bit addresses.
5. The next word fetched must match the key value for an 8-bit data stream (0x08AA). The least significant byte of this word is the byte read first and the most significant byte is the next byte fetched. This is true of all word transfers on the SPI. If the key value does not match, then the load is aborted and the device will branch to the flash entry point address.
6. The next 2 bytes fetched can be used to change the value of the low speed peripheral clock register (LOSPCP) and the SPI baud rate register (SPIBRR). The first byte read is the LOSPCP value and the second byte read is the SPIBRR value. The next 7 words are reserved for future enhancements. The SPI bootloader reads these 7 words and discards them.
7. The next 2 words makeup the 32-bit entry point address where execution will continue after the boot load process is complete. This is typically the entry point for the program being downloaded through the SPI port.
8. Multiple blocks of code and data are then copied into memory from the external serial SPI EEPROM through the SPI port. The blocks of code are organized in the standard data stream structure presented earlier. This is done until a block size of 0x0000 is encountered. At that point in time the entry point address is returned to the calling routine that then exits the bootloader and resumes execution at the address specified.

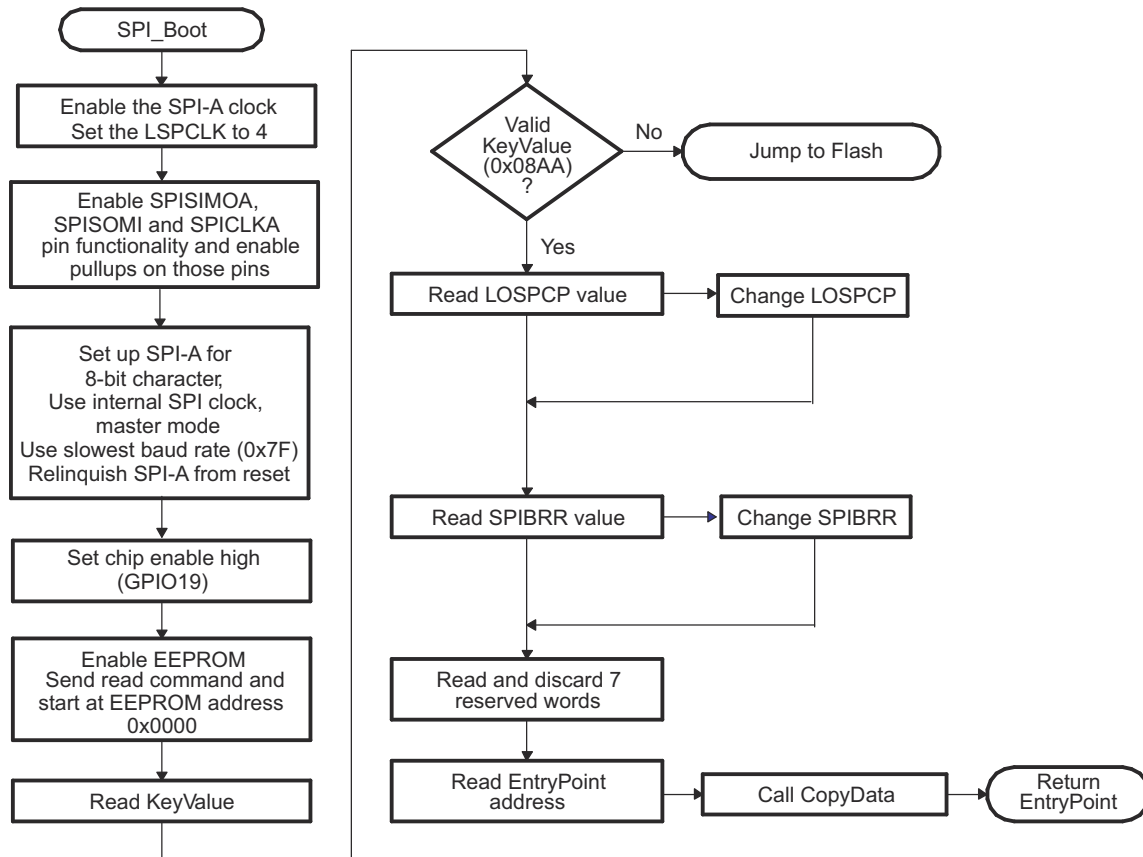


Figure 2-21. Data Transfer From EEPROM Flow

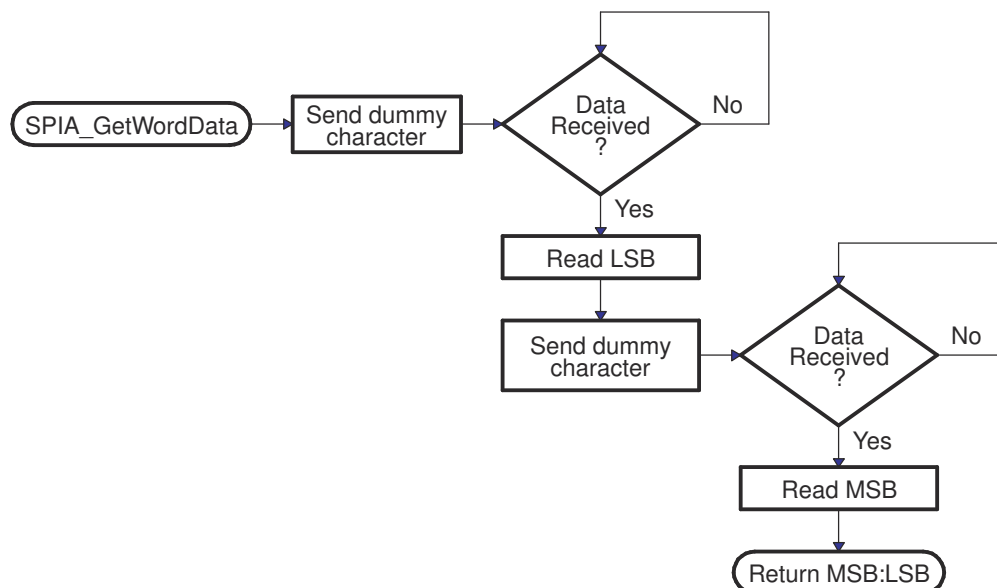
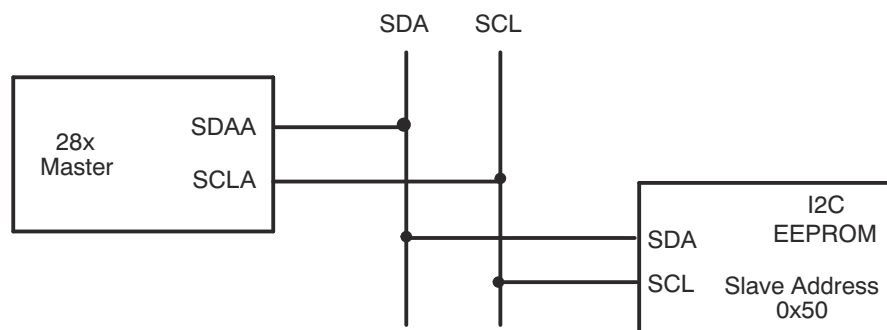


Figure 2-22. Overview of SPIA\_GetWordData Function

## 2.2.19 I2C Boot Function

The I2C bootloader expects an 8-bit wide I2C-compatible EEPROM device to be present at address 0x50 on the I2C-A bus as indicated in [Figure 2-23](#). The EEPROM must adhere to conventional I2C EEPROM protocol, as described in this section, with a 16-bit base address architecture.



**Figure 2-23. EEPROM Device at Address 0x50**

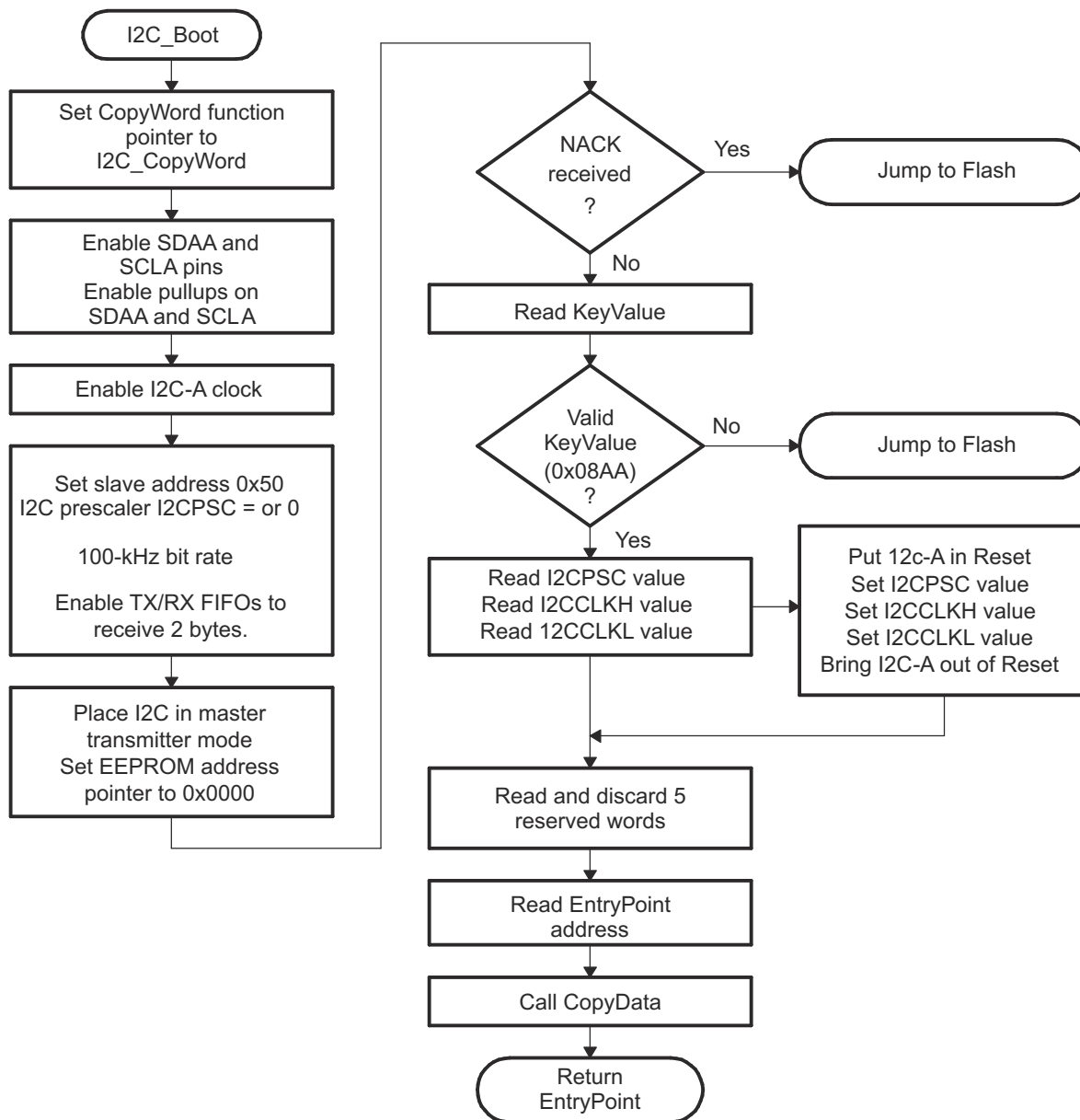
The I2C loader uses following pins:

- SDAA on GPIO 28
- SCLA on GPIO 29

If the download is to be performed from a device other than an EEPROM, then that device must be set up to operate in the slave mode and mimic the I2C EEPROM. Immediately after entering the I2C boot function, the GPIO pins are configured for I2C-A operation and the I2C is initialized. The following requirements must be met when booting from the I2C module:

- The input frequency to the device must be in the appropriate range.
- The EEPROM must be at slave address 0x50.





**Figure 2-24. Overview of I2C\_Boot Function**

The bit-period prescalers (I2CCLKH and I2CCLKL) are configured by the bootloader to run the I2C at a 50 percent duty cycle at 100-kHz bit rate (standard I2C mode) when the system clock is 10 MHz. These registers can be modified after receiving the first few bytes from the EEPROM. This allows the communication to be increased up to a 400-kHz bit rate (fast I2C mode) during the remaining data reads.

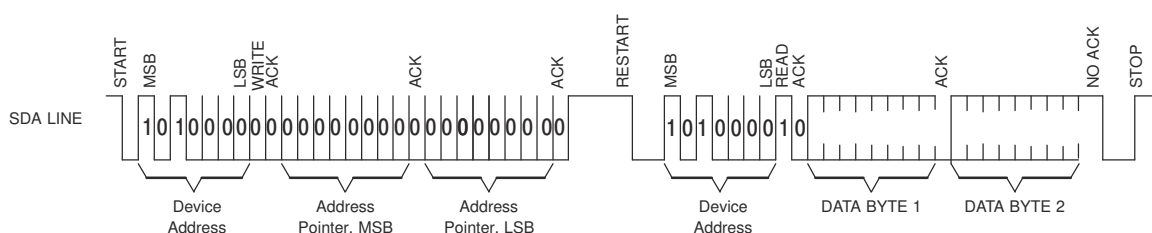
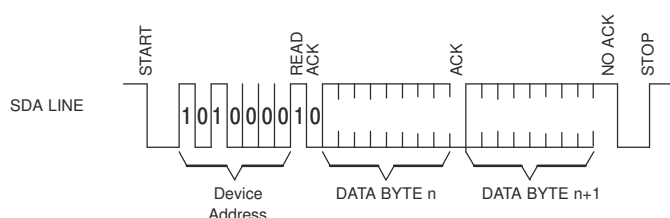
Arbitration, bus busy, and slave signals are not checked. Therefore, no other master is allowed to control the bus during this initialization phase. If the application requires another master during I2C boot mode, that master must be configured to hold off sending any I2C messages until the application software signals that it is past the bootloader portion of initialization.

The non-acknowledgment bit is checked only during the first message sent to initialize the EEPROM base address. This is to make sure that an EEPROM is present at address 0x50 before continuing. If an EEPROM is not present, code will The non-acknowledgment bit is not checked during the address phase of the data read messages (I2C\_Get Word). If a non acknowledgment is received during the data read messages, the I2C bus will hang. [Table 2-13](#) shows the 8-bit data stream used by the I2C.

**Table 2-13. I2C 8-Bit Data Stream**

| Byte     | Contents                                                                                                    |
|----------|-------------------------------------------------------------------------------------------------------------|
| 1        | LSB: AA (KeyValue for memory width = 8 bits)                                                                |
| 2        | MSB: 08h (KeyValue for memory width = 8 bits)                                                               |
| 3        | LSB: I2CPSC[7:0]                                                                                            |
| 4        | reserved                                                                                                    |
| 5        | LSB: I2CCLKH[7:0]                                                                                           |
| 6        | MSB: I2CCLKH[15:8]                                                                                          |
| 7        | LSB: I2CCLKL[7:0]                                                                                           |
| 8        | MSB: I2CCLKL[15:8]                                                                                          |
| ...      | ...                                                                                                         |
| ...      | Data for this section.                                                                                      |
| ...      | ...                                                                                                         |
| 17       | LSB: Reserved for future use                                                                                |
| 18       | MSB: Reserved for future use                                                                                |
| 19       | LSB: Upper half of entry point PC                                                                           |
| 20       | MSB: Upper half of entry point PC[22:16] (Note: Always 0x00)                                                |
| 21       | LSB: Lower half of entry point PC[15:8]                                                                     |
| 22       | MSB: Lower half of entry point PC[7:0]                                                                      |
| ...      | ...                                                                                                         |
| ...      | Data for this section.                                                                                      |
| ...      | ...                                                                                                         |
| ...      | Blocks of data in the format size/destination address/data as shown in the generic data stream description. |
| ...      | ...                                                                                                         |
| ...      | Data for this section.                                                                                      |
| ...      | ...                                                                                                         |
| LSB: 00h |                                                                                                             |
| n+1      | MSB: 00h - indicates the end of the source                                                                  |

The I2C EEPROM protocol required by the I2C bootloader is shown in [Figure 2-25](#) and [Figure 2-26](#). The first communication, which sets the EEPROM address pointer to 0x0000 and reads the KeyValue (0x08AA) from it, is shown in [Figure 2-25](#). All subsequent reads are shown in [Figure 2-26](#) and are read two bytes at a time.

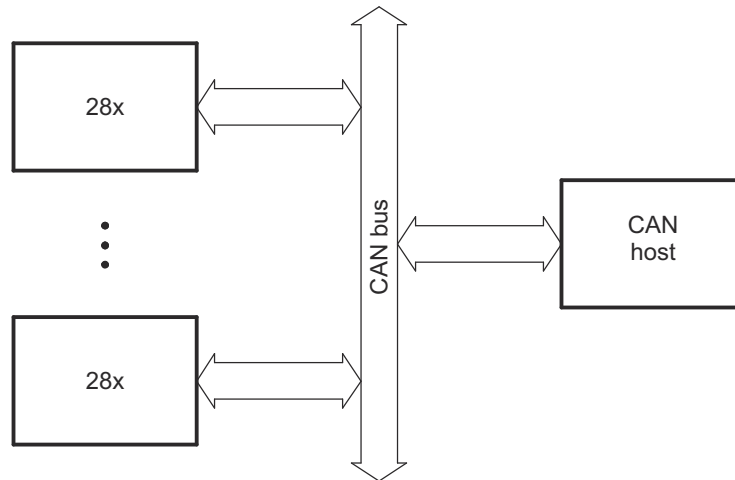

**Figure 2-25. Random Read**

**Figure 2-26. Sequential Read**

### 2.2.20 eCAN Boot Function

The eCAN bootloader asynchronously transfers code from eCAN-A to internal memory. The host can be any CAN node. The communication is first done with 11-bit standard identifiers (with a MSGID of 0x1) using two bytes per data frame. The host can download a kernel to reconfigure the eCAN if higher data throughput is desired.

The eCAN-A loader uses following pins:

- CANRXA on GPIO30
- CANTXA on GPIO31



**Figure 2-27. Overview of eCAN-A bootloader Operation**

The bit-timing registers are programmed in such a way that a valid bit-rate is achieved for a 10 MHz internal oscillator frequency as shown in [Table 2-14](#).

**Table 2-14. Bit-Rate Value for Internal Oscillators**

| OSCCLK | SYSCLKOUT | Bit Rate |
|--------|-----------|----------|
| 10 MHz | 10 MHz    | 100 kbps |

The SYSCLKOUT values shown are the reset values with the default PLL setting. The  $BRP_{reg}$  and bit-time values are hard-coded to 1 and 25, respectively.

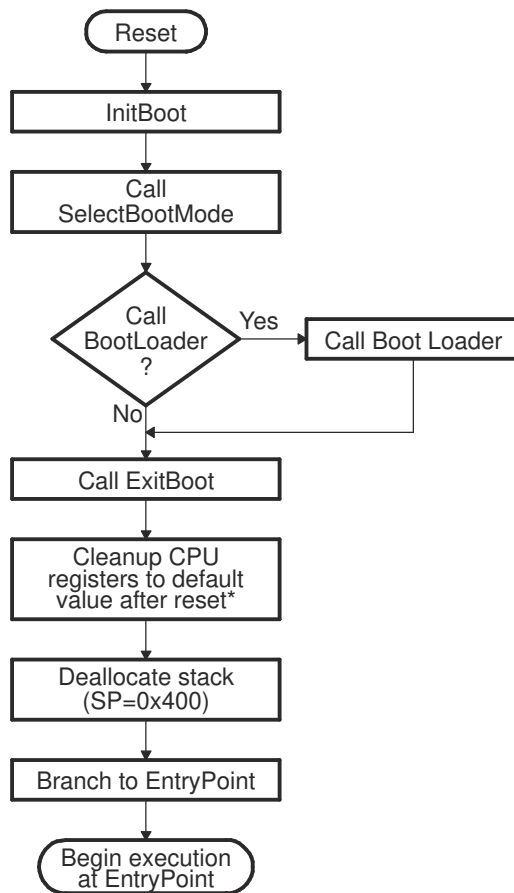
Mailbox 1 is programmed with a standard MSGID of 0x1 for boot-loader communication. The CAN host should transmit only 2 bytes at a time, LSB first and MSB next. For example, to transmit the word 0x08AA to the device, transmit AA first, followed by 08. The program flow of the CAN bootloader is identical to the SCI bootloader. The data sequence for the CAN bootloader is shown in [Table 2-15](#):

**Table 2-15. eCAN 8-Bit Data Stream**

| Bytes   | Byte 1 of 2 | Byte 2 of 2 | Description                                                                           |
|---------|-------------|-------------|---------------------------------------------------------------------------------------|
| 1 2     | AA          | 08          | 0x08AA (KeyValue for memory width = 16bits)                                           |
| 3 4     | 00          | 00          | reserved                                                                              |
| 5 6     | 00          | 00          | reserved                                                                              |
| 7 8     | 00          | 00          | reserved                                                                              |
| 9 10    | 00          | 00          | reserved                                                                              |
| 11 12   | 00          | 00          | reserved                                                                              |
| 13 14   | 00          | 00          | reserved                                                                              |
| 15 16   | 00          | 00          | reserved                                                                              |
| 17 18   | 00          | 00          | reserved                                                                              |
| 19 20   | BB          | 00          | Entry point PC[22:16]                                                                 |
| 21 22   | DD          | CC          | Entry point PC[15:0] (PC = 0xAABBCCDD)                                                |
| 23 24   | NN          | MM          | Block size of the first block of data to load = 0xMMNN words                          |
| 25 26   | BB          | AA          | Destination address of first block Addr[31:16]                                        |
| 27 28   | DD          | CC          | Destination address of first block Addr[15:0] (Addr = 0xAABBCCDD)                     |
| 29 30   | BB          | AA          | First word of the first block in the source being loaded = 0xAABB                     |
| ...     |             |             | ....                                                                                  |
| ...     |             |             | Data for this section.                                                                |
| ...     |             |             | ...                                                                                   |
| .       | BB          | AA          | Last word of the first block of the source being loaded = 0xAABB                      |
| .       | NN          | MM          | Block size of the 2nd block to load = 0xMMNN words                                    |
| .       | BB          | AA          | Destination address of second block Addr[31:16]                                       |
| .       | DD          | CC          | Destination address of second block Addr[15:0]                                        |
| .       | BB          | AA          | First word of the second block in the source being loaded                             |
| .       |             |             | ...                                                                                   |
| n n+1   | BB          | AA          | Last word of the last block of the source being loaded<br>(More sections if required) |
| n+2 n+3 | 00          | 00          | Block size of 0000h - indicates end of the source program                             |

### 2.2.21 ExitBoot Assembly Routine

The Boot ROM includes an ExitBoot routine that restores the CPU registers to their default state at reset. This is performed on all registers with one exception. The OBJMODE bit in ST1 is left set so that the device remains configured for C28x operation. This flow is detailed in the following diagram:



**Figure 2-28. ExitBoot Procedure Flow**

The following CPU registers are restored to their default values:

- ACC = 0x0000 0000
- RPC = 0x0000 0000
- P = 0x0000 0000
- XT = 0x0000 0000
- ST0 = 0x0000
- ST1 = 0x0A0B
- XAR0 = XAR7 = 0x0000 0000

After the ExitBoot routine completes and the program flow is redirected to the entry point address, the CPU registers will have the values in [Table 2-16](#).

**Table 2-16. CPU Register Restored Values**

| Register  | Value         | Register | Value          |
|-----------|---------------|----------|----------------|
| ACC       | 0x0000 0000   | P        | 0x0000 0000    |
| XT        | 0x0000 0000   | RPC      | 0x00 0000      |
| XAR0-XAR7 | 0x0000 0000   | DP       | 0x0000         |
| ST0       | 0x0000        | ST1      | 0x0A0B         |
|           | 15:10 OVC = 0 |          | 15:13 ARP = 0  |
|           | 9:7 PM = 0    |          | 12 XF = 0      |
|           | 6 V = 0       |          | 11 M0M1MAP = 1 |
|           | 5 N = 0       |          | 10 reserved    |
|           | 4 Z = 0       |          | 9 OBJMODE = 1  |
|           | 3 C = 0       |          | 8 AMODE = 0    |
|           | 2 TC = 0      |          | 7 IDLESTAT = 0 |
|           | 1 OVM = 0     |          | 6 EALLOW = 0   |
|           | 0 SXM = 0     |          | 5 LOOP = 0     |
|           |               |          | 4 SPA = 0      |
|           |               |          | 3 VMAP = 1     |
|           |               |          | 2 PAGE0 = 0    |
|           |               |          | 1 DBG0 = 1     |
|           |               |          | 0 INTM = 1     |

## 2.3 Building the Boot Table

This chapter explains how to generate the data stream and boot table required for the bootloader.

### 2.3.1 The C2000 Hex Utility

To use the features of the bootloader, you must generate a data stream and boot table as described in [Section 2.2.11](#). The hex conversion utility tool, included with the 28x code generation tools, can generate the required data stream including the required boot table. This section describes the hex2000 utility. An example of a file conversion performed by hex2000 is described in [Section 2.3.2](#).

The hex utility supports creation of the boot table required for the SCI, SPI, I2C, eCAN, and parallel I/O loaders. That is, the hex utility adds the required information to the file such as the key value, reserved bits, entry point, address, block start address, block length and terminating value. The contents of the boot table vary slightly depending on the boot mode and the options selected when running the hex conversion utility. The actual file format required by the host (ASCII, binary, hex, etc.) will differ from one specific application to another and some additional conversion may be required.

To build the boot table, follow these steps:

- 1. Assemble or compile the code.**

This creates the object files that will then be used by the linker to create a single output file.

- 2. Link the file.**

The linker combines all of the object files into a single output file in common object file format (COFF). The specified linker command file is used by the linker to allocate the code sections to different memory blocks. Each block of the boot table data corresponds to an initialized section in the COFF file. Uninitialized sections are not converted by the hex conversion utility. The following options may be useful:

The linker -m option can be used to generate a map file. This map file will show all of the sections that were created, their location in memory and their length. It can be useful to check this file to make sure that the initialized sections are where you expect them to be.

The linker -w option is also very useful. This option indicates if the linker has assigned a section to a memory region on its own. For example, if you have a section in your code called ramfuncs.

### 3. Run the hex conversion utility.

Choose the appropriate options for the desired boot mode and run the hex conversion utility to convert the COFF file produced by the linker to a boot table.

See the [TMS320C28x Assembly Language Tools User's Guide](#) and the [TMS320C28x Optimizing C/C++ Compiler User's Guide](#) for more information on the compiling and linking process.

Table 2-17 summarizes the hex conversion utility options available for the bootloader. See the [TMS320C28x Assembly Language Tools User's Guide](#) for a detailed description of the hex2000 operations used to generate a boot table. Updates will be made to support the I2C boot. See the Codegen release notes for the latest information.

**Table 2-17. Boot Loader Options**

| Option         | Description                                                                                                                                                                                                                                                                                                                                                                    |
|----------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| -boot          | Convert all sections into bootable form (use instead of a SECTIONS directive)                                                                                                                                                                                                                                                                                                  |
| -sci8          | Specify the source of the bootloader table as the SCI-A port, 8-bit mode                                                                                                                                                                                                                                                                                                       |
| -spi8          | Specify the source of the bootloader table as the SPI-A port, 8-bit mode                                                                                                                                                                                                                                                                                                       |
| -gpio8         | Specify the source of the bootloader table as the GPIO port, 8-bit mode                                                                                                                                                                                                                                                                                                        |
| -gpio16        | Specify the source of the bootloader table as the GPIO port, 16-bit mode                                                                                                                                                                                                                                                                                                       |
| -bootorg value | Specify the source address of the bootloader table                                                                                                                                                                                                                                                                                                                             |
| -lospcp value  | Specify the initial value for the LOSPCP register. This value is used only for the spi8 boot table format and ignored for all other formats. If the value is greater than 0x7F, the value is truncated to 0x7F.                                                                                                                                                                |
| -spibrr value  | Specify the initial value for the SPIBRR register. This value is used only for the spi8 boot table format and ignored for all other formats. If the value is greater than 0x7F, the value is truncated to 0x7F.                                                                                                                                                                |
| -e value       | Specify the entry point at which to begin execution after boot loading. The value can be an address or a global symbol. This value is optional. The entry point can be defined at compile time using the linker -e option to assign the entry point to a global symbol. The entry point for a C program is normally _c_int00 unless defined otherwise by the -e linker option. |
| -i2c8          | Specify the source of the bootloader table as the I2C-A port, 8-bit                                                                                                                                                                                                                                                                                                            |
| -i2cpsc value  | Specify the value for the I2CPSC register. This value will be loaded and take effect after all I2C options are loaded, prior to reading data from the EEPROM. This value will be truncated to the least significant eight bits and should be set to maintain an I2C module clock of 7-12 MHz.                                                                                  |
| -i2cclkh value | Specify the value for the I2CCLKH register. This value will be loaded and take effect after all I2C options are loaded, prior to reading data from the EEPROM.                                                                                                                                                                                                                 |
| -i2cckl value  | Specify the value for the I2CCLKL register. This value will be loaded and take effect after all I2C options are loaded, prior to reading data from the EEPROM.                                                                                                                                                                                                                 |

### 2.3.2 Example: Preparing a COFF File For eCAN Bootloading

This section shows how to convert a COFF file into a format suitable for CAN based bootloading. This example assumes that the host sending the data stream is capable of reading an ASCII hex format file. An example COFF file named GPIO34TOG.out has been used for the conversion.

Build the project and link using the -m linker option to generate a map file. Examine the .map file produced by the linker. The information shown in [Example 2-8](#) has been copied from the example map file (GPIO34TOG.map). This shows the section allocation map for the code. The map file includes the following information:

- **Output Section**

This is the name of the output section specified with the SECTIONS directive in the linker command file.

- **Origin**

The first origin listed for each output section is the starting address of that entire output section. The following origin values are the starting address of that portion of the output section.

- **Length**

The first length listed for each output section is the length for that entire output section. The following length values are the lengths associated with that portion of the output section.

- **Attributes/input sections**

This lists the input files that are part of the section or any value associated with an output section.

See the [TMS320C28x Assembly Language Tools User's Guide](#) for detailed information on generating a linker command file and a memory map.

All sections shown in [Example 2-8](#) that are initialized need to be loaded into the DSP in order for the code to execute properly. In this case, the codestart, ramfuncs, .cinit, myreset, and .text sections need to be loaded. The other sections are uninitialized and will not be included in the loading process. The map file also indicates the size of each section and the starting address. For example, the .text section has 0x155 words and starts at 0x3FA000.

#### Example 2-8. GPIO34TOG Map File

| output<br>section | page | origin   | length   | attributes/<br>input sections           |
|-------------------|------|----------|----------|-----------------------------------------|
| -----             | ---  | -----    | -----    | -----                                   |
| codestart         | 0    | 00000000 | 00000002 |                                         |
|                   |      | 00000000 | 00000002 | DSP280x_CodeStartBranch.obj (codestart) |
| .pinit            | 0    | 00000002 | 00000000 |                                         |
| .switch           | 0    | 00000002 | 00000000 | UNINITIALIZED                           |
| ramfuncs          | 0    | 00000002 | 00000016 |                                         |
|                   |      | 00000002 | 00000016 | DSP280x_SysCtrl.obj (ramfuncs)          |
| .cinit            | 0    | 00000018 | 00000019 |                                         |
|                   |      | 00000018 | 0000000e | rts2800_ml.lib : exit.obj (.cinit)      |
|                   |      | 00000026 | 0000000a | : _lock.obj (.cinit)                    |
|                   |      | 00000030 | 00000001 | --HOLE-- [fill = 0]                     |
| myreset           | 0    | 00000032 | 00000002 |                                         |
|                   |      | 00000032 | 00000002 | DSP280x_CodeStartBranch.obj (myreset)   |
| IQmath            | 0    | 003fa000 | 00000000 | UNINITIALIZED                           |
| .text             | 0    | 003fa000 | 00000155 |                                         |
|                   |      | 003fa000 | 00000046 | rts2800_ml.lib : boot.obj (.text)       |

To load the code using the CAN bootloader, the host must send the data in the format that the bootloader understands. That is, the data must be sent as blocks of data with a size, starting address followed by the data. A block size of 0 indicates the end of the data. The HEX2000.exe utility can be used to convert the COFF file into a format that includes this boot information. The following command syntax has been used to convert the application into an ASCII hex format file that includes all of the required information for the bootloader.



### Example 2-9. HEX2000.exe Command Syntax

```
C: HEX2000 GPIO34TOG.OUT -boot -gpio8 -a
Where:
- boot   Convert all sections into bootable form.
- gpio8  Use the GPIO in 8-bit mode data format. The eCAN
         uses the same data format as the GPIO in 8-bit mode.
- a      Select ASCII-Hex as the output format.
```

The command line shown in [Example 2-9](#) will generate an ASCII-Hex output file called GPIO34TOG.a00, whose contents are explained in [Example 2-10](#). This example assumes that the host will be able to read an ASCII hex format file. The format may differ for your application. Each section of data loaded can be tied back to the map file described in [Example 2-8](#). After the data stream is loaded, the boot ROM will jump to the Entrypoint address that was read as part of the data stream. In this case, execution will begin at 0x3FA000.

### Example 2-10. GPIO34TOG Data Stream

```
AA 08                                ;Keyvalue
00 00 00 00 00 00 00 00            ;8 reserved words
00 00 00 00 00 00 00 00
3F 00 00 A0                          ;Entrypoint 0x003FA000
02 00                                ;Load 2 words - codestart section
00 00 00 00                        ;Load block starting at 0x000000
7F 00 9A A0                          ;Data block 0x007F, 0xA09A
16 00                                ;Load 0x0016 words - ramfuncs section
00 00 02 00                        ;Load block starting at 0x000002
22 76 1F 76 2A 00 00 1A 01 00 06 CC F0 ;Data = 0x7522, 0x761F etc...
FF 05 50 06 96 06 CC FF F0 A9 1A 00 05
06 96 04 1A FF 00 05 1A FF 00 1A 76 07
F6 00 77 06 00
55 01                                ;Load 0x0155 words - .text section
3F 00 00 A0                          ;Load block starting at 0x003FA000
AD 28 00 04 69 FF 1F 56 16 56 1A 56 40 ;Data = 0x28AD, 0x4000 etc...
29 1F 76 00 00 02 29 1B 76 22 76 A9 28
18 00 A8 28 00 00 01 09 1D 61 C0 76 18
00 04 29 0F 6F 00 9B A9 24 01 DF 04 6C
04 29 A8 24 01 DF A6 1E A1 F7 86 24 A7
06 .. ..
.. .. ..
.. .. ..
FC 63 E6 6F
19 00 ;Load 0x0019 words - .cinit section
00 00 18 00                        ;Load block starting at 0x000018
FF FF 00 B0 3F 00 00 00 FE FF 02 B0 3F ;Data = 0xFFFF, 0xB000 etc...
00 00 00 00 00 FE FF 04 B0 3F 00 00 00
00 00 FE FF .. .. ..
.. .. ..
3F 00 00 00
02 00                                ;Load 0x0002 words - myreset section
00 00 32 00                        ;Load block starting at 0x000032
00 00 00 00                        ;Data = 0x0000, 0x0000
00 00                                ;Block size of 0 - end of data
```

## 2.4 Bootloader Code Overview

This chapter contains information on the Boot ROM version, checksum, and code.

### 2.4.1 Boot ROM Version and Checksum Information

The boot ROM contains its own version number located at address 0x3F FFBA. This version number starts at 1 and will be incremented any time the boot ROM code is modified. The next address, 0x3F FFBB contains the month and year (MM/YY in decimal) that the boot code was released. The next four memory locations contain a checksum value for the boot ROM. Taking a 64-bit summation of all addresses within the ROM, except for the checksum locations, generates this checksum.

**Table 2-18. Bootloader Revision and Checksum Information**

| Address   | Contents                           |
|-----------|------------------------------------|
| 0x3F FFB9 |                                    |
| 0x3F FFBA | Boot ROM Version Number            |
| 0x3F FFBB | MM/YY of release (in decimal)      |
| 0x3F FFBC | Least-significant word of checksum |
| 0x3F FFBD | ...                                |
| 0x3F FFBE | ...                                |
| 0x3F FFBF | Most-significant word of checksum  |



## Enhanced Pulse Width Modulator (ePWM) Module

The enhanced pulse width modulator (ePWM) module is a key element in controlling many of the power electronic systems found in both commercial and industrial equipments. These systems include digital motor control, switch mode power supply control, uninterruptible power supplies (UPS), and other forms of power conversion. The ePWM module performs a digital-to-analog (DAC) function, where the duty cycle is equivalent to a DAC analog value; it is sometimes referred to as a Power DAC.

This chapter is applicable for ePWM type 1. See the [C2000 Real-Time Control Peripheral Reference Guide](#) for a list of all devices with an ePWM module of the same type, to determine the differences between the types, and for a list of device-specific differences within a type.

This chapter includes an overview of the module and information about each of the submodules:

- Time-Base Module
- Counter-Compare Module
- Action-Qualifier Module
- Dead-Band Generator Module
- PWM-Chopper (PC) Module
- Trip-Zone Module
- Event-Trigger Module

ePWM Type 1 is fully compatible to the Type 0 module. Type 1 has the following enhancements in addition to the Type 0 module:

- **Increased Dead-Band Resolution:** The dead-band clocking has been enhanced to allow half-cycle clocking to double resolution.
- **Enhanced Interrupt and SOC Generation:** Interrupts and ADC start-of-conversion can now be generated on both the TBCTR == zero and TBCTR == period events. This feature enables dual edge PWM control. Additionally, the ADC start-of-conversion can be generated from an event defined in the digital compare sub-module.
- **High-Resolution Period Capability:** Provides the ability to enable high-resolution period. This is discussed in more detail in [Chapter 4](#).
- **Digital Compare Sub-module:** The digital compare sub-module enhances the event triggering and trip zone sub-modules by providing filtering, blanking and improved trip functionality to digital compare signals. Such features are essential for peak current mode control and for support of analog comparators.

|                                           |     |
|-------------------------------------------|-----|
| 3.1 Introduction.....                     | 242 |
| 3.2 ePWM Submodules.....                  | 249 |
| 3.3 Applications to Power Topologies..... | 302 |
| 3.4 Registers.....                        | 328 |

## 3.1 Introduction

An effective PWM peripheral must be able to generate complex pulse width waveforms with minimal CPU overhead or intervention. It needs to be highly programmable and very flexible while being easy to understand and use. The ePWM unit described here addresses these requirements by allocating all needed timing and control resources on a per PWM channel basis. Cross coupling or sharing of resources has been avoided; instead, the ePWM is built up from smaller single channel modules with separate resources that can operate together as required to form a system. This modular approach results in an orthogonal architecture and provides a more transparent view of the peripheral structure, helping users to understand its operation quickly.

In this document the letter x within a signal or module name is used to indicate a generic ePWM instance on a device. For example output signals EPWMxA and EPWMxB refer to the output signals from the ePWMx instance. Thus, EPWM1A and EPWM1B belong to ePWM1 and likewise EPWM4A and EPWM4B belong to ePWM4.

### 3.1.1 EPWM Related Collateral

#### Foundational Materials

- [Real-Time Control Reference Guide](#)
  - Refer to the EPWM section

#### Getting Started Materials

- [Flexible PWMs Enable Multi-Axis Drives, Multi-Level Inverters Application Report](#)
- [Getting Started with the C2000 ePWM Module](#) (Video)
- [Using PWM Output as a Digital-to-Analog Converter on a TMS320F280x Digital Signal Control Application Report](#)
  - Chapters 1 to 6 are Fundamental material, derivations, and explanations that are useful for learning about how PWM can be used to implement a DAC. Subsequent chapters are Getting Started and Expert material for implementing in a system.
- [Using the Enhanced Pulse Width Modulator \(ePWM\) Module Application Report](#)

#### Expert Materials

- [C2000 real-time microcontrollers - Reference designs](#)
  - See TI designs related to specific end applications used.

### 3.1.2 Submodule Overview

The ePWM module represents one complete PWM channel composed of two PWM outputs: EPWMxA and EPWMxB. Multiple ePWM modules are instanced within a device as shown in [Figure 3-1](#). Each ePWM instance is identical with one exception. Some instances include a hardware extension that allows more precise control of the PWM outputs. This extension is the high-resolution pulse width modulator (HRPWM) and is described in [Chapter 4](#). See your device-specific data sheet to determine which ePWM instances include this feature. Each ePWM module is indicated by a numerical value starting with 1. For example, ePWM1 is the first instance and ePWM3 is the third instance in the system and ePWMx indicates any instance.

The ePWM modules are chained together via a clock synchronization scheme that allows them to operate as a single system when required. Additionally, this synchronization scheme can be extended to the capture peripheral modules (eCAP). The number of modules is device-dependent and based on target application needs. Modules can also operate stand-alone.

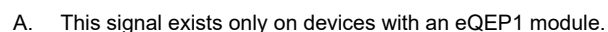
Each ePWM module supports the following features:

- Dedicated 16-bit time-base counter with period and frequency control.
- Two PWM outputs (EPWMxA and EPWMxB) that can be used in the following configurations:
  - Two independent PWM outputs with single-edge operation
  - Two independent PWM outputs with dual-edge symmetric operation
  - One independent PWM output with dual-edge asymmetric operation

- Asynchronous override control of PWM signals through software.
- Programmable phase-control support for lag or lead operation relative to other ePWM modules.
- Hardware-locked (synchronized) phase relationship on a cycle-by-cycle basis.
- Dead-band generation with independent rising and falling edge delay control.
- Programmable trip zone allocation of both cycle-by-cycle trip and one-shot trip on fault conditions.
- A trip condition can force either high, low, or high-impedance state logic levels at PWM outputs.
- Comparator module outputs and trip zone inputs can generate events, filtered events, or trip conditions.
- All events can trigger both CPU interrupts and ADC start of conversion (SOC).
- Programmable event prescaling minimizes CPU overhead on interrupts.
- PWM chopping by high-frequency carrier signal, useful for pulse transformer gate drives.

Each ePWM module is connected to the input/output signals shown in [Figure 3-1](#). The signals are described in detail in subsequent sections.

The order in which the ePWM modules are connected may differ from what is shown in [Figure 3-1](#). See [Section 3.2.2.3.3](#) for the synchronization scheme for a particular device. Each ePWM module consists of eight submodules and is connected within a system with the signals shown in [Figure 3-2](#).



### Figure 3-1. Multiple ePWM Modules

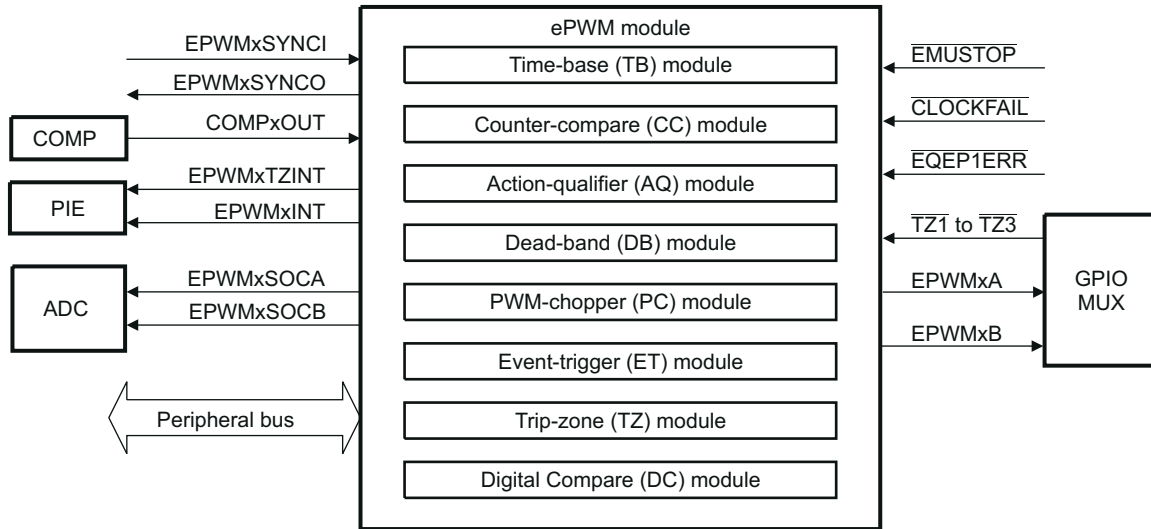
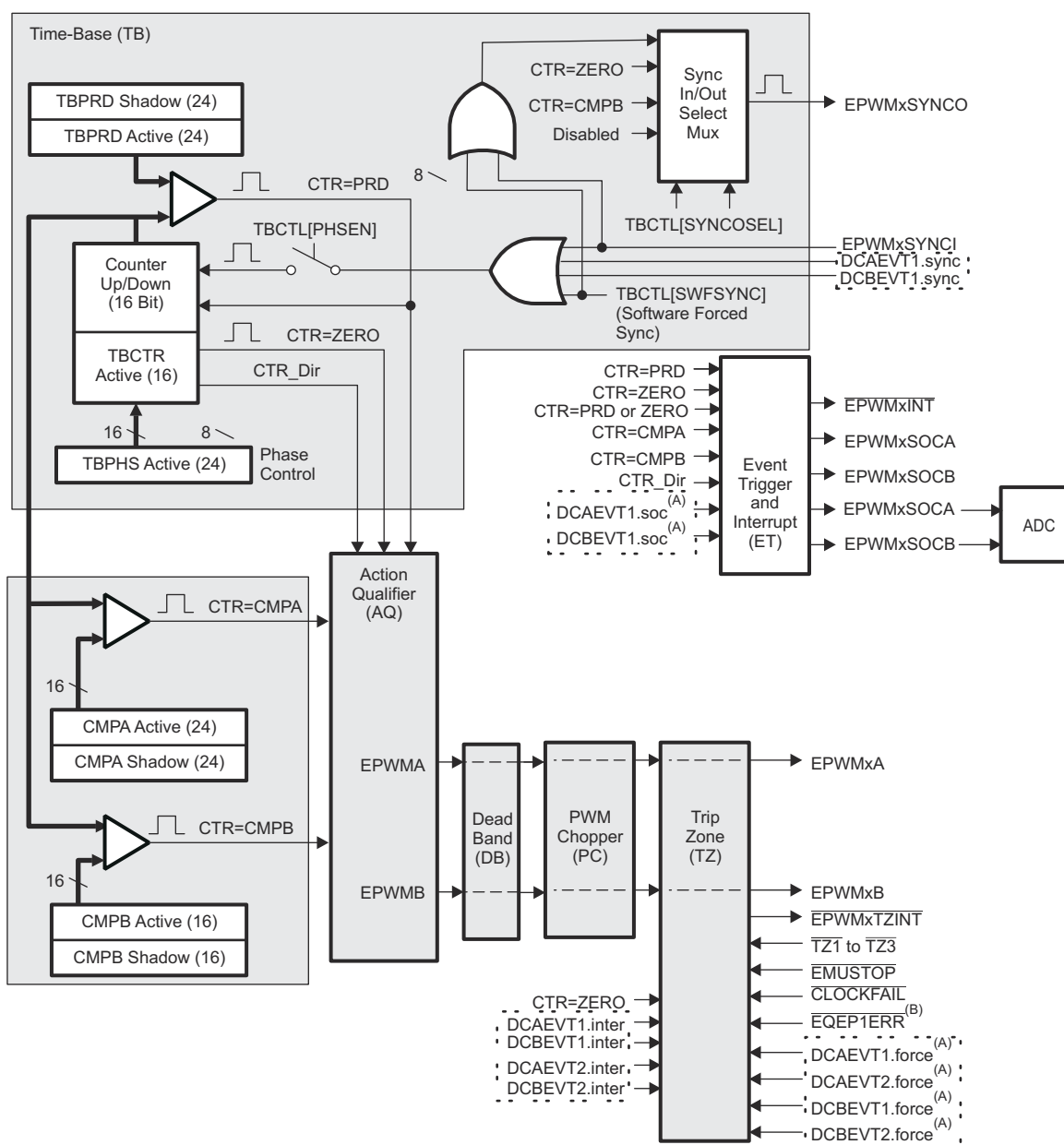


Figure 3-2. Submodules and Signal Connections for an ePWM Module

Figure 3-3 shows more internal details of a single ePWM module. The main signals used by the ePWM module are:

- **PWM output signals (EPWMxA and EPWMxB):** The PWM output signals are made available external to the device through the GPIO peripheral described in the system control and interrupts guide for your device.
- **Trip-zone signals (TZ1 to TZ6):** These input signals alert the ePWM module of fault conditions external to the ePWM module. Each module on a device can be configured to either use or ignore any of the trip-zone signals. The TZ1 to TZ3 trip-zone signals can be configured as asynchronous inputs through the GPIO peripheral. TZ4 is connected to an inverted EQEP1 error signal (EQEP1ERR) from the EQEP1 module (for those devices with an EQEP1 module). TZ5 is connected to the system clock fail logic, and TZ6 is connected to the EMUSTOP output from the CPU. This allows you to configure a trip action when the clock fails or the CPU halts.
- **Time-base synchronization input (EPWMxSYNCl) and output (EPWMxSYNCO) signals:** The synchronization signals daisy chain the ePWM modules together. Each module can be configured to either use or ignore its synchronization input. The clock synchronization input and output signal are brought out to pins only for ePWM1 (ePWM module #1). The synchronization output for ePWM1 (EPWM1SYNCO) is also connected to the SYNCl of the first enhanced capture module (eCAP1).
- **ADC start-of-conversion signals (EPWMxSOCA and EPWMxSOCB):** Each ePWM module has two ADC start of conversion signals. Any ePWM module can trigger a start of conversion. Whichever event triggers the start of conversion is configured in the Event-Trigger submodule of the ePWM.
- **Comparator output signals (COMPxOUT):** Output signals from the comparator module in conjunction with the trip zone signals can generate digital compare events.
- **Peripheral Bus:** The peripheral bus is 32-bits wide and allows both 16-bit and 32-bit writes to the ePWM register file.

Figure 3-3 also shows the key internal submodule interconnect signals. Each submodule is described in detail in its respective section.



- A. These events are generated by the type 1 ePWM digital compare (DC) submodule based on the levels of the COMPxOUT and TZ signals.
- B. This signal exists only on devices with in eQEP1 module.

**Figure 3-3. ePWM Submodules and Critical Internal Signal Interconnects**



### 3.1.3 Register Mapping

The complete ePWM module control and status register set is grouped by submodule as shown in [Table 3-1](#). Each register set is duplicated for each instance of the ePWM module. The start address for each ePWM register file instance on a device is specified in the appropriate data manual.

**Table 3-1. ePWM Module Control and Status Register Set Grouped by Submodule**

| Name                                                                     | Offset <sup>(1)</sup> | Size<br>(x16) | Shadow | EALLOW | Description                                                   |
|--------------------------------------------------------------------------|-----------------------|---------------|--------|--------|---------------------------------------------------------------|
| <b>Time-Base Submodule Registers</b>                                     |                       |               |        |        |                                                               |
| TBCTL                                                                    | 0x0000                | 1             |        |        | Time-Base Control Register                                    |
| TBSTS                                                                    | 0x0001                | 1             |        |        | Time-Base Status Register                                     |
| TBPHSHR                                                                  | 0x0002                | 1             |        |        | Extension for HRPWM Phase Register <sup>(2)</sup>             |
| TBPHS                                                                    | 0x0003                | 1             |        |        | Time-Base Phase Register                                      |
| TBCTR                                                                    | 0x0004                | 1             |        |        | Time-Base Counter Register                                    |
| TBPRD                                                                    | 0x0005                | 1             | Yes    |        | Time-Base Period Register                                     |
| TBPRDHR                                                                  | 0x0006                | 1             | Yes    |        | Time Base Period High Resolution Register <sup>(3)</sup>      |
| <b>Counter-Compare Submodule Registers</b>                               |                       |               |        |        |                                                               |
| CMPCTL                                                                   | 0x0007                | 1             |        |        | Counter-Compare Control Register                              |
| CMPAHR                                                                   | 0x0008                | 1             | Yes    |        | Extension for HRPWM Counter-Compare A Register <sup>(2)</sup> |
| CMPA                                                                     | 0x0009                | 1             | Yes    |        | Counter-Compare A Register                                    |
| CMPB                                                                     | 0x000A                | 1             | Yes    |        | Counter-Compare B Register                                    |
| <b>Action-Qualifier Submodule Registers</b>                              |                       |               |        |        |                                                               |
| AQCTLA                                                                   | 0x000B                | 1             |        |        | Action-Qualifier Control Register for Output A (EPWMxA)       |
| AQCTLB                                                                   | 0x000C                | 1             |        |        | Action-Qualifier Control Register for Output B (EPWMxB)       |
| AQSFRC                                                                   | 0x000D                | 1             |        |        | Action-Qualifier Software Force Register                      |
| AQCSFRC                                                                  | 0x000E                | 1             | Yes    |        | Action-Qualifier Continuous S/W Force Register Set            |
| <b>Dead-Band Generator Submodule Registers</b>                           |                       |               |        |        |                                                               |
| DBCTL                                                                    | 0x000F                | 1             |        |        | Dead-Band Generator Control Register                          |
| DBRED                                                                    | 0x0010                | 1             |        |        | Dead-Band Generator Rising Edge Delay Count Register          |
| DBFED                                                                    | 0x0011                | 1             |        |        | Dead-Band Generator Falling Edge Delay Count Register         |
| <b>Trip-Zone Submodule Registers</b>                                     |                       |               |        |        |                                                               |
| TZSEL                                                                    | 0x0012                | 1             |        | Yes    | Trip-Zone Select Register                                     |
| TZDCSEL                                                                  | 0x0013                | 1             |        | Yes    | Trip Zone Digital Compare Select Register                     |
| TZCTL                                                                    | 0x0014                | 1             |        | Yes    | Trip-Zone Control Register <sup>(3)</sup>                     |
| TZEINT                                                                   | 0x0015                | 1             |        | Yes    | Trip-Zone Enable Interrupt Register <sup>(3)</sup>            |
| TZFLG                                                                    | 0x0016                | 1             |        |        | Trip-Zone Flag Register <sup>(3)</sup>                        |
| TZCLR                                                                    | 0x0017                | 1             |        | Yes    | Trip-Zone Clear Register <sup>(3)</sup>                       |
| TZFRC                                                                    | 0x0018                | 1             |        | Yes    | Trip-Zone Force Register <sup>(3)</sup>                       |
| <b>Event-Trigger Submodule Registers</b>                                 |                       |               |        |        |                                                               |
| ETSEL                                                                    | 0x0019                | 1             |        |        | Event-Trigger Selection Register                              |
| ETPS                                                                     | 0x001A                | 1             |        |        | Event-Trigger Pre-Scale Register                              |
| ETFLG                                                                    | 0x001B                | 1             |        |        | Event-Trigger Flag Register                                   |
| ETCLR                                                                    | 0x001C                | 1             |        |        | Event-Trigger Clear Register                                  |
| ETFRC                                                                    | 0x001D                | 1             |        |        | Event-Trigger Force Register                                  |
| <b>PWM-Chopper Submodule Registers</b>                                   |                       |               |        |        |                                                               |
| PCCTL                                                                    | 0x001E                | 1             |        |        | PWM-Chopper Control Register                                  |
| <b>High-Resolution Pulse Width Modulator (HRPWM) Extension Registers</b> |                       |               |        |        |                                                               |
| HRCNFG                                                                   | 0x0020                | 1             |        | Yes    | HRPWM Configuration Register <sup>(2)</sup> <sup>(3)</sup>    |
| HRPWR                                                                    | 0x0021                | 1             |        | Yes    | HRPWM Power Register <sup>(3)</sup> <sup>(4)</sup>            |

**Table 3-1. ePWM Module Control and Status Register Set Grouped by Submodule (continued)**

| Name                                   | Offset <sup>(1)</sup> | Size (x16) | Shadow | EALLOW | Description                                                     |
|----------------------------------------|-----------------------|------------|--------|--------|-----------------------------------------------------------------|
| HRMSTEP                                | 0x0026                | 1          |        | Yes    | HRPWM MEP Step Register <sup>(3) (4)</sup>                      |
| HRPCTL                                 | 0x0028                | 1          |        | Yes    | High Resolution Period Control Register <sup>(3)</sup>          |
| TBPRDHRM                               | 0x002A                | 1          | Writes |        | Time Base Period High Resolution Register Mirror <sup>(3)</sup> |
| TBPRDM                                 | 0x002B                | 1          | Writes |        | Time Base Period Register Mirror                                |
| CMPAHRM                                | 0x002C                | 1          | Writes |        | Compare A High Resolution Register Mirror <sup>(3)</sup>        |
| CMPAM                                  | 0x002D                | 1          | Writes |        | Compare A Register Mirror                                       |
| <b>Digital Compare Event Registers</b> |                       |            |        |        |                                                                 |
| DCTRIPSEL                              | 0x0030                | 1          |        | Yes    | Digital Compare Trip Select Register                            |
| DCACTL                                 | 0x0031                | 1          |        | Yes    | Digital Compare A Control Register                              |
| DCBCTL                                 | 0x0032                | 1          |        | Yes    | Digital Compare B Control Register                              |
| DCFCTL                                 | 0x0033                | 1          |        | Yes    | Digital Compare Filter Control Register                         |
| DCCAPCTL                               | 0x0034                | 1          |        | Yes    | Digital Compare Capture Control Register                        |
| DCFOFFSET                              | 0x0035                | 1          | Writes |        | Digital Compare Filter Offset Register                          |
| DCFOFFSETCNT                           | 0x0036                | 1          |        |        | Digital Compare Filter Offset Counter Register                  |
| DCFWINDOW                              | 0x0037                | 1          |        |        | Digital Compare Filter Window Register                          |
| DCFWINDOWCNT                           | 0x0038                | 1          |        |        | Digital Compare Filter Window Counter Register                  |
| DCCAP                                  | 0x0039                | 1          | Yes    |        | Digital Compare Counter Capture Register                        |

(1) Locations not shown are reserved.

(2) These registers are only available on ePWM instances that include the high-resolution PWM extension. Otherwise these locations are reserved. These registers are also described in [Chapter 4](#). See your device-specific data sheet to determine which instances include the HRPWM.

(3) EALLOW protected registers as described in the *System Control and Interrupts* chapter.

(4) These registers only exist in the ePWM1 register space. They cannot be accessed from any other ePWM module's register space.

The CMPA , CMPAHR, TBPRD, and TBPRDHR registers are mirrored in the register map (Mirror registers include an "-M" suffix - CMPAM, CMPAHRM, TBPRDM, and TBPRDHRM). Note in the tables below, that in both Immediate mode and Shadow mode, reads from these mirror registers result in the active value of the register or a TI internal test value.

In Immediate Mode:

| Register | Offset | Write  | Read   | Register | Offset | Write  | Read        |
|----------|--------|--------|--------|----------|--------|--------|-------------|
| TBPRDHR  | 0x06   | Active | Active | TBPRDHRM | 0x2A   | Active | TI_Internal |
| TBPRD    | 0x05   | Active | Active | TBPRDM   | 0x2B   | Active | Active      |
| CMPAHR   | 0x08   | Active | Active | CMPAHRM  | 0x2C   | Active | TI_Internal |
| CMPA     | 0x09   | Active | Active | CMPAM    | 0x2D   | Active | Active      |

In Shadow Mode:

| Register | Offset | Write  | Read   | Register | Offset | Write  | Read        |
|----------|--------|--------|--------|----------|--------|--------|-------------|
| TBPRDHR  | 0x06   | Shadow | Shadow | TBPRDHRM | 0x2A   | Shadow | TI_Internal |
| TBPRD    | 0x05   | Shadow | Shadow | TBPRDM   | 0x2B   | Shadow | Active      |
| CMPAHR   | 0x08   | Shadow | Shadow | CMPAHRM  | 0x2C   | Shadow | TI_Internal |
| CMPA     | 0x09   | Shadow | Shadow | CMPAM    | 0x2D   | Shadow | Active      |

## 3.2 ePWM Submodules

Eight submodules are included in every ePWM peripheral. Each of these submodules performs specific tasks that can be configured by software.

### 3.2.1 Overview

[Table 3-2](#) lists the eight key submodules together with a list of their main configuration parameters. For example, if you need to adjust or control the duty cycle of a PWM waveform, then you should see the counter-compare submodule in [Section 3.2.3](#) for relevant details.

**Table 3-2. Submodule Configuration Parameters**

| Submodule             | Configuration Parameter or Option                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|-----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Time-base (TB)        | <ul style="list-style-type: none"> <li>Scale the time-base clock (TBCLK) relative to the system clock (SYSCLKOUT).</li> <li>Configure the PWM time-base counter (TBCTR) frequency or period.</li> <li>Set the mode for the time-base counter: <ul style="list-style-type: none"> <li>count-up mode: used for asymmetric PWM</li> <li>count-down mode: used for asymmetric PWM</li> <li>count-up-and-down mode: used for symmetric PWM</li> </ul> </li> <li>Configure the time-base phase relative to another ePWM module.</li> <li>Synchronize the time-base counter between modules through hardware or software.</li> <li>Configure the direction (up or down) of the time-base counter after a synchronization event.</li> <li>Configure how the time-base counter will behave when the device is halted by a debug probe.</li> <li>Specify the source for the synchronization output of the ePWM module: <ul style="list-style-type: none"> <li>Synchronization input signal</li> <li>Time-base counter equal to zero</li> <li>Time-base counter equal to counter-compare B (CMPB)</li> <li>No output synchronization signal generated.</li> </ul> </li> </ul> |
| Counter-compare (CC)  | <ul style="list-style-type: none"> <li>Specify the PWM duty cycle for output EPWMxA and/or output EPWMxB</li> <li>Specify the time at which switching events occur on the EPWMxA or EPWMxB output</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| Action-qualifier (AQ) | <ul style="list-style-type: none"> <li>Specify the type of action taken when a time-base or counter-compare submodule event occurs: <ul style="list-style-type: none"> <li>No action taken</li> <li>Output EPWMxA and/or EPWMxB switched high</li> <li>Output EPWMxA and/or EPWMxB switched low</li> <li>Output EPWMxA and/or EPWMxB toggled</li> </ul> </li> <li>Force the PWM output state through software control</li> <li>Configure and control the PWM dead-band through software</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| Dead-band (DB)        | <ul style="list-style-type: none"> <li>Control of traditional complementary dead-band relationship between upper and lower switches</li> <li>Specify the output rising-edge-delay value</li> <li>Specify the output falling-edge delay value</li> <li>Bypass the dead-band module entirely. In this case the PWM waveform is passed through without modification.</li> <li>Option to enable half-cycle clocking for double resolution.</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| PWM-chopper (PC)      | <ul style="list-style-type: none"> <li>Create a chopping (carrier) frequency.</li> <li>Pulse width of the first pulse in the chopped pulse train.</li> <li>Duty cycle of the second and subsequent pulses.</li> <li>Bypass the PWM-chopper module entirely. In this case the PWM waveform is passed through without modification.</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |

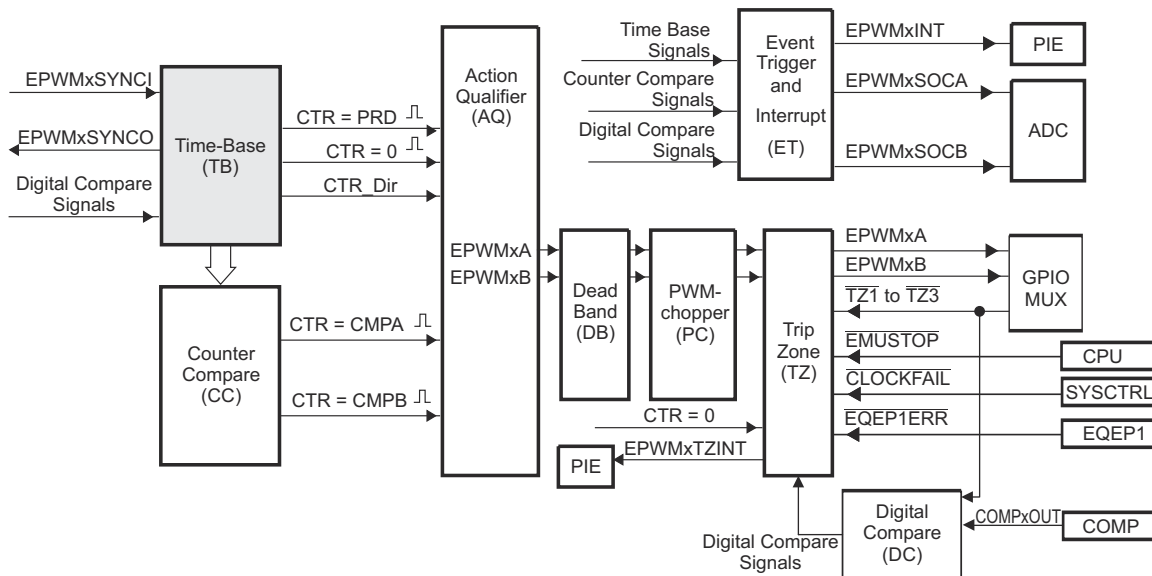
**Table 3-2. Submodule Configuration Parameters (continued)**

| Submodule            | Configuration Parameter or Option                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Trip-zone (TZ)       | <ul style="list-style-type: none"> <li>• Configure the ePWM module to react to one, all, or none of the trip-zone signals or digital compare events.</li> <li>• Specify the tripping action taken when a fault occurs: <ul style="list-style-type: none"> <li>– Force EPWMxA and/or EPWMxB high</li> <li>– Force EPWMxA and/or EPWMxB low</li> <li>– Force EPWMxA and/or EPWMxB to a high-impedance state</li> <li>– Configure EPWMxA and/or EPWMxB to ignore any trip condition.</li> </ul> </li> <li>• Configure how often the ePWM will react to each trip-zone signal: <ul style="list-style-type: none"> <li>– One-shot</li> <li>– Cycle-by-cycle</li> </ul> </li> <li>• Enable the trip-zone to initiate an interrupt.</li> <li>• Bypass the trip-zone module entirely.</li> </ul> |
| Event-trigger (ET)   | <ul style="list-style-type: none"> <li>• Enable the ePWM events that will trigger an interrupt.</li> <li>• Enable ePWM events that will trigger an ADC start-of-conversion event.</li> <li>• Specify the rate at which events cause triggers (every occurrence or every second or third occurrence)</li> <li>• Poll, set, or clear event flags</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                                                |
| Digital-compare (DC) | <ul style="list-style-type: none"> <li>• Enables comparator (COMP) module outputs and trip zone signals to create events and filtered events</li> <li>• Specify event-filtering options to capture TBCTR counter or generate blanking window</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |

Code examples are provided in the remainder of this document that show how to implement various ePWM module configurations. These examples use the constant definitions in the device *EPwm\_defines.h* file in the device-specific header file and peripheral examples software package.

### 3.2.2 Time-Base (TB) Submodule

Each ePWM module has its own time-base submodule that determines all of the event timing for the ePWM module. Built-in synchronization logic allows the time-base of multiple ePWM modules to work together as a single system. Figure 3-4 illustrates the time-base module's place within the ePWM.



**Figure 3-4. Time-Base Submodule Block Diagram**

#### 3.2.2.1 Purpose of the Time-Base Submodule

You can configure the time-base submodule for the following:

- Specify the ePWM time-base counter (TBCTR) frequency or period to control how often events occur.
- Manage time-base synchronization with other ePWM modules.
- Maintain a phase relationship with other ePWM modules.
- Set the time-base counter to count-up, count-down, or count-up-and-down mode.
- Generate the following events:
  - CTR = PRD: Time-base counter equal to the specified period (TBCTR = TBPRD) .
  - CTR = Zero: Time-base counter equal to zero (TBCTR = 0x0000).
- Configure the rate of the time-base clock; a prescaled version of the CPU system clock (SYSCLKOUT). This allows the time-base counter to increment/decrement at a slower rate.



**Table 3-4. Key Time-Base Signals**

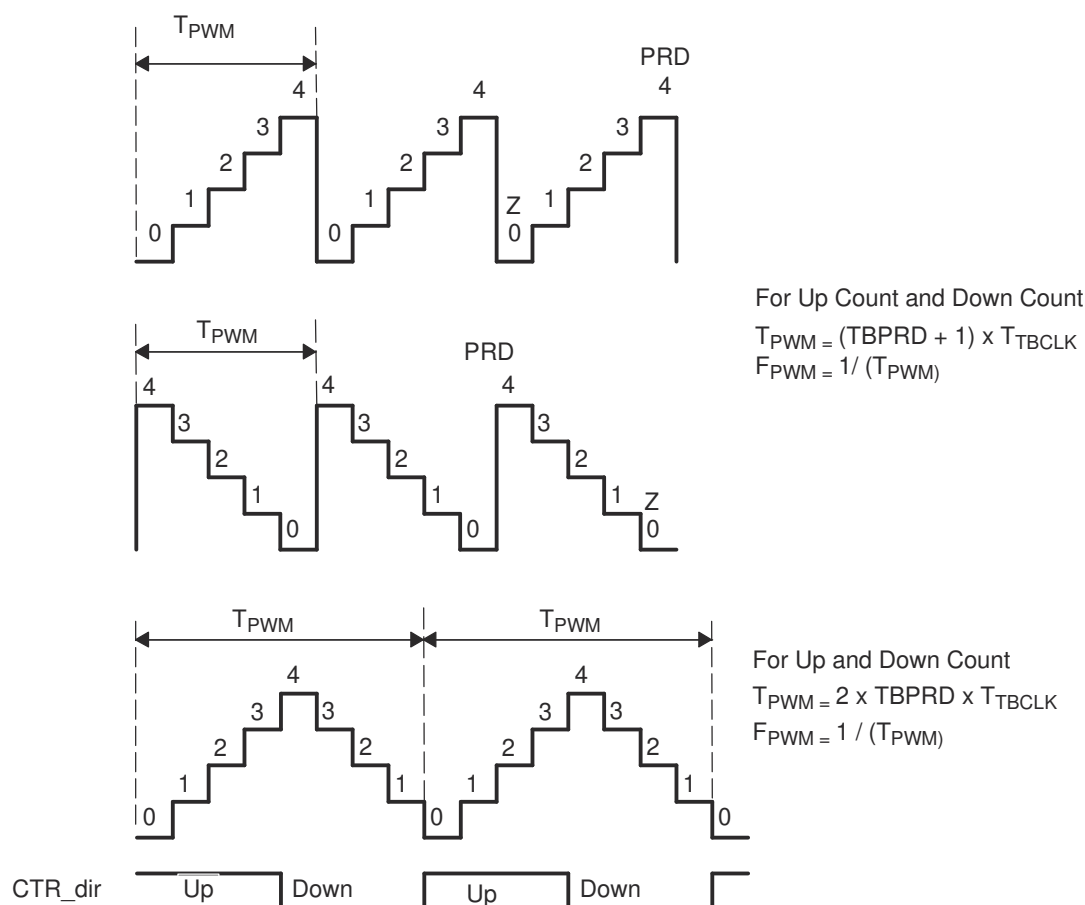
| Signal     | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
|------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| EPWMxSYNCl | Time-base synchronization input.<br><br>Input pulse used to synchronize the time-base counter with the counter of ePWM module earlier in the synchronization chain. An ePWM peripheral can be configured to use or ignore this signal. For the first ePWM module (EPWM1) this signal comes from a device pin. For subsequent ePWM modules this signal is passed from another ePWM peripheral. For example, EPWM2SYNCl is generated by the ePWM1 peripheral, EPWM3SYNCl is generated by ePWM2 and so forth. See <a href="#">Section 3.2.2.3.3</a> for information on the synchronization order of a particular device. |
| EPWMxSYNCO | Time-base synchronization output.<br><br>This output pulse is used to synchronize the counter of an ePWM module later in the synchronization chain. The ePWM module generates this signal from one of three event sources:<br><ol style="list-style-type: none"> <li>1. EPWMxSYNCl (Synchronization input pulse)</li> <li>2. CTR = Zero: The time-base counter equal to zero (TBCTR = 0x0000).</li> <li>3. CTR = CMPB: The time-base counter equal to the counter-compare B (TBCTR = CMPB) register.</li> </ol>                                                                                                       |
| CTR = PRD  | Time-base counter equal to the specified period.<br><br>This signal is generated whenever the counter value is equal to the active period register value. That is when TBCTR = TBPRD.                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| CTR = Zero | Time-base counter equal to zero<br><br>This signal is generated whenever the counter value is zero. That is when TBCTR equals 0x0000.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| CTR = CMPB | Time-base counter equal to active counter-compare B register (TBCTR = CMPB).<br><br>This event is generated by the counter-compare submodule and used by the synchronization out logic                                                                                                                                                                                                                                                                                                                                                                                                                                |
| CTR_dir    | Time-base counter direction.<br><br>Indicates the current direction of the ePWM's time-base counter. This signal is high when the counter is increasing and low when it is decreasing.                                                                                                                                                                                                                                                                                                                                                                                                                                |
| CTR_max    | Time-base counter equal max value. (TBCTR = 0xFFFF)<br><br>Generated event when the TBCTR value reaches its maximum value. This signal is only used only as a status bit                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| TBCLK      | Time-base clock.<br><br>This is a prescaled version of the system clock (SYSCLKOUT) and is used by all submodules within the ePWM. This clock determines the rate at which time-base counter increments or decrements.                                                                                                                                                                                                                                                                                                                                                                                                |

### 3.2.2.3 Calculating PWM Period and Frequency

The frequency of PWM events is controlled by the time-base period (TBPRD) register and the mode of the time-base counter. [Figure 3-6](#) shows the period ( $T_{pwm}$ ) and frequency ( $F_{pwm}$ ) relationships for the up-count, down-count, and up-down-count time-base counter modes when the period is set to 4 (TBPRD = 4). The time increment for each step is defined by the time-base clock (TBCLK) which is a prescaled version of the system clock (SYSCLKOUT).

The time-base counter has three modes of operation selected by the time-base control register (TBCTL):

- **Up-Down-Count Mode:** In up-down-count mode, the time-base counter starts from zero and increments until the period (TBPRD) value is reached. When the period value is reached, the time-base counter then decrements until it reaches zero. At this point the counter repeats the pattern and begins to increment.
- **Up-Count Mode:** In this mode, the time-base counter starts from zero and increments until it reaches the value in the period register (TBPRD). When the period value is reached, the time-base counter resets to zero and begins to increment once again.
- **Down-Count Mode:** In down-count mode, the time-base counter starts from the period (TBPRD) value and decrements until it reaches zero. When it reaches zero, the time-base counter is reset to the period value and it begins to decrement once again.


**Figure 3-6. Time-Base Frequency and Period**

### 3.2.2.3.1 Time-Base Period Shadow Register

The time-base period register (TBPRD) has a shadow register. Shadowing allows the register update to be synchronized with the hardware. The following definitions are used to describe all shadow registers in the ePWM module:

- **Active Register:** The active register controls the hardware and is responsible for actions that the hardware causes or invokes.
- **Shadow Register:** The shadow register buffers or provides a temporary holding location for the active register. It has no direct effect on any control hardware. At a strategic point in time the shadow register's content is transferred to the active register. This prevents corruption or spurious operation due to the register being asynchronously modified by software.

The memory address of the shadow period register is the same as the active register. Which register is written to or read from is determined by the TBCTL[PRDL] bit. This bit enables and disables the TBPRD shadow register as follows:

- **Time-Base Period Shadow Mode:** The TBPRD shadow register is enabled when TBCTL[PRDL] = 0. Reads from and writes to the TBPRD memory address go to the shadow register. The shadow register contents are transferred to the active register (TBPRD (Active) ← TBPRD (shadow)) when the time-base counter equals zero (TBCTR = 0x0000). By default the TBPRD shadow register is enabled.
- **Time-Base Period Immediate Load Mode:** If immediate load mode is selected (TBCTL[PRDL] = 1), then a read from or a write to the TBPRD memory address goes directly to the active register.



### 3.2.2.3.2 Time-Base Clock Synchronization

The TBCLKSYNC bit in the peripheral clock enable registers allows all users to globally synchronize all enabled ePWM modules to the time-base clock (TBCLK). When set, all enabled ePWM module clocks are started with the first rising edge of TBCLK aligned. For perfectly synchronized TBCLKs, the prescalers for each ePWM module must be set identically.

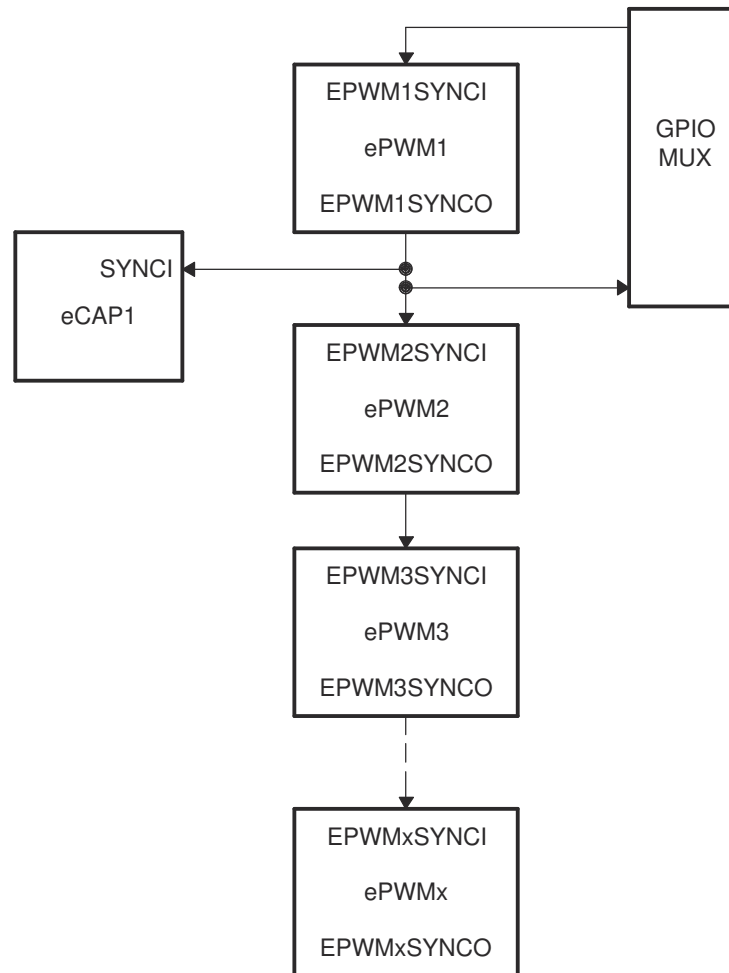
The proper procedure for enabling ePWM clocks is as follows:

1. Enable ePWM module clocks in the PCLKCRx register
2. Set TBCLKSYNC= 0
3. Configure ePWM modules
4. Set TBCLKSYNC=1

### 3.2.2.3.3 Time-Base Counter Synchronization

A time-base synchronization scheme connects all of the ePWM modules on a device. Each ePWM module has a synchronization input (EPWMxSYNCl) and a synchronization output (EPWMxSYNCO). The input synchronization for the first instance (ePWM1) comes from an external pin. The possible synchronization connections for the remaining ePWM modules are shown in [Figure 3-7](#).

Scheme 1 is shown in [Figure 3-7](#).



**Figure 3-7. Time-Base Counter Synchronization Scheme 1**

Each ePWM module can be configured to use or ignore the synchronization input. If the TBCTL[PHSEN] bit is set, then the time-base counter (TBCTR) of the ePWM module will be automatically loaded with the phase register (TBPHS) contents when one of the following conditions occur:

- **EPWMxSYNCl: Synchronization Input Pulse:** The value of the phase register is loaded into the counter register when an input synchronization pulse is detected (TBPHS → TBCTR). This operation occurs on the next valid time-base clock (TBCLK) edge. The source of the EPWMSYNCl pulse is selected by setting a GPIO to the EPWMSYNCl option per the GPIO Mux.

The delay from internal master module to slave modules is given by:

- if ( TBCLK = SYSCLKOUT): 2 x SYSCLKOUT
- if ( TBCLK != SYSCLKOUT): 1 TBCLK
- **Software Forced Synchronization Pulse:** Writing a 1 to the TBCTL[SWFSYNC] control bit invokes a software forced synchronization. This pulse is ORed with the synchronization input signal, and therefore has the same effect as a pulse on EPWMxSYNCl.
- **Digital Compare Event Synchronization Pulse:** DCAEVT1 and DCBEVT1 digital compare events can be configured to generate synchronization pulses which have the same affect as EPWMxSYNCl.

This feature enables the ePWM module to be automatically synchronized to the time base of another ePWM module. Lead or lag phase control can be added to the waveforms generated by different ePWM modules to synchronize them. In up-down-count mode, the TBCTL[PSHDIR] bit configures the direction of the time-base counter immediately after a synchronization event. The new direction is independent of the direction prior to the synchronization event. The PSHDIR bit is ignored in count-up or count-down modes. See [Figure 3-8](#) through [Figure 3-11](#) for examples.

Clearing the TBCTL[PHSEN] bit configures the ePWM to ignore the synchronization input pulse. The synchronization pulse can still be allowed to flow-through to the EPWMxSYNCO and be used to synchronize other ePWM modules. In this way, you can set up a master time-base (for example, ePWM1) and downstream modules (ePWM2 - ePWMx) may elect to run in synchronization with the master. See [Section 3.3](#) for more details on synchronization strategies.

### 3.2.2.4 Phase Locking the Time-Base Clocks of Multiple ePWM Modules

The TBCLKSYNC bit can be used to globally synchronize the time-base clocks of all enabled ePWM modules on a device. This bit is part of the device's clock enable registers and is described in the *System Control and Interrupts* section of this manual. When TBCLKSYNC = 0, the time-base clock of all ePWM modules is stopped (default). When TBCLKSYNC = 1, all ePWM time-base clocks are started with the rising edge of TBCLK aligned. For perfectly synchronized TBCLKs, the prescaler bits in the TBCTL register of each ePWM module must be set identically. The proper procedure for enabling the ePWM clocks is as follows:

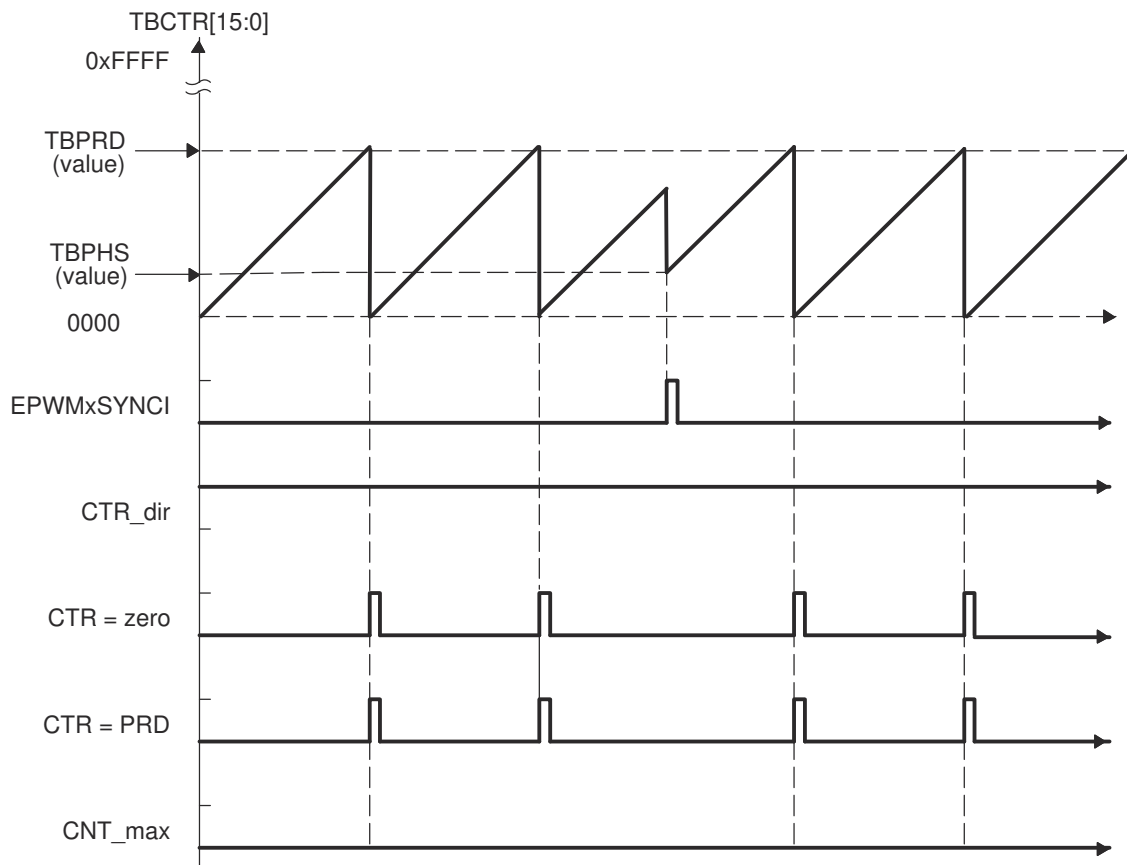
1. Enable the individual ePWM module clocks. This is described in the device-specific version of the *System Control and Interrupts* section.
2. Set TBCLKSYNC = 0. This will stop the time-base clock within any enabled ePWM module.
3. Configure the prescaler values and desired ePWM modes.
4. Set TBCLKSYNC = 1.

### 3.2.2.5 Time-base Counter Modes and Timing Waveforms

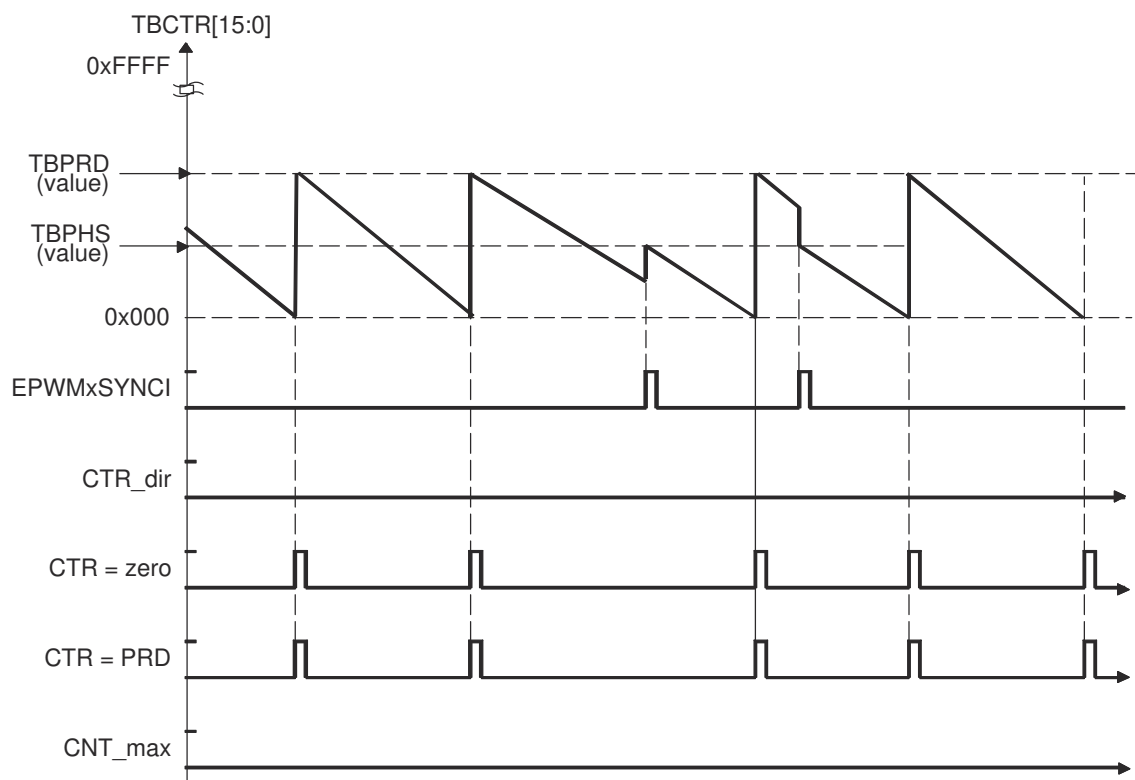
The time-base counter operates in one of four modes:

- Up-count mode which is asymmetrical
- Down-count mode which is asymmetrical
- Up-down-count which is symmetrical
- Frozen where the time-base counter is held constant at the current value

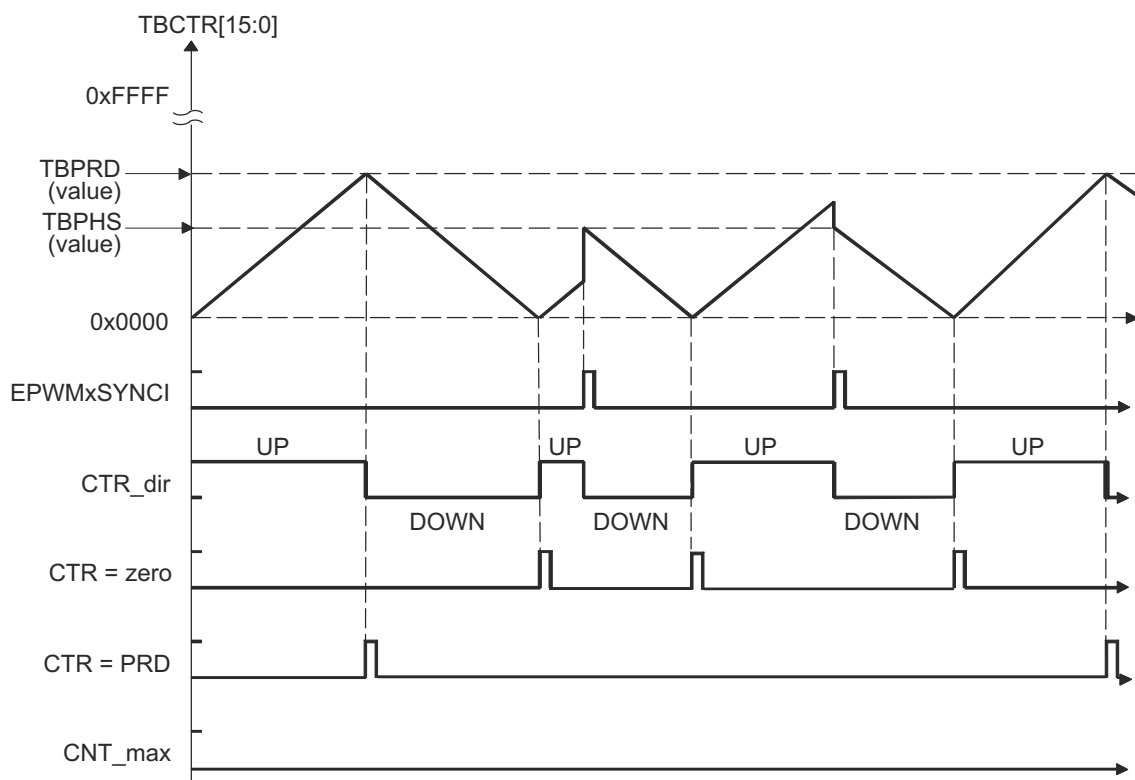
To illustrate the operation of the first three modes, the following timing diagrams show when events are generated and how the time-base responds to an EPWMxSYNCl signal.



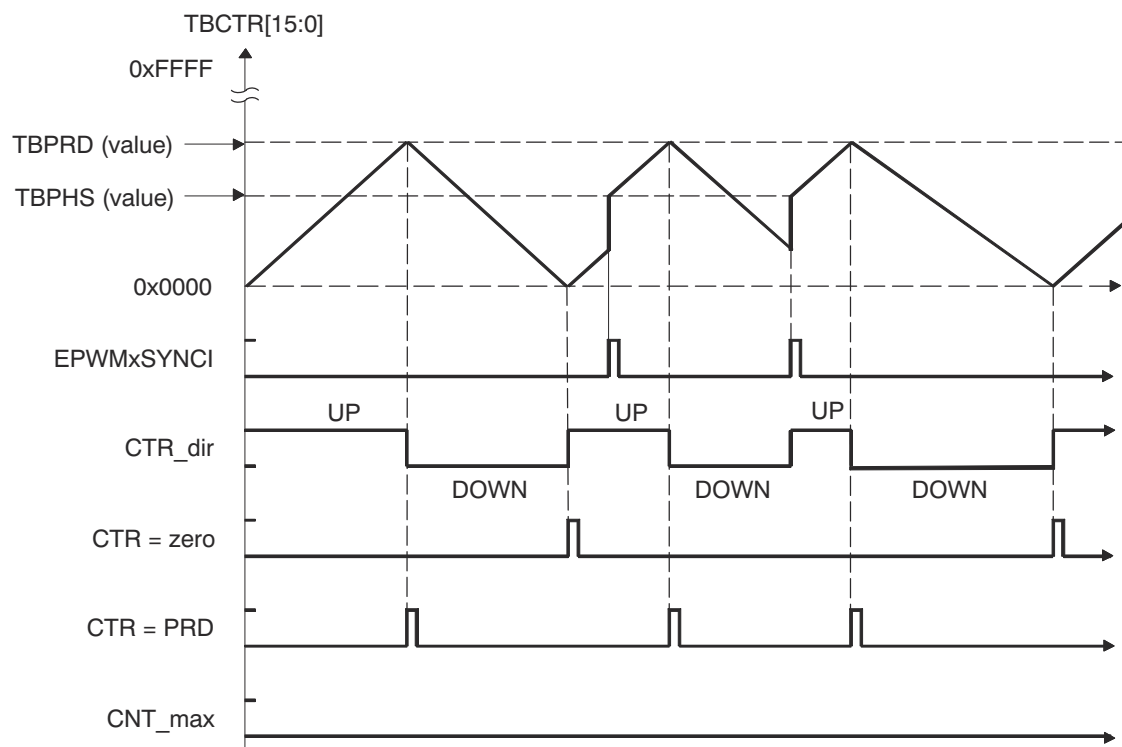
**Figure 3-8. Time-Base Up-Count Mode Waveforms**



**Figure 3-9. Time-Base Down-Count Mode Waveforms**



**Figure 3-10. Time-Base Up-Down-Count Waveforms, TBCTL[PHSDIR = 0] Count Down On Synchronization Event**

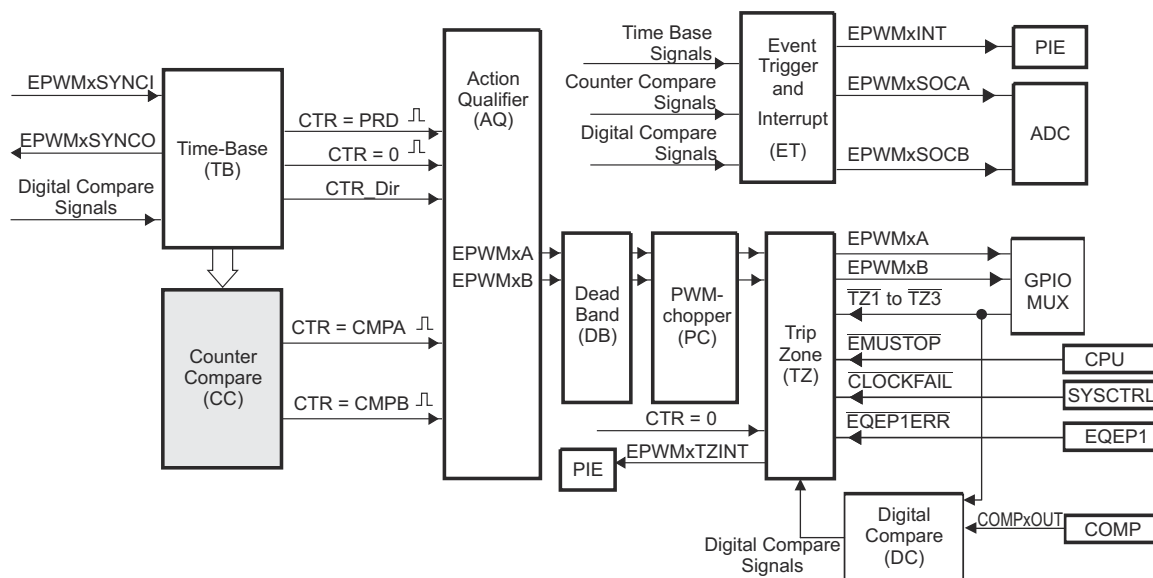


**Figure 3-11. Time-Base Up-Down Count Waveforms, TBCTL[PHSDIR = 1] Count Up On Synchronization Event**

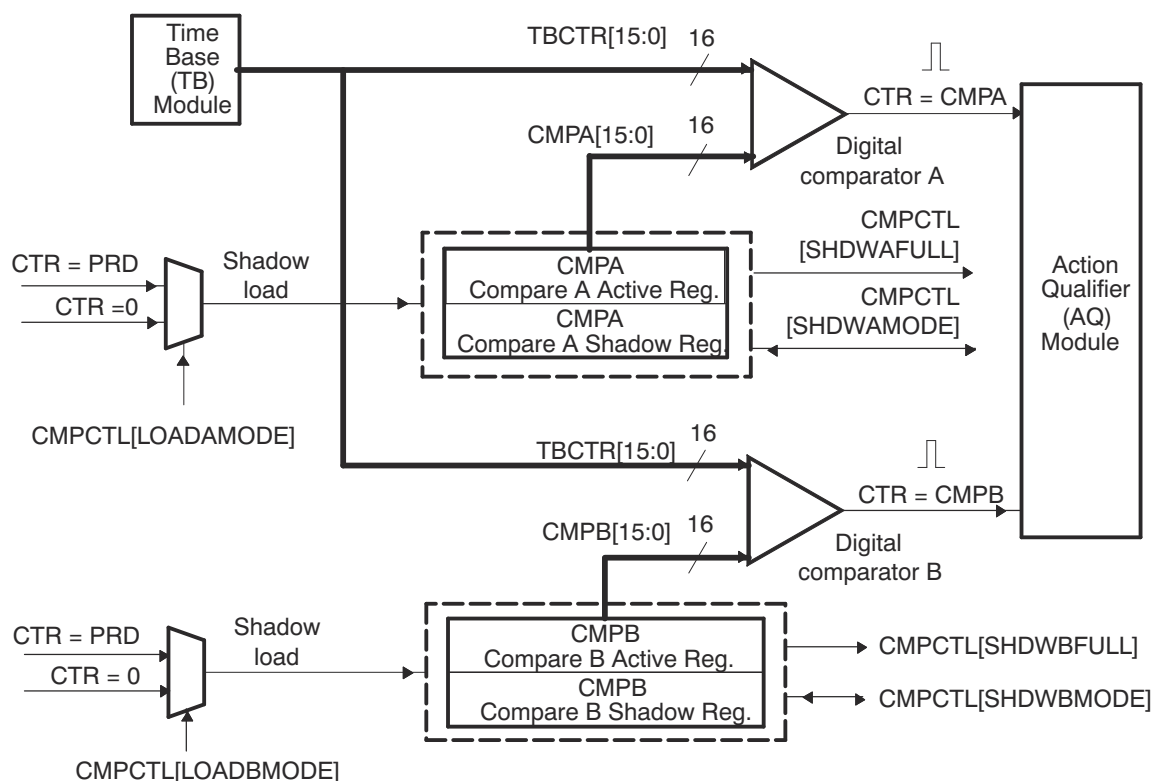
### 3.2.3 Counter-Compare (CC) Submodule

Figure 3-12 shows the counter-compare submodule within the ePWM.

Figure 3-13 shows the basic structure of the counter-compare submodule.



**Figure 3-12. Counter-Compare Submodule**



**Figure 3-13. Detailed View of the Counter-Compare Submodule**

### 3.2.3.1 Purpose of the Counter-Compare Submodule

The counter-compare submodule takes as input the time-base counter value. This value is continuously compared to the counter-compare A (CMPA) and counter-compare B (CMPB) registers. When the time-base counter is equal to one of the compare registers, the counter-compare unit generates an appropriate event.

The counter-compare:

- Generates events based on programmable time stamps using the CMPA and CMPB registers
  - CTR = CMPA: Time-base counter equals counter-compare A register (TBCTR = CMPA)
  - CTR = CMPB: Time-base counter equals counter-compare B register (TBCTR = CMPB)
- Controls the PWM duty cycle if the action-qualifier submodule is configured appropriately
- Shadows new compare values to prevent corruption or glitches during the active PWM cycle

### 3.2.3.2 Controlling and Monitoring the Counter-Compare Submodule

The counter-compare submodule operation is controlled and monitored by the registers shown in [Table 3-5](#).

**Table 3-5. Counter-Compare Submodule Registers**

| Register | Address Offset | Shadowed | Description                                                      | Bit Description                 |
|----------|----------------|----------|------------------------------------------------------------------|---------------------------------|
| CMPCTL   | 0x0007         | No       | Counter-Compare Control Register.                                | <a href="#">Section 3.4.2.1</a> |
| CMPAHR   | 0x0008         | Yes      | HRPWM Counter-Compare A Extension Register <sup>(1)</sup>        | <a href="#">Section 3.4.2.2</a> |
| CMPA     | 0x0009         | Yes      | Counter-Compare A Register                                       | <a href="#">Section 3.4.2.3</a> |
| CMPB     | 0x000A         | Yes      | Counter-Compare B Register                                       | <a href="#">Section 3.4.2.4</a> |
| CMPAHRM  | 0x002C         | Writes   | HRPWM counter-compare A Extension Mirror Register <sup>(1)</sup> | <a href="#">Section 3.4.2.5</a> |
| CMPAM    | 0x002D         | Writes   | Counter-compare A mirror Register                                | <a href="#">Section 3.4.2.6</a> |

- (1) This register is available only on ePWM modules with the high-resolution extension (HRPWM). On ePWM modules that do not include the HRPWM, this location is reserved. This register is also described in [Chapter 4](#). Refer to your device-specific data sheet to determine which ePWM instances include this feature.

The key signals associated with the counter-compare submodule are described in [Table 3-6](#).

**Table 3-6. Counter-Compare Submodule Key Signals**

| Signal     | Description of Event                                                                                                            | Registers Compared |
|------------|---------------------------------------------------------------------------------------------------------------------------------|--------------------|
| CTR = CMPA | Time-base counter equal to the active counter-compare A value                                                                   | TBCTR = CMPA       |
| CTR = CMPB | Time-base counter equal to the active counter-compare B value                                                                   | TBCTR = CMPB       |
| CTR = PRD  | Time-base counter equal to the active period.<br>Used to load active counter-compare A and B registers from the shadow register | TBCTR = TBPRD      |
| CTR = ZERO | Time-base counter equal to zero.<br>Used to load active counter-compare A and B registers from the shadow register              | TBCTR = 0x0000     |

### 3.2.3.3 Operational Highlights for the Counter-Compare Submodule

The counter-compare submodule is responsible for generating two independent compare events based on two compare registers:

1. CTR = CMPA: Time-base counter equal to counter-compare A register (TBCTR = CMPA)
2. CTR = CMPB: Time-base counter equal to counter-compare B register (TBCTR = CMPB)

For up-count or down-count mode, each event occurs only once per cycle. For up-down-count mode each event occurs twice per cycle if the compare value is between 0x0000-TBPRD and once per cycle if the compare value is equal to 0x0000 or equal to TBPRD. These events are fed into the action-qualifier submodule where they are qualified by the counter direction and converted into actions if enabled. Refer to [Section 3.2.4.1](#) for more details.

The counter-compare registers CMPA and CMPB each have an associated shadow register. Shadowing provides a way to keep updates to the registers synchronized with the hardware. When shadowing is used, updates to the active registers only occur at strategic points. This prevents corruption or spurious operation due to the register being asynchronously modified by software. The memory address of the active register and the shadow register is identical. Which register is written to or read from is determined by the CMPCTL[SHDWAMODE] and CMPCTL[SHDWBMODE] bits. These bits enable and disable the CMPA shadow register and CMPB shadow register respectively. The behavior of the two load modes is:

**Shadow Mode:** The shadow mode for the CMPA is enabled by clearing the CMPCTL[SHDWAMODE] bit and the shadow register for CMPB is enabled by clearing the CMPCTL[SHDWBMODE] bit. Shadow mode is enabled by default for both CMPA and CMPB.

If the shadow register is enabled then the content of the shadow register is transferred to the active register on one of the following events as specified by the CMPCTL[LOADAMODE] and CMPCTL[LOADBMODE] register bits:

- CTR = PRD: Time-base counter equal to the period (TBCTR = TBPRD)
- CTR = Zero: Time-base counter equal to zero (TBCTR = 0x0000)
- Both CTR = PRD and CTR = Zero

Only the active register contents are used by the counter-compare submodule to generate events to be sent to the action-qualifier.

**Immediate Load Mode:** If immediate load mode is selected (TBCTL[SHADWAMODE] = 1 or TBCTL[SHADWBMODE] = 1), then a read from or a write to the register will go directly to the active register.

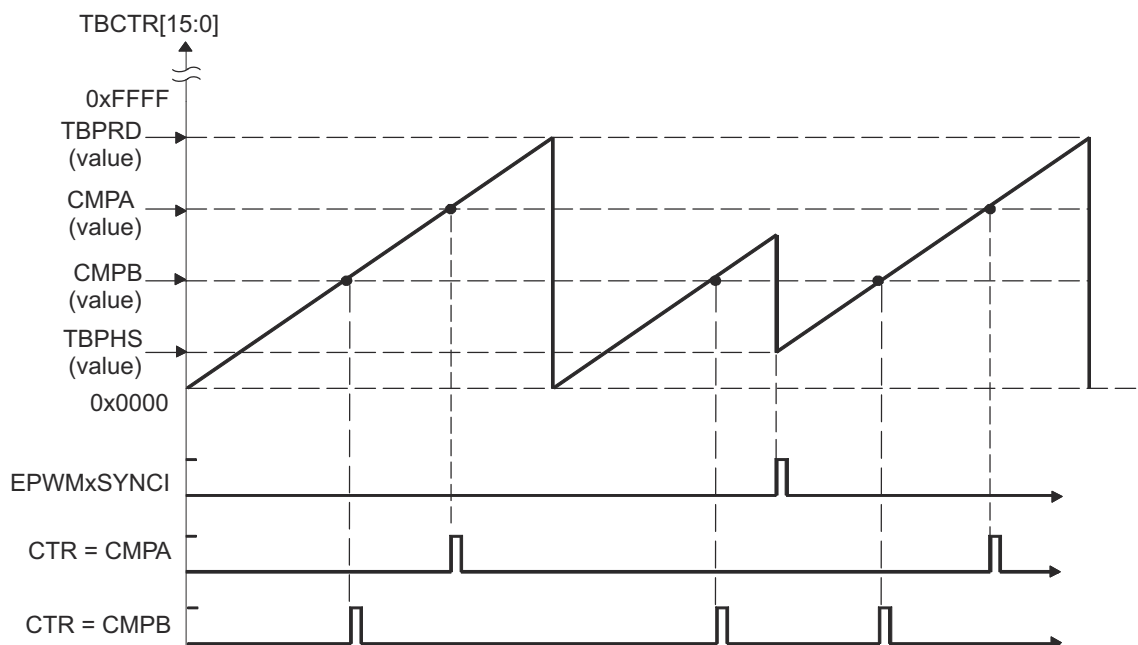
### 3.2.3.4 Count Mode Timing Waveforms

The counter-compare module can generate compare events in all three count modes:

- Up-count mode: used to generate an asymmetrical PWM waveform
- Down-count mode: used to generate an asymmetrical PWM waveform
- Up-down-count mode: used to generate a symmetrical PWM waveform

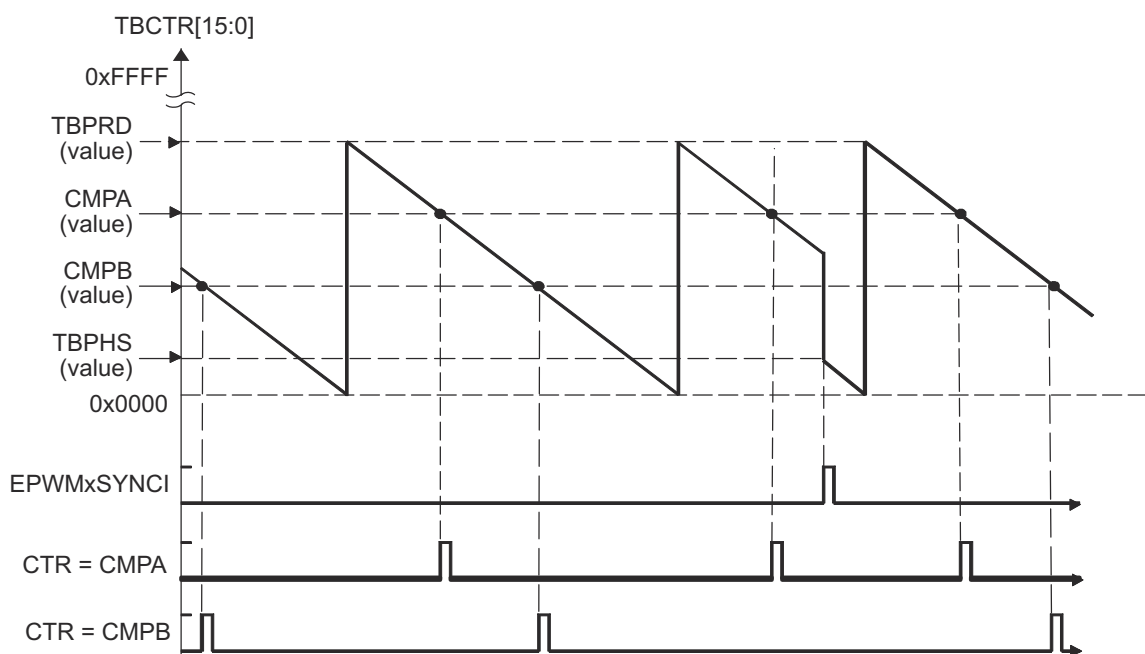
To best illustrate the operation of the first three modes, the timing diagrams in [Figure 3-14](#) through [Figure 3-17](#) show when events are generated and how the EPWMxSYNCI signal interacts.



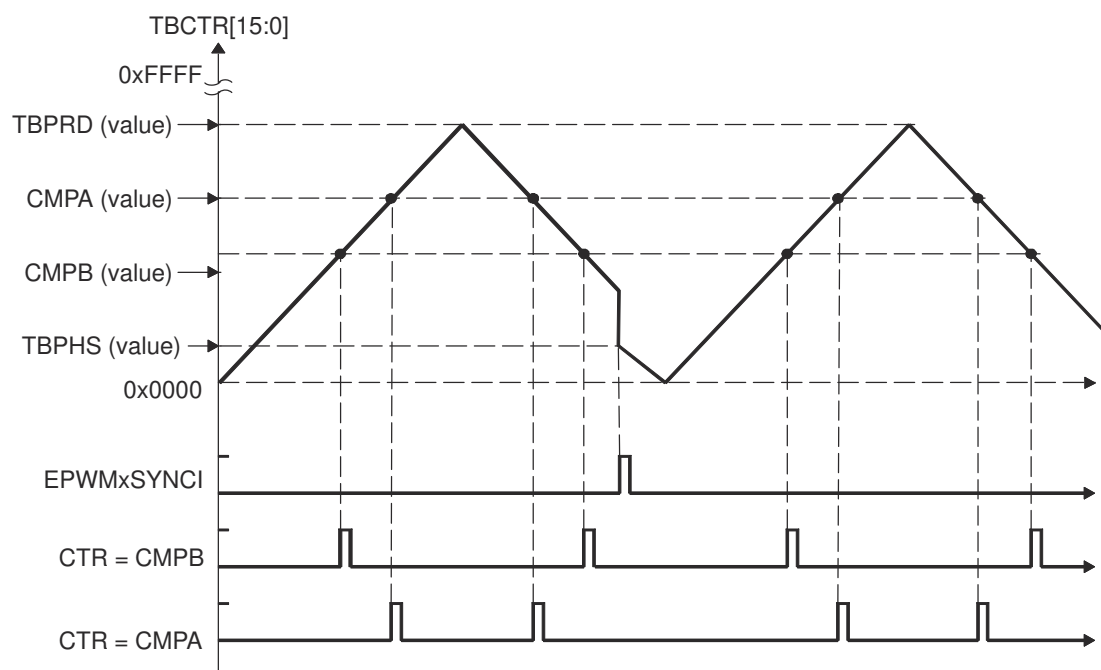


An EPWMxSYNCl external synchronization event can cause a discontinuity in the TBCTR count sequence. This can lead to a compare event being skipped. This skipping is considered normal operation and must be taken into account.

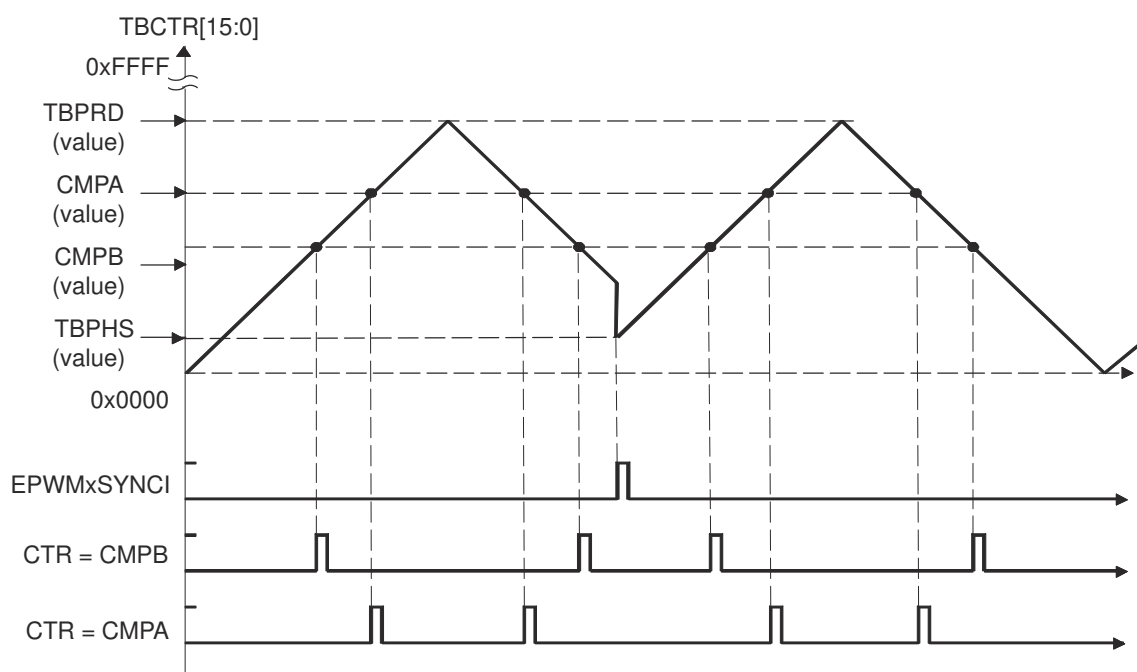
**Figure 3-14. Counter-Compare Event Waveforms in Up-Count Mode**



**Figure 3-15. Counter-Compare Events in Down-Count Mode**



**Figure 3-16. Counter-Compare Events In Up-Down-Count Mode, TBCTL[PHSDIR = 0] Count Down On Synchronization Event**

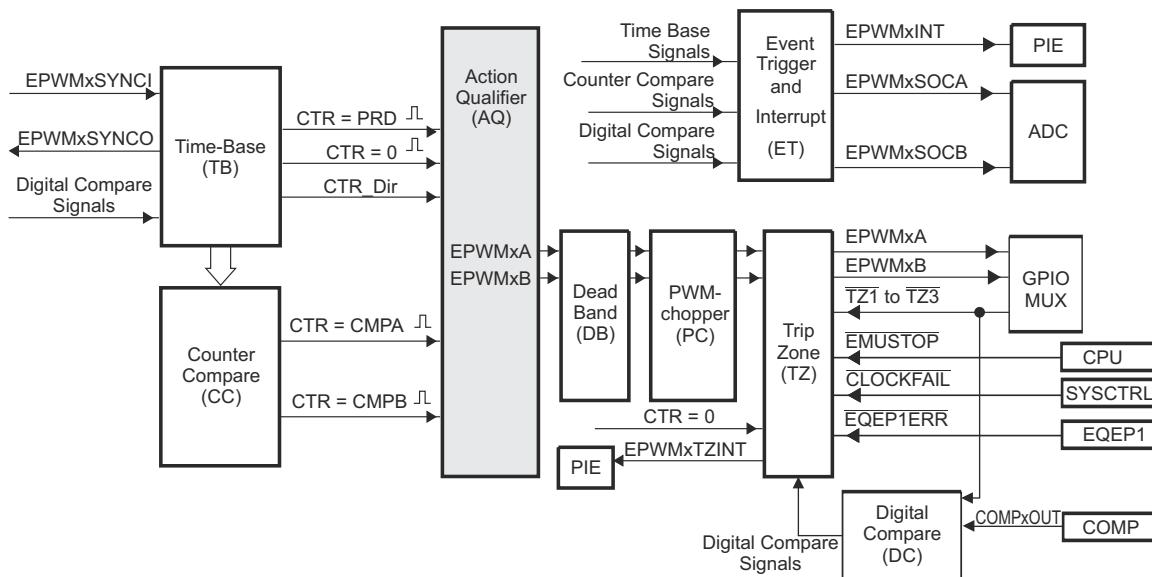


**Figure 3-17. Counter-Compare Events In Up-Down-Count Mode, TBCTL[PHSDIR = 1] Count Up On Synchronization Event**

### 3.2.4 Action-Qualifier (AQ) Submodule

Figure 3-18 shows the action-qualifier (AQ) submodule (see shaded block) in the ePWM system.

The action-qualifier submodule has the most important role in waveform construction and PWM generation. It decides which events are converted into various action types, thereby producing the required switched waveforms at the EPWMxA and EPWMxB outputs.



**Figure 3-18. Action-Qualifier Submodule**

#### 3.2.4.1 Purpose of the Action-Qualifier Submodule

The action-qualifier submodule is responsible for the following:

- Qualifying and generating actions (set, clear, toggle) based on the following events:
  - CTR = PRD: Time-base counter equal to the period (TBCTR = TBPRD).
  - CTR = Zero: Time-base counter equal to zero (TBCTR = 0x0000)
  - CTR = CMPA: Time-base counter equal to the counter-compare A register (TBCTR = CMPA)
  - CTR = CMPB: Time-base counter equal to the counter-compare B register (TBCTR = CMPB)
- Managing priority when these events occur concurrently
- Providing independent control of events when the time-base counter is increasing and when it is decreasing

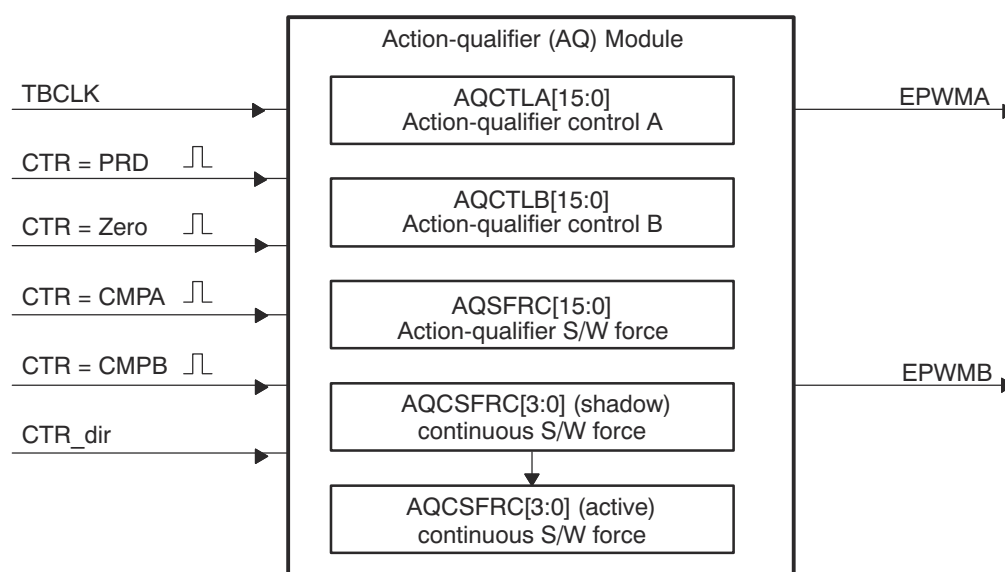
### 3.2.4.2 Action-Qualifier Submodule Control and Status Register Definitions

The action-qualifier submodule operation is controlled and monitored via the registers in [Table 3-7](#).

**Table 3-7. Action-Qualifier Submodule Registers**

| Register | Address Offset | Shadowed | Description                                             | Bit Description                 |
|----------|----------------|----------|---------------------------------------------------------|---------------------------------|
| AQCTLA   | 0x000B         | No       | Action-Qualifier Control Register for Output A (EPWMxA) | <a href="#">Section 3.4.3.1</a> |
| AQCTLB   | 0x000C         | No       | Action-Qualifier Control Register for Output B (EPWMxB) | <a href="#">Section 3.4.3.2</a> |
| AQSFRC   | 0x000D         | No       | Action-Qualifier Software Force Register                | <a href="#">Section 3.4.3.3</a> |
| AQCSFRC  | 0x000E         | Yes      | Action-Qualifier Continuous Software Force              | <a href="#">Section 3.4.3.4</a> |

The action-qualifier submodule is based on event-driven logic. It can be thought of as a programmable cross switch with events at the input and actions at the output, all of which are software controlled via the set of registers shown in [Table 3-7](#).



**Figure 3-19. Action-Qualifier Submodule Inputs and Outputs**

For convenience, the possible input events are summarized again in [Table 3-8](#).

**Table 3-8. Action-Qualifier Submodule Possible Input Events**

| Signal                | Description                                      | Registers Compared |
|-----------------------|--------------------------------------------------|--------------------|
| CTR = PRD             | Time-base counter equal to the period value      | TBCTR = TBPRD      |
| CTR = Zero            | Time-base counter equal to zero                  | TBCTR = 0x0000     |
| CTR = CMPA            | Time-base counter equal to the counter-compare A | TBCTR = CMPA       |
| CTR = CMPB            | Time-base counter equal to the counter-compare B | TBCTR = CMPB       |
| Software forced event | Asynchronous event initiated by software         |                    |

The software forced action is a useful asynchronous event. This control is handled by registers AQSFRC and AQCSFRC.

The action-qualifier submodule controls how the two outputs EPWMxA and EPWMxB behave when a particular event occurs. The event inputs to the action-qualifier submodule are further qualified by the counter direction (up or down). This allows for independent action on outputs on both the count-up and count-down phases.

The possible actions imposed on outputs EPWMxA and EPWMxB are:

- **Set High:** Set output EPWMxA or EPWMxB to a high level.
- **Clear Low:** Set output EPWMxA or EPWMxB to a low level.
- **Toggle:** If EPWMxA or EPWMxB is currently pulled high, then pull the output low. If EPWMxA or EPWMxB is currently pulled low, then pull the output high.
- **Do Nothing:** Keep outputs EPWMxA and EPWMxB at same level as currently set. Although the "Do Nothing" option prevents an event from causing an action on the EPWMxA and EPWMxB outputs, this event can still trigger interrupts and ADC start of conversion. See the Event-trigger Submodule description in [Section 3.2.8](#) for details.

Actions are specified independently for either output (EPWMxA or EPWMxB). Any or all events can be configured to generate actions on a given output. For example, both CTR = CMPA and CTR = CMPB can operate on output EPWMxA. All qualifier actions are configured via the control registers found at the end of this section.

For clarity, the drawings in this document use a set of symbolic actions. These symbols are summarized in [Figure 3-20](#). Each symbol represents an action as a marker in time. Some actions are fixed in time (zero and period) while the CMPA and CMPB actions are moveable and their time positions are programmed via the counter-compare A and B registers, respectively. To turn off or disable an action, use the "Do Nothing option"; it is the default at reset.

| S/W<br>force                                                                        | TB Counter equals:                                                                  |                                                                                     |                                                                                     |                                                                                       | Actions    |
|-------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|------------|
|                                                                                     | Zero                                                                                | Comp<br>A                                                                           | Comp<br>B                                                                           | Period                                                                                |            |
|  |  |  |  |  | Do Nothing |
|  |  |  |  |  | Clear Low  |
|  |  |  |  |  | Set High   |
|  |  |  |  |  | Toggle     |

**Figure 3-20. Possible Action-Qualifier Actions for EPWMxA and EPWMxB Outputs**

### 3.2.4.3 Action-Qualifier Event Priority

It is possible for the ePWM action qualifier to receive more than one event at the same time. In this case events are assigned a priority by the hardware. The general rule is events occurring later in time have a higher priority and software forced events always have the highest priority. The event priority levels for up-down-count mode are shown in [Table 3-9](#). A priority level of 1 is the highest priority and level 7 is the lowest. The priority changes slightly depending on the direction of TBCTR.

**Table 3-9. Action-Qualifier Event Priority for Up-Down-Count Mode**

| Priority Level | Event If TBCTR is Incrementing<br>TBCTR = Zero up to TBCTR = TBPRD | Event If TBCTR is Decrementing<br>TBCTR = TBPRD down to TBCTR = 1 |
|----------------|--------------------------------------------------------------------|-------------------------------------------------------------------|
| 1 (Highest)    | Software forced event                                              | Software forced event                                             |
| 2              | Counter equals CMPB on up-count (CBU)                              | Counter equals CMPB on down-count (CBD)                           |
| 3              | Counter equals CMPA on up-count (CAU)                              | Counter equals CMPA on down-count (CAD)                           |
| 4              | Counter equals zero                                                | Counter equals period (TBPRD)                                     |
| 5              | Counter equals CMPB on down-count (CBD)                            | Counter equals CMPB on up-count (CBU)                             |
| 6 (Lowest)     | Counter equals CMPA on down-count (CAD)                            | Counter equals CMPA on up-count (CBU)                             |

[Table 3-10](#) shows the action-qualifier priority for up-count mode. In this case, the counter direction is always defined as up and thus down-count events will never be taken.

**Table 3-10. Action-Qualifier Event Priority for Up-Count Mode**

| Priority Level | Event                                   |
|----------------|-----------------------------------------|
| 1 (Highest)    | Software forced event                   |
| 2              | Counter equal to period (TBPRD)         |
| 3              | Counter equal to CMPB on up-count (CBU) |
| 4              | Counter equal to CMPA on up-count (CAU) |
| 5 (Lowest)     | Counter equal to Zero                   |

[Table 3-11](#) shows the action-qualifier priority for down-count mode. In this case, the counter direction is always defined as down and thus up-count events will never be taken.

**Table 3-11. Action-Qualifier Event Priority for Down-Count Mode**

| Priority Level | Event                                     |
|----------------|-------------------------------------------|
| 1 (Highest)    | Software forced event                     |
| 2              | Counter equal to Zero                     |
| 3              | Counter equal to CMPB on down-count (CBD) |
| 4              | Counter equal to CMPA on down-count (CAD) |
| 5 (Lowest)     | Counter equal to period (TBPRD)           |

It is possible to set the compare value greater than the period. In this case the action will take place as shown in [Table 3-12](#).

**Table 3-12. Behavior if CMPA/CMPB is Greater than the Period**

| Counter Mode  | Compare on Up-Count Event<br>CAU/CBU                                                                                                                     | Compare on Down-Count Event<br>CAD/CBD |
|---------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------|
| Up-Count Mode | If CMPA/CMPB $\leq$ TBPRD period, then the event occurs on a compare match (TBCTR=CMPA or CMPB).<br>If CMPA/CMPB > TBPRD, then the event will not occur. | Never occurs.                          |

**Table 3-12. Behavior if CMPA/CMPB is Greater than the Period (continued)**

| Counter Mode       | Compare on Up-Count Event<br>CAU/CBU                                                                                                                                                                              | Compare on Down-Count Event<br>CAD/CBD                                                                                                                                                                   |
|--------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Down-Count Mode    | Never occurs.                                                                                                                                                                                                     | <p>If CMPA/CMPB &lt; TBPRD, the event will occur on a compare match (TBCTR=CMPA or CMPB).</p> <p>If CMPA/CMPB ≥ TBPRD, the event will occur on a period match (TBCTR=TBPRD).</p>                         |
| Up-Down-Count Mode | <p>If CMPA/CMPB &lt; TBPRD and the counter is incrementing, the event occurs on a compare match (TBCTR=CMPA or CMPB).</p> <p>If CMPA/CMPB is ≥ TBPRD, the event will occur on a period match (TBCTR = TBPRD).</p> | <p>If CMPA/CMPB &lt; TBPRD and the counter is decrementing, the event occurs on a compare match (TBCTR=CMPA or CMPB).</p> <p>If CMPA/CMPB ≥ TBPRD, the event occurs on a period match (TBCTR=TBPRD).</p> |

### 3.2.4.4 Waveforms for Common Configurations

#### Note

The waveforms in this document show the ePWMs behavior for a static compare register value. In a running system, the active compare registers (CMPA and CMPB) are typically updated from their respective shadow registers once every period. The user specifies when the update will take place; either when the time-base counter reaches zero or when the time-base counter reaches period. There are some cases when the action based on the new value can be delayed by one period or the action based on the old value can take effect for an extra period. Some PWM configurations avoid this situation. These include, but are not limited to, the following:

#### Use up-down-count mode to generate a symmetric PWM:

- If you load CMPA/CMPB on zero, then use CMPA/CMPB values greater than or equal to 1.
- If you load CMPA/CMPB on period, then use CMPA/CMPB values less than or equal to TBPRD-1.

This means there will always be a pulse of at least one TBCLK cycle in a PWM period which, when very short, tend to be ignored by the system.

#### Use up-down-count mode to generate an asymmetric PWM:

- To achieve 50%-0% asymmetric PWM use the following configuration: Load CMPA/CMPB on period and use the period action to clear the PWM and a compare-up action to set the PWM. Modulate the compare value from 0 to TBPRD to achieve 50%-0% PWM duty.

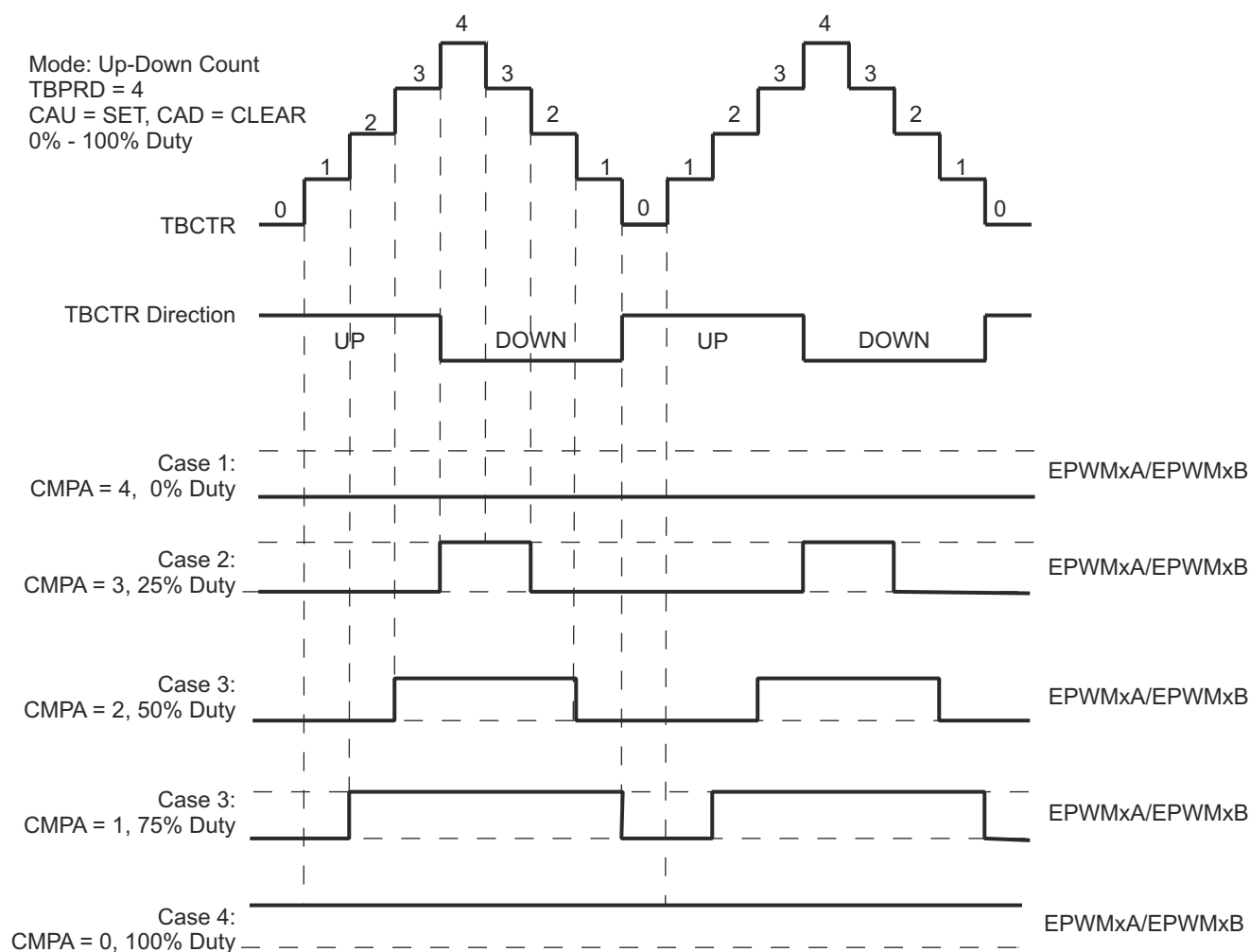
#### When using up-count mode to generate an asymmetric PWM:

- To achieve 0-100% asymmetric PWM use the following configuration: Load CMPA/CMPB on TBPRD. Use the Zero action to set the PWM and a compare-up action to clear the PWM. Modulate the compare value from 0 to TBPRD+1 to achieve 0-100% PWM duty.

See [Using the ePWM Module for 0% - 100% Duty Cycle Control](#). The software configurations described in this application report are not applicable when changing compare registers from a non-zero value to zero. However, they still apply when changing from a compare value of 0 to a non-zero value.

Figure 3-21 shows how a symmetric PWM waveform can be generated using the up-down-count mode of the TBCTR. In this mode 0%-100% DC modulation is achieved by using equal compare matches on the up count and down count portions of the waveform. In the example shown, CMPA is used to make the comparison. When the counter is incrementing the CMPA match will pull the PWM output high. Likewise, when the counter is decrementing the compare match will pull the PWM signal low. When CMPA = TBPRD, the PWM signal is low for the entire period giving the 0% duty waveform. When CMPA = 0, the PWM signal is high achieving 100% duty.

When using this configuration in practice, if you load CMPA/CMPB on zero, then use CMPA/CMPB values greater than or equal to 1. If you load CMPA/CMPB on period, then use CMPA/CMPB values less than or equal to TBPRD-1. This means there will always be a pulse of at least one TBCLK cycle in a PWM period which, when very short, tend to be ignored by the system.



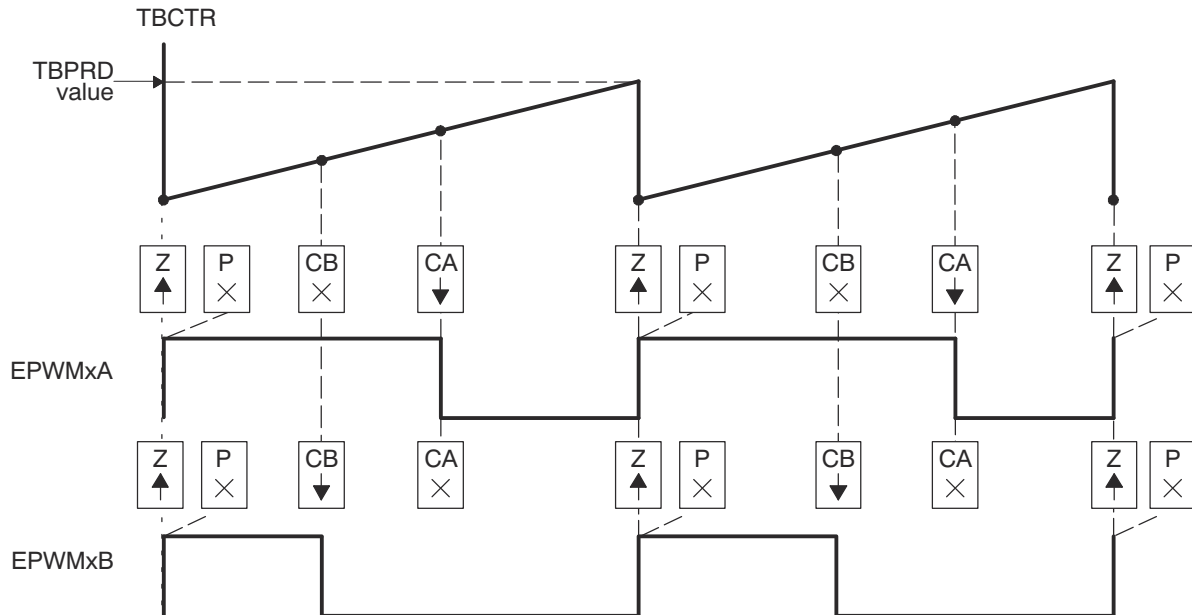
**Figure 3-21. Up-Down-Count Mode Symmetrical Waveform**

The PWM waveforms in [Figure 3-22](#) through [Figure 3-27](#) show some common action-qualifier configurations. The C-code samples in [Example 3-1](#) through [Example 3-6](#) shows how to configure an ePWM module for each case. Some conventions used in the figures and examples are as follows:

- TBPRD, CMPA, and CMPB refer to the value written in their respective registers. The active register, not the shadow register, is used by the hardware.
- CMPx, refers to either CMPA or CMPB.
- EPWMxA and EPWMxB refer to the output signals from ePWMx
- Up-Down means Count-up-and-down mode, Up means up-count mode and Dwn means down-count mode
- Sym = Symmetric, Asym = Asymmetric



Example 3-1 contains a code sample showing initialization and run time for the waveforms in Figure 3-22.



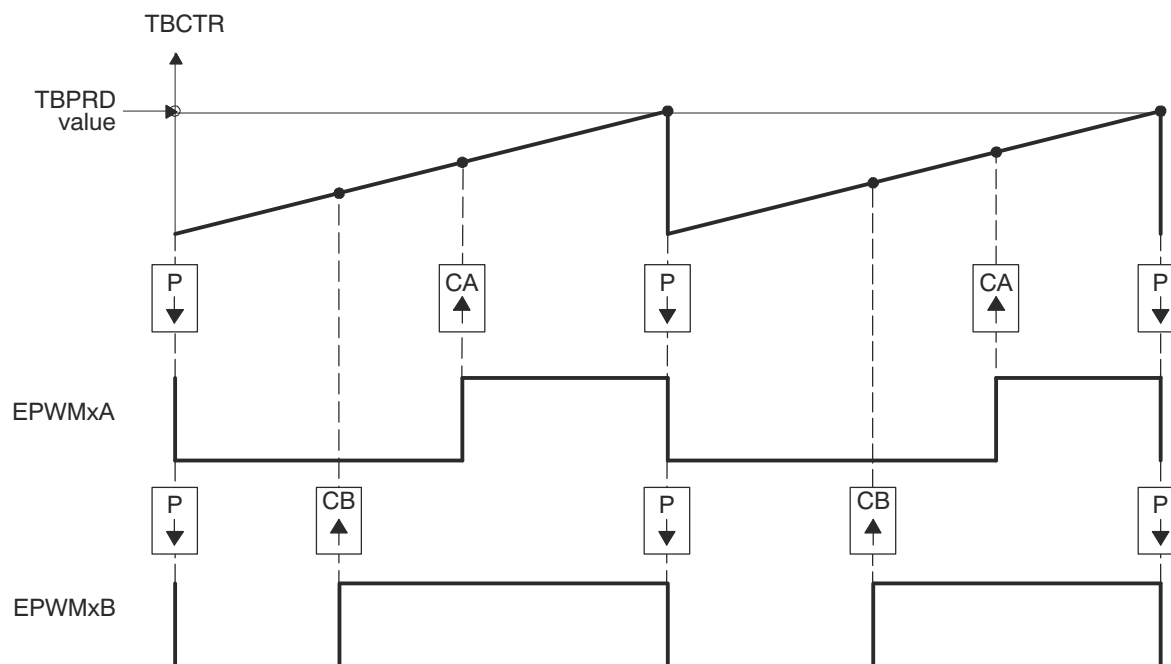
- PWM period =  $(TBPRD + 1) \times T_{TBCLK}$
- Duty modulation for EPWMxA is set by CMPA, and is active high (that is, high time duty proportional to CMPA).
- Duty modulation for EPWMxB is set by CMPB and is active high (that is, high time duty proportional to CMPB).
- The "Do Nothing" actions (X) are shown for completeness, but will not be shown on subsequent diagrams.
- Actions at zero and period, although appearing to occur concurrently, are actually separated by one TBCLK period. TBCTR wraps from period to 0000.

**Figure 3-22. Up, Single Edge Asymmetric Waveform, With Independent Modulation on EPWMxA and EPWMxB—Active High**

### Example 3-1. Code Sample for Figure 3-22

```
// Initialization Time
// =====
EPwm1Regs.TBPRD = 600;           // Period = 601 TBCLK counts
EPwm1Regs.CMPA.half.CMPA = 350; // Compare A = 350 TBCLK counts
EPwm1Regs.CMPB = 200;           // Compare B = 200 TBCLK counts
EPwm1Regs.TBPHS = 0;            // Set Phase register to zero
EPwm1Regs.TBCTR = 0;            // clear TB counter
EPwm1Regs.TBCTL.bit.CTRMODE = TB_COUNT_UP;
EPwm1Regs.TBCTL.bit.PHSEN = TB_DISABLE; // Phase loading disabled
EPwm1Regs.TBCTL.bit.PRDL = TB_SHADOW;
EPwm1Regs.TBCTL.bit.SYNCSEL = TB_SYNC_DISABLE;
EPwm1Regs.TBCTL.bit.HSPCLKDIV = TB_DIV1; // TBCLK = SYSCLK
EPwm1Regs.TBCTL.bit.CLKDIV = TB_DIV1;
EPwm1Regs.CMPCTL.bit.SHDWAMODE = CC_SHADOW;
EPwm1Regs.CMPCTL.bit.SHDWBMODE = CC_SHADOW;
EPwm1Regs.CMPCTL.bit.LOADAMODE = CC_CTR_ZERO; // load on CTR = Zero
EPwm1Regs.CMPCTL.bit.LOADBMODE = CC_CTR_ZERO; // load on CTR = Zero
EPwm1Regs.AQCTLA.bit.ZRO = AQ_SET;
EPwm1Regs.AQCTLA.bit.CAU = AQ_CLEAR;
EPwm1Regs.AQCTLB.bit.ZRO = AQ_SET;
EPwm1Regs.AQCTLB.bit.CBU = AQ_CLEAR;
//
// Run Time
// =====
EPwm1Regs.CMPA.half.CMPA = Duty1A; // adjust duty for output EPWM1A
EPwm1Regs.CMPB = Duty1B;           // adjust duty for output EPWM1B
```

Example 3-2 contains a code sample showing initialization and run time for the waveforms in Figure 3-23.



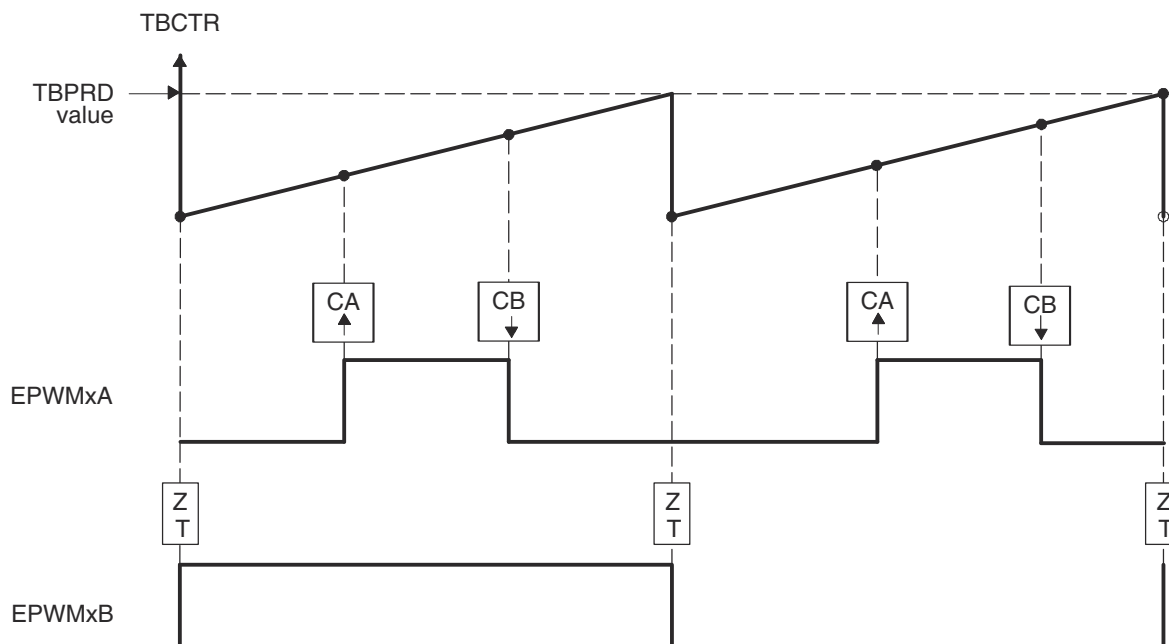
- PWM period =  $(TBPRD + 1) \times T_{TBCLK}$
- Duty modulation for EPWMxA is set by CMPA, and is active low (that is, the low time duty is proportional to CMPA).
- Duty modulation for EPWMxB is set by CMPB and is active low (that is, the low time duty is proportional to CMPB).
- Actions at zero and period, although appearing to occur concurrently, are actually separated by one TBCLK period. TBCTR wraps from period to 0000.

**Figure 3-23. Up, Single Edge Asymmetric Waveform With Independent Modulation on EPWMxA and EPWMxB—Active Low**

**Example 3-2. Code Sample for [Figure 3-23](#)**

```
// Initialization Time
// =====
EPwm1Regs.TBPRD = 600;                // Period = 601 TBCLK counts
EPwm1Regs.CMPA.half.CMPA = 350;      // Compare A = 350 TBCLK counts
EPwm1Regs.CMPB = 200;                // Compare B = 200 TBCLK counts
EPwm1Regs.TBPHS = 0;                 // Set Phase register to zero
EPwm1Regs.TBCTR = 0;                 // clear TB counter
EPwm1Regs.TBCTL.bit.CTRMODE = TB_COUNT_UP;
EPwm1Regs.TBCTL.bit.PHSEN = TB_DISABLE; // Phase loading disabled
EPwm1Regs.TBCTL.bit.PRDL = TB_SHADOW;
EPwm1Regs.TBCTL.bit.SYNCSEL = TB_SYNC_DISABLE;
EPwm1Regs.TBCTL.bit.HSPCLKDIV = TB_DIV1; // TBCLK = SYSCLKOUT
EPwm1Regs.TBCTL.bit.CLKDIV = TB_DIV1;
EPwm1Regs.CMPCTL.bit.SHDWAMODE = CC_SHADOW;
EPwm1Regs.CMPCTL.bit.SHDWBMODE = CC_SHADOW;
EPwm1Regs.CMPCTL.bit.LOADAMODE = CC_CTR_ZERO; // load on TBCTR = Zero
EPwm1Regs.CMPCTL.bit.LOADBMODE = CC_CTR_ZERO; // load on TBCTR = Zero
EPwm1Regs.AQCTLA.bit.PR = AQ_CLEAR;
EPwm1Regs.AQCTLA.bit.CAU = AQ_SET;
EPwm1Regs.AQCTLB.bit.PR = AQ_CLEAR;
EPwm1Regs.AQCTLB.bit.CBU = AQ_SET;
//
// Run Time
// =====
EPwm1Regs.CMPA.half.CMPA = Duty1A;    // adjust duty for output EPWM1A
EPwm1Regs.CMPB = Duty1B;              // adjust duty for output EPWM1B
```

**Example 3-3** contains a code sample showing initialization and run time for the waveforms [Figure 3-24](#). Use the constant definitions in the device-specific header file.



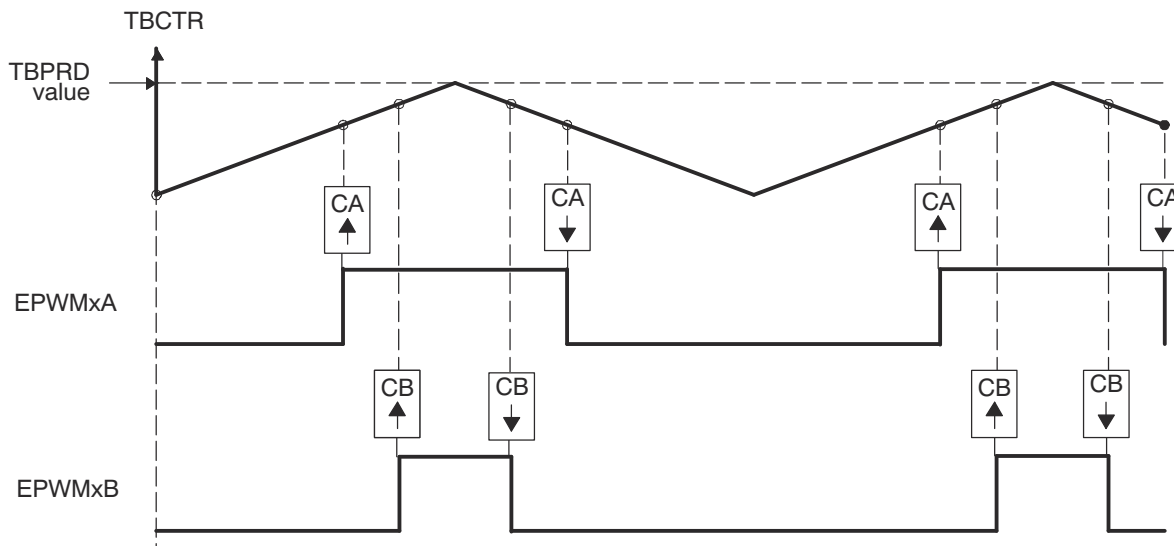
- PWM frequency =  $1 / ((TBPRD + 1) \times T_{TBCLK})$
- Pulse can be placed anywhere within the PWM cycle (0000 - TBPRD)
- High time duty proportional to (CMPB - CMPA)
- EPWMxB can be used to generate a 50% duty square wave with frequency =  $\frac{1}{2} \times ((TBPRD + 1) \times TBCLK)$

**Figure 3-24. Up-Count, Pulse Placement Asymmetric Waveform With Independent Modulation on EPWMxA**

### Example 3-3. Code Sample for [Figure 3-24](#)

```
// Initialization Time
// =====
EPwm1Regs.TBPRD = 600; // Period = 601 TBCLK counts
EPwm1Regs.CMPA.half.CMPA = 200; // Compare A = 200 TBCLK counts
EPwm1Regs.CMPB = 400; // Compare B = 400 TBCLK counts
EPwm1Regs.TBPHS = 0; // Set Phase register to zero
EPwm1Regs.TBCTR = 0; // clear TB counter
EPwm1Regs.TBCTL.bit.CTRMODE = TB_COUNT_UP;
EPwm1Regs.TBCTL.bit.PHSEN = TB_DISABLE; // Phase loading disabled
EPwm1Regs.TBCTL.bit.PRDL = TB_SHADOW;
EPwm1Regs.TBCTL.bit.SYNCSEL = TB_SYNC_DISABLE;
EPwm1Regs.TBCTL.bit.HSPCLKDIV = TB_DIV1; // TBCLK = SYSCLKOUT
EPwm1Regs.TBCTL.bit.CLKDIV = TB_DIV1;
EPwm1Regs.CMPCTL.bit.SHDWAMODE = CC_SHADOW;
EPwm1Regs.CMPCTL.bit.SHDWBMODE = CC_SHADOW;
EPwm1Regs.CMPCTL.bit.LOADAMODE = CC_CTR_ZERO; // load on TBCTR = Zero
EPwm1Regs.CMPCTL.bit.LOADBMODE = CC_CTR_ZERO; // load on TBCTR = Zero
EPwm1Regs.AQCTLA.bit.CAU = AQ_SET;
EPwm1Regs.AQCTLA.bit.CBU = AQ_CLEAR;
EPwm1Regs.AQCTLB.bit.ZRO = AQ_TOGGLE;
//
// Run Time
// =====
EPwm1Regs.CMPA.half.CMPA = EdgePosA; // adjust duty for output EPWM1A only
EPwm1Regs.CMPB = EdgePosB;
```

**Example 3-4** contains a code sample showing initialization and run time for the waveforms in [Figure 3-25](#). Use the constant definitions in the device-specific header file.



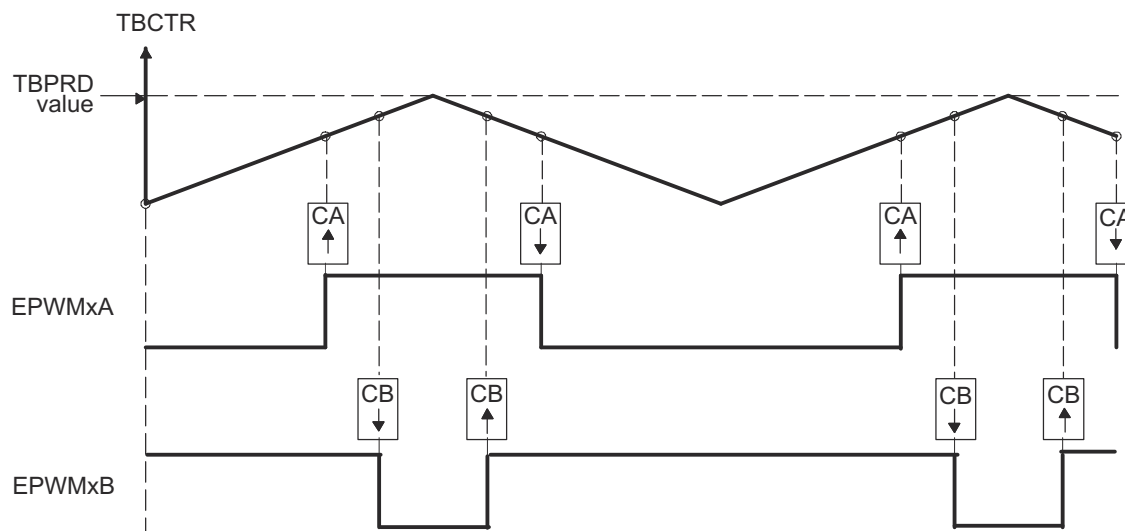
- PWM period =  $2 \times \text{TBPRD} \times T_{\text{TBCLK}}$
- Duty modulation for EPWMxA is set by CMPA, and is active low (that is, the low time duty is proportional to CMPA).
- Duty modulation for EPWMxB is set by CMPB and is active low (that is, the low time duty is proportional to CMPB).
- Outputs EPWMxA and EPWMxB can drive independent power switches

**Figure 3-25. Up-Down-Count, Dual Edge Symmetric Waveform, With Independent Modulation on EPWMxA and EPWMxB — Active Low**

#### Example 3-4. Code Sample for [Figure 3-25](#)

```
// Initialization Time
// =====
EPwm1Regs.TBPRD = 600; // Period = 2'600 TBCLK counts
EPwm1Regs.CMPA.half.CMPA = 400; // Compare A = 400 TBCLK counts
EPwm1Regs.CMPB = 500; // Compare B = 500 TBCLK counts
EPwm1Regs.TBPHS = 0; // Set Phase register to zero
EPwm1Regs.TBCTR = 0; // clear TB counter
EPwm1Regs.TBCTL.bit.CTRMODE = TB_COUNT_UPDOWN; // Symmetric
xEPwm1Regs.TBCTL.bit.PHSEN = TB_DISABLE; // Phase loading disabled
xEPwm1Regs.TBCTL.bit.PRDL = TB_SHADOW;
EPwm1Regs.TBCTL.bit.SYNCSEL = TB_SYNC_DISABLE;
EPwm1Regs.TBCTL.bit.HSPCLKDIV = TB_DIV1; // TBCLK = SYSCLKOUT
EPwm1Regs.TBCTL.bit.CLKDIV = TB_DIV1;
EPwm1Regs.CMPCTL.bit.SHDWAMODE = CC_SHADOW;
EPwm1Regs.CMPCTL.bit.SHDWBMODE = CC_SHADOW;
EPwm1Regs.CMPCTL.bit.LOADAMODE = CC_CTR_ZERO; // load on CTR = Zero
EPwm1Regs.CMPCTL.bit.LOADBMODE = CC_CTR_ZERO; // load on CTR = Zero
EPwm1Regs.AQCTLA.bit.CAU = AQ_SET;
EPwm1Regs.AQCTLA.bit.CAD = AQ_CLEAR;
EPwm1Regs.AQCTLB.bit.CBU = AQ_SET;
EPwm1Regs.AQCTLB.bit.CBD = AQ_CLEAR;
//
// Run Time
// =====
EPwm1Regs.CMPA.half.CMPA = Duty1A; // adjust duty for output EPWM1A
EPwm1Regs.CMPB = Duty1B; // adjust duty for output EPWM1B
```

**Example 3-5** contains a code sample showing initialization and run time for the waveforms in [Figure 3-26](#). Use the constant definitions in the device-specific header file.



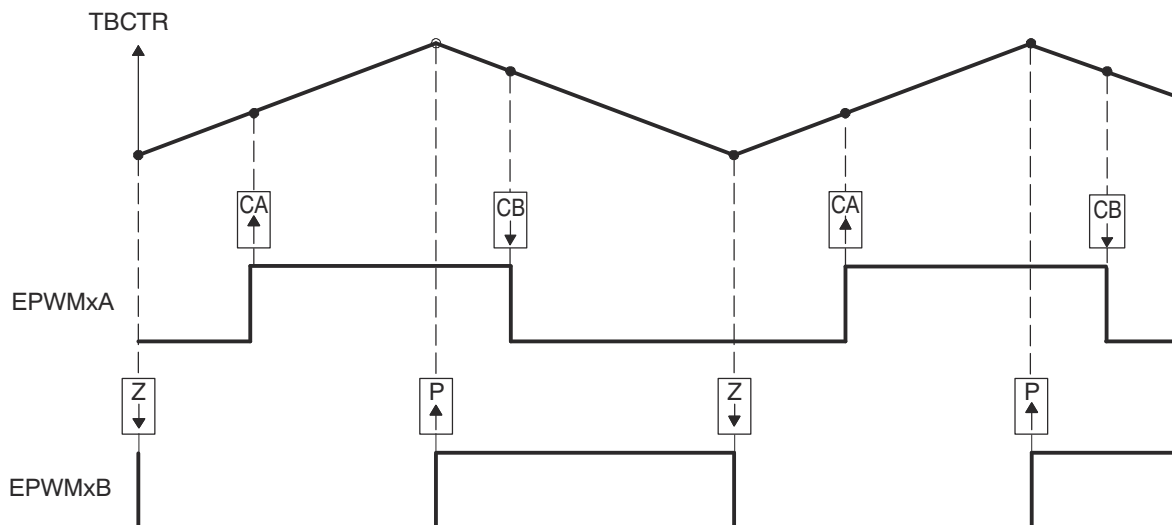
- PWM period =  $2 \times \text{TBPRD} \times T_{\text{TBCLK}}$
- Duty modulation for EPWMxA is set by CMPA, and is active low, that is, low time duty proportional to CMPA
- Duty modulation for EPWMxB is set by CMPB and is active high, that is, high time duty proportional to CMPB
- Outputs EPWMx can drive upper/lower (complementary) power switches
- Dead-band = CMPB - CMPA (fully programmable edge placement by software). Note the dead-band module is also available if the more classical edge delay method is required.

**Figure 3-26. Up-Down-Count, Dual Edge Symmetric Waveform, With Independent Modulation on EPWMxA and EPWMxB — Complementary**

### Example 3-5. Code Sample for [Figure 3-26](#)

```
// Initialization Time
// =====
EPwm1Regs.TBPRD = 600; // Period = 2'600 TBCLK counts
EPwm1Regs.CMPA.half.CMPA = 350; // Compare A = 350 TBCLK counts
EPwm1Regs.CMPB = 400; // Compare B = 400 TBCLK counts
EPwm1Regs.TBPHS = 0; // Set Phase register to zero
EPwm1Regs.TBCTR = 0; // clear TB counter
EPwm1Regs.TBCTL.bit.CTRMODE = TB_COUNT_UPDOWN; // Symmetric
EPwm1Regs.TBCTL.bit.PHSEN = TB_DISABLE; // Phase loading disabled
EPwm1Regs.TBCTL.bit.PRDL = TB_SHADOW;
EPwm1Regs.TBCTL.bit.SYNCSEL = TB_SYNC_DISABLE;
EPwm1Regs.TBCTL.bit.HSPCLKDIV = TB_DIV1; // TBCLK = SYSCLKOUT
EPwm1Regs.TBCTL.bit.CLKDIV = TB_DIV1;
EPwm1Regs.CMPCTL.bit.SHDWAMODE = CC_SHADOW;
EPwm1Regs.CMPCTL.bit.SHDWBMODE = CC_SHADOW;
EPwm1Regs.CMPCTL.bit.LOADAMODE = CC_CTR_ZERO; // load on CTR = Zero
EPwm1Regs.CMPCTL.bit.LOADBMODE = CC_CTR_ZERO; // load on CTR = Zero
EPwm1Regs.AQCTLA.bit.CAU = AQ_SET;
EPwm1Regs.AQCTLA.bit.CAD = AQ_CLEAR;
EPwm1Regs.AQCTLB.bit.CBU = AQ_CLEAR;
EPwm1Regs.AQCTLB.bit.CBD = AQ_SET;
// Run Time
// =====
EPwm1Regs.CMPA.half.CMPA = Duty1A; // adjust duty for output EPWM1A
EPwm1Regs.CMPB = Duty1B; // adjust duty for output EPWM1B
```

**Example 3-6** contains a code sample showing initialization and run time for the waveforms in [Figure 3-27](#). Use the constant definitions in the device-specific header file.



- PWM period =  $2 \times \text{TBPRD} \times \text{TBCLK}$
- Rising edge and falling edge can be asymmetrically positioned within a PWM cycle. This allows for pulse placement techniques.
- Duty modulation for EPWMxA is set by CMPA and CMPB.
- Low time duty for EPWMxA is proportional to (CMPA + CMPB).
- To change this example to active high, CMPA and CMPB actions need to be inverted (that is, Set ! Clear and Clear Set).
- Duty modulation for EPWMxB is fixed at 50% (utilizes spare action resources for EPWMxB)

**Figure 3-27. Up-Down-Count, Dual Edge Asymmetric Waveform, With Independent Modulation on EPWMxA—Active Low**

#### Example 3-6. Code Sample for [Figure 3-27](#)

```
// Initialization Time
// =====
EPwm1Regs.TBPRD = 600;           // Period = 2 * 600 TBCLK counts
EPwm1Regs.CMPA.half.CMPA = 250; // Compare A = 250 TBCLK counts
EPwm1Regs.CMPB = 450;           // Compare B = 450 TBCLK counts
EPwm1Regs.TBPHS = 0;            // Set Phase register to zero
EPwm1Regs.TBCTR = 0;            // clear TB counter
EPwm1Regs.TBCTL.bit.CTRMODE = TB_COUNT_UPDOWN; // Symmetric
EPwm1Regs.TBCTL.bit.PHSEN = TB_DISABLE; // Phase loading disabled
EPwm1Regs.TBCTL.bit.PRDL = TB_SHADOW;
EPwm1Regs.TBCTL.bit.SYNCSEL = TB_SYNC_DISABLE;
EPwm1Regs.TBCTL.bit.HSPCLKDIV = TB_DIV1; // TBCLK = SYSCLKOUT
EPwm1Regs.TBCTL.bit.CLKDIV = TB_DIV1;
EPwm1Regs.CMPCTL.bit.SHDWAMODE = CC_SHADOW;
EPwm1Regs.CMPCTL.bit.SHDWBMODE = CC_SHADOW;
EPwm1Regs.CMPCTL.bit.LOADAMODE = CC_CTR_ZERO; // load on CTR = Zero
EPwm1Regs.CMPCTL.bit.LOADBMODE = CC_CTR_ZERO; // load on CTR = Zero
EPwm1Regs.AQCTLA.bit.CAU = AQ_SET;
EPwm1Regs.AQCTLA.bit.CBD = AQ_CLEAR;
EPwm1Regs.AQCTLB.bit.ZRO = AQ_CLEAR;
EPwm1Regs.AQCTLB.bit.PR = AQ_SET;
// Run Time
// =====
EPwm1Regs.CMPA.half.CMPA = EdgePosA; // adjust duty for output EPWM1A only
EPwm1Regs.CMPB = EdgePosB;
```





### 3.2.5.3 Operational Highlights for the Dead-Band Submodule

The following sections provide the operational highlights. The dead-band submodule has two groups of independent selection options as shown in Figure 3-29.

- **Input Source Selection:** The input signals to the dead-band module are the EPWMxA and EPWMxB output signals from the action-qualifier. In this section they will be referred to as EPWMxA In and EPWMxB In. Using the DBCTL[IN\_MODE] control bits, the signal source for each delay, falling-edge or rising-edge, can be selected:
  - EPWMxA In is the source for both falling-edge and rising-edge delay. This is the default mode.
  - EPWMxA In is the source for falling-edge delay, EPWMxB In is the source for rising-edge delay.
  - EPWMxA In is the source for rising edge delay, EPWMxB In is the source for falling-edge delay.
  - EPWMxB In is the source for both falling-edge and rising-edge delay.
- **Half Cycle Clocking:** The dead-band submodule can be clocked using half cycle clocking to double the resolution (that is, counter clocked at  $2 \times \text{TBCLK}$ ).
- **Output Mode Control:** The output mode is configured by way of the DBCTL[OUT\_MODE] bits. These bits determine if the falling-edge delay, rising-edge delay, neither, or both are applied to the input signals.
- **Polarity Control:** The polarity control (DBCTL[POLSEL]) allows you to specify whether the rising-edge delayed signal and/or the falling-edge delayed signal is to be inverted before being sent out of the dead-band submodule.

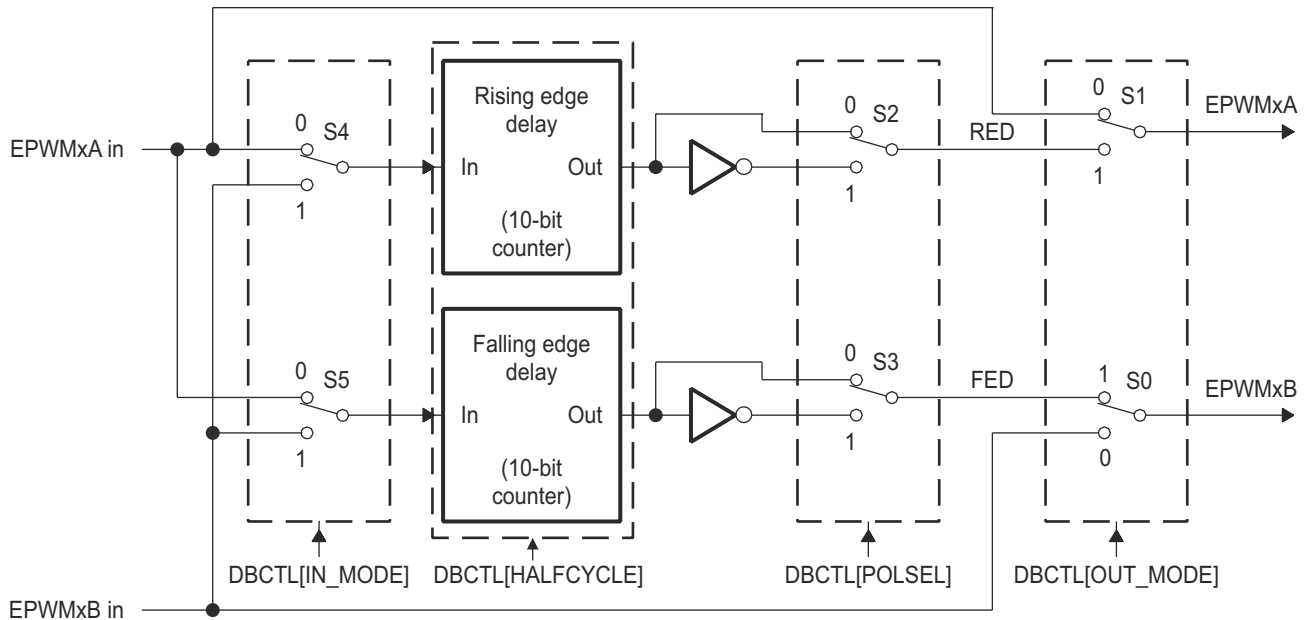


Figure 3-29. Configuration Options for the Dead-Band Submodule

Although all combinations are supported, not all are typical usage modes. Table 3-14 documents some classical dead-band configurations. These modes assume that the DBCTL[IN\_MODE] is configured such that EPWMxA In is the source for both falling-edge and rising-edge delay. Enhanced, or non-traditional modes can be achieved by changing the input signal source. The modes shown in Table 3-14 fall into the following categories:

- **Mode 1: Bypass both falling-edge delay (FED) and rising-edge delay (RED).** Allows you to fully disable the dead-band submodule from the PWM signal path.
- **Mode 2-5: Classical Dead-Band Polarity Settings.** These represent typical polarity configurations that should address all the active high/low modes required by available industry power switch gate drivers. The waveforms for these typical cases are shown in Figure 3-30. Note that to generate equivalent waveforms to Figure 3-30, configure the action-qualifier submodule to generate the signal as shown for EPWMxA.

- **Mode 6: Bypass rising-edge-delay and Mode 7: Bypass falling-edge-delay.** Finally the last two entries in [Table 3-14](#) show combinations where either the falling-edge-delay (FED) or rising-edge-delay (RED) blocks are bypassed.

**Table 3-14. Classical Dead-Band Operating Modes**

| Mode | Mode Description                               | DBCTL[POLSEL] |        | DBCTL[OUT_MODE] |    |
|------|------------------------------------------------|---------------|--------|-----------------|----|
|      |                                                | S3            | S2     | S1              | S0 |
| 1    | EPWMxA and EPWMxB Passed Through (No Delay)    | X             | X      | 0               | 0  |
| 2    | Active High Complementary (AHC)                | 1             | 0      | 1               | 1  |
| 3    | Active Low Complementary (ALC)                 | 0             | 1      | 1               | 1  |
| 4    | Active High (AH)                               | 0             | 0      | 1               | 1  |
| 5    | Active Low (AL)                                | 1             | 1      | 1               | 1  |
| 6    | EPWMxA Out = EPWMxA In (No Delay)              | 0 or 1        | 0 or 1 | 0               | 1  |
|      | EPWMxB Out = EPWMxA In with Falling Edge Delay |               |        |                 |    |
| 7    | EPWMxA Out = EPWMxA In with Rising Edge Delay  | 0 or 1        | 0 or 1 | 1               | 0  |
|      | EPWMxB Out = EPWMxB In with No Delay           |               |        |                 |    |

The dead-band submodule supports independent values for rising-edge (RED) and falling-edge (FED) delays. The amount of delay is programmed using the DBRED and DBFED registers. These are 10-bit registers and their value represents the number of time-base clock, TBCLK, periods a signal edge is delayed by. For example, the formulas to calculate falling-edge-delay and rising-edge-delay are:

$$\text{FED} = \text{DBFED} \times T_{\text{TBCLK}}$$

$$\text{RED} = \text{DBRED} \times T_{\text{TBCLK}}$$

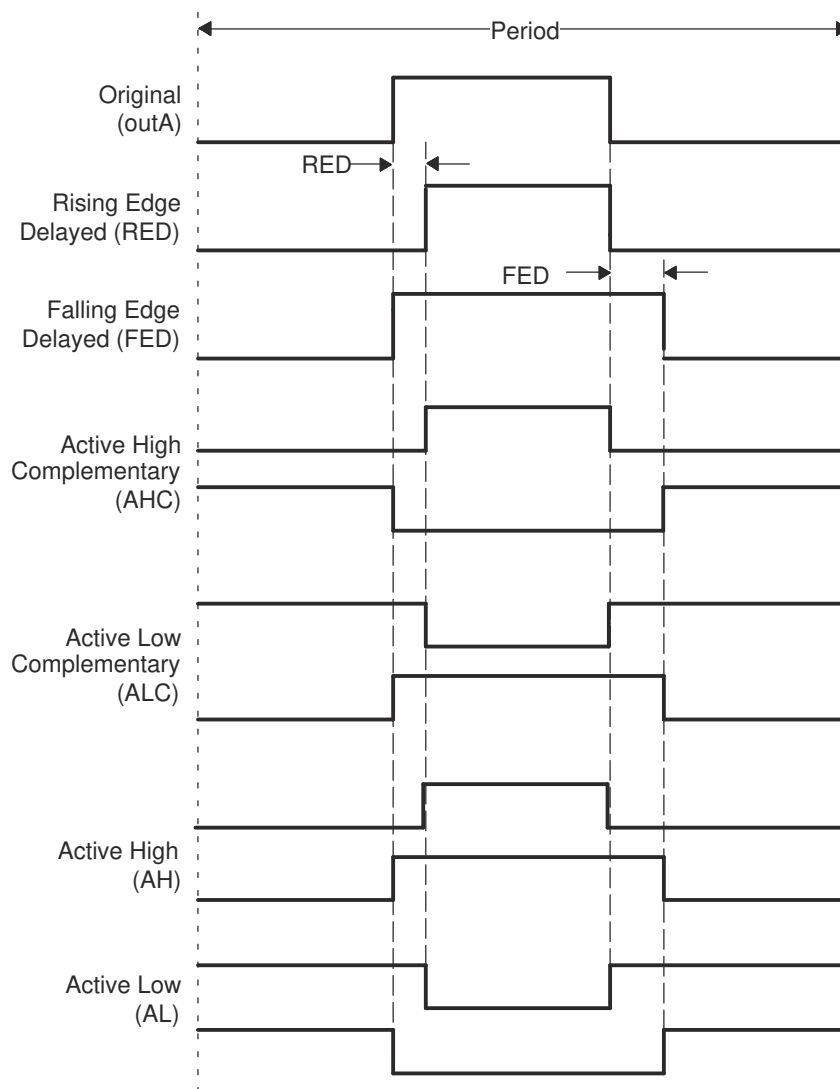
Where  $T_{\text{TBCLK}}$  is the period of TBCLK, the prescaled version of SYSCLKOUT.

When half-cycle clocking is enabled, the formula to calculate the falling-edge-delay and rising-edge-delay becomes:

$$\text{FED} = \text{DBFED} \times T_{\text{TBCLK}}/2$$

$$\text{RED} = \text{DBRED} \times T_{\text{TBCLK}}/2$$

[Figure 3-30](#) shows waveforms for typical cases where  $0\% < \text{duty} < 100\%$ .

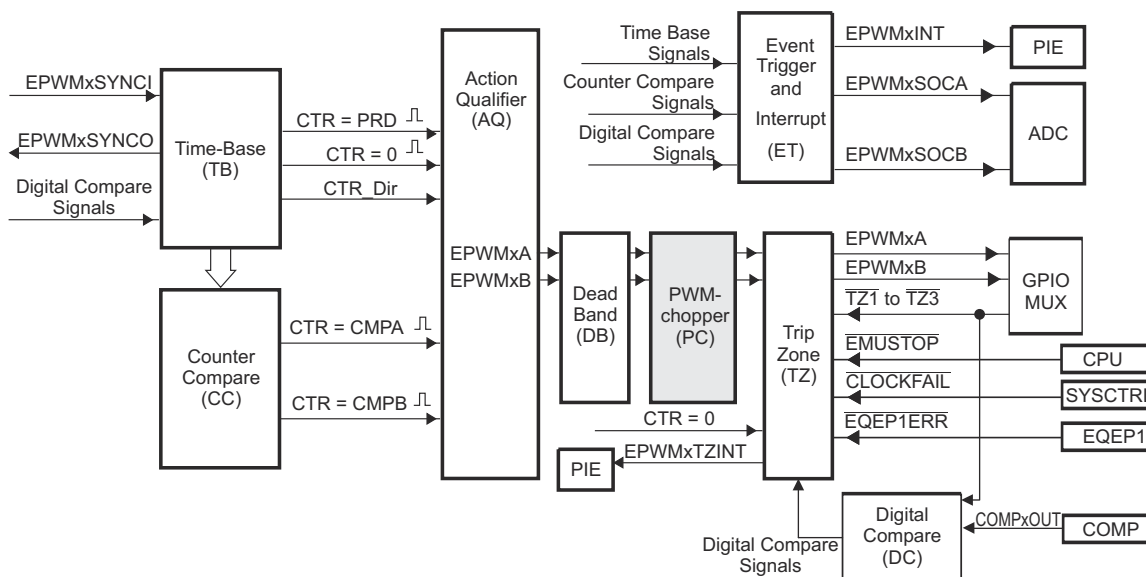


**Figure 3-30. Dead-Band Waveforms for Typical Cases (0% < Duty < 100%)**

### 3.2.6 PWM-Chopper (PC) Submodule

Figure 3-31 shows the PWM-chopper (PC) submodule within the ePWM module.

The PWM-chopper submodule allows a high-frequency carrier signal to modulate the PWM waveform generated by the action-qualifier and dead-band submodules. This capability is important if you need pulse transformer-based gate drivers to control the power switching elements.



**Figure 3-31. PWM-Chopper Submodule**

#### 3.2.6.1 Purpose of the PWM-Chopper Submodule

The key functions of the PWM-chopper submodule are:

- Programmable chopping (carrier) frequency
- Programmable pulse width of first pulse
- Programmable duty cycle of second and subsequent pulses
- Can be fully bypassed if not required

#### 3.2.6.2 Controlling the PWM-Chopper Submodule

The PWM-chopper submodule operation is controlled via the registers in Table 3-15.

**Table 3-15. PWM-Chopper Submodule Registers**

| Register | Address Offset | Shadowed | Description                  | Bit Description                 |
|----------|----------------|----------|------------------------------|---------------------------------|
| PCCTL    | 0x001E         | No       | PWM-chopper Control Register | <a href="#">Section 3.4.7.1</a> |

#### 3.2.6.3 Operational Highlights for the PWM-Chopper Submodule

Figure 3-32 shows the operational details of the PWM-chopper submodule. The carrier clock is derived from SYSCLKOUT. Its frequency and duty cycle are controlled via the CHPFREQ and CHPDUTY bits in the PCCTL register. The one-shot block is a feature that provides a high energy first pulse to ensure hard and fast power switch turn on, while the subsequent pulses sustain pulses, ensuring the power switch remains on. The one-shot width is programmed via the OSHTWTH bits. The PWM-chopper submodule can be fully disabled (bypassed) via the CHPEN bit.

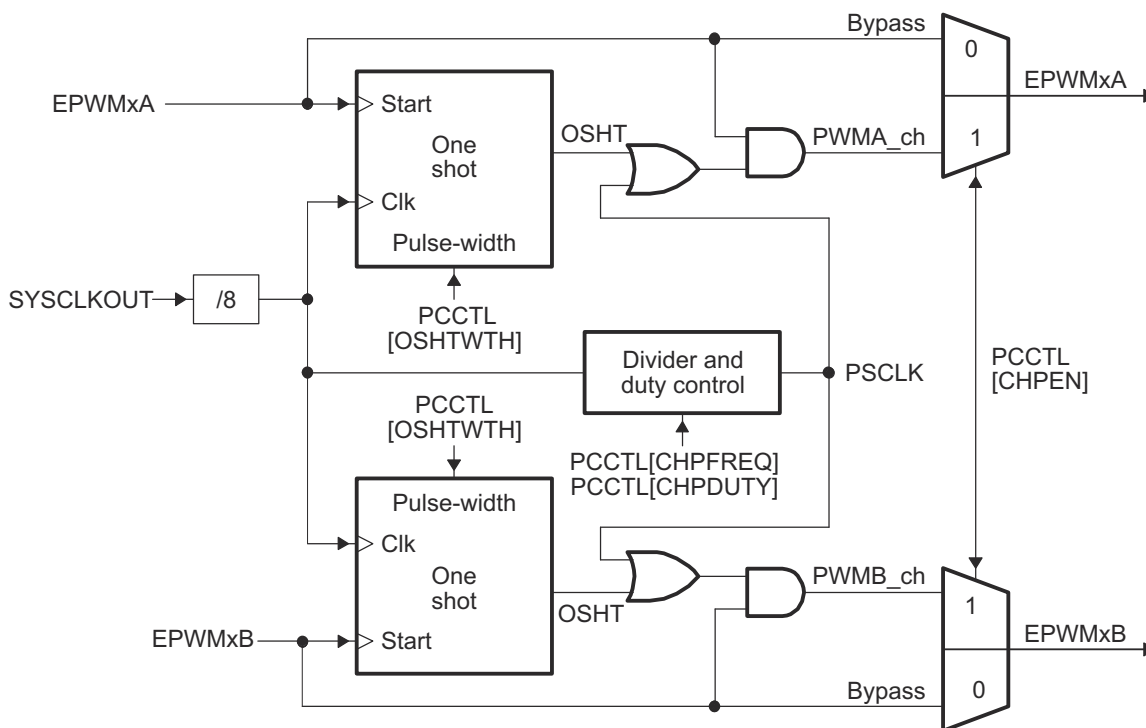


Figure 3-32. PWM-Chopper Submodule Operational Details

### 3.2.6.4 Waveforms

Figure 3-33 shows simplified waveforms of the chopping action only; one-shot and duty-cycle control are not shown. Details of the one-shot and duty-cycle control are discussed in the following sections.

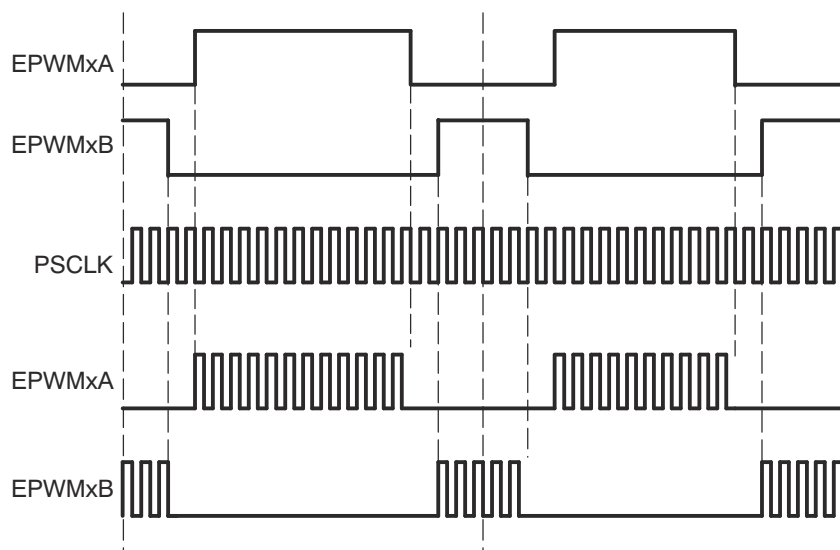


Figure 3-33. Simple PWM-Chopper Submodule Waveforms Showing Chopping Action Only

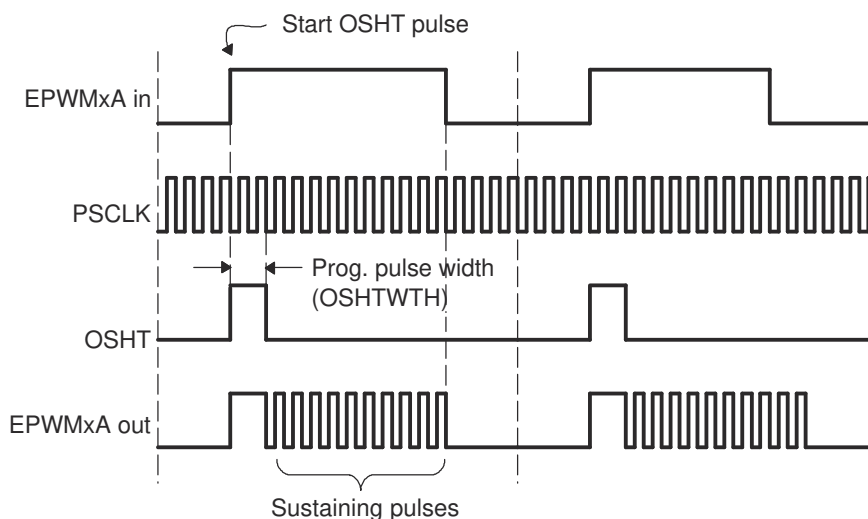
### 3.2.6.4.1 One-Shot Pulse

The width of the first pulse can be programmed to any of 16 possible pulse width values. The width or period of the first pulse is given by:

$$T_{1stpulse} = T_{SYSCLKOUT} \times 8 \times OSHTWTH$$

Where  $T_{SYSCLKOUT}$  is the period of the system clock (SYSCLKOUT) and OSHTWTH is the four control bits (value from 1 to 16).

Figure 3-34 shows the first and subsequent sustaining pulses and Table 3-16 gives the possible pulse width values for a SYSCLKOUT = 90 MHz.



**Figure 3-34. PWM-Chopper Submodule Waveforms Showing the First Pulse and Subsequent Sustaining Pulses**

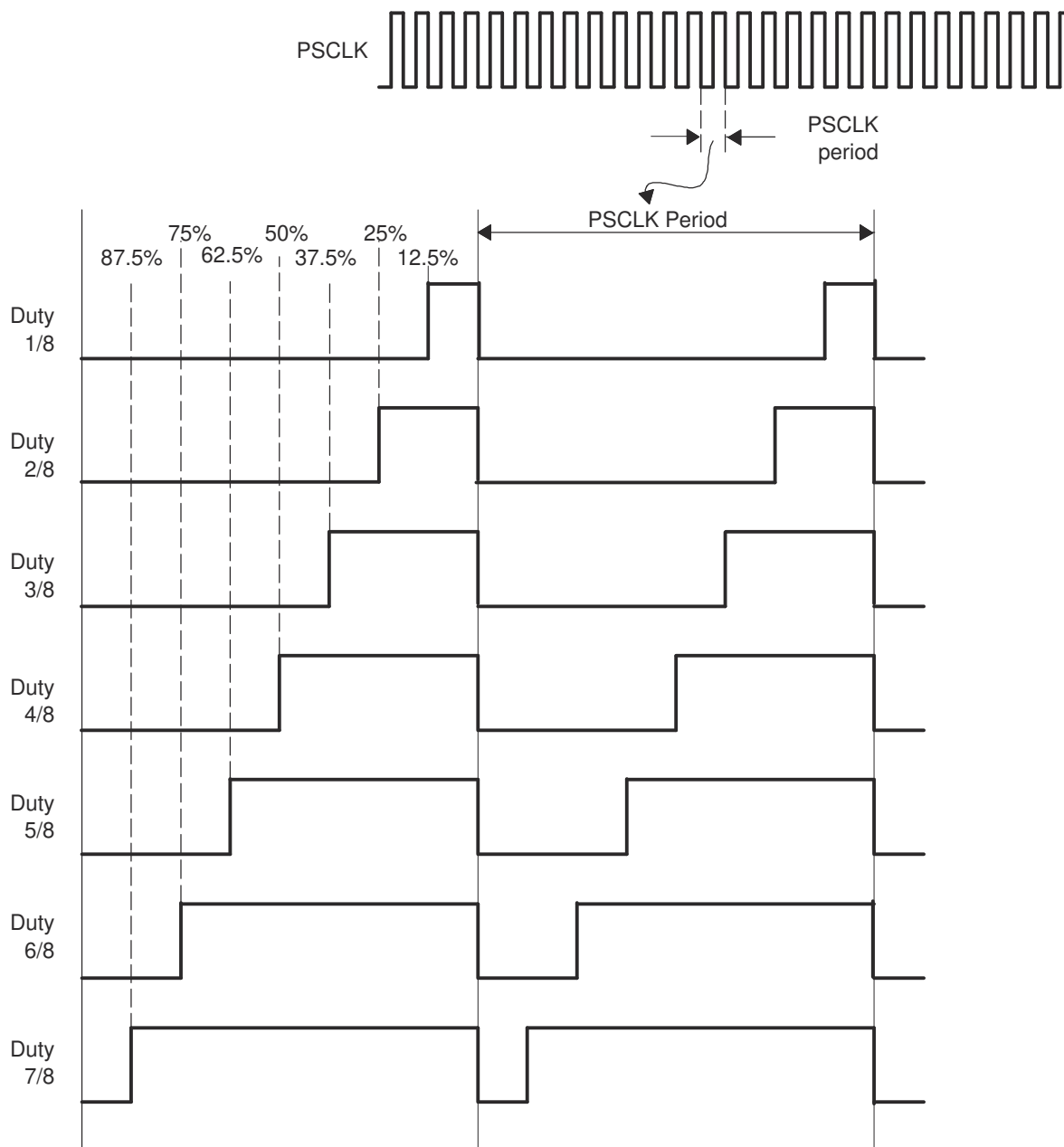
**Table 3-16. Possible Pulse Width Values for  
SYSCLKOUT = 90 MHz**

| OSHTWTHz<br>(hex) | Pulse Width<br>(nS) |
|-------------------|---------------------|
| 0                 | 89                  |
| 1                 | 178                 |
| 2                 | 267                 |
| 3                 | 356                 |
| 4                 | 445                 |
| 5                 | 533                 |
| 6                 | 622                 |
| 7                 | 711                 |
| 8                 | 800                 |
| 9                 | 889                 |
| A                 | 978                 |
| B                 | 1067                |
| C                 | 1156                |
| D                 | 1244                |
| E                 | 1333                |
| F                 | 1422                |

### 3.2.6.4.2 Duty Cycle Control

Pulse transformer-based gate drive designs need to comprehend the magnetic properties or characteristics of the transformer and associated circuitry. Saturation is one such consideration. To assist the gate drive designer, the duty cycles of the second and subsequent pulses have been made programmable. These sustaining pulses ensure the correct drive strength and polarity is maintained on the power switch gate during the on period, and hence a programmable duty cycle allows a design to be tuned or optimized via software control.

Figure 3-35 shows the duty cycle control that is possible by programming the CHPDUTY bits. One of seven possible duty ratios can be selected ranging from 12.5% to 87.5%.

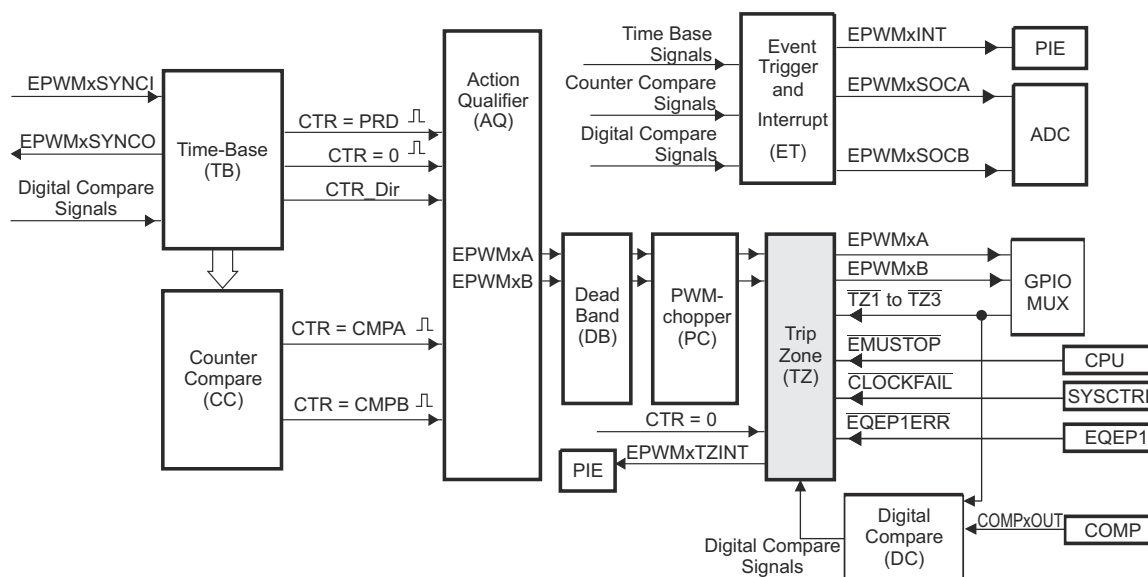


**Figure 3-35. PWM-Chopper Submodule Waveforms Showing the Pulse Width (Duty Cycle) Control of Sustaining Pulses**

### 3.2.7 Trip-Zone (TZ) Submodule

Figure 3-36 shows how the trip-zone (TZ) submodule fits within the ePWM module.

Each ePWM module is connected to six  $\overline{TZn}$  signals ( $\overline{TZ1}$  to  $\overline{TZ6}$ ).  $\overline{TZ1}$  to  $\overline{TZ3}$  are sourced from the GPIO mux.  $\overline{TZ4}$  is sourced from an inverted EQEP1ERR signal on those devices with an EQEP1 module.  $\overline{TZ5}$  is connected to the system clock fail logic, and  $\overline{TZ6}$  is sourced from the EMUSTOP output from the CPU. These signals indicate external fault or trip conditions, and the ePWM outputs can be programmed to respond accordingly when faults occur.



**Figure 3-36. Trip-Zone Submodule**

#### 3.2.7.1 Purpose of the Trip-Zone Submodule

The key functions of the Trip-Zone submodule are:

- Trip inputs  $\overline{TZ1}$  to  $\overline{TZ6}$  can be flexibly mapped to any ePWM module.
- Upon a fault condition, outputs EPWMxA and EPWMxB can be forced to one of the following:
  - High
  - Low
  - High-impedance
  - No action taken
- Support for one-shot trip (OSHT) for major short circuits or over-current conditions.
- Support for cycle-by-cycle tripping (CBC) for current limiting operation.
- Support for digital compare tripping (DC) based on state of on-chip analog comparator module outputs and/or  $\overline{TZ1}$  to  $\overline{TZ3}$  signals.
- Each trip-zone input and digital compare (DC) submodule DCAEVT1/2 or DCBEVT1/2 force event can be allocated to either one-shot or cycle-by-cycle operation.
- Interrupt generation is possible on any trip-zone input.
- Software-forced tripping is also supported.
- The trip-zone submodule can be fully bypassed if it is not required.



### 3.2.7.2 Controlling and Monitoring the Trip-Zone Submodule

The trip-zone submodule operation is controlled and monitored through the following registers:

**Table 3-17. Trip-Zone Submodule Registers**

| Register | Address Offset | Shadowed | Description <sup>(1)</sup>                               | Bit Description                 |
|----------|----------------|----------|----------------------------------------------------------|---------------------------------|
| TZSEL    | 0x0012         | No       | Trip-Zone Select Register                                | <a href="#">Section 3.4.5.1</a> |
| TZDCSEL  | 0x0013         | No       | Trip-zone Digital Compare Select Register <sup>(2)</sup> | <a href="#">Section 3.4.5.2</a> |
| TZCTL    | 0x0014         | No       | Trip-Zone Control Register                               | <a href="#">Section 3.4.5.3</a> |
| TZEINT   | 0x0015         | No       | Trip-Zone Enable Interrupt Register                      | <a href="#">Section 3.4.5.4</a> |
| TZFLG    | 0x0016         | No       | Trip-Zone Flag Register                                  | <a href="#">Section 3.4.5.5</a> |
| TZCLR    | 0x0017         | No       | Trip-Zone Clear Register                                 | <a href="#">Section 3.4.5.6</a> |
| TZFRC    | 0x0018         | No       | Trip-Zone Force Register                                 | <a href="#">Section 3.4.5.7</a> |

(1) All trip-zone registers are EALLOW protected and can be modified only after executing the EALLOW instruction. For more information, see the *System Control and Interrupts* chapter.

(2) This register is discussed in more detail in [Section 3.2.9](#).

### 3.2.7.3 Operational Highlights for the Trip-Zone Submodule

The following sections describe the operational highlights and configuration options for the trip-zone submodule.

The trip-zone signals  $\overline{TZ1}$  to  $\overline{TZ6}$  (also collectively referred to as  $\overline{TZn}$ ) are active low input signals. When one of these signals goes low, or when a DCAEVT1/2 or DCBEVT1/2 force happens based on the TZDCSEL register event selection, it indicates that a trip event has occurred. Each ePWM module can be individually configured to ignore or use each of the trip-zone signals or DC events. Which trip-zone signals or DC events are used by a particular ePWM module is determined by the TZSEL register for that specific ePWM module. The trip-zone signals may or may not be synchronized to the system clock (SYSCLKOUT) and digitally filtered within the GPIO MUX block. A minimum of 3\*TBCLK low pulse width on  $\overline{TZn}$  inputs is sufficient to trigger a fault condition on the ePWM module. If the pulse width is less than this, the trip condition may not be latched by CBC or OST latches. The asynchronous trip makes sure that if clocks are missing for any reason, the outputs can still be tripped by a valid event present on  $\overline{TZn}$  inputs. The GPIOs or peripherals must be appropriately configured. For more information, see the *System Control and Interrupts* chapter.

Each  $\overline{TZn}$  input can be individually configured to provide either a cycle-by-cycle or one-shot trip event for an ePWM module. DCAEVT1 and DCBEVT1 events can be configured to directly trip an ePWM module or provide a one-shot trip event to the module. Likewise, DCAEVT2 and DCBEVT2 events can also be configured to directly trip an ePWM module or provide a cycle-by-cycle trip event to the module. This configuration is determined by the TZSEL[DCAEVT1/2], TZSEL[DCBEVT1/2], TZSEL[CBCn], and TZSEL[OSHTn] control bits (where n corresponds to the trip input) respectively.

- **Cycle-by-Cycle (CBC):** When a cycle-by-cycle trip event occurs, the action specified in the TZCTL[TZA] and TZCTL[TZB] bits is carried out immediately on the EPWMxA and/or EPWMxB output. [Table 3-18](#) lists the possible actions. In addition, the cycle-by-cycle trip event flag (TZFLG[CBC]) is set and a EPWMx\_TZINT interrupt is generated if it is enabled in the TZEINT register and PIE peripheral.

If the CBC interrupt is enabled via the TZEINT register, and DCAEVT2 or DCBEVT2 are selected as CBC trip sources via the TZSEL register, it is not necessary to also enable the DCAEVT2 or DCBEVT2 interrupts in the TZEINT register, as the DC events trigger interrupts through the CBC mechanism.

The specified condition on the inputs is automatically cleared when the ePWM time-base counter reaches zero (TBCTR = 0x0000) if the trip event is no longer present. Therefore, in this mode, the trip event is cleared or reset every PWM cycle. The TZFLG[CBC] flag bit will remain set until it is manually cleared by writing to the TZCLR[CBC] bit. If the cycle-by-cycle trip event is still present when the TZFLG[CBC] bit is cleared, then it will again be immediately set.

- **One-Shot (OSHT):** When a one-shot trip event occurs, the action specified in the TZCTL[TZA] and TZCTL[TZB] bits is carried out immediately on the EPWMxA and/or EPWMxB output. [Table 3-18](#) lists the possible actions. In addition, the one-shot trip event flag (TZFLG[OST]) is set and a EPWMx\_TZINT interrupt is generated if it is enabled in the TZEINT register and PIE peripheral. The one-shot trip condition must be cleared manually by writing to the TZCLR[OST] bit.

If the one-shot interrupt is enabled via the TZEINT register, and DCAEVT1 or DCBEVT1 are selected as OSHT trip sources via the TZSEL register, it is not necessary to also enable the DCAEVT1 or DCBEVT1 interrupts in the TZEINT register, as the DC events trigger interrupts through the OSHT mechanism.

- **Digital Compare Events (DCAEVT1/2 and DCBEVT1/2):** A digital compare DCAEVT1/2 or DCBEVT1/2 event is generated based on a combination of the DCAH/DCAL and DCBH/DCBL signals as selected by the TZDCSEL register. The signals which source the DCAH/DCAL and DCBH/DCBL signals are selected via the DCTRISEL register and can be either trip zone input pins or analog comparator COMPxOUT signals. For more information on the digital compare submodule signals, see [Section 3.2.9](#).

When a digital compare event occurs, the action specified in the TZCTL[DCAEVT1/2] and TZCTL[DCBEVT1/2] bits is carried out immediately on the EPWMxA and/or EPWMxB output. [Table 3-18](#) lists the possible actions. In addition, the relevant DC trip event flag (TZFLG[DCAEVT1/2] / TZFLG[DCBEVT1/2]) is set and a EPWMx\_TZINT interrupt is generated if it is enabled in the TZEINT register and PIE peripheral.

The specified condition on the pins is automatically cleared when the DC trip event is no longer present. The TZFLG[DCAEVT1/2] or TZFLG[DCBEVT1/2] flag bit will remain set until it is manually cleared by writing to the TZCLR[DCAEVT1/2] or TZCLR[DCBEVT1/2] bit. If the DC trip event is still present when the TZFLG[DCAEVT1/2] or TZFLG[DCBEVT1/2] flag is cleared, then it will again be immediately set.

The action taken when a trip event occurs can be configured individually for each of the ePWM output pins by way of the TZCTL register bit fields. One of four possible actions, shown in [Table 3-18](#), can be taken on a trip event.

**Table 3-18. Possible Actions On a Trip Event**

| TZCTL Register Bit Settings | EPWMxA and/or EPWMxB | Comment                                         |
|-----------------------------|----------------------|-------------------------------------------------|
| 0,0                         | High-Impedance       | Tripped                                         |
| 0,1                         | Force to High State  | Tripped                                         |
| 1,0                         | Force to Low State   | Tripped                                         |
| 1,1                         | No Change            | Do Nothing.<br>No change is made to the output. |

### Example 3-7. Trip-Zone Configurations

#### Scenario A:

A one-shot trip event on  $\overline{TZ1}$  pulls both EPWM1A, EPWM1B low and also forces EPWM2A and EPWM2B high.

- Configure the ePWM1 registers as follows:
  - TZSEL[OSHT1] = 1: enables  $\overline{TZ1}$  as a one-shot event source for ePWM1
  - TZCTL[TZA] = 2: EPWM1A will be forced low on a trip event.
  - TZCTL[TZB] = 2: EPWM1B will be forced low on a trip event.
- Configure the ePWM2 registers as follows:
  - TZSEL[OSHT1] = 1: enables  $\overline{TZ1}$  as a one-shot event source for ePWM2
  - TZCTL[TZA] = 1: EPWM2A will be forced high on a trip event.
  - TZCTL[TZB] = 1: EPWM2B will be forced high on a trip event.

#### Scenario B:

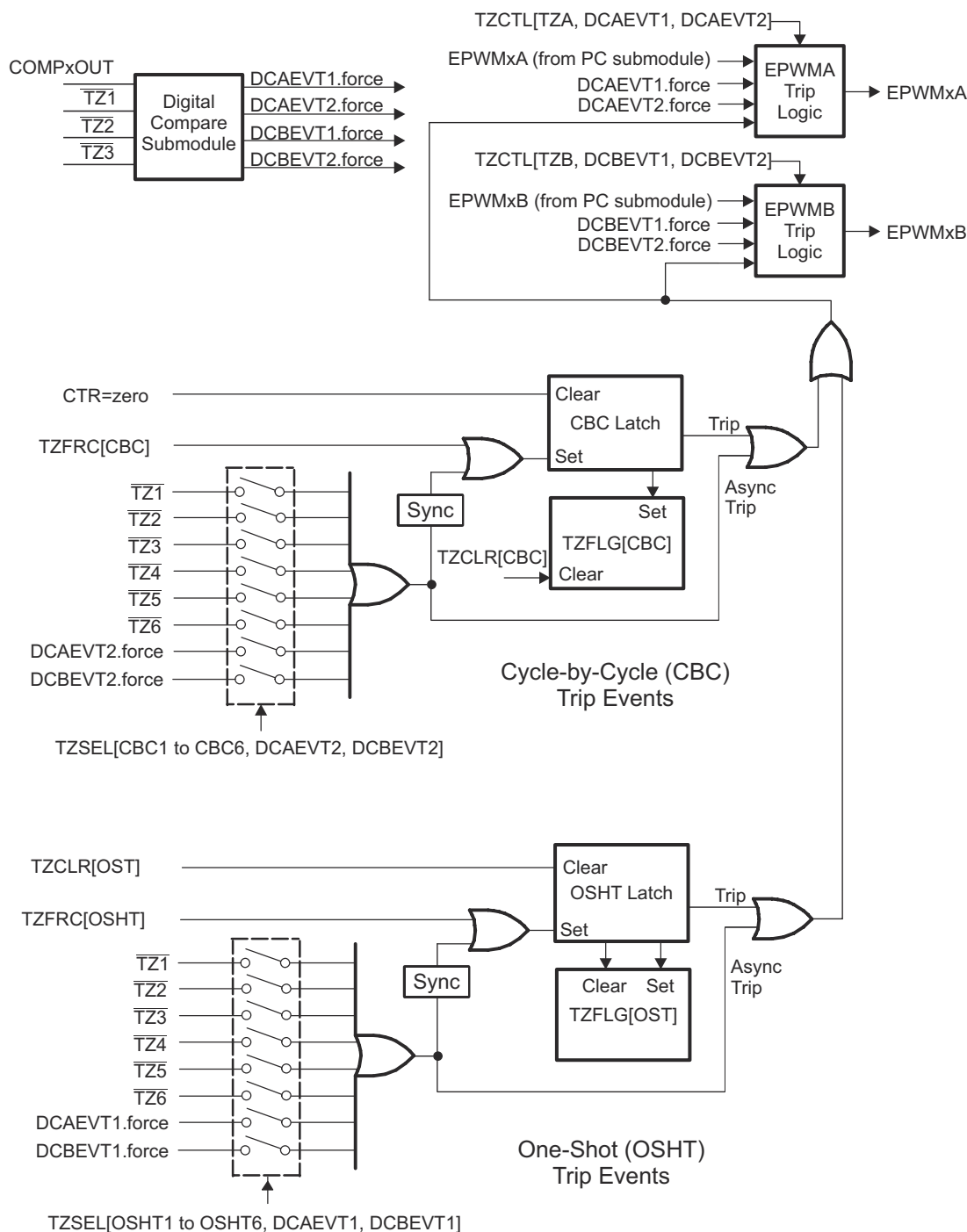
A cycle-by-cycle event on  $\overline{TZ5}$  pulls both EPWM1A, EPWM1B low.

A one-shot event on  $\overline{TZ1}$  or  $\overline{TZ6}$  puts EPWM2A into a high impedance state.

- Configure the ePWM1 registers as follows:
  - TZSEL[CBC5] = 1: enables  $\overline{TZ5}$  as a one-shot event source for ePWM1
  - TZCTL[TZA] = 2: EPWM1A will be forced low on a trip event.
  - TZCTL[TZB] = 2: EPWM1B will be forced low on a trip event.
- Configure the ePWM2 registers as follows:
  - TZSEL[OSHT1] = 1: enables  $\overline{TZ1}$  as a one-shot event source for ePWM2
  - TZSEL[OSHT6] = 1: enables  $\overline{TZ6}$  as a one-shot event source for ePWM2
  - TZCTL[TZA] = 0: EPWM2A will be put into a high-impedance state on a trip event.
  - TZCTL[TZB] = 3: EPWM2B will ignore the trip event.

### 3.2.7.4 Generating Trip Event Interrupts

Figure 3-37 and Figure 3-38 illustrate the trip-zone submodule control and interrupt logic, respectively. DCAEVT1/2 and DCBEVT1/2 signals are described in further detail in Section 3.2.9.



**Figure 3-37. Trip-Zone Submodule Mode Control Logic**

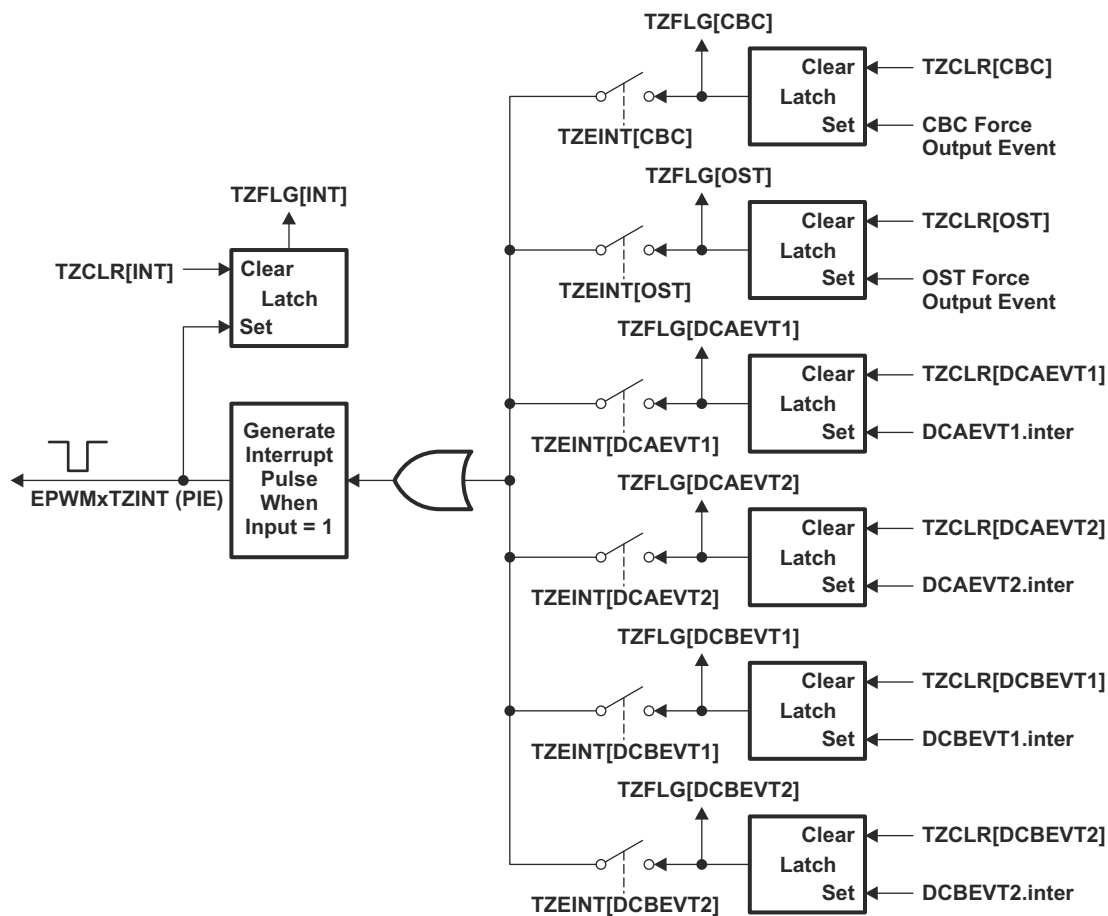


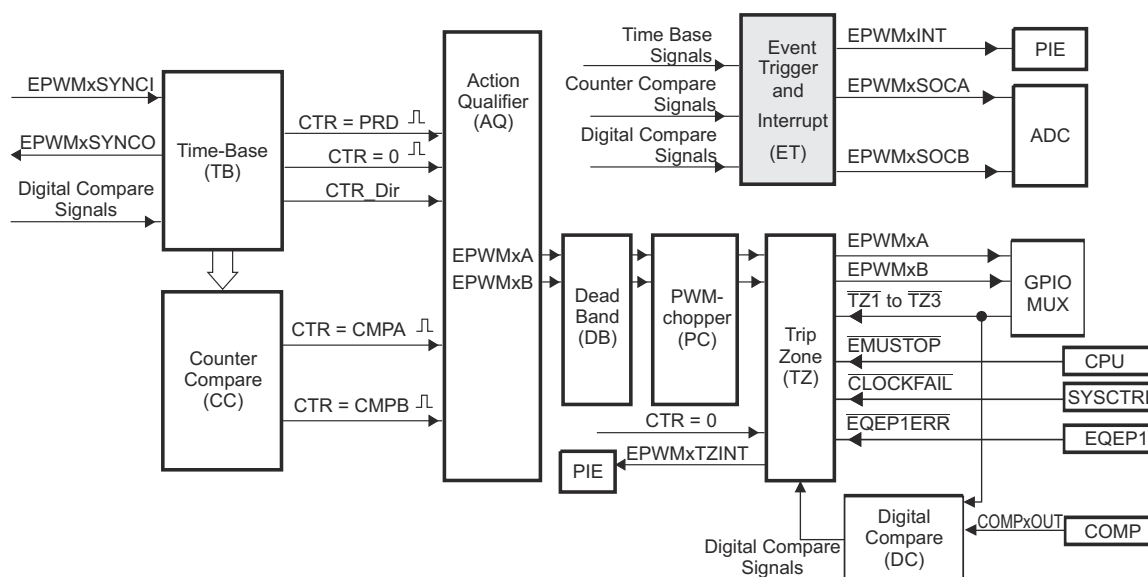
Figure 3-38. Trip-Zone Submodule Interrupt Logic

### 3.2.8 Event-Trigger (ET) Submodule

The key functions of the event-trigger submodule are:

- Receives event inputs generated by the time-base, counter-compare, and digital-compare submodules
- Uses the time-base direction information for up/down event qualification
- Uses prescaling logic to issue interrupt requests and ADC start of conversion at:
  - Every event
  - Every second event
  - Every third event
- Provides full visibility of event generation via event counters and flags
- Allows software forcing of Interrupts and ADC start of conversion

The event-trigger submodule manages the events generated by the time-base submodule, the counter-compare submodule, and the digital-compare submodule to generate an interrupt to the CPU and/or a start of conversion pulse to the ADC when a selected event occurs. Figure 3-39 illustrates where the event-trigger submodule fits within the ePWM system.

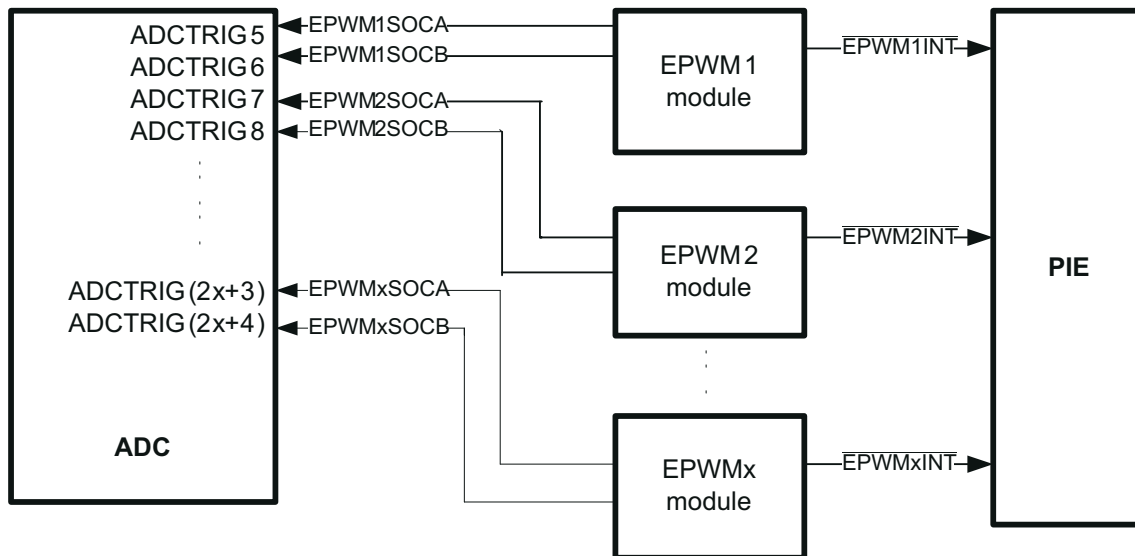


**Figure 3-39. Event-Trigger Submodule**

#### 3.2.8.1 Purpose of the Event-Trigger Submodule

The following sections describe the event-trigger submodule.

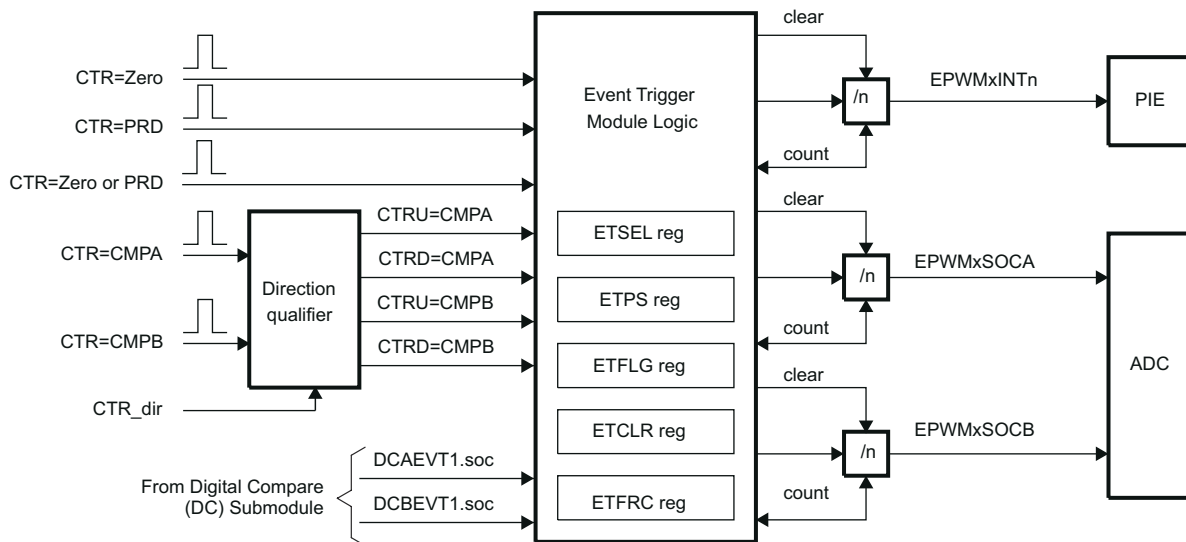
Each ePWM module has one interrupt request line connected to the PIE and two start of conversion signals connected to the ADC module. As shown in Figure 3-40, ADC start of conversion for all ePWM modules are connected to individual ADC trigger inputs to the ADC, and hence multiple modules can initiate an ADC start of conversion via the ADC trigger inputs.



**Figure 3-40. Event-Trigger Submodule Inter-Connectivity of ADC Start of Conversion**

The event-trigger submodule monitors various event conditions (the left side inputs to event-trigger submodule shown in [Figure 3-41](#)) and can be configured to prescale these events before issuing an Interrupt request or an ADC start of conversion. The event-trigger prescaling logic can issue Interrupt requests and ADC start of conversion at:

- Every event
- Every second event
- Every third event



**Figure 3-41. Event-Trigger Submodule Showing Event Inputs and Prescaled Outputs**

### 3.2.8.2 Controlling and Monitoring the Event-Trigger Submodule

The key registers used to configure the event-trigger submodule are shown in [Table 3-19](#).

**Table 3-19. Event-Trigger Submodule Registers**

| Register | Address Offset | Shadowed | Description                      | Bit Description                 |
|----------|----------------|----------|----------------------------------|---------------------------------|
| ETSEL    | 0x0019         | No       | Event-trigger Selection Register | <a href="#">Section 3.4.6.1</a> |
| ETPS     | 0x001A         | No       | Event-trigger Prescale Register  | <a href="#">Section 3.4.6.2</a> |
| ETFLG    | 0x001B         | No       | Event-trigger Flag Register      | <a href="#">Section 3.4.6.3</a> |
| ETCLR    | 0x001C         | No       | Event-trigger Clear Register     | <a href="#">Section 3.4.6.4</a> |
| ETFRC    | 0x001D         | No       | Event-trigger Force Register     | <a href="#">Section 3.4.6.5</a> |

- ETSEL—This selects which of the possible events will trigger an interrupt or start an ADC conversion
- ETPS—This programs the event prescaling options mentioned above.
- ETFLG—These are flag bits indicating status of the selected and prescaled events.
- ETCLR—These bits allow you to clear the flag bits in the ETFLG register via software.
- ETFRC—These bits allow software forcing of an event. Useful for debugging or s/w intervention.

### 3.2.8.3 Operational Highlights for the Event-Trigger Submodule

A more detailed look at how the various register bits interact with the Interrupt and ADC start of conversion logic are shown in [Figure 3-42](#), [Figure 3-43](#), and [Figure 3-44](#).

[Figure 3-42](#) shows the event-trigger's interrupt generation logic. The interrupt-period (ETPS[INTPRD]) bits specify the number of events required to cause an interrupt pulse to be generated. The choices available are:

- Do not generate an interrupt.
- Generate an interrupt on every event
- Generate an interrupt on every second event
- Generate an interrupt on every third event

Which event can cause an interrupt is configured by the interrupt selection (ETSEL[INTSEL]) bits. The event can be one of the following:

- Time-base counter equal to zero (TBCTR = 0x0000).
- Time-base counter equal to period (TBCTR = TBPRD).
- Time-base counter equal to zero or period (TBCTR = 0x0000 || TBCTR = TBPRD)
- Time-base counter equal to the compare A register (CMPA) when the timer is incrementing.
- Time-base counter equal to the compare A register (CMPA) when the timer is decrementing.
- Time-base counter equal to the compare B register (CMPB) when the timer is incrementing.
- Time-base counter equal to the compare B register (CMPB) when the timer is decrementing.

The number of events that have occurred can be read from the interrupt event counter (ETPS[INTCNT]) register bits. That is, when the specified event occurs the ETPS[INTCNT] bits are incremented until they reach the value specified by ETPS[INTPRD]. When ETPS[INTCNT] = ETPS[INTPRD] the counter stops counting and its output is set. The counter is only cleared when an interrupt is sent to the PIE.

When ETPS[INTCNT] reaches ETPS[INTPRD] the following behaviors will occur:

- If interrupts are enabled, ETSEL[INTEN] = 1 and the interrupt flag is clear, ETFLG[INT] = 0, then an interrupt pulse is generated and the interrupt flag is set, ETFLG[INT] = 1, and the event counter is cleared ETPS[INTCNT] = 0. The counter will begin counting events again.
- If interrupts are disabled, ETSEL[INTEN] = 0, or the interrupt flag is set, ETFLG[INT] = 1, the counter stops counting events when it reaches the period value ETPS[INTCNT] = ETPS[INTPRD].
- If interrupts are enabled, but the interrupt flag is already set, then the counter will hold its output high until the ETFLG[INT] flag is cleared. This allows for one interrupt to be pending while one is serviced.



When writing INTPRD values the following occur:

- Writing to the INTPRD bits will automatically clear the counter  $INTCNT = 0$  and the counter output will be reset (so no interrupts are generated).
- Writing an INTPRD value that is GREATER or equal to the current counter value will reset the  $INTCNT = 0$ .
- Writing an INTPRD value that is equal to the current counter value will trigger an interrupt if it is enabled and the status flag is cleared (and  $INTCNT$  will also be cleared to 0)
- Writing an INTPRD value that is LESS than the current counter value will result in undefined behavior (that is,  $INTCNT$  stops counting because INTPRD is below  $INTCNT$ , and interrupt will never fire).
- Writing a 1 to the ETFRC[INT] bit will increment the event counter  $INTCNT$ . The counter will behave as described above when  $INTCNT = INTPRD$ .
- When  $INTPRD = 0$ , the counter is disabled and hence no events will be detected and the ETFRC[INT] bit is also ignored.

The above definition means that you can generate an interrupt on every event, on every second event, or on every third event. An interrupt cannot be generated on every fourth or more events.

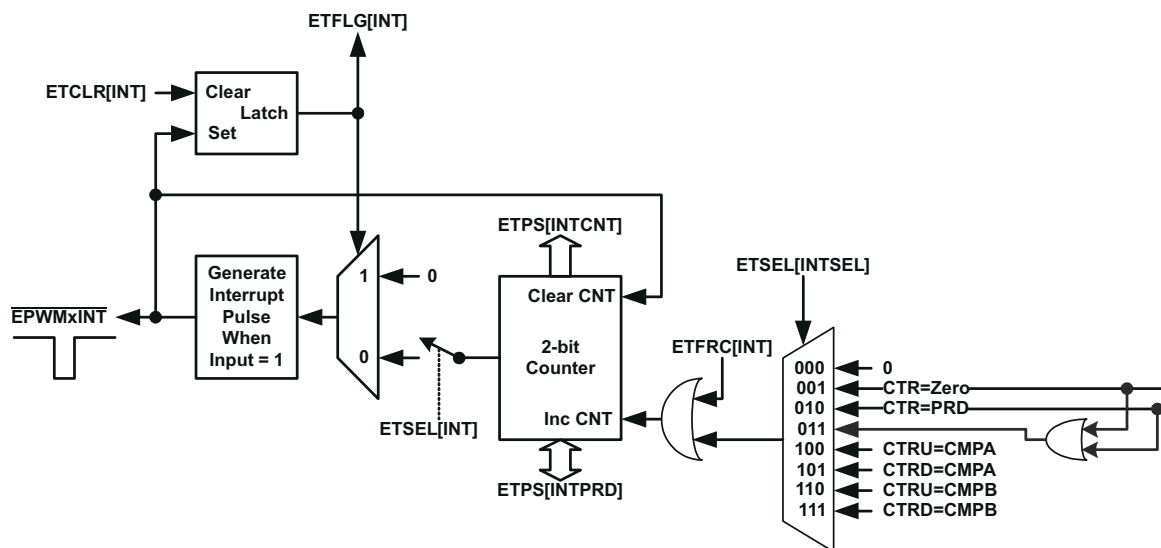
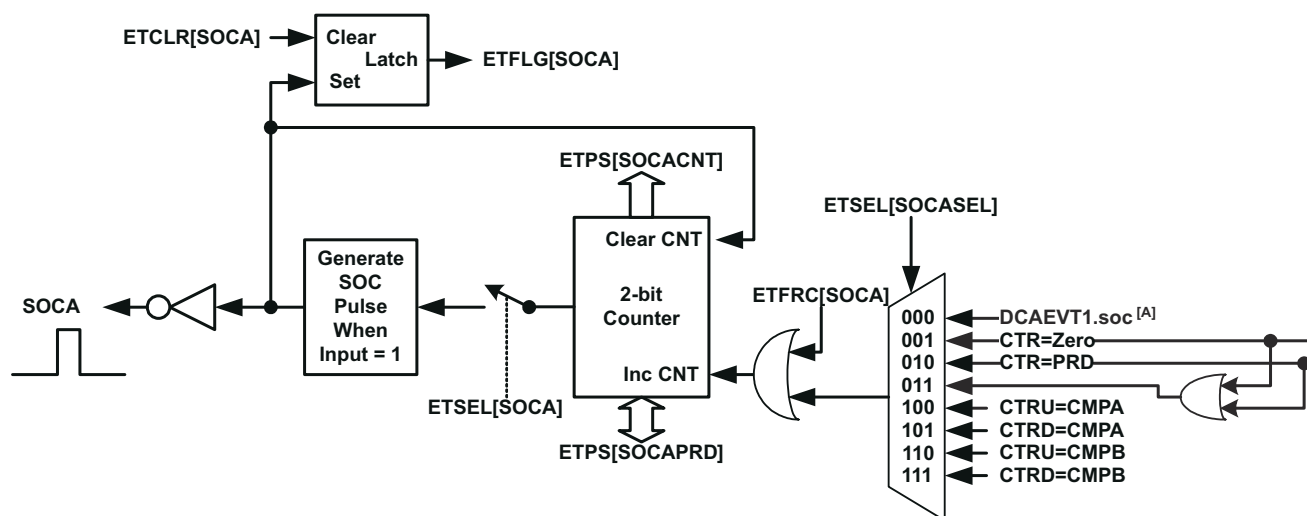


Figure 3-42. Event-Trigger Interrupt Generator

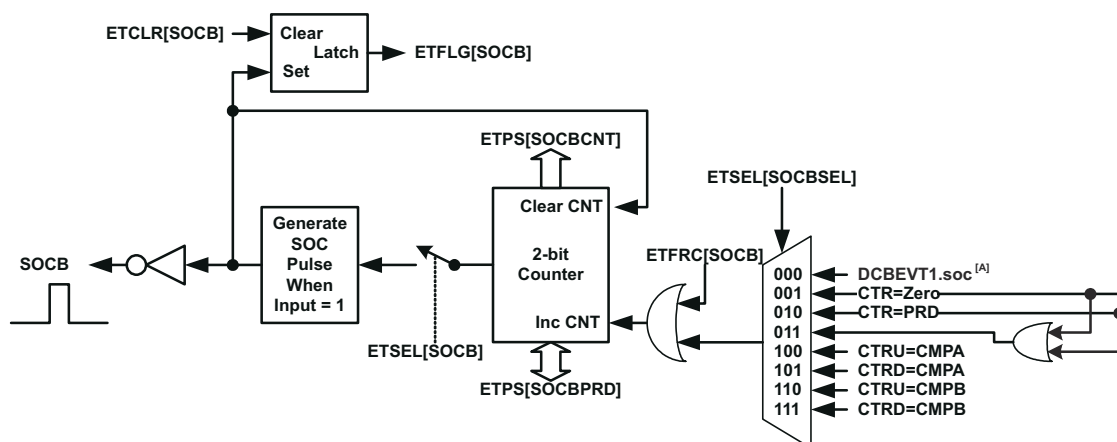
Figure 3-43 shows the operation of the event-trigger's start-of-conversion-A (SOCA) pulse generator. The ETPS[SOCACNT] counter and ETPS[SOCAPRD] period values behave similarly to the interrupt generator except that the pulses are continuously generated. That is, the pulse flag ETFLG[SOCA] is latched when a pulse is generated, but it does not stop further pulse generation. The enable/disable bit ETSEL[SOCAEN] stops pulse generation, but input events can still be counted until the period value is reached as with the interrupt generation logic. The event that will trigger an SOCA and SOCB pulse can be configured separately in the ETSEL[SOCASEL] and ETSEL[SOCBSEL] bits. The possible events are the same events that can be specified for the interrupt generation logic with the addition of the DCAEVT1.soc and DCBEVT1.soc event signals from the digital compare (DC) submodule.



A. The DCAEVT1.soc signals are signals generated by the Digital compare (DC) submodule described later in [Section 3.2.9](#)

**Figure 3-43. Event-Trigger SOCA Pulse Generator**

Figure 3-44 shows the operation of the event-trigger's start-of-conversion-B (SOCB) pulse generator. The event-trigger's SOCB pulse generator operates the same way as the SOCA.



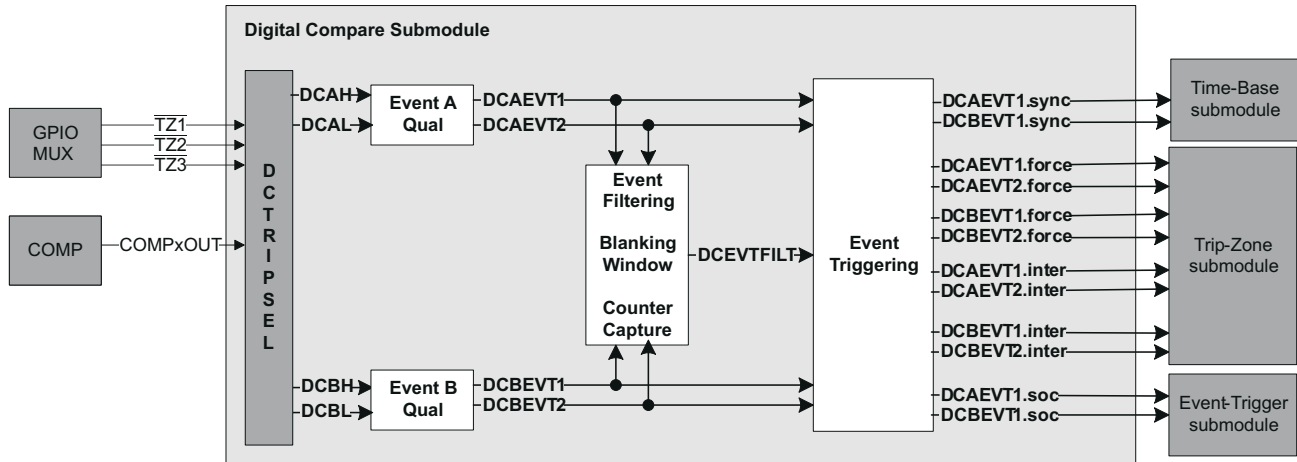
A. The DCBEVT1.soc signals are signals generated by the Digital compare (DC) submodule described later in [Section 3.2.9](#)

**Figure 3-44. Event-Trigger SOCB Pulse Generator**

### 3.2.9 Digital Compare (DC) Submodule

Figure 3-45 illustrates where the digital compare (DC) submodule signals interface to other submodules in the ePWM system.

The digital compare (DC) submodule compares signals external to the ePWM module (for instance, COMPxOUT signals from the analog comparators) to directly generate PWM events/actions which then feed to the event-trigger, trip-zone, and time-base submodules. Additionally, blanking window functionality is supported to filter noise or unwanted pulses from the DC event signals.



**Figure 3-45. Digital-Compare Submodule High-Level Block Diagram**

#### 3.2.9.1 Purpose of the Digital Compare Submodule

The key functions of the digital compare submodule are:

- Analog Comparator (COMP) module outputs and  $\overline{TZ1}$ ,  $\overline{TZ2}$ , and  $\overline{TZ3}$  inputs generate Digital Compare A High/Low (DCAH, DCAL) and Digital Compare B High/Low (DCBH, DCBL) signals.
- DCAH/L and DCBH/L signals trigger events which can then either be filtered or fed directly to the trip-zone, event-trigger, and time-base submodules to:
  - generate a trip zone interrupt
  - generate an ADC start of conversion
  - force an event
  - generate a synchronization event for synchronizing the ePWM module TBCTR.
- Event filtering (blanking window logic) can optionally blank the input signal to remove noise.

### 3.2.9.2 Controlling and Monitoring the Digital Compare Submodule

The digital compare submodule operation is controlled and monitored through the following registers.

**Table 3-20. Digital Compare Submodule Registers**

| Register                   | Address Offset | Shadowed | Description                                    | Bit Description                  |
|----------------------------|----------------|----------|------------------------------------------------|----------------------------------|
| TZDCSEL <sup>(1) (2)</sup> | 0x13           | No       | Trip Zone Digital Compare Select Register      | <a href="#">Section 3.4.5.2</a>  |
| DCTRISEL <sup>(2)</sup>    | 0x30           | No       | Digital Compare Trip Select Register           | <a href="#">Section 3.4.8.1</a>  |
| DCACTL <sup>(2)</sup>      | 0x31           | No       | Digital Compare A Control Register             | <a href="#">Section 3.4.8.2</a>  |
| DCBCTL <sup>(2)</sup>      | 0x32           | No       | Digital Compare B Control Register             | <a href="#">Section 3.4.8.3</a>  |
| DCFCTL <sup>(2)</sup>      | 0x33           | No       | Digital Compare Filter Control Register        | <a href="#">Section 3.4.8.4</a>  |
| DCCAPCTL <sup>(2)</sup>    | 0x34           | No       | Digital Compare Capture Control Register       | <a href="#">Section 3.4.8.5</a>  |
| DCOFFSET                   | 0x35           | Writes   | Digital Compare Filter Offset Register         | <a href="#">Section 3.4.8.6</a>  |
| DCOFFSETCNT                | 0x36           | No       | Digital Compare Filter Offset Counter Register | <a href="#">Section 3.4.8.7</a>  |
| DCFWINDOW                  | 0x37           | No       | Digital Compare Filter Window Register         | <a href="#">Section 3.4.8.8</a>  |
| DCFWINDOWCNT               | 0x38           | No       | Digital Compare Filter Window Counter Register | <a href="#">Section 3.4.8.9</a>  |
| DCCAP                      | 0x39           | Yes      | Digital Compare Counter Capture Register       | <a href="#">Section 3.4.8.10</a> |

(1) The TZDCSEL register is part of the trip-zone submodule but is shown here because of its functional significance to the digital compare submodule.

(2) These registers are EALLOW protected and can be modified only after executing the EALLOW instruction. For more information, see the *System Control and Interrupts* chapter.

### 3.2.9.3 Operation Highlights of the Digital Compare Submodule

The following sections describe the operational highlights and configuration options for the digital compare submodule.

#### 3.2.9.3.1 Digital Compare Events

As shown in [Figure 3-45](#) earlier in this section, trip zone inputs ( $\overline{TZ1}$ ,  $\overline{TZ2}$ , and  $\overline{TZ3}$ ) and COMPxOUT signals from the analog comparator (COMP) module can be selected via the DCTRISEL bits to generate the Digital Compare A High and Low (DCAH/L) and Digital Compare B High and Low (DCBH/L) signals. Then, the configuration of the TZDCSEL register qualifies the actions on the selected DCAH/L and DCBH/L signals, which generate the DCAEVT1/2 and DCBEVT1/2 events (Event Qualification A and B).

#### Note

The  $\overline{TZn}$  signals, when used as a DCEVT tripping functions, are treated as a normal input signal and can be defined to be active high or active low inputs. EPWM outputs are asynchronously tripped when either the  $\overline{TZn}$ , DCAEVTx.force, or DCBEVTx.force signals are active. For the condition to remain latched, a minimum of 3\*TBCLK sync pulse width is required. If pulse width is < 3\*TBCLK sync pulse width, the trip condition may or may not get latched by CBC or OST latches.

The DCAEVT1/2 and DCBEVT1/2 events can then be filtered to provide a filtered version of the event signals (DCEVTFILT) or the filtering can be bypassed. Filtering is discussed further in [Section 3.2.9.3.2](#). Either the DCAEVT1/2 and DCBEVT1/2 event signals or the filtered DCEVTFILT event signals can generate a force to the trip zone module, a TZ interrupt, an ADC SOC, or a PWM sync signal.

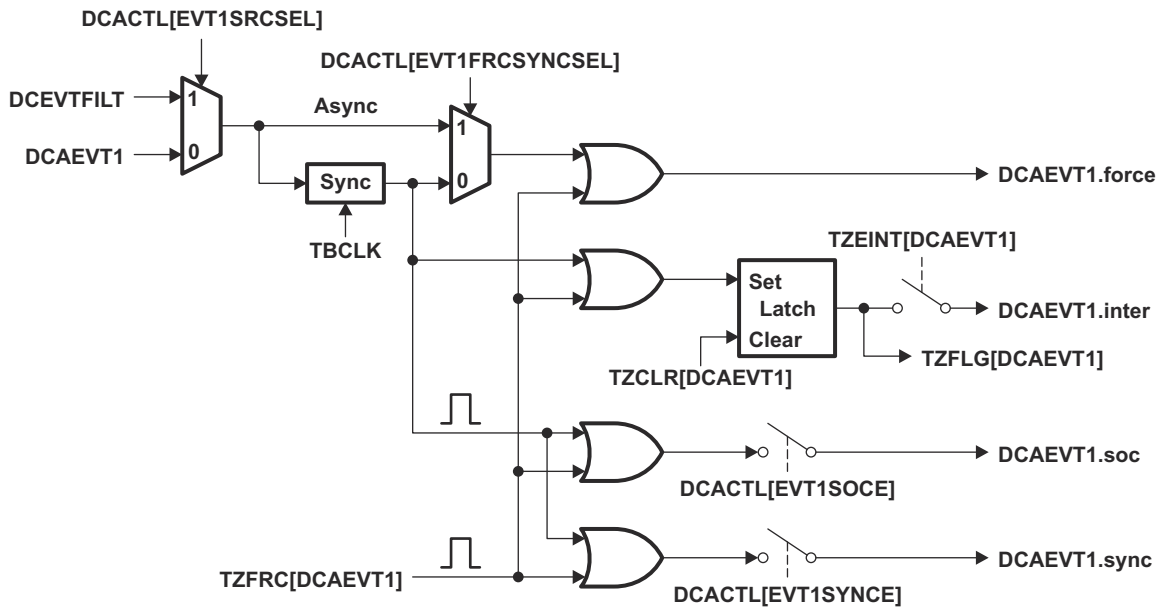
- **force signal:** DCAEVT1/2.force signals force trip zone conditions which either directly influence the output on the EPWMxA pin (via TZCTL[DCAEVT1 or DCAEVT2] configurations) or, if the DCAEVT1/2 signals are selected as one-shot or cycle-by-cycle trip sources (via the TZSEL register), the DCAEVT1/2.force signals can effect the trip action via the TZCTL[TZA] configuration. The DCBEVT1/2.force signals behaves similarly, but affect the EPWMxB output pin instead of the EPWMxA output pin.

The priority of conflicting actions on the TZCTL register is as follows (highest priority overrides lower priority):

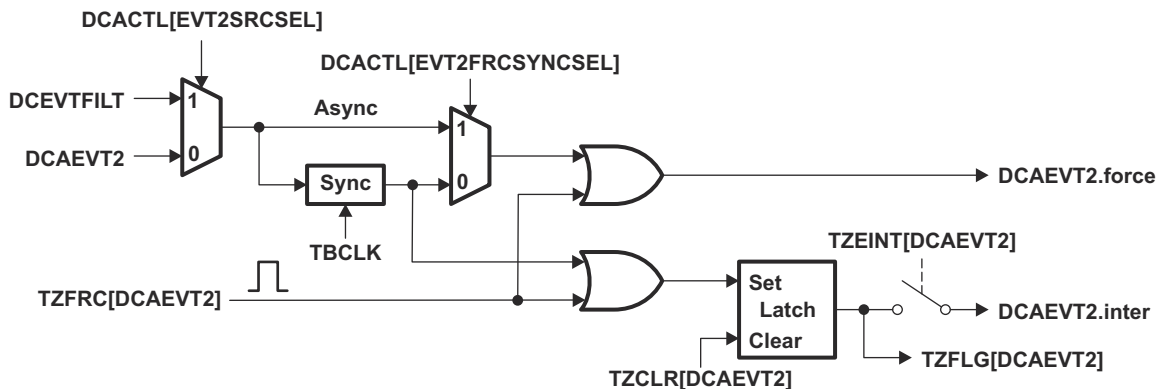
- Output EPWMxA: TZA (highest) -> DCAEVT1 -> DCAEVT2 (lowest)
- Output EPWMxB: TzB (highest) -> DCBEVT1 -> DCBEVT2 (lowest)

- **interrupt signal:** DCAEVT1/2.interrupt signals generate trip zone interrupts to the PIE. To enable the interrupt, the user must set the DCAEVT1, DCAEVT2, DCBEVT1, or DCBEVT2 bits in the TZEINT register. Once one of these events occurs, an EPWMxTZINT interrupt is triggered, and the corresponding bit in the TZCLR register must be set in order to clear the interrupt.
- **soc signal:** The DCAEVT1.soc signal interfaces with the event-trigger submodule and can be selected as an event which generates an ADC start-of-conversion-A (SOCA) pulse via the ETSEL[SOCASEL] bit. Likewise, the DCBEVT1.soc signal can be selected as an event which generates an ADC start-of-conversion-B (SOCB) pulse via the ETSEL[SOCBSEL] bit.
- **sync signal:** The DCAEVT1.sync and DCBEVT1.sync events are ORed with the EPWMxSYNCl input signal and the TBCTL[SWFSYNC] signal to generate a synchronization pulse to the time-base counter.

Figure 3-46 and Figure 3-47 show how the DCAEVT1, DCAEVT2, and DCEVTFILT signals are processed to generate the digital compare A event force, interrupt, soc, and sync signals.



### Figure 3-46. DCAEVT1 Event Triggering



### Figure 3-47. DCAEVT2 Event Triggering

Figure 3-48 and Figure 3-49 show how the DCBEVT1, DCBEVT2, and DCEVTFILT signals are processed to generate the digital compare B event force, interrupt, soc and sync signals.

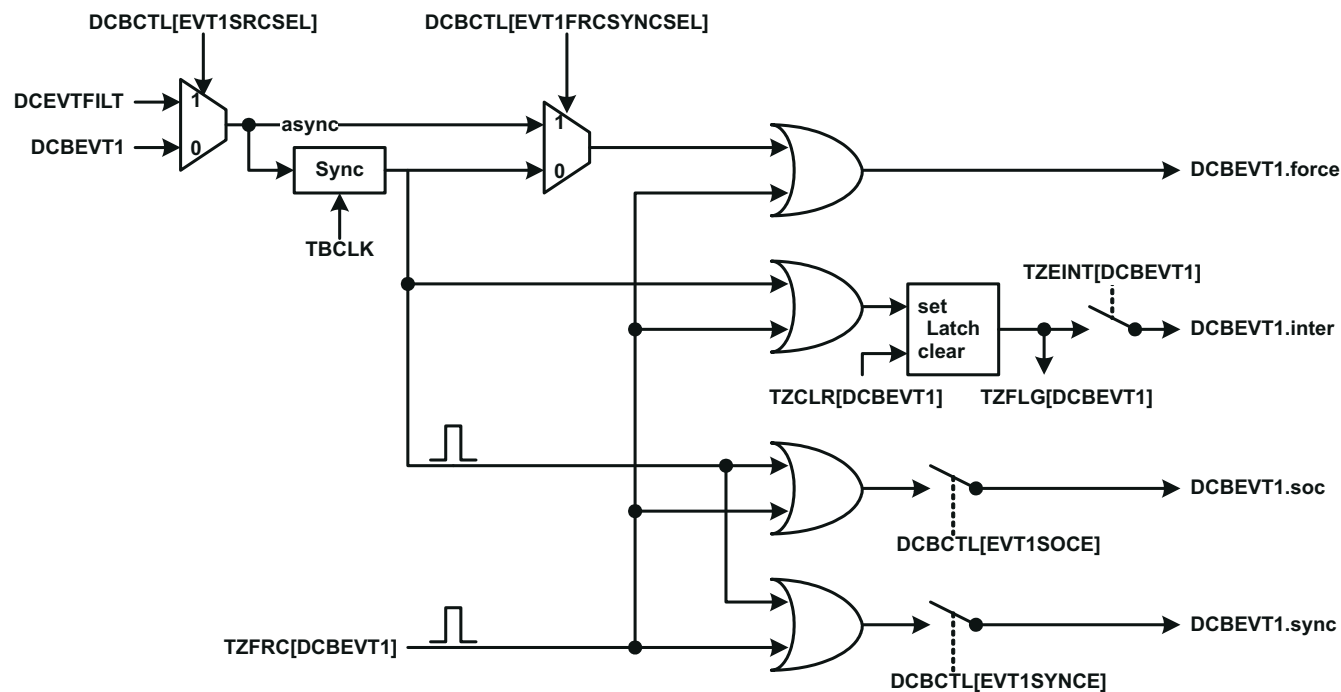


Figure 3-48. DCBEVT1 Event Triggering

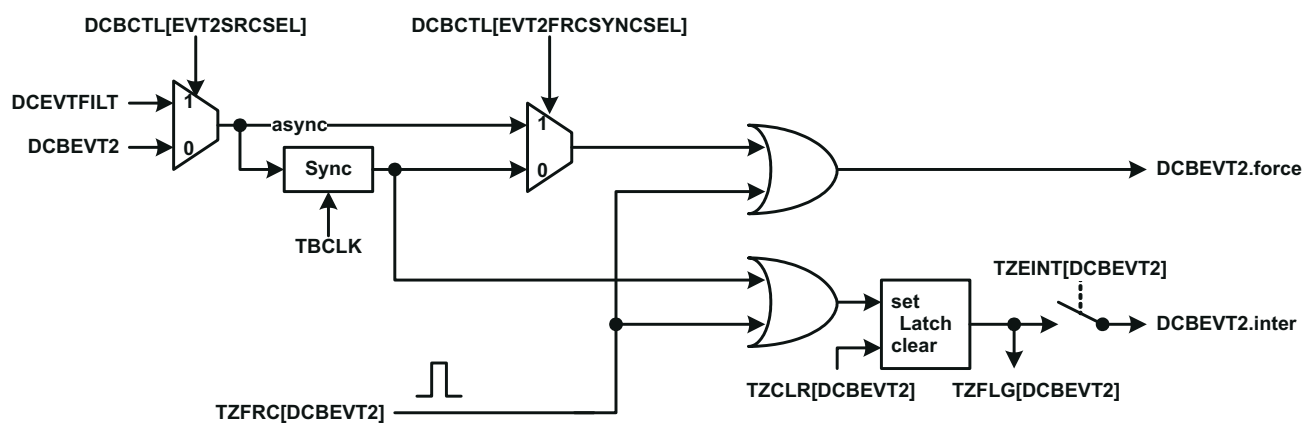
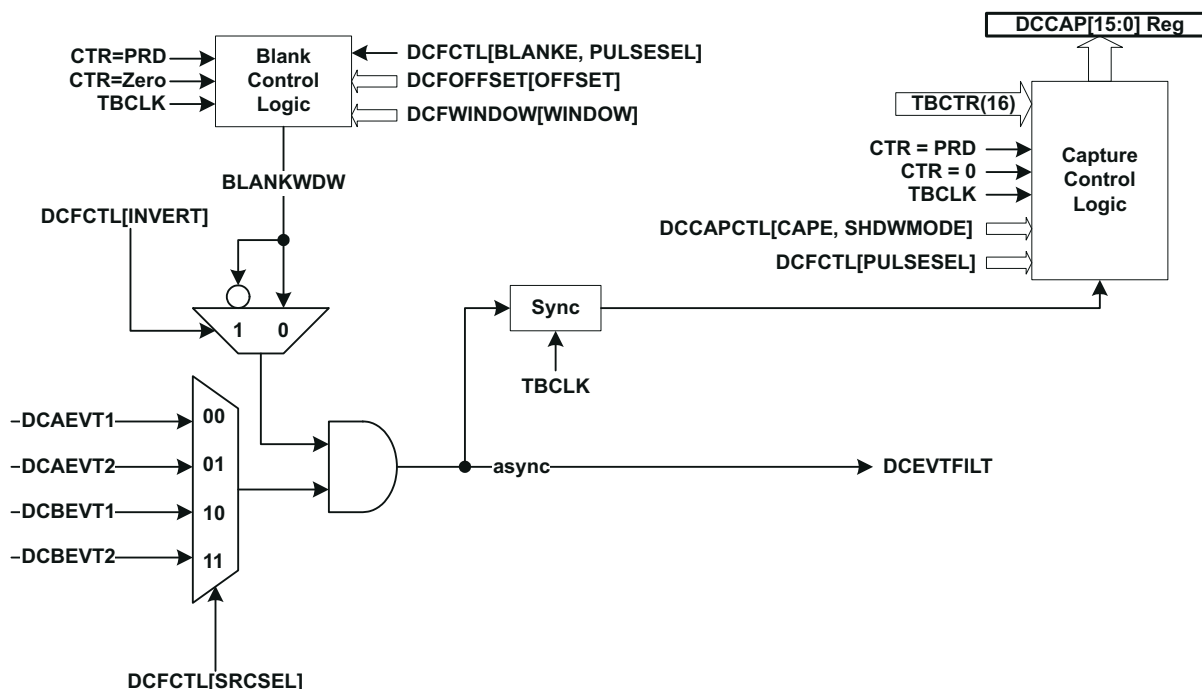


Figure 3-49. DCBEVT2 Event Triggering

### 3.2.9.3.2 Event Filtering

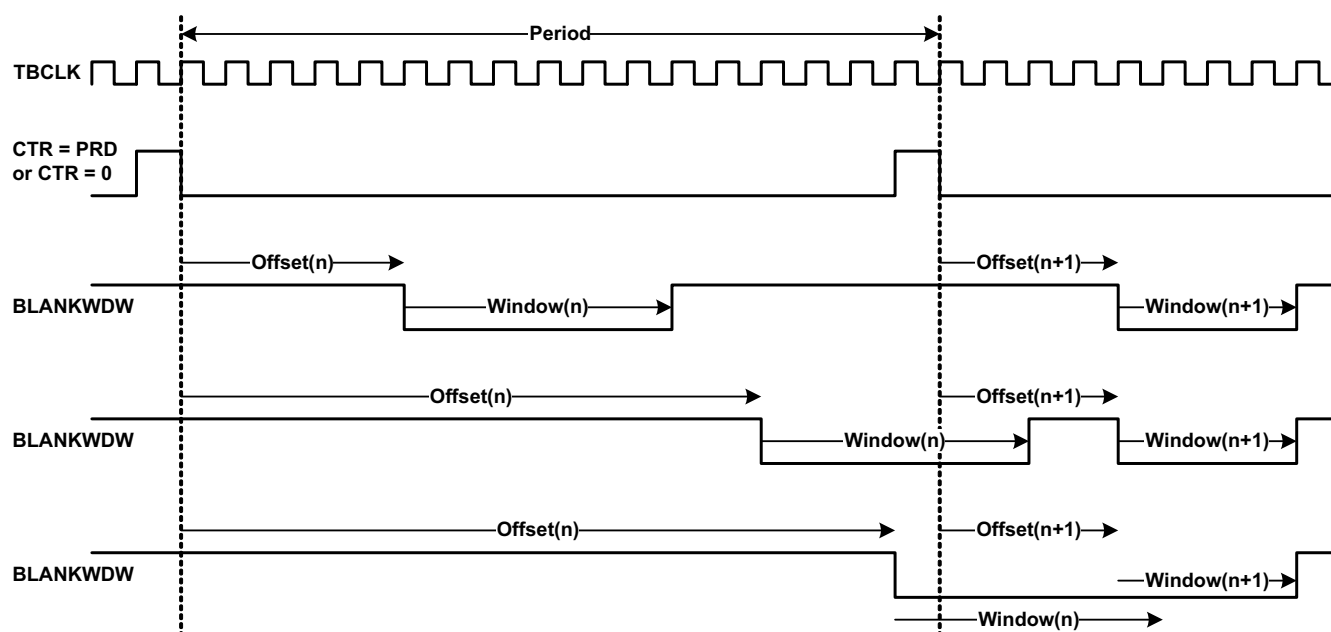
The DCAEVT1/2 and DCBEVT1/2 events can be filtered via event filtering logic to remove noise by optionally blanking events for a certain period of time. This is useful for cases where the analog comparator outputs may be selected to trigger DCAEVT1/2 and DCBEVT1/2 events, and the blanking logic is used to filter out potential noise on the signal prior to tripping the PWM outputs or generating an interrupt or ADC start-of-conversion. The event filtering can also capture the TBCTR value of the trip event. Figure 3-50 shows the details of the event filtering logic.



**Figure 3-50. Event Filtering**

If the blanking logic is enabled, one of the digital compare events – DCAEVT1, DCAEVT2, DCBEVT1, DCBEVT2 – is selected for filtering. The blanking window, which filters out all event occurrences on the signal while it is active, will be aligned to either a CTR = PRD pulse or a CTR = 0 pulse (configured by the DCFCTL[PULSESEL] bits). An offset value in TBCLK counts is programmed into the DCFOFFSET register, which determines at what point after the CTR = PRD or CTR = 0 pulse the blanking window starts. The duration of the blanking window, in number of TBCLK counts after the offset counter expires, is written to the DCFWINDOW register by the application. During the blanking window, all events are ignored. Before and after the blanking window ends, events can generate soc, sync, interrupt, and force signals as before.

Figure 3-51 illustrates several timing conditions for the offset and blanking window within an ePWM period. Notice that if the blanking window crosses the CTR = 0 or CTR = PRD boundary, the next window still starts at the same offset value after the CTR = 0 or CTR = PRD pulse.



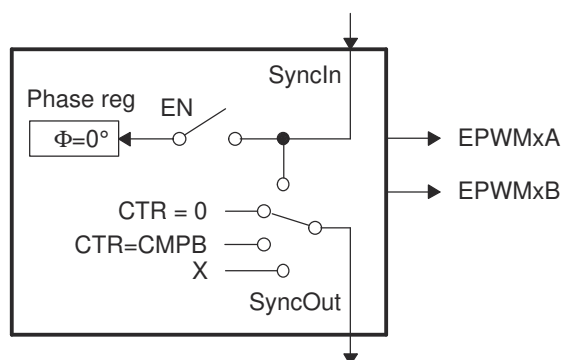
**Figure 3-51. Blanking Window Timing Diagram**

### 3.3 Applications to Power Topologies

An ePWM module has all the local resources necessary to operate completely as a standalone module or to operate in synchronization with other identical ePWM modules.

#### 3.3.1 Overview of Multiple Modules

Previously all discussions have described the operation of a single module. To facilitate the understanding of multiple modules working together in a system, the ePWM module described in reference is represented by the more simplified block diagram shown in [Figure 3-52](#). This simplified ePWM block shows only the key resources needed to explain how a multiswitch power topology is controlled with multiple ePWM modules working together.



**Figure 3-52. Simplified ePWM Module**

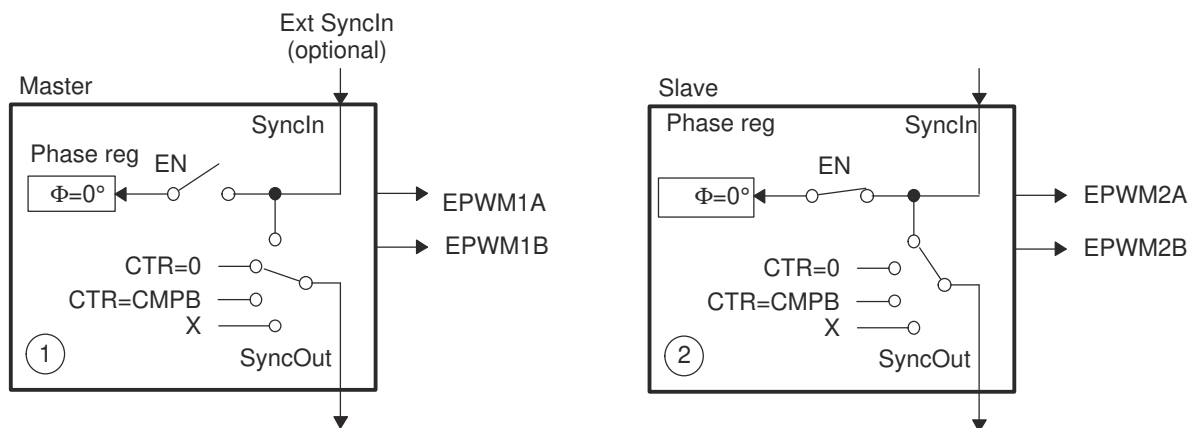


### 3.3.2 Key Configuration Capabilities

The key configuration choices available to each module are as follows:

- Options for SyncIn
  - Load own counter with phase register on an incoming sync strobe—enable (EN) switch closed
  - Do nothing or ignore incoming sync strobe—enable switch open
  - Sync flow-through - SyncOut connected to SyncIn
  - Master mode, provides a sync at PWM boundaries—SyncOut connected to CTR = PRD
  - Master mode, provides a sync at any programmable point in time—SyncOut connected to CTR = CMPB
  - Module is in standalone mode and provides No sync to other modules—SyncOut connected to X (disabled)
- Options for SyncOut
  - Sync flow-through - SyncOut connected to SyncIn
  - Master mode, provides a sync at PWM boundaries—SyncOut connected to CTR = PRD
  - Master mode, provides a sync at any programmable point in time—SyncOut connected to CTR = CMPB
  - Module is in standalone mode and provides No sync to other modules—SyncOut connected to X (disabled)

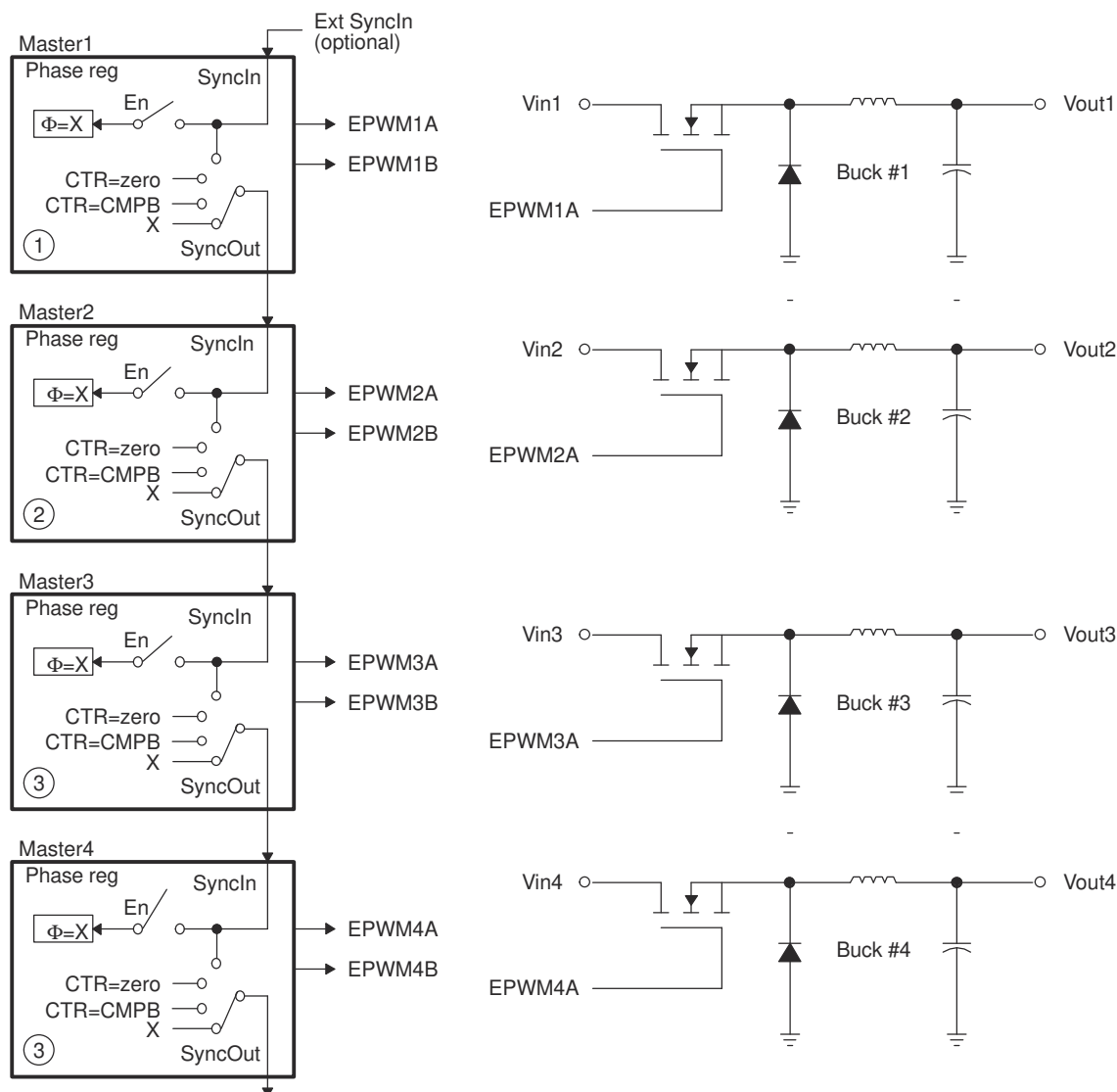
For each choice of SyncOut, a module may also choose to load its own counter with a new phase value on a SyncIn strobe input or choose to ignore it, that is, with the enable switch. Although various combinations are possible, the two most common—master module and slave module modes—are shown in [Figure 3-53](#).



**Figure 3-53. EPWM1 Configured as a Typical Master, EPWM2 Configured as a Slave**

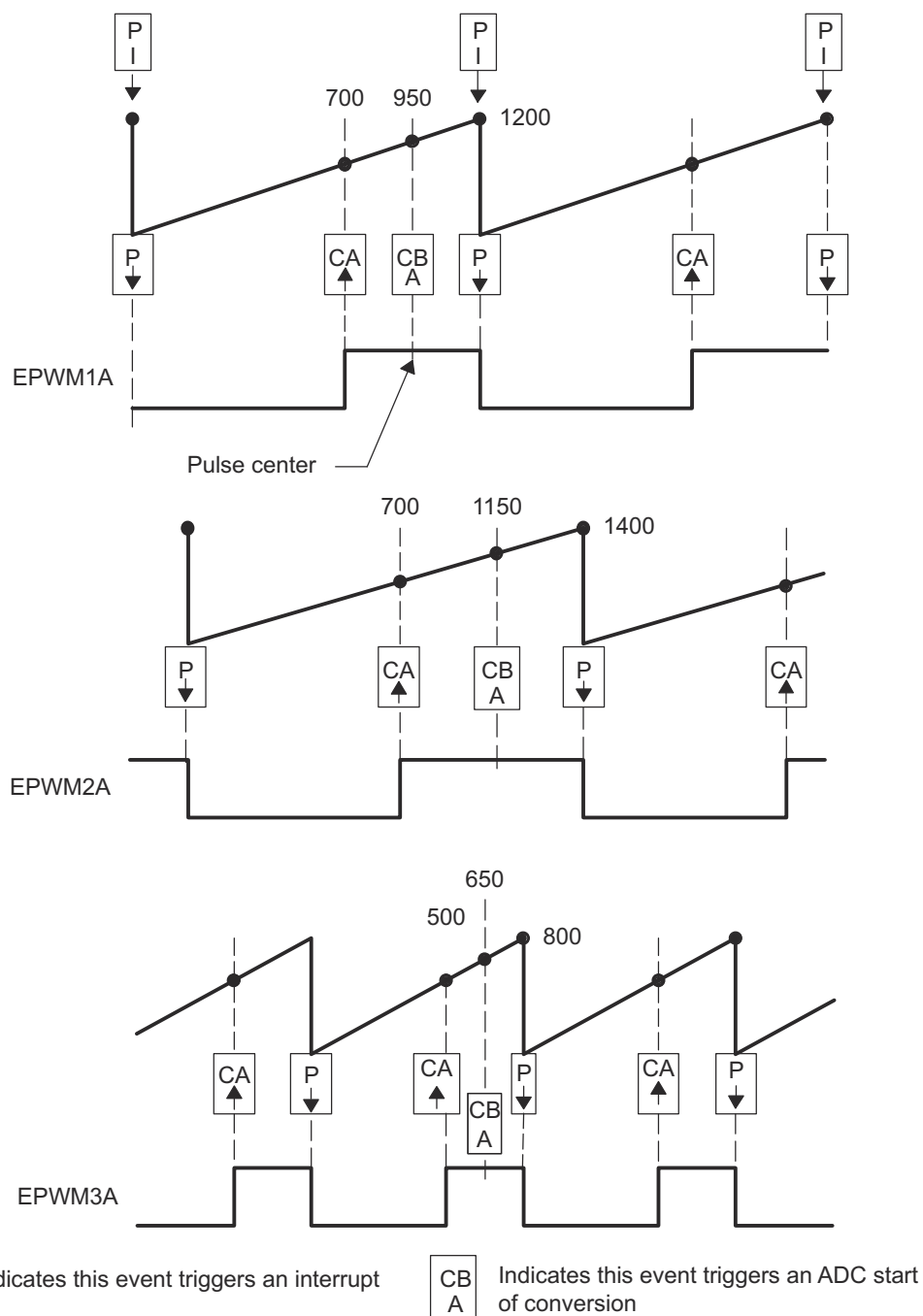
### 3.3.3 Controlling Multiple Buck Converters With Independent Frequencies

One of the simplest power converter topologies is the buck. A single ePWM module configured as a master can control two buck stages with the same PWM frequency. If independent frequency control is required for each buck converter, then one ePWM module must be allocated for each converter stage. Figure 3-54 shows four buck stages, each running at independent frequencies. In this case, all four ePWM modules are configured as masters and no synchronization is used. Figure 3-55 shows the waveforms generated by the setup shown in Figure 3-54; note that only three waveforms are shown, although there are four stages.



A.  $\Phi = X$  indicates value in phase register is a "don't care"

**Figure 3-54. Control of Four Buck Stages. Here  $F_{PWM1} \neq F_{PWM2} \neq F_{PWM3} \neq F_{PWM4}$**



**Figure 3-55. Buck Waveforms for Figure 3-54 (Note: Only three bucks shown here)**

### Example 3-8. Code Snippet for Configuration in Figure 3-54

```
//=====
// (Note: code for only 3 modules shown)
// Initialization Time
//=====
// EPWM Module 1 config
EPwm1Regs.TBPRD = 1200; // Period = 1201 TBCLK counts
EPwm1Regs.TBPHS.half.TBPHS = 0; // Set Phase register to zero
EPwm1Regs.TBCTL.bit.CTRMODE = TB_COUNT_UP; // Asymmetrical mode
EPwm1Regs.TBCTL.bit.PHSEN = TB_DISABLE; // Phase loading disabled
EPwm1Regs.TBCTL.bit.PRDL = TB_SHADOW;
EPwm1Regs.TBCTL.bit.SYNCSEL = TB_SYNC_DISABLE;
EPwm1Regs.CMPCTL.bit.SHDWAMODE = CC_SHADOW;
EPwm1Regs.CMPCTL.bit.SHDWBMODE = CC_SHADOW;
EPwm1Regs.CMPCTL.bit.LOADAMODE = CC_CTR_ZERO; // load on CTR=Zero
EPwm1Regs.CMPCTL.bit.LOADBMODE = CC_CTR_ZERO; // load on CTR=Zero
EPwm1Regs.AQCTLA.bit.PR = AQ_CLEAR;
EPwm1Regs.AQCTLA.bit.CAU = AQ_SET;
// EPWM Module 2 config
EPwm2Regs.TBPRD = 1400; // Period = 1401 TBCLK counts
EPwm2Regs.TBPHS.half.TBPHS = 0; // Set Phase register to zero
EPwm2Regs.TBCTL.bit.CTRMODE = TB_COUNT_UP; // Asymmetrical mode
EPwm2Regs.TBCTL.bit.PHSEN = TB_DISABLE; // Phase loading disabled
EPwm2Regs.TBCTL.bit.PRDL = TB_SHADOW;
EPwm2Regs.TBCTL.bit.SYNCSEL = TB_SYNC_DISABLE;
EPwm2Regs.CMPCTL.bit.SHDWAMODE = CC_SHADOW;
EPwm2Regs.CMPCTL.bit.SHDWBMODE = CC_SHADOW;
EPwm2Regs.CMPCTL.bit.LOADAMODE = CC_CTR_ZERO; // load on CTR=Zero
EPwm2Regs.CMPCTL.bit.LOADBMODE = CC_CTR_ZERO; // load on CTR=Zero
EPwm2Regs.AQCTLA.bit.PR = AQ_CLEAR;
EPwm2Regs.AQCTLA.bit.CAU = AQ_SET;
// EPWM Module 3 config
EPwm3Regs.TBPRD = 800; // Period = 801 TBCLK counts
EPwm3Regs.TBPHS.half.TBPHS = 0; // Set Phase register to zero
EPwm3Regs.TBCTL.bit.CTRMODE = TB_COUNT_UP;
EPwm3Regs.TBCTL.bit.PHSEN = TB_DISABLE; // Phase loading disabled
EPwm3Regs.TBCTL.bit.PRDL = TB_SHADOW;
EPwm3Regs.TBCTL.bit.SYNCSEL = TB_SYNC_DISABLE;
EPwm3Regs.CMPCTL.bit.SHDWAMODE = CC_SHADOW;
EPwm3Regs.CMPCTL.bit.SHDWBMODE = CC_SHADOW;
EPwm3Regs.CMPCTL.bit.LOADAMODE = CC_CTR_ZERO; // load on CTR=Zero
EPwm3Regs.CMPCTL.bit.LOADBMODE = CC_CTR_ZERO; // load on CTR=Zero
EPwm3Regs.AQCTLA.bit.PR = AQ_CLEAR;
EPwm3Regs.AQCTLA.bit.CAU = AQ_SET;
//
// Run Time (Note: Example execution of one run-time instant)
//=====
EPwm1Regs.CMPA.half.CMPA = 700; // adjust duty for output EPWM1A
EPwm2Regs.CMPA.half.CMPA = 700; // adjust duty for output EPWM2A
EPwm3Regs.CMPA.half.CMPA = 500; // adjust duty for output EPWM3A
```

### 3.3.4 Controlling Multiple Buck Converters With Same Frequencies

If synchronization is a requirement, ePWM module 2 can be configured as a slave and can operate at integer multiple (N) frequencies of module 1. The sync signal from master to slave ensures these modules remain locked. Figure 3-56 shows such a configuration; Figure 3-57 shows the waveforms generated by the configuration.

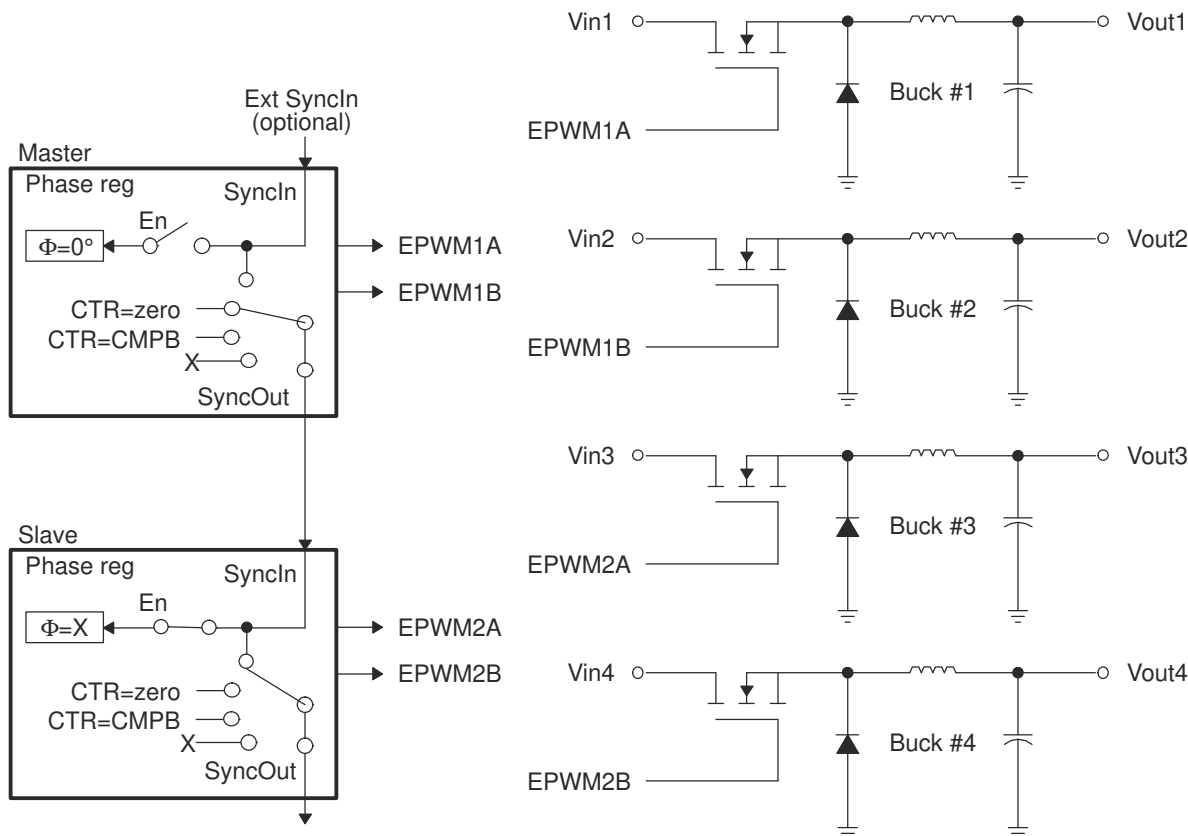


Figure 3-56. Control of Four Buck Stages. (Note:  $F_{PWM2} = N \times F_{PWM1}$ )

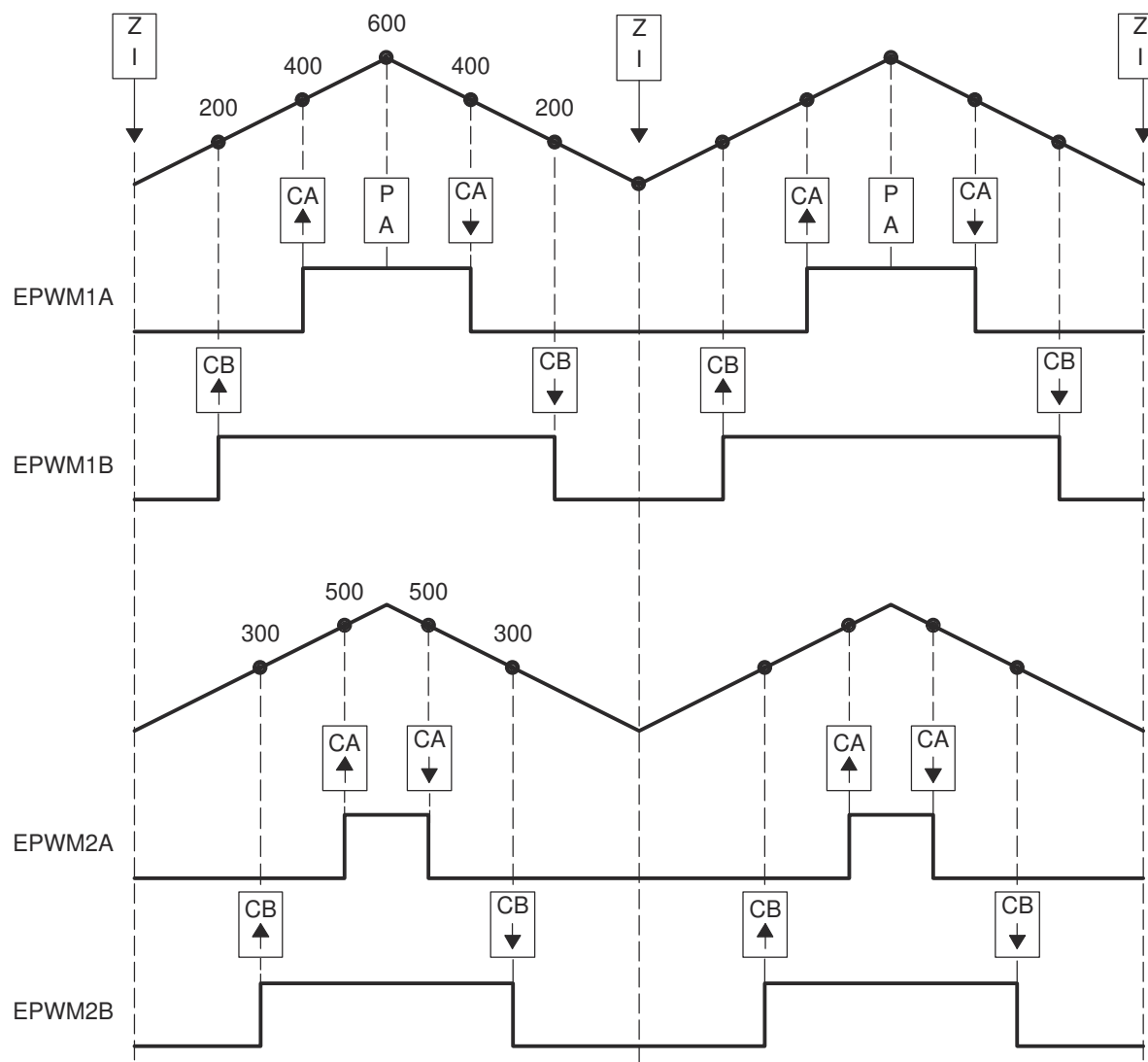


Figure 3-57. Buck Waveforms for Figure 3-56 (Note:  $F_{PWM2} = F_{PWM1}$ )

**Example 3-9. Code Snippet for Configuration in Figure 3-56**

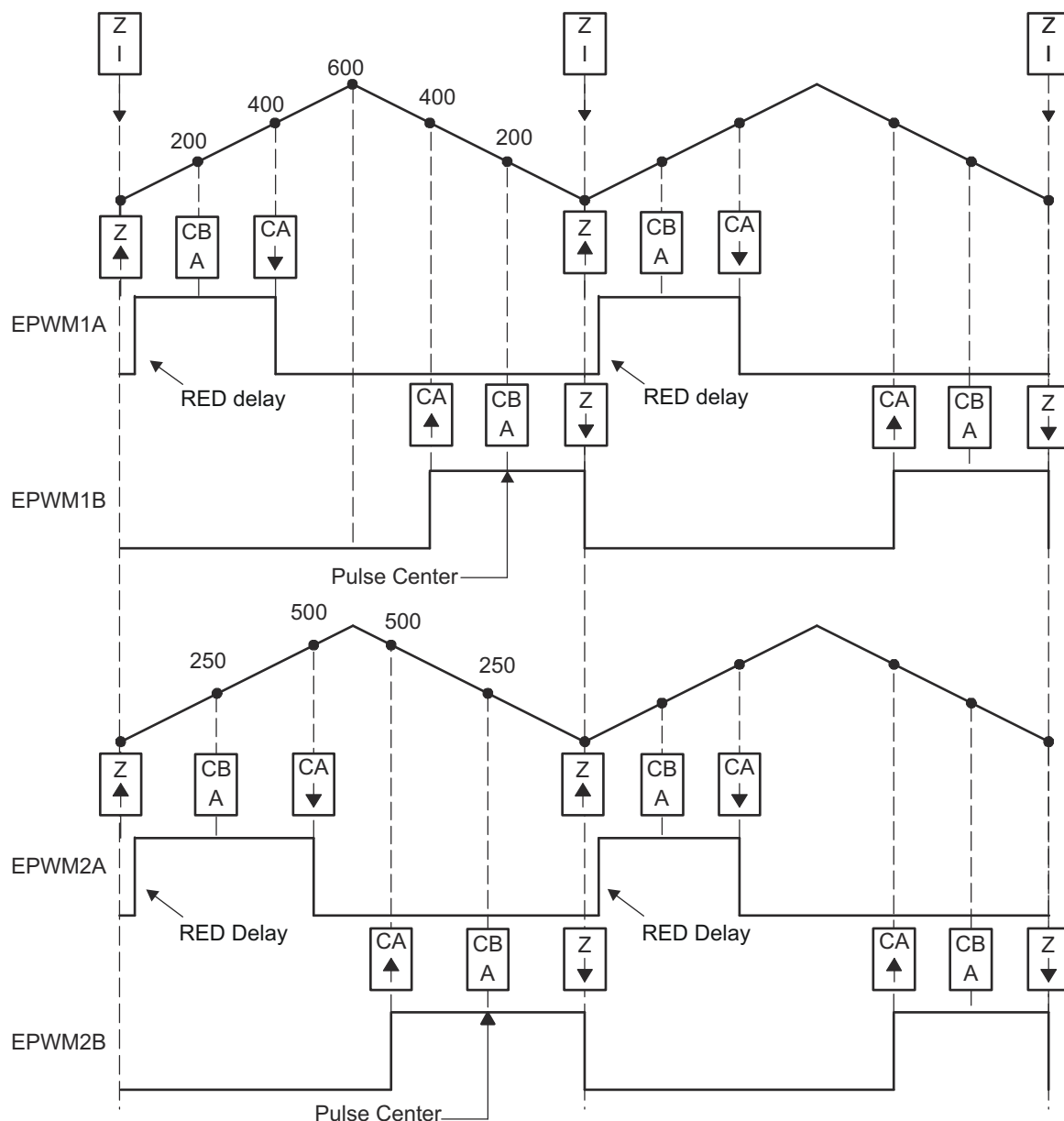
```
//=====
// EPWM Module 1 config
EPwm1Regs.TBPRD = 600; // Period = 1200 TBCLK counts
EPwm1Regs.TBPHS.half.TBPHS = 0; // Set Phase register to zero
EPwm1Regs.TBCTL.bit.CTRMODE = TB_COUNT_UPDOWN; // Symmetrical mode
EPwm1Regs.TBCTL.bit.PHSEN = TB_DISABLE; // Master module
EPwm1Regs.TBCTL.bit.PRDL = TB_SHADOW;
EPwm1Regs.TBCTL.bit.SYNCSEL = TB_CTR_ZERO; // Sync down-stream module
EPwm1Regs.CMPCTL.bit.SHDWAMODE = CC_SHADOW;
EPwm1Regs.CMPCTL.bit.SHDWBMODE = CC_SHADOW;
EPwm1Regs.CMPCTL.bit.LOADAMODE = CC_CTR_ZERO; // load on CTR=Zero
EPwm1Regs.CMPCTL.bit.LOADBMODE = CC_CTR_ZERO; // load on CTR=Zero
EPwm1Regs.AQCTLA.bit.CAU = AQ_SET; // set actions for EPWM1A
EPwm1Regs.AQCTLA.bit.CAD = AQ_CLEAR;
EPwm1Regs.AQCTLB.bit.CBU = AQ_SET; // set actions for EPWM1B
EPwm1Regs.AQCTLB.bit.CBD = AQ_CLEAR;

// EPWM Module 2 config
EPwm2Regs.TBPRD = 600; // Period = 1200 TBCLK counts
EPwm2Regs.TBPHS.half.TBPHS = 0; // Set Phase register to zero
EPwm2Regs.TBCTL.bit.CTRMODE = TB_COUNT_UPDOWN; // Symmetrical mode
EPwm2Regs.TBCTL.bit.PHSEN = TB_ENABLE; // Slave module
EPwm2Regs.TBCTL.bit.PRDL = TB_SHADOW;
EPwm2Regs.TBCTL.bit.SYNCSEL = TB_SYNC_IN; // sync flow-through
EPwm2Regs.CMPCTL.bit.SHDWAMODE = CC_SHADOW;
EPwm2Regs.CMPCTL.bit.SHDWBMODE = CC_SHADOW;
EPwm2Regs.CMPCTL.bit.LOADAMODE = CC_CTR_ZERO; // load on CTR=Zero
EPwm2Regs.CMPCTL.bit.LOADBMODE = CC_CTR_ZERO; // load on CTR=Zero
EPwm2Regs.AQCTLA.bit.CAU = AQ_SET; // set actions for EPWM2A
EPwm2Regs.AQCTLA.bit.CAD = AQ_CLEAR;
EPwm2Regs.AQCTLB.bit.CBU = AQ_SET; // set actions for EPWM2B
EPwm2Regs.AQCTLB.bit.CBD = AQ_CLEAR;

//
// Run Time (Note: Example execution of one run-time instance)
//=====
EPwm1Regs.CMPA.half.CMPA = 400; // adjust duty for output EPWM1A
EPwm1Regs.CMPB = 200; // adjust duty for output EPWM1B
EPwm2Regs.CMPA.half.CMPA = 500; // adjust duty for output EPWM2A
EPwm2Regs.CMPB = 300; // adjust duty for output EPWM2B
```







**Figure 3-59. Half-H Bridge Waveforms for Figure 3-58 (Note: Here  $F_{PWM2} = F_{PWM1}$  )**

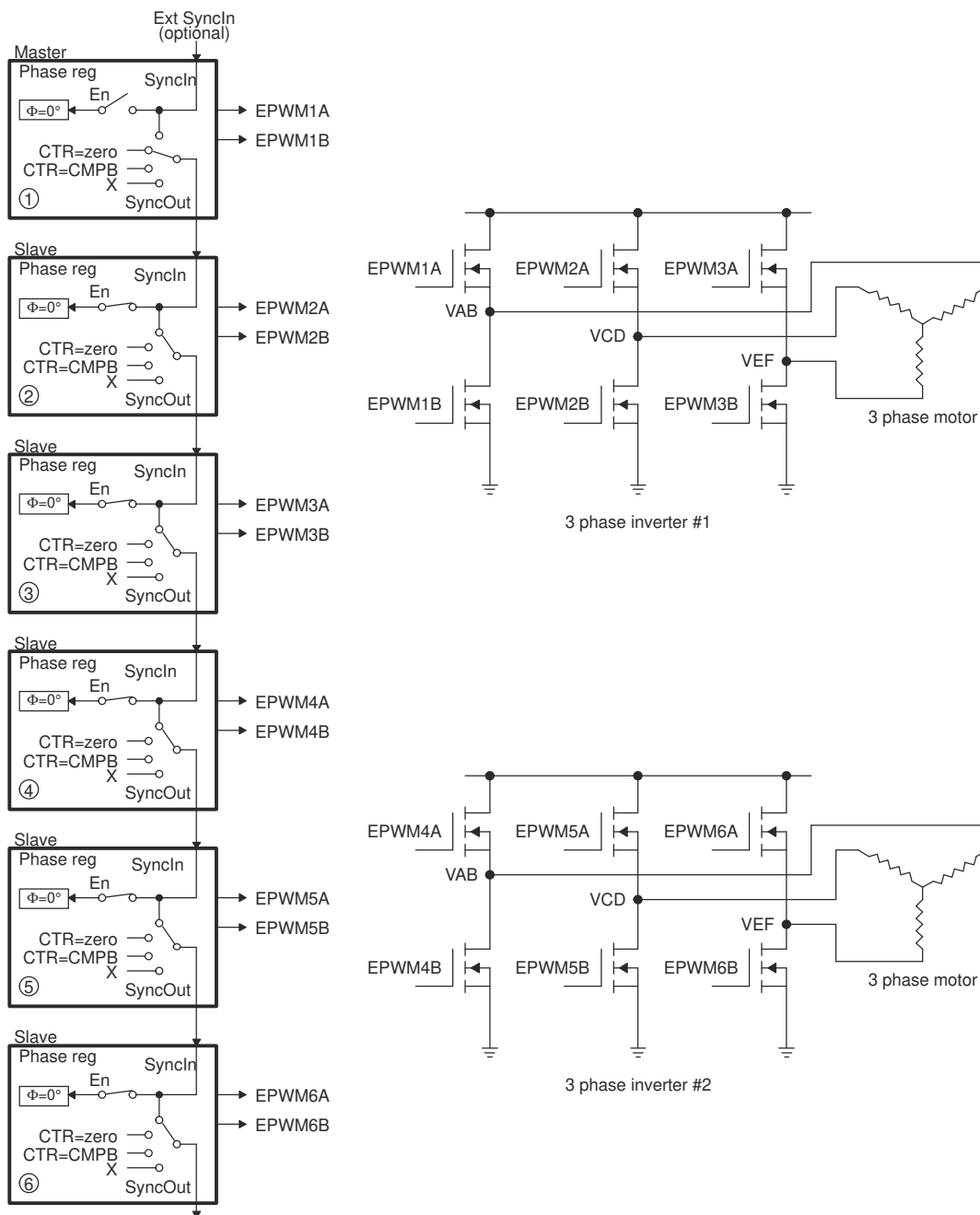
### Example 3-10. Code Snippet for Configuration in Figure 3-58

```
//=====
// Config
//=====
// Initialization Time
//=====
// EPWM Module 1 config
EPwm1Regs.TBPRD = 600; // Period = 1200 TBCLK counts
EPwm1Regs.TBPHS.half.TBPHS = 0; // Set Phase register to zero
EPwm1Regs.TBCTL.bit.CTRMODE = TB_COUNT_UPDOWN; // Symmetrical mode
EPwm1Regs.TBCTL.bit.PHSEN = TB_DISABLE; // Master module
EPwm1Regs.TBCTL.bit.PRDL = TB_SHADOW;
EPwm1Regs.TBCTL.bit.SYNCSEL = TB_CTR_ZERO; // Sync down-stream module
EPwm1Regs.CMPCTL.bit.SHDWAMODE = CC_SHADOW;
EPwm1Regs.CMPCTL.bit.SHDWBMODE = CC_SHADOW;
EPwm1Regs.CMPCTL.bit.LOADAMODE = CC_CTR_ZERO; // load on CTR=Zero
EPwm1Regs.CMPCTL.bit.LOADBMODE = CC_CTR_ZERO; // load on CTR=Zero
EPwm1Regs.AQCTLA.bit.ZRO = AQ_SET; // set actions for EPWM1A
EPwm1Regs.AQCTLA.bit.CAU = AQ_CLEAR;
EPwm1Regs.AQCTLB.bit.ZRO = AQ_CLEAR; // set actions for EPWM1B
EPwm1Regs.AQCTLB.bit.CAD = AQ_SET;
EPwm1Regs.DBCTL.bit.OUT_MODE = 2; // delay on EPWM1A
EPwm1Regs.DBRED = 10; // delay value
// EPWM Module 2 config
EPwm2Regs.TBPRD = 600; // Period = 1200 TBCLK counts
EPwm2Regs.TBPHS.half.TBPHS = 0; // Set Phase register to zero
EPwm2Regs.TBCTL.bit.CTRMODE = TB_COUNT_UPDOWN; // Symmetrical mode
EPwm2Regs.TBCTL.bit.PHSEN = TB_ENABLE; // Slave module
EPwm2Regs.TBCTL.bit.PRDL = TB_SHADOW;
EPwm2Regs.TBCTL.bit.SYNCSEL = TB_SYNC_IN; // sync flow-through
EPwm2Regs.CMPCTL.bit.SHDWAMODE = CC_SHADOW;
EPwm2Regs.CMPCTL.bit.SHDWBMODE = CC_SHADOW;
EPwm2Regs.CMPCTL.bit.LOADAMODE = CC_CTR_ZERO; // load on CTR=Zero
EPwm2Regs.CMPCTL.bit.LOADBMODE = CC_CTR_ZERO; // load on CTR=Zero
EPwm2Regs.AQCTLA.bit.ZRO = AQ_SET; // set actions for EPWM1A
EPwm2Regs.AQCTLA.bit.CAU = AQ_CLEAR;
EPwm2Regs.AQCTLB.bit.ZRO = AQ_CLEAR; // set actions for EPWM1B
EPwm2Regs.AQCTLB.bit.CAD = AQ_SET;
EPwm2Regs.DBCTL.bit.OUT_MODE = 2; // delay on EPWM2A
EPwm2Regs.DBRED = 10; // delay value
//=====
EPwm1Regs.CMPA.half.CMPA = 400; // adjust duty for output EPWM1A & EPWM1B
EPwm1Regs.CMPB = 200; // adjust point-in-time for ADCSOC trigger
EPwm2Regs.CMPA.half.CMPA = 500; // adjust duty for output EPWM2A & EPWM2B
EPwm2Regs.CMPB = 250; // adjust point-in-time for ADCSOC trigger
```

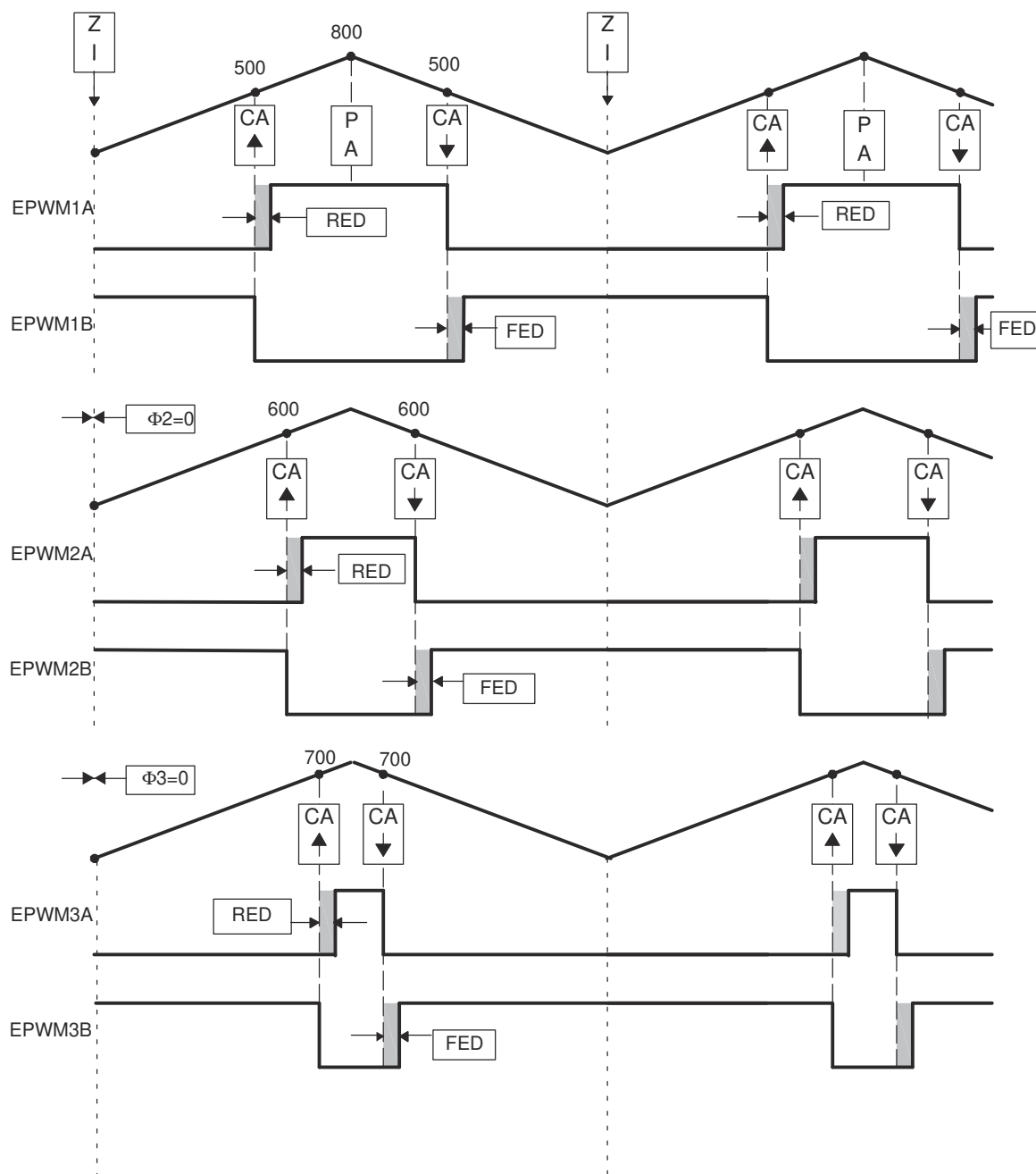
### 3.3.6 Controlling Dual 3-Phase Inverters for Motors (ACI and PMSM)

The idea of multiple modules controlling a single power stage can be extended to the 3-phase Inverter case. In such a case, six switching elements can be controlled using three PWM modules, one for each leg of the inverter. Each leg must switch at the same frequency and all legs must be synchronized. A master + two slaves configuration can easily address this requirement. Figure 3-60 shows how six PWM modules can control two independent 3-phase Inverters; each running a motor.

As in the cases shown in the previous sections, we have a choice of running each inverter at a different frequency (module 1 and module 4 are masters as in Figure 3-60), or both inverters can be synchronized by using one master (module 1) and five slaves. In this case, the frequency of modules 4, 5, and 6 (all equal) can be integer multiples of the frequency for modules 1, 2, 3 (also all equal).



**Figure 3-60. Control of Dual 3-Phase Inverter Stages as Is Commonly Used in Motor Control**



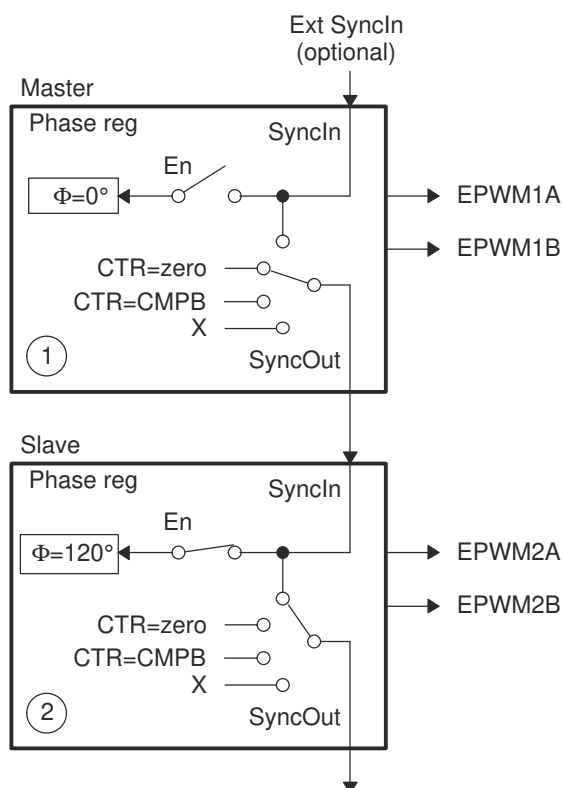
**Figure 3-61. 3-Phase Inverter Waveforms for Figure 3-60 (Only One Inverter Shown)**

**Example 3-11. Code Snippet for Configuration in Figure 3-60**

```
//=====
// Configuration
//=====
// Initialization Time
//=====
// EPWM Module 1 config
    EPwm1Regs.TBPRD = 800; // Period = 1600 TBCLK counts
    EPwm1Regs.TBPHS.half.TBPHS = 0; // Set Phase register to zero
    EPwm1Regs.TBCTL.bit.CTRMODE = TB_COUNT_UPDOWN; // Symmetrical mode
    EPwm1Regs.TBCTL.bit.PHSEN = TB_DISABLE; // Master module
    EPwm1Regs.TBCTL.bit.PRDL = TB_SHADOW;
    EPwm1Regs.TBCTL.bit.SYNCSEL = TB_CTR_ZERO; // Sync down-stream module
    EPwm1Regs.CMPCTL.bit.SHDWAMODE = CC_SHADOW;
    EPwm1Regs.CMPCTL.bit.SHDWBMODE = CC_SHADOW;
    EPwm1Regs.CMPCTL.bit.LOADAMODE = CC_CTR_ZERO; // load on CTR=Zero
    EPwm1Regs.CMPCTL.bit.LOADBMODE = CC_CTR_ZERO; // load on CTR=Zero
    EPwm1Regs.AQCTLA.bit.CAU = AQ_SET; // set actions for EPWM1A
    EPwm1Regs.AQCTLA.bit.CAD = AQ_CLEAR;
    EPwm1Regs.DBCTL.bit.OUT_MODE = DB_FULL_ENABLE; // enable Dead-band module
    EPwm1Regs.DBCTL.bit.POLSEL = DB_ACTV_HIC; // Active Hi complementary
    EPwm1Regs.DBFED = 50; // FED = 50 TBCLKs
    EPwm1Regs.DBRED = 50; // RED = 50 TBCLKs
// EPWM Module 2 config
    EPwm2Regs.TBPRD = 800; // Period = 1600 TBCLK counts
    EPwm2Regs.TBPHS.half.TBPHS = 0; // Set Phase register to zero
    EPwm2Regs.TBCTL.bit.CTRMODE = TB_COUNT_UPDOWN; // Symmetrical mode
    EPwm2Regs.TBCTL.bit.PHSEN = TB_ENABLE; // Slave module
    EPwm2Regs.TBCTL.bit.PRDL = TB_SHADOW;
    EPwm2Regs.TBCTL.bit.SYNCSEL = TB_SYNC_IN; // sync flow-through
    EPwm2Regs.CMPCTL.bit.SHDWAMODE = CC_SHADOW;
    EPwm2Regs.CMPCTL.bit.SHDWBMODE = CC_SHADOW;
    EPwm2Regs.CMPCTL.bit.LOADAMODE = CC_CTR_ZERO; // load on CTR=Zero
    EPwm2Regs.CMPCTL.bit.LOADBMODE = CC_CTR_ZERO; // load on CTR=Zero
    EPwm2Regs.AQCTLA.bit.CAU = AQ_SET; // set actions for EPWM2A
    EPwm2Regs.AQCTLA.bit.CAD = AQ_CLEAR;
    EPwm2Regs.DBCTL.bit.OUT_MODE = DB_FULL_ENABLE; // enable Dead-band module
    EPwm2Regs.DBCTL.bit.POLSEL = DB_ACTV_HIC; // Active Hi complementary
    EPwm2Regs.DBFED = 50; // FED = 50 TBCLKs
    EPwm2Regs.DBRED = 50; // RED = 50 TBCLKs
// EPWM Module 3 config
    EPwm3Regs.TBPRD = 800; // Period = 1600 TBCLK counts
    EPwm3Regs.TBPHS.half.TBPHS = 0; // Set Phase register to zero
    EPwm3Regs.TBCTL.bit.CTRMODE = TB_COUNT_UPDOWN; // Symmetrical mode
    EPwm3Regs.TBCTL.bit.PHSEN = TB_ENABLE; // Slave module
    EPwm3Regs.TBCTL.bit.PRDL = TB_SHADOW;
    EPwm3Regs.TBCTL.bit.SYNCSEL = TB_SYNC_IN; // sync flow-through
    EPwm3Regs.CMPCTL.bit.SHDWAMODE = CC_SHADOW;
    EPwm3Regs.CMPCTL.bit.SHDWBMODE = CC_SHADOW;
    EPwm3Regs.CMPCTL.bit.LOADAMODE = CC_CTR_ZERO; // load on CTR=Zero
    EPwm3Regs.CMPCTL.bit.LOADBMODE = CC_CTR_ZERO; // load on CTR=Zero
    EPwm3Regs.AQCTLA.bit.CAU = AQ_SET; // set actions for EPWM3A
    EPwm3Regs.AQCTLA.bit.CAD = AQ_CLEAR;
    EPwm3Regs.DBCTL.bit.OUT_MODE = DB_FULL_ENABLE; // enable Dead-band module
    EPwm3Regs.DBCTL.bit.POLSEL = DB_ACTV_HIC; // Active Hi complementary
    EPwm3Regs.DBFED = 50; // FED = 50 TBCLKs
    EPwm3Regs.DBRED = 50; // RED = 50 TBCLKs
// Run Time (Note: Example execution of one run-time instant)
//=====
    EPwm1Regs.CMPA.half.CMPA = 500; // adjust duty for output EPWM1A
    EPwm2Regs.CMPA.half.CMPA = 600; // adjust duty for output EPWM2A
    EPwm3Regs.CMPA.half.CMPA = 700; // adjust duty for output EPWM3A
```

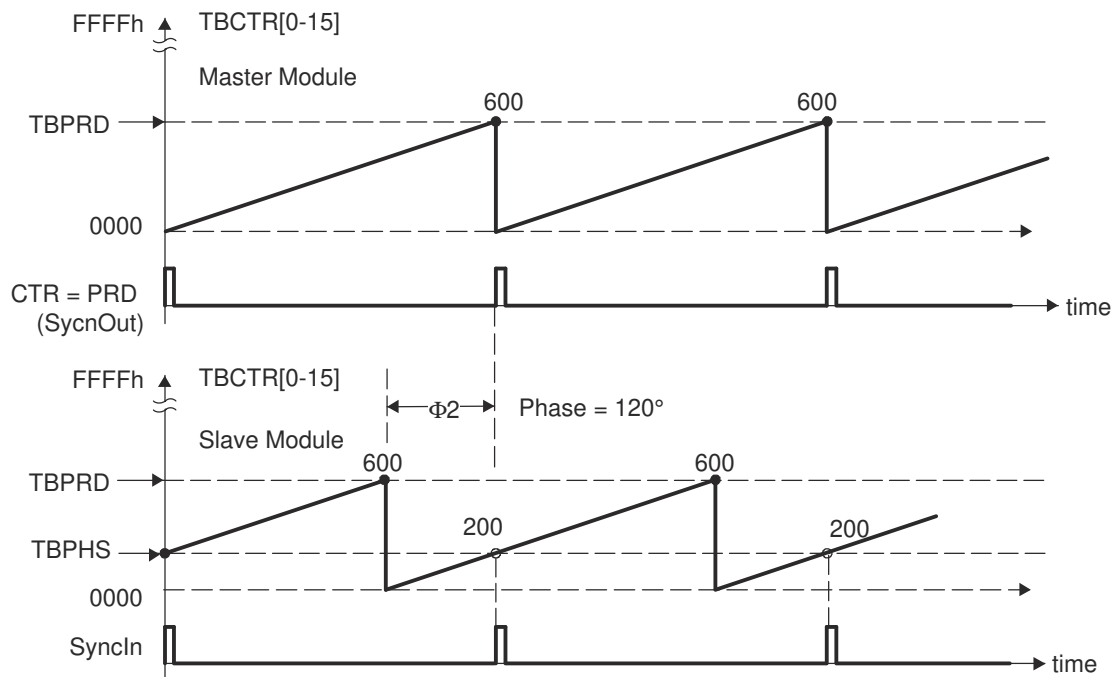
### 3.3.7 Practical Applications Using Phase Control Between PWM Modules

So far, none of the examples have made use of the phase register (TBPHS). It has either been set to zero or its value has been a don't care. However, by programming appropriate values into TBPHS, multiple PWM modules can address another class of power topologies that rely on phase relationship between legs (or stages) for correct operation. As described in the TB module section, a PWM module can be configured to allow a SyncIn pulse to cause the TBPHS register to be loaded into the TBCTR register. To illustrate this concept, [Figure 3-62](#) shows a master and slave module with a phase relationship of 120°, that is, the slave leads the master.



**Figure 3-62. Configuring Two PWM Modules for Phase Control**

Figure 3-63 shows the associated timing waveforms for this configuration. Here, TBPRD = 600 for both master and slave. For the slave, TBPHS = 200 ( $200/600 \times 360^\circ = 120^\circ$ ). Whenever the master generates a SyncIn pulse (CTR = PRD), the value of TBPHS = 200 is loaded into the slave TBCTR register so the slave time-base is always leading the master's time-base by  $120^\circ$ .



**Figure 3-63. Timing Waveforms Associated With Phase Control Between 2 Modules**

### 3.3.8 Controlling a 3-Phase Interleaved DC/DC Converter

A popular power topology that makes use of phase-offset between modules is shown in Figure 3-64. This system uses three PWM modules, with module 1 configured as the master. To work, the phase relationship between adjacent modules must be  $F = 120^\circ$ . This is achieved by setting the slave TBPHS registers 2 and 3 with values of 1/3 and 2/3 of the period value, respectively. For example, if the period register is loaded with a value of 600 counts, then TBPHS (slave 2) = 200 and TBPHS (slave 3) = 400. Both slave modules are synchronized to the master 1 module.

This concept can be extended to four or more phases, by setting the TBPHS values appropriately. The following formula gives the TBPHS values for N phases:

$$\text{TBPHS}(N,M) = (\text{TBPRD}/N) \times (M-1)$$

Where:

N = number of phases

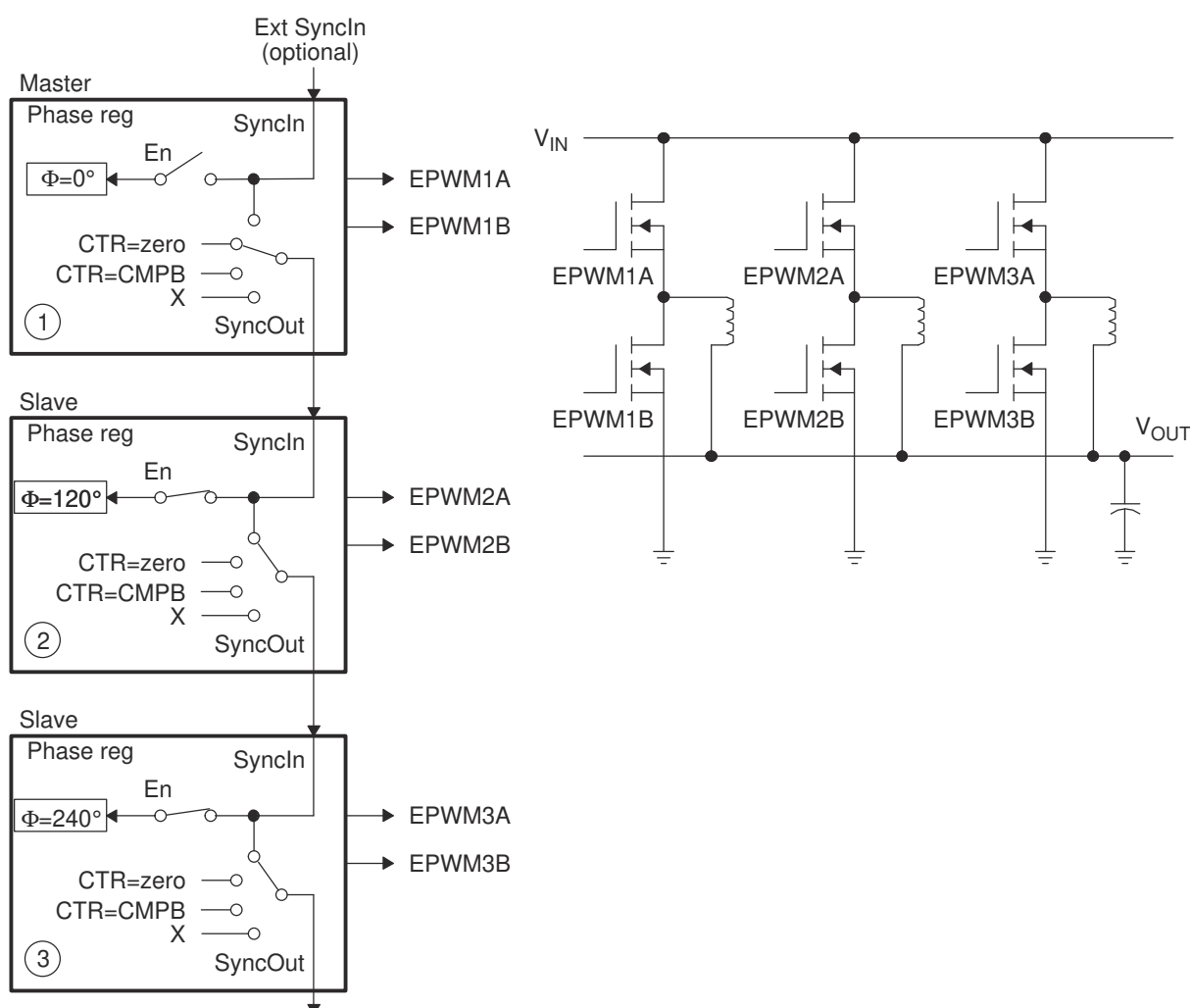
M = PWM module number

For example, for the 3-phase case (N=3), TBPRD = 600,

$\text{TBPHS}(3,2) = (600/3) \times (2-1) = 200$  (that is, Phase value for Slave module 2)

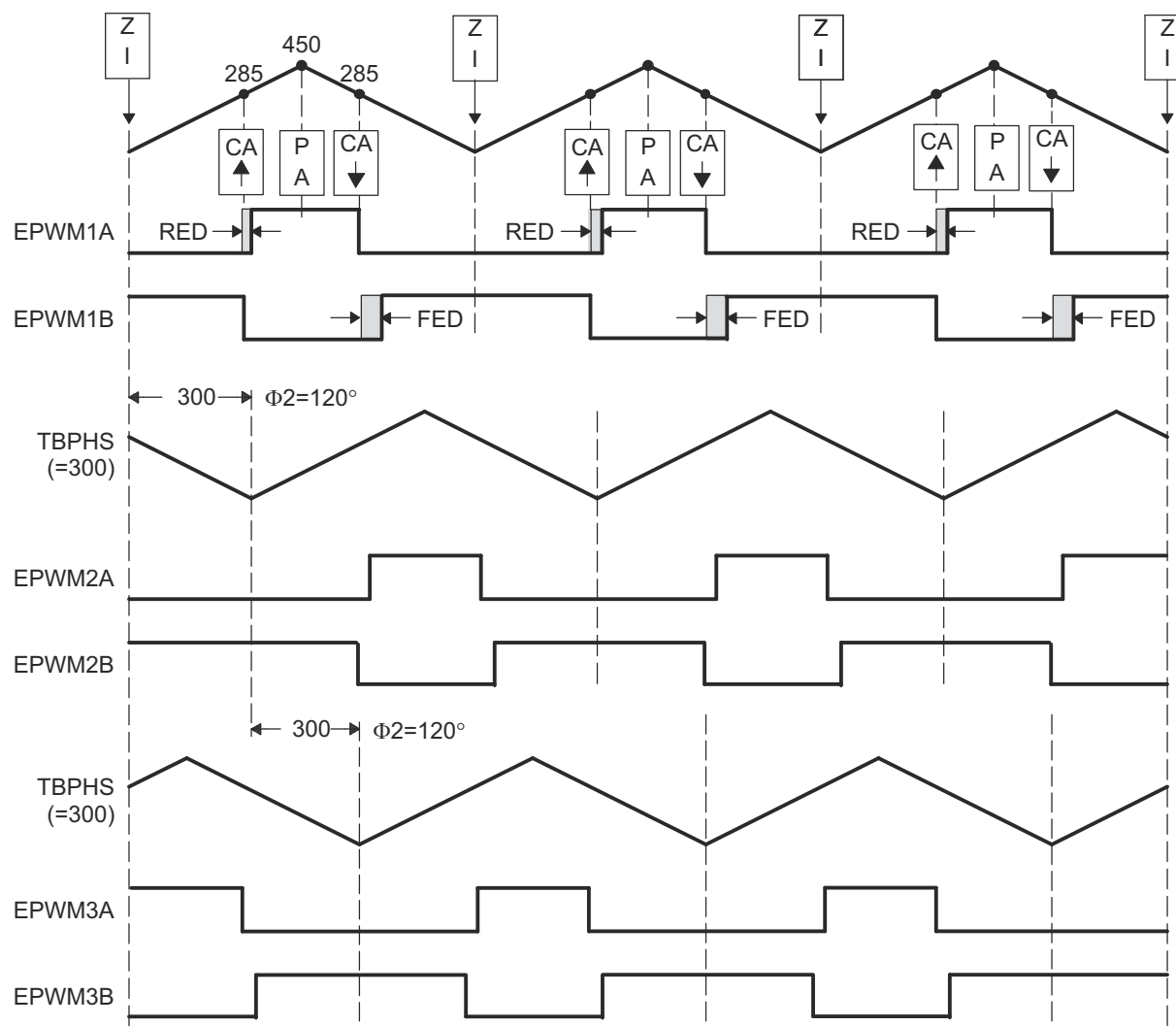
$\text{TBPHS}(3,3) = 400$  (Phase value for Slave module 3)

Figure 3-65 shows the waveforms for the configuration in Figure 3-64.



**Figure 3-64. Control of a 3-Phase Interleaved DC/DC Converter**





**Figure 3-65. 3-Phase Interleaved DC/DC Converter Waveforms for [Figure 3-64](#)**

**Example 3-12. Code Snippet for Configuration in Figure 3-64**

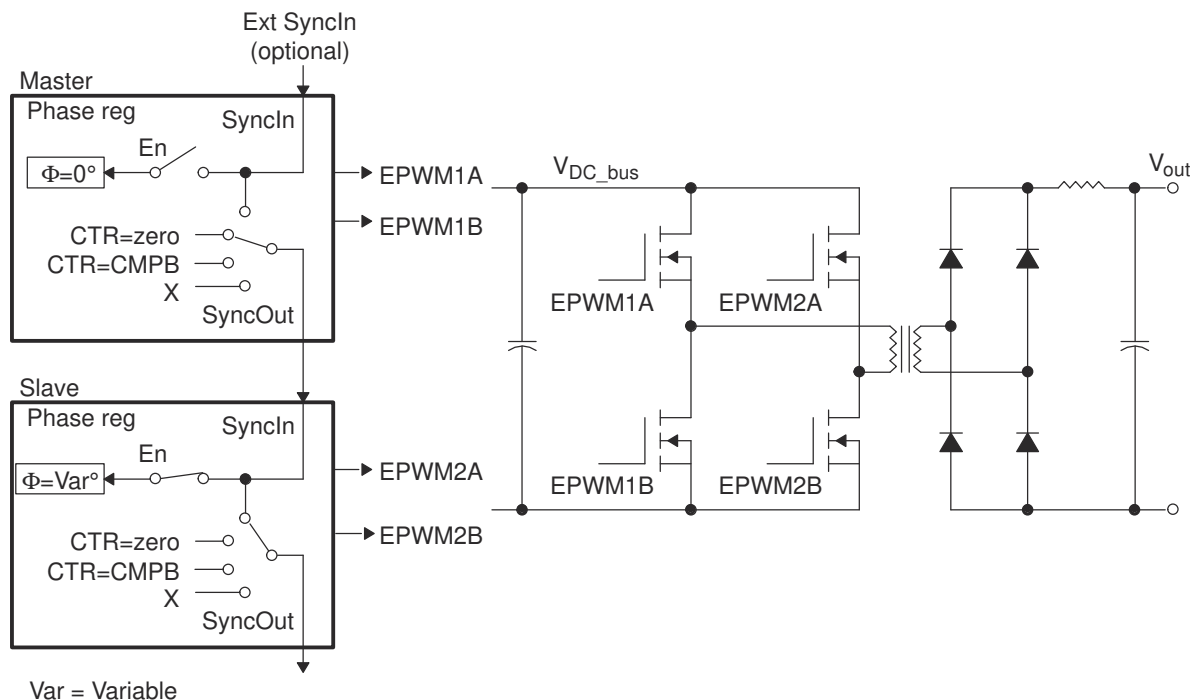
```
//=====
// Config
// Initialization Time
//=====
// EPWM Module 1 config
EPwm1Regs.TBPRD = 450; // Period = 900 TBCLK counts
EPwm1Regs.TBPHS.half.TBPHS = 0; // Set Phase register to zero
EPwm1Regs.TBCTL.bit.CTRMODE = TB_COUNT_UPDOWN; // Symmetrical mode
EPwm1Regs.TBCTL.bit.PHSEN = TB_DISABLE; // Master module
EPwm1Regs.TBCTL.bit.PRDL = TB_SHADOW;
EPwm1Regs.TBCTL.bit.SYNCSEL = TB_CTR_ZERO; // Sync down-stream module
EPwm1Regs.CMPCTL.bit.SHDWAMODE = CC_SHADOW;
EPwm1Regs.CMPCTL.bit.SHDWBMODE = CC_SHADOW;
EPwm1Regs.CMPCTL.bit.LOADAMODE = CC_CTR_ZERO; // load on CTR=Zero
EPwm1Regs.CMPCTL.bit.LOADBMODE = CC_CTR_ZERO; // load on CTR=Zero
EPwm1Regs.AQCTLA.bit.CAU = AQ_SET; // set actions for EPWM1A
EPwm1Regs.AQCTLA.bit.CAD = AQ_CLEAR;
EPwm1Regs.DBCTL.bit.OUT_MODE = DB_FULL_ENABLE; // enable Dead-band module
EPwm1Regs.DBCTL.bit.POLSEL = DB_ACTV_HIC; // Active Hi complementary
EPwm1Regs.DBFED = 20; // FED = 20 TBCLKs
EPwm1Regs.DBRED = 20; // RED = 20 TBCLKs

EPwm2Regs.TBPRD = 450; // EPWM Module 2 config
EPwm2Regs.TBPHS.half.TBPHS = 300; // Period = 900 TBCLK counts
EPwm2Regs.TBCTL.bit.CTRMODE = TB_COUNT_UPDOWN; // Phase = 300/900 * 360 = 120 deg
EPwm2Regs.TBCTL.bit.PHSEN = TB_ENABLE; // Symmetrical mode
EPwm2Regs.TBCTL.bit.PHSDIR = TB_DOWN; // Slave module
EPwm2Regs.TBCTL.bit.PRDL = TB_SHADOW; // Count DOWN on sync (=120 deg)
EPwm2Regs.TBCTL.bit.SYNCSEL = TB_SYNC_IN; // sync flow-through
EPwm2Regs.CMPCTL.bit.SHDWAMODE = CC_SHADOW;
EPwm2Regs.CMPCTL.bit.SHDWBMODE = CC_SHADOW;
EPwm2Regs.CMPCTL.bit.LOADAMODE = CC_CTR_ZERO; // load on CTR=Zero
EPwm2Regs.CMPCTL.bit.LOADBMODE = CC_CTR_ZERO; // load on CTR=Zero
EPwm2Regs.AQCTLA.bit.CAU = AQ_SET; // set actions for EPWM2A
EPwm2Regs.AQCTLA.bit.CAD = AQ_CLEAR;
EPwm2Regs.DBCTL.bit.OUT_MODE = DB_FULL_ENABLE; // enable dead-band module
EPwm2Regs.DBCTL.bit.POLSEL = DB_ACTV_HIC; // Active Hi Complementary
EPwm2Regs.DBFED = 20; // FED = 20 TBCLKs
EPwm2Regs.DBRED = 20; // RED = 20 TBCLKs

EPwm3Regs.TBPRD = 450; // EPWM Module 3 config
EPwm3Regs.TBPHS.half.TBPHS = 300; // Period = 900 TBCLK counts
EPwm3Regs.TBCTL.bit.CTRMODE = TB_COUNT_UPDOWN; // Phase = 300/900 * 360 = 120 deg
EPwm3Regs.TBCTL.bit.PHSEN = TB_ENABLE; // Symmetrical mode
EPwm3Regs.TBCTL.bit.PHSDIR = TB_UP; // Slave module
EPwm3Regs.TBCTL.bit.PRDL = TB_SHADOW; // Count UP on sync (=240 deg)
EPwm3Regs.TBCTL.bit.SYNCSEL = TB_SYNC_IN; // sync flow-through
EPwm3Regs.CMPCTL.bit.SHDWAMODE = CC_SHADOW;
EPwm3Regs.CMPCTL.bit.SHDWBMODE = CC_SHADOW;
EPwm3Regs.CMPCTL.bit.LOADAMODE = CC_CTR_ZERO; // load on CTR=Zero
EPwm3Regs.CMPCTL.bit.LOADBMODE = CC_CTR_ZERO; // load on CTR=Zero
EPwm3Regs.AQCTLA.bit.CAU = AQ_SET; // set actions for EPWM3Ai
EPwm3Regs.AQCTLA.bit.CAD = AQ_CLEAR;
EPwm3Regs.DBCTL.bit.OUT_MODE = DB_FULL_ENABLE; // enable Dead-band module
EPwm3Regs.DBCTL.bit.POLSEL = DB_ACTV_HIC; // Active Hi complementary
EPwm3Regs.DBFED = 20; // FED = 20 TBCLKs
EPwm3Regs.DBRED = 20; // RED = 20 TBCLKs
// Run Time (Note: Example execution of one run-time instant)
//=====
EPwm1Regs.CMPA.half.CMPA = 285; // adjust duty for output EPWM1A
EPwm2Regs.CMPA.half.CMPA = 285; // adjust duty for output EPWM2A
EPwm3Regs.CMPA.half.CMPA = 285; // adjust duty for output EPWM3A
```

### 3.3.9 Controlling Zero Voltage Switched Full Bridge (ZVSFB) Converter

The example given in [Figure 3-66](#) assumes a static or constant phase relationship between legs (modules). In such a case, control is achieved by modulating the duty cycle. It is also possible to dynamically change the phase value on a cycle-by-cycle basis. This feature lends itself to controlling a class of power topologies known as *phase-shifted full bridge*, or *zero voltage switched full bridge*. Here the controlled parameter is not duty cycle (this is kept constant at approximately 50 percent); instead it is the phase relationship between legs. Such a system can be implemented by allocating the resources of two PWM modules to control a single power stage, which in turn requires control of four switching elements. [Figure 3-67](#) shows a master/slave module combination synchronized together to control a full H-bridge. In this case, both master and slave modules are required to switch at the same PWM frequency. The phase is controlled by using the slave's phase register (TBPHS). The master's phase register is not used and therefore can be initialized to zero.



**Figure 3-66. Controlling a Full-H Bridge Stage ( $F_{PWM2} = F_{PWM1}$ )**

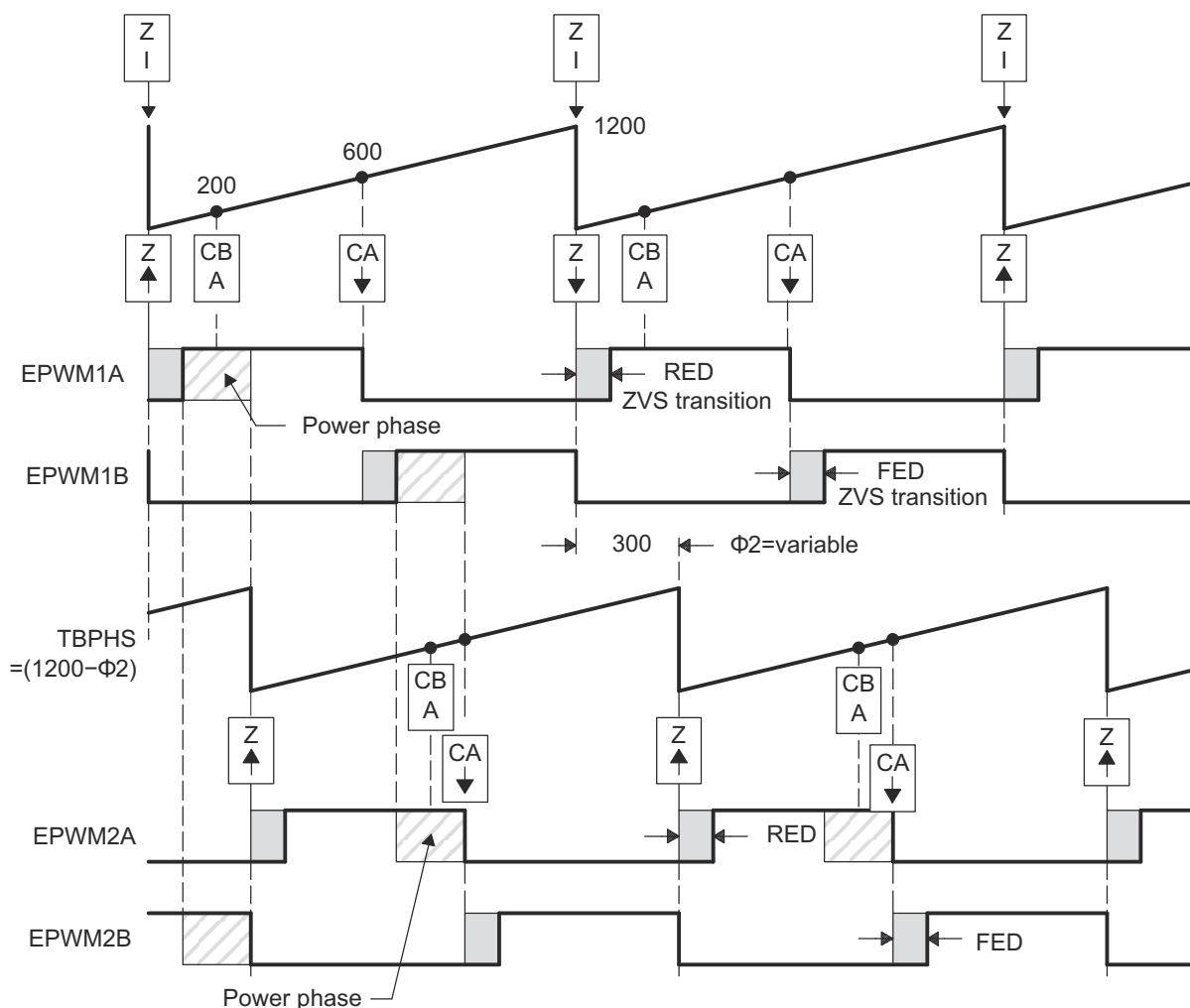


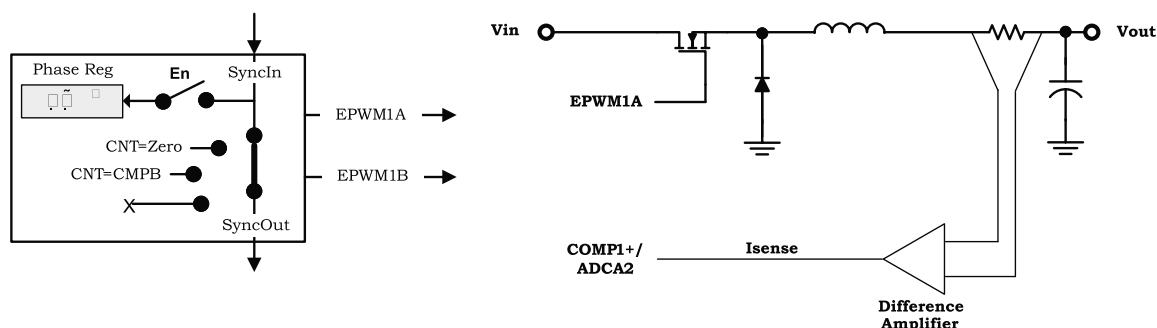
Figure 3-67. ZVS Full-H Bridge Waveforms

**Example 3-13. Code Snippet for Configuration in Figure 3-66**

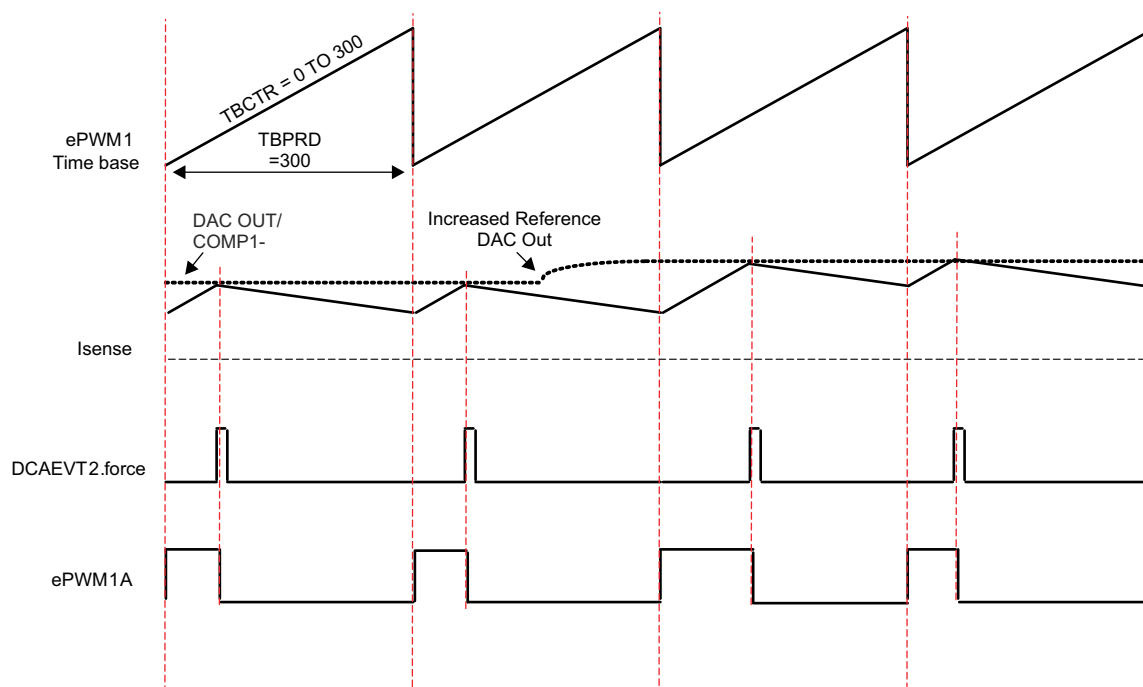
```
//=====
// Config
//=====
// Initialization Time
//=====
// EPWM Module 1 config
EPwm1Regs.TBPRD = 1200; // Period = 1201 TBCLK counts
EPwm1Regs.CMPA.half.CMPA = 600; // Set 50% fixed duty for EPWM1A
EPwm1Regs.TBPHS.half.TBPHS = 0; // Set Phase register to zero
EPwm1Regs.TBCTL.bit.CTRMODE = TB_COUNT_UP; // Asymmetrical mode
EPwm1Regs.TBCTL.bit.PHSEN = TB_DISABLE; // Master module
EPwm1Regs.TBCTL.bit.PRDL = TB_SHADOW;
EPwm1Regs.TBCTL.bit.SYNCSEL = TB_CTR_ZERO; // Sync down-stream module
EPwm1Regs.CMPCTL.bit.SHDWAMODE = CC_SHADOW;
EPwm1Regs.CMPCTL.bit.SHDWBMODE = CC_SHADOW;
EPwm1Regs.CMPCTL.bit.LOADAMODE = CC_CTR_ZERO; // load on CTR=Zero
EPwm1Regs.CMPCTL.bit.LOADBMODE = CC_CTR_ZERO; // load on CTR=Zero
EPwm1Regs.AQCTLA.bit.ZRO = AQ_SET; // set actions for EPWM1A
EPwm1Regs.AQCTLA.bit.CAU = AQ_CLEAR;
EPwm1Regs.DBCTL.bit.OUT_MODE = DB_FULL_ENABLE; // enable Dead-band module
EPwm1Regs.DBCTL.bit.POLSEL = DB_ACTV_HIC; // Active Hi complementary
EPwm1Regs.DBFED = 50; // FED = 50 TBCLKs initially
EPwm1Regs.DBRED = 70; // RED = 70 TBCLKs initially
// EPWM Module 2 config
EPwm2Regs.TBPRD = 1200; // Period = 1201 TBCLK counts
EPwm2Regs.CMPA.half.CMPA = 600; // Set 50% fixed duty EPWM2A
EPwm2Regs.TBPHS.half.TBPHS = 0; // Set Phase register to zero initially
EPwm2Regs.TBCTL.bit.CTRMODE = TB_COUNT_UP; // Asymmetrical mode
EPwm2Regs.TBCTL.bit.PHSEN = TB_ENABLE; // Slave module
EPwm2Regs.TBCTL.bit.PRDL = TB_SHADOW;
EPwm2Regs.TBCTL.bit.SYNCSEL = TB_SYNC_IN; // sync flow-through
EPwm2Regs.CMPCTL.bit.SHDWAMODE = CC_SHADOW;
EPwm2Regs.CMPCTL.bit.SHDWBMODE = CC_SHADOW;
EPwm2Regs.CMPCTL.bit.LOADAMODE = CC_CTR_ZERO; // load on CTR=Zero
EPwm2Regs.CMPCTL.bit.LOADBMODE = CC_CTR_ZERO; // load on CTR=Zero
EPwm2Regs.AQCTLA.bit.ZRO = AQ_SET; // set actions for EPWM2A
EPwm2Regs.AQCTLA.bit.CAU = AQ_CLEAR;
EPwm2Regs.DBCTL.bit.OUT_MODE = DB_FULL_ENABLE; // enable Dead-band module
EPwm2Regs.DBCTL.bit.POLSEL = DB_ACTV_HIC; // Active Hi complementary
EPwm2Regs.DBFED = 30; // FED = 30 TBCLKs initially
EPwm2Regs.DBRED = 40; // RED = 40 TBCLKs initially
// Run Time (Note: Example execution of one run-time instant)
//=====
EPwm2Regs.TBPHS = 1200-300; // Set Phase reg to
// 300/1200 * 360 = 90 deg
EPwm1Regs.DBFED = FED1_NewValue; // Update ZVS transition interval
EPwm1Regs.DBRED = RED1_NewValue; // Update ZVS transition interval
EPwm2Regs.DBFED = FED2_NewValue; // Update ZVS transition interval
EPwm2Regs.DBRED = RED2_NewValue; // Update ZVS transition interval
EPwm1Regs.CMPB = 200; // Adjust point-in-time for ADCSOC
trigger
```

### 3.3.10 Controlling a Peak Current Mode Controlled Buck Module

Peak current control techniques offer a number of benefits like automatic over current limiting, fast correction for input voltage variations, and reducing magnetic saturation. Figure 3-68 shows the use of ePWM1A along with the on-chip analog comparator for buck converter topology. The output current is sensed through a current sense resistor and fed to the positive terminal of the on-chip comparator. The internal programmable 10-bit DAC can be used to provide a reference peak current at the negative terminal of the comparator. Alternatively, an external reference could be connected at this input. The comparator output is an input to the digital compare submodule. The ePWM module is configured in such a way so as to trip the ePWM1A output as soon as the sensed current reaches the peak reference value. A cycle-by-cycle trip mechanism is used. Figure 3-69 shows the waveforms generated by the configuration.



**Figure 3-68. Peak Current Mode Control of a Buck Converter**



**Figure 3-69. Peak Current Mode Control Waveforms for Figure 3-68**

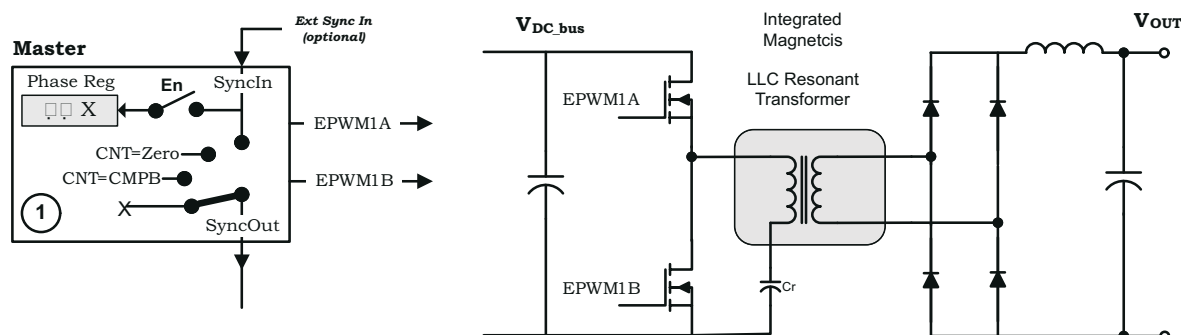
**Example 3-14. Code Snippet for Configuration in Figure 3-68**

```
//=====
// Config
// Initialization Time
//=====
EPwm1Regs.TBPRD = 300;
// Period = 300 TBCLK counts // (200 KHz @ 60MHz clock)
EPwm1Regs.TBPHS.half.TBPHS = 0; // Set Phase register to zero
EPwm1Regs.TBCTL.bit.CTRMODE = TB_COUNT_UP; // Asymmetrical mode
EPwm1Regs.TBCTL.bit.PHSEN = TB_DISABLE; // Phase loading disabled
EPwm1Regs.TBCTL.bit.HSPCLKDIV = TB_DIV1; // Clock ratio to SYSCLKOUT
EPwm1Regs.TBCTL.bit.CLKDIV = TB_DIV1;
EPwm1Regs.TBCTL.bit.PRDL = TB_SHADOW;
Pwm1Regs.AQCTLA.bit.ZRO = AQ_SET; // Set PWM1A on Zero
// Define an event (DCAEVT2) based on
// Comparator 1 Output
EPwm1Regs.DCTRIPSEL.bit.DCAHCOMPSEL = DC_COMP1OUT; // DCAH = Comparator 1 output

EPwm1Regs.TZDCSEL.bit.DCAEVT2 = TZ_DCAH_HI; // DCAEVT2 = DCAH high(will become active
// as Comparator output goes high)
EPwm1Regs.DCACTL.bit.EVT2SRCSEL = DC_EVT2; // DCAEVT2 = DCAEVT2 (not filtered)
EPwm1Regs.DCACTL.bit.EVT2FRCSYNCSSEL = DC_EVT_ASYNC; // Take async path // Enable DCAEVT2 as a
// one-shot trip source
// Note: DCxEVT1 events can be defined as
// one-shot.
// DCxEVT2 events can be defined as
// cycle-by-cycle.
EPwm1Regs.TZSEL.bit.DCAEVT2 = 1; // What do we want the DCAEVT1 and DCBEVT1
// events to do?
// DCAEVTx events can force EPWMxA
// DCBEVTx events can force EPWMxB
EPwm1Regs.TZCTL.bit.TZA = TZ_FORCE_LO; // EPWM1A will go low
//=====
// Run Time
//=====
// Adjust reference peak current to Comparator 1 negative input
```

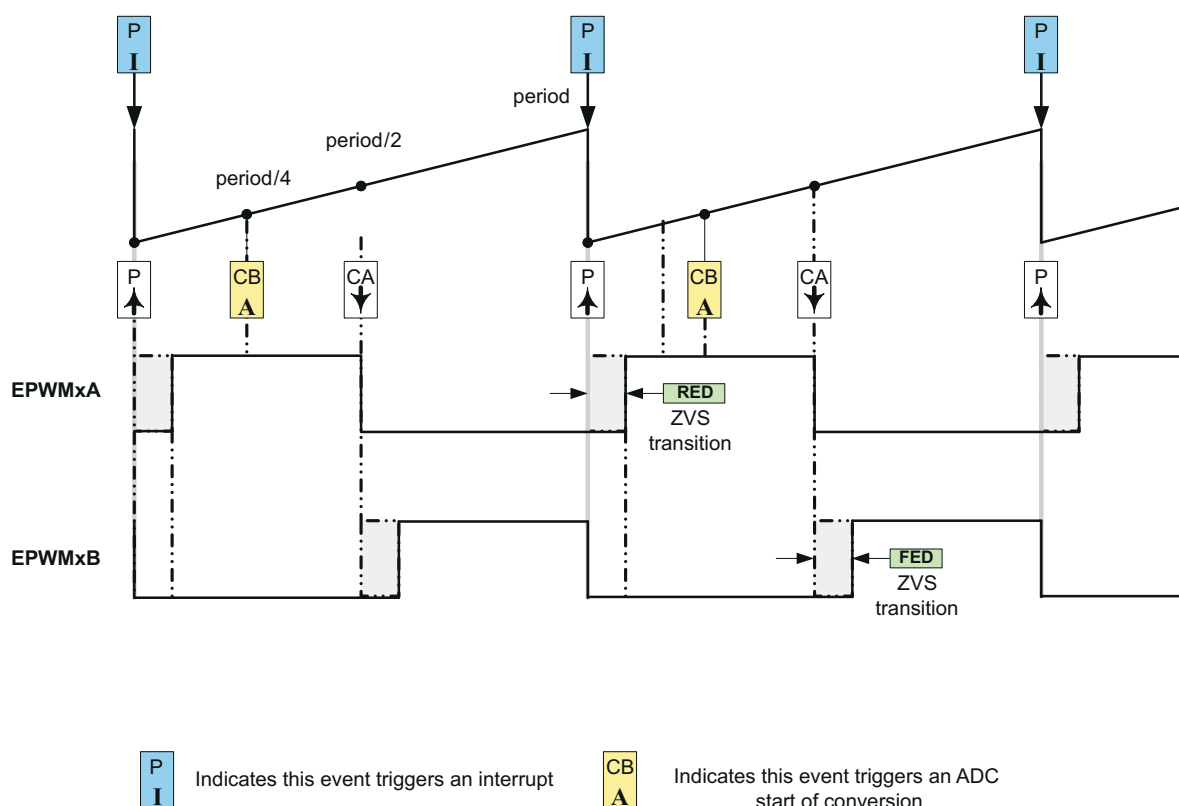
### 3.3.11 Controlling H-Bridge LLC Resonant Converter

For many years, various topologies of resonant converters have been well-known in the field of power electronics. In addition to these, H-bridge LLC resonant converter topology has recently gained popularity in many consumer electronics applications where high efficiency and power density are required. In this example, the single channel configuration of ePWM1 is detailed, yet the configuration can easily be extended to multichannel. Here, the controlled parameter is not duty cycle (this is kept constant at approximately 50 percent); instead it is frequency. Although the deadband is not controlled and kept constant as 300ns (that is, 18@60MHz TBCLK), it is up to the user to update it in real time to enhance the efficiency by adjusting enough time delay for soft switching.



NOTE:  $\Theta = X$  indicates value in phase register is 'don't care'

**Figure 3-70. Control of Two Resonant Converter Stages**



**Figure 3-71. H-Bridge LLC Resonant Converter PWM Waveforms**



**Example 3-15. Code Snippet for Configuration in Figure 3-70**

```
//=====
// Config
//===== //
Initialization Time
//===== //
EPWMxA & EPWMxB config
EPwm1Regs.TBCTL.bit.PRDL = TB_IMMEDIATE; // Set immediate load
EPwm1Regs.TBPRD = period; // PWM frequency = 1 / period
EPwm1Regs.CMPA.half.CMPA = period/2; // Set duty as 50%
EPwm1Regs.CMPB = period/4; // Set duty as 25%
EPwm1Regs.TBPHS.half.TBPHS = 0; // Set as master, phase =0
EPwm1Regs.TBCTR = 0; // Time base counter =0
EPwm1Regs.TBCTL.bit.CTRMODE = TB_COUNT_UP; // Count-up mode: used for asymmetric PWM
EPwm1Regs.TBCTL.bit.PHSEN = TB_DISABLE; // Disable phase loading
EPwm1Regs.TBCTL.bit.SYNCSEL = TB_CTR_ZERO; // Used to sync EPWM(n+1) "down-stream"
EPwm1Regs.TBCTL.bit.HSPCLKDIV = TB_DIV1; // Set the clock rate
EPwm1Regs.TBCTL.bit.CLKDIV = TB_DIV1; // Set the clock rate
EPwm1Regs.CMPCTL.bit.LOADAMODE = CC_CTR_PRD; // Load on CTR=PRD
EPwm1Regs.CMPCTL.bit.LOADBMODE = CC_CTR_PRD; // Load on CTR=PRD
EPwm1Regs.CMPCTL.bit.SHADOWMODE = CC_SHADOW; // Shadow mode. Operates as a double buffer.
EPwm1Regs.CMPCTL.bit.SHDBMODE = CC_SHADOW; // Shadow mode. Operates as a double buffer.
EPwm1Regs.AQCTLA.bit.ZRO = AQ_SET; // Set PWM1A on Zero
EPwm1Regs.AQCTLA.bit.CAU = AQ_CLEAR; // Clear PWM1A on event A, up count
EPwm1Regs.AQCTLB.bit.CAU = AQ_SET; // Set PWM1B on event A, up count
EPwm1Regs.AQCTLB.bit.PR = AQ_CLEAR; // Clear PWM1B on PRD
EPwm1Regs.DBCTL.bit.IN_MODE = DBA_ALL; // EPWMxA is the source for both delays
EPwm1Regs.DBCTL.bit.OUT_MODE = DB_FULL_ENABLE; // Enable Dead-band module
EPwm1Regs.DBCTL.bit.POLSEL = DB_ACTV_HIC; // Active High Complementary (AHC)
EPwm1Regs.DBRED = 30; // RED = 30 TBCLKs initially
EPwm1Regs.DBFED = 30; // FED = 30 TBCLKs initially
// Configure TZ1 for short cct
// protection EALLOW;
// one-shot source EPwm1Regs.TZCTL.bit.TZA =

EPwm1Regs.TZSEL.bit.OSHT1 = 1;
TZ_FORCE_LO;

// set EPWM1A to low at fault
// set EPWM1B to low at fault instant
// Enable TZ interrupt EDIS;
// Enable HiRes option EALLOW;

EPwm1Regs.HRCNFG.all = 0x0;
EPwm1Regs.HRCNFG.bit.EDGMODE = HR_FEP;
EPwm1Regs.HRCNFG.bit.CTLMODE = HR_CMP;
EPwm1Regs.HRCNFG.bit.HRLOAD = HR_CTR_PRD;
EDIS; // Run Time (Note: Example execution of
// one run-time instant)

//=====
EPwm1Regs.TBPRD = period_new value; // Update new period
// EPwm1Regs.CMPA.half.CMPA= period_new
// value/2;
// Update new CMPA EPwm1Regs.CMPB= period_new
// value/4;
// Update new CMPB
// Update new CMPB
// Update new CMPB
```

## 3.4 Registers

This section includes the register layouts and bit description for the submodules.

### 3.4.1 Time-Base Submodule Registers

This section describes the time-base submodule registers.

#### 3.4.1.1 Time-Base Control Register (TBCTL)

**Figure 3-72. Time-Base Control Register (TBCTL)**

|            |         |          |        |       |           |         |   |
|------------|---------|----------|--------|-------|-----------|---------|---|
| 15         | 14      | 13       | 12     | 10    | 9         | 8       |   |
| FREE, SOFT |         | PHSDIR   | CLKDIV |       | HSPCLKDIV |         |   |
| R/W-0      |         | R/W-0    | R/W-0  |       | R/W-0,0,1 |         |   |
| 7          | 6       | 5        | 4      | 3     | 2         | 1       | 0 |
| HSPCLKDIV  | SWFSYNC | SYNCOSEL |        | PRDLD | PHSEN     | CTRMODE |   |
| R/W-0,0,1  | R/W-0   | R/W-0    |        | R/W-0 | R/W-0     | R/W-11  |   |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 3-21. Time-Base Control Register (TBCTL) Field Descriptions**

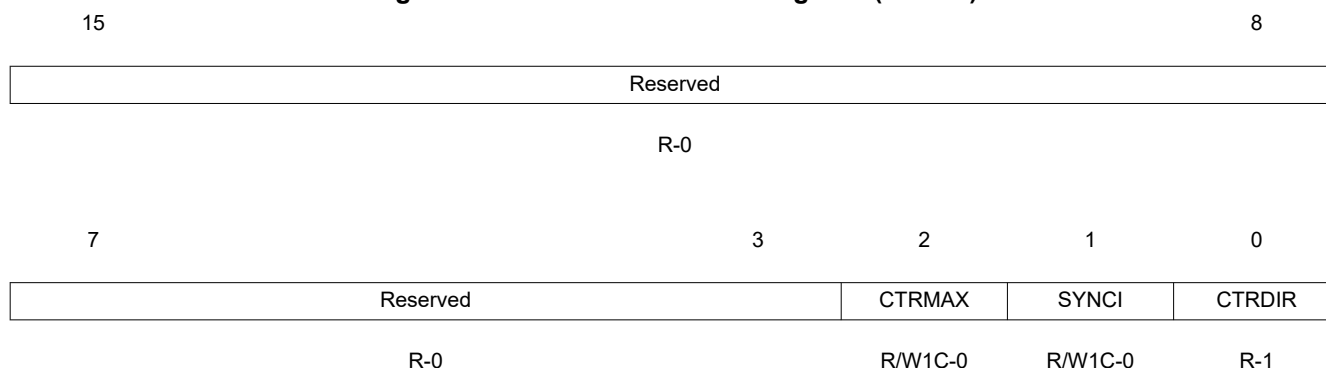
| Bit   | Field      | Value                                                | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|-------|------------|------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-14 | FREE, SOFT | 00<br>01<br>1x                                       | Emulation Mode Bits. These bits select the behavior of the ePWM time-base counter during emulation events:<br>00 Stop after the next time-base counter increment or decrement<br>01 Stop when counter completes a whole cycle:<br><ul style="list-style-type: none"> <li>Up-count mode: stop when the time-base counter = period (TBCTR = TBPRD)</li> <li>Down-count mode: stop when the time-base counter = 0x0000 (TBCTR = 0x0000)</li> <li>Up-down-count mode: stop when the time-base counter = 0x0000 (TBCTR = 0x0000)</li> </ul> 1x Free run           |
| 13    | PHSDIR     | 0<br>1                                               | Phase Direction Bit.<br>This bit is only used when the time-base counter is configured in the up-down-count mode. The PHSDIR bit indicates the direction the time-base counter (TBCTR) will count after a synchronization event occurs and a new phase value is loaded from the phase (TBPHS) register. This is irrespective of the direction of the counter before the synchronization event..<br>In the up-count and down-count modes this bit is ignored.<br>0 Count down after the synchronization event.<br>1 Count up after the synchronization event. |
| 12-10 | CLKDIV     | 000<br>001<br>010<br>011<br>100<br>101<br>110<br>111 | Time-base Clock Prescale Bits<br>These bits determine part of the time-base clock prescale value.<br>$TBCLK = SYSCLKOUT / (HSPCLKDIV \times CLKDIV)$<br>/1 (default on reset)<br>/2<br>/4<br>/8<br>/16<br>/32<br>/64<br>/128                                                                                                                                                                                                                                                                                                                                 |

**Table 3-21. Time-Base Control Register (TBCTL) Field Descriptions (continued)**

| Bit | Field     | Value                                                                                                                                                                 | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|-----|-----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 9-7 | HSPCLKDIV | <div>000 /1</div> <div>001 /2 (default on reset)</div> <div>010 /4</div> <div>011 /6</div> <div>100 /8</div> <div>101 /10</div> <div>110 /12</div> <div>111 /14</div> | <div>High Speed Time-base Clock Prescale Bits</div> <div>These bits determine part of the time-base clock prescale value.<br/>TBCLK = SYSCLKOUT / (HSPCLKDIV × CLKDIV)</div> <div>This divisor emulates the HSPCLK in the TMS320x281x system as used on the Event Manager (EV) peripheral.</div>                                                                                                                                                                                                                                                           |
| 6   | SWFSYNC   | <div>0</div> <div>1</div>                                                                                                                                             | <div>Software Forced Synchronization Pulse</div> <div>Writing a 0 has no effect and reads always return a 0.</div> <div>Writing a 1 forces a one-time synchronization pulse to be generated.</div> <div>This event is ORed with the EPWMxSYNCl input of the ePWM module.</div> <div>SWFSYNC is valid (operates) only when EPWMxSYNCl is selected by SYNCOSSEL = 00.</div>                                                                                                                                                                                  |
| 5-4 | SYNCOSSEL | <div>00</div> <div>01</div> <div>10</div> <div>11</div>                                                                                                               | <div>Synchronization Output Select. These bits select the source of the EPWMxSYNCO signal.</div> <div>EPWMxSYNC:</div> <div>CTR = zero: Time-base counter equal to zero (TBCTR = 0x0000)</div> <div>CTR = CMPB : Time-base counter equal to counter-compare B (TBCTR = CMPB)</div> <div>Disable EPWMxSYNCO signal</div>                                                                                                                                                                                                                                    |
| 3   | PRDLD     | <div>0</div> <div>1</div>                                                                                                                                             | <div>Active Period Register Load From Shadow Register Select</div> <div>The period register (TBPRD) is loaded from its shadow register when the time-base counter, TBCTR, is equal to zero.</div> <div>A write or read to the TBPRD register accesses the shadow register.</div> <div>Load the TBPRD register immediately without using a shadow register.</div> <div>A write or read to the TBPRD register directly accesses the active register.</div>                                                                                                   |
| 2   | PHSEN     | <div>0</div> <div>1</div>                                                                                                                                             | <div>Counter Register Load From Phase Register Enable</div> <div>Do not load the time-base counter (TBCTR) from the time-base phase register (TBPHS)</div> <div>Load the time-base counter with the phase register when an EPWMxSYNCl input signal occurs or when a software synchronization is forced by the SWFSYNC bit, or when a digital compare sync event occurs.</div>                                                                                                                                                                              |
| 1-0 | CTRMODE   | <div>00</div> <div>01</div> <div>10</div> <div>11</div>                                                                                                               | <div>Counter Mode</div> <div>The time-base counter mode is normally configured once and not changed during normal operation. If you change the mode of the counter, the change will take effect at the next TBCLK edge and the current counter value shall increment or decrement from the value before the mode change.</div> <div>These bits set the time-base counter mode of operation as follows:</div> <div>Up-count mode</div> <div>Down-count mode</div> <div>Up-down-count mode</div> <div>Stop-freeze counter operation (default on reset)</div> |

### 3.4.1.2 Time-Base Status Register (TBSTS)

**Figure 3-73. Time-Base Status Register (TBSTS)**



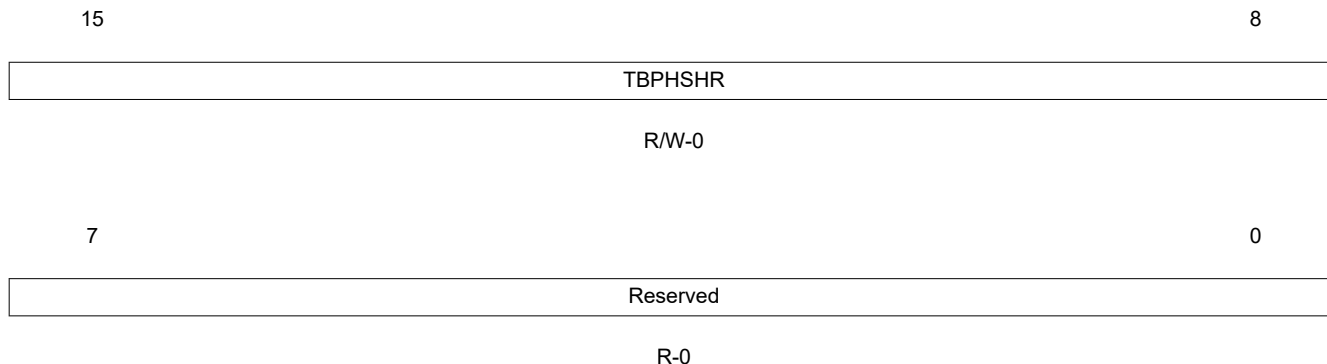
LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 3-22. Time-Base Status Register (TBSTS) Field Descriptions**

| Bit  | Field    | Value  | Description                                                                                                                                                                                                                                                                                                     |
|------|----------|--------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-3 | Reserved |        | Reserved                                                                                                                                                                                                                                                                                                        |
| 2    | CTRMAX   | 0<br>1 | Time-Base Counter Max Latched Status Bit<br>Reading a 0 indicates the time-base counter never reached its maximum value. Writing a 0 will have no effect.<br>Reading a 1 on this bit indicates that the time-base counter reached the max value 0xFFFF. Writing a 1 to this bit will clear the latched event.   |
| 1    | SYNCI    | 0<br>1 | Input Synchronization Latched Status Bit<br>Writing a 0 will have no effect. Reading a 0 indicates no external synchronization event has occurred.<br>Reading a 1 on this bit indicates that an external synchronization event has occurred (EPWMxSYNCI). Writing a 1 to this bit will clear the latched event. |
| 0    | CTRDIR   | 0<br>1 | Time-Base Counter Direction Status Bit. At reset, the counter is frozen; therefore, this bit has no meaning. To make this bit meaningful, you must first set the appropriate mode via TBCTL[CTRMODE].<br>0 Time-Base Counter is currently counting down.<br>1 Time-Base Counter is currently counting up.       |

### 3.4.1.3 Time-Base Phase High Resolution Register (TBPHSHR)

**Figure 3-74. Time-Base Phase High Resolution Register (TBPHSHR)**



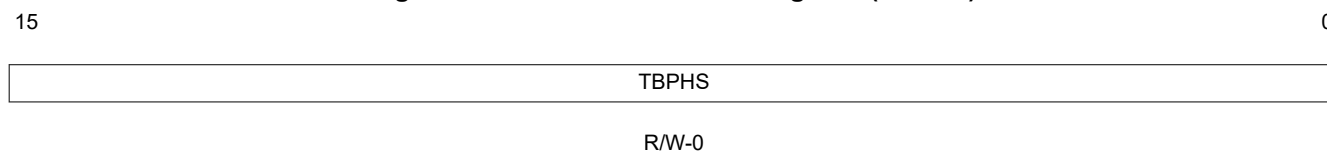
LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 3-23. Time-Base Phase High Resolution Register (TBPHSHR) Field Descriptions**

| Bit  | Field    | Value  | Description                          |
|------|----------|--------|--------------------------------------|
| 15-8 | TBPHSHR  | 00-FFh | Time base phase high-resolution bits |
| 7-0  | Reserved |        | Reserved                             |

### 3.4.1.4 Time-Base Phase Register (TBPHS)

**Figure 3-75. Time-Base Phase Register (TBPHS)**



LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 3-24. Time-Base Phase Register (TBPHS) Field Descriptions**

| Bits | Name  | Value     | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|------|-------|-----------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-0 | TBPHS | 0000-FFFF | <p>These bits set time-base counter phase of the selected ePWM relative to the time-base that is supplying the synchronization input signal.</p> <ul style="list-style-type: none"> <li>If TBCTL[PHSEN] = 0, then the synchronization event is ignored and the time-base counter is not loaded with the phase.</li> <li>If TBCTL[PHSEN] = 1, then the time-base counter (TBCTR) will be loaded with the phase (TBPHS) when a synchronization event occurs. The synchronization event can be initiated by the input synchronization signal (EPWMxSYNCl) or by a software forced synchronization.</li> </ul> |

### 3.4.1.5 Time-Base Counter Register (TBCTR)

**Figure 3-76. Time-Base Counter Register (TBCTR)**

15

0

|       |
|-------|
| TBCTR |
|-------|

R/W-0

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 3-25. Time-Base Counter Register (TBCTR) Field Descriptions**

| Bits | Name  | Value     | Description                                                                                                                                                                                                                                                                                |
|------|-------|-----------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-0 | TBCTR | 0000-FFFF | <p>Reading these bits gives the current time-base counter value.</p> <p>Writing to these bits sets the current time-base counter value. The update happens as soon as the write occurs; the write is NOT synchronized to the time-base clock (TBCLK) and the register is not shadowed.</p> |

### 3.4.1.6 Time-Base Period Register (TBPRD)

**Figure 3-77. Time-Base Period Register (TBPRD)**

15

0

|       |
|-------|
| TBPRD |
|-------|

R/W-0

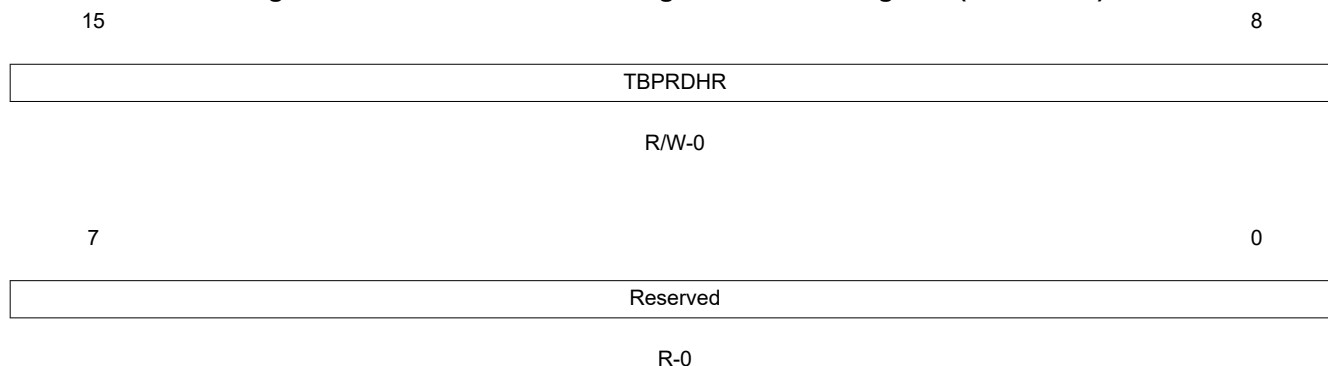
LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 3-26. Time-Base Period Register (TBPRD) Field Descriptions**

| Bit  | Field | Value      | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|------|-------|------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-0 | TBPRD | 0000-FFFFh | <p>These bits determine the period of the time-base counter. This sets the PWM frequency. Shadowing of this register is enabled and disabled by the TBCTL[PRDLD] bit. By default this register is shadowed.</p> <ul style="list-style-type: none"> <li>If TBCTL[PRDLD] = 0, then the shadow is enabled and any write or read will automatically go to the shadow register. In this case, the active register will be loaded from the shadow register when the time-base counter equals zero.</li> <li>If TBCTL[PRDLD] = 1, then the shadow is disabled and any write or read will go directly to the active register, that is the register actively controlling the hardware.</li> <li>The active and shadow registers share the same memory map address.</li> </ul> |

### 3.4.1.7 Time Base Period High Resolution Register (TBPRDHR)

**Figure 3-78. Time Base Period High Resolution Register (TBPRDHR)**



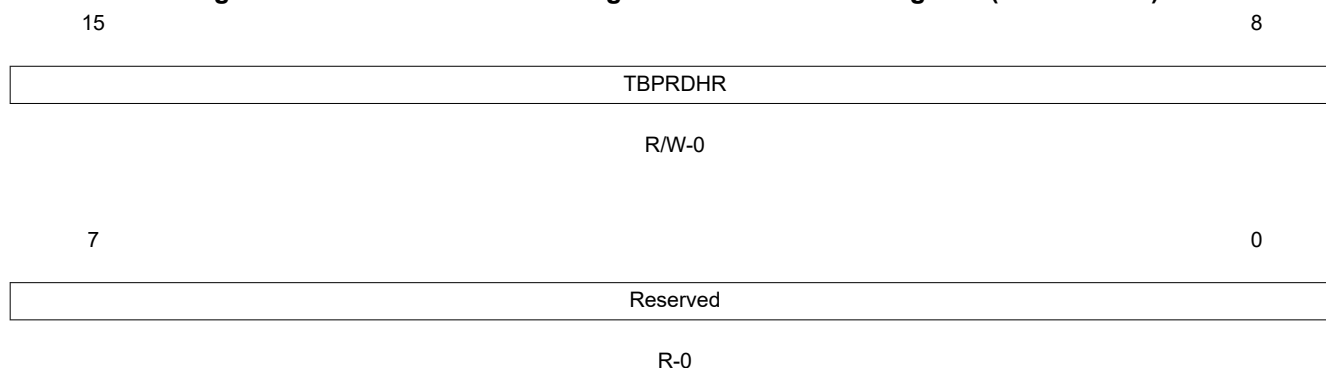
LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 3-27. Time Base Period High Resolution Register (TBPRDHR) Field Descriptions**

| Bit  | Field    | Value  | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|------|----------|--------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-8 | TBPRDHR  | 00-FFh | Period High Resolution Bits<br>These 8-bits contain the high-resolution portion of the period value.<br>The TBPRDHR register is not affected by the TBCTL[PRDL] bit. Reads from this register always reflect the shadow register. Likewise writes are also to the shadow register. The TBPRDHR register is only used when the high resolution period feature is enabled.<br>This register is only available with ePWM modules which support high-resolution period control. |
| 7-0  | Reserved | 0      | Reserved                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |

### 3.4.1.8 Time-Base Period High Resolution Mirror Register (TBPRDHRM)

**Figure 3-79. Time-Base Period High Resolution Mirror Register (TBPRDHRM)**



LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 3-28. Time-Base Period High Resolution Mirror Register (TBPRDHRM) Field Descriptions**

| Bit  | Field    | Value  | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|------|----------|--------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-8 | TBPRDHR  | 00-FFh | <p>Period High Resolution Bits</p> <p>These 8-bits contain the high-resolution portion of the period value</p> <p>TBPRD provides backwards compatibility with earlier ePWM modules. The mirror registers (TBPRDM and TBPRDHRM) allow for 32-bit writes to TBPRDHR in one access. Due to the odd-numbered memory address location of the TBPRD legacy register, a 32-bit write is not possible with TBPRD and TBPRDHR.</p> <p>The TBPRDHRM register is not affected by the TBCTL[PRDLD] bit</p> <p>Writes to both the TBPRDHR and TBPRDM locations access the high-resolution (least significant 8-bit) portion of the Time Base Period value. The only difference is that unlike TBPRDHR, reads from the mirror register TBPRDHRM, are indeterminate (reserved for TI Test).</p> <p>The TBPRDHRM register is available with ePWM modules which support high-resolution period control and is used only when the high resolution period feature is enabled.</p> |
| 7-0  | Reserved | 00-FFh | Reserved for TI Test                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |



### 3.4.1.9 Time-Base Period Mirror Register (TBPRDM)

**Figure 3-80. Time-Base Period Mirror Register (TBPRDM)**

15

0

|       |
|-------|
| TBPRD |
|-------|

R/W-0

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 3-29. Time-Base Period Mirror Register (TBPRDM) Field Descriptions**

| Bit  | Field | Value      | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|------|-------|------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-0 | TBPRD | 0000-FFFFh | <p>TBPRDM and TBPRD can both be used to access the time-base period.</p> <p>TBPRD provides backwards compatibility with earlier ePWM modules. The mirror registers (TBPRDM and TBPRDHRM) allow for 32-bit writes to TBPRDHR in one access. Due to the odd address memory location of the TBPRD legacy register, a 32-bit write is not possible.</p> <p>By default writes to this register are shadowed. Unlike the TBPRD register, reads of TBPRDM always return the active register value. Shadowing is enabled and disabled by the TBCTL[PRDL] bit.</p> <ul style="list-style-type: none"> <li>If TBCTL[PRDL] = 0, then the shadow is enabled and any write will automatically go to the shadow register. In this case the active register will be loaded from the shadow register when the time-base counter equals zero. Reads return the active value.</li> <li>If TBCTL[PRDL] = 1, then the shadow is disabled and any write to this register will go directly to the active register controlling the hardware. Likewise reads return the active value.</li> </ul> |

### 3.4.2 Counter-Compare Submodule Registers

This section describes the counter-compare submodule control and status registers.

#### 3.4.2.1 Counter-Compare Control (CMPCTL) Register

**Figure 3-81. Counter-Compare Control (CMPCTL) Register**

|          |  |           |  |          |  |           |  |           |           |           |  |
|----------|--|-----------|--|----------|--|-----------|--|-----------|-----------|-----------|--|
| 15       |  |           |  | 10       |  |           |  | 9         |           | 8         |  |
| Reserved |  |           |  |          |  |           |  | SHDWBFULL | SHDWAFULL |           |  |
| R-0      |  |           |  |          |  |           |  | R-0       |           | R-0       |  |
| 7        |  | 6         |  | 5        |  | 4         |  | 3         |           | 2         |  |
|          |  |           |  |          |  |           |  | 1         |           | 0         |  |
| Reserved |  | SHDWBMODE |  | Reserved |  | SHDWAMODE |  | LOADBMODE |           | LOADAMODE |  |
| R-0      |  | R/W-0     |  | R-0      |  | R/W-0     |  | R/W-0     |           | R/W-0     |  |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

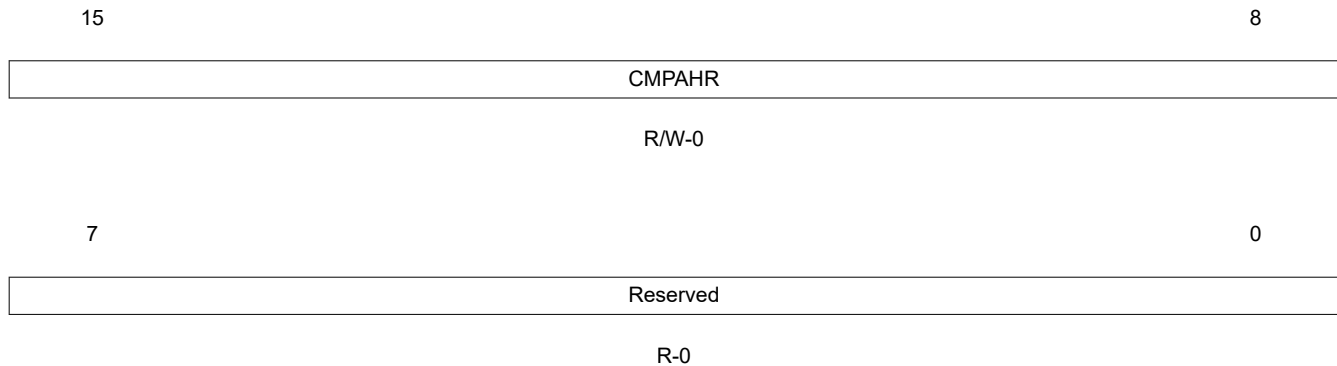
**Table 3-30. Counter-Compare Control (CMPCTL) Register Field Descriptions**

| Bits  | Name      | Value                | Description                                                                                                                                                                                                                                                                                                                                                                                                              |
|-------|-----------|----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-10 | Reserved  |                      | Reserved                                                                                                                                                                                                                                                                                                                                                                                                                 |
| 9     | SHDWBFULL | 0<br>1               | Counter-compare B (CMPB) Shadow Register Full Status Flag<br>This bit self clears once a load-strobe occurs.<br>CMPB shadow FIFO not full yet<br>Indicates the CMPB shadow FIFO is full; a CPU write will overwrite current shadow value.                                                                                                                                                                                |
| 8     | SHDWAFULL | 0<br>1               | Counter-compare A (CMPA) Shadow Register Full Status Flag<br>The flag bit is set when a 32-bit write to CMPA:CMPAHR register or a 16-bit write to CMPA register is made. A 16-bit write to CMPAHR register will not affect the flag.<br>This bit self clears once a load-strobe occurs.<br>CMPA shadow FIFO not full yet<br>Indicates the CMPA shadow FIFO is full, a CPU write will overwrite the current shadow value. |
| 7     | Reserved  |                      | Reserved                                                                                                                                                                                                                                                                                                                                                                                                                 |
| 6     | SHDWBMODE | 0<br>1               | Counter-compare B (CMPB) Register Operating Mode<br>Shadow mode. Operates as a double buffer. All writes via the CPU access the shadow register.<br>Immediate mode. Only the active compare B register is used. All writes and reads directly access the active register for immediate compare action.                                                                                                                   |
| 5     | Reserved  |                      | Reserved                                                                                                                                                                                                                                                                                                                                                                                                                 |
| 4     | SHDWAMODE | 0<br>1               | Counter-compare A (CMPA) Register Operating Mode<br>Shadow mode. Operates as a double buffer. All writes via the CPU access the shadow register.<br>Immediate mode. Only the active compare register is used. All writes and reads directly access the active register for immediate compare action                                                                                                                      |
| 3-2   | LOADBMODE | 00<br>01<br>10<br>11 | Active Counter-Compare B (CMPB) Load From Shadow Select Mode<br>This bit has no effect in immediate mode (CMPCTL[SHDWBMODE] = 1).<br>Load on CTR = Zero: Time-base counter equal to zero (TBCTR = 0x0000)<br>Load on CTR = PRD: Time-base counter equal to period (TBCTR = TBPRD)<br>Load on either CTR = Zero or CTR = PRD<br>Freeze (no loads possible)                                                                |

**Table 3-30. Counter-Compare Control (CMPCTL) Register Field Descriptions (continued)**

| Bits | Name      | Value | Description                                                                                                                     |
|------|-----------|-------|---------------------------------------------------------------------------------------------------------------------------------|
| 1-0  | LOADAMODE |       | Active Counter-Compare A (CMPA) Load From Shadow Select Mode. This bit has no effect in immediate mode (CMPCTL[SHDWAMODE] = 1). |
|      |           | 00    | Load on CTR = Zero: Time-base counter equal to zero (TBCTR = 0x0000)                                                            |
|      |           | 01    | Load on CTR = PRD: Time-base counter equal to period (TBCTR = TBPRD)                                                            |
|      |           | 10    | Load on either CTR = Zero or CTR = PRD                                                                                          |
|      |           | 11    | Freeze (no loads possible)                                                                                                      |

### 3.4.2.2 Compare A High Resolution (CMPAHR) Register

**Figure 3-82. Compare A High Resolution (CMPAHR) Register**


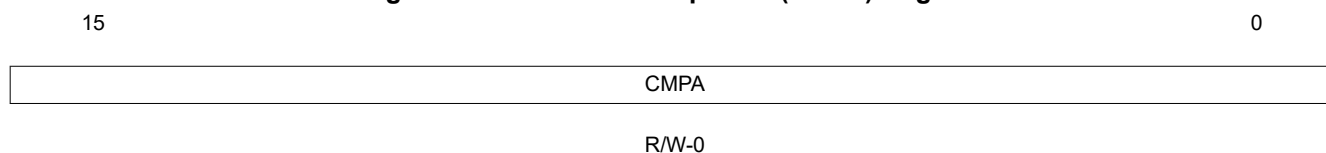
LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 3-31. Compare A High Resolution (CMPAHR) Register Field Descriptions**

| Bit  | Field    | Value  | Description                                                                                                                                                                                                                                                               |
|------|----------|--------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-8 | CMPAHR   | 00-FFh | These 8-bits contain the high-resolution portion (least significant 8-bits) of the counter-compare A value. CMPA:CMPAHR can be accessed in a single 32-bit read/write. Shadowing is enabled and disabled by the CMPCTL[SHDWAMODE] bit as described for the CMPA register. |
| 7-0  | Reserved |        | Reserved for TI Test                                                                                                                                                                                                                                                      |

### 3.4.2.3 Counter-Compare A (CMPA) Register

**Figure 3-83. Counter-Compare A (CMPA) Register**



LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 3-32. Counter-Compare A (CMPA) Register Field Descriptions**

| Bits | Name | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|------|------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-0 | CMPA | <p>The value in the active CMPA register is continuously compared to the time-base counter (TBCTR). When the values are equal, the counter-compare module generates a "time-base counter equal to counter compare A" event. This event is sent to the action-qualifier where it is qualified and converted it into one or more actions. These actions can be applied to either the EPWMxA or the EPWMxB output depending on the configuration of the AQCTLA and AQCTLB registers. The actions that can be defined in the AQCTLA and AQCTLB registers include:</p> <ul style="list-style-type: none"> <li>• Do nothing; the event is ignored.</li> <li>• Clear: Pull the EPWMxA and/or EPWMxB signal low</li> <li>• Set: Pull the EPWMxA and/or EPWMxB signal high</li> <li>• Toggle the EPWMxA and/or EPWMxB signal</li> </ul> <p>Shadowing of this register is enabled and disabled by the CMPCTL[SHDWAMODE] bit. By default this register is shadowed.</p> <ul style="list-style-type: none"> <li>• If CMPCTL[SHDWAMODE] = 0, then the shadow is enabled and any write or read will automatically go to the shadow register. In this case, the CMPCTL[LOADAMODE] bit field determines which event will load the active register from the shadow register.</li> <li>• Before a write, the CMPCTL[SHDWAFULL] bit can be read to determine if the shadow register is currently full.</li> <li>• If CMPCTL[SHDWAMODE] = 1, then the shadow register is disabled and any write or read will go directly to the active register, that is the register actively controlling the hardware.</li> <li>• In either mode, the active and shadow registers share the same memory map address.</li> </ul> |

### 3.4.2.4 Counter-Compare B (CMPB) Register

**Figure 3-84. Counter-Compare B (CMPB) Register**

15

0

|      |
|------|
| CMPB |
|------|

R/W-0

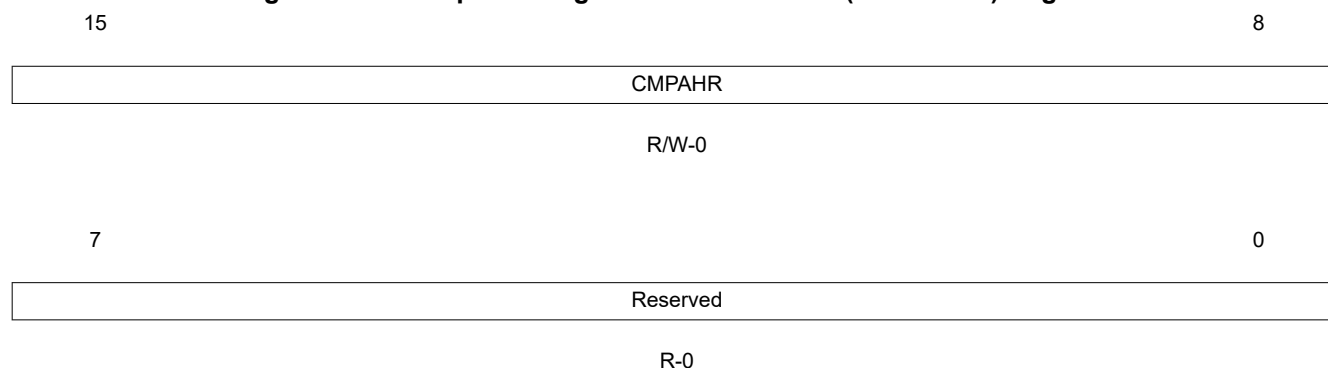
LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 3-33. Counter-Compare B (CMPB) Register Field Descriptions**

| Bits | Name | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
|------|------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-0 | CMPB | <p>The value in the active CMPB register is continuously compared to the time-base counter (TBCTR). When the values are equal, the counter-compare module generates a "time-base counter equal to counter compare B" event. This event is sent to the action-qualifier where it is qualified and converted it into one or more actions. These actions can be applied to either the EPWMxA or the EPWMxB output depending on the configuration of the AQCTLA and AQCTLB registers. The actions that can be defined in the AQCTLA and AQCTLB registers include:</p> <ul style="list-style-type: none"> <li>Do nothing. event is ignored.</li> <li>Clear: Pull the EPWMxA and/or EPWMxB signal low</li> <li>Set: Pull the EPWMxA and/or EPWMxB signal high</li> <li>Toggle the EPWMxA and/or EPWMxB signal</li> </ul> <p>Shadowing of this register is enabled and disabled by the CMPCTL[SHDWBMODE] bit. By default this register is shadowed.</p> <ul style="list-style-type: none"> <li>If CMPCTL[SHDWBMODE] = 0, then the shadow is enabled and any write or read will automatically go to the shadow register. In this case, the CMPCTL[LOADBMODE] bit field determines which event will load the active register from the shadow register:</li> <li>Before a write, the CMPCTL[SHDWBFULL] bit can be read to determine if the shadow register is currently full.</li> <li>If CMPCTL[SHDWBMODE] = 1, then the shadow register is disabled and any write or read will go directly to the active register, that is the register actively controlling the hardware.</li> <li>In either mode, the active and shadow registers share the same memory map address.</li> </ul> |

### 3.4.2.5 Compare A High-Resolution Mirror (CMPAHRM) Register

**Figure 3-85. Compare A High-Resolution Mirror (CMPAHRM) Register**



LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 3-34. Compare A High-Resolution Mirror (CMPAHRM) Register Field Descriptions**

| Bit  | Field    | Value  | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|------|----------|--------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-8 | CMPAHR   | 00-FFh | Compare A High Resolution Bits<br>Writes to both the CMPAHR and CMPAHRM locations access the high-resolution (least significant 8-bit) portion of the Counter Compare A value. The only difference is that unlike CMPAHR, reads from the mirror register, CMPAHRM, are indeterminate (reserved for TI Test).<br>By default writes to this register are shadowed. Shadowing is enabled and disabled by the CMPCTL[SHDWAMODE] bit as described for the CMPAM register. |
| 7-0  | Reserved |        | Reserved for TI Test                                                                                                                                                                                                                                                                                                                                                                                                                                                 |

### 3.4.2.6 Counter-Compare A Mirror (CMPAM) Register

**Figure 3-86. Counter-Compare A Mirror (CMPAM) Register**

15

0

|      |
|------|
| CMPA |
|------|

R/W-0

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 3-35. Counter-Compare A Mirror (CMPAM) Register Field Descriptions**

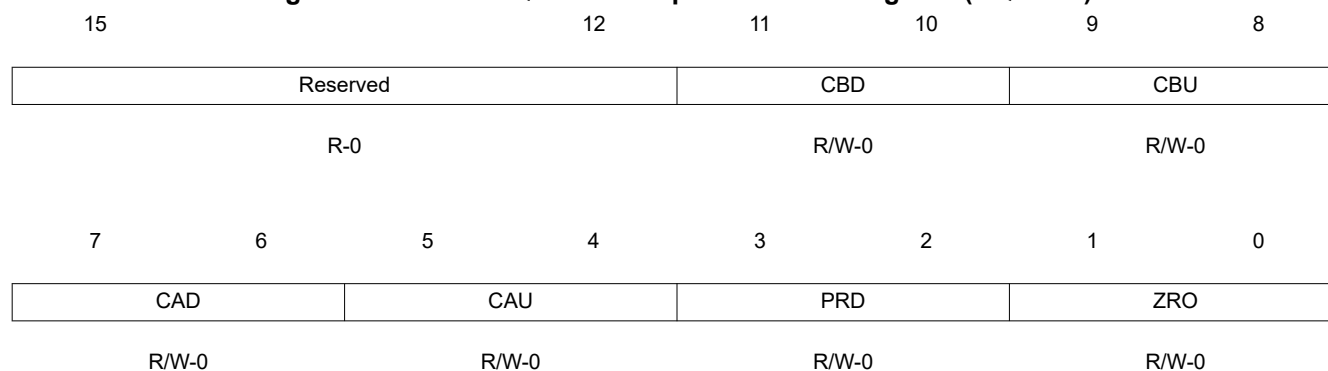
| Bit  | Field | Value      | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|------|-------|------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-0 | CMPA  | 0000-FFFFh | <p>CMPA and CMPAM can both be used to access the counter-compare A value. The only difference is that the mirror register always reads back the active value.</p> <p>By default writes to this register are shadowed. Unlike the CMPA register, reads of CMPAM always return the active register value. Shadowing is enabled and disabled by the CMPCTL[SHDWAMODE] bit.</p> <ul style="list-style-type: none"> <li>If CMPCTL[SHDWAMODE] = 0, then the shadow is enabled and any write will automatically go to the shadow register. All reads will reflect the active register value. In this case, the CMPCTL[LOADAMODE] bit field determines which event will load the active register from the shadow register.</li> <li>Before a write, the CMPCTL[SHDWAFULL] bit can be read to determine if the shadow register is currently full.</li> <li>If CMPCTL[SHDWAMODE] = 1, then the shadow register is disabled and any write will go directly to the active register, that is the register actively controlling the hardware.</li> </ul> |

### 3.4.3 Action-Qualifier Submodule Registers

This section describes the action-qualifier submodule registers.

#### 3.4.3.1 Action-Qualifier Output A Control Register (AQCTLA)

**Figure 3-87. Action-Qualifier Output A Control Register (AQCTLA)**



LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 3-36. Action-Qualifier Output A Control Register (AQCTLA) Field Descriptions**

| Bits  | Name     | Value                                                                                                                                                                                                               | Description                                                                                                                                                             |
|-------|----------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-12 | Reserved |                                                                                                                                                                                                                     | Reserved                                                                                                                                                                |
| 11-10 | CBD      | 00 Do nothing (action disabled)<br>01 Clear: force EPWMxA output low.<br>10 Set: force EPWMxA output high.<br>11 Toggle EPWMxA output: low output signal will be forced high, and a high signal will be forced low. | Action when the time-base counter equals the active CMPB register and the counter is decrementing.                                                                      |
| 9-8   | CBU      | 00 Do nothing (action disabled)<br>01 Clear: force EPWMxA output low.<br>10 Set: force EPWMxA output high.<br>11 Toggle EPWMxA output: low output signal will be forced high, and a high signal will be forced low. | Action when the counter equals the active CMPB register and the counter is incrementing.                                                                                |
| 7-6   | CAD      | 00 Do nothing (action disabled)<br>01 Clear: force EPWMxA output low.<br>10 Set: force EPWMxA output high.<br>11 Toggle EPWMxA output: low output signal will be forced high, and a high signal will be forced low. | Action when the counter equals the active CMPA register and the counter is decrementing.                                                                                |
| 5-4   | CAU      | 00 Do nothing (action disabled)<br>01 Clear: force EPWMxA output low.<br>10 Set: force EPWMxA output high.<br>11 Toggle EPWMxA output: low output signal will be forced high, and a high signal will be forced low. | Action when the counter equals the active CMPA register and the counter is incrementing.                                                                                |
| 3-2   | PRD      | 00 Do nothing (action disabled)<br>01 Clear: force EPWMxA output low.<br>10 Set: force EPWMxA output high.<br>11 Toggle EPWMxA output: low output signal will be forced high, and a high signal will be forced low. | Action when the counter equals the period.<br>Note: By definition, in count up-down mode when the counter equals period the direction is defined as 0 or counting down. |



**Table 3-36. Action-Qualifier Output A Control Register (AQCTLA) Field Descriptions (continued)**

| Bits | Name | Value | Description                                                                                                                                                |
|------|------|-------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1-0  | ZRO  |       | Action when counter equals zero.<br><br>Note: By definition, in count up-down mode when the counter equals 0 the direction is defined as 1 or counting up. |
|      |      | 00    | Do nothing (action disabled)                                                                                                                               |
|      |      | 01    | Clear: force EPWMxA output low.                                                                                                                            |
|      |      | 10    | Set: force EPWMxA output high.                                                                                                                             |
|      |      | 11    | Toggle EPWMxA output: low output signal will be forced high, and a high signal will be forced low.                                                         |

### 3.4.3.2 Action-Qualifier Output B Control Register (AQCTLB)

**Figure 3-88. Action-Qualifier Output B Control Register (AQCTLB)**

|          |  |  |  |       |  |  |  |       |  |  |  |       |  |  |  |       |  |  |  |   |  |  |  |   |  |  |  |   |  |  |  |
|----------|--|--|--|-------|--|--|--|-------|--|--|--|-------|--|--|--|-------|--|--|--|---|--|--|--|---|--|--|--|---|--|--|--|
| 15       |  |  |  | 12    |  |  |  | 11    |  |  |  | 10    |  |  |  | 9     |  |  |  | 8 |  |  |  |   |  |  |  |   |  |  |  |
| Reserved |  |  |  |       |  |  |  | CBD   |  |  |  |       |  |  |  | CBU   |  |  |  |   |  |  |  |   |  |  |  |   |  |  |  |
| R-0      |  |  |  |       |  |  |  | R/W-0 |  |  |  |       |  |  |  | R/W-0 |  |  |  |   |  |  |  |   |  |  |  |   |  |  |  |
| 7        |  |  |  | 6     |  |  |  | 5     |  |  |  | 4     |  |  |  | 3     |  |  |  | 2 |  |  |  | 1 |  |  |  | 0 |  |  |  |
| CAD      |  |  |  | CAU   |  |  |  | PRD   |  |  |  | ZRO   |  |  |  |       |  |  |  |   |  |  |  |   |  |  |  |   |  |  |  |
| R/W-0    |  |  |  | R/W-0 |  |  |  | R/W-0 |  |  |  | R/W-0 |  |  |  |       |  |  |  |   |  |  |  |   |  |  |  |   |  |  |  |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 3-37. Action-Qualifier Output B Control Register (AQCTLB) Field Descriptions**

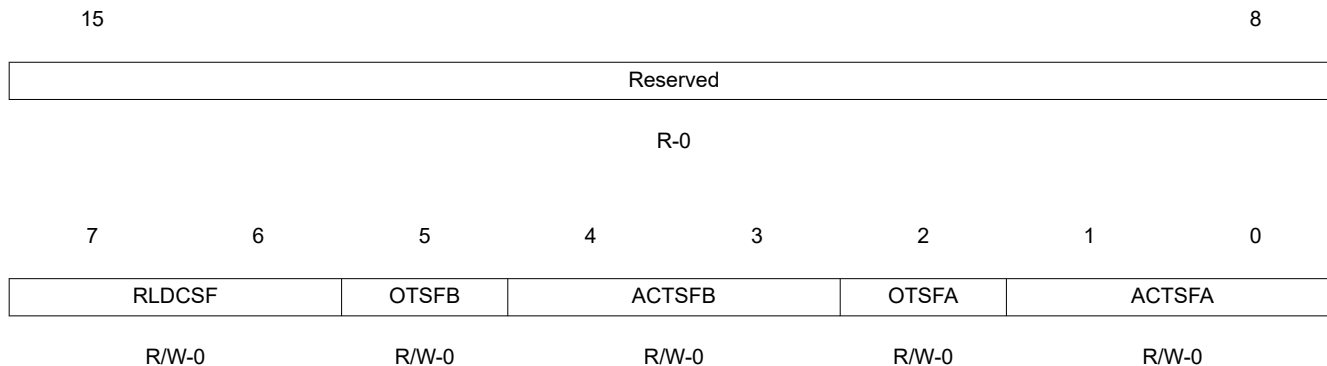
| Bits  | Name     | Value | Description                                                                                                                                                                                                                                                                                                         |
|-------|----------|-------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-12 | Reserved |       |                                                                                                                                                                                                                                                                                                                     |
| 11-10 | CBD      |       | Action when the counter equals the active CMPB register and the counter is decrementing.<br><br>00 Do nothing (action disabled)<br>01 Clear: force EPWMxB output low.<br>10 Set: force EPWMxB output high.<br>11 Toggle EPWMxB output: low output signal will be forced high, and a high signal will be forced low. |
| 9-8   | CBU      |       | Action when the counter equals the active CMPB register and the counter is incrementing.<br><br>00 Do nothing (action disabled)<br>01 Clear: force EPWMxB output low.<br>10 Set: force EPWMxB output high.<br>11 Toggle EPWMxB output: low output signal will be forced high, and a high signal will be forced low. |
| 7-6   | CAD      |       | Action when the counter equals the active CMPA register and the counter is decrementing.<br><br>00 Do nothing (action disabled)<br>01 Clear: force EPWMxB output low.<br>10 Set: force EPWMxB output high.<br>11 Toggle EPWMxB output: low output signal will be forced high, and a high signal will be forced low. |

**Table 3-37. Action-Qualifier Output B Control Register (AQCTLB) Field Descriptions (continued)**

| Bits | Name | Value | Description                                                                                                                                                             |
|------|------|-------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 5-4  | CAU  |       | Action when the counter equals the active CMPA register and the counter is incrementing.                                                                                |
|      |      | 00    | Do nothing (action disabled)                                                                                                                                            |
|      |      | 01    | Clear: force EPWMxB output low.                                                                                                                                         |
|      |      | 10    | Set: force EPWMxB output high.                                                                                                                                          |
|      |      | 11    | Toggle EPWMxB output: low output signal will be forced high, and a high signal will be forced low.                                                                      |
| 3-2  | PRD  |       | Action when the counter equals the period.<br>Note: By definition, in count up-down mode when the counter equals period the direction is defined as 0 or counting down. |
|      |      | 00    | Do nothing (action disabled)                                                                                                                                            |
|      |      | 01    | Clear: force EPWMxB output low.                                                                                                                                         |
|      |      | 10    | Set: force EPWMxB output high.                                                                                                                                          |
|      |      | 11    | Toggle EPWMxB output: low output signal will be forced high, and a high signal will be forced low.                                                                      |
| 1-0  | ZRO  |       | Action when counter equals zero.<br>Note: By definition, in count up-down mode when the counter equals 0 the direction is defined as 1 or counting up.                  |
|      |      | 00    | Do nothing (action disabled)                                                                                                                                            |
|      |      | 01    | Clear: force EPWMxB output low.                                                                                                                                         |
|      |      | 10    | Set: force EPWMxB output high.                                                                                                                                          |
|      |      | 11    | Toggle EPWMxB output: low output signal will be forced high, and a high signal will be forced low.                                                                      |

### 3.4.3.3 Action-Qualifier Software Force Register (AQSFRC)

**Figure 3-89. Action-Qualifier Software Force Register (AQSFRC)**



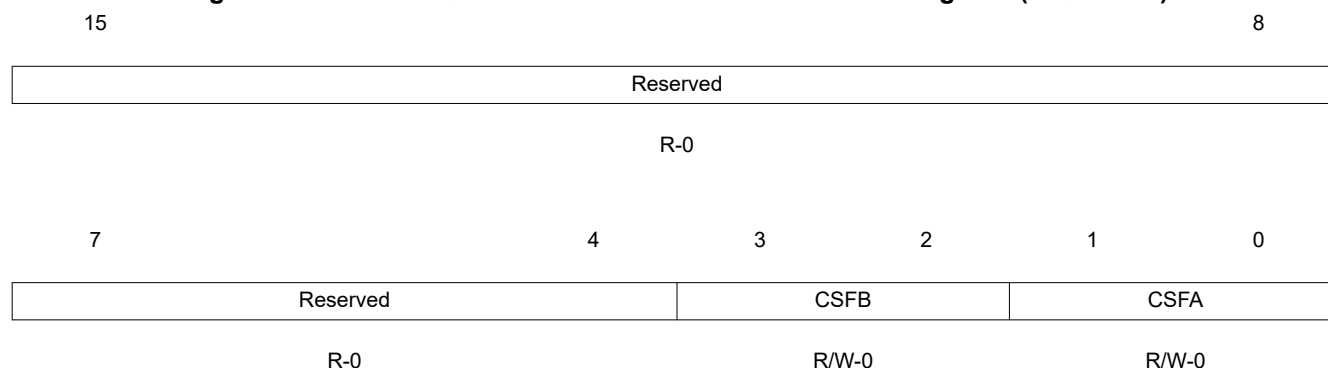
LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 3-38. Action-Qualifier Software Force Register (AQSFRC) Field Descriptions**

| Bit  | Field    | Value                                                                                                                                                                                                                                                                                                              | Description |
|------|----------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------|
| 15-8 | Reserved |                                                                                                                                                                                                                                                                                                                    |             |
| 7-6  | RLDCSF   | 00 Load on event counter equals zero<br>01 Load on event counter equals period<br>10 Load on event counter equals zero or counter equals period<br>11 Load immediately (the active register is directly accessed by the CPU and is not loaded from the shadow register).                                           |             |
| 5    | OTSFB    | 0 Writing a 0 (zero) has no effect. Always reads back a 0<br>This bit is auto cleared once a write to this register is complete (that is, a forced event is initiated).<br>This is a one-shot forced event. It can be overridden by another subsequent event on output B.<br>1 Initiates a single s/w forced event |             |
| 4-3  | ACTSFB   | 00 Does nothing (action disabled)<br>01 Clear (low)<br>10 Set (high)<br>11 Toggle (Low -> High, High -> Low)<br><b>Note:</b> This action is not qualified by counter direction (CNT_dir)                                                                                                                           |             |
| 2    | OTSFA    | 0 Writing a 0 (zero) has no effect. Always reads back a 0.<br>This bit is auto cleared once a write to this register is complete (that is, a forced event is initiated).<br>1 Initiates a single software forced event                                                                                             |             |
| 1-0  | ACTSFA   | 00 Does nothing (action disabled)<br>01 Clear (low)<br>10 Set (high)<br>11 Toggle (Low → High, High → Low)<br><b>Note:</b> This action is not qualified by counter direction (CNT_dir)                                                                                                                             |             |

### 3.4.3.4 Action-Qualifier Continuous Software Force Register (AQCSFRC)

**Figure 3-90. Action-Qualifier Continuous Software Force Register (AQCSFRC)**



LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 3-39. Action-Qualifier Continuous Software Force Register (AQCSFRC) Field Descriptions**

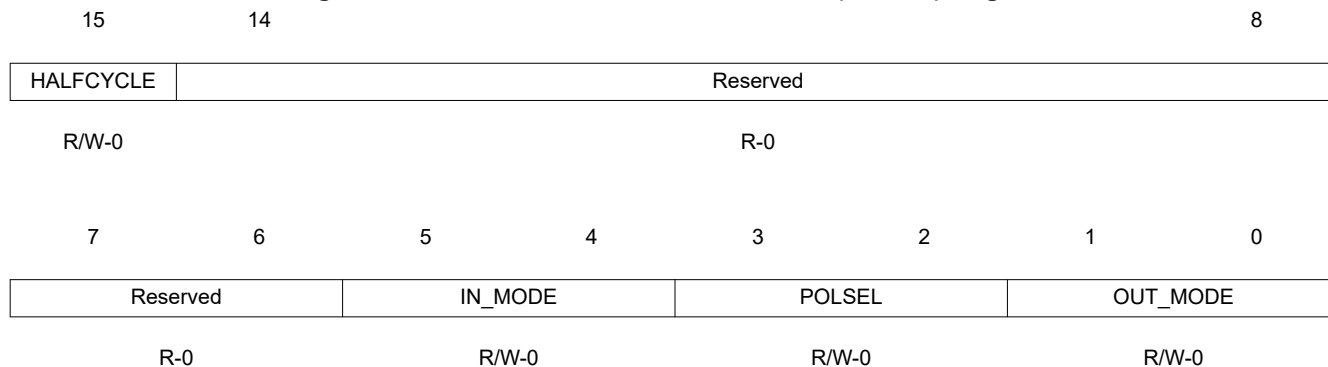
| Bits | Name     | Value                                                                                                                                                                                       | Description                                                                                                                                                                                                                                                                               |
|------|----------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-4 | Reserved |                                                                                                                                                                                             | Reserved                                                                                                                                                                                                                                                                                  |
| 3-2  | CSFB     | 00 Software forcing is disabled and has no effect<br>01 Forces a continuous low on output B<br>10 Forces a continuous high on output B<br>11 Software forcing is disabled and has no effect | Continuous Software Force on Output B<br>In immediate mode, a continuous force takes effect on the next TBCLK edge.<br>In shadow mode, a continuous force takes effect on the next TBCLK edge after a shadow load into the active register. To configure shadow mode, use AQSFRC[RLDCSF]. |
| 1-0  | CSFA     | 00 Software forcing is disabled and has no effect<br>01 Forces a continuous low on output A<br>10 Forces a continuous high on output A<br>11 Software forcing is disabled and has no effect | Continuous Software Force on Output A<br>In immediate mode, a continuous force takes effect on the next TBCLK edge.<br>In shadow mode, a continuous force takes effect on the next TBCLK edge after a shadow load into the active register.                                               |

### 3.4.4 Dead-Band Submodule Registers

This section describes the dead-band submodule registers.

#### 3.4.4.1 Dead-Band Generator Control (DBCTL) Register

**Figure 3-91. Dead-Band Generator Control (DBCTL) Register**



LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 3-40. Dead-Band Generator Control (DBCTL) Register Field Descriptions**

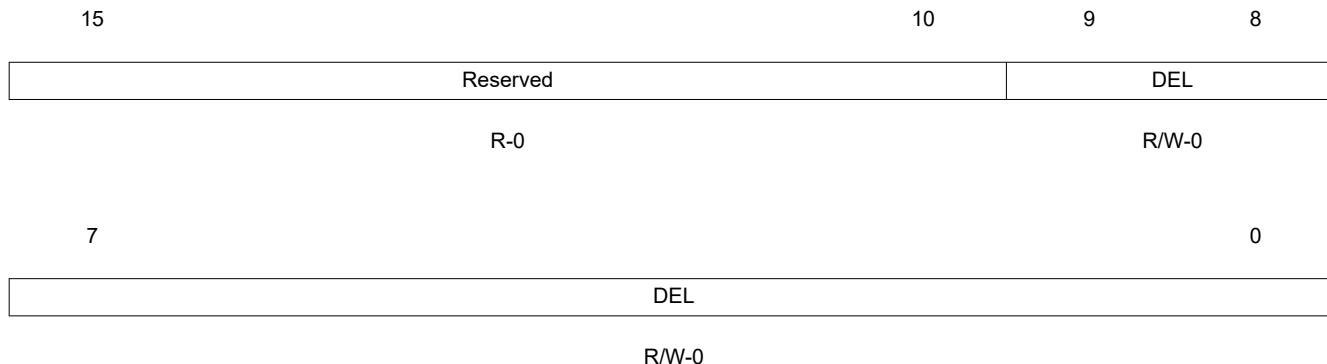
| Bits | Name      | Value                | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|------|-----------|----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15   | HALFCYCLE | 0<br>1               | Half Cycle Clocking Enable Bit:<br>Full cycle clocking enabled. The dead-band counters are clocked at the TBCLK rate.<br>Half cycle clocking enabled. The dead-band counters are clocked at TBCLK*2.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| 14-6 | Reserved  |                      | Reserved                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| 5-4  | IN_MODE   | 00<br>01<br>10<br>11 | Dead Band Input Mode Control<br>Bit 5 controls the S5 switch and bit 4 controls the S4 switch shown in <a href="#">Figure 3-29</a> .<br>This allows you to select the input source to the falling-edge and rising-edge delay.<br>To produce classical dead-band waveforms the default is EPWMxA In is the source for both falling and rising-edge delays.<br>EPWMxA In (from the action-qualifier) is the source for both falling-edge and rising-edge delay.<br>EPWMxB In (from the action-qualifier) is the source for rising-edge delayed signal.<br>EPWMxA In (from the action-qualifier) is the source for falling-edge delayed signal.<br>EPWMxA In (from the action-qualifier) is the source for rising-edge delayed signal.<br>EPWMxB In (from the action-qualifier) is the source for falling-edge delayed signal.<br>EPWMxB In (from the action-qualifier) is the source for both rising-edge delay and falling-edge delayed signal. |
| 3-2  | POLSEL    | 00<br>01<br>10<br>11 | Polarity Select Control<br>Bit 3 controls the S3 switch and bit 2 controls the S2 switch shown in <a href="#">Figure 3-29</a> .<br>This allows you to selectively invert one of the delayed signals before it is sent out of the dead-band submodule.<br>The following descriptions correspond to classical upper/lower switch control as found in one leg of a digital motor control inverter.<br>These assume that DBCTL[OUT_MODE] = 1,1 and DBCTL[IN_MODE] = 0,0. Other enhanced modes are also possible, but not regarded as typical usage modes.<br>Active high (AH) mode. Neither EPWMxA nor EPWMxB is inverted (default).<br>Active low complementary (ALC) mode. EPWMxA is inverted.<br>Active high complementary (AHC). EPWMxB is inverted.<br>Active low (AL) mode. Both EPWMxA and EPWMxB are inverted.                                                                                                                             |

**Table 3-40. Dead-Band Generator Control (DBCTL) Register Field Descriptions (continued)**

| Bits | Name     | Value | Description                                                                                                                                                                                                                                                                                 |
|------|----------|-------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1-0  | OUT_MODE |       | <p>Dead-band Output Mode Control</p> <p>Bit 1 controls the S1 switch and bit 0 controls the S0 switch shown in <a href="#">Figure 3-29</a>.</p> <p>This allows you to selectively enable or bypass the dead-band generation for the falling-edge and rising-edge delay.</p>                 |
|      |          | 00    | <p>Dead-band generation is bypassed for both output signals. In this mode, both the EPWMxA and EPWMxB output signals from the action-qualifier are passed directly to the PWM-chopper submodule.</p> <p>In this mode, the POLSEL and IN_MODE bits have no effect.</p>                       |
|      |          | 01    | <p>Disable rising-edge delay. The EPWMxA signal from the action-qualifier is passed straight through to the EPWMxA input of the PWM-chopper submodule.</p> <p>The falling-edge delayed signal is seen on output EPWMxB. The input signal for the delay is determined by DBCTL[IN_MODE].</p> |
|      |          | 10    | <p>The rising-edge delayed signal is seen on output EPWMxA. The input signal for the delay is determined by DBCTL[IN_MODE].</p> <p>Disable falling-edge delay. The EPWMxB signal from the action-qualifier is passed straight through to the EPWMxB input of the PWM-chopper submodule.</p> |
|      |          | 11    | <p>Dead-band is fully enabled for both rising-edge delay on output EPWMxA and falling-edge delay on output EPWMxB. The input signal for the delay is determined by DBCTL[IN_MODE].</p>                                                                                                      |

### 3.4.4.2 Dead-Band Generator Rising Edge Delay (DBRED) Register

**Figure 3-92. Dead-Band Generator Rising Edge Delay (DBRED) Register**



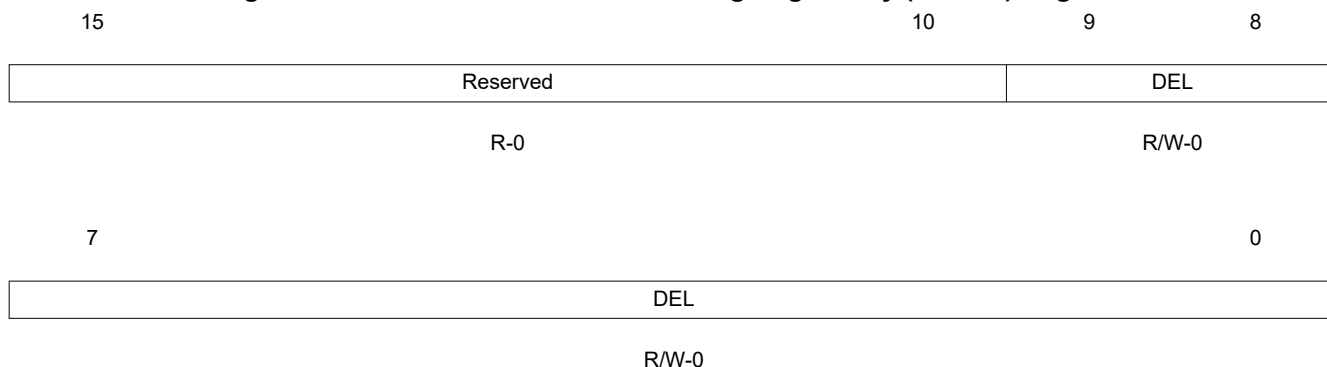
LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 3-41. Dead-Band Generator Rising Edge Delay (DBRED) Register Field Descriptions**

| Bits  | Name     | Value | Description                              |
|-------|----------|-------|------------------------------------------|
| 15-10 | Reserved |       | Reserved                                 |
| 9-0   | DEL      |       | Rising Edge Delay Count. 10-bit counter. |

### 3.4.4.3 Dead-Band Generator Falling Edge Delay (DBFED) Register

**Figure 3-93. Dead-Band Generator Falling Edge Delay (DBFED) Register**



LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 3-42. Dead-Band Generator Falling Edge Delay (DBFED) Register Field Descriptions**

| Bits  | Name     | Description                              |
|-------|----------|------------------------------------------|
| 15-10 | Reserved | Reserved                                 |
| 9-0   | DEL      | Falling Edge Delay Count. 10-bit counter |

### 3.4.5 Trip-Zone Submodule Control and Status Registers

This section describes the trip-zone submodule control and status registers.

#### 3.4.5.1 Trip-Zone Select Register (TZSEL)

**Figure 3-94. Trip-Zone Select Register (TZSEL)**

|         |         |       |       |       |       |       |       |
|---------|---------|-------|-------|-------|-------|-------|-------|
| 15      | 14      | 13    | 12    | 11    | 10    | 9     | 8     |
| DCBEVT1 | DCAEVT1 | OSHT6 | OSHT5 | OSHT4 | OSHT3 | OSHT2 | OSHT1 |
| R-0     | R-0     | R/W-0 | R/W-0 | R/W-0 | R/W-0 | R/W-0 | R/W-0 |
| 7       | 6       | 5     | 4     | 3     | 2     | 1     | 0     |
| DCBEVT2 | DCAEVT2 | CBC6  | CBC5  | CBC4  | CBC3  | CBC2  | CBC1  |
| R-0     |         | R/W-0 | R/W-0 | R/W-0 | R/W-0 | R/W-0 | R/W-0 |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 3-43. Trip-Zone Submodule Select Register (TZSEL) Field Descriptions**

| Bits                                                                                                                                                                                                                                                                                                                                                                                            | Name    | Value  | Description                                                                                                                                                                                    |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|--------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>One-Shot (OSHT) Trip-zone enable/disable. When any of the enabled pins go low, a one-shot trip event occurs for this ePWM module. When the event occurs, the action defined in the TZCTL register (Section 3.4.5.3) is taken on the EPWMxA and EPWMxB outputs. The one-shot trip condition remains latched until the user clears the condition via the TZCLR register (Section 3.4.5.6).</b> |         |        |                                                                                                                                                                                                |
| 15                                                                                                                                                                                                                                                                                                                                                                                              | DCBEVT1 | 0<br>1 | Digital Compare Output B Event 1 Select<br>Disable DCBEVT1 as one-shot-trip source for this ePWM module.<br>Enable DCBEVT1 as one-shot-trip source for this ePWM module.                       |
| 14                                                                                                                                                                                                                                                                                                                                                                                              | DCAEVT1 | 0<br>1 | Digital Compare Output A Event 1 Select<br>Disable DCAEVT1 as one-shot-trip source for this ePWM module.<br>Enable DCAEVT1 as one-shot-trip source for this ePWM module.                       |
| 13                                                                                                                                                                                                                                                                                                                                                                                              | OSHT6   | 0<br>1 | Trip-zone 6 ( $\overline{TZ6}$ ) Select<br>Disable $\overline{TZ6}$ as a one-shot trip source for this ePWM module.<br>Enable $\overline{TZ6}$ as a one-shot trip source for this ePWM module. |
| 12                                                                                                                                                                                                                                                                                                                                                                                              | OSHT5   | 0<br>1 | Trip-zone 5 ( $\overline{TZ5}$ ) Select<br>Disable $\overline{TZ5}$ as a one-shot trip source for this ePWM module<br>Enable $\overline{TZ5}$ as a one-shot trip source for this ePWM module   |
| 11                                                                                                                                                                                                                                                                                                                                                                                              | OSHT4   | 0<br>1 | Trip-zone 4 ( $\overline{TZ4}$ ) Select<br>Disable $\overline{TZ4}$ as a one-shot trip source for this ePWM module<br>Enable $\overline{TZ4}$ as a one-shot trip source for this ePWM module   |
| 10                                                                                                                                                                                                                                                                                                                                                                                              | OSHT3   | 0<br>1 | Trip-zone 3 ( $\overline{TZ3}$ ) Select<br>Disable $\overline{TZ3}$ as a one-shot trip source for this ePWM module<br>Enable $\overline{TZ3}$ as a one-shot trip source for this ePWM module   |
| 9                                                                                                                                                                                                                                                                                                                                                                                               | OSHT2   | 0<br>1 | Trip-zone 2 ( $\overline{TZ2}$ ) Select<br>Disable $\overline{TZ2}$ as a one-shot trip source for this ePWM module<br>Enable $\overline{TZ2}$ as a one-shot trip source for this ePWM module   |
| 8                                                                                                                                                                                                                                                                                                                                                                                               | OSHT1   | 0<br>1 | Trip-zone 1 ( $\overline{TZ1}$ ) Select<br>Disable $\overline{TZ1}$ as a one-shot trip source for this ePWM module<br>Enable $\overline{TZ1}$ as a one-shot trip source for this ePWM module   |



**Table 3-43. Trip-Zone Submodule Select Register (TZSEL) Field Descriptions (continued)**

| Bits                                                                                                                                                                                                                                                                                                                                                                               | Name    | Value  | Description                                                                                                                                                                        |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|--------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Cycle-by-Cycle (CBC) Trip-zone enable/disable. When any of the enabled pins go low, a cycle-by-cycle trip event occurs for this ePWM module. When the event occurs, the action defined in the TZCTL register (Section 3.4.5.3) is taken on the EPWMxA and EPWMxB outputs. A cycle-by-cycle trip condition is automatically cleared when the time-base counter reaches zero.</b> |         |        |                                                                                                                                                                                    |
| 7                                                                                                                                                                                                                                                                                                                                                                                  | DCBEVT2 | 0<br>1 | Digital Compare Output B Event 2 Select<br>Disable DCBEVT2 as a CBC trip source for this ePWM module<br>Enable DCBEVT2 as a CBC trip source for this ePWM module                   |
| 6                                                                                                                                                                                                                                                                                                                                                                                  | DCAEVT2 | 0<br>1 | Digital Compare Output A Event 2 Select<br>Disable DCAEVT2 as a CBC trip source for this ePWM module<br>Enable DCAEVT2 as a CBC trip source for this ePWM module                   |
| 5                                                                                                                                                                                                                                                                                                                                                                                  | CBC6    | 0<br>1 | Trip-zone 6 ( $\overline{TZ6}$ ) Select<br>Disable $\overline{TZ6}$ as a CBC trip source for this ePWM module<br>Enable $\overline{TZ6}$ as a CBC trip source for this ePWM module |
| 4                                                                                                                                                                                                                                                                                                                                                                                  | CBC5    | 0<br>1 | Trip-zone 5 ( $\overline{TZ5}$ ) Select<br>Disable $\overline{TZ5}$ as a CBC trip source for this ePWM module<br>Enable $\overline{TZ5}$ as a CBC trip source for this ePWM module |
| 3                                                                                                                                                                                                                                                                                                                                                                                  | CBC4    | 0<br>1 | Trip-zone 4 ( $\overline{TZ4}$ ) Select<br>Disable $\overline{TZ4}$ as a CBC trip source for this ePWM module<br>Enable $\overline{TZ4}$ as a CBC trip source for this ePWM module |
| 2                                                                                                                                                                                                                                                                                                                                                                                  | CBC3    | 0<br>1 | Trip-zone 3 ( $\overline{TZ3}$ ) Select<br>Disable $\overline{TZ3}$ as a CBC trip source for this ePWM module<br>Enable $\overline{TZ3}$ as a CBC trip source for this ePWM module |
| 1                                                                                                                                                                                                                                                                                                                                                                                  | CBC2    | 0<br>1 | Trip-zone 2 ( $\overline{TZ2}$ ) Select<br>Disable $\overline{TZ2}$ as a CBC trip source for this ePWM module<br>Enable $\overline{TZ2}$ as a CBC trip source for this ePWM module |
| 0                                                                                                                                                                                                                                                                                                                                                                                  | CBC1    | 0<br>1 | Trip-zone 1 ( $\overline{TZ1}$ ) Select<br>Disable $\overline{TZ1}$ as a CBC trip source for this ePWM module<br>Enable $\overline{TZ1}$ as a CBC trip source for this ePWM module |

### 3.4.5.2 Trip Zone Digital Compare Event Select Register (TZDCSEL)

**Figure 3-95. Trip Zone Digital Compare Event Select Register (TZDCSEL)**

|          |    |         |   |         |   |         |   |         |   |
|----------|----|---------|---|---------|---|---------|---|---------|---|
| 15       | 12 | 11      | 9 | 8       | 6 | 5       | 3 | 2       | 0 |
| Reserved |    | DCBEVT2 |   | DCBEVT1 |   | DCAEVT2 |   | DCAEVT1 |   |
| R-0      |    | R/W-0   |   | R/W-0   |   | R/W-0   |   | R/W-0   |   |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 3-44. Trip Zone Digital Compare Event Select Register (TZDCSEL) Field Descriptions**

| Bit   | Field    | Value | Description |
|-------|----------|-------|-------------|
| 15-12 | Reserved |       | Reserved    |

**Table 3-44. Trip Zone Digital Compare Event Select Register (TZDCSEL) Field Descriptions (continued)**

| Bit  | Field   | Value | Description                                |
|------|---------|-------|--------------------------------------------|
| 11-9 | DCBEVT2 |       | Digital Compare Output B Event 2 Selection |
|      |         | 000   | Event disabled                             |
|      |         | 001   | DCBH = low, DCBL = don't care              |
|      |         | 010   | DCBH = high, DCBL = don't care             |
|      |         | 011   | DCBL = low, DCBH = don't care              |
|      |         | 100   | DCBL = high, DCBH = don't care             |
|      |         | 101   | DCBL = high, DCBH = low                    |
|      |         | 110   | reserved                                   |
|      |         | 111   | reserved                                   |
| 8-6  | DCBEVT1 |       | Digital Compare Output B Event 1 Selection |
|      |         | 000   | Event disabled                             |
|      |         | 001   | DCBH = low, DCBL = don't care              |
|      |         | 010   | DCBH = high, DCBL = don't care             |
|      |         | 011   | DCBL = low, DCBH = don't care              |
|      |         | 100   | DCBL = high, DCBH = don't care             |
|      |         | 101   | DCBL = high, DCBH = low                    |
|      |         | 110   | reserved                                   |
|      |         | 111   | reserved                                   |
| 5-3  | DCAEVT2 |       | Digital Compare Output A Event 2 Selection |
|      |         | 000   | Event disabled                             |
|      |         | 001   | DCAH = low, DCAL = don't care              |
|      |         | 010   | DCAH = high, DCAL = don't care             |
|      |         | 011   | DCAL = low, DCAH = don't care              |
|      |         | 100   | DCAL = high, DCAH = don't care             |
|      |         | 101   | DCAL = high, DCAH = low                    |
|      |         | 110   | reserved                                   |
|      |         | 111   | reserved                                   |
| 2-0  | DCAEVT1 |       | Digital Compare Output A Event 1 Selection |
|      |         | 000   | Event disabled                             |
|      |         | 001   | DCAH = low, DCAL = don't care              |
|      |         | 010   | DCAH = high, DCAL = don't care             |
|      |         | 011   | DCAL = low, DCAH = don't care              |
|      |         | 100   | DCAL = high, DCAH = don't care             |
|      |         | 101   | DCAL = high, DCAH = low                    |
|      |         | 110   | reserved                                   |
|      |         | 111   | reserved                                   |

### 3.4.5.3 Trip-Zone Control Register (TZCTL)

**Figure 3-96. Trip-Zone Control Register (TZCTL)**

|          |   |   |         |    |         |       |   |    |         |   |  |   |
|----------|---|---|---------|----|---------|-------|---|----|---------|---|--|---|
| 15       |   |   |         | 12 |         | 11    |   | 10 |         | 9 |  | 8 |
| Reserved |   |   |         |    | DCBEVT2 |       |   |    | DCBEVT1 |   |  |   |
| R-0      |   |   |         |    | R/W-0   |       |   |    | R/W-0   |   |  |   |
| 7        | 6 | 5 | 4       | 3  | 2       | 1     | 0 |    |         |   |  |   |
| DCAEVT2  |   |   | DCAEVT1 |    |         | TZB   |   |    | TZA     |   |  |   |
| R/W-0    |   |   | R/W-0   |    |         | R/W-0 |   |    | R/W-0   |   |  |   |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

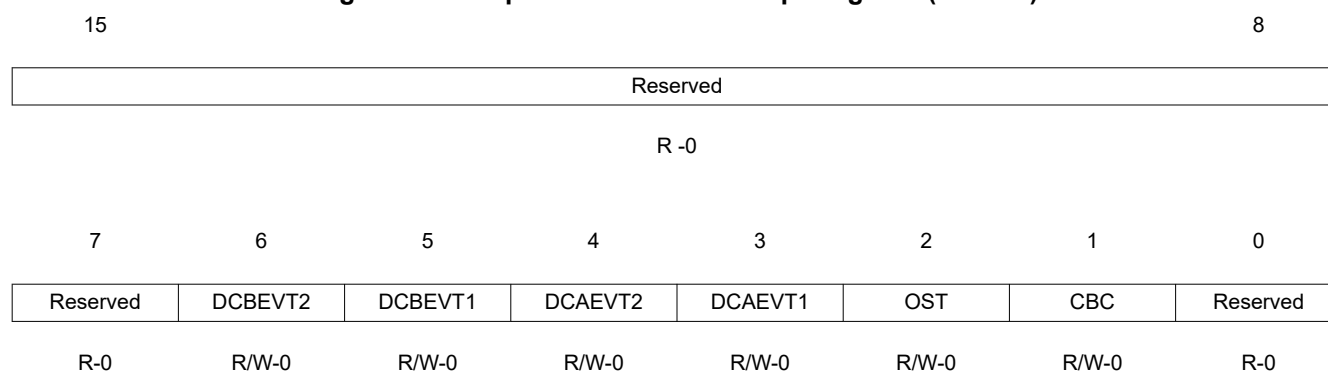
**Table 3-45. Trip-Zone Control Register (TZCTL) Field Descriptions**

| Bit   | Field    | Value                                                                                                                                                                  | Description                                                                                                                                        |
|-------|----------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-12 | Reserved |                                                                                                                                                                        | Reserved                                                                                                                                           |
| 11-10 | DCBEVT2  | 00 High-impedance (EPWMxB = High-impedance state)<br>01 Force EPWMxB to a high state.<br>10 Force EPWMxB to a low state.<br>11 Do Nothing, trip action is disabled     | Digital Compare Output B Event 2 Action On EPWMxB:                                                                                                 |
| 9-8   | DCBEVT1  | 00 High-impedance (EPWMxB = High-impedance state)<br>01 Force EPWMxB to a high state.<br>10 Force EPWMxB to a low state.<br>11 Do Nothing, trip action is disabled     | Digital Compare Output B Event 1 Action On EPWMxB:                                                                                                 |
| 7-6   | DCAEVT2  | 00 High-impedance (EPWMxA = High-impedance state)<br>01 Force EPWMxA to a high state.<br>10 Force EPWMxA to a low state.<br>11 Do Nothing, trip action is disabled     | Digital Compare Output A Event 2 Action On EPWMxA:                                                                                                 |
| 5-4   | DCAEVT1  | 00 High-impedance (EPWMxA = High-impedance state)<br>01 Force EPWMxA to a high state.<br>10 Force EPWMxA to a low state.<br>11 Do Nothing, trip action is disabled     | Digital Compare Output A Event 1 Action On EPWMxA:                                                                                                 |
| 3-2   | TZB      | 00 High-impedance (EPWMxB = High-impedance state)<br>01 Force EPWMxB to a high state<br>10 Force EPWMxB to a low state<br>11 Do nothing, no action is taken on EPWMxB. | When a trip event occurs the following action is taken on output EPWMxB. Which trip-zone pins can cause an event is defined in the TZSEL register. |

**Table 3-45. Trip-Zone Control Register (TZCTL) Field Descriptions (continued)**

| Bit | Field | Value | Description                                                                                                                                        |
|-----|-------|-------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| 1-0 | TZA   |       | When a trip event occurs the following action is taken on output EPWMxA. Which trip-zone pins can cause an event is defined in the TZSEL register. |
|     |       | 00    | High-impedance (EPWMxA = High-impedance state)                                                                                                     |
|     |       | 01    | Force EPWMxA to a high state                                                                                                                       |
|     |       | 10    | Force EPWMxA to a low state                                                                                                                        |
|     |       | 11    | Do nothing, no action is taken on EPWMxA.                                                                                                          |

### 3.4.5.4 Trip-Zone Enable Interrupt Register (TZEINT)

**Figure 3-97. Trip-Zone Enable Interrupt Register (TZEINT)**


LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 3-46. Trip-Zone Enable Interrupt Register (TZEINT) Field Descriptions**

| Bits | Name     | Value  | Description                                                                                                                                                                                    |
|------|----------|--------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-3 | Reserved |        | Reserved                                                                                                                                                                                       |
| 6    | DCBEVT2  | 0<br>1 | Digital Comparator Output B Event 2 Interrupt Enable<br>Disabled<br>Enabled                                                                                                                    |
| 5    | DCBEVT1  | 0<br>1 | Digital Comparator Output B Event 1 Interrupt Enable<br>Disabled<br>Enabled                                                                                                                    |
| 4    | DCAEVT2  | 0<br>1 | Digital Comparator Output A Event 2 Interrupt Enable<br>Disabled<br>Enabled                                                                                                                    |
| 3    | DCAEVT1  | 0<br>1 | Digital Comparator Output A Event 1 Interrupt Enable<br>Disabled<br>Enabled                                                                                                                    |
| 2    | OST      | 0<br>1 | Trip-zone One-Shot Interrupt Enable<br>Disable one-shot interrupt generation<br>Enable Interrupt generation; a one-shot trip event will cause a EPWMx_TZINT PIE interrupt.                     |
| 1    | CBC      | 0<br>1 | Trip-zone Cycle-by-Cycle Interrupt Enable<br>Disable cycle-by-cycle interrupt generation.<br>Enable interrupt generation; a cycle-by-cycle trip event will cause an EPWMx_TZINT PIE interrupt. |
| 0    | Reserved |        | Reserved                                                                                                                                                                                       |

### 3.4.5.5 Trip-Zone Flag Register (TZFLG)

**Figure 3-98. Trip-Zone Flag Register (TZFLG)**

|          |         |         |         |         |     |     |     |
|----------|---------|---------|---------|---------|-----|-----|-----|
| Reserved |         |         |         |         |     |     |     |
| R-0      |         |         |         |         |     |     |     |
| 7        | 6       | 5       | 4       | 3       | 2   | 1   | 0   |
| Reserved | DCBEVT2 | DCBEVT1 | DCAEVT2 | DCAEVT1 | OST | CBC | INT |
| R-0      | R-0     | R-0     | R-0     | R-0     | R-0 | R-0 | R-0 |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 3-47. Trip-Zone Flag Register (TZFLG) Field Descriptions**

| Bit  | Field    | Value  | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|------|----------|--------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-7 | Reserved |        | Reserved                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| 6    | DCBEVT2  | 0<br>1 | Latched Status Flag for Digital Compare Output B Event 2<br>Indicates no trip event has occurred on DCBEVT2<br>Indicates a trip event has occurred for the event defined for DCBEVT2                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| 5    | DCBEVT1  | 0<br>1 | Latched Status Flag for Digital Compare Output B Event 1<br>Indicates no trip event has occurred on DCBEVT1<br>Indicates a trip event has occurred for the event defined for DCBEVT1                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| 4    | DCAEVT2  | 0<br>1 | Latched Status Flag for Digital Compare Output A Event 2<br>Indicates no trip event has occurred on DCAEVT2<br>Indicates a trip event has occurred for the event defined for DCAEVT2                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| 3    | DCAEVT1  | 0<br>1 | Latched Status Flag for Digital Compare Output A Event 1<br>Indicates no trip event has occurred on DCAEVT1<br>Indicates a trip event has occurred for the event defined for DCAEVT1                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| 2    | OST      | 0<br>1 | Latched Status Flag for A One-Shot Trip Event<br>No one-shot trip event has occurred.<br>Indicates a trip event has occurred on a pin selected as a one-shot trip source.<br>This bit is cleared by writing the appropriate value to the TZCLR register .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| 1    | CBC      | 0<br>1 | Latched Status Flag for Cycle-By-Cycle Trip Event<br>No cycle-by-cycle trip event has occurred.<br>Indicates a trip event has occurred on a signal selected as a cycle-by-cycle trip source. The TZFLG[CBC] bit will remain set until it is manually cleared by the user. If the cycle-by-cycle trip event is still present when the CBC bit is cleared, then CBC will be immediately set again. The specified condition on the signal is automatically cleared when the ePWM time-base counter reaches zero (TBCTR = 0x0000) if the trip condition is no longer present. The condition on the signal is only cleared when the TBCTR = 0x0000 no matter where in the cycle the CBC flag is cleared.<br>This bit is cleared by writing the appropriate value to the TZCLR register . |

**Table 3-47. Trip-Zone Flag Register (TZFLG) Field Descriptions (continued)**

| Bit | Field | Value | Description                                                                                                                                                                                                                                                                                                                                                                                                                      |
|-----|-------|-------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 0   | INT   | 0     | Latched Trip Interrupt Status Flag<br>Indicates no interrupt has been generated.                                                                                                                                                                                                                                                                                                                                                 |
|     |       | 1     | Indicates an EPWMx_TZINT PIE interrupt was generated because of a trip condition.<br>No further EPWMx_TZINT PIE interrupts will be generated until this flag is cleared. If the interrupt flag is cleared when either CBC or OST is set, then another interrupt pulse will be generated. Clearing all flag bits will prevent further interrupts.<br>This bit is cleared by writing the appropriate value to the TZCLR register . |

### 3.4.5.6 Trip-Zone Clear Register (TZCLR)

**Figure 3-99. Trip-Zone Clear Register (TZCLR)**

|          |         |         |         |         |       |       |       |
|----------|---------|---------|---------|---------|-------|-------|-------|
| Reserved |         |         |         |         |       |       |       |
| R-0      |         |         |         |         |       |       |       |
| 7        | 6       | 5       | 4       | 3       | 2     | 1     | 0     |
| Reserved | DCBEVT2 | DCBEVT1 | DCAEVT2 | DCAEVT1 | OST   | CBC   | INT   |
| R-0      | R/W1C-0 | R/W1C-0 | R/W1C-0 | R/W1C-0 | R/W-0 | R/W-0 | R/W-0 |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 3-48. Trip-Zone Clear Register (TZCLR) Field Descriptions**

| Bit  | Field    | Value  | Description                                                                                                                                                                                                                                                                                                                                                                                                                   |
|------|----------|--------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-7 | Reserved |        | Reserved                                                                                                                                                                                                                                                                                                                                                                                                                      |
| 6    | DCBEVT2  | 0<br>1 | Clear Flag for Digital Compare Output B Event 2<br>Writing 0 has no effect. This bit always reads back 0.<br>Writing 1 clears the DCBEVT2 event trip condition.                                                                                                                                                                                                                                                               |
| 5    | DCBEVT1  | 0<br>1 | Clear Flag for Digital Compare Output B Event 1<br>Writing 0 has no effect. This bit always reads back 0.<br>Writing 1 clears the DCBEVT1 event trip condition.                                                                                                                                                                                                                                                               |
| 4    | DCAEVT2  | 0<br>1 | Clear Flag for Digital Compare Output A Event 2<br>Writing 0 has no effect. This bit always reads back 0.<br>Writing 1 clears the DCAEVT2 event trip condition.                                                                                                                                                                                                                                                               |
| 3    | DCAEVT1  | 0<br>1 | Clear Flag for Digital Compare Output A Event 1<br>Writing 0 has no effect. This bit always reads back 0.<br>Writing 1 clears the DCAEVT1 event trip condition.                                                                                                                                                                                                                                                               |
| 2    | OST      | 0<br>1 | Clear Flag for One-Shot Trip (OST) Latch<br>Has no effect. Always reads back a 0.<br>Clears this Trip (set) condition.                                                                                                                                                                                                                                                                                                        |
| 1    | CBC      | 0<br>1 | Clear Flag for Cycle-By-Cycle (CBC) Trip Latch<br>Has no effect. Always reads back a 0.<br>Clears this Trip (set) condition.                                                                                                                                                                                                                                                                                                  |
| 0    | INT      | 0<br>1 | Global Interrupt Clear Flag<br>Has no effect. Always reads back a 0.<br>Clears the trip-interrupt flag for this ePWM module (TZFLG[INT]).<br><b>NOTE:</b> No further EPWMx_TZINT PIE interrupts will be generated until the flag is cleared. If the TZFLG[INT] bit is cleared and any of the other flag bits are set, then another interrupt pulse will be generated. Clearing all flag bits will prevent further interrupts. |

### 3.4.5.7 Trip-Zone Force Register (TZFRC)

**Figure 3-100. Trip-Zone Force Register (TZFRC)**

|          |  |         |  |         |  |         |  |         |  |       |  |       |  |          |  |
|----------|--|---------|--|---------|--|---------|--|---------|--|-------|--|-------|--|----------|--|
| 15       |  |         |  |         |  |         |  | 8       |  |       |  |       |  |          |  |
| Reserved |  |         |  |         |  |         |  |         |  |       |  |       |  |          |  |
| R-0      |  |         |  |         |  |         |  |         |  |       |  |       |  |          |  |
| 7        |  | 6       |  | 5       |  | 4       |  | 3       |  | 2     |  | 1     |  | 0        |  |
| Reserved |  | DCBEVT2 |  | DCBEVT1 |  | DCAEVT2 |  | DCAEVT1 |  | OST   |  | CBC   |  | Reserved |  |
| R-0      |  | R/W-0   |  | R/W-0   |  | R/W-0   |  | R/W-0   |  | R/W-0 |  | R/W-0 |  | R- 0     |  |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 3-49. Trip-Zone Force Register (TZFRC) Field Descriptions**

| Bits | Name     | Value  | Description                                                                                                                                                                                     |
|------|----------|--------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-7 | Reserved |        | Reserved                                                                                                                                                                                        |
| 6    | DCBEVT2  | 0<br>1 | Force Flag for Digital Compare Output B Event 2<br>Writing 0 has no effect. This bit always reads back 0.<br>Writing 1 forces the DCBEVT2 event trip condition and sets the TZFLG[DCBEVT2] bit. |
| 5    | DCBEVT1  | 0<br>1 | Force Flag for Digital Compare Output B Event 1<br>Writing 0 has no effect. This bit always reads back 0.<br>Writing 1 forces the DCBEVT1 event trip condition and sets the TZFLG[DCBEVT1] bit. |
| 4    | DCAEVT2  | 0<br>1 | Force Flag for Digital Compare Output A Event 2<br>Writing 0 has no effect. This bit always reads back 0.<br>Writing 1 forces the DCAEVT2 event trip condition and sets the TZFLG[DCAEVT2] bit. |
| 3    | DCAEVT1  | 0<br>1 | Force Flag for Digital Compare Output A Event 1<br>Writing 0 has no effect. This bit always reads back 0.<br>Writing 1 forces the DCAEVT1 event trip condition and sets the TZFLG[DCAEVT1] bit. |
| 2    | OST      | 0<br>1 | Force a One-Shot Trip Event via Software<br>Writing of 0 is ignored. Always reads back a 0.<br>Forces a one-shot trip event and sets the TZFLG[OST] bit.                                        |
| 1    | CBC      | 0<br>1 | Force a Cycle-by-Cycle Trip Event via Software<br>Writing of 0 is ignored. Always reads back a 0.<br>Forces a cycle-by-cycle trip event and sets the TZFLG[CBC] bit.                            |
| 0    | Reserved |        | Reserved                                                                                                                                                                                        |



### 3.4.6 Event-Trigger Submodule Registers

This section describes the event-trigger submodule registers.

#### 3.4.6.1 Event-Trigger Selection Register (ETSEL)

**Figure 3-101. Event-Trigger Selection Register (ETSEL)**

|          |         |    |        |         |   |
|----------|---------|----|--------|---------|---|
| 15       | 14      | 12 | 11     | 10      | 8 |
| SOCBEN   | SOCBSEL |    | SOCAEN | SOCASEL |   |
| R/W-0    | R/W-0   |    | R/W-0  | R/W-0   |   |
| 7        |         | 4  | 3      | 2       | 0 |
| Reserved |         |    | INTEN  | INTSEL  |   |
| R-0      |         |    | R/W-0  | R/W-0   |   |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 3-50. Event-Trigger Selection Register (ETSEL) Field Descriptions**

| Bits  | Name     | Value                                                | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|-------|----------|------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15    | SOCBEN   | 0<br>1                                               | Enable the ADC Start of Conversion B (EPWMxSOCB) Pulse<br>Disable EPWMxSOCB.<br>Enable EPWMxSOCB pulse.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| 14-12 | SOCBSEL  | 000<br>001<br>010<br>011<br>100<br>101<br>110<br>111 | EPWMxSOCB Selection Options<br>These bits determine when a EPWMxSOCB pulse will be generated.<br>Enable DCBEVT1.soc event<br>Enable event time-base counter equal to zero. (TBCTR = 0x0000)<br>Enable event time-base counter equal to period (TBCTR = TBPRD)<br>Enable event time-base counter equal to zero or period (TBCTR = 0x0000 or TBCTR = TBPRD). This mode is useful in up-down count mode.<br>Enable event time-base counter equal to CMPA when the timer is incrementing.<br>Enable event time-base counter equal to CMPA when the timer is decrementing.<br>Enable event: time-base counter equal to CMPB when the timer is incrementing.<br>Enable event: time-base counter equal to CMPB when the timer is decrementing. |
| 11    | SOCAEN   | 0<br>1                                               | Enable the ADC Start of Conversion A (EPWMxSOCA) Pulse<br>Disable EPWMxSOCA.<br>Enable EPWMxSOCA pulse.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| 10-8  | SOCASEL  | 000<br>001<br>010<br>011<br>100<br>101<br>110<br>111 | EPWMxSOCA Selection Options<br>These bits determine when a EPWMxSOCA pulse will be generated.<br>Enable DCAEVT1.soc event<br>Enable event time-base counter equal to zero. (TBCTR = 0x0000)<br>Enable event time-base counter equal to period (TBCTR = TBPRD)<br>Enable event time-base counter equal to zero or period (TBCTR = 0x0000 or TBCTR = TBPRD). This mode is useful in up-down count mode.<br>Enable event time-base counter equal to CMPA when the timer is incrementing.<br>Enable event time-base counter equal to CMPA when the timer is decrementing.<br>Enable event: time-base counter equal to CMPB when the timer is incrementing.<br>Enable event: time-base counter equal to CMPB when the timer is decrementing. |
| 7-4   | Reserved |                                                      | Reserved                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |

**Table 3-50. Event-Trigger Selection Register (ETSEL) Field Descriptions (continued)**

| Bits | Name   | Value                                                | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|------|--------|------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 3    | INTEN  | 0<br>1                                               | Enable ePWM Interrupt (EPWMx_INT) Generation<br>Disable EPWMx_INT generation<br>Enable EPWMx_INT generation                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| 2-0  | INTSEL | 000<br>001<br>010<br>011<br>100<br>101<br>110<br>111 | ePWM Interrupt (EPWMx_INT) Selection Options<br>Reserved<br>Enable event time-base counter equal to zero. (TBCTR = 0x0000)<br>Enable event time-base counter equal to period (TBCTR = TBPRD)<br>Enable event time-base counter equal to zero or period (TBCTR = 0x0000 or TBCTR = TBPRD). This mode is useful in up-down count mode.<br>Enable event time-base counter equal to CMPA when the timer is incrementing.<br>Enable event time-base counter equal to CMPA when the timer is decrementing.<br>Enable event: time-base counter equal to CMPB when the timer is incrementing.<br>Enable event: time-base counter equal to CMPB when the timer is decrementing. |

### 3.4.6.2 Event-Trigger Prescale Register (ETPS)

**Figure 3-102. Event-Trigger Prescale Register (ETPS)**

|          |    |         |    |         |    |         |   |
|----------|----|---------|----|---------|----|---------|---|
| 15       | 14 | 13      | 12 | 11      | 10 | 9       | 8 |
| SOCBCNT  |    | SOCBPRD |    | SOCACNT |    | SOCAPRD |   |
| R-0      |    | R/W-0   |    | R-0     |    | R/W-0   |   |
| 7        | 4  |         |    | 3       | 2  | 1       | 0 |
| Reserved |    |         |    | INTCNT  |    | INTPRD  |   |
| R-0      |    |         |    | R-0     |    | R/W-0   |   |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 3-51. Event-Trigger Prescale Register (ETPS) Field Descriptions**

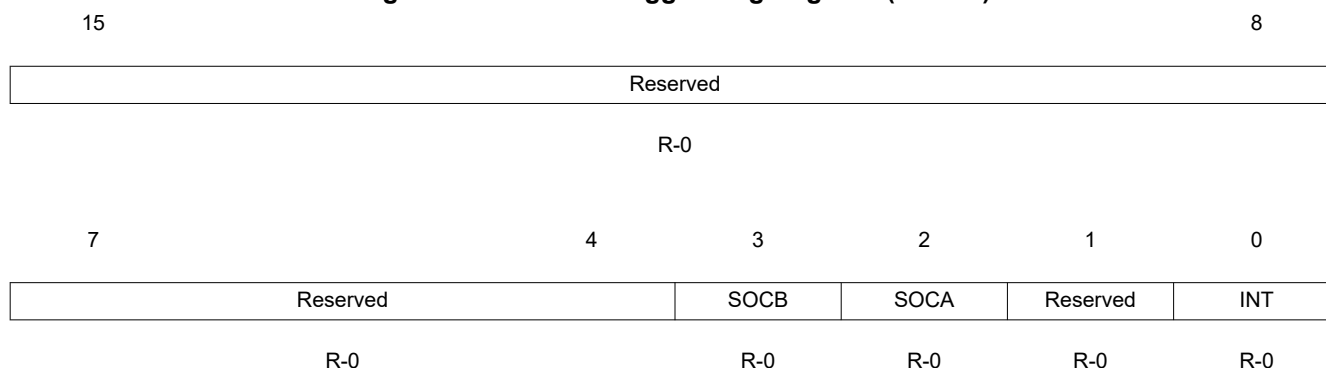
| Bits  | Name    | Value                | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|-------|---------|----------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-14 | SOCBCNT | 00<br>01<br>10<br>11 | ePWM ADC Start-of-Conversion B Event (EPWMxSOCB) Counter Register<br>These bits indicate how many selected ETSEL[SOCBSEL] events have occurred:<br>No events have occurred.<br>1 event has occurred.<br>2 events have occurred.<br>3 events have occurred.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| 13-12 | SOCBPRD | 00<br>01<br>10<br>11 | ePWM ADC Start-of-Conversion B Event (EPWMxSOCB) Period Select<br>These bits determine how many selected ETSEL[SOCBSEL] events need to occur before an EPWMxSOCB pulse is generated. To be generated, the pulse must be enabled (ETSEL[SOCBEN] = 1). The SOCB pulse will be generated even if the status flag is set from a previous start of conversion (ETFLG[SOCB] = 1). Once the SOCB pulse is generated, the ETPS[SOCBCNT] bits will automatically be cleared.<br>Disable the SOCB event counter. No EPWMxSOCB pulse will be generated<br>Generate the EPWMxSOCB pulse on the first event: ETPS[SOCBCNT] = 0,1<br>Generate the EPWMxSOCB pulse on the second event: ETPS[SOCBCNT] = 1,0<br>Generate the EPWMxSOCB pulse on the third event: ETPS[SOCBCNT] = 1,1 |

**Table 3-51. Event-Trigger Prescale Register (ETPS) Field Descriptions (continued)**

| Bits  | Name     |                                                                                                                                                                                                                                                                                                                              | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|-------|----------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 11-10 | SOCACNT  | <p>00 No events have occurred.</p> <p>01 1 event has occurred.</p> <p>10 2 events have occurred.</p> <p>11 3 events have occurred.</p>                                                                                                                                                                                       | <p>ePWM ADC Start-of-Conversion A Event (EPWMxSOCA) Counter Register</p> <p>These bits indicate how many selected ETSEL[SOCASEL] events have occurred:</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| 9-8   | SOCAPRD  | <p>00 Disable the SOCA event counter. No EPWMxSOCA pulse will be generated</p> <p>01 Generate the EPWMxSOCA pulse on the first event: ETPS[SOCACNT] = 0,1</p> <p>10 Generate the EPWMxSOCA pulse on the second event: ETPS[SOCACNT] = 1,0</p> <p>11 Generate the EPWMxSOCA pulse on the third event: ETPS[SOCACNT] = 1,1</p> | <p>ePWM ADC Start-of-Conversion A Event (EPWMxSOCA) Period Select</p> <p>These bits determine how many selected ETSEL[SOCASEL] events need to occur before an EPWMxSOCA pulse is generated. To be generated, the pulse must be enabled (ETSEL[SOCASEN] = 1). The SOCA pulse will be generated even if the status flag is set from a previous start of conversion (ETFLG[SOCASEN] = 1). Once the SOCA pulse is generated, the ETPS[SOCACNT] bits will automatically be cleared.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| 7-4   | Reserved |                                                                                                                                                                                                                                                                                                                              | Reserved                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| 3-2   | INTCNT   | <p>00 No events have occurred.</p> <p>01 1 event has occurred.</p> <p>10 2 events have occurred.</p> <p>11 3 events have occurred.</p>                                                                                                                                                                                       | <p>ePWM Interrupt Event (EPWMx_INT) Counter Register</p> <p>These bits indicate how many selected ETSEL[INTSEL] events have occurred. These bits are automatically cleared when an interrupt pulse is generated. If interrupts are disabled, ETSEL[INT] = 0 or the interrupt flag is set, ETFLG[INT] = 1, the counter will stop counting events when it reaches the period value ETPS[INTCNT] = ETPS[INTPRD].</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| 1-0   | INTPRD   | <p>00 Disable the interrupt event counter. No interrupt will be generated and ETFRC[INT] is ignored.</p> <p>01 Generate an interrupt on the first event INTCNT = 01 (first event)</p> <p>10 Generate interrupt on ETPS[INTCNT] = 1,0 (second event)</p> <p>11 Generate interrupt on ETPS[INTCNT] = 1,1 (third event)</p>     | <p>ePWM Interrupt (EPWMx_INT) Period Select</p> <p>These bits determine how many selected ETSEL[INTSEL] events need to occur before an interrupt is generated. To be generated, the interrupt must be enabled (ETSEL[INT] = 1). If the interrupt status flag is set from a previous interrupt (ETFLG[INT] = 1) then no interrupt will be generated until the flag is cleared via the ETCLR[INT] bit. This allows for one interrupt to be pending while another is still being serviced. Once the interrupt is generated, the ETPS[INTCNT] bits will automatically be cleared.</p> <p>Writing a INTPRD value that is the same as the current counter value will trigger an interrupt if it is enabled and the status flag is clear.</p> <p>Writing a INTPRD value that is less than the current counter value will result in an undefined state.</p> <p>If a counter event occurs at the same instant as a new zero or non-zero INTPRD value is written, the counter is incremented.</p> |

### 3.4.6.3 Event-Trigger Flag Register (ETFLG)

**Figure 3-103. Event-Trigger Flag Register (ETFLG)**



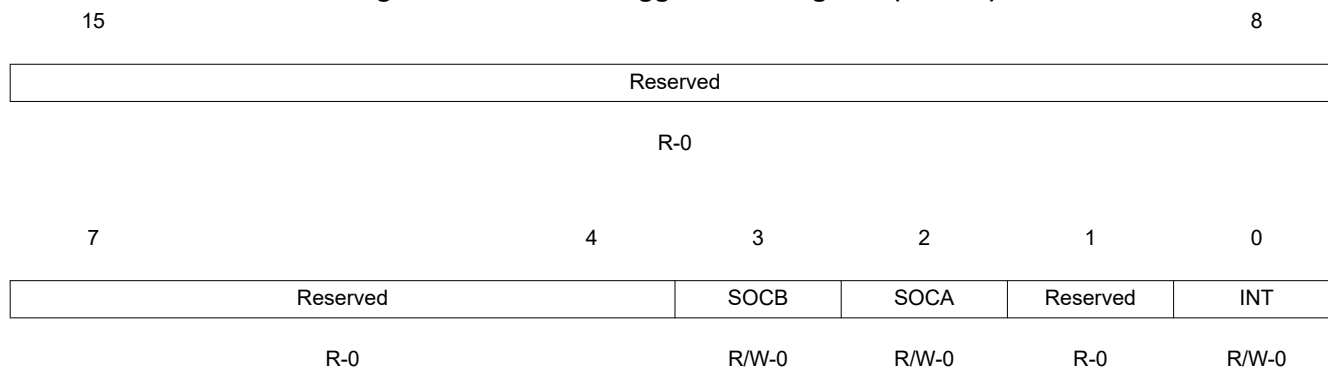
LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 3-52. Event-Trigger Flag Register (ETFLG) Field Descriptions**

| Bits | Name     | Value  | Description                                                                                                                                                                                                                                                                                                                                                                                                                      |
|------|----------|--------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-4 | Reserved |        | Reserved                                                                                                                                                                                                                                                                                                                                                                                                                         |
| 3    | SOCB     | 0<br>1 | Latched ePWM ADC Start-of-Conversion B (EPWMxSOCB) Status Flag<br>Indicates no EPWMxSOCB event occurred<br>Indicates that a start of conversion pulse was generated on EPWMxSOCB. The EPWMxSOCB output will continue to be generated even if the flag bit is set.                                                                                                                                                                |
| 2    | SOCA     | 0<br>1 | Latched ePWM ADC Start-of-Conversion A (EPWMxSOCA) Status Flag<br>Unlike the ETFLG[INT] flag, the EPWMxSOCA output will continue to pulse even if the flag bit is set.<br>Indicates no event occurred<br>Indicates that a start of conversion pulse was generated on EPWMxSOCA. The EPWMxSOCA output will continue to be generated even if the flag bit is set.                                                                  |
| 1    | Reserved |        | Reserved                                                                                                                                                                                                                                                                                                                                                                                                                         |
| 0    | INT      | 0<br>1 | Latched ePWM Interrupt (EPWMx_INT) Status Flag<br>Indicates no event occurred<br>Indicates that an ePWMx interrupt (EPWMx_INT) was generated. No further interrupts will be generated until the flag bit is cleared. Up to one interrupt can be pending while the ETFLG[INT] bit is still set. If an interrupt is pending, it will not be generated until after the ETFLG[INT] bit is cleared. See <a href="#">Figure 3-42</a> . |

### 3.4.6.4 Event-Trigger Clear Register (ETCLR)

**Figure 3-104. Event-Trigger Clear Register (ETCLR)**



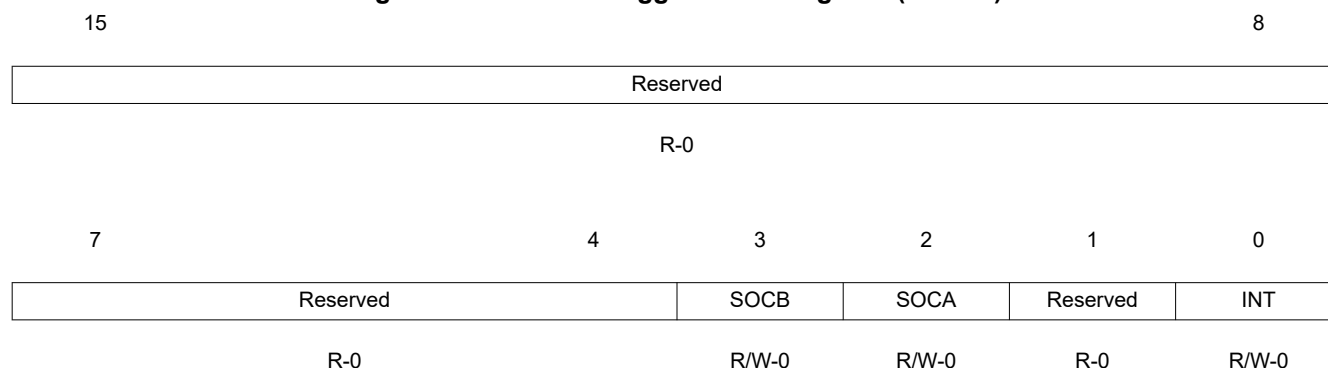
LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 3-53. Event-Trigger Clear Register (ETCLR) Field Descriptions**

| Bits | Name     | Value  | Description                                                                                                                                                                          |
|------|----------|--------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-4 | Reserved |        | Reserved                                                                                                                                                                             |
| 3    | SOCB     | 0<br>1 | ePWM ADC Start-of-Conversion B (EPWMxSOCB) Flag Clear Bit<br>Writing a 0 has no effect. Always reads back a 0<br>Clears the ETFLG[SOCB] flag bit                                     |
| 2    | SOCA     | 0<br>1 | ePWM ADC Start-of-Conversion A (EPWMxSOCA) Flag Clear Bit<br>Writing a 0 has no effect. Always reads back a 0<br>Clears the ETFLG[SOCA] flag bit                                     |
| 1    | Reserved |        | Reserved                                                                                                                                                                             |
| 0    | INT      | 0<br>1 | ePWM Interrupt (EPWMx_INT) Flag Clear Bit<br>Writing a 0 has no effect. Always reads back a 0<br>Clears the ETFLG[INT] flag bit and enable further interrupts pulses to be generated |

### 3.4.6.5 Event-Trigger Force Register (ETFRC)

**Figure 3-105. Event-Trigger Force Register (ETFRC)**



LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 3-54. Event-Trigger Force Register (ETFRC) Field Descriptions**

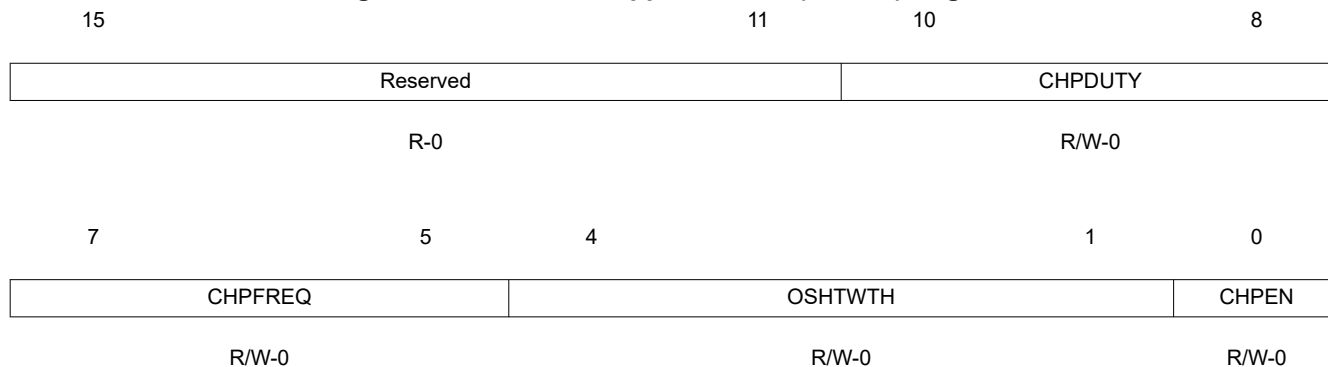
| Bits | Name     | Value  | Description                                                                                                                                                                                                                                                                                                           |
|------|----------|--------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-4 | Reserved |        | Reserved                                                                                                                                                                                                                                                                                                              |
| 3    | SOCB     | 0<br>1 | SOCB Force Bit. The SOCB pulse will only be generated if the event is enabled in the ETSEL register. The ETFLG[SOCB] flag bit will be set regardless.<br>Has no effect. Always reads back a 0.<br>Generates a pulse on EPWMxSOCB and sets the SOCBFLG bit. This bit is used for test purposes.                        |
| 2    | SOCA     | 0<br>1 | SOCA Force Bit. The SOCA pulse will only be generated if the event is enabled in the ETSEL register. The ETFLG[SOCA] flag bit will be set regardless.<br>Writing 0 to this bit will be ignored. Always reads back a 0.<br>Generates a pulse on EPWMxSOCA and set the SOCAFLG bit. This bit is used for test purposes. |
| 1    | Reserved | 0      | Reserved                                                                                                                                                                                                                                                                                                              |
| 0    | INT      | 0<br>1 | INT Force Bit. The interrupt will only be generated if the event is enabled in the ETSEL register. The INT flag bit will be set regardless.<br>Writing 0 to this bit will be ignored. Always reads back a 0.<br>Generates an interrupt on EPWMxINT and set the INT flag bit. This bit is used for test purposes.      |

### 3.4.7 PWM-Chopper Submodule Control Register

This section describes the PWM-chopper submodule control register.

#### 3.4.7.1 PWM-Chopper Control (PCCTL) Register

### Figure 3-106. PWM-Chopper Control (PCCTL) Register



LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

### Table 3-55. PWM-Chopper Control (PCCTL) Register Field Descriptions

| Bits  | Name     | Value                                                | Description                                                                                                                                                                                                                                                                                                                                                                                                              |
|-------|----------|------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-11 | Reserved |                                                      | Reserved                                                                                                                                                                                                                                                                                                                                                                                                                 |
| 10-8  | CHPDUTY  | 000<br>001<br>010<br>011<br>100<br>101<br>110<br>111 | Chopping Clock Duty Cycle<br>Duty = 1/8 (12.5%)<br>Duty = 2/8 (25.0%)<br>Duty = 3/8 (37.5%)<br>Duty = 4/8 (50.0%)<br>Duty = 5/8 (62.5%)<br>Duty = 6/8 (75.0%)<br>Duty = 7/8 (87.5%)<br>Reserved                                                                                                                                                                                                                          |
| 7-5   | CHPFREQ  | 000<br>001<br>010<br>011<br>100<br>101<br>110<br>111 | Chopping Clock Frequency<br>Divide by 1 (no prescale, = 11.25 MHz at 90 MHz SYSCLKOUT)<br>Divide by 2 (5.63 MHz at 90 MHz SYSCLKOUT)<br>Divide by 3 (3.75 MHz at 90 MHz SYSCLKOUT)<br>Divide by 4 (2.81 MHz at 90 MHz SYSCLKOUT)<br>Divide by 5 (2.25 MHz at 90 MHz SYSCLKOUT)<br>Divide by 6 (1.88 MHz at 90 MHz SYSCLKOUT)<br>Divide by 7 (1.61 MHz at 90 MHz SYSCLKOUT)<br>Divide by 8 (1.41 MHz at 90 MHz SYSCLKOUT) |

**Table 3-55. PWM-Chopper Control (PCCTL) Register Field Descriptions (continued)**

| Bits | Name    | Value | Description                                               |
|------|---------|-------|-----------------------------------------------------------|
| 4-1  | OSHTWTH |       | One-Shot Pulse Width                                      |
|      |         | 0000  | 1 x SYSCLKOUT / 8 wide ( = 88.89 nS at 90 MHz SYSCLKOUT)  |
|      |         | 0001  | 2 x SYSCLKOUT / 8 wide ( = 177.8 nS at 90 MHz SYSCLKOUT)  |
|      |         | 0010  | 3 x SYSCLKOUT / 8 wide ( = 266.7 nS at 90 MHz SYSCLKOUT)  |
|      |         | 0011  | 4 x SYSCLKOUT / 8 wide ( = 355.6 nS at 90 MHz SYSCLKOUT)  |
|      |         | 0100  | 5 x SYSCLKOUT / 8 wide ( = 444.4 nS at 90 MHz SYSCLKOUT)  |
|      |         | 0101  | 6 x SYSCLKOUT / 8 wide ( = 533.3 nS at 90 MHz SYSCLKOUT)  |
|      |         | 0110  | 7 x SYSCLKOUT / 8 wide ( = 622.2 nS at 90 MHz SYSCLKOUT)  |
|      |         | 0111  | 8 x SYSCLKOUT / 8 wide ( = 711.1 nS at 90 MHz SYSCLKOUT)  |
|      |         | 1000  | 9 x SYSCLKOUT / 8 wide ( = 800 nS at 90 MHz SYSCLKOUT)    |
|      |         | 1001  | 10 x SYSCLKOUT / 8 wide ( = 888.9 nS at 90 MHz SYSCLKOUT) |
|      |         | 1010  | 11 x SYSCLKOUT / 8 wide ( = 977.8 nS at 90 MHz SYSCLKOUT) |
|      |         | 1011  | 12 x SYSCLKOUT / 8 wide ( = 1067 nS at 90 MHz SYSCLKOUT)  |
|      |         | 1100  | 13 x SYSCLKOUT / 8 wide ( = 1156 nS at 90 MHz SYSCLKOUT)  |
|      |         | 1101  | 14 x SYSCLKOUT / 8 wide ( = 1244 nS at 90 MHz SYSCLKOUT)  |
|      |         | 1110  | 15 x SYSCLKOUT / 8 wide ( = 1333 nS at 90 MHz SYSCLKOUT)  |
|      |         | 1111  | 16 x SYSCLKOUT / 8 wide ( = 1422 nS at 90 MHz SYSCLKOUT)  |
| 0    | CHPEN   |       | PWM-chopping Enable                                       |
|      |         | 0     | Disable (bypass) PWM chopping function                    |
|      |         | 1     | Enable chopping function                                  |

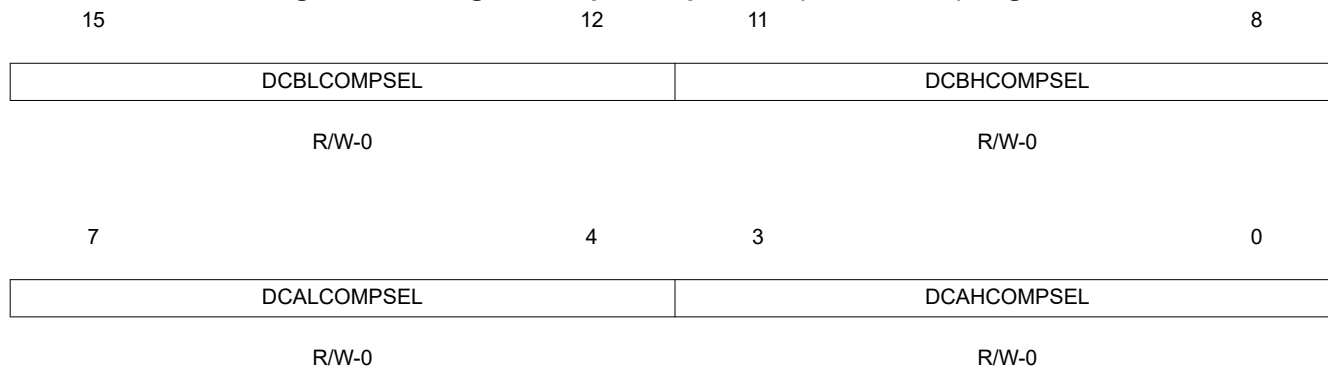


### 3.4.8 Digital Compare Submodule Registers

This section describes the digital compare submodule registers.

#### 3.4.8.1 Digital Compare Trip Select (DCTRIPSEL) Register

**Figure 3-107. Digital Compare Trip Select (DCTRIPSEL) Register**



LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 3-56. Digital Compare Trip Select (DCTRIPSEL) Register Field Descriptions**

| Bit   | Field       | Value                                                                                                                                                                                                                                                                             | Description                                                                                                                                                                                            |
|-------|-------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-12 | DCBLCOMPSEL | 0000 $\overline{TZ1}$ input<br>0001 $\overline{TZ2}$ input<br>0010 $\overline{TZ3}$ input<br>1000 COMP1OUT input<br>1001 COMP2OUT input<br>1010 COMP3OUT input<br>Values not shown are reserved. If a device does not have a particular comparator, then that option is reserved. | Digital Compare B Low Input Select<br>Defines the source for the DCBL input. The TZ signals, when used as trip signals, are treated as normal inputs and can be defined as active high or active low.  |
| 11-8  | DCBHCOMPSEL | 0000 $\overline{TZ1}$ input<br>0001 $\overline{TZ2}$ input<br>0010 $\overline{TZ3}$ input<br>1000 COMP1OUT input<br>1001 COMP2OUT input<br>1010 COMP3OUT input<br>Values not shown are reserved. If a device does not have a particular comparator, then that option is reserved. | Digital Compare B High Input Select<br>Defines the source for the DCBH input. The TZ signals, when used as trip signals, are treated as normal inputs and can be defined as active high or active low. |

**Table 3-56. Digital Compare Trip Select (DCTRIPSEL) Register Field Descriptions (continued)**

| Bit | Field       | Value | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|-----|-------------|-------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 7-4 | DCALCOMPSEL |       | <p>Digital Compare A Low Input Select</p> <p>Defines the source for the DCAL input. The TZ signals, when used as trip signals, are treated as normal inputs and can be defined as active high or active low.</p> <p>0000 <math>\overline{TZ1}</math> input</p> <p>0001 <math>\overline{TZ2}</math> input</p> <p>0010 <math>\overline{TZ3}</math> input</p> <p>1000 COMP1OUT input</p> <p>1001 COMP2OUT input</p> <p>1010 COMP3OUT input</p> <p>Values not shown are reserved. If a device does not have a particular comparator, then that option is reserved.</p>  |
| 3-0 | DCAHCOMPSEL |       | <p>Digital Compare A High Input Select</p> <p>Defines the source for the DCAH input. The TZ signals, when used as trip signals, are treated as normal inputs and can be defined as active high or active low.</p> <p>0000 <math>\overline{TZ1}</math> input</p> <p>0001 <math>\overline{TZ2}</math> input</p> <p>0010 <math>\overline{TZ3}</math> input</p> <p>1000 COMP1OUT input</p> <p>1001 COMP2OUT input</p> <p>1010 COMP3OUT input</p> <p>Values not shown are reserved. If a device does not have a particular comparator, then that option is reserved.</p> |

### 3.4.8.2 Digital Compare A Control (DCACTL) Register

**Figure 3-108. Digital Compare A Control (DCACTL) Register**

|          |  |  |   |           |          |                    |            |       |
|----------|--|--|---|-----------|----------|--------------------|------------|-------|
| 15       |  |  |   | 10        |          | 9                  |            | 8     |
| Reserved |  |  |   |           |          | EVT2FRC<br>SYNCSEL | EVT2SRCSEL |       |
| R-0      |  |  |   |           |          | R/W-0              |            | R/W-0 |
| 7        |  |  | 4 | 3         | 2        | 1                  |            | 0     |
| Reserved |  |  |   | EVT1SYNCE | EVT1SOCE | EVT1FRC<br>SYNCSEL | EVT1SRCSEL |       |
| R-0      |  |  |   | R/W-0     | R/W-0    | R/W-0              | R/W-0      |       |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 3-57. Digital Compare A Control (DCACTL) Register Field Descriptions**

| Bit   | Field              | Value  | Description                                                                                                  |
|-------|--------------------|--------|--------------------------------------------------------------------------------------------------------------|
| 15-10 | Reserved           |        | Reserved                                                                                                     |
| 9     | EVT2FRC<br>SYNCSEL | 0<br>1 | DCAEVT2 Force Synchronization Signal Select<br>Source Is Synchronous Signal<br>Source Is Asynchronous Signal |
| 8     | EVT2SRCSEL         | 0<br>1 | DCAEVT2 Source Signal Select<br>Source Is DCAEVT2 Signal<br>Source Is DCEVTFILT Signal                       |
| 7-4   | Reserved           |        | Reserved                                                                                                     |
| 3     | EVT1SYNCE          | 0<br>1 | DCAEVT1 SYNC, Enable/Disable<br>SYNC Generation Disabled<br>SYNC Generation Enabled                          |
| 2     | EVT1SOCE           | 0<br>1 | DCAEVT1 SOC, Enable/Disable<br>SOC Generation Disabled<br>SOC Generation Enabled                             |
| 1     | EVT1FRC<br>SYNCSEL | 0<br>1 | DCAEVT1 Force Synchronization Signal Select<br>Source Is Synchronous Signal<br>Source Is Asynchronous Signal |
| 0     | EVT1SRCSEL         | 0<br>1 | DCAEVT1 Source Signal Select<br>Source Is DCAEVT1 Signal<br>Source Is DCEVTFILT Signal                       |

### 3.4.8.3 Digital Compare B Control (DCBCTL) Register

**Figure 3-109. Digital Compare B Control (DCBCTL) Register**

|          |  |  |   |           |          |                    |            |   |
|----------|--|--|---|-----------|----------|--------------------|------------|---|
| 15       |  |  |   | 10        |          | 9                  |            | 8 |
| Reserved |  |  |   |           |          | EVT2FRC<br>SYNCSEL | EVT2SRCSEL |   |
| R-0      |  |  |   |           |          | R/W-0              | R/W-0      |   |
| 7        |  |  | 4 | 3         | 2        | 1                  |            | 0 |
| Reserved |  |  |   | EVT1SYNCE | EVT1SOCE | EVT1FRC<br>SYNCSEL | EVT1SRCSEL |   |
| R-0      |  |  |   | R/W-0     | R/W-0    | R/W-0              | R/W-0      |   |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 3-58. Digital Compare B Control (DCBCTL) Register Field Descriptions**

| Bit   | Field              | Value  | Description                                                                                                  |
|-------|--------------------|--------|--------------------------------------------------------------------------------------------------------------|
| 15-10 | Reserved           |        | Reserved                                                                                                     |
| 9     | EVT2FRC<br>SYNCSEL | 0<br>1 | DCBEVT2 Force Synchronization Signal Select<br>Source Is Synchronous Signal<br>Source Is Asynchronous Signal |
| 8     | EVT2SRCSEL         | 0<br>1 | DCBEVT2 Source Signal Select<br>Source Is DCBEVT2 Signal<br>Source Is DCEVTFILT Signal                       |
| 7-4   | Reserved           |        | Reserved                                                                                                     |
| 3     | EVT1SYNCE          | 0<br>1 | DCBEVT1 SYNC, Enable/Disable<br>SYNC Generation Disabled<br>SYNC Generation Enabled                          |
| 2     | EVT1SOCE           | 0<br>1 | DCBEVT1 SOC, Enable/Disable<br>SOC Generation Disabled<br>SOC Generation Enabled                             |
| 1     | EVT1FRC<br>SYNCSEL | 0<br>1 | DCBEVT1 Force Synchronization Signal Select<br>Source Is Synchronous Signal<br>Source Is Asynchronous Signal |
| 0     | EVT1SRCSEL         | 0<br>1 | DCBEVT1 Source Signal Select<br>Source Is DCBEVT1 Signal<br>Source Is DCEVTFILT Signal                       |

### 3.4.8.4 Digital Compare Filter Control (DCFCTL) Register

**Figure 3-110. Digital Compare Filter Control (DCFCTL) Register**

|          |          |          |   |          |        |        |   |   |
|----------|----------|----------|---|----------|--------|--------|---|---|
| 15       |          | 13       |   | 12       |        |        |   | 8 |
| Reserved |          |          |   | Reserved |        |        |   |   |
| R-0      |          |          |   | R-0      |        |        |   |   |
| 7        | 6        | 5        | 4 | 3        | 2      | 1      | 0 |   |
| Reserved | Reserved | PULSESEL |   | BLANKINV | BLANKE | SRCSEL |   |   |
| R-0      | R-0      | R/W-0    |   | R/W-0    | R/W-0  | R/W-0  |   |   |

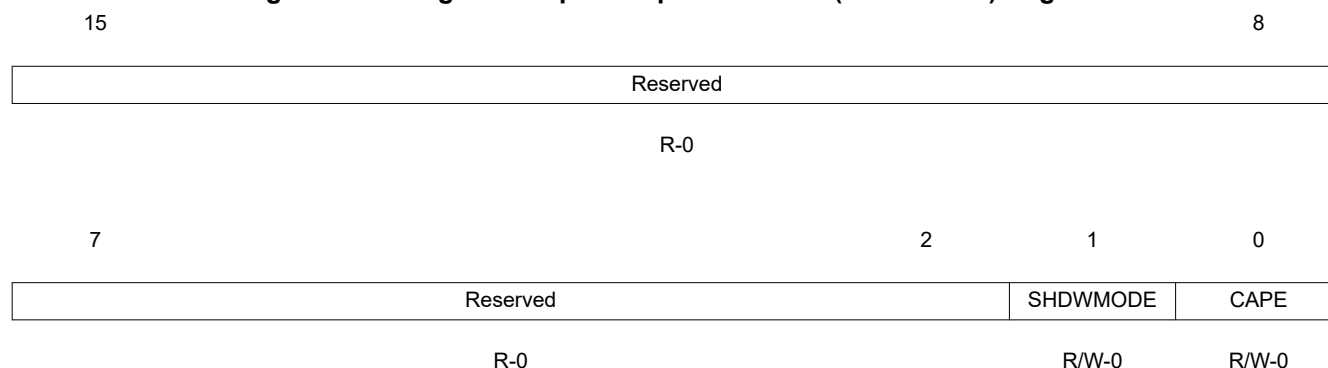
LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 3-59. Digital Compare Filter Control (DCFCTL) Register Field Descriptions**

| Bit   | Field    | Value                | Description                                                                                                                                                                    |
|-------|----------|----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-13 | Reserved |                      | Reserved                                                                                                                                                                       |
| 12-8  | Reserved |                      | Reserved for TI Test                                                                                                                                                           |
| 7     | Reserved |                      | Reserved                                                                                                                                                                       |
| 6     | Reserved |                      | Reserved for TI Test                                                                                                                                                           |
| 5-4   | PULSESEL | 00<br>01<br>10<br>11 | Pulse Select For Blanking & Capture Alignment<br>Time-base counter equal to period (TBCTR = TBPRD)<br>Time-base counter equal to zero (TBCTR = 0x0000)<br>Reserved<br>Reserved |
| 3     | BLANKINV | 0<br>1               | Blanking Window Inversion<br>Blanking window not inverted<br>Blanking window inverted                                                                                          |
| 2     | BLANKE   | 0<br>1               | Blanking Window Enable/Disable<br>Blanking window is disabled<br>Blanking window is enabled                                                                                    |
| 1-0   | SRCSEL   | 00<br>01<br>10<br>11 | Filter Block Signal Source Select<br>Source Is DCAEVT1 Signal<br>Source Is DCAEVT2 Signal<br>Source Is DCBEVT1 Signal<br>Source Is DCBEVT2 Signal                              |

### 3.4.8.5 Digital Compare Capture Control (DCCAPCTL) Register

**Figure 3-111. Digital Compare Capture Control (DCCAPCTL) Register**



LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 3-60. Digital Compare Capture Control (DCCAPCTL) Register Field Descriptions**

| Bit  | Field    | Value  | Description                                                                                                                                                                                                                                                                                                                                                                                                                            |
|------|----------|--------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-2 | Reserved |        | Reserved                                                                                                                                                                                                                                                                                                                                                                                                                               |
| 1    | SHDWMODE | 0<br>1 | TBCTR Counter Capture Shadow Select Mode<br>0 Enable shadow mode. The DCCAP active register is copied to shadow register on a TBCTR = TBPRD or TBCTR = zero event as defined by the DCFCTL[PULSESEL] bit. CPU reads of the DCCAP register will return the shadow register contents.<br>1 Active Mode. In this mode the shadow register is disabled. CPU reads from the DCCAP register will always return the active register contents. |
| 0    | CAPE     | 0<br>1 | TBCTR Counter Capture Enable/Disable<br>0 Disable the time-base counter capture.<br>1 Enable the time-base counter capture.                                                                                                                                                                                                                                                                                                            |

### 3.4.8.6 Digital Compare Filter Offset (DCOFFSET) Register

**Figure 3-112. Digital Compare Filter Offset (DCOFFSET) Register**

15

0

|          |
|----------|
| DCOFFSET |
|----------|

R-0

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 3-61. Digital Compare Filter Offset (DCOFFSET) Register Field Descriptions**

| Bit  | Field  | Value       | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
|------|--------|-------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-0 | OFFSET | 0000- FFFFh | <p>Blanking Window Offset</p> <p>These 16-bits specify the number of TBCLK cycles from the blanking window reference to the point when the blanking window is applied. The blanking window reference is either period or zero as defined by the DCFCTL[PULSESEL] bit.</p> <p>This offset register is shadowed and the active register is loaded at the reference point defined by DCFCTL[PULSESEL]. The offset counter is also initialized and begins to count down when the active register is loaded. When the counter expires, the blanking window is applied. If the blanking window is currently active, then the blanking window counter is restarted.</p> |

### 3.4.8.7 Digital Compare Filter Offset Counter (DCOFFSETCNT) Register

**Figure 3-113. Digital Compare Filter Offset Counter (DCOFFSETCNT) Register**

15

0

|           |
|-----------|
| OFFSETCNT |
|-----------|

R-0

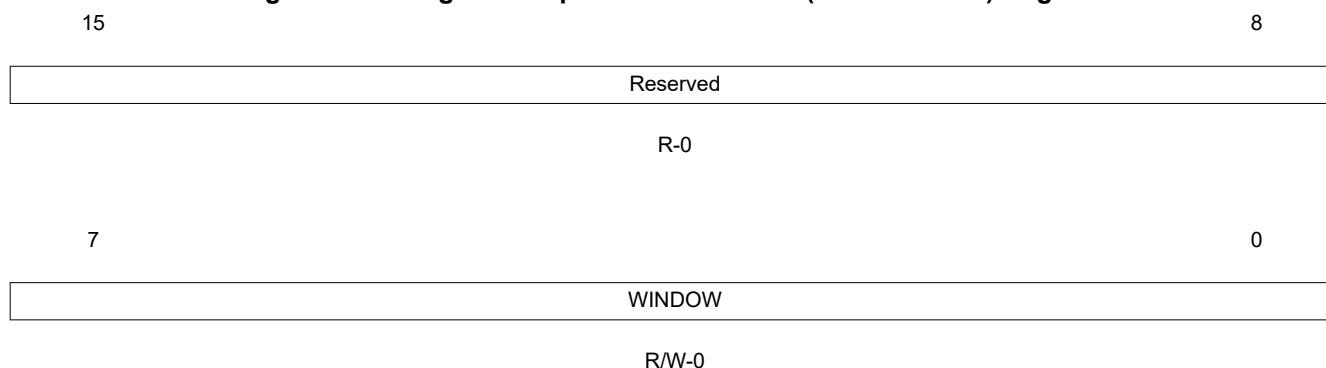
LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 3-62. Digital Compare Filter Offset Counter (DCOFFSETCNT) Register Field Descriptions**

| Bit  | Field     | Value       | Description                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|------|-----------|-------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-0 | OFFSETCNT | 0000- FFFFh | <p>Blanking Offset Counter</p> <p>These 16-bits are read only and indicate the current value of the offset counter. The counter counts down to zero and then stops until it is re-loaded on the next period or zero event as defined by the DCFCTL[PULSESEL] bit.</p> <p>The offset counter is not affected by the free/soft emulation bits. That is, it will always continue to count down if the device is halted by an emulation stop.</p> |

### 3.4.8.8 Digital Compare Filter Window (DCFWINDOW) Register

**Figure 3-114. Digital Compare Filter Window (DCFWINDOW) Register**



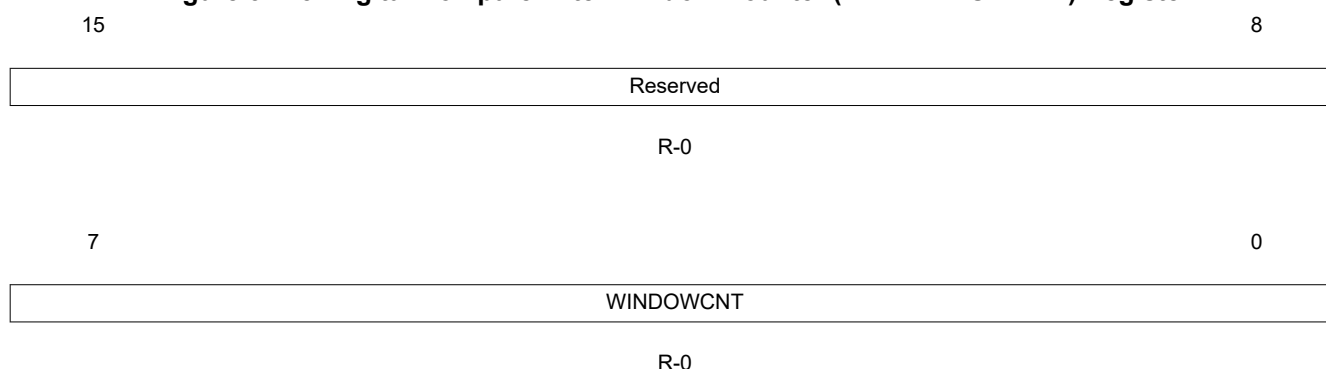
LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 3-63. Digital Compare Filter Window (DCFWINDOW) Register Field Descriptions**

| Bit  | Field    | Value         | Description                                                                                                                                                                                                                                                                                                                                                                                                                                |
|------|----------|---------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-8 | Reserved |               | Reserved                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| 7-0  | WINDOW   | 00h<br>01-FFh | Blanking Window Width<br>No blanking window is generated.<br>Specifies the width of the blanking window in TBCLK cycles. The blanking window begins when the offset counter expires. When this occurs, the window counter is loaded and begins to count down. If the blanking window is currently active and the offset counter expires, the blanking window counter is restarted.<br>The blanking window can cross a PWM period boundary. |

### 3.4.8.9 Digital Compare Filter Window Counter (DCFWINDOWCNT) Register

**Figure 3-115. Digital Compare Filter Window Counter (DCFWINDOWCNT) Register**



LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 3-64. Digital Compare Filter Window Counter (DCFWINDOWCNT) Register Field Descriptions**

| Bit  | Field     | Value | Description                                                                                                                                                                                                                  |
|------|-----------|-------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-8 | Reserved  |       | Any writes to these bit(s) must always have a value of 0.                                                                                                                                                                    |
| 7-0  | WINDOWCNT | 00-FF | Blanking Window Counter<br>These 8 bits are read only and indicate the current value of the window counter. The counter counts down to zero and then stops until it is re-loaded when the offset counter reaches zero again. |



### 3.4.8.10 Digital Compare Counter Capture (DCCAP) Register

**Figure 3-116. Digital Compare Counter Capture (DCCAP) Register**

15

0

|       |
|-------|
| DCCAP |
| R-0   |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 3-65. Digital Compare Counter Capture (DCCAP) Register Field Descriptions**

| Bit  | Field | Value      | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|------|-------|------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-0 | DCCAP | 0000-FFFFh | <p>Digital Compare Time-Base Counter Capture</p> <p>To enable time-base counter capture, set the DCCAPCLT[CAPE] bit to 1.</p> <p>If enabled, reflects the value of the time-base counter (TBCTR) on the low to high edge transition of a filtered (DCEVTFLT) event. Further capture events are ignored until the next period or zero as selected by the DCFCTL[PULSESEL] bit.</p> <p>Shadowing of DCCAP is enabled and disabled by the DCCAPCTL[SHDWMODE] bit. By default this register is shadowed.</p> <ul style="list-style-type: none"> <li>If DCCAPCTL[SHDWMODE] = 0, then the shadow is enabled. In this mode, the active register is copied to the shadow register on the TBCTR = TBPRD or TBCTR = zero as defined by the DCFCTL[PULSESEL] bit. CPU reads of this register will return the shadow register value.</li> <li>If DCCAPCTL[SHDWMODE] = 1, then the shadow register is disabled. In this mode, CPU reads will return the active register value.</li> </ul> <p>The active and shadow registers share the same memory map address.</p> |

### 3.4.9 Proper Interrupt Initialization Procedure

When the ePWM peripheral clock is enabled it may be possible that interrupt flags may be set due to spurious events due to the ePWM registers not being properly initialized. The proper procedure for initializing the ePWM peripheral is as follows:

1. Disable global interrupts (CPU INTM flag)
2. Disable ePWM interrupts
3. Set TBCLKSYNC=0
4. Initialize peripheral registers
5. Set TBCLKSYNC=1
6. Clear any spurious ePWM flags (including PIEIFR)
7. Enable ePWM interrupts
8. Enable global interrupts

This page intentionally left blank.



## High-Resolution Pulse Width Modulator (HRPWM)

This chapter is used in conjunction with [Chapter 3](#). The HRPWM module is a type 1 HRPWM. See the [C2000 Real-Time Control Peripheral Reference Guide](#) for a list of all devices with an HRPWM module of the same type, to determine the differences between types, and for a list of device-specific differences within a type.

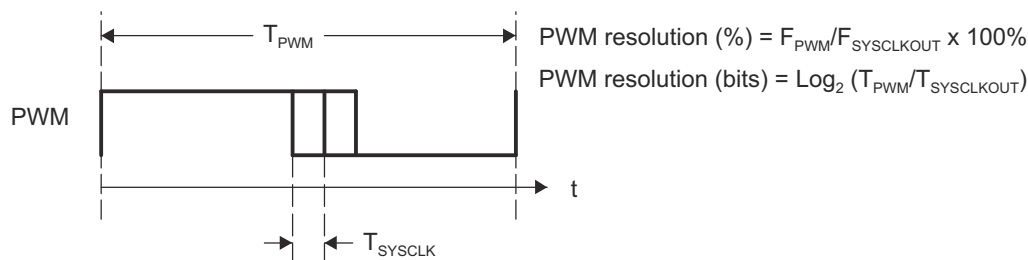
The HRPWM module extends the time resolution capabilities of the conventionally derived digital pulse width modulator (PWM). HRPWM is typically used when PWM resolution falls below ~9-10 bits. The key features of HRPWM are:

- Extended time resolution capability
- Used in both duty cycle and phase-shift control methods
- Finer time granularity control or edge positioning using extensions to the Compare A and Phase registers
- Implemented using the A signal path of PWM, that is, on the EPWMxA output
- Self-check diagnostics software mode to check if the micro edge positioner (MEP) logic is running optimally
- Enables high-resolution output on B signal path of PWM via PWM A and B channel path swapping
- Enables high-resolution output on B signal output via inversion of A signal output
- Enables high-resolution period control on the ePWMxA output on devices with a type 1 ePWM module. See the device-specific data manual to determine if your device has a type 1 ePWM module for high-resolution period support. The ePWMxB output will have a  $\pm 1$ -2 cycle jitter in this mode.

|                                                            |            |
|------------------------------------------------------------|------------|
| <b>4.1 Introduction.....</b>                               | <b>378</b> |
| <b>4.2 Operational Description of HRPWM.....</b>           | <b>379</b> |
| <b>4.3 SFO Library Software - SFO_TI_Build_V6.lib.....</b> | <b>397</b> |
| <b>4.4 HRPWM Registers.....</b>                            | <b>401</b> |

## 4.1 Introduction

The ePWM peripheral is used to perform a function that is mathematically equivalent to a digital-to-analog converter (DAC). As shown in [Figure 4-1](#), the effective resolution for conventionally generated PWM is a function of PWM frequency (or period) and system clock frequency.



**Figure 4-1. Resolution Calculations for Conventionally Generated PWM**

If the required PWM operating frequency does not offer sufficient resolution in PWM mode, you may want to consider HRPWM. As an example of improved performance offered by HRPWM, [Table 4-1](#) shows resolution in bits for various PWM frequencies. These values assume a MEP step size of 180 ps. See your device-specific data sheet for typical and maximum performance specifications for the MEP.

**Table 4-1. Resolution for PWM and HRPWM**

| PWM Frequency<br>(kHz) | Regular Resolution (PWM) |     | High Resolution (HRPWM) |       |
|------------------------|--------------------------|-----|-------------------------|-------|
|                        | 90 MHz SYCLKOUT          |     |                         |       |
|                        | Bits                     | %   | Bits                    | %     |
| 20                     | 12.1                     | 0.0 | 18.1                    | 0.000 |
| 50                     | 10.8                     | 0.1 | 16.8                    | 0.001 |
| 100                    | 9.8                      | 0.1 | 15.8                    | 0.002 |
| 150                    | 9.2                      | 0.2 | 15.2                    | 0.003 |
| 200                    | 8.8                      | 0.2 | 14.8                    | 0.004 |
| 250                    | 8.5                      | 0.3 | 14.4                    | 0.005 |
| 500                    | 7.5                      | 0.6 | 13.4                    | 0.009 |
| 1000                   | 6.5                      | 1.1 | 12.4                    | 0.018 |
| 1500                   | 5.9                      | 1.7 | 11.9                    | 0.027 |
| 2000                   | 5.5                      | 2.2 | 11.4                    | 0.036 |

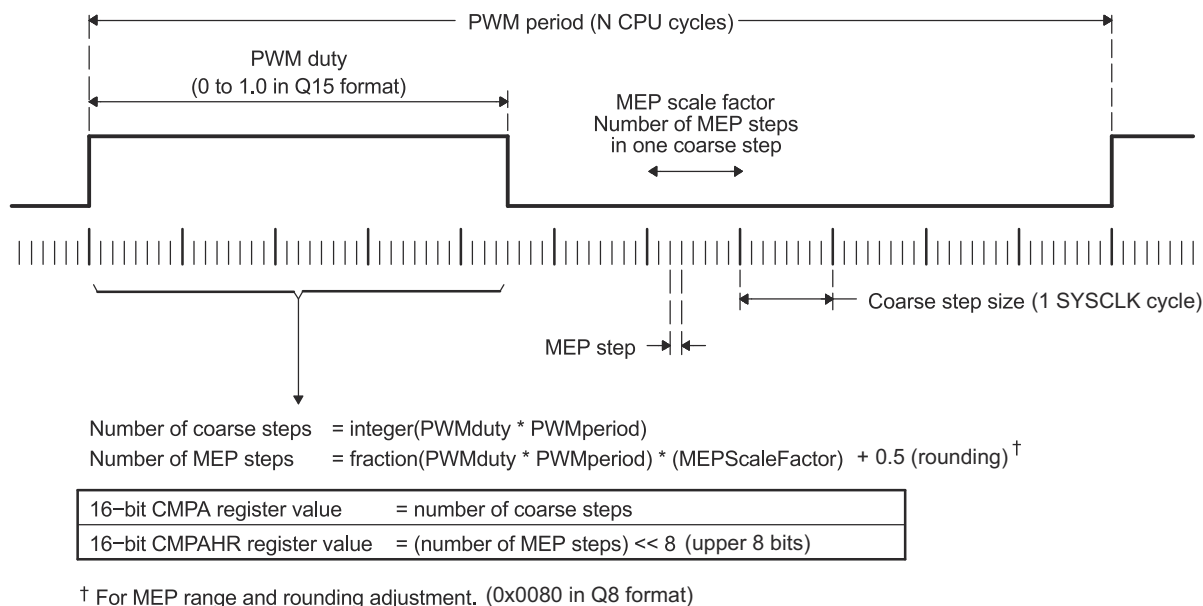
Although each application may differ, typical low-frequency PWM operation (below 250 kHz) may not require HRPWM. HRPWM capability is most useful for high-frequency PWM requirements of power conversion topologies such as:

- Single-phase buck, boost, and flyback
- Multi-phase buck, boost, and flyback
- Phase-shifted full bridge
- Direct modulation of D-Class power amplifiers

## 4.2 Operational Description of HRPWM

The HRPWM is based on micro edge positioner (MEP) technology. MEP logic is capable of positioning an edge very finely by sub-dividing one coarse system clock of a conventional PWM generator. The time step accuracy is on the order of 150 ps. See the device-specific data sheet for the typical MEP step size on a particular device. The HRPWM also has a self-check software diagnostics mode to check if the MEP logic is running optimally, under all operating conditions. Details on software diagnostics and functions are in [Section 4.2.4](#).

[Figure 4-2](#) shows the relationship between one coarse system clock and edge position in terms of MEP steps, which are controlled with an 8-bit field in the Compare A extension register (CMPAHR).



**Figure 4-2. Operating Logic Using MEP**

To generate an HRPWM waveform, configure the TBM, CCM, and AQM registers as you would to generate a conventional PWM of a given frequency and polarity. The HRPWM works together with the TBM, CCM, and AQM registers to extend edge resolution, and should be configured accordingly. Although many programming combinations are possible, only a few are needed and practical. These methods are described in [Section 4.2.5](#).

Registers discussed but not found in this chapter are in [Chapter 3](#).

The HRPWM operation is controlled and monitored using the registers listed in [Table 4-2](#).

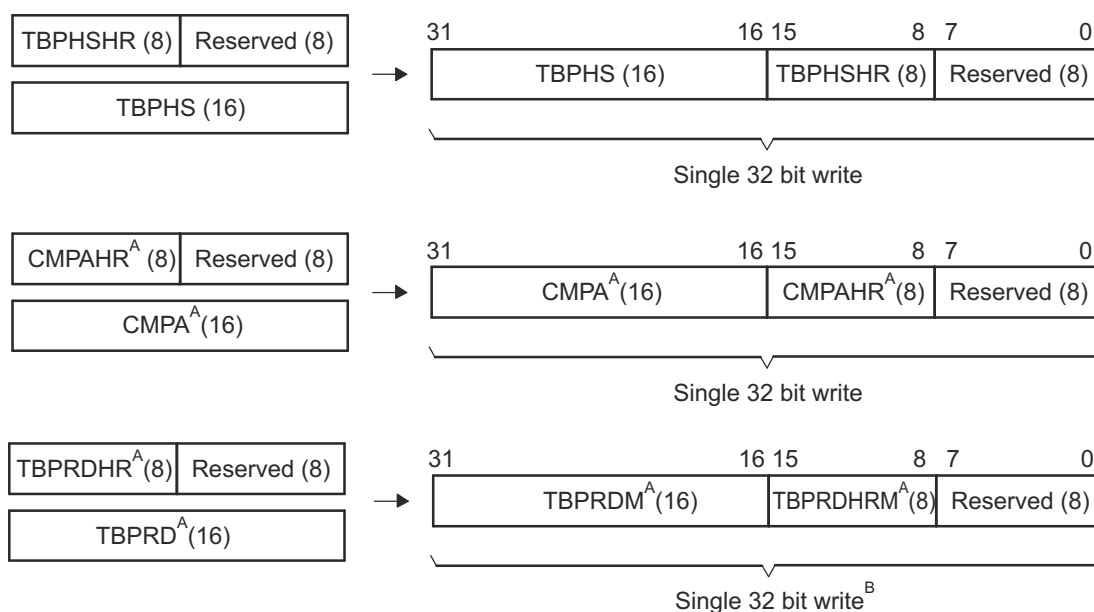
**Table 4-2. HRPWM Registers**

| Register | Address Offset | Shadowed | Description                                         |
|----------|----------------|----------|-----------------------------------------------------|
| TBPHSHR  | 0x0002         | No       | Extension Register for HRPWM Phase (8 bits)         |
| TBPRDHR  | 0x0006         | Yes      | Extension Register for HRPWM Period (8 bits)        |
| CMPAHR   | 0x0008         | Yes      | Extension Register for HRPWM Duty (8 bits)          |
| HRCNFG   | 0x0020         | No       | HRPWM Configuration Register                        |
| HRMSTEP  | 0x0026         | No       | HRPWM MEP Step Register                             |
| HRPCTL   | 0x0028         | No       | High Resolution Period Control Register             |
| TBPRDHRM | 0x002A         | Yes      | Extension Mirror Register for HRPWM Period (8 bits) |
| CMPAHRM  | 0x002C         | Yes      | Extension Mirror Register for HRPWM Duty (8 bits)   |

### 4.2.1 Controlling the HRPWM Capabilities

The MEP of the HRPWM is controlled by three extension registers, each 8-bits wide. These HRPWM registers are concatenated with the 16-bit TBPHS, TBPRD, and CMPA registers used to control PWM operation, see [Figure 4-3](#).

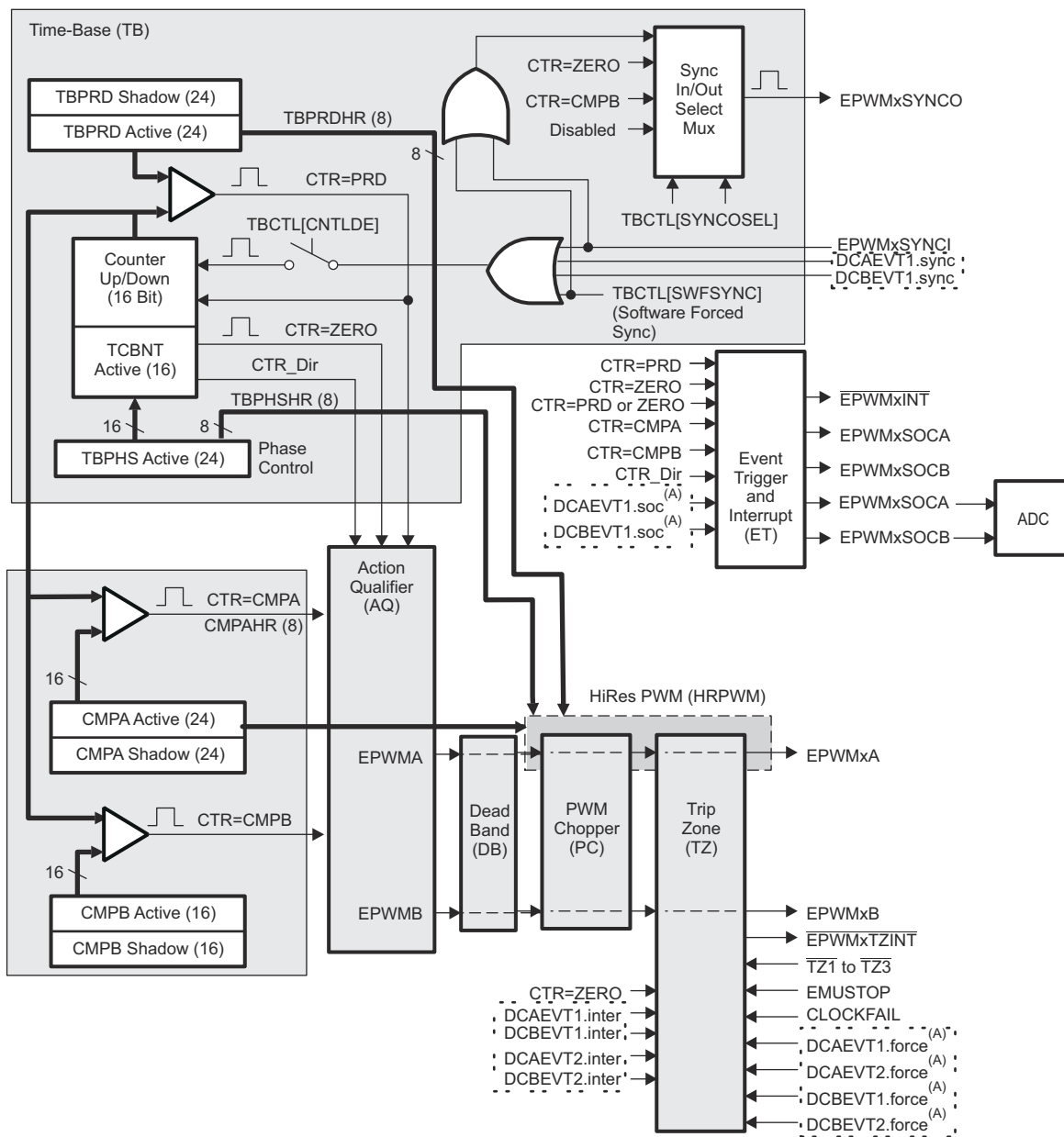
- TBPHSHR - Time-Base Phase High Resolution Register
- CMPAHR - Counter-Compare A High Resolution Register
- TBPRDHR - Time-Base Period High Resolution Register (available on some devices)



- A. These registers are mirrored and can be written to at two different memory locations (mirrored registers have an "M" suffix (that is, CMPA mirror = CMPAM). Reads of the high-resolution mirror registers will result in indeterminate values.
- B. TBPRDHR and TBPRD may be written to as a 32-bit value only at the mirrored address
- Not all devices may have TBPRD and TBPRDHR registers. See device-specific data sheet for more information

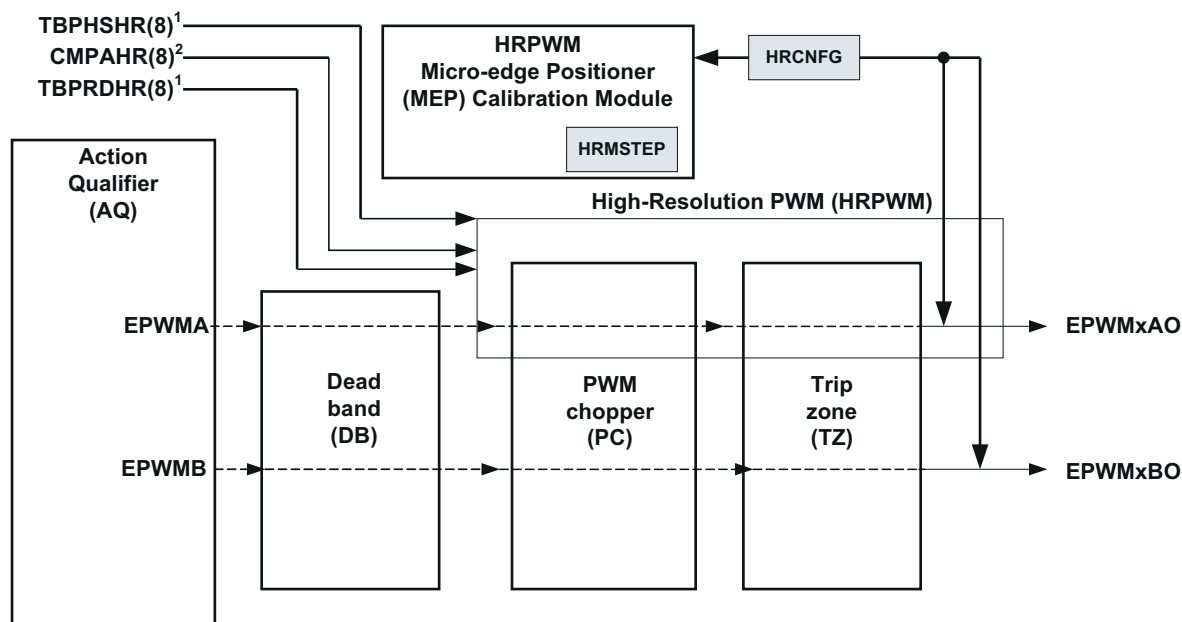
**Figure 4-3. HRPWM Extension Registers and Memory Configuration**

HRPWM capabilities are controlled using the Channel A PWM signal path. HRPWM support on the channel B signal path is available by properly configuring the HRCNFG register. [Figure 4-4](#) shows how the HRPWM interfaces with the 8-bit extension registers.



- A. These events are generated by the type 1 ePWM digital compare (DC) submodule based on the levels of the COMPxOUT and  $\overline{TZ}$  signals.

**Figure 4-4. HRPWM System Interface**



1. From ePWM time-base (TB) submodule.
2. From ePWM counter-compare (CC) submodule.

**Figure 4-5. HRPWM Block Diagram**



### 4.2.2 Configuring the HRPWM

Once the ePWM has been configured to provide conventional PWM of a given frequency and polarity, the HRPWM is configured by programming the HRCNFG register located at offset address 20h. This register provides the following configuration options:

|                                         |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|-----------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Edge Mode</b>                        | The MEP can be programmed to provide precise position control on the rising edge (RE), falling edge (FE), or both edges (BE) at the same time. FE and RE are used for power topologies requiring duty cycle control (CMPA high-resolution control), while BE is used for topologies requiring phase shifting, for example, phase shifted full bridge (TBPHS or TBPRD high-resolution control).                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| <b>Control Mode</b>                     | The MEP is programmed to be controlled either from the CMPAHR register (duty cycle control) or the TBPHSHR register (phase control). RE or FE control mode should be used with CMPAHR register. BE control mode should be used with TBPHSHR register. When the MEP is controlled from the TBPRDHR register (period control) the duty cycle and phase can also be controlled via their respective high-resolution registers.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| <b>Shadow Mode</b>                      | This mode provides the same shadowing (double buffering) option as in regular PWM mode. This option is valid only when operating from the CMPAHR and TBPRDHR registers and should be chosen to be the same as the regular load option for the CMPA register. If TBPHSHR is used, then this option has no effect.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| <b>High-Resolution B Signal Control</b> | The B signal path of an ePWM channel can generate a high-resolution output by either swapping the A and B outputs (the high-resolution signal will appear on ePWMxB instead of ePWMxA) or by outputting an inverted version of the high-resolution ePWMxA signal on the ePWMxB pin.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| <b>Auto-conversion Mode</b>             | This mode is used in conjunction with the scale factor optimization software only. For a type 1 HRPWM module, if auto-conversion is enabled, $CMPAHR = \text{fraction}(PWMduty * PWMperiod) < 8$ . The scale factor optimization software will calculate the MEP scale factor in background code and automatically update the HRMSTEP register with the calculated number of MEP steps per coarse step. The MEP Calibration Module will then use the values in the HRMSTEP and CMPAHR register to automatically calculate the appropriate number of MEP steps represented by the fractional duty cycle and move the high-resolution ePWM signal edge accordingly. If auto-conversion is disabled, the CMPAHR register behaves like a type 0 HRPWM module and $CMPAHR = (\text{fraction}(PWMduty * PWMperiod) * MEP \text{ Scale Factor} + 0.5) < 8$ . All of these calculations will need to be performed by user code in this mode, and the HRMSTEP register is ignored. Auto-conversion for high-resolution period has the same behavior as auto-conversion for high-resolution duty cycle. Auto-conversion must always be enabled for high-resolution period mode. |

### 4.2.3 Principle of Operation

The MEP logic is capable of placing an edge in one of 255 (8 bits) discrete time steps (see device-specific data sheet for typical MEP step size). The MEP works with the TBM and CCM registers to be certain that time steps are optimally applied and that edge placement accuracy is maintained over a wide range of PWM frequencies, system clock frequencies, and other operating conditions. Table 4-3 shows the typical range of operating frequencies supported by the HRPWM.

**Table 4-3. Relationship Between MEP Steps, PWM Frequency, and Resolution**

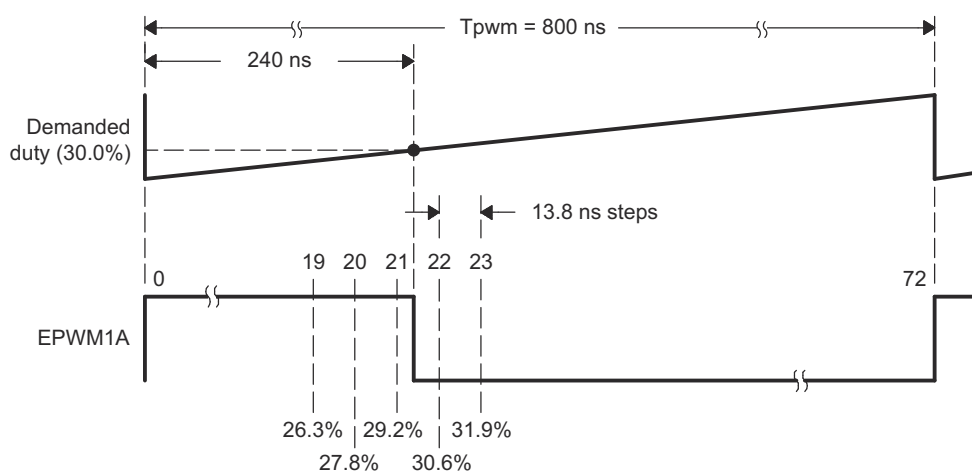
| System (MHz) | MEP Steps Per SYSCLKOUT <sup>(1) (2) (3)</sup> | PWM Minimum (Hz) <sup>(4)</sup> | PWM Maximum (MHz) | Resolution @ Maximum (Bits) <sup>(5)</sup> |
|--------------|------------------------------------------------|---------------------------------|-------------------|--------------------------------------------|
| 60.0         | 93                                             | 916                             | 3.00              | 10.9                                       |
| 70.0         | 79                                             | 1068                            | 3.50              | 10.6                                       |
| 80.0         | 69                                             | 1221                            | 4.00              | 10.4                                       |
| 90.0         | 62                                             | 1373                            | 4.50              | 10.3                                       |

- (1) System frequency = SYSCLKOUT, that is, CPU clock. TBCLK = SYSCLKOUT.
- (2) Table data based on a MEP time resolution of 180 ps (this is an example value. See the device-specific data sheet for MEP limits)
- (3) MEP steps applied =  $T_{SYSCLKOUT}/180$  ps in this example.
- (4) PWM minimum frequency is based on a maximum period value, that is, TBPRD = 65535. PWM mode is asymmetrical up-count.
- (5) Resolution in bits is given for the maximum PWM frequency stated.

#### 4.2.3.1 Edge Positioning

In a typical power control loop (for example, switch modes, digital motor control [DMC], uninterruptible power supply [UPS]), a digital controller (PID, 2pole/2zero, lag/lead, and so on) issues a duty command, usually expressed in a per unit or percentage terms. Assume that for a particular operating point, the demanded duty cycle is 0.300 or 30.0% on time and the required converter PWM frequency is 1.25 MHz. In conventional PWM generation with a system clock of 90 MHz, the duty cycle choices are in the vicinity of 30.0%. In Figure 4-6, a compare value of 22 counts (that is, duty = 30.6%) is the closest to 30.0% that you can attain. This is equivalent to an edge position of 244.4 ns instead of the desired 240.0 ns. This data is shown in Table 4-4.

By utilizing the MEP, you can achieve an edge position much closer to the desired point of 240 ns. Table 4-4 shows that in addition to the CMPA value of 21 (that is, duty = 29.2% and edge positioning at 233.3 ns), 37 steps of the MEP (CMPAHR register) will position the edge at 239.96 ns, resulting in almost zero error. In this example, it is assumed that the MEP has a step resolution of 180 ps.



**Figure 4-6. Required PWM Waveform for a Requested Duty = 30.0%**

**Table 4-4. CMPA versus Duty (left), and [CMPA:CMPAHR] versus Duty (right)**

| CMPA<br>(count) <sup>(1) (2) (3)</sup> | Duty   | High Time<br>(ns) | CMPA<br>(count) | CMPAHR<br>(count) | Duty   | High Time<br>(ns) |
|----------------------------------------|--------|-------------------|-----------------|-------------------|--------|-------------------|
| 17                                     | 23.61% | 188.9             | 21              | 31                | 29.86% | 238.88            |
| 18                                     | 25.0%  | 200.0             | 21              | 32                | 29.88% | 239.06            |
| 19                                     | 26.39% | 211.1             | 21              | 33                | 29.91% | 239.24            |
| 20                                     | 27.78% | 222.2             | 21              | 34                | 29.93% | 239.42            |
| 21                                     | 29.17% | 233.4             | 21              | 35                | 29.95% | 239.60            |
| 22                                     | 30.56% | 244.5             | 21              | 36                | 29.97% | 239.78            |
| 23                                     | 31.94% | 255.5             | 21              | 37                | 30.00% | 239.96            |
| 24                                     | 33.33% | 266.6             | 21              | 38                | 30.02% | 240.14            |
| 25                                     | 34.72% | 277.8             | 21              | 39                | 30.04% | 240.32            |
| Required                               |        |                   | 21              | 40                | 30.06% | 240.50            |
| 21.6                                   | 30.0%  | 240.0             | 21              | 41                | 30.09% | 240.68            |

(1) System clock, SYSCLKOUT and TBCLK = 90 MHz, 11.1 ns

(2) For a PWM Period register value of 72 counts, PWM Period = 72 x 11.1 ns = 800 ns, PWM frequency = 1/800 ns = 1.25 MHz

(3) Assumed MEP step size for the above example = 180 ps

See the device-specific data manual for typical and maximum MEP values.

#### 4.2.3.2 Scaling Considerations

The mechanics of how to position an edge precisely in time has been demonstrated using the resources of the standard CMPA and MEP (CMPAHR) registers. In a practical application, however, it is necessary to seamlessly provide the CPU a mapping function from a per-unit (fractional) duty cycle to a final integer (non-fractional) representation that is written to the [CMPA:CMPAHR] register combination. This section describes the mapping from a per-unit duty cycle only. The method for mapping from a per-unit period is described in [Section 4.2.3.4](#).

To do this, first examine the scaling or mapping steps involved. It is common in control software to express duty cycle in a per-unit or percentage basis. This has the advantage of performing all needed math calculations without concern for the final absolute duty cycle, expressed in clock counts or high time in ns. Furthermore, it makes the code more transportable across multiple converter types running different PWM frequencies.

To implement the mapping scheme, a two-step scaling procedure is required.

#### Assumptions for this example:

|                                                                                                                                                                                                                                                                                                                                    |                             |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------|
| System clock , SYSCLKOUT                                                                                                                                                                                                                                                                                                           | = 11.1 ns (90 MHz)          |
| PWM frequency                                                                                                                                                                                                                                                                                                                      | = 1.25 MHz (1/800 ns)       |
| Required PWM duty cycle, <b>PWMDuty</b>                                                                                                                                                                                                                                                                                            | = 0.300 (30.0%)             |
| PWM period in terms of coarse steps,<br><b>PWMperiod</b> (800 ns/ 11.1 ns)                                                                                                                                                                                                                                                         | = 72                        |
| Number of MEP steps per coarse step at<br>180 ps ( 11.1 ns/ 180 ps), <b>MEP_ScaleFactor</b>                                                                                                                                                                                                                                        | = 61                        |
| Value to keep CMPAHR within the range of 1-255 and<br>fractional rounding constant (default value). In the event that<br>$\text{frac}(\text{PWMDuty} * \text{PWMperiod}) * \text{MEP\_ScaleFactor}$ results in<br>a value with a decimal portion $\geq 0.5$ , this rounding constant<br>will round the CMPAHR value up 1 MEP step. | = 0.5 (0 080h in Q8 format) |

## Step 1: Percentage Integer Duty value conversion for CMPA register

CMPA register value =  $\text{int}(\text{PWMDuty} \times \text{PWMperiod})$ ; int means integer part  
 =  $\text{int}(0.300 \times 72)$   
 CMPA register value = 21 (15h)

## Step 2: Fractional value conversion for CMPAHR register

CMPAHR register value =  $(\text{frac}(\text{PWMDuty} \times \text{PWMperiod}) \times \text{MEP\_ScaleFactor} + 0.5) \ll 8$ ; frac means fractional part  
 =  $(\text{frac}(21.6) \times 72 + 0.5) \ll 8$ ; Shift is to move the value as CMPAHR high byte  
 =  $((0.6 \times 61 + 0.5) \ll 8)$   
 =  $((37.1 + 0.5) \ll 8)$   
 =  $37.6 \times 256$ ; Shifting left by 8 is the same as multiplying by 256.  
 = 9,625  
 CMPAHR value = 2559h; lower 8 bits will be ignored by hardware.

### Note

If the AUTOCONV bit (HRCNFG.6) is set and the MEP\_ScaleFactor is in the HRMSTEP register, then CMPAHR register value =  $\text{frac}(\text{PWMDuty} \times \text{PWMperiod} \ll 8)$ . The rest of the conversion calculations are performed automatically in hardware, and the correct MEP-scaled signal edge appears on the ePWM channel output. If AUTOCONV is not set, the above calculations must be performed by software.

### Note

The MEP scale factor (MEP\_ScaleFactor) varies with the system clock and DSP operating conditions. TI provides an MEP scale factor optimizing (SFO) software C function, which uses the built in diagnostics in each HRPWM and returns the best scale factor for a given operating point.

The scale factor varies slowly over a limited range so the optimizing C function can be run very slowly in a background loop.

The CMPA and CMPAHR registers are configured in memory so that the 32-bit data capability of the 28x CPU can write this as a single concatenated value, [CMPA:CMPAHR]. The TBPRDM and TBPRDHRM (mirror) registers are similarly configured in memory.

The mapping scheme has been implemented in both C and assembly, as shown in [Section 4.2.5](#). The actual implementation takes advantage of the 32-bit CPU architecture of the 28xx, and is somewhat different from the steps shown in [Section 4.2.3.2](#).

For time critical control loops where every cycle counts, the assembly version is recommended. This is a cycle optimized function (11 SYSCLKOUT cycles ) that takes a Q15 duty value as input and writes a single [CMPA:CMPAHR] value.

#### 4.2.3.3 Duty Cycle Range Limitation

In high resolution mode, the MEP is not active for 100% of the PWM period. It becomes operational:

- 3 SYSCLK cycles after the period starts when high-resolution period (TBPRDHR) control is not enabled.
- When high resolution period (TBPRDHR) control is enabled via the HRPCTL register:
  - In up-count mode: 3 SYSCLK cycles after the period starts until 3 SYSCLK cycles before the period ends.
  - In up-down count mode: when counting up, 3 cycles after CTR = 0 until 3 cycles before CTR = PRD, and when counting down, 3 cycles after CTR = PRD until 3 cycles before CTR = 0.

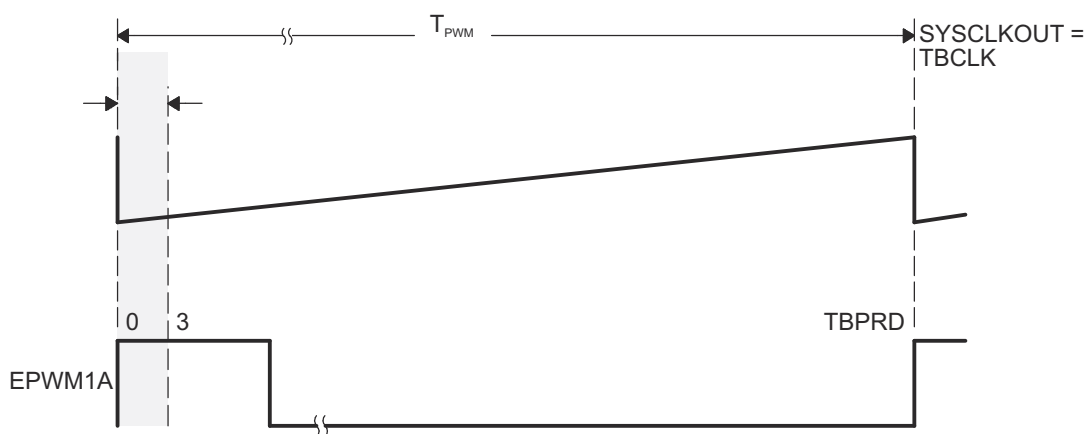
To better understand the useable duty cycle range, see [Table 4-5](#). Duty cycle range limitations are illustrated in [Figure 4-7](#) to [Figure 4-10](#). This limitation imposes a duty cycle limit on the MEP. For example, precision edge control is not available all the way down to 0% duty cycle. When high-resolution period control is disabled, although for the first three cycles, the HRPWM capabilities are not available, regular PWM duty control is still fully operational down to 0% duty. In most applications, this should not be an issue as the controller regulation point is usually not designed to be close to 0% duty cycle. When high-resolution period control is enabled (HRPCTL[HRPE]=1), the duty cycle must not fall within the restricted range. Otherwise, there may be undefined behavior on the ePWMxA output.

**Table 4-5. Duty Cycle Range Limitation for 3 SYSCLK/TBCLK Cycles**

| PWM Frequency <sup>(1)</sup><br>(kHz) | 3 Cycles<br>Minimum Duty | 3 Cycles<br>Maximum Duty <sup>(2)</sup> |
|---------------------------------------|--------------------------|-----------------------------------------|
| 200                                   | 0.67%                    | 99.33% 99.00%                           |
| 400                                   | 1.33%                    | 98.67%                                  |
| 600                                   | 2.00%                    | 98.00%                                  |
| 800                                   | 2.67%                    | 97.33%                                  |
| 1000                                  | 3.33%                    | 97.67%                                  |
| 1200                                  | 4.00%                    | 96.00%                                  |
| 1400                                  | 4.67%                    | 95.33%                                  |
| 1600                                  | 5.33%                    | 95.67%                                  |
| 1800                                  | 6.00%                    | 94.00%                                  |
| 2000                                  | 6.67%                    | 93.33%                                  |

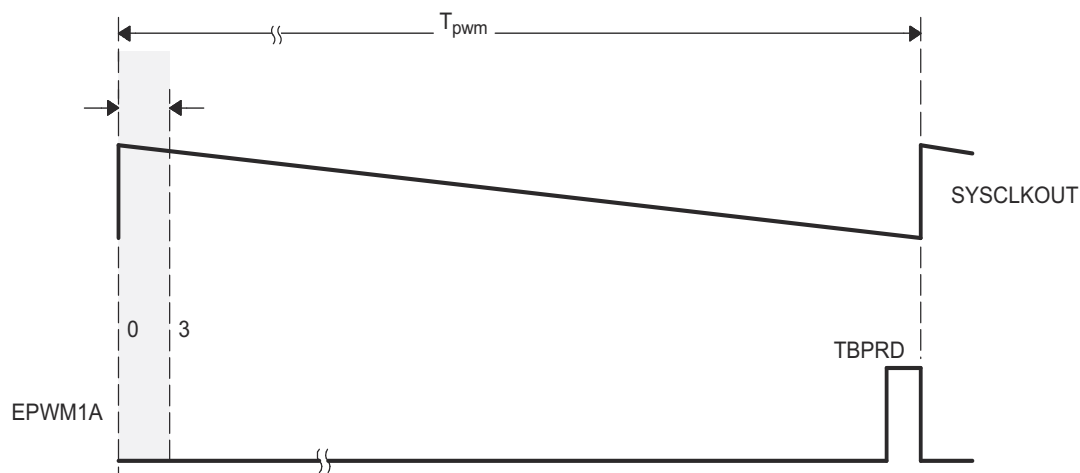
(1) System clock -  $T_{SYSCLKOUT} = 11.1$  ns System clock = TBCLK = 90 MHz

(2) This limitation applies only if high-resolution period (TBPRDHR) control is enabled.

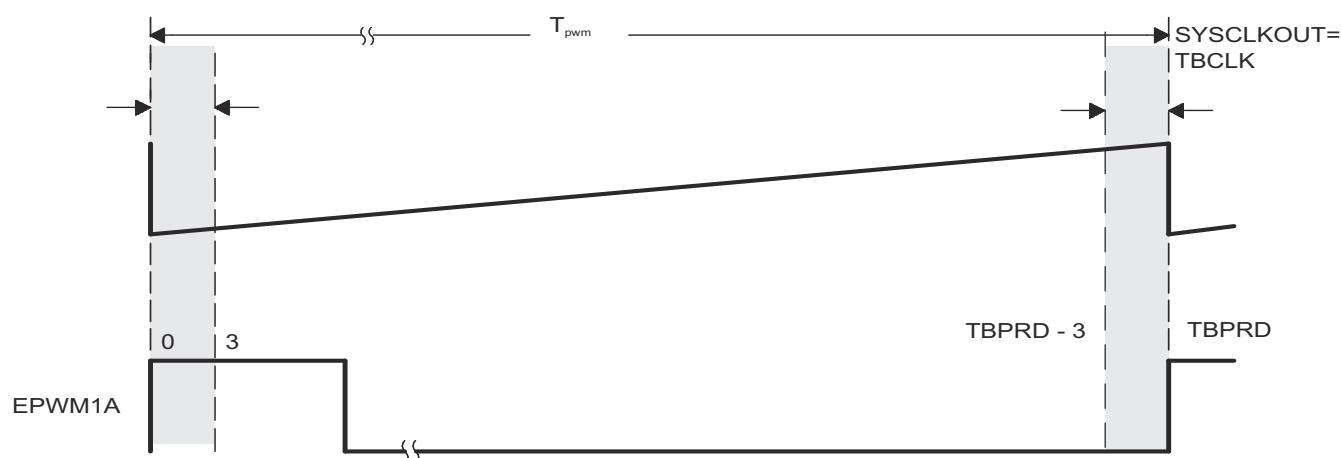


**Figure 4-7. Low % Duty Cycle Range Limitation Example (HRPCTL[HRPE] = 0)**

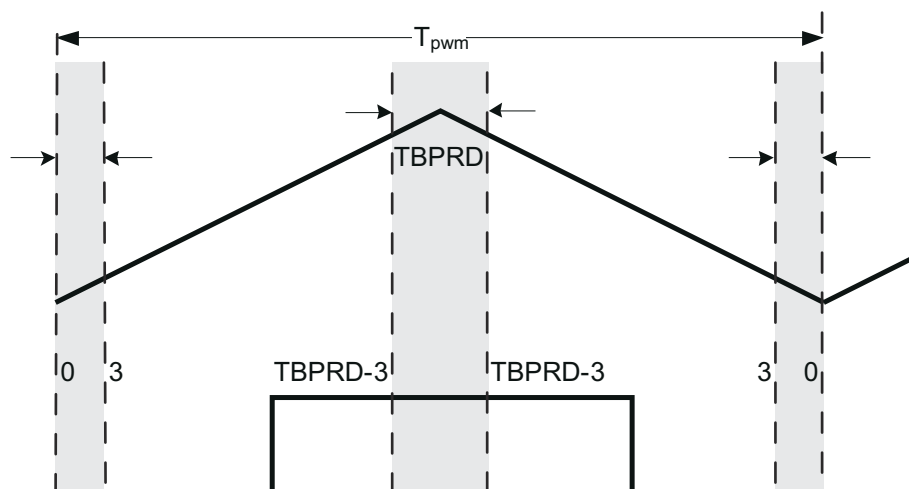
If the application demands HRPWM operation in the low percent duty cycle region, then the HRPWM can be configured to operate in count-down mode with the rising edge position (REP) controlled by the MEP when high-resolution period is disabled ( $\text{HRPCTL}[\text{HRPE}] = 0$ ). This is illustrated in [Figure 4-8](#). In this case, low percent duty limitation is no longer an issue. However, there will be a maximum duty limitation with same percent numbers as given in [Table 4-5](#).



**Figure 4-8. High % Duty Cycle Range Limitation Example ( $\text{HRPCTL}[\text{HRPE}] = 0$ )**



**Figure 4-9. Up-Count Duty Cycle Range Limitation Example ( $\text{HRPCTL}[\text{HRPE}] = 1$ )**



**Figure 4-10. Up-Down Count Duty Cycle Range Limitation Example (HRPCTL[HRPE]=1)**

**Note**

If the application has enabled high-resolution period control (HRPCTL[HRPE]=1), the duty cycle must not fall within the restricted range. Otherwise, there will be undefined behavior on the ePWM output.

#### 4.2.3.4 High Resolution Period

High resolution period control using the MEP logic is supported on devices with a Type 1 ePWM module via the TBPRDHR(M) register.

**Note**

When high-resolution period control is enabled, the ePWMxB output will have +/- 1 TBCLK cycle jitter in up-count mode and +/- 2 TBCLK cycle jitter in up-down count mode.

The scaling procedure described for duty cycle in [Section 4.2.3.2](#) applies for high-resolution period as well:

**Assumptions for this example:**

|                                                                                              |                                    |
|----------------------------------------------------------------------------------------------|------------------------------------|
| System clock , SYSCLKOUT                                                                     | = 11.1 ns (90 MHz)                 |
| Required PWM frequency                                                                       | = 175 kHz (TBPRD value of 514.286) |
| Number of MEP steps per coarse step at 180 ps (MEP_ScaleFactor)                              | = 61 (11.1 ns/180 ps)              |
| Value to keep TBPRDHR within range of 1-255 and fractional rounding constant (default value) | = 0.5 (0080h in Q8 format)         |

### Problem:

In up-count mode:

If TBPRD = 514, then PWM frequency = 174.75 kHz (period =  $(514+1) \cdot T_{TBCLK}$ ).

TBPRD = 513, then PWM frequency = 175.10 kHz (period =  $(513+1) \cdot T_{TBCLK}$ ).

In up-down count mode:

If TBPRD = 258, then PWM frequency = 174.42 kHz (period =  $(258*2) \cdot T_{TBCLK}$ ).

If TBPRD = 257, then PWM frequency = 175.10 kHz (period =  $(257*2) \cdot T_{TBCLK}$ ).

### Solution:

With 61 MEP steps per coarse step at 180 ps each:

#### Step 1: Percentage Integer Period value conversion for TBPRD register

Integer period value  $= 514 \cdot T_{TBCLK}$   
 $= \text{int}(514.286) \cdot T_{TBCLK}$   
 $= \text{int}(\text{PWMperiod}) \cdot T_{TBCLK}$

In up-count mode:

TBPRD register value  $= 513$  (TBPRD = period value - 1)  
 $= 0201\text{h}$

In up-down count mode:

TBPRD register value  $= 257$  (TBPRD = period value / 2)  
 $= 0101\text{h}$

#### Step 2: Fractional value conversion for TBPRDHR register

TBPRDHR register value  $= (\text{frac}(\text{PWMperiod}) \cdot \text{MEP\_ScaleFactor} + 0.5)$  (shift is to move the value as TBPRDHR high byte)

If auto-conversion enabled and HRMSTEP =

MEP\_ScaleFactor value (61):  $= \text{frac}(\text{PWMperiod}) \ll 8$

TBPRDHR register value  $= \text{frac}(514.286) \ll 8$

$= 0.286 \times 256$

$= 0049\text{h}$

The autoconversion will then automatically perform the calculation such that TBPRDHR MEP delay is scaled by hardware to:

$= ((\text{TBPRDHR}(15:0) \gg 8) \times \text{HRMSTEP} + 80\text{h}) \gg 8$

$= (0049\text{h} \times 61 + 80\text{h}) \gg 8$

$= (11\text{E}5\text{h}) \gg 8$

Period MEP delay

$= 0011\text{h}$  MEP Steps



#### 4.2.3.4.1 High-Resolution Period Configuration

To use High Resolution Period, the ePWMx module must be initialized, following the steps in this exact order:

1. Enable ePWMx clock
2. Disable TBCLKSYNC
3. Configure ePWMx registers - AQ, TBPRD, CC, etc.
  - ePWMx may only be configured for up-count or up-down count modes. High-resolution period is not compatible with down-count mode.
  - TBCLK must equal SYSCLKOUT
  - TBPRD and CC registers must be configured for shadow loads.
  - CMPCTL[LOADAMODE]
    - In up-count mode: CMPCTL[LOADAMODE] = 1 (load on CTR = PRD)
    - In up-down count mode: CMPCTL[LOADAMODE] = 2 (load on CTR=0 or CTR=PRD)
4. Configure HRPWM register such that:
  - HRCNFG[HRLOAD] = 2 (load on either CTR = 0 or CTR = PRD)
  - HRCNFG[AUTOCONV] = 1 (Enable auto-conversion)
  - HRCNFG[EDGMODE] = 3 (MEP control on both edges)
5. For TBPHS: TBPHSHR synchronization with high-resolution period, set both HRPCTL[TBPSHRLOADE] = 1 and TBCTL[PHSEN] = 1. In up-down count mode these bits must be set to 1 regardless of the contents of TBPHSHR.
6. Enable high-resolution period control (HRPCTL[HRPE] = 1)
7. Enable TBCLKSYNC
8. TBCTL[SWFSYNC] = 1
9. HRMSTEP must contain an accurate MEP scale factor (# of MEP steps per SYSCLKOUT coarse step) because auto-conversion is enabled. The MEP scale factor can be acquired via the SFO() function described in [Section 4.3](#).
10. To control high-resolution period, write to the TBPRDHR(M) registers.

#### Note

When high-resolution period mode is enabled, an EPWMxSYNC pulse will introduce +/- 1 - 2 cycle jitter to the PWM (+/- 1 cycle in up-count mode and +/- 2 cycle in up-down count mode). For this reason, TBCTL[SYNCOSEL] should not be set to 1 (CTR = 0 is EPWMxSYNCO source) or 2 (CTR = CMPB is EPWMxSYNCO source). Otherwise, the jitter will occur on every PWM cycle with the synchronization pulse.

When TBCTL[SYNCOSEL] = 0 (EPWMxSYNCO is EPWMxSYNCO source), a software synchronization pulse should be issued only once during high-resolution period initialization. If a software sync pulse is applied while the PWM is running, the jitter will appear on the PWM output at the time of the sync pulse.

#### 4.2.4 Scale Factor Optimizing Software (SFO)

The micro edge positioner (MEP) logic is capable of placing an edge in one of 255 discrete time steps. As previously mentioned, the size of these steps is on the order of 150 ps (see device-specific data sheet for typical MEP step size on your device). The MEP step size varies based on worst-case process parameters, operating temperature, and voltage. MEP step size increases with decreasing voltage and increasing temperature and decreases with increasing voltage and decreasing temperature. Applications that use the HRPWM feature should use the TI-supplied MEP scale factor optimizer (SFO) software function. The SFO function helps to dynamically determine the number of MEP steps per SYSCLKOUT period while the HRPWM is in operation.

To utilize the MEP capabilities effectively during the Q15 duty (or period) to [CMPA:CMPAHR] or [TBPRD(M):TBPRDHR(M)] mapping function (see [Section 4.2.3.2](#)), the correct value for the MEP scaling factor (MEP\_ScaleFactor) needs to be known by the software. To accomplish this, the HRPWM module has built in self-check and diagnostics capabilities that can be used to determine the optimum MEP\_ScaleFactor value for any operating condition. TI provides a C-callable library containing one SFO function that utilizes this hardware and determines the optimum MEP\_ScaleFactor. As such, MEP Control and Diagnostics registers are reserved for TI use.

A detailed description of the SFO library - SFO\_TI\_Build\_V6.lib software can be found in [Section 4.3](#).

#### 4.2.5 HRPWM Examples Using Optimized Assembly Code

The best way to understand how to use the HRPWM capabilities is through two real examples:

1. Simple buck converter using asymmetrical PWM (that is, count-up) with active high polarity.
2. DAC function using simple R+C reconstruction filter.

The following examples all have Initialization/configuration code written in C. To make these easier to understand, the #defines shown below are used. Note that the #defines introduced in [Chapter 3](#) are also used.

[Example 4-1](#) assumes an MEP step size of 150 ps and does not use the SFO library.

##### Example 4-1. #Defines for HRPWM Header Files

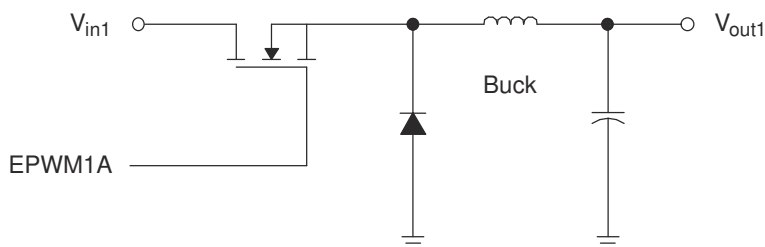
```
// HRPWM (High Resolution PWM) //
=====
// HRCNFG
#define HR_Disable 0x0
#define HR_REP 0x1           // Rising Edge position
#define HR_FEP 0x2           // Falling Edge position
#define HR_BEP 0x3           // Both Edge position #define HR_CMP 0x0 // CMPAHR controlled
#define HR_PHS 0x1           // TBPHSHR controlled #define HR_CTR_ZERO 0x0 // CTR = Zero event
#define HR_CTR_PRD 0x1        // CTR = Period event
#define HR_CTR_ZERO_PRD 0x2   // CTR = ZERO or Period event
#define HR_NORM_B 0x0         // Normal ePWMxB output
#define HR_INVERT_B 0x1       // ePWMxB is inverted ePWMxA output
```

#### 4.2.5.1 Implementing a Simple Buck Converter

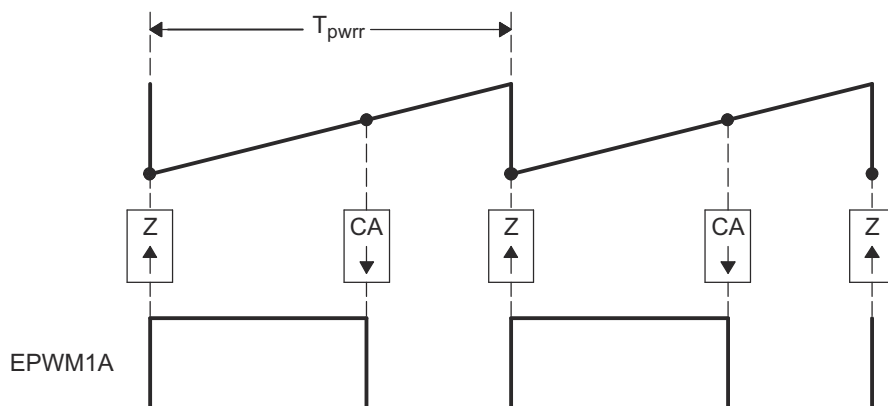
In this example, the PWM requirements for SYSCLKOUT = 80 MHz are:

- PWM frequency = 800 kHz (TBPRD = 100 )
- PWM mode = asymmetrical, up-count
- Resolution = 12.7 bits (with a MEP step size of 150 ps)

Figure 4-11 and Figure 4-12 show the required PWM waveform. As explained previously, configuration for the ePWM1 module is almost identical to the normal case except that the appropriate MEP options need to be enabled/selected.



**Figure 4-11. Simple Buck Controlled Converter Using a Single PWM**



**Figure 4-12. PWM Waveform Generated for Simple Buck Controlled Converter**

The example code shown consists of two main parts:

- Initialization code (executed once)
- Run time code (typically executed within an ISR)

**Example 4-2** shows the Initialization code. The first part is configured for conventional PWM. The second part sets up the HRPWM resources.

This example assumes an MEP step size of 150 ps and does not use the SFO library.

#### **Example 4-2. HRPWM Buck Converter Initialization Code**

```
void HrBuckDrvCnf(void)
{
    // Config for conventional PWM first
    EPwm1Regs.TBCTL.bit.PRDL = TB_IMMEDIATE;           // set Immediate load
    EPwm1Regs.TBPRD = 100;                             // Period set for 800 kHz PWM
    hrBuck_period = 200;                                // Used for Q15 to Q0 scaling
    EPwm1Regs.TBCTL.bit.CTRMODE = TB_COUNT_UP;
    EPwm1Regs.TBCTL.bit.PHSEN = TB_DISABLE;             // EPWM1 is the Master
    EPwm1Regs.TBCTL.bit.SYNCSEL = TB_SYNC_DISABLE;
    EPwm1Regs.TBCTL.bit.HSPCLKDIV = TB_DIV1;
    EPwm1Regs.TBCTL.bit.CLKDIV = TB_DIV1;
    // Note: ChB is initialized here only for comparison purposes, it is not required

    EPwm1Regs.CMPCTL.bit.LOADAMODE = CC_CTR_ZERO;
    EPwm1Regs.CMPCTL.bit.SHDWAMODE = CC_SHADOW;
    EPwm1Regs.CMPCTL.bit.LOADBMODE = CC_CTR_ZERO;       // optional
    EPwm1Regs.CMPCTL.bit.SHDWBMODE = CC_SHADOW;         // optional
    EPwm1Regs.AQCTLA.bit.ZRO = AQ_SET;
    EPwm1Regs.AQCTLA.bit.CAU = AQ_CLEAR;
    EPwm1Regs.AQCTLB.bit.ZRO = AQ_SET;                 // optional
    EPwm1Regs.AQCTLB.bit.CBU = AQ_CLEAR;               // optional
    // Now configure the HRPWM resources
    EALLOW;                                             // Note these registers are protected
                                                    // and act only on ChA
    EPwm1Regs.HRCNFG.all = 0x0;                        // clear all bits first
    EPwm1Regs.HRCNFG.bit.EDGMODE = HR_FEP;             // Control Falling Edge Position
    EPwm1Regs.HRCNFG.bit.CTLMODE = HR_CMP;             // CMPAHR controls the MEP
    EPwm1Regs.HRCNFG.bit.HRLOAD = HR_CTR_ZERO;         // Shadow load on CTR=Zero
    EDIS;
    MEP_ScaleFactor = 83*256;                          // Start with typical Scale Factor
                                                    // value for 80 MHz
                                                    // Note: Use SFO functions to update
                                                    // MEP_ScaleFactor dynamically
}
```

**Example 4-3** shows an assembly example of run-time code for the HRPWM buck converter.

#### **Example 4-3. HRPWM Buck Converter Run-Time Code**

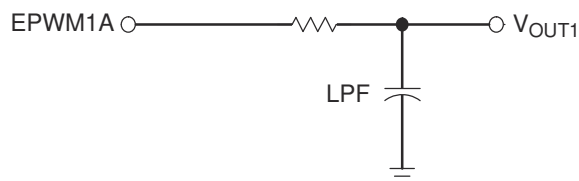
```
EPWM1_BASE .set 0x6800
CMPAHR1 .set EPWM1_BASE+0x8
;=====
HRBUCK_DRV; (can execute within an ISR or loop)
;=====
    MOVW DP, # HRBUCK_In
    MOVL XAR2,@ HRBUCK_In          ; Pointer to Input Q15 Duty (XAR2)
    MOVL XAR3,#CMPAHR1             ; Pointer to HRPWM CMPA reg (XAR3)
; Output for EPWM1A (HRPWM)
    MOV T,*XAR2 ; T <= Duty
    MPYU ACC,T,@ hrBuck_period ; Q15 to Q0 scaling based on Period
    MOV T,@ MEP_ScaleFactor       ; MEP scale factor (from optimizer s/w)
    MPYU P,T,@AL                 ; P <= T * AL, Optimizer scaling
    MOVH @AL,P                   ; AL <= P, move result back to ACC
    ADD ACC,#0x080                ; MEP range and rounding adjustment
    MOVL *XAR3,ACC                ; CMPA: CMPAHR(31:8) <= ACC
; Output for EPWM1B (Regular Res) Optional - for comparison purpose only
    MOV *+XAR3[2],AH              ; Store ACCH to regular CMPB
```

#### 4.2.5.2 Implementing a DAC Function Using an R+C Reconstruction Filter

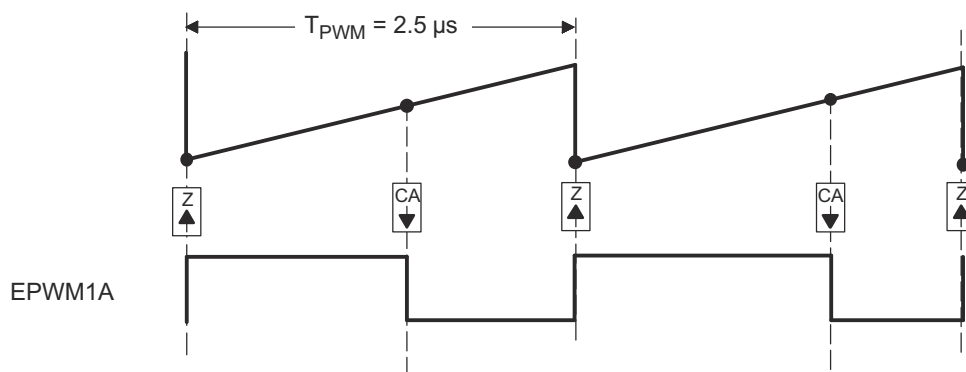
In this example, the PWM requirements are:

- PWM frequency = 533 kHz (that is, TBPRD = 150)
- PWM mode = Asymmetrical, Up-count
- Resolution = 14 bits (MEP step size = 150 ps)

Figure 4-13 and Figure 4-14 show the DAC function and the required PWM waveform. As explained previously, configuration for the ePWM1 module is almost identical to the normal case except that the appropriate MEP options need to be enabled/selected.



**Figure 4-13. Simple Reconstruction Filter for a PWM Based DAC**



**Figure 4-14. PWM Waveform Generated for the PWM DAC Function**

The example code shown consists of two main parts:

- Initialization code (executed once)
- Run time code (typically executed within an ISR)

This example assumes a typical MEP\_SP and does not use the SFO library.

Example 4-4 shows the Initialization code. The first part is configured for conventional PWM. The second part sets up the HRPWM resources.

#### Example 4-4. PWM DAC Function Initialization Code

```
void HrPwmDacDrvCnf(void)
{
    // Config for conventional PWM first
    EPwm1Regs.TBCTL.bit.PRDL = TB_IMMEDIATE; // Set Immediate load
    EPwm1Regs.TBPRD = 150; // Period set for 533 kHz PWM
    hrDAC_period = 150; // Used for Q15 to Q0 scaling
    EPwm1Regs.TBCTL.bit.CTRMODE = TB_COUNT_UP;
    EPwm1Regs.TBCTL.bit.PHSEN = TB_DISABLE; // EPWM1 is the Master
    EPwm1Regs.TBCTL.bit.SYNCSEL = TB_SYNC_DISABLE;
    EPwm1Regs.TBCTL.bit.HSPCLKDIV = TB_DIV1;
    EPwm1Regs.TBCTL.bit.CLKDIV = TB_DIV1;
    // Note: ChB is initialized here only for comparison purposes, it is not required
    EPwm1Regs.CMPCTL.bit.LOADAMODE = CC_CTR_ZERO;
    EPwm1Regs.CMPCTL.bit.SHDWAMODE = CC_SHADOW;
    EPwm1Regs.CMPCTL.bit.LOADBMODE = CC_CTR_ZERO; // optional
    EPwm1Regs.CMPCTL.bit.SHDWBMODE = CC_SHADOW; // optional
    EPwm1Regs.AQCTLA.bit.ZRO = AQ_SET;
    EPwm1Regs.AQCTLA.bit.CAU = AQ_CLEAR;
    EPwm1Regs.AQCTLB.bit.ZRO = AQ_SET; // optional
    EPwm1Regs.AQCTLB.bit.CBU = AQ_CLEAR; // optional
    // Now configure the HRPWM resources
    EALLOW; // Note these registers are protected
    // and act only on ChA.
    EPwm1Regs.HRCNFG.all = 0x0; // Clear all bits first
    EPwm1Regs.HRCNFG.bit.EDGMODE = HR_FEP; // Control falling edge position
    EPwm1Regs.HRCNFG.bit.CTLMODE = HR_CMP; // CMPAHR controls the MEP.
    EPwm1Regs.HRCNFG.bit.HRLOAD = HR_CTR_ZERO; // Shadow load on CTR=Zero.
    EDIS;
    MEP_ScaleFactor = 83*256; // Start with typical Scale Factor
    // value for 80 MHz.
    // Use SFO functions to update MEP_ScaleFactor
    // dynamically.
}
```

Example 4-5 shows an assembly example of run-time code that can execute in a high-speed ISR loop.

#### Example 4-5. PWM DAC Function Run-Time Code

```
EPWM1_BASE .set 0x6800
CMPAHR1 .set EPWM1_BASE+0x8
;=====
HRPWM_DAC_DRV; (can execute within an ISR or loop)
;=====
    MOVW DP, # HRDAC_In
    MOVL XAR2,@ HRDAC_In // Pointer to input Q15 duty (XAR2)
    MOVL XAR3,#CMPAHR1 // Pointer to HRPWM CMPA reg (XAR3)
; Output for EPWM1A (HRPWM
    MOV T,*XAR2 // T <= duty
    MPY ACC,T,@_hrDAC_period // Q15 to Q0 scaling based on period
    ADD ACC,@_hrDAC_period<<15 // Offset for bipolar operation
    MOV T,@_MEP_ScaleFactor // MEP scale factor (from optimizer s/w)
    MPYU P,T,@AL // P <= T * AL, optimizer scaling
    MOVH @AL,P // AL <= P, move result back to ACC
    ADD ACC,#0x080 // MEP range and rounding adjustment
    MOVL *XAR3,ACC // CMPA:CMPAHR(31:8) <= ACC
; Output for EPWM1B (Regular Res) Optional - for comparison purpose only
    MOV *+XAR3[2],AH // Store ACCH to regular CMPB
```

### 4.3 SFO Library Software - SFO\_TI\_Build\_V6.lib

Table 4-6 lists several features of the SFO\_TI\_Build\_V6.lib library.

**Table 4-6. SFO Library Features**

|                                                                                                      | SYSCCLK Freq | SFO_TI_Build_V6.lib | Unit         |
|------------------------------------------------------------------------------------------------------|--------------|---------------------|--------------|
| Maximum HRPWM channels supported                                                                     | -            | 8                   | channels     |
| Total static variable memory size                                                                    | -            | 11                  | words        |
| Completion-checking?                                                                                 | -            | Yes                 | -            |
| Typical time required for SFO() to update MEP_ScaleFactor, if called repetitively without interrupts | 80 MHz       | 1.3                 | milliseconds |
|                                                                                                      | 60 MHz       | 2.23                | milliseconds |

Following is a functional description of the SFO library routine, SFO().

#### 4.3.1 Scale Factor Optimizer Function - int SFO()

This routine drives the micro-edge positioner (MEP) calibration module to run SFO diagnostics and determine the appropriate MEP scale factor (number of MEP steps per coarse SYSCCLKOUT step) for a device at any given time.

If SYSCCLKOUT = TBCLK = 90 MHz and assuming the MEP step size is 150 ps, the typical scale factor value at 90 MHz = 74 MEP steps per TBCLK unit (11.1 ns)

The function returns a MEP scale factor value:

MEP\_ScaleFactor = Number of MEP steps/SYSCCLKOUT.

#### Constraints when using this function:

- SFO() can be used with a minimum SYSCCLKOUT = TBCLK = 50 MHz. MEP diagnostics logic uses SYSCCLKOUT and not TBCLK, so the SYSCCLKOUT restriction is an important constraint. Below 50 MHz, with device process variation, the MEP step size may decrease under cold temperature and high core voltage conditions to such a point, that 255 MEP steps will not span an entire SYSCCLKOUT cycle.
- At any time, SFO() can be called to run SFO diagnostics on the MEP calibration module

#### Usage:

- SFO() can be called at any time in the background while the ePWM channels are running in HRPWM mode. The scale factor result obtained in MEP\_ScaleFactor can be applied to all ePWM channels running in HRPWM mode because the function makes use of the diagnostics logic in the MEP calibration module (which runs independently of ePWM channels).
- This routine returns a 1 when calibration is finished, and a new scale factor has been calculated or a 0 if calibration is still running. The routine returns a 2 if there is an error, and the MEP\_ScaleFactor is greater than the maximum 255 fine steps per coarse SYSCCLKOUT cycle. In this case, the HRMSTEP register will maintain the last MEP scale factor value less than 256 for auto conversion.
- All ePWM modules operating in HRPWM incur only a 3-SYSCCLKOUT cycle minimum duty cycle limitation when high-resolution period control is not used. If high-resolution period control is enabled, there is an additional duty cycle limitation 3-SYSCCLKOUT cycles before the end of the PWM period (see [Section 4.2.3.3](#)).
- In SFO\_TI\_Build\_V6b.lib, the SFO() function also updates the HRMSTEP register with the scale factor result. If the HRCNFG[AUTOCONV] bit is set, the application software is responsible only for setting CMPAHR = fraction(PWMDuty\*PWMperiod)<<8 or TBPRDHR = fraction (PWMperiod) while running SFO() in the background. The MEP Calibration Module will then use the values in the HRMSTEP and CMPAHR/TBPRDHR register to automatically calculate the appropriate number of MEP steps represented by the fractional duty cycle or period and move the high-resolution ePWM signal edge accordingly. In SFO\_TI\_Build\_V6.lib, the SFO() function does not automatically update the HRMSTEP register. Therefore, after the SFO function completes, the application software must write MEP\_ScaleFactor to the HRMSTEP register (EALLOW-protected).

- If the HRCNFG[AUTOCONV] bit is clear, the HRMSTEP register is ignored. The application software will need to perform the necessary calculations manually so that:
  - $CMPAHR = (\text{fraction}(\text{PWMduty} * \text{PWMperiod}) * \text{MEP Scale Factor}) < 8 + 0x080$ .
  - Similar behavior applies for TBPHSHR. Auto-conversion must be enabled when using TBPRDHR.

The routine can be run as a background tasks in a slow loop requiring negligible CPU cycles. The repetition rate at which an SFO function needs to be executed depends on the application's operating environment. As with all digital CMOS devices temperature and supply voltage variations have an effect on MEP operation. However, in most applications these parameters vary slowly and therefore it is often sufficient to execute the SFO function once every 5 to 10 seconds or so. If more rapid variations are expected, then execution may have to be performed more frequently to match the application. Note, there is no high limit restriction on the SFO function repetition rate, hence it can execute as quickly as the background loop is capable.

While using the HRPWM feature, HRPWM logic will not be active for the first 3 SYSCLKOUT cycles of the PWM period (and the last 3 SYSCLKOUT cycles of the PWM period if TBPRDHR is used). While running the application in this configuration, if high-resolution period control is disabled (HRPCTL[HRPE=0]) and the CMPA register value is less than 3 cycles, then its CMPAHR register must be cleared to zero. If high-resolution period control is enabled (HRPCTL[HRPE=1]), the CMPA register value must not fall below 3 or above TBPRD-3. This would avoid any unexpected transitions on the PWM signal.

### 4.3.2 Software Usage

The software library function SFO(), calculates the MEP scale factor for the HRPWM-supported ePWM modules. The scale factor is an integer value in the range 1-255, and represents the number of micro step edge positions available for a system clock period. The scale factor value is returned in an integer variable called MEP\_ScaleFactor. For example, see [Table 4-7](#).

**Table 4-7. Factor Values**

| Software Function call | Functional Description                                                   | Updated Variables                     |
|------------------------|--------------------------------------------------------------------------|---------------------------------------|
| SFO()                  | Returns MEP scale factor in MEP_ScaleFactor                              | MEP_ScaleFactor and HRMSTEP register. |
|                        | Returns MEP scale factor in the HRMSTEP register in SFO_TI_Build_V6b.lib |                                       |

To use the HRPWM feature of the ePWMs it is recommended that the SFO function be used as described here.

#### Step 1. Add "Include" Files

The SFO\_V6.h file needs to be included as follows. This include file is mandatory while using the SFO library function. For the TMS320F2806x devices, the F2806x C/C++ Header Files and Peripheral Examples in controlSUITE F2806x\_Device.h and F2806x\_Epwm\_defines.h are necessary. For other device families, the device-specific equivalent files in the header files and peripheral examples software packages for those devices should be used. These include files are optional if customized header files are used in the end applications.

#### Example 4-6. A Sample of How to Add "Include" Files

```
#include "F2806x_Device.h"           // F2806x Headerfile
#include "F2806x_EPwm_defines.h"     // init defines
#include "SFO_V6.h"                  // SFO lib functions (needed for HRPWM)
```



## Step 2. Element Declaration

Declare an integer variable for the scale factor value as shown in [Example 4-7](#).

### Example 4-7. Declaring an Element

```
int MEP_ScaleFactor = 0;    //scale factor value
volatile struct EPWM_REGS *ePWM[] = {0, &EPwm1Regs, &EPwm2Regs, &EPwm3Regs,
&EPwm4Regs};
```

## Step 3. MEP\_ScaleFactor Initialization

The SFO() function does not require a starting scale factor value in MEP\_ScaleFactor. Prior to using the MEP\_ScaleFactor variable in application code, SFO() should be called to drive the MEP calibration module to calculate an MEP\_ScaleFactor value.

As part of the one-time initialization code prior to using MEP\_ScaleFactor, include the following shown in [Example 4-8](#).

### Example 4-8. Initializing With a Scale Factor Value

```
MEP_ScaleFactor initialized using function SFO ()
while (SFO() == 0) {} // MEP_ScaleFactor calculated by MEP Cal Module
```

## Step 4. Application Code

While the application is running, fluctuations in both device temperature and supply voltage may be expected. To be sure that optimal Scale Factors are used for each ePWM module, the SFO function should be re-run periodically as part of a slower back-ground loop. Some examples of this are shown in [Example 4-9](#).

### Note

See the HRPWM\_SFO example in the device-specific C/C++ header files and peripheral examples available from the TI website.

### Example 4-9. SFO Function Calls

```
main ()
{
    int status;
    // User code
    // ePWM1, 2, 3, 4 are running in HRPWM mode
    // The status variable returns 1 once a new MEP_ScaleFactor has been
    // calculated by the MEP Calibration Module running SFO
    // diagnostics.
    status = SFO();
    if(status==2) {ESTOP0;}    // The function returns a 2 if MEP_ScaleFactor is greater
                             // than the maximum 255 allowed (error condition)
}
```

### 4.3.3 SFO Library Version Software Differences

There are two different versions of the SFO library - SFO\_TI\_Build\_V6.lib, and SFO\_TI\_Build\_V6b.lib. SFO\_TI\_Build\_V6.lib does not update the HRMSTEP register with the value in MEP\_ScaleFactor, while SFO\_TI\_Build\_V6b.lib updates the register. Therefore, if using SFO\_TI\_Build\_V6.lib and auto-conversion is enabled, the application should write MEP\_Scalefactor in the HRMSTEP register as shown in [Example 4-10](#).

**Example 4-10. Manually Updating the HRMSTEP Register if Using SFO\_TI\_Build\_V6b.lib**

```
main ()
{
    int status;

    status = SFO_INCOMPLETE;
    while (status==SFO_INCOMPLETE) {
        status = SFO();
    }
    if(status!=SFO_ERROR) { // IF SFO() is complete with no errors
        EALLOW;
        EPwm1Regs.HRMSTEP=MEP_ScaleFactor;
        EDIS;
    }
}
```

## 4.4 HRPWM Registers

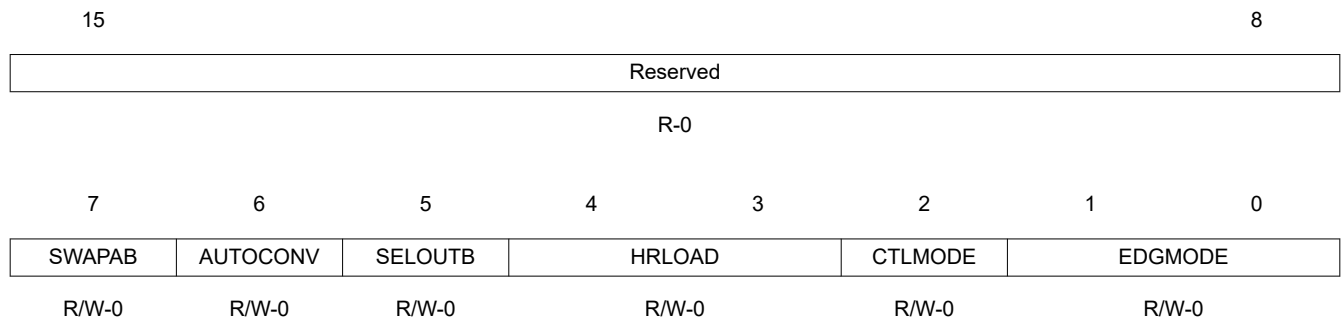
A summary of the registers required for the HRPWM is listed in [Table 4-8](#).

**Table 4-8. HRPWM Registers**

| Name                                               | Offset | Size(x16) | Shadow | Description                                       | Bit Description                 |
|----------------------------------------------------|--------|-----------|--------|---------------------------------------------------|---------------------------------|
| <b>Time-Base Registers</b>                         |        |           |        |                                                   |                                 |
| TBCTL                                              | 0x0000 | 1         |        | Time-Base Control Register                        | <a href="#">Section 3.4.1.1</a> |
| TBSTS                                              | 0x0001 | 1         |        | Time-Base Status Register                         | <a href="#">Section 3.4.1.2</a> |
| TBPHSHR                                            | 0x0002 | 1         |        | Time-Base Phase High Resolution Register          | <a href="#">Section 3.4.1.3</a> |
| TBPHS                                              | 0x0003 | 1         |        | Time-Base Phase Register                          | <a href="#">Section 3.4.1.4</a> |
| TBCTR                                              | 0x0004 | 1         |        | Time-Base Counter Register                        | <a href="#">Section 3.4.1.5</a> |
| TBPRD                                              | 0x0005 | 1         | Yes    | Time-Base Period Register Set                     | <a href="#">Section 3.4.1.6</a> |
| TBPRDHR                                            | 0x0006 | 1         | Yes    | Time-Base Period High Resolution Register Set     | <a href="#">Section 3.4.1.7</a> |
| <b>Counter-Compare Registers</b>                   |        |           |        |                                                   |                                 |
| CMPCTL                                             | 0x0007 | 1         |        | Counter-Compare Control Register                  | <a href="#">Section 3.4.2.1</a> |
| CMPAHR                                             | 0x0008 | 1         | Yes    | Counter-Compare A High Resolution Register        | <a href="#">Section 3.4.2.2</a> |
| CMPA                                               | 0x0009 | 1         | Yes    | Counter-Compare A Register                        | <a href="#">Section 3.4.2.3</a> |
| CMPB                                               | 0x000A | 1         | Yes    | Counter-Compare B Register                        | <a href="#">Section 3.4.2.4</a> |
| <b>HRPWM Registers</b>                             |        |           |        |                                                   |                                 |
| HRCNFG                                             | 0x0020 | 1         |        | HRPWM Configuration Register                      | <a href="#">Section 4.4.1</a>   |
| HRMSTEP                                            | 0x0026 | 1         |        | HRPWM MEP Step Register                           | <a href="#">Section 4.4.2</a>   |
| <b>High Resolution Period and Mirror Registers</b> |        |           |        |                                                   |                                 |
| HRPCTL                                             | 0x0028 | 1         |        | High Resolution Period Control Register           | <a href="#">Section 4.4.3</a>   |
| TBPRDHRM                                           | 0x002A | 1         | Writes | Time-Base Period High Resolution Mirror Register  | <a href="#">Section 3.4.1.8</a> |
| TBPRDM                                             | 0x002B | 1         | Writes | Time-Base Period Mirror Register                  | <a href="#">Section 3.4.1.9</a> |
| CMPAHRM                                            | 0x002C | 1         | Writes | Counter-Compare A High Resolution Mirror Register | <a href="#">Section 3.4.2.5</a> |
| CMPAM                                              | 0x002D | 1         | Writes | Counter-Compare A Mirror Register                 | <a href="#">Section 3.4.2.6</a> |

### 4.4.1 HRPWM Configuration (HRCNFG) Register

**Figure 4-15. HRPWM Configuration (HRCNFG) Register**



LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 4-9. HRPWM Configuration (HRCNFG) Register Field Descriptions**

| Bit  | Field    | Value                                                                                                                                                                                                                                                                                                                                                  | Description <sup>(1)</sup>                                                                                                                                                                                                                                                                                                                                                    |
|------|----------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-8 | Reserved |                                                                                                                                                                                                                                                                                                                                                        | Reserved                                                                                                                                                                                                                                                                                                                                                                      |
| 7    | SWAPAB   | 0 ePWMxA and ePWMxB outputs are unchanged.<br>1 ePWMxA signal appears on ePWMxB output and ePWMxB signal appears on ePWMxA output.                                                                                                                                                                                                                     | Swap ePWM A and B Output Signals<br>This bit enables the swapping of the A and B signal outputs. The selection is as follows:                                                                                                                                                                                                                                                 |
| 6    | AUTOCONV | 0 Automatic HRMSTEP scaling is disabled.<br>1 Automatic HRMSTEP scaling is enabled.<br>If application software is manually scaling the fractional duty cycle, or phase (that is, software sets $CMPAHR = (\text{fraction}(\text{PWMduty} * \text{PWMperiod}) * \text{MEP Scale Factor}) < 8 + 0x080$ for duty cycle), then this mode must be disabled. | Auto Convert Delay Line Value<br>Selects whether the fractional duty cycle/period/phase in the CMPAHR/TBPRDHR/TBPHSHR register is automatically scaled by the MEP scale factor in the HRMSTEP register or manually scaled by calculations in application software. The SFO library function automatically updates the HRMSTEP register with the appropriate MEP scale factor. |
| 5    | SELOUTB  | 0 ePWMxB output is normal.<br>1 ePWMxB output is inverted version of ePWMxA signal.                                                                                                                                                                                                                                                                    | ePWMxB Output Select Bit<br>This bit selects which signal is output on the ePWMxB channel output.                                                                                                                                                                                                                                                                             |
| 4-3  | HRLOAD   | 00 Load on CTR = Zero: Time-base counter equal to zero (TBCTR = 0x0000)<br>01 Load on CTR = PRD: Time-base counter equal to period (TBCTR = TBPRD)<br>10 Load on either CTR = Zero or CTR = PRD<br>11 Reserved                                                                                                                                         | Shadow Mode Bit<br>Selects the time event that loads the CMPAHR shadow value into the active register.                                                                                                                                                                                                                                                                        |
| 2    | CTLMODE  | 0 CMPAHR(8) or TBPRDHR(8) Register controls the edge position (that is, this is duty or period control mode). (Default on Reset)<br>1 TBPHSHR(8) Register controls the edge position (that is, this is phase control mode).                                                                                                                            | Control Mode Bits<br>Selects the register (CMP/TBPRD or TBPHS) that controls the MEP:                                                                                                                                                                                                                                                                                         |
| 1-0  | EDGMODE  | 00 HRPWM capability is disabled (default on reset)<br>01 MEP control of rising edge (CMPAHR)<br>10 MEP control of falling edge (CMPAHR)<br>11 MEP control of both edges (TBPHSHR or TBPRDHR)                                                                                                                                                           | Edge Mode Bits<br>Selects the edge of the PWM that is controlled by the micro-edge position (MEP) logic:                                                                                                                                                                                                                                                                      |

(1) This register is EALLOW protected.

#### 4.4.2 High Resolution Micro Step (HRMSTEP) Register

**Figure 4-16. High Resolution Micro Step (HRMSTEP) Register**

|          |   |         |   |
|----------|---|---------|---|
| 15       | 8 | 7       | 0 |
| Reserved |   | HRMSTEP |   |
| R-0      |   | R/W-0   |   |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 4-10. High Resolution Micro Step (HRMSTEP) Register Field Descriptions**

| Bit  | Field    | Value  | Description <sup>(1)</sup>                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|------|----------|--------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-8 | Reserved |        | Reserved                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| 7-0  | HRMSTEP  | 00-FFh | <p>High Resolution MEP Step</p> <p>When auto-conversion is enabled (HRCNFG[AUTOCONV] = 1), This 8-bit field contains the MEP_ScaleFactor (number of MEP steps per coarse steps) used by the hardware to automatically convert the value in the CMPAHR, TBPHSHR, or TBPRDHR register to a scaled micro-edge delay on the high-resolution ePWM output.</p> <p>The value in this register is written by the SFO calibration software at the end of each calibration run.</p> |

(1) This register is EALLOW protected.

### 4.4.3 High Resolution Period Control (HRPCTL) Register

**Figure 4-17. High Resolution Period Control (HRPCTL) Register**

|          |  |  |  |  |  |                  |   |            |  |       |  |   |
|----------|--|--|--|--|--|------------------|---|------------|--|-------|--|---|
| 15       |  |  |  |  |  |                  |   |            |  |       |  | 8 |
| Reserved |  |  |  |  |  |                  |   |            |  |       |  |   |
| R-0      |  |  |  |  |  |                  |   |            |  |       |  |   |
| 7        |  |  |  |  |  | 3                | 2 | 1          |  | 0     |  |   |
| Reserved |  |  |  |  |  | TBPHSHR<br>LOADE |   | PWMSYNCSEL |  | HRPE  |  |   |
| R-0      |  |  |  |  |  | R/W-0            |   | R/W-0      |  | R/W-0 |  |   |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 4-11. High Resolution Period Control (HRPCTL) Register Field Descriptions**

| Bit  | Field        | Value | Description <sup>(1)</sup> <sup>(2)</sup>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|------|--------------|-------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-3 | Reserved     |       | Reserved                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| 2    | TBPHSHRLOADE |       | <p>TBPHSHR Load Enable</p> <p>This bit allows you to synchronize ePWM modules with a high-resolution phase on a SYNCIN, TBCTL[SWFSYNC] or digital compare event. This allows for multiple ePWM modules operating at the same frequency to be phase aligned with high-resolution.</p> <p>0 Disables synchronization of high-resolution phase on a SYNCIN, TBCTL[SWFSYNC] or digital compare event:</p> <p>1 Synchronize the high-resolution phase on a SYNCIN, TBCTL[SWFSYNC] or digital comparator synchronization event. The phase is synchronized using the contents of the high-resolution phase TBPHSHR register.</p> <p>The TBCTL[PHSEN] bit which enables the loading of the TBCTR register with TBPHS register value on a SYNCIN or TBCTL[SWFSYNC] event works independently. However, users need to enable this bit also if they want to control phase in conjunction with the high-resolution period feature.</p> <p>This bit and the TBCTL[PHSEN] bit must be set to 1 when high-resolution period is enabled for up-down count mode even if TBPHSHR = 0x0000. This bit does not need to be set when only high-resolution duty is enabled.</p> |
| 1    | PWMSYNCSEL   |       | <p>PWMSYNC Source Select Bit</p> <p>This bit selects the source for the PWMSYNC signal.</p> <p>The PWMSYNC signal is used by external modules (such as COMP+DAC) for synchronizing timing to the selected ePWM module.</p> <p><b>Note:</b> This bit is not used for high-resolution period control.</p> <p>0 PWMSYNC is generated by TBCTR = PRD pulse.</p> <p>1 PWMSYNC is generated by TBCTR = 0 pulse.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| 0    | HRPE         |       | <p>High Resolution Period Enable Bit</p> <p>0 High resolution period feature disabled. In this mode the ePWM behaves as a Type 0 ePWM.</p> <p>1 High resolution period enabled. In this mode the HRPWM module can control high-resolution of both the duty and frequency.</p> <p>When high-resolution period is enabled, TBCTL[CTRMODE] = 0,1 (down-count mode) is not supported.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |

(1) This register is EALLOW protected.

(2) This register is used with Type 1 ePWM modules (support high-resolution period) only.

## High-Resolution Capture (HRCAP) Module

This chapter describes the operation of the high-resolution capture (HRCAP) module. The HRCAP module described here is a Type 0 HRCAP. HRCAP measures the width of external pulses with a typical resolution within hundreds of picoseconds. See the [TMS320x28xx, 28xxx DSP Peripheral Reference Guide](#) for a list of all devices with an HRCAP module of the same type, to determine the differences between types, and for a list of device-specific differences within a type.

|                                             |            |
|---------------------------------------------|------------|
| <b>5.1 Introduction.....</b>                | <b>406</b> |
| <b>5.2 Description.....</b>                 | <b>406</b> |
| <b>5.3 Operational Details.....</b>         | <b>407</b> |
| <b>5.4 HRCAP Calibration Library.....</b>   | <b>413</b> |
| <b>5.5 HRCAP Register Descriptions.....</b> | <b>421</b> |

## 5.1 Introduction

Uses for the HRCAP include:

- Capacitive touch applications
- High-resolution period and duty-cycle measurements of pulse-train cycles
- Instantaneous speed measurements
- Instantaneous frequency measurements
- Voltage measurements across an isolation boundary
- Distance and sonar measurement and scanning

### 5.1.1 Features

The HRCAP module includes the following features:

- Pulse-width capture in either non-high-resolution or high-resolution modes
- Difference (Delta) mode pulse-width capture
- Typical high-resolution capture on the order of 300 ps resolution on each edge
- Interrupt on either falling or rising edge
- Continuous mode capture of pulse widths in 2-deep buffer
- Calibration logic for precision high-resolution capture
- All of the above resources are dedicated to a single input pin.

### 5.1.2 HRCAP Related Collateral

#### Getting Started Materials

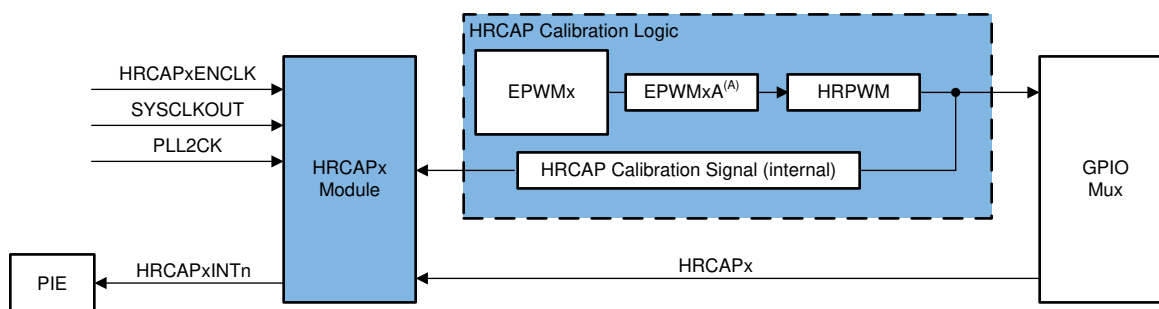
- [Leveraging High Resolution Capture \(HRCAP\) for Single Wire Data Transfer Application Report](#)

## 5.2 Description

The HRCAP module includes one capture channel in addition to a high-resolution calibration block, which connects internally to an HRPWM channel during calibration. See the device-specific data manual to determine which HRPWM channel output the HRCAP module is internally tied to during calibration.

Each HRCAP channel has the following independent key resources:

- Dedicated input capture pin
- 16-bit HRCAP clock (HCCAPCLK) that is either equal to the PLL2 output frequency (asynchronous to SYSCLK2) or equal to the SYSCLKOUT frequency .
- High-resolution pulse width capture in a two-deep buffer
- High-resolution calibration logic utilizing an internal connection to an HRPWM output



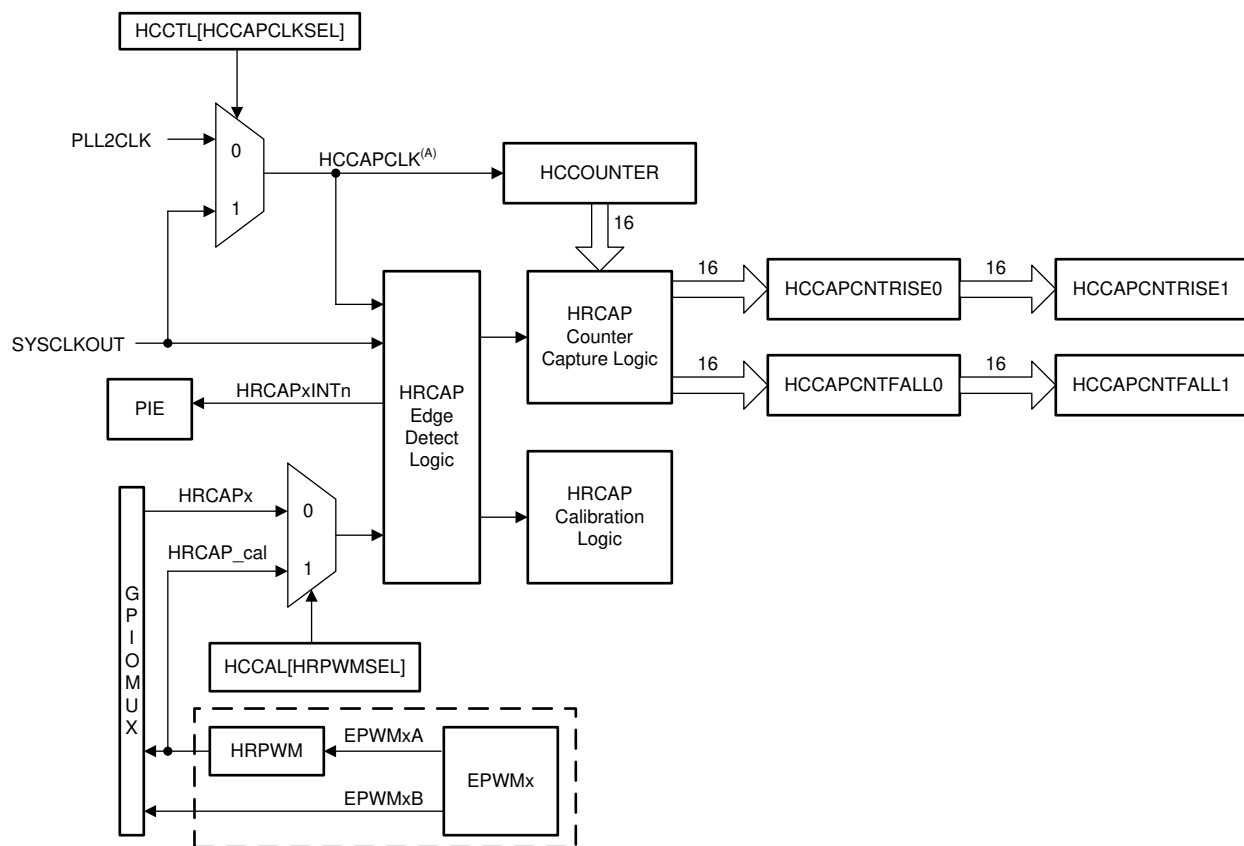
- A. In general, the largest numerical instance of an HRPWM module channel A output is the internal HRCAP calibration signal input. For instance, on devices where there are eight HRPWM instances, ePWM8A HRPWM output is the internal HRCAP calibration signal input.

**Figure 5-1. HRCAP Module System Block Diagram**



## 5.3 Operational Details

Figure 5-2 shows the various components that implement the high-resolution, pulse-width capture functionality of the module.



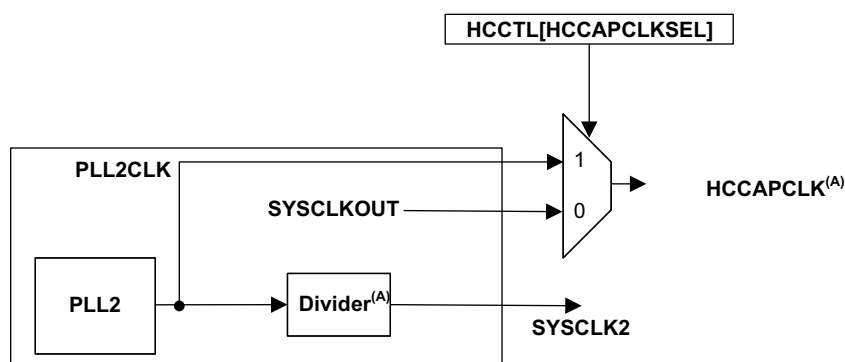
A. If PLLCLK is selected as the source for HCCAPCLK, HCCAPCLK is asynchronous to SYSCLK.

**Figure 5-2. HRCAP Block Diagram**

### 5.3.1 HRCAP Clocking

Although the HRCAP module is clocked by the system clock, the 16-bit counter (HCCOUNTER) and edge detection logic used for capturing high-resolution pulses is clocked by HCCAPCLK. HCCAPCLK must fall within the frequency range specified in the *Electricals* section of the device-specific data manual. HCCAPCLK can either be clocked by the system clock (SYSCLK), or the output of the PLL2 (PLL2CLK) before the divider is applied. If HCCAPCLK is fed from the PLL2CLK (HCCTL[HCCAPCLKSEL] = 1), then HCCAPCLK will be asynchronous to SYSCLK2. On this device, HCCAPCLK is clocked by SYSCLKOUT or PLL2.

Figure 5-3 shows how the HCCAPCLK that clocks the HCCOUNTER and edge detection logic is generated.



A. On this device, the clock divider is a default value of of /2 when DEVICECNF[SYSCLK2DIV2DIS] =0.

**Figure 5-3. HCCAPCLK Generation**

### 5.3.2 HRCAP Modes of Operation

The HRCAP module has two modes of operation:

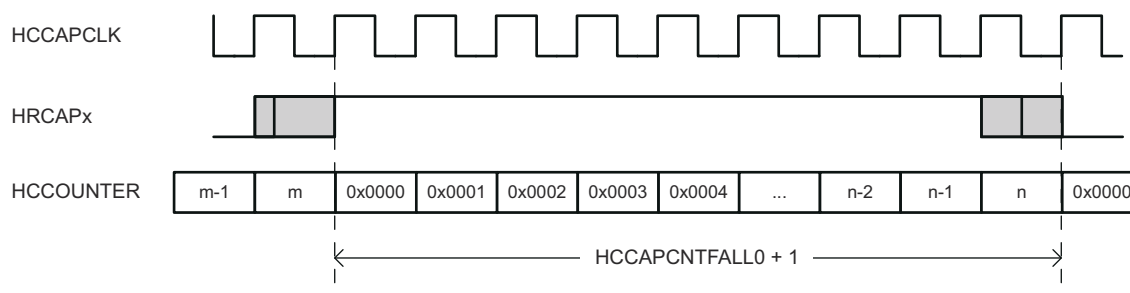
- **Normal capture mode:** The HRCAP module captures pulse widths in normal resolution within +/- 1 SYSCLK (where SYSCLK is sourced from the same PLL output clock that sources HCCAPCLK — on this device, this is SYSCLK2). This mode requires less software overhead than high-resolution capture mode.
- **High-resolution capture mode:** The HRCAP module captures pulse widths with the resolution of each edge captured within +/- 300 ps typical and requires the usage of the HCCal calibration library provided by Texas Instruments. In this mode, one HRCAP channel and the ePWM module connected to the HRCAP calibration input must be dedicated to HRCAP calibration and are not functionally available to the application during calibration.

#### 5.3.2.1 HRCAP Counter

Both modes of operation utilize HCCOUNTER, which resets to 0 and starts counting HCCAPCLK cycles again under the following conditions:

- SOFTRESET
- Detection of rising edge
- Detection of falling edge
- Device reset and reenale of HRCAP clock

When a rising edge is detected, the value in HCCOUNTER is captured into the 16-bit HCCAPCNTRISE0 register before the counter resets to 0. When a falling edge is detected, the value in HCCOUNTER is captured into the 16-bit HCCAPCNTFALL0 register before the counter resets to 0. Because the HCCOUNTER starts counting at 0 after an edge is detected, the actual low and high pulse widths (non-high-resolution) are HCCAPCNTFALL0 + 1 and HCCAPCNTRISE0 + 1, respectively, where the “+1” is added to account for the “0” HCCAPCLK cycle. This behavior is illustrated for high pulse width capture in Figure 5-4.



**Figure 5-4. HCCOUNTER Behavior During High Pulse Width Capture**

Because the HCCOUNTER starts counting immediately after SOFTRESET, the first capture result into the capture “0” registers should be discarded. The value captured will be the number of HCCAPCLK cycles since the last SOFTRESET or the HRCAPCLKEN bit was set rather than an actual pulse width measurement.

### 5.3.2.2 HCCAP0 - HCCAP1 Registers

The HRCAP capture registers include a 2-deep FIFO buffer to store two pulse widths' worth of data.

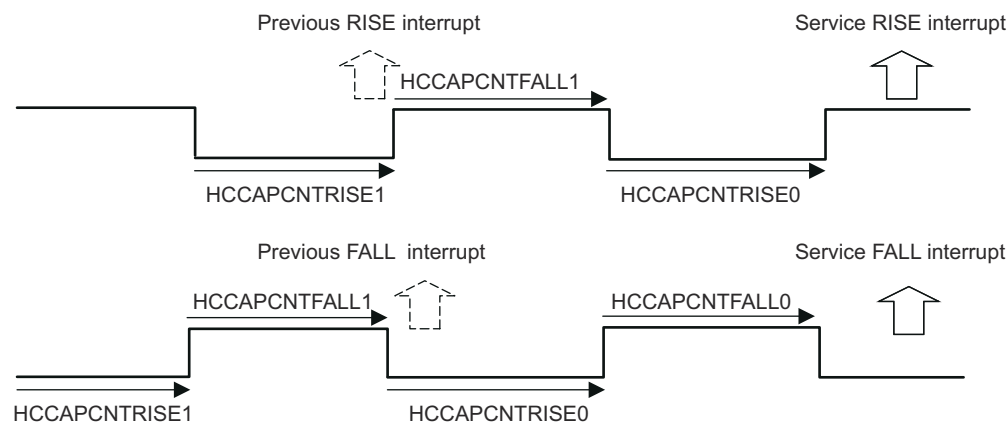
When a rising edge event occurs, HCCAPCNTRISE0 is loaded with the pulse width data from the last falling edge to the current rising edge (low pulse width). At the next rising edge event, the value in the HCCAPCNTRISE0 register is loaded into HCCAPCNTRISE1.

When a falling edge event occurs, HCCAPCNTFALL0 is loaded with the pulse width data from the last rising edge to the current falling edge (high pulse width). For falling edge events, the HRCAP logic operates such that the HCCAPCNTFALL0 register value is then loaded into the HCCAPCNTFALL1 register at the next rising edge event rather than waiting until the next falling edge event.

### 5.3.2.3 RISE versus FALL Capture Events

HRCAP capture registers can be read either during rising edge capture events or during falling edge capture events. They should not be read during both events.

There are a number of differences with regard to using RISE events to read captured registers versus using FALL events to read captured registers as shown in [Figure 5-5](#).



**Figure 5-5. Rise versus Fall Capture Events**

#### 5.3.2.3.1 RISE Capture Events

When a RISE event occurs, the application code has access to full valid capture data for two pulse widths (1 period) in high-resolution capture mode and three pulse widths (1.5 periods) in normal capture mode. HCCAPCNTFALL0 does not have valid data available on RISE events, as this values are not captured until the falling edge event after the current rising edge event (event has not yet occurred).

The application code has until the next RISE event to read all relevant capture data and clear the RISE event. Otherwise, the data will be overwritten and invalid. Therefore, RISE events are generally used to capture data for period signals where duty cycle may vary significantly.

#### Note

Because HCCOUNTER starts counting immediately after SOFTRESET, the first RISE capture result into HCCAPCNTRISE0 does not include valid pulse width data and should be discarded. When the second RISE capture event occurs, this invalid data is transferred to HCCAPCNTRISE1, and therefore the data in this register should also be discarded. After the second rise interrupt, all capture data is valid and can be used normally.

### 5.3.2.3.2 FALL Capture Events

When a FALL event occurs, the application code has access to full valid capture data for three pulse widths (1.5 periods) in high-resolution capture mode and four pulse widths (2 full periods) in normal capture mode.

The application code has only until the next RISE event to read all relevant registers and clear the FALL event. Otherwise, the data will be overwritten and invalid. Therefore FALL events are generally used to capture short pulse widths spaced far enough apart to read the registers safely.

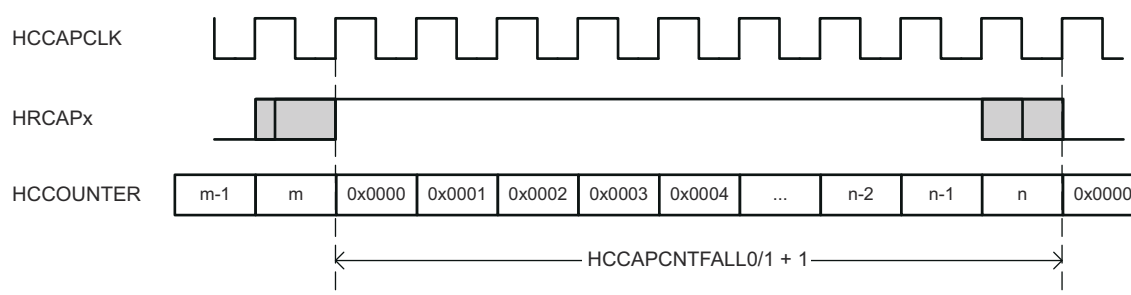
#### Note

Because HCCOUNTER starts counting immediately after SOFTRESET, the first FALL capture result into HCCAPCNTFALL0 does not include valid pulse width data and should be discarded. By the next FALL capture event, the invalid data in the “0” register has been transferred into HCCAPCNTFALL1. Therefore the data in this register should also be discarded. After the second FALL interrupt, all capture data is valid and can be used normally.

### 5.3.2.4 Normal Capture Mode

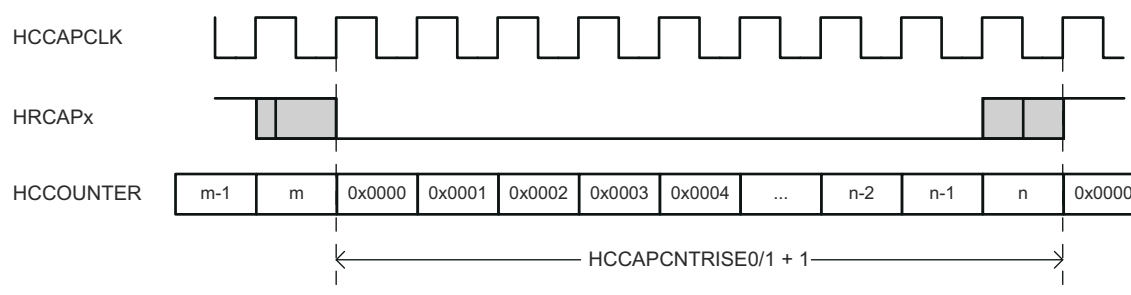
In normal capture mode, when a rise event (HCIFR[RISE]=1) or a fall event (HCIFR[FALL]=1) occurs, the application code reads the HCCAPCNTRISE0/1 and HCCAPCNTFALL0/1 registers and does not require the HCCal HRCAP calibration library. The resolution of the captured result will be accurate within  $\pm 1$  SYSCLK cycles (where SYSCLK is sourced by the same PLLCLK that generates HCCAPCLK – on this device, it is within  $\pm 1$  SYSCLK2 cycles).

High pulse widths are measured in number of HCCAPCLK cycles equal to  $1 + \text{HCCAPCNTFALL0}$  or  $1 + \text{HCCAPCNTFALL1}$  as shown in Figure 5-6.



**Figure 5-6. High Pulse Width Normal Mode Capture**

Low pulse widths are measured in number of HCCAPCLK cycles equal to  $1 + \text{HCCAPCNTRISE0}$  or  $1 + \text{HCCAPCNTRISE1}$  as shown in Figure 5-7.



**Figure 5-7. Low Pulse Width Normal Mode Capture**

In both cases, 1 is added to the value in the HCCAPCNT registers to account for the HCCAPCLK cycle in which HCCOUNTER = 0.

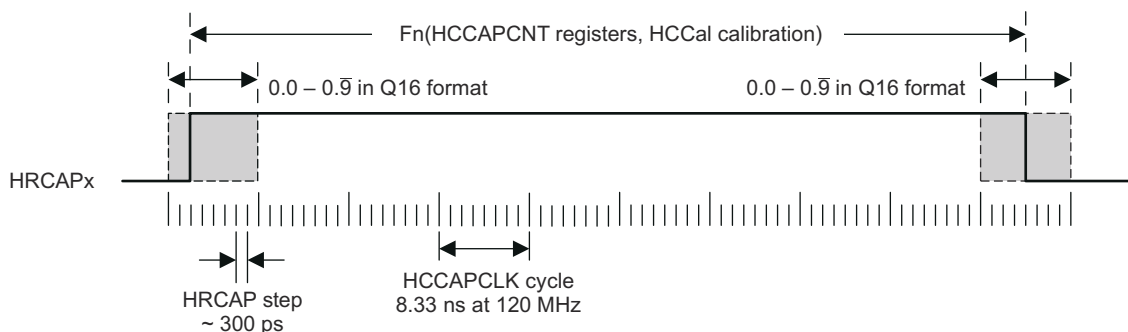
### 5.3.2.5 High Resolution Capture Mode

In high-resolution capture mode, the application code utilizes the HCCal HRCAP calibration library functions to capture the high-resolution pulse width with each edge captured at a typical resolution of +/- 300 ps (with two edges, the resolution of the measured pulse width could vary by +/- 600 ps). Note that although the HRCAP logic itself can be calibrated to capture high-resolution pulse widths, if the jitter on the input signal is greater than +/- 300 ps, the captured value will also vary according to the jitter on the input signal.

In order to use high-resolution capture mode, the high-resolution capture logic must be calibrated to scale the HRCAP step size to a Q16 fraction of the HCCAPCLK. One HRCAP module and the ePWM module internally connected to the HRCAP calibration input must be dedicated only to calibration and cannot be used functionally in the application during calibration.

Texas Instruments provides a calibration function in the HCCal HRCAP calibration library to perform this calibration once prior to using the HRCAP in high-resolution capture mode and periodically in a slow loop to account for changes in the HRCAP step size due to voltage and temperature changes while the application is running.

The library also provides functions to measure the high resolution high pulse width, low pulse width, and period. The pulse and period width measurement results are returned in Q16 fixed-point format with the fractional portion of the result representing a fraction of an HCCAPCLK cycle. (For instance a pulse width may appear in the form of 500.25 HCCAPCLK cycles). [Figure 5-8](#) shows how the high-resolution pulse width is a function of the calibration of the HRCAP step size and the values in the HCCAPCNT registers.



**Figure 5-8. HRCAP High-Resolution Mode Operating Logic**

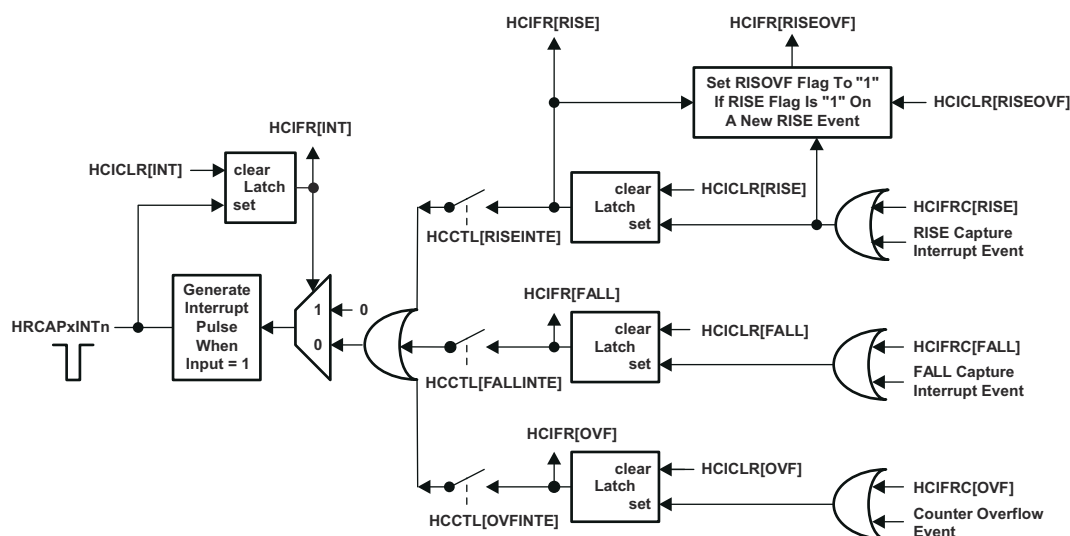
For details on using the HCCal HRCAP calibration library in high-resolution capture mode, see [Section 5.4](#).

### 5.3.3 HRCAP Interrupts

Rising edge capture (RISE), falling edge capture (FALL), and HCCOUNTER overflow (OVF) events can generate interrupts to the PIE from the HRCAP module. Additionally, if the rising edge capture flag is set (HCIFR[RISE]) when another rising edge capture event occurs, a rising edge overflow (RISEOVF) interrupt can also generate an interrupt to the PIE. The HRCAP interrupt logic is shown in Figure 5-9.

RISE/RISEOVF, FALL, and OVF events will only generate an interrupt if the corresponding interrupt enable bits in the HCCTL register are set to 1. Interrupt events can be cleared by writing a 1 to the corresponding bits in the HCICLR register. For testing purposes, interrupt events can be forced by writing a 1 to the corresponding bits in the HCIFRC register.

For proper operation, RISE and FALL interrupts should not be enabled at the same time. Capture registers should be read during rising edge interrupt events only, or during falling edge interrupt events only, and not during both interrupt events simultaneously. If RISEOVF interrupts are enabled, the RISE flag must always be acknowledged after a RISE event, otherwise a rise overflow condition will occur.



**Figure 5-9. Interrupts in HRCAP Module**

## 5.4 HRCAP Calibration Library

The HRCAP calibration (HCCal) logic is capable of capturing an edge in discrete time steps which subdivide an HCCAPCLK cycle. As previously mentioned, the size of each step is on the order of 300 ps (see device-specific data sheet for typical HRCAP step size on your device). The HRCAP\_Cal() function in the HRCAP calibration library must be run periodically to be certain that time steps are optimally applied and that the edge capture accuracy is maintained over a wide range of PWM frequencies, system clock frequencies, voltages, and temperatures. The HRCAP step size varies based on worst-case process parameters, operating temperature, and voltage. HRCAP step size increases with decreasing voltage and increasing temperature and decreases with increasing voltage and decreasing temperature.

Applications that use the HRCAP in high-resolution capture mode should use the TI-supplied HRCAP calibration (HCCal) software HRCAP\_Cal() function. The HRCAP\_Cal function helps to dynamically scale the HRCAP step size to a fraction of the HCCAPCLK cycle while the HRCAP is in high-resolution mode. To utilize the HCCal capabilities effectively during HRCAP operation, the HRCAP calibration logic uses built-in self-check and diagnostics capabilities to scale the HRCAP step size appropriately for any operating condition.

TI provides a C-callable library containing one HRCAP calibration function that utilizes this hardware and properly calibrates the internal HRCAP step logic as a fraction of a HCCAPCLK cycle. The library supplies additional high-resolution capture functions to calculate pulse widths captured in Q16 integer + fractional HCCAPCLK cycles based on the values in the HCCAPCNT registers and the calibration results.

The contents of these functions are proprietary to Texas Instruments and will not be published.

Currently, there is 1 released version of the HCCal Type 0 library, HCCal\_Type0\_V1.lib, which is located in the C2000ware software package under the \libraries\calibration\hrcap\ directory.

## 5.4.1 HRCAP Calibration Library Functions

### 5.4.1.1 HRCAP\_Cal

The HRCAP\_Cal() function runs calibration and self-check diagnostics logic on a given HRCAP module to internally scale the HCCal step logic as a fraction of a HCCAPCLK cycle.

#### Prototype:

```
Uint16 HRCAP_Cal(Uint16 HRCAPModule, Uint16 PLLClk, volatile struct EPWM_REGS *ePWMModule);
```

#### Parameters:

|             |                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|-------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| HRCAPModule | The HRCAP module number as an integer value (that is, "1" for HRCAP1 and "2" for HRCAP2). This HRCAP module will be dedicated to calibration only and cannot be used functionally to capture pulse widths.                                                                                                                                                                                                                                   |
| PLLClk      | If 0, then HCCAPCLK is clocked by SYSCLK, where SYSCLK is the system clock that clocks the HRCAP module. If 1, then HCCAPCLK is clocked by PLLCLK, where the PLLCLK frequency is a multiple of the system clock that clocks the HRCAP module.                                                                                                                                                                                                |
| ePWMModule  | A pointer to the address of the EPWM_REGS structure for the HRPWM module used to calibrate the HRCAP (that is, if HRPWM7A output is connected to the HRCAP's internal calibration logic, then &EPwm7Regs is passed into this parameter, and if HRPWM8A output is connected to the HRCAP's internal calibration logic, then &EPwm8Regs is passed into this parameter). The single ePWM module used for HRCAP calibration is device-dependent. |

#### Returns:

|   |                                                                                                                                                                                                                     |
|---|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 0 | If HCCal calibration is in progress without encountering errors.                                                                                                                                                    |
| 1 | If HCCal has exited with errors. User should check that PLL is configured such that the HCCAPCLK frequency falls within the frequency limits designated by the HRCAP <i>Electricals</i> section of the data manual. |
| 2 | If HCCal calibration has completed without errors.                                                                                                                                                                  |

#### Description:

This function drives the HRCAP calibration module logic to subdivide an HCCAPCLK cycle into HRCAP time steps equivalent to a fraction of an HCCAPCLK cycle at any given time.

HRCAP\_Cal() can only be used with HCCAPCLK between 98 MHz and 120 MHz (See your device-specific data manual's HRCAP *Electricals* section for the device-specific HCCAPCLK frequency limits).

The function can be called at any time on a single HRCAP module which is dedicated to calibration only.

The calibration HRCAP module cannot be used functionally to capture pulse widths. The calibration logic driven by this function uses a single ePWMxA HRPWM channel output connected internally to the HRCAP input to run diagnostics. During calibration, that ePWM module cannot be used for normal ePWM functions in the application. For instance, on this device, while the HRCAP is in use in high-resolution capture mode, ePWM8 cannot be used functionally by the application during calibration.



### 5.4.1.2 LowPulseWidth0

The LowPulseWidth0() function captures the high-resolution low pulse width around HCCAPCNTRISE0 in HCCAPCLK cycles.

#### Prototype:

```
Uint32 LowPulseWidth0 (Uint16 * ptrHRCAPmodule);
```

#### Parameters:

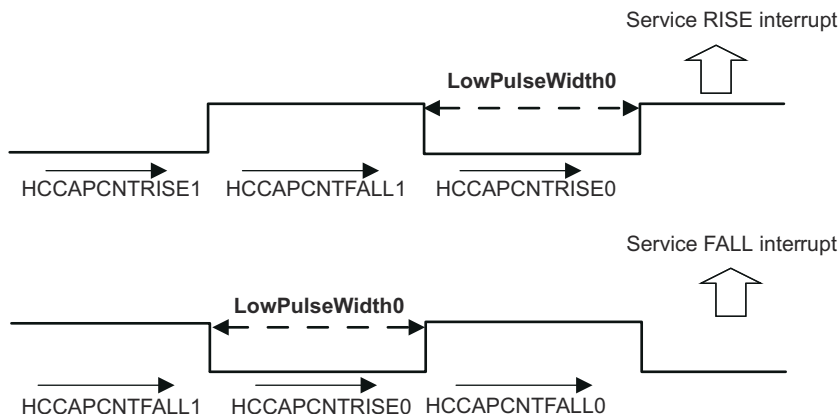
|                |                                                                                                               |
|----------------|---------------------------------------------------------------------------------------------------------------|
| ptrHRCAPmodule | A 16-bit pointer to the first address of the HRCAP register block of the HRCAP module used to capture pulses. |
|----------------|---------------------------------------------------------------------------------------------------------------|

#### Returns:

32-bit high-resolution low pulse width around HCCAPCNTRISE0 as Q16 fixed-point value in number of HCCAPCLK cycles.

#### Description:

This function can be called for any of the HRCAP modules not used for calibration to convert the HCCAPCNTRISE0 and HRCAP calibration results into a fixed-point Q16 integer + fractional high-resolution low-pulse width in HCCAPCLK cycles. [Figure 5-10](#) shows which low pulse widths can be captured on a RISE and FALL event.



**Figure 5-10. LowPulseWidth0 Capture on RISE and FALL Events**

#### 5.4.1.3 HighPulseWidth0 and HighPulseWidth1

The HighPulseWidth0() function captures the high-resolution high pulse width around HCCAPCNTFALL0 in HCCAPCLK cycles. The HighPulseWidth1() function captures the high-resolution high pulse width around HCCAPCNTFALL1 in HCCAPCLK cycles.

### Prototypes:

```

Uint32 HighPulseWidth0 (Uint16 * ptrHRCAPmodule);
Uint32 HighPulseWidth1 (Uint16 * ptrHRCAPmodule);

```

### Parameters:

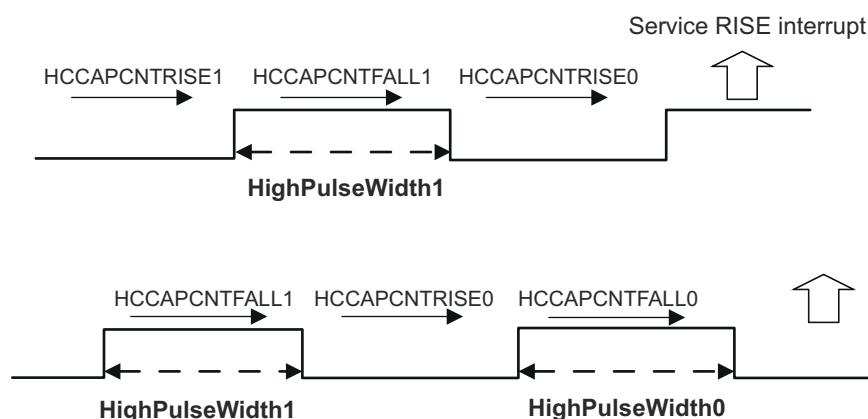
|                |                                                                                                               |
|----------------|---------------------------------------------------------------------------------------------------------------|
| ptrHRCAPmodule | A 16-bit pointer to the first address of the HRCAP register block of the HRCAP module used to capture pulses. |
|----------------|---------------------------------------------------------------------------------------------------------------|

**Returns:**

32-bit high-resolution high pulse width around HCCAPCNTFALL0/1 as Q16 fixed-point value in number of HCCAPCLK cycles.

**Description:**

These functions can be called for any of the HRCAP modules not used for calibration. They use the calibration logic to convert the HCCAPCNTFALL0/1 register value and HCCal calibration results into a fixed-point Q16 integer + fractional high-resolution high-pulse width in HCCAPCLK cycles. [Figure 5-11](#) shows which high pulse widths can be captured on a RISE and FALL event.



**Figure 5-11. HighPulseWidth0/1 Capture on RISE and FALL Events**

#### 5.4.1.4 PeriodWidthRise0

The PeriodWidthRise0() function captures the rising edge to rising edge high-resolution period width around HCCAPCNTRISE0 and HCCAPCNTFALL1 in HCCAPCLK cycles.

##### Prototype:

```
Uint32 PeriodWidthRise0 (Uint16 * ptrHRCAPmodule)
```

##### Parameters:

|                |                                                                                                               |
|----------------|---------------------------------------------------------------------------------------------------------------|
| ptrHRCAPmodule | A 16-bit pointer to the first address of the HRCAP register block of the HRCAP module used to capture pulses. |
|----------------|---------------------------------------------------------------------------------------------------------------|

##### Returns:

32-bit high-resolution rising edge to rising edge period width around HCCAPCNTRISE0+HCCAPCNTFALL1 as Q16 fixed-point value in number of HCCAPCLK cycles.

##### Description:

This function can be called for any of the HRCAP modules not used for calibration. It uses the calibration logic to convert the HCCAPCNTRISE0 and HCCAPCNTFALL1 register values and HCCal calibration results into the function into a fixed-point Q16 integer + fractional high-resolution period width in HCCAPCLK cycles. [Figure 5-12](#) shows which period widths can be captured on a RISE and FALL event.

### 5.4.1.5 PeriodWidthFall0

The PeriodWidthFall0() function captures the falling edge to falling edge high-resolution period width around HCCAPCNTFALL0 and HCCAPCNTRISE0 in HCCAPCLK cycles.

#### Prototype:

```
Uint32 PeriodWidthFall0 (Uint16 * ptrHRCAPmodule);
```

#### Parameters:

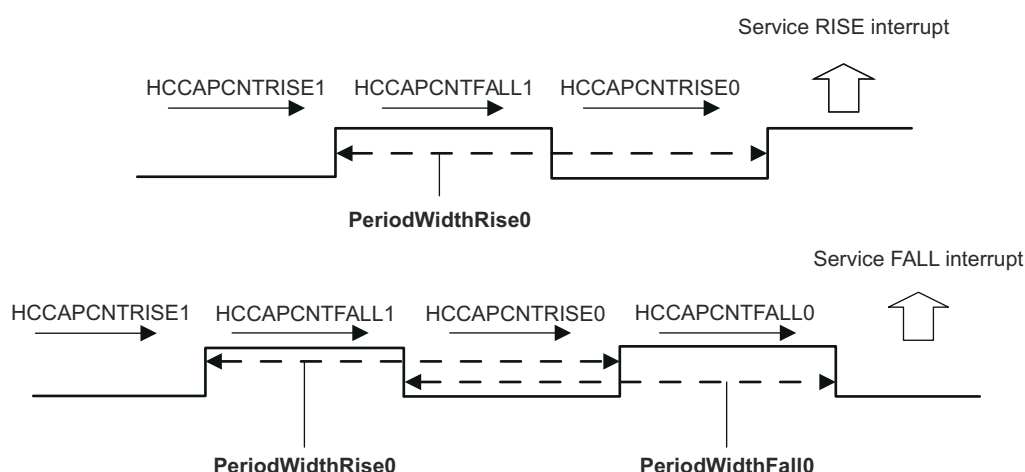
**ptrHRCAPmodule** A 16-bit pointer to the first address of the HRCAP register block of the HRCAP module used to capture pulses.

#### Returns:

32-bit high-resolution falling edge to falling edge period width around HCCAPCNTFALL0+HCCAPCNTRISE0 as Q16 fixed-point value in number of HCCAPCLK cycles.

#### Description:

This function can be called for any of the HRCAP modules not used for calibration. It uses the calibration logic to convert the HCCAPCNTFALL0 and HCCAPCNTRISE0 register values and HCCal calibration results into a fixed-point Q16 integer + fractional high-resolution period width in HCCAPCLK cycles. [Figure 5-12](#) shows which period widths can be captured on a RISE and FALL event.



**Figure 5-12. PeriodWidthRise0 and PeriodWidthFall0 Capture on RISE and FALL Events**

## 5.4.2 HRCAP Calibration Library Software Usage

To use the HRCAP in high-resolution mode, it is recommended that the HRCAP calibration functions be used as described here.

### Step 1: Add "Include" Files

The HCCal\_Type0\_V1.h file needs to be included as follows. This include file is mandatory while using the HRCAP calibration library functions. For these devices, the F2806x\_Device.h and F2806x\_Examples.h files in the F2806x C/C++ Header Files and Peripheral Examples package in C2000ware are necessary. For other device families, the device-specific equivalent files in the header files and peripheral examples software packages for those devices should be used. These include files are optional if customized header files are used in the end applications.

#### Example 1: A Sample of How to Add "Include" Files

```
#include "F2806x_Device.h"           // F2806x Headerfile
#include "F2806x_Examples.h"         // F2806x Examples Headerfile
#include "HCCal_Type0_V1.h"
```

### Step 2: HRCAP Register Array Declaration

Declare an array of pointers to HRCAP\_REGS structures which includes all available HRCAP modules on the device. Position 0 includes a 0 value which is not used by the HRCAP\_Cal function.

#### Example 2: A Sample of How to Declare an HRCAP Register Array

```
#define NUM_HRCAP 5           // # of HRCAP modules on 2806x + 1
volatile struct HRCAP_REGS *HRCAP[NUM_HRCAP] = {0, &HRCap1Regs,
    &HRCap2Regs, &HRCap3Regs, &HRCap4Regs};
```

### Step 3: HRCAP Pre-Calibration

Prior to using the HRCAP in high-resolution mode in application code, HRCAP\_Cal() should be called to calibrate the HRCAP step size subdivision into HCCAPCLK.

As part of the one-time pre-calibration prior to using the HRCAP in high-resolution mode, include the following:

#### Example 3: A Sample of HRCAP Pre-Calibration

```
while (status!= HCCAL_COMPLETE) // While calibration is incomplete
{
    // Use HRCAP2 to calibrate with:
    // HCCAPCLK = PLL2CLK
    // ePWM8A = HRCAP calibration input
    status = HRCAP_Cal(2,HCCAPCLK_PLLCLK, &EPwm8Regs);
    if (status == HCCAL_ERROR)
    {
        ESTOP0;           // Error, stop and check HCCAPCLK frequency
    }
}
```

## Step 4: Application Code Calibration

While the application is running, fluctuations in both device temperature and supply voltage may be expected. To be sure that optimal HRCAP step size is used for each HRCAP module, the HRCAP\_Cal function should be re-run periodically as part of a slower back-ground loop. Some examples of this are shown here.

### Note

**NOTE:** See the hrcap\_capture\_hrpwm example in the device-specific C/C++ header files and peripheral examples available in C2000ware from the TI website

### Example 4: HRCAP\_Cal Function Calls

```
main ()
{
    int status;
    // User code
    // HRCAP 1, 3, and 4 are running in high-resolution mode
    // The status variable returns 2 once calibration has been
    // completed by the HCCal Calibration Module running
    // diagnostics.
    status = HRCAP_Cal(2,HCCAPCLK_PLLCLK, &EPwm8Regs);
    // The function returns a 2 if HCCAPCLK is not within the
    // appropriate frequency range.
    if(status==HCCAL_ERROR) {ESTOP0;}
}
```

## Step 5: Application Code Pulse Width Measurement

While the application is running, when a RISE or FALL event occurs, pulse and period widths can be measured using the high resolution pulse width functions as shown below.

### Example 5: LowPulseWidth, HighPulseWidth, and PeriodWidth Function Calls

```
interrupt void HRCAP1_Isr (void)
{
    EALLOW;
    if (HRCap1Regs.HCIFR.bit.RISEOVF == 1) {
        ESTOP0; // Another rising edge detected
    }
    if (first < 1) {
        first++; // Discard first data (because first interrupt
                // after reset/clock enable measures time from
                // clock start to edge - invalid pulse width)
    } else {
        periodwidth = PeriodWidthRise0((Uint16 *)&HRCap1Regs);
        pulsedwidthlow = LowPulseWidth0((Uint16 *)&HRCap1Regs);
        pulsedwidthhigh = HighPulseWidth0((Uint16 *)&HRCap1Regs);
    }
    HRCap1Regs.HCICLR.bit.RISE=1;
    HRCap1Regs.HCICLR.bit.INT=1;
    PieCtrlRegs.PIEACK.bit.ACK4=1;
    EDIS;
}
```

## 5.5 HRCAP Register Descriptions

The HRCAP register set is shown in [Table 5-1](#).

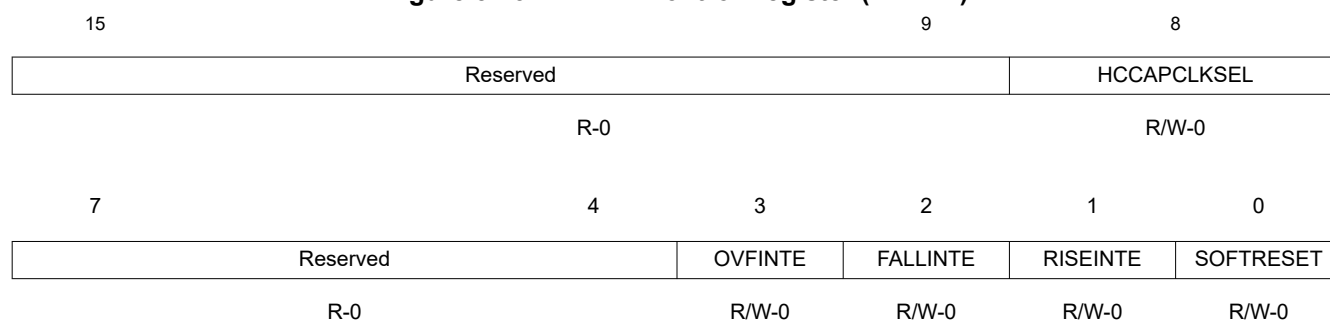
**Table 5-1. HRCAP Register Summary**

| Name          | Address Offset | Description                                      | Section                       |
|---------------|----------------|--------------------------------------------------|-------------------------------|
| HCCTL         | 0x00           | HRCAP Control Register                           | <a href="#">Section 5.5.1</a> |
| HCIFR         | 0x01           | HRCAP Interrupt Flag Register                    | <a href="#">Section 5.5.2</a> |
| HCICLR        | 0x02           | HRCAP Interrupt Clear Register                   | <a href="#">Section 5.5.3</a> |
| HCIFRC        | 0x03           | HRCAP Interrupt Force Register                   | <a href="#">Section 5.5.4</a> |
| HCCOUNTER     | 0x04           | HRCAP 16-bit Counter Register                    | <a href="#">Section 5.5.5</a> |
| HCCAPCNTRISE0 | 0x10           | HRCAP Capture Counter On Rising Edge 0 Register  | <a href="#">Section 5.5.6</a> |
| HCCAPCNTFALL0 | 0x12           | HRCAP Capture Counter On Falling Edge 0 Register | <a href="#">Section 5.5.7</a> |
| HCCAPCNTRISE1 | 0x18           | HRCAP Capture Counter On Rising Edge 1 Register  | <a href="#">Section 5.5.8</a> |
| HCCAPCNTFALL1 | 0x1A           | HRCAP Capture Counter On Falling Edge 1 Register | <a href="#">Section 5.5.9</a> |

### 5.5.1 HRCAP Control Register (HCCTL)

The HRCAP control register (HCCTL) is shown and described in the figure and table below.

**Figure 5-13. HRCAP Control Register (HCCTL)**



LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 5-2. HRCAP Control Register (HCCTL) Field Descriptions**

| Bit  | Field       | Value                                                                                                                                                          | Description <sup>(1)</sup>                                                                                                                                                                                                                                   |
|------|-------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-9 | Reserved    |                                                                                                                                                                | Reserved                                                                                                                                                                                                                                                     |
| 8    | HCCAPCLKSEL | 0 HCCAPCLK = SYSCLKOUT<br>1 HCCAPCLK = PLL2CLK                                                                                                                 | Capture clock select bit. This bit is used to select the clock source for HCCAPCLK. This bit should be set such that HCCAPCLK falls between the frequency range limits specified in the HRCAP <i>Electricals</i> section of the device-specific data manual. |
| 7-4  | Reserved    |                                                                                                                                                                | Reserved                                                                                                                                                                                                                                                     |
| 3    | OVFINTE     | 0 Disable counter overflow interrupt<br>1 Enable counter overflow interrupt                                                                                    | Counter overflow interrupt enable bit                                                                                                                                                                                                                        |
| 2    | FALLINTE    | 0 Disable falling edge capture interrupt<br>1 Enable rising edge capture interrupt                                                                             | Falling edge capture interrupt enable bit                                                                                                                                                                                                                    |
| 1    | RISEINTE    | 0 Disable rising edge capture interrupt<br>1 Enable rising edge capture interrupt                                                                              | Rising edge capture interrupt enable bit                                                                                                                                                                                                                     |
| 0    | SOFTRESET   | 0 Writes of "0" are ignored. This bit always reads "0".<br>1 Writes of "1" to this bit will clear HCCOUNTER, all capture registers, and the IFR register bits. | Soft reset                                                                                                                                                                                                                                                   |

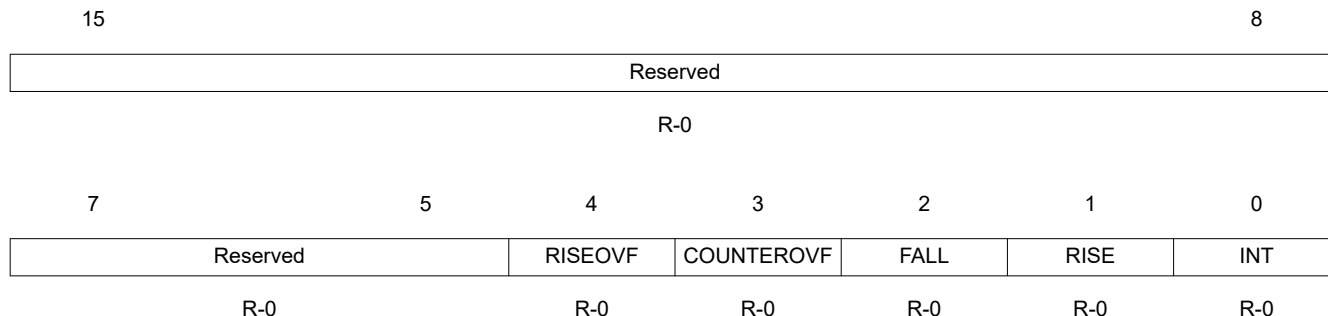
(1) This register is EALLOW protected.



## 5.5.2 HRCAP Interrupt Flag Register (HCIFR)

The HRCAP interrupt flag register (HCIFR) is shown and described in the figure and table below.

**Figure 5-14. HRCAP Interrupt Flag Register (HCIFR)**



LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 5-3. HRCAP Interrupt Flag Register (HCIFR) Field Descriptions**

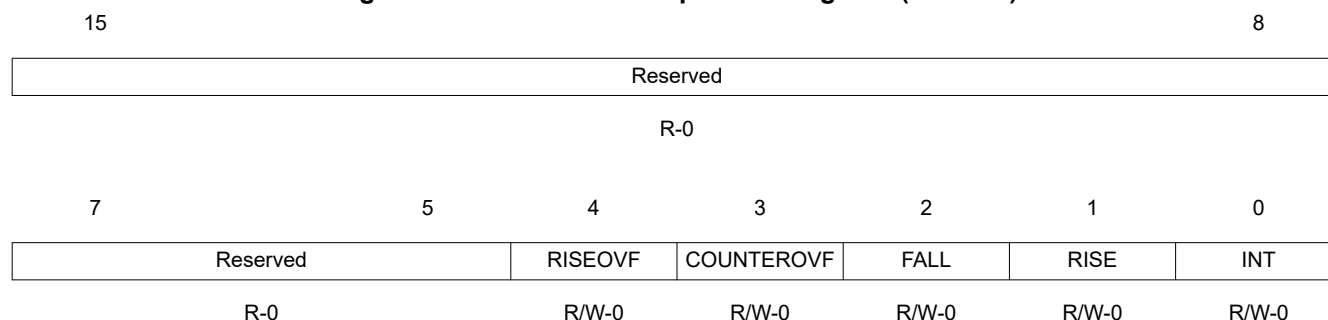
| Bit  | Field      | Value  | Description <sup>(1)</sup>                                                                                                                                                                                                                                                                                                                                                             |
|------|------------|--------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-5 | Reserved   |        | Reserved                                                                                                                                                                                                                                                                                                                                                                               |
| 4    | RISEOVF    | 0<br>1 | Rising edge interrupt overflow event flag<br>No rising edge interrupt overflow event has occurred. This bit is cleared to 0 by writing to the corresponding bit in the HCICLR register. This bit is also cleared by HCCTL[SOFTRESET].<br>This bit is set to "1" if the RISE flag is "1" when a new RISE event occurs.                                                                  |
| 3    | COUNTEROVF | 0<br>1 | Counter overflow interrupt flag<br>The HCCOUNTER has not overflowed. This bit is cleared to 0 by writing to the corresponding bit in the HCICLR register. This bit is also cleared by HCCTL[SOFTRESET].<br>This bit is set to 1 when the 16-bit HCCOUNTER overflows (from 0xFFFF to 0x0000). This bit can also be set to 1 by writing to the corresponding bit in the HCIFRC register. |
| 2    | FALL       | 0<br>1 | Falling edge capture interrupt flag:<br>No falling edge interrupt has occurred. This bit is cleared to 0 by writing to the corresponding bit in the HCICLR register. This bit is also cleared by HCCTL[SOFTRESET].<br>A falling edge input capture event has occurred. This bit can also be set to 1 by writing to the corresponding bit in the HCIFRC register.                       |
| 1    | RISE       | 0<br>1 | Rising edge capture interrupt flag<br>No rising edge interrupt has occurred. This bit is cleared to 0 by writing to the corresponding bit in the HCICLR register. This bit is also cleared by HCCTL[SOFTRESET].<br>A rising edge input capture event has occurred. This bit can also be set to 1 by writing to the corresponding bit in the HCIFRC register.                           |
| 0    | INT        | 0<br>1 | Global interrupt flag<br>No HRCAP interrupt has occurred. This bit is cleared to 0 by writing to the corresponding bit in the HCICLR register. This bit is also cleared by HCCTL[SOFTRESET].<br>An enabled RISE, FALL or COUNTEROVF interrupt has been generated. No further interrupts are generated until this bit is cleared.                                                       |

(1) This register is EALLOW protected.

### 5.5.3 HRCAP Interrupt Clear Register (HCICLR)

The HRCAP interrupt clear register (HCICLR) is shown and described in the figure and table below.

**Figure 5-15. HRCAP Interrupt Clear Register (HCICLR)**



LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 5-4. HRCAP Interrupt Clear Register (HCICLR) Field Descriptions**

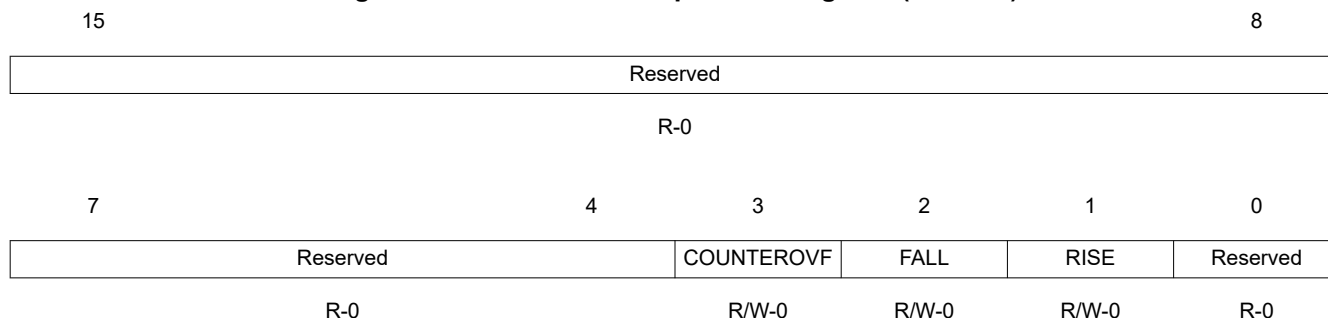
| Bit  | Field      | Value                                                                                                                                                                                                                                                                                                 | Description <sup>(1)</sup>                |
|------|------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------|
| 15-5 | Reserved   |                                                                                                                                                                                                                                                                                                       | Reserved                                  |
| 4    | RISEOVF    | 0<br>Writes of "0" are ignored. This bit always reads "0".<br>1<br>Writes of "1" to this bit will clear the corresponding RISEOVF flag bit in the HCIFR register to "0". The hardware setting of HCIFR[RISEOVF] flag bit has priority over the software clear if both happen on the same cycle.       | Rising edge interrupt overflow clear bit  |
| 3    | COUNTEROVF | 0<br>Writes of "0" are ignored. This bit always reads "0".<br>1<br>Writes of "1" to this bit will clear the corresponding COUNTEROVF flag bit in the HCIFR register to "0". The hardware setting of HCIFR[COUNTEROVF] flag bit has priority over the software clear if both happen on the same cycle. | Counter overflow interrupt clear bit      |
| 2    | FALL       | 0<br>Writes of "0" are ignored. This bit always reads "0".<br>1<br>Writes of "1" to this bit will clear the corresponding FALL flag bit in the HCIFR register to "0". The hardware setting of HCIFR[FALL] flag bit has priority over the software clear if both happen on the same cycle.             | Falling edge capture interrupt clear bit: |
| 1    | RISE       | 0<br>Writes of "0" are ignored. This bit always reads "0".<br>1<br>Writes of "1" to this bit will clear the corresponding RISE flag bit in the HCIFR register to "0". The hardware setting of HCIFR[RISE] flag bit has priority over the software clear if both happen on the same cycle.             | Rising edge capture interrupt clear bit   |
| 0    | INT        | 0<br>Writes of "0" are ignored. This bit always reads "0".<br>1<br>Writes of "1" to this bit will clear the corresponding INT flag bit in the HCIFR register to "0". The hardware setting of HCIFR[INT] flag bit has priority over the software clear if both happen on the same cycle.               | Global interrupt clear bit                |

(1) This register is EALLOW protected.

### 5.5.4 HRCAP Interrupt Force Register (HCIFRC)

The HRCAP interrupt force register (HCIFRC) is shown and described in the figure and table below.

**Figure 5-16. HRCAP Interrupt Force Register (HCIFRC)**



LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 5-5. HRCAP Interrupt Force Register (HCIFRC) Field Descriptions**

| Bit  | Field      | Value  | Description <sup>(1)</sup>                                                                                                                                                                                                                                                                                                                         |
|------|------------|--------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-4 | Reserved   |        | Reserved                                                                                                                                                                                                                                                                                                                                           |
| 3    | COUNTEROVF | 0<br>1 | Counter overflow interrupt force bits<br>Writes of "0" are ignored. This bit always reads "0".<br>Writes of "1" to this bit will force the corresponding COUNTEROVF flag bit in HCIFR register to "1". The software HCCTL[SOFTRESET] clearing of the HCIFR[COUNTEROVF] bit has priority over the hardware trying to set the bit in the same cycle. |
| 2    | FALL       | 0<br>1 | Falling edge interrupt force bits<br>Writes of "0" are ignored. This bit always reads "0".<br>Writes of "1" to this bit will force the corresponding FALL flag bit in HCIFR register to "1". The software HCCTL[SOFTRESET] clearing of the HCIFR[FALL] bit has priority over the hardware trying to set the bit in the same cycle.                 |
| 1    | RISE       | 0<br>1 | Rising edge interrupt force bits<br>Writes of "0" are ignored. This bit always reads "0".<br>Writes of "1" to this bit will force the corresponding RISE flag bit in HCIFR register to "1". The software HCCTL[SOFTRESET] clearing of the HCIFR[RISE] bit has priority over the hardware trying to set the bit in the same cycle.                  |
| 0    | Reserved   |        | Reserved                                                                                                                                                                                                                                                                                                                                           |

(1) This register is EALLOW protected.

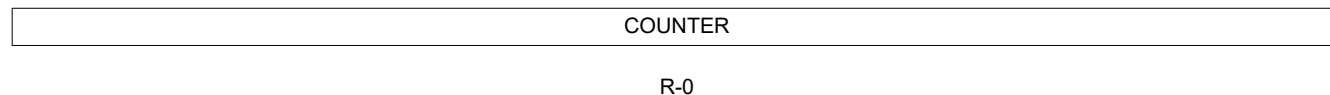
### 5.5.5 HRCAP Counter Register (HCCOUNTER)

The HRCAP counter register (HCCOUNTER) is shown and described in the figure and table below.

**Figure 5-17. HRCAP Counter Register (HCCOUNTER)**

15

0



LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 5-6. HRCAP Counter Register (HCCOUNTER) Field Descriptions**

| Bit  | Field   | Value | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
|------|---------|-------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-0 | COUNTER | 0     | <p>16-bit capture counter</p> <p>This free running counter is used to capture rising and falling edge events. The HCCOUNTER is incremented on every HCCAPCLK cycle. When the counter reaches 0xFFFF, it will overflow to 0x0000 on the next cycle and generate a COUNTEROVF interrupt event. The counter is reset to 0x0000 on every rising and falling edge event. The counter can also be reset to 0x0000 by a system reset or by setting the HCCTL[SOFTRESET] bit.</p> <p><b>NOTE:</b> Because the counter is clocked from HCCAPCLK, which can be asynchronous to SYSCLK, CPU reads to this register should not be performed unless the clocks to the HRCAP module are disabled (HRCAPxENCLK = 0).</p> |

### 5.5.6 HRCAP Capture Counter On Rising Edge 0 Register (HCCAPCNTRISE0)

The HRCAP capture counter on rising edge 0 register (HCCAPCNTRISE0) is shown and described in the figure and table below.

**Figure 5-18. HRCAP Capture Counter On Rising Edge 0 Register (HCCAPCNTRISE0)**

15

0

|               |  |
|---------------|--|
| HCCAPCNTRISE0 |  |
|---------------|--|

R-0

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 5-7. HRCAP Capture Counter On Rising Edge 0 Register (HCCAPCNTRISE0) Field Descriptions**

| Bit  | Field         | Value | Description                                                                                                                                |
|------|---------------|-------|--------------------------------------------------------------------------------------------------------------------------------------------|
| 15-0 | HCCAPCNTRISE0 | 0     | HRCAP capture counter on rising edge 0 register<br>This register captures the 16-bit HCCOUNTER value when a rising edge event is detected. |

### 5.5.7 HRCAP Capture Counter On Falling Edge 0 Register (HCCAPCNTFALL0)

The HRCAP capture counter on falling edge 0 register (HCCAPCNTFALL0) is shown and described in the figure and table below.

**Figure 5-19. HRCAP Capture Counter On Falling Edge 0 Register (HCCAPCNTFALL0)**

15

0

|               |  |
|---------------|--|
| HCCAPCNTFALL0 |  |
|---------------|--|

R-0

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 5-8. HRCAP Capture Counter On Falling Edge 0 Register (HCCAPCNTFALL0) Field Descriptions**

| Bit  | Field         | Value | Description                                                                                                                                  |
|------|---------------|-------|----------------------------------------------------------------------------------------------------------------------------------------------|
| 15-0 | HCCAPCNTFALL0 | 0     | HRCAP capture counter on falling edge 0 register<br>This register captures the 16-bit HCCOUNTER value when a Falling edge event is detected. |

### 5.5.8 HRCAP Capture Counter On Rising Edge 1 Register (HCCAPCNTRISE1)

The HRCAP capture counter on rising edge 1 register (HCCAPCNTRISE1) is shown and described in the figure and table below.

**Figure 5-20. HRCAP Capture Counter On Rising Edge 1 Register (HCCAPCNTRISE1)**

15

0

|               |  |
|---------------|--|
| HCCAPCNTRISE1 |  |
|---------------|--|

R-0

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 5-9. HRCAP Capture Counter On Rising Edge 1 Register (HCCAPCNTRISE1) Field Descriptions**

| Bit  | Field         | Value | Description                                                                                                                                                                                                                                 |
|------|---------------|-------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-0 | HCCAPCNTRISE1 | 0     | HRCAP capture counter on rising edge 1 register<br>On an input rising edge event, the value in the HCCAPCNTRISE0 register is copied into the HCCAPCNTRISE1 register before the HCCOUNTER value is captured into the HCCAPCNTRISE0 register. |

### 5.5.9 HRCAP Capture Counter On Falling Edge 1 Register (HCCAPCNTFALL1)

The HRCAP capture counter on falling edge 1 register (HCCAPCNTFALL1) is shown and described in the figure and table below.

**Figure 5-21. HRCAP Capture Counter On Falling Edge 1 Register (HCCAPCNTFALL1)**

15

0

|               |  |
|---------------|--|
| HCCAPCNTFALL1 |  |
|---------------|--|

R-0

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 5-10. HRCAP Capture Counter On Falling Edge 1 Register (HCCAPCNTFALL1) Field Descriptions**

| Bit  | Field         | Value | Description                                                                                                                                                                                                                                   |
|------|---------------|-------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-0 | HCCAPCNTFALL1 | 0     | HRCAP capture counter on falling edge 1 register<br>On an input falling edge event, the value in the HCCAPCNTFALL0 register is copied into the HCCAPCNTFALL1 register before the HCCOUNTER value is captured into the HCCAPCNTFALL0 register. |

This chapter describes the enhanced capture (eCAP) module, which is used in systems where accurate timing of external events is important.

The enhanced capture (eCAP) module is a Type 0 eCAP. See the [C2000 Real-Time Control Peripheral Reference Guide](#) for a list of all devices with an eCAP module of the same type, to determine the differences between the types, and for a list of device-specific differences within a type.

|                                                 |            |
|-------------------------------------------------|------------|
| <b>6.1 Introduction.....</b>                    | <b>430</b> |
| <b>6.2 Description.....</b>                     | <b>430</b> |
| <b>6.3 Capture and APWM Operating Mode.....</b> | <b>432</b> |
| <b>6.4 Capture Mode Description.....</b>        | <b>434</b> |
| <b>6.5 Application of the eCAP Module.....</b>  | <b>442</b> |
| <b>6.6 Application of the APWM Mode.....</b>    | <b>446</b> |
| <b>6.7 eCAP Registers.....</b>                  | <b>446</b> |

## 6.1 Introduction

### 6.1.1 Features

The features of the eCAP module include:

- Speed measurements of rotating machinery (for example, toothed sprockets sensed by way of Hall sensors)
- Elapsed time measurements between position sensor pulses
- Period and duty cycle measurements of pulse train signals
- Decoding current or voltage amplitude derived from duty cycle encoded current/voltage sensors

The eCAP module features described in this chapter include:

- 4-event time-stamp registers (each 32 bits)
- Edge polarity selection for up to four sequenced time-stamp capture events
- Interrupt on either of the four events
- Single-shot capture of up to four event time-stamps
- Continuous mode capture of time stamps in a four-deep circular buffer
- Absolute time-stamp capture
- Difference (Delta) mode time-stamp capture
- All above resources are dedicated to a single input pin
- When not used in capture mode, the eCAP module can be configured as a single-channel PWM output

### 6.1.2 ECAP Related Collateral

#### Getting Started Materials

- [Leveraging High Resolution Capture \(HRCAP\) for Single Wire Data Transfer Application Report](#)

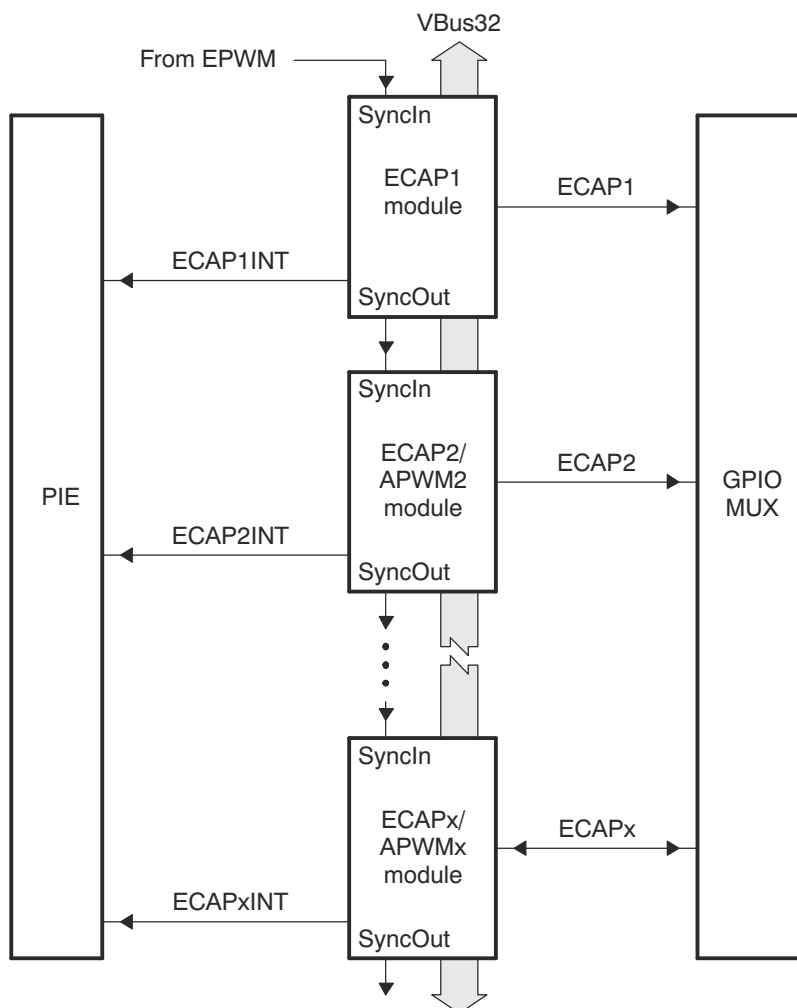
## 6.2 Description

The eCAP module represents one complete capture channel that can be instantiated multiple times, depending on the target device. In the context of this guide, one eCAP channel has the following independent key resources:

- Dedicated input capture pin
- 32-bit time base (counter)
- 4 x 32-bit time-stamp capture registers (CAP1-CAP4)
- Four-stage sequencer (modulo4 counter) that is synchronized to external events, eCAP pin rising/falling edges.
- Independent edge polarity (rising/falling edge) selection for all four events
- Input capture signal prescaling (from 2-62 or bypass)
- One-shot compare register (two bits) to freeze captures after 1-4 time-stamp events
- Control for continuous time-stamp captures using a four-deep circular buffer (CAP1-CAP4) scheme
- Interrupt capabilities on any of the four capture events

Multiple identical eCAP modules can be contained in a system as shown in [Figure 6-1](#). The number of modules is device-dependent and is based on target application needs.



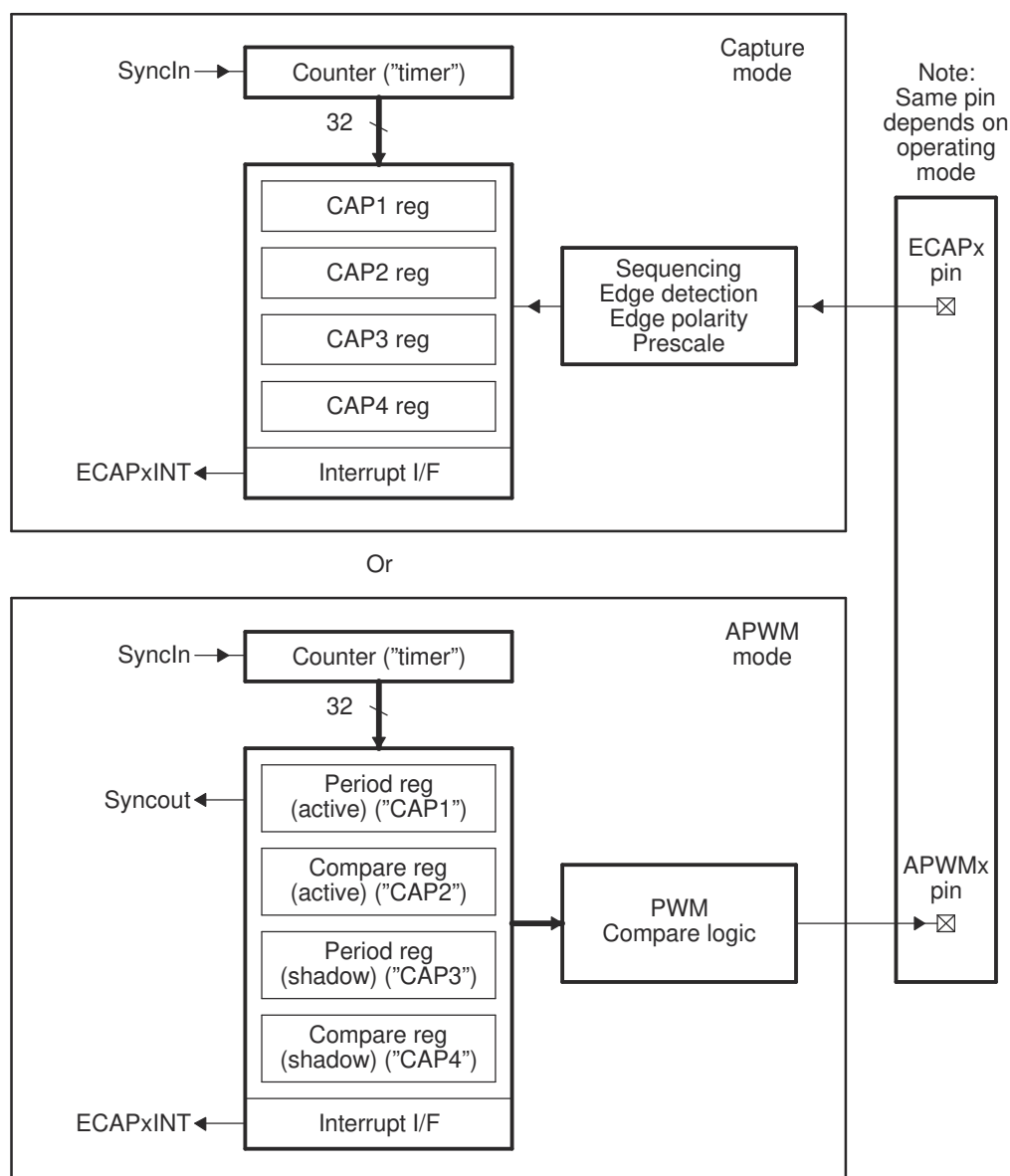


**Figure 6-1. Multiple eCAP Modules In A C28x System**

### 6.3 Capture and APWM Operating Mode

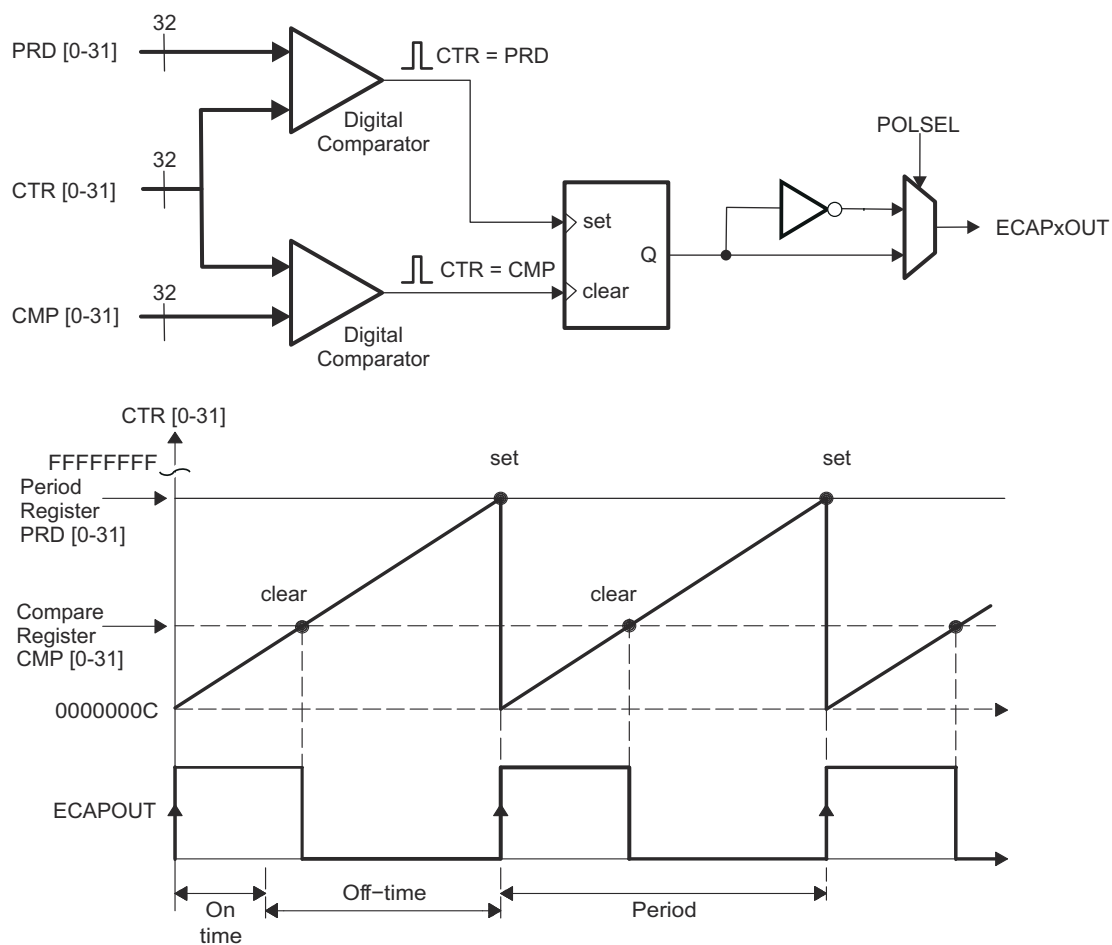
You can use the eCAP module resources to implement a single-channel PWM generator (with 32-bit capabilities) when it is not being used for input captures. The counter operates in count-up mode, providing a time-base for asymmetrical pulse width modulation (PWM) waveforms. The CAP1 and CAP2 registers become the active period and compare registers, respectively, while CAP3 and CAP4 registers become the period and compare shadow registers, respectively. Figure 6-2 is a high-level view of both the capture and auxiliary pulse-width modulator (APWM) modes of operation.

Figure 6-3 further describes the output of the eCAP in APWM mode based on the CMP and PRD values.



- A single pin is shared between CAP and APWM functions. In capture mode, it is an input; in APWM mode, it is an output.
- In APWM mode, writing any value to CAP1/CAP2 active registers also writes the same value to the corresponding shadow registers CAP3/CAP4. This emulates immediate mode. Writing to the shadow registers CAP3/CAP4 invokes the shadow mode.

**Figure 6-2. Capture and APWM Modes of Operation**



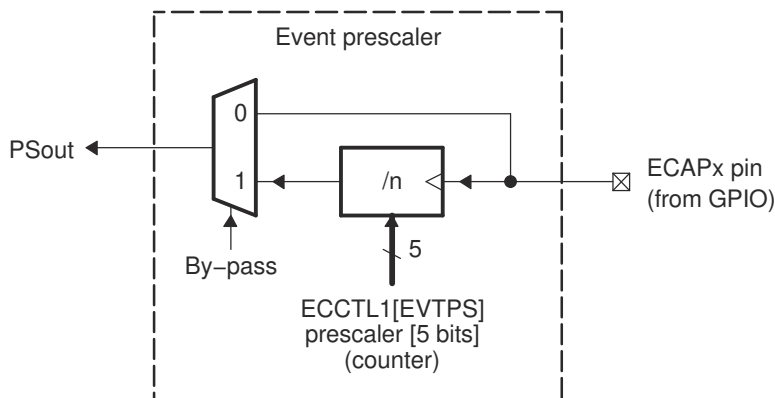
**Figure 6-3. Counter Compare and PRD Effects on the eCAP Output in APWM Mode**

Figure 6-4 shows the various components that implement the capture function.



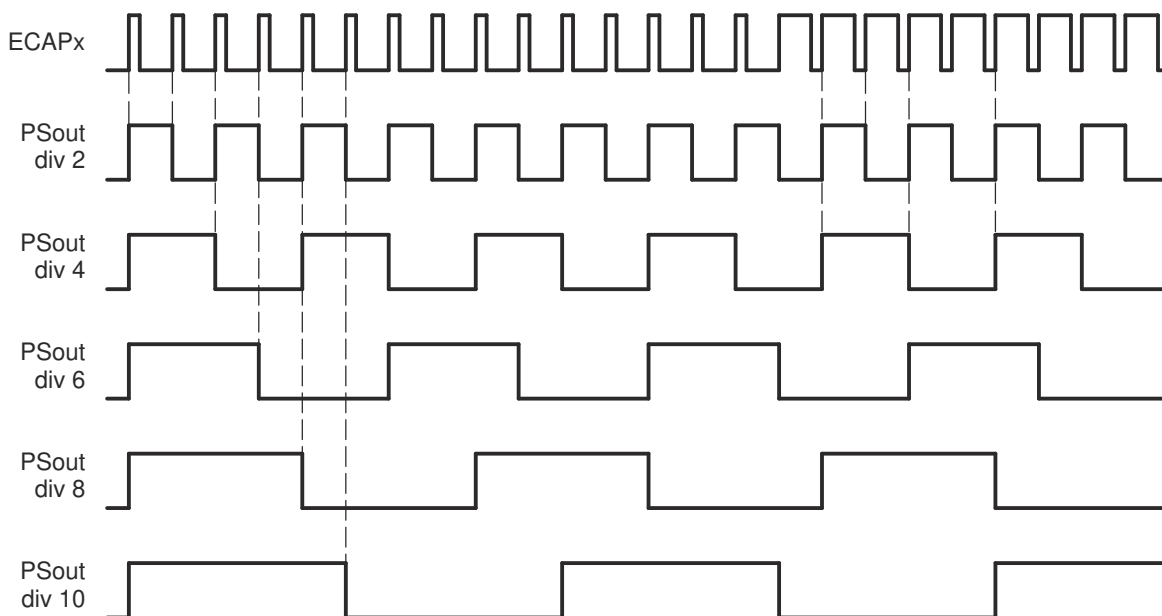
### 6.4.1 Event Prescaler

An input capture signal (pulse train) can be prescaled by  $N = 2-62$  (in multiples of 2) or can bypass the prescaler. This is useful when very high frequency signals are used as inputs. [Figure 6-5](#) shows a functional diagram and [Figure 6-6](#) shows the operation of the prescale function.



- When a prescale value of 1 is chosen (ECCTL1[13:9] = 0,0,0,0,0), the input capture signal bypasses the prescale logic completely.
- The first Rise edge after Prescale configuration change is not passed to Capture logic, prescaler value takes into effect on the second rising edge after the configuration.

**Figure 6-5. Event Prescale Control**



**Figure 6-6. Prescale Function Waveforms**

## 6.4.2 Edge Polarity Select and Qualifier

Functionality and features include:

- Four independent edge polarity (rising edge/falling edge) selection muxes are used, one for each capture event.
- Each edge (up to 4) is event qualified by the Modulo4 sequencer.
- The edge event is gated to its respective CAPx register by the Mod4 counter. The CAPx register is loaded on the falling edge.

## 6.4.3 Continuous/One-Shot Control

Operation of eCAP in Continuous/One-Shot mode:

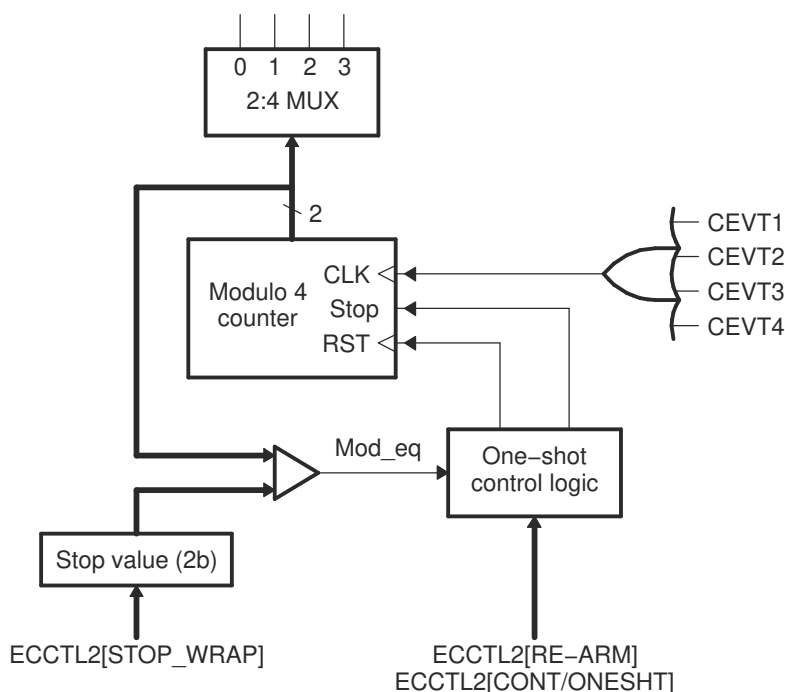
- The Mod4 (2-bit) counter is incremented via edge qualified events (CEVT1-CEVT4).
- The Mod4 counter continues counting (0->1->2->3->0) and wraps around unless stopped.
- During one-shot operation, a 2-bit stop register (STOP\_WRAP) is used to compare the Mod4 counter output, and when equal, stops the Mod4 counter and inhibits further loads of the CAP1-CAP4 registers. In this mode if TSCCTR counter is configured to reset on capture event (CEVTx) by configuring ECCTL1.CTRRSTx bit, it will still keep resetting the TSCCTR counter on capture event (CEVTx) after the STOP\_WRAP value is reached and re-arm (REARM) has not occurred.

The continuous/one-shot block controls the start, stop and reset (zero) functions of the Mod4 counter, via a mono-shot type of action that can be triggered by the stop-value comparator and re-armed via software control.

Once armed, the eCAP module waits for 1-4 (defined by stop-value) capture events before freezing both the Mod4 counter and contents of CAP1-4 registers (time stamps).

Re-arming prepares the eCAP module for another capture sequence. Also, re-arming clears (to zero) the Mod4 counter and permits loading of CAP1-4 registers again, providing the CAPLDEN bit is set.

In continuous mode, the Mod4 counter continues to run (0->1->2->3->0, the one-shot action is ignored, and capture values continue to be written to CAP1-4 in a circular buffer sequence.



**Figure 6-7. Details of the Continuous/One-shot Block**

#### 6.4.4 32-Bit Counter and Phase Control

This counter provides the time-base for event captures, and is clocked via the system clock.

A phase register is provided to achieve synchronization with other counters, via a hardware and software forced sync. This is useful in APWM mode when a phase offset between modules is needed.

On any of the four event loads, an option to reset the 32-bit counter is given. This is useful for time difference capture. The 32-bit counter value is captured first, then it is reset to 0 by any of the LD1-LD4 signals.

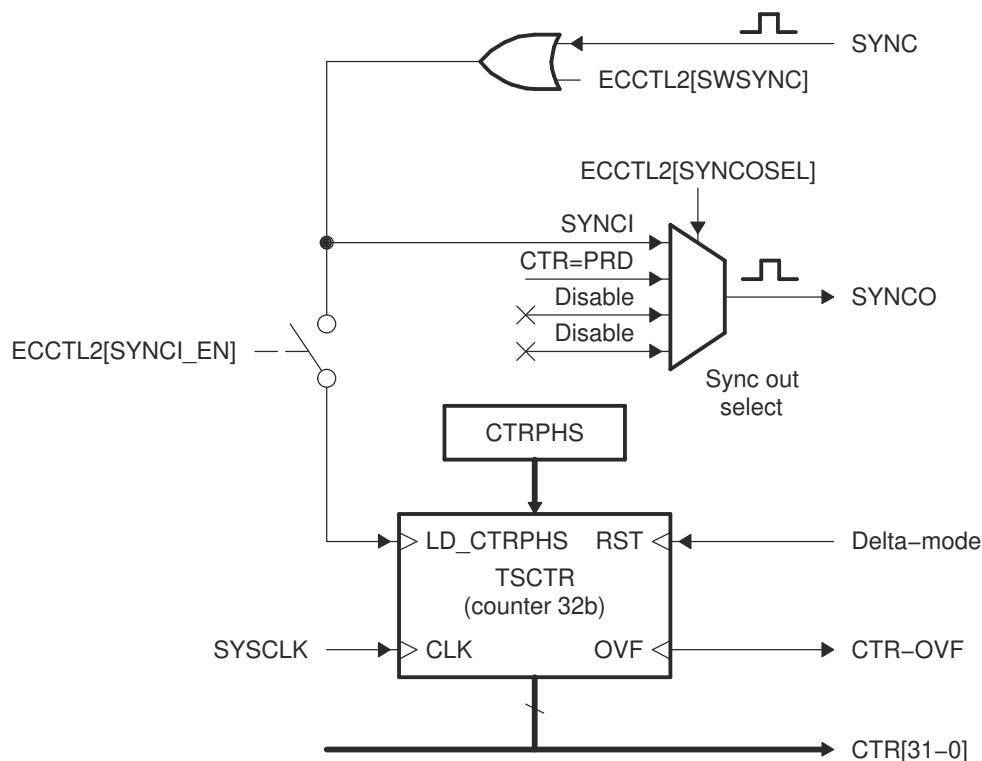


Figure 6-8. Details of the Counter and Synchronization Block

#### 6.4.5 CAP1-CAP4 Registers

These 32-bit registers are fed by the 32-bit counter timer bus, CTR[0-31] and are loaded (capture a time-stamp) when their respective LD inputs are strobed.

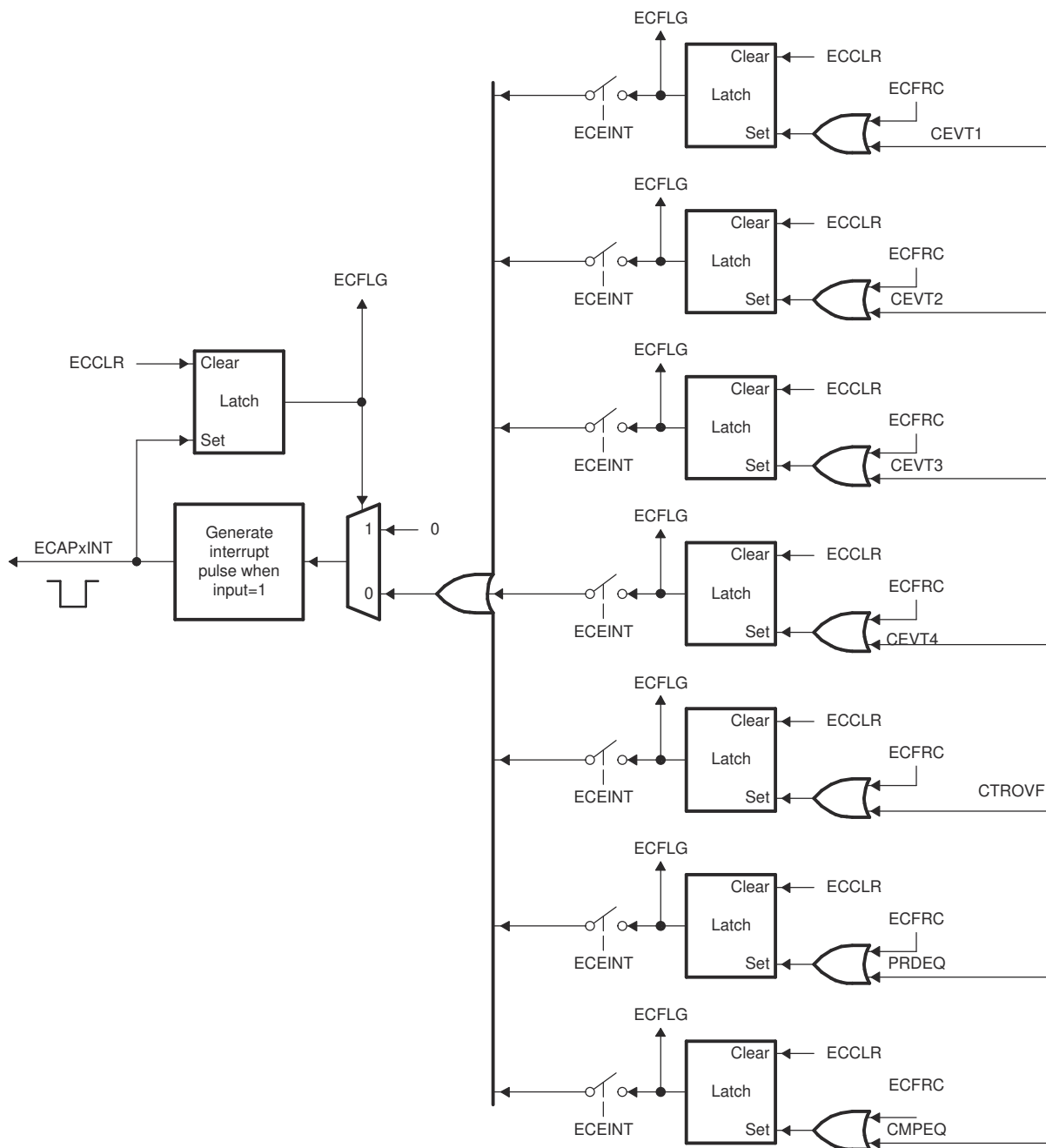
Control bit CAPLDEN can inhibit loading of the capture registers. During one-shot operation, this bit is cleared (loading is inhibited) automatically when a stop condition occurs, StopValue = Mod4.

CAP1 and CAP2 registers become the active period and compare registers, respectively, in APWM mode.

CAP3 and CAP4 registers become the respective shadow registers (APRD and ACMP) for CAP1 and CAP2 during APWM operation.







**Figure 6-9. Interrupts in eCAP Module**

#### 6.4.7 DMA Interrupt

On Type 0 eCAP modules, the CPU was required to begin data transfers using DMA. New to the Type 1 eCAP, a separate DMA Trigger (ECAP\_DMA\_INT) enables continuous transfer of capture data from eCAP registers to on-chip memory using DMA. Any one of the four available interrupt events ( ) can be selected as the trigger source for ECAP\_DMA\_INT using ECCTL2 [DMAEVTSEL].

### 6.4.8 Shadow Load and Lockout Control

In capture mode, this logic inhibits (locks out) any shadow loading of CAP1 or CAP2 from APRD and ACMP registers, respectively.

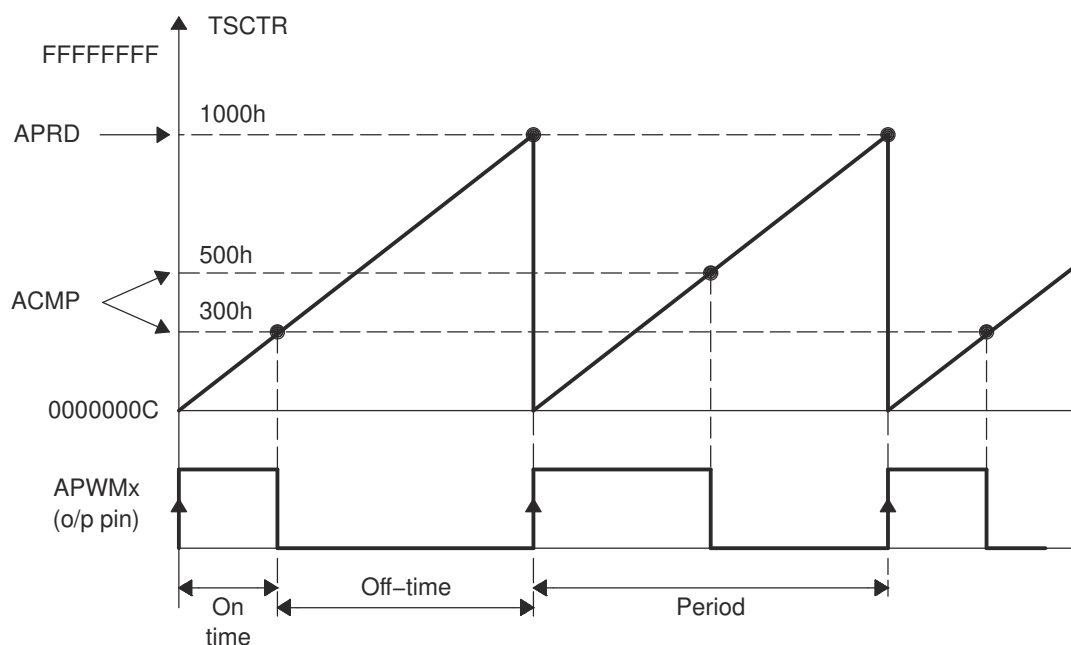
In APWM mode, shadow loading is active and two choices are permitted:

- Immediate - APRD or ACMP are transferred to CAP1 or CAP2 immediately upon writing a new value.
- On period equal,  $CTR[31:0] = PRD[31:0]$ .

### 6.4.9 APWM Mode Operation

Main operating highlights of the APWM section:

- The time-stamp counter bus is made available for comparison by way of 2 digital (32-bit) comparators.
- When CAP1/2 registers are not used in capture mode, their contents can be used as Period and Compare values in APWM mode.
- Double buffering is achieved via shadow registers APRD and ACMP (CAP3/4). The shadow register contents are transferred over to CAP1/2 registers, either immediately upon a write, or on a  $CTR = PRD$  trigger.
- In APWM mode, writing to CAP1/CAP2 active registers will also write the same value to the corresponding shadow registers CAP3/CAP4. This emulates immediate mode. Writing to the shadow registers CAP3/CAP4 will invoke the shadow mode.
- During initialization, you must write to the active registers for both period and compare. This automatically copies the initial values into the shadow values. For subsequent compare updates, during run-time, you only need to use the shadow registers.



**Figure 6-10. PWM Waveform Details Of APWM Mode Operation**

The behavior of APWM active high mode (APWMPOL == 0) is as follows:

CMP = 0x00000000, output low for duration of period (0% duty)

CMP = 0x00000001, output high 1 cycle

CMP = 0x00000002, output high 2 cycles

CMP = PERIOD, output high except for 1 cycle (<100% duty)

CMP = PERIOD+1, output high for complete period (100% duty)

CMP > PERIOD+1, output high for complete period

The behavior of APWM active low mode (APWMPOL == 1) is as follows:

CMP = 0x00000000, output high for duration of period (0% duty)

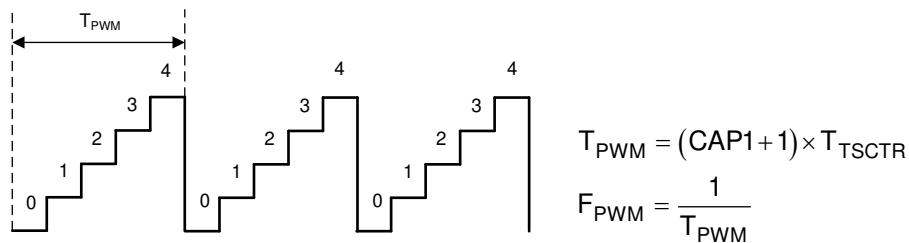
CMP = 0x00000001, output low 1 cycle

CMP = 0x00000002, output low 2 cycles

CMP = PERIOD, output low except for 1 cycle (<100% duty)

CMP = PERIOD+1, output low for complete period (100% duty)

CMP > PERIOD+1, output low for complete period



**Figure 6-11. Time-Base Frequency and Period Calculation**

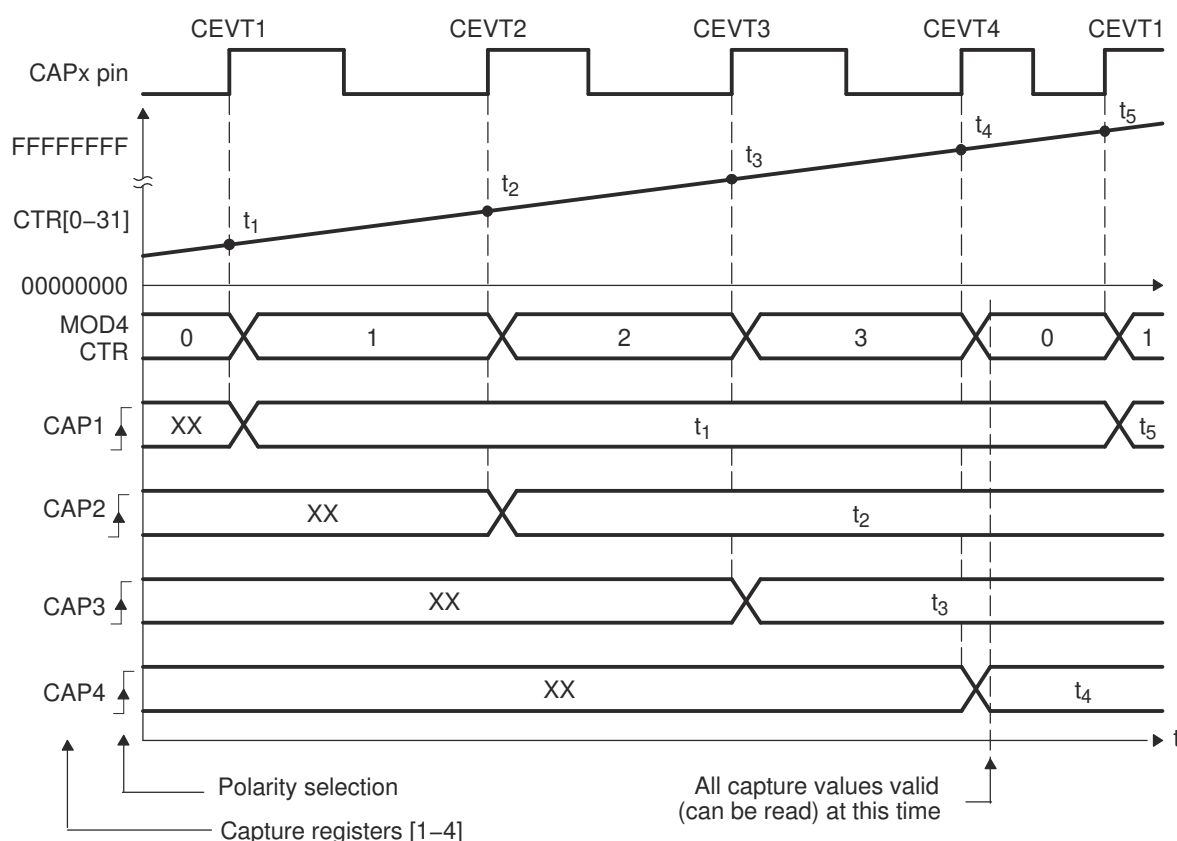
## 6.5 Application of the eCAP Module

The following sections will provide applications examples to show how to operate the eCAP module.

### 6.5.1 Example 1 - Absolute Time-Stamp Operation Rising Edge Trigger

Figure 6-12 shows an example of continuous capture operation (Mod4 counter wraps around). In this figure, TSCTR counts-up without resetting and capture events are qualified on the rising edge only, this gives period (and frequency) information.

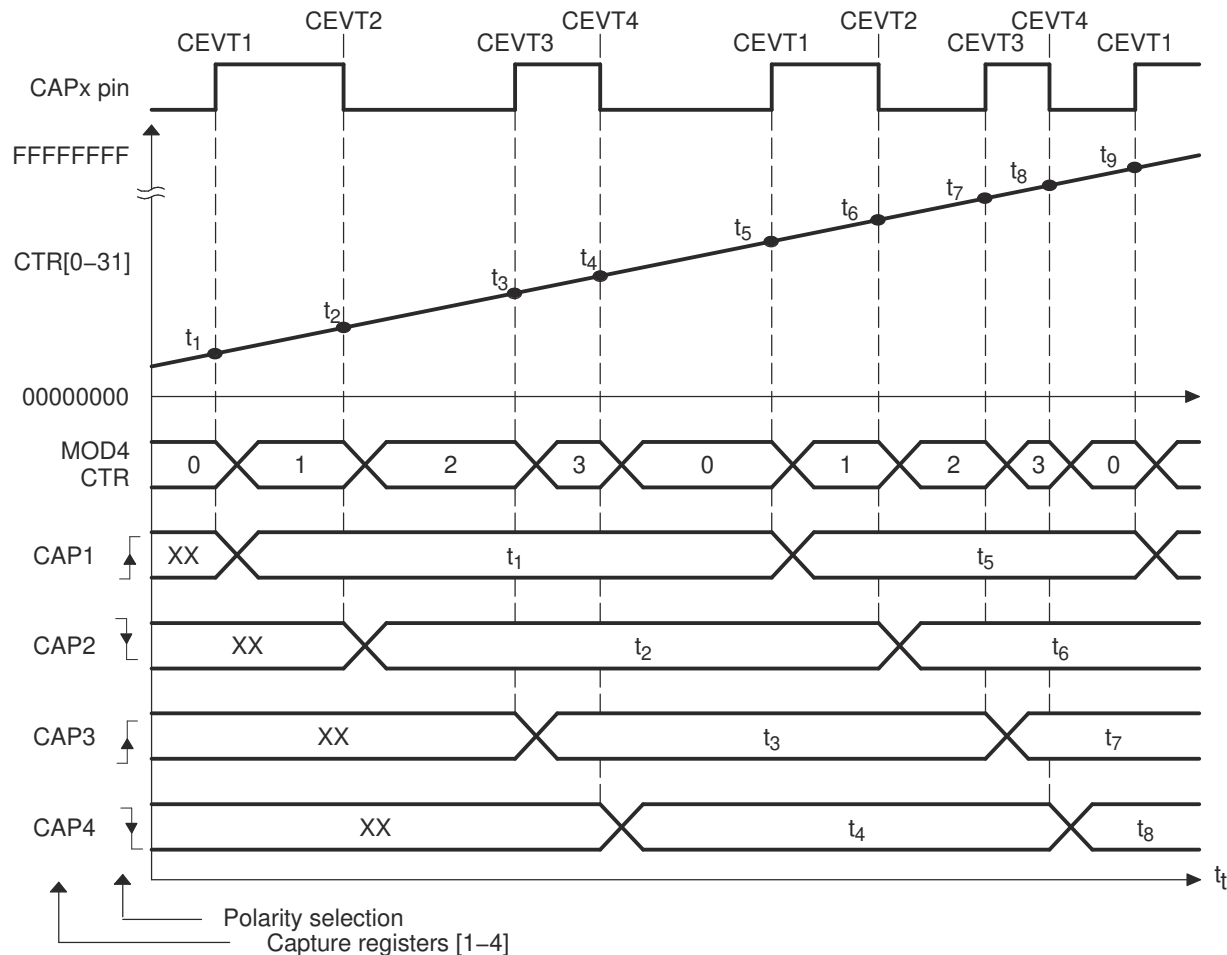
On an event, the TSCTR contents (time-stamp) is first captured, then Mod4 counter is incremented to the next state. When the TSCTR reaches FFFFFFFF (maximum value), it wraps around to 00000000 (not shown in Figure 6-12), if this occurs, the CTROVF (counter overflow) flag is set, and an interrupt (if enabled) occurs. Captured Time-stamps are valid at the point indicated by the diagram (after the 4th event), hence event CEVT4 can conveniently be used to trigger an interrupt and the CPU can read data from the CAPx registers.



**Figure 6-12. Capture Sequence for Absolute Time-stamp and Rising Edge Detect**

### 6.5.2 Example 2 - Absolute Time-Stamp Operation Rising and Falling Edge Trigger

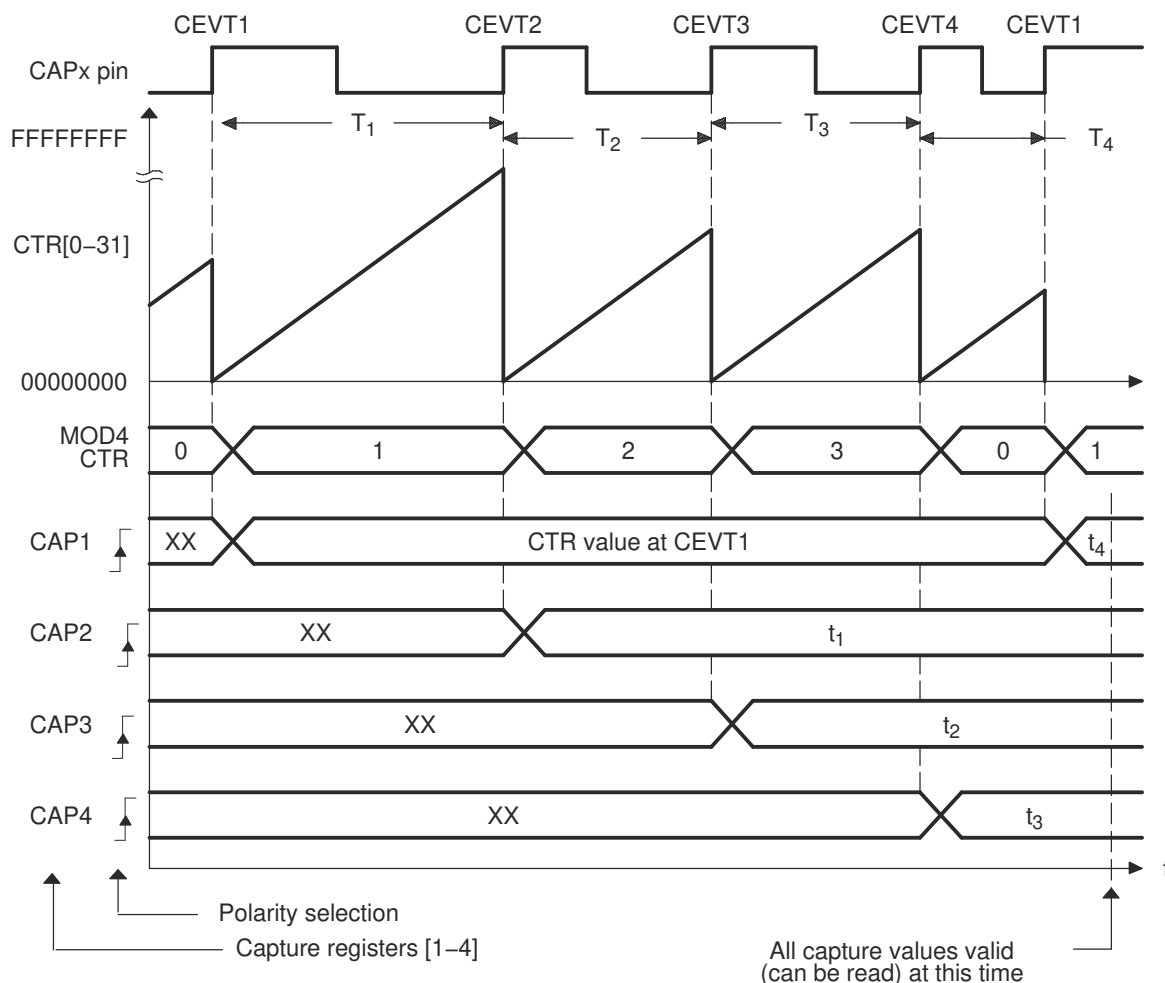
In Figure 6-13, the eCAP operating mode is almost the same as in the previous section except capture events are qualified as either rising or falling edge, this now gives both period and duty cycle information, that is: Period1 =  $t_3 - t_1$ , Period2 =  $t_5 - t_3$ , ...and so on. Duty Cycle1 (on-time %) =  $(t_2 - t_1) / \text{Period1} \times 100\%$ , and so on. Duty Cycle1 (off-time %) =  $(t_3 - t_2) / \text{Period1} \times 100\%$ , and so on.



**Figure 6-13. Capture Sequence for Absolute Time-stamp With Rising and Falling Edge Detect**

### 6.5.3 Example 3 - Time Difference (Delta) Operation Rising Edge Trigger

Figure 6-14 shows how the eCAP module can be used to collect Delta timing data from pulse train waveforms. Here Continuous Capture mode (TSCTR counts-up without resetting, and Mod4 counter wraps around) is used. In Delta-time mode, TSCTR is Reset back to Zero on every valid event. Here Capture events are qualified as Rising edge only. On an event, TSCTR contents (Time-Stamp) is captured first, and then TSCTR is reset to Zero. The Mod4 counter then increments to the next state. If TSCTR reaches FFFFFFFF (Max value), before the next event, it wraps around to 00000000 and continues, a CNTOVF (counter overflow) Flag is set, and an Interrupt (if enabled) occurs. The advantage of Delta-time Mode is that the CAPx contents directly give timing data without the need for CPU calculations, that is, Period1 =  $T_1$ , Period2 =  $T_2$ , and so on. As shown in Figure 6-14, the CEVT1 event is a good trigger point to read the timing data,  $T_1$ ,  $T_2$ ,  $T_3$ ,  $T_4$  are all valid here.

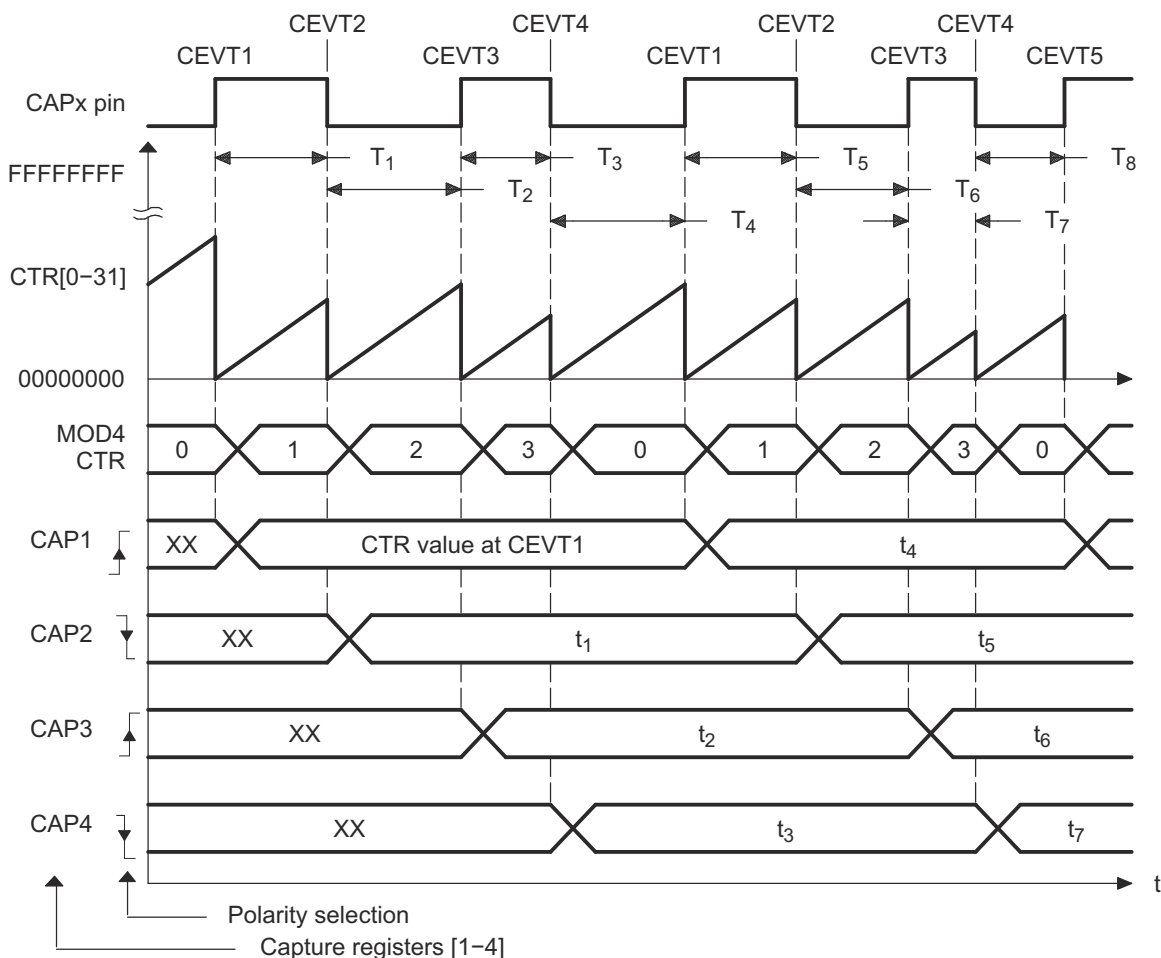


**Figure 6-14. Capture Sequence for Delta Mode Time-stamp and Rising Edge Detect**

#### 6.5.4 Example 4 - Time Difference (Delta) Operation Rising and Falling Edge Trigger

In Figure 6-15, the eCAP operating mode is almost the same as in previous section except Capture events are qualified as either Rising or Falling edge, this now gives both Period and Duty cycle information, that is: Period1 =  $T_1 + T_2$ , Period2 =  $T_3 + T_4$ , and so on. Duty Cycle1 (on-time %) =  $T_1 / \text{Period1} \times 100\%$ , Duty Cycle1 (off-time %) =  $T_2 / \text{Period1} \times 100\%$ , and so on.

During initialization, you must write to the active registers for both period and compare. This action will automatically copy the init values into the shadow values. For subsequent compare updates during run-time, the shadow registers must be used.

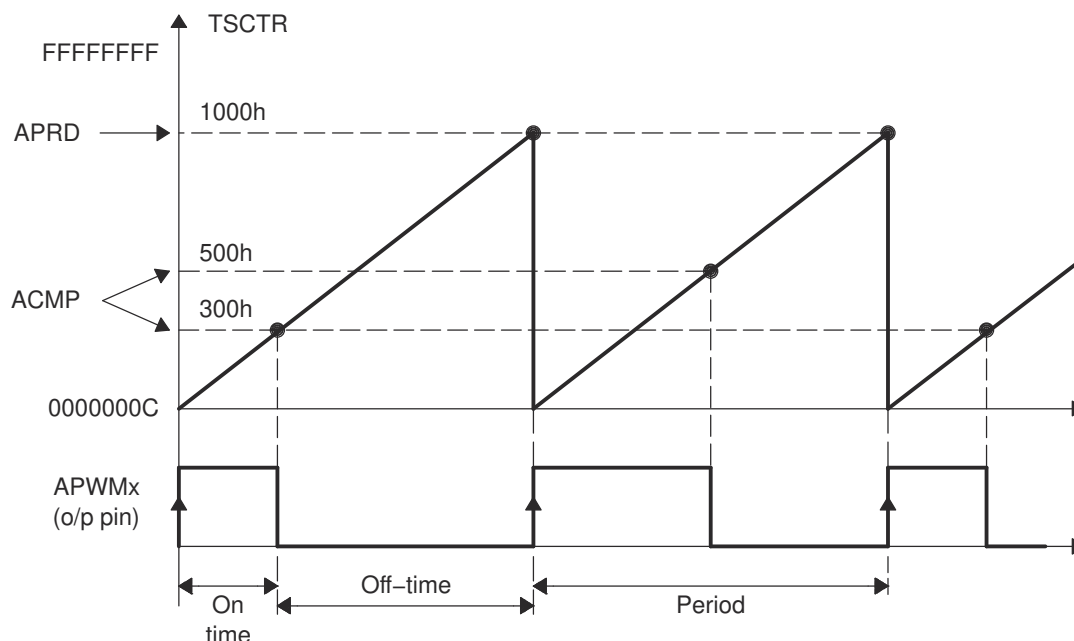


**Figure 6-15. Capture Sequence for Delta Mode Time-stamp With Rising and Falling Edge Detect**

## 6.6 Application of the APWM Mode

In this example, the eCAP module is configured to operate as a PWM generator. Here, a very simple single-channel PWM waveform is generated from the APWMx output pin. The PWM polarity is active high, which means that the compare value (CAP2 reg is now a compare register) represents the on-time (high level) of the period. Alternatively, if the APWMPOL bit is configured for active low, then the compare value represents the off-time.

### 6.6.1 Example 1 - Simple PWM Generation (Independent Channel/s)



**Figure 6-16. PWM Waveform Details of APWM Mode Operation**

## 6.7 eCAP Registers

This section describes the Enhanced Capture Registers.

### 6.7.1 ECAP Base Address Table

**Table 6-1. ECAP Base Address Table**

| Bit Field Name |           | Base Address |
|----------------|-----------|--------------|
| Instance       | Structure |              |
| ECap1Regs      | ECAP_REGS | 0x0000_6A00  |
| ECap2Regs      | ECAP_REGS | 0x0000_6A20  |
| ECap3Regs      | ECAP_REGS | 0x0000_6A40  |



## 6.7.2 ECAP\_REGS Registers

Table 6-2 lists the ECAP\_REGS registers. All register offset addresses not listed in Table 6-2 should be considered as reserved locations and the register contents should not be modified.

**Table 6-2. ECAP\_REGS Registers**

| Offset | Acronym | Register Name                       | Write Protection | Section            |
|--------|---------|-------------------------------------|------------------|--------------------|
| 0h     | TSCTR   | Time-Stamp Counter                  |                  | <a href="#">Go</a> |
| 2h     | CTRPHS  | Counter Phase Offset Value Register |                  | <a href="#">Go</a> |
| 4h     | CAP1    | Capture 1 Register                  |                  | <a href="#">Go</a> |
| 6h     | CAP2    | Capture 2 Register                  |                  | <a href="#">Go</a> |
| 8h     | CAP3    | Capture 3 Register                  |                  | <a href="#">Go</a> |
| Ah     | CAP4    | Capture 4 Register                  |                  | <a href="#">Go</a> |
| 14h    | ECCTL1  | Capture Control Register 1          |                  | <a href="#">Go</a> |
| 15h    | ECCTL2  | Capture Control Register 2          |                  | <a href="#">Go</a> |
| 16h    | ECEINT  | Capture Interrupt Enable Register   |                  | <a href="#">Go</a> |
| 17h    | ECFLG   | Capture Interrupt Flag Register     |                  | <a href="#">Go</a> |
| 18h    | ECCLR   | Capture Interrupt Clear Register    |                  | <a href="#">Go</a> |
| 19h    | ECFRC   | Capture Interrupt Force Register    |                  | <a href="#">Go</a> |

Complex bit access types are encoded to fit into small table cells. Table 6-3 shows the codes that are used for access types in this section.

**Table 6-3. ECAP\_REGS Access Type Codes**

| Access Type              | Code    | Description                                                                                                                                                                                                                                                                          |
|--------------------------|---------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Read Type                |         |                                                                                                                                                                                                                                                                                      |
| R                        | R       | Read                                                                                                                                                                                                                                                                                 |
| R-0                      | R<br>-0 | Read<br>Returns 0s                                                                                                                                                                                                                                                                   |
| Write Type               |         |                                                                                                                                                                                                                                                                                      |
| W                        | W       | Write                                                                                                                                                                                                                                                                                |
| W1C                      | W<br>1C | Write<br>1 to clear                                                                                                                                                                                                                                                                  |
| W1S                      | W<br>1S | Write<br>1 to set                                                                                                                                                                                                                                                                    |
| Reset or Default Value   |         |                                                                                                                                                                                                                                                                                      |
| -n                       |         | Value after reset or the default value                                                                                                                                                                                                                                               |
| Register Array Variables |         |                                                                                                                                                                                                                                                                                      |
| i,j,k,l,m,n              |         | When these variables are used in a register name, an offset, or an address, they refer to the value of a register array where the register is part of a group of repeating registers. The register groups form a hierarchical structure and the array is represented with a formula. |
| y                        |         | When this variable is used in a register name, an offset, or an address it refers to the value of a register array.                                                                                                                                                                  |

### 6.7.2.1 TSCTR Register (Offset = 0h) [reset = 0h]

TSCTR is shown in [Figure 6-17](#) and described in [Table 6-4](#).

Return to the [Summary Table](#).

Time-Stamp Counter

**Figure 6-17. TSCTR Register**

|        |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |   |   |   |   |   |   |   |   |   |   |
|--------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|---|---|---|---|---|---|---|---|---|---|
| 31     | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 | 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| TSCTR  |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |   |   |   |   |   |   |   |   |   |   |
| R/W-0h |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |   |   |   |   |   |   |   |   |   |   |

**Table 6-4. TSCTR Register Field Descriptions**

| Bit  | Field | Type | Reset | Description                                                                                |
|------|-------|------|-------|--------------------------------------------------------------------------------------------|
| 31-0 | TSCTR | R/W  | 0h    | Active 32-bit counter register that is used as the capture time-base<br>Reset type: SYSRSn |

### 6.7.2.2 CTRPHS Register (Offset = 2h) [reset = 0h]

CTRPHS is shown in [Figure 6-18](#) and described in [Table 6-5](#).

Return to the [Summary Table](#).

Counter Phase Offset Value Register

**Figure 6-18. CTRPHS Register**

|        |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |   |   |   |   |   |   |   |   |   |   |
|--------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|---|---|---|---|---|---|---|---|---|---|
| 31     | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 | 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| CTRPHS |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |   |   |   |   |   |   |   |   |   |   |
| R/W-0h |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |   |   |   |   |   |   |   |   |   |   |

**Table 6-5. CTRPHS Register Field Descriptions**

| Bit  | Field  | Type | Reset | Description                                                                                                                                                                                                                                                                                    |
|------|--------|------|-------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 31-0 | CTRPHS | R/W  | 0h    | Counter phase value register that can be programmed for phase lag/lead. This register CTRPHS is loaded into TSCTR upon either a SYNCI event or S/W force via a control bit. Used to achieve phase control synchronization with respect to other eCAP and EPWM timebases.<br>Reset type: SYSRSn |

### 6.7.2.3 CAP1 Register (Offset = 4h) [reset = 0h]

CAP1 is shown in [Figure 6-19](#) and described in [Table 6-6](#).

Return to the [Summary Table](#).

Capture 1 Register

**Figure 6-19. CAP1 Register**

|        |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |   |   |   |   |   |   |   |   |   |   |
|--------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|---|---|---|---|---|---|---|---|---|---|
| 31     | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 | 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| CAP1   |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |   |   |   |   |   |   |   |   |   |   |
| R/W-0h |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |   |   |   |   |   |   |   |   |   |   |

**Table 6-6. CAP1 Register Field Descriptions**

| Bit  | Field | Type | Reset | Description                                                                                                                                                                                                                                            |
|------|-------|------|-------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 31-0 | CAP1  | R/W  | 0h    | This register can be loaded (written) by:<br>- Time-Stamp counter value (TSCTR) during a capture event<br>- Software - may be useful for test purposes or initialization<br>- ARPD shadow register (CAP3) when used in APWM mode<br>Reset type: SYSRSn |

### 6.7.2.4 CAP2 Register (Offset = 6h) [reset = 0h]

CAP2 is shown in [Figure 6-20](#) and described in [Table 6-7](#).

Return to the [Summary Table](#).

Capture 2 Register

**Figure 6-20. CAP2 Register**

|        |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |   |   |   |   |   |   |   |   |   |   |
|--------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|---|---|---|---|---|---|---|---|---|---|
| 31     | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 | 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| CAP2   |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |   |   |   |   |   |   |   |   |   |   |
| R/W-0h |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |   |   |   |   |   |   |   |   |   |   |

**Table 6-7. CAP2 Register Field Descriptions**

| Bit  | Field | Type | Reset | Description                                                                                                                                                                                                                     |
|------|-------|------|-------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 31-0 | CAP2  | R/W  | 0h    | This register can be loaded (written) by:<br>- Time-Stamp ( counter value) during a capture event<br>- Software - may be useful for test purposes<br>- ACMP shadow register (CAP4) when used in APWM mode<br>Reset type: SYSRSn |

### 6.7.2.5 CAP3 Register (Offset = 8h) [reset = 0h]

CAP3 is shown in [Figure 6-21](#) and described in [Table 6-8](#).

Return to the [Summary Table](#).

Capture 3 Register

**Figure 6-21. CAP3 Register**

|        |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |   |   |   |   |   |   |   |   |   |   |
|--------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|---|---|---|---|---|---|---|---|---|---|
| 31     | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 | 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| CAP3   |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |   |   |   |   |   |   |   |   |   |   |
| R/W-0h |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |   |   |   |   |   |   |   |   |   |   |

**Table 6-8. CAP3 Register Field Descriptions**

| Bit  | Field | Type | Reset | Description                                                                                                                                                                                                                             |
|------|-------|------|-------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 31-0 | CAP3  | R/W  | 0h    | In CMP mode, this is a time-stamp capture register.<br>In APWM mode, this is the period shadow (APRD) register. You can update the PWM period value through this register. CAP3 (APRD) shadows CAP1 in this mode.<br>Reset type: SYSRSn |

### 6.7.2.6 CAP4 Register (Offset = Ah) [reset = 0h]

CAP4 is shown in [Figure 6-22](#) and described in [Table 6-9](#).

Return to the [Summary Table](#).

Capture 4 Register

**Figure 6-22. CAP4 Register**

|        |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |   |   |   |   |   |   |   |   |   |   |
|--------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|---|---|---|---|---|---|---|---|---|---|
| 31     | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 | 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| CAP4   |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |   |   |   |   |   |   |   |   |   |   |
| R/W-0h |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |   |   |   |   |   |   |   |   |   |   |

**Table 6-9. CAP4 Register Field Descriptions**

| Bit  | Field | Type | Reset | Description                                                                                                                                                                                                                           |
|------|-------|------|-------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 31-0 | CAP4  | R/W  | 0h    | In CMP mode, this is a time-stamp capture register.<br>In APWM mode, this is the compare shadow (ACMP) register. You can update the PWM compare value via this register. CAP4 (ACMP) shadows CAP2 in this mode.<br>Reset type: SYSRSn |

### 6.7.2.7 ECCTL1 Register (Offset = 14h) [reset = 0h]

ECCTL1 is shown in [Figure 6-23](#) and described in [Table 6-10](#).

Return to the [Summary Table](#).

Capture Control Register 1

**Figure 6-23. ECCTL1 Register**

| 15        | 14      | 13       | 12      | 11      | 10      | 9       | 8       |
|-----------|---------|----------|---------|---------|---------|---------|---------|
| FREE_SOFT |         | PRESCALE |         |         |         |         | CAPLDEN |
| R/W-0h    |         | R/W-0h   |         |         |         |         | R/W-0h  |
| 7         | 6       | 5        | 4       | 3       | 2       | 1       | 0       |
| CTRRST4   | CAP4POL | CTRRST3  | CAP3POL | CTRRST2 | CAP2POL | CTRRST1 | CAP1POL |
| R/W-0h    | R/W-0h  | R/W-0h   | R/W-0h  | R/W-0h  | R/W-0h  | R/W-0h  | R/W-0h  |

**Table 6-10. ECCTL1 Register Field Descriptions**

| Bit   | Field     | Type | Reset | Description                                                                                                                                                                                                                                                                                                           |
|-------|-----------|------|-------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-14 | FREE_SOFT | R/W  | 0h    | Emulation Control<br>Reset type: SYSRSn<br>0h (R/W) = TSCTR counter stops immediately on emulation suspend<br>1h (R/W) = TSCTR counter runs until = 0<br>2h (R/W) = TSCTR counter is unaffected by emulation suspend (Run Free)<br>3h (R/W) = TSCTR counter is unaffected by emulation suspend (Run Free)             |
| 13-9  | PRESCALE  | R/W  | 0h    | Event Filter prescale select<br>Reset type: SYSRSn<br>0h (R/W) = Divide by 1 (that is, no prescale, by-pass the prescaler)<br>1h (R/W) = Divide by 2<br>2h (R/W) = Divide by 4<br>3h (R/W) = Divide by 6<br>4h (R/W) = Divide by 8<br>5h (R/W) = Divide by 10<br>1Eh (R/W) = Divide by 60<br>1Fh (R/W) = Divide by 62 |
| 8     | CAPLDEN   | R/W  | 0h    | Enable Loading of CAP1-4 registers on a capture event. Note that this bit does not disable CEVTn events from being generated.<br>Reset type: SYSRSn<br>0h (R/W) = Disable CAP1-4 register loads at capture event time.<br>1h (R/W) = Enable CAP1-4 register loads at capture event time.                              |
| 7     | CTRRST4   | R/W  | 0h    | Counter Reset on Capture Event 4<br>Reset type: SYSRSn<br>0h (R/W) = Do not reset counter on Capture Event 4 (absolute time stamp operation)<br>1h (R/W) = Reset counter after Capture Event 4 time-stamp has been captured (used in difference mode operation)                                                       |
| 6     | CAP4POL   | R/W  | 0h    | Capture Event 4 Polarity select<br>Reset type: SYSRSn<br>0h (R/W) = Capture Event 4 triggered on a rising edge (RE)<br>1h (R/W) = Capture Event 4 triggered on a falling edge (FE)                                                                                                                                    |

**Table 6-10. ECCTL1 Register Field Descriptions (continued)**

| Bit | Field   | Type | Reset | Description                                                                                                                                                                                                                                   |
|-----|---------|------|-------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 5   | CTRRST3 | R/W  | 0h    | Counter Reset on Capture Event 3<br>Reset type: SYSRSn<br>0h (R/W) = Do not reset counter on Capture Event 3 (absolute time stamp)<br>1h (R/W) = Reset counter after Event 3 time-stamp has been captured (used in difference mode operation) |
| 4   | CAP3POL | R/W  | 0h    | Capture Event 3 Polarity select<br>Reset type: SYSRSn<br>0h (R/W) = Capture Event 3 triggered on a rising edge (RE)<br>1h (R/W) = Capture Event 3 triggered on a falling edge (FE)                                                            |
| 3   | CTRRST2 | R/W  | 0h    | Counter Reset on Capture Event 2<br>Reset type: SYSRSn<br>0h (R/W) = Do not reset counter on Capture Event 2 (absolute time stamp)<br>1h (R/W) = Reset counter after Event 2 time-stamp has been captured (used in difference mode operation) |
| 2   | CAP2POL | R/W  | 0h    | Capture Event 2 Polarity select<br>Reset type: SYSRSn<br>0h (R/W) = Capture Event 2 triggered on a rising edge (RE)<br>1h (R/W) = Capture Event 2 triggered on a falling edge (FE)                                                            |
| 1   | CTRRST1 | R/W  | 0h    | Counter Reset on Capture Event 1<br>Reset type: SYSRSn<br>0h (R/W) = Do not reset counter on Capture Event 1 (absolute time stamp)<br>1h (R/W) = Reset counter after Event 1 time-stamp has been captured (used in difference mode operation) |
| 0   | CAP1POL | R/W  | 0h    | Capture Event 1 Polarity select<br>Reset type: SYSRSn<br>0h (R/W) = Capture Event 1 triggered on a rising edge (RE)<br>1h (R/W) = Capture Event 1 triggered on a falling edge (FE)                                                            |

### 6.7.2.8 ECCTL2 Register (Offset = 15h) [reset = 6h]

ECCTL2 is shown in [Figure 6-24](#) and described in [Table 6-11](#).

Return to the [Summary Table](#).

Capture Control Register 2

**Figure 6-24. ECCTL2 Register**

| 15        | 14       | 13        | 12         | 11        | 10      | 9            | 8          |
|-----------|----------|-----------|------------|-----------|---------|--------------|------------|
| RESERVED  |          |           |            |           | APWMPOL | CAP_APWM     | SWSYNC     |
| R-0h      |          |           |            |           | R/W-0h  | R/W-0h       | R-0/W1S-0h |
| 7         | 6        | 5         | 4          | 3         | 2       | 1            | 0          |
| SYNCO_SEL | SYNCL_EN | TSCTRSTOP | REARM      | STOP_WRAP |         | CONT_ONESHOT |            |
| R/W-0h    | R/W-0h   | R/W-0h    | R-0/W1S-0h | R/W-3h    |         | R/W-0h       |            |

**Table 6-11. ECCTL2 Register Field Descriptions**

| Bit   | Field    | Type    | Reset | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|-------|----------|---------|-------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-11 | RESERVED | R       | 0h    | Reserved                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| 10    | APWMPOL  | R/W     | 0h    | APWM output polarity select. This is applicable only in APWM operating mode.<br>Reset type: SYSRSn<br>0h (R/W) = Output is active high (Compare value defines high time)<br>1h (R/W) = Output is active low (Compare value defines low time)                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| 9     | CAP_APWM | R/W     | 0h    | CAP/APWM operating mode select<br>Reset type: SYSRSn<br>0h (R/W) = ECAP module operates in capture mode. This mode forces the following configuration:<br>- Inhibits TSCTR resets via CTR = PRD event<br>- Inhibits shadow loads on CAP1 and 2 registers<br>- Permits user to enable CAP1-4 register load<br>- CAPx/APWMx pin operates as a capture input<br>1h (R/W) = ECAP module operates in APWM mode. This mode forces the following configuration:<br>- Resets TSCTR on CTR = PRD event (period boundary)<br>- Permits shadow loading on CAP1 and 2 registers<br>- Disables loading of time-stamps into CAP1-4 registers<br>- CAPx/APWMx pin operates as a APWM output |
| 8     | SWSYNC   | R-0/W1S | 0h    | Software-forced Counter (TSCTR) Synchronizer. This provides the user a method to generate a synchronization pulse through software. In APWM mode, the synchronization pulse can also be sourced from the CTR = PRD event.<br>Reset type: SYSRSn<br>0h (R/W) = Writing a zero has no effect. Reading always returns a zero<br>1h (R/W) = Writing a one forces a TSCTR shadow load of current ECAP module and any ECAP modules down-stream providing the SYNCO_SEL bits are 0,0. After writing a 1, this bit returns to a zero.<br>Note: Selection CTR = PRD is meaningful only in APWM mode however, you can choose it in CAP mode if you find doing so useful.               |

**Table 6-11. ECCTL2 Register Field Descriptions (continued)**

| Bit | Field       | Type    | Reset | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
|-----|-------------|---------|-------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 7-6 | SYNCO_SEL   | R/W     | 0h    | Sync-Out Select<br>Reset type: SYSRSn<br>0h (R/W) = Select sync-in event to be the sync-out signal (pass through)<br>1h (R/W) = Select CTR = PRD event to be the sync-out signal<br>2h (R/W) = Disable sync out signal<br>3h (R/W) = Disable sync out signal                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| 5   | SYNCI_EN    | R/W     | 0h    | Counter (TSCTR) Sync-In select mode<br>Reset type: SYSRSn<br>0h (R/W) = Disable sync-in option<br>1h (R/W) = Enable counter (TSCTR) to be loaded from CTRPHS register upon either a SYNCI signal or a S/W force event.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| 4   | TSCTRSTOP   | R/W     | 0h    | Time Stamp (TSCTR) Counter Stop (freeze) Control<br>Reset type: SYSRSn<br>0h (R/W) = TSCTR stopped<br>1h (R/W) = TSCTR free-running                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| 3   | REARM       | R-0/W1S | 0h    | Re-Arming Control. Note: The re-arm function is valid in one shot or continuous mode.<br>Reset type: SYSRSn<br>0h (R/W) = Has no effect (reading always returns a 0)<br>1h (R/W) = Arms the one-shot sequence as follows:<br>1) Resets the Mod4 counter to zero<br>2) Unfreezes the Mod4 counter<br>3) Enables capture register loads                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| 2-1 | STOP_WRAP   | R/W     | 3h    | Stop value for one-shot mode. This is the number (between 1-4) of captures allowed to occur before the CAP(1-4) registers are frozen, that is, capture sequence is stopped.<br>Wrap value for continuous mode. This is the number (between 1-4) of the capture register in which the circular buffer wraps around and starts again.<br>Notes: STOP_WRAP is compared to Mod4 counter and, when equal, 2 actions occur:<br>- Mod4 counter is stopped (frozen)<br>- Capture register loads are inhibited<br>In one-shot mode, further interrupt events are blocked until re-armed.<br>Reset type: SYSRSn<br>0h (R/W) = Stop after Capture Event 1 in one-shot mode<br>Wrap after Capture Event 1 in continuous mode.<br>1h (R/W) = Stop after Capture Event 2 in one-shot mode<br>Wrap after Capture Event 2 in continuous mode.<br>2h (R/W) = Stop after Capture Event 3 in one-shot mode<br>Wrap after Capture Event 3 in continuous mode.<br>3h (R/W) = Stop after Capture Event 4 in one-shot mode<br>Wrap after Capture Event 4 in continuous mode. |
| 0   | CONT_ONESHT | R/W     | 0h    | Continuous or one-shot mode control (applicable only in capture mode)<br>Reset type: SYSRSn<br>0h (R/W) = Operate in continuous mode<br>1h (R/W) = Operate in one-Shot mode                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |



### 6.7.2.9 ECEINT Register (Offset = 16h) [reset = 0h]

ECEINT is shown in [Figure 6-25](#) and described in [Table 6-12](#).

Return to the [Summary Table](#).

The interrupt enable bits (CEVT1, ...) block any of the selected events from generating an interrupt. Events will still be latched into the flag bit (ECFLG register) and can be forced or cleared with the ECFRC or ECCLR registers, respectively.

The proper procedure for configuring peripheral modes and interrupts is as follows:

1. Disable global interrupts
2. Stop eCAP counter
3. Disable eCAP interrupts
4. Configure peripheral registers
5. Clear spurious eCAP interrupt flags
6. Enable eCAP interrupts
7. Start eCAP counter
8. Enable global interrupts

**Figure 6-25. ECEINT Register**

|            |            |        |        |        |        |        |          |
|------------|------------|--------|--------|--------|--------|--------|----------|
| 15         | 14         | 13     | 12     | 11     | 10     | 9      | 8        |
| RESERVED   |            |        |        |        |        |        |          |
| R-0h       |            |        |        |        |        |        |          |
| 7          | 6          | 5      | 4      | 3      | 2      | 1      | 0        |
| CTR_EQ_CMP | CTR_EQ_PRD | CTROVF | CEVT4  | CEVT3  | CEVT2  | CEVT1  | RESERVED |
| R/W-0h     | R/W-0h     | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R-0h     |

**Table 6-12. ECEINT Register Field Descriptions**

| Bit  | Field      | Type | Reset | Description                                                                                                                                                                         |
|------|------------|------|-------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-8 | RESERVED   | R    | 0h    | Reserved                                                                                                                                                                            |
| 7    | CTR_EQ_CMP | R/W  | 0h    | Counter Equal Compare Interrupt Enable<br>Reset type: SYSRSn<br>0h (R/W) = Disable Compare Equal as an Interrupt source<br>1h (R/W) = Enable Compare Equal as an Interrupt source   |
| 6    | CTR_EQ_PRD | R/W  | 0h    | Counter Equal Period Interrupt Enable<br>Reset type: SYSRSn<br>0h (R/W) = Disable Period Equal as an Interrupt source<br>1h (R/W) = Enable Period Equal as an Interrupt source      |
| 5    | CTROVF     | R/W  | 0h    | Counter Overflow Interrupt Enable<br>Reset type: SYSRSn<br>0h (R/W) = Disabled counter Overflow as an Interrupt source<br>1h (R/W) = Enable counter Overflow as an Interrupt source |
| 4    | CEVT4      | R/W  | 0h    | Capture Event 4 Interrupt Enable<br>Reset type: SYSRSn<br>0h (R/W) = Disable Capture Event 4 as an Interrupt source<br>1h (R/W) = Capture Event 4 Interrupt Enable                  |
| 3    | CEVT3      | R/W  | 0h    | Capture Event 3 Interrupt Enable<br>Reset type: SYSRSn<br>0h (R/W) = Disable Capture Event 3 as an Interrupt source<br>1h (R/W) = Enable Capture Event 3 as an Interrupt source     |

**Table 6-12. ECEINT Register Field Descriptions (continued)**

| Bit | Field    | Type | Reset | Description                                                                                                                                                                     |
|-----|----------|------|-------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 2   | CEVT2    | R/W  | 0h    | Capture Event 2 Interrupt Enable<br>Reset type: SYSRSn<br>0h (R/W) = Disable Capture Event 2 as an Interrupt source<br>1h (R/W) = Enable Capture Event 2 as an Interrupt source |
| 1   | CEVT1    | R/W  | 0h    | Capture Event 1 Interrupt Enable<br>Reset type: SYSRSn<br>0h (R/W) = Disable Capture Event 1 as an Interrupt source<br>1h (R/W) = Enable Capture Event 1 as an Interrupt source |
| 0   | RESERVED | R    | 0h    | Reserved                                                                                                                                                                        |

### 6.7.2.10 ECFLG Register (Offset = 17h) [reset = 0h]

ECFLG is shown in [Figure 6-26](#) and described in [Table 6-13](#).

Return to the [Summary Table](#).

Capture Interrupt Flag Register

**Figure 6-26. ECFLG Register**

|          |         |        |       |       |       |       |      |
|----------|---------|--------|-------|-------|-------|-------|------|
| 15       | 14      | 13     | 12    | 11    | 10    | 9     | 8    |
| RESERVED |         |        |       |       |       |       |      |
| R-0h     |         |        |       |       |       |       |      |
| 7        | 6       | 5      | 4     | 3     | 2     | 1     | 0    |
| CTR_CMP  | CTR_PRD | CTROVF | CEVT4 | CEVT3 | CEVT2 | CEVT1 | INT  |
| R-0h     | R-0h    | R-0h   | R-0h  | R-0h  | R-0h  | R-0h  | R-0h |

**Table 6-13. ECFLG Register Field Descriptions**

| Bit  | Field    | Type | Reset | Description                                                                                                                                                                                                                                  |
|------|----------|------|-------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-8 | RESERVED | R    | 0h    | Reserved                                                                                                                                                                                                                                     |
| 7    | CTR_CMP  | R    | 0h    | Compare Equal Compare Status Flag. This flag is active only in APWM mode.<br>Reset type: SYSRSn<br>0h (R/W) = Indicates no event occurred<br>1h (R/W) = Indicates the counter (TSCTR) reached the compare register value (ACMP)              |
| 6    | CTR_PRD  | R    | 0h    | Counter Equal Period Status Flag. This flag is only active in APWM mode.<br>Reset type: SYSRSn<br>0h (R/W) = Indicates no event occurred<br>1h (R/W) = Indicates the counter (TSCTR) reached the period register value (APRD) and was reset. |
| 5    | CTROVF   | R    | 0h    | Counter Overflow Status Flag. This flag is active in CAP and APWM mode.<br>Reset type: SYSRSn<br>0h (R/W) = Indicates no event occurred<br>1h (R/W) = Indicates the counter (TSCTR) has made the transition from FFFFFFFF " 00000000         |
| 4    | CEVT4    | R    | 0h    | Capture Event 4 Status Flag This flag is only active in CAP mode.<br>Reset type: SYSRSn<br>0h (R/W) = Indicates no event occurred<br>1h (R/W) = Indicates the fourth event occurred at ECAPx pin                                             |
| 3    | CEVT3    | R    | 0h    | Capture Event 3 Status Flag. This flag is active only in CAP mode.<br>Reset type: SYSRSn<br>0h (R/W) = Indicates no event occurred<br>1h (R/W) = Indicates the third event occurred at ECAPx pin.                                            |
| 2    | CEVT2    | R    | 0h    | Capture Event 2 Status Flag. This flag is only active in CAP mode.<br>Reset type: SYSRSn<br>0h (R/W) = Indicates no event occurred<br>1h (R/W) = Indicates the second event occurred at ECAPx pin.                                           |
| 1    | CEVT1    | R    | 0h    | Capture Event 1 Status Flag. This flag is only active in CAP mode.<br>Reset type: SYSRSn<br>0h (R/W) = Indicates no event occurred<br>1h (R/W) = Indicates the first event occurred at ECAPx pin.                                            |

**Table 6-13. ECFLG Register Field Descriptions (continued)**

| Bit | Field | Type | Reset | Description                                                                                                                                           |
|-----|-------|------|-------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| 0   | INT   | R    | 0h    | Global Interrupt Status Flag<br>Reset type: SYSRSn<br>0h (R/W) = Indicates no event occurred<br>1h (R/W) = Indicates that an interrupt was generated. |

### 6.7.2.11 ECCLR Register (Offset = 18h) [reset = 0h]

ECCLR is shown in [Figure 6-27](#) and described in [Table 6-14](#).

Return to the [Summary Table](#).

Capture Interrupt Clear Register

**Figure 6-27. ECCLR Register**

|            |            |            |            |            |            |            |            |
|------------|------------|------------|------------|------------|------------|------------|------------|
| 15         | 14         | 13         | 12         | 11         | 10         | 9          | 8          |
| RESERVED   |            |            |            |            |            |            |            |
| R-0h       |            |            |            |            |            |            |            |
| 7          | 6          | 5          | 4          | 3          | 2          | 1          | 0          |
| CTR_CMP    | CTR_PRD    | CTROVF     | CEVT4      | CEVT3      | CEVT2      | CEVT1      | INT        |
| R-0/W1C-0h | R-0/W1C-0h | R-0/W1C-0h | R-0/W1C-0h | R-0/W1C-0h | R-0/W1C-0h | R-0/W1C-0h | R-0/W1C-0h |

**Table 6-14. ECCLR Register Field Descriptions**

| Bit  | Field    | Type    | Reset | Description                                                                                                                                                                                                                                                |
|------|----------|---------|-------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-8 | RESERVED | R       | 0h    | Reserved                                                                                                                                                                                                                                                   |
| 7    | CTR_CMP  | R-0/W1C | 0h    | Counter Equal Compare Status Clear<br>Reset type: SYSRSn<br>0h (R/W) = Writing a 0 has no effect. Always reads back a 0<br>1h (R/W) = Writing a 1 clears the CTR=COMP flag.                                                                                |
| 6    | CTR_PRD  | R-0/W1C | 0h    | Counter Equal Period Status Clear<br>Reset type: SYSRSn<br>0h (R/W) = Writing a 0 has no effect. Always reads back a 0<br>1h (R/W) = Writing a 1 clears the CTR=PRD flag.                                                                                  |
| 5    | CTROVF   | R-0/W1C | 0h    | Counter Overflow Status Clear<br>Reset type: SYSRSn<br>0h (R/W) = Writing a 0 has no effect. Always reads back a 0<br>1h (R/W) = Writing a 1 clears the CTROVF flag.                                                                                       |
| 4    | CEVT4    | R-0/W1C | 0h    | Capture Event 4 Status Clear<br>Reset type: SYSRSn<br>0h (R/W) = Writing a 0 has no effect. Always reads back a 0<br>1h (R/W) = Writing a 1 clears the CEVT4 flag.                                                                                         |
| 3    | CEVT3    | R-0/W1C | 0h    | Capture Event 3 Status Clear<br>Reset type: SYSRSn<br>0h (R/W) = Writing a 0 has no effect. Always reads back a 0<br>1h (R/W) = Writing a 1 clears the CEVT3 flag.                                                                                         |
| 2    | CEVT2    | R-0/W1C | 0h    | Capture Event 2 Status Clear<br>Reset type: SYSRSn<br>0h (R/W) = Writing a 0 has no effect. Always reads back a 0<br>1h (R/W) = Writing a 1 clears the CEVT2 flag.                                                                                         |
| 1    | CEVT1    | R-0/W1C | 0h    | Capture Event 1 Status Clear<br>Reset type: SYSRSn<br>0h (R/W) = Writing a 0 has no effect. Always reads back a 0<br>1h (R/W) = Writing a 1 clears the CEVT1 flag.                                                                                         |
| 0    | INT      | R-0/W1C | 0h    | ECAP Global Interrupt Status Clear<br>Reset type: SYSRSn<br>0h (R/W) = Writing a 0 has no effect. Always reads back a 0<br>1h (R/W) = Writing a 1 clears the INT flag and enable further interrupts to be generated if any of the event flags are set to 1 |

### 6.7.2.12 ECFRC Register (Offset = 19h) [reset = 0h]

ECFRC is shown in [Figure 6-28](#) and described in [Table 6-15](#).

Return to the [Summary Table](#).

Capture Interrupt Force Register

**Figure 6-28. ECFRC Register**

|            |            |            |            |            |            |            |          |
|------------|------------|------------|------------|------------|------------|------------|----------|
| 15         | 14         | 13         | 12         | 11         | 10         | 9          | 8        |
| RESERVED   |            |            |            |            |            |            |          |
| R-0h       |            |            |            |            |            |            |          |
| 7          | 6          | 5          | 4          | 3          | 2          | 1          | 0        |
| CTR_CMP    | CTR_PRD    | CTROVF     | CEVT4      | CEVT3      | CEVT2      | CEVT1      | RESERVED |
| R-0/W1S-0h | R-0/W1S-0h | R-0/W1S-0h | R-0/W1S-0h | R-0/W1S-0h | R-0/W1S-0h | R-0/W1S-0h | R-0h     |

**Table 6-15. ECFRC Register Field Descriptions**

| Bit  | Field    | Type    | Reset | Description                                                                                                                                                                                           |
|------|----------|---------|-------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-8 | RESERVED | R       | 0h    | Reserved                                                                                                                                                                                              |
| 7    | CTR_CMP  | R-0/W1S | 0h    | Force Counter Equal Compare Interrupt. This event is only active in APWM mode.<br>Reset type: SYSRSn<br>0h (R/W) = No effect. Always reads back a 0.<br>1h (R/W) = Writing a 1 sets the CTR=CMF flag. |
| 6    | CTR_PRD  | R-0/W1S | 0h    | Force Counter Equal Period Interrupt. This event is only active in APWM mode.<br>Reset type: SYSRSn<br>0h (R/W) = No effect. Always reads back a 0.<br>1h (R/W) = Writing a 1 sets the CTR=PRD flag.  |
| 5    | CTROVF   | R-0/W1S | 0h    | Force Counter Overflow.<br>Reset type: SYSRSn<br>0h (R/W) = No effect. Always reads back a 0.<br>1h (R/W) = Writing a 1 to this bit sets the CTROVF flag.                                             |
| 4    | CEVT4    | R-0/W1S | 0h    | Force Capture Event 4. This event is only active in CAP mode.<br>Reset type: SYSRSn<br>0h (R/W) = No effect. Always reads back a 0.<br>1h (R/W) = Writing a 1 sets the CEVT4 flag.                    |
| 3    | CEVT3    | R-0/W1S | 0h    | Force Capture Event 3. This event is only active in CAP mode.<br>Reset type: SYSRSn<br>0h (R/W) = No effect. Always reads back a 0.<br>1h (R/W) = Writing a 1 sets the CEVT3 flag.                    |
| 2    | CEVT2    | R-0/W1S | 0h    | Force Capture Event 2. This event is only active in CAP mode.<br>Reset type: SYSRSn<br>0h (R/W) = No effect. Always reads back a 0.<br>1h (R/W) = Writing a 1 sets the CEVT2 flag.                    |
| 1    | CEVT1    | R-0/W1S | 0h    | Force Capture Event 1. This event is only active in CAP mode.<br>Reset type: SYSRSn<br>0h (R/W) = No effect. Always reads back a 0.<br>1h (R/W) = Sets the CEVT1 flag.                                |
| 0    | RESERVED | R       | 0h    | Reserved                                                                                                                                                                                              |

## Enhanced Quadrature Encoder Pulse (eQEP)

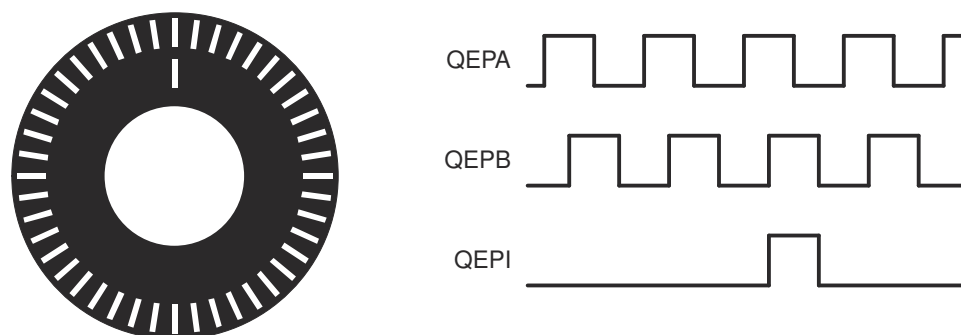
The enhanced Quadrature Encoder Pulse (eQEP) module described here is a Type 0 eQEP. See the [C2000 Real-Time Control Peripheral Reference Guide](#) for a list of all devices with a module of the same type to determine the differences between types and for a list of device-specific differences within a type.

The enhanced quadrature encoder pulse (eQEP) module is used for direct interface with a linear or rotary incremental encoder to get position, direction, and speed information from a rotating machine for use in a high-performance motion and position-control system.

|                                                     |            |
|-----------------------------------------------------|------------|
| <b>7.1 Introduction</b>                             | <b>462</b> |
| <b>7.2 Configuring Device Pins</b>                  | <b>464</b> |
| <b>7.3 Description</b>                              | <b>465</b> |
| <b>7.4 Quadrature Decoder Unit (QDU)</b>            | <b>468</b> |
| <b>7.5 Position Counter and Control Unit (PCCU)</b> | <b>471</b> |
| <b>7.6 eQEP Edge Capture Unit</b>                   | <b>478</b> |
| <b>7.7 eQEP Watchdog</b>                            | <b>482</b> |
| <b>7.8 eQEP Unit Timer Base</b>                     | <b>482</b> |
| <b>7.9 eQEP Interrupt Structure</b>                 | <b>483</b> |
| <b>7.10 eQEP Registers</b>                          | <b>483</b> |

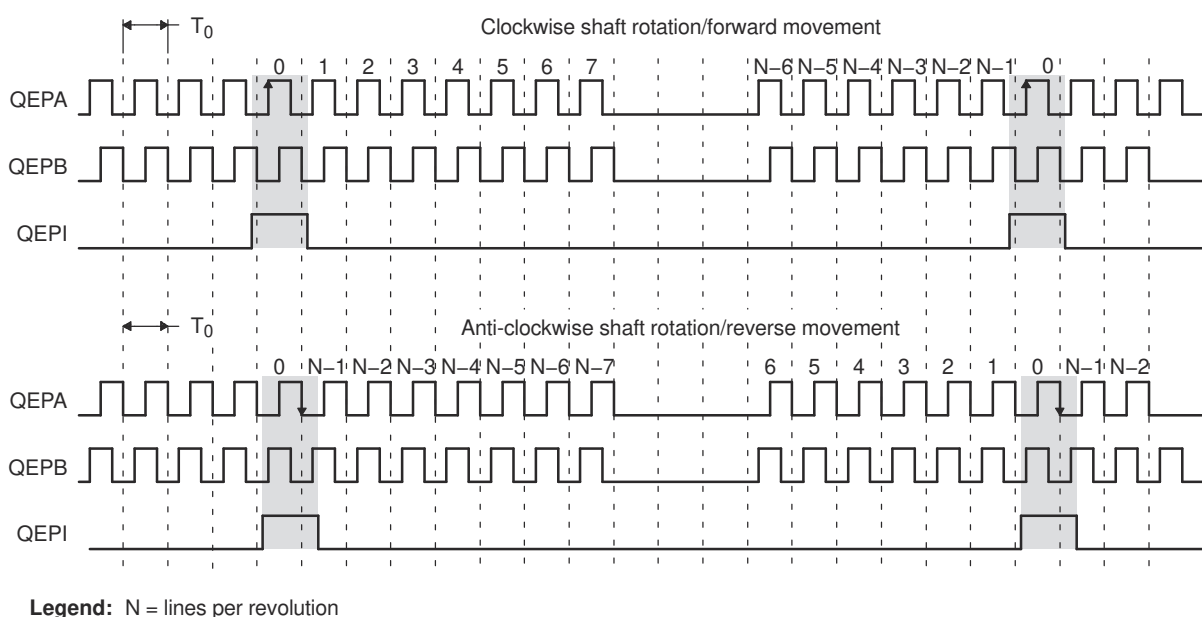
## 7.1 Introduction

An incremental encoder disk is patterned with a track of slots along its periphery, as shown in [Figure 7-1](#). These slots create an alternating pattern of dark and light lines. The disk count is defined as the number of dark and light line pairs that occur per revolution (lines per revolution). As a rule, a second track is added to generate a signal that occurs once per revolution (index signal: QEPI), which can be used to indicate an absolute position. Encoder manufacturers identify the index pulse using different terms such as index, marker, home position, and zero reference



**Figure 7-1. Optical Encoder Disk**

To derive direction information, the lines on the disk are read out by two different photo-elements that "look" at the disk pattern with a mechanical shift of 1/4 the pitch of a line pair between them. This shift is detected with a reticle or mask that restricts the view of the photo-element to the desired part of the disk lines. As the disk rotates, the two photo-elements generate signals that are shifted 90° out of phase from each other. These are commonly called the quadrature QEPA and QEPB signals. The clockwise direction for most encoders is defined as the QEPA channel going positive before the QEPB channel and conversely, as shown in [Figure 7-2](#).



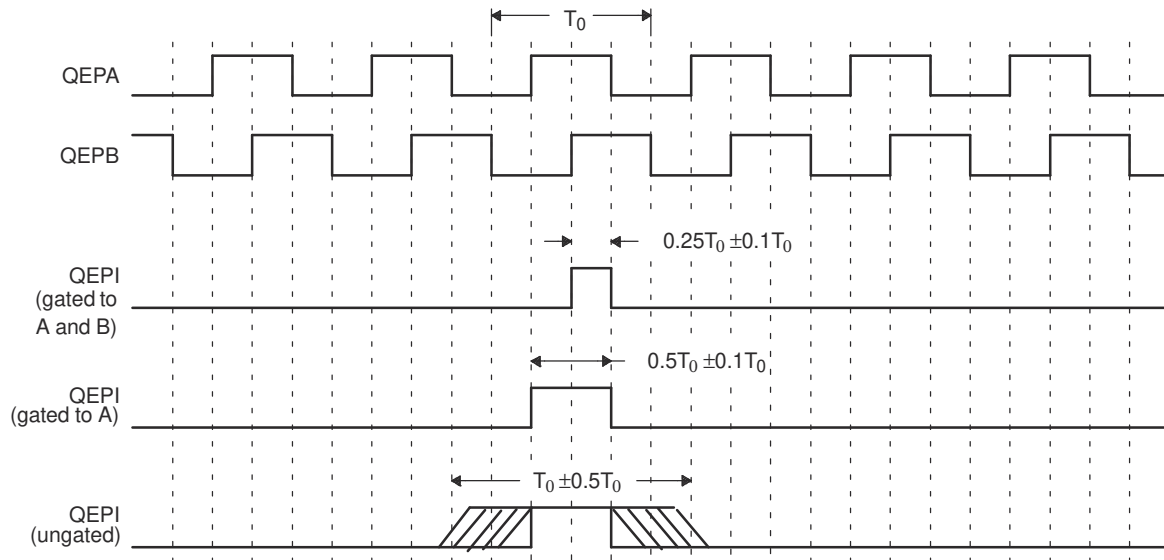
**Figure 7-2. QEP Encoder Output Signal for Forward/Reverse Movement**

The encoder wheel typically makes one revolution for every revolution of the motor, or the wheel may be at a geared rotation ratio with respect to the motor. Therefore, the frequency of the digital signal coming from the QEPA and QEPB outputs varies proportionally with the velocity of the motor. For example, a 2000-line encoder directly coupled to a motor running at 5000 revolutions per minute (rpm) results in a frequency of 166.6 kHz, so



by measuring the frequency of either the QEPA or QEPB output, the processor can determine the velocity of the motor.

Quadrature encoders from different manufacturers come with two forms of index pulse (gated index pulse or ungated index pulse) as shown in Figure 7-3. A nonstandard form of index pulse is ungated. In the ungated configuration, the index edges are not necessarily coincident with A and B signals. The gated index pulse is aligned to any of the four quadrature edges and width of the index pulse and can be equal to a quarter, half, or full period of the quadrature signal.



**Figure 7-3. Index Pulse Example**

Some typical applications of shaft encoders include robotics and computer input in the form of a mouse. Inside your mouse you can see where the mouse ball spins a pair of axles (a left/right, and an up/down axle). These axles are connected to optical shaft encoders that effectively tell the computer how fast and in what direction the mouse is moving.

**General Issues:** Estimating velocity from a digital position sensor is a cost-effective strategy in motor control. Two different first order approximations for velocity may be written as:

$$v(k) \approx \frac{x(k) - x(k-1)}{T} = \frac{\Delta X}{T} \quad (1)$$

$$v(k) \approx \frac{X}{t(k) - t(k-1)} = \frac{X}{\Delta T} \quad (2)$$

where:

- $v(k)$  = Velocity at time instant  $k$
- $x(k)$  = Position at time instant  $k$
- $x(k-1)$  = Position at time instant  $k-1$
- $T$  = Fixed unit time or inverse of velocity calculation rate
- $\Delta X$  = Incremental position movement in unit time
- $t(k)$  = Time instant " $k$ "
- $t(k-1)$  = Time instant " $k-1$ "
- $X$  = Fixed unit position
- $\Delta T$  = Incremental time elapsed for unit position movement

Equation 1 is the conventional approach to velocity estimation and it requires a time base to provide a unit time event for velocity calculation. Unit time is basically the inverse of the velocity calculation rate.

The encoder count (position) is read once during each unit time event. The quantity  $[x(k) - x(k-1)]$  is formed by subtracting the previous reading from the current reading. Then the velocity estimate is computed by multiplying by the known constant  $1/T$  (where  $T$  is the constant time between unit time events and is known in advance).

Estimation based on [Equation 1](#) has an inherent accuracy limit directly related to the resolution of the position sensor and the unit time period  $T$ . For example, consider a 500-line per revolution quadrature encoder with a velocity calculation rate of 400 Hz. When used for position, the quadrature encoder gives a four-fold increase in resolution; in this case, 2000 counts per revolution. The minimum rotation that can be detected is therefore 0.0005 revolutions, which gives a velocity resolution of 12 rpm when sampled at 400 Hz. While this resolution may be satisfactory at moderate or high speeds, for example 1% error at 1200 rpm, it would clearly prove inadequate at low speeds. In fact, at speeds below 12 rpm, the speed estimate would erroneously be zero much of the time.

At low speed, [Equation 2](#) provides a more accurate approach. It requires a position sensor that outputs a fixed interval pulse train, such as the aforementioned quadrature encoder. The width of each pulse is defined by motor speed for a given sensor resolution. [Equation 2](#) can be used to calculate motor speed by measuring the elapsed time between successive quadrature pulse edges. However, this method suffers from the opposite limitation, as does [Equation 1](#). A combination of relatively large motor speeds and high sensor resolution makes the time interval  $\Delta T$  small, and thus more greatly influenced by the timer resolution. This can introduce considerable error into high-speed estimates.

For systems with a large speed range (that is, speed estimation is needed at both low and high speeds), one approach is to use [Equation 2](#) at low speed and have the DSP software switch over to [Equation 1](#) when the motor speed rises above some specified threshold.

### 7.1.1 EQEP Related Collateral

#### Foundational Materials

- [Real-Time Control Reference Guide](#)
  - Refer to the Encoders section

#### Getting Started Materials

- [C2000 Position Manager PTO API Reference Guide Application Report](#)

#### Expert Materials

- [CW/CCW Support on the C2000 eQEP Module Application Report](#)

## 7.2 Configuring Device Pins

The GPIO mux registers must be configured to connect this peripheral to the device pins.

For proper operation of the eQEP module, input GPIO pins must be configured via the GPxQSELn registers for synchronous input mode (with or without qualification). The asynchronous mode should not be used for eQEP input pins. The internal pullups can be configured in the GPyPUD register.

See the *GPIO* chapter for more details on GPIO mux and settings.

## 7.3 Description

This section provides the eQEP inputs, memory map, and functional description.

### 7.3.1 EQEP Inputs

The eQEP inputs include two pins for quadrature-clock mode or direction-count mode, an index (or 0 marker), and a strobe input. The eQEP module requires that the QEPA, QEPB, and QEPI inputs are synchronized to SYSCLK prior to entering the module. The application code should enable the synchronous GPIO input feature on any eQEP-enabled GPIO pins (see the *System Control and Interrupts* chapter for more details).

- **QEPA/XCLK and QEPB/XDIR**

These two pins can be used in quadrature-clock mode or direction-count mode.

- Quadrature-clock Mode

The eQEP encoders provide two square wave signals (A and B) 90 electrical degrees out of phase.

This phase relationship is used to determine the direction of rotation of the input shaft and number of eQEP pulses from the index position to derive the relative position information. For forward or clockwise rotation, QEPA signal leads QEPB signal and conversely. The quadrature decoder uses these two inputs to generate quadrature-clock and direction signals.

- Direction-count Mode

In direction-count mode, direction and clock signals are provided directly from the external source. Some position encoders have this type of output instead of quadrature output. The QEPA pin provides the clock input and the QEPB pin provides the direction input.

- **QEPI: Index or Zero Marker**

The eQEP encoder uses an index signal to assign an absolute start position from which position information is incrementally encoded using quadrature pulses. This pin is connected to the index output of the eQEP encoder to optionally reset the position counter for each revolution. This signal can be used to initialize or latch the position counter on the occurrence of a desired event on the index pin.

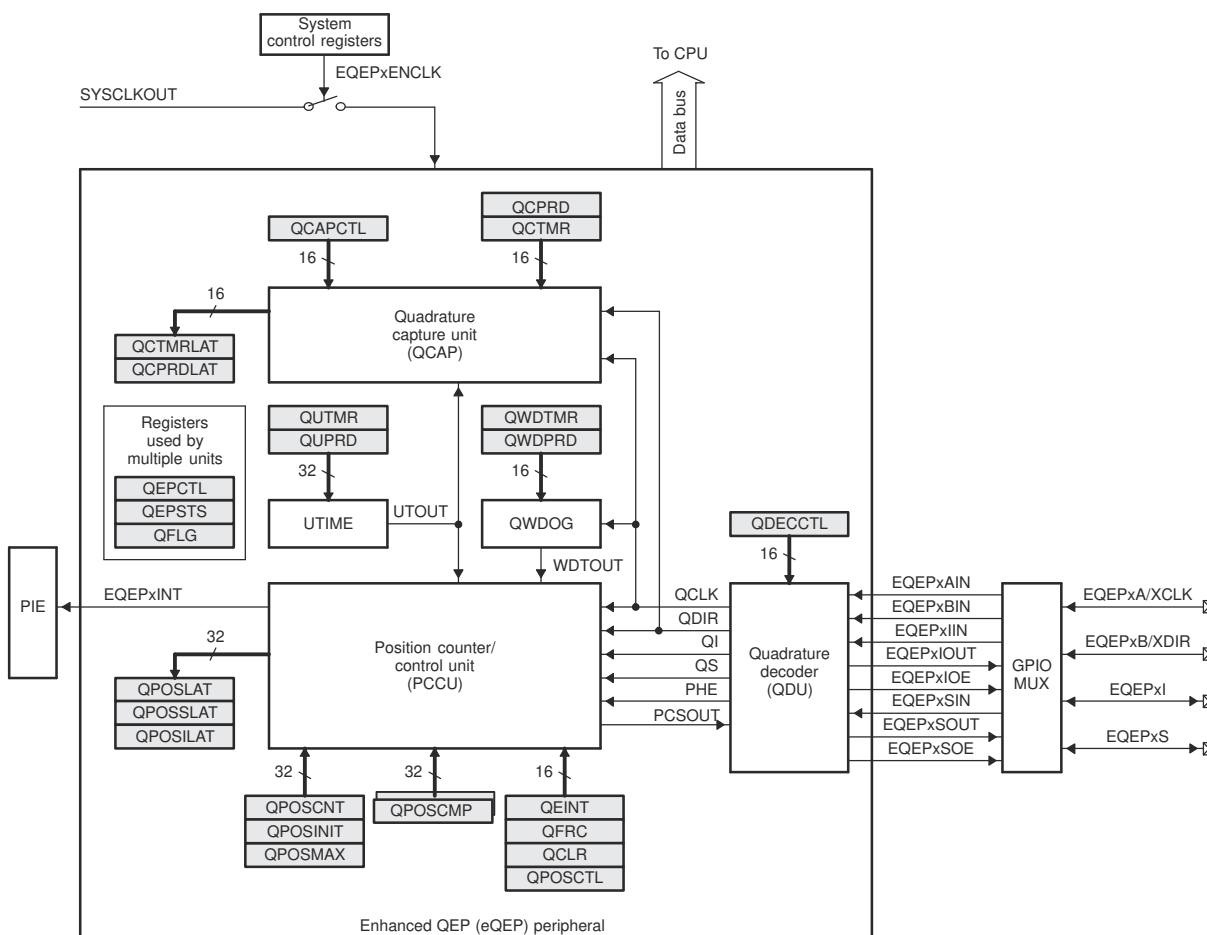
- **QEPS: Strobe Input**

This general-purpose strobe signal can initialize or latch the position counter on the occurrence of a desired event on the strobe pin. This signal is typically connected to a sensor or limit switch to notify that the motor has reached a defined position.

### 7.3.2 Functional Description

The eQEP peripheral contains the following major functional units (as shown in Figure 7-4):

- Programmable input qualification for each pin (part of the GPIO MUX)
- Quadrature decoder unit (QDU)
- Position counter and control unit for position measurement (PCCU)
- Quadrature edge-capture unit for low-speed measurement (QCAP)
- Unit time base for speed/frequency measurement (UTIME)
- Watchdog timer for detecting stalls (QWDOG)



**Figure 7-4. Functional Block Diagram of the eQEP Peripheral**

### 7.3.3 eQEP Memory Map

Table 7-1 lists the registers with their memory locations, sizes, and reset values.

**Table 7-1. EQEP Memory Map**

| Name     | Offset             | Size(x16)/<br>#shadow | Reset      | Register Description                   |
|----------|--------------------|-----------------------|------------|----------------------------------------|
| QPOSCNT  | 0x00               | 2/0                   | 0x00000000 | eQEP Position Counter                  |
| QPOSINIT | 0x02               | 2/0                   | 0x00000000 | eQEP Initialization Position Count     |
| QPOSMAX  | 0x04               | 2/0                   | 0x00000000 | eQEP Maximum Position Count            |
| QPOSCMP  | 0x06               | 2/1                   | 0x00000000 | eQEP Position-compare                  |
| QPOSILAT | 0x08               | 2/0                   | 0x00000000 | eQEP Index Position Latch              |
| QPOSSLAT | 0x0A               | 2/0                   | 0x00000000 | eQEP Strobe Position Latch             |
| QPOSLAT  | 0x0C               | 2/0                   | 0x00000000 | eQEP Position Latch                    |
| QUTMR    | 0x0E               | 2/0                   | 0x00000000 | QEP Unit Timer                         |
| QUPRD    | 0x10               | 2/0                   | 0x00000000 | eQEP Unit Period Register              |
| QWDTMR   | 0x12               | 1/0                   | 0x0000     | eQEP Watchdog Timer                    |
| QWDPRD   | 0x13               | 1/0                   | 0x0000     | eQEP Watchdog Period Register          |
| QDECCTL  | 0x14               | 1/0                   | 0x0000     | eQEP Decoder Control Register          |
| QEPCTL   | 0x15               | 1/0                   | 0x0000     | eQEP Control Register                  |
| QCAPCTL  | 0x16               | 1/0                   | 0x0000     | eQEP Capture Control Register          |
| QPOSCTL  | 0x17               | 1/0                   | 0x00000    | eQEP Position-compare Control Register |
| QEINT    | 0x18               | 1/0                   | 0x0000     | eQEP Interrupt Enable Register         |
| QFLG     | 0x19               | 1/0                   | 0x0000     | eQEP Interrupt Flag Register           |
| QCLR     | 0x1A               | 1/0                   | 0x0000     | eQEP Interrupt Clear Register          |
| QFRC     | 0x1B               | 1/0                   | 0x0000     | eQEP Interrupt Force Register          |
| QEPSTS   | 0x1C               | 1/0                   | 0x0000     | eQEP Status Register                   |
| QCTMR    | 0x1D               | 1/0                   | 0x0000     | eQEP Capture Timer                     |
| QCPRD    | 0x1E               | 1/0                   | 0x0000     | eQEP Capture Period Register           |
| QCTMRLAT | 0x1F               | 1/0                   | 0x0000     | eQEP Capture Timer Latch               |
| QCPRDLAT | 0x20               | 1/0                   | 0x0000     | eQEP Capture Period Latch              |
| reserved | 0x21<br>to<br>0x3F | 31/0                  |            |                                        |

## 7.4 Quadrature Decoder Unit (QDU)

Figure 7-5 shows a functional block diagram of the QDU.

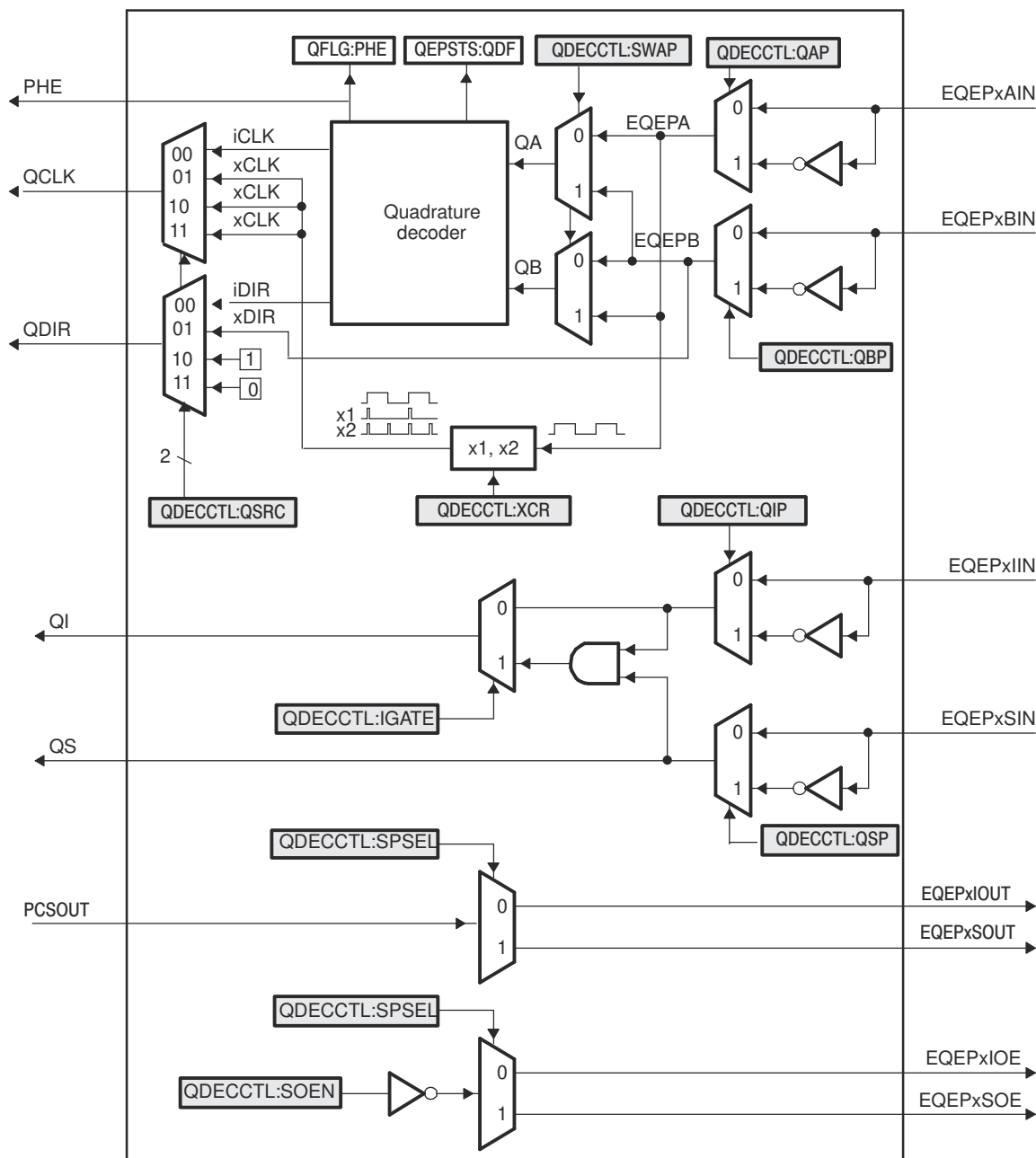


Figure 7-5. Functional Block Diagram of Decoder Unit

### 7.4.1 Position Counter Input Modes

Clock and direction input to the position counter is selected using QDECCTL[QSRC] bits, based on interface input requirement as follows:

- Quadrature-count mode
- Direction-count mode
- UP-count mode

- DOWN-count mode

#### 7.4.1.1 Quadrature Count Mode

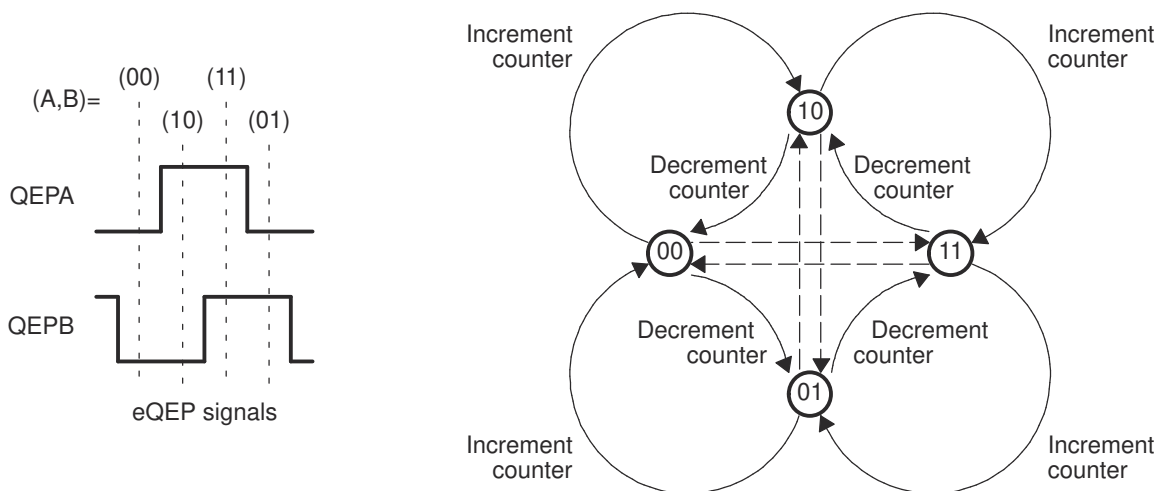
The quadrature decoder generates the direction and clock to the position counter in quadrature count mode.

##### Direction Decoding

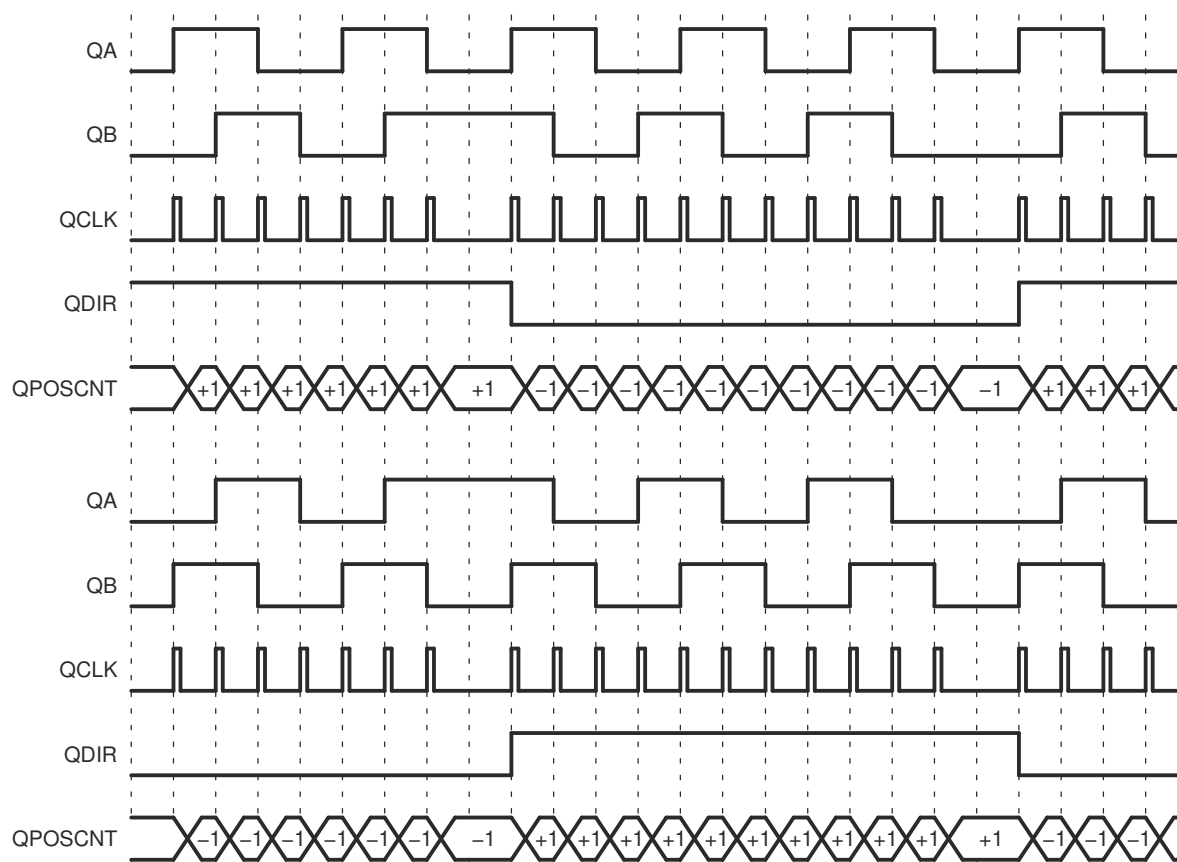
The direction decoding logic of the eQEP circuit determines which one of the sequences (QEPA, QEPB) is the leading sequence and accordingly updates the direction information in the QEPSTS[QDF] bit. Table 7-2 and Figure 7-6 show the direction decoding logic in truth table and state machine form. Both edges of the QEPA and QEPB signals are sensed to generate count pulses for the position counter. Therefore, the frequency of the clock generated by the eQEP logic is four times that of each input sequence. Figure 7-7 shows the direction decoding and clock generation from the eQEP input signals.

**Table 7-2. Quadrature Decoder Truth Table**

| Previous Edge | Present Edge | QDIR   | QPOSCNT                |
|---------------|--------------|--------|------------------------|
| QA↑           | QB↑          | UP     | Increment              |
|               | QB↓          | DOWN   | Decrement              |
|               | QA↓          | TOGGLE | Increment or Decrement |
| QA↓           | QB↓          | UP     | Increment              |
|               | QB↑          | DOWN   | Decrement              |
|               | QA↑          | TOGGLE | Increment or Decrement |
| QB↑           | QA↑          | DOWN   | Decrement              |
|               | QA↓          | UP     | Increment              |
|               | QB↓          | TOGGLE | Increment or Decrement |
| QB↓           | QA↓          | DOWN   | Decrement              |
|               | QA↑          | UP     | Increment              |
|               | QB↑          | TOGGLE | Increment or Decrement |



**Figure 7-6. Quadrature Decoder State Machine**


**Figure 7-7. Quadrature-clock and Direction Decoding**

**Phase Error Flag** In normal operating conditions, quadrature inputs QEPA and QEPB will be 90 degrees out of phase. The phase error flag (PHE) is set in the QFLG register and the QPOSCNT value can be incorrect and offset by multiples of 1 or 3. That is, when edge transition is detected simultaneously on the QEPA and QEPB signals to optionally generate interrupts. State transitions marked by dashed lines in [Figure 7-6](#) are invalid transitions that generate a phase error.

**Count Multiplication** The eQEP position counter provides 4x times the resolution of an input clock by generating a quadrature-clock (QCLK) on the rising/falling edges of both eQEP input clocks (QEPA and QEPB) as shown in [Figure 7-7](#).

**Reverse Count** In normal quadrature count operation, QEPA input is fed to the QA input of the quadrature decoder and the QEPB input is fed to the QB input of the quadrature decoder. Reverse counting is enabled by setting the SWAP bit in the QDECCTL register. This will swap the input to the quadrature decoder, thereby reversing the counting direction.

### 7.4.1.2 Direction-Count Mode

Some position encoders provide direction and clock outputs, instead of quadrature outputs. In such cases, direction-count mode can be used. QEPA input will provide the clock for the position counter and the QEPB input will have the direction information. The position counter is incremented on every rising edge of a QEPA input when the direction input is high, and decremented when the direction input is low.



### 7.4.1.3 Up-Count Mode

The counter direction signal is hard-wired for up-count and the position counter is used to measure the frequency of the QEPA input. Clearing the QDECCTL[XCR] bit enables clock generation to the position counter on both edges of the QEPA input, thereby increasing the measurement resolution by a factor of 2x. In up-count mode, it is recommended that the application not configure QEPB as a GPIO mux option, or ensure that a signal edge is not generated on the QEPB input.

### 7.4.1.4 Down-Count Mode

The counter direction signal is hardwired for a down-count and the position counter is used to measure the frequency of the QEPA input. Clearing the QDECCTL[XCR] bit enables clock generation to the position counter on both edges of a QEPA input, thereby increasing the measurement resolution by a factor of 2x. In down-count mode, it is recommended that the application not configure QEPB as a GPIO mux option, or ensure that a signal edge is not generated on the QEPB input.

## 7.4.2 eQEP Input Polarity Selection

Each eQEP input can be inverted using QDECCTL[8:5] control bits. As an example, setting the QDECCTL[QIP] bit will invert the index input.

## 7.4.3 Position-Compare Sync Output

The enhanced eQEP peripheral includes a position-compare unit that is used to generate the position-compare sync signal on compare match between the position-counter register (QPOSCNT) and the position-compare register (QPOSCMP). This sync signal can be output using an index pin or strobe pin of the EQEP peripheral.

Setting the QDECCTL[SOEN] bit enables the position-compare sync output and the QDECCTL[SPSEL] bit selects either an eQEP index pin or an eQEP strobe pin.

## 7.5 Position Counter and Control Unit (PCCU)

The position-counter and control unit provides two configuration registers (QEPCTL and QPOSCTL) for setting up position-counter operational modes, position-counter initialization/latch modes and position-compare logic for sync signal generation.

### 7.5.1 Position Counter Operating Modes

Position-counter data may be captured in different manners. In some systems, the position counter is accumulated continuously for multiple revolutions and the position-counter value provides the position information with respect to the known reference. An example of this is the quadrature encoder mounted on the motor controlling the print head in the printer. Here the position counter is reset by moving the print head to the home position and then the position counter provides absolute position information with respect to home position.

In other systems, the position counter is reset on every revolution using index pulse, and the position counter provides a rotor angle with respect to the index pulse position.

The position counter can be configured to operate in following four modes

- Position-Counter Reset on Index Event
- Position-Counter Reset on Maximum Position
- Position-Counter Reset on the first Index Event
- Position-Counter Reset on Unit Time Out Event (Frequency Measurement)

In all the above operating modes, the position counter is reset to 0 on overflow and to the QPOS MAX register value on underflow. Overflow occurs when the position counter counts up after the QPOS MAX value. Underflow occurs when the position counter counts down after "0". The Interrupt flag is set to indicate overflow/underflow in QFLG register.

### 7.5.1.1 Position Counter Reset on Index Event (QEPCTL[PCRM]=00)

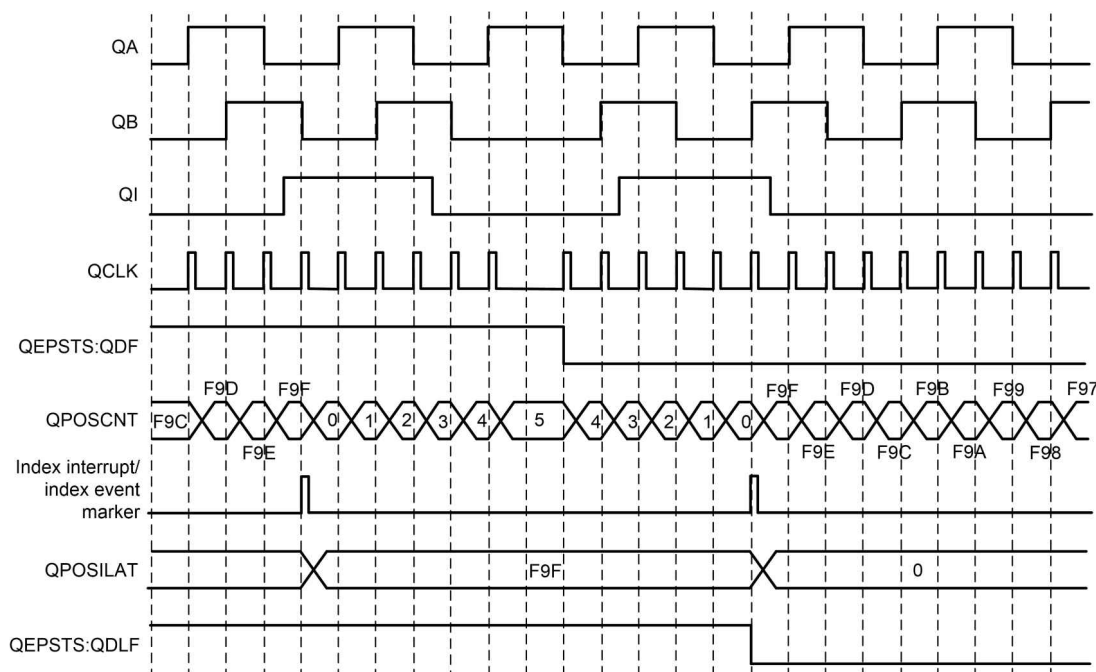
If the index event occurs during the forward movement, then the position counter is reset to 0 on the next eQEP clock. If the index event occurs during the reverse movement, then the position counter is reset to the value in the QPOS MAX register on the next eQEP clock.

First index marker is defined as the quadrature edge following the first index edge. The eQEP peripheral records the occurrence of the first index marker (QEPSTS[FIMF]) and direction on the first index event marker (QEPSTS[FIDF]) in QEPSTS registers, it also remembers the quadrature edge on the first index marker so that same relative quadrature transition is used for index event reset operation.

For example, if the first reset operation occurs on the falling edge of QEPB during the forward direction, then all the subsequent reset must be aligned with the falling edge of QEPB for the forward rotation and on the rising edge of QEPB for the reverse rotation as shown in Figure 7-8.

The position-counter value is latched to the QPOSILAT register and direction information is recorded in the QEPSTS[QDLF] bit on every index event marker. The position-counter error flag (QEPSTS[PCEF]) and error interrupt flag (QFLG[PCE]) are set if the latched value is not equal to 0 or QPOS MAX. The position-counter error flag (QEPSTS[PCEF]) is updated on every index event marker and an interrupt flag (QFLG[PCE]) will be set on error that can be cleared only through software.

The index event latch configuration QEPCTL[IEL] must be configured to '00' or '11' when pcr=0 and the position counter error flag/interrupt flag are generated only in index event reset mode. The position counter value is latched into the IPOS LAT register on every index marker.



**Figure 7-8. Position Counter Reset by Index Pulse for 1000 Line Encoder (QPOS MAX = 3999 or 0xF9F)**

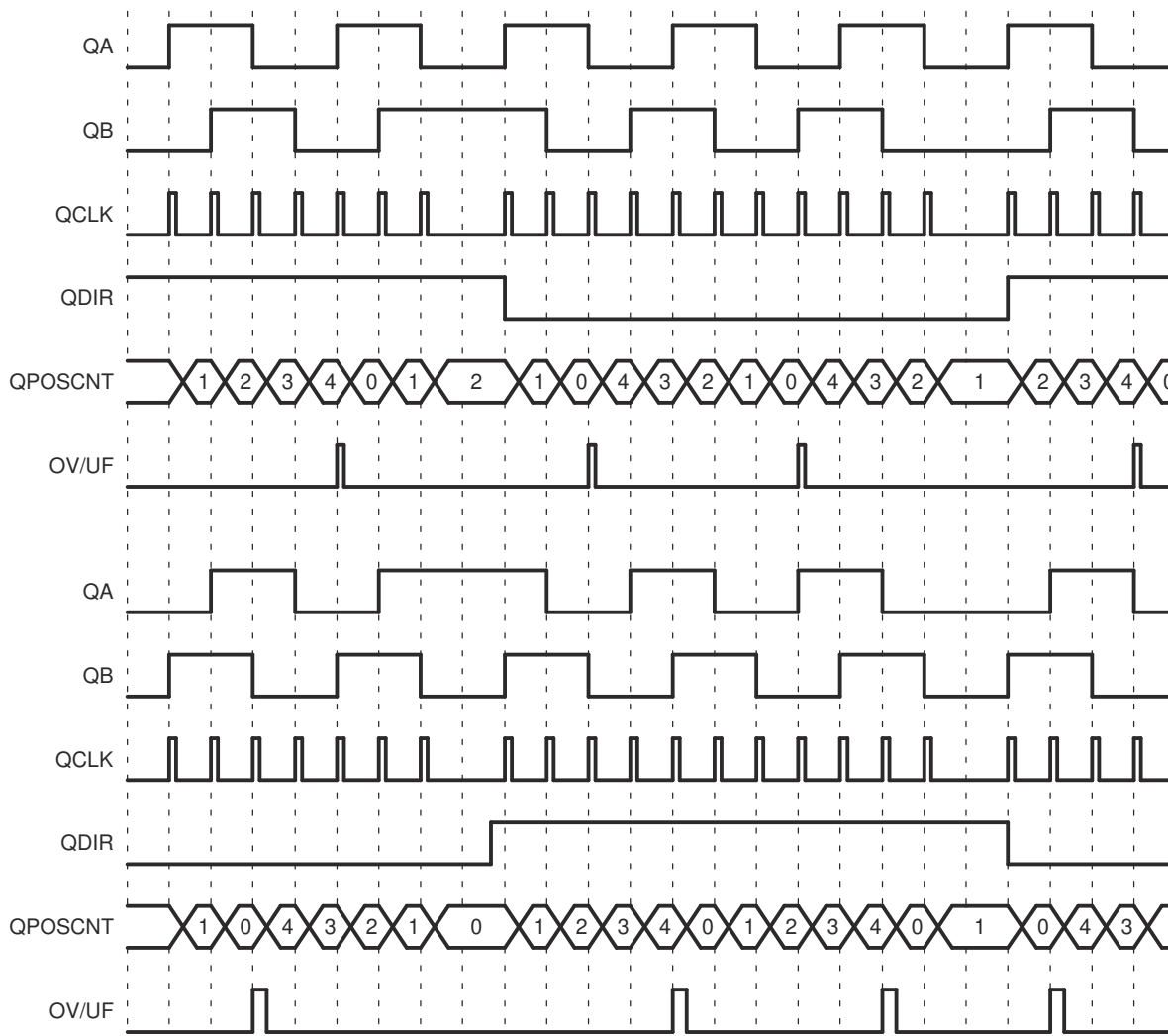
#### Note

In case of a boundary condition where the time period between the Index Event and the previous QCLK edge is less than SYSCLK period, then QPOSCNT gets reset to zero or QPOS MAX in the same SYSCLK cycle and does not wait for the next QCLK edge to occur.

### 7.5.1.2 Position Counter Reset on Maximum Position (QEPCTL[PCRM]=01)

If the position counter is equal to QPOS MAX, then the position counter is reset to 0 on the next eQEP clock for forward movement and position counter overflow flag is set. If the position counter is equal to ZERO, then the position counter is reset to QPOS MAX on the next QEP clock for reverse movement and position-counter underflow flag is set. [Figure 7-9](#) shows the position-counter reset operation in this mode.

The first index marker fields (QEPSTS[FIDF] and QEPSTS[FIMF]) are not applicable in this mode.



**Figure 7-9. Position Counter Underflow/Overflow (QPOS MAX = 4)**

### 7.5.1.3 Position Counter Reset on the First Index Event (QEPCTL[PCRM] = 10)

If the index event occurs during forward movement, then the position counter is reset to 0 on the next eQEP clock. If the index event occurs during the reverse movement, then the position counter is reset to the value in the QPOS MAX register on the next eQEP clock. Note that this is done only on the first occurrence and subsequently the position-counter value is not reset on an index event; rather, it is reset based on maximum position as described in [Section 7.5.1.2](#).

The first index marker fields (QEPSTS[FIDF] and QEPSTS[FIMF]) are not applicable in this mode.

### 7.5.1.4 Position Counter Reset on Unit Time out Event (QEPCTL[PCRM] = 11)

In this mode, QPOSCNT is set to 0 or QPOMAX, depending on the direction mode selected by QDECCTL[QSRC] bits on a unit time event. This is useful for frequency measurement.

### 7.5.2 Position Counter Latch

The eQEP index and strobe input can be configured to latch the position counter (QPOSCNT) into QPOSILAT and QPOSSLAT, respectively, on occurrence of a definite event on these pins.

#### 7.5.2.1 Index Event Latch

In some applications, it may not be desirable to reset the position counter on every index event and instead it may be required to operate the position counter in full 32-bit mode (QEPCTL[PCRM] = 01 and QEPCTL[PCRM] = 10 modes).

In such cases, the eQEP position counter can be configured to latch on the following events and direction information is recorded in the QEPSTS[QDLF] bit on every index event marker.

- Latch on Rising edge (QEPCTL[IEL]=01)
- Latch on Falling edge (QEPCTL[IEL]=10)
- Latch on Index Event Marker (QEPCTL[IEL]=11)

This is particularly useful as an error checking mechanism to check if the position counter accumulated the correct number of counts between index events. As an example, the 1000-line encoder must count 4000 times when moving in the same direction between the index events.

The index event latch interrupt flag (QFLG[IEL]) is set when the position counter is latched to the QPOSILAT register. The index event latch configuration bits (QEPCTZ[IEL]) are ignored when QEPCTL[PCRM] = 00.

#### Latch on Rising Edge (QEPCTL[IEL]=01)

The position-counter value (QPOSCNT) is latched to the QPOSILAT register on every rising edge of an index input.

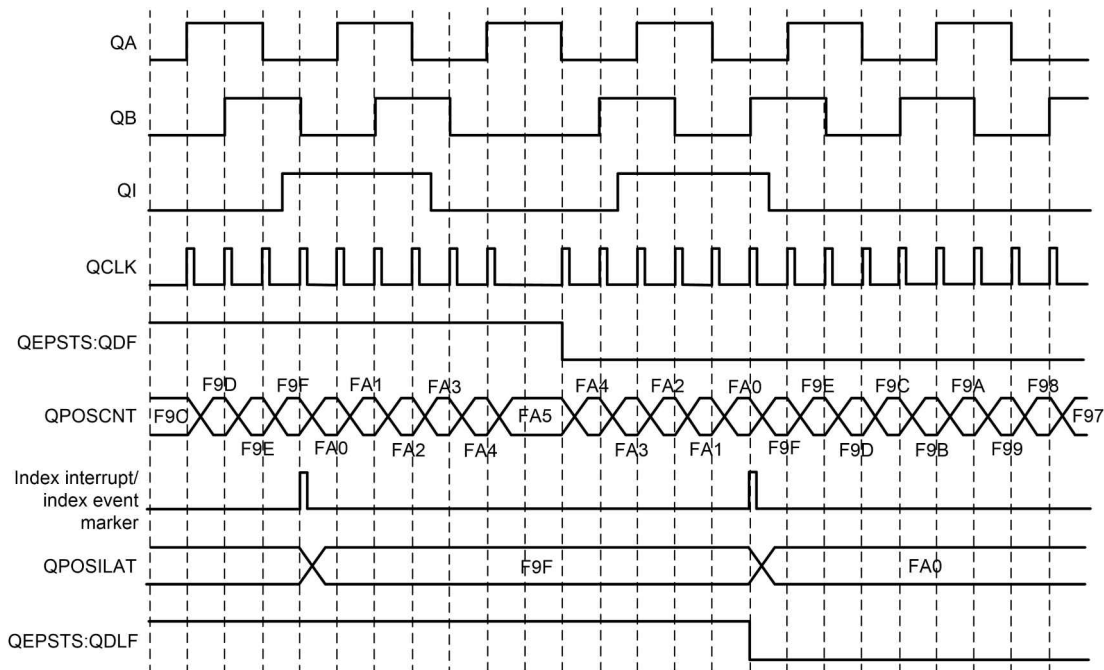
#### Latch on Falling Edge (QEPCTL[IEL] = 10)

The position-counter value (QPOSCNT) is latched to the QPOSILAT register on every falling edge of index input.

#### Latch on Index Event Marker/Software Index Marker (QEPCTL[IEL] = 11)

The first index marker is defined as the quadrature edge following the first index edge. The eQEP peripheral records the occurrence of the first index marker (QEPSTS[FIMF]) and direction on the first index event marker (QEPSTS[FIDF]) in the QEPSTS registers. It also remembers the quadrature edge on the first index marker so that same relative quadrature transition is used for latching the position counter (QEPCTL[IEL]=11).

Figure 7-10 shows the position counter latch using an index event marker.



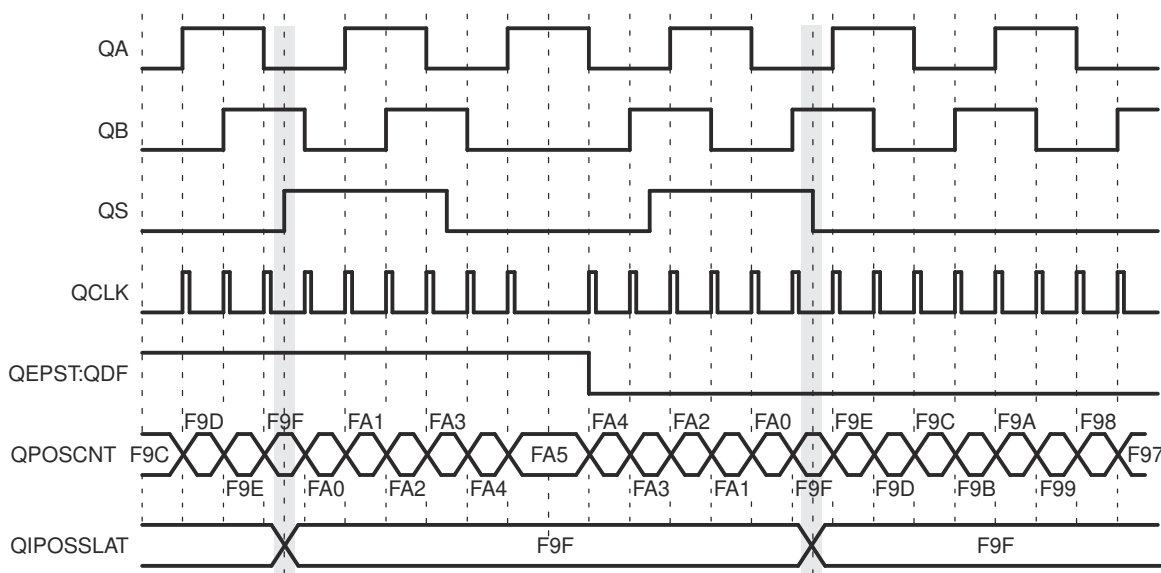
**Figure 7-10. Software Index Marker for 1000-line Encoder (QEPCNT[IEL] = 1)**

### 7.5.2.2 Strobe Event Latch

The position-counter value is latched to the QPOSSLAT register on the rising edge of the strobe input by clearing the QEPCNT[SEL] bit.

If the QEPCNT[SEL] bit is set, then the position-counter value is latched to the QPOSSLAT register on the rising edge of the strobe input for forward direction, and on the falling edge of the strobe input for reverse direction as shown in Figure 7-11.

The strobe event latch interrupt flag (QFLG[SEL]) is set when the position counter is latched to the QPOSSLAT register.



**Figure 7-11. Strobe Event Latch (QEPCNT[SEL] = 1)**

### 7.5.3 Position Counter Initialization

The position counter can be initialized using following events:

- Index event
- Strobe event
- Software initialization

#### Index Event

##### Initialization (IEI)

The QEPI index input can be used to trigger the initialization of the position counter at the rising or falling edge of the index input. If the QEPCTL[IEI] bits are 10, then the position counter (QPOSCNT) is initialized with a value in the QPOSINIT register on the rising edge of index input. Conversely, if the QEPCTL[IEI] bits are 11, initialization will be on the falling edge of the index input.

#### Strobe Event

##### Initialization

##### (SEI)

If the QEPCTL[SEI] bits are 10, then the position counter is initialized with a value in the QPOSINIT register on the rising edge of strobe input.

If QEPCTL[SEL] bits are 11, then the position counter is initialized with a value in the QPOSINIT register on the rising edge of strobe input for forward direction and on the falling edge of strobe input for reverse direction.

#### Software

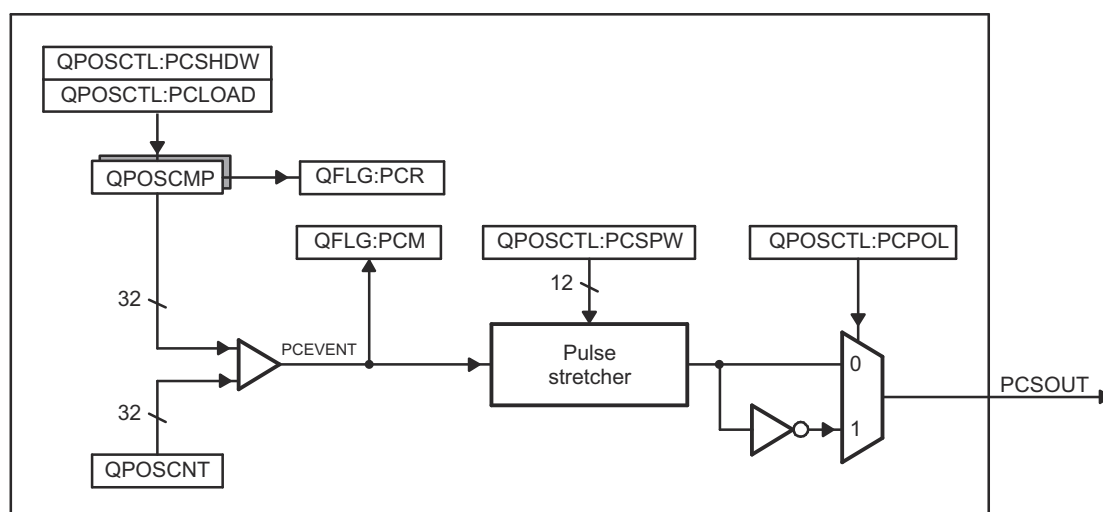
##### Initialization

##### (SWI)

The position counter can be initialized in software by writing a 1 to the QEPCTL[SWI] bit. This bit is not automatically cleared. While the bit is still set, if a 1 is written to it again, the position counter will be re-initialized.

### 7.5.4 eQEP Position-compare Unit

The eQEP peripheral includes a position-compare unit that is used to generate a sync output and/or interrupt on a position-compare match. Figure 7-12 shows a diagram. The position-compare (QPOSCMP) register is shadowed and shadow mode can be enabled or disabled using the QPOSCTL[PSSHDW] bit. If the shadow mode is not enabled, the CPU writes directly to the active position compare register.



**Figure 7-12. eQEP Position-compare Unit**

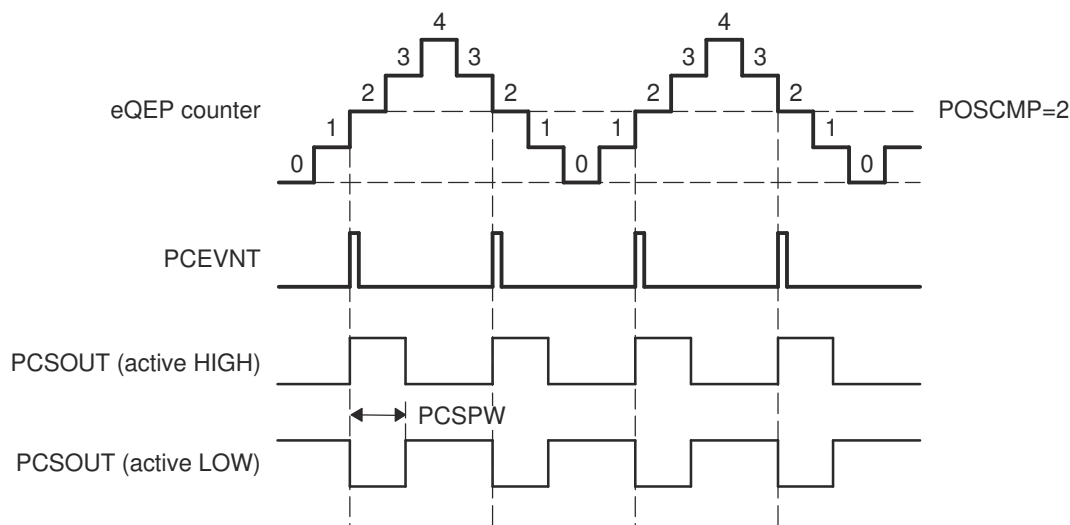
In shadow mode, you can configure the position-compare unit (QPOSCTL[PCLOAD]) to load the shadow register value into the active register on the following events, and to generate the position-compare ready (QFLG[PCR]) interrupt after loading.

- Load on compare match
- Load on position-counter zero event

The position-compare match (QFLG[PCM]) is set when the position-counter value (QPOSCNT) matches with the active position-compare register (QPOSCMP) and the position-compare sync output of the programmable pulse width is generated on compare-match to trigger an external device.

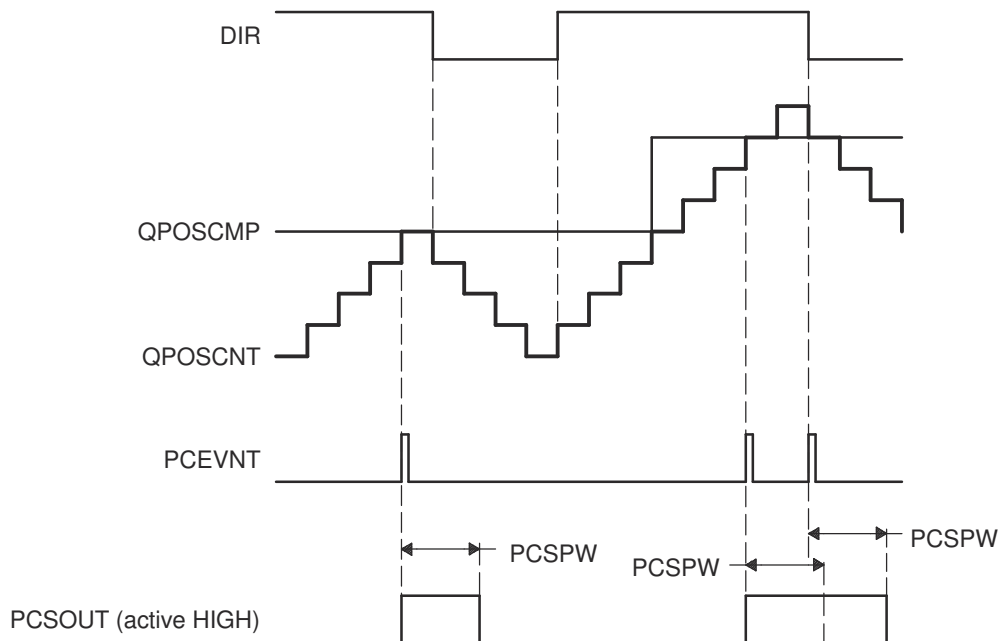
For example, if QPOSCMP = 2, the position-compare unit generates a position-compare event on 1 to 2 transitions of the eQEP position counter for forward counting direction and on 3 to 2 transitions of the eQEP position counter for reverse counting direction (see [Figure 7-13](#)).

See the register section for the layout of the eQEP Position-Compare Control Register (QPOSCTL) and description of the QPOSCTL bit fields.



**Figure 7-13. eQEP Position-compare Event Generation Points**

The pulse stretcher logic in the position-compare unit generates a programmable position-compare sync pulse output on the position-compare match. In the event of a new position-compare match while a previous position-compare pulse is still active, then the pulse stretcher generates a pulse of specified duration from the new position-compare event as shown in [Figure 7-14](#).



**Figure 7-14. eQEP Position-compare Sync Output Pulse Stretcher**



## 7.6 eQEP Edge Capture Unit

The eQEP peripheral includes an integrated edge capture unit to measure the elapsed time between the unit position events as shown in [Figure 7-15](#). This feature is typically used for low-speed measurement using the following formula:

$$v(k) = \frac{X}{t(k) - t(k - 1)} = \frac{X}{\Delta T} \quad (3)$$

where:

- $X$  = Unit position is defined by integer multiple of quadrature edges (see [Figure 7-16](#))
- $\Delta T$  = Elapsed time between unit position events
- $v(k)$  = Velocity at time instant "k"

The eQEP capture timer (QCTMR) runs from prescaled SYSCLKOUT and the prescaler is programmed by the QCAPCTL[CCPS] bits. The capture timer (QCTMR) value is latched into the capture period register (QCPRD) on every unit position event and then the capture timer is reset, a flag is set in QEPSTS:UPEVNT to indicate that new value is latched into the QCPRD register. Software can check this status flag before reading the period register for low speed measurement, and clear the flag by writing 1.

Time measurement ( $\Delta T$ ) between unit position events will be correct if the following conditions are met:

- No more than 65,535 counts have occurred between unit position events.
- No direction change between unit position events.

If the QEP capture timer overflows between unit position events, then it sets the QEP capture overflow flag (QEPSTS[COEF]) in the status register and the QCPRDLAT register is set to 0xFFFF. If direction change occurs between the unit position events, then the error flag is set in the status register (QEPSTS[CDEF]) and the QCPRDLAT register is set to 0xFFFF.

The Capture Timer (QCTMR) and Capture Period register (QCPRD) can be configured to latch on following events.

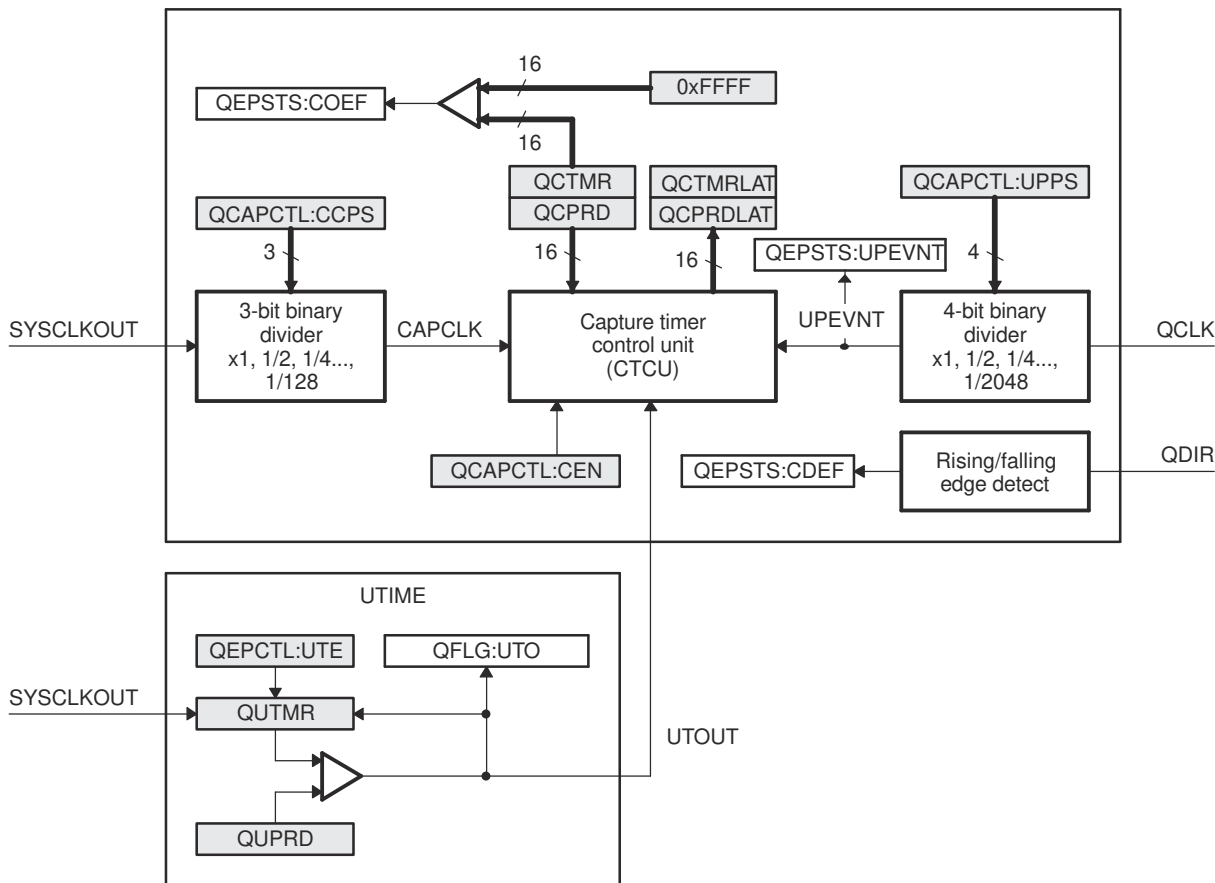
- CPU read of QPOSCNT register
- Unit time-out event

If the QEPCTL[QCLM] bit is cleared, then the capture timer and capture period values are latched into the QCTMRLAT and QCPRDLAT registers, respectively, when the CPU reads the position counter (QPOSCNT).

If the QEPCTL[QCLM] bit is set, then the position counter, capture timer, and capture period values are latched into the QPOSLAT, QCTMRLAT and QCPRDLAT registers, respectively, on unit time out.

[Figure 7-17](#) shows the capture unit operation along with the position counter.

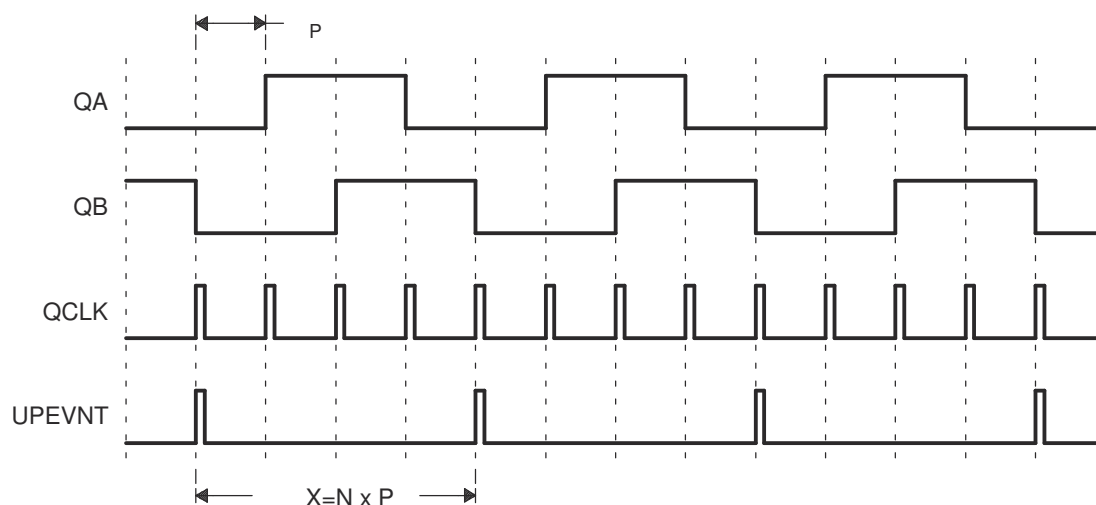




**Figure 7-15. eQEP Edge Capture Unit**

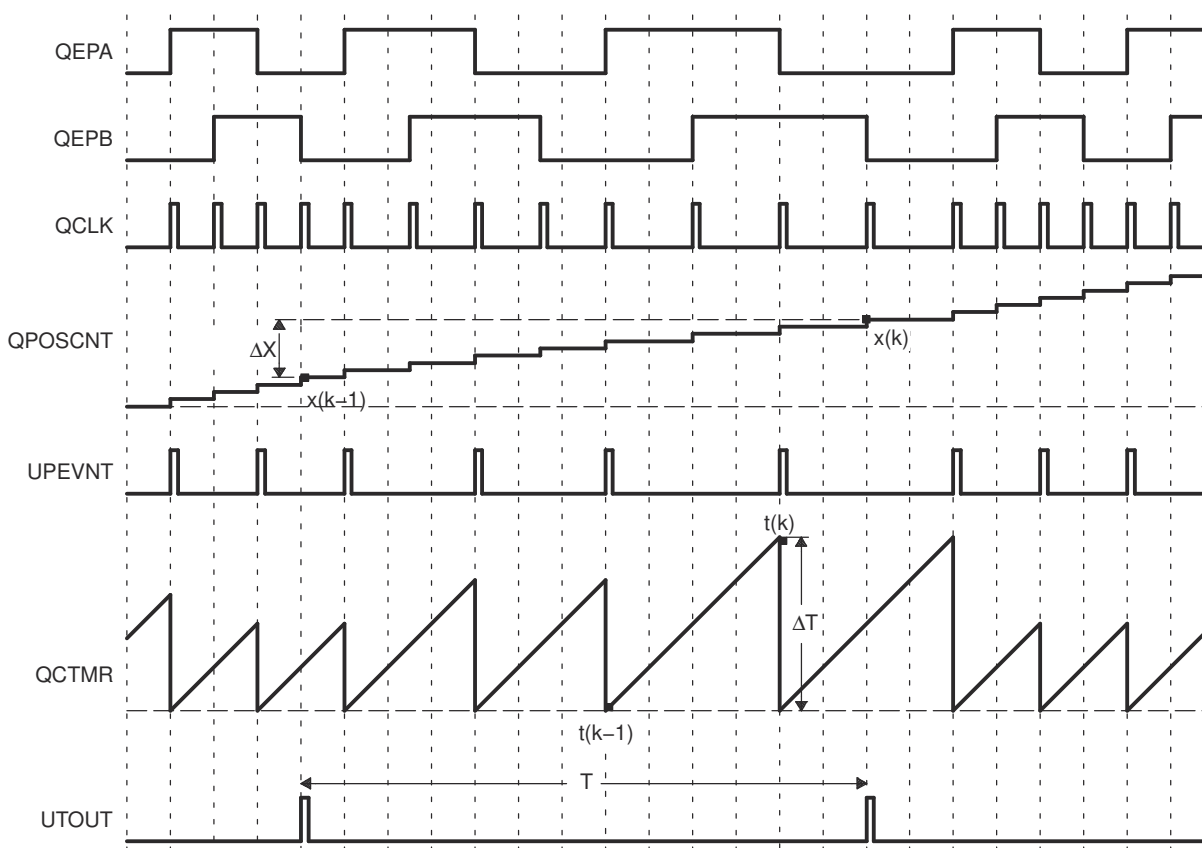
**Note**

The QCAPCTL[UPPS] prescaler should not be modified dynamically (such as switching the unit event prescaler from QCLK/4 to QCLK/8). Doing so may result in undefined behavior. The QCAPCTL[CCPS] prescaler can be modified dynamically (such as switching CAPCLK prescaling mode from SYSCLK/4 to SYSCLK/8) only after the capture unit is disabled.



$N$  = Number of quadrature periods selected using QCAPCTL[UPPS] bits

**Figure 7-16. Unit Position Event for Low Speed Measurement (QCAPCTL[UPPS] = 0010)**



**Figure 7-17. eQEP Edge Capture Unit - Timing Details**

Velocity calculation equation:

$$v(k) = \frac{x(k) - x(k-1)}{T} = \frac{\Delta X}{T} \quad (4)$$

where:

- $v(k)$  = Velocity at time instant  $k$
- $x(k)$  = Position at time instant  $k$
- $x(k-1)$  = Position at time instant  $k-1$
- $T$  = Fixed unit time or inverse of velocity calculation rate
- $\Delta X$  = Incremental position movement in unit time
- $X$  = Fixed unit position
- $\Delta T$  = Incremental time elapsed for unit position movement
- $t(k)$  = Time instant " $k$ "
- $t(k-1)$  = Time instant " $k-1$ "

Unit time ( $T$ ) and unit period ( $X$ ) are configured using the QUPRD and QCAPCTL[Upps] registers. Incremental position output and incremental time output is available in the QPOSLAT and QCPRDLAT registers.

| Parameter  | Relevant Register to Configure or Read the Information                  |
|------------|-------------------------------------------------------------------------|
| $T$        | Unit Period Register (QUPRD)                                            |
| $\Delta X$ | Incremental Position = QPOSLAT( $k$ ) - QPOSLAT( $k-1$ )                |
| $X$        | Fixed unit position defined by sensor resolution and ZCAPCTL[Upps] bits |
| $\Delta T$ | Capture Period Latch (QCPRDLAT)                                         |

## 7.7 eQEP Watchdog

The eQEP peripheral contains a 16-bit watchdog timer (Figure 7-18) that monitors the quadrature clock to indicate proper operation of the motion-control system. The eQEP watchdog timer is clocked from SYSCLKOUT/64 and the quadrature clock event (pulse) resets the watchdog timer. If no quadrature clock event is detected until a period match ( $QWDPRD = QWDTMR$ ), then the watchdog timer times out and the watchdog interrupt flag is set ( $QFLG[WTO]$ ). The time-out value is programmable through the watchdog period register ( $QWDPRD$ ).

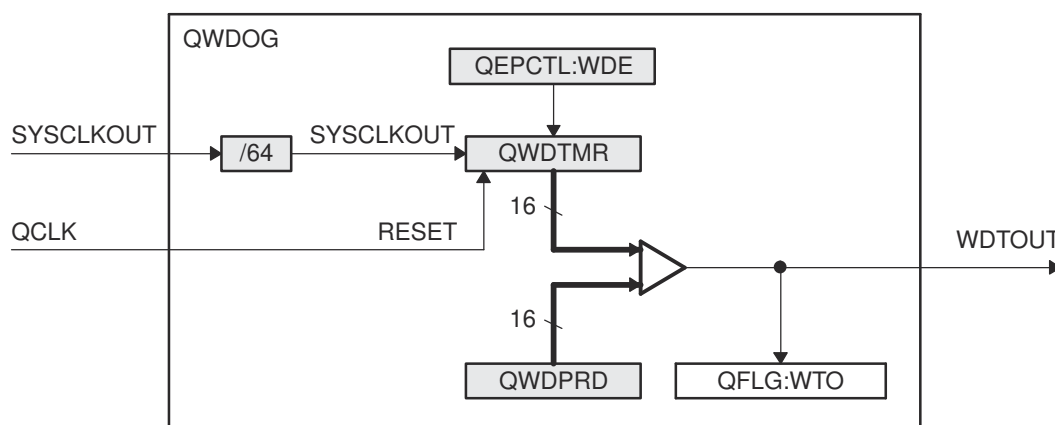


Figure 7-18. eQEP Watchdog Timer

## 7.8 eQEP Unit Timer Base

The eQEP peripheral includes a 32-bit timer (QUTMR) that is clocked by SYSCLKOUT to generate periodic interrupts for velocity calculations, see Figure 7-19. Whenever the unit timer (QUTMR) matches the unit period register (QUPRD), it resets the unit timer (QUTMR) and also generates the unit time out interrupt flag ( $QFLG[UTO]$ ). The unit timer gets reset whenever timer value equals to configured period value.

The eQEP peripheral can be configured to latch the position counter, capture timer, and capture period values on a unit time out event so that latched values are used for velocity calculation as described in Section 7.6.

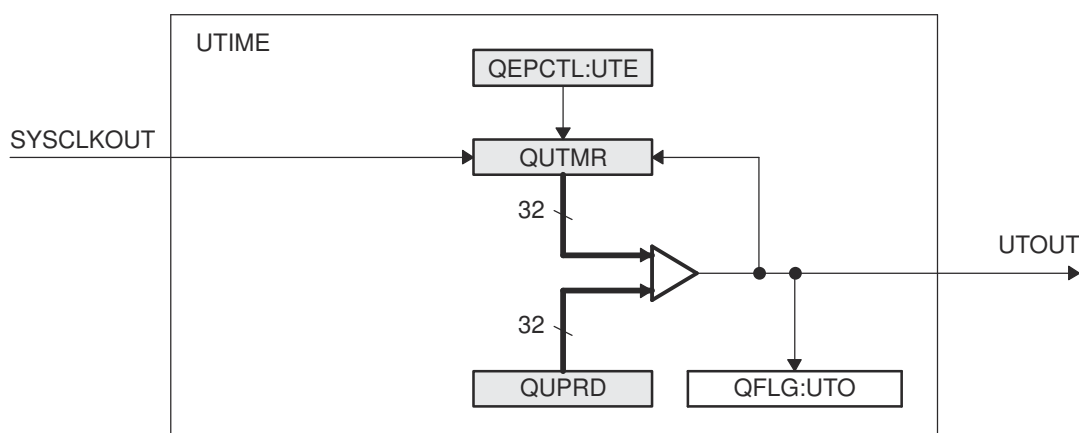


Figure 7-19. eQEP Unit Timer Base



## 7.10.2 EQEP\_REGS Registers

Table 7-4 lists the EQEP\_REGS registers. All register offset addresses not listed in Table 7-4 should be considered as reserved locations and the register contents should not be modified.

**Table 7-4. EQEP\_REGS Registers**

| Offset | Acronym  | Register Name              | Write Protection | Section            |
|--------|----------|----------------------------|------------------|--------------------|
| 0h     | QPOSCNT  | Position Counter           |                  | <a href="#">Go</a> |
| 2h     | QPOSINIT | Position Counter Init      |                  | <a href="#">Go</a> |
| 4h     | QPOSMAX  | Maximum Position Count     |                  | <a href="#">Go</a> |
| 6h     | QPOSCMP  | Position Compare           |                  | <a href="#">Go</a> |
| 8h     | QPOSILAT | Index Position Latch       |                  | <a href="#">Go</a> |
| Ah     | QPOSSLAT | Strobe Position Latch      |                  | <a href="#">Go</a> |
| Ch     | QPOSLAT  | Position Latch             |                  | <a href="#">Go</a> |
| Eh     | QUTMR    | QEP Unit Timer             |                  | <a href="#">Go</a> |
| 10h    | QUPRD    | QEP Unit Period            |                  | <a href="#">Go</a> |
| 12h    | QWDTMR   | QEP Watchdog Timer         |                  | <a href="#">Go</a> |
| 13h    | QWDPRD   | QEP Watchdog Period        |                  | <a href="#">Go</a> |
| 14h    | QDECCTL  | Quadrature Decoder Control |                  | <a href="#">Go</a> |
| 15h    | QEPCTL   | QEP Control                |                  | <a href="#">Go</a> |
| 16h    | QCAPCTL  | Quadrature Capture Control |                  | <a href="#">Go</a> |
| 17h    | QPOSCTL  | Position Compare Control   |                  | <a href="#">Go</a> |
| 18h    | QEINT    | QEP Interrupt Control      |                  | <a href="#">Go</a> |
| 19h    | QFLG     | QEP Interrupt Flag         |                  | <a href="#">Go</a> |
| 1Ah    | QCLR     | QEP Interrupt Clear        |                  | <a href="#">Go</a> |
| 1Bh    | QFRC     | QEP Interrupt Force        |                  | <a href="#">Go</a> |
| 1Ch    | QEPSTS   | QEP Status                 |                  | <a href="#">Go</a> |
| 1Dh    | QCTMR    | QEP Capture Timer          |                  | <a href="#">Go</a> |
| 1Eh    | QCPRD    | QEP Capture Period         |                  | <a href="#">Go</a> |
| 1Fh    | QCTMRLAT | QEP Capture Latch          |                  | <a href="#">Go</a> |
| 20h    | QCPRLAT  | QEP Capture Period Latch   |                  | <a href="#">Go</a> |

Complex bit access types are encoded to fit into small table cells. [Table 7-5](#) shows the codes that are used for access types in this section.

**Table 7-5. EQEP\_REGS Access Type Codes**

| Access Type              | Code    | Description                                                                                                                                                                                                                                                                          |
|--------------------------|---------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Read Type                |         |                                                                                                                                                                                                                                                                                      |
| R                        | R       | Read                                                                                                                                                                                                                                                                                 |
| R-0                      | R<br>-0 | Read<br>Returns 0s                                                                                                                                                                                                                                                                   |
| Write Type               |         |                                                                                                                                                                                                                                                                                      |
| W                        | W       | Write                                                                                                                                                                                                                                                                                |
| W1S                      | W<br>1S | Write<br>1 to set                                                                                                                                                                                                                                                                    |
| Reset or Default Value   |         |                                                                                                                                                                                                                                                                                      |
| -n                       |         | Value after reset or the default value                                                                                                                                                                                                                                               |
| Register Array Variables |         |                                                                                                                                                                                                                                                                                      |
| i,j,k,l,m,n              |         | When these variables are used in a register name, an offset, or an address, they refer to the value of a register array where the register is part of a group of repeating registers. The register groups form a hierarchical structure and the array is represented with a formula. |
| y                        |         | When this variable is used in a register name, an offset, or an address it refers to the value of a register array.                                                                                                                                                                  |

### 7.10.2.1 QPOSCNT Register (Offset = 0h) [reset = 0h]

Position Counter

**Figure 7-21. QPOSCNT Register**

|         |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |   |   |   |   |   |   |   |   |   |   |
|---------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|---|---|---|---|---|---|---|---|---|---|
| 31      | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 | 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| QPOSCNT |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |   |   |   |   |   |   |   |   |   |   |
| R/W-0h  |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |   |   |   |   |   |   |   |   |   |   |

**Table 7-6. QPOSCNT Register Field Descriptions**

| Bit  | Field   | Type | Reset | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|------|---------|------|-------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 31-0 | QPOSCNT | R/W  | 0h    | <p>Position Counter</p> <p>This 32-bit position counter register counts up/down on every eQEP pulse based on direction input. This counter acts as a position integrator whose count value is proportional to position from a give reference point. This Register acts as a Read ONLY register while counter is counting up/down.</p> <p>Note: It is recommended to only write to the position counter register (QPOSCNT) during initialization, that is, when the eQEP position counter is disabled (QPEN bit of QEPCNTL is zero). Once the position counter is enabled (QPEN bit is one), writing to the eQEP position counter register (QPOSCNT) may cause unexpected results.</p> <p>Reset type: SYSRSn</p> |

### 7.10.2.2 QPOSINIT Register (Offset = 2h) [reset = 0h]

Position Counter Initialize

**Figure 7-22. QPOSINIT Register**

|          |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |   |   |   |   |   |   |   |   |   |   |
|----------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|---|---|---|---|---|---|---|---|---|---|
| 31       | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 | 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| QPOSINIT |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |   |   |   |   |   |   |   |   |   |   |
| R/W-0h   |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |   |   |   |   |   |   |   |   |   |   |

**Table 7-7. QPOSINIT Register Field Descriptions**

| Bit  | Field    | Type | Reset | Description                                                                                                                                                                                                                                                                                                             |
|------|----------|------|-------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 31-0 | QPOSINIT | R/W  | 0h    | <p>Position Counter Init</p> <p>This register contains the position value that is used to initialize the position counter based on external strobe or index event. The position counter can be initialized through software. Writes to this register should always be full 32-bit writes.</p> <p>Reset type: SYSRSn</p> |



### 7.10.2.3 QPOSMAX Register (Offset = 4h) [reset = 0h]

Maximum Position Count

**Figure 7-23. QPOSMAX Register**

| 31      | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 | 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|---|---|---|---|---|---|---|---|---|---|
| QPOSMAX |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |   |   |   |   |   |   |   |   |   |   |
| R/W-0h  |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |   |   |   |   |   |   |   |   |   |   |

**Table 7-8. QPOSMAX Register Field Descriptions**

| Bit  | Field   | Type | Reset | Description                                                                                                                                                             |
|------|---------|------|-------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 31-0 | QPOSMAX | R/W  | 0h    | Maximum Position Count<br>This register contains the maximum position counter value. Writes to this register should always be full 32-bit writes.<br>Reset type: SYSRSn |

### 7.10.2.4 QPOSCMP Register (Offset = 6h) [reset = 0h]

Position Compare

**Figure 7-24. QPOSCMP Register**

| 31      | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 | 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|---|---|---|---|---|---|---|---|---|---|
| QPOSCMP |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |   |   |   |   |   |   |   |   |   |   |
| R/W-0h  |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |   |   |   |   |   |   |   |   |   |   |

**Table 7-9. QPOSCMP Register Field Descriptions**

| Bit  | Field   | Type | Reset | Description                                                                                                                                                                                      |
|------|---------|------|-------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 31-0 | QPOSCMP | R/W  | 0h    | Position Compare<br>The position-compare value in this register is compared with the position counter (QPOSCNT) to generate sync output and/or interrupt on compare match.<br>Reset type: SYSRSn |

### 7.10.2.5 QPOSILAT Register (Offset = 8h) [reset = 0h]

Index Position Latch

**Figure 7-25. QPOSILAT Register**

| 31       | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 | 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|----------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|---|---|---|---|---|---|---|---|---|---|
| QPOSILAT |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |   |   |   |   |   |   |   |   |   |   |
| R-0h     |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |   |   |   |   |   |   |   |   |   |   |

**Table 7-10. QPOSILAT Register Field Descriptions**

| Bit  | Field    | Type | Reset | Description                                                                                                                                                  |
|------|----------|------|-------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 31-0 | QPOSILAT | R    | 0h    | Index Position Latch<br>The position-counter value is latched into this register on an index event as defined by the QEPCTL[IEL] bits.<br>Reset type: SYSRSn |

### 7.10.2.6 QPOSSLAT Register (Offset = Ah) [reset = 0h]

Strobe Position Latch

**Figure 7-26. QPOSSLAT Register**

| 31       | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 | 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|----------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|---|---|---|---|---|---|---|---|---|---|
| QPOSSLAT |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |   |   |   |   |   |   |   |   |   |   |
| R-0h     |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |   |   |   |   |   |   |   |   |   |   |

**Table 7-11. QPOSSLAT Register Field Descriptions**

| Bit  | Field    | Type | Reset | Description                                                                                                                                                   |
|------|----------|------|-------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 31-0 | QPOSSLAT | R    | 0h    | Strobe Position Latch<br>The position-counter value is latched into this register on a strobe event as defined by the QEPCTL[SEL] bits.<br>Reset type: SYSRSn |

### 7.10.2.7 QPOSLAT Register (Offset = Ch) [reset = 0h]

Position Latch

**Figure 7-27. QPOSLAT Register**

| 31      | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 | 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|---|---|---|---|---|---|---|---|---|---|
| QPOSLAT |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |   |   |   |   |   |   |   |   |   |   |
| R-0h    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |   |   |   |   |   |   |   |   |   |   |

**Table 7-12. QPOSLAT Register Field Descriptions**

| Bit  | Field   | Type | Reset | Description                                                                                                                |
|------|---------|------|-------|----------------------------------------------------------------------------------------------------------------------------|
| 31-0 | QPOSLAT | R    | 0h    | Position Latch<br>The position-counter value is latched into this register on a unit time out event.<br>Reset type: SYSRSn |

### 7.10.2.8 QUTMR Register (Offset = Eh) [reset = 0h]

QEP Unit Timer

**Figure 7-28. QUTMR Register**

| 31     | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 | 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|--------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|---|---|---|---|---|---|---|---|---|---|
| QUTMR  |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |   |   |   |   |   |   |   |   |   |   |
| R/W-0h |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |   |   |   |   |   |   |   |   |   |   |

**Table 7-13. QUTMR Register Field Descriptions**

| Bit  | Field | Type | Reset | Description                                                                                                                                                                                      |
|------|-------|------|-------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 31-0 | QUTMR | R/W  | 0h    | QEP Unit Timer<br>This register acts as time base for unit time event generation. When this timer value matches the unit time period value a unit time event is generated.<br>Reset type: SYSRSn |

### 7.10.2.9 QUPRD Register (Offset = 10h) [reset = 0h]

QEP Unit Period

**Figure 7-29. QUPRD Register**

|        |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |   |   |   |   |   |   |   |   |   |   |
|--------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|---|---|---|---|---|---|---|---|---|---|
| 31     | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 | 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| QUPRD  |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |   |   |   |   |   |   |   |   |   |   |
| R/W-0h |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |   |   |   |   |   |   |   |   |   |   |

**Table 7-14. QUPRD Register Field Descriptions**

| Bit  | Field | Type | Reset | Description                                                                                                                                                                                                                                                                                                          |
|------|-------|------|-------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 31-0 | QUPRD | R/W  | 0h    | QEP Unit Period<br>This register contains the period count for the unit timer to generate periodic unit time events. These events latch the eQEP position information at periodic intervals and optionally generate an interrupt. Writes to this register should always be full 32-bit writes.<br>Reset type: SYSRSn |

### 7.10.2.10 QWDTMR Register (Offset = 12h) [reset = 0h]

QEP Watchdog Timer

**Figure 7-30. QWDTMR Register**

|        |    |    |    |    |    |   |   |
|--------|----|----|----|----|----|---|---|
| 15     | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
| QWDTMR |    |    |    |    |    |   |   |
| R/W-0h |    |    |    |    |    |   |   |
| 7      | 6  | 5  | 4  | 3  | 2  | 1 | 0 |
| QWDTMR |    |    |    |    |    |   |   |
| R/W-0h |    |    |    |    |    |   |   |

**Table 7-15. QWDTMR Register Field Descriptions**

| Bit  | Field  | Type | Reset | Description                                                                                                                                                                                                                                                                                                           |
|------|--------|------|-------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-0 | QWDTMR | R/W  | 0h    | QEP Watchdog Timer<br>This register acts as time base for the watchdog to detect motor stalls. When this timer value matches with the watchdog's period value a watchdog timeout interrupt is generated. This register is reset upon edge transition in quadrature-clock indicating the motion.<br>Reset type: SYSRSn |

### 7.10.2.11 QWDPRD Register (Offset = 13h) [reset = 0h]

QEP Watchdog Period

**Figure 7-31. QWDPRD Register**

|        |    |    |    |    |    |   |   |
|--------|----|----|----|----|----|---|---|
| 15     | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
| QWDPRD |    |    |    |    |    |   |   |
| R/W-0h |    |    |    |    |    |   |   |
| 7      | 6  | 5  | 4  | 3  | 2  | 1 | 0 |
| QWDPRD |    |    |    |    |    |   |   |
| R/W-0h |    |    |    |    |    |   |   |

**Table 7-16. QWDPRD Register Field Descriptions**

| Bit  | Field  | Type | Reset | Description                                                                                                                                                                                                                                    |
|------|--------|------|-------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-0 | QWDPRD | R/W  | 0h    | QEP Watchdog Period<br>This register contains the time-out count for the eQEP peripheral watch dog timer.<br>When the watchdog timer value matches the watchdog period value, a watchdog timeout interrupt is generated.<br>Reset type: SYSRSn |

### 7.10.2.12 QDECCTL Register (Offset = 14h) [reset = 0h]

Quadrature Decoder Control

**Figure 7-32. QDECCTL Register**

| 15     | 14     | 13     | 12       | 11     | 10     | 9      | 8      |
|--------|--------|--------|----------|--------|--------|--------|--------|
| QSRC   |        | SOEN   | SPSEL    | XCR    | SWAP   | IGATE  | QAP    |
| R/W-0h |        | R/W-0h | R/W-0h   | R/W-0h | R/W-0h | R/W-0h | R/W-0h |
| 7      | 6      | 5      | 4        | 3      | 2      | 1      | 0      |
| QBP    | QIP    | QSP    | RESERVED |        |        |        |        |
| R/W-0h | R/W-0h | R/W-0h | R-0h     |        |        |        |        |

**Table 7-17. QDECCTL Register Field Descriptions**

| Bit   | Field | Type | Reset | Description                                                                                                                                                                                                                                                                                                                                        |
|-------|-------|------|-------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-14 | QSRC  | R/W  | 0h    | Position-counter source selection<br>Reset type: SYSRSn<br>0h (R/W) = Quadrature count mode (QCLK = iCLK, QDIR = iDIR)<br>1h (R/W) = Direction-count mode (QCLK = xCLK, QDIR = xDIR)<br>2h (R/W) = UP count mode for frequency measurement (QCLK = xCLK, QDIR = 1)<br>3h (R/W) = DOWN count mode for frequency measurement (QCLK = xCLK, QDIR = 0) |
| 13    | SOEN  | R/W  | 0h    | Sync output-enable<br>Reset type: SYSRSn<br>0h (R/W) = Disable position-compare sync output<br>1h (R/W) = Enable position-compare sync output                                                                                                                                                                                                      |
| 12    | SPSEL | R/W  | 0h    | Sync output pin selection<br>Reset type: SYSRSn<br>0h (R/W) = Index pin is used for sync output<br>1h (R/W) = Strobe pin is used for sync output                                                                                                                                                                                                   |
| 11    | XCR   | R/W  | 0h    | External Clock Rate<br>Reset type: SYSRSn<br>0h (R/W) = 2x resolution: Count the rising/falling edge<br>1h (R/W) = 1x resolution: Count the rising edge only                                                                                                                                                                                       |
| 10    | SWAP  | R/W  | 0h    | CLK/DIR Signal Source for Position Counter<br>Reset type: SYSRSn<br>0h (R/W) = Quadrature-clock inputs are not swapped<br>1h (R/W) = Quadrature-clock inputs are swapped                                                                                                                                                                           |
| 9     | IGATE | R/W  | 0h    | Index pulse gating option<br>Reset type: SYSRSn<br>0h (R/W) = Disable gating of Index pulse<br>1h (R/W) = Gate the index pin with strobe                                                                                                                                                                                                           |
| 8     | QAP   | R/W  | 0h    | QEPA input polarity<br>Reset type: SYSRSn<br>0h (R/W) = No effect<br>1h (R/W) = Negates QEPA input                                                                                                                                                                                                                                                 |
| 7     | QBP   | R/W  | 0h    | QEPB input polarity<br>Reset type: SYSRSn<br>0h (R/W) = No effect<br>1h (R/W) = Negates QEPB input                                                                                                                                                                                                                                                 |

**Table 7-17. QDECCTL Register Field Descriptions (continued)**

| Bit | Field    | Type | Reset | Description                                                                                        |
|-----|----------|------|-------|----------------------------------------------------------------------------------------------------|
| 6   | QIP      | R/W  | 0h    | QEPI input polarity<br>Reset type: SYSRSn<br>0h (R/W) = No effect<br>1h (R/W) = Negates QEPI input |
| 5   | QSP      | R/W  | 0h    | QEPS input polarity<br>Reset type: SYSRSn<br>0h (R/W) = No effect<br>1h (R/W) = Negates QEPS input |
| 4-0 | RESERVED | R    | 0h    | Reserved                                                                                           |

### 7.10.2.13 QEPCNTL Register (Offset = 15h) [reset = 0h]

QEPCNTL Control

**Figure 7-33. QEPCNTL Register**

| 15        | 14     | 13     | 12 | 11     | 10     | 9      | 8      |
|-----------|--------|--------|----|--------|--------|--------|--------|
| FREE_SOFT |        | PCRM   |    | SEI    |        | IEI    |        |
| R/W-0h    |        | R/W-0h |    | R/W-0h |        | R/W-0h |        |
| 7         | 6      | 5      | 4  | 3      | 2      | 1      | 0      |
| SWI       | SEL    | IEL    |    | QOPEN  | QCLM   | UTE    | WDE    |
| R/W-0h    | R/W-0h | R/W-0h |    | R/W-0h | R/W-0h | R/W-0h | R/W-0h |

**Table 7-18. QEPCNTL Register Field Descriptions**

| Bit   | Field     | Type | Reset | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|-------|-----------|------|-------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-14 | FREE_SOFT | R/W  | 0h    | Emulation mode<br>Reset type: SYSRSn<br>0h (R/W) = QPOSCNT behavior<br>Position counter stops immediately on emulation suspend<br>0h (R/W) = QWDTMR behavior<br>Watchdog counter stops immediately<br>0h (R/W) = QUTMR behavior<br>Unit timer stops immediately<br>0h (R/W) = QCTMR behavior<br>Capture Timer stops immediately<br>1h (R/W) = QPOSCNT behavior<br>Position counter continues to count until the rollover<br>1h (R/W) = QWDTMR behavior<br>Watchdog counter counts until WD period match roll over<br>1h (R/W) = QUTMR behavior<br>Unit timer counts until period rollover<br>1h (R/W) = QCTMR behavior<br>Capture Timer counts until next unit period event<br>2h (R/W) = QPOSCNT behavior<br>Position counter is unaffected by emulation suspend<br>2h (R/W) = QWDTMR behavior<br>Watchdog counter is unaffected by emulation suspend<br>2h (R/W) = QUTMR behavior<br>Unit timer is unaffected by emulation suspend<br>2h (R/W) = QCTMR behavior<br>Capture Timer is unaffected by emulation suspend<br>3h (R/W) = Same as FREE_SOFT_2 |
| 13-12 | PCRM      | R/W  | 0h    | Position counter reset<br>Reset type: SYSRSn<br>0h (R/W) = Position counter reset on an index event<br>1h (R/W) = Position counter reset on the maximum position<br>2h (R/W) = Position counter reset on the first index event<br>3h (R/W) = Position counter reset on a unit time event                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |

**Table 7-18. QEPCTL Register Field Descriptions (continued)**

| Bit   | Field | Type | Reset | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|-------|-------|------|-------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 11-10 | SEI   | R/W  | 0h    | Strobe event initialization of position counter<br>Reset type: SYSRSn<br>0h (R/W) = Does nothing (action disabled)<br>1h (R/W) = Does nothing (action disabled)<br>2h (R/W) = Initializes the position counter on rising edge of the QEPS signal<br>3h (R/W) = Clockwise Direction:<br>Initializes the position counter on the rising edge of QEPS strobe<br>Counter Clockwise Direction:<br>Initializes the position counter on the falling edge of QEPS strobe                                                                                           |
| 9-8   | IEI   | R/W  | 0h    | Index event init of position count<br>Reset type: SYSRSn<br>0h (R/W) = Do nothing (action disabled)<br>1h (R/W) = Do nothing (action disabled)<br>2h (R/W) = Initializes the position counter on the rising edge of the QEPI signal (QPOSCNT = QPOSINIT)<br>3h (R/W) = Initializes the position counter on the falling edge of QEPI signal (QPOSCNT = QPOSINIT)                                                                                                                                                                                            |
| 7     | SWI   | R/W  | 0h    | Software init position counter<br>Reset type: SYSRSn<br>0h (R/W) = Do nothing (action disabled)<br>1h (R/W) = Initialize position counter (QPOSCNT=QPOSINIT). This bit is not cleared automatically                                                                                                                                                                                                                                                                                                                                                        |
| 6     | SEL   | R/W  | 0h    | Strobe event latch of position counter<br>Reset type: SYSRSn<br>0h (R/W) = The position counter is latched on the rising edge of QEPS strobe (QPOSSLAT = POSCCNT). Latching on the falling edge can be done by inverting the strobe input using the QSP bit in the QDECCTL register<br>1h (R/W) = Clockwise Direction:<br>Position counter is latched on rising edge of QEPS strobe<br>Counter Clockwise Direction:<br>Position counter is latched on falling edge of QEPS strobe                                                                          |
| 5-4   | IEL   | R/W  | 0h    | Index event latch of position counter (software index marker)<br>Reset type: SYSRSn<br>0h (R/W) = Reserved<br>1h (R/W) = Latches position counter on rising edge of the index signal<br>2h (R/W) = Latches position counter on falling edge of the index signal<br>3h (R/W) = Software index marker. Latches the position counter and quadrature direction flag on index event marker. The position counter is latched to the QPOSILAT register and the direction flag is latched in the QEPSTS[QDLF] bit. This mode is useful for software index marking. |



**Table 7-18. QEPCTL Register Field Descriptions (continued)**

| Bit | Field | Type | Reset | Description                                                                                                                                                                                                                                                                                                                                                                                                                              |
|-----|-------|------|-------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 3   | QPEN  | R/W  | 0h    | <p>Quadrature position counter enable/software reset</p> <p>Reset type: SYSRSn</p> <p>0h (R/W) = Reset the eQEP peripheral internal operating flags/read-only registers. Control/configuration registers are not disturbed by a software reset.</p> <p>When QPEN is disabled, some flags in the QFLG register do not get reset or cleared and show the actual state of that flag.</p> <p>1h (R/W) = eQEP position counter is enabled</p> |
| 2   | QCLM  | R/W  | 0h    | <p>QEP capture latch mode</p> <p>Reset type: SYSRSn</p> <p>0h (R/W) = Latch on position counter read by CPU. Capture timer and capture period values are latched into QCTMRLAT and QCPRDLAT registers when CPU reads the QPOSCNT register.</p> <p>1h (R/W) = Latch on unit time out. Position counter, capture timer and capture period values are latched into QPOSLAT, QCTMRLAT and QCPRDLAT registers on unit time out.</p>           |
| 1   | UTE   | R/W  | 0h    | <p>QEP unit timer enable</p> <p>Reset type: SYSRSn</p> <p>0h (R/W) = Disable eQEP unit timer</p> <p>1h (R/W) = Enable unit timer</p>                                                                                                                                                                                                                                                                                                     |
| 0   | WDE   | R/W  | 0h    | <p>QEP watchdog enable</p> <p>Reset type: SYSRSn</p> <p>0h (R/W) = Disable the eQEP watchdog timer</p> <p>1h (R/W) = Enable the eQEP watchdog timer</p>                                                                                                                                                                                                                                                                                  |

### 7.10.2.14 QCAPCTL Register (Offset = 16h) [reset = 0h]

Quadrature Capture Control

**Figure 7-34. QCAPCTL Register**

|          |          |    |    |        |    |   |   |
|----------|----------|----|----|--------|----|---|---|
| 15       | 14       | 13 | 12 | 11     | 10 | 9 | 8 |
| CEN      | RESERVED |    |    |        |    |   |   |
| R/W-0h   | R-0h     |    |    |        |    |   |   |
| 7        | 6        | 5  | 4  | 3      | 2  | 1 | 0 |
| RESERVED | CCPS     |    |    | UPPS   |    |   |   |
| R-0h     | R/W-0h   |    |    | R/W-0h |    |   |   |

**Table 7-19. QCAPCTL Register Field Descriptions**

| Bit  | Field    | Type | Reset | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|------|----------|------|-------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15   | CEN      | R/W  | 0h    | Enable eQEP capture<br>Reset type: SYSRStn<br>0h (R/W) = eQEP capture unit is disabled<br>1h (R/W) = eQEP capture unit is enabled                                                                                                                                                                                                                                                                                                                                                                                                       |
| 14-7 | RESERVED | R    | 0h    | Reserved                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| 6-4  | CCPS     | R/W  | 0h    | eQEP capture timer clock prescaler<br>Reset type: SYSRStn<br>0h (R/W) = CAPCLK = SYSCLKOUT/1<br>1h (R/W) = CAPCLK = SYSCLKOUT/2<br>2h (R/W) = CAPCLK = SYSCLKOUT/4<br>3h (R/W) = CAPCLK = SYSCLKOUT/8<br>4h (R/W) = CAPCLK = SYSCLKOUT/16<br>5h (R/W) = CAPCLK = SYSCLKOUT/32<br>6h (R/W) = CAPCLK = SYSCLKOUT/64<br>7h (R/W) = CAPCLK = SYSCLKOUT/128                                                                                                                                                                                  |
| 3-0  | UPPS     | R/W  | 0h    | Unit position event prescaler<br>Reset type: SYSRStn<br>0h (R/W) = UPEVNT = QCLK/1<br>1h (R/W) = UPEVNT = QCLK/2<br>2h (R/W) = UPEVNT = QCLK/4<br>3h (R/W) = UPEVNT = QCLK/8<br>4h (R/W) = UPEVNT = QCLK/16<br>5h (R/W) = UPEVNT = QCLK/32<br>6h (R/W) = UPEVNT = QCLK/64<br>7h (R/W) = UPEVNT = QCLK/128<br>8h (R/W) = UPEVNT = QCLK/256<br>9h (R/W) = UPEVNT = QCLK/512<br>Ah (R/W) = UPEVNT = QCLK/1024<br>Bh (R/W) = UPEVNT = QCLK/2048<br>Ch (R/W) = Reserved<br>Dh (R/W) = Reserved<br>Eh (R/W) = Reserved<br>Fh (R/W) = Reserved |

### 7.10.2.15 QPOSCTL Register (Offset = 17h) [reset = 0h]

Position Compare Control

**Figure 7-35. QPOSCTL Register**

|        |        |        |        |        |    |   |   |
|--------|--------|--------|--------|--------|----|---|---|
| 15     | 14     | 13     | 12     | 11     | 10 | 9 | 8 |
| PCSHDW | PCLOAD | PCPOL  | PCE    | PCSPW  |    |   |   |
| R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h |    |   |   |
| 7      | 6      | 5      | 4      | 3      | 2  | 1 | 0 |
| PCSPW  |        |        |        |        |    |   |   |
| R/W-0h |        |        |        |        |    |   |   |

**Table 7-20. QPOSCTL Register Field Descriptions**

| Bit  | Field  | Type | Reset | Description                                                                                                                                                                                     |
|------|--------|------|-------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15   | PCSHDW | R/W  | 0h    | Position compare of shadow enable<br>Reset type: SYSRSn<br>0h (R/W) = Shadow disabled, load Immediate<br>1h (R/W) = Shadow enabled                                                              |
| 14   | PCLOAD | R/W  | 0h    | Position compare of shadow load<br>Reset type: SYSRSn<br>0h (R/W) = Load on QPOSCNT = 0<br>1h (R/W) = Load when QPOSCNT = QPOSCMP                                                               |
| 13   | PCPOL  | R/W  | 0h    | Polarity of sync output<br>Reset type: SYSRSn<br>0h (R/W) = Active HIGH pulse output<br>1h (R/W) = Active LOW pulse output                                                                      |
| 12   | PCE    | R/W  | 0h    | Position compare enable/disable<br>Reset type: SYSRSn<br>0h (R/W) = Disable position compare unit<br>1h (R/W) = Enable position compare unit                                                    |
| 11-0 | PCSPW  | R/W  | 0h    | Select-position-compare sync output pulse width<br>Reset type: SYSRSn<br>0h (R/W) = 1 * 4 * SYSCLKOUT cycles<br>1h (R/W) = 2 * 4 * SYSCLKOUT cycles<br>FFFh (R/W) = 4096 * 4 * SYSCLKOUT cycles |

### 7.10.2.16 QEINT Register (Offset = 18h) [reset = 0h]

QEP Interrupt Control

**Figure 7-36. QEINT Register**

| 15       | 14     | 13     | 12     | 11     | 10     | 9      | 8        |
|----------|--------|--------|--------|--------|--------|--------|----------|
| RESERVED |        |        |        | UTO    | IEL    | SEL    | PCM      |
| R-0h     |        |        |        | R/W-0h | R/W-0h | R/W-0h | R/W-0h   |
| 7        | 6      | 5      | 4      | 3      | 2      | 1      | 0        |
| PCR      | PCO    | PCU    | WTO    | QDC    | QPE    | PCE    | RESERVED |
| R/W-0h   | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R-0h     |

**Table 7-21. QEINT Register Field Descriptions**

| Bit   | Field    | Type | Reset | Description                                                                                                                               |
|-------|----------|------|-------|-------------------------------------------------------------------------------------------------------------------------------------------|
| 15-12 | RESERVED | R    | 0h    | Reserved                                                                                                                                  |
| 11    | UTO      | R/W  | 0h    | Unit time out interrupt enable<br>Reset type: SYSRSn<br>0h (R/W) = Interrupt is disabled<br>1h (R/W) = Interrupt is enabled               |
| 10    | IEL      | R/W  | 0h    | Index event latch interrupt enable<br>Reset type: SYSRSn<br>0h (R/W) = Interrupt is disabled<br>1h (R/W) = Interrupt is enabled           |
| 9     | SEL      | R/W  | 0h    | Strobe event latch interrupt enable<br>Reset type: SYSRSn<br>0h (R/W) = Interrupt is disabled<br>1h (R/W) = Interrupt is enabled          |
| 8     | PCM      | R/W  | 0h    | Position-compare match interrupt enable<br>Reset type: SYSRSn<br>0h (R/W) = Interrupt is disabled<br>1h (R/W) = Interrupt is enabled      |
| 7     | PCR      | R/W  | 0h    | Position-compare ready interrupt enable<br>Reset type: SYSRSn<br>0h (R/W) = Interrupt is disabled<br>1h (R/W) = Interrupt is enabled      |
| 6     | PCO      | R/W  | 0h    | Position counter overflow interrupt enable<br>Reset type: SYSRSn<br>0h (R/W) = Interrupt is disabled<br>1h (R/W) = Interrupt is enabled   |
| 5     | PCU      | R/W  | 0h    | Position counter underflow interrupt enable<br>Reset type: SYSRSn<br>0h (R/W) = Interrupt is disabled<br>1h (R/W) = Interrupt is enabled  |
| 4     | WTO      | R/W  | 0h    | Watchdog time out interrupt enable<br>Reset type: SYSRSn<br>0h (R/W) = Interrupt is disabled<br>1h (R/W) = Interrupt is enabled           |
| 3     | QDC      | R/W  | 0h    | Quadrature direction change interrupt enable<br>Reset type: SYSRSn<br>0h (R/W) = Interrupt is disabled<br>1h (R/W) = Interrupt is enabled |

**Table 7-21. QEINT Register Field Descriptions (continued)**

| Bit | Field    | Type | Reset | Description                                                                                                                          |
|-----|----------|------|-------|--------------------------------------------------------------------------------------------------------------------------------------|
| 2   | QPE      | R/W  | 0h    | Quadrature phase error interrupt enable<br>Reset type: SYSRSn<br>0h (R/W) = Interrupt is disabled<br>1h (R/W) = Interrupt is enabled |
| 1   | PCE      | R/W  | 0h    | Position counter error interrupt enable<br>Reset type: SYSRSn<br>0h (R/W) = Interrupt is disabled<br>1h (R/W) = Interrupt is enabled |
| 0   | RESERVED | R    | 0h    | Reserved                                                                                                                             |

### 7.10.2.17 QFLG Register (Offset = 19h) [reset = 0h]

QEP Interrupt Flag

**Figure 7-37. QFLG Register**

| 15       | 14   | 13   | 12   | 11   | 10   | 9    | 8    |
|----------|------|------|------|------|------|------|------|
| RESERVED |      |      |      | UTO  | IEL  | SEL  | PCM  |
| R-0h     |      |      |      | R-0h | R-0h | R-0h | R-0h |
| 7        | 6    | 5    | 4    | 3    | 2    | 1    | 0    |
| PCR      | PCO  | PCU  | WTO  | QDC  | PHE  | PCE  | INT  |
| R-0h     | R-0h | R-0h | R-0h | R-0h | R-0h | R-0h | R-0h |

**Table 7-22. QFLG Register Field Descriptions**

| Bit   | Field    | Type | Reset | Description                                                                                                                                 |
|-------|----------|------|-------|---------------------------------------------------------------------------------------------------------------------------------------------|
| 15-12 | RESERVED | R    | 0h    | Reserved                                                                                                                                    |
| 11    | UTO      | R    | 0h    | Unit time out interrupt flag<br>Reset type: SYSRSn<br>0h (R/W) = No interrupt generated<br>1h (R/W) = Interrupt was generated               |
| 10    | IEL      | R    | 0h    | Index event latch interrupt flag<br>Reset type: SYSRSn<br>0h (R/W) = No interrupt generated<br>1h (R/W) = Interrupt was generated           |
| 9     | SEL      | R    | 0h    | Strobe event latch interrupt flag<br>Reset type: SYSRSn<br>0h (R/W) = No interrupt generated<br>1h (R/W) = Interrupt was generated          |
| 8     | PCM      | R    | 0h    | eQEP compare match event interrupt flag<br>Reset type: SYSRSn<br>0h (R/W) = No interrupt generated<br>1h (R/W) = Interrupt was generated    |
| 7     | PCR      | R    | 0h    | Position-compare ready interrupt flag<br>Reset type: SYSRSn<br>0h (R/W) = No interrupt generated<br>1h (R/W) = Interrupt was generated      |
| 6     | PCO      | R    | 0h    | Position counter overflow interrupt flag<br>Reset type: SYSRSn<br>0h (R/W) = No interrupt generated<br>1h (R/W) = Interrupt was generated   |
| 5     | PCU      | R    | 0h    | Position counter underflow interrupt flag<br>Reset type: SYSRSn<br>0h (R/W) = No interrupt generated<br>1h (R/W) = Interrupt was generated  |
| 4     | WTO      | R    | 0h    | Watchdog timeout interrupt flag<br>Reset type: SYSRSn<br>0h (R/W) = No interrupt generated<br>1h (R/W) = Interrupt was generated            |
| 3     | QDC      | R    | 0h    | Quadrature direction change interrupt flag<br>Reset type: SYSRSn<br>0h (R/W) = No interrupt generated<br>1h (R/W) = Interrupt was generated |

**Table 7-22. QFLG Register Field Descriptions (continued)**

| Bit | Field | Type | Reset | Description                                                                                                                            |
|-----|-------|------|-------|----------------------------------------------------------------------------------------------------------------------------------------|
| 2   | PHE   | R    | 0h    | Quadrature phase error interrupt flag<br>Reset type: SYSRSn<br>0h (R/W) = No interrupt generated<br>1h (R/W) = Interrupt was generated |
| 1   | PCE   | R    | 0h    | Position counter error interrupt flag<br>Reset type: SYSRSn<br>0h (R/W) = No interrupt generated<br>1h (R/W) = Interrupt was generated |
| 0   | INT   | R    | 0h    | Global interrupt status flag<br>Reset type: SYSRSn<br>0h (R/W) = No interrupt generated<br>1h (R/W) = Interrupt was generated          |

### 7.10.2.18 QCLR Register (Offset = 1Ah) [reset = 0h]

QEP Interrupt Clear

**Figure 7-38. QCLR Register**

| 15         | 14         | 13         | 12         | 11         | 10         | 9          | 8          |
|------------|------------|------------|------------|------------|------------|------------|------------|
| RESERVED   |            |            |            | UTO        | IEL        | SEL        | PCM        |
| R-0h       |            |            |            | R-0/W1S-0h | R-0/W1S-0h | R-0/W1S-0h | R-0/W1S-0h |
| 7          | 6          | 5          | 4          | 3          | 2          | 1          | 0          |
| PCR        | PCO        | PCU        | WTO        | QDC        | PHE        | PCE        | INT        |
| R-0/W1S-0h | R-0/W1S-0h | R-0/W1S-0h | R-0/W1S-0h | R-0/W1S-0h | R-0/W1S-0h | R-0/W1S-0h | R-0/W1S-0h |

**Table 7-23. QCLR Register Field Descriptions**

| Bit   | Field    | Type    | Reset | Description                                                                                                                            |
|-------|----------|---------|-------|----------------------------------------------------------------------------------------------------------------------------------------|
| 15-12 | RESERVED | R       | 0h    | Reserved                                                                                                                               |
| 11    | UTO      | R-0/W1S | 0h    | Clear unit time out interrupt flag<br>Reset type: SYSRSn<br>0h (R/W) = No effect<br>1h (R/W) = Clears the interrupt flag               |
| 10    | IEL      | R-0/W1S | 0h    | Clear index event latch interrupt flag<br>Reset type: SYSRSn<br>0h (R/W) = No effect<br>1h (R/W) = Clears the interrupt flag           |
| 9     | SEL      | R-0/W1S | 0h    | Clear strobe event latch interrupt flag<br>Reset type: SYSRSn<br>0h (R/W) = No effect<br>1h (R/W) = Clears the interrupt flag          |
| 8     | PCM      | R-0/W1S | 0h    | Clear eQEP compare match event interrupt flag<br>Reset type: SYSRSn<br>0h (R/W) = No effect<br>1h (R/W) = Clears the interrupt flag    |
| 7     | PCR      | R-0/W1S | 0h    | Clear position-compare ready interrupt flag<br>Reset type: SYSRSn<br>0h (R/W) = No effect<br>1h (R/W) = Clears the interrupt flag      |
| 6     | PCO      | R-0/W1S | 0h    | Clear position counter overflow interrupt flag<br>Reset type: SYSRSn<br>0h (R/W) = No effect<br>1h (R/W) = Clears the interrupt flag   |
| 5     | PCU      | R-0/W1S | 0h    | Clear position counter underflow interrupt flag<br>Reset type: SYSRSn<br>0h (R/W) = No effect<br>1h (R/W) = Clears the interrupt flag  |
| 4     | WTO      | R-0/W1S | 0h    | Clear watchdog timeout interrupt flag<br>Reset type: SYSRSn<br>0h (R/W) = No effect<br>1h (R/W) = Clears the interrupt flag            |
| 3     | QDC      | R-0/W1S | 0h    | Clear quadrature direction change interrupt flag<br>Reset type: SYSRSn<br>0h (R/W) = No effect<br>1h (R/W) = Clears the interrupt flag |



**Table 7-23. QCLR Register Field Descriptions (continued)**

| Bit | Field | Type    | Reset | Description                                                                                                                       |
|-----|-------|---------|-------|-----------------------------------------------------------------------------------------------------------------------------------|
| 2   | PHE   | R-0/W1S | 0h    | Clear quadrature phase error interrupt flag<br>Reset type: SYSRSn<br>0h (R/W) = No effect<br>1h (R/W) = Clears the interrupt flag |
| 1   | PCE   | R-0/W1S | 0h    | Clear position counter error interrupt flag<br>Reset type: SYSRSn<br>0h (R/W) = No effect<br>1h (R/W) = Clears the interrupt flag |
| 0   | INT   | R-0/W1S | 0h    | Global interrupt clear flag<br>Reset type: SYSRSn<br>0h (R/W) = No effect<br>1h (R/W) = Clears the interrupt flag                 |

### 7.10.2.19 QFRC Register (Offset = 1Bh) [reset = 0h]

QEP Interrupt Force

**Figure 7-39. QFRC Register**

| 15       | 14     | 13     | 12     | 11     | 10     | 9      | 8        |
|----------|--------|--------|--------|--------|--------|--------|----------|
| RESERVED |        |        |        | UTO    | IEL    | SEL    | PCM      |
| R-0h     |        |        |        | R/W-0h | R/W-0h | R/W-0h | R/W-0h   |
| 7        | 6      | 5      | 4      | 3      | 2      | 1      | 0        |
| PCR      | PCO    | PCU    | WTO    | QDC    | PHE    | PCE    | RESERVED |
| R/W-0h   | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R-0h     |

**Table 7-24. QFRC Register Field Descriptions**

| Bit   | Field    | Type | Reset | Description                                                                                                                 |
|-------|----------|------|-------|-----------------------------------------------------------------------------------------------------------------------------|
| 15-12 | RESERVED | R    | 0h    | Reserved                                                                                                                    |
| 11    | UTO      | R/W  | 0h    | Force unit time out interrupt<br>Reset type: SYSRSn<br>0h (R/W) = No effect<br>1h (R/W) = Force the interrupt               |
| 10    | IEL      | R/W  | 0h    | Force index event latch interrupt<br>Reset type: SYSRSn<br>0h (R/W) = No effect<br>1h (R/W) = Force the interrupt           |
| 9     | SEL      | R/W  | 0h    | Force strobe event latch interrupt<br>Reset type: SYSRSn<br>0h (R/W) = No effect<br>1h (R/W) = Force the interrupt          |
| 8     | PCM      | R/W  | 0h    | Force position-compare match interrupt<br>Reset type: SYSRSn<br>0h (R/W) = No effect<br>1h (R/W) = Force the interrupt      |
| 7     | PCR      | R/W  | 0h    | Force position-compare ready interrupt<br>Reset type: SYSRSn<br>0h (R/W) = No effect<br>1h (R/W) = Force the interrupt      |
| 6     | PCO      | R/W  | 0h    | Force position counter overflow interrupt<br>Reset type: SYSRSn<br>0h (R/W) = No effect<br>1h (R/W) = Force the interrupt   |
| 5     | PCU      | R/W  | 0h    | Force position counter underflow interrupt<br>Reset type: SYSRSn<br>0h (R/W) = No effect<br>1h (R/W) = Force the interrupt  |
| 4     | WTO      | R/W  | 0h    | Force watchdog time out interrupt<br>Reset type: SYSRSn<br>0h (R/W) = No effect<br>1h (R/W) = Force the interrupt           |
| 3     | QDC      | R/W  | 0h    | Force quadrature direction change interrupt<br>Reset type: SYSRSn<br>0h (R/W) = No effect<br>1h (R/W) = Force the interrupt |

**Table 7-24. QFRC Register Field Descriptions (continued)**

| Bit | Field    | Type | Reset | Description                                                                                                            |
|-----|----------|------|-------|------------------------------------------------------------------------------------------------------------------------|
| 2   | PHE      | R/W  | 0h    | Force quadrature phase error interrupt<br>Reset type: SYSRSn<br>0h (R/W) = No effect<br>1h (R/W) = Force the interrupt |
| 1   | PCE      | R/W  | 0h    | Force position counter error interrupt<br>Reset type: SYSRSn<br>0h (R/W) = No effect<br>1h (R/W) = Force the interrupt |
| 0   | RESERVED | R    | 0h    | Reserved                                                                                                               |

### 7.10.2.20 QEPSTS Register (Offset = 1Ch) [reset = 0h]

QEP Status

**Figure 7-40. QEPSTS Register**

| 15       | 14   | 13   | 12   | 11     | 10     | 9      | 8    |
|----------|------|------|------|--------|--------|--------|------|
| RESERVED |      |      |      |        |        |        |      |
| R-0h     |      |      |      |        |        |        |      |
| 7        | 6    | 5    | 4    | 3      | 2      | 1      | 0    |
| UPEVNT   | FIDF | QDF  | QDLF | COEF   | CDEF   | FIMF   | PCEF |
| R/W-0h   | R-0h | R-0h | R-0h | R/W-0h | R/W-0h | R/W-0h | R-0h |

**Table 7-25. QEPSTS Register Field Descriptions**

| Bit  | Field    | Type | Reset | Description                                                                                                                                                                                                                                                                                                  |
|------|----------|------|-------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-8 | RESERVED | R    | 0h    | Reserved                                                                                                                                                                                                                                                                                                     |
| 7    | UPEVNT   | R/W  | 0h    | Unit position event flag<br>Reset type: SYSRSn<br>0h (R/W) = No unit position event detected<br>1h (R/W) = Unit position event detected. Write 1 to clear                                                                                                                                                    |
| 6    | FIDF     | R    | 0h    | Direction on the first index marker<br>Status of the direction is latched on the first index event marker.<br>Reset type: SYSRSn<br>0h (R/W) = Counter-clockwise rotation (or reverse movement) on the first index event<br>1h (R/W) = Clockwise rotation (or forward movement) on the first index event     |
| 5    | QDF      | R    | 0h    | Quadrature direction flag<br>Reset type: SYSRSn<br>0h (R/W) = Counter-clockwise rotation (or reverse movement)<br>1h (R/W) = Clockwise rotation (or forward movement)                                                                                                                                        |
| 4    | QDLF     | R    | 0h    | eQEP direction latch flag<br>Reset type: SYSRSn<br>0h (R/W) = Counter-clockwise rotation (or reverse movement) on index event marker<br>1h (R/W) = Clockwise rotation (or forward movement) on index event marker                                                                                            |
| 3    | COEF     | R/W  | 0h    | Capture overflow error flag<br>Reset type: SYSRSn<br>0h (R/W) = Overflow has not occurred.<br>1h (R/W) = Overflow occurred in eQEP Capture timer (QEPCTMR).                                                                                                                                                  |
| 2    | CDEF     | R/W  | 0h    | Capture direction error flag<br>Reset type: SYSRSn<br>0h (R/W) = Capture direction error has not occurred.<br>1h (R/W) = Direction change occurred between the capture position event.                                                                                                                       |
| 1    | FIMF     | R/W  | 0h    | First index marker flag<br>Note: Once this flag has been set, if the flag is cleared the flag will not be set again until the module is reset by a peripheral or system reset.<br>Reset type: SYSRSn<br>0h (R/W) = First index pulse has not occurred.<br>1h (R/W) = Set by first occurrence of index pulse. |

**Table 7-25. QEPSTS Register Field Descriptions (continued)**

| Bit | Field | Type | Reset | Description                                                                                                                                                                                                                 |
|-----|-------|------|-------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 0   | PCEF  | R    | 0h    | Position counter error flag.<br>This bit is not sticky and it is updated for every index event.<br>Reset type: SYSRSn<br>0h (R/W) = No error occurred during the last index transition<br>1h (R/W) = Position counter error |

#### 7.10.2.21 QCTMR Register (Offset = 1Dh) [reset = 0h]

QEP Capture Timer

**Figure 7-41. QCTMR Register**

|        |    |    |    |    |    |   |   |
|--------|----|----|----|----|----|---|---|
| 15     | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
| QCTMR  |    |    |    |    |    |   |   |
| R/W-0h |    |    |    |    |    |   |   |
| 7      | 6  | 5  | 4  | 3  | 2  | 1 | 0 |
| QCTMR  |    |    |    |    |    |   |   |
| R/W-0h |    |    |    |    |    |   |   |

**Table 7-26. QCTMR Register Field Descriptions**

| Bit  | Field | Type | Reset | Description                                                                   |
|------|-------|------|-------|-------------------------------------------------------------------------------|
| 15-0 | QCTMR | R/W  | 0h    | This register provides time base for edge capture unit.<br>Reset type: SYSRSn |

#### 7.10.2.22 QCPRD Register (Offset = 1Eh) [reset = 0h]

QEP Capture Period

**Figure 7-42. QCPRD Register**

|        |    |    |    |    |    |   |   |
|--------|----|----|----|----|----|---|---|
| 15     | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
| QCPRD  |    |    |    |    |    |   |   |
| R/W-0h |    |    |    |    |    |   |   |
| 7      | 6  | 5  | 4  | 3  | 2  | 1 | 0 |
| QCPRD  |    |    |    |    |    |   |   |
| R/W-0h |    |    |    |    |    |   |   |

**Table 7-27. QCPRD Register Field Descriptions**

| Bit  | Field | Type | Reset | Description                                                                                                       |
|------|-------|------|-------|-------------------------------------------------------------------------------------------------------------------|
| 15-0 | QCPRD | R/W  | 0h    | This register holds the period count value between the last successive eQEP position events<br>Reset type: SYSRSn |

### 7.10.2.23 QCTMRLAT Register (Offset = 1Fh) [reset = 0h]

QEP Capture Latch

**Figure 7-43. QCTMRLAT Register**

|          |    |    |    |    |    |   |   |
|----------|----|----|----|----|----|---|---|
| 15       | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
| QCTMRLAT |    |    |    |    |    |   |   |
| R-0h     |    |    |    |    |    |   |   |
| 7        | 6  | 5  | 4  | 3  | 2  | 1 | 0 |
| QCTMRLAT |    |    |    |    |    |   |   |
| R-0h     |    |    |    |    |    |   |   |

**Table 7-28. QCTMRLAT Register Field Descriptions**

| Bit  | Field    | Type | Reset | Description                                                                                                                                                     |
|------|----------|------|-------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-0 | QCTMRLAT | R    | 0h    | The eQEP capture timer value can be latched into this register on two events viz., unit timeout event, reading the eQEP position counter.<br>Reset type: SYSRSn |

### 7.10.2.24 QCPRDLAT Register (Offset = 20h) [reset = 0h]

QEP Capture Period Latch

**Figure 7-44. QCPRDLAT Register**

|          |    |    |    |    |    |   |   |
|----------|----|----|----|----|----|---|---|
| 15       | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
| QCPRDLAT |    |    |    |    |    |   |   |
| R-0h     |    |    |    |    |    |   |   |
| 7        | 6  | 5  | 4  | 3  | 2  | 1 | 0 |
| QCPRDLAT |    |    |    |    |    |   |   |
| R-0h     |    |    |    |    |    |   |   |

**Table 7-29. QCPRDLAT Register Field Descriptions**

| Bit  | Field    | Type | Reset | Description                                                                                                                                                  |
|------|----------|------|-------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-0 | QCPRDLAT | R    | 0h    | eQEP capture period value can be latched into this register on two events viz., unit timeout event, reading the eQEP position counter.<br>Reset type: SYSRSn |

## Analog-to-Digital Converter (ADC)

The Analog-to-Digital Converter (ADC) module is a Type 3 ADC. See the [C2000 Real-Time Control Peripherals Reference Guide](#) for a list of all devices with modules of the same type, to determine the differences between the types, and for a list of device-specific differences within a type.

|                                                         |     |
|---------------------------------------------------------|-----|
| 8.1 Introduction.....                                   | 510 |
| 8.2 SOC Principle of Operation.....                     | 513 |
| 8.3 ONESHOT Single Conversion Support.....              | 519 |
| 8.4 ADC Conversion Priority.....                        | 520 |
| 8.5 Sequential Sampling Mode.....                       | 523 |
| 8.6 Simultaneous Sampling Mode.....                     | 523 |
| 8.7 EOC and Interrupt Operation.....                    | 524 |
| 8.8 Power-Up Sequence.....                              | 525 |
| 8.9 ADC Calibration.....                                | 525 |
| 8.10 Internal/External Reference Voltage Selection..... | 526 |
| 8.11 ADC Timings.....                                   | 527 |
| 8.12 Internal Temperature Sensor.....                   | 532 |
| 8.13 ADC Registers.....                                 | 534 |

## 8.1 Introduction

The ADC module described in this chapter is a 12-bit recyclic ADC; part SAR, part pipelined. The analog circuits of this converter, referred to as the "core" in this document, include the front-end analog multiplexers (MUXs), sample-and-hold (S+H) circuits, the conversion core, voltage regulators, and other analog supporting circuits. Digital circuits, referred to as the "wrapper" in this document, include programmable conversions, result registers, interface to analog circuits, interface to device peripheral bus, and interface to other on-chip modules.

### 8.1.1 Features

The core of the ADC contains a single 12-bit converter fed by two sample and hold circuits. The sample and hold circuits can be sampled simultaneously or sequentially. These, in turn, are fed by a total of up to 16 analog input channels. See your device-specific data sheet for the specific number of channels available. The converter can be configured to run with an internal bandgap reference to create true-voltage based conversions or with a pair of external voltage references (VREFHI/LO) to create ratiometric based conversions.

Contrary to previous ADC types, this ADC is not sequencer based. It is easy for the user to create a series of conversions from a single trigger. However, the basic principle of operation is centered around the configurations of individual conversions, called SOC's, or Start-Of-Conversions.

Functions of the ADC module include:

- 12-bit ADC core with built-in dual sample-and-hold (S+H)
- Simultaneous sampling or sequential sampling modes
- Full range analog input: 0 V to 3.3 V fixed, or VREFHI/VREFLO ratiometric
- Up to 16-channel, multiplexed inputs
- 16 SOC's, configurable for trigger, sample window, and channel
- 16 result registers (individually addressable) to store conversion values
- Multiple trigger sources
  - S/W - software immediate start
  - ePWM 1-8
  - GPIO XINT2
  - CPU Timers 0/1/2
  - ADCINT1/2
- 9 flexible PIE interrupts, can configure interrupt request after any conversion

### 8.1.2 ADC Related Collateral

#### Foundational Materials

- [ADC Input Circuit Evaluation for C2000 MCUs \(TINA-TI\) Application Report](#)
- [PSpice for TI design and simulation tool](#)
- [Real-Time Control Reference Guide](#)
  - Refer to the ADC section
- [TI Precision Labs - ADCs](#) (Video)
  - Start with the "Introduction to analog-to-digital converters (ADCs)" section.
- [TI Precision Labs - ADCs](#)
- [TI Precision Labs: Driving the reference input on a SAR ADC](#) (Video)
- [TI Precision Labs: Introduction to analog-to-digital converters \(ADCs\)](#) (Video)
- [TI Precision Labs: SAR ADC input driver design](#) (Video)
- [TI e2e: Connecting VDDA to VREFHI](#)
- [TI e2e: Topologies for ADC Input Protection](#)
- [TI e2e: Why does the ADC Input Voltage drop with sampling?](#)
  - Sampling a high impedance voltage divider with ADC
- [Understanding Data Converters Application Report](#)



**Getting Started Materials**

- [ADC-PWM Synchronization Using ADC Interrupt](#)
  - NOTE: This is a non-TI (third party) site.
- [C2000 ADC \(Type-3\) Performance Versus ACQPS Application Report](#)
- [Hardware Design Guide for F2800x C2000 Real-Time MCU Series](#)

**Expert Materials**

- [Analog Engineer's Calculator](#)
- [Analog Engineer's Pocket Reference](#)
- [Debugging an integrated ADC in a microcontroller using an oscilloscope](#)
- [TI Precision Labs: ADC AC specifications \(Video\)](#)
- [TI Precision Labs: ADC Error sources \(Video\)](#)
- [TI Precision Labs: ADC Noise \(Video\)](#)
- [TI Precision Labs: Analog-to-digital converter \(ADC\) drive topologies \(Video\)](#)
- [TI Precision Labs: Electrical overstress on data converters \(Video\)](#)
- [TI Precision Labs: High-speed ADC fundamentals \(Video\)](#)
- [TI Precision Labs: SAR & Delta-Sigma: Understanding the Difference \(Video\)](#)
- [TI e2e: ADC Bandwidth Clarification](#)
- [TI e2e: ADC Calibration and Total Unadjusted Error](#)
- [TI e2e: ADC Reference Driver Options](#)
- [TI e2e: ADC Resolution with Oversampling](#)
- [TI e2e: ADC configuration for interleaved mode](#)
- [TI e2e: Simultaneous Sampling with Single ADC](#)

### 8.1.3 Block Diagram

Figure 8-1 shows the block diagram of the ADC module.

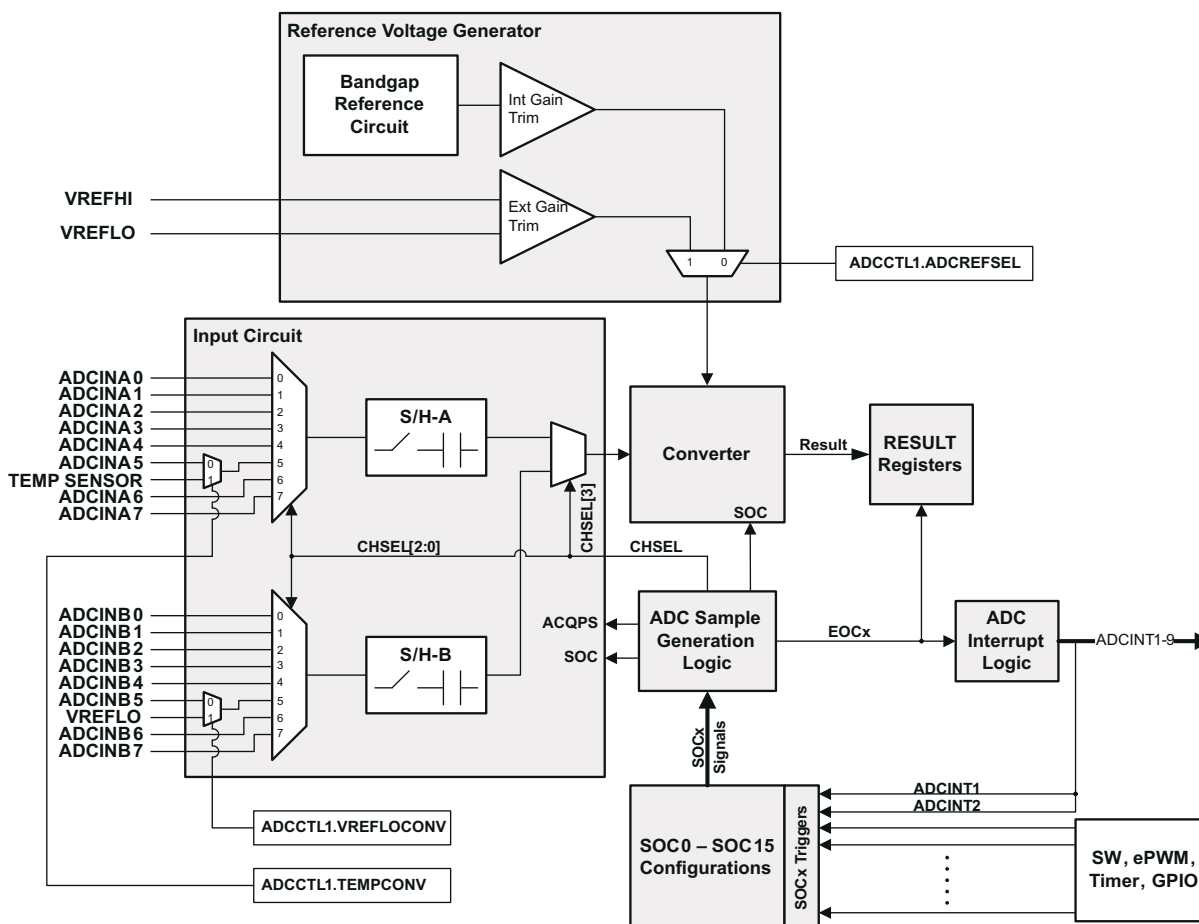
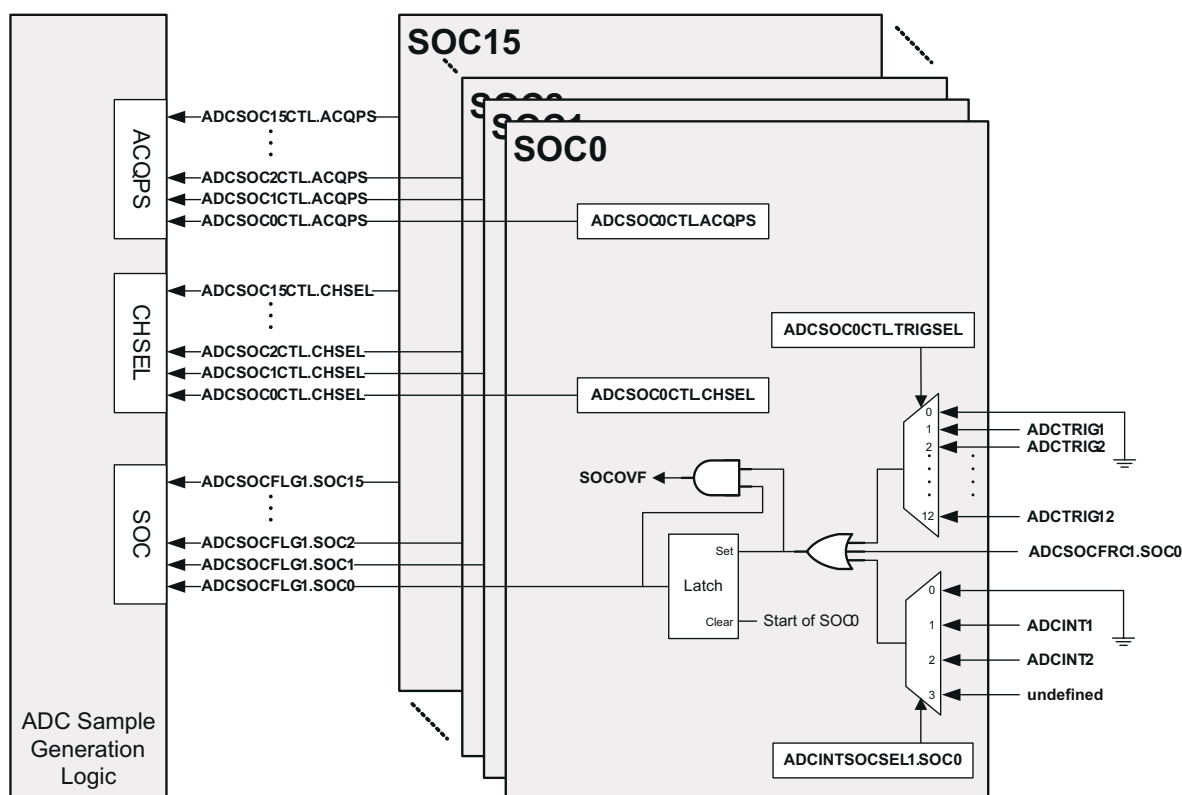


Figure 8-1. ADC Block Diagram

## 8.2 SOC Principle of Operation

Contrary to previous ADC types, this ADC is not sequencer based. Instead, it is SOC based. The term SOC is configuration set defining the single conversion of a single channel. In that set there are three configurations: the trigger source that starts the conversion, the channel to convert, and the acquisition (sample) window size. Each SOC is independently configured and can have any combination of the trigger, channel, and sample window size available. Multiple SOC's can be configured for the same trigger, channel, and/or acquisition window as desired. This provides a very flexible means of configuring conversions ranging from individual samples of different channels with different triggers, to oversampling the same channel using a single trigger, to creating your own series of conversions of different channels all from a single trigger.

The trigger source for SOCx is configured by a combination of the TRIGSEL field in the ADCSOCxCTL register and the appropriate bits in the ADCINTSOCSEL1 or ADCINTSOCSEL2 register. Software can also force an SOC event with the ADCSOCFRC1 register. The channel and sample window size for SOCx are configured with the CHSEL and ACQPS fields of the ADCSOCxCTL register.



**Figure 8-2. SOC Block Diagram**

For example, to configure a single conversion on channel ADCINA1 to occur when the ePWM3 timer reaches its period match, you must first setup ePWM3 to output an SOCA or SOCBA signal on a period match (see [Chapter 3](#)). In this case, we will use SOCA. Then, set up one of the SOC's using its ADCSOCxCTL register. It makes no difference which SOC we choose, so we will use SOC0. The fastest allowable sample window for the ADC is 7 cycles. Choosing the fastest time for the sample window, channel ADCINA1 for the channel to convert, and ePWM3 for the SOC0 trigger, we'll set the ACQPS field to 6, the CHSEL field to 1, and the TRIGSEL field to 9, respectively. The resulting value written into the register is:

```
ADCSOC0CTL = 4846h; // (ACQPS=6, CHSEL=1, TRIGSEL=9)
```

When configured as such, a single conversion of ADCINA1 will be started on an ePWM3 SOCA event with the resulting value stored in the ADCRESULT0 register.

If instead ADCINA1 needed to be oversampled by 3X, then SOC1, SOC2, and SOC3 could all be given the same configuration as SOC0.

```
ADCSOC1CTL = 4846h;           // (ACQPS=6, CHSEL=1, TRIGSEL=9)
ADCSOC2CTL = 4846h;           // (ACQPS=6, CHSEL=1, TRIGSEL=9)
ADCSOC3CTL = 4846h;           // (ACQPS=6, CHSEL=1, TRIGSEL=9)
```

When configured as such, four conversions of ADCINA1 will be started in series on an ePWM3 SOCA event with the resulting values stored in the ADCRESULT0 – ADCRESULT3 registers.

Another application may require 3 different signals to be sampled from the same trigger. This can be done by simply changing the CHSEL field for SOC0-SOC2 while leaving the TRIGSEL field unchanged.

```
ADCSOC0CTL = 4846h;           // (ACQPS=6, CHSEL=1, TRIGSEL=9)
ADCSOC1CTL = 4886h;           // (ACQPS=6, CHSEL=2, TRIGSEL=9)
ADCSOC2CTL = 48C6h;           // (ACQPS=6, CHSEL=3, TRIGSEL=9)
```

When configured this way, three conversions will be started in series on an ePWM3 SOCA event. The result of the conversion on channel ADCINA1 will show up in ADCRESULT0. The result of the conversion on channel ADCINA2 will show up in ADCRESULT1. The result of the conversion on channel ADCINA3 will show up in ADCRESULT2. The channel converted and the trigger have no bearing on where the result of the conversion shows up. The RESULT register is associated with the SOC.

#### Note

These examples are incomplete. Clocks must be enabled via the PCLKCR0 register and the ADC must be powered to work correctly. For a description of the PCLKCR0 register, see the *System Control and Interrupts* chapter. For the power-up sequence of the ADC, see [Section 8.8](#). The CLKDIV2EN bit in the ADCCTL2 register must also be set to a proper value to obtain correct frequency of operation. For more information on the ADCCTL2 register, refer to [Section 8.13](#).

### 8.2.1 ADC Acquisition (Sample and Hold) Window

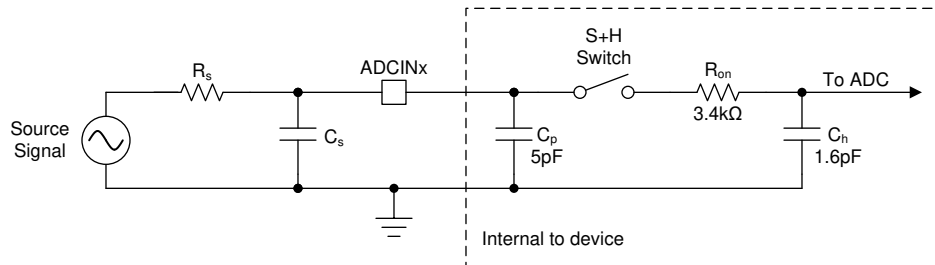
External drivers vary in their ability to drive an analog signal quickly and effectively. Some circuits require longer times to properly transfer the charge into the sampling capacitor of an ADC. To address this, the ADC supports control over the sample window length for each individual SOC configuration. Each ADCSOCxCTL register has a 6-bit field, ACQPS, that determines the sample and hold (S+H) window size. The value written to this field is one less than the number of cycles desired for the sampling window for that SOC. Thus, a value of 15 in this field will give 16 clock cycles of sample time. The minimum number of sample cycles allowed is 7 (ACQPS=6). The total sampling time is found by adding the sample window size to the conversion time of the ADC, 13 ADC clocks. Examples of various sample times are shown in [Table 8-1](#).

**Table 8-1. Sample Timings with Different Values of ACQPS**

| SYSCLKOUT | ADC Clock | ACQPS | Sample Window | Conversion Time<br>(13 cycles) | Total Time to Process<br>Analog Voltage <sup>(1)</sup> |
|-----------|-----------|-------|---------------|--------------------------------|--------------------------------------------------------|
| 90 MHz    | 45 MHz    | 6     | 155.56 ns     | 288.89 ns                      | 444.44 ns                                              |
| 90 MHz    | 45 MHz    | 25    | 577.78 ns     | 288.89 ns                      | 866.67 ns                                              |

(1) The total times are for a single conversion and do not include pipelining effects that increase the average speed over time.

As shown in [Figure 8-3](#), the ADCIN pins can be modeled as an RC circuit. With VREFLO connected to ground, a voltage swing from 0 to 3.3 V on ADCIN yields a typical RC time constant of 2 ns.



**Figure 8-3. ADCINx Input Model**

**Note**

The ADC does not precondition the Ch capacitor voltage before conversions, therefore the following behaviors apply:

1. There is no predetermined ADC conversion value when the ADCIN pin is not connected to a Source Signal.
2. Residual charge will remain on Ch between ADC conversions.
3. Sequential conversions may suffer from cross-talk if the ACQPS window is too short for Ch to settle.

For correct operation, the input signal to the ADC must be allowed adequate time to charge the sample and hold capacitor,  $C_h$ . Typically, the S+H duration is chosen such that  $C_h$  will be charged to within  $\frac{1}{2}$  LSB or  $\frac{1}{4}$  LSB of the final value, depending on the tolerable settling error.

The S+H time required to satisfy the settling error is largely influenced by the bandwidth of the source signal. Therefore, the following recommendations for approximating the S+H duration will be simplified into two practical scenarios of either high bandwidth or low bandwidth signals. A high bandwidth source signal will be characterized as being able to meet the settling error and real-time requirements of the system using a supported ACQPS setting. A low bandwidth source signal is one that requires a longer S+H duration than is acceptable.

### 8.2.1.1 ACQPS Approximation for High Bandwidth Signals

Signals that must be sampled frequently with minimal phase delay (such as feedback sensors used in control-loop calculations) are high bandwidth signals. These signal paths require a small  $R_s C_s$  time constant as seen by the ADCINx pin. An external signal buffer (such as an op-amp) may be used to boost the sampling bandwidth; such buffers should ideally have a bandwidth that is high enough to fully charge  $C_h$  within the selected ACQPS S+H window.

#### 8.2.1.1.1 ACQPS Approximation Equations for High Bandwidth Signals

An approximation of the required settling time can be determined using an RC settling model. The time constant ( $\tau$ ) for the model is given by the equation:

$$\tau = (R_s + R_{on})(C_h) + (R_s)(C_s + C_p)$$

And the number of time constants needed is given by the equation:

$$k = \ln\left(\frac{2^n}{\text{settling error}}\right) - \ln\left(\frac{C_s + C_p}{C_h}\right)$$

So the total S+H time ( $t_{s+h}$ ) should be set to at least:

$$t_{s+h} = k \cdot \tau$$

Finally,  $t_{S+H}$  is used to determine the minimum value to program into the ACQPS field of the ADCSOCxCTL registers:

$$ACQPS = (t_{S+H} \cdot f_{ADCCLK}) - 1$$

Where the following parameters are provided by the ADC input model:

- $n$  = ADC resolution (in bits)
- $R_{on}$  = ADC sampling switch resistance
- $C_h$  = ADC sampling capacitor
- $C_p$  = ADC parasitic pin capacitance for the channel

And the following parameters are dependent on the application design:

- settling error = tolerable settling error (in LSBs)
- $R_s$  = ADC driving circuit source impedance
- $C_s$  = ADC driving circuit source capacitance on ADC input pin
- $f_{ADCCLK}$  = ADC clock frequency

#### 8.2.1.1.2 ACQPS Approximation Example for High Bandwidth Signals

For example, assuming the following parameters:

- $n$  = 12-bits
- settling error =  $\frac{1}{4}$  LSB
- $R_{on} = 3400\Omega$
- $C_h = 1.6pF$
- $C_p = 5pF$
- $R_s = 56\Omega$
- $C_s = 2.2nF$
- $f_{ADCCLK} = 30MHz$

The time constant would be calculated as:

$$\tau = (56\Omega + 3400\Omega)(1.6pF) + (56\Omega)(2200pF + 5pF) = 5.5ns + 123.5ns = 129ns$$

And the number of required time constants would be:

$$k = \ln\left(\frac{2^{12} \text{ LSB}}{0.25 \text{ LSB}}\right) - \ln\left(\frac{2200pF + 5pF}{1.6pF}\right) = 9.7 - 7.2 = 2.5$$

So the S+H time should be set to at least:

$$t_{S+H} = 2.5 \cdot 129ns = 322.5ns$$

Finally, the minimum ACQPS value is calculated and rounded up to the nearest supported value:

$$ACQPS = (322.5ns \cdot 30MHz) - 1 = 8.7 \rightarrow 9$$

While this gives a rough estimate of the required acquisition window, a better method would be to setup a circuit with the ADC input model, a model of the source impedance/capacitance, and any board parasitics in SPICE (or similar software) and simulate to verify that the sampling capacitor settles to the desired accuracy.

### 8.2.1.2 ACQPS Approximation for Low Bandwidth Signals

Signals that are sampled infrequently and are tolerant of low-pass filtering (such as ambient temperature sensors) can be treated as low bandwidth signals. A large  $C_s$  that is sized to be much larger than  $C_h$  will allow the ADC to sample the  $R_s C_s$  filtered signal quickly at the expense of increased phase delay.  $C_h$  will receive the bulk of its charge from  $C_s$  during the S+H window, and  $C_s$  will recover its charge through  $R_s$  between ADC samples.

#### 8.2.1.2.1 ACQPS Approximation Equations for Low Bandwidth Signals

The desired settling accuracy will determine the value of  $C_s$ :

$$C_s = C_h \left( \frac{2^n}{\text{settling error}} \right)$$

The desired recovery time and acceptable charge error will determine the value of  $R_s$ :

$$R_s = \frac{t_{\text{recovery}}}{n_\tau \cdot C_s}$$

With this configuration,  $C_s$  acts as the effective source signal, which simplifies the equations for calculating (k) and ( $\tau$ ) as follows:

$$\tau = R_{\text{on}} \cdot C_h$$

$$k = \ln \left( \frac{2^n}{\text{settling error}} \right)$$

So the total S+H time ( $t_{\text{S+H}}$ ) should be set to at least:

$$t_{\text{S+H}} = k \cdot \tau$$

Finally,  $t_{\text{S+H}}$  is used to determine the minimum value to program into the ACQPS field of the ADCSOCxCTL registers:

$$\text{ACQPS} = (t_{\text{S+H}} \cdot f_{\text{ADCCLK}}) - 1$$

Where the following parameters are provided by the ADC input model:

- $n$  = ADC resolution (in bits)
- $R_{\text{on}}$  = ADC sampling switch resistance
- $C_h$  = ADC sampling capacitor
- $C_p$  = ADC parasitic pin capacitance for the channel

And the following parameters are dependent on the application design:

- settling error = tolerable settling error (in LSBs)
- $t_{\text{recovery}}$  = amount of time between ADC samples
- $n_\tau$  = number of RC time constants that comprise  $t_{\text{recovery}}$
- $R_s$  = ADC driving circuit source impedance
- $C_s$  = ADC driving circuit source capacitance on ADC input pin
- $f_{\text{ADCCLK}}$  = ADC clock frequency

The selection of  $n_\tau$  will determine how much the voltage on  $C_s$  is able to recover between samples. An insufficient amount of recovery time will introduce a droop error by way of an undercharged  $C_s$ . [Table 8-2](#) shows the relationship between  $n_\tau$  and the estimated droop error.

**Table 8-2. Estimated Droop Error from  $n_T$  Value**

| $n_T$ | Droop Error<br>(% of Settling Error) | Droop Error for $C_s$ sized to<br>¼ LSB Settling Error (LSB) | Total Error (Droop Error +<br>¼ LSB Settling Error) |
|-------|--------------------------------------|--------------------------------------------------------------|-----------------------------------------------------|
| 0.25  | 352%                                 | 0.88 LSB                                                     | 1.13 LSB                                            |
| 0.50  | 154%                                 | 0.39 LSB                                                     | 0.64 LSB                                            |
| 0.75  | 90%                                  | 0.23 LSB                                                     | 0.48 LSB                                            |
| 1.00  | 58%                                  | 0.15 LSB                                                     | 0.40 LSB                                            |
| 2.00  | 16%                                  | 0.04 LSB                                                     | 0.29 LSB                                            |
| 3.00  | 5%                                   | 0.01 LSB                                                     | 0.26 LSB                                            |
| 4.00  | 2%                                   | 0.01 LSB                                                     | 0.26 LSB                                            |
| 5.00  | 1%                                   | 0.00 LSB                                                     | 0.25 LSB                                            |

### 8.2.1.2.2 ACQPS Approximation Example for Low Bandwidth Signals

For example, assuming the following parameters:

- $n = 12$ -bits
- settling error = ¼ LSB
- $t_{\text{recovery}} = 1$  ms
- $n_T = 2$
- $R_{\text{on}} = 3400 \Omega$
- $C_h = 1.6$  pF
- $C_p = 5$  pF
- $f_{\text{ADCCLK}} = 30$  MHz

The minimum source capacitance would be calculated and rounded up to a common value:

$$C_s = 1.6pF \left( \frac{2^{12} \text{ LSB}}{0.25 \text{ LSB}} \right) = 26.2nF \rightarrow 33nF$$

Then the maximum source resistance would be calculated and rounded down to a common value:

$$R_s = \frac{1ms}{2 \cdot 33nF} = 15.2k\Omega \rightarrow 12k\Omega$$

Now the time constant ( $\tau$ ) and multiple ( $k$ ) can be calculated as:

$$\tau = 3400\Omega \cdot 1.6pF = 5.4ns$$

$$k = \ln \left( \frac{2^{12} \text{ LSB}}{0.25 \text{ LSB}} \right) = 9.7$$

So the S+H time should be set to at least:

$$t_{s+h} = 9.7 \cdot 5.4ns = 52.4ns$$

Finally, the minimum ACQPS value is calculated and rounded up to the nearest supported value:

$$ACQPS = (52.4ns \cdot 30MHz) - 1 = 0.6 \rightarrow 6$$

While this gives a rough estimate of the required acquisition window, a better method would be to setup a circuit with the ADC input model, a model of the source impedance/capacitance, and any board parasitics in SPICE (or similar software) and simulate to verify that the sampling capacitor settles to the desired accuracy.



### 8.2.2 Trigger Operation

Each SOC can be configured to start on one of many input triggers. Multiple SOC's can be configured for the same channel if desired. Following is a list of the available input triggers:

- Software
- CPU Timers 0/1/2 interrupts
- XINT2 SOC
- ePWM1-8 SOCA and SOCB

See the ADCSOCxCTL register bit definitions for the configuration details of these triggers.

Additionally ADCINT1 and ADCINT2 can be fed back to trigger another conversion. This configuration is controlled in the ADCINTSOCSEL1/2 registers. This mode is useful if a continuous stream of conversions is desired. See [Section 8.7](#) for information on the ADC interrupt signals.

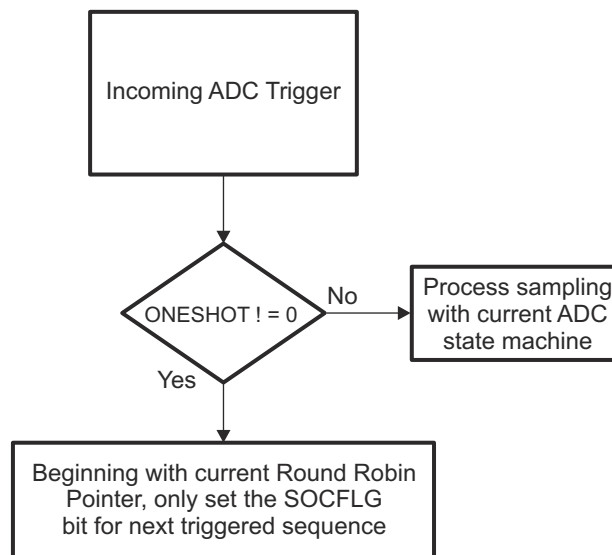
### 8.2.3 Channel Selection

Each SOC can be configured to convert any of the available ADCIN input channels. When an SOC is configured for sequential sampling mode, the four bit CHSEL field of the ADCSOCxCTL register defines which channel to convert. When an SOC is configured for simultaneous sampling mode, the most significant bit of the CHSEL field is dropped and the lower three bits determine which pair of channels are converted.

ADCINA0 is shared with VREFHI, and therefore cannot be used as a variable input source when using external reference voltage mode. See [Section 8.10](#) for details on this mode.

### 8.3 ONESHOT Single Conversion Support

This mode will allow you to perform a single conversion on the next triggered SOC in the round robin scheme. The ONESHOT mode is only valid for channels present in the round robin wheel. Channels which are not configured for triggered SOC in the round robin scheme will get priority based on contents of the SOC PRIORITY field in the ADCSOC PRIORITY CTL register.



**Figure 8-4. ONESHOT Single Conversion**

The effect of ONESHOT mode on Sequential Mode and Simultaneous Mode is explained below.

**Sequential mode:** Only the next active SOC in RR mode (one up from current RR pointer) will be allowed to generate SOC; all other triggers for other SOC slots will be ignored.

**Simultaneous mode:** If current RR pointer has SOC with simultaneous enabled; active SOC will be incremented by 2 from the current RR pointer. This is because simultaneous mode will create result for SOCx and SOCx+1, and SOCx+1 will never be triggered by the user.

---

#### Note

ONESHOT = 1 and SOC PRIORITY = 10h is not a valid combination for above implementation reasons. This should not be a desired mode of operation by the user in any case. The limitation of the above is that the next SOC's must eventually be triggered, or else the ADC will not generate new SOC's for other out-of-order triggers. Any non-orthogonal channels should be placed in the priority mode which is unaffected by ONESHOT mode

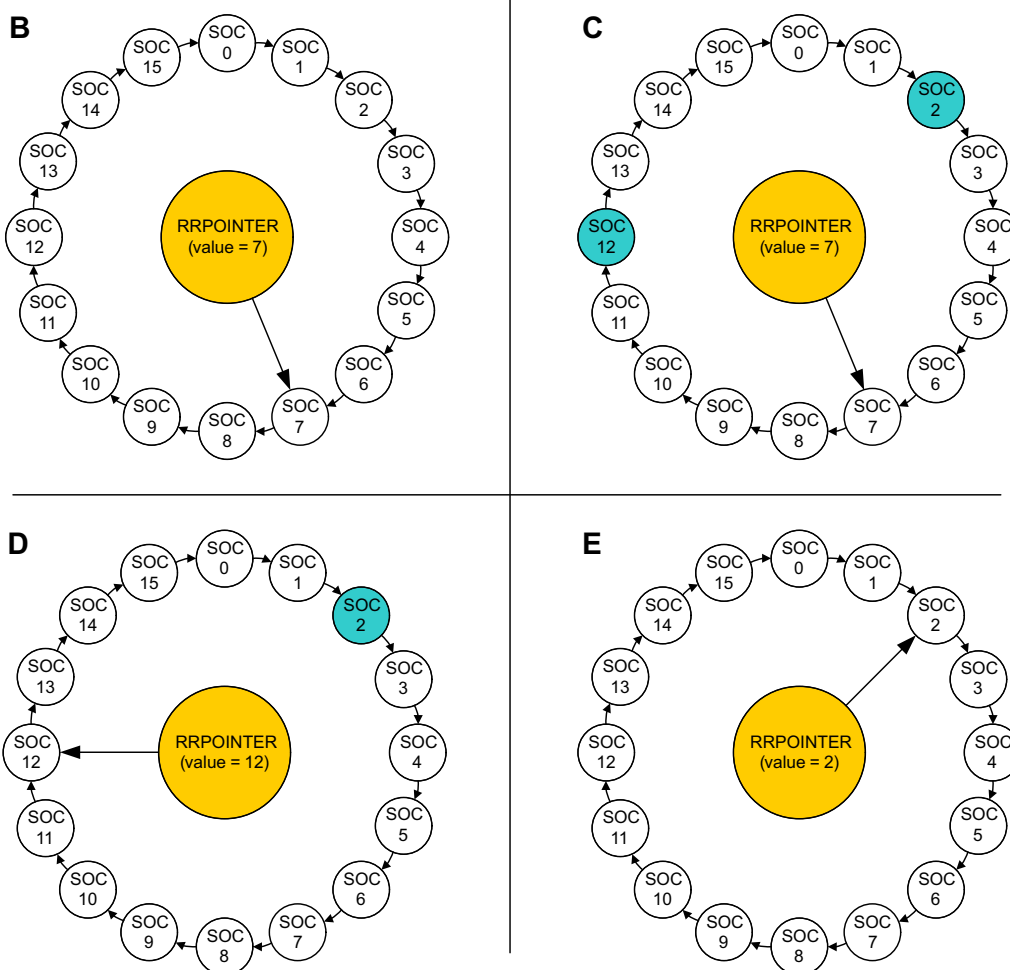
---

## 8.4 ADC Conversion Priority

When multiple SOC flags are set at the same time, one of two forms of priority determines the order in which they are converted. The default priority method is round robin. In this scheme, no SOC has an inherent higher priority than another. Priority depends on the round robin pointer (RR POINTER). The RR POINTER reflected in the ADCSOC PRIORITY CTL register points to the last SOC converted. The highest priority SOC is given to the next value greater than the RR POINTER value, wrapping around back to SOC0 after SOC15. At reset the value is 16 since 0 indicates a conversion has already occurred. When RR POINTER equals 16, the highest priority is given to SOC0. The RR POINTER is reset by a device reset, when the ADCCTL1.RESET bit is set, or when the SOC PRIORITY CTL register is written.

An example of the round robin priority method is given in [Figure 8-5](#).

- A** After reset, SOC0 is highest priority SOC ;  
SOC7 receives trigger;  
SOC7 configured channel is converted immediately .
- B** RRPOINTER changes to point to SOC 7;  
SOC8 is now highest priority SOC .
- C** SOC2 & SOC12 triggers rcvd. simultaneously ;  
SOC12 is first on round robin wheel ;  
SOC12 configured channel is converted while  
SOC2 stays pending .
- D** RRPOINTER changes to point to SOC 12;  
SOC2 configured channel is now converted .
- E** RRPOINTER changes to point to SOC 2;  
SOC3 is now highest priority SOC .



**Figure 8-5. Round Robin Priority Example**

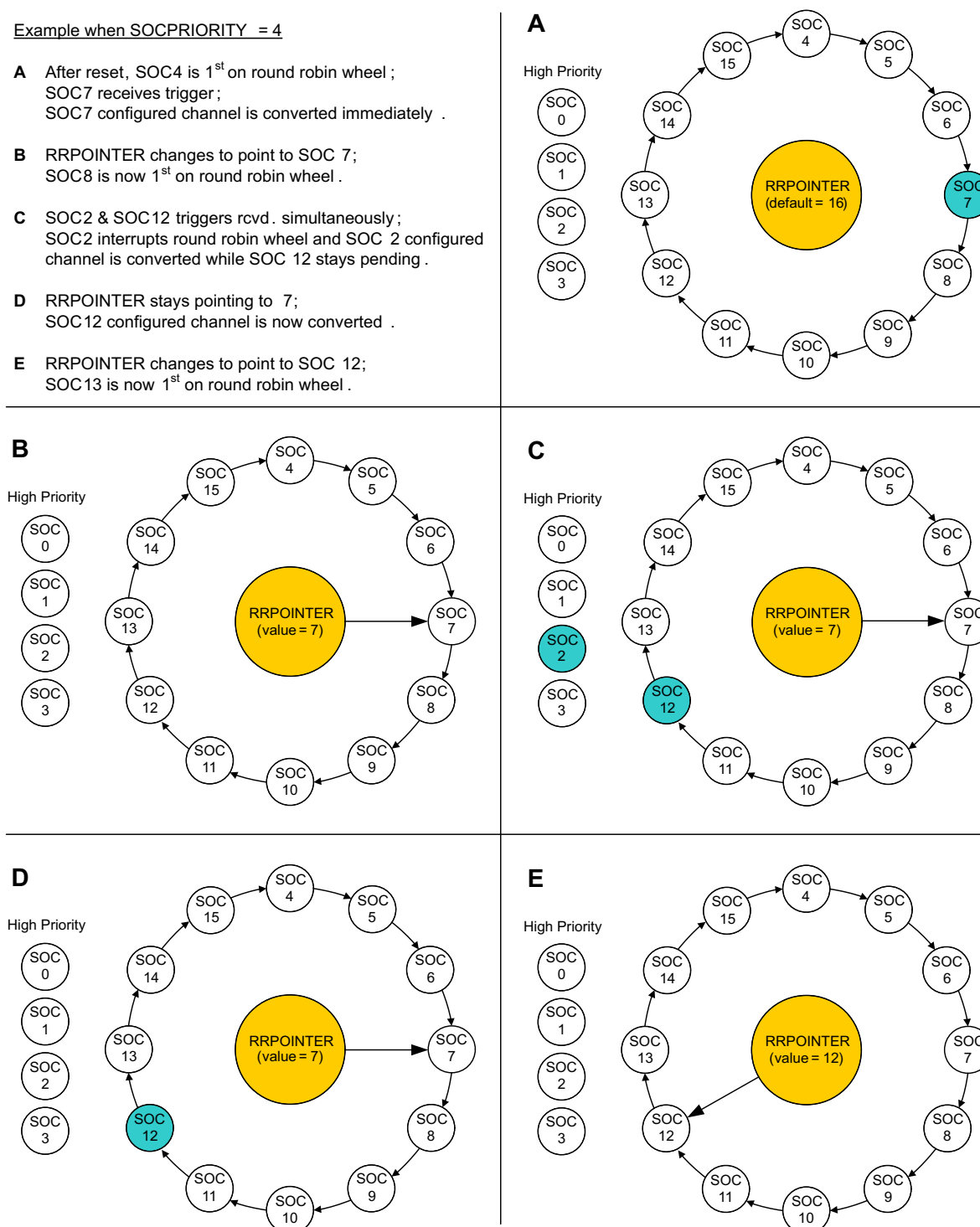
The SOC PRIORITY field in the ADCSOC PRIORITY CTL register can be used to assign high priority from a single to all of the SOC's. When configured as high priority, an SOC will interrupt the round robin wheel after any current conversion completes and insert itself in as the next conversion. After its conversion completes, the round robin wheel will continue where it was interrupted. If two high priority SOC's are triggered at the same time, the SOC with the lower number will take precedence.

High priority mode is assigned first to SOC0, then in increasing numerical order. The value written in the SOC PRIORITY field defines the first SOC that is not high priority. In other words, if a value of 4 is written into SOC PRIORITY, then SOC0, SOC1, SOC2, and SOC3 are defined as high priority, with SOC0 the highest.

An example using high priority SOC's is given in Figure 8-6 .

#### Example when SOC PRIORITY = 4

- A** After reset, SOC 4 is 1<sup>st</sup> on round robin wheel ;  
SOC 7 receives trigger ;  
SOC 7 configured channel is converted immediately .
- B** RRPOINTER changes to point to SOC 7 ;  
SOC 8 is now 1<sup>st</sup> on round robin wheel .
- C** SOC 2 & SOC 12 triggers rcvd . simultaneously ;  
SOC 2 interrupts round robin wheel and SOC 2 configured channel is converted while SOC 12 stays pending .
- D** RRPOINTER stays pointing to 7 ;  
SOC 12 configured channel is now converted .
- E** RRPOINTER changes to point to SOC 12 ;  
SOC 13 is now 1<sup>st</sup> on round robin wheel .



**Figure 8-6. High Priority Example**

## 8.5 Sequential Sampling Mode

The default behavior of the ADC is to treat triggered SOC's as single conversions to be processed sequentially. Sequential sampling can convert both A-channels and B-channels without restriction on ordering.

However, a by-product of supporting the Simultaneous Sampling Mode is that the sampling capacitor from the paired simultaneous channel will also be connected to its respective input at the same time as the desired sequential sampling ACQPS window; the ADC will not convert the sample from the paired channel. (The Simultaneous Sampling Mode is described in [Section 8.6](#), and the sampling capacitor is denoted as Ch in [Figure 8-3](#).)

For example, assume that SOC0 is configured to convert ADCINB3, and SOC1 is configured to convert ADCINA5. If SOC0 and SOC1 are triggered together, the following sequence of simplified events would take place:

1. SOC0 Sample: A-channel and B-channel sampling capacitors are connected to ADCINA3 and ADCINB3 for SOC0 ACQPS window
2. SOC0 Convert: ADC converts B-channel sampling capacitor and stores result to ADCRESULT0
3. SOC1 Sample: A-channel and B-channel sampling capacitors are connected to ADCINA5 and ADCINB5 for SOC1 ACQPS window
4. SOC1 Convert: ADC converts A-channel sampling capacitor and stores result to ADCRESULT1

The extraneous sampling capacitor exposure should be taken into consideration for input signals that have a slow recovery or settling time. Typical examples of slow inputs are sensors with high impedance outputs and signals that are conditioned with low-pass filters.

## 8.6 Simultaneous Sampling Mode

In some applications it is important to keep the delay between the sampling of two signals minimal. The ADC contains dual sample and hold circuits to allow two different channels to be sampled simultaneously. Simultaneous sampling mode is configured for a pair of SOC's with the ADCSAMPLEMODE register. The even-numbered SOCx and the following odd-numbered SOCx (SOC0 and SOC1) are coupled together with one enable bit (SIMULEN0, in this case). The coupling behavior is as follows:

- Either SOCx's trigger will start a pair of conversions.
- The pair of channels converted will consist of the A-channel and the B-channel corresponding to the value of the CHSEL field of the triggered SOCx. The valid values in this mode are 0-7.
- Both channels will be sampled simultaneously.
- The A channel will always convert first.
- The even EOCx pulse will be generated based off of the A-channel conversion, the odd EOCx pulse will be generated off of the B-channel conversion. See [Section 8.7](#) for an explanation of the EOCx signals.
- The result of the A-channel conversion is placed in the even ADCRESULTx register and the result of the B-channel conversion is written to the odd ADCRESULTx register.

For example, if the ADCSAMPLEMODE.SIMULEN0 bit is set, and SOC0 is configured as follows:

- CHSEL = 2 (ADCINA2/ADCINB2 pair)
- TRIGSEL = 5 (ADCTRIG5 = ePWM1.ADCSOCA)

When the ePWM1 sends out an ADCSOCA trigger, both ADCINA2 and ADCINB2 will be sampled simultaneously (assuming priority). Immediately after, the ADCINA2 channel will be converted and its value will be stored in the ADCRESULT0 register. Depending on the ADCCTL1.INTPULSEPOS setting, the EOC0 pulse will either occur when the conversion of ADCINA2 begins or completes. Then the ADCINB2 channel will be converted and its value will be stored in the ADCRESULT1 register. Depending on the ADCCTL1.INTPULSEPOS setting, the EOC1 pulse will either occur when the conversion of ADCINB2 begins or completes.

Typically in an application it is expected that only the even SOCx of the pair will be used. However, it is possible to use the odd SOCx instead, or even both. In the latter case, both SOCx triggers will start a conversion. Therefore, caution is urged as both SOCx's will store their results to the same ADCRESULTx registers, possibly overwriting each other.

The rules of priority for the SOCx's remain the same as in sequential sampling mode.

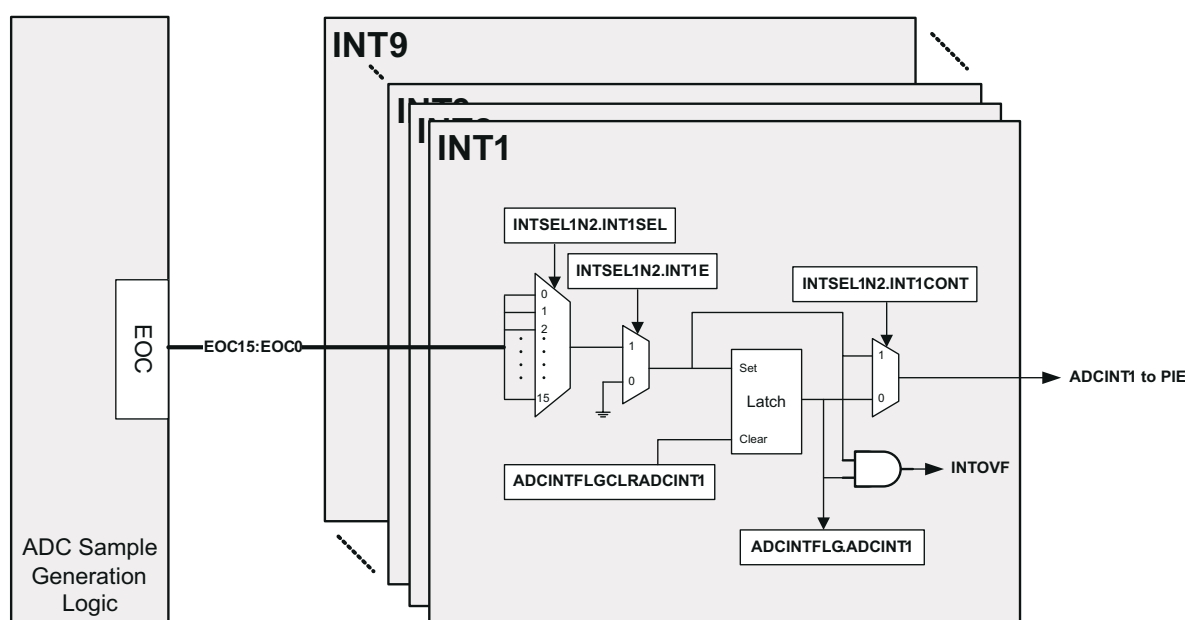
[Section 8.11](#) shows the timing of simultaneous sampling mode.

## 8.7 EOC and Interrupt Operation

Just as there are 16 independent SOCx configuration sets, there are 16 EOCx pulses. In sequential sampling mode, the EOCx is associated directly with the SOCx. In simultaneous sampling mode, the even and the following odd EOCx pair are associated with the even and the following odd SOCx pair, as described in [Section 8.6](#). Depending on the ADCCTL1.INTPULSEPOS setting, the EOCx pulse will occur either at the beginning of a conversion or the end. See [Section 8.11](#) for exact timings on the EOCx pulses.

The ADC contains 9 interrupts that can be flagged and/or passed on to the PIE. Each of these interrupts can be configured to accept any of the available EOCx signals as its source. The configuration of which EOCx is the source is done in the INTSELxNy registers. Additionally, the ADCINT1 and ADCINT2 signals can be configured to generate an SOCx trigger. This is beneficial to creating a continuous stream of conversions.

[Figure 8-7](#) shows a block diagram of the interrupt structure of the ADC.



**Figure 8-7. Interrupt Structure**

### Note

Interrupt generation may be disrupted in non-continuous conversion mode when the interrupt overflow bit in ADCINTOVF is set.

## 8.8 Power-Up Sequence

The ADC resets to the ADC off state. Before writing to any of the ADC registers the ADCENCLK bit in the PCLKCR0 register must be set. For a description of the PCLKCR0 register, see the System Control and Interrupts section in this manual. When powering up the ADC, use the following sequence:

1. If an external reference is desired, enable this mode using bit 3 (ADCREFSSEL) in the ADCCTL1 register.
2. Power up the reference, bandgap, and analog circuits together by setting bits 7-5 (ADCPWDN, ADCBGPWD, ADCREFPWD) in the ADCCTL1 register.
3. Enable the ADC by setting bit 14 (ADCENABLE) of the ADCCTL1 register.
4. Before performing the first conversion, a delay of 1 millisecond after step 2 is required.

Alternatively, steps 1 through 3 can be performed simultaneously.

When powering down the ADC, all three bits in step 2 can be cleared simultaneously. The ADC power levels must be controlled via software and they are independent of the state of the device power modes.

---

### Note

This type ADC requires a 1ms delay after all of the circuits are powered up. This differs from the previous type ADC's.

---

## 8.9 ADC Calibration

Inherent in any converter is a zero offset error and a full scale gain error. The ADC is factory calibrated at 30-degrees Celsius to correct both of these while allowing the user to modify the offset correction for any application environmental effects, such as the ambient temperature. Except under certain emulation conditions, or unless a modification from the factory settings is desired, the user is not required to perform any specific action. The ADC will be properly calibrated during the device boot process.

---

### Note

If the system is reset or the ADC module is reset using Bit 15 (RESET) from the ADC Control Register 1, the Device\_cal() routine must be repeated.

---

### 8.9.1 Factory Settings and Calibration Function

During the fabrication and test process Texas Instruments calibrates several ADC settings along with a couple of internal oscillator settings. These settings are embedded into the TI reserved OTP memory as part of a C-callable function named Device\_cal(). Called during the startup boot procedure in the Boot ROM this function writes the factory settings into their respective active registers. Until this occurs, the ADC and the internal oscillators will not adhere to their specified parameters. If the boot process is skipped during emulation, the user must ensure the trim settings are written to their respective registers to ensure the ADC and the internal oscillators meet the specifications in the data sheet. This can be done either by calling this function manually or in the application itself, or by a direct write via CCS. A gel function for device calibration is included in CCS when the appropriate .ccxml file is created for the target MCU.

For more information on the Device\_cal() function refer to the Boot ROM section in this manual.

Texas Instruments cannot assure the parameters specified in the data sheet, if a value other than the factory settings contained in the TI reserved OTP memory is written into the ADC trim registers.

### 8.9.2 ADC Zero Offset Calibration

Zero offset error is defined as the resultant digital value that occurs when converting a voltage at VREFLO. This base error affects all conversions of the ADC and together with the full scale gain and linearity specifications, determine the DC accuracy of a converter. The zero offset error can be positive, meaning that a positive digital value is output when VREFLO is presented, or negative, meaning that a voltage higher than a one step above VREFLO still reads as a digital zero value. To correct this error, the two's complement of the error is written into the ADCOFFTRIM register. The value contained in this register will be applied before the results are available in the ADC result registers. This operation is fully contained within the ADC core, so the timing for the results



will not be affected and the full dynamic range of the ADC will be maintained for any trim value. Calling the Device\_cal() function writes the ADCOFFTRIM register with the factory calibrated offset error correction, but the user can modify the ADCOFFTRIM register to compensate for additional offset error induced by the application environment. This can be done without sacrificing an ADC channel by using the VREFLOCONV bit in the ADCCTRL1 register.

Use the following procedure to re-calibrate the ADC offset:

1. **Set ADCOFFTRIM to 80 (50h).** This adds an artificial offset to account for negative offset that may reside in the ADC core.
2. **Set ADCCTL1.VREFLOCONV to 1.** This internally connects VREFLO to input channel B5. See the ADCCTL1 register description.
3. **Perform multiple conversions on B5 (sample VREFLO) and take an average to account for board noise.** See [Section 8.2](#) on how to setup and initiate the ADC to sample B5.
4. **Set ADCOFFTRIM to 80 (50h) minus the average obtained in step 3.** This removes the artificial offset from step 1 and creates a two's complement of the offset error.
5. **Set ADCCTL1.VREFLOCONV to 0.** This connects B5 back to the external ADCINB5 input pin.

#### Note

The AdcOffsetSelfCal() function located in F2806x\_Adc.c in the common header files performs these steps.

### 8.9.3 ADC Full Scale Gain Calibration

Gain error occurs as an incremental error as the voltage input is increased. Full scale gain error occurs at the maximum input voltage. As in offset error, gain error can be positive or negative. A positive full scale gain error means that the full scale digital result is reached before the maximum voltage is input. A negative full scale error implies that the full digital result will never be achieved. The calibration function Device\_cal() writes a factory trim value to correct the ADC full scale gain error into the ADCREFTRIM register. This register should not be modified after the Device\_cal() function is called.

### 8.9.4 ADC Bias Current Calibration

To further increase the accuracy of the ADC, the calibration function Device\_cal() also writes a factory trim value to an ADC register for the ADC bias currents. This register should not be modified after the Device\_cal() function is called.

## 8.10 Internal/External Reference Voltage Selection

### 8.10.1 Internal Reference Voltage

The ADC can operate in two different reference modes, selected by the ADCCTL1.ADCREFSEL bit. By default the internal bandgap is chosen to generate the reference voltage for the ADC. This will convert the voltage presented according to a fixed scale 0 to 3.3 V range. The equation governing conversions in this mode is:

|                                                                                          |                                                   |
|------------------------------------------------------------------------------------------|---------------------------------------------------|
| Digital Value = 0                                                                        | when Input $\leq$ 0 V                             |
| Digital Value = $4096 \left[ \frac{\text{Input} - \text{VREFLO}}{3.3 \text{ V}} \right]$ | when $0 \text{ V} < \text{Input} < 3.3 \text{ V}$ |
| Digital Value = 4095,                                                                    | when Input $\geq$ 3.3 V                           |

\*All fractional values are truncated

\*\*VREFLO must be tied to ground in this mode. This is done internally on some devices.



### 8.10.2 External Reference Voltage

To convert the voltage presented as a ratiometric signal, the external VREFHI/VREFLO pins should be chosen to generate the reference voltage. In contrast with the fixed 0 to 3.3 V input range of the internal bandgap mode, the ratiometric mode has an input range from VREFLO to VREFHI. Converted values will scale to this range. For instance, if VREFLO is set to 0.5 V and VREFHI is 3.0 V, a voltage of 1.75 V will be converted to the digital result of 2048. See the device data sheet for the allowable ranges of VREFLO and VREFHI. On some devices VREFLO is tied to ground internally, and hence limited to 0 V. The equation governing the conversions in this mode is:

Digital Value = 0

when Input ≤ VREFLO

Digital Value = 4096 [(Input – VREFLO)/(VREFHI – VREFLO)]

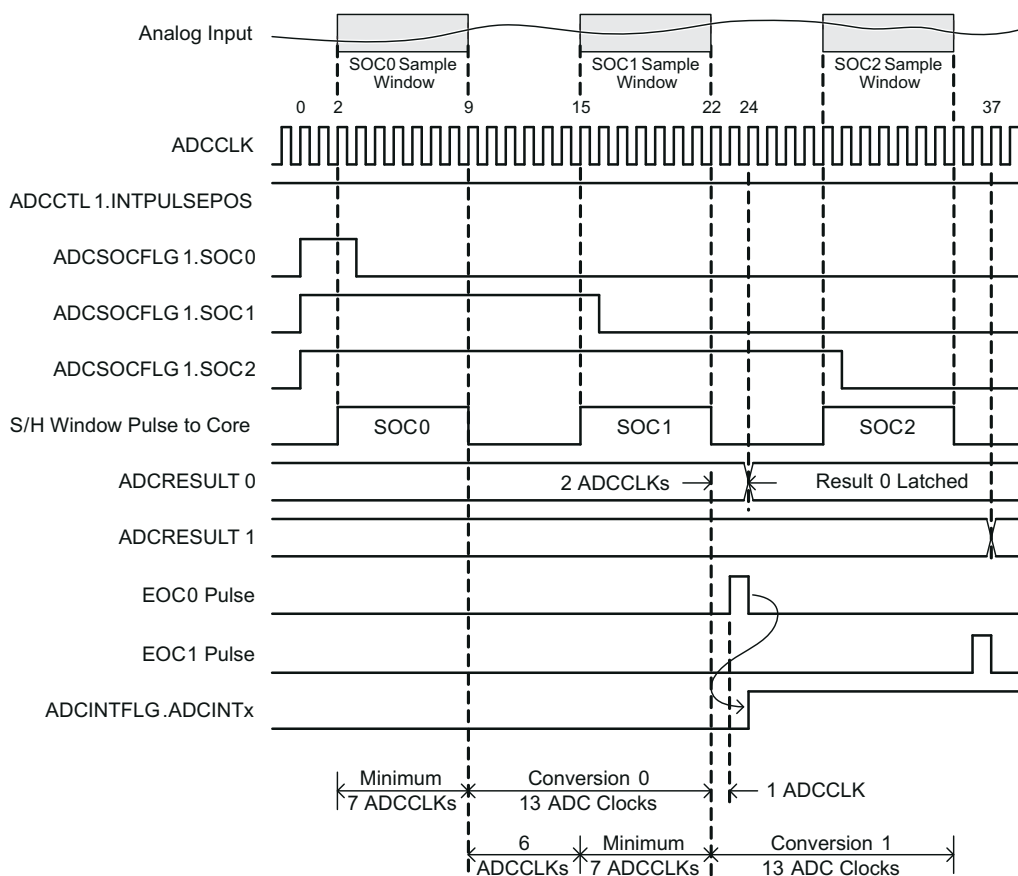
when VREFLO < Input < VREFHI

Digital Value = 4095,

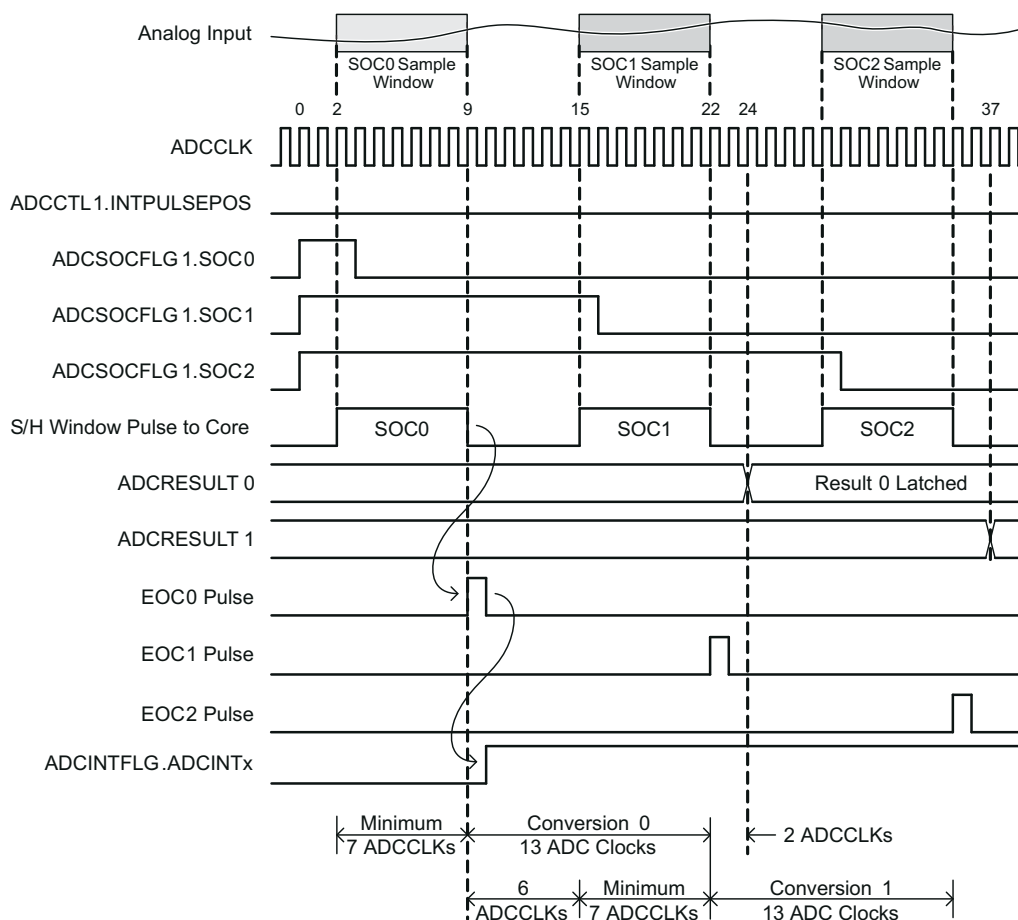
when Input ≥ VREFHI

\*All fractional values are truncated

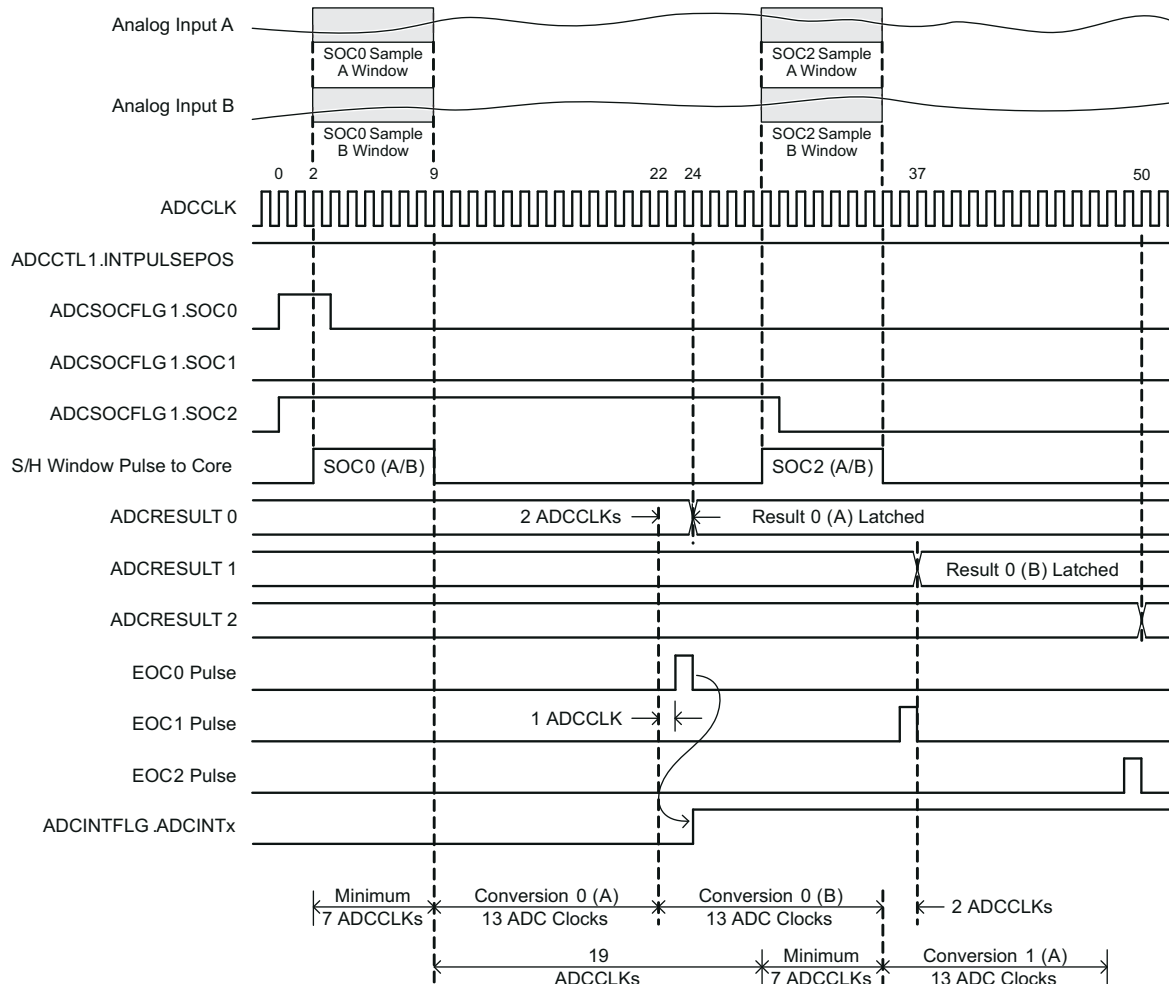
### 8.11 ADC Timings



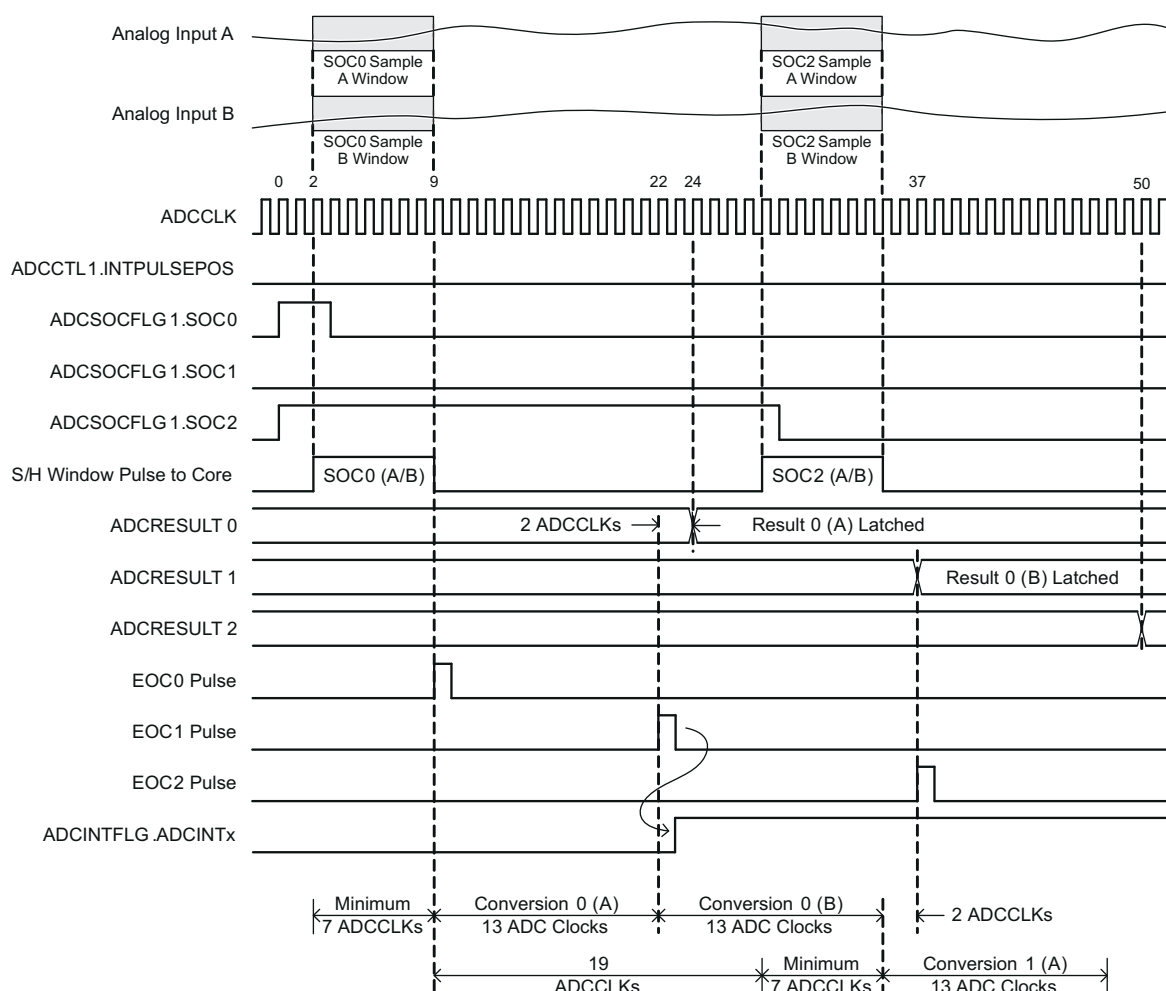
**Figure 8-8. Timing Example For Sequential Mode / Late Interrupt Pulse**



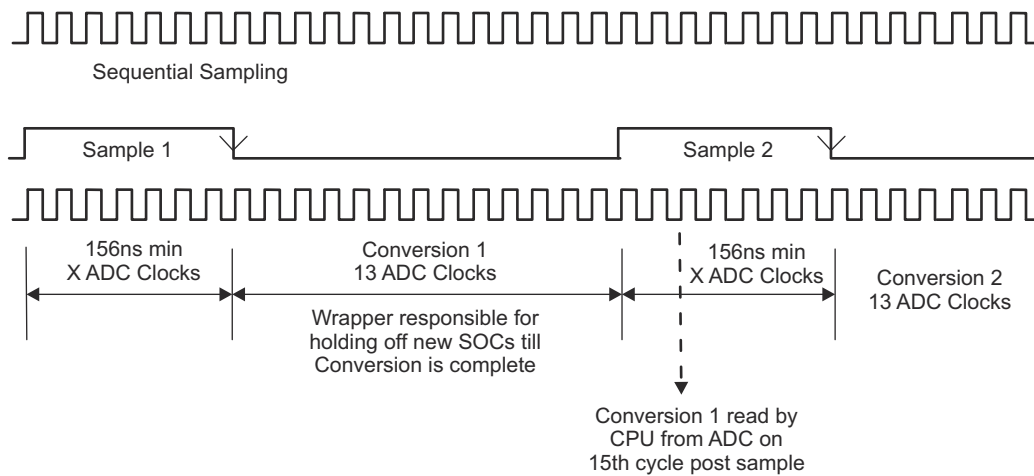
**Figure 8-9. Timing Example For Sequential Mode / Early Interrupt Pulse**



**Figure 8-10. Timing Example For Simultaneous Mode / Late Interrupt Pulse**



**Figure 8-11. Timing Example For Simultaneous Mode / Early Interrupt Pulse**



**Figure 8-12. Timing Example for NONOVERLAP Mode**

**Note**

The NONOVERLAP bit in the ADCCTL2 register, when enabled, removes the overlap of sampling and conversion stages.

## 8.12 Internal Temperature Sensor

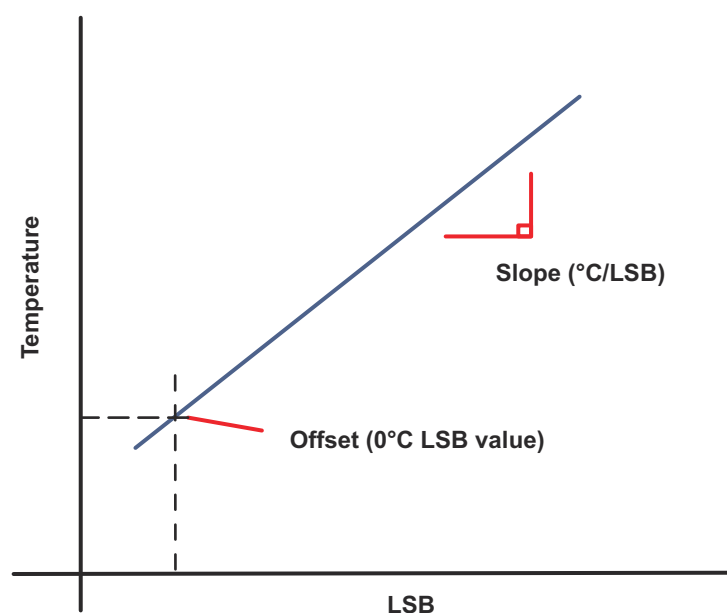
The internal temperature sensor measures the junction temperature of the device. The sensor output can be sampled with the ADC on channel A5 using a switch controlled by the ADCCTL1.TEMPCONV bit. The switch allows A5 to be used both as an external ADC input pin and the temperature sensor access point. When sampling the temperature sensor, the external circuitry on ADCINA5 has no effect on the sample. Refer to [Section 8.13.1](#) for information about switching between the external ADCINA5 input pin and the internal temperature sensor.

### 8.12.1 Transfer Function

The temperature sensor output and the resulting ADC values increase with increasing junction temperature. The offset is defined as the 0 °C LSB crossing as illustrated in [Figure 8-13](#). This information can be used to convert the ADC sensor sample into a temperature unit.

The transfer function to determine a temperature is defined as:

$$\text{Temperature} = (\text{sensor} - \text{Offset}) * \text{Slope}$$



**Figure 8-13. Temperature Sensor Transfer Function**

Refer to the electrical characteristics section in the [TMS320F2806x Real-Time Microcontrollers Data Manual](#) for the slope and offset, or use the stored slope and offset calibrated per device in the factory that can be extracted by a function at the following locations.

For F2806x:

- 0x3D7E82 - Slope (°C / LSB, fixed-point Q15 format)
- 0x3D7E85 - Offset (0 °C LSB value)

The values listed are assuming a 3.3-V full-scale range. Using the internal reference mode automatically achieves this fixed range, but if using the external mode, the temperature sensor values must be adjusted accordingly to the external reference voltages.

## Example

The header files include an example project to easily sample the temperature sensor and convert the result into two different temperature units. There are three steps to using the temperature sensor:

1. Configure the ADC to sample the temperature sensor
2. Sample the temperature sensor
3. Convert the result into a temperature unit, such as °C.

Here is an example of these steps:

```
// Configure the ADC to sample the temperature sensor EALLOW;
AdcRegs.ADCCTL1.bit.TEMPCONV = 1; //Connect A5 - temp sensor
AdcRegs.ADCSOC0CTL.bit.CHSEL = 5; //Set SOC0 to sample A5
AdcRegs.ADCSOC1CTL.bit.CHSEL = 5; //Set SOC1 to sample A5
AdcRegs.ADCSOC0CTL.bit.ACQPS = 6; //Set SOC0 ACQPS to 7 ADCCLK
AdcRegs.ADCSOC1CTL.bit.ACQPS = 6; //Set SOC1 ACQPS to 7 ADCCLK
AdcRegs.INTSEL1N2.bit.INT1SEL = 1; //Connect ADCINT1 to EOC1
AdcRegs.INTSEL1N2.bit.INT1E = 1; //Enable ADCINT1
EDIS;

// Sample the temperature sensor
AdcRegs.ADCSOCFRC1.all = 0x03; //Sample temp sensor
while(AdcRegs.ADCINTFLG.bit.ADCINT1 == 0){} //Wait for ADCINT1
AdcRegs.ADCINTFLGCLR.bit.ADCINT1 = 1; //Clear ADCINT1
sensorSample = AdcResult.ADCRESULT1; //Get temp sensor sample result

//Convert raw temperature sensor output to a temperature (degC)
DegreesC = (sensorSample - TempSensorOffset) * TempSensorSlope;

For the F2806x, call the below factory stored slope and offset get functions:
//Slope of temperature sensor (deg. C / ADC code, fixed pt Q15 format)
#define getTempSlope() (*(int (*)(void))0x3D7E82)()

//ADC code corresponding to temperature sensor output at 0-degreesC
#define getTempOffset() (*(int (*)(void))0x3D7E85)()
```

## 8.13 ADC Registers

This section contains the ADC registers and bit definitions with the registers grouped by function. All of the ADC registers are located in Peripheral Frame 2 except the ADCRESULTx registers, which are found in Peripheral Frame 0. See the device data sheet for specific addresses.

**Table 8-3. ADC Configuration and Control Registers (AdcRegs and AdcResult)**

| Register                 | Address Offset             | Size (x16) | Description                                                                        | Bit Description                  |
|--------------------------|----------------------------|------------|------------------------------------------------------------------------------------|----------------------------------|
| ADCCTL1                  | 0x00                       | 1          | Control 1 Register <sup>(1)</sup>                                                  | <a href="#">Section 8.13.1</a>   |
| ADCCTL2                  | 0x01                       | 1          | Control 2 Register <sup>(1)</sup>                                                  | <a href="#">Section 8.13.2</a>   |
| ADCINTFLG                | 0x04                       | 1          | Interrupt Flag Register                                                            | <a href="#">Section 8.13.3.1</a> |
| ADCINTFLGCLR             | 0x05                       | 1          | Interrupt Flag Clear Register                                                      | <a href="#">Section 8.13.3.2</a> |
| ADCINTOVF                | 0x06                       | 1          | Interrupt Overflow Register                                                        | <a href="#">Section 8.13.3.3</a> |
| ADCINTOVFCLR             | 0x07                       | 1          | Interrupt Overflow Clear Register                                                  | <a href="#">Section 8.13.3.4</a> |
| INTSEL1N2                | 0x08                       | 1          | Interrupt 1 and 2 Selection Register <sup>(1)</sup>                                | <a href="#">Section 8.13.3.5</a> |
| INTSEL3N4                | 0x09                       | 1          | Interrupt 3 and 4 Selection Register <sup>(1)</sup>                                | <a href="#">Section 8.13.3.5</a> |
| INTSEL5N6                | 0x0A                       | 1          | Interrupt 5 and 6 Selection Register <sup>(1)</sup>                                | <a href="#">Section 8.13.3.5</a> |
| INTSEL7N8                | 0x0B                       | 1          | Interrupt 7 and 8 Selection Register <sup>(1)</sup>                                | <a href="#">Section 8.13.3.5</a> |
| INTSEL9N10               | 0x0C                       | 1          | Interrupt 9 Selection Register<br>(reserved Interrupt 10 Selection) <sup>(1)</sup> | <a href="#">Section 8.13.3.5</a> |
| SOCPRCTL                 | 0x10                       | 1          | Start of Conversion Priority Control Register <sup>(1)</sup>                       | <a href="#">Section 8.13.4</a>   |
| ADCSAMPLEMODE            | 0x12                       | 1          | Sampling Mode Register <sup>(1)</sup>                                              | <a href="#">Section 8.13.5.1</a> |
| ADCINTSOCSEL1            | 0x14                       | 1          | Interrupt SOC Selection 1 Register (for 8 channels) <sup>(1)</sup>                 | <a href="#">Section 8.13.5.2</a> |
| ADCINTSOCSEL2            | 0x15                       | 1          | Interrupt SOC Selection 2 Register (for 8 channels) <sup>(1)</sup>                 | <a href="#">Section 8.13.5.3</a> |
| ADCSOCFLG1               | 0x18                       | 1          | SOC Flag 1 Register (for 16 channels)                                              | <a href="#">Section 8.13.5.4</a> |
| ADCSOCFRC1               | 0x1A                       | 1          | SOC Force 1 Register (for 16 channels)                                             | <a href="#">Section 8.13.5.5</a> |
| ADCSOCOVF1               | 0x1C                       | 1          | SOC Overflow 1 Register (for 16 channels)                                          | <a href="#">Section 8.13.5.6</a> |
| ADCSOCOVFCLR1            | 0x1E                       | 1          | SOC Overflow Clear 1 Register (for 16 channels)                                    | <a href="#">Section 8.13.5.7</a> |
| ADCSOC0CTL - ADCSOC15CTL | 0x20 - 0x2F                | 1          | SOC0 Control Register to SOC15 Control Register <sup>(1)</sup>                     | <a href="#">Section 8.13.5.8</a> |
| ADCREFTIM                | 0x40                       | 1          | Reference Trim Register <sup>(1)</sup>                                             | <a href="#">Section 8.13.6.1</a> |
| ADCOFFTRIM               | 0x41                       | 1          | Offset Trim Register <sup>(1)</sup>                                                | <a href="#">Section 8.13.6.2</a> |
| COMPHYSTCTL              | 0x4C                       | 1          | Comp Hysteresis Control Register <sup>(1)</sup>                                    | <a href="#">Section 8.13.7</a>   |
| ADCREV – reserved        | 0x4F                       | 1          | Revision Register                                                                  | <a href="#">Section 8.13.8</a>   |
| ADCRESULT0 - ADCRESULT15 | 0x00 - 0x0F <sup>(2)</sup> | 1          | ADC Result 0 Register to ADC Result 15 Register                                    | <a href="#">Section 8.13.9</a>   |

(1) This register is EALLOW protected.

(2) The base address of the ADCRESULT registers differs from the base address of the other ADC registers. In the header files, the ADCRESULT registers are found in the AdcResult register file, not in the AdcRegs.



### 8.13.1 ADC Control Register 1 (ADCCTL1)

**Figure 8-14. ADC Control Register 1 (ADCCTL1)**

|         |           |           |           |           |             |             |          |
|---------|-----------|-----------|-----------|-----------|-------------|-------------|----------|
| 15      | 14        | 13        | 12        |           |             |             | 8        |
| RESET   | ADCENABLE | ADCBSY    | ADCBSYCHN |           |             |             |          |
| R-0/W-1 | R-1       | R-0       | R-0       |           |             |             |          |
| 7       | 6         | 5         | 4         | 3         | 2           | 1           | 0        |
| ADCPWN  | ADCBGPWD  | ADCREFPWD | Reserved  | ADCREFSEL | INTPULSEPOS | VREFLO CONV | TEMPCONV |
| R/W-0   | R/W-0     | R/W-0     | R-0       | R/W-0     | R/W-0       | R/W-0       | R/W-0    |

LEGEND: R/W = Read/Write; R = Read only; R-0/W-1 = always read as 0, write 1 to set; -n = value after reset

**Table 8-4. ADC Control Register 1 (ADCCTL1) Field Descriptions**

| Bit | Field     | Value                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   | Description <sup>(1)</sup> |
|-----|-----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------|
| 15  | RESET     | <div>0no effect</div> <div>1Resets entire ADC module (bit is then set back to 0 by ADC logic)</div> <p><b>Note:</b> The ADC module is reset during a system reset. If an ADC module reset is desired at any other time, you can do so by writing a 1 to this bit. After two clock cycles, you can then write the appropriate values to the ADCCTL1 register bits. Assembly code:</p> <p>MOV ADCCTL1, #1xxxxxxxxxxxxxb ; Resets the ADC (RESET = 1)</p> <p>NOP ; Delay two cycles</p> <p>NOP</p> <p>MOV ADCCTL1, #0xxxxxxxxxxxxxb ; Set to user-desired value</p> <p><b>Note:</b> The second MOV is not required if the default configuration is sufficient.</p> <p><b>Note:</b> If the system is reset or the ADC module is reset using Bit 15 (RESET) from the ADC Control Register 1, the Device_cal() routine must be repeated .</p> |                            |
| 14  | ADCENABLE | <div>0ADC disabled (<b>does not</b> power down ADC)</div> <div>1ADC Enabled. Must set before an ADC conversion (recommend that it be set directly after setting ADC power-up bits)</div>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |                            |
| 13  | ADCBSY    | <div>ADC Busy</div> <p>Set when ADC SOC is generated, cleared per below. Used by the ADC state machine to determine if ADC is available to sample.</p> <p>Sequential Mode: Cleared 4 ADC clocks after negative edge of S+H pulse</p> <p>Simultaneous Mode: Cleared 14 ADC clocks after negative edge of S+H pulse</p> <div><div>0ADC is available to sample next channel</div><div>1ADC is busy and cannot sample another channel</div></div>                                                                                                                                                                                                                                                                                                                                                                                           |                            |

**Table 8-4. ADC Control Register 1 (ADCCTL1) Field Descriptions (continued)**

| Bit  | Field       | Value                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             | Description <sup>(1)</sup>                                                                                                                                                                                                                                                                   |
|------|-------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 12-8 | ADCB SYCHN  | <p>00h SOC0 is currently processing or was last SOC converted</p> <p>01h SOC1 is currently processing or was last SOC converted</p> <p>02h SOC2 is currently processing or was last SOC converted</p> <p>03h SOC3 is currently processing or was last SOC converted</p> <p>04h SOC4 is currently processing or was last SOC converted</p> <p>05h SOC5 is currently processing or was last SOC converted</p> <p>06h SOC6 is currently processing or was last SOC converted</p> <p>07h SOC7 is currently processing or was last SOC converted</p> <p>08h SOC8 is currently processing or was last SOC converted</p> <p>09h SOC9 is currently processing or was last SOC converted</p> <p>0Ah SOC10 is currently processing or was last SOC converted</p> <p>0Bh SOC11 is currently processing or was last SOC converted</p> <p>0Ch SOC12 is currently processing or was last SOC converted</p> <p>0Dh SOC13 is currently processing or was last SOC converted</p> <p>0Eh SOC14 is currently processing or was last SOC converted</p> <p>0Fh ADCINB15 is currently processing or was last SOC converted</p> <p>1xh Invalid value</p> | <p>Set when ADC SOC for current SOC is generated</p> <p>When ADCBSY = 0: holds the value of the last converted SOC</p> <p>When ADCBSY = 1: reflects SOC currently being processed</p>                                                                                                        |
| 7    | ADCPWDN     | <p>0 All analog circuitry inside the core except the bandgap and reference circuitry is powered down</p> <p>1 The analog circuitry inside the core is powered up</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              | <p>ADC power down (active low).</p> <p>This bit controls the power up and power down of all the analog circuitry inside the analog core except the bandgap and reference circuitry</p>                                                                                                       |
| 6    | ADCBGPWD    | <p>0 Bandgap circuitry is powered down</p> <p>1 Bandgap buffer's circuitry inside core is powered up</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          | <p>Bandgap circuit power down (active low)</p>                                                                                                                                                                                                                                               |
| 5    | ADCREFPWD   | <p>0 Reference buffers circuitry is powered down</p> <p>1 Reference buffers circuitry inside the core is powered up</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           | <p>Reference buffers circuit power down (active low)</p>                                                                                                                                                                                                                                     |
| 4    | Reserved    | 0                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 | Reads return a zero; Writes have no effect.                                                                                                                                                                                                                                                  |
| 3    | ADCREFSEL   | <p>0 Internal Bandgap used for reference generation</p> <p>1 External VREFHI or VREFLO pins used for reference generation. On some devices the VREFHI pin is shared with ADCINA0. In this case ADCINA0 will not be available for conversions in this mode. On some devices the VREFLO pin is shared with VSSA. In this case the VREFLO voltage cannot be varied.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              | <p>Internal or external reference select</p>                                                                                                                                                                                                                                                 |
| 2    | INTPULSEPOS | <p>0 INT pulse generation occurs when ADC begins conversion (negative edge of sample pulse of the sampled signal)</p> <p>1 INT pulse generation occurs 1 cycle prior to ADC result latching into its result register</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          | <p>INT Pulse Generation control</p>                                                                                                                                                                                                                                                          |
| 1    | VREFLOCONV  | <p>0 ADCINB5 is passed to the ADC module as normal, VREFLO connection to ADCINB5 is disabled</p> <p>1 VREFLO internally connected to the ADC for sampling</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     | <p>VREFLO Convert.</p> <p>When enabled, internally connects VREFLO to the ADC channel B5 and disconnects the ADCINB5 pin from the ADC. Whether the pin ADCINB5 exists on the device does not affect this function. Any external circuitry on the ADCINB5 pin is unaffected by this mode.</p> |

**Table 8-4. ADC Control Register 1 (ADCCTL1) Field Descriptions (continued)**

| Bit | Field    | Value | Description <sup>(1)</sup>                                                                                                                                                                                                                                                                                    |
|-----|----------|-------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 0   | TEMPCONV |       | Temperature sensor convert. When enabled internally connects the internal temperature sensor to ADC channel A5 and disconnects the ADCINA5 pin from the ADC. Whether the pin ADCINA5 exists on the device does not affect this function. Any external circuitry on the ADCINA5 pin is unaffected by this mode |
|     |          | 0     | ADCINA5 is passed to the ADC module as normal, internal temperature sensor connection to ADCINA5 is disabled.                                                                                                                                                                                                 |
|     |          | 1     | Temperature sensor is internally connected to the ADC for sampling                                                                                                                                                                                                                                            |

(1) This register is EALLOW protected.

### 8.13.2 ADC Control Register 2 (ADCCTL2)

**Figure 8-15. ADC Control Register 2 (ADCCTL2)**

|          |  |   |           |               |           |
|----------|--|---|-----------|---------------|-----------|
| 15       |  | 3 | 2         | 1             | 0         |
| Reserved |  |   | CLKDIV4EN | ADCNONOVERLAP | CLKDIV2EN |
| R-0      |  |   | R/W-0     | R/W-0         | R/W-0     |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 8-5. ADC Control Register 2 (ADCCTL2) Field Descriptions**

| Bit  | Field         | Value | Description <sup>(1)</sup>                                                                                            |
|------|---------------|-------|-----------------------------------------------------------------------------------------------------------------------|
| 15-3 | Reserved      | 0     | Reads return a zero; writes have no effect.                                                                           |
| 2    | CLKDIV4EN     |       | ADC Clock Prescaler. Used in conjunction with CLKDIV2EN to divide ADCCLK from SYSCLK. See <a href="#">Table 8-6</a> . |
|      |               | 0     | ADCCLK = SYSCLK or SYSCLK / 2                                                                                         |
|      |               | 1     | ADCCLK = SYSCLK or SYSCLK / 4                                                                                         |
| 1    | ADCNONOVERLAP |       | ADCNONOVERLAP control bit                                                                                             |
|      |               | 0     | Overlap of sample and conversion is allowed                                                                           |
|      |               | 1     | Overlap of sample is not allowed                                                                                      |
| 0    | CLKDIV2EN     |       | ADC Clock Prescaler. Used in conjunction with CLKDIV4EN to divide ADCCLK from SYSCLK. See <a href="#">Table 8-6</a> . |
|      |               | 0     | ADCCLK = SYSCLK                                                                                                       |
|      |               | 1     | ADCCLK = SYSCLK / 2 or SYSCLK / 4                                                                                     |

(1) This register is EALLOW protected.

**Table 8-6. CLKDIV2EN and CLKDIV4EN Usage**

| CLKDIV2EN | CLKDIV4EN | ADCCLK     |
|-----------|-----------|------------|
| 0         | 0         | SYSCLK     |
| 0         | 1         | SYSCLK     |
| 1         | 0         | SYSCLK / 2 |
| 1         | 1         | SYSCLK / 4 |

### 8.13.3 ADC Interrupt Registers

#### 8.13.3.1 ADC Interrupt Flag Register (ADCINTFLG)

**Figure 8-16. ADC Interrupt Flag Register (ADCINTFLG)**

|          |         |         |         |         |         |         |         |         |
|----------|---------|---------|---------|---------|---------|---------|---------|---------|
| 15       |         |         |         |         |         |         | 9       | 8       |
| Reserved |         |         |         |         |         |         |         | ADCINT9 |
| R-0      |         |         |         |         |         |         |         | R-0     |
| 7        | 6       | 5       | 4       | 3       | 2       | 1       | 0       |         |
| ADCINT8  | ADCINT7 | ADCINT6 | ADCINT5 | ADCINT4 | ADCINT3 | ADCINT2 | ADCINT1 |         |
| R-0      | R-0     | R-0     | R-0     | R-0     | R-0     | R-0     | R-0     |         |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 8-7. ADC Interrupt Flag Register (ADCINTFLG) Field Descriptions**

| Bit  | Field                   | Value  | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|------|-------------------------|--------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-9 | Reserved                | 0      | Reads return a zero; Writes have no effect.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| 8-0  | ADCINTx<br>(x = 9 to 1) | 0<br>1 | <p>ADC Interrupt Flag Bits: Reading this bit indicates if an ADCINT pulse was generated</p> <p>No ADC interrupt pulse generated</p> <p>ADC Interrupt pulse generated</p> <p>If the ADC interrupt is placed in continuous mode (INTSELxNy register) then further interrupt pulses are generated whenever a selected EOC event occurs even if the flag bit is set.</p> <p>If the continuous mode is not enabled, then no further interrupt pulses are generated until the user clears this flag bit using the ADCINTFLGCLR register. The ADCINTOVF flag will be set if EOC events are generated while the ADCINTFLG flag is set. Both ADCINTFLG and ADCINTOVF flags must be cleared before normal interrupt operation can resume in non-continuous mode.</p> |

### 8.13.3.2 ADC Interrupt Flag Clear Register (ADCINTFLGCLR)

**Figure 8-17. ADC Interrupt Flag Clear Register (ADCINTFLGCLR)**

|          |         |         |         |         |         |         |         |   |  |
|----------|---------|---------|---------|---------|---------|---------|---------|---|--|
| 15       |         |         |         |         |         | 9       |         | 8 |  |
| Reserved |         |         |         |         |         |         | ADCINT9 |   |  |
| R-0      |         |         |         |         |         |         | R/W-0   |   |  |
| 7        |         | 6       | 5       | 4       | 3       | 2       | 1       | 0 |  |
| ADCINT8  | ADCINT7 | ADCINT6 | ADCINT5 | ADCINT4 | ADCINT3 | ADCINT2 | ADCINT1 |   |  |
| R/W-0    | R/W-0   | R/W-0   | R/W-0   | R/W-0   | R/W-0   | R/W-0   | R/W-0   |   |  |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 8-8. ADC Interrupt Flag Clear Register (ADCINTFLGCLR) Field Descriptions**

| Bit  | Field                   | Value  | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|------|-------------------------|--------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-9 | Reserved                | 0      | Reads return a zero; Writes have no effect.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| 8-0  | ADCINTx<br>(x = 9 to 1) | 0<br>1 | <p>ADC interrupt Flag Clear Bit</p> <p>No action.</p> <p>Clears respective flag bit in the ADCINTFLG register.</p> <p><b>Boundary condition for clearing or setting flag bits:</b> If hardware tries to set a flag bit while software tries to clear the flag bit in the same cycle, the following will take place:</p> <ol style="list-style-type: none"> <li>SW has priority, and will clear the flag</li> <li>HW set will be discarded, no signal will propagate to the PIE from the latch</li> <li>Overflow flag or condition will be generated</li> </ol> |

### 8.13.3.3 ADC Interrupt Overflow Register (ADCINTOVF)

**Figure 8-18. ADC Interrupt Overflow Register (ADCINTOVF)**

|          |  |         |  |         |  |         |  |         |  |         |  |         |  |         |  |   |  |
|----------|--|---------|--|---------|--|---------|--|---------|--|---------|--|---------|--|---------|--|---|--|
| 15       |  |         |  | 9       |  |         |  | 8       |  |         |  |         |  |         |  |   |  |
| Reserved |  |         |  |         |  |         |  | ADCINT9 |  |         |  |         |  |         |  |   |  |
| R-0      |  |         |  |         |  |         |  | R-0     |  |         |  |         |  |         |  |   |  |
| 7        |  |         |  | 6       |  | 5       |  | 4       |  | 3       |  | 2       |  | 1       |  | 0 |  |
| ADCINT8  |  | ADCINT7 |  | ADCINT6 |  | ADCINT5 |  | ADCINT4 |  | ADCINT3 |  | ADCINT2 |  | ADCINT1 |  |   |  |
| R-0      |  | R-0     |  | R-0     |  | R-0     |  | R-0     |  | R-0     |  | R-0     |  | R-0     |  |   |  |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 8-9. ADC Interrupt Overflow Register (ADCINTOVF) Field Descriptions**

| Bit  | Field                   | Value  | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|------|-------------------------|--------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-9 | Reserved                | 0      | Reserved                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| 8-0  | ADCINTx<br>(x = 9 to 1) | 0<br>1 | ADC Interrupt Overflow Bits.<br>Indicates if an overflow occurred when generating ADCINT pulses. If the respective ADCINTFLG bit is set and a selected additional EOC trigger is generated, then an overflow condition occurs.<br>No ADC interrupt overflow event detected.<br>ADC Interrupt overflow event detected.<br>The overflow bit does not care about the continuous mode bit state. An overflow condition is generated irrespective of this mode selection. Both ADCINTFLG and ADCINTOVF flags must be cleared before normal interrupt operation can resume in non-continuous mode. |

### 8.13.3.4 ADC Interrupt Overflow Clear Register (ADCINTOVFCLR)

**Figure 8-19. ADC Interrupt Overflow Clear Register (ADCINTOVFCLR)**

|          |  |  |  |         |  |  |  |         |  |  |  |         |  |  |  |         |  |  |  |         |  |  |  |         |  |  |  |         |  |  |  |
|----------|--|--|--|---------|--|--|--|---------|--|--|--|---------|--|--|--|---------|--|--|--|---------|--|--|--|---------|--|--|--|---------|--|--|--|
| 15       |  |  |  |         |  |  |  |         |  |  |  | 9       |  |  |  | 8       |  |  |  |         |  |  |  |         |  |  |  |         |  |  |  |
| Reserved |  |  |  |         |  |  |  |         |  |  |  | ADCINT9 |  |  |  |         |  |  |  |         |  |  |  |         |  |  |  |         |  |  |  |
| R-0      |  |  |  |         |  |  |  |         |  |  |  | R-0/W-1 |  |  |  |         |  |  |  |         |  |  |  |         |  |  |  |         |  |  |  |
| 7        |  |  |  | 6       |  |  |  | 5       |  |  |  | 4       |  |  |  | 3       |  |  |  | 2       |  |  |  | 1       |  |  |  | 0       |  |  |  |
| ADCINT8  |  |  |  | ADCINT7 |  |  |  | ADCINT6 |  |  |  | ADCINT5 |  |  |  | ADCINT4 |  |  |  | ADCINT3 |  |  |  | ADCINT2 |  |  |  | ADCINT1 |  |  |  |
| R-0/W-1  |  |  |  | R-0/W-1 |  |  |  | R-0/W-1 |  |  |  | R-0/W-1 |  |  |  | R-0/W-1 |  |  |  | R-0/W-1 |  |  |  | R-0/W-1 |  |  |  | R-0/W-1 |  |  |  |

LEGEND: R/W = Read/Write; R = Read only; R-0/W-1 =always read 0, write 1 to set; -n = value after reset

**Table 8-10. ADC Interrupt Overflow Clear Register (ADCINTOVFCLR) Field Descriptions**

| Bit  | Field                   | Value | Description                                                                                                                                                                                                                                                  |
|------|-------------------------|-------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-9 | Reserved                | 0     | Reads return a zero; Writes have no effect.                                                                                                                                                                                                                  |
| 8-0  | ADCINTx<br>(x = 9 to 1) | 0     | ADC Interrupt Overflow Clear Bits.<br>No action.                                                                                                                                                                                                             |
|      |                         | 1     | Clears the respective overflow bit in the ADCINTOVF register. If software tries to set this bit on the same clock cycle that hardware tries to set the overflow bit in the ADCINTOVF register, then hardware has priority and the ADCINTOVF bit will be set. |

### 8.13.3.5 Interrupt Select Registers (INTSELxNy)

**Figure 8-20. Interrupt Select 1 and 2 Register (INTSEL1N2)**

|          |          |       |         |   |
|----------|----------|-------|---------|---|
| 15       | 14       | 13    | 12      | 8 |
| Reserved | INT2CONT | INT2E | INT2SEL |   |
| R-0      | R/W-0    | R/W-0 | R/W-0   |   |
| 7        | 6        | 5     | 4       | 0 |
| Reserved | INT1CONT | INT1E | INT1SEL |   |
| R-0      | R/W-0    | R/W-0 | R/W-0   |   |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Figure 8-21. Interrupt Select 3 and 4 Register (INTSEL3N4)**

|          |          |       |         |   |
|----------|----------|-------|---------|---|
| 15       | 14       | 13    | 12      | 8 |
| Reserved | INT4CONT | INT4E | INT4SEL |   |
| R-0      | R/W-0    | R/W-0 | R/W-0   |   |
| 7        | 6        | 5     | 4       | 0 |
| Reserved | INT3CONT | INT3E | INT3SEL |   |
| R-0      | R/W-0    | R/W-0 | R/W-0   |   |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Figure 8-22. Interrupt Select 5 and 6 Register (INTSEL5N6)**

|          |          |       |         |   |
|----------|----------|-------|---------|---|
| 15       | 14       | 13    | 12      | 8 |
| Reserved | INT6CONT | INT6E | INT6SEL |   |
| R-0      | R/W-0    | R/W-0 | R/W-0   |   |
| 7        | 6        | 5     | 4       | 0 |
| Reserved | INT5CONT | INT5E | INT5SEL |   |
| R-0      | R/W-0    | R/W-0 | R/W-0   |   |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset



**Figure 8-23. Interrupt Select 7 and 8 Register (INTSEL7N8)**

|          |          |       |         |   |
|----------|----------|-------|---------|---|
| 15       | 14       | 13    | 12      | 8 |
| Reserved | INT8CONT | INT8E | INT8SEL |   |
| R-0      | R/W-0    | R/W-0 | R/W-0   |   |
| 7        | 6        | 5     | 4       | 0 |
| Reserved | INT7CONT | INT7E | INT7SEL |   |
| R-0      | R/W-0    | R/W-0 | R/W-0   |   |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Figure 8-24. Interrupt Select 9 and 10 Register (INTSEL9N10)**

|          |  |          |  |       |  |         |  |   |  |
|----------|--|----------|--|-------|--|---------|--|---|--|
| 15       |  |          |  | 8     |  |         |  |   |  |
| Reserved |  |          |  |       |  |         |  |   |  |
| R-0      |  |          |  |       |  |         |  |   |  |
| 7        |  | 6        |  | 5     |  | 4       |  | 0 |  |
| Reserved |  | INT9CONT |  | INT9E |  | INT9SEL |  |   |  |
| R-0      |  | R/W-0    |  | R/W-0 |  | R/W-0   |  |   |  |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 8-11. Interrupt Select Register (INTSELxNy) Field Descriptions**

| Bit | Field    | Value | Description <sup>(1)</sup>                                                                                                               |
|-----|----------|-------|------------------------------------------------------------------------------------------------------------------------------------------|
| 15  | Reserved | 0     | Reserved                                                                                                                                 |
| 14  | INTyCONT | 0     | ADCINTy Continuous Mode Enable<br>No further ADCINTy pulses are generated until ADCINTy flag (in ADCINTFLG register) is cleared by user. |
|     |          | 1     | ADCINTy pulses are generated whenever an EOC pulse is generated irrespective if the flag bit is cleared or not.                          |
| 13  | INTyE    | 0     | ADCINTy Interrupt Enable<br>ADCINTy is disabled.                                                                                         |
|     |          | 1     | ADCINTy is enabled.                                                                                                                      |

**Table 8-11. Interrupt Select Register (INTSELxNy) Field Descriptions (continued)**

| Bit  | Field    | Value | Description <sup>(1)</sup>                                                                                      |
|------|----------|-------|-----------------------------------------------------------------------------------------------------------------|
| 12-8 | INTySEL  | 00h   | ADCINTy EOC Source Select                                                                                       |
|      |          | 01h   | EOC0 is trigger for ADCINTy                                                                                     |
|      |          | 02h   | EOC1 is trigger for ADCINTy                                                                                     |
|      |          | 03h   | EOC2 is trigger for ADCINTy                                                                                     |
|      |          | 04h   | EOC3 is trigger for ADCINTy                                                                                     |
|      |          | 05h   | EOC4 is trigger for ADCINTy                                                                                     |
|      |          | 06h   | EOC5 is trigger for ADCINTy                                                                                     |
|      |          | 07h   | EOC6 is trigger for ADCINTy                                                                                     |
|      |          | 08h   | EOC7 is trigger for ADCINTy                                                                                     |
|      |          | 09h   | EOC8 is trigger for ADCINTy                                                                                     |
|      |          | 0Ah   | EOC9 is trigger for ADCINTy                                                                                     |
|      |          | 0Bh   | EOC10 is trigger for ADCINTy                                                                                    |
|      |          | 0Ch   | EOC11 is trigger for ADCINTy                                                                                    |
|      |          | 0Dh   | EOC12 is trigger for ADCINTy                                                                                    |
|      |          | 0Eh   | EOC13 is trigger for ADCINTy                                                                                    |
|      |          | 0Fh   | EOC14 is trigger for ADCINTy                                                                                    |
|      |          | 1xh   | Invalid value.                                                                                                  |
| 7    | Reserved | 0     | Reads return a zero; Writes have no effect.                                                                     |
| 6    | INTxCONT | 0     | ADCINTx Continuous Mode Enable.                                                                                 |
|      |          | 1     | No further ADCINTx pulses are generated until ADCINTx flag (in ADCINTFLG register) is cleared by user.          |
| 5    | INTxE    | 0     | ADCINTx pulses are generated whenever an EOC pulse is generated irrespective if the flag bit is cleared or not. |
|      |          | 1     | ADCINTx Interrupt Enable                                                                                        |
| 4-0  | INTxSEL  | 00h   | ADCINTx is disabled.                                                                                            |
|      |          | 01h   | ADCINTx is enabled .                                                                                            |
|      |          | 02h   | ADCINTx EOC Source Select                                                                                       |
|      |          | 03h   | EOC0 is trigger for ADCINTx                                                                                     |
|      |          | 04h   | EOC1 is trigger for ADCINTx                                                                                     |
|      |          | 05h   | EOC2 is trigger for IADCNTx                                                                                     |
|      |          | 06h   | EOC3 is trigger for ADCINTx                                                                                     |
|      |          | 07h   | EOC4 is trigger for ADCINTx                                                                                     |
|      |          | 08h   | EOC5 is trigger for ADCINTx                                                                                     |
|      |          | 09h   | EOC6 is trigger for ADCINTx                                                                                     |
|      |          | 0Ah   | EOC7 is trigger for ADCINTx                                                                                     |
|      |          | 0Bh   | EOC8 is trigger for ADCINTx                                                                                     |
|      |          | 0Ch   | EOC9 is trigger for ADCINTx                                                                                     |
|      |          | 0Dh   | EOC10 is trigger for ADCINTx                                                                                    |
|      |          | 0Eh   | EOC11 is trigger for ADCINTx                                                                                    |
|      |          | 0Fh   | EOC12 is trigger for ADCINTx                                                                                    |
|      |          | 1xh   | EOC13 is trigger for ADCINTx                                                                                    |
|      |          | 1xh   | Invalid value.                                                                                                  |

(1) This register is EALLOW protected.

### 8.13.4 ADC Start of Conversion Priority Control Register (SOCPRCTL)

**Figure 8-25. ADC Start of Conversion Priority Control Register (SOCPRCTL)**

|         |          |           |    |   |             |   |
|---------|----------|-----------|----|---|-------------|---|
| 15      | 14       | 11        | 10 | 5 | 4           | 0 |
| ONESHOT | Reserved | RRPOINTER |    |   | SOCPRIORITY |   |
| R/W-0   | R-0      | R-20h     |    |   | R/W-0       |   |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 8-12. SOCPRCTL Register Field Descriptions**

| Bit   | Field     | Value                                                                                                                                | Description <sup>(1)</sup>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|-------|-----------|--------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15    | ONESHOT   | 0<br>1                                                                                                                               | One shot mode disabled<br>One shot mode enabled                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| 14-11 | Reserved  |                                                                                                                                      | Reads return a zero; Writes have no effect.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| 10-5  | RRPOINTER | 00h<br>01h<br>02h<br>03h<br>04h<br>05h<br>06h<br>07h<br>08h<br>09h<br>0Ah<br>0Bh<br>0Ch<br>0Dh<br>0Eh<br>0Fh<br>1xh<br>20h<br>Others | Round Robin Pointer. Holds the value of the last converted round robin SOCx to be used by the round robin scheme to determine order of conversions.<br>SOC0 was last round robin SOC to convert. SOC1 is highest round robin priority.<br>SOC1 was last round robin SOC to convert. SOC2 is highest round robin priority.<br>SOC2 was last round robin SOC to convert. SOC3 is highest round robin priority.<br>SOC3 was last round robin SOC to convert. SOC4 is highest round robin priority.<br>SOC4 was last round robin SOC to convert. SOC5 is highest round robin priority.<br>SOC5 was last round robin SOC to convert. SOC6 is highest round robin priority.<br>SOC6 was last round robin SOC to convert. SOC7 is highest round robin priority.<br>SOC7 was last round robin SOC to convert. SOC8 is highest round robin priority.<br>SOC8 was last round robin SOC to convert. SOC9 is highest round robin priority.<br>SOC9 was last round robin SOC to convert. SOC10 is highest round robin priority.<br>SOC10 was last round robin SOC to convert. SOC11 is highest round robin priority.<br>SOC11 was last round robin SOC to convert. SOC12 is highest round robin priority.<br>SOC12 was last round robin SOC to convert. SOC13 is highest round robin priority.<br>SOC13 was last round robin SOC to convert. SOC14 is highest round robin priority.<br>SOC14 was last round robin SOC to convert. SOC15 is highest round robin priority.<br>SOC15 was last round robin SOC to convert. SOC0 is highest round robin priority.<br>Invalid value<br>Reset value to indicate no SOC has been converted. SOC0 is highest round robin priority. Set to this value when the device is reset, when the ADCCTL1.RESET bit is set, or when the SOCPRCTL register is written. In the latter case, if a conversion is currently in progress, it will complete and then the new priority will take effect.<br>Invalid selection. |

**Table 8-12. SOCPRCTL Register Field Descriptions (continued)**

| Bit | Field       | Value  | Description <sup>(1)</sup>                                                                          |
|-----|-------------|--------|-----------------------------------------------------------------------------------------------------|
| 4-0 | SOCPRIORITY |        | SOC Priority.<br>Determines the cutoff point for priority mode and round robin arbitration for SOCx |
|     |             | 00h    | SOC priority is handled in round robin mode for all channels.                                       |
|     |             | 01h    | SOC0 is high priority, rest of channels are in round robin mode.                                    |
|     |             | 02h    | SOC0-SOC1 are high priority, SOC2-SOC15 are in round robin mode.                                    |
|     |             | 03h    | SOC0-SOC2 are high priority, SOC3-SOC15 are in round robin mode.                                    |
|     |             | 04h    | SOC0-SOC3 are high priority, SOC4-SOC15 are in round robin mode.                                    |
|     |             | 05h    | SOC0-SOC4 are high priority, SOC5-SOC15 are in round robin mode.                                    |
|     |             | 06h    | SOC0-SOC5 are high priority, SOC6-SOC15 are in round robin mode.                                    |
|     |             | 07h    | SOC0-SOC6 are high priority, SOC7-SOC15 are in round robin mode.                                    |
|     |             | 08h    | SOC0-SOC7 are high priority, SOC8-SOC15 are in round robin mode.                                    |
|     |             | 09h    | SOC0-SOC8 are high priority, SOC9-SOC15 are in round robin mode.                                    |
|     |             | 0Ah    | SOC0-SOC9 are high priority, SOC10-SOC15 are in round robin mode.                                   |
|     |             | 0Bh    | SOC0-SOC10 are high priority, SOC11-SOC15 are in round robin mode.                                  |
|     |             | 0Ch    | SOC0-SOC11 are high priority, SOC12-SOC15 are in round robin mode.                                  |
|     |             | 0Dh    | SOC0-SOC12 are high priority, SOC13-SOC15 are in round robin mode.                                  |
|     |             | 0Eh    | SOC0-SOC13 are high priority, SOC14-SOC15 are in round robin mode.                                  |
|     |             | 0Fh    | SOC0-SOC14 are high priority, SOC15 is in round robin mode.                                         |
|     |             | 10h    | All SOCx are in high priority mode, arbitrated by SOC number                                        |
|     |             | Others | Invalid selection.                                                                                  |

(1) This register is EALLOW protected.

## 8.13.5 ADC SOC Registers

### 8.13.5.1 ADC Sample Mode Register (ADCSAMPLEMODE)

**Figure 8-26. ADC Sample Mode Register (ADCSAMPLEMODE)**

|           |           |           |          |          |          |          |          |
|-----------|-----------|-----------|----------|----------|----------|----------|----------|
| Reserved  |           |           |          |          |          |          |          |
| R-0       |           |           |          |          |          |          |          |
| 7         | 6         | 5         | 4        | 3        | 2        | 1        | 0        |
| SIMULEN14 | SIMULEN12 | SIMULEN10 | SIMULEN8 | SIMULEN6 | SIMULEN4 | SIMULEN2 | SIMULEN0 |
| R/W-0     | R/W-0     | R/W-0     | R/W-0    | R/W-0    | R/W-0    | R/W-0    | R/W-0    |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 8-13. ADC Sample Mode Register (ADCSAMPLEMODE) Field Descriptions**

| Bit  | Field     | Value | Description <sup>(1)</sup>                                                                                                                                                                                                                     |
|------|-----------|-------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15:8 | Reserved  | 0     | Reserved                                                                                                                                                                                                                                       |
| 7    | SIMULEN14 | 0     | Simultaneous sampling enable for SOC14/SOC15. Couples SOC14 and SOC15 in simultaneous sampling mode. See <a href="#">Section 8.5</a> for details. This bit should not be set when the ADC is actively converting SOC14 or SOC15.               |
|      |           | 1     | Single sample mode set for SOC14 and SOC15. All bits of CHSEL field define channel to be converted. EOC14 associated with SOC14. EOC15 associated with SOC15. SOC14 result placed in ADCRESULT14 register. SOC15 result placed in ADCRESULT15. |
| 6    | SIMULEN12 | 0     | Simultaneous sampling enable for SOC12/SOC13. Couples SOC12 and SOC13 in simultaneous sampling mode. See <a href="#">Section 8.5</a> for details. This bit should not be set when the ADC is actively converting SOC12 or SOC13.               |
|      |           | 1     | Single sample mode set for SOC12 and SOC13. All bits of CHSEL field define channel to be converted. EOC12 associated with SOC12. EOC13 associated with SOC13. SOC12 result placed in ADCRESULT12 register. SOC13 result placed in ADCRESULT13. |
| 5    | SIMULEN10 | 0     | Simultaneous sampling enable for SOC10/SOC11. Couples SOC10 and SOC11 in simultaneous sampling mode. See <a href="#">Section 8.5</a> for details. This bit should not be set when the ADC is actively converting SOC10 or SOC11.               |
|      |           | 1     | Single sample mode set for SOC10 and SOC11. All bits of CHSEL field define channel to be converted. EOC10 associated with SOC10. EOC11 associated with SOC11. SOC10 result placed in ADCRESULT10 register. SOC11 result placed in ADCRESULT11. |
| 4    | SIMULEN8  | 0     | Simultaneous sampling enable for SOC8/SOC9. Couples SOC8 and SOC9 in simultaneous sampling mode. See <a href="#">Section 8.5</a> for details. This bit should not be set when the ADC is actively converting SOC8 or SOC9.                     |
|      |           | 1     | Single sample mode set for SOC8 and SOC9. All bits of CHSEL field define channel to be converted. EOC8 associated with SOC8. EOC9 associated with SOC9. SOC8 result placed in ADCRESULT8 register. SOC9 result placed in ADCRESULT9.           |

**Table 8-13. ADC Sample Mode Register (ADCSAMPLEMODE) Field Descriptions (continued)**

| Bit | Field    | Value             | Description <sup>(1)</sup>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|-----|----------|-------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 3   | SIMULEN6 | <p>0</p> <p>1</p> | <p>Simultaneous sampling enable for SOC6/SOC7. Couples SOC6 and SOC7 in simultaneous sampling mode. See <a href="#">Section 8.5</a> for details. This bit should not be set when the ADC is actively converting SOC6 or SOC7.</p> <p>Single sample mode set for SOC6 and SOC7. All bits of CHSEL field define channel to be converted. EOC6 associated with SOC6. EOC7 associated with SOC7. SOC6 result placed in ADCRESULT6 register. SOC7 result placed in ADCRESULT7.</p> <p>Simultaneous sample for SOC6 and SOC7. Lowest three bits of CHSEL field define the pair of channels to be converted. EOC6 and EOC7 associated with SOC6 and SOC7 pair. SOC6 and SOC7 results will be placed in ADCRESULT6 and ADCRESULT7 registers, respectively.</p> |
| 2   | SIMULEN4 | <p>0</p> <p>1</p> | <p>Simultaneous sampling enable for SOC4/SOC5. Couples SOC4 and SOC5 in simultaneous sampling mode. See <a href="#">Section 8.5</a> for details. This bit should not be set when the ADC is actively converting SOC4 or SOC5.</p> <p>Single sample mode set for SOC4 and SOC5. All bits of CHSEL field define channel to be converted. EOC4 associated with SOC4. EOC5 associated with SOC5. SOC4 result placed in ADCRESULT4 register. SOC5 result placed in ADCRESULT5.</p> <p>Simultaneous sample for SOC4 and SOC5. Lowest three bits of CHSEL field define the pair of channels to be converted. EOC4 and EOC5 associated with SOC4 and SOC5 pair. SOC4 and SOC5 results will be placed in ADCRESULT4 and ADCRESULT5 registers, respectively.</p> |
| 1   | SIMULEN2 | <p>0</p> <p>1</p> | <p>Simultaneous sampling enable for SOC2/SOC3. Couples SOC2 and SOC3 in simultaneous sampling mode. See <a href="#">Section 8.5</a> for details. This bit should not be set when the ADC is actively converting SOC2 or SOC3.</p> <p>Single sample mode set for SOC2 and SOC3. All bits of CHSEL field define channel to be converted. EOC2 associated with SOC2. EOC3 associated with SOC3. SOC2 result placed in ADCRESULT2 register. SOC3 result placed in ADCRESULT3.</p> <p>Simultaneous sample for SOC2 and SOC3. Lowest three bits of CHSEL field define the pair of channels to be converted. EOC2 and EOC3 associated with SOC2 and SOC3 pair. SOC2 and SOC3 results will be placed in ADCRESULT2 and ADCRESULT3 registers, respectively.</p> |
| 0   | SIMULEN0 | <p>0</p> <p>1</p> | <p>Simultaneous sampling enable for SOC0/SOC1. Couples SOC0 and SOC1 in simultaneous sampling mode. See <a href="#">Section 8.5</a> for details. This bit should not be set when the ADC is actively converting SOC0 or SOC1.</p> <p>Single sample mode set for SOC0 and SOC1. All bits of CHSEL field define channel to be converted. EOC0 associated with SOC0. EOC1 associated with SOC1. SOC0 result placed in ADCRESULT0 register. SOC1 result placed in ADCRESULT1.</p> <p>Simultaneous sample for SOC0 and SOC1. Lowest three bits of CHSEL field define the pair of channels to be converted. EOC0 and EOC1 associated with SOC0 and SOC1 pair. SOC0 and SOC1 results will be placed in ADCRESULT0 and ADCRESULT1 registers, respectively.</p> |

(1) This register is EALLOW protected.

### 8.13.5.2 ADC Interrupt Trigger SOC Select 1 Register (ADCINTSOCSEL1)

**Figure 8-27. ADC Interrupt Trigger SOC Select 1 Register (ADCINTSOCSEL1)**

|       |       |       |       |       |       |       |       |   |   |   |   |   |   |   |   |
|-------|-------|-------|-------|-------|-------|-------|-------|---|---|---|---|---|---|---|---|
| 15    | 14    | 13    | 12    | 11    | 10    | 9     | 8     | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| SOC7  | SOC6  | SOC5  | SOC4  | SOC3  | SOC2  | SOC1  | SOC0  |   |   |   |   |   |   |   |   |
| R/W-0 | R/W-0 | R/W-0 | R/W-0 | R/W-0 | R/W-0 | R/W-0 | R/W-0 |   |   |   |   |   |   |   |   |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 8-14. ADC Interrupt Trigger SOC Select 1 Register (ADCINTSOCSEL1) Register Field Descriptions**

| Bit   | Field                | Value | Description <sup>(1)</sup>                                                                                                                                                                                                                          |
|-------|----------------------|-------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15--0 | SOCx<br>(x = 7 to 0) |       | SOCx ADC Interrupt Trigger Select. Select ADCINT to trigger SOCx. The ADCINT trigger is OR'ed with the trigger selected by the TRIGSEL field in the ADCSOCxCTL register, as well as the software force trigger signal from the ADCSOCFRC1 register. |
|       |                      | 00    | No ADCINT will trigger SOCx.                                                                                                                                                                                                                        |
|       |                      | 01    | ADCINT1 will trigger SOCx.                                                                                                                                                                                                                          |
|       |                      | 10    | ADCINT2 will trigger SOCx.                                                                                                                                                                                                                          |
|       |                      | 11    | Invalid selection.                                                                                                                                                                                                                                  |

(1) This register is EALLOW protected.

### 8.13.5.3 ADC Interrupt Trigger SOC Select 2 Register (ADCINTSOCSEL2)

**Figure 8-28. ADC Interrupt Trigger SOC Select 2 Register (ADCINTSOCSEL2)**

|       |       |       |       |       |       |       |       |   |   |   |   |   |   |   |   |
|-------|-------|-------|-------|-------|-------|-------|-------|---|---|---|---|---|---|---|---|
| 15    | 14    | 13    | 12    | 11    | 10    | 9     | 8     | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| SOC15 | SOC14 | SOC13 | SOC12 | SOC11 | SOC10 | SOC9  | SOC8  |   |   |   |   |   |   |   |   |
| R/W-0 | R/W-0 | R/W-0 | R/W-0 | R/W-0 | R/W-0 | R/W-0 | R/W-0 |   |   |   |   |   |   |   |   |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 8-15. ADC Interrupt Trigger SOC Select 2 Register (ADCINTSOCSEL2) Field Descriptions**

| Bit  | Field                 | Value | Description <sup>(1)</sup>                                                                                                                                                                                                                          |
|------|-----------------------|-------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-0 | SOCx<br>(x = 15 to 8) |       | SOCx ADC Interrupt Trigger Select. Select ADCINT to trigger SOCx. The ADCINT trigger is OR'ed with the trigger selected by the TRIGSEL field in the ADCSOCxCTL register, as well as the software force trigger signal from the ADCSOCFRC1 register. |
|      |                       | 00    | No ADCINT will trigger SOCx.                                                                                                                                                                                                                        |
|      |                       | 01    | ADCINT1 will trigger SOCx.                                                                                                                                                                                                                          |
|      |                       | 10    | ADCINT2 will trigger SOCx.                                                                                                                                                                                                                          |
|      |                       | 11    | Invalid selection.                                                                                                                                                                                                                                  |

(1) This register is EALLOW protected.

### 8.13.5.4 ADC SOC Flag 1 Register (ADCSOCFLG1)

**Figure 8-29. ADC SOC Flag 1 Register (ADCSOCFLG1)**

|       |       |       |       |       |       |      |      |
|-------|-------|-------|-------|-------|-------|------|------|
| 15    | 14    | 13    | 12    | 11    | 10    | 9    | 8    |
| SOC15 | SOC14 | SOC13 | SOC12 | SOC11 | SOC10 | SOC9 | SOC8 |
| R-0   | R-0   | R-0   | R-0   | R-0   | R-0   | R-0  | R-0  |
| 7     | 6     | 5     | 4     | 3     | 2     | 1    | 0    |
| SOC7  | SOC6  | SOC5  | SOC4  | SOC3  | SOC2  | SOC1 | SOC0 |
| R-0   | R-0   | R-0   | R-0   | R-0   | R-0   | R-0  | R-0  |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 8-16. ADC SOC Flag 1 Register (ADCSOCFLG1) Field Descriptions**

| Bit  | Field                 | Value  | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|------|-----------------------|--------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-0 | SOCx<br>(x = 15 to 0) | 0<br>1 | <p>SOCx Start of Conversion Flag. Indicates the state of individual SOC conversions.</p> <p>No sample pending for SOCx.</p> <p>Trigger has been received and sample is pending for SOCx.</p> <p>The bit will be automatically cleared when the respective SOCx conversion is started. If contention exists where this bit receives both a request to set <b>and</b> a request to clear on the same cycle, regardless of the source of either, this bit will be set and the request to clear will be ignored. In this case the overflow bit in the ADCSOCOVF1 register will not be affected regardless of whether this bit was previously set or not.</p> |



### 8.13.5.5 ADC SOC Force 1 Register (ADCSOCFRC1)

**Figure 8-30. ADC SOC Force 1 Register (ADCSOCFRC1)**

|       |       |       |       |       |       |       |       |
|-------|-------|-------|-------|-------|-------|-------|-------|
| 15    | 14    | 13    | 12    | 11    | 10    | 9     | 8     |
| SOC15 | SOC14 | SOC13 | SOC12 | SOC11 | SOC10 | SOC9  | SOC8  |
| R/W-0 | R/W-0 | R/W-0 | R/W-0 | R/W-0 | R/W-0 | R/W-0 | R/W-0 |
| 7     | 6     | 5     | 4     | 3     | 2     | 1     | 0     |
| SOC7  | SOC6  | SOC5  | SOC4  | SOC3  | SOC2  | SOC1  | SOC0  |
| R/W-0 | R/W-0 | R/W-0 | R/W-0 | R/W-0 | R/W-0 | R/W-0 | R/W-0 |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 8-17. ADC SOC Force 1 Register (ADCSOCFRC1) Field Descriptions**

| Bit  | Field                 | Value | Description                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|------|-----------------------|-------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-0 | SOCx<br>(x = 15 to 0) | 0     | SOCx Force Start of Conversion Flag. Writing a 1 will force to 1 the respective SOCx flag bit in the ADCSOCFLG1 register. This can be used to initiate a software initiated conversion. Writes of 0 are ignored.<br>No action.                                                                                                                                                                                                                      |
|      |                       | 1     | Force SOCx flag bit to 1. This will cause a conversion to start once priority is given to SOCx.<br>If software tries to set this bit on the same clock cycle that hardware tries to clear the SOCx bit in the ADCSOCFLG1 register, then software has priority and the ADCSOCFLG1 bit will be set. In this case the overflow bit in the ADCSOCOVF1 register will not be affected regardless of whether the ADCSOCFLG1 bit was previously set or not. |

### 8.13.5.6 ADC SOC Overflow 1 Register (ADCSOCOVF1)

**Figure 8-31. ADC SOC Overflow 1 Register (ADCSOCOVF1)**

|       |       |       |       |       |       |      |      |
|-------|-------|-------|-------|-------|-------|------|------|
| 15    | 14    | 13    | 12    | 11    | 10    | 9    | 8    |
| SOC15 | SOC14 | SOC13 | SOC12 | SOC11 | SOC10 | SOC9 | SOC8 |
| R-0   | R-0   | R-0   | R-0   | R-0   | R-0   | R-0  | R-0  |
| 7     | 6     | 5     | 4     | 3     | 2     | 1    | 0    |
| SOC7  | SOC6  | SOC5  | SOC4  | SOC3  | SOC2  | SOC1 | SOC0 |
| R-0   | R-0   | R-0   | R-0   | R-0   | R-0   | R-0  | R-0  |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 8-18. ADC SOC Overflow 1 Register (ADCSOCOVF1) Field Descriptions**

| Bit  | Field                 | Value  | Description                                                                                                                                                                                                                                                                                                     |
|------|-----------------------|--------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-0 | SOCx<br>(x = 15 to 0) | 0<br>1 | SOCx Start of Conversion Overflow Flag. Indicates an SOCx event was generated while an existing SOCx event was already pending.<br>No SOCx event overflow<br>SOCx event overflow<br>An overflow condition does not stop SOCx events from being processed. It simply is an indication that a trigger was missed. |

### 8.13.5.7 ADC SOC Overflow Clear 1 Register (ADCSOCOVFCLR1)

**Figure 8-32. ADC SOC Overflow Clear 1 Register (ADCSOCOVFCLR1)**

|       |       |       |       |       |       |       |       |
|-------|-------|-------|-------|-------|-------|-------|-------|
| 15    | 14    | 13    | 12    | 11    | 10    | 9     | 8     |
| SOC15 | SOC14 | SOC13 | SOC12 | SOC11 | SOC10 | SOC9  | SOC8  |
| R/W-0 | R/W-0 | R/W-0 | R/W-0 | R/W-0 | R/W-0 | R/W-0 | R/W-0 |
| 7     | 6     | 5     | 4     | 3     | 2     | 1     | 0     |
| SOC7  | SOC6  | SOC5  | SOC4  | SOC3  | SOC2  | SOC1  | SOC0  |
| R/W-0 | R/W-0 | R/W-0 | R/W-0 | R/W-0 | R/W-0 | R/W-0 | R/W-0 |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 8-19. ADC SOC Overflow Clear 1 Register (ADCSOCOVFCLR1) Field Descriptions**

| Bit  | Field                 | Value  | Description                                                                                                                                                                                                                                                                                                                                                                                                |
|------|-----------------------|--------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-0 | SOCx<br>(x = 15 to 0) | 0<br>1 | SOCx Clear Start of Conversion Overflow Flag. Writing a 1 will clear the respective SOCx overflow flag in the ADCSOCOVF1 register. Writes of 0 are ignored.<br>No action.<br>Clear SOCx overflow flag.<br>If software tries to set this bit on the same clock cycle that hardware tries to set the overflow bit in the ADCSOCOVF1 register, then hardware has priority and the ADCSOCOVF1 bit will be set. |

### 8.13.5.8 ADC SOC0-SOC15 Control Registers (ADCSOCxCTL)

**Figure 8-33. ADC SOC0-SOC15 Control Registers (ADCSOCxCTL)**

|         |          |       |       |   |   |   |
|---------|----------|-------|-------|---|---|---|
| 15      | 11       | 10    | 9     | 6 | 5 | 0 |
| TRIGSEL | Reserved | CHSEL | ACQPS |   |   |   |
| R/W-0   | R-0      | R/W-0 | R/W-0 |   |   |   |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 8-20. ADC SOC0-SOC15 Control Registers (ADCSOCxCTL) Register Field Descriptions**

| Bit   | Field    | Value | Description <sup>(1)</sup>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|-------|----------|-------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-11 | TRIGSEL  |       | <p>SOCx Trigger Source Select.</p> <p>Configures which trigger will set the respective SOCx flag in the ADCSOCFLG1 register to initiate a conversion to start once priority is given to SOCx. This setting can be overridden by the respective SOCx field in the ADCINTSOCSEL1 or ADCINTSOCSEL2 register.</p> <p>00h ADCTRIG0 - Software only.</p> <p>01h ADCTRIG1 - CPU Timer 0, TINT0n</p> <p>02h ADCTRIG2 - CPU Timer 1, TINT1n</p> <p>03h ADCTRIG3 - CPU Timer 2, TINT2n</p> <p>04h ADCTRIG4 - XINT2, XINT2SOC</p> <p>05h ADCTRIG5 - ePWM1, ADCSOCA</p> <p>06h ADCTRIG6 - ePWM1, ADCSOCA</p> <p>07h ADCTRIG7 - ePWM2, ADCSOCA</p> <p>08h ADCTRIG8 - ePWM2, ADCSOCA</p> <p>09h ADCTRIG9 - ePWM3, ADCSOCA</p> <p>0Ah ADCTRIG10 - ePWM3, ADCSOCA</p> <p>0Bh ADCTRIG11 - ePWM4, ADCSOCA</p> <p>0Ch ADCTRIG12 - ePWM4, ADCSOCA</p> <p>0Dh ADCTRIG13 - ePWM5, ADCSOCA</p> <p>0Eh ADCTRIG14 - ePWM5, ADCSOCA</p> <p>0Fh ADCTRIG15 - ePWM6, ADCSOCA</p> <p>10h ADCTRIG16 - ePWM6, ADCSOCA</p> <p>11h ADCTRIG17 - ePWM7, ADCSOCA</p> <p>12h ADCTRIG18 - ePWM7, ADCSOCA</p> <p>13h ADCTRIG19 - ePWM8, ADCSOCA</p> <p>14h ADCTRIG20 - ePWM8, ADCSOCA</p> <p>Others Invalid selection.</p> |
| 10    | Reserved |       | Reads return a zero; Writes have no effect.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |

**Table 8-20. ADC SOC0-SOC15 Control Registers (ADC SOCxCTL) Register Field Descriptions (continued)**

| Bit | Field | Value | Description <sup>(1)</sup>                                                                 |
|-----|-------|-------|--------------------------------------------------------------------------------------------|
| 9-6 | CHSEL |       | SOCx Channel Select. Selects the channel to be converted when SOCx is received by the ADC. |
|     |       |       | Sequential Sampling Mode (SIMULENx = 0):                                                   |
|     |       | 0h    | ADCINA0                                                                                    |
|     |       | 1h    | ADCINA1                                                                                    |
|     |       | 2h    | ADCINA2                                                                                    |
|     |       | 3h    | ADCINA3                                                                                    |
|     |       | 4h    | ADCINA4                                                                                    |
|     |       | 5h    | ADCINA5                                                                                    |
|     |       | 6h    | ADCINA6                                                                                    |
|     |       | 7h    | ADCINA7                                                                                    |
|     |       | 8h    | ADCINB0                                                                                    |
|     |       | 9h    | ADCINB1                                                                                    |
|     |       | Ah    | ADCINB2                                                                                    |
|     |       | Bh    | ADCINB3                                                                                    |
|     |       | Ch    | ADCINB4                                                                                    |
|     |       | Dh    | ADCINB5                                                                                    |
|     |       | Eh    | ADCINB6                                                                                    |
|     |       | Fh    | ADCINB7                                                                                    |
|     |       |       | Simultaneous Sampling Mode (SIMULENx = 1):                                                 |
|     |       | 0h    | ADCINA0/ADCINB0 pair                                                                       |
|     |       | 1h    | ADCINA1/ADCINB1 pair                                                                       |
|     |       | 2h    | ADCINA2/ADCINB2 pair                                                                       |
|     |       | 3h    | ADCINA3/ADCINB3 pair                                                                       |
|     |       | 4h    | ADCINA4/ADCINB4 pair                                                                       |
|     |       | 5h    | ADCINA5/ADCINB5 pair                                                                       |
|     |       | 6h    | ADCINA6/ADCINB6 pair                                                                       |
|     |       | 7h    | ADCINA7/ADCINB7 pair                                                                       |
|     |       | 8h    | Invalid selection.                                                                         |
|     |       | 9h    | Invalid selection.                                                                         |
|     |       | Ah    | Invalid selection.                                                                         |
|     |       | Bh    | Invalid selection.                                                                         |
|     |       | Ch    | Invalid selection.                                                                         |
|     |       | Dh    | Invalid selection.                                                                         |
|     |       | Eh    | Invalid selection.                                                                         |
|     |       | Fh    | Invalid selection.                                                                         |

**Table 8-20. ADC SOC0-SOC15 Control Registers (ADCSOCxCTL) Register Field Descriptions (continued)**

| Bit                                                                                                                          | Field | Value | Description <sup>(1)</sup>                                               |
|------------------------------------------------------------------------------------------------------------------------------|-------|-------|--------------------------------------------------------------------------|
| 5-0                                                                                                                          | ACQPS |       | SOCx Acquisition Prescale. Controls the sample and hold window for SOCx. |
|                                                                                                                              |       | 00h   | Invalid selection.                                                       |
|                                                                                                                              |       | 01h   | Invalid selection.                                                       |
|                                                                                                                              |       | 02h   | Invalid selection.                                                       |
|                                                                                                                              |       | 03h   | Invalid selection.                                                       |
|                                                                                                                              |       | 04h   | Invalid selection.                                                       |
|                                                                                                                              |       | 05h   | Invalid selection.                                                       |
|                                                                                                                              |       | 06h   | Sample window is 7 cycles long (6 + 1 clock cycles).                     |
|                                                                                                                              |       | 07h   | Sample window is 8 cycles long (7 + 1 clock cycles).                     |
|                                                                                                                              |       | 08h   | Sample window is 9 cycles long (8 + 1 clock cycles).                     |
|                                                                                                                              |       | 09h   | Sample window is 10 cycles long (9 + 1 clock cycles).                    |
|                                                                                                                              |       | ...   | ...                                                                      |
|                                                                                                                              |       | 3Fh   | Sample window is 64 cycles long (63 + 1 clock cycles).                   |
| Other invalid selections: 10h, 11h, 12h, 13h, 14h, 1Dh, 1Eh, 1Fh, 20h, 21h, 2Ah, 2Bh, 2Ch, 2Dh, 2Eh, 37h, 38h, 39h, 3Ah, 3Bh |       |       |                                                                          |

(1) This register is EALLOW protected.

## 8.13.6 ADC Calibration Registers

### 8.13.6.1 ADC Reference/Gain Trim Register (ADCREFTTRIM)

**Figure 8-34. ADC Reference/Gain Trim Register (ADCREFTTRIM)**

|          |    |    |                  |   |                |   |              |
|----------|----|----|------------------|---|----------------|---|--------------|
| 15       | 14 | 13 | 9                | 8 | 5              | 4 | 0            |
| Reserved |    |    | EXTREF_FINE_TRIM |   | BG_COARSE_TRIM |   | BG_FINE_TRIM |
| R-0      |    |    | R/W-0            |   | R/W-0          |   | R/W-0        |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 8-21. ADC Reference/Gain Trim Register (ADCREFTTRIM) Field Descriptions**

| Bit   | Field            | Value | Description <sup>(1)</sup>                                                                                                                                               |
|-------|------------------|-------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-14 | Reserved         |       | Reads return a zero; Writes have no effect.                                                                                                                              |
| 13-9  | EXTREF_FINE_TRIM |       | ADC External reference Fine Trim. These bits should not be modified after device boot code loads them with the factory trim setting.                                     |
| 8-5   | BG_COARSE_TRIM   |       | ADC Internal Bandgap Fine Trim. These bits should not be modified after device boot code loads them with the factory trim setting.                                       |
| 4-0   | BG_FINE_TRIM     |       | ADC Internal Bandgap Coarse Trim. A maximum value of 30 is supported. These bits should not be modified after device boot code loads them with the factory trim setting. |

(1) This register is EALLOW protected.

### 8.13.6.2 ADC Offset Trim Register (ADCOFFTRIM)

**Figure 8-35. ADC Offset Trim Register (ADCOFFTRIM)**

|          |   |         |   |
|----------|---|---------|---|
| 15       | 9 | 8       | 0 |
| Reserved |   | OFFTRIM |   |
| R-0      |   | R/W-0   |   |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 8-22. ADC Offset Trim Register (ADCOFFTRIM) Field Descriptions**

| Bit  | Field    | Value | Description <sup>(1)</sup>                                                                                                                                                                                                         |
|------|----------|-------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-9 | Reserved |       | Reads return a zero; Writes have no effect.                                                                                                                                                                                        |
| 8-0  | OFFTRIM  |       | ADC Offset Trim. 2's complement of ADC offset. Range is -256 to +255. These bits are loaded by device boot code with a factory trim setting. Modification of this default setting can be made to correct any board induced offset. |

(1) This register is EALLOW protected.

### 8.13.7 Comparator Hysteresis Control Register (COMPHYSTCTL)

**Figure 8-36. Comparator Hysteresis Control Register (COMPHYSTCTL)**

|          |                    |          |                    |          |                    |          |        |     |     |
|----------|--------------------|----------|--------------------|----------|--------------------|----------|--------|-----|-----|
| 15       | 12                 | 11       | 10                 | 7        | 6                  | 5        | 2      | 1   | 0   |
| Reserved | COMP3_HYST_DISABLE | Reserved | COMP2_HYST_DISABLE | Reserved | COMP1_HYST_DISABLE | Reserved |        |     |     |
| R-0      | R/W--0             | R-0      | R/W--0             | R-0      | R/W--0             | R-0      | R/W--0 | R-0 | R-0 |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

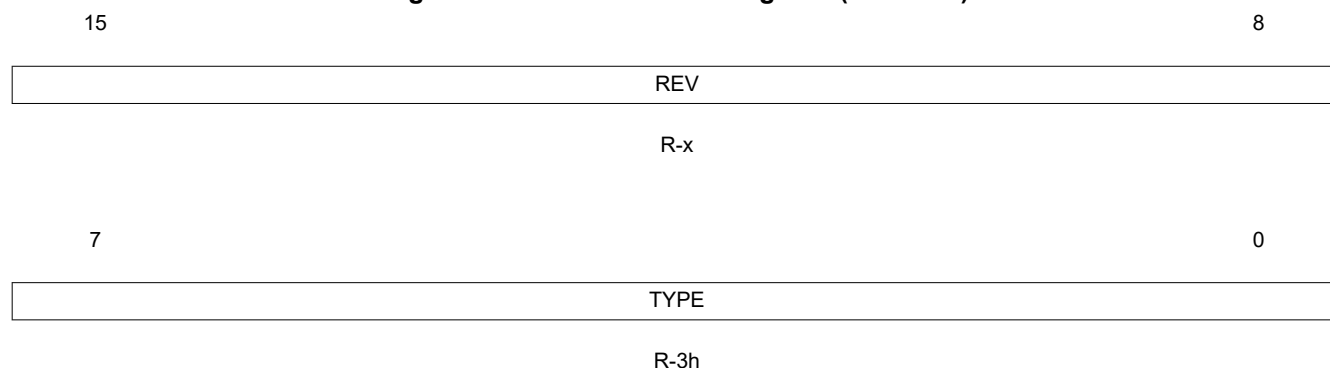
**Table 8-23. Comparator Hysteresis Control Register (COMPHYSTCTL) Field Descriptions**

| Bit   | Field              | Value  | Description <sup>(1)</sup>                                                   |
|-------|--------------------|--------|------------------------------------------------------------------------------|
| 15-12 | Reserved           |        | Reads return a zero; Writes have no effect.                                  |
| 11    | COMP3_HYST_DISABLE | 0<br>1 | Comparator 3 Hysteresis disable<br>Hysteresis enabled<br>Hysteresis disabled |
| 10-7  | Reserved           |        | Reserved                                                                     |
| 6     | COMP2_HYST_DISABLE | 0<br>1 | Comparator 2 Hysteresis disable<br>Hysteresis enabled<br>Hysteresis disabled |
| 5-2   | Reserved           |        | Reserved                                                                     |
| 1     | COMP1_HYST_DISABLE | 0<br>1 | Comparator 1 Hysteresis disable<br>Hysteresis enabled<br>Hysteresis disabled |
| 0     | Reserved           |        | Reserved                                                                     |

(1) This register is EALLOW protected.

### 8.13.8 ADC Revision Register (ADCREV)

**Figure 8-37. ADC Revision Register (ADCREV)**



LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

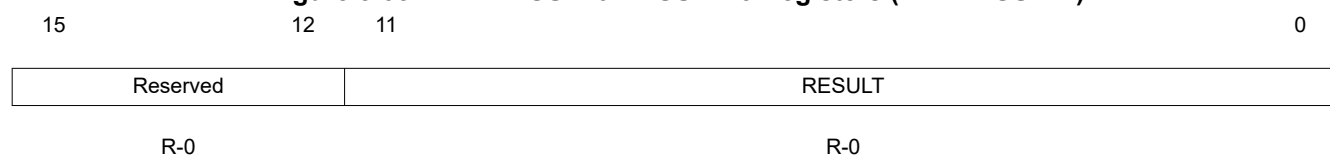
**Table 8-24. ADC Revision Register (ADCREV) Field Descriptions**

| Bit  | Field | Value | Description                                                                                             |
|------|-------|-------|---------------------------------------------------------------------------------------------------------|
| 15-8 | REV   |       | ADC Revision. To allow documentation of differences between revisions. First version is labeled as 00h. |
| 7-0  | TYPE  | 3     | ADC Type. Always set to 3 for this type ADC.                                                            |

### 8.13.9 ADC RESULT0-RESULT15 Registers (ADCRESULTx)

The ADC Result Registers are found in Peripheral Frame 0 (PF0). In the header files, the ADCRESULTx registers are located in the AdcResult register file, not AdcRegs.

**Figure 8-38. ADC RESULT0-RESULT15 Registers (ADCRESULTx)**



LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 8-25. ADC RESULT0-ADCRESULT15 Registers (ADCRESULTx) Field Descriptions**

| Bit   | Field    | Value | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|-------|----------|-------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-12 | Reserved |       | Reads return a zero; Writes have no effect.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| 11-0  | RESULT   |       | 12-bit right-justified ADC result<br>Sequential Sampling Mode (SIMULENx = 0):<br>After the ADC completes a conversion of an SOCx, the digital result is placed in the corresponding ADCRESULTx register. For example, if SOC4 is configured to sample ADCINA1, the completed result of that conversion will be placed in ADCRESULT4.<br>Simultaneous Sampling Mode (SIMULENx = 1):<br>After the ADC completes a conversion of a channel pair, the digital results are found in the corresponding ADCRESULTx and ADCRESULTx+1 registers (assuming x is even). For example, for SOC4, the completed results of those conversions will be placed in ADCRESULT4 and ADCRESULT5. See 1.11 for timings of when this register is written. |



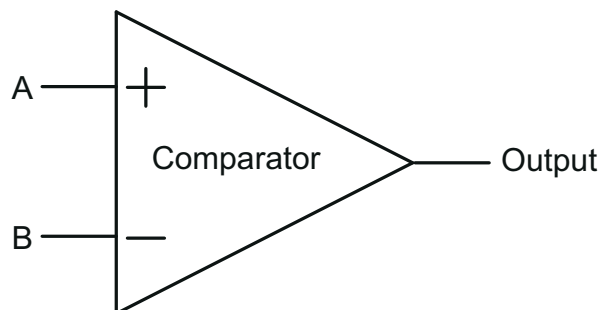
The Comparator (COMP) module described in this chapter is a Type 0 Comparator. See the [C2000 Real-Time Control Peripherals Reference Guide](#) for a list of all devices with modules of the same type, to determine the differences between the types, and for a list of device-specific differences within a type.

|                                             |            |
|---------------------------------------------|------------|
| <b>9.1 Introduction.....</b>                | <b>560</b> |
| <b>9.2 Comparator Function.....</b>         | <b>561</b> |
| <b>9.3 DAC Reference.....</b>               | <b>561</b> |
| <b>9.4 Ramp Generator Input.....</b>        | <b>562</b> |
| <b>9.5 Initialization.....</b>              | <b>563</b> |
| <b>9.6 Digital Domain Manipulation.....</b> | <b>563</b> |
| <b>9.7 Comparator Registers.....</b>        | <b>564</b> |



## 9.2 Comparator Function

The comparator is an analog comparator module (Figure 9-2) and the output is asynchronous to the system clock. The truth table for the comparator is shown in Table 9-1.



**Figure 9-2. Comparator**

**Table 9-1. Comparator Truth Table**

| Voltages              | Output |
|-----------------------|--------|
| Voltage A > Voltage B | 1      |
| Voltage B > Voltage A | 0      |

There is no definition for the condition Voltage A = Voltage B, since there is hysteresis in the response of the comparator output. Refer to the device data sheet for the value of this hysteresis. This also limits the sensitivity of the comparator output to noise on the input voltages.

The output state of the comparator, after qualification, is reflected by the COMPSTS bit in the COMPSTS register. The COMPSTS register will not update if the module clock is not enabled.

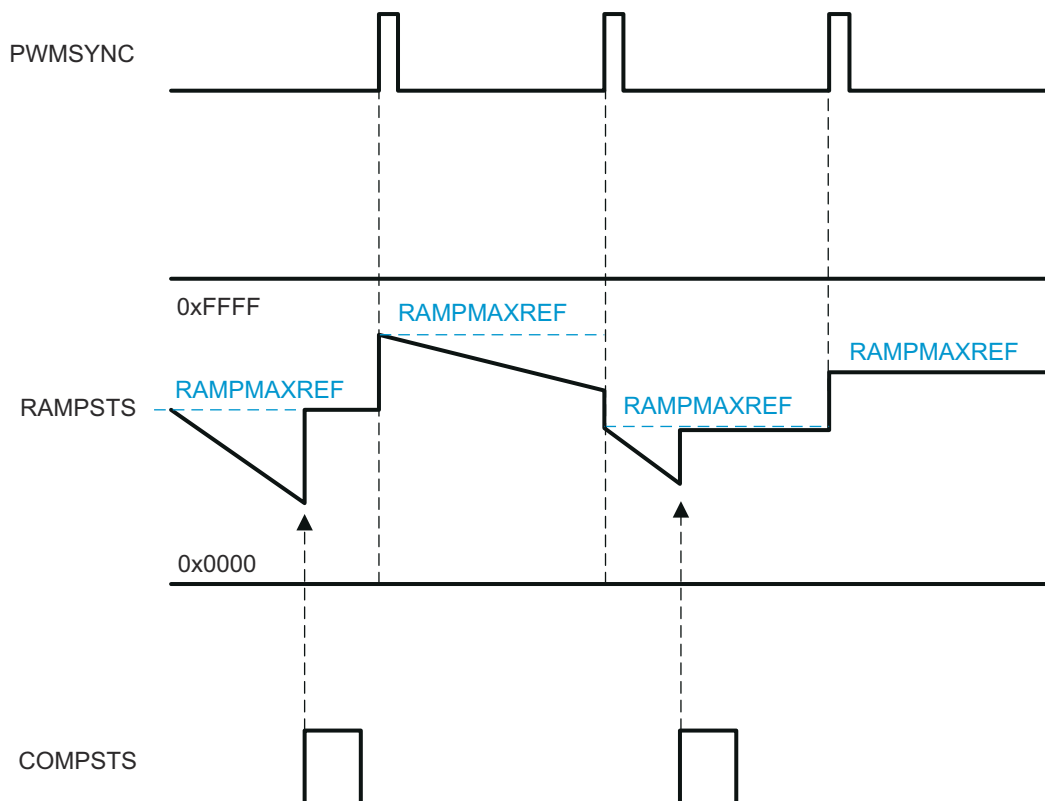
## 9.3 DAC Reference

Each comparator block contains an internal 10-bit voltage DAC reference that can be used to supply the inverting input (B side input) of the comparator. The voltage output of the DAC is controlled by the DACVAL bit field in the DACVAL register. The output of the DAC is given by the equation:

$$V = \frac{\text{DACVAL} * (\text{VDDA} - \text{VSSA})}{1023}$$

Since the DAC is also in the analog domain it does not require a clock to maintain its voltage output. A clock is required, however, to modify the digital inputs that control the DAC.





**Figure 9-4. Ramp Generator Behavior**

## 9.5 Initialization

There are 2 steps that must be performed prior to using the comparator block:

1. Enable the Band Gap inside the ADC by writing a 1 to the ADCBGPWD bit inside ADCCTL1.
2. Enable the comparator block by writing a 1 to the COMPDACEN bit in the COMPCTL register.

## 9.6 Digital Domain Manipulation

At the output of the comparator there are two more functional blocks that can be used to influence the behavior of the comparator output. They are:

1. Inverter circuit: Controlled by the CMPINV bit in the COMPCTL register; will apply a logical NOT to the output of the comparator. This function is asynchronous, while its control requires a clock present in order to change its value.
2. Qualification block: Controlled by the QUALSEL bit field in the COMPCTL register, and gated by the SYNCSEL bit in the COMPCTL register. This block can be used as a simple filter to only pass the output of the comparator once it is synchronized to the system clock. and qualified by the number of system clocks defined in QUALSEL bit field.

## 9.7 Comparator Registers

This device has the comparator modules listed in [Table 9-2](#) and the registers listed in [Table 9-3](#).

**Table 9-2. Comparator Modules**

| Name  | Address Range | Size(x16) | Description |
|-------|---------------|-----------|-------------|
| COMP1 | 6400h – 641Fh | 1         | Comparator  |
| COMP2 | 6420h – 643Fh | 1         | Comparator  |
| COMP3 | 6440h – 645Fh | 1         | Comparator  |

**Table 9-3. Comparator Module Registers**

| Name              | Address Range (base) | Size (x16) | Description                               | Bit Description               |
|-------------------|----------------------|------------|-------------------------------------------|-------------------------------|
| COMPCTL           | 0x00                 | 1          | Comparator Control <sup>(1)</sup>         | <a href="#">Section 9.7.1</a> |
| Reserved          | 0x01                 | 1          | Reserved                                  | -                             |
| COMPSTS           | 0x02                 | 1          | Compare Output Status                     | <a href="#">Section 9.7.2</a> |
| Reserved          | 0x03                 | 1          | Reserved                                  | -                             |
| DACCTL            | 0x04                 | 1          | DAC Control <sup>(1)</sup>                | <a href="#">Section 9.7.3</a> |
| Reserved          | 0x05                 | 1          | Reserved                                  | -                             |
| DACVAL            | 0x06                 | 1          | 10-bit DAC Value                          | <a href="#">Section 9.7.4</a> |
| Reserved          | 0x07                 | 1          | Reserved                                  | -                             |
| RAMPMAXREF_ACTIVE | 0x08                 | 1          | Ramp Generator Maximum Reference (Active) | <a href="#">Section 9.7.5</a> |
| Reserved          | 0x09                 | 1          | Reserved                                  | -                             |
| RAMPMAXREF_SHDW   | 0x0A                 | 1          | Ramp Generator Maximum Reference (Shadow) | <a href="#">Section 9.7.6</a> |
| Reserved          | 0x0B                 | 1          | Reserved                                  | -                             |
| RAMPDECVAL_ACTIVE | 0x0C                 | 1          | Ramp Generator Decrement Value (Active)   | <a href="#">Section 9.7.7</a> |
| Reserved          | 0x0D                 | 1          | Reserved                                  | -                             |
| RAMPDECVAL_SHDW   | 0x0E                 | 1          | Ramp Generator Decrement Value (Shadow)   | <a href="#">Section 9.7.8</a> |
| Reserved          | 0x0F                 | 1          | Reserved                                  | -                             |
| RAMPSTS           | 0x10                 | 1          | Ramp Generator Status                     | <a href="#">Section 9.7.9</a> |
| Reserved          | 0x11-0x1F            | 15         | Reserved                                  | -                             |

(1) This register is EALLOW protected.

### 9.7.1 Comparator Control (COMPCTL) Register

**Figure 9-5. Comparator Control (COMPCTL) Register**

|          |  |   |        |   |            |   |           |  |         |
|----------|--|---|--------|---|------------|---|-----------|--|---------|
| 15       |  |   |        |   |            |   | 9         |  | 8       |
| Reserved |  |   |        |   |            |   |           |  | SYNCSEL |
| R-0      |  |   |        |   |            |   |           |  | R/W-0   |
| 7        |  | 3 |        | 2 |            | 1 |           |  | 0       |
| QUALSEL  |  |   | CMPINV |   | COMPSOURCE |   | COMPDACEN |  |         |
| R/W-0    |  |   | R/W-0  |   | R/W-0      |   | R/W-0     |  |         |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 9-4. Comparator Control (COMPCTL) Register Field Descriptions**

| Bit  | Field      | Value                        | Description <sup>(1)</sup>                                                                                                                                                                                                                                                                                                                                                                                                                              |
|------|------------|------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-9 | Reserved   |                              | Reads return a 0; Writes have no effect.                                                                                                                                                                                                                                                                                                                                                                                                                |
| 8    | SYNCSEL    | 0<br>1                       | Synchronization select for output of the comparator before being passed to EPWM/GPIO blocks<br>Asynchronous version of Comparator output is passed<br>Synchronous version of comparator output is passed                                                                                                                                                                                                                                                |
| 7-3  | QUALSEL    | 0h<br>1h<br>2h<br>...<br>1Fh | Qualification Period for synchronized output of the comparator<br>Synchronized value of comparator is passed through<br>Input to the block must be consistent for 2 consecutive clocks before output of Qual block can change<br>Input to the block must be consistent for 3 consecutive clocks before output of Qual block can change<br>...<br>Input to the block must be consistent for 32 consecutive clocks before output of Qual block can change |
| 2    | CMPINV     | 0<br>1                       | Invert select for Comparator<br>Output of comparator is passed<br>Inverted output of comparator is passed                                                                                                                                                                                                                                                                                                                                               |
| 1    | COMPSOURCE | 0<br>1                       | Source select for comparator inverting input<br>Inverting input of comparator connected to internal DAC<br>Inverting input connected to external pin                                                                                                                                                                                                                                                                                                    |
| 0    | COMPDACEN  | 0<br>1                       | Comparator/DAC Enable<br>Comparator/DAC logic is powered down.<br>Comparator/DAC logic is powered up.                                                                                                                                                                                                                                                                                                                                                   |

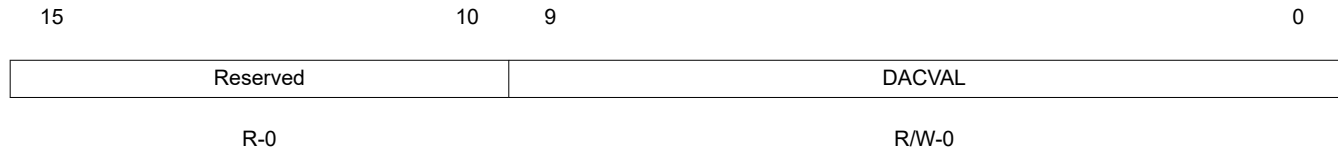
(1) This register is EALLOW protected.





#### 9.7.4 DAC Value (DACVAL) Register

**Figure 9-8. DAC Value (DACVAL) Register**



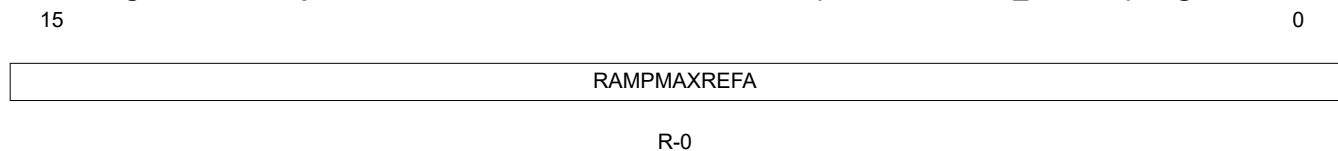
LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 9-7. DAC Value (DACVAL) Register Field Descriptions**

| Bit   | Field    | Value  | Description                                                 |
|-------|----------|--------|-------------------------------------------------------------|
| 15-10 | Reserved |        | Reads return zero and writes have no effect.                |
| 9-0   | DACVAL   | 0-3FFh | DAC Value bits, scales the output of the DAC from 0 – 1023. |

#### 9.7.5 Ramp Generator Maximum Reference Active (RAMPMAXREF\_ACTIVE) Register

**Figure 9-9. Ramp Generator Maximum Reference Active (RAMPMAXREF\_ACTIVE) Register**



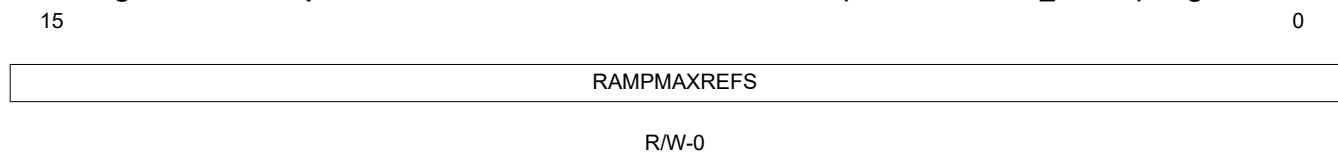
LEGEND: R = Read only; -n = value after reset

**Table 9-8. Ramp Generator Maximum Reference Active (RAMPMAXREF\_ACTIVE) Register Field Descriptions**

| Bit  | Field       | Value   | Description                                                                                                                                      |
|------|-------------|---------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-0 | RAMPMAXREFA | 0-FFFFh | 16-bit maximum reference active value for down ramp generator.<br>This value is loaded from RAMPMAXREF_SHDW when the PWMSYNC signal is received. |

#### 9.7.6 Ramp Generator Maximum Reference Shadow (RAMPMAXREF\_SHDW) Register

**Figure 9-10. Ramp Generator Maximum Reference Shadow (RAMPMAXREF\_SHDW) Register**



LEGEND: R/W = Read/Write; -n = value after reset

**Table 9-9. Ramp Generator Maximum Reference Shadow (RAMPMAXREF\_SHDW) Register Field Descriptions**

| Bit  | Field       | Value   | Description                                                    |
|------|-------------|---------|----------------------------------------------------------------|
| 15-0 | RAMPMAXREFS | 0-FFFFh | 16-bit maximum reference shadow value for down ramp generator. |

### 9.7.7 Ramp Generator Decrement Value Active (RAMPDECVAL\_ACTIVE) Register

**Figure 9-11. Ramp Generator Decrement Value Active (RAMPDECVAL\_ACTIVE) Register**

15

0

|             |
|-------------|
| RAMPDECVALA |
|-------------|

R-0

LEGEND: R = Read only; -n = value after reset

**Table 9-10. Ramp Generator Decrement Value Active (RAMPDECVAL\_ACTIVE) Register Field Descriptions**

| Bit  | Field       | Value   | Description                                                                                                                              |
|------|-------------|---------|------------------------------------------------------------------------------------------------------------------------------------------|
| 15-0 | RAMPDECVALA | 0-FFFFh | 16-bit decrement active value for down ramp generator.<br>This value is loaded from RAMPDECVAL_SHDW when the PWMSYNC signal is received. |

### 9.7.8 Ramp Generator Decrement Value Shadow (RAMPDECVAL\_SHDW) Register

**Figure 9-12. Ramp Generator Decrement Value Shadow (RAMPDECVAL\_SHDW) Register**

15

0

|             |
|-------------|
| RAMPDECVALS |
|-------------|

R/W-0

LEGEND: R/W = Read/Write; -n = value after reset

**Table 9-11. Ramp Generator Decrement Value Shadow (RAMPDECVAL\_SHDW) Register Field Descriptions**

| Bit  | Field       | Value   | Description                                            |
|------|-------------|---------|--------------------------------------------------------|
| 15-0 | RAMPDECVALS | 0-FFFFh | 16-bit decrement shadow value for down ramp generator. |

### 9.7.9 Ramp Generator Status (RAMPSTS) Register

**Figure 9-13. Ramp Generator Status (RAMPSTS) Register**

15

0

|           |
|-----------|
| RAMPVALUE |
|-----------|

R-0

LEGEND: R = Read only; -n = value after reset

**Table 9-12. Ramp Generator Status (RAMPSTS) Register Field Descriptions**

| Bit  | Field     | Value   | Description                          |
|------|-----------|---------|--------------------------------------|
| 15-0 | RAMPVALUE | 0-FFFFh | 16-bit value of down ramp generator. |

The Control Law Accelerator (CLA) Type-0 is an independent, fully-programmable, 32-bit floating-point math processor that brings concurrent control-loop execution to the C28x family. The low interrupt latency of the CLA allows it to read ADC samples "just-in-time." This significantly reduces the ADC sample to output delay to enable faster system response and higher MHz control loops. By using the CLA to service time-critical control loops, the main CPU is free to perform other system tasks such as communications and diagnostics. This chapter provides an overview of the architectural structure and components of the control law accelerator.

|                                              |            |
|----------------------------------------------|------------|
| <b>10.1 Introduction.....</b>                | <b>570</b> |
| <b>10.2 CLA Interface.....</b>               | <b>572</b> |
| <b>10.3 CLA and CPU Arbitration.....</b>     | <b>575</b> |
| <b>10.4 CLA Configuration and Debug.....</b> | <b>578</b> |
| <b>10.5 Pipeline.....</b>                    | <b>582</b> |
| <b>10.6 Instruction Set.....</b>             | <b>588</b> |
| <b>10.7 CLA Registers.....</b>               | <b>709</b> |

## 10.1 Introduction

The Control Law Accelerator extends the capabilities of the C28x CPU by adding parallel processing. Time-critical control loops serviced by the CLA can achieve low ADC sample to output delay. Thus, the CLA enables faster system response and higher frequency control loops. Utilizing the CLA for time-critical tasks frees up the main CPU to perform other system and communication functions concurrently.

### 10.1.1 Features

The following is a list of major features of the CLA:

- C compilers are available for CLA software development.
- Clocked at the same rate as the main CPU (SYSCLKOUT).
- An independent architecture allowing CLA algorithm execution independent of the main C28x CPU.
  - Complete bus architecture:
    - Program Address Bus (PAB) and Program Data Bus (PDB)
    - Data Read Address Bus (DRAB), Data Read Data Bus (DRDB), Data Write Address Bus (DWAB), and Data Write Data Bus (DWDB)
  - Independent eight stage pipeline.
  - 12-bit program counter (MPC)
  - Four 32-bit result registers (MR0-MR3)
  - Two 16-bit auxiliary registers (MAR0, MAR1)
  - Status register (MSTF)
- Instruction set includes:
  - IEEE single-precision (32-bit) floating-point math operations
  - Floating-point math with parallel load or store
  - Floating-point multiply with parallel add or subtract
  - 1/X and 1/sqrt(X) estimations
  - Data type conversions.
  - Conditional branch and call
  - Data load/store operations
- The CLA program code can consist of up to eight tasks or interrupt service routines
  - The start address of each task is specified by the MVECT registers.
  - No limit on task size as long as the tasks fit within the configurable CLA program memory space.
  - One task is serviced at a time until its completion. There is no nesting of tasks.
  - Upon task completion a task-specific interrupt is flagged within the PIE.
  - When a task finishes the next highest-priority pending task is automatically started.
- Task trigger mechanisms:
  - C28x CPU via the IACK instruction
  - Task1 to Task8: trigger sources from peripherals connected to the shared bus on which the CLA assumes secondary ownership.
- Memory and Shared Peripherals:
  - Two dedicated message RAMs for communication between the CLA and the main CPU.
  - The C28x CPU can map CLA program and data memory to the main CPU space or CLA space.

### 10.1.2 CLA Related Collateral

#### Foundational Materials

- [C2000 CLA C Compiler Series](#) (Video)
- [CLA Hands On Workshop](#) (Video)
- [CLA usage in Valley Switching Boost Power Factor Correction \(PFC\) Reference Design](#) (Video)
- [Control Law Accelerator Debug in CCS 4](#) (Video)
- [Control Law Accelerator Example Framework](#) (Video)
- [Enhancing the Computational Performance of the C2000 Microcontroller Family Application Report](#)

## Getting Started Materials

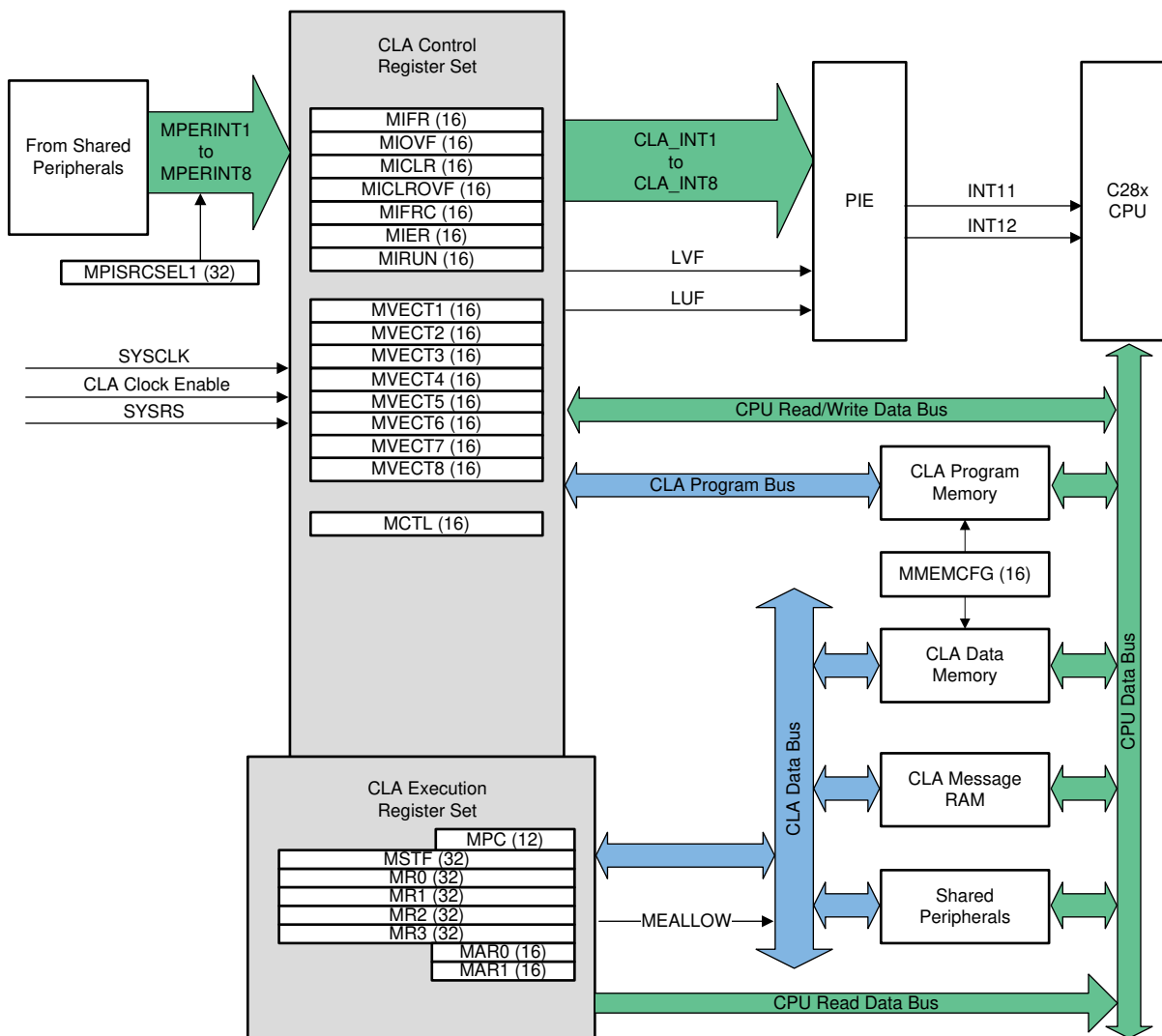
- [CLA Software Development Guide](#)
- [Software Examples to Showcase Unique Capabilities of TI's C2000 CLA Application Report](#)

## Expert Materials

- [Digital Control of Two Phase Interleaved PFC and Motor Drive Using MCU With CLA Application Report](#)
- [Sensorless Field Oriented Control: 3-Phase Perm. Magnet Synch. Motors With CLA Application Report](#)

### 10.1.3 Block Diagram

Figure 10-1 is a block diagram of the CLA.



**Figure 10-1. CLA (Type 0) Block Diagram**

## 10.2 CLA Interface

This section describes how the C28x main CPU can interface to the CLA and conversely.

### 10.2.1 CLA Memory

The CLA can access three types of memory: program, data and message RAMs. The behavior and arbitration for each type of memory is described in this chapter. The CLA RAMs are protected by the CSM module. Refer to the Code Security Module (CSM) section of the *System Control and Interrupts* chapter for more details on the security scheme.

- **CLA Program Memory**

The CLA program can be loaded to a designated memory block. At reset, all memory blocks are mapped to the CPU. While mapped to the CPU space, the CPU can copy the CLA program code into the memory. During debug, the memory can also be loaded directly by the Code Composer Studio™ IDE.

Once the memory is initialized with CLA code, the CPU maps it to the CLA program space by setting the MMEMCFG[PROGE] bit.

When a memory block is configured as CLA program memory, debug accesses are allowed only on cycles where the CLA is not fetching a new instruction. A detailed explanation of the memory configurations and access arbitration (CPU, CLA, and DEBUG) process can be found in the Memory Controller Module section of the *System Control and Interrupts* chapter.

All CLA program fetches are performed as 32-bit read operations and all opcodes must be aligned to an even address. Since all CLA opcodes are 32-bits, this alignment occurs naturally.

- **CLA Data Memory**

Designated memory locations can serve as data memory blocks to the CLA. At reset, all blocks are mapped to the CPU memory space, whereby the CPU can initialize the memory with data tables, coefficients, and so on, for the CLA to use.

Once the memory is initialized with CLA data, the CPU maps it to the CLA data space by setting the respective MMEMCFG[RAMxE] bit.

When a memory block is configured as CLA data memory, CLA read and write accesses are arbitrated along with CPU accesses. The user has the option of turning on CPU fetch or write protection to the memory by writing to the appropriate MMEMCFG[RAMnCPUE] bits. A detailed explanation of the memory configurations and access arbitration (CPU, CLA, and DEBUG) process can be found in the Memory Controller Module section of the *System Control and Interrupts* chapter.

- **CLA Shared Message RAMs**

There are two memory blocks for data sharing and communication between the CLA and the CPU. The message RAMs are always mapped to both CPU and CLA memory spaces, and only data access is allowed; no program fetches can be performed.

- **CLA to CPU Message RAM:** The CLA can use this block to pass data to the CPU. This block is both readable and writable by the CLA. This block is also readable by the CPU but writes by the CPU are ignored.
- **CPU to CLA Message RAM:** The CPU can use this block to pass data and messages to the CLA. This message RAM is both readable and writable by the CPU. The CLA can perform reads but writes by the CLA are ignored.

### 10.2.2 CLA Memory Bus

The CLA has dedicated bus architecture similar to that of the C28x CPU where there are separate program read, data read, and data write buses. Thus, there can be simultaneous instruction fetch, data read, and data write in a single cycle. Like the C28x CPU, the CLA expects memory logic to align any 32-bit read or write to an even address. If the address-generation logic generates an odd address, the CLA will begin reading or writing at the previous even address. This alignment does not affect the address values generated by the address-generation logic.

- **CLA Program Bus**

The CLA program bus has an access range of 2048 32-bit instructions. Since all CLA instructions are 32 bits, this bus always fetches 32 bits at a time and the opcodes must be even-word aligned. The amount of program space available for the CLA is limited to the number of allocated memory blocks. This number is device-dependent and will be described in the device-specific data manual.

- **CLA Data Read Bus**

The CLA data read bus has a 64K x 16 address range. The bus can perform 16 or 32-bit reads and will automatically stall if there are memory access conflicts. The data read bus has access to both the message RAMs, CLA data memory, and the shared peripherals.

- **CLA Data Write Bus**

The CLA data write bus has a 64K x 16 address range. This bus can perform 16 or 32-bit writes. The bus will automatically stall if there are memory access conflicts. The data write bus has access to the CLA to CPU message RAM, CLA data memory, and the shared peripherals.

### 10.2.3 Shared Peripherals and EALLOW Protection

The CPU and CLA share access to some peripherals. describes the arbitration between the CPU and CLA.

Refer to the device data sheet for the list of peripherals connected to the bus.

Several peripheral control registers are protected from spurious 28x CPU writes by the EALLOW protection mechanism. These same registers are also protected from spurious CLA writes. The EALLOW bit in the CPU status register 1 (ST1) indicates the state of protection for the CPU. Likewise the MEALLOW bit in the CLA status register (MSTF) indicates the state of write protection for the CLA. The MEALLOW CLA instruction enables write access by the CLA to EALLOW protected registers. Likewise the MEDIS CLA instruction will disable write access. This way the CLA can enable/disable write access independent of the CPU.

The ADC offers the option to generate an early interrupt pulse at the start of a sample conversion. If this option is used to start an ADC-triggered CLA task, the user may use the intervening cycles, until the completion of the conversion, to perform preliminary calculations or loads and stores before finally reading the ADC value. The CLA pipeline activity for this scenario is shown in [Section 10.5](#).

## 10.2.4 CLA Tasks and Interrupt Vectors

The CLA program code is divided up into tasks or interrupt service routines. Tasks do not have a fixed starting location or length. The CLA program memory can be divided up as desired. The CLA uses the contents of the interrupt vectors (MVECT1 to MVECT8) to determine where a task begins; tasks are terminated by the MSTOP instruction.

The CLA supports eight tasks. Task 1 has the highest priority and task 8 has the lowest priority.

A task can be requested by a peripheral interrupt or by software:

- **Peripheral interrupt trigger**

Each task can be triggered by software-selectable interrupt sources. The trigger for each task is defined by writing an appropriate value to the MPISRCSEL1[PERINTnSEL] bit field. Each option specifies an interrupt source from a specific peripheral on the shared bus. The peripheral interrupt triggers are listed in [Section 10.7.3.3](#).

For example, Task 1 (MVECT1) can be set to trigger on EPWM1\_INT by writing 2 to MPISRCSEL1[PERINT1SEL]. To disable the triggering of a task by a peripheral, the user must configure the MPISRCSEL1[PERINTxSEL] bit field to a "No interrupt source" selection.

- **Software Trigger**

CPU software can trigger tasks by writing to the MIFRC register or by the IACK instruction. Using the IACK instruction is more efficient because it does not require you to issue an EALLOW to set MIFR bits. Set the MCTL[IACKE] bit to enable the IACK feature. Each bit in the operand of the IACK instruction corresponds to a task. For example IACK #0x0001 will set bit 0 in the MIFR register to start task 1. Likewise IACK #0x0003 will set bits 0 and 1 in the MIFR register to start task 1 and task 2.

The CLA has its own fetch mechanism and can run and execute a task independent of the CPU. Only one task is serviced at a time; there is no nesting of tasks. The task currently running is indicated in the MIRUN register. Interrupts that have been received but not yet serviced are indicated in the flag register (MIFR). If an interrupt request from a peripheral is received and that same task is already flagged, then the overflow flag bit is set. Overflow flags will remain set until they are cleared by the CPU.

If the CLA is idle (no task is currently running) then the highest priority interrupt request that is both flagged (MIFR) and enabled (MIER) will start.

The flow is as follows:

1. The associated RUN register bit is set (MIRUN) and the flag bit (MIFR) is cleared.
2. The CLA begins execution at the location indicated by the associated interrupt vector (MVECTx). MVECTx contains the absolute 16-bit address of the task in the lower 64K memory space.
3. The CLA executes instructions until the MSTOP instruction is found. This indicates the end of the task.
4. The MIRUN bit is cleared.
5. The task-specific interrupt to the PIE is issued. This informs the main CPU that the task has completed.
6. The CLA returns to idle.

Once a task completes the next highest-priority pending task is automatically serviced and this sequence repeats.



## 10.3 CLA and CPU Arbitration

Typically, CLA activity is independent of the CPU activity. Under the circumstance where the CLA or CPU attempt to concurrently access memory or a peripheral register within the same interface, an arbitration procedure will occur. This section describes this arbitration.

The arbitration follows a fixed arbitration scheme with highest priority first:

They are covered in detail in the Memory Controller Module section of the *System Control and Interrupts* chapter.

### 10.3.1 CLA Message RAM

Message RAMs consist of blocks:

- CLA to CPU Message RAM
- CPU to CLA Message RAM

These blocks are useful for passing data between the CLA and CPU. No opcode fetches, from either the CLA or CPU, are allowed from the message RAMs. A write protection violation is not generated if the CLA attempts to write to the CPU to CLA message RAM, but the write is ignored. The arbitration scheme for the message RAMs are the same as those for the shared memories, described in the Memory Controller Module section of the *System Control and Interrupts* chapter.

The message RAMs have the following characteristics:

- CLA to CPU Message RAM:  
The following accesses are allowed:
  - CPU reads
  - CLA data reads and writes
  - CPU debug reads and writes  
The following accesses are ignored:
  - CPU writes
- CPU to CLA Message RAM:  
The following accesses are allowed:
  - CPU reads and writes
  - CLA reads
  - CPU debug reads and writes  
The following accesses are ignored:
  - CLA writes

### 10.3.2 CLA Program Memory

The behavior of the program memory depends on the state of the MMEMCFG[PROGE] bit. This bit controls whether the memory is mapped to CLA space or CPU space.

#### • MMEMCFG[PROGE] == 0

In this case the memory is mapped to the CPU. The CLA will be halted and no tasks should be incoming.

- Any CLA fetch will be treated as an illegal opcode condition as described in [Section 10.4.4](#). This condition will not occur if the proper procedure is followed to map the program memory.
- CLA reads and writes cannot occur
- The memory block behaves as any normal RAM block mapped to CPU memory space.

Priority of accesses are (highest priority first):

1. CPU data write, program write, debug write
2. CPU data read, program read, debug read
3. CPU fetch, program read

#### • MMEMCFG[PROGE] == 1

In this case the memory block is mapped to CLA space. The CPU can only make debug accesses.

- CLA reads and writes cannot occur
- CLA fetches are allowed
- CPU fetches return 0 which is an illegal opcode and will cause an ITRAP interrupt.
- CPU data reads and program reads return 0
- CPU data writes and program writes are ignored

Priority of accesses are (highest priority first):

1. CLA fetch
2. CPU debug write
3. CPU debug read

---

#### Note

Because the CLA fetch has higher priority than CPU debug reads, it is possible for the CLA to permanently block debug accesses if the CLA is executing in a loop. This might occur when initially developing CLA code due to a bug. To avoid this issue, the program memory will return all 0x0000 for CPU debug reads (ignore writes) when the CLA is running. When the CLA is halted or idle then normal CPU debug read and write access can be performed.

---

### 10.3.3 CLA Data Memory

There are three independent data memory blocks. The behavior of the data memory depends on the state of the MMEMCFG[RAM0E], MMEMCFG[RAM1E] and MMEMCFG[RAM2E] bits. These bits determine whether the memory blocks are mapped to CLA space or CPU space.

#### • MMEMCFG[RAMxE] == 0, MMEMCFG[RAMxCPUE] = 0/1

In this case the memory block is mapped to the CPU.

- CLA fetches cannot occur to this block.
- CLA reads return 0.
- CLA writes are ignored.
- The memory block behaves as any normal RAM block mapped to the CPU memory space.

Priority of accesses are (highest priority first):

1. CPU data write/program write/debug access write
2. CPU data read/debug access read
3. CPU fetch/program read

- **MMEMCFG[RAMxE] == 1, MMEMCFG[RAMxCPUE] = 0**

In this case the memory block is mapped to CLA space. The CPU can make only debug accesses.

- CLA fetches cannot occur to this block.
- CLA read and CLA writes are allowed.
- CPU fetches return 0
- CPU data reads and program reads return 0.
- CPU data writes and program writes are ignored.

Priority of accesses are (highest priority first):

1. CLA data write
2. CPU debug write
3. CPU debug read
4. CLA read

- **MMEMCFG[RAMxE] == 1, MMEMCFG[RAMxCPUE] = 1**

In this case the memory block is mapped to CLA space. The CPU has read and write access to the memory in addition to debug accesses.

- CLA fetches cannot occur to this block.
- CLA read and CLA writes are allowed.
- CPU fetches return 0
- CPU data reads and writes are allowed.
- CPU program reads return 0 while program writes are ignored.

Priority of accesses are (highest priority first):

1. CLA data write
2. CPU debug access write/CPU data write
3. CPU debug access read/ CPU data read
4. CLA read

### 10.3.4 Peripheral Registers (ePWM, HRPWM, Comparator, eCAP, eQEP)

Accesses to the registers follow these rules:

- If both the CPU and CLA request access at the same time, then the CLA will have priority and the main CPU is stalled.
- If a CPU access is in progress and another CPU access is pending, then the CLA will have priority over the pending CPU access. In this case the CLA access will begin when the current CPU access completes.
- While a CPU access is in progress any incoming CLA access will be stalled.
- While a CLA access is in progress any incoming CPU access will be stalled.
- A CPU write operation has priority over a CPU read operation.
- A CLA write operation has priority over a CLA read operation.
- If the CPU is performing a read-modify-write operation and the CLA performs a write to the same location, the CLA write may be lost if the operation occurs in-between the CPU read and write. For this reason, you should not mix CPU and CLA accesses to same location.

## 10.4 CLA Configuration and Debug

This section discusses the steps necessary to configure and debug the CLA.

### 10.4.1 Building a CLA Application

The control law accelerator can be programmed in either CLA assembly code, using the instructions described in [Section 10.6](#), or a reduced subset of the C language. CLA assembly code resides in the same project with C28x code. The only restriction is the CLA code must be in its own assembly section. This can be easily done using the **.sect** assembly directive. This does not prevent CLA and C28x code from being linked into the same memory region in the linker command file.

System and CLA initialization are performed by the main CPU. This would typically be done in C or C++ but can also include C28x assembly code. The main CPU will also copy the CLA code to the program memory and, if needed, initialize the CLA data RAM(s). Once system initialization is complete and the application begins, the CLA will service its interrupts using the CLA assembly code (or tasks). The main CPU can perform other tasks concurrently with CLA program execution.

The CLA Type 0 requires Codegen V5.2.0 or later with the compiler switch: `--cla_support=cla0`.

### 10.4.2 Typical CLA Initialization Sequence

A typical CLA initialization sequence is performed by the main CPU as described in this section.

#### 1. Copy CLA code into the CLA program RAM

The source for the CLA code can initially reside in the Flash or a data stream from a communications peripheral or anywhere the main CPU can access it. The debugger can also be used to load code directly to the CLA program RAM during development.

#### 2. Initialize CLA data RAM, if necessary

Populate the CLA data RAM with any required data coefficients or constants.

#### 3. Configure the CLA registers

Configure the CLA registers, but keep interrupts disabled until later (leave MIER = 0):

- **Enable the CLA peripheral clock using the assigned PCLKCRn register**

The peripheral clock control (PCLKCRn) registers are defined in the *System Control and Interrupts* chapter.

- **Populate the CLA task interrupt vectors**

- MVECT1 to MVECT8

Each vector needs to be initialized with the start address of the task to be executed when the CLA receives the associated interrupt. The address is an offset from the base address of the assigned CLA Program memory block.

- **Select the task interrupt sources**

For each task select the interrupt source in the CLA1TASKSRCSELx register. If a task is software triggered, select no interrupt.

- **Enable IACK to start a task from software, if desired**

To enable the IACK instruction to start a task set the MCTL[IACKE] bit. Using the IACK instruction avoids having to set and clear the EALLOW bit.

- **Map CLA data RAM to CLA space, if necessary**

Map the data RAM to the CLA space by writing a 1 to the MMEMCFG[RAMxE] bit. The CPU will have restricted access to the memory block once the MMEMCFG[RAMxE] bit is set. CPU access to CLA data RAM can be granted through the MMEMCFG[RAMxCPUE] bit. Allow two SYSCLK cycles for MMEMCFG updates to take effect.

- **Map CLA program RAM to CLA space**

Map the CLA program RAM to CLA space by setting the MMEMCFG[PROGE] bit. The CPU will only have debug access to program RAM once the MMEMCFG[PROGE] bit is set. Allow two SYSCLK cycles for MMEMCFG updates to take effect.

#### 4. Initialize the PIE vector table and registers

When a CLA task completes, the associated interrupt in the PIE will be flagged. The CLA overflow and underflow flags also have associated interrupts within the PIE.

#### 5. Enable CLA tasks/interrupts

Set appropriate bits in the interrupt enable register (MIER) to allow the CLA to service interrupts. It should be noted that a CLA task only triggers on a level transition (a falling edge) of the configured interrupt source. If a peripheral is enabled and an interrupt fires before the CLA is configured, then the CLA will not see the interrupt edge and will not respond. To avoid this, configure the CLA before the peripherals or clear any pending peripheral interrupts before setting bits in the MIER register.

#### 6. Initialize other peripherals

Initialize any peripherals (such as ePWM, ADC, and others) that will generate interrupt triggers for enabled CLA tasks.

The CLA is now ready to service interrupts and the message RAMs can be used to pass data between the CPU and the CLA. Mapping of the CLA program and data RAMs typically occurs only during the initialization process. If the RAM mapping needs to be changed after initialization, the CLA interrupts must be disabled and all tasks must be completed (by checking the MIRUN register) prior to modifying the RAM ownership.

### 10.4.3 Debugging CLA Code

Debugging the CLA code is a simple process that occurs independently of the main CPU. .

#### 10.4.3.1 Breakpoint Support (MDEBUGSTOP)

##### 1. Insert a breakpoint in CLA code

Insert a CLA breakpoint (MDEBUGSTOP instruction) into the code where you want the CLA to halt, then rebuild and reload the code. Because the CLA does not flush its pipeline when you single-step, the MDEBUGSTOP instruction must be inserted as part of the code. The debugger cannot insert it as needed.

If CLA breakpoints are not enabled, then the MDEBUGSTOP will be ignored and is treated as a MNOP. The MDEBUGSTOP instruction can be placed anywhere in the CLA code as long as it is not within three instructions of a MBCNDD, MCCNDD, or MRCNDD instruction. When programming in C, the user can use the `__mdebugstop()` intrinsic instead; the compiler will ensure that the placement of the MDEBUSTOP instruction in the generated assembly does not violate any of the pipeline restrictions.

##### 2. Enable CLA breakpoints

Enable the CLA breakpoints in the debugger. In the Code Composer Studio™ IDE, this is done by connecting to the CLA core (or tap) from the debug perspective. Breakpoints are disabled when the core is disconnected.

##### 3. Start the task

There are three ways to start the task:

- The peripheral can assert an interrupt,
- The main CPU can execute an IACK instruction, or
- The user can manually write to the MIFRC register in the debugger window

When the task starts, the CLA will execute instructions until the MDEBUGSTOP is in the D2 phase of the pipeline. At this point, the CLA will halt and the pipeline will be frozen. The MPC register will reflect the address of the MDEBUGSTOP instruction.

#### 4. Single-step the CLA code

Once halted, the user can single-step the CLA code. The behavior of a CLA single-step is different than the main C28x. When issuing a CLA single-step, the pipeline is clocked only one cycle and then again frozen. On the C28x CPU, the pipeline is flushed for each single-step.

You can also run to the next MDEBUSTOP or to the end of the task. If another task is pending, it will automatically start when you run to the end of the task.

---

#### Note

A CLA fetch has higher priority than CPU debug reads. For this reason, it is possible for the CLA to permanently block CPU debug accesses if the CLA is executing in a loop. This might occur when initially developing CLA code due to a bug that causes an infinite loop. To avoid locking up the main CPU, the program memory will return all 0x0000 for CPU debug reads when the CLA is running. When the CLA is halted or idle then normal CPU debug read and write access to CLA program memory can be performed.

If the CLA gets caught in an infinite loop, you can use a soft or hard reset to exit the condition. A debugger reset will also exit the condition.

---

There are special cases that can occur when single-stepping a task such that the program counter, MPC, reaches the MSTOP instruction at the end of the task.

- **MPC halts at or after the MSTOP with a task already pending**

If you are single-stepping or halted in "task A" and "task B" comes in before the MPC reaches the MSTOP, then "task B" will start if you continue to step through the MSTOP instruction. Basically if "task B" is pending before the MPC reaches MSTOP in "task A" then there is no issue in "task B" starting and no special action is required.

- **MPC halts at or after the MSTOP with no task pending**

In this case you have single-stepped or halted in "task A" and the MPC has reached the MSTOP with no tasks pending. If "task B" comes in at this point, it will be flagged in the MIFR register but it may or may not start if you continue to single-step through the MSTOP instruction of "task A."

It depends on exactly when the new task comes in. To reliably start "task B" perform a soft reset and reconfigure the MIER bits. Once this is done, you can start single-stepping "task B."

This case can be handled slightly differently if there is control over when "task B" comes in (for example, using the IACK instruction to start the task). In this case you have single-stepped or halted in "task A" and the MPC has reached the MSTOP with no tasks pending. Before forcing "task B," run free to force the CLA out of the debug state. Once this is done you can force "task B" and continue debugging.

#### 5. Disable CLA breakpoints, if desired

In the Code Composer Studio™ IDE, you can disable the CLA breakpoints by disconnecting the CLA core in the debug perspective. Make sure to first issue a run or reset; otherwise, the CLA will be halted and no other tasks will start.

#### 10.4.4 CLA Illegal Opcode Behavior

If the CLA fetches an opcode that does not correspond to a legal instruction, it will behave as follows:

- The CLA will halt with the illegal opcode in the D2 phase of the pipeline as if it were a breakpoint. This will occur whether CLA breakpoints are enabled or not.
- The CLA will issue the task-specific interrupt to the PIE.
- The MIRUN bit for the task will remain set.

Further single-stepping is ignored once execution halts due to an illegal op-code. To exit this situation, issue either a soft or hard reset of the CLA as described in [Section 10.4.5](#).

### 10.4.5 Resetting the CLA

There may be times when you need to reset the CLA. For example, during code debug the CLA may enter an infinite loop due to a code bug. The CLA has two types of resets: hard and soft. Both of these resets can be performed by the debugger or by the main CPU.

- **Hard Reset**

Writing a 1 to the MCTL[HARDRESET] bit will perform a hard reset of the CLA. The behavior of a hard reset is the same as a system reset (via  $\overline{\text{XRS}}$  or the debugger). In this case all CLA configuration and execution registers will be set to their default state and CLA execution will halt.

- **Soft Reset**

Writing a 1 to the MCTL[SOFTRESET] bit performs a soft reset of the CLA. If a task is executing it will halt and the associated MIRUN bit will be cleared. All bits within the interrupt enable (MIER) register will also be cleared so that no new tasks start.

## 10.5 Pipeline

This section describes the CLA pipeline stages and presents cases where pipeline alignment must be considered.

### 10.5.1 Pipeline Overview

The CLA pipeline is very similar to the C28x pipeline with eight stages:

1. **Fetch 1 (F1):** During the F1 stage the program read address is placed on the CLA program address bus.
2. **Fetch 2 (F2):** During the F2 stage the instruction is read using the CLA program data bus.
3. **Decode 1 (D1):** During D1 the instruction is decoded.
4. **Decode 2 (D2):** Generate the data read address. Changes to MAR0 and MAR1 due to post-increment using indirect addressing takes place in the D2 phase. Conditional branch decisions are also made at this stage based on the MSTF register flags.
5. **Read 1 (R1):** Place the data read address on the CLA data-read address bus. If a memory conflict exists, the R1 stage will be stalled.
6. **Read 2 (R2):** Read the data value using the CLA data read data bus.
7. **Execute (EXE):** Execute the operation. Changes to MAR0 and MAR1 due to loading an immediate value or value from memory take place in this stage.
8. **Write (W):** Place the write address and write data on the CLA write data bus. If a memory conflict exists, the W stage will be stalled.



## 10.5.2 CLA Pipeline Alignment

The majority of the CLA instructions do not require any special pipeline considerations. This section lists the few operations that do require special consideration.

- Write Followed by Read**

In both the C28x pipeline and the CLA pipeline, the read operation occurs before the write. This means that if a read operation immediately follows a write, then the read will complete first as shown in [Table 10-1](#). In most cases this does not cause a problem since the contents of one memory location does not depend on the state of another. For accesses to peripherals where a write to one location can affect the value in another location, the code must wait for the write to complete before issuing the read as shown in [Table 10-2](#).

This behavior is different for the C28x CPU. For the C28x CPU, any write followed by read to the same location is protected by what is called write-followed-by-read protection. This protection automatically stalls the pipeline so that the write will complete before the read. In addition, some peripheral frames are protected such that a C28x CPU write to one location within the frame will always complete before a read to the frame. The CLA does not have this protection mechanism. Instead the code must wait to perform the read.

**Table 10-1. Write Followed by Read - Read Occurs First**

| Instruction          | F1 | F2 | D1 | D2 | R1 | R2 | E  | W  |
|----------------------|----|----|----|----|----|----|----|----|
| I1 MMOV16 @Reg1, MR3 | I1 |    |    |    |    |    |    |    |
| I2 MMOV16 MR2, @Reg2 | I2 | I1 |    |    |    |    |    |    |
|                      |    | I2 | I1 |    |    |    |    |    |
|                      |    |    | I2 | I1 |    |    |    |    |
|                      |    |    |    | I2 | I1 |    |    |    |
|                      |    |    |    |    | I2 | I1 |    |    |
|                      |    |    |    |    |    | I2 | I1 |    |
|                      |    |    |    |    |    |    | I2 | I1 |

**Table 10-2. Write Followed by Read - Write Occurs First**

| Instruction          | F1 | F2 | D1 | D2 | R1 | R2 | E  | W  |
|----------------------|----|----|----|----|----|----|----|----|
| I1 MMOV16 @Reg1, MR3 | I1 |    |    |    |    |    |    |    |
| I2                   | I2 | I1 |    |    |    |    |    |    |
| I3                   | I3 | I2 | I1 |    |    |    |    |    |
| I4                   | I4 | I3 | I2 | I1 |    |    |    |    |
| I5 MMOV16 MR2, @Reg2 | I5 | I4 | I3 | I2 | I1 |    |    |    |
|                      |    | I5 | I4 | I3 | I2 | I1 |    |    |
|                      |    |    | I5 | I4 | I3 | I2 | I1 |    |
|                      |    |    |    | I5 | I4 | I3 | I2 | I1 |
|                      |    |    |    |    | I5 | I4 | I3 |    |
|                      |    |    |    |    |    | I5 | I4 |    |
|                      |    |    |    |    |    |    | I5 |    |

## • Delayed Conditional instructions: MBCNDD, MCCNDD, and MRCNDD

Referring to [Example 10-1](#), the following applies to delayed conditional instructions:

### – I1

I1 is the last instruction that can effect the CNDF flags for the branch, call or return instruction. The CNDF flags are tested in the D2 phase of the pipeline. That is, a decision is made whether to branch or not when MBCNDD, MCCNDD, or MRCNDD is in the D2 phase.

### – I2, I3, and I4

The three instructions preceding MBCNDD can change MSTF flags but will have no effect on whether the MBCNDD instruction branches or not. This is because the flag modification will occur after the D2 phase of the branch, call or return instruction. These three instructions must not be a MSTOP, MDEBUGSTOP, MBCNDD, MCCNDD, or MRCNDD.

### – I5, I6, and I7

The three instructions following a branch, call or return are always executed irrespective of whether the condition is true or not. These instructions must not be MSTOP, MDEBUGSTOP, MBCNDD, MCCNDD or MRCNDD.

For a more detailed description refer to the functional description for [MBCNDD](#), [MCCNDD](#), and [MRCNDD](#).

### **Example 10-1. Code Fragment For MBCNDD, MCCNDD, or MRCNDD**

```
<Instruction 1>      ; I1 Last instruction that can affect flags for
                    ;   the branch, call or return operation
<Instruction 2>      ; I2 Cannot be stop, branch, call or return
<Instruction 3>      ; I3 Cannot be stop, branch, call or return
<Instruction 4>      ; I4 Cannot be stop, branch, call or return
<branch/call/ret>    ; MBCNDD, MCCNDD or MRCNDD
                    ; I5-I7: Three instructions after are always
                    ;   executed whether the branch/call or return is
                    ;   taken or not
<Instruction 5>      ; I5 Cannot be stop, branch, call or return
<Instruction 6>      ; I6 Cannot be stop, branch, call or return
<Instruction 7>      ; I7 Cannot be stop, branch, call or return
<Instruction 8>      ; I8
<Instruction 9>      ; I9
....
```

## • Stop or Halting a Task: MSTOP and MDEBUGSTOP

The MSTOP and MDEBUGSTOP instructions cannot be placed three instructions before or after a conditional branch, call or return instruction (MBCNDD, MCCNDD, or MRCNDD). Refer to [Example 10-1](#). To single-step through a branch/call or return, insert the MDEBUGSTOP at least four instructions back and step from there.

- **Loading MAR0 or MAR1**

A load of auxiliary register MAR0 or MAR1 will occur in the EXE phase of the pipeline. Any post increment of MAR0 or MAR1 using indirect addressing will occur in the D2 phase of the pipeline. Referring to [Example 10-2](#), the following applies when loading the auxiliary registers:

- **I1 and I2**

The two instructions following the load instruction will use the value in MAR0 or MAR1 before the update occurs.

- **I3**

Loading of an auxiliary register occurs in the EXE phase while updates due to post-increment addressing occur in the D2 phase. Thus I3 cannot use the auxiliary register or there will be a conflict. In the case of a conflict, the update due to address-mode post increment will win and the auxiliary register will not be updated with #\_X.

- **I4**

Starting with the 4th instruction MAR0 or MAR1 will have the new value.

**Example 10-2. Code Fragment for Loading MAR0 or MAR1**

```
; Assume MAR0 is 50 and #_X is 20

MMOVI16 MAR0, #_X ; Load MAR0 with address of X (20)
<Instruction 1>    ; I1 Will use the old value of MAR0 (50)
<Instruction 2>    ; I2 Will use the old value of MAR0 (50)
<Instruction 3>    ; I3 Cannot use MAR0
<Instruction 4>    ; I4 Will use the new value of MAR0 (20)
<Instruction 5>    ; I5 Will use the new value of MAR0 (20)
....
```

### 10.5.2.1 ADC Early Interrupt to CLA Response

The ADC can be configured to generate an early interrupt pulse before the ADC conversion completes. If this option is used to start a CLA task, the CLA will be able to read the result as soon as the conversion result is available in the ADC result register. This combination of just-in-time sampling along with the low interrupt response of the CLA enable faster system response and higher frequency control loops. The CLA task trigger to first instruction fetch interrupt latency is 4 cycles.

Timings for ADC conversions are shown in the timing diagrams of the ADC chapter. If the ADCCLK is a divided down version of the SYSCLK, the user will have to account for the conversion time in SYSCLK cycles.

For example, if using the ADC with ADCCLK at  $\text{SYSCLK} / 2$ , it would take  $13 \text{ ADCCLK} \times 2 \text{ SYSCLK} = 26$  SYSCLK cycles to complete a conversion.

From a CLA perspective, the pipeline activity is shown in [Table 10-3](#) for an N-cycle (SYSCLK) ADC conversion. The N-2 instruction will arrive in the R2 phase just in time to read the result register. While the prior instructions will enter the R2 phase of the pipeline too soon to read the conversion, they can be efficiently used for pre-processing calculations needed by the task.

**Table 10-3. ADC to CLA Early Interrupt Response**

| ADC Activity                      | CLA Activity             | F1                       | F2                       | D1                       | D2                       | R1                        | R2                        | E | W |
|-----------------------------------|--------------------------|--------------------------|--------------------------|--------------------------|--------------------------|---------------------------|---------------------------|---|---|
| Sample                            |                          |                          |                          |                          |                          |                           |                           |   |   |
| Sample                            |                          |                          |                          |                          |                          |                           |                           |   |   |
| ...                               |                          |                          |                          |                          |                          |                           |                           |   |   |
| Sample                            |                          |                          |                          |                          |                          |                           |                           |   |   |
| Conversion <sub>(Cycle 1)</sub>   | Interrupt Received       |                          |                          |                          |                          |                           |                           |   |   |
| Conversion <sub>(Cycle 2)</sub>   | Task Startup             |                          |                          |                          |                          |                           |                           |   |   |
| Conversion <sub>(Cycle 3)</sub>   | Task Startup             |                          |                          |                          |                          |                           |                           |   |   |
| Conversion <sub>(Cycle 4)</sub>   | I <sub>(Cycle 4)</sub>   | I <sub>(Cycle 4)</sub>   |                          |                          |                          |                           |                           |   |   |
| Conversion <sub>(Cycle 5)</sub>   | I <sub>(Cycle 5)</sub>   | I <sub>(Cycle 5)</sub>   | I <sub>(Cycle 4)</sub>   |                          |                          |                           |                           |   |   |
| Conversion <sub>(...)</sub>       | ...                      | ...                      | ...                      | ...                      | ...                      | ...                       | ...                       |   |   |
| Conversion <sub>(Cycle N-6)</sub> | I <sub>(Cycle N-6)</sub> | I <sub>(Cycle N-6)</sub> | I <sub>(Cycle N-7)</sub> | I <sub>(Cycle N-8)</sub> | I <sub>(Cycle N-9)</sub> | I <sub>(Cycle N-10)</sub> | I <sub>(Cycle N-11)</sub> |   |   |
| Conversion <sub>(Cycle N-5)</sub> | I <sub>(Cycle N-5)</sub> | I <sub>(Cycle N-5)</sub> | I <sub>(Cycle N-6)</sub> | I <sub>(Cycle N-7)</sub> | I <sub>(Cycle N-8)</sub> | I <sub>(Cycle N-9)</sub>  | I <sub>(Cycle N-10)</sub> |   |   |
| Conversion <sub>(Cycle N-4)</sub> | I <sub>(Cycle N-4)</sub> | I <sub>(Cycle N-4)</sub> | I <sub>(Cycle N-5)</sub> | I <sub>(Cycle N-6)</sub> | I <sub>(Cycle N-7)</sub> | I <sub>(Cycle N-8)</sub>  | I <sub>(Cycle N-9)</sub>  |   |   |
| Conversion <sub>(Cycle N-3)</sub> | I <sub>(Cycle N-3)</sub> | I <sub>(Cycle N-3)</sub> | I <sub>(Cycle N-4)</sub> | I <sub>(Cycle N-5)</sub> | I <sub>(Cycle N-6)</sub> | I <sub>(Cycle N-7)</sub>  | I <sub>(Cycle N-8)</sub>  |   |   |
| Conversion <sub>(Cycle N-2)</sub> | <b>Read RESULT</b>       | <b>Read RESULT</b>       | I <sub>(Cycle N-3)</sub> | I <sub>(Cycle N-4)</sub> | I <sub>(Cycle N-5)</sub> | I <sub>(Cycle N-6)</sub>  | I <sub>(Cycle N-7)</sub>  |   |   |
| Conversion <sub>(Cycle N-1)</sub> |                          |                          | <b>Read RESULT</b>       | I <sub>(Cycle N-3)</sub> | I <sub>(Cycle N-4)</sub> | I <sub>(Cycle N-5)</sub>  | I <sub>(Cycle N-6)</sub>  |   |   |
| Conversion <sub>(Cycle N-0)</sub> |                          |                          |                          | <b>Read RESULT</b>       | I <sub>(Cycle N-3)</sub> | I <sub>(Cycle N-4)</sub>  | I <sub>(Cycle N-5)</sub>  |   |   |
| Conversion Complete               |                          |                          |                          |                          | <b>Read RESULT</b>       | I <sub>(Cycle N-3)</sub>  | I <sub>(Cycle N-4)</sub>  |   |   |
| RESULT Latched                    |                          |                          |                          |                          |                          | <b>Read RESULT</b>        | I <sub>(Cycle N-3)</sub>  |   |   |
| <b>RESULT Available</b>           |                          |                          |                          |                          |                          |                           | <b>Read RESULT</b>        |   |   |

### 10.5.3 Parallel Instructions

Parallel instructions are single opcodes that perform two operations in parallel. The following types of parallel instructions are available: math operation in parallel with a move operation, or two math operations in parallel. Both operations complete in a single cycle and there are no special pipeline alignment requirements.

#### Example 10-3. Math Operation with Parallel Load

```
; MADDF32 || MMOV32 instruction: 32-bit floating-point add with parallel move
; MADDF32 is a 1 cycle operation
; MMOV32 is a 1 cycle operation
    MADDF32    MR0, MR1, #2      ; MR0 = MR1 + 2,
    || MMOV32  MR1, @Val         ; MR1 gets the contents of Val
                                   ; <-- MMOV32 completes here (MR1 is valid)
                                   ; <-- DDF32 completes here (MR0 is valid)
    MMPYF32 MR0, MR0, MR1       ; Any instruction, can use MR1 and/or MR0
```

#### Example 10-4. Multiply with Parallel Add

```
; MMPYF32 || MADDF32 instruction: 32-bit floating-point multiply with parallel add
; MMPYF32 is a 1 cycle operation
; MADDF32 is a 1 cycle operation
    MMPYF32 MR0, MR1, MR3       ; MR0 = MR1 * MR3
    || MADDF32 MR1, MR2, MR0     ; MR1 = MR2 + MR0 (Uses value of MR0 before MMPYF32)
                                   ; <-- MMPYF32 and MADDF32 complete here (MR0 and MR1 are valid)
    MMPYF32 MR1, MR1, MR0       ; Any instruction, can use MR1 and/or MR0
```

## 10.6 Instruction Set

This section describes the assembly language instructions of the control law accelerator. Also described are parallel operations, conditional operations, resource constraints, and addressing modes. The instructions listed here are independent from C28x and C28x+FPU instruction sets.

### 10.6.1 Instruction Descriptions

This section gives detailed information on the instruction set. Each instruction may present the following information:

- Operands
- Opcode
- Description
- Exceptions
- Pipeline
- Examples
- See also

The example INSTRUCTION is shown to familiarize you with the way each instruction is described. The example describes the kind of information you will find in each part of the individual instruction description and where to obtain more information. CLA instructions follow the same format as the C28x; the source operand(s) are always on the right and the destination operand(s) are on the left.

The explanations for the syntax of the operands used in the instruction descriptions for the C28x CLA are given in [Table 10-4](#).

**Table 10-4. Operand Nomenclature**

| Symbol    | Description                                                                                                                                                           |
|-----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| #16FHi    | 16-bit immediate (hex or float) value that represents the upper 16-bits of an IEEE 32-bit floating-point value. Lower 16-bits of the mantissa are assumed to be zero. |
| #16FHiHex | 16-bit immediate hex value that represents the upper 16-bits of an IEEE 32-bit floating-point value. Lower 16-bits of the mantissa are assumed to be zero.            |
| #16FLoHex | A 16-bit immediate hex value that represents the lower 16-bits of an IEEE 32-bit floating-point value                                                                 |
| #32Fhex   | 32-bit immediate value that represents an IEEE 32-bit floating-point value                                                                                            |
| #32F      | Immediate float value represented in floating-point representation                                                                                                    |
| #0.0      | Immediate zero                                                                                                                                                        |
| #SHIFT    | Immediate value of 1 to 32 used for arithmetic and logical shifts.                                                                                                    |
| addr      | Opcode field indicating the addressing mode                                                                                                                           |
| CNDF      | Condition to test the flags in the MSTF register                                                                                                                      |
| FLAG      | Selected flags from MSTF register (OR) 8 bit mask indicating which floating-point status flags to change                                                              |
| MAR0      | auxiliary register 0                                                                                                                                                  |
| MAR1      | auxiliary register 1                                                                                                                                                  |
| MARx      | Either MAR0 or MAR1                                                                                                                                                   |
| mem16     | 16-bit memory location accessed using direct, indirect, or offset addressing modes                                                                                    |
| mem32     | 32-bit memory location accessed using direct, indirect, or offset addressing modes                                                                                    |
| MRa       | MR0 to MR3 registers                                                                                                                                                  |
| MRb       | MR0 to MR3 registers                                                                                                                                                  |
| MRc       | MR0 to MR3 registers                                                                                                                                                  |
| MRd       | MR0 to MR3 registers                                                                                                                                                  |
| MRe       | MR0 to MR3 registers                                                                                                                                                  |
| MRf       | MR0 to MR3 registers                                                                                                                                                  |
| MSTF      | CLA Floating-point Status Register                                                                                                                                    |
| shift     | Opcode field indicating the number of bits to shift.                                                                                                                  |

**Table 10-4. Operand Nomenclature (continued)**

| Symbol | Description                                                                              |
|--------|------------------------------------------------------------------------------------------|
| VALUE  | Flag value of 0 or 1 for selected flag (OR) 8 bit mask indicating the flag value; 0 or 1 |

Each instruction has a table that gives a list of the operands and a short description. Instructions always have their destination operand(s) first followed by the source operand(s).

**Table 10-5. INSTRUCTION dest, source1, source2 Short Description**

|              | Description                                                                                                                                                                                                                                                                          |
|--------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| dest1        | Description for the 1st operand for the instruction                                                                                                                                                                                                                                  |
| source1      | Description for the 2nd operand for the instruction                                                                                                                                                                                                                                  |
| source2      | Description for the 3rd operand for the instruction                                                                                                                                                                                                                                  |
| Opcode       | This section shows the opcode for the instruction                                                                                                                                                                                                                                    |
| Description  | Detailed description of the instruction execution is described. Any constraints on the operands imposed by the processor or the assembler are discussed.                                                                                                                             |
| Restrictions | Any constraints on the operands or use of the instruction imposed by the processor are discussed.                                                                                                                                                                                    |
| Pipeline     | This section describes the instruction in terms of pipeline cycles as described in <a href="#">Section 10.5</a>                                                                                                                                                                      |
| Example      | Examples of instruction execution. If applicable, register and memory values are given before and after instruction execution. Some examples are code fragments while other examples are full tasks that assume the CLA is correctly configured and the main CPU has passed it data. |
| Operands     | Each instruction has a table that gives a list of the operands and a short description. Instructions always have their destination operand(s) first followed by the source operand(s).                                                                                               |

## 10.6.2 Addressing Modes and Encoding

The CLA uses the same address to access data and registers as the main CPU. For example if the main CPU accesses an ePWM register at address 0x00 6800, then the CLA will access it using address 0x6800. Since all CLA accessible memory and registers are within the low 64k x 16 of memory, only the low 16-bits of the address are used by the CLA.

To address the CLA data memory, message RAMs and shared peripherals, the CLA supports two addressing modes:

- Direct addressing mode: Uses the address of the variable or register directly.
- Indirect addressing with 16-bit post increment. This mode uses either XAR0 or XAR1.

The CLA does not use a data page pointer or a stack pointer. The two addressing modes are encoded as shown [Table 10-6](#).

**Table 10-6. Addressing Modes**

| Addressing Mode | 'addr' Opcode Field Encode <sup>(1)</sup> | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|-----------------|-------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| @dir            | 0000                                      | <b>Direct Addressing Mode</b><br>Example 1: MMOV32 MR1, @_VarA<br>Example 2: MMOV32 MR1, @_EPwm1Regs.CMPA.all<br>In this case the 'mmmm mmmm mmmm mmmm' opcode field will be populated with the 16-bit address of the variable. This is the low 16-bits of the address that you would use to access the variable using the main CPU.<br>For example @_VarA will populate the address of the variable VarA. and @_EPwm1Regs.CMPA.all will populate the address of the CMPA register.                                                                                                                                                                                                                                                                                                                                                 |
| *MAR0[#imm16]++ | 0001                                      | <b>MAR0 Indirect Addressing with 16-bit Immediate Post Increment</b><br><b>MAR1 Indirect Addressing with 16-bit Immediate Post Increment</b><br>addr = MAR0 (or MAR1) Access memory using the address stored in MAR0 (or MAR1).<br>MAR0 (or MAR1) += #imm16 Then post increment MAR0 (or MAR1) by #imm16.<br>#imm16<br>Example 1: MMOV32 MR0, *MAR0[2]++<br>Example 2: MMOV32 MR1, *MAR1[-2]++<br>For a post increment of 0 the assembler will accept both *MAR0 and *MAR0[0]++.<br>The 'mmmm mmmm mmmm mmmm' opcode field will be populated with the signed 16-bit pointer offset. For example if #imm16 is 2, then the opcode field will be 0x0002. Likewise if #imm16 is -2, then the opcode field will be 0xFFFE.<br>If addition of the 16-bit immediate causes overflow, then the value will wrap around on a 16-bit boundary. |
| *MAR1[#imm16]++ | 0010                                      |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |

(1) Values not shown are reserved.

Encoding for the shift fields in the MASR32, MLSR32 and MLSL32 instructions is shown in [Table 10-7](#).

**Table 10-7. Shift Field Encoding**

| Shift Value | 'shift' Opcode Field Encode |
|-------------|-----------------------------|
| 1           | 0000                        |
| 2           | 0001                        |
| 3           | 0010                        |
| ....        | ....                        |
| 32          | 1111                        |

For instructions that use MR<sub>x</sub> (where x could be 'a' through 'f') as operands, the trailing alphabet appears in the opcode as a two-bit field. For example:

```
MMPYF32 MRa, MRb, MRc ||
MADDF32 MRd, MRe, MRf
```

whose opcode is,

```
LSW: 0000 ffee ddcc bbaa
MSW: 0111 1010 0000 0000
```

The two-bit field specifies one of four working registers according to [Table 10-8](#).



**Table 10-8. Operand Encoding**

| Two-Bit Field | Working Register |
|---------------|------------------|
| 00            | MR0              |
| 01            | MR1              |
| 10            | MR2              |
| 11            | MR3              |

Table 10-9 shows the condition field encoding for conditional instructions such as MNEGF, MSWAPF, MBCNDD, MCCNDD, and MRCNDD.

**Table 10-9. Condition Field Encoding**

| Encode <sup>(1)</sup> | CNDF                | Description                          | MSTF Flags Tested   |
|-----------------------|---------------------|--------------------------------------|---------------------|
| 0000                  | NEQ                 | Not equal to zero                    | ZF == 0             |
| 0001                  | EQ                  | Equal to zero                        | ZF == 1             |
| 0010                  | GT                  | Greater than zero                    | ZF == 0 AND NF == 0 |
| 0011                  | GEQ                 | Greater than or equal to zero        | NF == 0             |
| 0100                  | LT                  | Less than zero                       | NF == 1             |
| 0101                  | LEQ                 | Less than or equal to zero           | ZF == 1 OR NF == 1  |
| 1010                  | TF                  | Test flag set                        | TF == 1             |
| 1011                  | NTF                 | Test flag not set                    | TF == 0             |
| 1100                  | LU                  | Latched underflow                    | LUF == 1            |
| 1101                  | LV                  | Latched overflow                     | LVF == 1            |
| 1110                  | UNC                 | Unconditional                        | None                |
| 1111                  | UNCF <sup>(2)</sup> | Unconditional with flag modification | None                |

(1) Values not shown are reserved.

(2) This is the default operation if no CNDF field is specified. This condition will allow the ZF and NF flags to be modified when a conditional operation is executed. All other conditions will not modify these flags.

### 10.6.3 Instructions

The instructions are listed alphabetically.

#### Instruction Set Summary

|                                                                                                 |     |
|-------------------------------------------------------------------------------------------------|-----|
| MABSF32 MRa, MRb 32-Bit Floating-Point Absolute Value.....                                      | 594 |
| MADD32 MRa, MRb, MRc 32-Bit Integer Add.....                                                    | 595 |
| MADDF32 MRa, #16FHi, MRb 32-Bit Floating-Point Addition.....                                    | 596 |
| MADDF32 MRa, MRb, #16FHi 32-Bit Floating-Point Addition.....                                    | 597 |
| MADDF32 MRa, MRb, MRc 32-Bit Floating-Point Addition.....                                       | 599 |
| MADDF32 MRd, MRe, MRf  MMOV32 mem32, MRa 32-Bit Floating-Point Addition with Parallel Move..... | 600 |
| MADDF32 MRd, MRe, MRf  MMOV32 MRa, mem32 32-Bit Floating-Point Addition with Parallel Move..... | 601 |
| MAND32 MRa, MRb, MRc Bitwise AND.....                                                           | 603 |
| MASR32 MRa, #SHIFT Arithmetic Shift Right.....                                                  | 604 |
| MBCNDD 16BitDest {, CNDF} Branch Conditional Delayed.....                                       | 605 |
| MCCNDD 16BitDest {, CNDF} Call Conditional Delayed.....                                         | 610 |
| MCMP32 MRa, MRb 32-Bit Integer Compare for Equal, Less Than or Greater Than.....                | 614 |
| MCMPF32 MRa, MRb 32-Bit Floating-Point Compare for Equal, Less Than or Greater Than.....        | 616 |
| MCMPF32 MRa, #16FHi 32-Bit Floating-Point Compare for Equal, Less Than or Greater Than.....     | 617 |
| MDEBUGSTOP Debug Stop Task.....                                                                 | 619 |
| MEALLOW Enable CLA Write Access to EALLOW Protected Registers.....                              | 620 |
| MEDIS Disable CLA Write Access to EALLOW Protected Registers.....                               | 621 |

|                                                                                                                           |     |
|---------------------------------------------------------------------------------------------------------------------------|-----|
| MEINVF32 MRa, MRb 32-Bit Floating-Point Reciprocal Approximation.....                                                     | 622 |
| MEISQRTF32 MRa, MRb 32-Bit Floating-Point Square-Root Reciprocal Approximation.....                                       | 623 |
| MF32TOI16 MRa, MRb Convert 32-Bit Floating-Point Value to 16-Bit Integer.....                                             | 625 |
| MF32TOI16R MRa, MRb Convert 32-Bit Floating-Point Value to 16-Bit Integer and Round.....                                  | 626 |
| MF32TOI32 MRa, MRb Convert 32-Bit Floating-Point Value to 32-Bit Integer.....                                             | 627 |
| MF32TOUI16 MRa, MRb Convert 32-Bit Floating-Point Value to 16-bit Unsigned Integer .....                                  | 628 |
| MF32TOUI16R MRa, MRb Convert 32-Bit Floating-Point Value to 16-bit Unsigned Integer and Round.....                        | 629 |
| MF32TOUI32 MRa, MRb Convert 32-Bit Floating-Point Value to 32-Bit Unsigned Integer .....                                  | 630 |
| MFRACF32 MRa, MRb Fractional Portion of a 32-Bit Floating-Point Value.....                                                | 631 |
| MI16TOF32 MRa, MRb Convert 16-Bit Integer to 32-Bit Floating-Point Value .....                                            | 632 |
| MI16TOF32 MRa, mem16 Convert 16-Bit Integer to 32-Bit Floating-Point Value .....                                          | 633 |
| MI32TOF32 MRa, mem32 Convert 32-Bit Integer to 32-Bit Floating-Point Value .....                                          | 634 |
| MI32TOF32 MRa, MRb Convert 32-Bit Integer to 32-Bit Floating-Point Value .....                                            | 635 |
| MLSL32 MRa, #SHIFT Logical Shift Left.....                                                                                | 636 |
| MLSR32 MRa, #SHIFT Logical Shift Right.....                                                                               | 637 |
| MMACF32 MR3, MR2, MRd, MRe, MRf   MMOV32 MRa, mem32 32-Bit Floating-Point Multiply and Accumulate with Parallel Move..... | 638 |
| MMAXF32 MRa, MRb 32-Bit Floating-Point Maximum.....                                                                       | 641 |
| MMAXF32 MRa, #16FHi 32-Bit Floating-Point Maximum.....                                                                    | 643 |
| MMINF32 MRa, MRb 32-Bit Floating-Point Minimum.....                                                                       | 644 |
| MMINF32 MRa, #16FHi 32-Bit Floating-Point Minimum.....                                                                    | 646 |
| MMOV16 MARx, MRa, #16I Load the Auxiliary Register with MRa + 16-bit Immediate Value.....                                 | 647 |
| MMOV16 MARx, mem16 Load MAR1 with 16-bit Value.....                                                                       | 650 |
| MMOV16 mem16, MARx Move 16-Bit Auxiliary Register Contents to Memory.....                                                 | 653 |
| MMOV16 mem16, MRa Move 16-Bit Floating-Point Register Contents to Memory.....                                             | 654 |
| MMOV32 mem32, MRa Move 32-Bit Floating-Point Register Contents to Memory .....                                            | 656 |
| MMOV32 mem32, MSTF Move 32-Bit MSTF Register to Memory.....                                                               | 657 |
| MMOV32 MRa, mem32 {, CNDF} Conditional 32-Bit Move.....                                                                   | 658 |
| MMOV32 MRa, MRb {, CNDF} Conditional 32-Bit Move.....                                                                     | 660 |
| MMOV32 MSTF, mem32 Move 32-Bit Value from Memory to the MSTF Register.....                                                | 662 |
| MMOVD32 MRa, mem32 Move 32-Bit Value from Memory with Data Copy.....                                                      | 663 |
| MMOVF32 MRa, #32F Load the 32-Bits of a 32-Bit Floating-Point Register.....                                               | 665 |
| MMOVI16 MARx, #16I Load the Auxiliary Register with the 16-Bit Immediate Value.....                                       | 666 |
| MMOVI32 MRa, #32FHex Load the 32-Bits of a 32-Bit Floating-Point Register with the Immediate.....                         | 668 |
| MMOVIZ MRa, #16FHi Load the Upper 16-Bits of a 32-Bit Floating-Point Register .....                                       | 670 |
| MMOVZ16 MRa, mem16 Load MRx With 16-bit Value.....                                                                        | 671 |
| MMOVXI MRa, #16FLoHex Move Immediate to the Low 16-Bits of a Floating-Point Register.....                                 | 672 |
| MMPYF32 MRa, MRb, MRc 32-Bit Floating-Point Multiply.....                                                                 | 673 |
| MMPYF32 MRa, #16FHi, MRb 32-Bit Floating-Point Multiply .....                                                             | 674 |
| MMPYF32 MRa, MRb, #16FHi 32-Bit Floating-Point Multiply .....                                                             | 676 |
| MMPYF32 MRa, MRb, MRc  MADF32 MRd, MRe, MRf 32-Bit Floating-Point Multiply with Parallel Add.....                         | 678 |
| MMPYF32 MRd, MRe, MRf   MMOV32 MRa, mem32 32-Bit Floating-Point Multiply with Parallel Move.....                          | 680 |
| MMPYF32 MRd, MRe, MRf   MMOV32 mem32, MRa 32-Bit Floating-Point Multiply with Parallel Move.....                          | 682 |
| MMPYF32 MRa, MRb, MRc   MSUBF32 MRd, MRe, MRf 32-Bit Floating-Point Multiply with Parallel Subtract....                   | 683 |
| MNEGF32 MRa, MRb{, CNDF} Conditional Negation.....                                                                        | 684 |
| MNOP No Operation.....                                                                                                    | 686 |
| MOR32 MRa, MRb, MRc Bitwise OR.....                                                                                       | 687 |
| MRCNDD {CNDF} Return Conditional Delayed.....                                                                             | 688 |
| MSETFLG FLAG, VALUE Set or Clear Selected Floating-Point Status Flags.....                                                | 691 |
| MSTOP Stop Task.....                                                                                                      | 692 |
| MSUB32 MRa, MRb, MRc 32-Bit Integer Subtraction.....                                                                      | 694 |
| MSUBF32 MRa, MRb, MRc 32-Bit Floating-Point Subtraction.....                                                              | 695 |
| MSUBF32 MRa, #16FHi, MRb 32-Bit Floating-Point Subtraction.....                                                           | 696 |
| MSUBF32 MRd, MRe, MRf   MMOV32 MRa, mem32 32-Bit Floating-Point Subtraction with Parallel Move.....                       | 698 |

|                                                                                                     |     |
|-----------------------------------------------------------------------------------------------------|-----|
| MSUBF32 MRd, MRe, MRf   MMOV32 mem32, MRa 32-Bit Floating-Point Subtraction with Parallel Move..... | 699 |
| MSWAPF MRa, MRb {, CNDF} Conditional Swap.....                                                      | 700 |
| MTESTTF CNDF Test MSTF Register Flag Condition.....                                                 | 702 |
| MUI16TOF32 MRa, mem16 Convert Unsigned 16-Bit Integer to 32-Bit Floating-Point Value.....           | 704 |
| MUI16TOF32 MRa, MRb Convert Unsigned 16-Bit Integer to 32-Bit Floating-Point Value.....             | 705 |
| MUI32TOF32 MRa, mem32 Convert Unsigned 32-Bit Integer to 32-Bit Floating-Point Value.....           | 706 |
| MUI32TOF32 MRa, MRb Convert Unsigned 32-Bit Integer to 32-Bit Floating-Point Value.....             | 707 |
| MXOR32 MRa, MRb, MRc Bitwise Exclusive Or.....                                                      | 708 |

## MABSF32 MRa, MRb

### 32-Bit Floating-Point Absolute Value

#### Operands

|     |                                                      |
|-----|------------------------------------------------------|
| MRa | CLA floating-point destination register (MR0 to MR3) |
| MRb | CLA floating-point source register (MR0 to MR3)      |

#### Opcode

```
LSW: 0000 0000 0000 bbaa
MSW: 0111 1110 0010 0000
```

#### Description

The absolute value of MRb is loaded into MRa. Only the sign bit of the operand is modified by the MABSF32 instruction.

```
if (MRb < 0) {MRa = -MRb};
else {MRa = MRb};
```

#### Flags

This instruction modifies the following flags in the MSTF register:

| Flag     | TF | ZF  | NF  | LUF | LVF |
|----------|----|-----|-----|-----|-----|
| Modified | No | Yes | Yes | No  | No  |

The MSTF register flags are modified as follows:

```
NF = 0;
ZF = 0;
if ( MRa(30:23) == 0) ZF = 1;
```

#### Pipeline

This is a single-cycle instruction.

#### Example

```
MMOVIZ MR0, #-2.0 ; MR0 = -2.0 (0xC0000000)
MABSF32 MR0, MR0 ; MR0 = 2.0 (0x40000000), ZF = NF = 0
MMOVIZ MR0, #5.0 ; MR0 = 5.0 (0x40A00000)
MABSF32 MR0, MR0 ; MR0 = 5.0 (0x40A00000), ZF = NF = 0
MMOVIZ MR0, #0.0 ; MR0 = 0.0
MABSF32 MR0, MR0 ; MR0 = 0.0 ZF = 1, NF = 0
```

#### See also

[MNEGF32 MRa, MRb {, CNDF}](#)

## MADD32 MRa, MRb, MRc

### 32-Bit Integer Add

#### Operands

|     |                                                      |
|-----|------------------------------------------------------|
| MRa | CLA floating-point destination register (MR0 to MR3) |
| MRb | CLA floating-point destination register (MR0 to MR3) |
| MRc | CLA floating-point destination register (MR0 to MR3) |

#### Opcode

```
LSW: 0000 0000 000cc bbaa
MSW: 0111 1110 1100 0000
```

#### Description

32-bit integer addition of MRb and MRc.

```
MRa(31:0) = MRb(31:0) + MRc(31:0);
```

#### Flags

This instruction modifies the following flags in the MSTF register:

| Flag     | TF | ZF  | NF  | LUF | LVF |
|----------|----|-----|-----|-----|-----|
| Modified | No | Yes | Yes | No  | No  |

The MSTF register flags are modified based on the integer results of the operation.

```
NF = MRa(31);
ZF = 0;
if(MRa(31:0) == 0) { ZF = 1; };
```

#### Pipeline

This is a single-cycle instruction.

#### Example

```
; Given A = (int32)1
; B = (int32)2
; C = (int32)-7
;
; Calculate Y2 = A + B + C
;
_ClalTask1:
    MOV32 MR0, @_A      ; MR0 = 1 (0x00000001)
    MOV32 MR1, @_B      ; MR1 = 2 (0x00000002)
    MOV32 MR2, @_C      ; MR2 = -7 (0xFFFFFFFF9)
    MADD32 MR3, MR0, MR1 ; A + B
    MADD32 MR3, MR2, MR3 ; A + B + C = -4 (0xFFFFFFFFC)
    MOV32 @_y2, MR3      ; Store y2
    MSTOP                ; end of task
```

#### See also

[MAND32 MRa, MRb, MRc](#)  
[MASR32 MRa, #SHIFT](#)  
[MLSL32 MRa, #SHIFT](#)  
[MLSR32 MRa, #SHIFT](#)  
[MOR32 MRa, MRb, MRc](#)  
[MXOR32 MRa, MRb, MRc](#)  
[MSUB32 MRa, MRb, MRc](#)

**MADDF32 MRa, #16FHi, MRb****32-Bit Floating-Point Addition****Operands**

|        |                                                                                                                                                             |
|--------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| MRa    | CLA floating-point destination register (MR0 to MR3)                                                                                                        |
| #16FHi | A 16-bit immediate value that represents the upper 16-bits of an IEEE 32-bit floating-point value. The low 16-bits of the mantissa are assumed to be all 0. |
| MRb    | CLA floating-point source register (MR0 to MR3)                                                                                                             |

**Opcode**

```
LSW: I I I I I I I I I I I I I I
MSW: 0 1 1 1 0 1 1 1 1 1 0 0 b b a a
```

**Description**

Add MRb to the floating-point value represented by the immediate operand. Store the result of the addition in MRa.

#16FHi is a 16-bit immediate value that represents the upper 16-bits of an IEEE 32-bit floating-point value. The low 16-bits of the mantissa are assumed to be all 0. #16FHi is most useful for representing constants where the lowest 16-bits of the mantissa are 0. Some examples are 2.0 (0x40000000), 4.0 (0x40800000), 0.5 (0x3F000000), and -1.5 (0xBFC00000). The assembler will accept either a hex or float as the immediate value. That is, the value -1.5 can be represented as #-1.5 or #0xBFC0.

```
MRa = MRb + #16FHi:0;
```

This instruction can also be written as MADDF32 MRa, MRb, #16FHi.

**Flags**

This instruction modifies the following flags in the MSTF register:

| Flag     | TF | ZF | NF | LUF | LVF |
|----------|----|----|----|-----|-----|
| Modified | No | No | No | Yes | Yes |

The MSTF register flags are modified as follows:

- LUF = 1 if MADDF32 generates an underflow condition.
- LVF = 1 if MADDF32 generates an overflow condition.

**Pipeline**

This is a single-cycle instruction.

**Example**

```
; Add to MR1 the value 2.0 in 32-bit floating-point format
; Store the result in MR0
MADDF32 MR0, #2.0, MR1 ; MR0 = 2.0 + MR1
; Add to MR3 the value -2.5 in 32-bit floating-point format
; Store the result in MR2
MADDF32 MR2, #-2.5, MR3 ; MR2 = -2.5 + MR3
; Add to MR3 the value 0x3FC00000 (1.5)
; Store the result in MR3
MADDF32 MR3, #0x3FC0, MR3 ; MR3 = 1.5 + MR3
```

**See also**

[MADDF32 MRa, MRb, #16FHi](#)  
[MADDF32 MRa, MRb, MRc](#)  
[MADDF32 MRd, MRe, MRf || MMOV32 MRa, mem32](#)  
[MADDF32 MRd, MRe, MRf || MMOV32 mem32, MRa](#)  
[MMPYF32 MRa, MRb, MRc || MADDF32 MRd, MRe, MRf](#)

## MADDF32 MRa, MRb, #16FHi

### 32-Bit Floating-Point Addition

#### Operands

|        |                                                                                                                                                             |
|--------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| MRa    | CLA floating-point destination register (MR0 to MR3)                                                                                                        |
| MRb    | CLA floating-point source register (MR0 to MR3)                                                                                                             |
| #16FHi | A 16-bit immediate value that represents the upper 16-bits of an IEEE 32-bit floating-point value. The low 16-bits of the mantissa are assumed to be all 0. |

#### Opcode

```
LSW: I I I I I I I I I I I I I I I I
MSW: 0 1 1 1 0 1 1 1 1 1 0 0 b b a a
```

#### Description

Add MRb to the floating-point value represented by the immediate operand. Store the result of the addition in MRa.

#16FHi is a 16-bit immediate value that represents the upper 16-bits of an IEEE 32-bit floating-point value. The low 16-bits of the mantissa are assumed to be all 0. #16FHi is most useful for representing constants where the lowest 16-bits of the mantissa are 0. Some examples are 2.0 (0x40000000), 4.0 (0x40800000), 0.5 (0x3F000000), and -1.5 (0xBFC00000). The assembler will accept either a hex or float as the immediate value. That is, the value -1.5 can be represented as #-1.5 or #0xBFC0.

```
MRa = MRb + #16FHi:0;
```

This instruction can also be written as MADDF32 MRa, #16FHi, MRb.

#### Flags

This instruction modifies the following flags in the MSTF register:

| Flag     | TF | ZF | NF | LUF | LVF |
|----------|----|----|----|-----|-----|
| Modified | No | No | No | Yes | Yes |

The MSTF register flags are modified as follows:

- LUF = 1 if MADDF32 generates an underflow condition.
- LVF = 1 if MADDF32 generates an overflow condition.

#### Pipeline

This is a single-cycle instruction.

## MADDF32 MRa, MRb, #16FHi (continued)

### 32-Bit Floating-Point Addition

#### Example 1

```
; X is an array of 32-bit floating-point values
; Find the maximum value in an array X
; and store it in Result
;
_ClalTask1:
    MMOV16    MAR1, #_X          ; Start address
    MUI16TOF32 MR0, @_len        ; Length of the array
    MNOP      ; delay for MAR1 load
    MNOP      ; delay for MAR1 load
    MMOV32    MR1, *MAR1[2]++    ; MR1 = X0
LOOP
    MMOV32    MR2, *MAR1[2]++    ; MR2 = next element
    MMAXF32   MR1, MR2          ; MR1 = MAX(MR1, MR2)
    MADDF32   MR0, MR0, #-1.0    ; Decrement the counter
    MCMPPF32  MR0, #0.0         ; Set/clear flags for MBCNDD
    MNOP
    MNOP
    MNOP
    MBCNDD   LOOP, NEQ          ; Branch if not equal to zero
    MMOV32   @_Result, MR1      ; Always executed
    MNOP      ; Always executed
    MNOP      ; Always executed
    MSTOP     ; End of task
```

#### Example 2

```
; Show the basic operation of MADDF32
;
; Add to MR1 the value 2.0 in 32-bit floating-point format
; Store the result in MR0
    MADDF32 MR0, MR1, #2.0      ; MR0 = MR1 + 2.0
; Add to MR3 the value -2.5 in 32-bit floating-point format
; Store the result in MR2
    MADDF32 MR2, MR3, #-2.5     ; MR2 = MR3 + (-2.5)
; Add to MR0 the value 0x3FC00000 (1.5)
; Store the result in MR0
    MADDF32 MR0, MR0, #0x3FC0   ; MR0 = MR0 + 1.5
```

#### See also

[MADDF32 MRa, #16FHi, MRb](#)  
[MADDF32 MRa, MRb, MRc](#)  
[MADDF32 MRd, MRe, MRf || MMOV32 MRa, mem32](#)  
[MADDF32 MRd, MRe, MRf || MMOV32 mem32, MRa](#)  
[MMPYF32 MRa, MRb, MRc || MADDF32 MRd, MRe, MRf](#)



## MADDF32 MRa, MRb, MRc

### 32-Bit Floating-Point Addition

#### Operands

|     |                                                      |
|-----|------------------------------------------------------|
| MRa | CLA floating-point destination register (MR0 to MR3) |
| MRb | CLA floating-point source register (MR0 to MR3)      |
| MRc | CLA floating-point source register (MR0 to MR3)      |

#### Opcode

```
LSW: 000 0000 00cc bbba
MSW: 0111 1100 0010 0000
```

#### Description

Add the contents of MRc to the contents of MRb and load the result into MRa.

```
MRa = MRb + MRc;
```

#### Flags

This instruction modifies the following flags in the MSTF register:

| Flag     | TF | ZF | NF | LUF | LVF |
|----------|----|----|----|-----|-----|
| Modified | No | No | No | Yes | Yes |

The MSTF register flags are modified as follows:

- LUF = 1 if MADDF32 generates an underflow condition.
- LVF = 1 if MADDF32 generates an overflow condition.

#### Pipeline

This is a single-cycle instruction.

#### Example

```
; Given M1, X1 and B1 are 32-bit floating point numbers
; Calculate Y1 = M1*X1+B1
;
_Cla1Task1:
    MMOV32 MR0,@M1      ; Load MR0 with M1
    MMOV32 MR1,@X1      ; Load MR1 with X1
    MPPYF32 MR1,MR1,MR0 ; Multiply M1*X1
    || MMOV32 MR0,@B1    ; and in parallel load MR0 with B1
    MADDF32 MR1,MR1,MR0 ; Add M*X1 to B1 and store in MR1
    MMOV32 @Y1,MR1      ; Store the result
    MSTOP                ; end of task
```

#### See also

[MADDF32 MRa, #16FHi, MRb](#)  
[MADDF32 MRa, MRb, #16FHi](#)  
[MADDF32 MRd, MRe, MRf || MMOV32 MRa, mem32](#)  
[MADDF32 MRd, MRe, MRf || MMOV32 mem32, MRa](#)  
[MPPYF32 MRa, MRb, MRc || MADDF32 MRd, MRe, MRf](#)

**MADDF32 MRd, MRe, MRf||MMOV32 mem32, MRa****32-Bit Floating-Point Addition with Parallel Move****Operands**

|       |                                                                                                                          |
|-------|--------------------------------------------------------------------------------------------------------------------------|
| MRd   | CLA floating-point destination register for the MADDF32 (MR0 to MR3)                                                     |
| MRe   | CLA floating-point source register for the MADDF32 (MR0 to MR3)                                                          |
| MRf   | CLA floating-point source register for the MADDF32 (MR0 to MR3)                                                          |
| mem32 | 32-bit memory location accessed using one of the available addressing modes. This will be the destination of the MMOV32. |
| MRa   | CLA floating-point source register for the MMOV32 (MR0 to MR3)                                                           |

**Opcode**

```
LSW: mmmm mmmm mmmm mmmm
MSW: 0101 ffee ddaa addr
```

**Description**

Perform an MADDF32 and a MMOV32 in parallel. Add MRf to the contents of MRe and store the result in MRd. In parallel move the contents of MRa to the 32-bit location mem32.

```
MRd = MRe + MRf;
[mem32] = MRa;
```

**Flags**

This instruction modifies the following flags in the MSTF register:

| Flag     | TF | ZF | NF | LUF | LVF |
|----------|----|----|----|-----|-----|
| Modified | No | No | No | Yes | Yes |

The MSTF register flags are modified as follows:

- LUF = 1 if MADDF32 generates an underflow condition.
- LVF = 1 if MADDF32 generates an overflow condition.

**Pipeline**

Both MADDF32 and MMOV32 complete in a single cycle.

**Example**

```
; Given A, B and C are 32-bit floating-point numbers
; Calculate Y2 = (A * B)
;           Y3 = (A * B) + C
;
;
_ClalTask2:
    MMOV32    MR0, @_A        ; Load MR0 with A
    MMOV32    MR1, @_B        ; Load MR1 with B
    MPPYF32   MR1, MR1, MR0    ; Multiply A*B
    || MMOV32  MR0, @_C        ; and in parallel load MR0 with C
    MADDF32   MR1, MR1, MR0    ; Add (A*B) to C
    || MMOV32  @_Y2, MR1       ; and in parallel store A*B
    MMOV32    @_Y3, MR1       ; Store the A*B + C
    MSTOP
```

**See also**

[MADDF32 MRa, #16FHi, MRb](#)  
[MADDF32 MRa, MRb, #16FHi](#)  
[MADDF32 MRa, MRb, MRc](#)  
[MPPYF32 MRa, MRb, MRc || MADDF32 MRd, MRe, MRf](#)  
[MADDF32 MRd, MRe, MRf || MMOV32 MRa, mem32](#)

## MADDF32 MRd, MRe, MRf ||MMOV32 MRa, mem32

### 32-Bit Floating-Point Addition with Parallel Move

#### Operands

|       |                                                                                                                  |
|-------|------------------------------------------------------------------------------------------------------------------|
| MRd   | CLA floating-point destination register for the MADDF32 (MR0 to MR3).<br>MRd cannot be the same register as MRa. |
| MRe   | CLA floating-point source register for the MADDF32 (MR0 to MR3)                                                  |
| MRf   | CLA floating-point source register for the MADDF32 (MR0 to MR3)                                                  |
| MRa   | CLA floating-point destination register for the MMOV32 (MR0 to MR3).<br>MRa cannot be the same register as MRd.  |
| mem32 | 32-bit memory location accessed using one of the available addressing modes. This is the source for the MMOV32.  |

#### Opcode

```
LSW: mmmm mmmm mmmm mmmm
MSW: 0001 ffee ddaa addr
```

#### Description

Perform an MADDF32 and a MMOV32 operation in parallel. Add MRf to the contents of MRe and store the result in MRd. In parallel move the contents of the 32-bit location mem32 to MRa.

```
MRd = MRe + MRf;
MRa = [mem32];
```

#### Restrictions

The destination register for the MADDF32 and the MMOV32 must be unique. That is, MRa and MRd cannot be the same register.

#### Flags

This instruction modifies the following flags in the MSTF register:

| Flag     | TF | ZF  | NF  | LUF | LVF |
|----------|----|-----|-----|-----|-----|
| Modified | No | Yes | Yes | Yes | Yes |

The MSTF register flags are modified as follows:

- LUF = 1 if MADDF32 generates an underflow condition.
- LVF = 1 if MADDF32 generates an overflow condition.

The MMOV32 Instruction will set the NF and ZF flags as follows:

```
NF = MRa(31);
ZF = 0;
if(MRa(30:23) == 0) { ZF = 1; NF = 0; };
```

#### Pipeline

The MADDF32 and the MMOV32 both complete in a single cycle.

## MADDF32 MRd, MRe, MRf || MMOV32 MRa, mem32 (continued)

### 32-Bit Floating-Point Addition with Parallel Move

#### Example 1

```
; Given A, B and C are 32-bit floating-point numbers
; Calculate Y1 = A + 4B
;           Y2 = A + C
;
;
_Cla1Task1:
    MMOV32 MR0, @A          ; Load MR0 with A
    MMOV32 MR1, @B          ; Load MR1 with B
    MMPYF32 MR1, MR1, #4.0  ; Multiply 4 * B
||   MMOV32 MR2, @C          ; and in parallel load C
    MADDF32 MR3, MR0, MR1   ; Add A + 4B
    MADDF32 MR3, MR0, MR2   ; Add A + C
||   MMOV32 @Y1, MR3        ; and in parallel store A+4B
    MMOV32 @Y2, MR3        ; store A + C
                                ; end of task
                                ; MSTOP
```

#### Example 2

```
; Given A, B and C are 32-bit floating-point numbers
; Calculate Y3 = (A + B)
;           Y4 = (A + B) * C
;
;
_Cla1Task2:
    MMOV32 MR0, @A          ; Load MR0 with A
    MMOV32 MR1, @B          ; Load MR1 with B
    MADDF32 MR1, MR1, MR0   ; Add A+B
||   MMOV32 MR0, @C          ; and in parallel load MR0 with C
    MMPYF32 MR1, MR1, MR0   ; Multiply (A+B) by C
||   MMOV32 @Y3, MR1        ; and in parallel store A+B
    MMOV32 @Y4, MR1        ; Store the (A+B) * C
    MSTOP                  ; end of task
```

#### See also

[MADDF32 MRa, #16FHi, MRb](#)  
[MADDF32 MRa, MRb, #16FHi](#)  
[MADDF32 MRa, MRb, MRc](#)  
[MADDF32 MRd, MRe, MRf || MMOV32 mem32, MRa](#)  
[MMPYF32 MRa, MRb, MRc || MADDF32 MRd, MRe, MRf](#)

## MAND32 MRa, MRb, MRc

### Bitwise AND

#### Operands

|     |                                                      |
|-----|------------------------------------------------------|
| MRa | CLA floating-point destination register (MR0 to MR3) |
| MRb | CLA floating-point source register (MR0 to MR3)      |
| MRc | CLA floating-point source register (MR0 to MR3)      |

#### Opcode

```
LSW: 0000 0000 00cc bbaa
MSW: 0111 1100 0110 0000
```

#### Description

Bitwise AND of MRb with MRc.

```
MRa(31:0) = MRb(31:0) AND MRc(31:0);
```

#### Flags

This instruction modifies the following flags in the MSTF register:

| Flag     | TF | ZF  | NF  | LUF | LVF |
|----------|----|-----|-----|-----|-----|
| Modified | No | Yes | Yes | No  | No  |

The MSTF register flags are modified based on the integer results of the operation.

```
NF = MRa(31);
ZF = 0;
if(MRa(31:0) == 0) { ZF = 1; }
```

#### Pipeline

This is a single-cycle instruction.

#### Example

```
MMOVIZ MR0, #0x5555 ; MR0 = 0x5555AAAA
MMOVXI MR0, #0xAAAA
MMOVIZ MR1, #0x5432 ; MR1 = 0x5432FEDC
MMOVXI MR1, #0xFEDC
; 0101 AND 0101 = 0101 (5)
; 0101 AND 0100 = 0100 (4)
; 0101 AND 0011 = 0001 (1)
; 0101 AND 0010 = 0000 (0)
; 1010 AND 1111 = 1010 (A)
; 1010 AND 1110 = 1010 (A)
; 1010 AND 1101 = 1000 (8)
; 1010 AND 1100 = 1000 (8)
MAND32 MR2, MR1, MR0 ; MR3 = 0x5410AA88
```

#### See also

[MADD32 MRa, MRb, MRc](#)  
[MASR32 MRa, #SHIFT](#)  
[MLSL32 MRa, #SHIFT](#)  
[MLSR32 MRa, #SHIFT](#)  
[MOR32 MRa, MRb, MRc](#)  
[MXOR32 MRa, MRb, MRc](#)  
[MSUB32 MRa, MRb, MRc](#)

## MASR32 MRa, #SHIFT

### Arithmetic Shift Right

#### Operands

|        |                                                             |
|--------|-------------------------------------------------------------|
| MRa    | CLA floating-point source/destination register (MR0 to MR3) |
| #SHIFT | Number of bits to shift (1 to 32)                           |

#### Opcode

```
LSW: 0000 0000 0shi ftaa
MSW: 0111 1011 0100 0000
```

#### Description

Arithmetic shift right of MRa by the number of bits indicated. The number of bits can be 1 to 32.

```
MARa(31:0) = Arithmetic Shift(MARa(31:0) by #SHIFT bits);
```

#### Flags

This instruction modifies the following flags in the MSTF register:

| Flag     | TF | ZF  | NF  | LUF | LVF |
|----------|----|-----|-----|-----|-----|
| Modified | No | Yes | Yes | No  | No  |

The MSTF register flags are modified based on the integer results of the operation.

```
NF = MRa(31);
ZF = 0;
if(MRa(31:0) == 0) { ZF = 1; }
```

#### Pipeline

This is a single-cycle instruction.

#### Example

```
; Given m2 = (int32)32
; x2 = (int32)64
; b2 = (int32)-128
;
; Calculate
; m2 = m2/2
; x2 = x2/4
; b2 = b2/8
;
_Cla1Task2:
  MMOV32 MR0, @_m2 ; MR0 = 32 (0x00000020)
  MMOV32 MR1, @_x2 ; MR1 = 64 (0x00000040)
  MMOV32 MR2, @_b2 ; MR2 = -128 (0xFFFFF80)
  MASR32 MR0, #1 ; MR0 = 16 (0x00000010)
  MASR32 MR1, #2 ; MR1 = 16 (0x00000010)
  MASR32 MR2, #3 ; MR2 = -16 (0xFFFFF0)
  MMOV32 @_m2, MR0 ; store results
  MMOV32 @_x2, MR1
  MMOV32 @_b2, MR2
  MSTOP ; end of task
```

#### See also

[MADD32 MRa, MRb, MRc](#)  
[MAND32 MRa, MRb, MRc](#)  
[MLSL32 MRa, #SHIFT](#)  
[MLSR32 MRa, #SHIFT](#)  
[MOR32 MRa, MRb, MRc](#)  
[MXOR32 MRa, MRb, MRc](#)  
[MSUB32 MRa, MRb, MRc](#)

## MBCNDD 16BitDest {, CNDF}

### Branch Conditional Delayed

#### Operands

|           |                                         |
|-----------|-----------------------------------------|
| 16BitDest | 16-bit destination if condition is true |
| CNDF      | Optional condition tested               |

#### Opcode

```
LSW: dest dest dest dest
MSW: 0111 1001 1000 cndf
```

#### Description

If the specified condition is true, then branch by adding the signed 16BitDest value to the MPC value. Otherwise, continue without branching. If the address overflows, it wraps around. Therefore a value of "0xFFFFE" will put the MPC back to the MBCNDD instruction.

Please refer to the pipeline section for important information regarding this instruction.

```
if (CNDF == TRUE) MPC += 16BitDest;
```

CNDF is one of the following conditions:

| Encode <sup>(1)</sup> | CNDF                | Description                          | MSTF Flags Tested   |
|-----------------------|---------------------|--------------------------------------|---------------------|
| 0000                  | NEQ                 | Not equal to zero                    | ZF == 0             |
| 0001                  | EQ                  | Equal to zero                        | ZF == 1             |
| 0010                  | GT                  | Greater than zero                    | ZF == 0 AND NF == 0 |
| 0011                  | GEQ                 | Greater than or equal to zero        | NF == 0             |
| 0100                  | LT                  | Less than zero                       | NF == 1             |
| 0101                  | LEQ                 | Less than or equal to zero           | ZF == 1 OR NF == 1  |
| 1010                  | TF                  | Test flag set                        | TF == 1             |
| 1011                  | NTF                 | Test flag not set                    | TF == 0             |
| 1100                  | LU                  | Latched underflow                    | LUF == 1            |
| 1101                  | LV                  | Latched overflow                     | LVF == 1            |
| 1110                  | UNC                 | Unconditional                        | None                |
| 1111                  | UNCF <sup>(2)</sup> | Unconditional with flag modification | None                |

(1) Values not shown are reserved.

(2) This is the default operation, if no CNDF field is specified. This condition will allow the ZF and NF flags to be modified when a conditional operation is executed. All other conditions will not modify these flags.

#### Restrictions

The MBCNDD instruction is not allowed three instructions before or after a MBCNDD, MCCNDD or MRCNDD instruction. Refer to the pipeline section for more information.

#### Flags

This instruction does not modify flags in the MSTF register.

| Flag     | TF | ZF | NF | LUF | LVF |
|----------|----|----|----|-----|-----|
| Modified | No | No | No | No  | No  |

**MBCNDD 16BitDest {, CNDF}** (continued)**Branch Conditional Delayed****Pipeline**

The MBCNDD instruction by itself is a single-cycle instruction. As shown in [Table 10-10](#) for each branch 6 instruction slots are executed; three before the branch instruction (I2-I4) and three after the branch instruction (I5-I7). The total number of cycles for a branch taken or not taken depends on the usage of these slots. That is, the number of cycles depends on how many slots are filled with a MNOP as well as which slots are filled. The effective number of cycles for a branch can, therefore, range from 1 to 7 cycles. The number of cycles for a branch taken may not be the same as for a branch not taken.

Referring to [Table 10-10](#) and [Table 10-11](#), the instructions before and after MBCNDD have the following properties:

- **I1**
  - I1 is the last instruction that can effect the CNDF flags for the MBCNDD instruction. The CNDF flags are tested in the D2 phase of the pipeline. That is, a decision is made whether to branch or not when MBCNDD is in the D2 phase.
  - There are no restrictions on the type of instruction for I1.
- **I2, I3 and I4**
  - The three instructions proceeding MBCNDD can change MSTF flags but will have no effect on whether the MBCNDD instruction branches or not. This is because the flag modification will occur after the D2 phase of the MBCNDD instruction.
  - These instructions must not be the following: MSTOP, MDEBUGSTOP, MBCNDD, MCCNDD or MRCNDD.
- **I5, I6 and I7**
  - The three instructions following MBCNDD are always executed irrespective of whether the branch is taken or not.
  - These instructions must not be the following: MSTOP, MDEBUGSTOP, MBCNDD, MCCNDD or MRCNDD.

```

<Instruction 1> ; I1 Last instruction that can affect flags for
                ; the MBCNDD operation
<Instruction 2> ; I2 Cannot be stop, branch, call or return
<Instruction 3> ; I3 Cannot be stop, branch, call or return
<Instruction 4> ; I4 Cannot be stop, branch, call or return
MBCNDD _Skip, NEQ ; Branch to Skip if not equal to zero
                ; Three instructions after MBCNDD are always
                ; executed whether the branch is taken or not

<Instruction 5> ; I5 Cannot be stop, branch, call or return
<Instruction 6> ; I6 Cannot be stop, branch, call or return
<Instruction 7> ; I7 Cannot be stop, branch, call or return
<Instruction 8> ; I8
<Instruction 9> ; I9
....
_Skip:
<Destination 1> ; d1 Can be any instruction
<Destination 2> ; d2
<Destination 3> ; d3
....
....
MSTOP
....

```



**MBCNDD 16BitDest {, CNDF}** (continued)

**Branch Conditional Delayed**
**Table 10-10. Pipeline Activity For MBCNDD, Branch Not Taken**

| Instruction | F1     | F2     | D1     | D2     | R1  | R2  | E   | W |
|-------------|--------|--------|--------|--------|-----|-----|-----|---|
| I1          | I1     |        |        |        |     |     |     |   |
| I2          | I2     | I1     |        |        |     |     |     |   |
| I3          | I3     | I2     | I1     |        |     |     |     |   |
| I4          | I4     | I3     | I2     | I1     |     |     |     |   |
| MBCNDD      | MBCNDD | I4     | I3     | I2     | I1  |     |     |   |
| I5          | I5     | MBCNDD | I4     | I3     | I2  | I1  |     |   |
| I6          | I6     | I5     | MBCNDD | I4     | I3  | I2  | I1  |   |
| I7          | I7     | I6     | I5     | MBCNDD | I4  | I3  | I2  |   |
| I8          | I8     | I7     | I6     | I5     | -   | I4  | I3  |   |
| I9          | I9     | I8     | I7     | I6     | I5  | -   | I4  |   |
| I10         | I10    | I9     | I8     | I7     | I6  | I5  | -   |   |
|             |        | I10    | I9     | I8     | I7  | I6  | I5  |   |
|             |        |        | I10    | I9     | I8  | I7  | I6  |   |
|             |        |        |        | I10    | I9  | I8  | I7  |   |
|             |        |        |        |        | I10 | I9  | I8  |   |
|             |        |        |        |        |     | I10 | I9  |   |
|             |        |        |        |        |     |     | I10 |   |

**Table 10-11. Pipeline Activity For MBCNDD, Branch Taken**

| Instruction | F1     | F2     | D1     | D2     | R1 | R2 | E  | W |
|-------------|--------|--------|--------|--------|----|----|----|---|
| I1          | I1     |        |        |        |    |    |    |   |
| I2          | I2     | I1     |        |        |    |    |    |   |
| I3          | I3     | I2     | I1     |        |    |    |    |   |
| I4          | I4     | I3     | I2     | I1     |    |    |    |   |
| MBCNDD      | MBCNDD | I4     | I3     | I2     | I1 |    |    |   |
| I5          | I5     | MBCNDD | I4     | I3     | I2 | I1 |    |   |
| I6          | I6     | I5     | MBCNDD | I4     | I3 | I2 | I1 |   |
| I7          | I7     | I6     | I5     | MBCNDD | I4 | I3 | I2 |   |
| d1          | d1     | I7     | I6     | I5     | -  | I4 | I3 |   |
| d2          | d2     | d1     | I7     | I6     | I5 | -  | I4 |   |
| d3          | d3     | d2     | d1     | I7     | I6 | I5 | -  |   |
|             |        | d3     | d2     | d1     | I7 | I6 | I5 |   |
|             |        |        | d3     | d2     | d1 | I7 | I6 |   |
|             |        |        |        | d3     | d2 | d1 | I7 |   |
|             |        |        |        |        | d3 | d2 | d1 |   |
|             |        |        |        |        |    | d3 | d2 |   |
|             |        |        |        |        |    |    | d3 |   |

**MBCNDD 16BitDest {, CNDF} (continued)**
**Branch Conditional Delayed**
**Example 1**

```

; if (State == 0.1)
; RampState = RampState || RAMPMASK
; else if (State == 0.01)
; CoastState = CoastState || COASTMASK
; else
; SteadyState = SteadyState || STEADYMASK
;
_ClalTask1:
MMOV32 MR0, @State
MCMPPF32 MR0, #0.1          ; Affects flags for 1st MBCNDD (A)
MNOP
MNOP
MNOP
MBCNDD Skip1, NEQ           ; (A) If State != 0.1, go to Skip1
MNOP ; Always executed
MNOP ; Always executed
MNOP ; Always executed
MMOV32 MR1, @RampState      ; Execute if (A) branch not taken
MMOVXI MR2, #RAMPMASK      ; Execute if (A) branch not taken
MOR32 MR1, MR2              ; Execute if (A) branch not taken
MMOV32 @RampState, MR1      ; Execute if (A) branch not taken
MSTOP                      ; end of task if (A) branch not taken
Skip1:
MCMPPF32 MR0, #0.01         ; Affects flags for 2nd MBCNDD (B)
MNOP
MNOP
MNOP
MBCNDD Skip2, NEQ           ; (B) If State != 0.01, go to Skip2
MNOP ; Always executed
MNOP ; Always executed
MNOP ; Always executed
MMOV32 MR1, @CoastState     ; Execute if (B) branch not taken
MMOVXI MR2, #COASTMASK      ; Execute if (B) branch not taken
MOR32 MR1, MR2              ; Execute if (B) branch not taken
MMOV32 @CoastState, MR1     ; Execute if (B) branch not taken
MSTOP
Skip2:
MMOV32 MR3, @SteadyState    ; Executed if (B) branch taken
MMOVXI MR2, #STEADYMASK     ; Executed if (B) branch taken
MOR32 MR3, MR2              ; Executed if (B) branch taken
MMOV32 @SteadyState, MR3    ; Executed if (B) branch taken
MSTOP

```

## MBCNDD 16BitDest {, CNDF} (continued)

### Branch Conditional Delayed

#### Example 2

```
; This example is the same as Example 1, except
; the code is optimized to take advantage of delay slots
;
; if (State == 0.1)
; RampState = RampState || RAMPMASK
; else if (State == 0.01)
; CoastState = CoastState || COASTMASK
; else
; SteadyState = SteadyState || STEADYMASK
;
_ClalTask2:
_MMOV32 MR0, @State
_MCMPPF32 MR0, #0.1           ; Affects flags for 1st MBCNDD (A)
_MCMPPF32 MR0, #0.01          ; Check used by 2nd MBCNDD (B)
_MTESTTF EQ                   ; Store EQ flag in TF for 2nd MBCNDD (B)
_MNOP
_MBCNDD Skip1, NEQ             ; (A) If State != 0.1, go to Skip1
_MMOV32 MR1, @RampState        ; Always executed
_MMOVXI MR2, #RAMPMASK         ; Always executed
_MOR32 MR1, MR2                ; Always executed
_MMOV32 @RampState, MR1        ; Execute if (A) branch not taken
_MSTOP                        ; end of task if (A) branch not taken
Skip1:
_MMOV32 MR3, @SteadyState
_MMOVXI MR2, #STEADYMASK
_MOR32 MR3, MR2
_MBCNDD Skip2, NTF             ; (B) if State != .01, go to Skip2
_MMOV32 MR1, @CoastState       ; Always executed
_MMOVXI MR2, #COASTMASK        ; Always executed
_MOR32 MR1, MR2                ; Always executed
_MMOV32 @CoastState, MR1       ; Execute if (B) branch not taken
_MSTOP                        ; end of task if (B) branch not taken
Skip2:
_MMOV32 @SteadyState, MR3      ; Executed if (B) branch taken
_MSTOP
```

#### See also

[MCCNDD 16BitDest, CNDF](#)  
[MRCNDD CNDF](#)

**MCCNDD 16BitDest {, CNDF}****Call Conditional Delayed****Operands**

|           |                                         |
|-----------|-----------------------------------------|
| 16BitDest | 16-bit destination if condition is true |
| CNDF      | Optional condition to be tested         |

**Opcode**

```
LSW: dest dest dest dest
MSW: 0111 1001 1001 cndf
```

**Description**

If the specified condition is true, then store the return address in the RPC field of MSTF and make the call by adding the signed 16BitDest value to the MPC value. Otherwise, continue code execution without making the call. If the address overflows, it wraps around. Therefore a value of "0xFFFFE" will put the MPC back to the MCCNDD instruction.

Please refer to the pipeline section for important information regarding this instruction.

```
if (CNDF == TRUE)
{
    RPC = return address;
    MPC += 16BitDest;
};
```

CNDF is one of the following conditions:

| Encode <sup>(1)</sup> | CNDF                | Description                          | MSTF Flags Tested   |
|-----------------------|---------------------|--------------------------------------|---------------------|
| 0000                  | NEQ                 | Not equal to zero                    | ZF == 0             |
| 0001                  | EQ                  | Equal to zero                        | ZF == 1             |
| 0010                  | GT                  | Greater than zero                    | ZF == 0 AND NF == 0 |
| 0011                  | GEQ                 | Greater than or equal to zero        | NF == 0             |
| 0100                  | LT                  | Less than zero                       | NF == 1             |
| 0101                  | LEQ                 | Less than or equal to zero           | ZF == 1 OR NF == 1  |
| 1010                  | TF                  | Test flag set                        | TF == 1             |
| 1011                  | NTF                 | Test flag not set                    | TF == 0             |
| 1100                  | LU                  | Latched underflow                    | LUF == 1            |
| 1101                  | LV                  | Latched overflow                     | LVF == 1            |
| 1110                  | UNC                 | Unconditional                        | None                |
| 1111                  | UNCF <sup>(2)</sup> | Unconditional with flag modification | None                |

(1) Values not shown are reserved.

(2) This is the default operation if no CNDF field is specified. This condition will allow the ZF and NF flags to be modified when a conditional operation is executed. All other conditions will not modify these flags.

**Restrictions**

The MCCNDD instruction is not allowed three instructions before or after a MBCNDD, MCCNDD, or MRCNDD instruction. Refer to the Pipeline section for more details.

**Flags**

This instruction does not modify flags in the MSTF register.

| Flag     | TF | ZF | NF | LUF | LVF |
|----------|----|----|----|-----|-----|
| Modified | No | No | No | No  | No  |

## MCCNDD 16BitDest {, CNDF} (continued)

### Call Conditional Delayed

#### Pipeline

The MCCNDD instruction by itself is a single-cycle instruction. As shown in [Table 10-12](#), for each call 6 instruction slots are executed; three before the call instruction (I2-I4) and three after the call instruction (I5-I7). The total number of cycles for a call taken or not taken depends on the usage of these slots. That is, the number of cycles depends on how many slots are filled with a MNOP as well as which slots are filled. The effective number of cycles for a call can, therefore, range from 1 to 7 cycles. The number of cycles for a call taken may not be the same as for a call not taken.

Referring to the following code fragment and the pipeline diagrams in [Table 10-12](#) and [Table 10-13](#), the instructions before and after MCCNDD have the following properties:

- **I1**
  - I1 is the last instruction that can effect the CNDF flags for the MCCNDD instruction. The CNDF flags are tested in the D2 phase of the pipeline. That is, a decision is made whether to branch or not when MCCNDD is in the D2 phase.
  - There are no restrictions on the type of instruction for I1.
- **I2, I3 and I4**
  - The three instructions proceeding MCCNDD can change MSTF flags but will have no effect on whether the MCCNDD instruction makes the call or not. This is because the flag modification will occur after the D2 phase of the MCCNDD instruction.
  - These instructions must not be the following: MSTOP, MDEBUGSTOP, MBCNDD, MCCNDD or MRCNDD.
- **I5, I6 and I7**
  - The three instructions following MBCNDD are always executed irrespective of whether the branch is taken or not.
  - These instructions must not be the following: MSTOP, MDEBUGSTOP, MBCNDD, MCCNDD or MRCNDD.

```

<Instruction 1> ; I1 Last instruction that can affect flags for
                ; the MCCNDD operation
<Instruction 2> ; I2 Cannot be stop, branch, call or return
<Instruction 3> ; I3 Cannot be stop, branch, call or return
<Instruction 4> ; I4 Cannot be stop, branch, call or return
MCCNDD _func, NEQ ; Call to func if not equal to zero
                ; Three instructions after MCCNDD are always
                ; executed whether the call is taken or not
<Instruction 5> ; I5 Cannot be stop, branch, call or return
<Instruction 6> ; I6 Cannot be stop, branch, call or return
<Instruction 7> ; I7 Cannot be stop, branch, call or return
<Instruction 8> ; I8 The address of this instruction is saved
                ; in the RPC field of the MSTF register.
                ; Upon return this value is loaded into MPC
                ; and fetching continues from this point.
<Instruction 9> ; I9
....
func:
<Destination 1> ; d1 Can be any instruction
<Destination 2> ; d2
<Destination 3> ; d3
<Destination 4> ; d4 Last instruction that can affect flags for
                ; the MRCNDD operation
<Destination 5> ; d5 Cannot be stop, branch, call or return
<Destination 6> ; d6 Cannot be stop, branch, call or return
<Destination 7> ; d7 Cannot be stop, branch, call or return
MRCNDD UNC      ; Return to <Instruction 8>, unconditional
                ; Three instructions after MRCNDD are always
                ; executed whether the return is taken or not
<Destination 8> ; d8 Cannot be stop, branch, call or return
<Destination 9> ; d9 Cannot be stop, branch, call or return

```

**MCCNDD 16BitDest {, CNDF}** (continued)**Call Conditional Delayed**

```

<Destination 10> ; d10 Cannot be stop, branch, call or return
<Destination 11> ; d11
....
MSTOP

```

**Table 10-12. Pipeline Activity For MCCNDD, Call Not Taken**

| Instruction | F1     | F2     | D1     | D2     | R1  | R2  | E   | W |
|-------------|--------|--------|--------|--------|-----|-----|-----|---|
| I1          | I1     |        |        |        |     |     |     |   |
| I2          | I2     | I1     |        |        |     |     |     |   |
| I3          | I3     | I2     | I1     |        |     |     |     |   |
| I4          | I4     | I3     | I2     | I1     |     |     |     |   |
| MCCNDD      | MCCNDD | I4     | I3     | I2     | I1  |     |     |   |
| I5          | I5     | MCCNDD | I4     | I3     | I2  | I1  |     |   |
| I6          | I6     | I5     | MCCNDD | I4     | I3  | I2  | I1  |   |
| I7          | I7     | I6     | I5     | MCCNDD | I4  | I3  | I2  |   |
| I8          | I8     | I7     | I6     | I5     | -   | I4  | I3  |   |
| I9          | I9     | I8     | I7     | I6     | I5  | -   | I4  |   |
| I10         | I10    | I9     | I8     | I7     | I6  | I5  | -   |   |
| etc ....    |        | I10    | I9     | I8     | I7  | I6  | I5  |   |
| ....        |        |        | I10    | I9     | I8  | I7  | I6  |   |
| ....        |        |        |        | I10    | I9  | I8  | I7  |   |
| ....        |        |        |        |        | I10 | I9  | I8  |   |
|             |        |        |        |        |     | I10 | I9  |   |
|             |        |        |        |        |     |     | I10 |   |

**Table 10-13. Pipeline Activity For MCCNDD, Call Taken**

| Instruction       | F1     | F2     | D1     | D2     | R1 | R2 | E  | W |
|-------------------|--------|--------|--------|--------|----|----|----|---|
| I1                | I1     |        |        |        |    |    |    |   |
| I2                | I2     | I1     |        |        |    |    |    |   |
| I3                | I3     | I2     | I1     |        |    |    |    |   |
| I4                | I4     | I3     | I2     | I1     |    |    |    |   |
| MCCNDD            | MCCNDD | I4     | I3     | I2     | I1 |    |    |   |
| I5                | I5     | MCCNDD | I4     | I3     | I2 | I1 |    |   |
| I6                | I6     | I5     | MCCNDD | I4     | I3 | I2 | I1 |   |
| I7 <sup>(1)</sup> | I7     | I6     | I5     | MCCNDD | I4 | I3 | I2 |   |
| d1                | d1     | I7     | I6     | I5     | -  | I4 | I3 |   |
| d2                | d2     | d1     | I7     | I6     | I5 | -  | I4 |   |
| d3                | d3     | d2     | d1     | I7     | I6 | I5 | -  |   |
| etc ....          |        | d3     | d2     | d1     | I7 | I6 | I5 |   |
| ....              |        |        | d3     | d2     | d1 | I7 | I6 |   |
| ....              |        |        |        | d3     | d2 | d1 | I7 |   |
| ....              |        |        |        |        | d3 | d2 | d1 |   |
|                   |        |        |        |        |    | d3 | d2 |   |
|                   |        |        |        |        |    |    | d3 |   |

(1) The RPC value in the MSTF register will point to the instruction following I7 (instruction I8).

**See also**

[MBCNDD #16BitDest, CNDF](#)  
[MMOV32 mem32, MSTF](#)  
[MMOV32 MSTF, mem32](#)

**MCCNDD 16BitDest {, CNDF}** (continued)

**Call Conditional Delayed**

---

[MRCNDD CNDF](#)

**MCMP32 MRa, MRb****32-Bit Integer Compare for Equal, Less Than or Greater Than****Operands**

|     |                                                 |
|-----|-------------------------------------------------|
| MRa | CLA floating-point source register (MR0 to MR3) |
| MRb | CLA floating-point source register (MR0 to MR3) |

**Opcode**

```
LSW: 0000 0000 0000 bbaa
MSW: 0111 1111 0010 0000
```

**Description**

Set ZF and NF flags on the result of MRa - MRb where MRa and MRb are 32-bit integers. For a floating point compare refer to [MCMFP32](#).

**Note**

A known hardware issue exists in the MCMP32 instruction. Signed integer comparisons using MCMP32 by itself will set the status bits in a way that is not useful for comparison when the difference between the two operands is too large, such as when the inputs have opposite sign and are near the extreme 32-bit signed values. This affects both signed and unsigned integer comparisons.

The compiler (version 18.1.5.LTS or higher) has implemented a workaround for this issue. The compiler checks the upper bits of the operands by performing a floating point comparison before proceeding to do the integer comparison or subtraction.

The compiler flag `--cla_signed_compare_workaround` enables this workaround.

**Flags**

This instruction modifies the following flags in the MSTF register:

| Flag     | TF | ZF  | NF  | LUF | LVF |
|----------|----|-----|-----|-----|-----|
| Modified | No | Yes | Yes | No  | No  |

The MSTF register flags are modified based on the integer results of the operation.

```
If (MRa == MRb) {ZF=1; NF=0;}
If (MRa > MRb) {ZF=0; NF=0;}
If (MRa < MRb) {ZF=0; NF=1;}
```

**Pipeline**

This is a single-cycle instruction.

**Example**

```
; Behavior of ZF and NF flags for different comparisons
;
; Given A = (int32)1
;       B = (int32)2
;       C = (int32)-7
;
MMOV32 MR0, @_A ; MR0 = 1 (0x00000001)
MMOV32 MR1, @_B ; MR1 = 2 (0x00000002)
MMOV32 MR2, @_C ; MR2 = -7 (0xFFFFFFFF9)
MCMP32 MR2, MR2 ; NF = 0, ZF = 1
MCMP32 MR0, MR1 ; NF = 1, ZF = 0
MCMP32 MR1, MR0 ; NF = 0, ZF = 0
```

**See also**

[MADD32 MRa, MRb, MRc](#)



**MCMP32 MRa, MRb** (continued)

***32-Bit Integer Compare for Equal, Less Than or Greater Than***

---

[MSUB32 MRa, MRb, MRc](#)

**MCMPF32 MRa, MRb****32-Bit Floating-Point Compare for Equal, Less Than or Greater Than****Operands**

|     |                                                 |
|-----|-------------------------------------------------|
| MRa | CLA floating-point source register (MR0 to MR3) |
| MRb | CLA floating-point source register (MR0 to MR3) |

**Opcode**

```
LSW: 0000 0000 0000 bbaa
MSW: 0111 1101 0000 0000
```

**Description**

Set ZF and NF flags on the result of MRa - MRb. The MCMPF32 instruction is performed as a logical compare operation. This is possible because of the IEEE format offsetting the exponent. Basically the bigger the binary number, the bigger the floating-point value.

Special cases for inputs:

- Negative zero will be treated as positive zero.
- A denormalized value will be treated as positive zero.
- Not-a-Number (NaN) will be treated as infinity.

**Flags**

This instruction modifies the following flags in the MSTF register:

| Flag     | TF | ZF  | NF  | LUF | LVF |
|----------|----|-----|-----|-----|-----|
| Modified | No | Yes | Yes | No  | No  |

The MSTF register flags are modified as follows:

```
If(MRa == MRb) {ZF=1; NF=0;}
If(MRa > MRb) {ZF=0; NF=0;}
If(MRa < MRb) {ZF=0; NF=1;}
```

**Pipeline**

This is a single-cycle instruction.

**Example**

```
; Behavior of ZF and NF flags for different comparisons
MMOVIZ MR1, #-2.0 ; MR1 = -2.0 (0xC0000000)
MMOVIZ MR0, #5.0 ; MR0 = 5.0 (0x40A00000)
MCMPF32 MR1, MR0 ; ZF = 0, NF = 1
MCMPF32 MR0, MR1 ; ZF = 0, NF = 0
MCMPF32 MR0, MR0 ; ZF = 1, NF = 0
```

**See also**

[MCMPF32 MRa, #16FHi](#)  
[MMAXF32 MRa, #16FHi](#)  
[MMAXF32 MRa, MRb](#)  
[MMINF32 MRa, #16FHi](#)  
[MMINF32 MRa, MRb](#)

## MCMPF32 MRa, #16FHi

### 32-Bit Floating-Point Compare for Equal, Less Than or Greater Than

#### Operands

|        |                                                                                                                                                             |
|--------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| MRa    | CLA floating-point source register (MR0 to MR3)                                                                                                             |
| #16FHi | A 16-bit immediate value that represents the upper 16-bits of an IEEE 32-bit floating-point value. The low 16-bits of the mantissa are assumed to be all 0. |

#### Opcode

```
LSW: I I I I I I I I I I I I I I I I
MSW: 0111 1000 1100 00aa
```

#### Description

Compare the value in MRa with the floating-point value represented by the immediate operand. Set the ZF and NF flags on (MRa - #16FHi:0).

#16FHi is a 16-bit immediate value that represents the upper 16-bits of an IEEE 32-bit floating-point value. The low 16-bits of the mantissa are assumed to be all 0. This addressing mode is most useful for constants where the lowest 16-bits of the mantissa are 0. Some examples are 2.0 (0x40000000), 4.0 (0x40800000), 0.5 (0x3F000000), and -1.5 (0xBFC00000). The assembler will accept either a hex or float as the immediate value. That is, -1.5 can be represented as #-1.5 or #0xBFC0.

The MCMPF32 instruction is performed as a logical compare operation. This is possible because of the IEEE floating-point format offsets the exponent. Basically the bigger the binary number, the bigger the floating-point value.

Special cases for inputs:

- Negative zero will be treated as positive zero.
- Denormalized value will be treated as positive zero.
- Not-a-Number (NaN) will be treated as infinity.

#### Flags

This instruction modifies the following flags in the MSTF register:

| Flag     | TF | ZF  | NF  | LUF | LVF |
|----------|----|-----|-----|-----|-----|
| Modified | No | Yes | Yes | No  | No  |

The MSTF register flags are modified as follows:

```
If (MRa == #16FHi:0) {ZF=1, NF=0;}
If (MRa > #16FHi:0) {ZF=0, NF=0;}
If (MRa < #16FHi:0) {ZF=0, NF=1;}
```

#### Pipeline

This is a single-cycle instruction

#### Example 1

```
; Behavior of ZF and NF flags for different comparisons
MMOVIZ    MR1, #-2.0 ; MR1 = -2.0 (0xC0000000)
MMOVIZ    MR0, #5.0  ; MR0 = 5.0 (0x40A00000)
MCMPF32   MR1, #-2.2 ; ZF = 0, NF = 0
MCMPF32   MR0, #6.5  ; ZF = 0, NF = 1
MCMPF32   MR0, #5.0  ; ZF = 1, NF = 0
```

## MCMPF32 MRa, #16FHi (continued)

### 32-Bit Floating-Point Compare for Equal, Less Than or Greater Than

#### Example 2

```

; X is an array of 32-bit floating-point values
; and has len elements. Find the maximum value in
; the array and store it in Result
;
; Note: MCMPF32 and MSWAPF can be replaced with MMAXF32
;
_Cla1Task1:
    MMovi16 MAR1, # X          ; Start address
    Mui16TOF32 MR0, @_len      ; Length of the array
    MNOP                      ; delay for MAR1 load
    MNOP                      ; delay for MAR1 load
    MMov32 MR1, *MAR1[2]++     ; MR1 = X0
LOOP
    MMov32 MR2, *MAR1[2]++     ; MR2 = next element
    MCMPF32 MR2, MR1           ; Compare MR2 with MR1
    MSWAPF MR1, MR2, GT        ; MR1 = MAX(MR1, MR2)
    MADDF32 MR0, MR0, #-1.0    ; Decrement the counter
    MCMPF32 MR0, #0.0          ; Set/clear flags for MBCND
    MNOP
    MNOP
    MNOP
    MBCND LOOP, NEQ            ; Branch if not equal to zero
    MMov32 @_Result, MR1       ; Always executed
    MNOP                      ; Always executed
    MNOP                      ; Always executed
    MSTOP                     ; End of task

```

#### See also

[MCMPF32 MRa, MRb](#)  
[MMAXF32 MRa, #16FHi](#)  
[MMAXF32 MRa, MRb](#)  
[MMINF32 MRa, #16FHi](#)  
[MMINF32 MRa, MRb](#)

## MDEBUGSTOP

### Debug Stop Task

#### Operands

|      |                                             |
|------|---------------------------------------------|
| none | This instruction does not have any operands |
|------|---------------------------------------------|

#### Opcode

|      |      |      |      |      |
|------|------|------|------|------|
| LSW: | 0000 | 0000 | 0000 | 0000 |
| MSW: | 0111 | 1111 | 0110 | 0000 |

#### Description

When CLA breakpoints are enabled, the MDEBUGSTOP instruction is used to halt a task so that it can be debugged. That is, MDEBUGSTOP is the CLA breakpoint. If CLA breakpoints are not enabled, the MDEBUGSTOP instruction behaves like a MNOP. Unlike the MSTOP, the MIRUN flag is not cleared and an interrupt is not issued. A single-step or run operation will continue execution of the task.

#### Restrictions

The MDEBUGSTOP instruction cannot be placed 3 instructions before or after a [MBCNDD](#), [MCCNDD](#), or [MRCNDD](#) instruction.

#### Flags

This instruction does not modify flags in the MSTF register.

| Flag     | TF | ZF | NF | LUF | LVF |
|----------|----|----|----|-----|-----|
| Modified | No | No | No | No  | No  |

#### Pipeline

This is a single-cycle instruction.

#### See also

[MSTOP](#)

## MEALLOW

### Enable CLA Write Access to EALLOW Protected Registers

#### Operands

|      |                                             |
|------|---------------------------------------------|
| none | This instruction does not have any operands |
|------|---------------------------------------------|

#### Opcode

|                          |
|--------------------------|
| LSW: 0000 0000 0000 0000 |
| MSW: 0111 1111 1001 0000 |

#### Description

This instruction sets the MEALLOW bit in the CLA status register MSTF. When this bit is set, the CLA is allowed write access to EALLOW protected registers. To again protect against CLA writes to protected registers, use the MEDIS instruction.

MEALLOW and MEDIS only control CLA write access; reads are allowed even if MEALLOW has not been executed. MEALLOW and MEDIS are also independent from the main CPU's EALLOW/EDIS. This instruction does not modify the EALLOW bit in the main CPU's status register. The MEALLOW bit in MSTF only controls access for the CLA while the EALLOW bit in the ST1 register only controls access for the main CPU.

As with EALLOW, the MEALLOW bit is overridden via the JTAG port, allowing full control of register accesses during debug from Code Composer Studio.

#### Flags

This instruction does not modify flags in the MSTF register.

| Flag     | TF | ZF | NF | LUF | LVF |
|----------|----|----|----|-----|-----|
| Modified | No | No | No | No  | No  |

#### Pipeline

This is a single-cycle instruction.

#### Example

```

; C header file including definition of
; the EPwm1Regs structure
;
; The ePWM TZSEL register is EALLOW protected
;
.cdecls C,LIST,"CLAShared.h"
...
_Cla1Task1:
...
MEALLOW                ; Allow CLA write access
MMOV16 @_EPwm1Regs.TZSEL.all, MR3 ; Write to TZSEL
MEDIS                  ; Disallow CLA write access
...
...
MSTOP

```

#### See also

[MEDIS](#)

## MEDIS

### Disable CLA Write Access to EALLOW Protected Registers

#### Operands

|      |                                             |
|------|---------------------------------------------|
| none | This instruction does not have any operands |
|------|---------------------------------------------|

#### Opcode

|                          |
|--------------------------|
| LSW: 0000 0000 0000 0000 |
| MSW: 0111 1111 1011 0000 |

#### Description

This instruction clears the MEALLOW bit in the CLA status register MSTF. When this bit is clear, the CLA is not allowed write access to EALLOW-protected registers. To enable CLA writes to protected registers, use the MEALLOW instruction.

MEALLOW and MEDIS only control CLA write access; reads are allowed even if MEALLOW has not been executed. MEALLOW and MEDIS are also independent from the main CPU's EALLOW/EDIS. This instruction does not modify the EALLOW bit in the main CPU's status register. The MEALLOW bit in MSTF only controls access for the CLA while the EALLOW bit in the ST1 register only controls access for the main CPU.

As with EALLOW, the MEALLOW bit is overridden via the JTAG port, allowing full control of register accesses during debug from the Code Composer Studio™ IDE.

#### Flags

This instruction does not modify flags in the MSTF register.

| Flag     | TF | ZF | NF | LUF | LVF |
|----------|----|----|----|-----|-----|
| Modified | No | No | No | No  | No  |

#### Pipeline

This is a single-cycle instruction.

#### Example

```

; C header file including definition of
; the EPwm1Regs structure
;
; The ePWM TZSEL register is EALLOW protected
;
.cdecls C,LIST,"CLAShared.h"
...
_Cla1Task1:
...
MEALLOW                ; Allow CLA write access
MMOV16 @_EPwm1Regs.TZSEL.all, MR3 ; Write to TZSEL
MEDIS                  ; Disallow CLA write access
...
...
MSTOP

```

#### See also

[MEALLOW](#)

**MEINVF32 MRa, MRb****32-Bit Floating-Point Reciprocal Approximation****Operands**

|     |                                                      |
|-----|------------------------------------------------------|
| MRa | CLA floating-point destination register (MR0 to MR3) |
| MRb | CLA floating-point source register (MR0 to MR3)      |

**Opcode**

```
LSW: 0000 0000 0000 bbaa
MSW: 0111 1111 0000 0000
```

**Description**

This operation generates an estimate of  $1/X$  in 32-bit floating-point format accurate to approximately 8 bits. This value can be used in a Newton-Raphson algorithm to get a more accurate answer. That is:

```
Ye = Estimate(1/X);
Ye = Ye*(2.0 - Ye*X);
Ye = Ye*(2.0 - Ye*X);
```

After two iterations of the Newton-Raphson algorithm, you will get an exact answer accurate to the 32-bit floating-point format. On each iteration the mantissa bit accuracy approximately doubles. The MEINVF32 operation will not generate a negative zero, DeNorm or NaN value.

```
MRa = Estimate of 1/MRb;
```

**Flags**

This instruction modifies the following flags in the MSTF register:

| Flag     | TF | ZF | NF | LUF | LVF |
|----------|----|----|----|-----|-----|
| Modified | No | No | No | Yes | Yes |

The MSTF register flags are modified as follows:

- LUF = 1 if MEINVF32 generates an underflow condition.
- LVF = 1 if MEINVF32 generates an overflow condition.

**Pipeline**

This is a single-cycle instruction.

**Example**

```
; Calculate Num/Den using a Newton-Raphson algorithm for 1/Den
; Ye = Estimate(1/X)
; Ye = Ye*(2.0 - Ye*X)
; Ye = Ye*(2.0 - Ye*X)
;
_Cla1Task1:
  _MMOV32 MR1, @_Den      ; MR1 = Den
  MEINVF32 MR2, MR1       ; MR2 = Ye = Estimate(1/Den)
  MPPYF32 MR3, MR2, MR1   ; MR3 = Ye*Den
  MSUBF32 MR3, #2.0, MR3  ; MR3 = 2.0 - Ye*Den
  MPPYF32 MR2, MR2, MR3   ; MR2 = Ye = Ye*(2.0 - Ye*Den)
  MPPYF32 MR3, MR2, MR1   ; MR3 = Ye*Den
  || _MMOV32 MR0, @_Num    ; MR0 = Num
  MSUBF32 MR3, #2.0, MR3  ; MR3 = 2.0 - Ye*Den
  MPPYF32 MR2, MR2, MR3   ; MR2 = Ye = Ye*(2.0 - Ye*Den)
  || _MMOV32 MR1, @_Den    ; Reload Den To Set Sign
  MNEGF32 MR0, MR0, EQ    ; if(Den == 0.0) Change Sign Of Num
  MPPYF32 MR0, MR0, MR2   ; MR0 = Y = Ye*Num
  _MMOV32 @_Dest, MR0     ; Store result
  MSTOP                  ; end of task
```

**See also**

[MEISQRTF32 MRa, MRb](#)



## MEISQRTF32 MRa, MRb

### 32-Bit Floating-Point Square-Root Reciprocal Approximation

#### Operands

|     |                                                      |
|-----|------------------------------------------------------|
| MRa | CLA floating-point destination register (MR0 to MR3) |
| MRb | CLA floating-point source register (MR0 to MR3)      |

#### Opcode

```
LSW: 0000 0000 0000 bbaa
MSW: 0111 1110 0100 0000
```

#### Description

This operation generates an estimate of  $1/\sqrt{X}$  in 32-bit floating-point format accurate to approximately 8 bits. This value can be used in a Newton-Raphson algorithm to get a more accurate answer. That is:

```
Ye = Estimate(1/sqrt(X));
Ye = Ye*(1.5 - Ye*Ye*X/2.0);
Ye = Ye*(1.5 - Ye*Ye*X/2.0);
```

After 2 iterations of the Newton-Raphson algorithm, you will get an exact answer accurate to the 32-bit floating-point format. On each iteration the mantissa bit accuracy approximately doubles. The MEISQRTF32 operation will not generate a negative zero, DeNorm or NaN value.

```
MRa = Estimate of 1/sqrt (MRb);
```

#### Flags

This instruction modifies the following flags in the MSTF register:

| Flag     | TF | ZF | NF | LUF | LVF |
|----------|----|----|----|-----|-----|
| Modified | No | No | No | Yes | Yes |

The MSTF register flags are modified as follows:

- LUF = 1 if MEISQRTF32 generates an underflow condition.
- LVF = 1 if MEISQRTF32 generates an overflow condition.

#### Pipeline

This is a single-cycle instruction.

#### Example

```
; Y = sqrt(X)
; Ye = Estimate(1/sqrt(X));
; Ye = Ye*(1.5 - Ye*Ye*X*0.5)
; Ye = Ye*(1.5 - Ye*Ye*X*0.5)
; Y = X*Ye
;
_ClalTask3:
    MMOV32 MR0, @_x          ; MR0 = X
    MEISQRTF32 MR1, MR0      ; MR1 = Ye = Estimate(1/sqrt(X))
    MMOV32 MR1, @_x, EQ      ; if(X == 0.0) Ye = 0.0
    MMPYF32 MR3, MR0, #0.5    ; MR3 = X*0.5
    MMPYF32 MR2, MR1, MR3     ; MR2 = Ye*X*0.5
    MMPYF32 MR2, MR1, MR2     ; MR2 = Ye*Ye*X*0.5
    MSUBF32 MR2, #1.5, MR2    ; MR2 = 1.5 - Ye*Ye*X*0.5
    MMPYF32 MR1, MR1, MR2     ; MR1 = Ye = Ye*(1.5 - Ye*Ye*X*0.5)
    MMPYF32 MR2, MR1, MR3     ; MR2 = Ye*X*0.5
    MMPYF32 MR2, MR1, MR2     ; MR2 = Ye*Ye*X*0.5
    MSUBF32 MR2, #1.5, MR2    ; MR2 = 1.5 - Ye*Ye*X*0.5
    MMPYF32 MR1, MR1, MR2     ; MR1 = Ye = Ye*(1.5 - Ye*Ye*X*0.5)
    MMPYF32 MR0, MR1, MR0     ; MR0 = Y = Ye*X
    MMOV32 @_y, MR0          ; Store Y = sqrt(X)
    MSTOP                    ; end of task
```

**MEISQRTF32 MRa, MRb** (continued)

***32-Bit Floating-Point Square-Root Reciprocal Approximation***

---

**See also**

[MEINVF32 MRa, MRb](#)

## MF32TOI16 MRa, MRb

### Convert 32-Bit Floating-Point Value to 16-Bit Integer

#### Operands

|     |                                                      |
|-----|------------------------------------------------------|
| MRa | CLA floating-point destination register (MR0 to MR3) |
| MRb | CLA floating-point source register (MR0 to MR3)      |

#### Opcode

```
LSW: 0000 0000 0000 bbaa
MSW: 0111 1101 1110 0000
```

#### Description

Convert a 32-bit floating point value in MRb to a 16-bit integer and truncate. The result will be stored in MRa.

```
MRa(15:0) = F32TOI16(MRb);
MRa(31:16) = sign extension of MRa(15);
```

#### Flags

This instruction does not affect any flags:

| Flag     | TF | ZF | NF | LUF | LVF |
|----------|----|----|----|-----|-----|
| Modified | No | No | No | No  | No  |

#### Pipeline

This is a single-cycle instruction.

#### Example

```
MMOVIZ    MR0, #5.0 ; MR0      = 5.0 (0x40A00000)
MF32TOI16 MR1, MR0  ; MR1(15:0) = MF32TOI16(MR0) = 0x0005
              ; MR1(31:16) = Sign extension of MR1(15) = 0x0000
MMOVIZ    MR2, #-5.0 ; MR2      = -5.0 (0xC0A00000)
MF32TOI16 MR3, MR2  ; MR3(15:0) = MF32TOI16(MR2) = -5 (0xFFFFB)
              ; MR3(31:16) = Sign extension of MR3(15) = 0xFFFF
```

#### See also

[MF32TOI16R MRa, MRb](#)  
[MF32TOUI16 MRa, MRb](#)  
[MF32TOUI16R MRa, MRb](#)  
[MI16TOF32 MRa, MRb](#)  
[MI16TOF32 MRa, mem16](#)  
[MUI16TOF32 MRa, mem16](#)  
[MUI16TOF32 MRa, MRb](#)

**MF32TOI16R MRa, MRb****Convert 32-Bit Floating-Point Value to 16-Bit Integer and Round****Operands**

|     |                                                      |
|-----|------------------------------------------------------|
| MRa | CLA floating-point destination register (MR0 to MR3) |
| MRb | CLA floating-point source register (MR0 to MR3)      |

**Opcode**

```
LSW: 0000 0000 0000 bbaa
MSW: 0111 1110 0110 0000
```

**Description**

Convert the 32-bit floating point value in MRb to a 16-bit integer and round to the nearest even value. The result is stored in MRa.

```
MRa(15:0) = F32TOI16round(MRb);
MRa(31:16) = sign extension of MRa(15);
```

**Flags**

This instruction does not affect any flags:

| Flag     | TF | ZF | NF | LUF | LVF |
|----------|----|----|----|-----|-----|
| Modified | No | No | No | No  | No  |

**Pipeline**

This is a single-cycle instruction.

**Example**

```
MMOVIZ MR0, #0x3FD9 ; MR0(31:16) = 0x3FD9
MMOVXI MR0, #0x999A ; MR0(15:0) = 0x999A
                    ; MR0 = 1.7 (0x3FD9999A)
MF32TOI16R MR1, MR0 ; MR1(15:0) = MF32TOI16round (MR0) = 2 (0x0002)
                    ; MR1(31:16) = Sign extension of MR1(15) = 0x0000
MMOVF32 MR2, #-1.7 ; MR2 = -1.7 (0xBFD9999A)
MF32TOI16R MR3, MR2 ; MR3(15:0) = MF32TOI16round (MR2) = -2 (0xFFFFE)
                    ; MR3(31:16) = Sign extension of MR2(15) = 0xFFFF
```

**See also**

[MF32TOI16 MRa, MRb](#)  
[MF32TOUI16 MRa, MRb](#)  
[MF32TOUI16R MRa, MRb](#)  
[MI16TOF32 MRa, MRb](#)  
[MI16TOF32 MRa, mem16](#)  
[MUI16TOF32 MRa, mem16](#)  
[MUI16TOF32 MRa, MRb](#)

## MF32TOI32 MRa, MRb

### Convert 32-Bit Floating-Point Value to 32-Bit Integer

#### Operands

|     |                                                      |
|-----|------------------------------------------------------|
| MRa | CLA floating-point destination register (MR0 to MR3) |
| MRb | CLA floating-point source register (MR0 to MR3)      |

#### Opcode

```
LSW: 0000 0000 0000 bbaa
MSW: 0111 1101 0110 0000
```

#### Description

Convert the 32-bit floating-point value in MRb to a 32-bit integer value and truncate. Store the result in MRa.

```
MRa = F32TOI32 (MRb) ;
```

#### Flags

This instruction does not affect any flags:

| Flag     | TF | ZF | NF | LUF | LVF |
|----------|----|----|----|-----|-----|
| Modified | No | No | No | No  | No  |

#### Pipeline

This is a single-cycle instruction.

#### Example 1

```
MMOVF32    MR2, #11204005.0 ; MR2 = 11204005.0 (0x4B2AF5A5)
MF32TOI32  MR3, MR2         ; MR3 = MF32TOI32(MR2) = 11204005 (0x00AAF5A5)
MMOVF32    MR0, #-11204005.0 ; MR0 = -11204005.0 (0xCB2AF5A5)
MF32TOI32  MR1, MR0         ; MR1 = MF32TOI32(MR0) = -11204005 (0xFF550A5B)
```

#### Example 2

```
; Given X, M and B are IQ24 numbers:
; X = IQ24(+2.5) = 0x02800000
; M = IQ24(+1.5) = 0x01800000
; B = IQ24(-0.5) = 0xFF800000
;
; Calculate Y = X * M + B
;
; Convert M, X and B from IQ24 to float
;
_Cla1Task2:
  MI32TOF32 MR0, @_M          ; MR0 = 0x4BC00000
  MI32TOF32 MR1, @_X          ; MR1 = 0x4C200000
  MI32TOF32 MR2, @_B          ; MR2 = 0xCB000000
  MMPYF32   MR0, MR0, #0x3380 ; M = 1/(1*2^24) * iqm = 1.5 (0x3FC00000)
  MMPYF32   MR1, MR1, #0x3380 ; X = 1/(1*2^24) * iqx = 2.5 (0x40200000)
  MMPYF32   MR2, MR2, #0x3380 ; B = 1/(1*2^24) * iqb = -1.5 (0xBF000000)
  MMPYF32   MR3, MR0, MR1      ; M*X
  MADDF32   MR2, MR2, MR3      ; Y=MX+B = 3.25 (0x40500000)
; Convert Y from float32 to IQ24
  MMPYF32   MR2, MR2, #0x4B80 ; Y * 1*2^24
  MF32TOI32 MR2, MR2          ; IQ24(Y) = 0x03400000
  MMOV32    @_Y, MR2          ; store result
  MSTOP                                           ; end of task
```

#### See also

[MF32TOUI32 MRa, MRb](#)  
[MI32TOF32 MRa, MRb](#)  
[MI32TOF32 MRa, mem32](#)  
[MUI32TOF32 MRa, MRb](#)  
[MUI32TOF32 MRa, mem32](#)

## MF32TOUI16 MRa, MRb

### Convert 32-Bit Floating-Point Value to 16-bit Unsigned Integer

#### Operands

|     |                                                      |
|-----|------------------------------------------------------|
| MRa | CLA floating-point destination register (MR0 to MR3) |
| MRb | CLA floating-point source register (MR0 to MR3)      |

#### Opcode

```
LSW: 0000 0000 0000 bbaa
MSW: 0111 1110 1010 0000
```

#### Description

Convert the 32-bit floating point value in MRb to an unsigned 16-bit integer value and truncate to zero. The result will be stored in MRa. To instead round the integer to the nearest even value use the MF32TOUI16R instruction.

```
MRa(15:0) = F32TOUI16(MRb);
MRa(31:16) = 0x0000;
```

#### Flags

This instruction does not affect any flags:

| Flag     | TF | ZF | NF | LUF | LVF |
|----------|----|----|----|-----|-----|
| Modified | No | No | No | No  | No  |

#### Pipeline

This is a single-cycle instruction.

#### Example

```
MMOVIZ      MR0, #9.0      ; MR0 = 9.0 (0x41100000)
MF32TOUI16   MR1, MR0      ; MR1(15:0) = MF32TOUI16(MR0) = 9 (0x0009)
                          ; MR1(31:16) = 0x0000
MMOVIZ      MR2, #-9.0     ; MR2 = -9.0 (0xC1100000)
MF32TOUI16   MR3, MR2      ; MR3(15:0) = MF32TOUI16(MR2) = 0 (0x0000)
                          ; MR3(31:16) = 0x0000
```

#### See also

[MF32TOI16 MRa, MRb](#)  
[MF32TOUI16 MRa, MRb](#)  
[MF32TOUI16R MRa, MRb](#)  
[MI16TOF32 MRa, MRb](#)  
[MI16TOF32 MRa, mem16](#)  
[MUI16TOF32 MRa, mem16](#)  
[MUI16TOF32 MRa, MRb](#)

## MF32TOUI16R MRa, MRb

### Convert 32-Bit Floating-Point Value to 16-bit Unsigned Integer and Round

#### Operands

|     |                                                      |
|-----|------------------------------------------------------|
| MRa | CLA floating-point destination register (MR0 to MR3) |
| MRb | CLA floating-point source register (MR0 to MR3)      |

#### Opcode

```
LSW: 0000 0000 0000 bbaa
MSW: 0111 1110 1100 0000
```

#### Description

Convert the 32-bit floating-point value in MRb to an unsigned 16-bit integer and round to the closest even value. The result will be stored in MRa. To instead truncate the converted value, use the MF32TOUI16 instruction.

```
MRa(15:0) = MF32TOUI16round(MRb);
MRa(31:16) = 0x0000;
```

#### Flags

This instruction does not affect any flags:

| Flag     | TF | ZF | NF | LUF | LVF |
|----------|----|----|----|-----|-----|
| Modified | No | No | No | No  | No  |

#### Pipeline

This is a single-cycle instruction.

#### Example

```
MMOVIZ      MR0, #0x412C      ; MR0 = 0x412C
MMOVXI      MR0, #0xCCCCD     ; MR0 = 0xCCCCD ; MR0 = 10.8 (0x412CCCCD)
MF32TOUI16R  MR1, MR0         ; MR1(15:0) = MF32TOUI16round(MR0) = 11 (0x000B)
                                   ; MR1(31:16) = 0x0000
MMOVF32     MR2, #-10.8       ; MR2 = -10.8 (0x0xC12CCCCD)
MF32TOUI16R  MR3, MR2         ; MR3(15:0) = MF32TOUI16round(MR2) = 0 (0x0000)
                                   ; MR3(31:16) = 0x0000
```

#### See also

[MF32TOI16 MRa, MRb](#)  
[MF32TOI16R MRa, MRb](#)  
[MF32TOUI16 MRa, MRb](#)  
[MI16TOF32 MRa, MRb](#)  
[MI16TOF32 MRa, mem16](#)  
[MUI16TOF32 MRa, mem16](#)  
[MUI16TOF32 MRa, MRb](#)

## MF32TOUI32 MRa, MRb

### Convert 32-Bit Floating-Point Value to 32-Bit Unsigned Integer

#### Operands

|     |                                                      |
|-----|------------------------------------------------------|
| MRa | CLA floating-point destination register (MR0 to MR3) |
| MRb | CLA floating-point source register (MR0 to MR3)      |

#### Opcode

```
LSW: 0000 0000 0000 bbaa
MSW: 0111 1101 1010 0000
```

#### Description

Convert the 32-bit floating-point value in MRb to an unsigned 32-bit integer and store the result in MRa.

```
MRa = F32TOUI32 (MRb);
```

#### Flags

This instruction does not affect any flags:

| Flag     | TF | ZF | NF | LUF | LVF |
|----------|----|----|----|-----|-----|
| Modified | No | No | No | No  | No  |

#### Pipeline

This is a single-cycle instruction.

#### Example

```
MMOVIZ      MR0, #12.5 ; MR0 = 12.5 (0x41480000)
MF32TOUI32  MR0, MR0   ; MR0 = MF32TOUI32 (MR0) = 12 (0x0000000C)
MMOVIZ      MR1, #-6.5 ; MR1 = -6.5 (0xC0D00000)
MF32TOUI32  MR2, MR1   ; MR2 = MF32TOUI32 (MR1) = 0.0 (0x00000000)
```

#### See also

[MF32TOI32 MRa, MRb](#)  
[MI32TOF32 MRa, MRb](#)  
[MI32TOF32 MRa, mem32](#)  
[MUI32TOF32 MRa, MRb](#)  
[MUI32TOF32 MRa, mem32](#)



## MFRACF32 MRa, MRb

### Fractional Portion of a 32-Bit Floating-Point Value

#### Operands

|     |                                                      |
|-----|------------------------------------------------------|
| MRa | CLA floating-point destination register (MR0 to MR3) |
| MRb | CLA floating-point source register (MR0 to MR3)      |

#### Opcode

```
LSW: 0000 0000 0000 bbaa
MSW: 0111 1110 0000 0000
```

#### Description

Returns in MRa the fractional portion of the 32-bit floating-point value in MRb

#### Flags

This instruction does not affect any flags:

| Flag     | TF | ZF | NF | LUF | LVF |
|----------|----|----|----|-----|-----|
| Modified | No | No | No | No  | No  |

#### Pipeline

This is a single-cycle instruction.

#### Example

```
MMOVIZ MR2, #19.625 ; MR2 = 19.625 (0x419D0000)
MFRACF32 MR3, MR2 ; MR3 = MFRACF32(MR2) = 0.625 (0x3F200000) 0)
```

**MI16TOF32 MRa, MRb****Convert 16-Bit Integer to 32-Bit Floating-Point Value****Operands**

|     |                                                      |
|-----|------------------------------------------------------|
| MRa | CLA floating-point destination register (MR0 to MR3) |
| MRb | CLA floating-point source register (MR0 to MR3)      |

**Opcode**

```
LSW: 0000 0000 0000 bbaa
MSW: 0111 1110 1000 0000
```

**Description**

Convert the 16-bit signed integer in MRb to a 32-bit floating point value and store the result in MRa.

```
MRa = MI16TOF32 (MRb);
```

**Flags**

This instruction does not affect any flags:

| Flag     | TF | ZF | NF | LUF | LVF |
|----------|----|----|----|-----|-----|
| Modified | No | No | No | No  | No  |

**Pipeline**

This is a single-cycle instruction.

**Example**

```
MMOVIZ    MR0, #0x0000    ; MR0 (31:16) = 0.0 (0x0000)
MMOVXI    MR0, #0x0004    ; MR0 (15:0) = 4.0 (0x0004)
MI16TOF32 MR1, MR0        ; MR1 = MI16TOF32 (MR0) = 4.0 (0x40800000)
MMOVIZ    MR2, #0x0000    ; MR2 (31:16) = 0.0 (0x0000)
MMOVXI    MR2, #0xFFFC    ; MR2 (15:0) = -4.0 (0xFFFC)
MI16TOF32 MR3, MR2        ; MR3 = MI16TOF32 (MR2) = -4.0 (0xC0800000)
MSTOP
```

**See also**

[MF32TOI16 MRa, MRb](#)  
[MF32TOI16R MRa, MRb](#)  
[MF32TOUI16 MRa, MRb](#)  
[MF32TOUI16R MRa, MRb](#)  
[MI16TOF32 MRa, mem16](#)  
[MUI16TOF32 MRa, mem16](#)  
[MUI16TOF32 MRa, MRb](#)

## MI16TOF32 MRa, mem16

### Convert 16-Bit Integer to 32-Bit Floating-Point Value

#### Operands

|       |                                                      |
|-------|------------------------------------------------------|
| MRa   | CLA floating-point destination register (MR0 to MR3) |
| mem16 | 16-bit source memory location to be converted        |

#### Opcode

```
LSW: mmmm mmmm mmmm mmmm
MSW: 0111 0101 00aa addr
```

#### Description

Convert the 16-bit signed integer indicated by the mem16 pointer to a 32-bit floating-point value and store the result in MRa.

```
MRa = MI16TOF32[mem16];
```

#### Flags

This instruction does not affect any flags:

| Flag     | TF | ZF | NF | LUF | LVF |
|----------|----|----|----|-----|-----|
| Modified | No | No | No | No  | No  |

#### Pipeline

This is a single-cycle instruction:

#### Example

```
; Assume A = 4 (0x0004)
;      B = -4 (0xFFFFC)
MI16TOF32 MR0, @_A ; MR0 = MI16TOF32(A) = 4.0 (0x40800000)
MI16TOF32 MR1, @_B ; MR1 = MI16TOF32(B) = -4.0 (0xC0800000)
```

#### See also

[MF32TOI16 MRa, MRb](#)  
[MF32TOI16R MRa, MRb](#)  
[MF32TOUI16 MRa, MRb](#)  
[MF32TOUI16R MRa, MRb](#)  
[MI16TOF32 MRa, MRb](#)  
[MUI16TOF32 MRa, mem16](#)  
[MUI16TOF32 MRa, MRb](#)

**MI32TOF32 MRa, mem32****Convert 32-Bit Integer to 32-Bit Floating-Point Value****Operands**

|       |                                                      |
|-------|------------------------------------------------------|
| MRa   | CLA floating-point destination register (MR0 to MR3) |
| mem32 | 32-bit memory source for the MMOV32 operation.       |

**Opcode**

```
LSW: rrrrr rrrrr rrrrr rrrrr
MSW: 0111 0100 01aa addr
```

**Description**

Convert the 32-bit signed integer indicated by mem32 to a 32-bit floating point value and store the result in MRa.

```
MRa = MI32TOF32[mem32];
```

**Flags**

This instruction does not affect any flags:

| Flag     | TF | ZF | NF | LUF | LVF |
|----------|----|----|----|-----|-----|
| Modified | No | No | No | No  | No  |

**Pipeline**

This is a single-cycle instruction.

**Example**

```
; Given X, M and B are IQ24 numbers:
; X = IQ24(+2.5) = 0x02800000
; M = IQ24(+1.5) = 0x01800000
; B = IQ24(-0.5) = 0xFF800000
;
; Calculate Y = X * M + B
;
; Convert M, X and B from IQ24 to float
;
_Cla1Task3:
    MI32TOF32 MR0, @_M      ; MR0 = 0x4BC00000
    MI32TOF32 MR1, @_X      ; MR1 = 0x4C200000
    MI32TOF32 MR2, @_B      ; MR2 = 0xCB000000
    MMPYF32 MR0, MR0, #0x3380 ; M = 1/(1*2^24) * iqm = 1.5 (0x3FC00000)
    MMPYF32 MR1, MR1, #0x3380 ; X = 1/(1*2^24) * iqx = 2.5 (0x40200000)
    MMPYF32 MR2, MR2, #0x3380 ; B = 1/(1*2^24) * iqb = -.5 (0xBF000000)
    MMPYF32 MR3, MR0, MR1    ; M*X
    MADDF32 MR2, MR2, MR3    ; Y=MX+B = 3.25 (0x40500000)
; Convert Y from float32 to IQ24
    MMPYF32 MR2, MR2, #0x4B80 ; Y * 1*2^24
    MF32TOI32 MR2, MR2        ; IQ24(Y) = 0x03400000
    MMOV32 @_Y, MR2           ; store result
    MSTOP                    ; end of task
```

**See also**

[MF32TOI32 MRa, MRb](#)  
[MF32TOUI32 MRa, MRb](#)  
[MI32TOF32 MRa, MRb](#)  
[MUI32TOF32 MRa, MRb](#)  
[MUI32TOF32 MRa, mem32](#)

## MI32TOF32 MRa, MRb

### Convert 32-Bit Integer to 32-Bit Floating-Point Value

#### Operands

|     |                                                      |
|-----|------------------------------------------------------|
| MRa | CLA floating-point destination register (MR0 to MR3) |
| MRb | CLA floating-point source register (MR0 to MR3)      |

#### Opcode

```
LSW: 0000 0000 0000 bbaa
MSW: 0111 1101 1000 0000
```

#### Description

Convert the signed 32-bit integer in MRb to a 32-bit floating-point value and store the result in MRa.

```
MRa = MI32TOF32 (MRb);
```

#### Flags

This instruction does not affect any flags:

| Flag     | TF | ZF | NF | LUF | LVF |
|----------|----|----|----|-----|-----|
| Modified | No | No | No | No  | No  |

#### Pipeline

This is a single-cycle instruction.

#### Example

```
MMOVIZ    MR2, #0x1111 ; MR2(31:16) = 4369 (0x1111)
MMOVXI    MR2, #0x1111 ; MR2(15:0) = 4369 (0x1111)
           ; MR2 = +286331153 (0x11111111)
MI32TOF32 MR3, MR2     ; MR3 = MI32TOF32 (MR2) = 286331153.0 (0x4D888888)
```

#### See also

[MF32TOI32 MRa, MRb](#)  
[MF32TOUI32 MRa, MRb](#)  
[MI32TOF32 MRa, mem32](#)  
[MUI32TOF32 MRa, MRb](#)  
[MUI32TOF32 MRa, mem32](#)

## MLSL32 MRa, #SHIFT

### Logical Shift Left

#### Operands

|        |                                                             |
|--------|-------------------------------------------------------------|
| MRa    | CLA floating-point source/destination register (MR0 to MR3) |
| #SHIFT | Number of bits to shift (1 to 32)                           |

#### Opcode

```
LSW: 0000 0000 0shi ftaa
MSW: 0111 1011 1100 0000
```

#### Description

Logical shift left of MRa by the number of bits indicated. The number of bits can be 1 to 32.

```
MARa(31:0) = Logical Shift Left(MARa(31:0) by #SHIFT bits);
```

#### Flags

This instruction modifies the following flags in the MSTF register:

| Flag     | TF | ZF  | NF  | LUF | LVF |
|----------|----|-----|-----|-----|-----|
| Modified | No | Yes | Yes | No  | No  |

The MSTF register flags are modified based on the integer results of the operation.

```
NF = MRa(31);
ZF = 0;
if(MRa(31:0) == 0) { ZF = 1; }
```

#### Pipeline

This is a single-cycle instruction.

#### Example

```
; Given m2 = (int32)32
; x2 = (int32)64
; b2 = (int32)-128
;
; Calculate:
; m2 = m2*2
; x2 = x2*4
; b2 = b2*8
;
_Cla1Task3:
  MMIOV32 MR0, @_m2 ; MR0 = 32 (0x00000020)
  MMIOV32 MR1, @_x2 ; MR1 = 64 (0x00000040)
  MMIOV32 MR2, @_b2 ; MR2 = -128 (0xFFFFF80)
  MLSL32 MR0, #1 ; MR0 = 64 (0x00000040)
  MLSL32 MR1, #2 ; MR1 = 256 (0x00000100)
  MLSL32 MR2, #3 ; MR2 = -1024 (0xFFFFFC00)
  MMIOV32 @_m2, MR0 ; Store results
  MMIOV32 @_x2, MR1
  MMIOV32 @_b2, MR2
  MSTOP ; end of task
```

#### See also

[MADD32 MRa, MRb, MRc](#)  
[MASR32 MRa, #SHIFT](#)  
[MAND32 MRa, MRb, MRc](#)  
[MLSR32 MRa, #SHIFT](#)  
[MOR32 MRa, MRb, MRc](#)  
[MXOR32 MRa, MRb, MRc](#)  
[MSUB32 MRa, MRb, MRc](#)

## MLSR32 MRa, #SHIFT

### Logical Shift Right

#### Operands

|        |                                                             |
|--------|-------------------------------------------------------------|
| MRa    | CLA floating-point source/destination register (MR0 to MR3) |
| #SHIFT | Number of bits to shift (1 to 32)                           |

#### Opcode

```
LSW: 0000 0000 0shi ftaa
MSW: 0111 1011 1000 0000
```

#### Description

Logical shift right of MRa by the number of bits indicated. The number of bits can be 1 to 32. Unlike the arithmetic shift (MASR32), the logical shift does not preserve the number's sign bit. Every bit in the operand is moved the specified number of bit positions, and the vacant bit-positions are filled in with zeros

```
MARa(31:0) = Logical Shift Right(MARa(31:0) by #SHIFT bits);
```

#### Flags

This instruction modifies the following flags in the MSTF register:

| Flag     | TF | ZF  | NF  | LUF | LVF |
|----------|----|-----|-----|-----|-----|
| Modified | No | Yes | Yes | No  | No  |

The MSTF register flags are modified based on the integer results of the operation.

```
NF = MRa(31);
ZF = 0;
if(MRa(31:0) == 0) { ZF = 1; }
```

#### Pipeline

This is a single-cycle instruction.

#### Example

```
; Illustrate the difference between MASR32 and MLSR32
MMOVIZ MR0, #0xAAAA ; MR0 = 0xAAAA5555
MMOVXI MR0, #0x5555
MMOV32 MR1, MR0 ; MR1 = 0xAAAA5555
MMOV32 MR2, MR0 ; MR2 = 0xAAAA5555
MASR32 MR1, #1 ; MR1 = 0xD5552AAA
MLSR32 MR2, #1 ; MR2 = 0x55552AAA
MASR32 MR1, #1 ; MR1 = 0xEAAA9555
MLSR32 MR2, #1 ; MR2 = 0x2AAA9555
MASR32 MR1, #6 ; MR1 = 0xFFAAA555
MLSR32 MR2, #6 ; MR2 = 0x00AAA555
```

#### See also

[MADD32 MRa, MRb, MRc](#)  
[MASR32 MRa, #SHIFT](#)  
[MAND32 MRa, MRb, MRc](#)  
[MLSL32 MRa, #SHIFT](#)  
[MOR32 MRa, MRb, MRc](#)  
[MXOR32 MRa, MRb, MRc](#)  
[MSUB32 MRa, MRb, MRc](#)

**MMACF32 MR3, MR2, MRd, MRe, MRf ||MMOV32 MRa, mem32**
**32-Bit Floating-Point Multiply and Accumulate with Parallel Move**
**Operands**

|       |                                                                                                                                  |
|-------|----------------------------------------------------------------------------------------------------------------------------------|
| MR3   | floating-point destination/source register MR3 for the add operation                                                             |
| MR2   | CLA floating-point source register MR2 for the add operation                                                                     |
| MRd   | CLA floating-point destination register (MR0 to MR3) for the multiply operation<br>MRd cannot be the same register as MRa        |
| MRe   | CLA floating-point source register (MR0 to MR3) for the multiply operation                                                       |
| MRf   | CLA floating-point source register (MR0 to MR3) for the multiply operation                                                       |
| MRa   | CLA floating-point destination register for the MMOV32 operation (MR0 to MR3).<br>MRa cannot be MR3 or the same register as MRd. |
| mem32 | 32-bit source for the MMOV32 operation                                                                                           |

**Opcode**

```
LSW: mmmm mmmm mmmm mmmm
MSW: 0011 ffee ddaa addr
```

**Description**

Multiply and accumulate the contents of floating-point registers and move from register to memory. The destination register for the MMOV32 cannot be the same as the destination registers for the MMACF32.

```
MR3 = MR3 + MR2;
MRd = MRe * MRf;
MRa = [mem32];
```

**Restrictions**

The destination registers for the MMACF32 and the MMOV32 must be unique. That is, MRa cannot be MR3 and MRa cannot be the same register as MRd.

**Flags**

This instruction modifies the following flags in the MSTF register:

| Flag     | TF | ZF  | NF  | LUF | LVF |
|----------|----|-----|-----|-----|-----|
| Modified | No | Yes | Yes | Yes | Yes |

The MSTF register flags are modified as follows:

- LUF = 1 if MMACF32 (add or multiply) generates an underflow condition.
- LVF = 1 if MMACF32 (add or multiply) generates an overflow condition.

MMOV32 sets the NF and ZF flags as follows:

```
NF = MRa(31);
ZF = 0;
if(MRa(30:23) == 0) { ZF = 1; NF = 0; }
```

**Pipeline**

MMACF32 and MMOV32 complete in a single cycle.



**MMACF32 MR3, MR2, MRd, MRe, MRf ||MMOV32 MRa, mem32 (continued)**
**32-Bit Floating-Point Multiply and Accumulate with Parallel Move**
**Example 1**

```
; Perform 5 multiply and accumulate operations:
;
; X and Y are 32-bit floating point arrays
;
; 1st multiply: A = X0 * Y0
; 2nd multiply: B = X1 * Y1
; 3rd multiply: C = X2 * Y2
; 4th multiply: D = X3 * Y3
; 5th multiply: E = X3 * Y3
;
; Result = A + B + C + D + E
;
_Cla1Task1:
    MMOV32 MAR0, #_X          ; MAR0 points to X array
    MMOV32 MAR1, #_Y          ; MAR1 points to Y array
    MNOP                      ; Delay for MAR0, MAR1 load
    MNOP                      ; Delay for MAR0, MAR1 load
    ; <-- MAR0 valid
    MMOV32 MR0, *MAR0[2]++    ; MR0 = X0, MAR0 += 2
    ; <-- MAR1 valid
    MMOV32 MR1, *MAR1[2]++    ; MR1 = Y0, MAR1 += 2
    MMPYF32 MR2, MR0, MR1     ; MR2 = A = X0 * Y0
    || MMOV32 MR0, *MAR0[2]++  ; In parallel MR0 = X1, MAR0 += 2
    MMOV32 MR1, *MAR1[2]++    ; MR1 = Y1, MAR1 += 2
    MMPYF32 MR3, MR0, MR1     ; MR3 = B = X1 * Y1
    || MMOV32 MR0, *MAR0[2]++  ; In parallel MR0 = X2, MAR0 += 2
    MMOV32 MR1, *MAR1[2]++    ; MR1 = Y2, MAR2 += 2
    MMACF32 MR3, MR2, MR2, MR0, MR1 ; MR3 = A + B, MR2 = C = X2 * Y2
    || MMOV32 MR0, *MAR0[2]++  ; In parallel MR0 = X3
    MMOV32 MR1, *MAR1[2]++    ; MR1 = Y3 M
    MACF32 MR3, MR2, MR2, MR0, MR1 ; MR3 = (A + B) + C, MR2 = D = X3 * Y3
    || MMOV32 MR0, *MAR0      ; In parallel MR0 = X4
    MMOV32 MR1, *MAR1         ; MR1 = Y4
    MMPYF32 MR2, MR0, MR1     ; MR2 = E = X4 * Y4
    || MADDF32 MR3, MR3, MR2   ; in parallel MR3 = (A + B + C) + D
    MADDF32 MR3, MR3, MR2     ; MR3 = (A + B + C + D) + E
    MMOV32 @_Result, MR3      ; Store the result
    MSTOP                    ; end of task
```

**Example 2**

```
; sum = X0*B0 + X1*B1 + X2*B2 + Y1*A1 + Y2*B2
;
; X2 = X1
; X1 = X0
; Y2 = Y1 ; Y1 = sum
;
_ClaTask2:
    MMOV32 MR0, @_B2          ; MR0 = B2
    MMOV32 MR1, @_X2          ; MR1 = X2
    MMPYF32 MR2, MR1, MR0     ; MR2 = X2*B2
    || MMOV32 MR0, @_B1        ; MR0 = B1
    MMOV32 MR1, @_X1          ; MR1 = X1, X2 = X1
    MMPYF32 MR3, MR1, MR0     ; MR3 = X1*B1
    || MMOV32 MR0, @_B0        ; MR0 = B0
    MMOV32 MR1, @_X0          ; MR1 = X0, X1 = X0
    ; MR3 = X1*B1 + X2*B2, MR2 = X0*B0
    ; MR0 = A2
    MMACF32 MR3, MR2, MR2, MR1, MR0
    || MMOV32 MR0, @_A2 M
    ; MR1 = Y2
    MOV32 MR1, @_Y2           ; MR1 = Y2
    ; MR3 = X0*B0 + X1*B1 + X2*B2, MR2 = Y2*A2
    ; MR0 = A1
    MMACF32 MR3, MR2, MR2, MR1, MR0
    || MMOV32 MR0, @_A1
    MMOV32 MR1, @_Y1          ; MR1 = Y1, Y2 = Y1
    MADDF32 MR3, MR3, MR2     ; MR3 = Y2*A2 + X0*B0 + X1*B1 + X2*B2
    || MMPYF32 MR2, MR1, MR0   ; MR2 = Y1*A1
    MADDF32 MR3, MR3, MR2     ; MR3 = Y1*A1 + Y2*A2 + X0*B0 + X1*B1 + X2*B2
```

**MMACF32 MR3, MR2, MRd, MRe, MRf || MMOV32 MRa, mem32** (continued)**32-Bit Floating-Point Multiply and Accumulate with Parallel Move**

```
MMOV32 @_Y1, MR3      ; Y1 = MR3  
MSTOP                  ; end of task
```

**See also**[MMPYF32 MRa, MRb, MRc || MADDF32 MRd, MRe, MRf](#)

## MMAXF32 MRa, MRb

### 32-Bit Floating-Point Maximum

#### Operands

|     |                                                             |
|-----|-------------------------------------------------------------|
| MRa | CLA floating-point source/destination register (MR0 to MR3) |
| MRb | CLA floating-point source register (MR0 to MR3)             |

#### Opcode

```
LSW: 0000 0000 0000 bbaa
MSW: 0111 1101 0010 0000
```

#### Description

```
if (MRa < MRb) MRa = MRb;
```

Special cases for the output from the MMAXF32 operation:

- NaN output will be converted to infinity
- A denormalized output will be converted to positive zero.

#### Flags

This instruction modifies the following flags in the MSTF register:

| Flag     | TF | ZF  | NF  | LUF | LVF |
|----------|----|-----|-----|-----|-----|
| Modified | No | Yes | Yes | No  | No  |

The ZF and NF flags are configured on the result of the operation, not the result stored in the destination register.

```
if (MRa == MRb) {ZF=1; NF=0;}
if (MRa > MRb) {ZF=0; NF=0;}
if (MRa < MRb) {ZF=0; NF=1;}
```

#### Pipeline

This is a single-cycle instruction.

#### Example 1

```
MMOVIZ MR0, #5.0 ; MR0 = 5.0 (0x40A00000)
MMOVIZ MR1, #-2.0 ; MR1 = -2.0 (0xC0000000)
MMOVIZ MR2, #-1.5 ; MR2 = -1.5 (0xBF000000)
MMAXF32 MR2, MR1 ; MR2 = -1.5, ZF = NF = 0
MMAXF32 MR1, MR2 ; MR1 = -1.5, ZF = 0, NF = 1
MMAXF32 MR2, MR0 ; MR2 = 5.0, ZF = 0, NF = 1
MAXF32 MR0, MR2 ; MR2 = 5.0, ZF = 1, NF = 0
```

**MMAXF32 MRa, MRb** (continued)

**32-Bit Floating-Point Maximum**
**Example 2**

```

; X is an array of 32-bit floating-point values
; Find the maximum value in an array X
; and store it in Result
;
_Cla1Task1:
  MMIOV16    MAR1, #_X          ; Start address
  MUI16TOF32 MR0, @_len         ; Length of the array
  MNOP                          ; delay for MAR1 load
  MNOP                          ; delay for MAR1 load
  MMOV32     MR1, *MAR1[2]++     ; MR1 = X0
LOOP
  MMOV32     MR2, *MAR1[2]++     ; MR2 = next element
  MMAXF32    MR1, MR2           ; MR1 = MAX(MR1, MR2)
  MADDF32    MR0, MR0, #-1.0     ; Decrement the counter
  MCMPPF32   MR0, #0.0          ; Set/clear flags for MBCNDD
  MNOP
  MNOP
  MNOP
  MBCNDD     LOOP, NEQ          ; Branch if not equal to zero
  MMOV32     @_Result, MR1      ; Always executed
  MNOP                          ; Always executed
  MNOP                          ; Always executed
  MSTOP
  ; End of task

```

**See also**

[MCMPPF32 MRa, MRb](#)  
[MCMPPF32 MRa, #16FHi](#)  
[MMAXF32 MRa, #16FHi](#)  
[MMINF32 MRa, MRb](#)  
[MMINF32 MRa, #16FHi](#)

## MMAXF32 MRa, #16FHi

### 32-Bit Floating-Point Maximum

#### Operands

|        |                                                                                                                                                             |
|--------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| MRa    | CLA floating-point source/destination register (MR0 to MR3)                                                                                                 |
| #16FHi | A 16-bit immediate value that represents the upper 16-bits of an IEEE 32-bit floating-point value. The low 16-bits of the mantissa are assumed to be all 0. |

#### Opcode

```
LSW: I I I I I I I I I I I I I I I I
MSW: 0111 1001 0000 00aa
```

#### Description

Compare MRa with the floating-point value represented by the immediate operand. If the immediate value is larger, then load it into MRa.

```
if(MRa < #16FHi:0) MRa = #16FHi:0;
```

#16FHi is a 16-bit immediate value that represents the upper 16-bits of an IEEE 32-bit floating-point value. The low 16-bits of the mantissa are assumed to be all 0. This addressing mode is most useful for constants where the lowest 16-bits of the mantissa are 0. Some examples are 2.0 (0x40000000), 4.0 (0x40800000), 0.5 (0x3F000000), and -1.5 (0xBFC00000). The assembler will accept either a hex or float as the immediate value. That is, -1.5 can be represented as #-1.5 or #0xBFC0.

Special cases for the output from the MMAXF32 operation:

- NaN output will be converted to infinity
- A denormalized output will be converted to positive zero.

#### Flags

This instruction modifies the following flags in the MSTF register:

| Flag     | TF | ZF  | NF  | LUF | LVF |
|----------|----|-----|-----|-----|-----|
| Modified | No | Yes | Yes | No  | No  |

The ZF and NF flags are configured on the result of the operation, not the result stored in the destination register.

```
if(MRa == #16FHi:0) {ZF=1; NF=0;}
if(MRa > #16FHi:0) {ZF=0; NF=0;}
if(MRa < #16FHi:0) {ZF=0; NF=1;}
```

#### Pipeline

This is a single-cycle instruction.

#### Example

```
MMOVIZ MR0, #5.0 ; MR0 = 5.0 (0x40A00000)
MMOVIZ MR1, #4.0 ; MR1 = 4.0 (0x40800000)
MMOVIZ MR2, #-1.5 ; MR2 = -1.5 (0xBFC00000)
MMAXF32 MR0, #5.5 ; MR0 = 5.5, ZF = 0, NF = 1
MMAXF32 MR1, #2.5 ; MR1 = 4.0, ZF = 0, NF = 0
MMAXF32 MR2, #-1.0 ; MR2 = -1.0, ZF = 0, NF = 1
MMAXF32 MR2, #-1.0 ; MR2 = -1.5, ZF = 1, NF = 0
```

#### See also

[MMAXF32 MRa, MRb](#)  
[MMINF32 MRa, MRb](#)  
[MMINF32 MRa, #16FHi](#)

## MMINF32 MRa, MRb

### 32-Bit Floating-Point Minimum

#### Operands

|     |                                                             |
|-----|-------------------------------------------------------------|
| MRa | CLA floating-point source/destination register (MR0 to MR3) |
| MRb | CLA floating-point source register (MR0 to MR3)             |

#### Opcode

```
LSW: 0000 0000 0000 bbaa
MSW: 0111 1101 0100 0000
```

#### Description

```
if (MRa > MRb) MRa = MRb;
```

Special cases for the output from the MMINF32 operation:

- NaN output will be converted to infinity
- A denormalized output will be converted to positive zero.

#### Flags

This instruction modifies the following flags in the MSTF register:

| Flag     | TF | ZF  | NF  | LUF | LVF |
|----------|----|-----|-----|-----|-----|
| Modified | No | Yes | Yes | No  | No  |

The ZF and NF flags are configured on the result of the operation, not the result stored in the destination register.

```
if (MRa == MRb) {ZF=1; NF=0;}
if (MRa > MRb) {ZF=0; NF=0;}
if (MRa < MRb) {ZF=0; NF=1;}
```

#### Pipeline

This is a single-cycle instruction.

#### Example 1

```
MMOVIZ MR0, #5.0 ; MR0 = 5.0 (0x40A00000)
MMOVIZ MR1, #4.0 ; MR1 = 4.0 (0x40800000)
MMOVIZ MR2, #-1.5 ; MR2 = -1.5 (0xBFC00000)
MMINF32 MR0, MR1 ; MR0 = 4.0, ZF = 0, NF = 0
MMINF32 MR1, MR2 ; MR1 = -1.5, ZF = 0, NF = 0
MMINF32 MR2, MR1 ; MR2 = -1.5, ZF = 1, NF = 0
MMINF32 MR1, MR0 ; MR2 = -1.5, ZF = 0, NF = 1
```

## MMINF32 MRa, MRb (continued)

### 32-Bit Floating-Point Minimum

#### Example 2

```

;
; X is an array of 32-bit floating-point values
; Find the minimum value in an array X
; and store it in Result
;
;_ClalTask1:
MMOVI16    MAR1,#_X           ; Start address
MUI16TOF32 MR0,@_len         ; Length of the array
MNOP                               ; delay for MAR1 load
MNOP                               ; delay for MAR1 load
MMOV32     MR1,*MAR1[2]++     ; MR1 = X0
LOOP
MMOV32     MR2,*MAR1[2]++     ; MR2 = next element
MMINF32    MR1,MR2            ; MR1 = MAX(MR1, MR2)
MADDF32    MR0,MR0,#-1.0      ; Decrement the counter
MCMPPF32   MR0,#0.0           ; Set/clear flags for MBCNDD
MNOP
MNOP
MNOP
MBCNDD     LOOP,NEQ           ; Branch if not equal to zero
MMOV32     @_Result,MR1       ; Always executed
MNOP                               ; Always executed
MNOP                               ; Always executed
MSTOP
; End of task

```

#### See also

[MMAXF32 MRa, MRb](#)  
[MMAXF32 MRa, #16FHi](#)  
[MMINF32 MRa, #16FHi](#)

**MMINF32 MRa, #16FHi****32-Bit Floating-Point Minimum****Operands**

|        |                                                                                                                                                             |
|--------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| MRa    | floating-point source/destination register (MR0 to MR3)                                                                                                     |
| #16FHi | A 16-bit immediate value that represents the upper 16-bits of an IEEE 32-bit floating-point value. The low 16-bits of the mantissa are assumed to be all 0. |

**Opcode**

```
LSW: I I I I I I I I I I I I I I I I
MSW: 0111 1001 0100 00aa
```

**Description**

Compare MRa with the floating-point value represented by the immediate operand. If the immediate value is smaller, then load it into MRa.

```
if(MRa > #16FHi:0) MRa = #16FHi:0;
```

#16FHi is a 16-bit immediate value that represents the upper 16-bits of an IEEE 32-bit floating-point value. The low 16-bits of the mantissa are assumed to be all 0. This addressing mode is most useful for constants where the lowest 16-bits of the mantissa are 0. Some examples are 2.0 (0x40000000), 4.0 (0x40800000), 0.5 (0x3F000000), and -1.5 (0xBFC00000). The assembler will accept either a hex or float as the immediate value. That is, -1.5 can be represented as #-1.5 or #0xBFC0.

Special cases for the output from the MMINF32 operation:

- NaN output will be converted to infinity
- A denormalized output will be converted to positive zero.

**Flags**

This instruction modifies the following flags in the MSTF register:

| Flag     | TF | ZF  | NF  | LUF | LVF |
|----------|----|-----|-----|-----|-----|
| Modified | No | Yes | Yes | No  | No  |

The ZF and NF flags are configured on the result of the operation, not the result stored in the destination register.

```
if(MRa == #16FHi:0) {ZF=1; NF=0;}
if(MRa > #16FHi:0) {ZF=0; NF=0;}
if(MRa < #16FHi:0) {ZF=0; NF=1;}
```

**Pipeline**

This is a single-cycle instruction.

**Example**

```
MMOVIZ MR0, #5.0 ; MR0 = 5.0 (0x40A00000)
MMOVIZ MR1, #4.0 ; MR1 = 4.0 (0x40800000)
MMOVIZ MR2, #-1.5 ; MR2 = -1.5 (0xBFC00000)
MMINF32 MR0, #5.5 ; MR0 = 5.0, ZF = 0, NF = 1
MMINF32 MR1, #2.5 ; MR1 = 2.5, ZF = 0, NF = 0
MMINF32 MR2, #-1.0 ; MR2 = -1.5, ZF = 0, NF = 1
MMINF32 MR2, #-1.5 ; MR2 = -1.5, ZF = 1, NF = 0
```

**See also**

[MMAXF32 MRa, #16FHi](#)  
[MMAXF32 MRa, MRb](#)  
[MMINF32 MRa, MRb](#)



## MMOV16 MARx, MRa, #16I

### Load the Auxiliary Register with MRa + 16-bit Immediate Value

#### Operands

|      |                                          |
|------|------------------------------------------|
| MARx | Auxiliary register MAR0 or MAR1          |
| MRa  | CLA Floating-point register (MR0 to MR3) |
| #16I | 16-bit immediate value                   |

#### Opcode

```
LSW: I I I I I I I I I I I I I I (opcode of MMOV16 MAR0, MRa, #16I)
MSW: 0111 1111 1101 00AA
LSW: I I I I I I I I I I I I I I (opcode of MMOV16 MAR1, MRa, #16I)
MSW: 0111 1111 1111 00AA
```

#### Description

Load the auxiliary register, MAR0 or MAR1, with MRa(15:0) + 16-bit immediate value. Refer to the pipeline section for important information regarding this instruction.

```
MARx = MRa(15:0) + #16I;
```

#### Flags

This instruction does not modify flags in the MSTF register:

| Flag     | TF | ZF | NF | LUF | LVF |
|----------|----|----|----|-----|-----|
| Modified | No | No | No | No  | No  |

#### Pipeline

This is a single-cycle instruction. The load of MAR0 or MAR1 will occur in the EXE phase of the pipeline. Any post increment of MAR0 or MAR1 using indirect addressing will occur in the D2 phase of the pipeline. Therefore the following applies when loading the auxiliary registers:

- I1 and I2**

The two instructions following MMOV16 will use MAR0/MAR1 before the update occurs. Thus these two instructions will use the old value of MAR0 or MAR1.

- I3**

Loading of an auxiliary register occurs in the EXE phase while updates due to post-increment addressing occur in the D2 phase. Thus I3 cannot use the auxiliary register or there will be a conflict. In the case of a conflict, the update due to address-mode post increment will win and the auxiliary register will not be updated with #\_X.

- I4**

Starting with the 4th instruction MAR0 or MAR1 will be the new value loaded with MMOV16.

```
; Assume MAR0 is 50, MR0 is 10, and #_X is 20
MMOV16 MAR0, MR0, #_X ; Load MAR0 with address of X (20) + MR0 (10)
<Instruction 1> ; I1 Will use the old value of MAR0 (50)
<Instruction 2> ; I2 Will use the old value of MAR0 (50)
<Instruction 3> ; I3 Cannot use MAR0
<Instruction 4> ; I4 Will use the new value of MAR0 (30)
<Instruction 5> ; I5
```

**MMOV16 MARx, MRa, #16I** (continued)**Load the Auxiliary Register with MRa + 16-bit Immediate Value****Table 10-14. Pipeline Activity For MMOV16 MARx, MRa , #16I**

| Instruction              | F1     | F2     | D1     | D2     | R1     | R2     | E      | W |
|--------------------------|--------|--------|--------|--------|--------|--------|--------|---|
| MMOV16 MAR0, MR0,<br>#_X | MMOV16 |        |        |        |        |        |        |   |
| I1                       | I1     | MMOV16 |        |        |        |        |        |   |
| I2                       | I2     | I1     | MMOV16 |        |        |        |        |   |
| I3                       | I3     | I2     | I1     | MMOV16 |        |        |        |   |
| I4                       | I4     | I3     | I2     | I1     | MMOV16 |        |        |   |
| I5                       | I5     | I4     | I3     | I2     | I1     | MMOV16 |        |   |
| I6                       | I6     | I5     | I4     | I3     | I2     | I1     | MMOV16 |   |

**Example 1**

```

; Calculate an offset into a sin/cos table
;
_ClalTask1:
    MMOV32 MR0,@_rad           ; MR0 = rad
    MMOV32 MR1,@_TABLE_SIZEDivTwoPi ; MR1 = TABLE_SIZE/(2*Pi)
    MPPYF32 MR1,MR0,MR1       ; MR1 = rad* TABLE_SIZE/(2*Pi)
|| MMOV32 MR2,@_TABLE_MASK    ; MR2 = TABLE_MASK
    MF32TOI32 MR3,MR1         ; MR3 = K=int(rad*TABLE_SIZE/(2*Pi))
    MAND32 MR3,MR3,MR2        ; MR3 = K & TABLE_MASK
    MSL32 MR3,#1              ; MR3 = K * 2
    MMOV16 MAR0,MR3,#_Cos0    ; MAR0 K*2+addr of table.Cos0
    MFRACF32 MR1,MR1          ; I1
    MMOV32 MR0,@_TwoPiDivTABLE_SIZE ; I2
    MPPYF32 MR1,MR1,MR0       ; I3
|| MMOV32 MR0,@_Coef3
    MMOV32 MR2,*MAR0[#-64]++   ; MR2 = *MAR0, MAR0 += (-64)
    ...
    ...
    MSTOP ; end of task

```

**MMOV16 MARx, MRa, #16I (continued)**
**Load the Auxiliary Register with MRa + 16-bit Immediate Value**
**Example 2**

```

; This task logs the last NUM_DATA_POINTS
; ADCRESULT1 values in the array VoltageCLA
;
; When the last element in the array has been
; filled, the task will go back to the
; the first element.
;
; Before starting the ADC conversions, force
; Task 8 to initialize the ConversionCount to zero
;
; The ADC is set to sample (acquire) for 15 SYSCLK cycles
; or 75ns. After the capacitor has captured the analog
; value, the ADC will trigger this task early.
; It takes 10.5 ADCCLKs to complete a conversion,
; the ADCCLK being SYSCLK/4
;   T_sys = 1/200MHz = 5ns
;   T_adc = 4*T_sys = 20ns
; The ADC will take 10.5 * 4 or 42 SYSCLK cycles to complete
; a conversion. The ADC result register may be read on the
; 36th instruction after the task begins.
;
_Cla1Task2:
    .asg          0, N
    .loop
    MNOP
result
    .eval         N + 1, N
    .break        N = 28
    .endloop
    MMOVZ16      MR0, @_ConversionCount           ;I29 Current Conversion
    MMOV16       MAR1, MR0, #_VoltageCLA          ;I30 Next array location
    MUI16TOF32   MR0, MR0                        ;I31 Convert count to float32
    MADDF32      MR0, MR0, #1.0                  ;I32 Add 1 to conversion count
    MCMPPF32     MR0, #NUM_DATA_POINTS.0         ;I33 Compare count to max
    MF32TOUI16   MR0, MR0                        ;I34 Convert count to Uint16
    MNOP
result
    MMOVZ16      MR2, @_AdcaResultRegs.ADCRESULT1 ;I36 Read ADCRESULT1
    MMOV16       *MAR1, MR2                      ; Store ADCRESULT1
    MBCNDD       _RestartCount, GEQ              ; If count >= NUM_DATA_POINTS
    MMOVIZ       MR1, #0.0                       ; Always executed: MR1=0
    MNOP
    MNOP
    MMOV16       @_ConversionCount, MR0          ; If branch not taken
    MSTOP
result
    MMOV16       @_ConversionCount, MR1          ; If branch taken, restart
count
    MSTOP
; This task initializes the ConversionCount
; to zero
;
_Cla1Task8:
    MMOVIZ       MR0, #0.0
    MMOV16       @_ConversionCount, MR0
    MSTOP
_ClaT8End:

```

## MMOV16 MARx, mem16

### Load MAR1 with 16-bit Value

#### Operands

|       |                                                                                |
|-------|--------------------------------------------------------------------------------|
| MARx  | CLA auxiliary register MAR0 or MAR1                                            |
| mem16 | 16-bit destination memory accessed using one of the available addressing modes |

#### Opcode

```
LSW: mmmmm mmmmm mmmmm mmmmm (Opcode for MMOV16 MAR0, mem16)
MSW: 0111 0110 0000 addr
LSW: mmmmm mmmmm mmmmm mmmmm (Opcode for MMOV16 MAR1, mem16)
MSW: 0111 0110 0100 addr
```

#### Description

Load MAR0 or MAR1 with the 16-bit value pointed to by mem16. Refer to the pipeline section for important information regarding this instruction.

```
MAR1 = [mem16];
```

#### Flags

No flags MSTF flags are affected.

| Flag     | TF | ZF | NF | LUF | LVF |
|----------|----|----|----|-----|-----|
| Modified | No | No | No | No  | No  |

#### Pipeline

This is a single-cycle instruction. The load of MAR0 or MAR1 will occur in the EXE phase of the pipeline. Any post increment of MAR0 or MAR1 using indirect addressing will occur in the D2 phase of the pipeline. Therefore the following applies when loading the auxiliary registers:

- I1 and I2**

The two instructions following MMOV16 will use MAR0/MAR1 before the update occurs. Thus these two instructions will use the old value of MAR0 or MAR1.

- I3**

Loading of an auxiliary register occurs in the EXE phase while updates due to post-increment addressing occur in the D2 phase. Thus I3 cannot use the auxiliary register or there will be a conflict. In the case of a conflict, the update due to address-mode post increment will win send the auxiliary register will not be updated with #\_X.

- I4**

Starting with the 4th instruction MAR0 or MAR1 will be the new value loaded with MMOV16.

```
; Assume MAR0 is 50 and @_X is 20
MMOV16 MAR0, @_X ; Load MAR0 with the contents of X (20)
<Instruction 1> ; I1 Will use the old value of MAR0 (50)
<Instruction 2> ; I2 Will use the old value of MAR0 (50)
<Instruction 3> ; I3 Cannot use MAR0
<Instruction 4> ; I4 Will use the new value of MAR0 (20)
<Instruction 5> ; I5
....
```

**MMOV16 MARx, mem16** (continued)

**Load MAR1 with 16-bit Value**
**Table 10-15. Pipeline Activity For MMOV16 MAR0/MAR1, mem16**

| Instruction      | F1     | F2     | D1     | D2     | R1     | R2     | E      | W |
|------------------|--------|--------|--------|--------|--------|--------|--------|---|
| MMOV16 MAR0, @_X | MMOV16 |        |        |        |        |        |        |   |
| I1               | I1     | MMOV16 |        |        |        |        |        |   |
| I2               | I2     | I1     | MMOV16 |        |        |        |        |   |
| I3               | I3     | I2     | I1     | MMOV16 |        |        |        |   |
| I4               | I4     | I3     | I2     | I1     | MMOV16 |        |        |   |
| I5               | I5     | I4     | I3     | I2     | I1     | MMOV16 |        |   |
| I6               | I6     | I5     | I4     | I3     | I2     | I1     | MMOV16 |   |

**Example**

```

; This task logs the last NUM_DATA_POINTS
; ADCRESULT1 values in the array VoltageCLA
;
; When the last element in the array has been
; filled, the task will go back to the
; the first element.
;
; Before starting the ADC conversions, force
; Task 8 to initialize the ConversionCount to zero
;
; The ADC is set to sample (acquire) for 15 SYSCLK cycles
; or 75ns. After the capacitor has captured the analog
; value, the ADC will trigger this task early.
; It takes 10.5 ADCCLKs to complete a conversion,
; the ADCCLK being SYSCLK/4
; T_sys = 1/200MHz = 5ns
; T_adc = 4*T_sys = 20ns
; The ADC will take 10.5 * 4 or 42 SYSCLK cycles to complete
; a conversion. The ADC result register may be read on the
; 36th instruction after the task begins.
;
_ClalTask2:
    .asg      0, N
    .loop
    MNOP
;I1 - I28 Wait till I36 to read result
    .eval      N + 1, N
    .break      N = 28
    .endloop
    MMOVZ16    MR0, @_ConversionCount           ;I29 Current Conversion
    MMOV16     MAR1, MR0, #_VoltageCLA          ;I30 Next array location
    MUI16TOF32 MR0, MR0                         ;I31 Convert count to float32
    MADDF32    MR0, MR0, #1.0                   ;I32 Add 1 to conversion count
    MCMPPF32   MR0, #NUM_DATA_POINTS.0         ;I33 Compare count to max
    MF32TOUI16 MR0, MR0                         ;I34 Convert count to Uint16
    MNOP       ;I35 Wait till I36 to read
result
    MMOVZ16    MR2, @_AdcaResultRegs.ADCRESULT1 ;I36 Read ADCRESULT1
    MMOV16     *MAR1, MR2                      ; Store ADCRESULT1
    MBCNDD     _RestartCount, GEQ              ; If count >= NUM_DATA_POINTS
    MMOVIZ     MR1, #0.0                       ; Always executed: MR1=0
    MNOP
    MNOP
    MMOV16     @_ConversionCount, MR0          ; If branch not taken
    MSTOP                                           ; store current count
_RestartCount
    MMOV16     @_ConversionCount, MR1          ; If branch taken, restart
count
    MSTOP                                           ; end of task
; This task initializes the ConversionCount
; to zero
;
_ClalTask8:

```

**MMOV16 MARx, mem16** (continued)**Load MAR1 with 16-bit Value**

```
MMOVIZ    MR0, #0.0
MMOV16    @_ConversionCount, MR0
MSTOP
_ClaT8End:
```

## MMOV16 mem16, MARx

### Move 16-Bit Auxiliary Register Contents to Memory

#### Operands

|       |                                                                                |
|-------|--------------------------------------------------------------------------------|
| mem16 | 16-bit destination memory accessed using one of the available addressing modes |
| MARx  | CLA auxiliary register MAR0 or MAR1                                            |

#### Opcode

```

LSW: mmmm mmmm mmmm mmmm (Opcode for MMOV16 mem16, MAR0)
MSW: 0111 0110 1000 addr
LSW: mmmm mmmm mmmm mmmm (Opcode for MMOV16 mem16, MAR1)
MSW: 0111 0110 1100 addr

```

#### Description

Store the contents of MAR0 or MAR1 in the 16-bit memory location pointed to by mem16.

```
[mem16] = MAR0;
```

#### Flags

No flags MSTF flags are affected.

| Flag     | TF | ZF | NF | LUF | LVF |
|----------|----|----|----|-----|-----|
| Modified | No | No | No | No  | No  |

#### Pipeline

This is a single-cycle instruction.

**MMOV16 mem16, MRa****Move 16-Bit Floating-Point Register Contents to Memory****Operands**

|       |                                                                                |
|-------|--------------------------------------------------------------------------------|
| mem16 | 16-bit destination memory accessed using one of the available addressing modes |
| MRa   | CLA floating-point source register (MR0 to MR3)                                |

**Opcode**

```
LSW: mmmmm mmmmm mmmmm mmmmm
MSW: 0111 0101 11aa addr
```

**Description**

Move 16-bit value from the lower 16-bits of the floating-point register (MRa(15:0)) to the location pointed to by mem16.

```
[mem16] = MRa(15:0);
```

**Flags**

No flags MSTF flags are affected.

| Flag     | TF | ZF | NF | LUF | LVF |
|----------|----|----|----|-----|-----|
| Modified | No | No | No | No  | No  |

**Pipeline**

This is a single-cycle instruction.

**Example**

```
; This task logs the last NUM_DATA_POINTS
; ADCRESULT1 values in the array VoltageCLA
;
; When the last element in the array has been
; filled, the task will go back to the
; the first element.
;
; Before starting the ADC conversions, force
; Task 8 to initialize the ConversionCount to zero
;
; The ADC is set to sample (acquire) for 15 SYSCLK cycles
; or 75ns. After the capacitor has captured the analog
; value, the ADC will trigger this task early.
; It takes 10.5 ADCCLKs to complete a conversion,
; the ADCCLK being SYSCLK/4
; T_sys = 1/200MHz = 5ns
; T_adc = 4*T_sys = 20ns
; The ADC will take 10.5 * 4 or 42 SYSCLK cycles to complete
; a conversion. The ADC result register may be read on the
; 36th instruction after the task begins.
;
_ClalTask2:
    .asg          0, N
    .loop
    MNOP
;I1 - I28 Wait till I36 to read result
    .eval        N + 1, N
    .break       N = 28
    .endloop
MMOVZ16 MR0, @_ConversionCount           ;I29 Current Conversion
MMOV16  MAR1, MR0, #_VoltageCLA          ;I30 Next array location
MUI16TOF32 MR0, MR0                     ;I31 Convert count to float32
MADDF32  MR0, MR0, #1.0                  ;I32 Add 1 to conversion count
MCMPPF32 MR0, #NUM_DATA_POINTS.0        ;I33 Compare count to max
MF32TOUI16 MR0, MR0                     ;I34 Convert count to Uint16
MNOP                                           ;I35 Wait till I36 to read
result
MMOVZ16  MR2, @_AdcaResultRegs.ADCRESULT1 ;I36 Read ADCRESULT1
MMOV16   *MAR1, MR2                       ; Store ADCRESULT1
MBCNDD   _RestartCount, GEQ               ; If count >= NUM_DATA_POINTS
MMOVIZ   MR1, #0.0                       ; Always executed: MR1=0
```



## MMOV16 mem16, MRa (continued)

### Move 16-Bit Floating-Point Register Contents to Memory

```

        MNOP
        MNOP
        MMOV16    @_ConversionCount, MR0        ; If branch not taken
        MSTOP      ; store current count
        _RestartCount
        MMOV16    @_ConversionCount, MR1        ; If branch taken, restart
count
        MSTOP      ; end of task
; This task initializes the ConversionCount
; to zero
;
_Cla1Task8:
        MMOVIZ    MR0, #0.0
        MMOV16    @_ConversionCount, MR0
        MSTOP
_ClaT8End:

```

## See also

[MMOVIZ MRa, #16FHi](#)
[MMOVXI MRa, #16FLoHex](#)

**MMOV32 mem32, MRa****Move 32-Bit Floating-Point Register Contents to Memory****Operands**

|       |                                                                                |
|-------|--------------------------------------------------------------------------------|
| MRa   | floating-point register (MR0 to MR3)                                           |
| mem32 | 32-bit destination memory accessed using one of the available addressing modes |

**Opcode**

```
LSW: mmmmm mmmmm mmmmm mmmmm
MSW: 0111 0100 11aa addr
```

**Description**

Move from MRa to 32-bit memory location indicated by mem32.

```
[mem32] = MRa;
```

**Flags**

This instruction modifies the following flags in the MSTF register:

| Flag     | TF | ZF | NF | LUF | LVF |
|----------|----|----|----|-----|-----|
| Modified | No | No | No | No  | No  |

No flags affected.

**Pipeline**

This is a single-cycle instruction.

**Example**

```
; Perform 5 multiply and accumulate operations:
;
; X and Y are 32-bit floating point arrays;
; 1st multiply: A = X0 * Y0
; 2nd multiply: B = X1 * Y1
; 3rd multiply: C = X2 * Y2
; 4th multiply: D = X3 * Y3
; 5th multiply: E = X3 * Y3;
; Result = A + B + C + D + E
;
;_ClaiTask1:
- MMOV32      MAR0, # X           ; MAR0 points to X array
  MMOV32      MAR1, # _Y         ; MAR1 points to Y array
  MNOP                     ; Delay for MAR0, MAR1 load
  MNOP                     ; Delay for MAR0, MAR1 load
  MMOV32      MR0, *MAR0[2]++    ; <-- MAR0 valid
                                ; MR0 = X0, MAR0 += 2
                                ; <-- MAR1 valid
  MMOV32      MR1, *MAR1[2]++    ; MR1 = Y0, MAR1 += 2
  MPMYF32     MR2, MR0, MR1      ; MR2 = A = X0 * Y0
| MMOV32      MR0, *MAR0[2]++    ; In parallel MR0 = X1, MAR0 += 2
  MMOV32      MR1, *MAR1[2]++    ; MR1 = Y1, MAR1 += 2
  MPMYF32     MR3, MR0, MR1      ; MR3 = B = X1 * Y1
| MMOV32      MR0, *MAR0[2]++    ; In parallel MR0 = X2, MAR0 += 2
  MMOV32      MR1, *MAR1[2]++    ; MR1 = Y2, MAR2 += 2
  MMACF32     MR3, MR2, MR2, MR0, MR1 ; MR3 = A + B, MR2 = C = X2 * Y2
| MMOV32      MR0, *MAR0[2]++    ; In parallel MR0 = X3
  MMOV32      MR1, *MAR1[2]++    ; MR1 = Y3
  MMACF32     MR3, MR2, MR2, MR0, MR1 ; MR3 = (A + B) + C, MR2 = D = X3 * Y3
| MMOV32      MR0, *MAR0
  MMOV32      MR1, *MAR1         ; In parallel MR0 = X4
  MPMYF32     MR2, MR0, MR1      ; MR2 = E = X4 * Y4
| MADD32      MR3, MR3, MR2      ; in parallel MR3 = (A + B + C) + D
  MADD32      MR3, MR3, MR2      ; MR3 = (A + B + C + D) + E
  MMOV32      @_Result, MR3      ; Store the result MSTOP ; end of task
```

**See also**[MMOV32 mem32, MSTF](#)

## MMOV32 mem32, MSTF

### Move 32-Bit MSTF Register to Memory

#### Operands

|       |                                |
|-------|--------------------------------|
| MSTF  | floating-point status register |
| mem32 | 32-bit destination memory      |

#### Opcode

```
LSW: mmmm mmmm mmmm mmmm
MSW: 0111 0111 0100 addr
```

#### Description

Copy the CLA's floating-point status register, MSTF, to memory.

```
[mem32] = MSTF;
```

#### Flags

This instruction does not modify flags in the MSTF register:

| Flag     | TF | ZF | NF | LUF | LVF |
|----------|----|----|----|-----|-----|
| Modified | No | No | No | No  | No  |

#### Pipeline

This is a single-cycle instruction.

One of the uses of this instruction is to save off the return PC (RPC) prior to calling a function. The decision to jump to a function is made when the MCCNDD is in the decode2 (D2) phase of the pipeline; the RPC is also updated in this phase. The actual jump occurs three cycles later when MCCNDD enters its execution (E) phase. The user must, therefore, save the old RPC before MCCNDD updates it in the D2 phase, that is, it must save MSTF three instructions prior to the function call.

#### Example

The following example illustrates the pipeline flow for the context save (of the flags and RPC) prior to a function call. The first column in the comments shows the pipeline stages for the MMOV32 instruction while the second column pertains to the MCCNDD instruction.

```
MMOV32 @_temp, MSTF ; D2| |
MNOP                ; R1|F1| MCCNDD is fetched
MNOP                ; R2|F2|
MNOP                ; E |D1|
MCCNDD _bar, UNC    ; W |D2| old RPC written to memory,
                   ;   |   RPC updated with MPC+1
MNOP                ;   |R1|
MNOP                ;   |R2|
MNOP                ;   |E | execution branches to _bar
```

#### See also

[MMOV32 mem32, MRa](#)

**MMOV32 MRa, mem32 {, CNDF}****Conditional 32-Bit Move****Operands**

|       |                                                                             |
|-------|-----------------------------------------------------------------------------|
| MRa   | CLA floating-point destination register (MR0 to MR3)                        |
| mem32 | 32-bit memory location accessed using one of the available addressing modes |
| CNDF  | optional condition.                                                         |

**Opcode**

```
LSW: mmmm mmmm mmmm mmmm
MSW: 0111 00cn dfaa addr
```

**Description**

If the condition is true, then move the 32-bit value referenced by mem32 to the floating-point register indicated by MRa.

```
if (CNDF == TRUE) MRa = [mem32];
```

CNDF is one of the following conditions:

| Encode <sup>(1)</sup> | CNDF                | Description                          | MSTF Flags Tested   |
|-----------------------|---------------------|--------------------------------------|---------------------|
| 0000                  | NEQ                 | Not equal to zero                    | ZF == 0             |
| 0001                  | EQ                  | Equal to zero                        | ZF == 1             |
| 0010                  | GT                  | Greater than zero                    | ZF == 0 AND NF == 0 |
| 0011                  | GEQ                 | Greater than or equal to zero        | NF == 0             |
| 0100                  | LT                  | Less than zero                       | NF == 1             |
| 0101                  | LEQ                 | Less than or equal to zero           | ZF == 1 OR NF == 1  |
| 1010                  | TF                  | Test flag set                        | TF == 1             |
| 1011                  | NTF                 | Test flag not set                    | TF == 0             |
| 1100                  | LU                  | Latched underflow                    | LUF == 1            |
| 1101                  | LV                  | Latched overflow                     | LVF == 1            |
| 1110                  | UNC                 | Unconditional                        | None                |
| 1111                  | UNCF <sup>(2)</sup> | Unconditional with flag modification | None                |

(1) Values not shown are reserved.

(2) This is the default operation if no CNDF field is specified. This condition will allow the ZF and NF flags to be modified when a conditional operation is executed. All other conditions will not modify these flags.

**Flags**

This instruction modifies the following flags in the MSTF register:

| Flag     | TF | ZF  | NF  | LUF | LVF |
|----------|----|-----|-----|-----|-----|
| Modified | No | Yes | Yes | No  | No  |

```
if (CNDF == UNCF)
{
    NF = MRa(31);
    ZF = 0;
    if (MRa(30:23) == 0) { ZF = 1; NF = 0; }
}
else No flags modified;
```

**Pipeline**

This is a single-cycle instruction.

## ***MMOV32 MRa, mem32 {, CNDF}*** (continued)

### ***Conditional 32-Bit Move***

#### **Example**

```

; Given A, B, X, M1 and M2 are 32-bit floating-point
; numbers
;
; if(A == B) calculate Y = X*M1
; if(A! = B) calculate Y = X*M2
;
_ClalTask5:
    MMOV32    MR0, @_A
    MMOV32    MR1, @_B
    MCMPPF32  MR0, MR1
    MMOV32    MR2, @_M1, EQ    ; if A == B, MR2 = M1
                                ;      Y = M1*X
    MMOV32    MR2, @_M2, NEQ   ; if A! = B, MR2 = M2
                                ;      Y = M2*X

    MMOV32    MR3, @_X
    MMPYF32   MR3, MR2, MR3    ; Calculate Y
    MMOV32    @_Y, MR3        ; Store Y
    MSTOP                                ; end of task

```

#### **See also**

[MMOV32 MRa, MRb {, CNDF}](#)  
[MMOVD32 MRa, mem32](#)

## MMOV32 MRa, MRb {, CNDF}

### Conditional 32-Bit Move

#### Operands

|      |                                                      |
|------|------------------------------------------------------|
| MRa  | CLA floating-point destination register (MR0 to MR3) |
| MRb  | CLA floating-point source register (MR0 to MR3)      |
| CNDF | optional condition.                                  |

#### Opcode

```
LSW: 0000 0000 cndf bbaa
MSW: 0111 1010 1100 0000
```

#### Description

If the condition is true, then move the 32-bit value in MRb to the floating-point register indicated by MRa.

```
if (CNDF == TRUE) MRa = MRb;
```

CNDF is one of the following conditions:

| Encode <sup>(1)</sup> | CNDF                | Description                          | MSTF Flags Tested   |
|-----------------------|---------------------|--------------------------------------|---------------------|
| 0000                  | NEQ                 | Not equal to zero                    | ZF == 0             |
| 0001                  | EQ                  | Equal to zero                        | ZF == 1             |
| 0010                  | GT                  | Greater than zero                    | ZF == 0 AND NF == 0 |
| 0011                  | GEQ                 | Greater than or equal to zero        | NF == 0             |
| 0100                  | LT                  | Less than zero                       | NF == 1             |
| 0101                  | LEQ                 | Less than or equal to zero           | ZF == 1 OR NF == 1  |
| 1010                  | TF                  | Test flag set                        | TF == 1             |
| 1011                  | NTF                 | Test flag not set                    | TF == 0             |
| 1100                  | LU                  | Latched underflow                    | LUF == 1            |
| 1101                  | LV                  | Latched overflow                     | LVF == 1            |
| 1110                  | UNC                 | Unconditional                        | None                |
| 1111                  | UNCF <sup>(2)</sup> | Unconditional with flag modification | None                |

(1) Values not shown are reserved.

(2) This is the default operation if no CNDF field is specified. This condition will allow the ZF, and NF flags to be modified when a conditional operation is executed. All other conditions will not modify these flags.

#### Flags

This instruction modifies the following flags in the MSTF register:

| Flag     | TF | ZF  | NF  | LUF | LVF |
|----------|----|-----|-----|-----|-----|
| Modified | No | Yes | Yes | No  | No  |

```
if (CNDF == UNCF)
{
    NF = MRa(31); ZF = 0;
    if (MRa(30:23) == 0) {ZF = 1; NF = 0;}
}
else No flags modified;
```

#### Pipeline

This is a single-cycle instruction.

## ***MMOV32 MRa, MRb {, CNDF}*** (continued)

### ***Conditional 32-Bit Move***

#### **Example**

```
; Given: X = 8.0
;        Y = 7.0
;        A = 2.0
;        B = 5.0
; _ClaTask1
; MMOV32 MR3, @_X      ; MR3 = X = 8.0
; MMOV32 MR0, @_Y      ; MR0 = Y = 7.0
; MMAXF32 MR3, MR0     ; ZF = 0, NF = 0, MR3 = 8.0
; MMOV32 MR1, @_A, GT   ; true, MR1 = A = 2.0
; MMOV32 MR1, @_B, LT   ; false, does not load MR1
; MMOV32 MR2, MR1, GT   ; true, MR2 = MR1 = 2.0
; MMOV32 MR2, MR0, LT   ; false, does not load MR2
; MSTOP
```

#### **See also**

[MMOV32 MRa, mem32 {,CNDF}](#)

## MMOV32 MSTF, mem32

### Move 32-Bit Value from Memory to the MSTF Register

#### Operands

|       |                               |
|-------|-------------------------------|
| MSTF  | CLA status register           |
| mem32 | 32-bit source memory location |

#### Opcode

```
LSW: mmmm mmmm mmmm mmmm
MSW: 0111 0111 0000 addr
```

#### Description

Move from memory to the CLA's status register MSTF. This instruction is most useful when nesting function calls (via MCCNDD).

```
MSTF = [mem32];
```

#### Flags

This instruction modifies the following flags in the MSTF register:

| Flag     | TF  | ZF  | NF  | LUF | LVF |
|----------|-----|-----|-----|-----|-----|
| Modified | Yes | Yes | Yes | Yes | Yes |

Loading the status register will overwrite all flags and the RPC field. The MEALLOW field is not affected.

#### Pipeline

This is a single-cycle instruction.

#### See also

[MMOV32 mem32, MSTF](#)



## MMOVD32 MRa, mem32

### Move 32-Bit Value from Memory with Data Copy

#### Operands

|       |                                                                             |
|-------|-----------------------------------------------------------------------------|
| MRa   | CLA floating-point register (MR0 to MR3)                                    |
| mem32 | 32-bit memory location accessed using one of the available addressing modes |

#### Opcode

```
LSW: mmmm mmmm mmmm mmmm
MSW: 0111 0100 00aa addr
```

#### Description

Move the 32-bit value referenced by mem32 to the floating-point register indicated by MRa.

```
MRa = [mem32];
[mem32+2] = [mem32];
```

#### Flags

This instruction modifies the following flags in the MSTF register:

| Flag     | TF | ZF  | NF  | LUF | LVF |
|----------|----|-----|-----|-----|-----|
| Modified | No | Yes | Yes | No  | No  |

```
NF = MRa(31);
ZF = 0;
if(MRa(30:23) == 0){ ZF = 1; NF = 0; }
```

#### Pipeline

This is a single-cycle instruction.

#### Example

```
; sum = X0*B0 + X1*B1 + X2*B2 + Y1*A1 + Y2*B2
;
;      X2 = X1
;      X1 = X0
;      Y2 = Y1
;      Y1 = sum
;
_Cla1Task2:
  MMOV32 MR0, @_B2      ; MR0 = B2
  MMOV32 MR1, @_X2      ; MR1 = X2
  MMPYF32 MR2, MR1, MR0 ; MR2 = X2*B2
|| MMOV32 MR0, @_B1      ; MR0 = B1
  MMOVD32 MR1, @_X1      ; MR1 = X1, X2 = X1
  MMPYF32 MR3, MR1, MR0 ; MR3 = X1*B1
|| MMOV32 MR0, @_B0      ; MR0 = B0
  MMOVD32 MR1, @_X0      ; MR1 = X0, X1 = X0
; MR3 = X1*B1 + X2*B2, MR2 = X0*B0
; MR0 = A2
  MMACF32 MR3, MR2, MR2, MR1, MR0
|| MMOV32 MR0, @_A2

  MMOV32 MR1, @_Y2      ; MR1 = Y2
; MR3 = X0*B0 + X1*B1 + X2*B2, MR2 = Y2*A2
; MR0 = A1
  MMACF32 MR3, MR2, MR2, MR1, MR0
|| MMOV32 MR0, @_A1
  MMOVD32 MR1, @_Y1      ; MR1 = Y1, Y2 = Y1
  MADDF32 MR3, MR3, MR2 ; MR3 = Y2*A2 + X0*B0 + X1*B1 + X2*B2
|| MMPYF32 MR2, MR1, MR0 ; MR2 = Y1*A1
  MADDF32 MR3, MR3, MR2 ; MR3 = Y1*A1 + Y2*A2 + X0*B0 + X1*B1 + X2*B2
  MMOV32 @_Y1, MR3      ; Y1 = MR3
  MSTOP                  ; end of task
```

**MMOVD32 MRa, mem32** (continued)

***Move 32-Bit Value from Memory with Data Copy***

---

**See also**

[MMOV32 MRa, mem32 {,CNDF}](#)

## MMOVF32 MRa, #32F

### Load the 32-Bits of a 32-Bit Floating-Point Register

#### Operands

This instruction is an alias for MMOVIZ and MMOVXI instructions. The second operand is translated by the assembler such that the instruction becomes:

```
MMOVIZ MRa, #16FHiHex MMOVXI MRa, #16FLoHex
```

|      |                                                                    |
|------|--------------------------------------------------------------------|
| MRa  | CLA floating-point destination register (MR0 to MR3)               |
| #32F | immediate float value represented in floating-point representation |

#### Opcode

```
LSW: I I I I I I I I I I I I I I (opcode of MMOVIZ MRa, #16FHiHex)
MSW: 0 1 1 1 1 0 0 0 0 1 0 0 0 0 a a
LSW: I I I I I I I I I I I I I I (opcode of MMOVXI MRa, #16FLoHex)
MSW: 0 1 1 1 1 0 0 0 1 0 0 0 0 0 a a
```

#### Description

Note: This instruction accepts the immediate operand only in floating-point representation. To specify the immediate value as a hex value (IEEE 32-bit floating-point format) use the MOV132 MRa, #32FHex instruction.

Load the 32-bits of MRa with the immediate float value represented by #32F.

#32F is a float value represented in floating-point representation. The assembler will only accept a float value represented in floating-point representation. That is, 3.0 can only be represented as #3.0. #0x40400000 will result in an error.

```
MRa = #32F;
```

#### Flags

This instruction modifies the following flags in the MSTF register:

| Flag     | TF | ZF | NF | LUF | LVF |
|----------|----|----|----|-----|-----|
| Modified | No | No | No | No  | No  |

#### Pipeline

Depending on #32FH, this instruction takes one or two cycles. If all of the lower 16-bits of the IEEE 32-bit floating-point format of #32F are zeros, then the assembler will convert MMOVF32 into only MMOVIZ instruction. If the lower 16-bits of the IEEE 32-bit floating-point format of #32F are not zeros, then the assembler will convert MMOVF32 into MMOVIZ and MMOVXI instructions.

#### Example

```
MMOVF32 MR1, #3.0      ; MR1 = 3.0 (0x40400000)
                        ; Assembler converts this instruction as
                        ; MMOVIZ MR1, #0x4040
MMOVF32 MR2, #0.0      ; MR2 = 0.0 (0x00000000)
                        ; Assembler converts this instruction as
                        ; MMOVIZ MR2, #0x0
MMOVF32 MR3, #12.265   ; MR3 = 12.625 (0x41443D71)
                        ; Assembler converts this instruction as
                        ; MMOVIZ MR3, #0x4144
                        ; MMOVXI MR3, #0x3D71
```

#### See also

[MMOVIZ MRa, #16FHi](#)  
[MMOVXI MRa, #16FLoHex](#)  
[MMOVI32 MRa, #32FHex](#)

**MMOVI16 MARx, #16I****Load the Auxiliary Register with the 16-Bit Immediate Value****Operands**

|      |                                 |
|------|---------------------------------|
| MARx | Auxiliary register MAR0 or MAR1 |
| #16I | 16-bit immediate value          |

**Opcode**

```

LSW: IIII IIII IIII IIII (opcode of MMOVI16 MAR0, #16I)
MSW: 0111 1111 1100 0000
LSW: IIII IIII IIII IIII (opcode of MMOVI16 MAR1, #16I)
MSW: 0111 1111 1110 0000

```

**Description**

Load the auxiliary register, MAR0 or MAR1, with a 16-bit immediate value. Refer to the pipeline section for important information regarding this instruction.

```
MARx = #16I;
```

**Flags**

This instruction does not modify flags in the MSTF register:

| Flag     | TF | ZF | NF | LUF | LVF |
|----------|----|----|----|-----|-----|
| Modified | No | No | No | No  | No  |

**Pipeline**

This is a single-cycle instruction. The immediate load of MAR0 or MAR1 will occur in the EXE phase of the pipeline. Any post increment of MAR0 or MAR1 using indirect addressing will occur in the D2 phase of the pipeline. Therefore the following applies when loading the auxiliary registers:

- I1 and I2**

The two instructions following MMOVI16 will use MAR0/MAR1 before the update occurs. Thus these two instructions will use the old value of MAR0 or MAR1.

- I3**

Loading of an auxiliary register occurs in the EXE phase while updates due to post-increment addressing occur in the D2 phase. Thus I3 cannot use the auxiliary register or there will be a conflict. In the case of a conflict, the update due to address-mode post increment will win and the auxiliary register will not be updated with #\_X.

- I4**

Starting with the 4th instruction MAR0 or MAR1 will be the new value loaded with MMOVI16.

```

; Assume MAR0 is 50 and #_X is 20
MMOVI16 MAR0, #_X          ; Load MAR0 with address of X (20)
<Instruction 1>              ; I1 Will use the old value of MAR0 (50)
<Instruction 2>              ; I2 Will use the old value of MAR0 (50)
<Instruction 3>              ; I3 Cannot use MAR0
<Instruction 4>              ; I4 Will use the new value of MAR0 (20)
<Instruction 5>              ; I5
....

```

**MMOVI16 MARx, #16I** (continued)

**Load the Auxiliary Register with the 16-Bit Immediate Value**

| Table 10-16. Pipeline Activity For MMOVI16 MAR0/MAR1, #16I |         |         |         |         |         |         |         |   |
|------------------------------------------------------------|---------|---------|---------|---------|---------|---------|---------|---|
| Instruction                                                | F1      | F2      | D1      | D2      | R1      | R2      | E       | W |
| MMOVI16 MAR0, #_X                                          | MMOVI16 |         |         |         |         |         |         |   |
| I1                                                         | I1      | MMOVI16 |         |         |         |         |         |   |
| I2                                                         | I2      | I1      | MMOVI16 |         |         |         |         |   |
| I3                                                         | I3      | I2      | I1      | MMOVI16 |         |         |         |   |
| I4                                                         | I4      | I3      | I2      | I1      | MMOVI16 |         |         |   |
| I5                                                         | I5      | I4      | I3      | I2      | I1      | MMOVI16 |         |   |
| I6                                                         | I6      | I5      | I4      | I3      | I2      | I1      | MMOVI16 |   |

**MMOVI32 MRa, #32FHex****Load the 32-Bits of a 32-Bit Floating-Point Register with the Immediate****Operands**

|         |                                                                               |
|---------|-------------------------------------------------------------------------------|
| MRa     | floating-point register (MR0 to MR3)                                          |
| #32FHex | A 32-bit immediate value that represents an IEEE 32-bit floating-point value. |

This instruction is an alias for MMOVIZ and MMOVXI instructions. The second operand is translated by the assembler such that the instruction becomes:

```
MMOVIZ MRa, #16FHiHex
MMOVXI MRa, #16FLHex
```

**Opcode**

```
LSW: I I I I I I I I I I I I I I (opcode of MMOVIZ MRa, #16FHiHex)
MSW: 0111 1000 0100 00aa
LSW: I I I I I I I I I I I I I I (opcode of MMOVXI MRa, #16FLHex)
MSW: 0111 1000 1000 00aa
```

**Description**

Note: This instruction only accepts a hex value as the immediate operand. To specify the immediate value with a floating-point representation use the MMOVF32 MRa, #32F instruction.

Load the 32-bits of MRa with the immediate 32-bit hex value represented by #32Fhex.

#32Fhex is a 32-bit immediate hex value that represents the IEEE 32-bit floating-point value of a floating-point number. The assembler will only accept a hex immediate value. That is, 3.0 can only be represented as #0x40400000. #3.0 will result in an error.

```
MRa = #32FHex;
```

**Flags**

This instruction modifies the following flags in the MSTF register:

| Flag     | TF | ZF | NF | LUF | LVF |
|----------|----|----|----|-----|-----|
| Modified | No | No | No | No  | No  |

**Pipeline**

Depending on #32FHex, this instruction takes one or two cycles. If all of the lower 16-bits of #32FHex are zeros, then assembler will convert MOVI32 to the MMOVIZ instruction. If the lower 16-bits of #32FHex are not zeros, then assembler will convert MOVI32 to a MMOVIZ and a MMOVXI instruction.

**Example**

```
MOVI32 MR1, #0x40400000 ; MR1 = 0x40400000
                        ; Assembler converts this instruction as
                        ; MMOVIZ MR1, #0x4040
MOVI32 MR2, #0x00000000 ; MR2 = 0x00000000
                        ; Assembler converts this instruction as
                        ; MMOVIZ MR2, #0x0
MOVI32 MR3, #0x40004001 ; MR3 = 0x40004001
                        ; Assembler converts this instruction as
                        ; MMOVIZ MR3, #0x4000
                        ; MMOVXI MR3, #0x4001
MOVI32 MR0, #0x00004040 ; MR0 = 0x00004040
                        ; Assembler converts this instruction as
                        ; MMOVIZ MR0, #0x0000
                        ; MMOVXI MR0, #0x4040
```

**MMOVI32 MRa, #32FHex** (continued)

***Load the 32-Bits of a 32-Bit Floating-Point Register with the Immediate***

---

**See also**

[MMOVIZ MRa, #16FHi](#)  
[MMOVXI MRa, #16FLoHex](#)  
[MMOVF32 MRa, #32F](#)

**MMOVIZ MRa, #16FHi****Load the Upper 16-Bits of a 32-Bit Floating-Point Register****Operands**

|        |                                                                                                                                                             |
|--------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| MRa    | floating-point register (MR0 to MR3)                                                                                                                        |
| #16FHi | A 16-bit immediate value that represents the upper 16-bits of an IEEE 32-bit floating-point value. The low 16-bits of the mantissa are assumed to be all 0. |

**Opcode**

```
LSW: I I I I I I I I I I I I I I I I
MSW: 0111 1000 0100 00aa
```

**Description**

Load the upper 16-bits of MRa with the immediate value #16FHi and clear the low 16-bits of MRa.

#16FHiHex is a 16-bit immediate value that represents the upper 16-bits of an IEEE 32-bit floating-point value. The low 16-bits of the mantissa are assumed to be all 0. The assembler will only accept a decimal or hex immediate value. That is, -1.5 can be represented as #-1.5 or #0xBFC0.

By itself, MMOVIZ is useful for loading a floating-point register with a constant in which the lowest 16-bits of the mantissa are 0. Some examples are 2.0 (0x40000000), 4.0 (0x40800000), 0.5 (0x3F000000), and -1.5 (0xBFC00000). If a constant requires all 32-bits of a floating-point register to be initialized, then use MMOVIZ along with the MMOVXI instruction.

```
MRa(31:16) = #16FHi;
MRa(15:0) = 0;
```

**Flags**

This instruction modifies the following flags in the MSTF register:

| Flag     | TF | ZF | NF | LUF | LVF |
|----------|----|----|----|-----|-----|
| Modified | No | No | No | No  | No  |

**Pipeline**

This is a single-cycle instruction.

**Example**

```
; Load MR0 and MR1 with -1.5 (0xBFC00000)
MMOVIZ MR0, #0xBFC0 ; MR0 = 0xBFC00000 (1.5)
MMOVIZ MR1, #-1.5 ; MR1 = -1.5 (0xBFC00000)
; Load MR2 with pi = 3.141593 (0x40490FDB)
MMOVIZ MR2, #0x4049 ; MR2 = 0x40490000
MMOVXI MR2, #0x0FDB ; MR2 = 0x40490FDB
```

**See also**

[MMOVF32 MRa, #32F](#)  
[MMOVI32 MRa, #32FHex](#)  
[MMOVXI MRa, #16FLoHex](#)



## MMOVZ16 MRa, mem16

### Load MRx With 16-bit Value

#### Operands

|       |                                                      |
|-------|------------------------------------------------------|
| MRa   | CLA floating-point destination register (MR0 to MR3) |
| mem16 | 16-bit source memory location                        |

#### Opcode

```
LSW: mmmm mmmm mmmm mmmm
MSW: 0111 0101 10aa addr
```

#### Description

Move the 16-bit value referenced by mem16 to the floating-point register indicated by MRa.

```
MRa(31:16) = 0;
MRa(15:0) = [mem16];
```

#### Flags

This instruction modifies the following flags in the MSTF register:

| Flag     | TF | ZF  | NF  | LUF | LVF |
|----------|----|-----|-----|-----|-----|
| Modified | No | Yes | Yes | No  | No  |

The MSTF register flags are modified based on the integer results of the operation.

```
NF = 0;
if (MRa(31:0) == 0) { ZF = 1; }
```

#### Pipeline

This is a single-cycle instruction.

## MMOVXI MRa, #16FLoHex

### Move Immediate to the Low 16-Bits of a Floating-Point Register

#### Operands

|           |                                                                                                                                                |
|-----------|------------------------------------------------------------------------------------------------------------------------------------------------|
| MRa       | CLA floating-point register (MR0 to MR3)                                                                                                       |
| #16FLoHex | A 16-bit immediate hex value that represents the lower 16-bits of an IEEE 32-bit floating-point value. The upper 16-bits will not be modified. |

#### Opcode

```
LSW: I I I I I I I I I I I I I I I I
MSW: 0111 1000 1000 00aa
```

#### Description

Load the low 16-bits of MRa with the immediate value #16FLoHex. #16FLoHex represents the lower 16-bits of an IEEE 32-bit floating-point value. The upper 16-bits of MRa will not be modified. MMOVXI can be combined with the MMOVIZ instruction to initialize all 32-bits of a MRa register.

```
MRa(15:0) = #16FLoHex;
MRa(31:16) = Unchanged;
```

#### Flags

| Flag     | TF | ZF | NF | LUF | LVF |
|----------|----|----|----|-----|-----|
| Modified | No | No | No | No  | No  |

#### Pipeline

This is a single-cycle instruction.

#### Example

```
; Load MR0 with pi = 3.141593 (0x40490FDB)
MMOVIZ    MR0, #0x4049    ; MR0 = 0x40490000
MMOVXI    MR0, #0x0FDB    ; MR0 = 0x40490FDB
```

#### See also

[MMOVIZ MRa, #16FHi](#)

## MMPYF32 MRa, MRb, MRc

### 32-Bit Floating-Point Multiply

#### Operands

|     |                                                      |
|-----|------------------------------------------------------|
| MRa | CLA floating-point destination register (MR0 to MR3) |
| MRb | CLA floating-point source register (MR0 to MR3)      |
| MRc | CLA floating-point source register (MR0 to MR3)      |

#### Opcode

```
LSW: 0000 0000 00cc bbaa
MSW: 0111 1100 0000 0000
```

#### Description

Multiply the contents of two floating-point registers.

```
MRa = MRb * MRc;
```

#### Flags

This instruction modifies the following flags in the MSTF register:

| Flag     | TF | ZF | NF | LUF | LVF |
|----------|----|----|----|-----|-----|
| Modified | No | No | No | Yes | Yes |

The MSTF register flags are modified as follows:

- LUF = 1 if MMPYF32 generates an underflow condition.
- LVF = 1 if MMPYF32 generates an overflow condition.

#### Pipeline

This is a single-cycle instruction.

#### Example

```
; Calculate Num/Den using a Newton-Raphson algorithm for 1/Den
; Ye = Estimate(1/X)
; Ye = Ye*(2.0 - Ye*X)
; Ye = Ye*(2.0 - Ye*X)
;
_Cla1Task1:
    MOV32    MR1, @_Den        ; MR1 = Den
    MEINVF32 MR2, MR1         ; MR2 = Ye = Estimate(1/Den)
    MMPYF32  MR3, MR2, MR1     ; MR3 = Ye*Den
    MSUBF32  MR3, #2.0, MR3    ; MR3 = 2.0 - Ye*Den
    MMPYF32  MR2, MR2, MR3     ; MR2 = Ye = Ye*(2.0 - Ye*Den)
    MMPYF32  MR3, MR2, MR1     ; MR3 = Ye*Den
    || MOV32  MR0, @_Num        ; MR0 = Num
    MSUBF32  MR3, #2.0, MR3    ; MR3 = 2.0 - Ye*Den
    MMPYF32  MR2, MR2, MR3     ; MR2 = Ye = Ye*(2.0 - Ye*Den)
    || MOV32  MR1, @_Den        ; Reload Den To Set Sign
    MNEGF32  MR0, MR0, EQ      ; if (Den == 0.0) Change Sign Of Num
    MMPYF32  MR0, MR2, MR0     ; MR0 = Y = Ye*Num
    MOV32    @_Dest, MR0      ; Store result
    MSTOP                                     ; end of task
```

#### See also

[MMPYF32 MRa, #16FHi, MRb](#)  
[MMPYF32 MRa, MRb, MRc || MADDF32 MRd, MRe, MRf](#)  
[MMPYF32 MRd, MRe, MRf || MOV32 MRa, mem32](#)  
[MMPYF32 MRd, MRe, MRf || MOV32 mem32, MRa](#)  
[MMPYF32 MRa, MRb, MRc || MSUBF32 MRd, MRe, MRf](#)  
[MMACF32 MR3, MR2, MRd, MRe, MRf || MOV32 MRa, mem32](#)

**MMPYF32 MRa, #16FHi, MRb****32-Bit Floating-Point Multiply****Operands**

|        |                                                                                                                                                             |
|--------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| MRa    | CLA floating-point destination register (MR0 to MR3)                                                                                                        |
| #16FHi | A 16-bit immediate value that represents the upper 16-bits of an IEEE 32-bit floating-point value. The low 16-bits of the mantissa are assumed to be all 0. |
| MRb    | CLA floating-point source register (MR0 to MR3)                                                                                                             |

**Opcode**

```
LSW: I I I I I I I I I I I I I I I I
MSW: 0 1 1 1 0 1 1 1 1 0 0 0 b b a a
```

**Description**

Multiply MRb with the floating-point value represented by the immediate operand. Store the result of the addition in MRa.

#16FHi is a 16-bit immediate value that represents the upper 16-bits of an IEEE 32-bit floating-point value. The low 16-bits of the mantissa are assumed to be all 0. #16FHi is most useful for representing constants where the lowest 16-bits of the mantissa are 0. Some examples are 2.0 (0x40000000), 4.0 (0x40800000), 0.5 (0x3F000000), and -1.5 (0xBFC00000). The assembler will accept either a hex or float as the immediate value. That is, the value -1.5 can be represented as #-1.5 or #0xBFC0.

```
MRa = MRb * #16FHi:0;
```

This instruction can also be written as MMPYF32 MRa, MRb, #16FHi.

**Flags**

This instruction modifies the following flags in the MSTF register:.

| Flag     | TF | ZF | NF | LUF | LVF |
|----------|----|----|----|-----|-----|
| Modified | No | No | No | Yes | Yes |

The MSTF register flags are modified as follows:

- LUF = 1 if MMPYF32 generates an underflow condition.
- LVF = 1 if MMPYF32 generates an overflow condition.

**Pipeline**

This is a single-cycle instruction.

**Example 1**

```
; Same as example 2 but #16FHi is represented in float
MMOVIZ    MR3, #2.0      ; MR3 = 2.0 (0x40000000)
MMPYF32    MR0, #3.0, MR3 ; MR0 = 3.0 * MR3 = 6.0 (0x40C00000)
MMOV32     @_X, MR0      ; Save the result in variable X
```

**Example 2**

```
; Same as example 1 but #16FHi is represented in Hex
MMOVIZ    MR3, #2.0      ; MR3 = 2.0 (0x40000000)
MMPYF32    MR0, #0x4040, MR3 ; MR0 = 0x4040 * MR3 = 6.0 (0x40C00000)
MMOV32     @_X, MR0      ; Save the result in variable X
```

**MMPYF32 MRa, #16FHi, MRb** (continued)

**32-Bit Floating-Point Multiply**
**Example 3**

```

; Given X, M and B are IQ24 numbers:
; X = IQ24(+2.5) = 0x02800000
; M = IQ24(+1.5) = 0x01800000
; B = IQ24(-0.5) = 0xFF800000
;
; Calculate Y = X * M + B
;
;
;_ClalTask2:
;
; Convert M, X and B from IQ24 to float
MI32TOF32 MR0, @_M ; MR0 = 0x4BC00000
MI32TOF32 MR1, @_X ; MR1 = 0x4C200000
MI32TOF32 MR2, @_B ; MR2 = 0xCB000000
MMPYF32 MR0, MR0, #0x3380 ; M = 1/(1*2^24) * iqm = 1.5 (0x3FC00000)
MMPYF32 MR1, MR1, #0x3380 ; X = 1/(1*2^24) * iqx = 2.5 (0x40200000)
MMPYF32 MR2, MR2, #0x3380 ; B = 1/(1*2^24) * iqb = -.5 (0xBF000000)
MMPYF32 MR3, MR0, MR1 ; M*X
MADDF32 MR2, MR2, MR3 ; Y=MX+B = 3.25 (0x40500000)
; Convert Y from float32 to IQ24
MMPYF32 MR2, MR2, #0x4B80 ; Y * 1*2^24
MF32TOI32 MR2, MR2 ; IQ24(Y) = 0x03400000
MMOV32 @_Y, MR2 ; store result
MSTOP ; end of task

```

**See also**
[MMPYF32 MRa, MRb, #16FHi](#)
[MMPYF32 MRa, MRb, MRc](#)
[MMPYF32 MRa, MRb, MRc || MADDF32 MRd, MRe, MRf](#)

**MMPYF32 MRa, MRb, #16FHi****32-Bit Floating-Point Multiply****Operands**

|        |                                                                                                                                                             |
|--------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| MRa    | CLA floating-point destination register (MR0 to MR3)                                                                                                        |
| MRb    | CLA floating-point source register (MR0 to MR3)                                                                                                             |
| #16FHi | A 16-bit immediate value that represents the upper 16-bits of an IEEE 32-bit floating-point value. The low 16-bits of the mantissa are assumed to be all 0. |

**Opcode**

```
LSW: I I I I I I I I I I I I I I I I
MSW: 0 1 1 1 0 1 1 1 1 0 0 0 b b a a
```

**Description**

Multiply MRb with the floating-point value represented by the immediate operand. Store the result of the addition in MRa.

#16FHi is a 16-bit immediate value that represents the upper 16-bits of an IEEE 32-bit floating-point value. The low 16-bits of the mantissa are assumed to be all 0. #16FHi is most useful for representing constants where the lowest 16-bits of the mantissa are 0. Some examples are 2.0 (0x40000000), 4.0 (0x40800000), 0.5 (0x3F000000), and -1.5 (0xBFC00000). The assembler will accept either a hex or float as the immediate value. That is, the value -1.5 can be represented as #-1.5 or #0xBFC0.

```
MRa = MRb * #16FHi:0;
```

This instruction can also be written as MMPYF32 MRa, #16FHi, MRb.

**Flags**

This instruction modifies the following flags in the MSTF register..

| Flag     | TF | ZF | NF | LUF | LVF |
|----------|----|----|----|-----|-----|
| Modified | No | No | No | Yes | Yes |

The MSTF register flags are modified as follows:

- LUF = 1 if MMPYF32 generates an underflow condition.
- LVF = 1 if MMPYF32 generates an overflow condition.

**Pipeline**

This is a single-cycle instruction.

**Example 1**

```
;Same as example 2 but #16FHi is represented in float
MMOVIZ MR3, #2.0 ; MR3 = 2.0 (0x40000000)
MMPYF32 MR0, MR3, #3.0 ; MR0 = MR3 * 3.0 = 6.0 (0x40C00000)
MMOV32 @_X, MR0 ; Save the result in variable X
```

**Example 2**

```
;Same as example 1 but #16FHi is represented in Hex
MMOVIZ MR3, #2.0 ; MR3 = 2.0 (0x40000000)
MMPYF32 MR0, MR3, #0x4040 ; MR0 = MR3 * 0x4040 = 6.0 (0x40C00000)
MMOV32 @_X, MR0 ; Save the result in variable X
```

**MMPYF32 MRa, MRb, #16FHi** (continued)

**32-Bit Floating-Point Multiply**
**Example 3**

```

; Given X, M and B are IQ24 numbers:
; X = IQ24(+2.5) = 0x02800000
; M = IQ24(+1.5) = 0x01800000
; B = IQ24(-0.5) = 0xFF800000
;
; Calculate Y = X * M + B
;
_ClalTask2:
;
; Convert M, X and B from IQ24 to float
MI32TOF32 MR0, @_M      ; MR0 = 0x4BC00000
MI32TOF32 MR1, @_X      ; MR1 = 0x4C200000
MI32TOF32 MR2, @_B      ; MR2 = 0xCB000000
MMPYF32 MR0, #0x3380, MR0 ; M = 1/(1*2^24) * iqm = 1.5 (0x3FC00000)
MMPYF32 MR1, #0x3380, MR1 ; X = 1/(1*2^24) * iqx = 2.5 (0x40200000)
MMPYF32 MR2, #0x3380, MR2 ; B = 1/(1*2^24) * iqb = -.5 (0xBF000000)
MMPYF32 MR3, MR0, MR1    ; M*X
MADDF32 MR2, MR2, MR3    ; Y=MX+B = 3.25 (0x40500000)
; Convert Y from float32 to IQ24
MMPYF32 MR2, #0x4B80, MR2 ; Y * 1*2^24
MF32TOI32 MR2, MR2      ; IQ24(Y) = 0x03400000
MMOV32 @_Y, MR2         ; store result
MSTOP                   ; end of task

```

**See also**

[MMPYF32 MRa, #16FHi, MRb](#)  
[MMPYF32 MRa, MRb, MRc](#)

**MMPYF32 MRa, MRb, MRc||MADDF32 MRd, MRe, MRf**
**32-Bit Floating-Point Multiply with Parallel Add**
**Operands**

|     |                                                                                                            |
|-----|------------------------------------------------------------------------------------------------------------|
| MRa | CLA floating-point destination register for MMPYF32 (MR0 to MR3)<br>MRa cannot be the same register as MRd |
| MRb | CLA floating-point source register for MMPYF32 (MR0 to MR3)                                                |
| MRc | CLA floating-point source register for MMPYF32 (MR0 to MR3)                                                |
| MRd | CLA floating-point destination register for MADDF32 (MR0 to MR3)<br>MRd cannot be the same register as MRa |
| MRe | CLA floating-point source register for MADDF32 (MR0 to MR3)                                                |
| MRf | CLA floating-point source register for MADDF32 (MR0 to MR3)                                                |

**Opcode**

```
LSW: 0000 ffee ddcc bbaa
MSW: 0111 1010 0000 0000
```

**Description**

Multiply the contents of two floating-point registers with parallel addition of two registers.

```
MRa = MRb * MRc;
MRd = MRe + MRf;
```

**Restrictions**

The destination register for the MMPYF32 and the MADDF32 must be unique. That is, MRa cannot be the same register as MRd.

**Flags**

This instruction modifies the following flags in the MSTF register:

| Flag     | TF | ZF | NF | LUF | LVF |
|----------|----|----|----|-----|-----|
| Modified | No | No | No | Yes | Yes |

The MSTF register flags are modified as follows:

- LUF = 1 if MMPYF32 or MADDF32 generates an underflow condition.
- LVF = 1 if MMPYF32 or MADDF32 generates an overflow condition.

**Pipeline**

Both MMPYF32 and MADDF32 complete in a single cycle.



**MMPYF32 MRa, MRb, MRC||MADDF32 MRd, MRe, MRf (continued)**
**32-Bit Floating-Point Multiply with Parallel Add**
**Example**

```

; Perform 5 multiply and accumulate operations:
;
; X and Y are 32-bit floating point arrays
;
; 1st multiply: A = X0 * Y0
; 2nd multiply: B = X1 * Y1
; 3rd multiply: C = X2 * Y2
; 4th multiply: D = X3 * Y3
; 5th multiply: E = X3 * Y3
;
; Result = A + B + C + D + E
;
_Cla1Task1:
    MMOV16    MAR0, #_X          ; MAR0 points to X array
    MMOV16    MAR1, #_Y          ; MAR1 points to Y array
    MNOP                      ; Delay for MAR0, MAR1 load
    MNOP                      ; Delay for MAR0, MAR1 load
    MMOV32    MR0, *MAR0[2]++    ; <-- MAR0 valid
                                ; MR0 = X0, MAR0 += 2
                                ; <-- MAR1 valid
    MMOV32    MR1, *MAR1[2]++    ; MR1 = Y0, MAR1 += 2
    MMPYF32   MR2, MR0, MR1      ; MR2 = A = X0 * Y0
|| MMOV32    MR0, *MAR0[2]++    ; In parallel MR0 = X1, MAR0 += 2
    MMOV32    MR1, *MAR1[2]++    ; MR1 = Y1, MAR1 += 2
    MMPYF32   MR3, MR0, MR1      ; MR3 = B = X1 * Y1
|| MMOV32    MR0, *MAR0[2]++    ; In parallel MR0 = X2, MAR0 += 2
    MMOV32    MR1, *MAR1[2]++    ; MR1 = Y2, MAR2 += 2
    MMACF32   MR3, MR2, MR2, MR0, MR1 ; MR3 = A + B, MR2 = C = X2 * Y2
|| MMOV32    MR0, *MAR0[2]++    ; In parallel MR0 = X3
    MMOV32    MR1, *MAR1[2]++    ; MR1 = Y3
    MMACF32   MR3, MR2, MR2, MR0, MR1 ; MR3 = (A + B) + C, MR2 = D = X3 * Y3
|| MMOV32    MR0, *MAR0         ; In parallel MR0 = X4
    MMOV32    MR1, *MAR1         ; MR1 = Y4
    MMPYF32   MR2, MR0, MR1      ; MR2 = E = X4 * Y4
|| MADDF32   MR3, MR3, MR2      ; in parallel MR3 = (A + B + C) + D

    MADDF32   MR3, MR3, MR2      ; MR3 = (A + B + C + D) + E
    MMOV32    @_Result, MR3      ; Store the result
    MSTOP                      ; end of task

```

**See also**
[MMACF32 MR3, MR2, MRd, MRe, MRf || MMOV32 MRa, mem32](#)

**MMPYF32 MRd, MRe, MRf ||MMOV32 MRa, mem32**
**32-Bit Floating-Point Multiply with Parallel Move**
**Operands**

|       |                                                                                                                 |
|-------|-----------------------------------------------------------------------------------------------------------------|
| MRd   | CLA floating-point destination register for MMPYF32 (MR0 to MR3)<br>MRd cannot be the same register as MRa      |
| MRe   | CLA floating-point source register for MMPYF32 (MR0 to MR3)                                                     |
| MRf   | CLA floating-point source register for MMPYF32 (MR0 to MR3)                                                     |
| MRa   | CLA floating-point destination register for MMOV32 (MR0 to MR3)<br>MRa cannot be the same register as MRd       |
| mem32 | 32-bit memory location accessed using one of the available addressing modes. This will be the source of MMOV32. |

**Opcode**

```
LSW: mmmm mmmm mmmm mmmm
MSW: 0000 ffee ddaa addr
```

**Description**

Multiply the contents of two floating-point registers and load another.

```
MRd = MRe * MRf;
MRa = [mem32];
```

**Restrictions**

The destination register for the MMPYF32 and the MMOV32 must be unique. That is, MRa cannot be the same register as MRd.

**Flags**

This instruction modifies the following flags in the MSTF register..

| Flag     | TF | ZF  | NF  | LUF | LVF |
|----------|----|-----|-----|-----|-----|
| Modified | No | Yes | Yes | Yes | Yes |

The MSTF register flags are modified as follows:

- LUF = 1 if MMPYF32 generates an underflow condition.
- LVF = 1 if MMPYF32 generates an overflow condition.

The MMOV32 Instruction will set the NF and ZF flags as follows:

```
NF = MRa(31);
ZF = 0;
if(MRa(30:23) == 0) { ZF = 1; NF = 0; }
```

**Pipeline**

Both MMPYF32 and MMOV32 complete in a single cycle.

**Example 1**

```
; Given M1, X1 and B1 are 32-bit floating point
; Calculate Y1 = M1*X1+B1
;
_ClalTask1:
    MMOV32    MR0, @M1        ; Load MR0 with M1
    MMOV32    MR1, @X1        ; Load MR1 with X1
    MMPYF32   MR1, MR1, MR0    ; Multiply M1*X1
    || MMOV32  MR0, @B1        ; and in parallel load MR0 with B1
    MADDF32   MR1, MR1, MR0    ; Add M*X1 to B1 and store in MR1
    MMOV32    @Y1, MR1        ; Store the result
    MSTOP                                ; end of task
```

## **MMPYF32 MRd, MRe, MRf || MMOV32 MRa, mem32 (continued)**

### **32-Bit Floating-Point Multiply with Parallel Move**

#### **Example 2**

```

; Given A, B and C are 32-bit floating-point numbers
; Calculate Y2 = (A * B)
;           Y3 = (A * B) * C
;
;
_Cla1Task2:
    _MMOV32    MR0, @A          ; Load MR0 with A
    _MMOV32    MR1, @B          ; Load MR1 with B
    _MMPYF32   MR1, MR1, MR0    ; Multiply A*B
||   _MMOV32    MR0, @C          ; and in parallel load MR0 with C
||   _MMPYF32   MR1, MR1, MR0    ; Multiply (A*B) by C
||   _MMOV32    @Y2, MR1        ; and in parallel store A*B
    _MMOV32    @Y3, MR1        ; Store the result
    _MSTOP
                                ; end of task

```

#### **See also**

[MMPYF32 MRd, MRe, MRf || MMOV32 mem32, MRa](#)  
[MMACF32 MR3, MR2, MRd, MRe, MRf || MMOV32 MRa, mem32](#)

**MMPYF32 MRd, MRe, MRf || MMOV32 mem32, MRa**
**32-Bit Floating-Point Multiply with Parallel Move**
**Operands**

|       |                                                                                                                      |
|-------|----------------------------------------------------------------------------------------------------------------------|
| MRd   | CLA floating-point destination register for MMPYF32 (MR0 to MR3)                                                     |
| MRe   | CLA floating-point source register for MMPYF32 (MR0 to MR3)                                                          |
| MRf   | CLA floating-point source register for MMPYF32 (MR0 to MR3)                                                          |
| mem32 | 32-bit memory location accessed using one of the available addressing modes. This will be the destination of MMOV32. |
| MRa   | CLA floating-point source register for MMOV32 (MR0 to MR3)                                                           |

**Opcode**

```
LSW: mmmm mmmm mmmm mmmm
MSW: 0100 ffee ddaa addr
```

**Description**

Multiply the contents of two floating-point registers and move from memory to register.

```
MRd = MRe * MRf;
[mem32] = MRa;
```

**Flags**

This instruction modifies the following flags in the MSTF register:

| Flag     | TF | ZF | NF | LUF | LVF |
|----------|----|----|----|-----|-----|
| Modified | No | No | No | Yes | Yes |

The MSTF register flags are modified as follows:

- LUF = 1 if MMPYF32 generates an underflow condition.
- LVF = 1 if MMPYF32 generates an overflow condition.

**Pipeline**

MMPYF32 and MMOV32 both complete in a single cycle.

**Example**

```
; Given A, B and C are 32-bit floating-point numbers
; Calculate Y2 = (A * B)
;           Y3 = (A * B) * C
;
;
_Cla1Task2:
    MMOV32    MR0, @A           ; Load MR0 with A
    MMOV32    MR1, @B           ; Load MR1 with B
    MMPYF32    MR1, MR1, MR0     ; Multiply A*B
||    MMOV32    MR0, @C           ; and in parallel load MR0 with C
    MMPYF32    MR1, MR1, MR0     ; Multiply (A*B) by C
||    MMOV32    @Y2, MR1         ; and in parallel store A*B
    MMOV32    @Y3, MR1         ; Store the result
    MSTOP                          ; end of task
```

**See also**

[MMPYF32 MRd, MRe, MRf || MMOV32 MRa, mem32](#)  
[MMACF32 MR3, MR2, MRd, MRe, MRf || MMOV32 MRa, mem32](#)

## MMPYF32 MRa, MRb, MRc || MSUBF32 MRd, MRe, MRf

### 32-Bit Floating-Point Multiply with Parallel Subtract

#### Operands

|     |                                                                                                            |
|-----|------------------------------------------------------------------------------------------------------------|
| MRa | CLA floating-point destination register for MMPYF32 (MR0 to MR3)<br>MRa cannot be the same register as MRd |
| MRb | CLA floating-point source register for MMPYF32 (MR0 to MR3)                                                |
| MRC | CLA floating-point source register for MMPYF32 (MR0 to MR3)                                                |
| MRd | CLA floating-point destination register for MSUBF32 (MR0 to MR3)<br>MRd cannot be the same register as MRa |
| MRe | CLA floating-point source register for MSUBF32 (MR0 to MR3)                                                |
| MRf | CLA floating-point source register for MSUBF32 (MR0 to MR3)                                                |

#### Opcode

```
LSW: 0000 ffee ddcc bbaa
MSW: 0111 1010 0100 0000
```

#### Description

Multiply the contents of two floating-point registers with parallel subtraction of two registers.

```
MRa = MRb * MRC;
MRd = MRe - MRf;
```

#### Restrictions

The destination register for the MMPYF32 and the MSUBF32 must be unique. That is, MRa cannot be the same register as MRd.

#### Flags

This instruction modifies the following flags in the MSTF register:

| Flag     | TF | ZF | NF | LUF | LVF |
|----------|----|----|----|-----|-----|
| Modified | No | No | No | Yes | Yes |

The MSTF register flags are modified as follows:

- LUF = 1 if MMPYF32 or MSUBF32 generates an underflow condition.
- LVF = 1 if MMPYF32 or MSUBF32 generates an overflow condition.

#### Pipeline

MMPYF32 and MSUBF32 both complete in a single cycle.

#### Example

```
; Given A, B and C are 32-bit floating-point numbers
; Calculate Y2 = (A * B)
;           Y3 = (A - B)
;
_ClalTask2:
    MMOV32    MR0, @A           ; Load MR0 with A
    MMOV32    MR1, @B           ; Load MR1 with B
    MMPYF32   MR2, MR0, MR1     ; Multiply (A*B)
    || MSUBF32 MR3, MR0, MR1     ; and in parallel Sub (A-B)
    MMOV32    @Y2, MR2          ; Store A*B
    MMOV32    @Y3, MR3          ; Store A-B
    MSTOP                                ; end of task
```

#### See also

MSUBF32 MRa, MRb, MRc  
MSUBF32 MRd, MRe, MRf || MMOV32 MRa, mem32  
MSUBF32 MRd, MRe, MRf || MMOV32 mem32, MRa

**MNEGF32 MRa, MRb{, CNDF}****Conditional Negation****Operands**

|      |                                                      |
|------|------------------------------------------------------|
| MRa  | CLA floating-point destination register (MR0 to MR3) |
| MRb  | CLA floating-point source register (MR0 to MR3)      |
| CNDF | condition tested                                     |

**Opcode**

```
LSW: 0000 0000 cndf bbaa
MSW: 0111 1010 1000 0000
```

**Description**

```
if (CNDF == true) {MRa = - MRb; }
else {MRa = MRb; }
```

CNDF is one of the following conditions:

| Encode <sup>(1)</sup> | CNDF                | Description                          | MSTF Flags Tested   |
|-----------------------|---------------------|--------------------------------------|---------------------|
| 0000                  | NEQ                 | Not equal to zero                    | ZF == 0             |
| 0001                  | EQ                  | Equal to zero                        | ZF == 1             |
| 0010                  | GT                  | Greater than zero                    | ZF == 0 AND NF == 0 |
| 0011                  | GEQ                 | Greater than or equal to zero        | NF == 0             |
| 0100                  | LT                  | Less than zero                       | NF == 1             |
| 0101                  | LEQ                 | Less than or equal to zero           | ZF == 1 OR NF == 1  |
| 1010                  | TF                  | Test flag set                        | TF == 1             |
| 1011                  | NTF                 | Test flag not set                    | TF == 0             |
| 1100                  | LU                  | Latched underflow                    | LUF == 1            |
| 1101                  | LV                  | Latched overflow                     | LVF == 1            |
| 1110                  | UNC                 | Unconditional                        | None                |
| 1111                  | UNCF <sup>(2)</sup> | Unconditional with flag modification | None                |

(1) Values not shown are reserved.

(2) This is the default operation if no CNDF field is specified. This condition will allow the ZF, and NF flags to be modified when a conditional operation is executed. All other conditions will not modify these flags.

**Flags**

This instruction modifies the following flags in the MSTF register:

| Flag     | TF | ZF  | NF  | LUF | LVF |
|----------|----|-----|-----|-----|-----|
| Modified | No | Yes | Yes | No  | No  |

**Pipeline**

This is a single-cycle instruction.

## MNEGF32 MRa, MRb{, CNDF} (continued)

### Conditional Negation

#### Example 1

```
; Show the basic operation of MNEGF32
;
MMOVIZ    MR0, #5.0      ; MR0 = 5.0 (0x40A00000)
MMOVIZ    MR1, #4.0      ; MR1 = 4.0 (0x40800000)
MMOVIZ    MR2, #-1.5     ; MR2 = -1.5 (0xBFC00000)
MMPYF32   MR3, MR1, MR2  ; MR3 = -6.0
MMPYF32   MR0, MR0, MR1  ; MR0 = 20.0
MMOVIZ    MR1, #0.0
MCMPIF32  MR3, MR1       ; NF = 1
MNEGF32   MR3, MR3, LT   ; if NF = 1, MR3 = 6.0
MCMPIF32  MR0, MR1       ; NF = 0
MNEGF32   MR0, MR0, GEQ  ; if NF = 0, MR0 = -20.0
```

#### Example 2

```
; Calculate Num/Den using a Newton-Raphson algorithm for 1/Den
; Ye = Estimate(1/X)
; Ye = Ye*(2.0 - Ye*X)
; Ye = Ye*(2.0 - Ye*X)
;
_Cla1Task1:
MMOV32    MR1, @_Den      ; MR1 = Den
MEINVF32  MR2, MR1        ; MR2 = Ye = Estimate(1/Den)
MMPYF32   MR3, MR2, MR1   ; MR3 = Ye*Den
MSUBF32   MR3, #2.0, MR3  ; MR3 = 2.0 - Ye*Den
MMPYF32   MR2, MR2, MR3   ; MR2 = Ye = Ye*(2.0 - Ye*Den)
MMPYF32   MR3, MR2, MR1   ; MR3 = Ye*Den
|| MMOV32  MR0, @_Num      ; MR0 = Num
MSUBF32   MR3, #2.0, MR3  ; MR3 = 2.0 - Ye*Den
MMPYF32   MR2, MR2, MR3   ; MR2 = Ye = Ye*(2.0 - Ye*Den)
|| MMOV32  MR1, @_Den      ; Reload Den To Set Sign
MNEGF32   MR0, MR0, EQ    ; if(Den == 0.0) Change Sign Of Num
MMPYF32   MR0, MR2, MR0   ; MR0 = Y = Ye*Num
MMOV32    @_Dest, MR0     ; Store result
MSTOP
```

#### See also

[MABSF32 MRa, MRb](#)

## MNOP

### No Operation

#### Operands

|      |                                             |
|------|---------------------------------------------|
| none | This instruction does not have any operands |
|------|---------------------------------------------|

#### Opcode

|                          |
|--------------------------|
| LSW: 0000 0000 0000 0000 |
| MSW: 0111 1111 1010 0000 |

#### Description

Do nothing. This instruction is used to fill required pipeline delay slots when other instructions are not available to fill the slots.

#### Flags

This instruction does not modify flags in the MSTF register.

| Flag     | TF | ZF | NF | LUF | LVF |
|----------|----|----|----|-----|-----|
| Modified | No | No | No | No  | No  |

#### Pipeline

This is a single-cycle instruction.

#### Example

```

; X is an array of 32-bit floating-point values
; Find the maximum value in an array X
; and store it in Result
;
_ClalTask1:
    MMOV16    MAR1, #_X          ; Start address
    MUI16TOF32 MR0, @_len        ; Length of the array
    MNOP      ; delay for MAR1 load
    MNOP      ; delay for MAR1 load
    MMOV32    MR1, *MAR1[2]++    ; MR1 = X0
LOOP
    MMOV32    MR2, *MAR1[2]++    ; MR2 = next element
    MMAXF32   MR1, MR2           ; MR1 = MAX(MR1, MR2)
    MADD32    MR0, MR0, #-1.0    ; Decrement the counter
    MCMPPF32  MR0 #0.0           ; Set/clear flags for MBCNDD
    MNOP      ; Too late to affect MBCNDD
    MNOP      ; Too late to affect MBCNDD
    MNOP      ; Too late to affect MBCNDD
    MBCNDD    LOOP, NEQ          ; Branch if not equal to zero
    MMOV32    @_Result, MR1      ; Always executed
    MNOP      ; Pad to separate MBCNDD and MSTOP
    MNOP      ; Pad to separate MBCNDD and MSTOP
    MSTOP     ; End of task

```



## MOR32 MRa, MRb, MRc

### Bitwise OR

#### Operands

|     |                                                      |
|-----|------------------------------------------------------|
| MRa | CLA floating-point destination register (MR0 to MR3) |
| MRb | CLA floating-point source register (MR0 to MR3)      |
| MRc | CLA floating-point source register (MR0 to MR3)      |

#### Opcode

```
LSW: 0000 0000 00cc bbaa
MSW: 0111 1100 1000 0000
```

#### Description

Bitwise OR of MRb with MRc.

```
MARa(31:0) = MARb(31:0) OR MRc(31:0);
```

#### Flags

This instruction modifies the following flags in the MSTF register:

| Flag     | TF | ZF  | NF  | LUF | LVF |
|----------|----|-----|-----|-----|-----|
| Modified | No | Yes | Yes | No  | No  |

The MSTF register flags are modified based on the integer results of the operation.

```
NF = MRa(31);
ZF = 0;
if(MRa(31:0) == 0) { ZF = 1; }
```

#### Pipeline

This is a single-cycle instruction.

#### Example

```
MMOVIZ MR0, #0x5555 ; MR0 = 0x5555AAAA
MMOVXI MR0, #0xAAAA
MMOVIZ MR1, #0x5432 ; MR1 = 0x5432FEDC
MMOVXI MR1, #0xFEDC
; 0101 OR 0101 = 0101 (5)
; 0101 OR 0100 = 0101 (5)
; 0101 OR 0011 = 0111 (7)
; 0101 OR 0010 = 0111 (7)
; 1010 OR 1111 = 1111 (F)
; 1010 OR 1110 = 1110 (E)
; 1010 OR 1101 = 1111 (F)
; 1010 OR 1100 = 1110 (E)
MOR32 MR2, MR1, MR0 ; MR3 = 0x5555FEFE
```

#### See also

[MAND32 MRa, MRb, MRc](#)  
[MXOR32 MRa, MRb, MRc](#)

**MRCNDD {CNDF}****Return Conditional Delayed****Operands**

|      |                     |
|------|---------------------|
| CNDF | optional condition. |
|------|---------------------|

**Opcode**

|                          |
|--------------------------|
| LSW: 0000 0000 0000 0000 |
| MSW: 0111 1001 1010 cndf |

**Description**

If the specified condition is true, then the RPC field of MSTF is loaded into MPC and fetching continues from that location. Otherwise program fetches will continue without the return.

Please refer to the pipeline section for important information regarding this instruction.

|                                           |
|-------------------------------------------|
| <code>if (CNDF == TRUE) MPC = RPC;</code> |
|-------------------------------------------|

CNDF is one of the following conditions:

| Encode <sup>(1)</sup> | CNDF                | Description                          | MSTF Flags Tested   |
|-----------------------|---------------------|--------------------------------------|---------------------|
| 0000                  | NEQ                 | Not equal to zero                    | ZF == 0             |
| 0001                  | EQ                  | Equal to zero                        | ZF == 1             |
| 0010                  | GT                  | Greater than zero                    | ZF == 0 AND NF == 0 |
| 0011                  | GEQ                 | Greater than or equal to zero        | NF == 0             |
| 0100                  | LT                  | Less than zero                       | NF == 1             |
| 0101                  | LEQ                 | Less than or equal to zero           | ZF == 1 OR NF == 1  |
| 1010                  | TF                  | Test flag set                        | TF == 1             |
| 1011                  | NTF                 | Test flag not set                    | TF == 0             |
| 1100                  | LU                  | Latched underflow                    | LUF == 1            |
| 1101                  | LV                  | Latched overflow                     | LVF == 1            |
| 1110                  | UNC                 | Unconditional                        | None                |
| 1111                  | UNCF <sup>(2)</sup> | Unconditional with flag modification | None                |

(1) Values not shown are reserved.

(2) This is the default operation if no CNDF field is specified. This condition will allow the ZF and NF flags to be modified when a conditional operation is executed. All other conditions will not modify these flags.

**Flags**

This instruction does not modify flags in the MSTF register.

| Flag     | TF | ZF | NF | LUF | LVF |
|----------|----|----|----|-----|-----|
| Modified | No | No | No | No  | No  |

**Pipeline**

The MRCNDD instruction by itself is a single-cycle instruction. As shown in [Table 10-17](#), for each return 6 instruction slots are executed; three before the return instruction (d5-d7) and three after the return instruction (d8-d10). The total number of cycles for a return taken or not taken depends on the usage of these slots. That is, the number of cycles depends on how many slots are filled with a MNOP as well as which slots are filled. The effective number of cycles for a return can, therefore, range from 1 to 7 cycles. The number of cycles for a return taken may not be the same as for a return not taken.

## MRCNDD {CNDF} (continued)

### Return Conditional Delayed

Referring to the following code fragment and the pipeline diagrams in [Table 10-17](#) and [Table 10-18](#), the instructions before and after MRCNDD have the following properties:

```

;
;
<Instruction 1>    ; I1 Last instruction that can affect flags for
                  ; the MCCNDD operation
<Instruction 2>    ; I2 Cannot be stop, branch, call or return
<Instruction 3>    ; I3 Cannot be stop, branch, call or return
<Instruction 4>    ; I4 Cannot be stop, branch, call or return
MCCNDD _func, NEQ  ; Call to func if not equal to zero
                  ; Three instructions after MCCNDD are always
                  ; executed whether the call is taken or not
<Instruction 5>    ; I5 Cannot be stop, branch, call or return
<Instruction 6>    ; I6 Cannot be stop, branch, call or return
<Instruction 7>    ; I7 Cannot be stop, branch, call or return
<Instruction 8>    ; I8 The address of this instruction is saved
                  ; in the RPC field of the MSTF register.
                  ; Upon return this value is loaded into MPC
                  ; and fetching continues from this point.
<Instruction 9>    ; I9
<Instruction 10>   ; I10
....
....
func:
<Destination 1>    ; d1 Can be any instruction
<Destination 2>    ; d2
<Destination 3>    ; d3
<Destination 4>    ; d4 Last instruction that can affect flags for
                  ; the MRCNDD operation
<Destination 5>    ; d5 Cannot be stop, branch, call or return
<Destination 6>    ; d6 Cannot be stop, branch, call or return
<Destination 7>    ; d7 Cannot be stop, branch, call or return
MRCNDD NEQ         ; Return to <Instruction 8> if not equal to zero
                  ; Three instructions after MRCNDD are always
                  ; executed whether the return is taken or not
<Destination 8>    ; d8 Cannot be stop, branch, call or return
<Destination 9>    ; d9 Cannot be stop, branch, call or return
<Destination 10>   ; d10 Cannot be stop, branch, call or return
<Destination 11>   ; d11
<Destination 12>   ; d12
....
....
MSTOP
....

```

- **d4**
  - d4 is the last instruction that can effect the CNDF flags for the MRCNDD instruction. The CNDF flags are tested in the D2 phase of the pipeline. That is, a decision is made whether to return or not when MRCNDD is in the D2 phase.
  - There are no restrictions on the type of instruction for d4.
- **d5, d6 and d7**
  - The three instructions proceeding MRCNDD can change MSTF flags but will have no effect on whether the MRCNDD instruction makes the return or not. This is because the flag modification will occur after the D2 phase of the MRCNDD instruction.
  - These instructions must not be the following: MSTOP, MDEBUGSTOP, MBCNDD, MCCNDD or MRCNDD.
- **d8, d9 and d10**
  - The three instructions following MRCNDD are always executed irrespective of whether the return is taken or not.
  - These instructions must not be the following: MSTOP, MDEBUGSTOP, MBCNDD, MCCNDD or MRCNDD.

**MRCNDD {CNDF}** (continued)

**Return Conditional Delayed**
**Table 10-17. Pipeline Activity For MRCNDD, Return Not Taken**

| Instruction | F1     | F2     | D1     | D2     | R1  | R2  | E   | W |
|-------------|--------|--------|--------|--------|-----|-----|-----|---|
| d4          | d4     | d3     | d2     | d1     | l7  | l6  | l5  |   |
| d5          | d5     | d4     | d3     | d2     | d1  | l7  | l6  |   |
| d6          | d6     | d5     | d4     | d3     | d2  | d1  | i7  |   |
| d7          | d7     | d6     | d5     | d4     | d3  | d2  | d1  |   |
| MRCNDD      | MRCNDD | d7     | d6     | d5     | d4  | d3  | d2  |   |
| d8          | d8     | MRCNDD | d7     | d6     | d5  | d4  | d3  |   |
| d9          | d9     | d8     | MRCNDD | d7     | d6  | d5  | d4  |   |
| d10         | d10    | d9     | d8     | MRCNDD | d7  | d6  | d5  |   |
| d11         | d11    | d10    | d9     | d8     | -   | d7  | d6  |   |
| d12         | d12    | d11    | d10    | d9     | d8  | -   | d7  |   |
| etc....     | ....   | d12    | d11    | d10    | d9  | d8  | -   |   |
| ....        | ....   | ....   | d12    | d11    | d10 | d9  | d8  |   |
| ....        | ....   | ....   | ....   | d12    | d11 | d10 | d9  |   |
|             |        |        |        |        | d12 | d11 | d10 |   |
|             |        |        |        |        |     | d12 | d11 |   |
|             |        |        |        |        |     |     | d12 |   |

**Table 10-18. Pipeline Activity For MRCNDD, Return Taken**

| Instruction | F1     | F2     | D1     | D2     | R1  | R2  | E   | W |
|-------------|--------|--------|--------|--------|-----|-----|-----|---|
| d4          | d4     | d3     | d2     | d1     | l7  | l6  | l5  |   |
| d5          | d5     | d4     | d3     | d2     | d1  | l7  | l6  |   |
| d6          | d6     | d5     | d4     | d3     | d2  | d1  | i7  |   |
| d7          | d7     | d6     | d5     | d4     | d3  | d2  | d1  |   |
| MRCNDD      | MRCNDD | d7     | d6     | d5     | d4  | d3  | d2  |   |
| d8          | d8     | MRCNDD | d7     | d6     | d5  | d4  | d3  |   |
| d9          | d9     | d8     | MRCNDD | d7     | d6  | d5  | d4  |   |
| d10         | d10    | d9     | d8     | MRCNDD | d7  | d6  | d5  |   |
| l8          | l8     | d10    | d9     | d8     | -   | d7  | d6  |   |
| l9          | l9     | l8     | d10    | d9     | d8  | -   | d7  |   |
| l10         | l10    | l9     | l8     | d10    | d9  | d8  | -   |   |
| etc....     | ....   | l10    | l9     | l8     | d10 | d9  | d8  |   |
| ....        | ....   |        | l10    | l9     | l8  | d10 | d9  |   |
| ....        | ....   |        |        | l10    | l9  | l8  | d10 |   |
|             |        |        |        |        | l10 | l9  | l8  |   |
|             |        |        |        |        |     | l10 | l9  |   |
|             |        |        |        |        |     |     | l10 |   |

**See also**

[MBCNDD #16BitDest, CNDF](#)  
[MCCNDD 16BitDest, CNDF](#)  
[MMOV32 mem32, MSTF](#)  
[MMOV32 MSTF, mem32](#)

## MSETFLG FLAG, VALUE

### Set or Clear Selected Floating-Point Status Flags

#### Operands

|       |                                                                    |
|-------|--------------------------------------------------------------------|
| FLAG  | 8-bit mask indicating which floating-point status flags to change. |
| VALUE | 8-bit mask indicating the flag value: 0 or 1.                      |

#### Opcode

```
LSW: FFFF FFFF VVVV VVVV
MSW: 0111 1001 1100 0000
```

#### Description

The MSETFLG instruction is used to set or clear selected floating-point status flags in the MSTF register. The FLAG field is an 11-bit value that indicates which flags will be changed. That is, if a FLAG bit is set to 1 it indicates that flag will be changed; all other flags will not be modified. The bit mapping of the FLAG field is:

| 9          | 8        | 7 | 6  | 5        | 4 | 3  | 2  | 1   | 0   |
|------------|----------|---|----|----------|---|----|----|-----|-----|
| RNDF<br>32 | Reserved |   | TF | Reserved |   | ZF | NF | LUF | LVF |

The VALUE field indicates the value the flag should be set to: 0 or 1.

#### Flags

This instruction modifies the following flags in the MSTF register:

| Flag     | TF  | ZF  | NF  | LUF | LVF |
|----------|-----|-----|-----|-----|-----|
| Modified | Yes | Yes | Yes | Yes | Yes |

Any flag can be modified by this instruction. The MEALLOW and RPC fields cannot be modified with this instruction.

#### Pipeline

This is a single-cycle instruction.

#### Example

To make it easier and legible, the assembler accepts a FLAG=VALUE syntax for the MSETFLG operation as shown below:

```
MSETFLG RNDF32=0, TF=0, NF=1; FLAG = 11000100; VALUE = 00XXX1XX;
```

#### See also

[MMOV32 mem32, MSTF](#)  
[MMOV32 MSTF, mem32](#)

## MSTOP

### Stop Task

#### Operands

|      |                                             |
|------|---------------------------------------------|
| none | This instruction does not have any operands |
|------|---------------------------------------------|

#### Opcode

|                          |
|--------------------------|
| LSW: 0000 0000 0000 0000 |
| MSW: 0111 1111 1000 0000 |

#### Description

The MSTOP instruction must be placed to indicate the end of each task. In addition, placing MSTOP in unused memory locations within the CLA program RAM can be useful for debugging and preventing run away CLA code. When MSTOP enters the D2 phase of the pipeline, the MIRUN flag for the task is cleared and the associated interrupt is flagged in the PIE vector table.

There are three special cases that can occur when single-stepping a task such that the MPC reaches the MSTOP instruction.

1. If you are single-stepping or halted in "task A" and "task B" comes in before the MPC reaches the MSTOP, then "task B" will start if you continue to step through the MSTOP instruction. Basically if "task B" is pending before the MPC reaches MSTOP in "task A" then there is no issue in "task B" starting and no special action is required.
2. In this case you have single-stepped or halted in "task A" and the MPC has reached the MSTOP with no tasks pending. If "task B" comes in at this point, it will be flagged in the MIFR register but it may or may not start if you continue to single-step through the MSTOP instruction of "task A". It depends on exactly when the new task comes in. To reliably start "task B" perform a soft reset and reconfigure the MIER bits. Once this is done, you can start single-stepping "task B".
3. Case 2 can be handled slightly differently if there is control over when "task B" comes in (for example using the IACK instruction to start the task). In this case you have single-stepped or halted in "task A" and the MPC has reached the MSTOP with no tasks pending. Before forcing "task B", run free to force the CLA out of the debug state. Once this is done you can force "task B" and continue debugging.

#### Restrictions

The MSTOP instruction cannot be placed 3 instructions before or after a [MBCNDD](#), [MCCNDD](#), or [MRCNDD](#) instruction.

#### Flags

This instruction does not modify flags in the MSTF register.

| Flag     | TF | ZF | NF | LUF | LVF |
|----------|----|----|----|-----|-----|
| Modified | No | No | No | No  | No  |

## MSTOP (continued)

### Stop Task

#### Pipeline

This is a single-cycle instruction. [Table 10-19](#) shows the pipeline behavior of the MSTOP instruction. The MSTOP instruction cannot be placed with 3 instructions of a [MBCNDD](#), [MCCNDD](#), or [MRCNDD](#) instruction.

**Table 10-19. Pipeline Activity For MSTOP**

| Instruction                         | F1    | F2    | D1    | D2    | R1 | R2 | E  | W |
|-------------------------------------|-------|-------|-------|-------|----|----|----|---|
| I1                                  | I1    |       |       |       |    |    |    |   |
| I2                                  | I2    | I1    |       |       |    |    |    |   |
| I3                                  | I3    | I2    | I1    |       |    |    |    |   |
| MSTOP                               | MSTOP | I3    | I2    | I1    |    |    |    |   |
| I4                                  | I4    | MSTOP | I3    | I2    | I1 |    |    |   |
| I5                                  | I5    | I4    | MSTOP | I3    | I2 | I1 |    |   |
| I6                                  | I6    | I5    | I4    | MSTOP | I3 | I2 | I1 |   |
| New Task Arbitrated and Prioritized | -     | -     | -     | -     | -  | I3 | I2 |   |
| New Task Arbitrated and Prioritized | -     | -     | -     | -     | -  | -  | I3 |   |
| I1                                  | I1    | -     | -     | -     | -  | -  | -  | - |
| I2                                  | I2    | I1    | -     | -     | -  | -  | -  | - |
| I3                                  | I3    | I2    | I1    | -     | -  | -  | -  | - |
| I4                                  | I4    | I3    | I2    | I1    | -  | -  | -  | - |
| I5                                  | I5    | I4    | I3    | I2    | I1 | -  | -  | - |
| I6                                  | I6    | I5    | I4    | I3    | I2 | I1 | -  | - |
| I7                                  | I7    | I6    | I5    | I4    | I3 | I2 | I1 | - |
| etc ....                            |       |       |       |       |    |    |    |   |

#### Example

```

; Given A = (int32)1
; B = (int32)2
; C = (int32)-7
;
; Calculate Y2 = A - B - C
_Cla1Task3:
    MMOV32 MR0, @_A      ; MR0 = 1 (0x00000001)
    MMOV32 MR1, @_B      ; MR1 = 2 (0x00000002)
    MMOV32 MR2, @_C      ; MR2 = -7 (0xFFFFFFFF9)
    MSUB32 MR3, MR0, MR1  ; A + B
    MSUB32 MR3, MR3, MR2  ; A + B + C = 6 (0x00000006)
    MMOV32 @_y2, MR3     ; Store y2
    MSTOP                ; End of task

```

#### See also

[MDEBUGSTOP](#)

**MSUB32 MRa, MRb, MRc****32-Bit Integer Subtraction****Operands**

|     |                                                      |
|-----|------------------------------------------------------|
| MRa | CLA floating-point destination register (MR0 to MR3) |
| MRb | CLA floating-point destination register (MR0 to MR3) |
| MRc | CLA floating-point destination register (MR0 to MR3) |

**Opcode**

```
LSW: 0000 0000 00cc bbaa
MSW: 0111 1100 1110 0000
```

**Description**

32-bit integer addition of MRb and MRc.

```
MARa(31:0) = MARb(31:0) - MRc(31:0);
```

**Flags**

This instruction modifies the following flags in the MSTF register:

| Flag     | TF | ZF  | NF  | LUF | LVF |
|----------|----|-----|-----|-----|-----|
| Modified | No | Yes | Yes | No  | No  |

The MSTF register flags are modified as follows:

```
NF = MRa(31);
ZF = 0;
if(MRa(31:0) == 0) { ZF = 1; }
```

**Pipeline**

This is a single-cycle instruction.

**Example**

```
; Given A = (int32)1
; B = (int32)2
; C = (int32)-7
;
;
; Calculate Y2 = A - B - C
;
_Cla1Task3:
    MMOV32 MR0, @_A      ; MR0 = 1 (0x00000001)
    MMOV32 MR1, @_B      ; MR1 = 2 (0x00000002)
    MMOV32 MR2, @_C      ; MR2 = -7 (0xFFFFFFFF9)
    MSUB32 MR3, MR0, MR1  ; A + B
    MSUB32 MR3, MR3, MR2  ; A + B + C = 6 (0x00000006)
    MMOV32 @_y2, MR3      ; Store y2
    MSTOP                 ; End of task
```

**See also**

[MADD32 MRa, MRb, MRc](#)  
[MAND32 MRa, MRb, MRc](#)  
[MASR32 MRa, #SHIFT](#)  
[MLSL32 MRa, #SHIFT](#)  
[MLSR32 MRa, #SHIFT](#)  
[MOR32 MRa, MRb, MRc](#)  
[MXOR32 MRa, MRb, MRc](#)



## MSUBF32 MRa, MRb, MRc

### 32-Bit Floating-Point Subtraction

#### Operands

|     |                                                     |
|-----|-----------------------------------------------------|
| MRa | CLA floating-point destination register (MR0 to R1) |
| MRb | CLA floating-point source register (MR0 to R1)      |
| MRc | CLA floating-point source register (MR0 to R1)      |

#### Opcode

```
LSW: 0000 0000 00cc bbaa
MSW: 0111 1100 0100 0000
```

#### Description

Subtract the contents of two floating-point registers

```
MRa = MRb - MRc;
```

#### Flags

This instruction modifies the following flags in the MSTF register:

| Flag     | TF | ZF | NF | LUF | LVF |
|----------|----|----|----|-----|-----|
| Modified | No | No | No | Yes | Yes |

The MSTF register flags are modified as follows:

- LUF = 1 if MSUBF32 generates an underflow condition.
- LVF = 1 if MSUBF32 generates an overflow condition.

#### Pipeline

This is a single-cycle instruction.

#### Example

```
; Given A, B and C are 32-bit floating-point numbers
; Calculate Y2 = A + B - C
;
_Cla1Task5:
    MMOV32    MR0, @_A      ; Load MR0 with A
    MMOV32    MR1, @_B      ; Load MR1 with B
    MADDF32   MR0, MR1, MR0 ; Add A + B
    || MMOV32  MR1, @_C      ; and in parallel load C
    MSUBF32   MR0, MR0, MR1 ; Subtract C from (A + B)
    MMOV32    @Y, MR0       ; (A+B) - C
    MSTOP                     ; end of task
```

#### See also

[MSUBF32 MRa, #16FHi, MRb](#)  
[MSUBF32 MRd, MRe, MRf || MMOV32 MRa, mem32](#)  
[MSUBF32 MRd, MRe, MRf || MMOV32 mem32, MRa](#)  
[MMPYF32 MRa, MRb, MRc || MSUBF32 MRd, MRe, MRf](#)

## MSUBF32 MRa, #16FHi, MRb

### 32-Bit Floating-Point Subtraction

#### Operands

|        |                                                                                                                                                             |
|--------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| MRa    | CLA floating-point destination register (MR0 to R1)                                                                                                         |
| #16FHi | A 16-bit immediate value that represents the upper 16-bits of an IEEE 32-bit floating-point value. The low 16-bits of the mantissa are assumed to be all 0. |
| MRb    | CLA floating-point source register (MR0 to R1)                                                                                                              |

#### Opcode

```
LSW: I I I I I I I I I I I I I I I I
MSW: 0111 1000 0000 baaa
```

#### Description

Subtract MRb from the floating-point value represented by the immediate operand. Store the result of the addition in MRa.

#16FHi is a 16-bit immediate value that represents the upper 16-bits of an IEEE 32-bit floating-point value. The low 16-bits of the mantissa are assumed to be all 0. #16FHi is most useful for representing constants where the lowest 16-bits of the mantissa are 0. Some examples are 2.0 (0x40000000), 4.0 (0x40800000), 0.5 (0x3F000000), and -1.5 (0xBFC00000). The assembler will accept either a hex or float as the immediate value. That is, the value -1.5 can be represented as #-1.5 or #0xBFC0.

```
MRa = #16FHi:0 - MRb;
```

#### Flags

This instruction modifies the following flags in the MSTF register:

| Flag     | TF | ZF | NF | LUF | LVF |
|----------|----|----|----|-----|-----|
| Modified | No | No | No | Yes | Yes |

The MSTF register flags are modified as follows:

- LUF = 1 if MSUBF32 generates an underflow condition.
- LVF = 1 if MSUBF32 generates an overflow condition.

#### Pipeline

This is a single-cycle instruction.

#### Example

```
; Y = sqrt(X)
; Ye = Estimate(1/sqrt(X));
; Ye = Ye*(1.5 - Ye*Ye*X*0.5)
; Ye = Ye*(1.5 - Ye*Ye*X*0.5)
; Y = X*Ye
;
_Cla1Task3:
  MMOV32    MR0, @_x          ; MR0 = X
  MEISQRTF32 MR1, MR0         ; MR1 = Ye = Estimate(1/sqrt(X))
  MMOV32    MR1, @_x, EQ      ; if(X == 0.0) Ye = 0.0
  MMPYF32   MR3, MR0, #0.5    ; MR3 = X*0.5
  MMPYF32   MR2, MR1, MR3     ; MR2 = Ye*X*0.5
  MMPYF32   MR2, MR1, MR2     ; MR2 = Ye*Ye*X*0.5
  MSUBF32   MR2, #1.5, MR2    ; MR2 = 1.5 - Ye*Ye*X*0.5
  MMPYF32   MR1, MR1, MR2     ; MR1 = Ye = Ye*(1.5 - Ye*Ye*X*0.5)
  MMPYF32   MR2, MR1, MR3     ; MR2 = Ye*X*0.5
  MMPYF32   MR2, MR1, MR2     ; MR2 = Ye*Ye*X*0.5
  MSUBF32   MR2, #1.5, MR2    ; MR2 = 1.5 - Ye*Ye*X*0.5
  MMPYF32   MR1, MR1, MR2     ; MR1 = Ye = Ye*(1.5 - Ye*Ye*X*0.5)
  MMPYF32   MR0, MR1, MR0     ; MR0 = Y = Ye*X
  MMOV32    @_y, MR0         ; Store Y = sqrt(X)
  MSTOP
```

**MSUBF32 MRa, #16FHi, MRb** (continued)

**32-Bit Floating-Point Subtraction**

---

**See also**

[MSUBF32 MRa, MRb, MRc](#)  
[MSUBF32 MRd, MRe, MRf || MMOV32 MRa, mem32](#)  
[MSUBF32 MRd, MRe, MRf || MMOV32 mem32, MRa](#)  
[MMPYF32 MRa, MRb, MRc || MSUBF32 MRd, MRe, MRf](#)

**MSUBF32 MRd, MRe, MRf || MMOV32 MRa, mem32****32-Bit Floating-Point Subtraction with Parallel Move****Operands**

|       |                                                                                                                          |
|-------|--------------------------------------------------------------------------------------------------------------------------|
| MRd   | CLA floating-point destination register (MR0 to MR3) for the MSUBF32 operation<br>MRd cannot be the same register as MRa |
| MRe   | CLA floating-point source register (MR0 to MR3) for the MSUBF32 operation                                                |
| MRf   | CLA floating-point source register (MR0 to MR3) for the MSUBF32 operation                                                |
| MRa   | CLA floating-point destination register (MR0 to MR3) for the MMOV32 operation<br>MRa cannot be the same register as MRd  |
| mem32 | 32-bit memory location accessed using one of the available addressing modes. Source for the MMOV32 operation.            |

**Opcode**

```
LSW: mmmm mmmm mmmm mmmm
MSW: 0010 ffee ddaa addr
```

**Description**

Subtract the contents of two floating-point registers and move from memory to a floating-point register.

```
MRd = MRe - MRf;
MRa = [mem32];
```

**Restrictions**

The destination register for the MSUBF32 and the MMOV32 must be unique. That is, MRa cannot be the same register as MRd.

**Flags**

This instruction modifies the following flags in the MSTF register:

| Flag     | TF | ZF  | NF  | LUF | LVF |
|----------|----|-----|-----|-----|-----|
| Modified | No | Yes | Yes | Yes | Yes |

The MSTF register flags are modified as follows:

- LUF = 1 if MSUBF32 generates an underflow condition.
- LVF = 1 if MSUBF32 generates an overflow condition.

The MMOV32 Instruction will set the NF and ZF flags.

**Pipeline**

Both MSUBF32 and MMOV32 complete in a single cycle.

**Example**

```
NF = MRa(31);
ZF = 0;
if (MRa(30:23) == 0) { ZF = 1; NF = 0; }
```

**See also**

[MSUBF32 MRa, MRb, MRc](#)  
[MSUBF32 MRa, #16FHi, MRb](#)  
[MMPYF32 MRa, MRb, MRc || MSUBF32 MRd, MRe, MRf](#)

**MSUBF32 MRd, MRe, MRf ||MMOV32 mem32, MRa**
**32-Bit Floating-Point Subtraction with Parallel Move**
**Operands**

|       |                                                                                |
|-------|--------------------------------------------------------------------------------|
| MRd   | CLA floating-point destination register (MR0 to MR3) for the MSUBF32 operation |
| MRe   | CLA floating-point source register (MR0 to MR3) for the MSUBF32 operation      |
| MRf   | CLA floating-point source register (MR0 to MR3) for the MSUBF32 operation      |
| mem32 | 32-bit destination memory location for the MMOV32 operation                    |
| MRa   | CLA floating-point source register (MR0 to MR3) for the MMOV32 operation       |

**Opcode**

```
LSW: mmmmm mmmmm mmmmm mmmmm
MSW: 0110 ffee ddaa addr
```

**Description**

Subtract the contents of two floating-point registers and move from a floating-point register to memory.

```
MRd = MRe - MRf;
[mem32] = MRa;
```

**Flags**

This instruction modifies the following flags in the MSTF register:

| Flag     | TF | ZF | NF | LUF | LVF |
|----------|----|----|----|-----|-----|
| Modified | No | No | No | Yes | Yes |

The MSTF register flags are modified as follows:

- LUF = 1 if MSUBF32 generates an underflow condition.
- LVF = 1 if MSUBF32 generates an overflow condition.

**Pipeline**

Both MSUBF32 and MMOV32 complete in a single cycle.

**See also**

[MSUBF32 MRa, MRb, MRc](#)  
[MSUBF32 MRa, #16FHi, MRb](#)  
[MSUBF32 MRd, MRe, MRf || MMOV32 MRa, mem32](#)  
[MMPYF32 MRa, MRb, MRc || MSUBF32 MRd, MRe, MRf](#)

**MSWAPF MRa, MRb {, CNDF}****Conditional Swap****Operands**

|      |                                                   |
|------|---------------------------------------------------|
| MRa  | CLA floating-point register (MR0 to MR3)          |
| MRb  | CLA floating-point register (MR0 to MR3)          |
| CNDF | Optional condition tested based on the MSTF flags |

**Opcode**

```
LSW: 0000 0000 CNDF bbaa
MSW: 0111 1011 0000 0000
```

**Description**

Conditional swap of MRa and MRb.

```
if (CNDF == true) swap MRa and MRb;
```

CNDF is one of the following conditions:

| Encode <sup>(1)</sup> | CNDF                | Description                          | MSTF Flags Tested   |
|-----------------------|---------------------|--------------------------------------|---------------------|
| 0000                  | NEQ                 | Not equal to zero                    | ZF == 0             |
| 0001                  | EQ                  | Equal to zero                        | ZF == 1             |
| 0010                  | GT                  | Greater than zero                    | ZF == 0 AND NF == 0 |
| 0011                  | GEQ                 | Greater than or equal to zero        | NF == 0             |
| 0100                  | LT                  | Less than zero                       | NF == 1             |
| 0101                  | LEQ                 | Less than or equal to zero           | ZF == 1 OR NF == 1  |
| 1010                  | TF                  | Test flag set                        | TF == 1             |
| 1011                  | NTF                 | Test flag not set                    | TF == 0             |
| 1100                  | LU                  | Latched underflow                    | LUF == 1            |
| 1101                  | LV                  | Latched overflow                     | LVF == 1            |
| 1110                  | UNC                 | Unconditional                        | None                |
| 1111                  | UNCF <sup>(2)</sup> | Unconditional with flag modification | None                |

(1) Values not shown are reserved.

(2) This is the default operation if no CNDF field is specified. This condition will allow the ZF and NF flags to be modified when a conditional operation is executed. All other conditions will not modify these flags.

**Flags**

This instruction modifies the following flags in the MSTF register:

| Flag     | TF | ZF | NF | LUF | LVF |
|----------|----|----|----|-----|-----|
| Modified | No | No | No | No  | No  |

No flags affected

**Pipeline**

This is a single-cycle instruction.

## MSWAPF MRa, MRb {, CNDF} (continued)

### Conditional Swap

#### Example

```
; X is an array of 32-bit floating-point values
; and has len elements. Find the maximum value in
; the array and store it in Result
;
; Note: MCMPPF32 and MSWAPF can be replaced by MMAXF32
;
_Cla1Task1:
    MMOV16    MAR1, # X          ; Start address
    MUI16TOF32 MR0, @_len       ; Length of the array
    MNOP      ; delay for MAR1 load
    MNOP      ; delay for MAR1 load
    MMOV32    MR1, *MAR1[2]++    ; MR1 = X0
LOOP
    MMOV32    MR2, *MAR1[2]++    ; MR2 = next element
    MCMPPF32  MR2, MR1           ; Compare MR2 with MR1
    MSWAPF    MR1, MR2, GT       ; MR1 = MAX(MR1, MR2)
    MADD32    MR0, MR0, #-1.0    ; Decrement the counter
    MCMPPF32  MR0, #0.0         ; Set/clear flags for MBCNDD
    MNOP
    MNOP
    MNOP
    MBCNDD    LOOP, NEQ         ; Branch if not equal to zero
    MMOV32    @_Result, MR1     ; Always executed
    MNOP      ; Always executed
    MNOP      ; Always executed
    MSTOP     ; End of task
```

**MTESTTF CNDF****Test MSTF Register Flag Condition****Operands**

|      |                                       |
|------|---------------------------------------|
| CNDF | condition to test based on MSTF flags |
|------|---------------------------------------|

**Opcode**

```
LSW: 0000 0000 0000 cndf
MSW: 0111 1111 0100 0000
```

**Description**

Test the CLA floating-point condition and if true, set the MSTF[TF] flag. If the condition is false, clear the MSTF[TF] flag. This is useful for temporarily storing a condition for later use.

```
if (CNDF == true) TF = 1;
else TF = 0;
```

CNDF is one of the following conditions:

| Encode <sup>(1)</sup> | CNDF                | Description                          | MSTF Flags Tested   |
|-----------------------|---------------------|--------------------------------------|---------------------|
| 0000                  | NEQ                 | Not equal to zero                    | ZF == 0             |
| 0001                  | EQ                  | Equal to zero                        | ZF == 1             |
| 0010                  | GT                  | Greater than zero                    | ZF == 0 AND NF == 0 |
| 0011                  | GEQ                 | Greater than or equal to zero        | NF == 0             |
| 0100                  | LT                  | Less than zero                       | NF == 1             |
| 0101                  | LEQ                 | Less than or equal to zero           | ZF == 1 OR NF == 1  |
| 1010                  | TF                  | Test flag set                        | TF == 1             |
| 1011                  | NTF                 | Test flag not set                    | TF == 0             |
| 1100                  | LU                  | Latched underflow                    | LUF == 1            |
| 1101                  | LV                  | Latched overflow                     | LVF == 1            |
| 1110                  | UNC                 | Unconditional                        | None                |
| 1111                  | UNCF <sup>(2)</sup> | Unconditional with flag modification | None                |

(1) Values not shown are reserved.

(2) This is the default operation if no CNDF field is specified. This condition will allow the ZF and NF flags to be modified when a conditional operation is executed. All other conditions will not modify these flags.

**Flags**

This instruction modifies the following flags in the MSTF register:

| Flag     | TF  | ZF | NF | LUF | LVF |
|----------|-----|----|----|-----|-----|
| Modified | Yes | No | No | No  | No  |

```
TF = 0;
if (CNDF == true) TF = 1;
```

Note: If (CNDF == UNC or UNCF), the TF flag will be set to 1.

**Pipeline**

This is a single-cycle instruction.



## MTESTTF CNDF (continued)

### Test MSTF Register Flag Condition

#### Example

```
; if (State == 0.1)
;   RampState = RampState || RAMPMASK
; else if (State == 0.01)
;   CoastState = CoastState || COASTMASK
; else
;   SteadyState = SteadyState || STEADYMASK
;
_ClalTask2:
  MMOV32    MR0, @ State
  MCMPF32   MR0, #0.1      ; Affects flags for 1st MBCNDD (A)
  MCMPF32   MR0, #0.01     ; Check used by 2nd MBCNDD (B)
  MTESTTF   EQ             ; Store EQ flag in TF for 2nd MBCNDD (B)
  MNOP
  MBCNDD    _Skip1, NEQ     ; (A) If State != 0.1, go to Skip1
  MMOV32    MR1, @ RampState ; Always executed
  MMOVXI    MR2, #RAMPMASK  ; Always executed
  MOR32     MR1, MR2        ; Always executed
  MMOV32    @_RampState, MR1 ; Execute if (A) branch not taken
  MSTOP     ; end of task if (A) branch not taken
_Skip1:
  MMOV32    MR3, @ SteadyState
  MMOVXI    MR2, #STEADYMASK
  MOR32     MR3, MR2
  MBCNDD    _Skip2, NTF     ; (B) if State != .01, go to Skip2
  MMOV32    MR1, @_CoastState ; Always executed
  MMOVXI    MR2, #COASTMASK  ; Always executed
  MOR32     MR1, MR2        ; Always executed
  MMOV32    @_CoastState, MR1 ; Execute if (B) branch not taken
  MSTOP     ; end of task if (B) branch not taken
_Skip2:
  MMOV32    @_SteadyState, MR3 ; Executed if (B) branch taken
  MSTOP
```

## MUI16TOF32 MRa, mem16

### Convert Unsigned 16-Bit Integer to 32-Bit Floating-Point Value

#### Operands

|       |                                                      |
|-------|------------------------------------------------------|
| MRa   | CLA floating-point destination register (MR0 to MR3) |
| mem16 | 16-bit source memory location                        |

#### Opcode

```
LSW: mmmm mmmm mmmm mmmm
MSW: 0111 0101 01aa addr
```

#### Description

When converting F32 to I16/UI16 data format, the MF32TOI16/UI16 operation truncates to zero while the MF32TOI16R/UI16R operation will round to nearest (even) value.

```
MRa = UI16TOF32[mem16];
```

#### Flags

This instruction does not affect any flags:

| Flag     | TF | ZF | NF | LUF | LVF |
|----------|----|----|----|-----|-----|
| Modified | No | No | No | No  | No  |

#### Pipeline

This is a single-cycle instruction.

#### See also

[MF32TOI16 MRa, MRb](#)  
[MF32TOI16R MRa, MRb](#)  
[MF32TOUI16 MRa, MRb](#)  
[MF32TOUI16R MRa, MRb](#)  
[MI16TOF32 MRa, MRb](#)  
[MI16TOF32 MRa, mem16](#)  
[MUI16TOF32 MRa, MRb](#)

## MUI16TOF32 MRa, MRb

### Convert Unsigned 16-Bit Integer to 32-Bit Floating-Point Value

#### Operands

|     |                                                      |
|-----|------------------------------------------------------|
| MRa | CLA floating-point destination register (MR0 to MR3) |
| MRb | CLA floating-point source register (MR0 to MR3)      |

#### Opcode

```
LSW: 0000 0000 0000 bbaa
MSW: 0111 1110 1110 0000
```

#### Description

Convert an unsigned 16-bit integer to a 32-bit floating-point value. When converting float32 to I16/UI16 data format, the MF32TOI16/UI16 operation truncates to zero while the MF32TOI16R/UI16R operation will round to nearest (even) value.

```
MRa = UI16TOF32[MRb];
```

#### Flags

This instruction does not affect any flags:

| Flag     | TF | ZF | NF | LUF | LVF |
|----------|----|----|----|-----|-----|
| Modified | No | No | No | No  | No  |

#### Pipeline

This is a single-cycle instruction.

#### Example

```
MMOVXI MR1, #0x800F ; MR1(15:0) = 32783 (0x800F)
MUI16TOF32 MR0, MR1 ; MR0 = UI16TOF32 (MR1(15:0))
                      ; = 32783.0 (0x47000F00)
```

#### See also

[MF32TOI16 MRa, MRb](#)  
[MF32TOI16R MRa, MRb](#)  
[MF32TOUI16 MRa, MRb](#)  
[MF32TOUI16R MRa, MRb](#)  
[MI16TOF32 MRa, MRb](#)  
[MI16TOF32 MRa, mem16](#)  
[MUI16TOF32 MRa, mem16](#)

**MUI32TOF32 MRa, mem32****Convert Unsigned 32-Bit Integer to 32-Bit Floating-Point Value****Operands**

|       |                                                                             |
|-------|-----------------------------------------------------------------------------|
| MRa   | CLA floating-point destination register (MR0 to MR3)                        |
| mem32 | 32-bit memory location accessed using one of the available addressing modes |

**Opcode**

```
LSW: mmmmm mmmmm mmmmm mmmmm
MSW: 0111 0100 10aa addr
```

**Description**

```
MRa = UI32TOF32[mem32];
```

**Flags**

This instruction does not affect any flags:

| Flag     | TF | ZF | NF | LUF | LVF |
|----------|----|----|----|-----|-----|
| Modified | No | No | No | No  | No  |

**Pipeline**

This is a single-cycle instruction.

**Example**

```
; Given x2, m2 and b2 are Uint32 numbers:
;
; x2 = Uint32(2) = 0x00000002
; m2 = Uint32(1) = 0x00000001
; b2 = Uint32(3) = 0x00000003
;
; Calculate y2 = x2 * m2 + b2
;
_Cla1Task1:
    MUI32TOF32    MR0, @_m2      ; MR0 = 1.0 (0x3F800000)
    MUI32TOF32    MR1, @_x2      ; MR1 = 2.0 (0x40000000)
    MUI32TOF32    MR2, @_b2      ; MR2 = 3.0 (0x40400000)
    MPYF32        MR3, MR0, MR1  ; M*X
    MADDF32        MR3, MR2, MR3  ; Y=MX+B = 5.0 (0x40A00000)
    MF32TOUI32    MR3, MR3       ; Y = Uint32(5.0) = 0x00000005
    MMOV32        @_y2, MR3      ; store result
    MSTOP
```

**See also**

[MF32TOI32 MRa, MRb](#)  
[MF32TOUI32 MRa, MRb](#)  
[MI32TOF32 MRa, mem32](#)  
[MI32TOF32 MRa, MRb](#)  
[MUI32TOF32 MRa, MRb](#)

## MUI32TOF32 MRa, MRb

### Convert Unsigned 32-Bit Integer to 32-Bit Floating-Point Value

#### Operands

|     |                                                      |
|-----|------------------------------------------------------|
| MRa | CLA floating-point destination register (MR0 to MR3) |
| MRb | CLA floating-point source register (MR0 to MR3)      |

#### Opcode

```
LSW: 0000 0000 0000 bbaa
MSW: 0111 1101 1100 0000
```

#### Description

```
MRa = UI32TOF32 [MRb];
```

#### Flags

This instruction does not affect any flags:

| Flag     | TF | ZF | NF | LUF | LVF |
|----------|----|----|----|-----|-----|
| Modified | No | No | No | No  | No  |

#### Pipeline

This is a single-cycle instruction.

#### Example

```
MMOVIZ    MR3, #0x8000 ; MR3(31:16) = 0x8000
MMOVXI    MR3, #0x1111 ; MR3(15:0) = 0x1111
           ; MR3 = 2147488017
MUI32TOF32 MR3, MR3    ; MR3 = MUI32TOF32 (MR3) = 2147488017.0 (0x4F000011)
```

#### See also

[MF32TOI32 MRa, MRb](#)  
[MF32TOUI32 MRa, MRb](#)  
[MI32TOF32 MRa, mem32](#)  
[MI32TOF32 MRa, MRb](#)  
[MUI32TOF32 MRa, mem32](#)

## MXOR32 MRa, MRb, MRc

### Bitwise Exclusive Or

#### Operands

|     |                                                      |
|-----|------------------------------------------------------|
| MRa | CLA floating-point destination register (MR0 to MR3) |
| MRb | CLA floating-point source register (MR0 to MR3)      |
| MRc | CLA floating-point source register (MR0 to MR3)      |

#### Opcode

```
LSW: 0000 0000 00cc bbaa
MSW: 0111 1100 1010 0000
```

#### Description

Bitwise XOR of MRb with MRc.

```
MARa(31:0) = MARb(31:0) XOR MRc(31:0);
```

#### Flags

This instruction modifies the following flags in the MSTF register:

| Flag     | TF | ZF  | NF  | LUF | LVF |
|----------|----|-----|-----|-----|-----|
| Modified | No | Yes | Yes | No  | No  |

The MSTF register flags are modified based on the integer results of the operation.

```
NF = MRa(31);
ZF = 0;
if(MRa(31:0) == 0) { ZF = 1; }
```

#### Pipeline

This is a single-cycle instruction.

#### Example

```
MMOVIZ MR0, #0x5555 ; MR0 = 0x5555AAAA
MMOVXI MR0, #0xAAAA
MMOVIZ MR1, #0x5432 ; MR1 = 0x5432FEDC
MMOVXI MR1, #0xFEDC
; 0101 XOR 0101 = 0000 (0)
; 0101 XOR 0100 = 0001 (1)
; 0101 XOR 0011 = 0110 (6)
; 0101 XOR 0010 = 0111 (7)
; 1010 XOR 1111 = 0101 (5)
; 1010 XOR 1110 = 0100 (4)
; 1010 XOR 1101 = 0111 (7)
; 1010 XOR 1100 = 0110 (6)
MXOR32 MR2, MR1, MR0 ; MR3 = 0x01675476
```

#### See also

[MAND32 MRa, MRb, MRc](#)  
[MOR32 MRa, MRb, MRc](#)

## 10.7 CLA Registers

This section describes the Control Law Accelerator registers.

### 10.7.1 Register Memory Mapping

[Table 10-20](#) lists the CLA module control and status register set.

**Table 10-20. CLA Module Control and Status Register Set**

| Acronym                                   | Offset | Size<br>(x16) | EALLOW | DCSM <sup>(1)</sup><br>Protected | Name                                          | Section                           |
|-------------------------------------------|--------|---------------|--------|----------------------------------|-----------------------------------------------|-----------------------------------|
| <b>Task Interrupt Vectors</b>             |        |               |        |                                  |                                               |                                   |
| MVECT1                                    | 0x0000 | 1             | Yes    | Yes                              | Task 1 Interrupt Vector                       | <a href="#">Section 10.7.2.1</a>  |
| MVECT2                                    | 0x0001 | 1             | Yes    | Yes                              | Task 2 Interrupt Vector                       | <a href="#">Section 10.7.2.1</a>  |
| MVECT3                                    | 0x0002 | 1             | Yes    | Yes                              | Task 3 Interrupt Vector                       | <a href="#">Section 10.7.2.1</a>  |
| MVECT4                                    | 0x0003 | 1             | Yes    | Yes                              | Task 4 Interrupt Vector                       | <a href="#">Section 10.7.2.1</a>  |
| MVECT5                                    | 0x0004 | 1             | Yes    | Yes                              | Task 5 Interrupt Vector                       | <a href="#">Section 10.7.2.1</a>  |
| MVECT6                                    | 0x0005 | 1             | Yes    | Yes                              | Task 6 Interrupt Vector                       | <a href="#">Section 10.7.2.1</a>  |
| MVECT7                                    | 0x0006 | 1             | Yes    | Yes                              | Task 7 Interrupt Vector                       | <a href="#">Section 10.7.2.1</a>  |
| MVECT8                                    | 0x0007 | 1             | Yes    | Yes                              | Task 8 Interrupt Vector                       | <a href="#">Section 10.7.2.1</a>  |
| <b>Configuration Registers</b>            |        |               |        |                                  |                                               |                                   |
| MCTL                                      | 0x0010 | 1             | Yes    | Yes                              | Control Register                              | <a href="#">Section 10.7.3.1</a>  |
| MMEMCFG                                   | 0x0011 | 1             | Yes    | Yes                              | Memory Configuration Register                 | <a href="#">Section 10.7.3.2</a>  |
| MPISRCSEL1                                | 0x0014 | 2             | Yes    | Yes                              | Peripheral Interrupt Source Select 1 Register | <a href="#">Section 10.7.3.3</a>  |
| MIFR                                      | 0x0020 | 1             | Yes    | Yes                              | Interrupt Flag Register                       | <a href="#">Section 10.7.3.4</a>  |
| MIOVF                                     | 0x0021 | 1             | Yes    | Yes                              | Interrupt Overflow Flag Register              | <a href="#">Section 10.7.3.5</a>  |
| MIFRC                                     | 0x0022 | 1             | Yes    | Yes                              | Interrupt Force Register                      | <a href="#">Section 10.7.3.6</a>  |
| MICLR                                     | 0x0023 | 1             | Yes    | Yes                              | Interrupt Flag Clear Register                 | <a href="#">Section 10.7.3.7</a>  |
| MICLROVF                                  | 0x0024 | 1             | Yes    | Yes                              | Interrupt Overflow Flag Clear Register        | <a href="#">Section 10.7.3.8</a>  |
| MIER                                      | 0x0025 | 1             | Yes    | Yes                              | Interrupt Enable Register                     | <a href="#">Section 10.7.3.9</a>  |
| MIRUN                                     | 0x0026 | 1             | Yes    | Yes                              | Interrupt Run Status Register                 | <a href="#">Section 10.7.3.10</a> |
| <b>Execution Registers <sup>(2)</sup></b> |        |               |        |                                  |                                               |                                   |
| MPC                                       | 0x0028 | 1             | -      | Yes                              | CLA Program Counter                           | <a href="#">Section 10.7.4.1</a>  |
| MAR0                                      | 0x0029 | 1             | -      | Yes                              | CLA Auxiliary Register 0                      | -                                 |
| MAR1                                      | 0x002A | 1             | -      | Yes                              | CLA Auxiliary Register 1                      | -                                 |
| MSTF                                      | 0x002E | 2             | -      | Yes                              | CLA Floating-Point Status Register            | <a href="#">Section 10.7.4.2</a>  |
| MR0                                       | 0x0030 | 2             | -      | Yes                              | CLA Floating-Point Result Register 0          | -                                 |
| MR1                                       | 0x0034 | 2             | -      | Yes                              | CLA Floating-Point Result Register 1          | -                                 |
| MR2                                       | 0x0038 | 2             | -      | Yes                              | CLA Floating-Point Result Register 2          | -                                 |
| MR3                                       | 0x003C | 2             | -      | Yes                              | CLA Floating-Point Result Register 3          | -                                 |

(1) The CLA is protected by Zone 1 code security only.

(2) The main C28x CPU only has read access to the CLA execution registers for debug purposes. The main CPU cannot perform CPU or debugger writes to these registers.

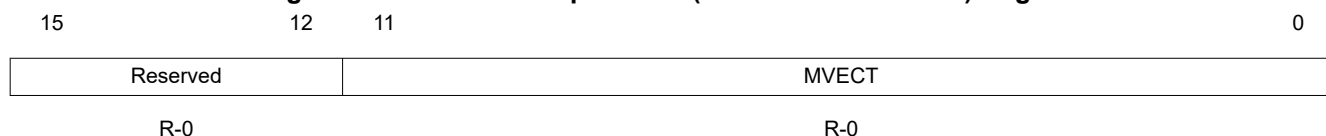
## 10.7.2 Task Interrupt Vector Registers

Each CLA interrupt has its own interrupt vector (MVECT1 to MVECT8). This interrupt vector points to the first instruction of the associated task. When a task begins, the CLA will start fetching instructions at the location indicated by the appropriate MVECT register .

### 10.7.2.1 Task Interrupt Vector (MVECT1/2/3/4/5/6/7/8) Register

The task interrupt vector registers (MVECT1/2/3/4/5/6/7/8) are shown in [Section 10.7.2.1](#) and described in [Figure 10-2](#).

**Figure 10-2. Task Interrupt Vector (MVECT1/2/3/4/5/6/7/8) Register**



LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 10-21. Task Interrupt Vector (MVECT1/2/3/4/5/6/7/8) Field Descriptions**

| Bits  | Name     | Value       | Description <sup>(1)</sup>                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|-------|----------|-------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-12 | Reserved |             | Any writes to these bit(s) must always have a value of 0.                                                                                                                                                                                                                                                                                                                                                                                                                        |
| 11-0  | MVECT    | 0000 - 0FFF | Offset of the first instruction in the associated task from the start of CLA program space. The CLA will begin instruction fetches from this location when the specific task begins.<br><br>For example: If CLA program memory begins at CPU address 0x009000 and the code for task 5 begins at CPU address 0x009120, then MVECT5 should be initialized with 0x0120.<br><br>There is one MVECT register per task. Interrupt 1 uses MVECT1, interrupt 2 uses MVECT2 and so forth. |

(1) These registers are protected by EALLOW and the dual code security module.



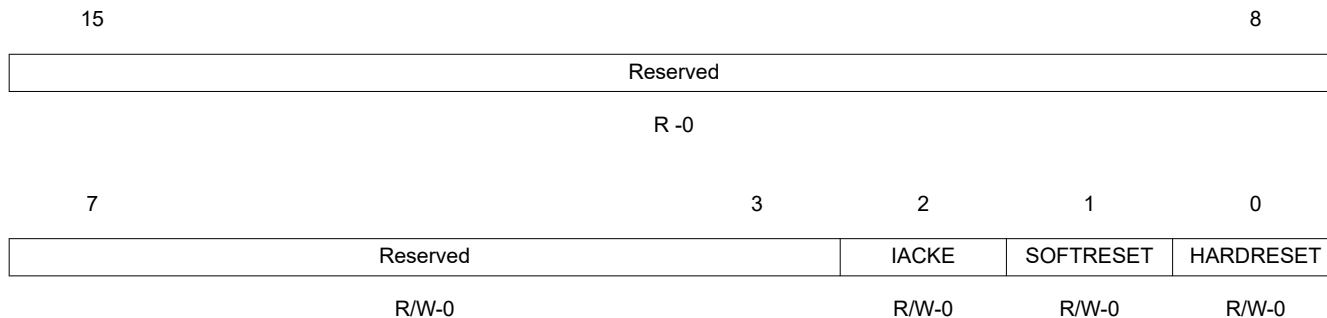
### 10.7.3 Configuration Registers

The configuration registers are described here.

#### 10.7.3.1 Control Register (MCTL)

The configuration control register (MCTL) is shown in [Figure 10-3](#) and described in [Table 10-22](#).

**Figure 10-3. Control Register (MCTL)**



LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 10-22. Control Register (MCTL) Field Descriptions**

| Bits | Name      | Value                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            | Description <sup>(1)</sup>                                |
|------|-----------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------|
| 15-3 | Reserved  |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  | Any writes to these bit(s) must always have a value of 0. |
| 2    | IACKE     | <div>0The CLA ignores the IACK instruction. (default)</div> <div>1Enable the main CPU to use the IACK #16bit instruction to set MIFRC bits in the same manner as writing to the MIFRC register. Each bit in the operand, #16bit, corresponds to a bit in the MIFRC register. Using IACK has the advantage of not having to first set the EALLOW bit. This allows the main CPU to efficiently trigger a CLA task through software.</div> <div>Examples     IACK #0x0001             Write a 1 to MIFRC bit 0 to force task 1</div> <div>                 IACK #0x0003             Write a 1 to MIFRC bit 0 and 1 to force task 1 and task 2</div> |                                                           |
| 1    | SOFTRESET | <div>0This bit always reads back 0 and writes of 0 are ignored.</div> <div>1Writing a 1 will cause a soft reset of the CLA. This will stop the current task, clear the MIRUN flag and clear all bits in the MIER register. After a soft reset you must wait at least 1 SYSCLKOUT cycle before reconfiguring the MIER bits. If these two operations are done back-to-back then the MIER bits will not get set.</div>                                                                                                                                                                                                                              |                                                           |
| 0    | HARDRESET | <div>0This bit always reads back 0 and writes of 0 are ignored.</div> <div>1Writing a 1 will cause a hard reset of the CLA. This will set all CLA registers to their default state.</div>                                                                                                                                                                                                                                                                                                                                                                                                                                                        |                                                           |

(1) This register is protected by EALLOW and the dual code security module.

### 10.7.3.2 Memory Configuration Register (MMEMCFG)

The MMEMCFG register is used to map the CLA program and data RAMs to either the CPU or the CLA memory space. Typically mapping of the CLA program and data RAMs occurs only during the initialization process. If after some time the you want to re-map these memories back to CPU space then disable interrupts (MIER) and make sure all tasks have completed by checking the MIRUN register. Allow two SYSCLKOUT cycles between changing the map configuration of these memories and accessing them. Refer to for CLA and CPU access arbitration details.

**Figure 10-4. Memory Configuration Register (MMEMCFG)**

|          |   |       |       |       |          |    |          |       |          |   |
|----------|---|-------|-------|-------|----------|----|----------|-------|----------|---|
| 15       |   |       |       | 11    |          | 10 |          | 9     |          | 8 |
| Reserved |   |       |       |       | RAM2CPUE |    | RAM1CPUE |       | RAM0CPUE |   |
| R-0      |   |       |       |       | R/W-0    |    | R/W-0    |       | R/W-0    |   |
| 7        | 6 | 5     | 4     | 3     |          |    |          | 1     |          | 0 |
| Reserved |   | RAM2E | RAM1E | RAM0E | Reserved |    |          | PROGE |          |   |
| R-0      |   | R/W-0 | R/W-0 | R/W-0 | R-0      |    |          | R/W-0 |          |   |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 10-23. Memory Configuration Register (MMEMCFG) Field Descriptions**

| Bits  | Field    | Value  | Description                                                                                                                                                                                                                                                                                                                                                                                      |
|-------|----------|--------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-11 | Reserved |        | Any writes to these bit(s) must always have a value of 0                                                                                                                                                                                                                                                                                                                                         |
| 10    | RAM2CPUE | 0<br>1 | CLA Data RAM 2 CPU Access Enable Bit:<br>Allow two SYSCLKOUT cycles between changing this bit and accessing the memory<br>RAM2E = 0, CPU data accesses to CLA Data RAM 2 are always allowed<br>RAM2E = 1, CPU data accesses to CLA Data RAM 2 are not allowed<br>RAM2E = 0, CPU data accesses to CLA Data RAM 2 are always allowed<br>RAM2E = 1, CPU data accesses to CLA Data RAM 2 are allowed |
| 9     | RAM1CPUE | 0<br>1 | CLA Data RAM 1 CPU Access Enable Bit:<br>Allow two SYSCLKOUT cycles between changing this bit and accessing the memory<br>RAM1E = 0, CPU data accesses to CLA Data RAM 1 are always allowed<br>RAM1E = 1, CPU data accesses to CLA Data RAM 1 are not allowed<br>RAM1E = 0, CPU data accesses to CLA Data RAM 1 are always allowed<br>RAM1E = 1, CPU data accesses to CLA Data RAM 1 are allowed |
| 8     | RAM0CPUE | 0<br>1 | CLA Data RAM 0 CPU Access Enable Bit:<br>Allow two SYSCLKOUT cycles between changing this bit and accessing the memory<br>RAM0E = 0, CPU data accesses to CLA Data RAM 0 are always allowed<br>RAM0E = 1, CPU data accesses to CLA Data RAM 0 are not allowed<br>RAM0E = 0, CPU data accesses to CLA Data RAM 0 are always allowed<br>RAM0E = 1, CPU data accesses to CLA Data RAM 0 are allowed |
| 7     | Reserved |        | Any writes to these bit(s) must always have a value of 0                                                                                                                                                                                                                                                                                                                                         |
| 6     | RAM2E    | 0<br>1 | CLA Data RAM 2 Enable<br>Allow two SYSCLKOUT cycles between changing this bit and accessing the memory<br>CLA data SARAM block 2 is mapped to the main CPU program and data space. CLA reads will return zero. (default)<br>CLA data SARAM block 2 is mapped to the CLA space. The RAM2CPUE bit determines the CPU access to this memory                                                         |

**Table 10-23. Memory Configuration Register (MMEMCFG) Field Descriptions (continued)**

| Bits | Field    | Value             | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|------|----------|-------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 5    | RAM1E    | <p>0</p> <p>1</p> | <p>CLA Data RAM 1 Enable</p> <p>Allow two SYSCLKOUT cycles between changing this bit and accessing the memory</p> <p>CLA data SARAM block 1 is mapped to the main CPU program and data space. CLA reads will return zero. (default)</p> <p>CLA data SARAM block 1 is mapped to the CLA space. The RAM1CPUE bit determines the CPU access to this memory</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| 4    | RAM0E    | <p>0</p> <p>1</p> | <p>CLA Data RAM 0 Enable</p> <p>Allow two SYSCLKOUT cycles between changing this bit and accessing the memory</p> <p>CLA data SARAM block 0 is mapped to the main CPU program and data space. CLA reads will return zero. (default)</p> <p>CLA data SARAM block 0 is mapped to the CLA space. The RAM0CPUE bit determines the CPU access to this memory</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| 3-1  | Reserved |                   | Any writes to these bit(s) must always have a value of 0                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| 0    | PROGE    | <p>0</p> <p>1</p> | <p>CLA Program Space Enable</p> <p>Allow two SYSCLKOUT cycles between changing this bit and accessing the memory</p> <p>CLA program SARAM is mapped to the main CPU program and data space. If the CLA attempts a program fetch, the result will be the same as an illegal opcode fetch</p> <p>CLA program SARAM is mapped to the CLA program space. The main SPU can only make debug accesses to this block</p> <p>In this state the CLA has higher priority than CPU debug reads. It is, therefore, possible for the CLA to permanently block debug accesses if the CLA is executing in a loop. This might occur when if the CLA code has a bug. To avoid this issue, the program memory will return 0x0000 for CPU debug reads (ignore writes) when the CLA is running. When the CLA is halted or idle then normal CPU debug read and write access can be performed</p> |

### 10.7.3.3 CLA Peripheral Interrupt Source Select 1 Register (MPISRCSEL1)

Each task has specific peripherals that can start it. For example, Task2 can be started by ADCINT2 or EPWM2\_INT. To configure which of the possible peripherals will start a task configure the MPISRCSEL1 register shown in [Figure 10-5](#). Choosing the option "no interrupt source" means that only the main CPU software will be able to start the given task.

It should be noted that a CLA task only triggers on a level transition (a falling edge) of the configured interrupt source. If a peripheral is enabled and an interrupt fires before the CLA is configured, then the CLA will not see the interrupt edge and will not respond. Refer to the initialization order shown in [Section 10.4.2](#).

**Figure 10-5. CLA Peripheral Interrupt Source Select 1 Register (MPISRCSEL1)**

|            |    |    |    |            |    |    |    |
|------------|----|----|----|------------|----|----|----|
| 31         | 28 | 27 | 24 | 23         | 20 | 19 | 16 |
| PERINT8SEL |    |    |    | PERINT7SEL |    |    |    |
| R/W-0      |    |    |    | R/W-0      |    |    |    |
| 15         | 12 | 11 | 8  | 7          | 4  | 3  | 0  |
| PERINT4SEL |    |    |    | PERINT3SEL |    |    |    |
| R/W-0      |    |    |    | R/W-0      |    |    |    |
| PERINT2SEL |    |    |    | PERINT1SEL |    |    |    |
| R/W-0      |    |    |    | R/W-0      |    |    |    |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 10-24. Peripheral Interrupt Source Select 1 (MPISRCSEL1) Register Field Descriptions**

| Bits  | Field      | Value <sup>(1)</sup> | Description <sup>(2)</sup>                                                                       |
|-------|------------|----------------------|--------------------------------------------------------------------------------------------------|
| 31-28 | PERINT8SEL | 0000                 | Task 8 Peripheral Interrupt Input Select<br>ADCINT8 is the input for interrupt task 8. (default) |
|       |            | 0010                 | CPU Timer 0 is the input for interrupt task 8. (TINT0)                                           |
|       |            | 0100                 | eQEP1 is the input for interrupt task 8. (EQEP1_INT)                                             |
|       |            | 0101                 | No interrupt source for task 8.                                                                  |
|       |            | 1000                 | eCAP1 is the input for interrupt task 8. (ECAP1_INT)                                             |
|       |            | Other                | No interrupt source for task 8.                                                                  |
|       |            |                      |                                                                                                  |
| 27-24 | PERINT7SEL | 0000                 | Task 7 Peripheral Interrupt Input Select<br>ADCINT7 is the input for interrupt task 7. (default) |
|       |            | 0010                 | ePWM7 is the input for interrupt task 7. (EPWM7_INT)                                             |
|       |            | 0100                 | eQEP1 is the input for interrupt task 7. (EQEP1_INT)                                             |
|       |            | 0101                 | No interrupt source for task 7.                                                                  |
|       |            | 1000                 | eCAP1 is the input for interrupt task 7. (ECAP1_INT)                                             |
|       |            | Other                | No interrupt source for task 7.                                                                  |
|       |            |                      |                                                                                                  |
| 23-20 | PERINT6SEL | 0000                 | Task 6 Peripheral Interrupt Input Select<br>ADCINT6 is the input for interrupt task 6. (default) |
|       |            | 0010                 | ePWM6 is the input for interrupt task 6. (EPWM6_INT)                                             |
|       |            | 0100                 | eQEP1 is the input for interrupt task 6. (EQEP1_INT)                                             |
|       |            | 0101                 | No interrupt source for task 6.                                                                  |
|       |            | 1000                 | eCAP1 is the input for interrupt task 6. (ECAP1_INT)                                             |
|       |            | Other                | No interrupt source for task 6.                                                                  |
|       |            |                      |                                                                                                  |

**Table 10-24. Peripheral Interrupt Source Select 1 (MPISRCSEL1) Register Field Descriptions (continued)**

| Bits  | Field      | Value <sup>(1)</sup>                          | Description <sup>(2)</sup>                                                                                                                                                                                                                                                                                                                     |
|-------|------------|-----------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 19-16 | PERINT5SEL | 0000<br>0010<br>0100<br>0101<br>1000<br>Other | Task 5 Peripheral Interrupt Input Select<br>ADCINT5 is the input for interrupt task 5. (default)<br>ePWM5 is the input for interrupt task 5. (EPWM5_INT)<br>eQEP1 is the input for interrupt task 5. (EQEP1_INT)<br>No interrupt source for task 5.<br>eCAP1 is the input for interrupt task 5. (ECAP1_INT)<br>No interrupt source for task 5. |
| 15-12 | PERINT4SEL | 0000<br>0010<br>0100<br>0101<br>1000<br>Other | Task 4 Peripheral Interrupt Input Select<br>ADCINT4 is the input for interrupt task 4. (default)<br>ePWM4 is the input for interrupt task 4. (EPWM4_INT)<br>eQEP1 is the input for interrupt task 4. (EQEP1_INT)<br>No interrupt source for task 4.<br>eCAP1 is the input for interrupt task 4. (ECAP1_INT)<br>No interrupt source for task 4. |
| 11-8  | PERINT3SEL | 0000<br>0010<br>xxx1                          | Task 3 Peripheral Interrupt Input Select<br>ADCINT3 is the input for interrupt task 3. (default)<br>ePWM3 is the input for interrupt task 3. (EPWM3_INT)<br>No interrupt source for task 3.                                                                                                                                                    |
| 7-4   | PERINT2SEL | 0000<br>0010<br>xxx1                          | Task 2 Peripheral Interrupt Input Select<br>ADCINT2 is the input for interrupt task 2. (default)<br>ePWM2 is the input for interrupt task 2. (EPWM2_INT)<br>No interrupt source for task 2.                                                                                                                                                    |
| 3-0   | PERINT1SEL | 0000<br>0010<br>xxx1                          | Task 1 Peripheral Interrupt Input Select<br>ADCINT1 is the input for interrupt task 1. (default)<br>ePWM1 is the input for interrupt task 1. (EPWM1_INT)<br>No interrupt source                                                                                                                                                                |

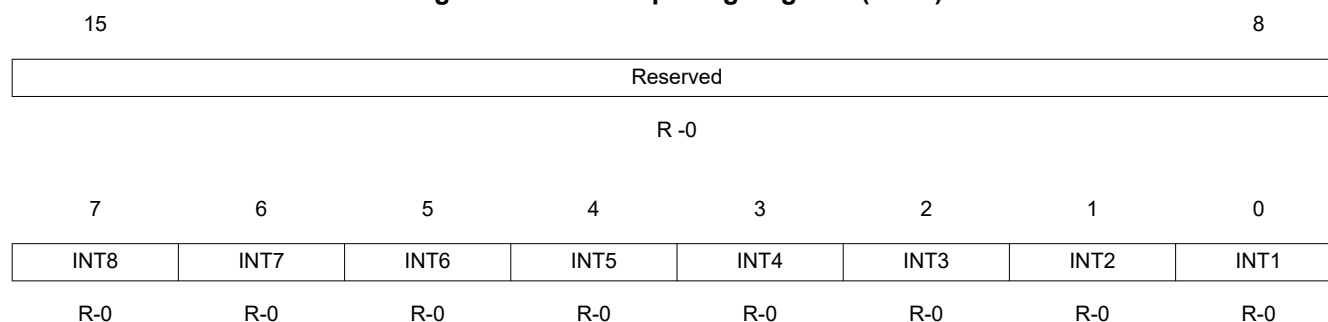
(1) All values not shown are reserved.

(2) This register is protected by EALLOW and the dual code security module.

#### 10.7.3.4 Interrupt Flag Register (MIFR)

Each bit in the interrupt flag register corresponds to a CLA task. The corresponding bit is automatically set when the task request is received from the peripheral interrupt. The bit can also be set by the main CPU writing to the MIFRC register or using the IACK instruction to start the task. To use the IACK instruction to begin a task first enable this feature in the MCTL register. If the bit is already set when a new peripheral interrupt is received, then the corresponding overflow bit will be set in the MIOVF register.

The corresponding MIFR bit is automatically cleared when the task begins execution. This will occur if the interrupt is enabled in the MIER register and no other higher priority task is pending. The bits can also be cleared manually by writing to the MICLR register. Writes to the MIFR register are ignored.

**Figure 10-6. Interrupt Flag Register (MIFR)**

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 10-25. Interrupt Flag Register (MIFR) Field Descriptions**

| Bits | Name     | Value  | Description <sup>(1)</sup>                                                                                                                        |
|------|----------|--------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-8 | Reserved |        | Any writes to these bit(s) must always have a value of 0.                                                                                         |
| 7    | INT8     | 0<br>1 | Task 8 Interrupt Flag<br>A task 8 interrupt is currently not flagged. (default)<br>A task 8 interrupt has been received and is pending execution. |
| 6    | INT7     | 0<br>1 | Task 7 Interrupt Flag<br>A task 7 interrupt is currently not flagged. (default)<br>A task 7 interrupt has been received and is pending execution. |
| 5    | INT6     | 0<br>1 | Task 6 Interrupt Flag<br>A task 6 interrupt is currently not flagged. (default)<br>A task 6 interrupt has been received and is pending execution. |
| 4    | INT5     | 0<br>1 | Task 5 Interrupt Flag<br>A task 5 interrupt is currently not flagged. (default)<br>A task 5 interrupt has been received and is pending execution. |
| 3    | INT4     | 0<br>1 | Task 4 Interrupt Flag<br>A task 4 interrupt is currently not flagged. (default)<br>A task 4 interrupt has been received and is pending execution. |
| 2    | INT3     | 0<br>1 | Task 3 Interrupt Flag<br>A task 3 interrupt is currently not flagged. (default)<br>A task 3 interrupt has been received and is pending execution. |
| 1    | INT2     | 0<br>1 | Task 2 Interrupt Flag<br>A task 2 interrupt is currently not flagged. (default)<br>A task 2 interrupt has been received and is pending execution. |
| 0    | INT1     | 0<br>1 | Task 1 Interrupt Flag<br>A task 1 interrupt is currently not flagged. (default)<br>A task 1 interrupt has been received and is pending execution. |

(1) This register is protected by the dual code security module.

### 10.7.3.5 Interrupt Overflow Flag Register (MIOVF)

Each bit in the overflow flag register corresponds to a CLA task. The bit is set when an interrupt overflow event has occurred for the specific task. An overflow event occurs when the MIFR register bit is already set when a new interrupt is received from a peripheral source. The MIOVF bits are only affected by peripheral interrupt events. They do not respond to a task request by the main CPU IACK instruction or by directly setting MIFR bits. The overflow flag will remain latched and can only be cleared by writing to the overflow flag clear (MICLROVF) register. Writes to the MIOVF register are ignored.

**Figure 10-7. Interrupt Overflow Flag Register (MIOVF)**

|          |  |      |  |      |  |      |  |      |  |      |  |      |  |      |  |
|----------|--|------|--|------|--|------|--|------|--|------|--|------|--|------|--|
| 15       |  |      |  |      |  |      |  | 8    |  |      |  |      |  |      |  |
| Reserved |  |      |  |      |  |      |  |      |  |      |  |      |  |      |  |
| R -0     |  |      |  |      |  |      |  |      |  |      |  |      |  |      |  |
| 7        |  | 6    |  | 5    |  | 4    |  | 3    |  | 2    |  | 1    |  | 0    |  |
| INT8     |  | INT7 |  | INT6 |  | INT5 |  | INT4 |  | INT3 |  | INT2 |  | INT1 |  |
| R-0      |  | R-0  |  | R-0  |  | R-0  |  | R-0  |  | R-0  |  | R-0  |  | R-0  |  |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 10-26. Interrupt Overflow Flag Register (MIOVF) Field Descriptions**

| Bits | Name     | Value  | Description <sup>(1)</sup>                                                                                                             |
|------|----------|--------|----------------------------------------------------------------------------------------------------------------------------------------|
| 15-8 | Reserved |        | Any writes to these bit(s) must always have a value of 0.                                                                              |
| 7    | INT8     | 0<br>1 | Task 8 Interrupt Overflow Flag<br>A task 8 interrupt overflow has not occurred. (default)<br>A task 8 interrupt overflow has occurred. |
| 6    | INT7     | 0<br>1 | Task 7 Interrupt Overflow Flag<br>A task 7 interrupt overflow has not occurred. (default)<br>A task 7 interrupt overflow has occurred. |
| 5    | INT6     | 0<br>1 | Task 6 Interrupt Overflow Flag<br>A task 6 interrupt overflow has not occurred. (default)<br>A task 6 interrupt overflow has occurred. |
| 4    | INT5     | 0<br>1 | Task 5 Interrupt Overflow Flag<br>A task 5 interrupt overflow has not occurred. (default)<br>A task 5 interrupt overflow has occurred. |
| 3    | INT4     | 0<br>1 | Task 4 Interrupt Overflow Flag<br>A task 4 interrupt overflow has not occurred. (default)<br>A task 4 interrupt overflow has occurred. |
| 2    | INT3     | 0<br>1 | Task 3 Interrupt Overflow Flag<br>A task 3 interrupt overflow has not occurred. (default)<br>A task 3 interrupt overflow has occurred. |
| 1    | INT2     | 0<br>1 | Task 2 Interrupt Overflow Flag<br>A task 2 interrupt overflow has not occurred. (default)<br>A task 2 interrupt overflow has occurred. |
| 0    | INT1     | 0<br>1 | Task 1 Interrupt Overflow Flag<br>A task 1 interrupt overflow has not occurred. (default)<br>A task 1 interrupt overflow has occurred. |

(1) This register is protected by the dual code security module.

### 10.7.3.6 Interrupt Force Register (MIFRC)

The interrupt force register can be used by the main CPU to start tasks through software. Writing a 1 to a MIFRC bit will set the corresponding bit in the MIFR register. Writes of 0 are ignored and reads always return 0. The IACK #16bit operation can also be used to start tasks and has the same effect as the MIFRC register. To enable IACK to set MIFR bits you must first set the MCTL[IACK] bit. Using IACK has the advantage of not having to first set the EALLOW bit. This allows the main CPU to efficiently trigger CLA tasks through software.

**Figure 10-8. Interrupt Force Register (MIFRC)**

|          |       |       |       |       |       |       |       |
|----------|-------|-------|-------|-------|-------|-------|-------|
| Reserved |       |       |       |       |       |       |       |
| R -0     |       |       |       |       |       |       |       |
| 7        | 6     | 5     | 4     | 3     | 2     | 1     | 0     |
| INT8     | INT7  | INT6  | INT5  | INT4  | INT3  | INT2  | INT1  |
| R/W-0    | R/W-0 | R/W-0 | R/W-0 | R/W-0 | R/W-0 | R/W-0 | R/W-0 |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 10-27. Interrupt Force Register (MIFRC) Field Descriptions**

| Bits | Name     | Value  | Description <sup>(1)</sup>                                                                                                         |
|------|----------|--------|------------------------------------------------------------------------------------------------------------------------------------|
| 15-8 | Reserved |        | Any writes to these bit(s) must always have a value of 0.                                                                          |
| 7    | INT8     | 0<br>1 | Task 8 Interrupt Force<br>This bit always reads back 0 and writes of 0 have no effect.<br>Write a 1 to force the task 8 interrupt. |
| 6    | INT7     | 0<br>1 | Task 7 Interrupt Force<br>This bit always reads back 0 and writes of 0 have no effect.<br>Write a 1 to force the task 7 interrupt. |
| 5    | INT6     | 0<br>1 | Task 6 Interrupt Force<br>This bit always reads back 0 and writes of 0 have no effect.<br>Write a 1 to force the task 6 interrupt. |
| 4    | INT5     | 0<br>1 | Task 5 Interrupt Force<br>This bit always reads back 0 and writes of 0 have no effect.<br>Write a 1 to force the task 5 interrupt. |
| 3    | INT4     | 0<br>1 | Task 4 Interrupt Force<br>This bit always reads back 0 and writes of 0 have no effect.<br>Write a 1 to force the task 4 interrupt. |
| 2    | INT3     | 0<br>1 | Task 3 Interrupt Force<br>This bit always reads back 0 and writes of 0 have no effect.<br>Write a 1 to force the task 3 interrupt. |
| 1    | INT2     | 0<br>1 | Task 2 Interrupt Force<br>This bit always reads back 0 and writes of 0 have no effect.<br>Write a 1 to force the task 2 interrupt. |
| 0    | INT1     | 0<br>1 | Task 1 Interrupt Force<br>This bit always reads back 0 and writes of 0 have no effect.<br>Write a 1 to force the task 1 interrupt. |

(1) This register is protected by EALLOW and the dual code security module.



### 10.7.3.7 Interrupt Flag Clear Register (MICLR)

Normally bits in the MIFR register are automatically cleared when a task begins. The interrupt flag clear register can be used to instead manually clear bits in the interrupt flag (MIFR) register. Writing a 1 to a MICLR bit will clear the corresponding bit in the MIFR register. Writes of 0 are ignored and reads always return 0.

**Figure 10-9. Interrupt Flag Clear Register (MICLR)**

|          |       |       |       |       |       |       |       |
|----------|-------|-------|-------|-------|-------|-------|-------|
| Reserved |       |       |       |       |       |       |       |
| R -0     |       |       |       |       |       |       |       |
| 7        | 6     | 5     | 4     | 3     | 2     | 1     | 0     |
| INT8     | INT7  | INT6  | INT5  | INT4  | INT3  | INT2  | INT1  |
| R/W-0    | R/W-0 | R/W-0 | R/W-0 | R/W-0 | R/W-0 | R/W-0 | R/W-0 |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 10-28. Interrupt Flag Clear Register (MICLR) Field Descriptions**

| Bits | Name     | Value  | Description <sup>(1)</sup>                                                                                                                   |
|------|----------|--------|----------------------------------------------------------------------------------------------------------------------------------------------|
| 15-8 | Reserved |        | Any writes to these bit(s) must always have a value of 0.                                                                                    |
| 7    | INT8     | 0<br>1 | Task 8 Interrupt Flag Clear<br>This bit always reads back 0 and writes of 0 have no effect.<br>Write a 1 to clear the task 8 interrupt flag. |
| 6    | INT7     | 0<br>1 | Task 7 Interrupt Flag Clear<br>This bit always reads back 0 and writes of 0 have no effect.<br>Write a 1 to clear the task 7 interrupt flag. |
| 5    | INT6     | 0<br>1 | Task 6 Interrupt Flag Clear<br>This bit always reads back 0 and writes of 0 have no effect.<br>Write a 1 to clear the task 6 interrupt flag. |
| 4    | INT5     | 0<br>1 | Task 5 Interrupt Flag Clear<br>This bit always reads back 0 and writes of 0 have no effect.<br>Write a 1 to clear the task 5 interrupt flag. |
| 3    | INT4     | 0<br>1 | Task 4 Interrupt Flag Clear<br>This bit always reads back 0 and writes of 0 have no effect.<br>Write a 1 to clear the task 4 interrupt flag. |
| 2    | INT3     | 0<br>1 | Task 3 Interrupt Flag Clear<br>This bit always reads back 0 and writes of 0 have no effect.<br>Write a 1 to clear the task 3 interrupt flag. |
| 1    | INT2     | 0<br>1 | Task 2 Interrupt Flag Clear<br>This bit always reads back 0 and writes of 0 have no effect.<br>Write a 1 to clear the task 2 interrupt flag. |
| 0    | INT1     | 0<br>1 | Task 1 Interrupt Flag Clear<br>This bit always reads back 0 and writes of 0 have no effect.<br>Write a 1 to clear the task 1 interrupt flag. |

(1) This register is protected by EALLOW and the dual code security module.

### 10.7.3.8 Interrupt Overflow Flag Clear Register (MICLROVF)

Overflow flag bits in the MIOVF register are latched until manually cleared using the MICLROVF register. Writing a 1 to a MICLROVF bit will clear the corresponding bit in the MIOVF register. Writes of 0 are ignored and reads always return 0.

**Figure 10-10. Interrupt Overflow Flag Clear Register (MICLROVF)**

|          |       |       |       |       |       |       |       |
|----------|-------|-------|-------|-------|-------|-------|-------|
| Reserved |       |       |       |       |       |       |       |
| R -0     |       |       |       |       |       |       |       |
| 7        | 6     | 5     | 4     | 3     | 2     | 1     | 0     |
| INT8     | INT7  | INT6  | INT5  | INT4  | INT3  | INT2  | INT1  |
| R/W-0    | R/W-0 | R/W-0 | R/W-0 | R/W-0 | R/W-0 | R/W-0 | R/W-0 |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 10-29. Interrupt Overflow Flag Clear Register (MICLROVF) Field Descriptions**

| Bits | Name     | Value  | Description <sup>(1)</sup>                                                                                                                                     |
|------|----------|--------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-8 | Reserved |        | Any writes to these bit(s) must always have a value of 0.                                                                                                      |
| 7    | INT8     | 0<br>1 | Task 8 Interrupt Overflow Flag Clear<br>This bit always reads back 0 and writes of 0 have no effect.<br>Write a 1 to clear the task 8 interrupt overflow flag. |
| 6    | INT7     | 0<br>1 | Task 7 Interrupt Overflow Flag Clear<br>This bit always reads back 0 and writes of 0 have no effect.<br>Write a 1 to clear the task 7 interrupt overflow flag. |
| 5    | INT6     | 0<br>1 | Task 6 Interrupt Overflow Flag Clear<br>This bit always reads back 0 and writes of 0 have no effect.<br>Write a 1 to clear the task 6 interrupt overflow flag. |
| 4    | INT5     | 0<br>1 | Task 5 Interrupt Overflow Flag Clear<br>This bit always reads back 0 and writes of 0 have no effect.<br>Write a 1 to clear the task 5 interrupt overflow flag. |
| 3    | INT4     | 0<br>1 | Task 4 Interrupt Overflow Flag Clear<br>This bit always reads back 0 and writes of 0 have no effect.<br>Write a 1 to clear the task 4 interrupt overflow flag. |
| 2    | INT3     | 0<br>1 | Task 3 Interrupt Overflow Flag Clear<br>This bit always reads back 0 and writes of 0 have no effect.<br>Write a 1 to clear the task 3 interrupt overflow flag. |
| 1    | INT2     | 0<br>1 | Task 2 Interrupt Overflow Flag Clear<br>This bit always reads back 0 and writes of 0 have no effect.<br>Write a 1 to clear the task 2 interrupt overflow flag. |
| 0    | INT1     | 0<br>1 | Task 1 Interrupt Overflow Flag Clear<br>This bit always reads back 0 and writes of 0 have no effect.<br>Write a 1 to clear the task 1 interrupt overflow flag. |

(1) This register is protected by EALLOW and the dual code security module.

### 10.7.3.9 Interrupt Enable Register (MIER)

Setting the bits in the interrupt enable register (MIER) allow an incoming interrupt or main CPU software to start the corresponding CLA task. Writing a 0 will block the task, but the interrupt request will still be latched in the flag register (MIFLG). Setting the MIER register bit to 0 while the corresponding task is executing will have no effect on the task. The task will continue to run until it hits the MSTOP instruction.

It should be noted that a CLA task only triggers on a level transition (a falling edge) of the configured interrupt source. If a peripheral is enabled and an interrupt fires before the CLA is configured, then the CLA will not see the interrupt edge and will not respond. Refer to the initialization order shown in [Section 10.4.2](#).

When a soft reset is issued, the MIER bits are cleared. There should always be at least a 1 SYSCLKOUT delay between issuing the soft reset and reconfiguring the MIER bits.

**Figure 10-11. Interrupt Enable Register (MIER)**

|          |  |       |  |       |  |       |  |       |  |       |  |       |  |       |  |
|----------|--|-------|--|-------|--|-------|--|-------|--|-------|--|-------|--|-------|--|
| 15       |  |       |  |       |  |       |  | 8     |  |       |  |       |  |       |  |
| Reserved |  |       |  |       |  |       |  |       |  |       |  |       |  |       |  |
| R -0     |  |       |  |       |  |       |  |       |  |       |  |       |  |       |  |
| 7        |  | 6     |  | 5     |  | 4     |  | 3     |  | 2     |  | 1     |  | 0     |  |
| INT8     |  | INT7  |  | INT6  |  | INT5  |  | INT4  |  | INT3  |  | INT2  |  | INT1  |  |
| R/W-0    |  | R/W-0 |  | R/W-0 |  | R/W-0 |  | R/W-0 |  | R/W-0 |  | R/W-0 |  | R/W-0 |  |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 10-30. Interrupt Enable Register (MIER) Field Descriptions**

| Bits | Name     | Value  | Description <sup>(1)</sup>                                                                         |
|------|----------|--------|----------------------------------------------------------------------------------------------------|
| 15-8 | Reserved |        | Any writes to these bit(s) must always have a value of 0.                                          |
| 7    | INT8     | 0<br>1 | Task 8 Interrupt Enable<br>Task 8 interrupt is disabled. (default)<br>Task 8 interrupt is enabled. |
| 6    | INT7     | 0<br>1 | Task 7 Interrupt Enable<br>Task 7 interrupt is disabled. (default)<br>Task 7 interrupt is enabled. |
| 5    | INT6     | 0<br>1 | Task 6 Interrupt Enable<br>Task 6 interrupt is disabled. (default)<br>Task 6 interrupt is enabled. |
| 4    | INT5     | 0<br>1 | Task 5 Interrupt Enable<br>Task 5 interrupt is disabled. (default)<br>Task 5 interrupt is enabled. |
| 3    | INT4     | 0<br>1 | Task 4 Interrupt Enable<br>Task 4 interrupt is disabled. (default)<br>Task 4 interrupt is enabled. |
| 2    | INT3     | 0<br>1 | Task 3 Interrupt Enable<br>Task 3 interrupt is disabled. (default)<br>Task 3 interrupt is enabled. |
| 1    | INT2     | 0<br>1 | Task 2 Interrupt Enable<br>Task 2 interrupt is disabled. (default)<br>Task 2 interrupt is enabled. |

**Table 10-30. Interrupt Enable Register (MIER) Field Descriptions (continued)**

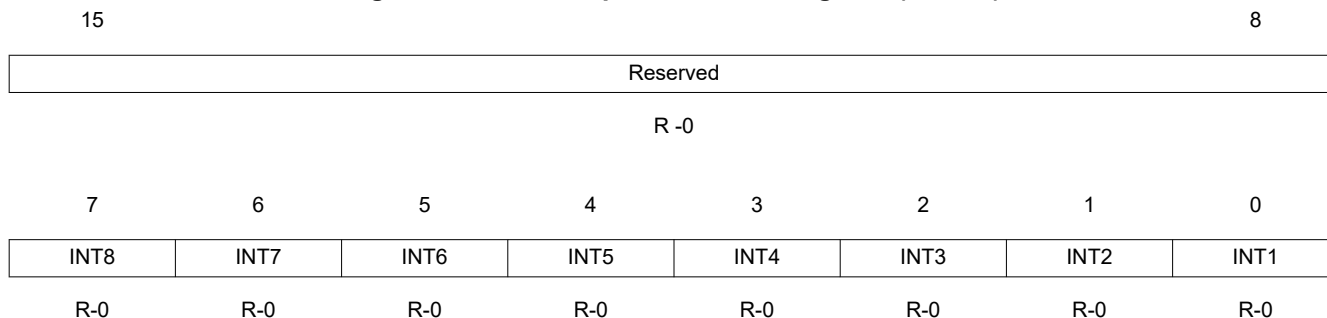
| Bits | Name | Value | Description <sup>(1)</sup>              |
|------|------|-------|-----------------------------------------|
| 0    | INT1 |       | Task 1 Interrupt Enable                 |
|      |      | 0     | Task 1 interrupt is disabled. (default) |
|      |      | 1     | Task 1 interrupt is enabled.            |

(1) This register is protected by EALLOW and the dual code security module.

### 10.7.3.10 Interrupt Run Status Register (MIRUN)

The interrupt run status register (MIRUN) indicates which task is currently executing. Only one MIRUN bit will ever be set to a 1 at any given time. The bit is automatically cleared when the task completes and the respective interrupt is fed to the peripheral interrupt expansion (PIE) block of the device. This lets the main CPU know when a task has completed. The main CPU can stop a currently running task by writing to the MCTL[SOFTRESET] bit. This will clear the MIRUN flag and stop the task. In this case no interrupt will be generated to the PIE.

**Figure 10-12. Interrupt Run Status Register (MIRUN)**



LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 10-31. Interrupt Run Status Register (MIRUN) Field Descriptions**

| Bits | Name     | Value  | Description <sup>(1)</sup>                                                      |
|------|----------|--------|---------------------------------------------------------------------------------|
| 15-8 | Reserved |        | Any writes to these bit(s) must always have a value of 0.                       |
| 7    | INT8     | 0<br>1 | Task 8 Run Status<br>Task 8 is not executing. (default)<br>Task 8 is executing. |
| 6    | INT7     | 0<br>1 | Task 7 Run Status<br>Task 7 is not executing. (default)<br>Task 7 is executing. |
| 5    | INT6     | 0<br>1 | Task 6 Run Status<br>Task 6 is not executing. (default)<br>Task 6 is executing. |
| 4    | INT5     | 0<br>1 | Task 5 Run Status<br>Task 5 is not executing. (default)<br>Task 5 is executing. |
| 3    | INT4     | 0<br>1 | Task 4 Run Status<br>Task 4 is not executing. (default)<br>Task 4 is executing. |
| 2    | INT3     | 0<br>1 | Task 3 Run Status<br>Task 3 is not executing. (default)<br>Task 3 is executing. |
| 1    | INT2     | 0<br>1 | Task 2 Run Status<br>Task 2 is not executing. (default)<br>Task 2 is executing. |
| 0    | INT1     | 0<br>1 | Task 1 Run Status<br>Task 1 is not executing. (default)<br>Task 1 is executing. |

(1) This register is protected by the dual code security module.

## 10.7.4 Execution Registers

The CLA program counter is initialized by the appropriate MVECTx register when an interrupt is received and a task begins execution. The MPC points to the instruction in the decode 2 (D2) stage of the CLA pipeline. After a MSTOP operation, if no other tasks are pending, the MPC will remain pointing to the MSTOP instruction. The MPC register can be read by the main C28x CPU for debug purposes. The main CPU cannot write to MPC.

### 10.7.4.1 Program Counter (MPC)

The MPC register is described in [Figure 10-13](#) and described in [Table 10-32](#).

**Figure 10-13. Program Counter (MPC)**

|          |    |     |   |
|----------|----|-----|---|
| 15       | 12 | 11  | 0 |
| Reserved |    | MPC |   |
| R-0      |    | R-0 |   |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 10-32. Program Counter (MPC) Field Descriptions**

| Bits  | Name     | Value       | Description <sup>(1)</sup>                                                                                                                              |
|-------|----------|-------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-12 | Reserved |             | Any writes to these bit(s) must always have a value of 0.                                                                                               |
| 11-0  | MPC      | 0000 - 0FFF | Points to the instruction currently in the decode 2 phase of the CLA pipeline. The value is the offset from the first address in the CLA program space. |

- (1) This register is protected by the dual code security module. The main CPU can read this register for debug purposes but it can not write to it.

### 10.7.4.2 CLA Status Register (MSTF)

The CLA status register (MSTF) reflects the results of different operations. These are the basic rules for the flags:

- Zero and negative flags are cleared or set based on:
  - floating-point moves to registers
  - the result of compare, minimum, maximum, negative and absolute value operations
  - the integer result of operations such as MMOV16, MAND32, MOR32, MXOR32, MCMP32, MASR32, MLSR32
- Overflow and underflow flags are set by floating-point math instructions such as multiply, add, subtract and 1/x. These flags may also be connected to the peripheral interrupt expansion (PIE) block on your device. This can be useful for debugging underflow and overflow conditions within an application.

The MSTF register is shown in [Figure 10-14](#) and described in [Table 10-33](#).

**Figure 10-14. CLA Status Register (MSTF)**

|          |  |  |  |         |  |      |  |       |  |          |  |       |  |          |  |       |  |       |  |       |  |       |  |   |  |   |  |   |  |
|----------|--|--|--|---------|--|------|--|-------|--|----------|--|-------|--|----------|--|-------|--|-------|--|-------|--|-------|--|---|--|---|--|---|--|
| 31       |  |  |  | 24      |  |      |  | 23    |  |          |  | 16    |  |          |  |       |  |       |  |       |  |       |  |   |  |   |  |   |  |
| Reserved |  |  |  |         |  |      |  | RPC   |  |          |  |       |  |          |  |       |  |       |  |       |  |       |  |   |  |   |  |   |  |
| R/W-0    |  |  |  |         |  |      |  | R/W-0 |  |          |  |       |  |          |  |       |  |       |  |       |  |       |  |   |  |   |  |   |  |
| 15       |  |  |  | 12      |  | 11   |  | 10    |  | 9        |  | 8     |  | 7        |  | 6     |  | 5     |  | 4     |  | 3     |  | 2 |  | 1 |  | 0 |  |
| RPC      |  |  |  | MEALLOW |  | Rsvd |  | RND32 |  | Reserved |  | TF    |  | Reserved |  | ZF    |  | NF    |  | LUF   |  | LVF   |  |   |  |   |  |   |  |
| R/W-0    |  |  |  | R/W-0   |  | R-0  |  | R/W-0 |  | R-0      |  | R/W-0 |  | R-0      |  | R/W-0 |  | R/W-0 |  | R/W-0 |  | R/W-0 |  |   |  |   |  |   |  |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 10-33. CLA Status (MSTF) Register Field Descriptions**

| Bits  | Field    | Value  | Description <sup>(1)</sup>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|-------|----------|--------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 31-24 | Reserved | 0      | Reserved                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| 23-12 | RPC      |        | Return program counter.<br>The RPC is used to save and restore the MPC address by the MCCNDD and MRCNDD operations.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| 11    | MEALLOW  | 0<br>1 | This bit enables and disables CLA write access to EALLOW protected registers. This is independent of the state of the EALLOW bit in the main CPU status register. This status bit can be saved and restored by the MMOV32 STF, mem32 instruction.<br><br>The CLA cannot write to EALLOW protected registers. This bit is cleared by the CLA instruction, MEDIS.<br><br>The CLA is allowed to write to EALLOW protected registers. This bit is set by the CLA instruction, MEALLOW.                                                                                                                                                                                             |
| 10    | Reserved | 0      | Any writes to these bit(s) must always have a value of 0.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| 9     | RND32    | 0<br>1 | Round 32-bit Floating-Point Mode<br>Use the MSETFLG and MMOV32 MSTF, mem32 instructions to change the rounding mode.<br><br>If this bit is zero, the MMPYF32, MADDF32, and MSUBF32 instructions will round to zero (truncate).<br><br>If this bit is one, the MMPYF32, MADDF32, and MSUBF32 instructions will round to the nearest even value.                                                                                                                                                                                                                                                                                                                                 |
| 8-7   | Reserved | 0      | Reserved                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| 6     | TF       | 0<br>1 | Test Flag<br>The MTESTTF instruction can modify this flag based on the condition tested. The MSETFLG and MMOV32 MSTF, mem32 instructions can also be used to modify this flag.<br><br>The condition tested with the MTESTTF instruction is false.<br><br>The condition tested with the MTESTTF instruction is true.                                                                                                                                                                                                                                                                                                                                                            |
| 5-4   | Reserved |        | These two bits may change based on integer results. These flags are not, however, used by the CLA and therefore marked as reserved.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| 3     | ZF       | 0<br>1 | Zero Flag <sup>(2) (3)</sup> <ul style="list-style-type: none"> <li>Instructions that modify this flag based on the floating-point value stored in the destination register: MMOV32, MMOV32D, MABSF32, MNEGF32</li> <li>Instructions that modify this flag based on the floating-point result of the operation: MCMPF32, MMAXF32, and MMINF32</li> <li>Instructions that modify this flag based on the integer result of the operation: MMOV16, MAND32, MOR32, MXOR32, MCMP32, MASR32, MLSR32, and MLSL32</li> </ul> The MSETFLG and MMOV32 MSTF, mem32 instructions can also be used to modify this flag<br><br>The value is not zero.<br><br>The value is zero.              |
| 2     | NF       | 0<br>1 | Negative Flag <sup>(2) (3)</sup> <ul style="list-style-type: none"> <li>Instructions that modify this flag based on the floating-point value stored in the destination register: MMOV32, MMOV32D, MABSF32, MNEGF32</li> <li>Instructions that modify this flag based on the floating-point result of the operation: MCMPF32, MMAXF32, and MMINF32</li> <li>Instructions that modify this flag based on the integer result of the operation: MMOV16, MAND32, MOR32, MXOR32, MCMP32, MASR32, MLSR32, and MLSL32</li> </ul> The MSETFLG and MMOV32 MSTF, mem32 instructions can also be used to modify this flag.<br><br>The value is not negative.<br><br>The value is negative. |

**Table 10-33. CLA Status (MSTF) Register Field Descriptions (continued)**

| Bits | Field | Value | Description <sup>(1)</sup>                                                                                                                                                                                                                                     |
|------|-------|-------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1    | LUF   |       | Latched Underflow Flag<br><br>The following instructions will set this flag to 1 if an underflow occurs: MMPYF32, MADDF32, MSUBF32, MMACF32, MEINVF32, MEISQRTF32<br><br>The MSETFLG and MMOV32 MSTF, mem32 instructions can also be used to modify this flag. |
|      |       | 0     | An underflow condition has not been latched.                                                                                                                                                                                                                   |
|      |       | 1     | An underflow condition has been latched.                                                                                                                                                                                                                       |
| 0    | LVF   |       | Latched Overflow Flag<br><br>The following instructions will set this flag to 1 if an overflow occurs: MMPYF32, MADDF32, MSUBF32, MMACF32, MEINVF32, MEISQRTF32<br><br>The MSETFLG and MMOV32 MSTF, mem32 instructions can also be used to modify this flag.   |
|      |       | 0     | An overflow condition has not been latched.                                                                                                                                                                                                                    |
|      |       | 1     | An overflow condition has been latched.                                                                                                                                                                                                                        |

- (1) This register is protected by the dual code security module. The main CPU can read this register for debug purposes but it can not write to it.
- (2) A negative zero floating-point value is treated as a positive zero value when configuring the ZF and NF flags.
- (3) A DeNorm floating-point value is treated as a positive zero value when configuring the ZF and NF flags.



The direct memory access (DMA) module provides a hardware method of transferring data between peripherals and/or memory without intervention from the CPU, thereby freeing up bandwidth for other system functions. Additionally, the DMA has the capability to orthogonally rearrange the data as it is transferred as well as “ping-pong” data between buffers. These features are useful for structuring data into blocks for optimal CPU processing.

|                                                       |            |
|-------------------------------------------------------|------------|
| <b>11.1 Introduction.....</b>                         | <b>728</b> |
| <b>11.2 DMA Overview.....</b>                         | <b>728</b> |
| <b>11.3 Architecture.....</b>                         | <b>729</b> |
| <b>11.4 Pipeline Timing and Throughput.....</b>       | <b>732</b> |
| <b>11.5 CPU Arbitration.....</b>                      | <b>733</b> |
| <b>11.6 Channel Priority.....</b>                     | <b>733</b> |
| <b>11.7 Address Pointer and Transfer Control.....</b> | <b>735</b> |
| <b>11.8 Overrun Detection Feature.....</b>            | <b>739</b> |
| <b>11.9 DMA Registers.....</b>                        | <b>740</b> |

## 11.1 Introduction

The strength of a controller is not measured purely in processor speed, but in total system capabilities. As a part of the equation, any time the CPU bandwidth for a given function can be reduced, the greater the system capabilities. Many times applications spend a significant amount of their bandwidth moving data, whether it is from off-chip memory to on-chip memory, or from a peripheral such as an analog-to-digital converter (ADC) to RAM, or even from one peripheral to another. Furthermore, many times this data comes in a format that is not conducive to the optimum processing powers of the CPU. The DMA module described in this reference guide has the ability to free up CPU bandwidth and rearrange the data into a pattern for more streamlined processing.

## 11.2 DMA Overview

The DMA module is an event-based machine, meaning it requires a peripheral interrupt trigger to start a DMA transfer. Although it can be made into a periodic time-driven machine by configuring a timer as the interrupt trigger source, there is no mechanism within the module itself to start memory transfers periodically. The interrupt trigger source for each of the six DMA channels can be configured separately and each channel contains its own independent PIE interrupt to let the CPU know when a DMA transfers has either started or completed. Five of the six channels are exactly the same, while Channel 1 has one additional feature: the ability to be configured at a higher priority than the others. At the heart of the DMA is a state machine and tightly coupled address control logic. It is this address control logic that allows for rearrangement of the block of data during the transfer as well as the process of *ping-ponging* data between buffers. Each of these features, along with others will be discussed in detail in this document.

- Six channels with independent PIE interrupts
- Peripheral interrupt trigger sources
  - ADC interrupts 1 and 2
  - Multichannel buffered serial port transmit and receive
  - XINT1-3
  - CPU Timers
  - ePWM2-7 ADCSOCA and ADSOCB signals
  - USB endpoints 1-3 transmit and receive
  - Software
- Data sources/destinations:
  - L5-L8 32K x 16 SARAM
  - ADC memory bus mapped result registers
  - McBSP transmit and receive buffers
  - ePWM1-8 / HRPWM1-8

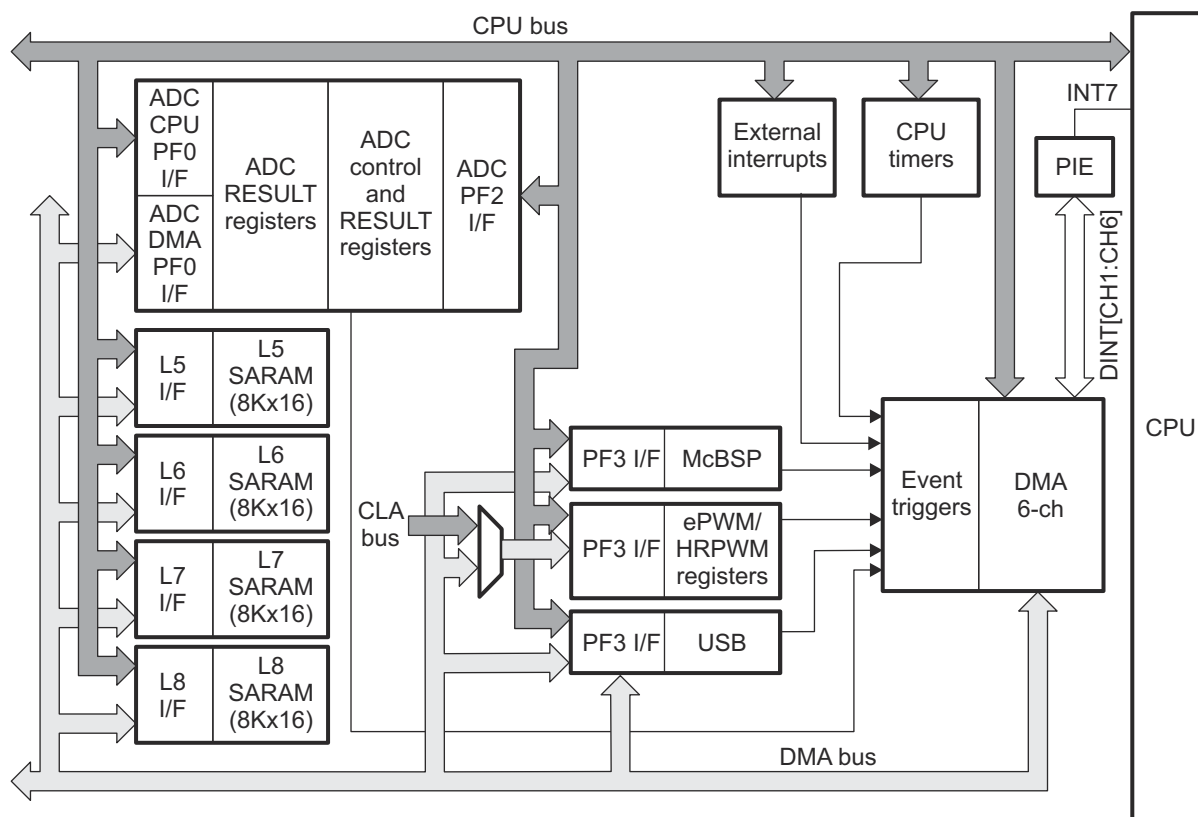
The DMA module can access all the PWM modules as shown in [Figure 11-1](#) . However, only 6 out of 8 PWM modules (ePWM2-ePWM7) can trigger DMA using PERINTSEL.

- Word Size: 16-bit or 32-bit (McBSP limited to 16-bit)
- Throughput: 4 cycles/word (5 cycles/word for McBSP reads)

## 11.3 Architecture

### 11.3.1 Block Diagram

Figure 11-1 shows a device level block diagram of the DMA.



**Figure 11-1. DMA Block Diagram**

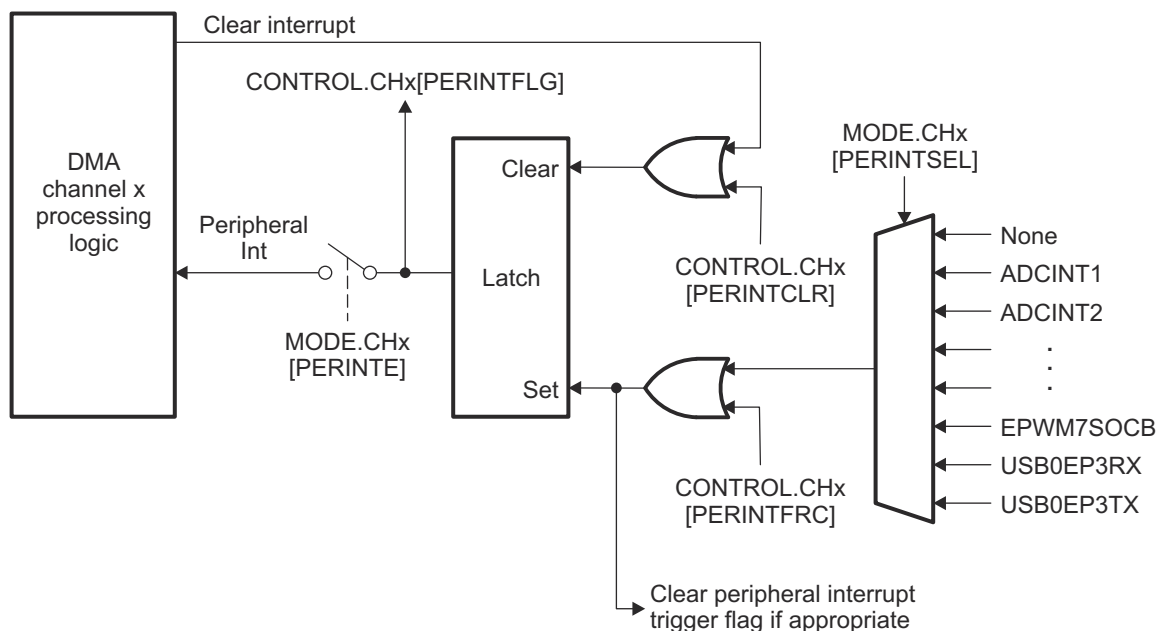
### 11.3.2 Peripheral Interrupt Event Trigger Sources

The peripheral interrupt event trigger can be independently configured as one twenty-nine different sources for each of the six DMA channels. Included in these sources are three external interrupt signals which can be connected to most of the general-purpose input/output (GPIO) pins on the device. This adds significant flexibility to the event trigger capabilities. A bit field called PERINTSEL in the MODE register of each channel is used to select that channels interrupt trigger source. An active peripheral interrupt trigger will be latched into the PERINTFLG bit of the CONTROL register, and if the respective interrupt and DMA channel is enabled (see the MODE.CHx[PERINTE] and CONTROL.CHx[RUNSTS] bits), it will be serviced by the DMA channel. Upon receipt of a peripheral interrupt event signal, the DMA will automatically send a clear signal to the interrupt source so that subsequent interrupt events will occur.

Regardless of the value of the MODE.CHx[PERINTSEL] bit field, software can always force a trigger by using the CONTROL.CHx[PERINTFRC] bit. Likewise, software can always clear a pending DMA trigger using the CONTROL.CHx[PERINTCLR] bit.

Once a particular interrupt trigger sets a channel's PERINTFLG bit, the bit stays pending until the priority logic of the state machine starts the burst transfer for that channel. Once the burst transfer starts, the flag is cleared. If a new interrupt trigger is generated while a burst is in progress, the burst will complete before responding to the new interrupt trigger (after proper prioritization). If a third interrupt trigger occurs before the pending interrupt is serviced, an error flag is set in the CONTROL.CHx[OVRFLG] bit. If a peripheral interrupt trigger occurs at the same time as the latched flag is being cleared, the peripheral interrupt trigger has priority and the PERINTFLG will remain set.

Figure 11-2 shows a diagram of the trigger select circuit. See the MODE.CHx[PERINTSEL] bit field description for the complete list of peripheral interrupt trigger sources.



**Figure 11-2. Peripheral Interrupt Trigger Input Diagram**

Table 11-1 shows the interrupt trigger source options that are available for each channel.

**Table 11-1. Peripheral Interrupt Trigger Source Options**

| Peripheral          | Interrupt Trigger Source                                             |
|---------------------|----------------------------------------------------------------------|
| CPU                 | DMA Software bit (CHx.CONTROL.PERINTFRC) only                        |
| ADC                 | ADCINT 1<br>ADCINT 2                                                 |
| External Interrupts | External Interrupt 1<br>External Interrupt 2<br>External Interrupt 3 |
| CPU Timers          | Timer 0 Overflow<br>Timer 1 Overflow<br>Timer 2 Overflow             |
| McBSP               | McBSP Transmit Buffer Empty<br>McBSP Receive Buffer Full             |
| ePWM2               | ADC Start of Conversion A<br>ADC Start of Conversion B               |
| ePWM3               | ADC Start of Conversion A<br>ADC Start of Conversion B               |
| ePWM4               | ADC Start of Conversion A<br>ADC Start of Conversion B               |
| ePWM5               | ADC Start of Conversion A<br>ADC Start of Conversion B               |
| ePWM6               | ADC Start of Conversion A<br>ADC Start of Conversion B               |
| ePWM7               | ADC Start of Conversion A<br>ADC Start of Conversion B               |
| USB                 | USB Endpoint 1 Receive Full                                          |
| USB                 | USB Endpoint 1 Transmit Empty                                        |
| USB                 | USB Endpoint 2 Receive Full                                          |
| USB                 | USB Endpoint 2 Transmit Empty                                        |
| USB                 | USB Endpoint 3 Receive Full                                          |
| USB                 | USB Endpoint 3 Transmit Empty                                        |

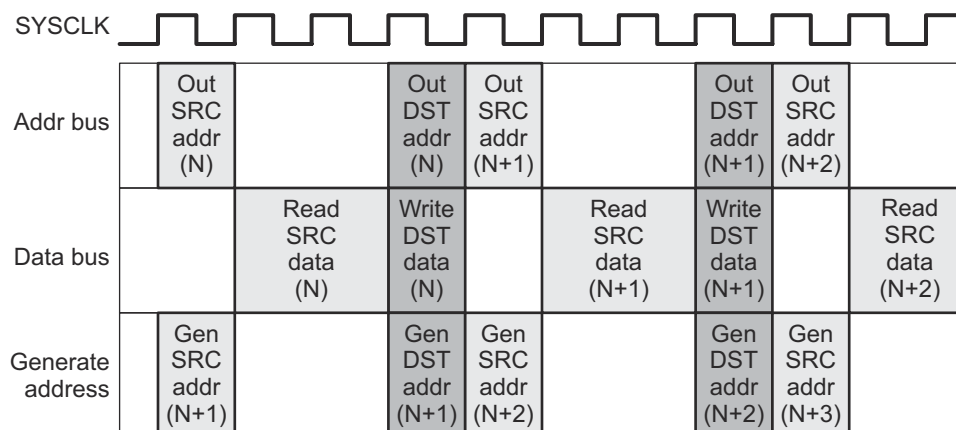
### 11.3.3 DMA Bus

The DMA bus architecture consists of a 22-bit address bus, a 32-bit data read bus, and a 32-bit data write bus. Memories and register locations connected to the DMA bus are via interfaces that sometimes share resources with the CPU memory or peripheral bus. Arbitration rules are defined in [Section 11.5](#). The following resources are connected to the DMA bus:

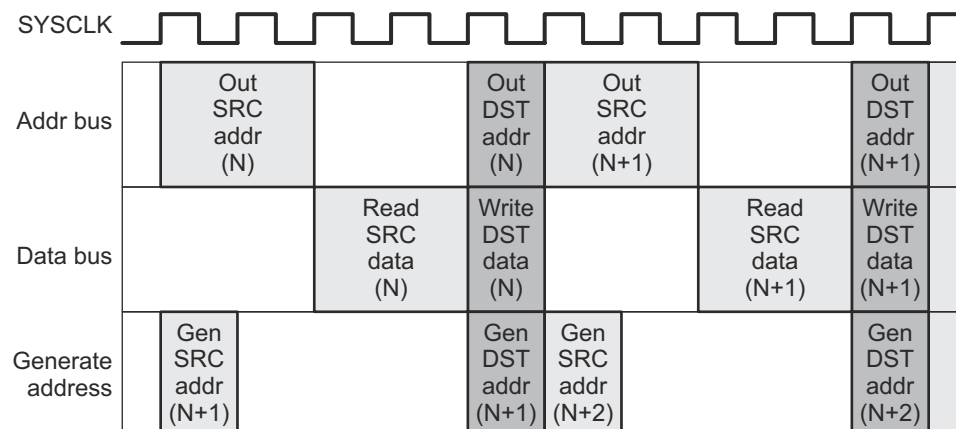
- L5 SARAM
- L6 SARAM
- L7 SARAM
- L8 SARAM
- ADC Memory Mapped Result Registers
- McBSP Data Receive Registers (DRR2/DRR1) and Data Transmit Registers (DXR2/DXR1)
- ePWM1-8/HRPWM1-8 Registers
- USB Transmit and Receive Endpoints 1-3

## 11.4 Pipeline Timing and Throughput

The DMA consists of a 4-stage pipeline as shown in [Figure 11-3](#). The one exception to this is when a DMA channel is configured to have the McBSP as its data source. A read of a McBSP DRR register stalls the DMA bus for one cycle during the read portion of the transfer, as shown in [Figure 11-4](#).



**Figure 11-3. 4-Stage Pipeline DMA Transfer**



**Figure 11-4. 4-Stage Pipeline With One Read Stall (McBSP as source)**

In addition to the pipeline there are a few other behaviors of the DMA that affect its total throughput

- A 1-cycle delay is added at the beginning of each burst
- A 1-cycle delay is added when returning from a CH1 high priority interrupt
- 32-bit transfers run at double the speed of a 16-bit transfer (that is, it takes the same amount of time to transfer a 32-bit word as it does a 16-bit word)
- Collisions with the CPU may add delay slots (see [Section 11.5](#))

For example, to transfer 128 16-bit words from ADC to RAM a channel can be configured to transfer 8 bursts of 16 words/burst. This will give:

$$8 \text{ bursts} * [(4 \text{ cycles/word} * 16 \text{ words/burst}) + 1] = 520 \text{ cycles}$$

If instead the channel were configured to transfer the same amount of data 32 bits at a time (the word size is configured to 32 bits) the transfer would take:

$$8 \text{ bursts} * [(4 \text{ cycles/word} * 8 \text{ words/burst}) + 1] = 264 \text{ cycles}$$

## 11.5 CPU Arbitration

Typically, DMA activity is independent of the CPU activity. Under the circumstance where both the DMA and the CPU are attempting to access memory or a peripheral register within the same interface concurrently, an arbitration procedure will occur. The one exception is with the memory mapped (PF0) ADC registers, which do not create a conflict when read by both the CPU and the DMA simultaneously, even if different addresses are accessed. Any combined accesses between the different interfaces, or where the CPU access is outside of the interface that the DMA is accessing do not create a conflict.

The interfaces which internally contain conflicts are:

- L5 RAM
- L6 RAM
- L7 RAM
- L8 RAM
- McBSP Peripheral Frame 3
- ePWM/HRPWM Peripheral Frame 3
- USB Peripheral Frame 3

If the CPU and the DMA make an access to the same interface in the same cycle, the DMA has priority and the CPU is stalled.

If a CPU access to an interface is in progress and another CPU access to the same interface is pending, for example, the CPU is performing a write operation and a read operation from the CPU is pending, then a DMA access to that same interface has priority over the pending CPU access when the current CPU access completes.

---

### Note

If the CPU is performing a read-modify-write operation and the DMA performs a write to the same location, the DMA write may be lost if the operation occurs in between the CPU read and the CPU write. For this reason, it is advised not to mix such CPU accesses with DMA accesses to the same locations.

---

In the case of RAM, a ping-pong scheme can be implemented to avoid the CPU and the DMA accessing the same RAM block concurrently, thus avoiding any stalls or corruption issues.

## 11.6 Channel Priority

Two priority schemes exist when determining channel priority: Round-robin mode and Channel 1 high-priority mode.

### 11.6.1 Round-Robin Mode

In this mode, all channels have *equal* priority and each enabled channel is serviced in round-robin fashion as follows:

CH1 → CH2 → CH3 → CH4 → CH5 → CH6 → CH1 → CH2 → ...

In the case above, after each channel has transferred a burst of words, the next channel is serviced. You can specify the size of the burst for each channel. Once CH6 (or the last enabled channel) has been serviced, and no other channels are pending, the round-robin state machine enters an idle state.

From the idle state, channel 1 (if enabled) is always serviced first. However, if the DMA is currently processing another channel x, all other pending channels between x and the end of the round are serviced before CH1. It is in this sense that all the channels are of *equal* priority. For instance, take an example where CH1, CH4, and CH5 are enabled in round-robin mode and CH4 is currently being processed. Then CH1 and CH5 both receive an interrupt trigger from their respective peripherals before CH4 completes. CH1 and CH5 are now both pending. When CH4 completes its burst, CH5 will be serviced next. Only after CH5 completes will CH1 be serviced. Upon completion of CH1, if there are no more channels pending, the round-robin state machine will enter an idle state.

A more complicated example is shown below:

- Assume all channels are enabled, and the DMA is in an idle state,
- Initially a trigger occurs on CH1, CH3, and CH5 on the same cycle,
- When the CH1 burst transfer starts, requests from CH3 and CH5 are pending,
- Before completion of the CH1 burst, the DMA receives a request from CH2. Now the pending requests are from CH2, CH3, and CH5,
- After completing the CH1 burst, CH2 will be serviced since it is next in the round-robin scheme after CH1.
- After the burst from CH2 is finished, the CH3 burst will be serviced, followed by CH5 burst.
- Now while the CH5 burst is being serviced, the DMA receives a request from CH1, CH3, and CH6.
- The burst from CH6 will start after the completion of the CH5 burst since it is the next channel after CH5 in the round-robin scheme.
- This will be followed by the CH1 burst and then the CH3 burst
- After the CH3 burst finishes, assuming no more triggers have occurred, the round-robin state machine will enter an idle state.

The round-robin state machine may be reset to the idle state via the `DMACTRL[PRIORITYRESET]` bit.

### 11.6.2 Channel 1 High Priority Mode

In this mode, if a CH1 trigger occurs, the current word transfer or the current + 1 word transfer (depends on which phase of the current DMA transfer the new CH1 trigger occurred) on any other channel is completed (not the complete burst), execution is halted, and CH1 is serviced for the complete burst count. When the CH1 burst is complete, execution returns to the channel that was active when the CH1 trigger occurred. All other channels have equal priority and each enabled channel is serviced in round-robin fashion as follows:

Higher Priority: CH1  
Lower priority: CH2 → CH3 → CH4 → CH5 → CH6 → CH2 → ...

Given an example where CH1, CH4 and CH5 are enabled in Channel 1 High Priority Mode and CH4 is currently being processed. Then CH1 and CH5 both receive an interrupt trigger from their respective peripherals before CH4 completes. CH1 and CH5 are now both pending. When the current CH4 word transfer is completed, regardless of whether the DMA has completed the entire CH4 burst, CH4 execution will be suspended and CH1 will be serviced. After the CH1 burst completes, CH4 will resume execution.

Upon completion of CH4, CH5 will be serviced. After CH5 completes, if there are no more channels pending, the round-robin state machine will enter an idle state.

Typically Channel 1 would be used in this mode for the ADC, since its data rate is so high. However, Channel 1 High Priority Mode may be used in conjunction with any peripheral.

---

#### Note

High-priority mode and ONESHOT mode may not be used at the same time on channel 1. Other channels may use ONESHOT mode when channel 1 is in high-priority mode.

---



## 11.7 Address Pointer and Transfer Control

The DMA state machine is, at its most basic level, two nested loops. The inner loop transfers a burst of data when a peripheral interrupt trigger is received. A burst is the smallest amount of data that can be transferred at one time and its size is defined by the BURST\_SIZE register for each channel. The BURST\_SIZE register allows a maximum of 32 sixteen-bit words to be transferred in one burst. The outer loop, whose size is set by the TRANSFER\_SIZE register for each channel, defines how many bursts are performed in the entire transfer. Since TRANSFER\_SIZE is a 16-bit register, the total size of a transfer allowed is well beyond any practical requirement. One CPU interrupt is generated, if enabled, for each transfer. This interrupt can be configured to occur at the beginning or the end of the transfer via the MODE.CHx[CHINTMODE] bit.

In the default setting of the MODE.CHx[ONESHOT] bit, the DMA transfers one burst of data each time a peripheral interrupt trigger is received. After the burst is completed, the state machine moves on to the next pending channel in the priority scheme, even if another trigger for the channel just completed is pending. This feature keeps any single channel from monopolizing the DMA bus. If a transfer of more than the maximum number of words per burst is desired for a single trigger, the MODE.CHx[ONESHOT] bit can be set to complete the entire transfer when triggered. Care is advised when using this mode, since this can create a condition where one trigger uses up the majority of the DMA bandwidth.

Each DMA channel contains a shadowed address pointer for the source and the destination address. These pointers, SRC\_ADDR and DST\_ADDR, can be independently controlled during the state machine operation. At the beginning of each transfer, the shadowed version of each pointer is copied into its respective active register. During the burst loop, after each word is transferred, the signed value contained in the appropriate source or destination BURST\_STEP register is added to the active SRC/DST\_ADDR register. During the transfer loop, after each burst is complete, there are two methods that can be used to modify the active address pointer. The first, and default, method is by adding the signed value contained in the SRC/DST\_TRANSFER\_STEP register to the appropriate pointer. The second is via a process called wrapping, where a wrap address is loaded into the active address pointer. When a wrap procedure occurs, the associated SRC/DST\_TRANSFER\_STEP register has no effect.

Address wrapping occurs when a number of bursts specified by the appropriate SRC/DST\_WRAP\_SIZE register completes. Each DMA channel contains two shadowed wrap address pointers, SRC\_BEG\_ADDR and DST\_BEG\_ADDR, allowing the source and destination wrapping to be independent of each other. Like the SRC\_ADDR and DST\_ADDR registers, the active SRC/DST\_BEG\_ADDR registers are loaded from their shadow counterpart at the beginning of a transfer. When the specified number of bursts has occurred, a two part wrap procedure takes place:

- The appropriate active SRC/DST\_BEG\_ADDR register is incremented by the signed value contained in the SRC/DST\_WRAP\_STEP register, then
- The new active SRC/DST\_BEG\_ADDR register is loaded into the active SRC/DST\_ADDR register.

Additionally the wrap counter (SRC/DST\_WRAP\_COUNT) register is reloaded with the SRC/DST\_WRAP\_SIZE value to setup the next wrap period. This allows the channel to wrap multiple times within a single transfer. Combined with the first bullet above, this allows the channel to address multiple buffers within a single transfer.

The DMA contains both an active and shadow set of the following address pointers. When a DMA transfer begins, the shadow register set is copied to the active working set of registers. This allows you to program the values of the shadow registers for the next transfer while the DMA works with the active set. It also allows you to implement Ping-Pong buffer schemes without disrupting the DMA channel execution.

### Source/Destination Address Pointers (SRC/DST\_ADDR)

The value written into the shadow register is the start address of the first location where data is read or written to.

At the beginning of a transfer the shadow register is copied into the active register. The active register performs as the current address pointer.

### Source/Destination Begin Address Pointers (SRC/ DST\_BEG\_ADDR)

This is the wrap pointer.

The value written into the shadow register will be loaded into the active register at the start of a transfer. On a wrap condition, the active register will be incremented by the signed value in the appropriate SRC/DST\_WRAP\_STEP register prior to being loaded into the active SRC/DST\_ADDR register.

For each channel, the transfer process can be controlled with the following size values:

|                                                              |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|--------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Source and Destination Burst Size (BURST_SIZE):</b>       | <p>This specifies the number of words to be transferred in a burst.</p> <p>This value is loaded into the BURST_COUNT register at the beginning of each burst. The BURST_COUNT decrements each word that is transferred and when it reaches a zero value, the burst is complete, indicating that the next channel can be serviced. The behavior of the current channel is defined by the ONE_SHOT bit in the MODE register. The maximum size of the burst is dictated by the type of peripheral. For the ADC, the burst size could be all 16 registers (if all 16 registers are used). For a McBSP peripheral, the burst size is limited to 1 since there is no FIFO and the receive or transmit data register must be loaded or copied every word transferred. For RAM the burst size can be up to the maximum allowed by the BURST_SIZE register, which is 32.</p> |
| <b>Source and Destination Transfer Size (TRANSFER_SIZE):</b> | <p>This specifies the number of bursts to be transferred before per CPU interrupt (if enabled).</p> <p>Whether this interrupt is generated at the beginning or the end of the transfer is defined in the CHINTMODE bit in the MODE register. Whether the channel remains enabled or not after the transfer is completed is defined by the CONTINUOUS bit in the MODE register. The TRANSFER_SIZE register is loaded into the TRANSFER_COUNT register at the beginning of each transfer. The TRANSFER_COUNT register keeps track of how many bursts of data the channel has transferred and when it reaches zero, the DMA transfer is complete.</p>                                                                                                                                                                                                                  |
| <b>Source/Destination Wrap Size (SRC/DST_WRAP_SIZE)</b>      | <p>This specifies the number of bursts to be transferred before the current address pointer wraps around to the beginning.</p> <p>This feature is used to implement a circular addressing type function. This value is loaded into the appropriate SRC/DST_WRAP_COUNT register at the beginning of each transfer. The SRC/DST_WRAP_COUNT registers keep track of how many bursts of data the channel has transferred and when they reaches zero, the wrap procedure is performed on the appropriate source or destination address pointer. A separate size and count register is allocated for source and destination pointers. To <i>disable</i> the wrap function, assign the value of these registers to be larger than the TRANSFER_SIZE.</p>                                                                                                                   |

### Note

The value written to the SIZE registers is one less than the intended size. So, to transfer three 16-bit words, the value 2 should be placed in the SIZE register.

Regardless of the state of the DATASIZE bit, the value specified in the SIZE registers are for 16-bit addresses. So, to transfer three 32-bit words, the value 5 should be placed in the SIZE register.

For each source/destination pointer, the address changes can be controlled with the following step values:

|                                                           |                                                                                                                                                                                                                                                                                       |
|-----------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Source/Destination Burst Step (SRC/DST_BURST_STEP)</b> | <p>Within each burst transfer, the address source and destination step sizes are specified by these registers.</p> <p>This value is a signed 2's compliment number so that the address pointer can be incremented or decremented as required. If no increment is desired, such as</p> |
|-----------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

when accessing the McBSP data receive or transmit registers, the value of these registers should be set to zero.

**Source/Destination  
Transfer Step (SRC/  
DST\_TRANSFER\_STEP)**

This specifies the address offset to start the next burst transfer after completing the current burst transfer.

This is used in cases where registers or data memory locations are spaced at constant intervals. This value is a signed 2's complement number so that the address pointer can be incremented or decremented as required.

**Source/Destination  
Wrap Step (SRC/  
DST\_WRAP\_STEP):**

When the wrap counter reaches zero, this value specifies the number of words to add/subtract from the BEG\_ADDR pointer and hence sets the new start address.

This implements a circular type of addressing mode, useful in many applications. This value is a signed 2's complement number so that the address pointer can be incremented or decremented as required.

---

**Note**

Regardless of the state of the DATASIZE bit, the value specified in the STEP registers are for 16-bit addresses. So, to increment one 32-bit address, a value of 2 should be placed in these registers.

---

Three modes are provided to control the way the state machine behaves during the burst loop and the transfer loop:

**One Shot Mode  
(ONESHOT)**

If one shot mode is enabled when an interrupt event trigger occurs, the DMA will continue transferring data in bursts until TRANSFER\_COUNT is zero. If one shot mode is disabled, then an interrupt event trigger is required for each burst transfer and this will continue until TRANSFER\_COUNT is zero.

---

**Note**

When ONESHOT mode is enabled, the DMA will continuously transfer bursts of data on the given channel until the TRANSFER\_COUNT value is zero. This could potentially hog the bandwidth of a peripheral and cause long CPU stalls to occur. To avoid this, you could configure a CPU timer (or similar) and disable ONESHOT so as to avoid this situation.

High-priority mode and ONESHOT mode may not be used at the same time on channel 1. Other channels may use ONESHOT mode when channel 1 is in high-priority mode.

---

**Continuous Mode  
(CONTINUOUS)**

If continuous mode is disabled the RUNSTS bit in the CONTROL register is cleared at the end of the transfer, disabling the DMA channel.

The channel must be re-enabled by setting the RUN bit in the CONTROL register before another transfer can be started on that channel. If the continuous mode is enabled the RUNSTS bit is not cleared at the end of the transfer.

**Channel  
Interrupt Mode  
(CHINTMODE)**

This mode bit selects whether the DMA interrupt from the respective channel is generated at the beginning of a new transfer or at the end of the transfer.

If implementing a ping-pong buffer scheme with continuous mode of operation, then the interrupt would be generated at the beginning, just after the working registers are copied to the shadow set. If the DMA does not operate in continuous mode, then the interrupt is typically generated at the end when the transfer is complete.

All of the above features and modes are shown in [Figure 11-5](#).

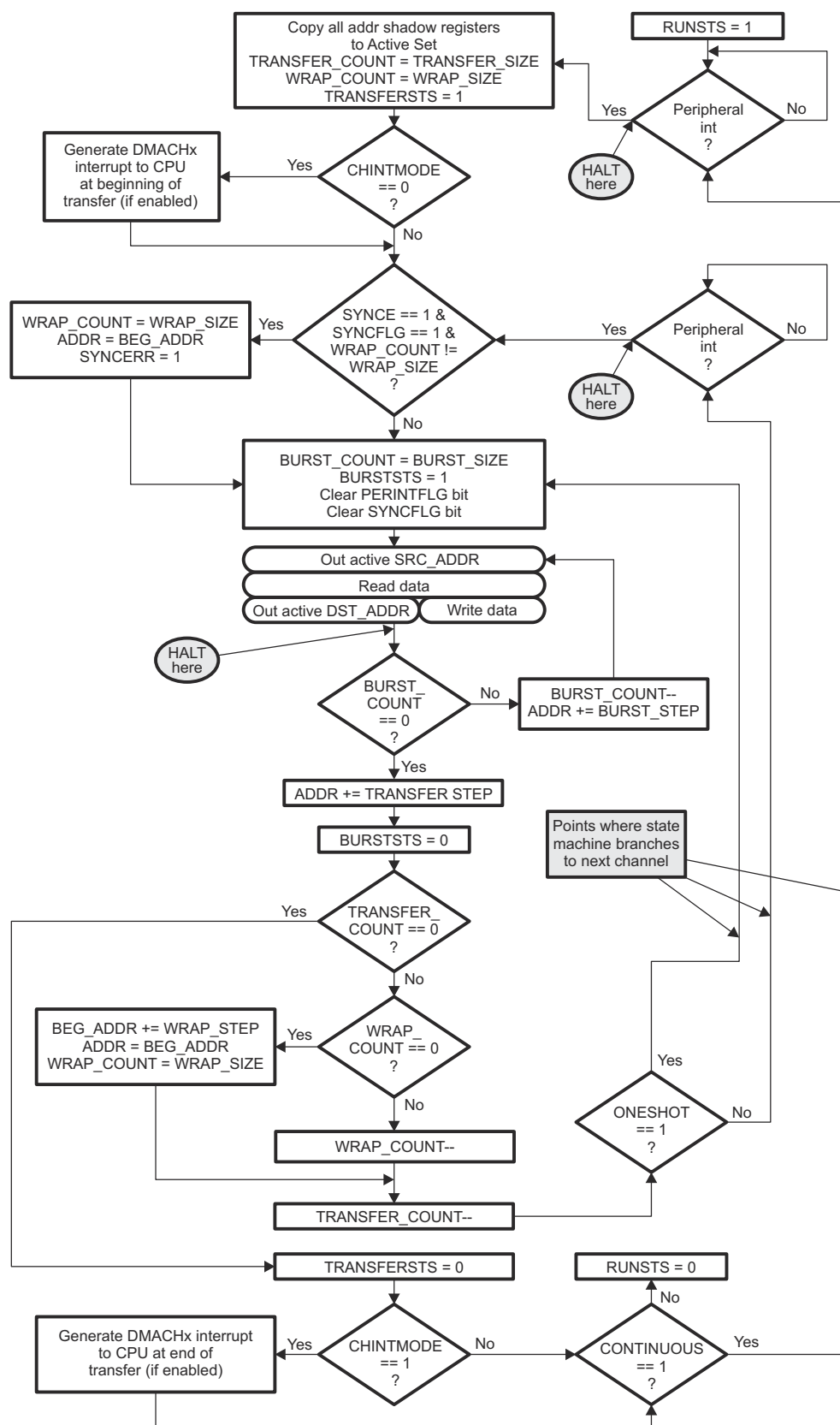


Figure 11-5. DMA State Diagram

The following items are in reference to [Figure 11-5](#).

- The *HALT* points represent where the channel halts operation when interrupted by a high priority channel 1 trigger, or when the HALT command is set, or when an emulation halt is issued and the FREE bit is cleared to 0.
- The ADDR registers are not affected by BEG\_ADDR at the start of a transfer. BEG\_ADDR only affects the ADDR registers on a wrap or sync error. Following is what happens to each of the ADDR registers when a transfer first starts:
  - BEG\_ADDR\_SHADOW remains unchanged.
  - ADDR\_SHADOW remains unchanged.
  - BEG\_ADDR = BEG\_ADDR\_SHADOW
  - ADDR = ADDR\_SHADOW
- The active registers get updated when a wrap occurs. The shadow registers remain unchanged. Specifically:
  - BEG\_ADDR\_SHADOW remains unchanged.
  - ADDR\_SHADOW remains unchanged.
  - BEG\_ADDR += WRAP\_STEP
  - ADDR = BEG\_ADDR
- The active registers get updated when a sync error occurs. The shadow registers remain unchanged. Specifically:
  - BEG\_ADDR\_SHADOW remains unchanged.
  - ADDR\_SHADOW remains unchanged.
  - BEG\_ADDR remains unchanged.
  - ADDR = BEG\_ADDR

Probably the easiest way to remember all this is that:

- The shadow registers never change except by software.
- The active registers never change except by hardware, and a shadow register is only copied into its own active register, never an active register by another name.

## 11.8 Overrun Detection Feature

The DMA contains overrun detection logic. When a peripheral event trigger is received by the DMA, the PERINTFLG bit in the CONTROL register is set, pending the channel to the DMA state machine. When the burst for that channel is started, the PERINTFLG is cleared. If however, between the time that the PERINTFLG bit is set by an event trigger and cleared by the start of the burst, an additional event trigger arrives, the second trigger will be lost. This condition will set the OVRFLG bit in the CONTROL register as in [Figure 11-6](#). If the overrun interrupt is enabled then the channel interrupt will be generated to the PIE module.

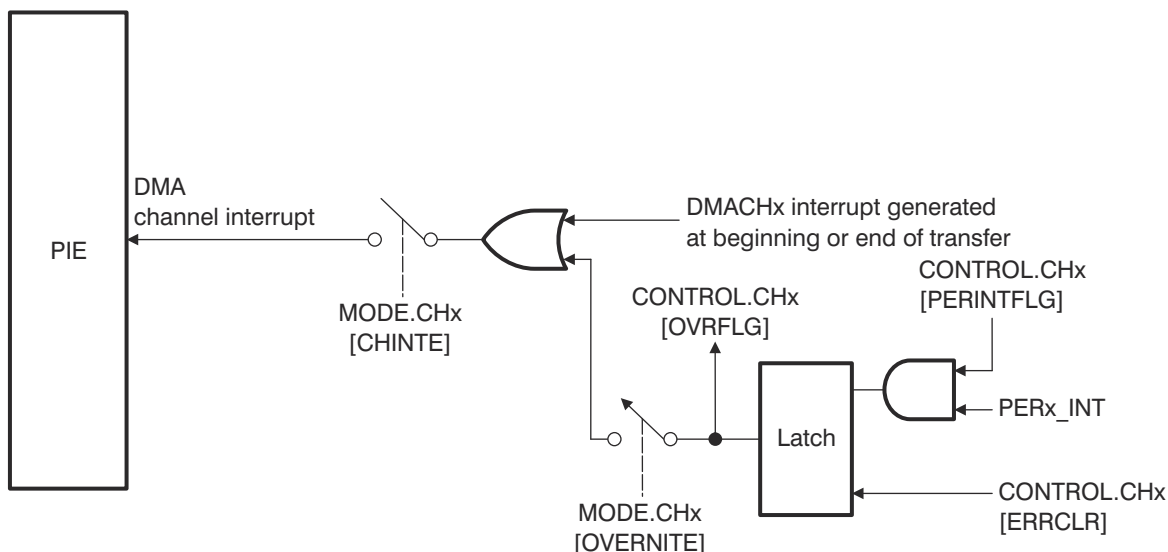


Figure 11-6. Overrun Detection Logic

## 11.9 DMA Registers

The complete DMA register set is shown in [Table 11-2](#).

**Table 11-2. DMA Register Summary**

| Address                                       | Acronym             | Description <sup>(1)</sup>                                     | Section                         |
|-----------------------------------------------|---------------------|----------------------------------------------------------------|---------------------------------|
| <b>DMA Control, Mode and Status Registers</b> |                     |                                                                |                                 |
| 0x1000                                        | DMACTRL             | DMA Control Register                                           | <a href="#">Section 11.9.1</a>  |
| 0x1001                                        | DEBUGCTRL           | Debug Control Register                                         | <a href="#">Section 11.9.2</a>  |
| 0x1002                                        | REVISION            | Peripheral Revision Register                                   | <a href="#">Section 11.9.3</a>  |
| 0x1003                                        | Reserved            | Reserved                                                       |                                 |
| 0x1004                                        | PRIORITYCTRL1       | Priority Control Register 1                                    | <a href="#">Section 11.9.4</a>  |
| 0x1005                                        | Reserved            | Reserved                                                       |                                 |
| 0x1006                                        | PRIORITYSTAT        | Priority Status Register                                       | <a href="#">Table 11-7</a>      |
| 0x1007                                        | Reserved            | Reserved                                                       |                                 |
| 0x101F                                        |                     |                                                                |                                 |
| <b>DMA Channel 1 Registers</b>                |                     |                                                                |                                 |
| 0x1020                                        | MODE                | Mode Register                                                  | <a href="#">Section 11.9.6</a>  |
| 0x1021                                        | CONTROL             | Control Register                                               | <a href="#">Section 11.9.7</a>  |
| 0x1022                                        | BURST_SIZE          | Burst Size Register                                            | <a href="#">Section 11.9.8</a>  |
| 0x1023                                        | BURST_COUNT         | Burst Count Register                                           | <a href="#">Section 11.9.9</a>  |
| 0x1024                                        | SRC_BURST_STEP      | Source Burst Step Size Register                                | <a href="#">Section 11.9.10</a> |
| 0x1025                                        | DST_BURST_STEP      | Destination Burst Step Size Register                           | <a href="#">Section 11.9.11</a> |
| 0x1026                                        | TRANSFER_SIZE       | Transfer Size Register                                         | <a href="#">Table 11-13</a>     |
| 0x1027                                        | TRANSFER_COUNT      | Transfer Count Register                                        | <a href="#">Section 11.9.13</a> |
| 0x1028                                        | SRC_TRANSFER_STEP   | Source Transfer Step Size Register                             | <a href="#">Section 11.9.14</a> |
| 0x1029                                        | DST_TRANSFER_STEP   | Destination Transfer Step Size Register                        | <a href="#">Section 11.9.15</a> |
| 0x102A                                        | SRC_WRAP_SIZE       | Source Wrap Size Register                                      | <a href="#">Section 11.9.16</a> |
| 0x102B                                        | SRC_WRAP_COUNT      | Source Wrap Count Register                                     | <a href="#">Section 11.9.17</a> |
| 0x102C                                        | SRC_WRAP_STEP       | Source Wrap Step Size Register                                 | <a href="#">Section 11.9.18</a> |
| 0x102D                                        | DST_WRAP_SIZE       | Destination Wrap Size Register                                 | <a href="#">Section 11.9.16</a> |
| 0x102E                                        | DST_WRAP_COUNT      | Destination Wrap Count Register                                | <a href="#">Section 11.9.17</a> |
| 0x102F                                        | DST_WRAP_STEP       | Destination Wrap Step Size Register                            | <a href="#">Section 11.9.18</a> |
| 0x1030                                        | SRC_BEG_ADDR_SHADOW | Shadow Source Begin and Current Address Pointer Registers      | <a href="#">Section 11.9.19</a> |
| 0x1032                                        | SRC_ADDR_SHADOW     |                                                                | <a href="#">Section 11.9.19</a> |
| 0x1034                                        | SRC_BEG_ADDR        | Active Source Begin and Current Address Pointer Registers      | <a href="#">Section 11.9.20</a> |
| 0x1036                                        | SRC_ADDR            |                                                                | <a href="#">Section 11.9.20</a> |
| 0x1038                                        | DST_BEG_ADDR_SHADOW | Shadow Destination Begin and Current Address Pointer Registers | <a href="#">Section 11.9.21</a> |

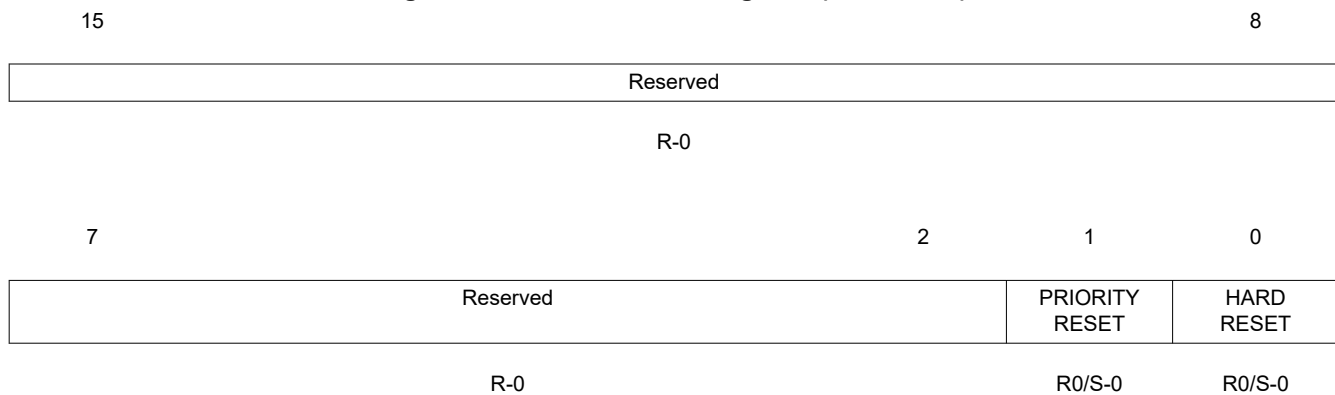
**Table 11-2. DMA Register Summary (continued)**

| Address                        | Acronym         | Description <sup>(1)</sup>                                     | Section                         |
|--------------------------------|-----------------|----------------------------------------------------------------|---------------------------------|
| 0x103A                         | DST_ADDR_SHADOW |                                                                | <a href="#">Section 11.9.21</a> |
| 0x103C                         | DST_BEG_ADDR    | Active Destination Begin and Current Address Pointer Registers | <a href="#">Section 11.9.22</a> |
| 0x103E                         | DST_ADDR        |                                                                | <a href="#">Section 11.9.22</a> |
| 0x103F                         | Reserved        | Reserved                                                       |                                 |
| <b>DMA Channel 2 Registers</b> |                 |                                                                |                                 |
| 0x1040                         | Same as above   |                                                                |                                 |
| 0x105F                         |                 |                                                                |                                 |
| <b>DMA Channel 3 Registers</b> |                 |                                                                |                                 |
| 0x1060                         | Same as above   |                                                                |                                 |
| 0x107F                         |                 |                                                                |                                 |
| <b>DMA Channel 4 Registers</b> |                 |                                                                |                                 |
| 0x1080                         | Same as above   |                                                                |                                 |
| 0x109F                         |                 |                                                                |                                 |
| <b>DMA Channel 5 Registers</b> |                 |                                                                |                                 |
| 0x10A0                         | Same as above   |                                                                |                                 |
| 0x10BF                         |                 |                                                                |                                 |
| <b>DMA Channel 6 Registers</b> |                 |                                                                |                                 |
| 0x10C0                         | Same as above   |                                                                |                                 |
| 0x10DF                         |                 |                                                                |                                 |

(1) All DMA register writes are EALLOW protected.

### 11.9.1 DMA Control Register (DMACTRL) — EALLOW Protected

The DMA control register (DMACTRL) is shown in [Figure 11-7](#) and described in [Table 11-3](#).

**Figure 11-7. DMA Control Register (DMACTRL)**


LEGEND: R0/S = Read 0/Set; R = Read only; -n = value after reset

**Table 11-3. DMA Control Register (DMACTRL) Field Descriptions**

| Bit  | Field    | Value | Description |
|------|----------|-------|-------------|
| 15-2 | Reserved |       | Reserved    |

**Table 11-3. DMA Control Register (DMACTRL) Field Descriptions (continued)**

| Bit | Field         | Value | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|-----|---------------|-------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1   | PRIORITYRESET | 0     | <p>The priority reset bit resets the round-robin state machine when a 1 is written. Service starts from the first enabled channel. Writes of 0 are ignored and this bit always reads back a 0.</p> <p>When a 1 is written to this bit, any pending burst transfer completes before resetting the channel priority machine. If CH1 is configured as a high priority channel, and this bit is written to while CH1 is servicing a burst, the CH1 burst is completed and then any lower priority channel burst is also completed (if CH1 interrupted in the middle of a burst), before the state machine is reset.</p> <p>In case CH1 is high priority, the <i>state machine</i> restarts from CH2 (or the next highest enabled channel).</p>                                                                     |
| 0   | HARDRESET     | 0     | <p>Writing a 1 to the hard reset bit resets the whole DMA and aborts any current access (similar to applying a device reset). Writes of 0 are ignored and this bit always reads back a 0.</p> <p>For a <i>soft</i> reset, a bit is provided for each channel to perform a gentler reset. Refer to the channel control registers.</p> <p>If the DMA was performing an access to the XINTF and the DMA access was stalled (XREADY not responding), then a HARDRESET would abort the access. The XINTF access would only complete if XREADY is released.</p> <p>When writing to this bit, there is a one cycle delay before it takes effect. Hence at least a one cycle delay (i.e., a NOP instruction) after writing to this bit should be introduced before attempting an access to any other DMA register.</p> |



### 11.9.2 Debug Control Register (DEBUGCTRL) — EALLOW Protected

The debug control register (DEBUGCTRL) is shown in [Figure 11-8](#) and described in [Table 11-4](#).

**Figure 11-8. Debug Control Register (DEBUGCTRL)**

|       |          |   |
|-------|----------|---|
| 15    | 14       | 0 |
| FREE  | Reserved |   |
| R/W-0 | R-0      |   |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 11-4. Debug Control Register (DEBUGCTRL) Field Descriptions**

| Bit  | Field    | Value  | Description                                                                                                                                                                                                                                                                                                                                    |
|------|----------|--------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15   | FREE     | 0<br>1 | Emulation Control Bit: This bit specifies the action when an emulation halt event occurs.<br>DMA runs until the current DMA read-write access is completed and the current status of a DMA is frozen. See the <i>HALT</i> points in <a href="#">Figure 11-5</a> for possible halt states.<br>DMA is unaffected by emulation suspend (run free) |
| 14-0 | Reserved |        | Reserved                                                                                                                                                                                                                                                                                                                                       |

### 11.9.3 Revision Register (REVISION)

The revision register (REVISION) is shown in [Figure 11-9](#) and described in [Table 11-5](#).

**Figure 11-9. Revision Register (REVISION)**

|      |   |     |   |
|------|---|-----|---|
| 15   | 8 | 7   | 0 |
| TYPE |   | REV |   |
| R    |   | R   |   |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

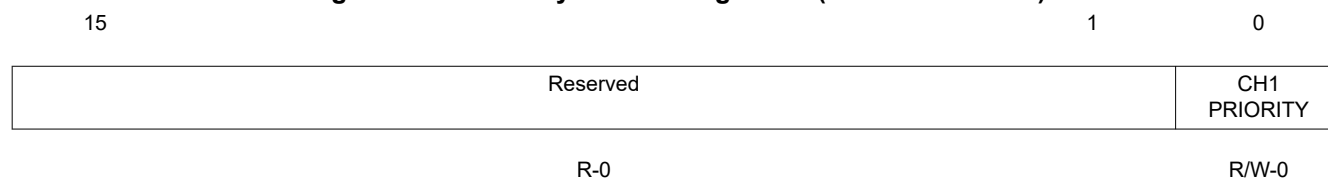
**Table 11-5. Revision Register (REVISION) Field Descriptions**

| Bit  | Field | Value  | Description                                                                                                                                                                                                                                                                                                                                                                                                     |
|------|-------|--------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-8 | TYPE  | 0x0000 | DMA Type Bits. A type change represents a major functional feature difference in a peripheral module. Within a peripheral type, there may be minor differences between devices which do not affect the basic functionality of the module. These device-specific differences are listed in the <a href="#">C2000 Real-Time Control MCU Peripherals Reference Guide</a> .<br>This document describes a Type0 DMA. |
| 7-0  | REV   | 0x0000 | DMA Silicon Revision Bits: These bits specify the DMA revision and are changed if any bug fixes are performed.<br>First release                                                                                                                                                                                                                                                                                 |

### 11.9.4 Priority Control Register 1 (PRIORITYCTRL1) — EALLOW Protected

The priority control register 1 (PRIORITYCTRL1) is shown in [Figure 11-10](#) and described in [Table 11-6](#).

**Figure 11-10. Priority Control Register 1 (PRIORITYCTRL1)**



LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

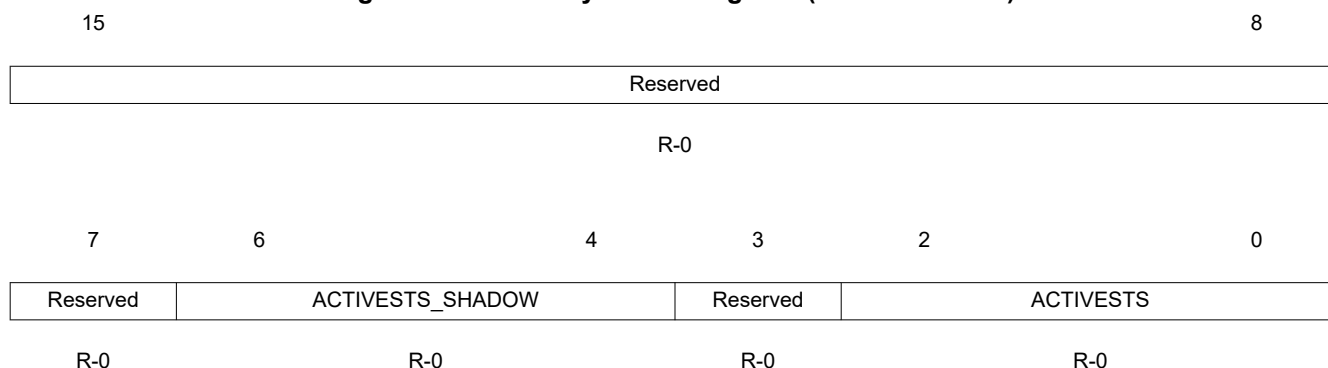
**Table 11-6. Priority Control Register 1 (PRIORITYCTRL1) Field Descriptions**

| Bit  | Field       | Value                     | Description                                                                                                                                                                                                                                                                                                          |
|------|-------------|---------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-1 | Reserved    |                           | Reserved                                                                                                                                                                                                                                                                                                             |
| 0    | CH1PRIORITY | <div>0</div> <div>1</div> | DMA Ch1 Priority: This bit selects whether channel 1 has higher priority or not:<br>Same priority as all other channels<br>Highest priority channel<br>Channel priority can only be changed when all channels are disabled. A priority reset should be performed before restarting channels after changing priority. |

### 11.9.5 Priority Status Register (PRIORITYSTAT)

The priority status register (PRIORITYSTAT) is shown in [Figure 11-11](#) and described in [Table 11-7](#).

**Figure 11-11. Priority Status Register (PRIORITYSTAT)**



LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 11-7. Priority Status Register (PRIORITYSTAT) Field Descriptions**

| Bit  | Field            | Value                                                                                                        | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|------|------------------|--------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-7 | Reserved         |                                                                                                              | Reserved                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| 6-4  | ACTIVESTS_SHADOW | 0,0,0 No channel pending<br>0,0,1 CH 1<br>0,1,0 CH 2<br>0,1,1 CH 3<br>1,0,0 CH 4<br>1,0,1 CH 5<br>1,1,0 CH 6 | Active Channel Status Shadow Bits: These bits are only useful when CH1 is enabled as a higher priority channel. When CH1 is serviced, the ACTIVESTS bits are copied to the shadow bits and indicate which channel was interrupted by CH1. When CH1 service is completed, the shadow bits are copied back to the ACTIVESTS bits. If this bit field is zero or the same as the ACTIVESTS bit field, then no channel is pending due to a CH1 interrupt. When CH1 is not a higher priority channel, these bits should be ignored: |
| 3    | Reserved         |                                                                                                              | Reserved                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| 2-0  | ACTIVESTS        | 0,0,0 no channel active<br>0,0,1 CH 1<br>0,1,0 CH 2<br>0,1,1 CH 3<br>1,0,0 CH 4<br>1,0,1 CH 5<br>1,1,0 CH 6  | Active Channel Status Bits: These bits indicate which channel is currently active or performing a transfer:                                                                                                                                                                                                                                                                                                                                                                                                                   |

### 11.9.6 Mode Register (MODE) — EALLOW Protected

The mode register (MODE) is shown in [Figure 11-12](#) and described in [Table 11-8](#). Only write to the MODE register when the RUNSTS bit is 0 (DMA channel stopped or halted). Typically though, the values should only be configured when the channel is stopped.

**Figure 11-12. Mode Register (MODE)**

|         |  |          |  |          |  |           |  |            |  |         |  |           |  |         |  |   |  |
|---------|--|----------|--|----------|--|-----------|--|------------|--|---------|--|-----------|--|---------|--|---|--|
| 15      |  | 14       |  | 13       |  | 12        |  | 11         |  | 10      |  | 9         |  | 8       |  |   |  |
| CHINTE  |  | DATASIZE |  | Reserved |  |           |  | CONTINUOUS |  | ONESHOT |  | CHINTMODE |  | PERINTE |  |   |  |
| R/W-0   |  | R/W-0    |  | R/W-0    |  |           |  | R/W-0      |  | R/W-0   |  | R/W-0     |  | R/W-0   |  |   |  |
| 7       |  | 6        |  | 5        |  | 4         |  |            |  |         |  |           |  |         |  | 0 |  |
| OVRINTE |  | Reserved |  |          |  | PERINTSEL |  |            |  |         |  |           |  |         |  |   |  |
| R/W-0   |  | R-0      |  |          |  | R/W-0     |  |            |  |         |  |           |  |         |  |   |  |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 11-8. Mode Register (MODE) Field Descriptions**

| Bit   | Field      | Value  | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|-------|------------|--------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15    | CHINTE     | 0<br>1 | Channel Interrupt Enable Bit: This bit enables/disables the respective DMA channel interrupt to the CPU (via the PIE).<br>Interrupt disabled<br>Interrupt enabled                                                                                                                                                                                                                                                                                                                                                                  |
| 14    | DATASIZE   | 0<br>1 | Data Size Mode Bit: This bit selects if the DMA channel transfers 16-bits or 32-bits of data at a time.<br>16-bit data transfer size<br>32-bit data transfer size<br><br><b>Note</b><br><br>Regardless of the value of this bit all of the registers in the DMA refer to 16-bit words. The only effect this bit causes is whether the data bus width is 16 or 32 bits.<br><br>It is up to you to configure the pointer step increment and size to accommodate 32-bit data transfers. See <a href="#">Section 11.7</a> for details. |
| 13-12 | Reserved   |        | Reserved                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| 11    | CONTINUOUS |        | Continuous Mode Bit: If this bit is set to 1, then DMA re-initializes when TRANSFER_COUNT is zero and waits for the next interrupt event trigger. If this bit is 0, then the DMA stops and clears the RUNSTS bit to 0.                                                                                                                                                                                                                                                                                                             |
| 10    | ONESHOT    |        | One Shot Mode Bit: If this bit is set to 1, then subsequent burst transfers occur without additional event triggers after the first event trigger. If this bit is 0 then only one burst transfer is performed per event trigger.<br>Note: High-priority mode and One-shot mode may not be used at the same time on CH1.                                                                                                                                                                                                            |
| 9     | CHINTMODE  | 0<br>1 | Channel Interrupt Generation Mode Bit: This bit specifies when the respective DMA channel interrupt should be generated to the CPU (via the PIE).<br>Generate interrupt at beginning of new transfer<br>Generate interrupt at end of transfer.                                                                                                                                                                                                                                                                                     |
| 8     | PERINTE    | 0<br>1 | Peripheral Interrupt Trigger Enable Bit: This bit enables/disables the selected peripheral interrupt trigger to the DMA.<br>Interrupt trigger disabled. Neither the selected peripheral nor software can start a DMA burst.<br>Interrupt trigger enabled.                                                                                                                                                                                                                                                                          |

**Table 11-8. Mode Register (MODE) Field Descriptions (continued)**

| Bit | Field                  | Value        | Description                                                                                                                                                                                                                                                                                                                                                                                                                       |
|-----|------------------------|--------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 7   | OVRINTE <sup>(1)</sup> | 0<br>1       | <p>Overflow Interrupt Enable: This bit when set to 1 enables the DMA to generate an interrupt when an overflow event is detected.</p> <p>Overflow interrupt disabled</p> <p>Overflow interrupt enabled</p> <p>An overflow interrupt is generated when the PERINTFLG is set and another interrupt event occurs. The PERINTFLG being set indicates a previous peripheral event is latched and has not been serviced by the DMA.</p> |
| 6-5 | Reserved               |              | Reserved                                                                                                                                                                                                                                                                                                                                                                                                                          |
| 4-0 | PERINTSEL              |              | Peripheral Interrupt Source Select Bits: These bits select which interrupt triggers a DMA burst for the given channel. Only one interrupt source can be selected. A DMA burst can also be forced via the PERINTFRC bit.                                                                                                                                                                                                           |
|     |                        | <b>Value</b> | <b>Interrupt</b>                                                                                                                                                                                                                                                                                                                                                                                                                  |
|     |                        | <b>Sync</b>  | <b>Peripheral</b>                                                                                                                                                                                                                                                                                                                                                                                                                 |
|     |                        | 0            | None                                                                                                                                                                                                                                                                                                                                                                                                                              |
|     |                        | 1            | ADCINT1                                                                                                                                                                                                                                                                                                                                                                                                                           |
|     |                        | 2            | ADCINT2                                                                                                                                                                                                                                                                                                                                                                                                                           |
|     |                        | 3            | XINT1                                                                                                                                                                                                                                                                                                                                                                                                                             |
|     |                        | 4            | XINT2                                                                                                                                                                                                                                                                                                                                                                                                                             |
|     |                        | 5            | XINT3                                                                                                                                                                                                                                                                                                                                                                                                                             |
|     |                        | 6            | Reserved                                                                                                                                                                                                                                                                                                                                                                                                                          |
|     |                        | 7            | USB0EP1RX                                                                                                                                                                                                                                                                                                                                                                                                                         |
|     |                        | 8            | USB0EP1TX                                                                                                                                                                                                                                                                                                                                                                                                                         |
|     |                        | 9            | USB0EP2RX                                                                                                                                                                                                                                                                                                                                                                                                                         |
|     |                        | 10           | USB0EP2TX                                                                                                                                                                                                                                                                                                                                                                                                                         |
|     |                        | 11           | TINT0                                                                                                                                                                                                                                                                                                                                                                                                                             |
|     |                        | 12           | TINT1                                                                                                                                                                                                                                                                                                                                                                                                                             |
|     |                        | 13           | TINT2                                                                                                                                                                                                                                                                                                                                                                                                                             |
|     |                        | 14           | MXEVTA                                                                                                                                                                                                                                                                                                                                                                                                                            |
|     |                        | 15           | MREVTA                                                                                                                                                                                                                                                                                                                                                                                                                            |
|     |                        | 16           | Reserved                                                                                                                                                                                                                                                                                                                                                                                                                          |
|     |                        | 17           | Reserved                                                                                                                                                                                                                                                                                                                                                                                                                          |
|     |                        | 18           | ePWM2SOCA                                                                                                                                                                                                                                                                                                                                                                                                                         |
|     |                        | 19           | ePWM2SOCB                                                                                                                                                                                                                                                                                                                                                                                                                         |
|     |                        | 20           | ePWM3SOCA                                                                                                                                                                                                                                                                                                                                                                                                                         |
|     |                        | 21           | ePWM3SOCB                                                                                                                                                                                                                                                                                                                                                                                                                         |
|     |                        | 22           | ePWM4SOCA                                                                                                                                                                                                                                                                                                                                                                                                                         |
|     |                        | 23           | ePWM4SOCB                                                                                                                                                                                                                                                                                                                                                                                                                         |
|     |                        | 24           | ePWM5SOCA                                                                                                                                                                                                                                                                                                                                                                                                                         |
|     |                        | 25           | ePWM5SOCB                                                                                                                                                                                                                                                                                                                                                                                                                         |
|     |                        | 26           | ePWM6SOCA                                                                                                                                                                                                                                                                                                                                                                                                                         |
|     |                        | 27           | ePWM6SOCB                                                                                                                                                                                                                                                                                                                                                                                                                         |
|     |                        | 28           | ePWM7SOCA                                                                                                                                                                                                                                                                                                                                                                                                                         |
|     |                        | 29           | ePWM7 SOCB                                                                                                                                                                                                                                                                                                                                                                                                                        |
|     |                        | 30           | USB0EP3RX                                                                                                                                                                                                                                                                                                                                                                                                                         |
|     |                        | 31           | USB0EP3TX                                                                                                                                                                                                                                                                                                                                                                                                                         |

(1) The overflow interrupt is ORed together with the DMACH interrupt as shown in [Figure 11-6](#).

(2) The ADCSYNC only works when the sequencer override bit is set in the ADC sequencer control registers.

### 11.9.7 Control Register (CONTROL) — EALLOW Protected

The control register (CONTROL) is shown in [Figure 11-13](#) and described in [Table 11-9](#).

**Figure 11-13. Control Register (CONTROL)**

|          |          |           |           |             |          |          |           |
|----------|----------|-----------|-----------|-------------|----------|----------|-----------|
| 15       | 14       | 13        | 12        | 11          | 10       | 9        | 8         |
| Reserved | OVRFLG   | RUNSTS    | BURSTSTS  | TRANSFERSTS | Reserved | Reserved | PERINTFLG |
| R-0      | R-0      | R-0       | R-0       | R-0         | R-0      | R-0      | R-0       |
| 7        | 6        | 5         | 4         | 3           | 2        | 1        | 0         |
| ERRCLR   | Reserved | PERINTCLR | PERINTFRC | SOFTRESET   | HALT     | RUN      |           |
| R0/S-0   | R-0      | R0/S-0    | R0/S-0    | R0/S-0      | R0/S-0   | R0/S-0   | R0/S-0    |

LEGEND: R0/S = Read 0/Set; R = Read only; -n = value after reset

**Table 11-9. Control Register (CONTROL) Field Descriptions**

| Bit  | Field       | Value  | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|------|-------------|--------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15   | Reserved    |        | Reserved                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| 14   | OVRFLG      | 0<br>1 | Overflow Flag Bit: This bit indicates if a peripheral interrupt event trigger is received from the selected peripheral and the PERINTFLG is already set.<br>No overflow event<br>Overflow event<br>The ERRCLR bit can be used to clear the state of this bit to 0. The OVRFLG bit is not affected by the PERINTFRC event.                                                                                                                                                                                      |
| 13   | RUNSTS      | 0<br>1 | Run Status Bit: This bit is set to 1 when the RUN bit is written to with a 1. This indicates the DMA channel is now ready to process peripheral interrupt event triggers. This bit is cleared to 0 when TRANSFER_COUNT reaches zero and CONTINUOUS mode bit is set to 0. This bit is also cleared to 0 when either the HARDRESET bit, the SOFTRESET bit, or the HALT bit is activated.<br>Channel is disabled.<br>Channel is enabled.                                                                          |
| 12   | BURSTSTS    | 0<br>1 | Burst Status Bit: This bit is set to 1 when a DMA burst transfer begins and the BURST_COUNT is initialized with the BURST_SIZE. This bit is cleared to zero when BURST_COUNT reaches zero. This bit is also cleared to 0 when either the HARDRESET or the SOFTRESET bit is activated.<br>No burst activity<br>The DMA is currently servicing or suspending a burst transfer from this channel.                                                                                                                 |
| 11   | TRANSFERSTS | 0<br>1 | Transfer Status Bit: This bit is set to 1 when a DMA transfer begins and the address registers are copied to the shadow set and the TRANSFER_COUNT is initialized with the TRANSFER_SIZE. This bit is cleared to zero when TRANSFER_COUNT reaches zero. This bit is also cleared to 0 when either the HARDRESET or the SOFTRESET bit is activated.<br>No transfer activity<br>The channel is currently in the middle of a transfer regardless of whether a burst of data is actively being transferred or not. |
| 10-9 | Reserved    |        | Reserved                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| 8    | PERINTFLG   | 0<br>1 | Peripheral Interrupt Trigger Flag Bit: This bit indicates if a peripheral interrupt event trigger has occurred. This flag is automatically cleared when the first burst transfer begins.<br>No interrupt event trigger<br>Interrupt event trigger<br>The PERINTFRC bit can be used to set the state of this bit to 1 and force a software DMA event. The PERINTCLR bit can be used to clear the state of this bit to 0.                                                                                        |

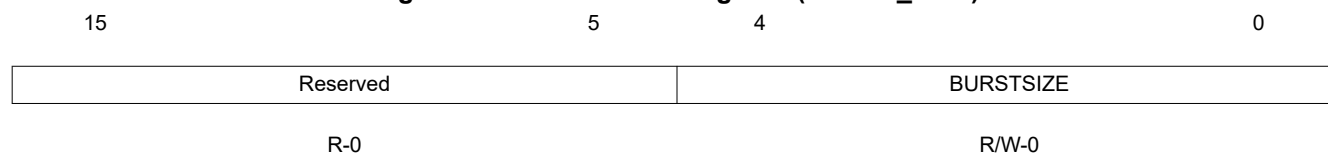
**Table 11-9. Control Register (CONTROL) Field Descriptions (continued)**

| Bit | Field     | Value | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|-----|-----------|-------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 7   | ERRCLR    | 0     | Error Clear Bit: Writing a 1 to this bit will clear any latched sync error event and clear the SYNCERR bit. This bit will also clear the OVRFLG bit. This bit would normally be used when initializing the DMA for the first time or if an overflow condition is detected. If an ADCSYNC error event or overflow event occurs at the same time as writing to this bit, the ADC or overrun has priority and the SYNCERR or OVRFLG bit is set.                                                                             |
| 6-5 | Reserved  |       | Reserved                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| 4   | PERINTCLR | 0     | Peripheral Interrupt Clear Bit: Writing a 1 to this bit clears any latched peripheral interrupt event and clears the PERINTFLG bit. This bit would normally be used when initializing the DMA for the first time. If a peripheral event occurs at the same time as writing to this bit, the peripheral has priority and the PERINTFLG bit is set.                                                                                                                                                                        |
| 3   | PERINTFRC | 0     | Peripheral Interrupt Force Bit: Writing a 1 to this bit latches a peripheral interrupt event trigger and sets the PERINTFLG bit. If the PERINTE bit is set, this bit can be used like a software force for a DMA burst transfer.                                                                                                                                                                                                                                                                                         |
| 2   | SOFTRESET | 0     | Channel Soft Reset Bit: Writing a 1 to this bit completes current read-write access and places the channel into a default state as follows:<br>RUNSTS = 0<br>TRANSFERSTS = 0<br>BURSTSTS = 0<br>BURST_COUNT = 0<br>TRANSFER_COUNT = 0<br>SRC_WRAP_COUNT = 0<br>DST_WRAP_COUNT = 0<br>This is a <i>soft</i> reset that basically allows the DMA to complete the current read-write access and then places the DMA channel into the default reset state.                                                                   |
| 1   | HALT      | 0     | Channel Halt Bit: Writing a 1 to this bit halts the DMA at the current state and any current read-write access is completed. See <a href="#">Figure 11-5</a> for the various positions the state machine can be at when HALTED. The RUNSTS bit is set to 0. To take the device out of HALT, the RUN bit needs to be activated.                                                                                                                                                                                           |
| 0   | RUN       | 0     | Channel Run Bit: Writing a 1 to this bit starts the DMA channel. The RUNSTS bit is set to 1. This bit is also used to take the device out of HALT.<br><br>The RUN bit is typically used to start the DMA running after you have configured the DMA. It will then wait for the first interrupt event (PERINTFLG == 1) to start operation. The RUN bit can also be used to take the DMA channel out of a HALT condition See <a href="#">Figure 11-5</a> for the various positions the state machine can be at when HALTED. |

### 11.9.8 Burst Size Register (BURST\_SIZE) — EALLOW Protected

The burst size register (BURST\_SIZE) is shown in [Figure 11-14](#) and described in [Table 11-10](#).

**Figure 11-14. Burst Size Register (BURST\_SIZE)**



LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

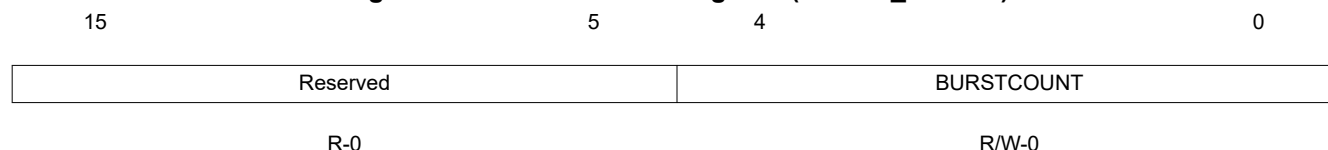
**Table 11-10. Burst Size Register (BURST\_SIZE) Field Descriptions**

| Bit  | Field     | Value               | Description                                                                                                                                     |
|------|-----------|---------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-5 | Reserved  |                     | Reserved                                                                                                                                        |
| 4-0  | BURSTSIZE | 0<br>1<br>...<br>31 | These bits specify the burst transfer size:<br>Transfer 1 word in a burst<br>Transfer 2 words in a burst<br>...<br>Transfer 32 words in a burst |

### 11.9.9 BURST\_COUNT Register

The burst count register (BURST\_COUNT) is shown in [Figure 11-15](#) and described in [Table 11-11](#).

**Figure 11-15. Burst Count Register (BURST\_COUNT)**



LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 11-11. Burst Count Register (BURST\_COUNT) Field Descriptions**

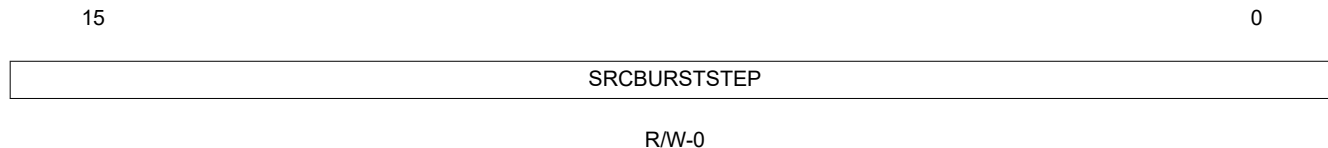
| Bit  | Field      | Value                    | Description                                                                                                                                                                                                                                           |
|------|------------|--------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-5 | Reserved   |                          | Reserved                                                                                                                                                                                                                                              |
| 4-0  | BURSTCOUNT | 0<br>1<br>2<br>...<br>31 | These bits indicate the current burst counter value:<br>0 word left in a burst<br>1 word left in a burst<br>2 words left in a burst<br>...<br>31 words left in a burst<br>The above values represent the state of the counter at the HALT conditions. |



### 11.9.10 Source Burst Step Register Size (SRC\_BURST\_STEP) — EALLOW Protected

The source burst step size register (SRC\_BURST\_STEP) is shown in [Figure 11-16](#) and described in [Table 11-12](#).

**Figure 11-16. Source Burst Step Size Register (SRC\_BURST\_STEP)**



LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 11-12. Source Burst Step Size Register (SRC\_BURST\_STEP) Field Descriptions**

| Bit  | Field        | Value  | Description                                                                                                |
|------|--------------|--------|------------------------------------------------------------------------------------------------------------|
| 15-0 | SRCBURSTSTEP |        | These bits specify the source address post-increment/decrement step size while processing a burst of data: |
|      |              | 0x0FFF | Add 4095 to address                                                                                        |
|      |              | ...    | ...                                                                                                        |
|      |              | 0x0002 | Add 2 to address                                                                                           |
|      |              | 0x0001 | Add 1 to address                                                                                           |
|      |              | 0x0000 | No address change                                                                                          |
|      |              | 0xFFFF | Sub 1 from address                                                                                         |
|      |              | 0xFFFE | Sub 2 from address                                                                                         |
|      |              | ...    | ...                                                                                                        |
|      |              | 0xF000 | Sub 4096 from address                                                                                      |
|      |              |        | Only values from -4096 to 4095 are valid.                                                                  |

### 11.9.11 Destination Burst Step Register Size (DST\_BURST\_STEP) — EALLOW Protected

The destination burst step register size (DST\_BURST\_STEP) is shown in [Figure 11-17](#) and described in [Table 11-13](#). Only write to the BURST registers when the RUNSTS bit is 0 (DMA channel stopped or halted). Typically though, the values should be configured only when the channel is stopped.

**Figure 11-17. Destination Burst Step Register Size (DST\_BURST\_STEP)**

15

0

|              |  |
|--------------|--|
| DSTBURSTSTEP |  |
| R/W-0        |  |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 11-13. Destination Burst Step Register Size (DST\_BURST\_STEP) Field Descriptions**

| Bit                                       | Field        | Value  | Description                                                                                                     |
|-------------------------------------------|--------------|--------|-----------------------------------------------------------------------------------------------------------------|
| 15-0                                      | DSTBURSTSTEP |        | These bits specify the destination address post-increment/decrement step size while processing a burst of data: |
|                                           |              | 0x0FFF | Add 4095 to address                                                                                             |
|                                           |              | ...    | ...                                                                                                             |
|                                           |              | 0x0002 | Add 2 to address                                                                                                |
|                                           |              | 0x0001 | Add 1 to address                                                                                                |
|                                           |              | 0x0000 | No address change                                                                                               |
|                                           |              | 0xFFFF | Sub 1 from address                                                                                              |
|                                           |              | 0xFFFE | Sub 2 from address                                                                                              |
|                                           |              | ...    | ...                                                                                                             |
|                                           |              | 0xF000 | Sub 4096 from address                                                                                           |
| Only values from -4096 to 4095 are valid. |              |        |                                                                                                                 |

### 11.9.12 Transfer Size Register (TRANSFER\_SIZE) — EALLOW Protected

The transfer size register (TRANSFER\_SIZE) is shown in [Figure 11-18](#) and described in [Table 11-14](#).

**Figure 11-18. Transfer Size Register (TRANSFER\_SIZE)**

15

0

|              |  |
|--------------|--|
| TRANSFERSIZE |  |
| R/W-0        |  |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 11-14. Transfer Size Register (TRANSFER\_SIZE) Field Descriptions**

| Bit  | Field        | Value  | Description                                          |
|------|--------------|--------|------------------------------------------------------|
| 15-0 | TRANSFERSIZE |        | These bits specify the number of bursts to transfer: |
|      |              | 0x0000 | Transfer 1 burst                                     |
|      |              | 0x0001 | Transfer 2 bursts                                    |
|      |              | 0x0002 | Transfer 3 bursts                                    |
|      |              | ...    | ...                                                  |
|      |              | 0xFFFF | Transfer 65536 bursts                                |

### 11.9.13 Transfer Count Register (TRANSFER\_COUNT)

The transfer count register (TRANSFER\_COUNT) is shown in [Figure 11-19](#) and described in [Table 11-15](#).

**Figure 11-19. Transfer Count Register (TRANSFER\_COUNT)**

15

0

|               |  |
|---------------|--|
| TRANSFERCOUNT |  |
| R/W-0         |  |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 11-15. Transfer Count Register (TRANSFER\_COUNT) Field Descriptions**

| Bit  | Field         | Value  | Description                                                                 |
|------|---------------|--------|-----------------------------------------------------------------------------|
| 15-0 | TRANSFERCOUNT |        | These bits specify the current transfer counter value:                      |
|      |               | 0x0000 | 0 bursts left to transfer                                                   |
|      |               | 0x0001 | 1 burst left to transfer                                                    |
|      |               | 0x0002 | 2 bursts left to transfer                                                   |
|      |               | ...    | ...                                                                         |
|      |               | 0xFFFF | 65535 bursts left to transfer                                               |
|      |               |        | The above values represent the state of the counter at the HALT conditions. |

### 11.9.14 Source Transfer Step Size Register (SRC\_TRANSFER\_STEP) — EALLOW Protected

The source transfer step size register (SRC\_TRANSFER\_STEP) is shown in [Figure 11-20](#) and described in [Table 11-16](#).

**Figure 11-20. Source Transfer Step Size Register (SRC\_TRANSFER\_STEP)**

15

0

|                 |  |
|-----------------|--|
| SRCTRANSFERSTEP |  |
| R/W-0           |  |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 11-16. Source Transfer Step Size Register (SRC\_TRANSFER\_STEP) Field Descriptions**

| Bit  | Field           | Value  | Description                                                                                                        |
|------|-----------------|--------|--------------------------------------------------------------------------------------------------------------------|
| 15-0 | SRCTRANSFERSTEP |        | These bits specify the source address pointer post-increment/decrement step size after processing a burst of data: |
|      |                 | 0x0FFF | Add 4095 to address                                                                                                |
|      |                 | ...    | ...                                                                                                                |
|      |                 | 0x0002 | Add 2 to address                                                                                                   |
|      |                 | 0x0001 | Add 1 to address                                                                                                   |
|      |                 | 0x0000 | No address change                                                                                                  |
|      |                 | 0xFFFF | Sub 1 from address                                                                                                 |
|      |                 | 0xFFFE | Sub 2 from address                                                                                                 |
|      |                 | ...    | ...                                                                                                                |
|      |                 | 0xF000 | Sub 4096 from address                                                                                              |
|      |                 |        | Only values from -4096 to 4095 are valid.                                                                          |

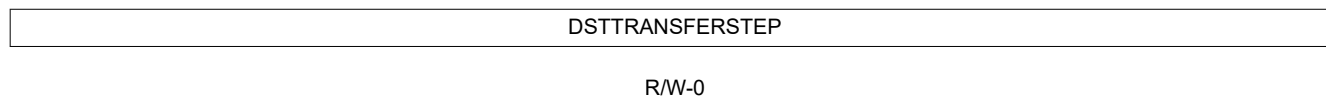
### 11.9.15 Destination Transfer Step Size Register (DST\_TRANSFER\_STEP) — EALLOW Protected

The destination transfer step size register (DST\_TRANSFER\_STEP) is shown in [Figure 11-21](#) and described in [Table 11-17](#). Only write to the TRANSFER register when the RUNSTS bit is 0 (DMA channel stopped or halted). Typically though, the values should only be configured when the channel is stopped.

**Figure 11-21. Destination Transfer Step Size Register (DST\_TRANSFER\_STEP)**

15

0



LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 11-17. Destination Transfer Step Size Register (DST\_TRANSFER\_STEP) Field Descriptions**

| Bit  | Field           | Value  | Description                                                                                                             |
|------|-----------------|--------|-------------------------------------------------------------------------------------------------------------------------|
| 15-0 | DSTTRANSFERSTEP |        | These bits specify the destination address pointer post-increment/decrement step size after processing a burst of data: |
|      |                 | 0x0FFF | Add 4095 to address                                                                                                     |
|      |                 | ...    | ...                                                                                                                     |
|      |                 | 0x0002 | Add 2 to address                                                                                                        |
|      |                 | 0x0001 | Add 1 to address                                                                                                        |
|      |                 | 0x0000 | No address change                                                                                                       |
|      |                 | 0xFFFF | Sub 1 from address                                                                                                      |
|      |                 | 0xFFFE | Sub 2 from address                                                                                                      |
|      |                 | ...    | ...                                                                                                                     |
|      |                 | 0xF000 | Sub 4096 from address                                                                                                   |
|      |                 |        | Only values from -4096 to 4095 are valid.                                                                               |

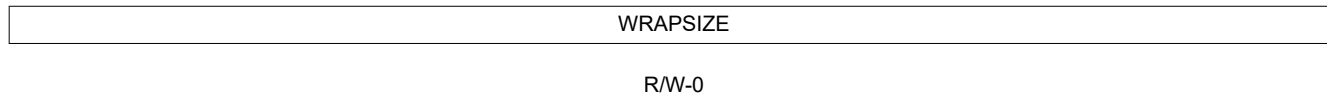
### 11.9.16 Source/Destination Wrap Size Register (SRC/DST\_WRAP\_SIZE) — EALLOW protected)

The source/destination wrap size register is shown in [Figure 11-22](#) and described in [Table 11-18](#).

**Figure 11-22. Source/Destination Wrap Size Register (SRC/DST\_WRAP\_SIZE)**

15

0



LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 11-18. Source/Destination Wrap Size Register (SRC/DST\_WRAP\_SIZE) Field Descriptions**

| Bit  | Field    | Value  | Description                                                                                                         |
|------|----------|--------|---------------------------------------------------------------------------------------------------------------------|
| 15-0 | WRAPSIZE |        | These bits specify the number of bursts to transfer before wrapping back to begin address pointer:                  |
|      |          | 0x0000 | Wrap after 1 burst                                                                                                  |
|      |          | 0x0001 | Wrap after 2 bursts                                                                                                 |
|      |          | 0x0002 | Wrap after 3 bursts                                                                                                 |
|      |          | ...    | ...                                                                                                                 |
|      |          | 0xFFFF | Wrap after 65536 bursts                                                                                             |
|      |          |        | To <i>disable</i> the wrap function, set the WRAPSIZE bit field to a number larger than the TRANSFERSIZE bit field. |

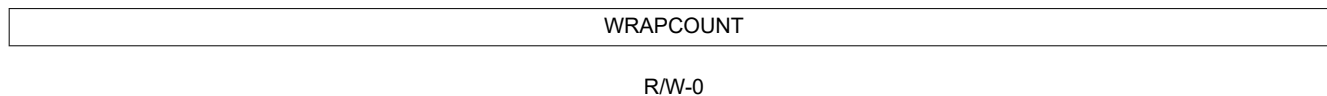
### 11.9.17 Source/Destination Wrap Count Register (SCR/DST\_WRAP\_COUNT)

The source/destination wrap count register (SCR/DST\_WRAP\_COUNT) is shown in [Figure 11-23](#) and described in [Table 11-19](#).

**Figure 11-23. Source/Destination Wrap Count Register (SCR/DST\_WRAP\_COUNT)**

15

0



LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 11-19. Source/Destination Wrap Count Register (SCR/DST\_WRAP\_COUNT) Field Descriptions**

| Bit  | Field     | Value  | Description                                                                 |
|------|-----------|--------|-----------------------------------------------------------------------------|
| 15-0 | WRAPCOUNT |        | These bits indicate the current wrap counter value:                         |
|      |           | 0x0000 | Wrap complete                                                               |
|      |           | 0x0001 | 1 burst left                                                                |
|      |           | 0x0002 | 2 burst left                                                                |
|      |           | ...    | ...                                                                         |
|      |           | 0xFFFF | 65535 burst left                                                            |
|      |           |        | The above values represent the state of the counter at the HALT conditions. |

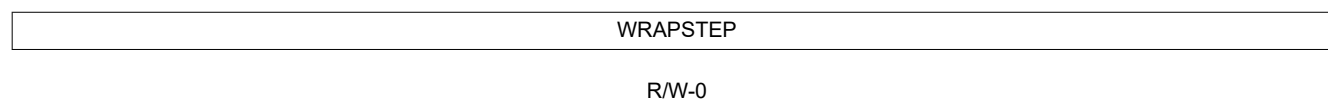
### 11.9.18 Source/Destination Wrap Step Size Registers (SRC/DST\_WRAP\_STEP) — EALLOW Protected

The source/destination wrap step size register (SRC/DST\_WRAP\_STEP) are shown in [Figure 11-24](#) and described in [Table 11-20](#). Only write to WRAP registers when RUNSTS bit is 0 (DMA channel stopped or halted). Typically though, the values should only be configured when the channel is stopped.

**Figure 11-24. Source/Destination Wrap Step Size Registers (SRC/DST\_WRAP\_STEP)**

15

0



LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 11-20. Source/Destination Wrap Step Size Registers (SRC/DST\_WRAP\_STEP) Field Descriptions**

| Bit  | Field    | Value  | Description                                                                                                        |
|------|----------|--------|--------------------------------------------------------------------------------------------------------------------|
| 15-0 | WRAPSTEP |        | These bits specify the source begin address pointer post-increment/decrement step size after wrap counter expires: |
|      |          | 0x0FFF | Add 4095 to address                                                                                                |
|      |          | ...    | ...                                                                                                                |
|      |          | 0x0002 | Add 2 to address                                                                                                   |
|      |          | 0x0001 | Add 1 to address                                                                                                   |
|      |          | 0x0000 | No address change                                                                                                  |
|      |          | 0xFFFF | Sub 1 from address                                                                                                 |
|      |          | 0xFFFE | Sub 2 from address                                                                                                 |
|      |          | ...    | ...                                                                                                                |
|      |          | 0xF000 | Sub 4096 from address                                                                                              |
|      |          |        | Only values from -4096 to 4095 are valid.                                                                          |

### 11.9.19 Shadow Source Begin and Current Address Pointer Registers (SRC\_BEG\_ADDR\_SHADOW/DST\_BEG\_ADDR\_SHADOW) — All EALLOW Protected

The shadow source begin and current address pointer registers (SRC\_BEG\_ADDR\_SHADOW/DST\_BEG\_ADDR\_SHADOW) are shown in [Figure 11-25](#) and described in [Table 11-21](#).

**Figure 11-25. Shadow Source Begin and Current Address Pointer Registers (SRC\_BEG\_ADDR\_SHADOW/DST\_BEG\_ADDR\_SHADOW)**

|          |    |         |   |
|----------|----|---------|---|
| 31       | 22 | 21      | 0 |
| Reserved |    | BEGADDR |   |
| R-0      |    | R/W-0   |   |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 11-21. Shadow Source Begin and Current Address Pointer Registers (SRC\_BEG\_ADDR\_SHADOW/DST\_BEG\_ADDR\_SHADOW) Field Descriptions**

| Bit   | Field    | Value | Description          |
|-------|----------|-------|----------------------|
| 31-22 | Reserved |       | Reserved             |
| 21-0  | BEGADDR  |       | 22-bit address value |

### 11.9.20 Active Source Begin and Current Address Pointer Registers (SRC\_BEG\_ADDR/DST\_BEG\_ADDR)

The active source begin and current address pointer registers (SRC\_BEG\_ADDR/DST\_BEG\_ADDR) are shown in [Table 11-22](#) and described in [Table 11-22](#).

**Figure 11-26. Active Source Begin and Current Address Pointer Registers (SRC\_BEG\_ADDR/DST\_BEG\_ADDR)**

|          |    |         |   |
|----------|----|---------|---|
| 31       | 22 | 21      | 0 |
| Reserved |    | BEGADDR |   |
| R-0      |    | R-0     |   |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 11-22. Active Source Begin and Current Address Pointer Registers (SRC\_BEG\_ADDR/DST\_BEG\_ADDR) Field Descriptions**

| Bit   | Field    | Value | Description          |
|-------|----------|-------|----------------------|
| 31-22 | Reserved |       | Reserved             |
| 21-0  | BEGADDR  |       | 22-bit address value |

### 11.9.21 Shadow Destination Begin and Current Address Pointer Registers (SRC\_ADDR\_SHADOW/DST\_ADDR\_SHADOW) — All EALLOW Protected

The shadow destination begin and current address pointer registers (SRC\_ADDR\_SHADOW/DST\_ADDR\_SHADOW) are shown in [Figure 11-27](#) and described in [Table 11-23](#).

**Figure 11-27. Shadow Destination Begin and Current Address Pointer Registers (SRC\_ADDR\_SHADOW/DST\_ADDR\_SHADOW)**

|          |    |       |   |
|----------|----|-------|---|
| 31       | 22 | 21    | 0 |
| Reserved |    | ADDR  |   |
| R-0      |    | R/W-0 |   |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 11-23. Shadow Destination Begin and Current Address Pointer Registers (SRC\_ADDR\_SHADOW/DST\_ADDR\_SHADOW) Field Descriptions**

| Bit   | Field    | Value | Description          |
|-------|----------|-------|----------------------|
| 31-22 | Reserved |       | Reserved             |
| 21-0  | ADDR     |       | 22-bit address value |

### 11.9.22 Active Destination Begin and Current Address Pointer Registers (SRC\_ADDR/DST\_ADDR)

The active destination begin and current address pointer registers (SRC\_ADDR/DST\_ADDR) are shown in [Figure 11-28](#) and described in [Table 11-24](#).

**Figure 11-28. Active Destination Begin and Current Address Pointer Registers (SRC\_ADDR/DST\_ADDR)**

|          |    |      |   |
|----------|----|------|---|
| 31       | 22 | 21   | 0 |
| Reserved |    | ADDR |   |
| R-0      |    | R-0  |   |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 11-24. Active Destination Begin and Current Address Pointer Registers (SRC\_ADDR/DST\_ADDR) Field Descriptions**

| Bit   | Field    | Value | Description          |
|-------|----------|-------|----------------------|
| 31-22 | Reserved |       | Reserved             |
| 21-0  | ADDR     |       | 22-bit address value |



This chapter describes the serial peripheral interface (SPI) which is a high-speed synchronous serial input and output (I/O) port that allows a serial bit stream of programmed length (one to 16 bits) to be shifted into and out of the device at a programmed bit-transfer rate. The SPI is normally used for communications between the MCU controller and external peripherals or another controller. Typical applications include external I/O or peripheral expansion using devices such as shift registers, display drivers, and analog-to-digital converters (ADCs). Multi-device communications are supported by the master or slave operation of the SPI. The port supports a 4-level, receive and transmit FIFO for reducing CPU servicing overhead.

|                                           |            |
|-------------------------------------------|------------|
| <b>12.1 Introduction.....</b>             | <b>760</b> |
| <b>12.2 System-Level Integration.....</b> | <b>762</b> |
| <b>12.3 SPI Operation.....</b>            | <b>766</b> |
| <b>12.4 Programming Procedure.....</b>    | <b>775</b> |
| <b>12.5 SPI Registers.....</b>            | <b>779</b> |

## 12.1 Introduction

### 12.1.1 Features

The SPI module features include:

- SPISOMI: SPI slave-output/master-input pin
- SPISIMO: SPI slave-input/master-output pin
- $\overline{\text{SPISTE}}$ : SPI slave transmit-enable pin
- SPICLK: SPI serial-clock pin

---

**Note**

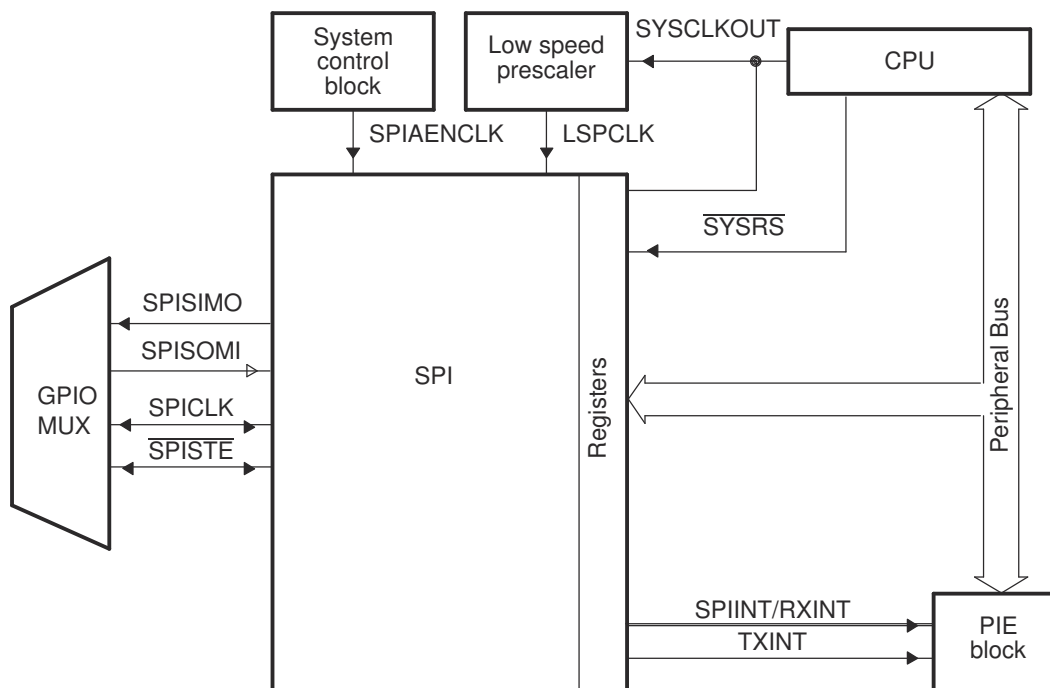
All four pins can be used as GPIO if the SPI module is not used.

---

- Two operational modes: Master and Slave
- Baud rate: 125 different programmable rates. The maximum baud rate that can be employed is limited by the maximum speed of the I/O buffers used on the SPI pins. See the device-specific data sheet for more details.
- Data word length: 1 to 16 data bits
- Four clocking schemes (controlled by clock polarity and clock phase bits) include:
  - Falling edge without phase delay: SPICLK active-high. SPI transmits data on the falling edge of the SPICLK signal and receives data on the rising edge of the SPICLK signal.
  - Falling edge with phase delay: SPICLK active-high. SPI transmits data one half-cycle ahead of the falling edge of the SPICLK signal and receives data on the falling edge of the SPICLK signal.
  - Rising edge without phase delay: SPICLK inactive-low. SPI transmits data on the rising edge of the SPICLK signal and receives data on the falling edge of the SPICLK signal.
  - Rising edge with phase delay: SPICLK inactive-low. SPI transmits data one half-cycle ahead of the rising edge of the SPICLK signal and receives data on the rising edge of the SPICLK signal.
- Simultaneous receive and transmit operation (transmit function can be disabled in software)
- Transmitter and receiver operations are accomplished through either interrupt- driven or polled algorithm
- 4-level transmit/receive FIFO
- Delayed transmit control
- 3-wire SPI mode
- $\overline{\text{SPISTE}}$  inversion for digital audio interface receive mode on devices with two SPI modules

## 12.1.2 Block Diagram

Figure 12-1 shows the SPI CPU interfaces.



**Figure 12-1. SPI CPU Interface**

## 12.2 System-Level Integration

This section describes the various functionality that is applicable to the device integration. These features require configuration of other modules in the device that are not within the scope of this chapter.

### 12.2.1 SPI Module Signals

[Table 12-1](#) classifies and provides a summary of the SPI module signals.

**Table 12-1. SPI Module Signal Summary**

| Signal Name                | Description                                                                                                      |
|----------------------------|------------------------------------------------------------------------------------------------------------------|
| <b>External Signals</b>    |                                                                                                                  |
| SPICLK                     | SPI clock                                                                                                        |
| SPISIMO                    | SPI slave in, master out                                                                                         |
| SPISOMI                    | SPI slave out, master in                                                                                         |
| $\overline{\text{SPISTE}}$ | SPI slave transmit enable                                                                                        |
| <b>Control</b>             |                                                                                                                  |
| SPI Clock Rate             | LSPCLK                                                                                                           |
| <b>Interrupt Signals</b>   |                                                                                                                  |
| SPIINT/SPIRXINT            | Transmit interrupt/ Receive Interrupt in non FIFO mode (referred to as SPIINT)<br>Receive interrupt in FIFO mode |
| SPITXINT                   | Transmit interrupt in FIFO mode                                                                                  |

### Special Considerations

The  $\overline{\text{SPISTE}}$  signal provides the ability to gate any spurious clock and data pulses when the SPI is in slave mode. A HIGH logic signal on  $\overline{\text{SPISTE}}$  will not allow the slave to receive data. This prevents the SPI slave from losing synchronization with the master. It is this reason that TI does not recommend that the  $\overline{\text{SPISTE}}$  always be tied to the active state.

If the SPI slave does ever lose synchronization with the master, toggling SPIWRESET resets the internal bit counter as well as the various status flags in the module. By resetting the bit counter, the SPI interprets the next clock transition as the first bit of a new transmission. The register bit fields that are reset by SPIWRESET are found in [Section 12.5](#).

### Configuring a GPIO to Emulate $\overline{\text{SPISTE}}$

In many systems, a SPI master may be connected to multiple SPI slaves using multiple instances of  $\overline{\text{SPISTE}}$ . Though this SPI module does not natively support multiple  $\overline{\text{SPISTE}}$  signals, it is possible to emulate this behavior in software using GPIOs. In this configuration, the SPI must be configured as the master. Rather than using the GPIO Mux to select  $\overline{\text{SPISTE}}$ , the application would configure pins to be GPIO outputs, one GPIO per SPI slave. Before transmitting any data, the application would drive the desired GPIO to the active state. Immediately after the transmission has been completed, the GPIO chip select would be driven to the inactive state. This process can be repeated for many slaves that share the SPICLK, SPISIMO, and SPISOMI lines.

### 12.2.2 Configuring Device Pins

The GPIO mux registers must be configured to connect this peripheral to the device pins.

Some IO functionality is defined by GPIO register settings independent of this peripheral. For input signals, the GPIO input qualification should be set to asynchronous mode by setting the appropriate GPxQSELn register bits to 11b. The internal pullups can be configured in the GPyPUD register.

See the *GPIO* chapter for more details on GPIO mux and settings.

### 12.2.3 SPI Interrupts

This section includes information on the available interrupts present in the SPI module.

The SPI module contains two interrupt lines: SPIINT/SPIRXINT and SPITXINT. When the SPI is operating in non-FIFO mode, all available interrupts are routed together to generate the single SPIINT interrupt. When FIFO mode is used, both SPIRXINT and SPITXINT can be generated.

#### SPIINT/SPIRXINT

When the SPI is operating in non-FIFO mode, the interrupt generated is called SPIINT. If FIFO enhancements are enabled, the interrupt is called SPIRXINT. These interrupts share the same interrupt vector in the Peripheral Interrupt Expansion (PIE) block.

In non-FIFO mode, two conditions can trigger an interrupt: a transmission is complete (INT\_FLAG), or there is overrun in the receiver (OVERRUN\_FLAG). Both of these conditions share the same interrupt vector: SPIINT.

The transmission complete flag (INT\_FLAG) indicates that the SPI has completed sending or receiving the last bit and is ready to be serviced. At the same time this bit is set, the received character is placed in the receiver buffer (SPIRXBUF). The INT\_FLAG will generate an interrupt on the SPIINT vector if the SPIINTENA bit is set.

The receiver overrun flag (OVERRUN\_FLAG) indicates that a transmit or receive operation has completed before the previous character has been read from the buffer. The OVERRUN\_FLAG will generate an interrupt on the SPIINT vector if the OVERRUNINTENA bit is set and OVERRUN\_FLAG was previously cleared.

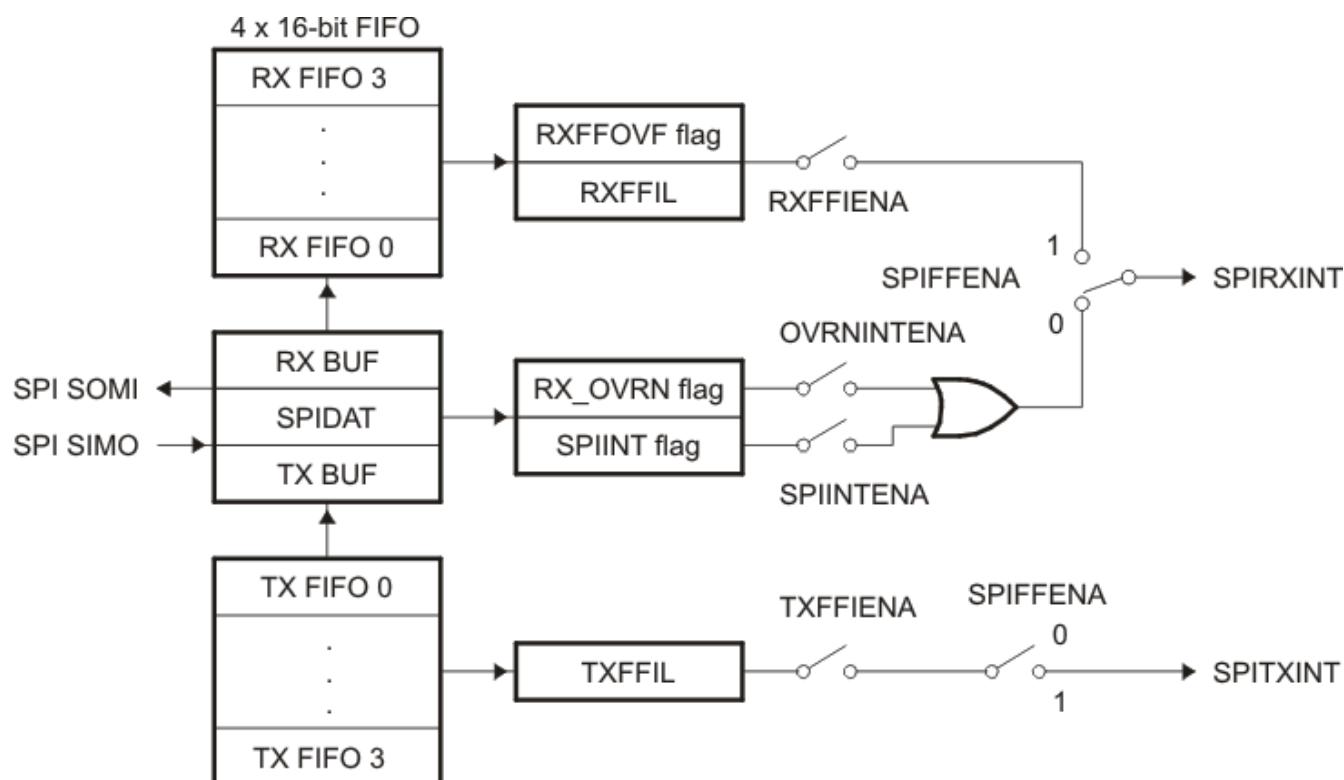
In FIFO mode, the SPI can interrupt the CPU upon a match condition between the current receive FIFO status (RXFFST) and the receive FIFO interrupt level (RXFFIL). If RXFFST is greater than or equal to RXFFIL, the receive FIFO interrupt flag (RXFFINT) will be set. SPIRXINT will be triggered in the PIE block if RXFFINT is set and the receive FIFO interrupt is enabled (RXFFIENA = 1).

#### SPITXINT

The SPITXINT interrupt is not available when the SPI is operating in non-FIFO mode.

In FIFO mode, the SPITXINT behavior is similar to the SPIRXINT. SPITXINT is generated upon a match condition between the current transmit FIFO status (TXFFST) and the transmit FIFO interrupt level (TXFFIL). If TXFFST is less than or equal to TXFFIL, the transmit FIFO interrupt flag (TXFFINT) will be set. SPITXINT will be triggered in the PIE block if TXFFINT is set and the transmit FIFO interrupt is enabled in the SPI module (TXFFIENA = 1).

[Figure 12-2](#) and [Table 12-2](#) show how these control bits influence the SPI interrupt generation.



**Figure 12-2. SPI Interrupt Flags and Enable Logic Generation**

**Table 12-2. SPI Interrupt Flag Modes**

| FIFO Options            | SPI Interrupt Source | Interrupt Flags | Interrupt Enables | FIFO Enable (SPIFFENA) | Interrupt Line <sup>(1)</sup> |
|-------------------------|----------------------|-----------------|-------------------|------------------------|-------------------------------|
| <b>SPI without FIFO</b> | Receive overrun      | RXOVRN          | OVRNINTENA        | 0                      | SPIRXINT                      |
|                         | Data receive         | SPIINT          | SPIINTENA         | 0                      | SPIRXINT                      |
|                         | Transmit empty       | SPIINT          | SPIINTENA         | 0                      | SPIRXINT                      |
| <b>SPI FIFO mode</b>    | FIFO receive         | RXFFIL          | RXFFIENA          | 1                      | SPIRXINT                      |
|                         | Transmit empty       | TXFFIL          | TXFFIENA          | 1                      | SPITXINT                      |

(1) In non-FIFO mode, SPIRXINT is the same name as the SPIINT interrupt in C28x devices.

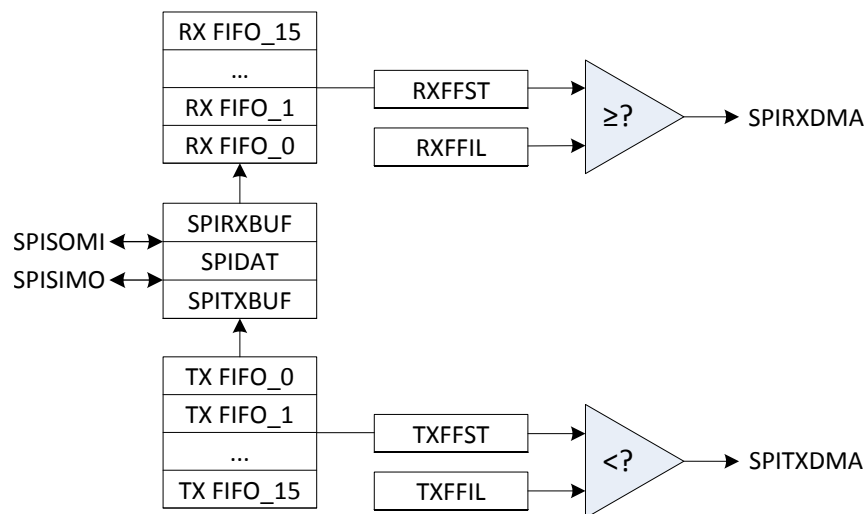
## 12.2.4 DMA Support

Both the CPU and DMA have access to the SPI data registers via the internal peripheral bus. This access is limited to 16-bit register read/writes. Each SPI module can generate two DMA events, SPITXDMA and SPIRXDMA. The DMA events are controlled by configuring the SPIFFTX.TXFFIL and SPIFFRX.RXFFIL appropriately. SPITXDMA activates when TXFFST is less than the interrupt level (TXFFIL). SPIRXDMA activates when RXFFST is greater than or equal to the interrupt level (RXFFIL).

The SPI must have FIFO enhancements enabled in order for the DMA triggers to be generated.

For more information on configuring the SPI for DMA transfers, refer to .

[Figure 12-3](#) is a block diagram showing the DMA trigger generation from the SPI module.



**Figure 12-3. SPI DMA Trigger Diagram**

## 12.3 SPI Operation

This section describes the various modes of operation of the SPI. Included are explanations of the operational modes, interrupts, data format, clock sources, and initialization. Typical timing diagrams for data transfers are given.

### 12.3.1 Introduction to Operation

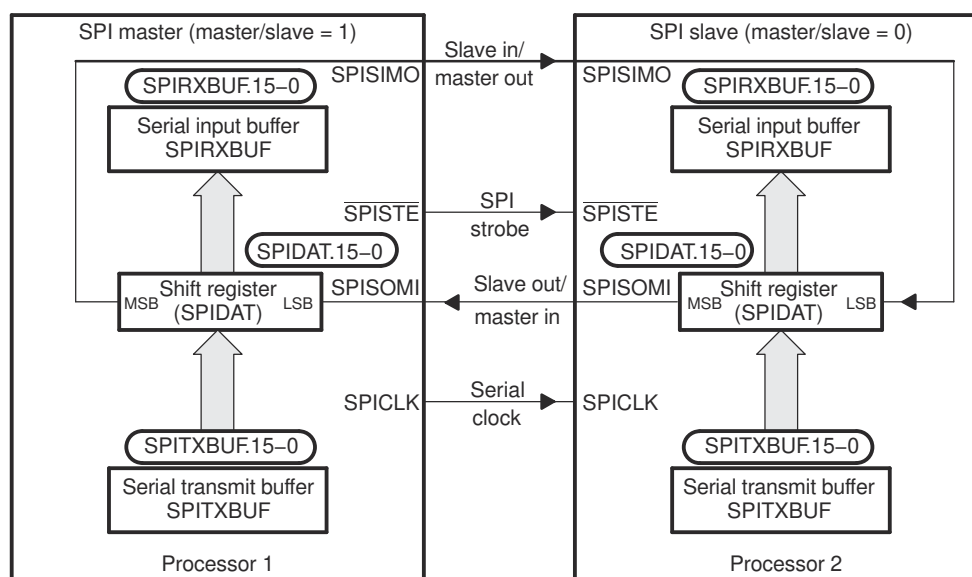
Figure 12-4 shows typical connections of the SPI for communications between two controllers: a master and a slave.

The master transfers data by sending the SPICLK signal. For both the slave and the master, data is shifted out of the shift registers on one edge of the SPICLK and latched into the shift register on the opposite SPICLK clock edge. If the CLK\_PHASE bit is high, data is transmitted and received a half-cycle before the SPICLK transition. As a result, both controllers send and receive data simultaneously. The application software determines whether the data is meaningful or dummy data. There are three possible methods for data transmission:

- Master sends data; slave sends dummy data.
- Master sends data; slave sends data.
- Master sends dummy data; slave sends data.

The master can initiate data transfer at any time because it controls the SPICLK signal. The software, however, determines how the master detects when the slave is ready to broadcast data.

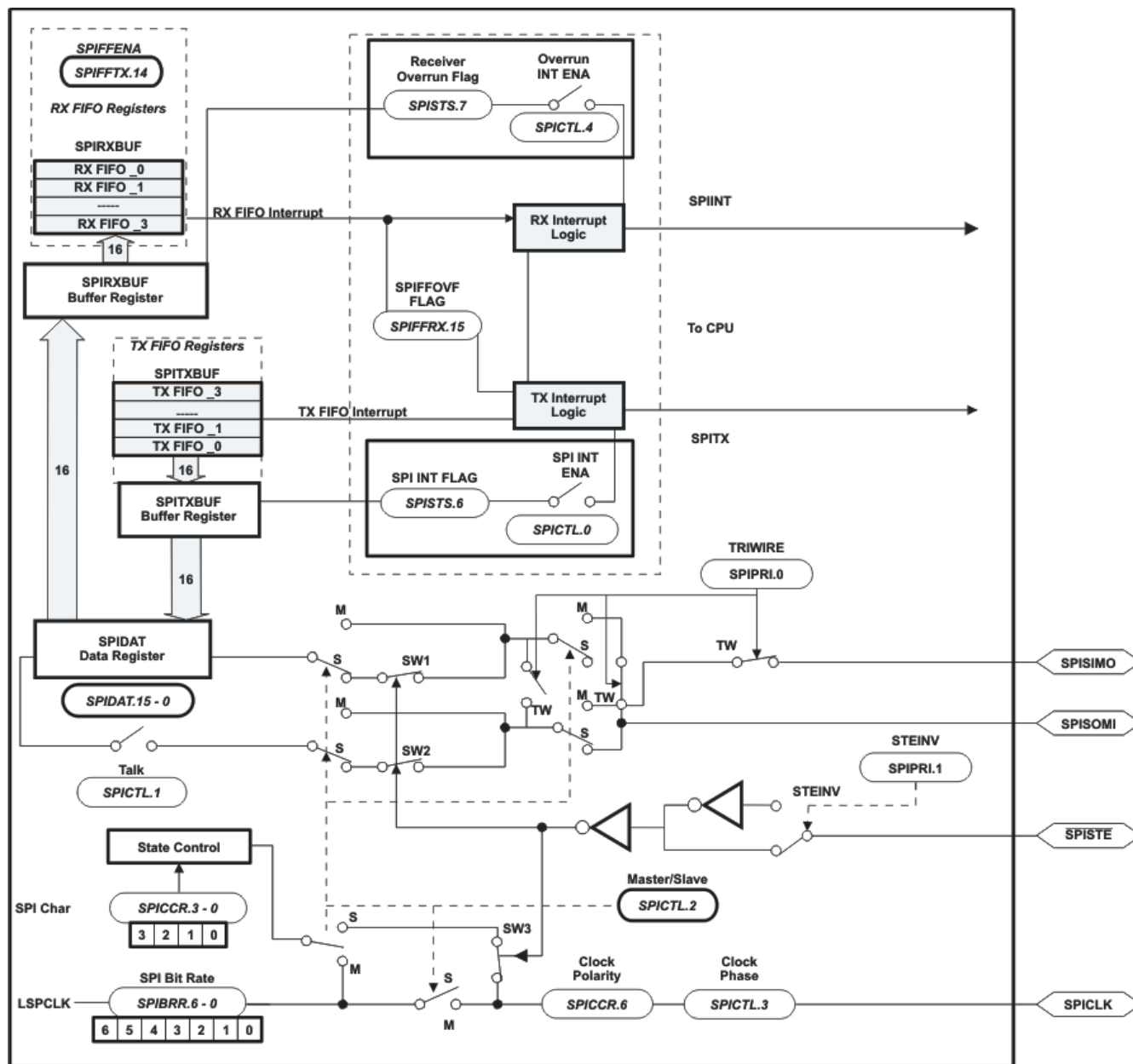
The SPI can operate in master or slave mode. The MASTER\_SLAVE bit selects the operating mode and the source of the SPICLK signal.



**Figure 12-4. SPI Master/Slave Connection**

Figure 12-5 is a block diagram of the SPI module showing all of the basic control blocks available on the SPI module.





### Figure 12-5. Serial Peripheral Interface Block Diagram

### 12.3.2 Master Mode

In master mode (MASTER\_SLAVE = 1), the SPI provides the serial clock on the SPICLK pin for the entire serial communications network. Data is output on the SPISIMO pin and latched from the SPISOMI pin.

The SPIBRR register determines both the transmit and receive bit transfer rate for the network. SPIBRR can select 125 different data transfer rates.

Data written to SPIDAT or SPITXBUF initiates data transmission on the SPISIMO pin, MSB (most-significant bit) first. Simultaneously, received data is shifted through the SPISOMI pin into the LSB (least-significant bit) of SPIDAT. When the selected number of bits has been transmitted, the received data is transferred to the SPIRXBUF (buffered receiver) for the CPU to read. Data is stored right-justified in SPIRXBUF.

When the specified number of data bits has been shifted through SPIDAT, the following events occur:

- SPIDAT contents are transferred to SPIRXBUF.
- INT\_FLAG bit is set to 1.
- If there is valid data in the transmit buffer SPITXBUF, as indicated by the transmit buffer full flag (BUFFULL\_FLAG), this data is transferred to SPIDAT and is transmitted; otherwise, SPICLK stops after all bits have been shifted out of SPIDAT.
- If the SPIINTENA bit is set to 1, an interrupt is asserted.

In a typical application, the  $\overline{\text{SPISTE}}$  pin serves as a chip-enable pin for a SPI slave device. This pin is driven low by the master before transmitting data to the slave and is taken high after the transmission is complete.

### 12.3.3 Slave Mode

In slave mode (MASTER\_SLAVE = 0), data shifts out on the SPISOMI pin and in on the SPISIMO pin. The SPICLK pin is used as the input for the serial shift clock, which is supplied from the external network master. The transfer rate is defined by this clock. The SPICLK input frequency should be no greater than the LSPCLK frequency divided by 4.

Data written to SPIDAT or SPITXBUF is transmitted to the network when appropriate edges of the SPICLK signal are received from the network master. A character written to the SPITXBUF register will be copied to the SPIDAT register when all bits of the current character in SPIDAT have been shifted out. If no character was previously copied to SPIDAT, then any character written to SPITXBUF will be immediately copied to SPIDAT. If a character was previously copied to SPIDAT, any data written to SPITXBUF will not be copied to SPIDAT until the current character in SPIDAT has been shifted out. To receive data, the SPI waits for the network master to send the SPICLK signal and then shifts the data on the SPISIMO pin into SPIDAT. If data is to be transmitted by the slave simultaneously, and SPIDAT has not been previously loaded, the character must be written to SPITXBUF before the beginning of the SPICLK signal.

When the TALK bit is cleared, data transmission is disabled, and the output line (SPISOMI) is put into the high-impedance state. If this occurs while a transmission is active, the current character is completely transmitted even though SPISOMI is forced into the high-impedance state. This ensures that the SPI is still able to receive incoming data correctly. This TALK bit allows many slave devices to be tied together on the network, but only one slave at a time is allowed to drive the SPISOMI line.

The  $\overline{\text{SPISTE}}$  pin operates as the slave-select pin. An active-low signal on the  $\overline{\text{SPISTE}}$  pin allows the slave SPI to transfer data to the serial data line; an inactive-high signal causes the slave SPI serial shift register to stop and its serial output pin to be put into the high-impedance state. This allows many slave devices to be tied together on the network, although only one slave device is selected at a time.

### 12.3.4 Data Format

The four-bit SPICHR register field specifies the number of bits in the data character (1 to 16). This information directs the state control logic to count the number of bits received or transmitted to determine when a complete character has been processed.

The following statements apply to characters with fewer than 16 bits:

- Data must be left-justified when written to SPIDAT and SPITXBUF.
- Data read back from SPIRXBUF is right-justified.
- SPIRXBUF contains the most recently received character, right-justified, plus any bits that remain from previous transmission(s) that have been shifted to the left (shown in [Example 12-1](#)).

#### Example 12-1. Transmission of Bit from SPIRXBUF

Conditions:

1. Transmission character length = 1 bit (specified in SPICHR bits)
2. The current value of SPIDAT = 737Bh

| SPIDAT (before transmission)  |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
|-------------------------------|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
|                               | 0 | 1 | 1 | 1 | 0 | 0 | 1 | 1 | 0 | 1 | 1 | 1 | 1 | 0 | 1 |
| SPIDAT (after transmission)   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
| (TXed) 0 ←                    | 1 | 1 | 1 | 0 | 0 | 1 | 1 | 0 | 1 | 1 | 1 | 1 | 0 | 1 | 1 |
|                               |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
| SPIRXBUF (after transmission) |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
|                               | 1 | 1 | 1 | 0 | 0 | 1 | 1 | 0 | 1 | 1 | 1 | 1 | 0 | 1 | 1 |

(1) x = 1, if SPISOMI data is high; x = 0, if SPISOMI data is low; master mode is assumed.

### 12.3.5 Baud Rate Selection

The SPI module supports 125 different baud rates and four different clock schemes. Depending on whether the SPI clock is in slave or master mode, the SPICLK pin can receive an external SPI clock signal or provide the SPI clock signal, respectively.

- In the slave mode, the SPI clock is received on the SPICLK pin from the external source and can be no greater than the LSPCLK frequency divided by 4.
- In the master mode, the SPI clock is generated by the SPI and is output on the SPICLK pin and can be no greater than the LSPCLK frequency divided by 4.

#### Note

The baud rate should be configured to not exceed the maximum rated GPIO toggle frequency. Refer to the device data sheet for the maximum GPIO toggle frequency

[Example 12-2](#) shows how to determine the SPI baud rates.

#### Example 12-2. Baud Rate Determination

For SPIBRR = 3 to 127:

$$\text{SPI Baud Rate} = \frac{\text{LSPCLK}}{(\text{SPIBRR} + 1)} \quad (5)$$

For SPIBRR = 0, 1, or 2:

$$\text{SPI Baud Rate} = \frac{\text{LSPCLK}}{4} \quad (6)$$

where:

LSPCLK = Low-speed peripheral clock frequency of the device

SPIBRR = Contents of the SPIBRR in the master SPI device

To determine what value to load into SPIBRR, you must know the device system clock (LSPCLK) frequency (that is device-specific) and the baud rate at which you will be operating.

[Example 12-3](#) shows how to calculate the baud rate of the SPI module .

#### Example 12-3. Baud Rate Calculation

$$\begin{aligned} \text{Maximum SPI Baud Rate} &= \frac{\text{LSPCLK}}{4} \\ &= \frac{40 \times 10^6}{4} \\ &= 10 \times 10^6 \text{ bauds} \end{aligned} \quad (7)$$

### 12.3.6 SPI Clocking Schemes

The clock polarity select bit (CLKPOLARITY) and the clock phase select bit (CLK\_PHASE) control four different clocking schemes on the SPICLK pin. CLKPOLARITY selects the active edge, either rising or falling, of the clock. CLK\_PHASE selects a half-cycle delay of the clock. The four different clocking schemes are as follows:

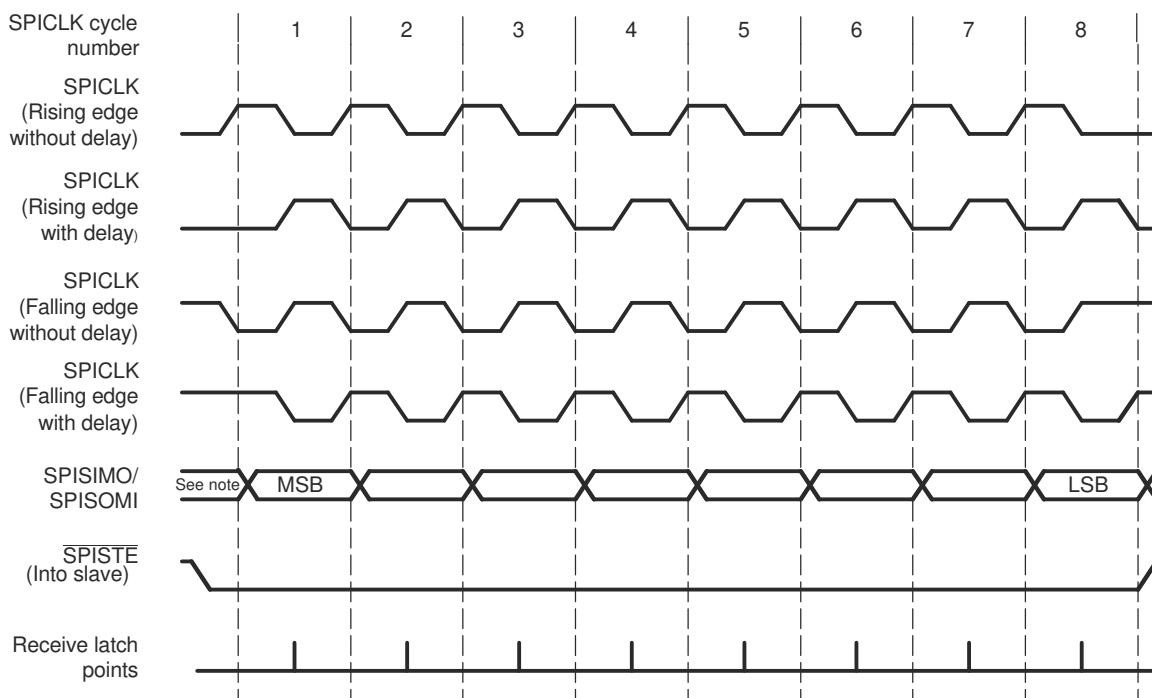
- **Falling Edge Without Delay.** The SPI transmits data on the falling edge of the SPICLK and receives data on the rising edge of the SPICLK.
- **Falling Edge With Delay.** The SPI transmits data one half-cycle ahead of the falling edge of the SPICLK signal and receives data on the falling edge of the SPICLK signal.
- **Rising Edge Without Delay.** The SPI transmits data on the rising edge of the SPICLK signal and receives data on the falling edge of the SPICLK signal.
- **Rising Edge With Delay.** The SPI transmits data one half-cycle ahead of the rising edge of the SPICLK signal and receives data on the rising edge of the SPICLK signal.

The selection procedure for the SPI clocking scheme is shown in [Table 12-3](#). Examples of these four clocking schemes relative to transmitted and received data are shown in [Figure 12-6](#).

**Table 12-3. SPI Clocking Scheme Selection Guide**

| SPICLK Scheme              | CLKPOLARITY | CLK_PHASE <sup>(1)</sup> |
|----------------------------|-------------|--------------------------|
| Rising edge without delay  | 0           | 0                        |
| Rising edge with delay     | 0           | 1                        |
| Falling edge without delay | 1           | 0                        |
| Falling edge with delay    | 1           | 1                        |

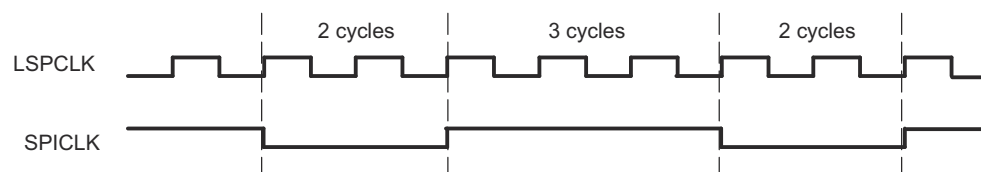
(1) The description of CLK\_PHASE and CLKPOLARITY differs between manufacturers. For proper operation, select the desired waveform to determine the clock phase and clock polarity settings.



**Note:** Previous data bit

**Figure 12-6. SPICLK Signal Options**

SPICLK symmetry is retained only when the result of  $(SPIBRR + 1)$  is an even value. When  $(SPIBRR + 1)$  is an odd value and  $SPIBRR$  is greater than 3, SPICLK becomes asymmetrical. The low pulse of SPICLK is one LSPCLK cycle longer than the high pulse when  $CLKPOLARITY$  bit is clear (0). When  $CLKPOLARITY$  bit is set to 1, the high pulse of the SPICLK is one LSPCLK cycle longer than the low pulse, as shown in [Figure 12-7](#).



**Figure 12-7. SPI: SPICLK-LSPCLK Characteristic When  $(BRR + 1)$  is Odd,  $BRR > 3$ , and  $CLKPOLARITY = 1$**

### 12.3.7 SPI FIFO Description

The following steps explain the FIFO features and help with programming the SPI FIFOs:

1. **Reset.** At reset the SPI powers up in standard SPI mode and the FIFO function is disabled. The FIFO registers SPIFFTX, SPIFFRX and SPIFFCT remain inactive.
2. **Standard SPI.** The standard 28x SPI mode will work with SPIINT/SPIRXINT as the interrupt source.
3. **Mode change.** FIFO mode is enabled by setting the SPIFFENA bit to 1 in the SPIFFTX register. SPIRST can reset the FIFO mode at any stage of its operation.
4. **Active registers.** All the SPI registers and SPI FIFO registers SPIFFTX, SPIFFRX, and SPIFFCT will be active.
5. **Interrupts.** FIFO mode has two interrupts one for the transmit FIFO, SPITXINT and one for the receive FIFO, SPIRXINT. SPIRXINT is the common interrupt for SPI FIFO receive, receive error and receive FIFO overflow conditions. The single SPIINT for both transmit and receive sections of the standard SPI will be disabled and this interrupt will service as SPI receive FIFO interrupt. For more information, refer to [Section 12.2.3](#).
6. **Buffers.** Transmit and receive buffers are each supplemented with a 4 word FIFO. The one-word transmit buffer (SPITXBUF) of the standard SPI functions as a transition buffer between the transmit FIFO and shift register. The one-word transmit buffer will be loaded from transmit FIFO only after the last bit of the shift register is shifted out.
7. **Delayed transfer.** The rate at which transmit words in the FIFO are transferred to transmit shift register is programmable. The SPIFFCT register bits (7–0) FFTXDLY7–FFTXDLY0 define the delay between the word transfer. The delay is defined in number SPI serial clock cycles. The 8-bit register could define a minimum delay of 0 SPICLK cycles and a maximum of 255 SPICLK cycles. With zero delay, the SPI module can transmit data in continuous mode with the FIFO words shifting out back to back. With the 255 clock delay, the SPI module can transmit data in a maximum delayed mode with the FIFO words shifting out with a delay of 255 SPICLK cycles between each words. The programmable delay facilitates glueless interface to various slow SPI peripherals, such as EEPROMs, ADC, DAC, and so on.
8. **FIFO status bits.** Both transmit and receive FIFOs have status bits TXFFST or RXFFST that define the number of words available in the FIFOs at any time. The transmit FIFO reset bit (TXFIFO) and receive reset bit (RXFIFO) will reset the FIFO pointers to zero when these bits are set to 1. The FIFOs will resume operation from start once these bits are cleared to zero.
9. **Programmable interrupt levels.** Both transmit and receive FIFOs can generate CPU interrupts. The transmit interrupt (SPITXINT) is generated whenever the transmit FIFO status bits (TXFFST) match (less than or equal to) the interrupt trigger level bits (TXFFIL). The receive interrupt (SPIRXINT) is generated whenever the receive FIFO status bits (RXFFST) match (greater than or equal to) the interrupt trigger level RXFFIL. This provides a programmable interrupt trigger for transmit and receive sections of the SPI. The default value for these trigger level bits will be 0x11111 for receive FIFO and 0x00000 for transmit FIFO, respectively.

### 12.3.8 SPI 3-Wire Mode Description

SPI 3-wire mode allows for SPI communication over three pins instead of the normal four pins.

In master mode, if the TRIWIRE bit is set, enabling 3-wire SPI mode, SPISIMOX becomes the bi-directional SPIMOMIx (SPI master out, master in) pin, and SPISOMIx is no longer used by the SPI. In slave mode, if the TRIWIRE bit is set, SPISOMIx becomes the bi-directional SPISISOx (SPI slave in, slave out) pin, and SPISIMOX is no longer used by the SPI.

Table 12-4 indicates the pin function differences between 3-wire and 4-wire SPI mode for a master and slave SPI.

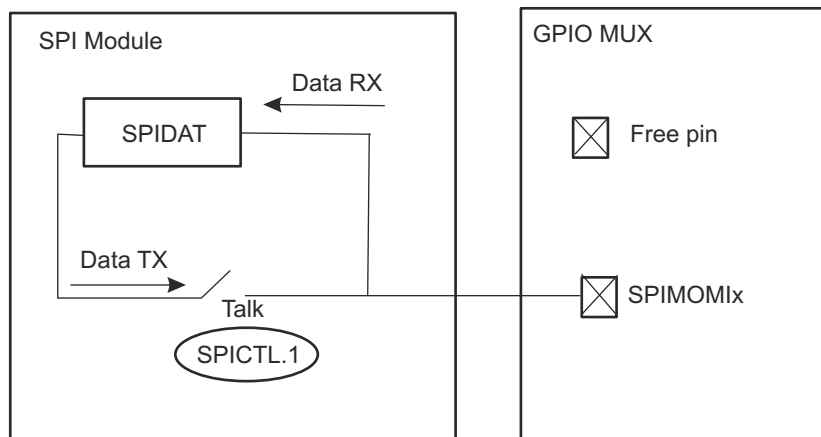
**Table 12-4. 4-wire vs. 3-wire SPI Pin Functions**

| 4-wire SPI | 3-wire SPI (Master) | 3-wire SPI (Slave) |
|------------|---------------------|--------------------|
| SPICLKx    | SPICLKx             | SPICLKx            |
| SPISTEx    | SPISTEx             | SPISTEx            |
| SPISIMOX   | SPIMOMIx            | Free               |
| SPISOMIx   | Free                | SPISISOx           |

Because in 3-wire mode, the receive and transmit paths within the SPI are connected, any data transmitted by the SPI module is also received by itself. The application software must take care to perform a dummy read to clear the SPI data register of the additional received data.

The TALK bit plays an important role in 3-wire SPI mode. The bit must be set to transmit data and cleared prior to reading data. In master mode, in order to initiate a read, the application software must write dummy data to the SPI data register (SPIDAT or SPIRXBUF) while the TALK bit is cleared (no data is transmitted out the SPIMOMI pin) before reading from the data register.

Figure 12-8 and Figure 12-9 illustrate 3-wire master and slave mode.



**Figure 12-8. SPI 3-wire Master Mode**

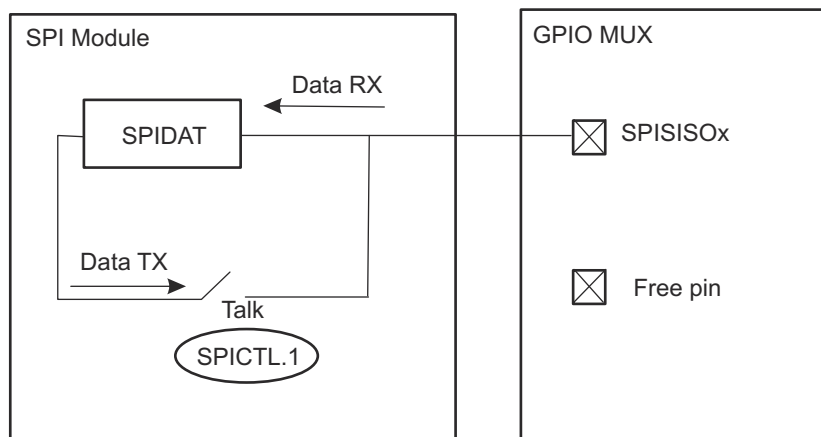

**Figure 12-9. SPI 3-wire Slave Mode**

Table 12-5 indicates how data is received or transmitted in the various SPI modes while the TALK bit is set or cleared.

**Table 12-5. 3-Wire SPI Pin Configuration**

| Pin Mode    | SPIPRI[TRIWIRE] | SPICTL[TALK] | SPISIMO             | SPISOMI             |
|-------------|-----------------|--------------|---------------------|---------------------|
| Master Mode |                 |              |                     |                     |
| 4-wire      | 0               | X            | TX                  | RX                  |
| 3-pin mode  | 1               | 0            | RX                  | Disconnect from SPI |
|             |                 | 1            | TX/RX               |                     |
| Slave Mode  |                 |              |                     |                     |
| 4-wire      | 0               | X            | RX                  | TX                  |
| 3-pin mode  | 1               | 0            | Disconnect from SPI | RX                  |
|             |                 | 1            |                     | TX/RX               |



## 12.4 Programming Procedure

This section describes the procedure for configuring the SPI for the various modes of operation.

### 12.4.1 Initialization Upon Reset

A system reset forces the SPI peripheral into the following default configuration:

- Unit is configured as a slave module (MASTER\_SLAVE = 0)
- Transmit capability is disabled (TALK = 0)
- Data is latched at the input on the falling edge of the SPICLK signal
- Character length is assumed to be one bit
- SPI interrupts are disabled
- Data in SPIDAT is reset to 0000h

### 12.4.2 Configuring the SPI

This section describes the procedure in which to configure the SPI module for operation. To prevent unwanted and unforeseen events from occurring during or as a result of initialization changes, clear the SPISWRESET bit before making initialization changes, and then set this bit after initialization is complete. While the SPI is held in reset (SPISWRESET = 0), configuration may be changed in any order. The following list shows the SPI configuration procedure in a logical order. However, the SPI registers can be written with single 16-bit writes, so the order is not required with the exception of SPISWRESET.

To change the SPI configuration:

1. Clear the SPI Software Reset bit (SPISWRESET) to 0 to force the SPI to the reset state.
2. Configure the SPI as desired:
  - Select either master or slave mode (MASTER\_SLAVE).
  - Choose SPICLK polarity and phase (CLKPOLARITY and CLK\_PHASE).
  - Set the desired baud rate (SPIBRR).
  - Set the SPI character length (SPICHR).
  - Clear the SPI Flags (OVERRUN\_FLAG, INT\_FLAG).
  - Enable  $\overline{\text{SPISTE}}$  inversion (STEINV), if needed.
  - Enable 3-wire mode (TRIWIRE), if needed.
  - If using FIFO enhancements:
    - Enable the FIFO enhancements (SPIFFENA).
    - Clear the FIFO Flags (TXFFINTCLR, RXFFOVFCLR, and RXFFINTCLR).
    - Release transmit and receive FIFO resets (TXFIFO and RXFIFORESET).
    - Release SPI FIFO channels from reset (SPIRST).
3. If interrupts are used:
  - In non-FIFO mode, enable the receiver overrun and/or SPI interrupts (OVERRUNINTENA and SPIINTENA).
  - In FIFO mode, set the transmit and receive interrupt levels (TXFFIL and RXFFIL) then enable the interrupts (TXFFIENA and RXFFIENA).
4. Set SPISWRESET to 1 to release the SPI from the reset state.

---

#### Note

Do not change the SPI configuration when communication is in progress.

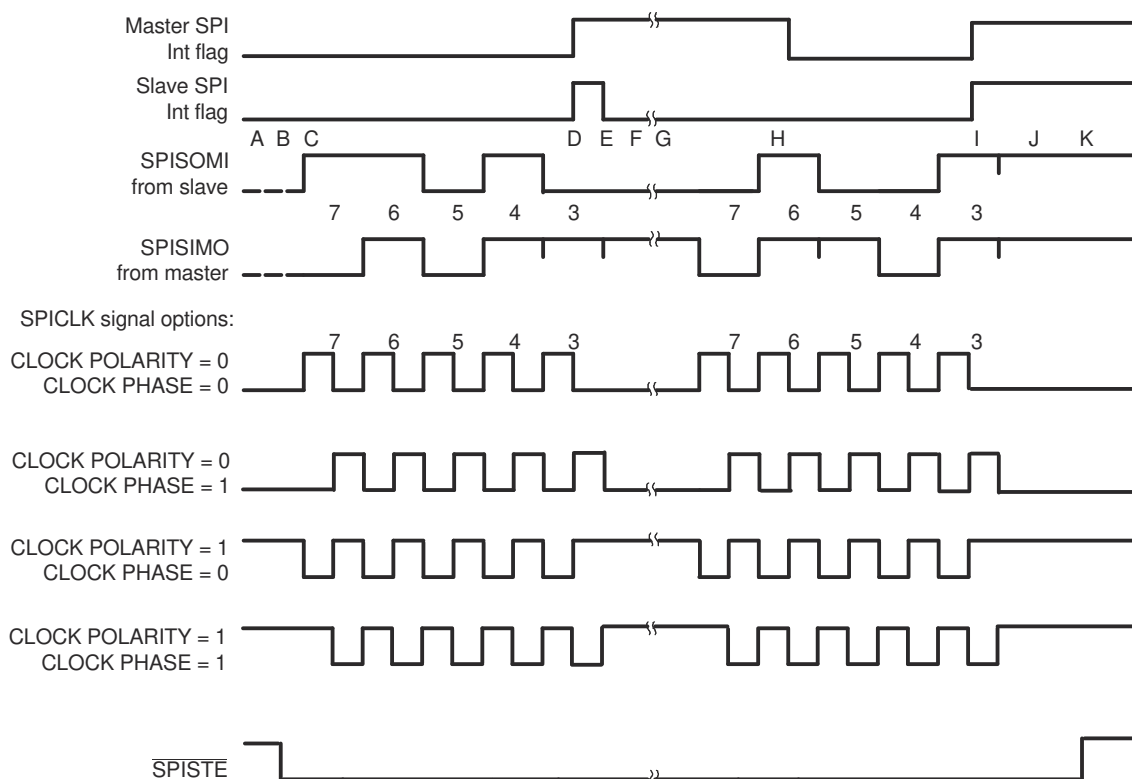
---

### 12.4.3 Data Transfer Example

The timing diagram shown in Figure 12-10 shows an SPI data transfer between two devices using a character length of five bits with the SPICLK being symmetrical.

The timing diagram with SPICLK asymmetrical (Figure 12-7) shares similar characterizations with Figure 12-10 except that the data transfer is one LSPCLK cycle longer per bit during the low pulse (CLKPOLARITY = 0) or during the high pulse (CLKPOLARITY = 1) of the SPICLK.

Figure 12-10 is applicable for 8-bit SPI only and is not for C28x devices that are capable of working with 16-bit data. The figure is shown for illustrative purposes only.



- A. Slave writes 0D0h to SPIDAT and waits for the master to shift out the data.
- B. Master sets the slave  $\overline{\text{SPISITE}}$  signal low (active).
- C. Master writes 058h to SPIDAT, which starts the transmission procedure.
- D. First byte is finished and sets the interrupt flags.
- E. Slave reads 0Bh from its SPIRXBUF (right-justified).
- F. Slave writes 04Ch to SPIDAT and waits for the master to shift out the data.
- G. Master writes 06Ch to SPIDAT, which starts the transmission procedure.
- H. Master reads 01Ah from the SPIRXBUF (right-justified).
- I. Second byte is finished and sets the interrupt flags.
- J. Master reads 89h and the slave reads 8Dh from their respective SPIRXBUF. After the user software masks off the unused bits, the master receives 09h and the slave receives 0Dh.
- K. Master clears the slave  $\overline{\text{SPISITE}}$  signal high (inactive).

**Figure 12-10. Five Bits per Character**

#### 12.4.4 SPI 3-Wire Mode Code Examples

In addition to the normal SPI initialization, to configure the SPI module for 3-wire mode, the TRIWIRE bit (SPIPRI.0) must be set to 1. After initialization, there are several considerations to take into account when transmitting and receiving data in 3-wire master and slave mode. The following examples demonstrate these considerations.

In 3-wire master mode, SPICLKx, SPISTEx, and SPISIMOX pins must be configured as SPI pins (SPISOMIx pin can be configured as non-SPI pin). When the master transmits, it receives the data it transmits (because SPISIMOX and SPISOMIx are connected internally in 3-wire mode). Therefore, the junk data received must be cleared from the receive buffer every time data is transmitted.

##### **Example 12-4. 3-Wire Master Mode Transmit**

```

Uint16 data;
Uint16 dummy;
    SpiaRegs.SPICTL.bit.TALK = 1;           // Enable Transmit path
    SpiaRegs.SPITXBUF = data; // Master transmits data
    while(SpiaRegs.SPISTS.bit.INT_FLAG !=1) {} // Waits until data rx'd
    dummy = SpiaRegs.SPIRXBUF;              // Clears junk data from itself
                                           // bc it rx'd same data tx'd

```

To receive data in 3-wire master mode, the master must clear the TALK (SPICTL.1) bit to 0 to close the transmit path and then transmit dummy data in order to initiate the transfer from the slave. Because the TALK bit is 0, unlike in transmit mode, the master dummy data does not appear on the SPISIMOX pin, and the master does not receive its own dummy data. Instead, the data from the slave is received by the master.

##### **Example 12-5. 3-Wire Master Mode Receive**

```

Uint16 rdata;
Uint16 dummy;
    SpiaRegs.SPICTL.bit.TALK = 0;           // Disable Transmit path
    SpiaRegs.SPITXBUF = dummy;              // Send dummy to start tx
    // NOTE: because TALK = 0, data does not tx onto SPISIMOA pin
    while(SpiaRegs.SPISTS.bit.INT_FLAG !=1) {} // Wait until data received
    rdata = SpiaRegs.SPIRXBUF;              // Master reads data

```

In 3-wire slave mode, SPICLKx, SPISTEx, and SPISOMIx pins must be configured as SPI pins (SPISIMOX pin can be configured as non-SPI pin). Like in master mode, when transmitting, the slave receives the data it transmits and must clear this junk data from its receive buffer.

##### **Example 12-6. 3-Wire Slave Mode Transmit**

```

Uint16 data;
Uint16 dummy;
    SpiaRegs.SPICTL.bit.TALK = 1;           // Enable Transmit path
    SpiaRegs.SPITXBUF = data;              // Slave transmits data
    while(SpiaRegs.SPISTS.bit.INT_FLAG !=1) {} // Wait until data rx'd
    dummy = SpiaRegs.SPIRXBUF;              // Clears junk data from itself

```

As in 3-wire master mode, the TALK bit must be cleared to 0. Otherwise, the slave receives data normally.

### Example 12-7. 3-Wire Slave Mode Receive

```

Uint16 rdata;
SpiRegs.SPICTL.bit.TALK = 0;           // Disable Transmit path
while(SpiRegs.SPISTS.bit.INT_FLAG !=1) {} // Waits until data rx'd
rdata = SpiRegs.SPIRXBUF;               // Slave reads data

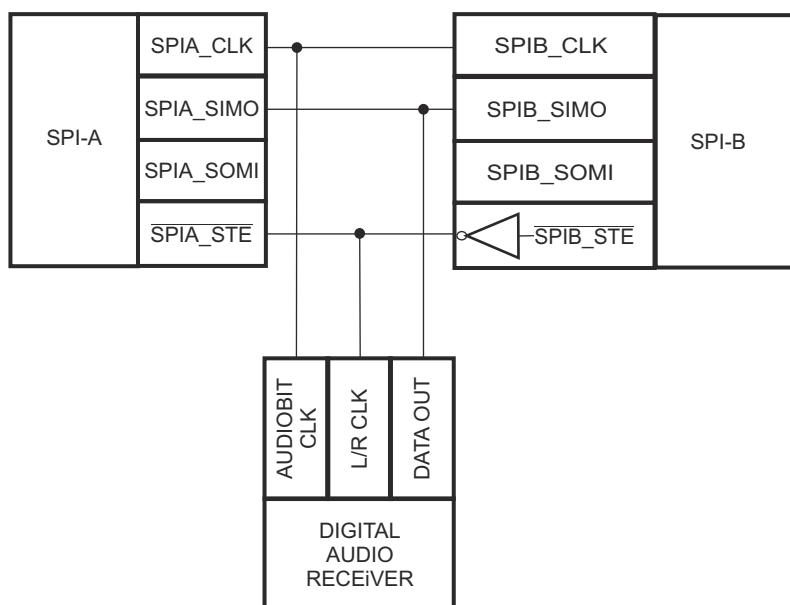
```

## 12.4.5 SPI STEINV Bit in Digital Audio Transfers

On those devices with two SPI modules, enabling the STEINV bit on one of the SPI modules allows the pair of SPIs to receive both left and right-channel digital audio data in slave mode. The SPI module that receives a normal active-low  $\overline{\text{SPISTE}}$  signal stores right-channel data, and the SPI module that receives an inverted active-high  $\overline{\text{SPISTE}}$  signal stores left-channel data from the master. To receive digital audio data from a digital audio interface receiver, the SPI modules can be connected as shown in Figure 12-11.

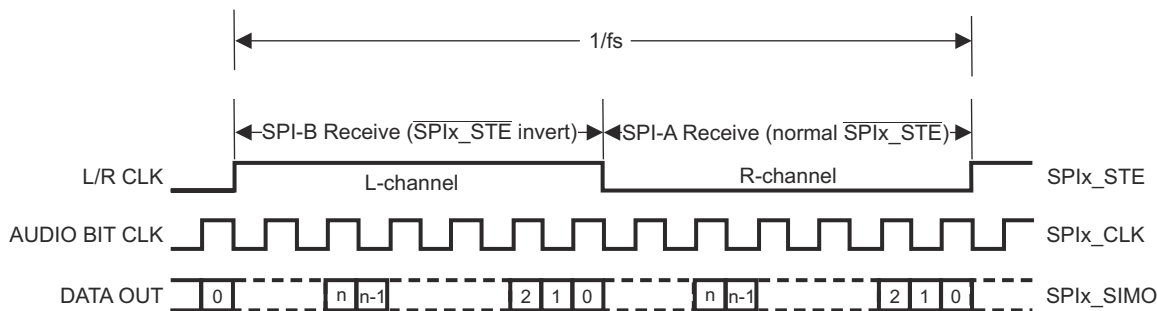
### Note

This configuration is only applicable to slave mode (MASTER\_SLAVE = 0). When the SPI is configured as master (MASTER\_SLAVE = 1), the STEINV bit will have no effect on the  $\overline{\text{SPISTE}}$  pin.



**Figure 12-11. SPI Digital Audio Receiver Configuration Using Two SPIs**

Standard C28x SPI timing requirements limit the number of digital audio interface formats supported using the 2-SPI configuration with the STEINV bit. See your device-specific data sheet electrical specifications for SPI timing requirements. With the SPI clock phase configured such that the CLKPOLARITY bit is 0 and the CLK\_PHASE bit is 1 (data latched on rising edge of clock), standard right-justified digital audio interface data format is supported as shown in Figure 12-12.



**Figure 12-12. Standard Right-Justified Digital Audio Data Format**

## 12.5 SPI Registers

This section describes the Serial Peripheral Interface registers. It is important to note that the SPI registers only allow 16-bit accesses.

### 12.5.1 SPI Base Addresses

**Table 12-6. SPI Base Address Table**

| Bit Field Name |           | Base Address |
|----------------|-----------|--------------|
| Instance       | Structure |              |
| SpiaRegs       | SPI_REGS  | 0x0000_7040  |
| SpibRegs       | SPI_REGS  | 0x0000_7740  |

### 12.5.2 SPI\_REGS Registers

[Table 12-7](#) lists the SPI\_REGS registers. All register offset addresses not listed in [Table 12-7](#) should be considered as reserved locations and the register contents should not be modified.

**Table 12-7. SPI\_REGS Registers**

| Offset | Acronym  | Register Name                      | Write Protection | Section                           |
|--------|----------|------------------------------------|------------------|-----------------------------------|
| 0h     | SPICCR   | SPI Configuration Control Register |                  | <a href="#">Section 12.5.2.1</a>  |
| 1h     | SPICTL   | SPI Operation Control Register     |                  | <a href="#">Section 12.5.2.2</a>  |
| 2h     | SPISTS   | SPI Status Register                |                  | <a href="#">Section 12.5.2.3</a>  |
| 4h     | SPIBRR   | SPI Baud Rate Register             |                  | <a href="#">Section 12.5.2.4</a>  |
| 6h     | SPIRXEMU | SPI Emulation Buffer Register      |                  | <a href="#">Section 12.5.2.5</a>  |
| 7h     | SPIRXBUF | SPI Serial Input Buffer Register   |                  | <a href="#">Section 12.5.2.6</a>  |
| 8h     | SPITXBUF | SPI Serial Output Buffer Register  |                  | <a href="#">Section 12.5.2.7</a>  |
| 9h     | SPIDAT   | SPI Serial Data Register           |                  | <a href="#">Section 12.5.2.8</a>  |
| Ah     | SPIFFTX  | SPI FIFO Transmit Register         |                  | <a href="#">Section 12.5.2.9</a>  |
| Bh     | SPIFFRX  | SPI FIFO Receive Register          |                  | <a href="#">Section 12.5.2.10</a> |
| Ch     | SPIFFCT  | SPI FIFO Control Register          |                  | <a href="#">Section 12.5.2.11</a> |
| Fh     | SPIPRI   | SPI Priority Control Register      |                  | <a href="#">Section 12.5.2.12</a> |

Complex bit access types are encoded to fit into small table cells. [Table 12-8](#) shows the codes that are used for access types in this section.

**Table 12-8. SPI\_REGS Access Type Codes**

| Access Type              | Code    | Description                                                                                                                                                                                                                                                                          |
|--------------------------|---------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Read Type                |         |                                                                                                                                                                                                                                                                                      |
| R                        | R       | Read                                                                                                                                                                                                                                                                                 |
| RC                       | R<br>C  | Read<br>to Clear                                                                                                                                                                                                                                                                     |
| Write Type               |         |                                                                                                                                                                                                                                                                                      |
| W                        | W       | Write                                                                                                                                                                                                                                                                                |
| W1C                      | W<br>1C | Write<br>1 to clear                                                                                                                                                                                                                                                                  |
| Reset or Default Value   |         |                                                                                                                                                                                                                                                                                      |
| -n                       |         | Value after reset or the default value                                                                                                                                                                                                                                               |
| Register Array Variables |         |                                                                                                                                                                                                                                                                                      |
| i,j,k,l,m,n              |         | When these variables are used in a register name, an offset, or an address, they refer to the value of a register array where the register is part of a group of repeating registers. The register groups form a hierarchical structure and the array is represented with a formula. |
| y                        |         | When this variable is used in a register name, an offset, or an address it refers to the value of a register array.                                                                                                                                                                  |

### 12.5.2.1 SPI Configuration Control Register (SPICCR) (Offset = 0h) [reset = 0h]

SPICCR controls the setup of the SPI for operation.

**Figure 12-13. SPI Configuration Control Register (SPICCR)**

|            |             |          |        |          |    |   |   |
|------------|-------------|----------|--------|----------|----|---|---|
| 15         | 14          | 13       | 12     | 11       | 10 | 9 | 8 |
| RESERVED   |             |          |        |          |    |   |   |
| R-0h       |             |          |        |          |    |   |   |
| 7          | 6           | 5        | 4      | 3        | 2  | 1 | 0 |
| SPISWRESET | CLKPOLARITY | RESERVED | SPILBK | SPICCHAR |    |   |   |
| R/W-0h     | R/W-0h      | R-0h     | R/W-0h | R/W-0h   |    |   |   |

**Table 12-9. SPI Configuration Control Register (SPICCR) Field Descriptions**

| Bit  | Field      | Type | Reset | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|------|------------|------|-------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-8 | RESERVED   | R    | 0h    | Reserved                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| 7    | SPISWRESET | R/W  | 0h    | <p>SPI Software Reset</p> <p>When changing configuration, you should clear this bit before the changes and set this bit before resuming operation.</p> <p>Reset type: SYSRSn</p> <p>0h (R/W) = Initializes the SPI operating flags to the reset condition. Specifically, the RECEIVER OVERRUN Flag bit (SPISTS.7), the SPI INT FLAG bit (SPISTS.6), and the TXBUF FULL Flag bit (SPISTS.5) are cleared. SPISTE will become inactive. SPICLK will be immediately driven to 0 regardless of the clock polarity. The SPI configuration remains unchanged.</p> <p>1h (R/W) = SPI is ready to transmit or receive the next character. When the SPI SW RESET bit is a 0, a character written to the transmitter will not be shifted out when this bit is set. A new character must be written to the serial data register. SPICLK will be returned to its inactive state one SPICLK cycle after this bit is set.</p> |

**Table 12-9. SPI Configuration Control Register (SPICCR) Field Descriptions (continued)**

| Bit | Field       | Type | Reset | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|-----|-------------|------|-------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 6   | CLKPOLARITY | R/W  | 0h    | <p>Shift Clock Polarity</p> <p>This bit controls the polarity of the SPICLK signal. CLOCK POLARITY and POLARITY CLOCK PHASE (SPICTL.3) control four clocking schemes on the SPICLK pin.</p> <p>Reset type: SYSRSn</p> <p>0h (R/W) = Data is output on rising edge and input on falling edge. When no SPI data is sent, SPICLK is at low level. The data input and output edges depend on the value of the CLOCK PHASE bit (SPICTL.3) as follows:</p> <ul style="list-style-type: none"> <li>- CLOCK PHASE = 0: Data is output on the rising edge of the SPICLK signal. Input data is latched on the falling edge of the SPICLK signal.</li> <li>- CLOCK PHASE = 1: Data is output one half-cycle before the first rising edge of the SPICLK signal and on subsequent falling edges of the SPICLK signal. Input data is latched on the rising edge of the SPICLK signal.</li> </ul> <p>1h (R/W) = Data is output on falling edge and input on rising edge. When no SPI data is sent, SPICLK is at high level. The data input and output edges depend on the value of the CLOCK PHASE bit (SPICTL.3) as follows:</p> <ul style="list-style-type: none"> <li>- CLOCK PHASE = 0: Data is output on the falling edge of the SPICLK signal. Input data is latched on the rising edge of the SPICLK signal.</li> <li>- CLOCK PHASE = 1: Data is output one half-cycle before the first falling edge of the SPICLK signal and on subsequent rising edges of the SPICLK signal. Input data is latched on the falling edge of the SPICLK signal.</li> </ul> |
| 5   | RESERVED    | R    | 0h    | Reserved                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| 4   | SPILBK      | R/W  | 0h    | <p>SPI Loopback Mode Select</p> <p>Loopback mode allows module validation during device testing. This mode is valid only in master mode of the SPI.</p> <p>Reset type: SYSRSn</p> <p>0h (R/W) = SPI loopback mode disabled. This is the default value after reset.</p> <p>1h (R/W) = SPI loopback mode enabled, SIMO/SOMI lines are connected internally. Used for module self-tests.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |



**Table 12-9. SPI Configuration Control Register (SPICCR) Field Descriptions (continued)**

| Bit | Field    | Type | Reset | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|-----|----------|------|-------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 3-0 | SPICCHAR | R/W  | 0h    | <p>Character Length Control Bits</p> <p>These four bits determine the number of bits to be shifted in or SPI CHAR0 out as a single character during one shift sequence.</p> <p>SPICCHAR = Word length - 1</p> <p>Reset type: SYSRSn</p> <p>0h (R/W) = 1-bit word</p> <p>1h (R/W) = 2-bit word</p> <p>2h (R/W) = 3-bit word</p> <p>3h (R/W) = 4-bit word</p> <p>4h (R/W) = 5-bit word</p> <p>5h (R/W) = 6-bit word</p> <p>6h (R/W) = 7-bit word</p> <p>7h (R/W) = 8-bit word</p> <p>8h (R/W) = 9-bit word</p> <p>9h (R/W) = 10-bit word</p> <p>Ah (R/W) = 11-bit word</p> <p>Bh (R/W) = 12-bit word</p> <p>Ch (R/W) = 13-bit word</p> <p>Dh (R/W) = 14-bit word</p> <p>Eh (R/W) = 15-bit word</p> <p>Fh (R/W) = 16-bit word</p> |

### 12.5.2.2 SPI Operation Control (SPICTL) Register (Offset = 1h) [reset = 0h]

SPICTL controls data transmission, the SPI's ability to generate interrupts, the SPICLK phase, and the operational mode (slave or master).

**Figure 12-14. SPI Operation Control (SPICTL) Register**

|          |    |    |                   |           |              |        |           |
|----------|----|----|-------------------|-----------|--------------|--------|-----------|
| 15       | 14 | 13 | 12                | 11        | 10           | 9      | 8         |
| RESERVED |    |    |                   |           |              |        |           |
| R-0h     |    |    |                   |           |              |        |           |
| 7        | 6  | 5  | 4                 | 3         | 2            | 1      | 0         |
| RESERVED |    |    | OVERRUNINT<br>ENA | CLK_PHASE | MASTER_SLAVE | TALK   | SPIINTENA |
| R-0h     |    |    | R/W-0h            | R/W-0h    | R/W-0h       | R/W-0h | R/W-0h    |

**Table 12-10. SPI Operation Control (SPICTL) Register Field Descriptions**

| Bit  | Field         | Type | Reset | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|------|---------------|------|-------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-5 | RESERVED      | R    | 0h    | Reserved                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| 4    | OVERRUNINTENA | R/W  | 0h    | <p>Overrun Interrupt Enable</p> <p>Overrun Interrupt Enable. Setting this bit causes an interrupt to be generated when the RECEIVER OVERRUN Flag bit (SPISTS.7) is set by hardware. Interrupts generated by the RECEIVER OVERRUN Flag bit and the SPI INT FLAG bit (SPISTS.6) share the same interrupt vector.</p> <p>Reset type: SYSRSn</p> <p>0h (R/W) = Disable RECEIVER OVERRUN interrupts.</p> <p>1h (R/W) = Enable RECEIVER_OVERRUN interrupts.</p>                                                                                                                                                                                                                         |
| 3    | CLK_PHASE     | R/W  | 0h    | <p>SPI Clock Phase Select</p> <p>This bit controls the phase of the SPICLK signal. CLOCK PHASE and CLOCK POLARITY (SPICCR.6) make four different clocking schemes possible (see clocking figures in SPI chapter). When operating with CLOCK PHASE high, the SPI (master or slave) makes the first bit of data available after SPIDAT is written and before the first edge of the SPICLK signal, regardless of which SPI mode is being used.</p> <p>Reset type: SYSRSn</p> <p>0h (R/W) = Normal SPI clocking scheme, depending on the CLOCK POLARITY bit (SPICCR.6).</p> <p>1h (R/W) = SPICLK signal delayed by one half-cycle. Polarity determined by the CLOCK POLARITY bit.</p> |
| 2    | MASTER_SLAVE  | R/W  | 0h    | <p>SPI Network Mode Control</p> <p>This bit determines whether the SPI is a network master or slave. SLAVE During reset initialization, the SPI is automatically configured as a network slave.</p> <p>Reset type: SYSRSn</p> <p>0h (R/W) = SPI is configured as a slave.</p> <p>1h (R/W) = SPI is configured as a master.</p>                                                                                                                                                                                                                                                                                                                                                    |

**Table 12-10. SPI Operation Control (SPICTL) Register Field Descriptions (continued)**

| Bit | Field     | Type | Reset | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|-----|-----------|------|-------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1   | TALK      | R/W  | 0h    | <p><b>Transmit Enable</b><br/>The TALK bit can disable data transmission (master or slave) by placing the serial data output in the high-impedance state. If this bit is disabled during a transmission, the transmit shift register continues to operate until the previous character is shifted out. When the TALK bit is disabled, the SPI is still able to receive characters and update the status flags. TALK is cleared (disabled) by a system reset.</p> <p>Reset type: SYSRSn</p> <p>0h (R/W) = Disables transmission:</p> <ul style="list-style-type: none"> <li>- Slave mode operation: If not previously configured as a general-purpose I/O pin, the SPISOMI pin will be put in the high-impedance state.</li> <li>- Master mode operation: If not previously configured as a general-purpose I/O pin, the SPISIMO pin will be put in the high-impedance state.</li> </ul> <p>1h (R/W) = Enables transmission For the 4-pin option, ensure to enable the receiver's SPISTEn input pin.</p> |
| 0   | SPIINTENA | R/W  | 0h    | <p><b>SPI Interrupt Enable</b><br/>This bit controls the SPI's ability to generate a transmit/receive interrupt. The SPI INT FLAG bit (SPISTS.6) is unaffected by this bit.</p> <p>Reset type: SYSRSn</p> <p>0h (R/W) = Disables the interrupt.</p> <p>1h (R/W) = Enables the interrupt.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |

### 12.5.2.3 SPI Status (SPISTS) Register (Offset = 2h) [reset = 0h]

SPISTS contains interrupt and status bits.

**Figure 12-15. SPI Status (SPISTS) Register**

|              |          |              |          |    |    |   |   |
|--------------|----------|--------------|----------|----|----|---|---|
| 15           | 14       | 13           | 12       | 11 | 10 | 9 | 8 |
| RESERVED     |          |              |          |    |    |   |   |
| R-0h         |          |              |          |    |    |   |   |
| 7            | 6        | 5            | 4        | 3  | 2  | 1 | 0 |
| OVERRUN_FLAG | INT_FLAG | BUFFULL_FLAG | RESERVED |    |    |   |   |
| W1C-0h       | RC-0h    | R-0h         | R-0h     |    |    |   |   |

**Table 12-11. SPI Status (SPISTS) Register Field Descriptions**

| Bit  | Field        | Type | Reset | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|------|--------------|------|-------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-8 | RESERVED     | R    | 0h    | Reserved                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| 7    | OVERRUN_FLAG | W1C  | 0h    | <p>SPI Receiver Overrun Flag</p> <p>This bit is a read/clear-only flag. The SPI hardware sets this bit when a receive or transmit operation completes before the previous character has been read from the buffer. The bit is cleared in one of three ways:</p> <ul style="list-style-type: none"> <li>- Writing a 1 to this bit</li> <li>- Writing a 0 to SPI SW RESET (SPICCR.7)</li> <li>- Resetting the system</li> </ul> <p>If the OVERRUN INT ENA bit (SPICTL.4) is set, the SPI requests only one interrupt upon the first occurrence of setting the RECEIVER OVERRUN Flag bit. Subsequent overruns will not request additional interrupts if this flag bit is already set. This means that in order to allow new overrun interrupt requests the user must clear this flag bit by writing a 1 to SPISTS.7 each time an overrun condition occurs. In other words, if the RECEIVER OVERRUN Flag bit is left set (not cleared) by the interrupt service routine, another overrun interrupt will not be immediately re-entered when the interrupt service routine is exited.</p> <p>Reset type: SYSRSn</p> <p>0h (R/W) = A receive overrun condition has not occurred.</p> <p>1h (R/W) = The last received character has been overwritten and therefore lost (when the SPIRXBUF was overwritten by the SPI module before the previous character was read by the user application).</p> <p>Writing a '1' will clear this bit. The RECEIVER OVERRUN Flag bit should be cleared during the interrupt service routine because the RECEIVER OVERRUN Flag bit and SPI INT FLAG bit (SPISTS.6) share the same interrupt vector. This will alleviate any possible doubt as to the source of the interrupt when the next byte is received.</p> |

**Table 12-11. SPI Status (SPISTS) Register Field Descriptions (continued)**

| Bit | Field        | Type | Reset | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|-----|--------------|------|-------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 6   | INT_FLAG     | RC   | 0h    | <p>SPI Interrupt Flag</p> <p>SPI INT FLAG is a read-only flag. Hardware sets this bit to indicate that the SPI has completed sending or receiving the last bit and is ready to be serviced. This flag causes an interrupt to be requested if the SPI INT ENA bit (SPICTL.0) is set. The received character is placed in the receiver buffer at the same time this bit is set. This bit is cleared in one of three ways:</p> <ul style="list-style-type: none"> <li>- Reading SPIRXBUF</li> <li>- Writing a 0 to SPI SW RESET (SPICCR.7)</li> <li>- Resetting the system</li> </ul> <p>Note: This bit should not be used if FIFO mode is enabled. The internal process of copying the received word from SPIRXBUF to the Receive FIFO will clear this bit. Use the FIFO status, or FIFO interrupt bits for similar functionality.</p> <p>Reset type: SYSRSn</p> <p>0h (R/W) = No full words have been received or transmitted.</p> <p>1h (R/W) = Indicates that the SPI has completed sending or receiving the last bit and is ready to be serviced.</p> |
| 5   | BUFFULL_FLAG | R    | 0h    | <p>SPI Transmit Buffer Full Flag</p> <p>This read-only bit gets set to 1 when a character is written to the SPI Transmit buffer SPITXBUF. It is cleared when the character is automatically loaded into SPIDAT when the shifting out of a previous character is complete.</p> <p>Reset type: SYSRSn</p> <p>0h (R/W) = Transmit buffer is not full.</p> <p>1h (R/W) = Transmit buffer is full.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| 4-0 | RESERVED     | R    | 0h    | Reserved                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |

### 12.5.2.4 SPI Baud Rate Register (SPIBRR) (Offset = 4h) [reset = 0h]

SPIBRR contains the bits used for baud-rate selection.

**Figure 12-16. SPI Baud Rate Register (SPIBRR)**

|          |              |    |    |    |    |   |   |
|----------|--------------|----|----|----|----|---|---|
| 15       | 14           | 13 | 12 | 11 | 10 | 9 | 8 |
| RESERVED |              |    |    |    |    |   |   |
| R-0h     |              |    |    |    |    |   |   |
| 7        | 6            | 5  | 4  | 3  | 2  | 1 | 0 |
| RESERVED | SPI_BIT_RATE |    |    |    |    |   |   |
| R-0h     | R/W-0h       |    |    |    |    |   |   |

**Table 12-12. SPI Baud Rate Register (SPIBRR) Field Descriptions**

| Bit  | Field        | Type | Reset | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|------|--------------|------|-------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-7 | RESERVED     | R    | 0h    | Reserved                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| 6-0  | SPI_BIT_RATE | R/W  | 0h    | <p>SPI Baud Rate Control</p> <p>These bits determine the bit transfer rate if the SPI is the network SPI BIT RATE 0 master. There are 125 data-transfer rates (each a function of the CPU clock, LSPCLK) that can be selected. One data bit is shifted per SPICLK cycle. (SPICLK is the baud rate clock output on the SPICLK pin.)</p> <p>If the SPI is a network slave, the module receives a clock on the SPICLK pin from the network master. Therefore, these bits have no effect on the SPICLK signal. The frequency of the input clock from the master should not exceed the slave SPI's LSPCLK signal divided by 4.</p> <p>In master mode, the SPI clock is generated by the SPI and is output on the SPICLK pin. The SPI baud rates are determined by the following formula:</p> <p>For SPIBRR = 3 to 127: SPI Baud Rate = LSPCLK / (SPIBRR + 1)</p> <p>For SPIBRR = 0, 1, or 2: SPI Baud Rate = LSPCLK / 4</p> <p>Reset type: SYSRSn</p> <p>3h (R/W) = SPI Baud Rate = LSPCLK/4</p> <p>4h (R/W) = SPI Baud Rate = LSPCLK/5</p> <p>7Eh (R/W) = SPI Baud Rate = LSPCLK/127</p> <p>7Fh (R/W) = SPI Baud Rate = LSPCLK/128</p> |

### 12.5.2.5 SPI Emulation Buffer (SPIRXEMU) Register (Offset = 6h) [reset = 0h]

SPIRXEMU contains the received data. Reading SPIRXEMU does not clear the SPI INT FLAG bit of SPISTS. This is not a real register but a dummy address from which the contents of SPIRXBUF can be read by the debug probe connection without clearing the SPI INT FLAG.

**Figure 12-17. SPI Emulation Buffer (SPIRXEMU) Register**

|       |    |    |    |    |    |   |   |
|-------|----|----|----|----|----|---|---|
| 15    | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
| ERXBn |    |    |    |    |    |   |   |
| R-0h  |    |    |    |    |    |   |   |
| 7     | 6  | 5  | 4  | 3  | 2  | 1 | 0 |
| ERXBn |    |    |    |    |    |   |   |
| R-0h  |    |    |    |    |    |   |   |

**Table 12-13. SPI Emulation Buffer (SPIRXEMU) Register Field Descriptions**

| Bit  | Field | Type | Reset | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|------|-------|------|-------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-0 | ERXBn | R    | 0h    | <p>Emulation Buffer Received Data</p> <p>SPIRXEMU functions almost identically to SPIRXBUF, except that reading SPIRXEMU does not clear the SPI INT FLAG bit (SPISTS.6). Once the SPIDAT has received the complete character, the character is transferred to SPIRXEMU and SPIRXBUF, where it can be read. At the same time, SPI INT FLAG is set.</p> <p>This mirror register was created to support emulation. Reading SPIRXBUF clears the SPI INT FLAG bit (SPISTS.6). In the normal operation with a debug probe connection, the control registers are read to continually update the contents of these registers on the display screen. SPIRXEMU was created so that the debug probe connection can read this register and properly update the contents on the display screen. Reading SPIRXEMU does not clear the SPI INT FLAG bit, but reading SPIRXBUF clears this flag. In other words, SPIRXEMU enables the debug probe connection to emulate the true operation of the SPI more accurately.</p> <p>It is recommended that you view SPIRXEMU when the debug probe is connected.</p> <p>Reset type: SYSRStn</p> |

### 12.5.2.6 SPI Serial Input Buffer (SPIRXBUF) Register (Offset = 7h) [reset = 0h]

SPIRXBUF contains the received data. Reading SPIRXBUF clears the SPI INT FLAG bit in SPISTS. If FIFO mode is enabled, reading this register will also decrement the RXFFST counter in SPIFFRX.

**Figure 12-18. SPI Serial Input Buffer (SPIRXBUF) Register**

|      |    |    |    |    |    |   |   |
|------|----|----|----|----|----|---|---|
| 15   | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
| RXBn |    |    |    |    |    |   |   |
| R-0h |    |    |    |    |    |   |   |
| 7    | 6  | 5  | 4  | 3  | 2  | 1 | 0 |
| RXBn |    |    |    |    |    |   |   |
| R-0h |    |    |    |    |    |   |   |

**Table 12-14. SPI Serial Input Buffer (SPIRXBUF) Register Field Descriptions**

| Bit  | Field | Type | Reset | Description                                                                                                                                                                                                                                                                                                                       |
|------|-------|------|-------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-0 | RXBn  | R    | 0h    | Received Data<br>Once SPIDAT has received the complete character, the character is transferred to SPIRXBUF, where it can be read. At the same time, the SPI INT FLAG bit (SPISTS.6) is set. Since data is shifted into the SPI's most significant bit first, it is stored right-justified in this register.<br>Reset type: SYSRSn |

### 12.5.2.7 SPI Serial Output Buffer (SPITXBUF) Register (Offset = 8h) [reset = 0h]

SPITXBUF stores the next character to be transmitted. Writing to this register sets the TX BUF FULL Flag bit in SPISTS. When the transmission of the current character is complete, the contents of this register are automatically loaded in SPIDAT and the TX BUF FULL Flag is cleared. If no transmission is currently active, data written to this register falls through into the SPIDAT register and the TX BUF FULL Flag is not set. In master mode, if no transmission is currently active, writing to this register initiates a transmission in the same manner that writing to SPIDAT does.

**Figure 12-19. SPI Serial Output Buffer (SPITXBUF) Register**

|        |    |    |    |    |    |   |   |
|--------|----|----|----|----|----|---|---|
| 15     | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
| TXBn   |    |    |    |    |    |   |   |
| R/W-0h |    |    |    |    |    |   |   |
| 7      | 6  | 5  | 4  | 3  | 2  | 1 | 0 |
| TXBn   |    |    |    |    |    |   |   |
| R/W-0h |    |    |    |    |    |   |   |

**Table 12-15. SPI Serial Output Buffer (SPITXBUF) Register Field Descriptions**

| Bit  | Field | Type | Reset | Description                                                                                                                                                                                                                                                                                                                                                         |
|------|-------|------|-------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-0 | TXBn  | R/W  | 0h    | Transmit Data Buffer<br>This is where the next character to be transmitted is stored. When the transmission of the current character has completed, if the TX BUF FULL Flag bit is set, the contents of this register is automatically transferred to SPIDAT, and the TX BUF FULL Flag is cleared. Writes to SPITXBUF must be left-justified.<br>Reset type: SYSRSn |



### 12.5.2.8 SPI Serial Data (SPIDAT) Register (Offset = 9h) [reset = 0h]

SPIDAT is the transmit and receive shift register. Data written to SPIDAT is shifted out (MSB) on subsequent SPICLK cycles. For every bit (MSB) shifted out of the SPI, a bit is shifted into the LSB end of the shift register.

**Figure 12-20. SPI Serial Data (SPIDAT) Register**

|        |    |    |    |    |    |   |   |
|--------|----|----|----|----|----|---|---|
| 15     | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
| SDATn  |    |    |    |    |    |   |   |
| R/W-0h |    |    |    |    |    |   |   |
| 7      | 6  | 5  | 4  | 3  | 2  | 1 | 0 |
| SDATn  |    |    |    |    |    |   |   |
| R/W-0h |    |    |    |    |    |   |   |

**Table 12-16. SPI Serial Data (SPIDAT) Register Field Descriptions**

| Bit  | Field | Type | Reset | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|------|-------|------|-------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-0 | SDATn | R/W  | 0h    | <p>Serial Data Shift Register</p> <ul style="list-style-type: none"> <li>- It provides data to be output on the serial output pin if the TALK bit (SPICTL.1) is set.</li> <li>- When the SPI is operating as a master, a data transfer is initiated. When initiating a transfer, check the CLOCK POLARITY bit (SPICCR.6) described in Section 10.2.1.1 and the CLOCK PHASE bit (SPICTL.3) described in Section 10.2.1.2, for the requirements. In master mode, writing dummy data to SPIDAT initiates a receiver sequence. Since the data is not hardware-justified for characters shorter than sixteen bits, transmit data must be written in left-justified form, and received data read in right-justified form.</li> </ul> <p>Reset type: SYSRSn</p> |

### 12.5.2.9 SPI FIFO Transmit (SPIFFTX) Register (Offset = Ah) [reset = A000h]

SPIFFTX contains both control and status bits related to the output FIFO buffer. This includes FIFO reset control, FIFO interrupt level control, FIFO level status, as well as FIFO interrupt enable and clear bits.

**Figure 12-21. SPI FIFO Transmit (SPIFFTX) Register**

| 15      | 14         | 13       | 12 | 11 | 10 | 9 | 8      |
|---------|------------|----------|----|----|----|---|--------|
| SPIRST  | SPIFFENA   | TXFIFO   |    |    |    |   | TXFFST |
| R/W-1h  | R/W-0h     | R/W-1h   |    |    |    |   | R-0h   |
| 7       | 6          | 5        | 4  | 3  | 2  | 1 | 0      |
| TXFFINT | TXFFINTCLR | TXFFIENA |    |    |    |   | TXFFIL |
| R-0h    | W-0h       | R/W-0h   |    |    |    |   | R/W-0h |

**Table 12-17. SPI FIFO Transmit (SPIFFTX) Register Field Descriptions**

| Bit  | Field      | Type | Reset | Description                                                                                                                                                                                                                                                                                        |
|------|------------|------|-------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15   | SPIRST     | R/W  | 1h    | SPI Reset<br>Reset type: SYSRSn<br>0h (R/W) = Write 0 to reset the SPI transmit and receive channels. The SPI FIFO register configuration bits will be left as is.<br>1h (R/W) = SPI FIFO can resume transmit or receive. No effect to the SPI registers bits.                                     |
| 14   | SPIFFENA   | R/W  | 0h    | SPI FIFO Enhancements Enable<br>Reset type: SYSRSn<br>0h (R/W) = SPI FIFO enhancements are disabled.<br>1h (R/W) = SPI FIFO enhancements are enabled.                                                                                                                                              |
| 13   | TXFIFO     | R/W  | 1h    | TX FIFO Reset<br>Reset type: SYSRSn<br>0h (R/W) = Write 0 to reset the FIFO pointer to zero, and hold in reset.<br>1h (R/W) = Release transmit FIFO from reset.                                                                                                                                    |
| 12-8 | TXFFST     | R    | 0h    | Transmit FIFO Status<br>Reset type: SYSRSn<br>0h (R/W) = Transmit FIFO is empty.<br>1h (R/W) = Transmit FIFO has 1 word.<br>2h (R/W) = Transmit FIFO has 2 words.<br>3h (R/W) = Transmit FIFO has 3 words.<br>4h (R/W) = Transmit FIFO has 4 words, which is the maximum.<br>1Fh (R/W) = Reserved. |
| 7    | TXFFINT    | R    | 0h    | TX FIFO Interrupt Flag<br>Reset type: SYSRSn<br>0h (R/W) = TXFIFO interrupt has not occurred, This is a read-only bit.<br>1h (R/W) = TXFIFO interrupt has occurred, This is a read-only bit.                                                                                                       |
| 6    | TXFFINTCLR | W    | 0h    | TXFIFO Interrupt Clear<br>Reset type: SYSRSn<br>0h (R/W) = Write 0 has no effect on TXFIFINT flag bit, Bit reads back a zero.<br>1h (R/W) = Write 1 to clear SPIFFTX[TXFFINT] flag.                                                                                                                |

**Table 12-17. SPI FIFO Transmit (SPIFFTX) Register Field Descriptions (continued)**

| Bit | Field    | Type | Reset | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|-----|----------|------|-------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 5   | TXFFIENA | R/W  | 0h    | TX FIFO Interrupt Enable<br>Reset type: SYSRSTn<br>0h (R/W) = TX FIFO interrupt based on TXFFIL match (less than or equal to) will be disabled.<br>1h (R/W) = TX FIFO interrupt based on TXFFIL match (less than or equal to) will be enabled.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| 4-0 | TXFFIL   | R/W  | 0h    | Transmit FIFO Interrupt Level Bits<br>Transmit FIFO will generate interrupt when the FIFO status bits (TXFFST4-0) and FIFO level bits (TXFFIL4-0) match (less than or equal to).<br>Reset type: SYSRSTn<br>0h (R/W) = A TX FIFO interrupt request is generated when there are no words remaining in the TX buffer.<br>1h (R/W) = A TX FIFO interrupt request is generated when there is 1 word or no words remaining in the TX buffer.<br>2h (R/W) = A TX FIFO interrupt request is generated when there is 2 words or fewer remaining in the TX buffer.<br>3h (R/W) = A TX FIFO interrupt request is generated when there are 3 words or fewer remaining in the TX buffer.<br>4h (R/W) = A TX FIFO interrupt request is generated when there are 4 words or fewer remaining in the TX buffer.<br>1Fh (R/W) = Reserved. |

### 12.5.2.10 SPI FIFO Receive (SPIFFRX) Register (Offset = Bh) [reset = 201Fh]

SPIFFRX contains both control and status bits related to the input FIFO buffer. This includes FIFO reset control, FIFO interrupt level control, FIFO level status, as well as FIFO interrupt enable and clear bits.

**Figure 12-22. SPI FIFO Receive (SPIFFRX) Register**

| 15      | 14         | 13          | 12      | 11 | 10 | 9 | 8 |
|---------|------------|-------------|---------|----|----|---|---|
| RXFFOVF | RXFFOVFCLR | RXFIFORESET | RXFFST  |    |    |   |   |
| R-0h    | W-0h       | R/W-1h      | R-0h    |    |    |   |   |
| 7       | 6          | 5           | 4       | 3  | 2  | 1 | 0 |
| RXFFINT | RXFFINTCLR | RXFFIENA    | RXFFIL  |    |    |   |   |
| R-0h    | W-0h       | R/W-0h      | R/W-1Fh |    |    |   |   |

**Table 12-18. SPI FIFO Receive (SPIFFRX) Register Field Descriptions**

| Bit  | Field       | Type | Reset | Description                                                                                                                                                                                                                                                                                  |
|------|-------------|------|-------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15   | RXFFOVF     | R    | 0h    | Receive FIFO Overflow Flag<br>Reset type: SYSRSn<br>0h (R/W) = Receive FIFO has not overflowed. This is a read-only bit.<br>1h (R/W) = Receive FIFO has overflowed, read-only bit. More than 4 words have been received in to the FIFO, and the first received word is lost.                 |
| 14   | RXFFOVFCLR  | W    | 0h    | Receive FIFO Overflow Clear<br>Reset type: SYSRSn<br>0h (R/W) = Write 0 does not affect RXFFOVF flag bit, Bit reads back a zero.<br>1h (R/W) = Write 1 to clear SPIFFRX[RXFFOVF].                                                                                                            |
| 13   | RXFIFORESET | R/W  | 1h    | Receive FIFO Reset<br>Reset type: SYSRSn<br>0h (R/W) = Write 0 to reset the FIFO pointer to zero, and hold in reset.<br>1h (R/W) = Re-enable receive FIFO operation.                                                                                                                         |
| 12-8 | RXFFST      | R    | 0h    | Receive FIFO Status<br>Reset type: SYSRSn<br>0h (R/W) = Receive FIFO is empty.<br>1h (R/W) = Receive FIFO has 1 word.<br>2h (R/W) = Receive FIFO has 2 words.<br>3h (R/W) = Receive FIFO has 3 words.<br>4h (R/W) = Receive FIFO has 4 words, which is the maximum.<br>1Fh (R/W) = Reserved. |
| 7    | RXFFINT     | R    | 0h    | Receive FIFO Interrupt Flag<br>Reset type: SYSRSn<br>0h (R/W) = RXFIFO interrupt has not occurred. This is a read-only bit.<br>1h (R/W) = RXFIFO interrupt has occurred. This is a read-only bit.                                                                                            |
| 6    | RXFFINTCLR  | W    | 0h    | Receive FIFO Interrupt Clear<br>Reset type: SYSRSn<br>0h (R/W) = Write 0 has no effect on RXFIFINT flag bit, Bit reads back a zero.<br>1h (R/W) = Write 1 to clear SPIFFRX[RXFFINT] flag                                                                                                     |

**Table 12-18. SPI FIFO Receive (SPIFFRX) Register Field Descriptions (continued)**

| Bit | Field    | Type | Reset | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|-----|----------|------|-------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 5   | RXFFIENA | R/W  | 0h    | <p>RX FIFO Interrupt Enable</p> <p>Reset type: SYSRSn</p> <p>0h (R/W) = RX FIFO interrupt based on RXFFIL match (greater than or equal to) will be disabled.</p> <p>1h (R/W) = RX FIFO interrupt based on RXFFIL match (greater than or equal to) will be enabled.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| 4-0 | RXFFIL   | R/W  | 1Fh   | <p>Receive FIFO Interrupt Level Bits</p> <p>Receive FIFO generates an interrupt when the FIFO status bits (RXFFST4-0) are greater than or equal to the FIFO level bits (RXFFIL4-0). The default value of these bits after reset is 11111.</p> <p>This avoids frequent interrupts after reset, as the receive FIFO will be empty most of the time.</p> <p>Reset type: SYSRSn</p> <p>0h (R/W) = A RX FIFO interrupt request is generated when there is 0 or more words in the RX buffer.</p> <p>1h (R/W) = A RX FIFO interrupt request is generated when there are 1 or more words in the RX buffer.</p> <p>2h (R/W) = A RX FIFO interrupt request is generated when there are 2 or more words in the RX buffer.</p> <p>3h (R/W) = A RX FIFO interrupt request is generated when there are 3 or more words in the RX buffer.</p> <p>4h (R/W) = A RX FIFO interrupt request is generated when there are 4 words in the RX buffer.</p> <p>1Fh (R/W) = Reserved.</p> |

### 12.5.2.11 SPI FIFO Control (SPIFFCT) Register (Offset = Ch) [reset = 0h]

SPIFFCT controls the FIFO transmit delay bits.

**Figure 12-23. SPI FIFO Control (SPIFFCT) Register**

|          |    |    |    |    |    |   |   |
|----------|----|----|----|----|----|---|---|
| 15       | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
| RESERVED |    |    |    |    |    |   |   |
| R-0h     |    |    |    |    |    |   |   |
| 7        | 6  | 5  | 4  | 3  | 2  | 1 | 0 |
| TXDLY    |    |    |    |    |    |   |   |
| R/W-0h   |    |    |    |    |    |   |   |

**Table 12-19. SPI FIFO Control (SPIFFCT) Register Field Descriptions**

| Bit  | Field    | Type | Reset | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|------|----------|------|-------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-8 | RESERVED | R    | 0h    | Reserved                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| 7-0  | TXDLY    | R/W  | 0h    | <p>FIFO Transmit Delay Bits</p> <p>These bits define the delay between every transfer from FIFO transmit buffer to transmit shift register. The delay is defined in number SPI serial clock cycles. The 8-bit register could define a minimum delay of 0 serial clock cycles and a maximum of 255 serial clock cycles. In FIFO mode, the buffer (TXBUF) between the shift register and the FIFO should be filled only after the shift register has completed shifting of the last bit. This is required to pass on the delay between transfers to the data stream. In the FIFO mode TXBUF should not be treated as one additional level of buffer.</p> <p>Reset type: SYSRStn</p> <p>0h (R/W) = The next word in the TX FIFO buffer is transferred to SPITXBUF immediately upon completion of transmission of the previous word.</p> <p>1h (R/W) = The next word in the TX FIFO buffer is transferred to SPITXBUF1 serial clock cycle after completion of transmission of the previous word.</p> <p>2h (R/W) = The next word in the TX FIFO buffer is transferred to SPITXBUF 2 serial clock cycles after completion of transmission of the previous word.</p> <p>FFh (R/W) = The next word in the TX FIFO buffer is transferred to SPITXBUF 255 serial clock cycles after completion of transmission of the previous word.</p> |

### 12.5.2.12 SPI Priority Control (SPIPRI) Register (Offset = Fh) [reset = 0h]

SPIPRI controls auxiliary functions for the SPI including emulation control, and 3-wire control.

**Figure 12-24. SPI Priority Control (SPIPRI) Register**

|          |          |        |        |          |    |        |          |
|----------|----------|--------|--------|----------|----|--------|----------|
| 15       | 14       | 13     | 12     | 11       | 10 | 9      | 8        |
| RESERVED |          |        |        |          |    |        |          |
| R-0h     |          |        |        |          |    |        |          |
| 7        | 6        | 5      | 4      | 3        | 2  | 1      | 0        |
| RESERVED | RESERVED | SOFT   | FREE   | RESERVED |    | STEINV | TRIWIRES |
| R-0h     | R/W-0h   | R/W-0h | R/W-0h | R-0h     |    | R/W-0h | R/W-0h   |

**Table 12-20. SPI Priority Control (SPIPRI) Register Field Descriptions**

| Bit  | Field    | Type | Reset | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|------|----------|------|-------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-7 | RESERVED | R    | 0h    | Reserved                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| 6    | RESERVED | R/W  | 0h    | Reserved                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| 5    | SOFT     | R/W  | 0h    | <p>Emulation Soft Run</p> <p>This bit only has an effect when the FREE bit is 0.</p> <p>Reset type: SYSRSn</p> <p>0h (R/W) = Transmission stops midway in the bit stream while TSUSPEND is asserted. Once TSUSPEND is deasserted without a system reset, the remainder of the bits pending in the DATBUF are shifted. Example: If SPIDAT has shifted 3 out of 8 bits, the communication freezes right there. However, if TSUSPEND is later deasserted without resetting the SPI, SPI starts transmitting from where it had stopped (fourth bit in this case) and will transmit 8 bits from that point.</p> <p>1h (R/W) = If the emulation suspend occurs before the start of a transmission, (that is, before the first SPICLK pulse) then the transmission will not occur. If the emulation suspend occurs after the start of a transmission, then the data will be shifted out to completion. When the start of transmission occurs is dependent on the baud rate used.</p> <p>Standard SPI mode: Stop after transmitting the words in the shift register and buffer. That is, after TXBUF and SPIDAT are empty.</p> <p>In FIFO mode: Stop after transmitting the words in the shift register and buffer. That is, after TX FIFO and SPIDAT are empty.</p> |
| 4    | FREE     | R/W  | 0h    | <p>Emulation Free Run</p> <p>These bits determine what occurs when an emulation suspend occurs (for example, when the debugger hits a breakpoint). The peripheral can continue whatever it is doing (free-run mode) or, if in stop mode, it can either stop immediately or stop when the current operation (the current receive/transmit sequence) is complete.</p> <p>Reset type: SYSRSn</p> <p>0h (R/W) = Emulation mode is selected by the SOFT bit</p> <p>1h (R/W) = Free run, continue SPI operation regardless of suspend or when the suspend occurred.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| 3-2  | RESERVED | R    | 0h    | Reserved                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |

**Table 12-20. SPI Priority Control (SPIPRI) Register Field Descriptions (continued)**

| Bit | Field   | Type | Reset | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|-----|---------|------|-------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1   | STEINV  | R/W  | 0h    | <p>SPISTEn Inversion Bit</p> <p>On devices with 2 SPI modules, inverting the SPISTE signal on one of the modules allows the device to receive left and right- channel digital audio data.</p> <p>This bit is only applicable to slave mode. Writing to this bit while configured as master (MASTER_SLAVE = 1) has no effect</p> <p>Reset type: SYSRSn</p> <p>0h (R/W) = SPISTEn is active low (normal)</p> <p>1h (R/W) = SPISTE is active high (inverted)</p> |
| 0   | TRIWIRE | R/W  | 0h    | <p>SPI 3-wire Mode Enable</p> <p>Reset type: SYSRSn</p> <p>0h (R/W) = Normal 4-wire SPI mode.</p> <p>1h (R/W) = 3-wire SPI mode enabled. The unused pin becomes a GPIO pin. In master mode, the SPISIMO pin becomes the SPIMOMI (master receive and transmit) pin and SPISOMI is free for non-SPI use. In slave mode, the SPISOMI pin becomes the SPISISO (slave receive and transmit) pin and SPISIMO is free for non-SPI use.</p>                           |



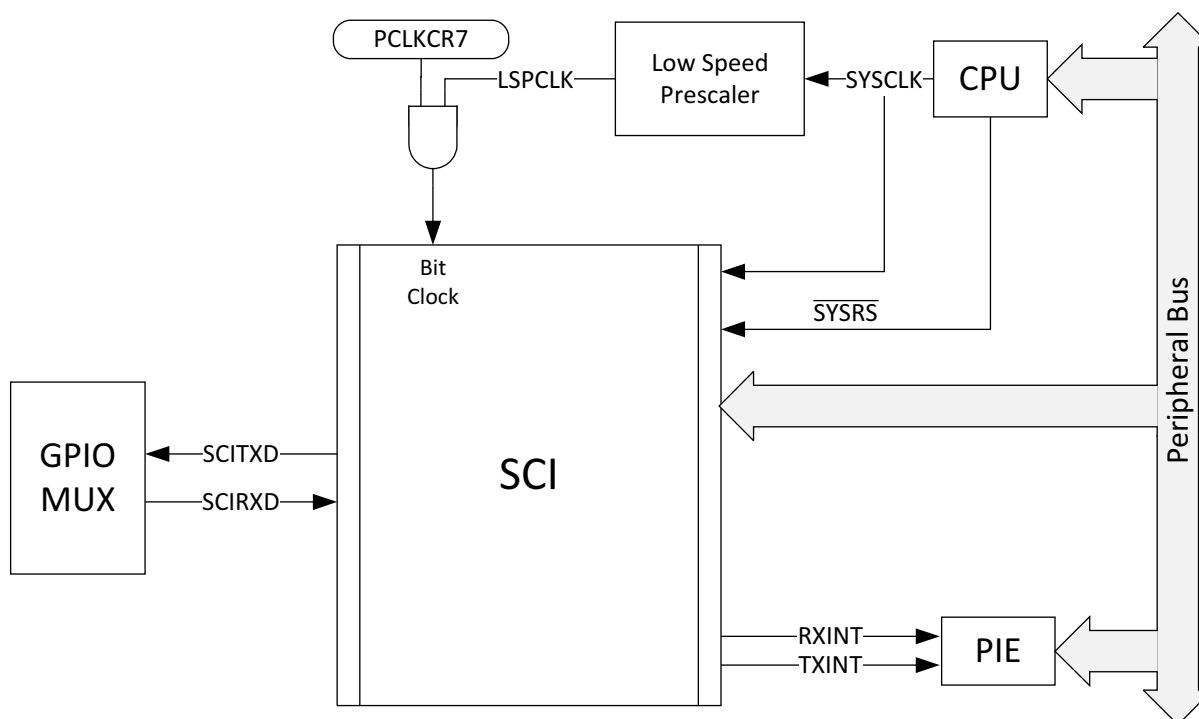
This chapter describes the features and operation of the serial communication interface (SCI) module. SCI is a two-wire asynchronous serial port, commonly known as a UART. The SCI modules support digital communications between the CPU and other asynchronous peripherals that use the standard non-return-to-zero (NRZ) format. The SCI receiver and transmitter each have a 4-level deep FIFO for reducing servicing overhead, and each has its own separate enable and interrupt bits. Both can be operated independently for half-duplex communication, or simultaneously for full-duplex communication.

To specify data integrity, the SCI checks received data for break detection, parity, overrun, and framing errors. The bit rate is programmable to different speeds through a 16-bit baud-select register.

|                                                                      |            |
|----------------------------------------------------------------------|------------|
| <b>13.1 Introduction.....</b>                                        | <b>800</b> |
| <b>13.2 Architecture.....</b>                                        | <b>801</b> |
| <b>13.3 SCI Module Signal Summary.....</b>                           | <b>801</b> |
| <b>13.4 Configuring Device Pins.....</b>                             | <b>802</b> |
| <b>13.5 Multiprocessor and Asynchronous Communication Modes.....</b> | <b>803</b> |
| <b>13.6 SCI Programmable Data Format.....</b>                        | <b>803</b> |
| <b>13.7 SCI Multiprocessor Communication.....</b>                    | <b>804</b> |
| <b>13.8 Idle-Line Multiprocessor Mode.....</b>                       | <b>805</b> |
| <b>13.9 Address-Bit Multiprocessor Mode.....</b>                     | <b>807</b> |
| <b>13.10 SCI Communication Format.....</b>                           | <b>808</b> |
| <b>13.11 SCI Port Interrupts.....</b>                                | <b>810</b> |
| <b>13.12 SCI Baud Rate Calculations.....</b>                         | <b>810</b> |
| <b>13.13 SCI Enhanced Features.....</b>                              | <b>811</b> |
| <b>13.14 SCI Registers.....</b>                                      | <b>814</b> |

## 13.1 Introduction

The SCI interfaces are shown in [Figure 13-1](#).



**Figure 13-1. SCI CPU Interface**

### 13.1.1 Features

Features of the SCI module include:

- Two external pins:
  - SCITXD: SCI transmit-output pin
  - SCIRXD: SCI receive-input pin
- Both pins can be used as GPIO if not used for SCI.
- Baud rate programmable to 64K different rates
- Data-word format
  - One start bit
  - Data-word length programmable from one to eight bits
  - Optional even/odd/no parity bit
  - One or two stop bits
  - An extra bit to distinguish addresses from data (address bit mode only)
- Four error-detection flags: parity, overrun, framing, and break detection
- Two wake-up multiprocessor modes: idle-line and address bit
- Half- or full-duplex operation
- Double-buffered receive and transmit functions
- Transmitter and receiver operations can be accomplished through interrupt-driven or polled algorithms with status flags
- Separate enable bits for transmitter and receiver interrupts (except BRKDT)
- NRZ (non-return-to-zero) format

Enhanced features include:

- Auto-baud-detect hardware logic
- 4-level transmit/receive FIFO

### 13.1.2 SCI Related Collateral

#### Foundational Materials

- [One Minute RS-485 Introduction](#) (Video)
- [RS-232, RS-422, RS-485: What Are the Differences?](#) (Video)

#### Getting Started Materials

- [\[FAQ\] My C2000 SCI is not Transmitting and/or Receiving data correctly, how do I fix this?](#)

### 13.1.3 Block Diagram

Figure 13-2 shows the SCI module block diagram. The SCI port operation is configured and controlled by the registers listed in Section 13.14.

## 13.2 Architecture

The major elements used in full-duplex operation are shown in Figure 13-2 and include:

- A transmitter (TX) and its major registers (upper half of Figure 13-2)
  - SCITXBUF — transmitter data buffer register. Contains data (loaded by the CPU) to be transmitted
  - TXSHF register — transmitter shift register. Accepts data from register SCITXBUF and shifts data onto the SCITXD pin, one bit at a time
- A receiver (RX) and its major registers (lower half of Figure 13-2)
  - RXSHF register — receiver shift register. Shifts data in from SCIRXD pin, one bit at a time
  - SCIRXBUF — receiver data buffer register. Contains data to be read by the CPU. Data from a remote processor is loaded into register RXSHF and then into registers SCIRXBUF and SCIRXEMU
- A programmable baud generator
- Control and status registers

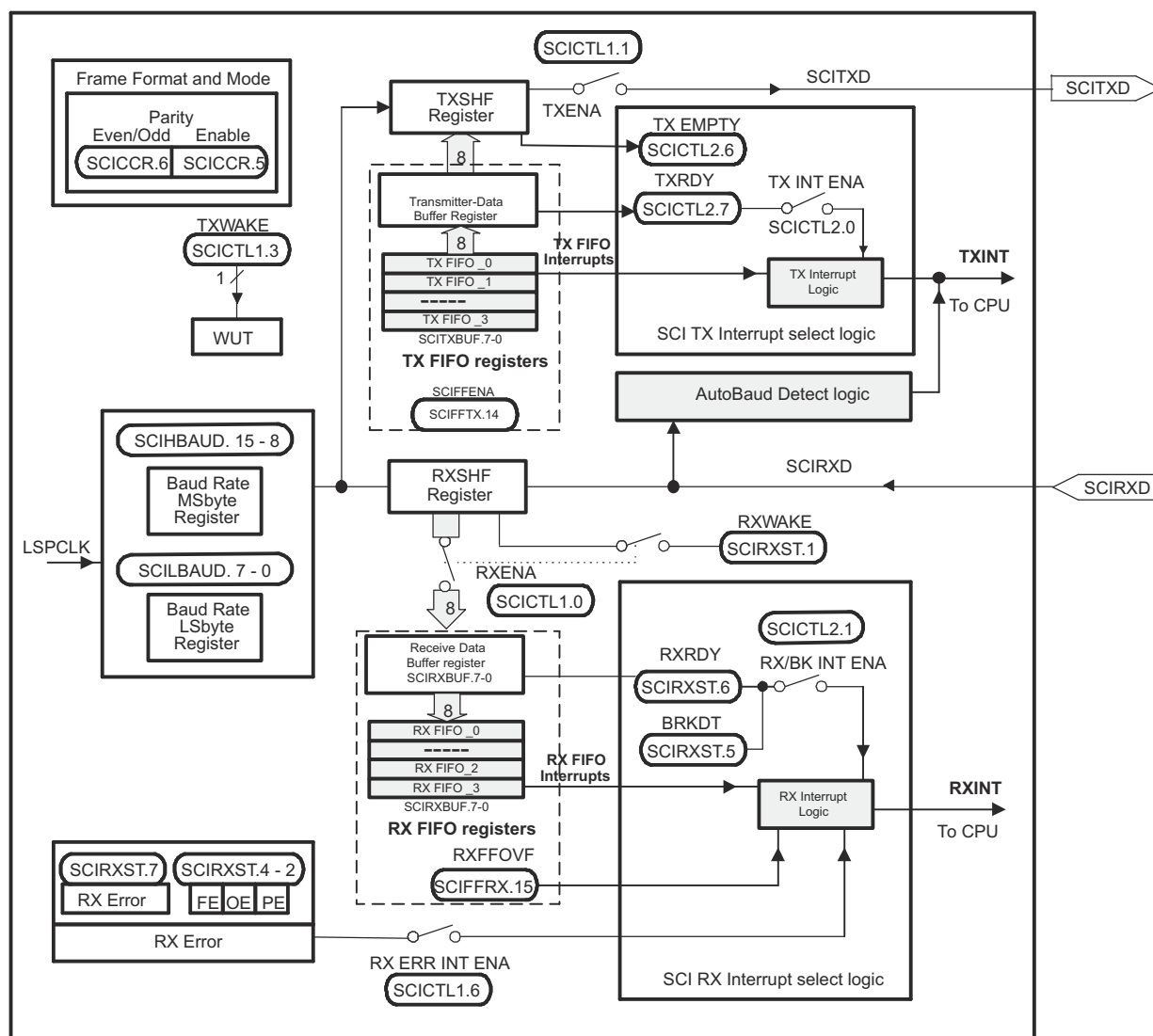
The SCI receiver and transmitter can operate either independently or simultaneously.

### 13.3 SCI Module Signal Summary

A summarized description of each SCI signal name is shown in Table 13-1.

**Table 13-1. SCI Module Signal Summary**

| Signal Name              | Description                                |
|--------------------------|--------------------------------------------|
| <b>External signals</b>  |                                            |
| SCIRXD                   | SCI Asynchronous Serial Port receive data  |
| SCITXD                   | SCI Asynchronous Serial Port transmit data |
| <b>Control</b>           |                                            |
| Baud clock               | LSPCLK Prescaled clock                     |
| <b>Interrupt signals</b> |                                            |
| TXINT                    | Transmit interrupt                         |
| RXINT                    | Receive Interrupt                          |



**Figure 13-2. Serial Communications Interface (SCI) Module Block Diagram**

## 13.4 Configuring Device Pins

The GPIO mux registers must be configured to connect this peripheral to the device pins. To avoid glitches on the pins, the GPyMUX bits must be configured first (while keeping the corresponding GPyMUX bits at the default of zero), followed by writing the GPyMUX register to the desired value.

Some IO functionality is defined by GPIO register settings independent of this peripheral. For input signals, the GPIO input qualification should be set to asynchronous mode by setting the appropriate GPxQSELn register bits to 11b. The internal pullups can be configured in the GPyPUD register.

See the *General-Purpose Input/Output (GPIO)* chapter for more details on GPIO mux and settings.

## 13.5 Multiprocessor and Asynchronous Communication Modes

The SCI has two multiprocessor protocols, the idle-line multiprocessor mode (see [Section 13.8](#)) and the address-bit multiprocessor mode (see [Section 13.9](#)). These protocols allow efficient data transfer between multiple processors.

The SCI offers the universal asynchronous receiver/transmitter (UART) communications mode for interfacing with many popular peripherals. The asynchronous mode (see [Section 13.10](#)) requires two lines to interface with many standard devices such as terminals and printers that use RS-232-C formats. Data transmission characteristics include:

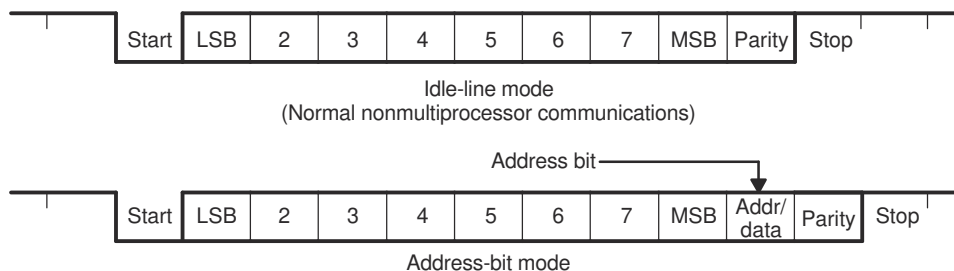
- One start bit
- One to eight data bits
- An even/odd parity bit or no parity bit
- One or two stop bits

## 13.6 SCI Programmable Data Format

SCI data, both receive and transmit, is in NRZ (non-return-to-zero) format. The NRZ data format, shown in [Figure 13-3](#), consists of:

- One start bit
- One to eight data bits
- An even/odd parity bit (optional)
- One or two stop bits
- An extra bit to distinguish addresses from data (address-bit mode only)

The basic unit of data is called a character and is one to eight bits in length. Each character of data is formatted with a start bit, one or two stop bits, and optional parity and address bits. A character of data with its formatting information is called a frame and is shown in [Figure 13-3](#).



**Figure 13-3. Typical SCI Data Frame Formats**

To program the data format, use the SCICCR register. The bits used to program the data format are shown in [Table 13-2](#).

**Table 13-2. Programming the Data Format Using SCICCR**

| Bit(s) | Bit Name              | Designation | Functions                                                                                                 |
|--------|-----------------------|-------------|-----------------------------------------------------------------------------------------------------------|
| 2-0    | SCICHAR               | SCICCR.2:0  | Select the character (data) length (one to eight bits).                                                   |
| 5      | PARITYENA<br>(ENABLE) | SCICCR.5    | Enables the parity function if set to 1, or disables the parity function if cleared to 0.                 |
| 6      | PARITY<br>(EVEN/ODD)  | SCICCR.6    | If parity is enabled, selects odd parity if cleared to 0; even parity if set to 1.                        |
| 7      | STOPBITS              | SCICCR.7    | Determines the number of stop bits transmitted—one stop bit if cleared to 0 or two stop bits if set to 1. |

## 13.7 SCI Multiprocessor Communication

The multiprocessor communication format allows one processor to efficiently send blocks of data to other processors on the same serial link. On one serial line, there should be only one transfer at a time. In other words, there can be only one talker on a serial line at a time.

### Address Byte

The first byte of a block of information that the talker sends contains an address byte that is read by all listeners. Only listeners with the correct address can be interrupted by the data bytes that follow the address byte. The listeners with an incorrect address remain uninterrupted until the next address byte.

### Sleep Bit

All processors on the serial link set the SCI SLEEP bit (bit 2 of SCICTL1) to 1 so that they are interrupted only when the address byte is detected. When a processor reads a block address that corresponds to the CPU device address as set by your application software, your program must clear the SLEEP bit to enable the SCI to generate an interrupt on receipt of each data byte.

Although the receiver still operates when the SLEEP bit is 1, it does not set RXRDY, RXINT, or any of the receiver error status bits to 1 unless the address byte is detected and the address bit in the received frame is a 1 (applicable to address-bit mode). The SCI does not alter the SLEEP bit; your software must alter the SLEEP bit.

### 13.7.1 Recognizing the Address Byte

A processor recognizes an address byte differently, depending on the multiprocessor mode used. For example:

- The idle-line mode ([Section 13.8](#)) leaves a quiet space before the address byte. This mode does not have an extra address/data bit and is more efficient than the address-bit mode for handling blocks that contain more than ten bytes of data. The idle-line mode should be used for typical non-multiprocessor SCI communication.
- The address-bit mode ([Section 13.9](#)) adds an extra bit (that is, an address bit) into every byte to distinguish addresses from data. This mode is more efficient in handling many small blocks of data because, unlike the idle mode, it does not have to wait between blocks of data. However, at a high transmit speed, the program is not fast enough to avoid a 10-bit idle in the transmission stream.

### 13.7.2 Controlling the SCI TX and RX Features

The multiprocessor mode is software selectable via the ADDR/IDLE MODE bit (SCICCR, bit 3). Both modes use the TXWAKE flag bit (SCICTL1, bit 3), RXWAKE flag bit (SCIRXST, bit1), and the SLEEP flag bit (SCICTL1, bit 2) to control the SCI transmitter and receiver features of these modes.

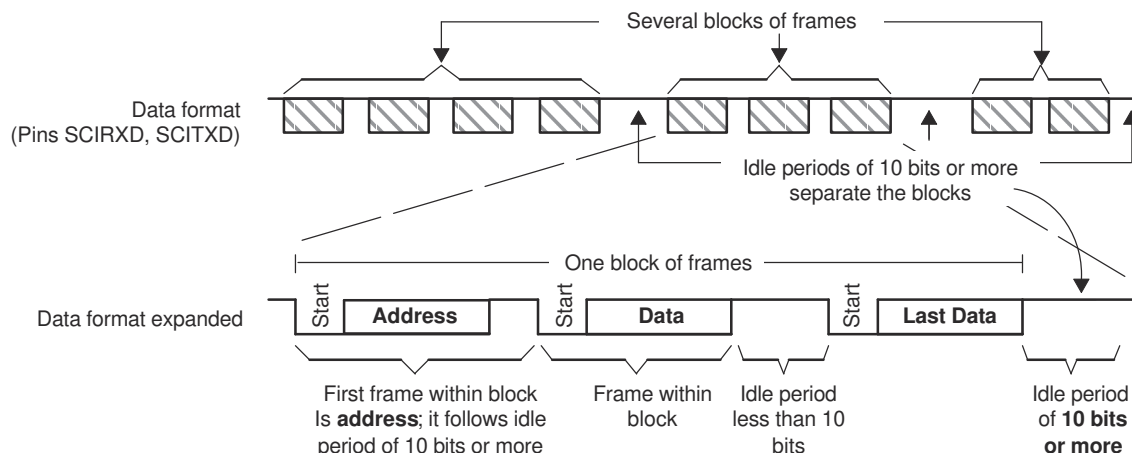
### 13.7.3 Receipt Sequence

In both multiprocessor modes, the receive sequence is as follows:

1. At the receipt of an address block, the SCI port wakes up and requests an interrupt (bit number 1 RX/BK INT ENA-of SCICTL2 must be enabled to request an interrupt). It reads the first frame of the block, which contains the destination address.
2. A software routine is entered through the interrupt and checks the incoming address. This address byte is checked against its device address byte stored in memory.
3. If the check shows that the block is addressed to the device CPU, the CPU clears the SLEEP bit and reads the rest of the block. If not, the software routine exits with the SLEEP bit still set, and does not receive interrupts until the next block start.

## 13.8 Idle-Line Multiprocessor Mode

In the idle-line multiprocessor protocol (ADDR/IDLE MODE bit=0), blocks are separated by having a longer idle time between the blocks than between frames in the blocks. An idle time of ten or more high-level bits after a frame indicates the start of a new block. The time of a single bit is calculated directly from the baud value (bits per second). The idle-line multiprocessor communication format is shown in Figure 13-4 (ADDR/IDLE MODE bit is bit 3 of SCICCR).



**Figure 13-4. Idle-Line Multiprocessor Communication Format**

### 13.8.1 Idle-Line Mode Steps

The steps followed by the idle-line mode:

1. SCI wakes up after receipt of the block-start signal.
2. The processor recognizes the next SCI interrupt.
3. The interrupt service routine compares the received address (sent by a remote transmitter) to its own.
4. If the CPU is being addressed, the service routine clears the SLEEP bit and receives the rest of the data block.
5. If the CPU is not being addressed, the SLEEP bit remains set. This lets the CPU continue to execute its main program without being interrupted by the SCI port until the next detection of a block start.

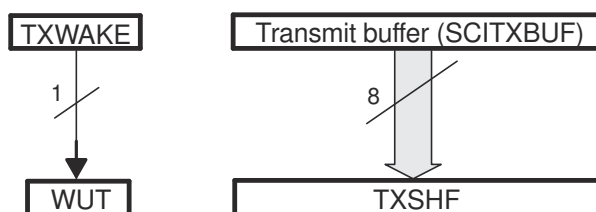
### 13.8.2 Block Start Signal

There are two ways to send a block-start signal:

- **Method 1:** Deliberately leave an idle time of ten bits or more by delaying the time between the transmission of the last frame of data in the previous block and the transmission of the address frame of the new block.
- **Method 2:** The SCI port first sets the TXWAKE bit (SCICTL1, bit 3) to 1 before writing to the SCITXBUF register. This sends an idle time of exactly 11 bits. In this method, the serial communications line is not idle any longer than necessary. (A don't care byte has to be written to SCITXBUF after setting TXWAKE, and before sending the address, so as to transmit the idle time.)

### 13.8.3 Wake-UP Temporary (WUT) Flag

Associated with the TXWAKE bit is the wake-up temporary (WUT) flag. WUT is an internal flag, double-buffered with TXWAKE. When TXSHF is loaded from SCITXBUF, WUT is loaded from TXWAKE, and the TXWAKE bit is cleared to 0. This arrangement is shown in [Figure 13-5](#).



**Figure 13-5. Double-Buffered WUT and TXSHF**

#### 13.8.3.1 Sending a Block Start Signal

To send out a block-start signal of exactly one frame time during a sequence of block transmissions:

1. Write a 1 to the TXWAKE bit.
2. Write a data word (content not important: a don't care) to the SCITXBUF register (transmit data buffer) to send a block-start signal. (The first data word written is suppressed while the block-start signal is sent out and ignored after that.) When the TXSHF (transmit shift register) is free again, SCITXBUF contents are shifted to TXSHF, the TXWAKE value is shifted to WUT, and then TXWAKE is cleared.

Because TXWAKE was set to a 1, the start, data, and parity bits are replaced by an idle period of 11 bits transmitted following the last stop bit of the previous frame.

3. Write a new address value to SCITXBUF.

A don't-care data word must first be written to register SCITXBUF so that the TXWAKE bit value can be shifted to WUT. After the don't-care data word is shifted to the TXSHF register, the SCITXBUF (and TXWAKE, if necessary) can be written to again because TXSHF and WUT are both double-buffered.

### 13.8.4 Receiver Operation

The receiver operates regardless of the SLEEP bit. However, the receiver neither sets RXRDY nor the error status bits, nor does it request a receive interrupt until an address frame is detected.



## 13.9 Address-Bit Multiprocessor Mode

In the address-bit protocol (ADDR/IDLE MODE bit=1), frames have an extra bit called an address bit that immediately follows the last data bit. The address bit is set to 1 in the first frame of the block and to 0 in all other frames. The idle period timing is irrelevant (see Figure 13-6).

### 13.9.1 Sending an Address

The TXWAKE bit value is placed in the address bit. During transmission, when the SCITXBUF register and TXWAKE are loaded into the TXSHF register and WUT respectively, TXWAKE is reset to 0 and WUT becomes the value of the address bit of the current frame. Thus, to send an address:

1. Set the TXWAKE bit to 1 and write the appropriate address value to the SCITXBUF register.

When this address value is transferred to the TXSHF register and shifted out, its address bit is sent as a 1. This flags the other processors on the serial link to read the address.

2. Write to SCITXBUF and TXWAKE after TXSHF and WUT are loaded. (Can be written to immediately since both TXSHF and WUT are both double-buffered.)
3. Leave the TXWAKE bit set to 0 to transmit non-address frames in the block.

#### Note

As a general rule, the address-bit format is typically used for data frames of 11 bytes or less. This format adds one bit value (1 for an address frame, 0 for a data frame) to all data bytes transmitted. The idle-line format is typically used for data frames of 12 bytes or more.

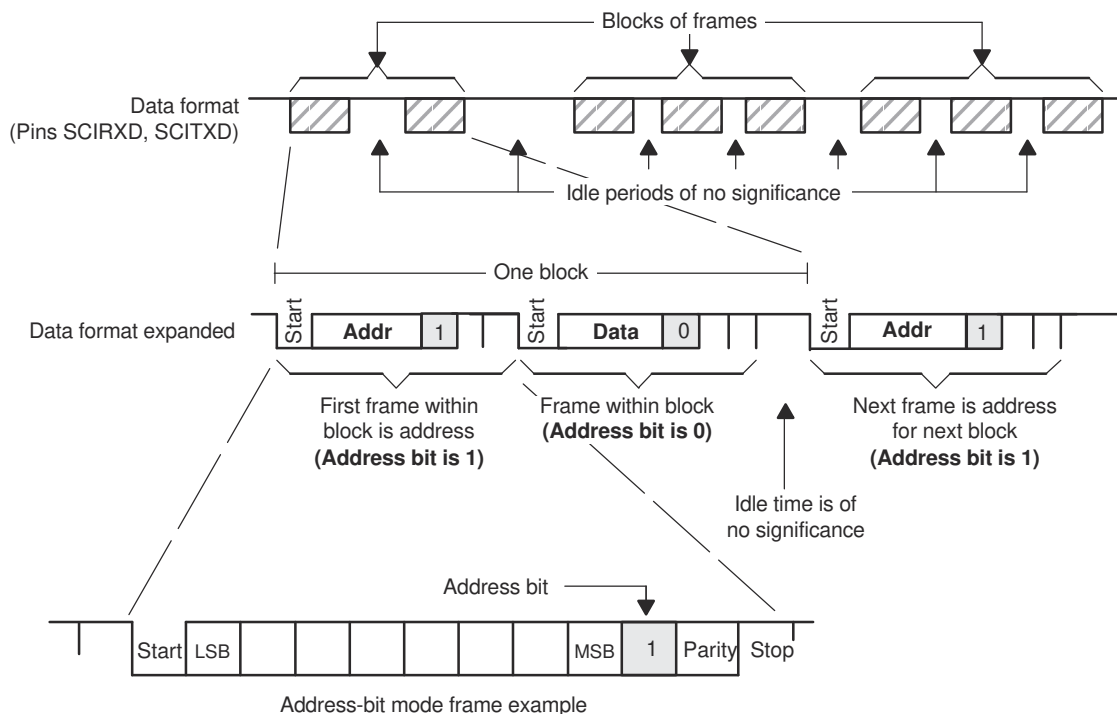


Figure 13-6. Address-Bit Multiprocessor Communication Format

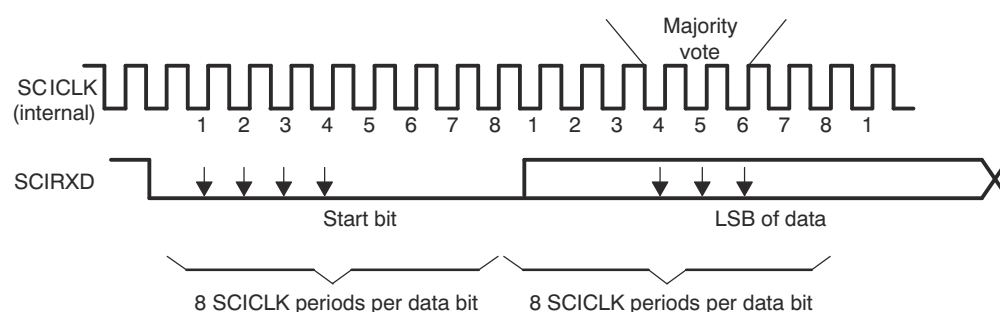
## 13.10 SCI Communication Format

The SCI asynchronous communication format uses either single line (one way) or two line (two way) communications. In this mode, the frame consists of a start bit, one to eight data bits, an optional even/odd parity bit, and one or two stop bits (shown in Figure 13-7). There are eight SCICLK periods per data bit.

The receiver begins operation on receipt of a valid start bit. A valid start bit is identified by four consecutive internal SCICLK periods of zero bits as shown in Figure 13-7. If any bit is not zero, then the processor starts over and begins looking for another start bit.

For the bits following the start bit, the processor determines the bit value by making three samples in the middle of the bits. These samples occur on the fourth, fifth, and sixth SCICLK periods, and bit-value determination is on a majority (two out of three) basis. Figure 13-7 illustrates the asynchronous communication format for this with a start bit showing where a majority vote is taken.

Since the receiver synchronizes itself to frames, the external transmitting and receiving devices do not have to use a synchronized serial clock. The clock can be generated locally.

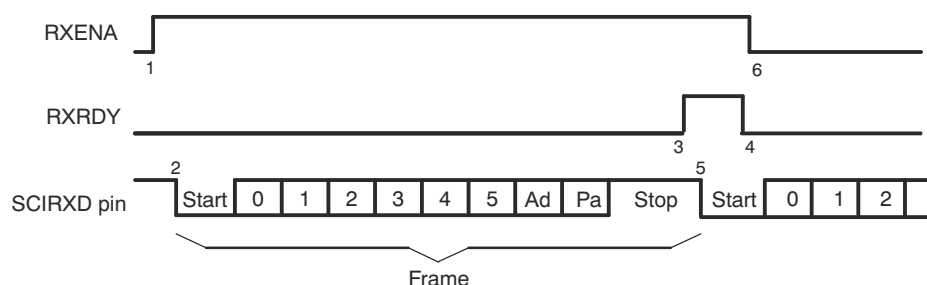


**Figure 13-7. SCI Asynchronous Communications Format**

### 13.10.1 Receiver Signals in Communication Modes

Figure 13-8 illustrates an example of receiver signal timing that assumes the following conditions:

- Address-bit wake-up mode (address bit does not appear in idle-line mode)
- Six bits per character



**Figure 13-8. SCI RX Signals in Communication Modes**

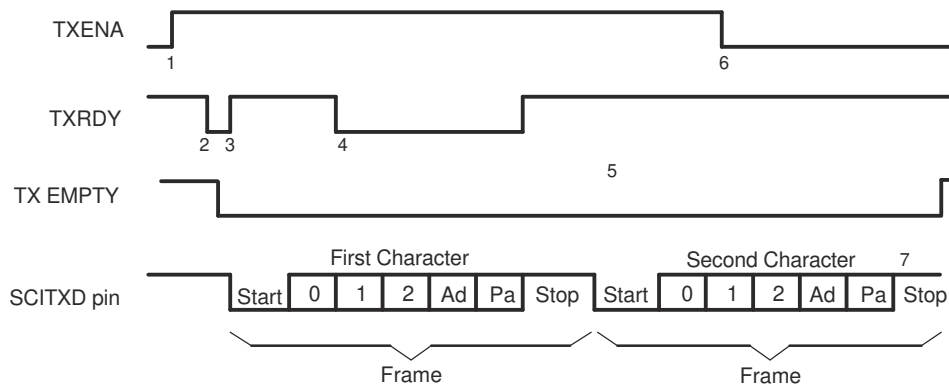
Notes:

1. Flag bit RXENA (SCICTL1, bit 0) goes high to enable the receiver.
2. Data arrives on the SCIRXD pin, start bit detected.
3. Data is shifted from RXSHF to the receiver buffer register (SCIRXBUF); an interrupt is requested. Flag bit RXRDY (SCIRXST, bit 6) goes high to signal that a new character has been received.
4. The program reads SCIRXBUF; flag RXRDY is automatically cleared.
5. The next byte of data arrives on the SCIRXD pin; the start bit is detected, then cleared.
6. Bit RXENA is brought low to disable the receiver. Data continues to be assembled in RXSHF but is not transferred to the receiver buffer register.

### 13.10.2 Transmitter Signals in Communication Modes

Figure 13-9 illustrates an example of transmitter signal timing that assumes the following conditions:

- Address-bit wake-up mode (address bit does not appear in idle-line mode)
- Three bits per character



**Figure 13-9. SCI TX Signals in Communications Mode**

#### Notes:

1. Bit TXENA (SCICTL1, bit 1) goes high, enabling the transmitter to send data.
2. SCITXBUF is written to; thus, (1) the transmitter is no longer empty, and (2) TXRDY goes low.
3. The SCI transfers data to the shift register (TXSHF). The transmitter is ready for a second character (TXRDY goes high), and it requests an interrupt (to enable an interrupt, bit TX INT ENA — SCICTL2, bit 0 — must be set).
4. The program writes a second character to SCITXBUF after TXRDY goes high (item 3). (TXRDY goes low again after the second character is written to SCITXBUF.)
5. Transmission of the first character is complete. Transfer of the second character to shift register TXSHF begins.
6. Bit TXENA goes low to disable the transmitter; the SCI finishes transmitting the current character.
7. Transmission of the second character is complete; transmitter is empty and ready for new character.

## 13.11 SCI Port Interrupts

The SCI receiver and transmitter can be interrupt controlled. The SCICTL2 register has one flag bit (TXRDY) that indicates active interrupt conditions, and the SCIRXST register has two interrupt flag bits (RXRDY and BRKDT), plus the RX ERROR interrupt flag that is a logical-OR of the FE, OE, BRKDT, and PE conditions. The transmitter and receiver have separate interrupt-enable bits. When not enabled, the interrupts are not asserted; however, the condition flags remain active, reflecting transmission and receipt status.

The SCI has independent peripheral interrupt vectors for the receiver and transmitter. Peripheral interrupt requests can be either high priority or low priority. This is indicated by the priority bits that are output from the peripheral to the PIE controller. When both RX and TX interrupt requests are made at the same priority level, the receiver always has higher priority than the transmitter, reducing the possibility of receiver overrun.

The operation of peripheral interrupts is described in the Peripheral Interrupts section of the *System Control and Interrupts* chapter.

- If the RX/BK INT ENA bit (SCICTL2, bit 1) is set, the receiver peripheral interrupt request is asserted when one of the following events occurs:
  - The SCI receives a complete frame and transfers the data in the RXSHF register to the SCIRXBUF register. This action sets the RXRDY flag (SCIRXST, bit 6) and initiates an interrupt.
  - A break detect condition occurs (the SCIRXD is low for 9.625 bit periods following a missing stop bit). This action sets the BRKDT flag bit (SCIRXST, bit 5) and initiates an interrupt.
- If the TX INT ENA bit (SCICTL2.0) is set, the transmitter peripheral interrupt request is asserted whenever the data in the SCITXBUF register is transferred to the TXSHF register, indicating that the CPU can write to SCITXBUF; this action sets the TXRDY flag bit (SCICTL2, bit 7) and initiates an interrupt.

### Note

Interrupt generation due to the RXRDY and BRKDT bits is controlled by the RX/BK INT ENA bit (SCICTL2, bit 1). Interrupt generation due to the RX ERROR bit is controlled by the RX ERR INT ENA bit (SCICTL1, bit 6).

## 13.12 SCI Baud Rate Calculations

The internally generated serial clock is determined by the low-speed peripheral clock (LSPCLK) and the baud-select registers. The SCI uses the 16-bit value of the baud-select registers to select one of the 64K different serial clock rates possible for a given LSPCLK.

See the bit descriptions in the baud-select registers, for the formula to use when calculating the SCI asynchronous baud. [Table 13-3](#) shows the baud-select values for common SCI bit rates. LSPCLK/16 is the maximum baud rate. For example, if LSPCLK is 100 MHz, then the maximum baud rate is 6.25 Mbps.

**Table 13-3. Asynchronous Baud Register Values for Common SCI Bit Rates**

| Ideal Baud | LSPCLK Clock Frequency, 100 MHz |             |         |
|------------|---------------------------------|-------------|---------|
|            | BRR                             | Actual Baud | % Error |
| 2400       | 5207 (1457h)                    | 2400        | 0       |
| 4800       | 2603 (A2Bh)                     | 4800        | 0       |
| 9600       | 1301 (515h)                     | 9601        | 0.01    |
| 19200      | 650 (28Ah)                      | 19201       | 0.01    |
| 38400      | 324 (144h)                      | 38462       | 0.16    |

## 13.13 SCI Enhanced Features

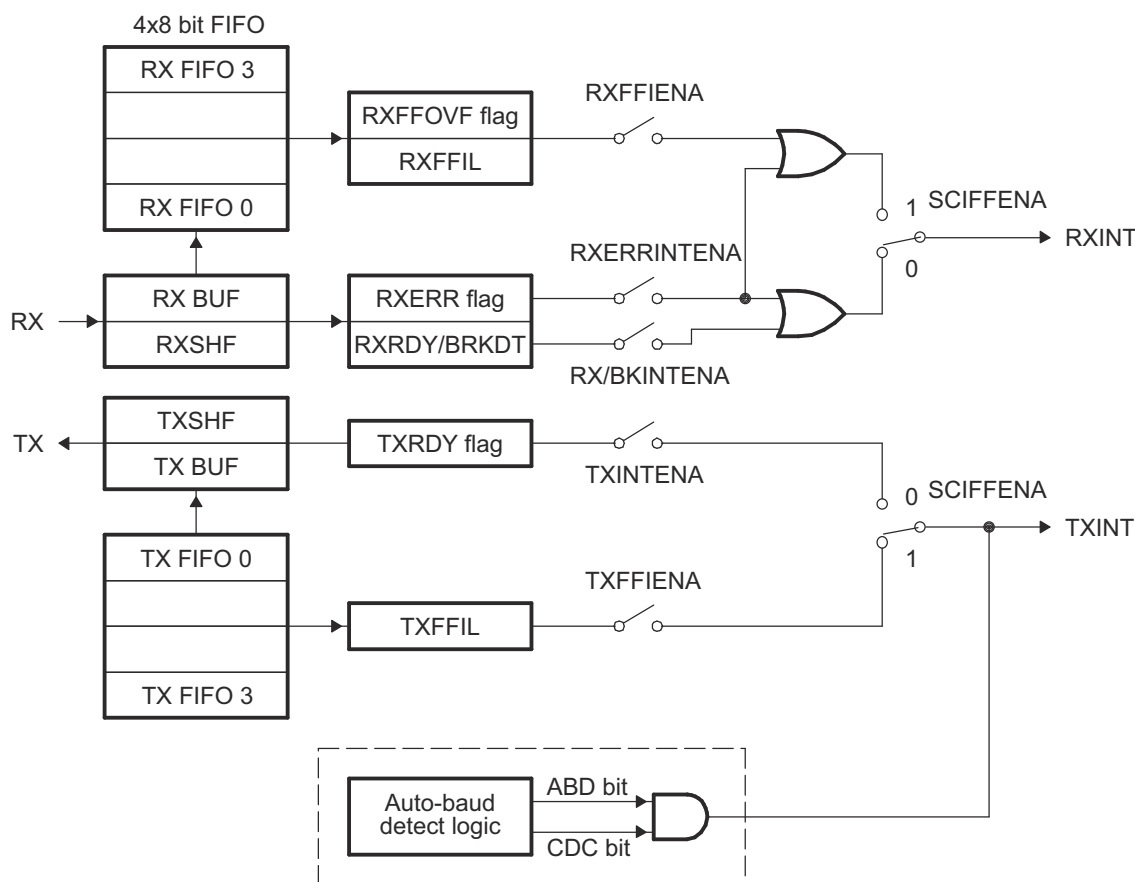
The C28x SCI features autobaud detection and transmit/receive FIFO. The following section explains the FIFO operation.

### 13.13.1 SCI FIFO Description

The following steps explain the FIFO features and help with programming the SCI with FIFOs.

1. **Reset.** At reset the SCI powers up in standard SCI mode and the FIFO function is disabled. The FIFO registers SCIFFTX, SCIFFRX, and SCIFFCT remain inactive.
2. **Standard SCI.** The standard SCI modes will work normally with TXINT/RXINT interrupts as the interrupt source for the module.
3. **FIFO enable.** FIFO mode is enabled by setting the SCIFFEN bit in the SCIFFTX register. SCIRST can reset the FIFO mode at any stage of its operation.
4. **Active registers.** All the SCI registers and SCI FIFO registers (SCIFFTX, SCIFFRX, and SCIFFCT) are active.
5. **Interrupts.** FIFO mode has two interrupts; one for transmit FIFO, TXINT and one for receive FIFO, RXINT. RXINT is the common interrupt for SCI FIFO receive, receive error, and receive FIFO overflow conditions. The TXINT of the standard SCI will be disabled and this interrupt will service as SCI transmit FIFO interrupt.
6. **Buffers.** Transmit and receive buffers are supplemented with two 4-level FIFOs. The transmit FIFO registers are 8 bits wide and receive FIFO registers are 10 bits wide. The one-word transmit buffer (SCITXBUF) of the standard SCI functions as a transition buffer before the transmit FIFO and shift register. SCITXBUF is loaded into either the FIFO (when FIFO is enabled) or TXSHF (when FIFO is disabled). When FIFO is enabled, SCITXBUF loads into the FIFO only after the last bit of the shift register is shifted out, so SCITXBUF should not be treated as an additional level of buffer. With the FIFO enabled, TXSHF is directly loaded from the FIFO (not TXBUF) after an optional delay value (SCIFFCT). When FIFO mode is enabled for SCI, characters written to SCITXBUF are queued in to SCI-TXFIFO and the characters received in SCI-RXFIFO can be read using SCIRXBUF.
7. **Delayed transfer.** The rate at which words in the FIFO are transferred to the transmit shift register is programmable. The SCIFFCT register bits (7–0) FFTXDLY7–FFTXDLY0 define the delay between the word transfer. The delay is defined in the number SCI baud clock cycles. The 8 bit register can define a minimum delay of 0 baud clock cycles and a maximum of 256-baud clock cycles. With zero delay, the SCI module can transmit data in continuous mode with the FIFO words shifting out back to back. With the 256 clock delay the SCI module can transmit data in a maximum delayed mode with the FIFO words shifting out with a delay of 256 baud clocks between each words. The programmable delay facilitates communication with slow SCI/UARTs with little CPU intervention.
8. **FIFO status bits.** Both the transmit and receive FIFOs have status bits TXFFST or RXFFST (bits 12–8) that define the number of words available in the FIFOs at any time. The transmit FIFO reset bit TXFIFO and receive reset bit RXFIFO reset the FIFO pointers to zero when these bits are cleared to 0. The FIFOs resumes operation from start once these bits are set to 1.
9. **Programmable interrupt levels.** Both transmit and receive FIFO can generate CPU interrupts. The interrupt trigger is generated whenever the transmit FIFO status bits TXFFST (bits 12–8) match (less than or equal to) the interrupt trigger level bits TXFFIL (bits 4–0). This provides a programmable interrupt trigger for transmit and receive sections of the SCI. Default value for these trigger level bits will be 0x11111 for receive FIFO and 0x00000 for transmit FIFO, respectively.

Figure 13-10 and Table 13-4 explain the operation/configuration of SCI interrupts in nonFIFO/FFO mode.


**Figure 13-10. SCI FIFO Interrupt Flags and Enable Logic**
**Table 13-4. SCI Interrupt Flags**

| FIFO Options <sup>(1)</sup> | SCI Interrupt Source            | Interrupt Flags      | Interrupt Enables | FIFO Enable<br>SCIFFENA | Interrupt Line |
|-----------------------------|---------------------------------|----------------------|-------------------|-------------------------|----------------|
| SCI without FIFO            | Receive error                   | RXERR <sup>(2)</sup> | RXERRINTENA       | 0                       | RXINT          |
|                             | Receive break                   | BRKDT                | RX/BKINTENA       | 0                       | RXINT          |
|                             | Data receive                    | RXRDY                | RX/BKINTENA       | 0                       | RXINT          |
|                             | Transmit empty                  | TXRDY                | TXINTENA          | 0                       | TXINT          |
| SCI with FIFO               | Receive error and receive break | RXERR                | RXERRINTENA       | 1                       | RXINT          |
|                             | FIFO receive                    | RXFFIL               | RXFFIENA          | 1                       | RXINT          |
|                             | Transmit empty                  | TXFFIL               | TXFFIENA          | 1                       | TXINT          |
| Auto-baud                   | Auto-baud detected              | ABD                  | Don't care        | x                       | TXINT          |

(1) FIFO mode TXSHF is directly loaded after delay value, TXBUF is not used.

(2) RXERR can be set by BRKDT, FE, OE, PE flags. In FIFO mode, BRKDT interrupt is only through RXERR flag.

### 13.13.2 SCI Auto-Baud

Most SCI modules do not have an auto-baud detect logic built-in hardware. These SCI modules are integrated with embedded controllers whose clock rates are dependent on PLL reset values. Often embedded controller clocks change after final design. In the enhanced feature set this module supports an autobaud-detect logic in hardware. The following section explains the enabling sequence for autobaud-detect feature.

#### 13.13.3 Autobaud-Detect Sequence

Bits ABD and CDC in SCIFFCT control the autobaud logic. The SCIRST bit should be enabled to make autobaud logic work.

If ABD is set while CDC is 1, which indicates auto-baud alignment, SCI transmit FIFO interrupt will occur (TXINT). After the interrupt service CDC bit has to be cleared by software. If CDC remains set even after interrupt service, there should be no repeat interrupts.

1. Enable autobaud-detect mode for the SCI by setting the CDC bit (bit 13) in SCIFFCT and clearing the ABD bit (Bit 15) by writing a 1 to ABDCLR bit (bit 14).
2. Initialize the baud register to be 1 or less than a baud rate limit of 500 Kbps.
3. Allow SCI to receive either character "A" or "a" from a host at the desired baud rate. If the first character is either "A" or "a", the autobaud-detect hardware will detect the incoming baud rate and set the ABD bit.
4. The auto-detect hardware will update the baud rate register with the equivalent baud value hex. The logic will also generate an interrupt to the CPU.
5. Respond to the interrupt clear ADB bit by writing a 1 to ABD CLR (bit 14) of SCIFFCT register and disable further autobaud locking by clearing CDC bit by writing a 0.
6. Read the receive buffer for character "A" or "a" to empty the buffer and buffer status.
7. If ABD is set while CDC is 1, which indicates autobaud alignment, the SCI transmit FIFO interrupt will occur (TXINT). After the interrupt service CDC bit must be cleared by software.

---

#### Note

At higher baud rates, the slew rate of the incoming data bits can be affected by transceiver and connector performance. While normal serial communications may work well, this slew rate may limit reliable autobaud detection at higher baud rates (typically beyond 100k baud) and cause the auto-baudlock feature to fail.

To avoid this, the following is recommended:

- Achieve a baud-lock between the host and C28x SCI boot loader using a lower baud rate.
  - The host may then handshake with the loaded C28x application to set the SCI baud rate register to the desired higher baud rate.
-

## 13.14 SCI Registers

The section describes the Serial Communication Interface module registers.

### 13.14.1 SCI Base Addresses

**Table 13-5. SCI Base Address Table**

| Bit Field Name |           | Base Address |
|----------------|-----------|--------------|
| Instance       | Structure |              |
| SciaRegs       | SCI_REGS  | 0x0000_7050  |
| ScibRegs       | SCI_REGS  | 0x0000_7750  |

### 13.14.2 SCI\_REGS Registers

[Table 13-6](#) lists the SCI\_REGS registers. All register offset addresses not listed in [Table 13-6](#) should be considered as reserved locations and the register contents should not be modified.

**Table 13-6. SCI\_REGS Registers**

| Offset | Acronym  | Register Name                         | Section                            |
|--------|----------|---------------------------------------|------------------------------------|
| 0h     | SCICCR   | SCI Communications Control Register   | <a href="#">Section 13.14.2.1</a>  |
| 1h     | SCICTL1  | SCI Control Register 1                | <a href="#">Section 13.14.2.2</a>  |
| 2h     | SCIHBAUD | SCI Baud Rate (high) Register         | <a href="#">Section 13.14.2.3</a>  |
| 3h     | SCILBAUD | SCI Baud Rate (low) Register          | <a href="#">Section 13.14.2.4</a>  |
| 4h     | SCICTL2  | SCI Control Register 2                | <a href="#">Section 13.14.2.5</a>  |
| 5h     | SCIRXST  | SCI Receive Status Register           | <a href="#">Section 13.14.2.6</a>  |
| 6h     | SCIRXEMU | SCI Receive Emulation Buffer Register | <a href="#">Section 13.14.2.7</a>  |
| 7h     | SCIRXBUF | SCI Receive Data Buffer Register      | <a href="#">Section 13.14.2.8</a>  |
| 9h     | SCITXBUF | SCI Transmit Data Buffer Register     | <a href="#">Section 13.14.2.9</a>  |
| Ah     | SCIFFTX  | SCI FIFO Transmit Register            | <a href="#">Section 13.14.2.10</a> |
| Bh     | SCIFFRX  | SCI FIFO Receive Register             | <a href="#">Section 13.14.2.11</a> |
| Ch     | SCIFFCT  | SCI FIFO Control Register             | <a href="#">Section 13.14.2.12</a> |
| Fh     | SCIPRI   | SCI Priority Control Register         | <a href="#">Section 13.14.2.13</a> |



Complex bit access types are encoded to fit into small table cells. [Table 13-7](#) shows the codes that are used for access types in this section.

**Table 13-7. SCI\_REGS Access Type Codes**

| Access Type              | Code | Description                                                                                                                                                                                                                                                                          |
|--------------------------|------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Read Type                |      |                                                                                                                                                                                                                                                                                      |
| R                        | R    | Read                                                                                                                                                                                                                                                                                 |
| R-0                      | R-0  | Read<br>Returns 0s                                                                                                                                                                                                                                                                   |
| Write Type               |      |                                                                                                                                                                                                                                                                                      |
| W                        | W    | Write                                                                                                                                                                                                                                                                                |
| W1S                      | W1S  | Write<br>1 to set                                                                                                                                                                                                                                                                    |
| Reset or Default Value   |      |                                                                                                                                                                                                                                                                                      |
| -n                       |      | Value after reset or the default value                                                                                                                                                                                                                                               |
| Register Array Variables |      |                                                                                                                                                                                                                                                                                      |
| i,j,k,l,m,n              |      | When these variables are used in a register name, an offset, or an address, they refer to the value of a register array where the register is part of a group of repeating registers. The register groups form a hierarchical structure and the array is represented with a formula. |
| y                        |      | When this variable is used in a register name, an offset, or an address it refers to the value of a register array.                                                                                                                                                                  |

### 13.14.2.1 SCI Communications Control Register (SCICCR) (Offset = 0h) [reset = 0h]

SCICCR defines the character format, protocol, and communications mode used by the SCI.

**Figure 13-11. SCI Communications Control Register (SCICCR)**

|          |        |           |           |               |          |   |   |
|----------|--------|-----------|-----------|---------------|----------|---|---|
| 15       | 14     | 13        | 12        | 11            | 10       | 9 | 8 |
| RESERVED |        |           |           |               |          |   |   |
| R-0h     |        |           |           |               |          |   |   |
| 7        | 6      | 5         | 4         | 3             | 2        | 1 | 0 |
| STOPBITS | PARITY | PARITYENA | LOOPBKENA | ADDRIDLE_MODE | SCICCHAR |   |   |
| R/W-0h   | R/W-0h | R/W-0h    | R/W-0h    | R/W-0h        | R/W-0h   |   |   |

**Table 13-8. SCI Communications Control Register (SCICCR) Field Descriptions**

| Bit  | Field         | Type | Reset | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|------|---------------|------|-------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-8 | RESERVED      | R    | 0h    | Reserved                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| 7    | STOPBITS      | R/W  | 0h    | SCI number of stop bits.<br>This bit specifies the number of stop bits transmitted. The receiver checks for only one stop bit.<br>Reset type: SYSRSn<br>0h (R/W) = One stop bit<br>1h (R/W) = Two stop bits                                                                                                                                                                                                                                                                                                                                                                                                                     |
| 6    | PARITY        | R/W  | 0h    | SCI parity odd/even selection.<br>If the PARITY ENABLE bit (SCICCR, bit 5) is set, PARITY (bit 6) designates odd or even parity (odd or even number of bits with the value of 1 in both transmitted and received characters).<br>Reset type: SYSRSn<br>0h (R/W) = Odd parity<br>1h (R/W) = Even parity                                                                                                                                                                                                                                                                                                                          |
| 5    | PARITYENA     | R/W  | 0h    | SCI parity enable.<br>This bit enables or disables the parity function. If the SCI is in the address bit multiprocessor mode (set using bit 3 of this register), the address bit is included in the parity calculation (if parity is enabled). For characters of less than eight bits, the remaining unused bits should be masked out of the parity calculation.<br>Reset type: SYSRSn<br>0h (R/W) = Parity disabled<br>no parity bit is generated during transmission or is expected during reception<br>1h (R/W) = Parity is enabled                                                                                          |
| 4    | LOOPBKENA     | R/W  | 0h    | Loop Back test mode enable.<br>This bit enables the Loop Back test mode where the Tx pin is internally connected to the Rx pin.<br>Reset type: SYSRSn<br>0h (R/W) = Loop Back test mode disabled<br>1h (R/W) = Loop Back test mode enabled                                                                                                                                                                                                                                                                                                                                                                                      |
| 3    | ADDRIDLE_MODE | R/W  | 0h    | SCI multiprocessor mode control bit.<br>This bit selects one of the multiprocessor protocols. Multiprocessor communication is different from the other communication modes because it uses SLEEP and TXWAKE functions (bits SCICTL1, bit 2 and SCICTL1, bit 3, respectively). The idle-line mode is usually used for normal communications because the address-bit mode adds an extra bit to the frame. The idle-line mode does not add this extra bit and is compatible with RS-232 type communications.<br>Reset type: SYSRSn<br>0h (R/W) = Idle-line mode protocol selected<br>1h (R/W) = Address-bit mode protocol selected |

**Table 13-8. SCI Communications Control Register (SCICCR) Field Descriptions (continued)**

| Bit | Field    | Type | Reset | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|-----|----------|------|-------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 2-0 | SCICCHAR | R/W  | 0h    | <p>Character-length control bits 2-0.</p> <p>These bits select the SCI character length from one to eight bits. Characters of less than eight bits are right-justified in SCIRXBUF and SCIRXEMU and are padded with leading zeros in SCIRXBUF. SCITXBUF doesn't need to be padded with leading zeros.</p> <p>Reset type: SYSRSn</p> <p>0h (R/W) = SCICCHAR_LENGTH_1</p> <p>1h (R/W) = SCICCHAR_LENGTH_2</p> <p>2h (R/W) = SCICCHAR_LENGTH_3</p> <p>3h (R/W) = SCICCHAR_LENGTH_4</p> <p>4h (R/W) = SCICCHAR_LENGTH_5</p> <p>5h (R/W) = SCICCHAR_LENGTH_6</p> <p>6h (R/W) = SCICCHAR_LENGTH_7</p> <p>7h (R/W) = SCICCHAR_LENGTH_8</p> |

### 13.14.2.2 SCI Control Register 1 (SCICTL1) (Offset = 1h) [reset = 0h]

SCICTL1 controls the receiver/transmitter enable, TXWAKE and SLEEP functions, and the SCI software reset.

**Figure 13-12. SCI Control Register 1 (SCICTL1)**

|          |             |         |          |        |        |        |        |
|----------|-------------|---------|----------|--------|--------|--------|--------|
| 15       | 14          | 13      | 12       | 11     | 10     | 9      | 8      |
| RESERVED |             |         |          |        |        |        |        |
| R-0h     |             |         |          |        |        |        |        |
| 7        | 6           | 5       | 4        | 3      | 2      | 1      | 0      |
| RESERVED | RXERRINTENA | SWRESET | RESERVED | TXWAKE | SLEEP  | TXENA  | RXENA  |
| R-0h     | R/W-0h      | R/W-0h  | R-0h     | R/W-0h | R/W-0h | R/W-0h | R/W-0h |

**Table 13-9. SCI Control Register 1 (SCICTL1) Field Descriptions**

| Bit  | Field       | Type | Reset | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
|------|-------------|------|-------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-7 | RESERVED    | R    | 0h    | Reserved                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| 6    | RXERRINTENA | R/W  | 0h    | SCI receive error interrupt enable.<br>Setting this bit enables an interrupt if the RX ERROR bit (SCIRXST, bit 7) becomes set because of errors occurring.<br>Reset type: SYSRSn<br>0h (R/W) = Receive error interrupt disabled<br>1h (R/W) = Receive error interrupt enabled                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| 5    | SWRESET     | R/W  | 0h    | SCI software reset (active low).<br>Writing a 0 to this bit initializes the SCI state machines and operating flags (registers SCICTL2 and SCIRXST) to the reset condition. The SW RESET bit does not affect any of the configuration bits.<br>All affected logic is held in the specified reset state until a 1 is written to SW RESET (the bit values following a reset are shown beneath each register diagram in this section). Thus, after a system reset, re-enable the SCI by writing a 1 to this bit. Clear this bit after a receiver break detect (BRKDT flag, bit SCIRXST, bit 5).<br>SW RESET affects the operating flags of the SCI, but it neither affects the configuration bits nor restores the reset values. Once SW RESET is asserted, the flags are frozen until the bit is deasserted.<br>The affected flags are as follows:<br>Value After SW SCI Flag Register Bit<br>RESET<br>1 TXRDY SCICTL2, bit 7<br>1 TX EMPTY SCICTL2, bit 6<br>0 RXWAKE SCIRXST, bit 1<br>0 PE SCIRXST, bit 2<br>0 OE SCIRXST, bit 3<br>0 FE SCIRXST, bit 4<br>0 BRKDT SCIRXST, bit 5<br>0 RXRDY SCIRXST, bit 6<br>0 RX ERROR SCIRXST, bit 7<br>Reset type: SYSRSn<br>0h (R/W) = Writing a 0 to this bit initializes the SCI state machines and operating flags (registers SCICTL2 and SCIRXST) to the reset condition.<br>1h (R/W) = After a system reset, re-enable the SCI by writing a 1 to this bit. |
| 4    | RESERVED    | R    | 0h    | Reserved                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |

**Table 13-9. SCI Control Register 1 (SCICTL1) Field Descriptions (continued)**

| Bit | Field  | Type | Reset | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|-----|--------|------|-------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 3   | TXWAKE | R/W  | 0h    | <p>SCI transmitter wake-up method select.</p> <p>The TXWAKE bit controls selection of the data-transmit feature, depending on which transmit mode (idle-line or address-bit) is specified at the ADDR/IDLE MODE bit (SCICCR, bit 3)</p> <p>Reset type: SYSRSn</p> <p>0h (R/W) = Transmit feature is not selected. In idle-line mode: write a 1 to TXWAKE, then write data to register SCITXBUF to generate an idle period of 11 data bits In address-bit mode: write a 1 to TXWAKE, then write data to SCITXBUF to set the address bit for that frame to 1</p> <p>1h (R/W) = Transmit feature selected is dependent on the mode, idle-line or address-bit: TXWAKE is not cleared by the SW RESET bit (SCICTL1, bit 5)</p> <p>it is cleared by a system reset or the transfer of TXWAKE to the WUT flag.</p>                        |
| 2   | SLEEP  | R/W  | 0h    | <p>SCI sleep.</p> <p>The TXWAKE bit controls selection of the data-transmit feature, depending on which transmit mode (idle-line or address-bit) is specified at the ADDR/IDLE MODE bit (SCICCR, bit 3). In a multiprocessor configuration, this bit controls the receiver sleep function. Clearing this bit brings the SCI out of the sleep mode. The receiver still operates when the SLEEP bit is set however, operation does not update the receiver buffer ready bit (SCIRXST, bit 6, RXRDY) or the error status bits (SCIRXST, bit 5-2: BRKDT, FE, OE, and PE) unless the address byte is detected. SLEEP is not cleared when the address byte is detected.</p> <p>Reset type: SYSRSn</p> <p>0h (R/W) = Sleep mode disabled</p> <p>1h (R/W) = Sleep mode enabled</p>                                                         |
| 1   | TXENA  | R/W  | 0h    | <p>SCI transmitter enable.</p> <p>Data is transmitted through the SCITXD pin only when TXENA is set. If reset, transmission is halted but only after all data previously written to SCITXBUF has been sent. Data written into SCITXBUF when TXENA is disabled will not be transmitted even if the TXENA is enabled later.</p> <p>Reset type: SYSRSn</p> <p>0h (R/W) = Transmitter disabled</p> <p>1h (R/W) = Transmitter enabled</p>                                                                                                                                                                                                                                                                                                                                                                                               |
| 0   | RXENA  | R/W  | 0h    | <p>SCI receiver enable.</p> <p>Data is received on the SCIRXD pin and is sent to the receiver shift register and then the receiver buffers. This bit enables or disables the receiver (transfer to the buffers).</p> <p>Clearing RXENA stops received characters from being transferred to the two receiver buffers and also stops the generation of receiver interrupts. However, the receiver shift register can continue to assemble characters. Thus, if RXENA is set during the reception of a character, the complete character will be transferred into the receiver buffer registers, SCIRXEMU and SCIRXBUF.</p> <p>Reset type: SYSRSn</p> <p>0h (R/W) = Prevent received characters from transfer into the SCIRXEMU and SCIRXBUF receiver buffers</p> <p>1h (R/W) = Send received characters to SCIRXEMU and SCIRXBUF</p> |

### 13.14.2.3 SCI Baud Rate (high) (SCIHBAUD) Register (Offset = 2h) [reset = 0h]

The values in SCIHBAUD and SCILBAUD specify the baud rate for the SCI.

**Figure 13-13. SCI Baud Rate (high) (SCIHBAUD) Register**

|          |    |    |    |    |    |   |   |
|----------|----|----|----|----|----|---|---|
| 15       | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
| RESERVED |    |    |    |    |    |   |   |
| R-0h     |    |    |    |    |    |   |   |
| 7        | 6  | 5  | 4  | 3  | 2  | 1 | 0 |
| BAUD     |    |    |    |    |    |   |   |
| R/W-0h   |    |    |    |    |    |   |   |

**Table 13-10. SCI Baud Rate (high) (SCIHBAUD) Register Field Descriptions**

| Bit  | Field    | Type | Reset | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|------|----------|------|-------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-8 | RESERVED | R    | 0h    | Reserved                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| 7-0  | BAUD     | R/W  | 0h    | <p>SCI 16-bit baud selection Registers SCIHBAUD (MSbyte). The internally-generated serial clock is determined by the low speed peripheral clock (LSPCLK) signal and the two baud-select registers. The SCI uses the 16-bit value of these registers to select one of 64K serial clock rates for the communication modes.</p> <p><math>BRR = (SCIHBAUD \ll 8) + (SCILBAUD)</math></p> <p>The SCI baud rate is calculated using the following equation:</p> <p>SCI Asynchronous Baud = <math>LSPCLK / ((BRR + 1) * 8)</math></p> <p>Alternatively,</p> <p><math>BRR = LSPCLK / (SCI Asynchronous Baud * 8) - 1</math></p> <p>Note that the above formulas are applicable only when <math>0 &lt; BRR &lt; 65536</math>. If <math>BRR = 0</math>, then</p> <p>SCI Asynchronous Baud = <math>LSPCLK / 16</math></p> <p>Where: BRR = the 16-bit value (in decimal) in the baud-select registers</p> <p>Reset type: SYSRSTn</p> |

### 13.14.2.4 SCI Baud Rate (low) (SCILBAUD) Register (Offset = 3h) [reset = 0h]

The values in SCIHBAUD and SCILBAUD specify the baud rate for the SCI.

**Figure 13-14. SCI Baud Rate (low) (SCILBAUD) Register**

|          |    |    |    |    |    |   |   |
|----------|----|----|----|----|----|---|---|
| 15       | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
| RESERVED |    |    |    |    |    |   |   |
| R-0h     |    |    |    |    |    |   |   |
| 7        | 6  | 5  | 4  | 3  | 2  | 1 | 0 |
| BAUD     |    |    |    |    |    |   |   |
| R/W-0h   |    |    |    |    |    |   |   |

**Table 13-11. SCI Baud Rate (low) (SCILBAUD) Register Field Descriptions**

| Bit  | Field    | Type | Reset | Description                                             |
|------|----------|------|-------|---------------------------------------------------------|
| 15-8 | RESERVED | R    | 0h    | Reserved                                                |
| 7-0  | BAUD     | R/W  | 0h    | See SCIHBAUD Detailed Description<br>Reset type: SYSRSn |

### 13.14.2.5 SCI Control Register 2 (SCICTL2) (Offset = 4h) [reset = C0h]

SCICTL2 enables the receive-ready, break-detect, and transmit-ready interrupts as well as transmitter-ready and -empty flags.

**Figure 13-15. SCI Control Register 2 (SCICTL2)**

|          |         |          |    |    |    |            |          |
|----------|---------|----------|----|----|----|------------|----------|
| 15       | 14      | 13       | 12 | 11 | 10 | 9          | 8        |
| RESERVED |         |          |    |    |    |            |          |
| R-0h     |         |          |    |    |    |            |          |
| 7        | 6       | 5        | 4  | 3  | 2  | 1          | 0        |
| TXRDY    | TXEMPTY | RESERVED |    |    |    | RXBKINTENA | TXINTENA |
| R-1h     | R-1h    | R-0h     |    |    |    | R/W-0h     | R/W-0h   |

**Table 13-12. SCI Control Register 2 (SCICTL2) Field Descriptions**

| Bit  | Field      | Type | Reset | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|------|------------|------|-------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-8 | RESERVED   | R    | 0h    | Reserved                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| 7    | TXRDY      | R    | 1h    | <p>Transmitter buffer register ready flag.</p> <p>When set, this bit indicates that the transmit data buffer register, SCITXBUF, is ready to receive another character. Writing data to the SCITXBUF automatically clears this bit. When set, this flag asserts a transmitter interrupt request if the interrupt-enable bit, TX INT ENA (SCICTL2.0), is also set. TXRDY is set to 1 by enabling the SW RESET bit (SCICTL1.5) or by a system reset.</p> <p>Reset type: SYSRSn</p> <p>0h (R/W) = SCITXBUF is full</p> <p>1h (R/W) = SCITXBUF is ready to receive the next character</p>                                                                                                                                                                                                                                                         |
| 6    | TXEMPTY    | R    | 1h    | <p>Transmitter empty flag.</p> <p>This flag's value indicates the contents of the transmitter's buffer register (SCITXBUF) and shift register (TXSHF). An active SW RESET (SCICTL1.5), or a system reset, sets this bit. This bit does not cause an interrupt request.</p> <p>Reset type: SYSRSn</p> <p>0h (R/W) = Transmitter buffer or shift register or both are loaded with data</p> <p>1h (R/W) = Transmitter buffer and shift registers are both empty</p>                                                                                                                                                                                                                                                                                                                                                                              |
| 5-2  | RESERVED   | R    | 0h    | Reserved                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| 1    | RXBKINTENA | R/W  | 0h    | <p>Receiver-buffer/break interrupt enable.</p> <p>This bit controls the interrupt request caused by either the RXRDY flag or the BRKDT flag (bits SCIRXST.6 and .5) being set. However, RX/BK INT ENA does not prevent the setting of these flags.</p> <p>Reset type: SYSRSn</p> <p>0h (R/W) = Disable RXRDY/BRKDT interrupt</p> <p>1h (R/W) = Enable RXRDY/BRKDT interrupt</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| 0    | TXINTENA   | R/W  | 0h    | <p>SCITXBUF-register interrupt enable.</p> <p>This bit controls the interrupt request caused by the setting of TXRDY flag bit (SCICTL2.7). However, it does not prevent the TXRDY flag from being set (which indicates SCITXBUF is ready to receive another character).</p> <p>0 Disable TXRDY interrupt</p> <p>1 Enable TXRDY interrupt.</p> <p>In non-FIFO mode, a dummy (or a valid) data has to be written to SCITXBUF for the first transmit interrupt to occur. This is the case when you enable the transmit interrupt for the first time and also when you re-enable (disable and then enable) the transmit interrupt. If TXINTENA is enabled after writing the data to SCITXBUF, it will not generate an interrupt.</p> <p>Reset type: SYSRSn</p> <p>0h (R/W) = Disable TXRDY interrupt</p> <p>1h (R/W) = Enable TXRDY interrupt</p> |



### 13.14.2.6 SCI Receive Status (SCIRXST) Register (Offset = 5h) [reset = 0h]

SCIRXST contains seven bits that are receiver status flags (two of which can generate interrupt requests). Each time a complete character is transferred to the receiver buffers (SCIRXEMU and SCIRXBUF), the status flags are updated.

**Figure 13-16. SCI Receive Status (SCIRXST) Register**

|          |       |       |      |      |      |        |          |
|----------|-------|-------|------|------|------|--------|----------|
| 15       | 14    | 13    | 12   | 11   | 10   | 9      | 8        |
| RESERVED |       |       |      |      |      |        |          |
| R-0h     |       |       |      |      |      |        |          |
| 7        | 6     | 5     | 4    | 3    | 2    | 1      | 0        |
| RXERROR  | RXRDY | BRKDT | FE   | OE   | PE   | RXWAKE | RESERVED |
| R-0h     | R-0h  | R-0h  | R-0h | R-0h | R-0h | R-0h   | R-0h     |

**Table 13-13. SCI Receive Status (SCIRXST) Register Field Descriptions**

| Bit  | Field    | Type | Reset | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|------|----------|------|-------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-8 | RESERVED | R    | 0h    | Reserved                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| 7    | RXERROR  | R    | 0h    | <p>SCI receiver error flag.</p> <p>The RX ERROR flag indicates that one of the error flags in the receiver status register is set. RX ERROR is a logical OR of the break detect, framing error, overrun, and parity error enable flags (bits 5-2: BRKDT, FE, OE, and PE).</p> <p>A 1 on this bit will cause an interrupt if the RX ERR INT ENA bit (SCICTL1.6) is set. This bit can be used for fast error-condition checking during the interrupt service routine. This error flag cannot be cleared directly</p> <p>it is cleared by an active SW RESET or by a system reset.</p> <p>Reset type: SYSRSn</p> <p>0h (R/W) = No error flags set</p> <p>1h (R/W) = Error flag(s) set</p>                                                                                                                                                                     |
| 6    | RXRDY    | R    | 0h    | <p>SCI receiver-ready flag.</p> <p>When a new character is ready to be read from the SCIRXBUF register, the receiver sets this bit, and a receiver interrupt is generated if the RX/BK INT ENA bit (SCICTL2.1) is a 1. RXRDY is cleared by a reading of the SCIRXBUF register, by an active SW RESET, or by a system reset.</p> <p>Reset type: SYSRSn</p> <p>0h (R/W) = No new character in SCIRXBUF</p> <p>1h (R/W) = Character ready to be read from SCIRXBUF</p>                                                                                                                                                                                                                                                                                                                                                                                        |
| 5    | BRKDT    | R    | 0h    | <p>SCI break-detect flag.</p> <p>The SCI sets this bit when a break condition occurs. A break condition occurs when the SCI receiver data line (SCIRXD) remains continuously low for at least ten bits, beginning after a missing first stop bit. The occurrence of a break causes a receiver interrupt to be generated if the RX/BK INT ENA bit is a 1, but it does not cause the receiver buffer to be loaded. A BRKDT interrupt can occur even if the receiver SLEEP bit is set to 1. BRKDT is cleared by an active SW RESET or by a system reset. It is not cleared by receipt of a character after the break is detected. In order to receive more characters, the SCI must be reset by toggling the SW RESET bit or by a system reset.</p> <p>Reset type: SYSRSn</p> <p>0h (R/W) = No break condition</p> <p>1h (R/W) = Break condition occurred</p> |

**Table 13-13. SCI Receive Status (SCIRXST) Register Field Descriptions (continued)**

| Bit | Field    | Type | Reset | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|-----|----------|------|-------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 4   | FE       | R    | 0h    | <p>SCI framing-error flag.</p> <p>The SCI sets this bit when an expected stop bit is not found. Only the first stop bit is checked. The missing stop bit indicates that synchronization with the start bit has been lost and that the character is incorrectly framed. The FE bit is reset by a clearing of the SW RESET bit or by a system reset.</p> <p>Reset type: SYSRSn</p> <p>0h (R/W) = No framing error detected</p> <p>1h (R/W) = Framing error detected</p>                                                                                                                                                                                                                                                                                                                   |
| 3   | OE       | R    | 0h    | <p>SCI overrun-error flag.</p> <p>The SCI sets this bit when a character is transferred into registers SCIRXEMU and SCIRXBUF before the previous character is fully read by the CPU or DMAC. The previous character is overwritten and lost. The OE flag bit is reset by an active SW RESET or by a system reset.</p> <p>Reset type: SYSRSn</p> <p>0h (R/W) = No overrun error detected</p> <p>1h (R/W) = Overrun error detected</p>                                                                                                                                                                                                                                                                                                                                                    |
| 2   | PE       | R    | 0h    | <p>SCI parity-error flag.</p> <p>This flag bit is set when a character is received with a mismatch between the number of 1s and its parity bit. The address bit is included in the calculation. If parity generation and detection is not enabled, the PE flag is disabled and read as 0. The PE bit is reset by an active SW RESET or a system reset.</p> <p>Reset type: SYSRSn</p> <p>0h (R/W) = No parity error or parity is disabled</p> <p>1h (R/W) = Parity error is detected</p>                                                                                                                                                                                                                                                                                                 |
| 1   | RXWAKE   | R    | 0h    | <p>Receiver wake-up-detect flag</p> <p>Reset type: SYSRSn</p> <p>0h (R/W) = No detection of a receiver wake-up condition</p> <p>1h (R/W) = A value of 1 in this bit indicates detection of a receiver wake-up condition. In the address-bit multiprocessor mode (SCICCR.3 = 1), RXWAKE reflects the value of the address bit for the character contained in SCIRXBUF. In the idle-line multiprocessor mode, RXWAKE is set if the SCIRXD data line is detected as idle.</p> <p>RXWAKE is a read-only flag, cleared by one of the following:</p> <ul style="list-style-type: none"> <li>- The transfer of the first byte after the address byte to SCIRXBUF (only in non-FIFO mode)</li> <li>- The reading of SCIRXBUF</li> <li>- An active SW RESET</li> <li>- A system reset</li> </ul> |
| 0   | RESERVED | R    | 0h    | Reserved                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |

### 13.14.2.7 SCI Receive Emulation Buffer (SCIRXEMU) Register (Offset = 6h) [reset = 0h]

Normal SCI data-receive operations read the data received from the SCIRXBUF register. The SCIRXEMU register is used principally during a debug connection because it can continuously read the data received for screen updates without clearing the RXRDY flag. SCIRXEMU is cleared by a system reset. This is the register that should be used in the CCS watch window to view the contents of the SCIRXBUF register. SCIRXEMU is not physically implemented

it is just a different address location to access the SCIRXBUF register without clearing the RXRDY flag.

**Figure 13-17. SCI Receive Emulation Buffer (SCIRXEMU) Register**

|          |    |    |    |    |    |   |   |
|----------|----|----|----|----|----|---|---|
| 15       | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
| RESERVED |    |    |    |    |    |   |   |
| R-0h     |    |    |    |    |    |   |   |
| 7        | 6  | 5  | 4  | 3  | 2  | 1 | 0 |
| ERXDT    |    |    |    |    |    |   |   |
| R-0h     |    |    |    |    |    |   |   |

**Table 13-14. SCI Receive Emulation Buffer (SCIRXEMU) Register Field Descriptions**

| Bit  | Field    | Type | Reset | Description                                         |
|------|----------|------|-------|-----------------------------------------------------|
| 15-8 | RESERVED | R    | 0h    | Reserved                                            |
| 7-0  | ERXDT    | R    | 0h    | Receive emulation buffer data<br>Reset type: SYSRSn |

### 13.14.2.8 SCI Receive Data Buffer (SCIRXBUF) Register (Offset = 7h) [reset = 0h]

When the current data received is shifted from RXSHF to the receiver buffer, flag bit RXRDY is set and the data is ready to be read. If the RXBKINTENA bit (SCICTL2.1) is set, this shift also causes an interrupt. When SCIRXBUF is read, the RXRDY flag is reset. SCIRXBUF is cleared by a system reset.

**Figure 13-18. SCI Receive Data Buffer (SCIRXBUF) Register**

|         |         |          |    |    |    |   |   |
|---------|---------|----------|----|----|----|---|---|
| 15      | 14      | 13       | 12 | 11 | 10 | 9 | 8 |
| SCIFFFE | SCIFFPE | RESERVED |    |    |    |   |   |
| R-0h    | R-0h    | R-0h     |    |    |    |   |   |
| 7       | 6       | 5        | 4  | 3  | 2  | 1 | 0 |
| SAR     |         |          |    |    |    |   |   |
| R-0h    |         |          |    |    |    |   |   |

**Table 13-15. SCI Receive Data Buffer (SCIRXBUF) Register Field Descriptions**

| Bit  | Field    | Type | Reset | Description                                                                                                                                                                                                                                                                                                                                                                                                    |
|------|----------|------|-------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15   | SCIFFFE  | R    | 0h    | SCIFFFE. SCI FIFO Framing error flag bit (applicable only if the FIFO is enabled)<br>Reset type: SYSRSn<br>0h (R/W) = No frame error occurred while receiving the character, in bits 7-0. This bit is associated with the character on the top of the FIFO.<br>1h (R/W) = A frame error occurred while receiving the character in bits 7-0. This bit is associated with the character on the top of the FIFO.  |
| 14   | SCIFFPE  | R    | 0h    | SCIFFPE. SCI FIFO parity error flag bit (applicable only if the FIFO is enabled)<br>Reset type: SYSRSn<br>0h (R/W) = No parity error occurred while receiving the character, in bits 7-0. This bit is associated with the character on the top of the FIFO.<br>1h (R/W) = A parity error occurred while receiving the character in bits 7-0. This bit is associated with the character on the top of the FIFO. |
| 13-8 | RESERVED | R    | 0h    | Reserved                                                                                                                                                                                                                                                                                                                                                                                                       |
| 7-0  | SAR      | R    | 0h    | Receive Character bits<br>Reset type: SYSRSn                                                                                                                                                                                                                                                                                                                                                                   |

### 13.14.2.9 SCI Transmit Data Buffer (SCITXBUF) Register (Offset = 9h) [reset = 0h]

Data bits to be transmitted are written to SCITXBUF. These bits must be right-justified because the leftmost bits are ignored for characters less than eight bits long. The transfer of data from this register to the TXSHF transmitter shift register sets the TXRDY flag (SCICTL2.7), indicating that SCITXBUF is ready to receive another set of data. If bit TXINTENA (SCICTL2.0) is set, this data transfer also causes an interrupt.

**Figure 13-19. SCI Transmit Data Buffer (SCITXBUF) Register**

|          |    |    |    |    |    |   |   |
|----------|----|----|----|----|----|---|---|
| 15       | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
| RESERVED |    |    |    |    |    |   |   |
| R-0h     |    |    |    |    |    |   |   |
| 7        | 6  | 5  | 4  | 3  | 2  | 1 | 0 |
| TXDT     |    |    |    |    |    |   |   |
| R/W-0h   |    |    |    |    |    |   |   |

**Table 13-16. SCI Transmit Data Buffer (SCITXBUF) Register Field Descriptions**

| Bit  | Field    | Type | Reset | Description                                |
|------|----------|------|-------|--------------------------------------------|
| 15-8 | RESERVED | R    | 0h    | Reserved                                   |
| 7-0  | TXDT     | R/W  | 0h    | Transmit data buffer<br>Reset type: SYSRSn |

### 13.14.2.10 SCI FIFO Transmit (SCIFFTX) Register (Offset = Ah) [reset = A000h]

SCIFFTX controls the transmit FIFO interrupt, FIFO enhancements, and reset for the SCI transmit and receive channels.

**Figure 13-20. SCI FIFO Transmit (SCIFFTX) Register**

|         |            |             |        |    |    |   |   |
|---------|------------|-------------|--------|----|----|---|---|
| 15      | 14         | 13          | 12     | 11 | 10 | 9 | 8 |
| SCIRST  | SCIFFENA   | TXFIFORESET | TXFFST |    |    |   |   |
| R/W-1h  | R/W-0h     | R/W-1h      | R-0h   |    |    |   |   |
| 7       | 6          | 5           | 4      | 3  | 2  | 1 | 0 |
| TXFFINT | TXFFINTCLR | TXFFIENA    | TXFFIL |    |    |   |   |
| R-0h    | R-0/W1S-0h | R/W-0h      | R/W-0h |    |    |   |   |

**Table 13-17. SCI FIFO Transmit (SCIFFTX) Register Field Descriptions**

| Bit  | Field       | Type    | Reset | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|------|-------------|---------|-------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15   | SCIRST      | R/W     | 1h    | SCI Reset<br>0 Write 0 to reset the SCI transmit and receive channels. SCI FIFO register configuration bits will be left as is.<br>1 SCI FIFO can resume transmit or receive. SCIRST should be 1 even for Autobaud logic to work.<br>Reset type: SYSRSn                                                                                                                                                                                                                                                                                                                       |
| 14   | SCIFFENA    | R/W     | 0h    | SCI FIFO enable<br>Reset type: SYSRSn<br>0h (R/W) = SCI FIFO enhancements are disabled<br>1h (R/W) = SCI FIFO enhancements are enabled                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| 13   | TXFIFORESET | R/W     | 1h    | Transmit FIFO reset<br>Reset type: SYSRSn<br>0h (R/W) = Reset the FIFO pointer to zero and hold in reset<br>1h (R/W) = Re-enable transmit FIFO operation                                                                                                                                                                                                                                                                                                                                                                                                                      |
| 12-8 | TXFFST      | R       | 0h    | FIFO status<br>Reset type: SYSRSn<br>0h (R/W) = Transmit FIFO is empty<br>1h (R/W) = Transmit FIFO has 1 words<br>2h (R/W) = Transmit FIFO has 2 words<br>3h (R/W) = Transmit FIFO has 3 words<br>4h (R/W) = Transmit FIFO has 4 words                                                                                                                                                                                                                                                                                                                                        |
| 7    | TXFFINT     | R       | 0h    | Transmit FIFO interrupt<br>Reset type: SYSRSn<br>0h (R/W) = TXFIFO interrupt has not occurred, read-only bit<br>1h (R/W) = TXFIFO interrupt has occurred, read-only bit                                                                                                                                                                                                                                                                                                                                                                                                       |
| 6    | TXFFINTCLR  | R-0/W1S | 0h    | Transmit FIFO clear<br>Reset type: SYSRSn<br>0h (R/W) = Write 0 has no effect on TXFIFINT flag bit, Bit reads back a zero<br>1h (R/W) = Write 1 to clear TXFFINT flag in bit 7                                                                                                                                                                                                                                                                                                                                                                                                |
| 5    | TXFFIENA    | R/W     | 0h    | Transmit FIFO interrupt enable<br>Reset type: SYSRSn<br>0h (R/W) = TX FIFO interrupt is disabled<br>1h (R/W) = TX FIFO interrupt is enabled. This interrupt is triggered whenever the transmit FIFO status (TXFFST) bits match (equal to or less than) the interrupt trigger level bits TXFFIL (bits 4-0).                                                                                                                                                                                                                                                                    |
| 4-0  | TXFFIL      | R/W     | 0h    | TXFFIL4-0 Transmit FIFO interrupt level bits.<br>The transmit FIFO generates an interrupt whenever the FIFO status bits (TXFFST4-0) are less than or equal to the FIFO level bits (TXFFIL4-0). The maximum value that can be assigned to these bits to generate an interrupt cannot be more than the depth of the TX FIFO. The default value of these bits after reset is 00000b. Users should set TXFFIL to best fit their application needs by weighing between the CPU overhead to service the ISR and the best possible usage of SCI bus bandwidth.<br>Reset type: SYSRSn |

### 13.14.2.11 SCI FIFO Receive (SCIFFRX) Register (Offset = Bh) [reset = 201Fh]

SCIFFTX controls the receive FIFO interrupt, receive FIFO reset, and status of the receive FIFO overflow.

**Figure 13-21. SCI FIFO Receive (SCIFFRX) Register**

| 15      | 14         | 13          | 12      | 11 | 10 | 9 | 8 |
|---------|------------|-------------|---------|----|----|---|---|
| RXFFOVF | RXFFOVRCLR | RXFIFORESET | RXFFST  |    |    |   |   |
| R-0h    | R-0/W1S-0h | R/W-1h      | R-0h    |    |    |   |   |
| 7       | 6          | 5           | 4       | 3  | 2  | 1 | 0 |
| RXFFINT | RXFFINTCLR | RXFFIENA    | RXFFIL  |    |    |   |   |
| R-0h    | W-0h       | R/W-0h      | R/W-1Fh |    |    |   |   |

**Table 13-18. SCI FIFO Receive (SCIFFRX) Register Field Descriptions**

| Bit  | Field       | Type    | Reset | Description                                                                                                                                                                                                                                                                                                                                                                                                                                               |
|------|-------------|---------|-------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15   | RXFFOVF     | R       | 0h    | Receive FIFO overflow.<br>This will function as flag, but cannot generate interrupt by itself. This condition will occur while receive interrupt is active. Receive interrupts should service this flag condition.<br>Reset type: SYSRSn<br>0h (R/W) = Receive FIFO has not overflowed, read-only bit<br>1h (R/W) = Receive FIFO has overflowed, read-only bit. More than 16 words have been received in to the FIFO, and the first received word is lost |
| 14   | RXFFOVRCLR  | R-0/W1S | 0h    | RXFFOVF clear<br>Reset type: SYSRSn<br>0h (R/W) = Write 0 has no effect on RXFFOVF flag bit, Bit reads back a zero<br>1h (R/W) = Write 1 to clear RXFFOVF flag in bit 15                                                                                                                                                                                                                                                                                  |
| 13   | RXFIFORESET | R/W     | 1h    | Receive FIFO reset<br>Reset type: SYSRSn<br>0h (R/W) = Write 0 to reset the FIFO pointer to zero, and hold in reset.<br>1h (R/W) = Re-enable receive FIFO operation                                                                                                                                                                                                                                                                                       |
| 12-8 | RXFFST      | R       | 0h    | FIFO status<br>Reset type: SYSRSn<br>0h (R/W) = Receive FIFO is empty<br>1h (R/W) = Receive FIFO has 1 words<br>2h (R/W) = Receive FIFO has 2 words<br>3h (R/W) = Receive FIFO has 3 words<br>4h (R/W) = Receive FIFO has 4 words                                                                                                                                                                                                                         |
| 7    | RXFFINT     | R       | 0h    | Receive FIFO interrupt<br>Reset type: SYSRSn<br>0h (R/W) = RXFIFO interrupt has not occurred, read-only bit<br>1h (R/W) = RXFIFO interrupt has occurred, read-only bit                                                                                                                                                                                                                                                                                    |
| 6    | RXFFINTCLR  | W       | 0h    | Receive FIFO interrupt clear<br>Reset type: SYSRSn<br>0h (R/W) = Write 0 has no effect on RXFIFINT flag bit. Bit reads back a zero.<br>1h (R/W) = Write 1 to clear RXFFINT flag in bit 7                                                                                                                                                                                                                                                                  |

**Table 13-18. SCI FIFO Receive (SCIFFRX) Register Field Descriptions (continued)**

| Bit | Field    | Type | Reset | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|-----|----------|------|-------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 5   | RXFFIENA | R/W  | 0h    | Receive FIFO interrupt enable<br>Reset type: SYSRSn<br>0h (R/W) = RX FIFO interrupt is disabled<br>1h (R/W) = RX FIFO interrupt is enabled. This interrupt is triggered whenever the receive FIFO status (RXFFST) bits match (equal to or greater than) the interrupt trigger level bits RXFFIL (bits 4-0).                                                                                                                                                                                                                                                        |
| 4-0 | RXFFIL   | R/W  | 1Fh   | Receive FIFO interrupt level bits<br>The receive FIFO generates an interrupt whenever the FIFO status bits (RXFFST4-0) are greater than or equal to the FIFO level bits (RXFFIL4-0). The maximum value that can be assigned to these bits to generate an interrupt cannot be more than the depth of the RX FIFO. The default value of these bits after reset is 1111b. Users should set RXFFIL to best fit their application needs by weighing between the CPU overhead to service the ISR and the best possible usage of received SCI data.<br>Reset type: SYSRSn |



### 13.14.2.12 SCI FIFO Control (SCIFFCT) Register (Offset = Ch) [reset = 0h]

SCIFFCT contains the status of auto-baud detect, clears the auto-baud flag, and calibrate for A-detect bit.

**Figure 13-22. SCI FIFO Control (SCIFFCT) Register**

|         |        |        |          |    |    |   |   |
|---------|--------|--------|----------|----|----|---|---|
| 15      | 14     | 13     | 12       | 11 | 10 | 9 | 8 |
| ABD     | ABDCLR | CDC    | RESERVED |    |    |   |   |
| R-0h    | W-0h   | R/W-0h | R-0h     |    |    |   |   |
| 7       | 6      | 5      | 4        | 3  | 2  | 1 | 0 |
| FFTXDLY |        |        |          |    |    |   |   |
| R/W-0h  |        |        |          |    |    |   |   |

**Table 13-19. SCI FIFO Control (SCIFFCT) Register Field Descriptions**

| Bit  | Field    | Type | Reset | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
|------|----------|------|-------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15   | ABD      | R    | 0h    | Auto-baud detect (ABD) bit<br>Reset type: SYSRSn<br>0h (R/W) = Auto-baud detection is not complete. "A","a" character has not been received successfully.<br>1h (R/W) = Auto-baud hardware has detected "A" or "a" character on the SCI receive register. Auto-detect is complete.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| 14   | ABDCLR   | W    | 0h    | ABD-clear bit<br>Reset type: SYSRSn<br>0h (R/W) = Write 0 has no effect on ABD flag bit. Bit reads back a zero.<br>1h (R/W) = Write 1 to clear ABD flag in bit 15.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| 13   | CDC      | R/W  | 0h    | CDC calibrate A-detect bit<br>Reset type: SYSRSn<br>0h (R/W) = Disables auto-baud alignment<br>1h (R/W) = Enables auto-baud alignment                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| 12-8 | RESERVED | R    | 0h    | Reserved                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| 7-0  | FFTXDLY  | R/W  | 0h    | FIFO transfer delay. These bits define the delay between every transfer from FIFO transmit buffer to transmit shift register. The delay is defined in the number of SCI serial baud clock cycles. The 8 bit register could define a minimum delay of 0 baud clock cycles and a maximum of 256 baud clock cycles.<br>In FIFO mode, the buffer (TXBUF) between the shift register and the FIFO should be filled only after the shift register has completed shifting of the last bit. This is required to pass on the delay between transfers to the data stream. In FIFO mode, TXBUF should not be treated as one additional level of buffer. The delayed transmit feature will help to create an auto-flow scheme without RTS/CTS controls as in standard UARTS.<br>When SCI is configured for one stop-bit, delay introduced by FFTXDLY between one frame and the next frame is equal to number of baud clock cycles that FFTXDLY is set to.<br>When SCI is configured for two stop-bits, delay introduced by FFTXDLY between one frame and the next frame is equal to number of baud clock cycles that FFTXDLY is set to minus 1.<br>Reset type: SYSRSn |

### 13.14.2.13 SCI Priority Control (SCIPRI) Register (Offset = Fh) [reset = 0h]

SCIPRI determines what happens when an emulation suspend event occurs.

**Figure 13-23. SCI Priority Control (SCIPRI) Register**

|          |    |    |          |    |          |   |   |
|----------|----|----|----------|----|----------|---|---|
| 15       | 14 | 13 | 12       | 11 | 10       | 9 | 8 |
| RESERVED |    |    |          |    |          |   |   |
| R-0h     |    |    |          |    |          |   |   |
| 7        | 6  | 5  | 4        | 3  | 2        | 1 | 0 |
| RESERVED |    |    | FREESOFT |    | RESERVED |   |   |
| R-0h     |    |    | R/W-0h   |    | R-0h     |   |   |

**Table 13-20. SCI Priority Control (SCIPRI) Register Field Descriptions**

| Bit  | Field    | Type | Reset | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|------|----------|------|-------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-8 | RESERVED | R    | 0h    | Reserved                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| 7-5  | RESERVED | R    | 0h    | Reserved                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| 4-3  | FREESOFT | R/W  | 0h    | These bits determine what occurs when an emulation suspend event occurs (for example, when the debugger hits a break point). The peripheral can continue whatever it is doing (free-run mode), or if in stop mode, it can either stop immediately or stop when the current operation (the current receive/transmit sequence) is complete.<br>Reset type: SYSRSn<br>0h (R/W) = Immediate stop on suspend<br>1h (R/W) = Complete current receive/transmit sequence before stopping<br>2h (R/W) = Free run<br>3h (R/W) = Free run |
| 2-0  | RESERVED | R    | 0h    | Reserved                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |

This chapter describes the features and operation of the inter-integrated circuit (I2C) module. The I2C module provides an interface between one of these devices and devices compliant with the NXP Semiconductors Inter-IC bus (I2C bus) specification version 2.1, and connected by way of an I2C bus. External components attached to this 2-wire serial bus can transmit/receive 1- to 8-bit data to/from the device through the I2C module. This chapter assumes the reader is familiar with the I2C bus specification.

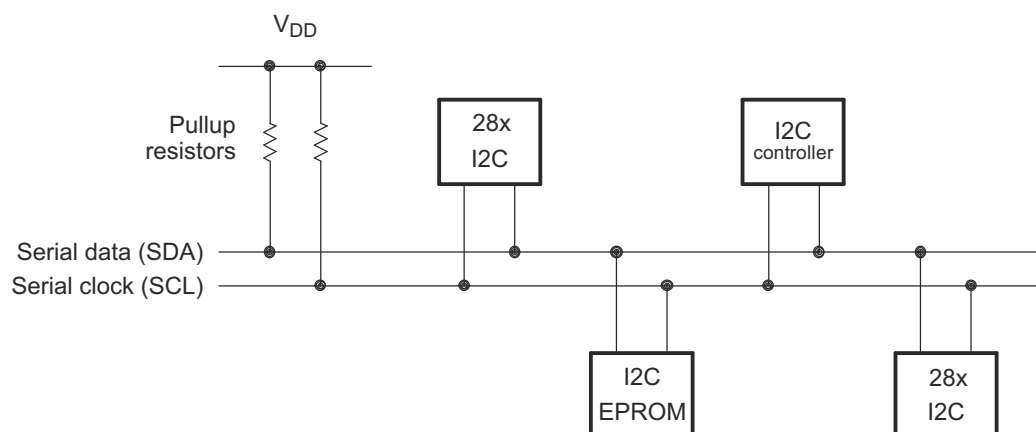
#### Note

A unit of data transmitted or received by the I2C module can have fewer than 8 bits; however, for convenience, a unit of data is called a data byte throughout this document. The number of bits in a data byte is selectable by way of the BC bits of the mode register, I2CMR.

|                                                                 |            |
|-----------------------------------------------------------------|------------|
| <b>14.1 Introduction.....</b>                                   | <b>834</b> |
| <b>14.2 Configuring Device Pins.....</b>                        | <b>839</b> |
| <b>14.3 I2C Module Operational Details.....</b>                 | <b>839</b> |
| <b>14.4 Interrupt Requests Generated by the I2C Module.....</b> | <b>851</b> |
| <b>14.5 Resetting or Disabling the I2C Module.....</b>          | <b>853</b> |
| <b>14.6 I2C Registers.....</b>                                  | <b>853</b> |

## 14.1 Introduction

The I2C module supports any slave or master I2C-compatible device. [Figure 14-1](#) shows an example of multiple I2C modules connected for a two-way transfer from one device to other devices.



**Figure 14-1. Multiple I2C Modules Connected**

### 14.1.1 I2C Related Collateral

#### Foundational Materials

- [I2C Hardware Overview](#) (Video)
- [I2C Protocol Overview](#) (Video)
- [Understanding the I2C Bus Application Report](#)

#### Getting Started Materials

- [Configuring the TMS320F280x DSP as an I2C Processor Application Report](#)
- [I2C Buffers Overview](#) (Video)
- [I2C Dynamic Addressing Application Report](#)
- [I2C translators overview](#) (Video)
- [Why, When, and How to use I2C Buffers Application Report](#)

#### Expert Materials

- [I2C Bus Pull-Up Resistor Calculation Application Report](#)
- [Maximum Clock Frequency of I2C Bus Using Repeaters Application Report](#)

### 14.1.2 Features

The I2C module has the following features:

- Compliance with the NXP Semiconductors I2C bus specification (version 2.1):
  - Support for 8-bit format transfers
  - 7-bit and 10-bit addressing modes
  - General call
  - START byte mode
  - Support for multiple master-transmitters and slave-receivers
  - Support for multiple slave-transmitters and master-receivers
  - Combined master transmit/receive and receive/transmit mode
  - Data transfer rate from 10 kbps up to 400 kbps (Fast-mode)
- Receive FIFO and Transmitter FIFO (4-deep x 8-bit FIFO)
- Supports two ePIE interrupts:
  - I2Cx Interrupt – Any of the following events can be configured to generate an I2Cx interrupt:
    - Transmit-data ready
    - Receive-data ready
    - Register-access ready
    - No-acknowledgment received
    - Arbitration lost
    - Stop condition detected
    - Addressed as slave
  - I2Cx\_FIFO interrupts:
    - Transmit FIFO interrupt
    - Receive FIFO interrupt
- Module enable and disable capability
- Free data format mode

### 14.1.3 Features Not Supported

The I2C module does not support:

- High-speed mode (Hs-mode)
- CBUS-compatibility mode

### 14.1.4 Functional Overview

Each device connected to an I2C bus is recognized by a unique address. Each device can operate as either a transmitter or a receiver, depending on the function of the device. A device connected to the I2C bus can also be considered as the master or the slave when performing data transfers. A master device is the device that initiates a data transfer on the bus and generates the clock signals to permit that transfer. During this transfer, any device addressed by this master is considered a slave. The I2C module supports the multi-master mode, in which one or more devices capable of controlling an I2C bus can be connected to the same I2C bus.

For data communication, the I2C module has a serial data pin (SDA) and a serial clock pin (SCL), as shown in [Figure 14-2](#). These two pins carry information between the C28x device and other devices connected to the I2C bus. The SDA and SCL pins both are bidirectional. They each must be connected to a positive supply voltage using a pull-up resistor. When the bus is free, both pins are high. The driver of these two pins has an open-drain configuration to perform the required wired-AND function.

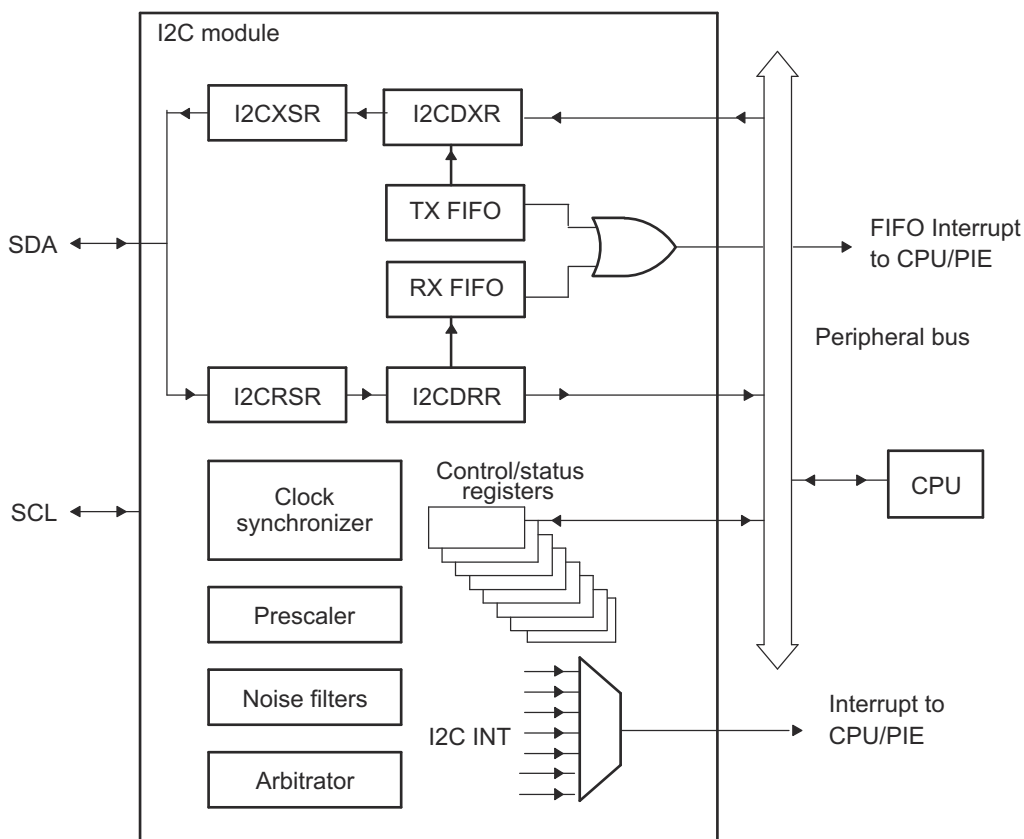
There are two major transfer techniques:

- Standard Mode: Send exactly n data values, where n is a value you program in an I2C module register. See the I2CCNT register in [Section 14.6](#) for more information.
- Repeat Mode: Keep sending data values until you use software to initiate a STOP condition or a new START condition. See the I2CMDR register in [Section 14.6](#) for RM bit information.

The I2C module consists of the following primary blocks:

- A serial interface: one data pin (SDA) and one clock pin (SCL)
- Data registers and FIFOs to temporarily hold receive data and transmit data traveling between the SDA pin and the CPU
- Control and status registers
- A peripheral bus interface to enable the CPU to access the I2C module registers and FIFOs.
- A clock synchronizer to synchronize the I2C input clock (from the device clock generator) and the clock on the SCL pin, and to synchronize data transfers with masters of different clock speeds
- A prescaler to divide down the input clock that is driven to the I2C module
- A noise filter on each of the two pins, SDA and SCL
- An arbitrator to handle arbitration between the I2C module (when it is a master) and another master
- Interrupt generation logic, so that an interrupt can be sent to the CPU
- FIFO interrupt generation logic, so that FIFO access can be synchronized to data reception and data transmission in the I2C module

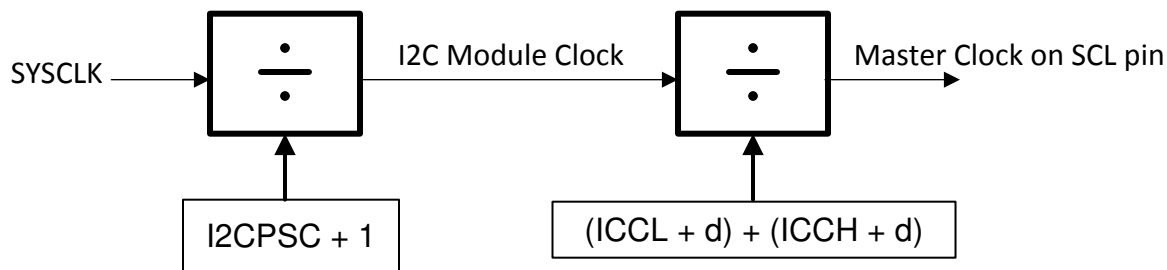
[Figure 14-2](#) shows the four registers used for transmission and reception in non-FIFO mode. The CPU writes data for transmission to I2CDXR and reads received data from I2CDRR. When the I2C module is configured as a transmitter, data written to I2CDXR is copied to I2CXSR and shifted out on the SDA pin one bit at a time. When the I2C module is configured as a receiver, received data is shifted into I2CRSR and then copied to I2CDRR.



**Figure 14-2. I2C Module Conceptual Block Diagram**

#### 14.1.5 Clock Generation

The I2C module clock determines the frequency at which the I2C module operates. A programmable prescaler in the I2C module divides down the SYSCLK to produce the I2C module clock and this I2C module clock is divided further to produce the I2C master clock on the SCL pin. [Figure 14-3](#) shows the clock generation diagram for I2C module.



**Figure 14-3. Clocking Diagram for the I2C Module**

#### Note

To meet all of the I2C protocol timing specifications, the I2C module clock must be between 7-12 MHz.

To specify the divide-down value, initialize the IPSC field of the prescaler register, I2CPSC. The resulting frequency is:

$$\text{I2C Module Clock (Fmod)} = \frac{\text{SYSCLK}}{(\text{I2CPSC} + 1)} \quad (8)$$

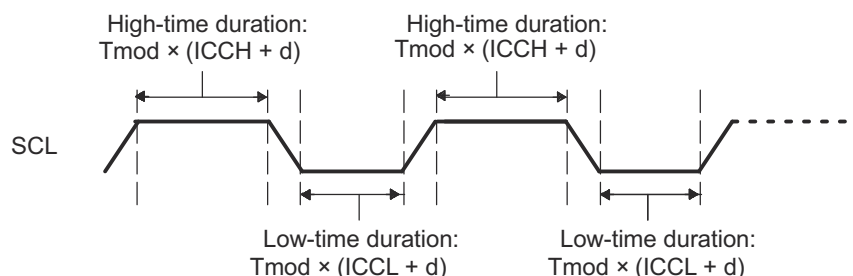
The prescaler must be initialized only while the I2C module is in the reset state (IRS = 0 in I2CMDR). The prescaled frequency takes effect only when IRS is changed to 1. Changing the IPSC value while IRS = 1 has no effect.

The master clock appears on the SCL pin when the I2C module is configured to be a master on the I2C bus. This clock controls the timing of communication between the I2C module and a slave. As shown in Figure 14-3, a second clock divider in the I2C module divides down the module clock to produce the master clock. The clock divider uses the ICCL value of I2CCLKL to divide down the low portion of the module clock signal and uses the ICCH value of I2CCLKH to divide down the high portion of the module clock signal. See Section 14.1.6 for the master clock frequency equation.

#### 14.1.6 I2C Clock Divider Registers (I2CCLKL and I2CCLKH)

As explained in Section 14.1.5, when the I2C module is a master, the I2C module clock is divided down further to use as the master clock on the SCL pin. As shown in Figure 14-4, the shape of the master clock depends on two divide-down values:

- ICCL in I2CCLKL. For each master clock cycle, ICCL determines the amount of time the signal is low.
- ICCH in I2CCLKH. For each master clock cycle, ICCH determines the amount of time the signal is high.



**Figure 14-4. Roles of the Clock Divide-Down Values (ICCL and ICCH)**

##### 14.1.6.1 Formula for the Master Clock Period

The master clock period (Tmst) is a multiple of the period of the I2C Module Clock (Tmod):

$$\text{Master Clock period (Tmst)} = \frac{[(\text{ICCH} + d) + (\text{ICCL} + d)]}{\text{I2C Module Clock (Fmod)}} \quad (9)$$

where d depends on the divide-down value IPSC, as shown in Table 14-1. IPSC is described in the I2CPSC register.

**Table 14-1. Dependency of Delay d on the Divide-Down Value IPSC**

| IPSC           | d |
|----------------|---|
| 0              | 7 |
| 1              | 6 |
| Greater than 1 | 5 |



## 14.2 Configuring Device Pins

The GPIO mux registers must be configured to connect this peripheral to the device pins.

Some IO functionality is defined by GPIO register settings independent of this peripheral. For input signals, the GPIO input qualification should be set to asynchronous mode by setting the appropriate GPxQSELn register bits to 11b. The internal pullups can be configured in the GPyPUD register.

See the *GPIO* chapter for more details on GPIO mux and settings.

## 14.3 I2C Module Operational Details

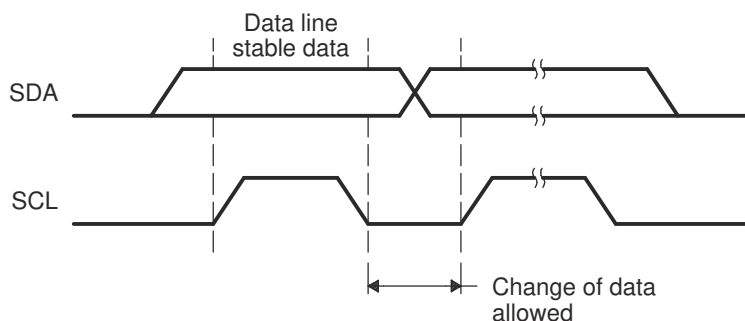
This section provides an overview of the I2C bus protocol and how it is implemented.

### 14.3.1 Input and Output Voltage Levels

One clock pulse is generated by the master device for each data bit transferred. Due to a variety of different technology devices that can be connected to the I2C bus, the levels of logic 0 (low) and logic 1 (high) are not fixed and depend on the associated level of  $V_{DD}$ . For details, see your device-specific data sheet.

### 14.3.2 Data Validity

The data on SDA must be stable during the high period of the clock (see [Figure 14-5](#)). The high or low state of the data line, SDA, should change only when the clock signal on SCL is low.



**Figure 14-5. Bit Transfer on the I2C bus**

### 14.3.3 Operating Modes

The I2C module has four basic operating modes to support data transfers as a master and as a slave. See [Table 14-2](#) for the names and descriptions of the modes.

If the I2C module is a master, it begins as a master-transmitter and typically transmits an address for a particular slave. When giving data to the slave, the I2C module must remain a master-transmitter. To receive data from a slave, the I2C module must be changed to the master-receiver mode.

If the I2C module is a slave, it begins as a slave-receiver and typically sends acknowledgment when it recognizes its slave address from a master. If the master will be sending data to the I2C module, the module must remain a slave-receiver. If the master has requested data from the I2C module, the module must be changed to the slave-transmitter mode.

**Table 14-2. Operating Modes of the I2C Module**

| Operating Mode          | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|-------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Slave-receiver mode     | <p>The I2C module is a slave and receives data from a master.</p> <p>All slaves begin in this mode. In this mode, serial data bits received on SDA are shifted in with the clock pulses that are generated by the master. As a slave, the I2C module does not generate the clock signal, but it can hold SCL low while the intervention of the device is required (RSFULL = 1 in I2CSTR) after a byte has been received. See <a href="#">Section 14.3.7</a> for more details.</p>                                                                                                                                                                                                                                                                                                                                                               |
| Slave-transmitter mode  | <p>The I2C module is a slave and transmits data to a master.</p> <p>This mode can be entered only from the slave-receiver mode; the I2C module must first receive a command from the master. When you are using any of the 7-bit/10-bit addressing formats, the I2C module enters its slave-transmitter mode if the slave address byte is the same as its own address (in I2COAR) and the master has transmitted R/ <math>\bar{W}</math> = 1. As a slave-transmitter, the I2C module then shifts the serial data out on SDA with the clock pulses that are generated by the master. While a slave, the I2C module does not generate the clock signal, but it can hold SCL low while the intervention of the device is required (XSMT = 0 in I2CSTR) after a byte has been transmitted. See <a href="#">Section 14.3.7</a> for more details.</p> |
| Master-receiver mode    | <p>The I2C module is a master and receives data from a slave.</p> <p>This mode can be entered only from the master-transmitter mode; the I2C module must first transmit a command to the slave. When you are using any of the 7-bit/10-bit addressing formats, the I2C module enters its master-receiver mode after transmitting the slave address byte and R/ <math>\bar{W}</math> = 1. Serial data bits on SDA are shifted into the I2C module with the clock pulses generated by the I2C module on SCL. The clock pulses are inhibited and SCL is held low when the intervention of the device is required (RSFULL = 1 in I2CSTR) after a byte has been received.</p>                                                                                                                                                                        |
| Master-transmitter mode | <p>The I2C module is a master and transmits control information and data to a slave.</p> <p>All masters begin in this mode. In this mode, data assembled in any of the 7-bit/10-bit addressing formats is shifted out on SDA. The bit shifting is synchronized with the clock pulses generated by the I2C module on SCL. The clock pulses are inhibited and SCL is held low when the intervention of the device is required (XSMT = 0 in I2CSTR) after a byte has been transmitted.</p>                                                                                                                                                                                                                                                                                                                                                         |

To summarize, SCL will be held low in the following conditions:

- When an overrun condition is detected (RSFULL = 1), in Slave-receiver mode.
- When an underflow condition is detected (XSMT = 0), in Slave-transmitter mode.

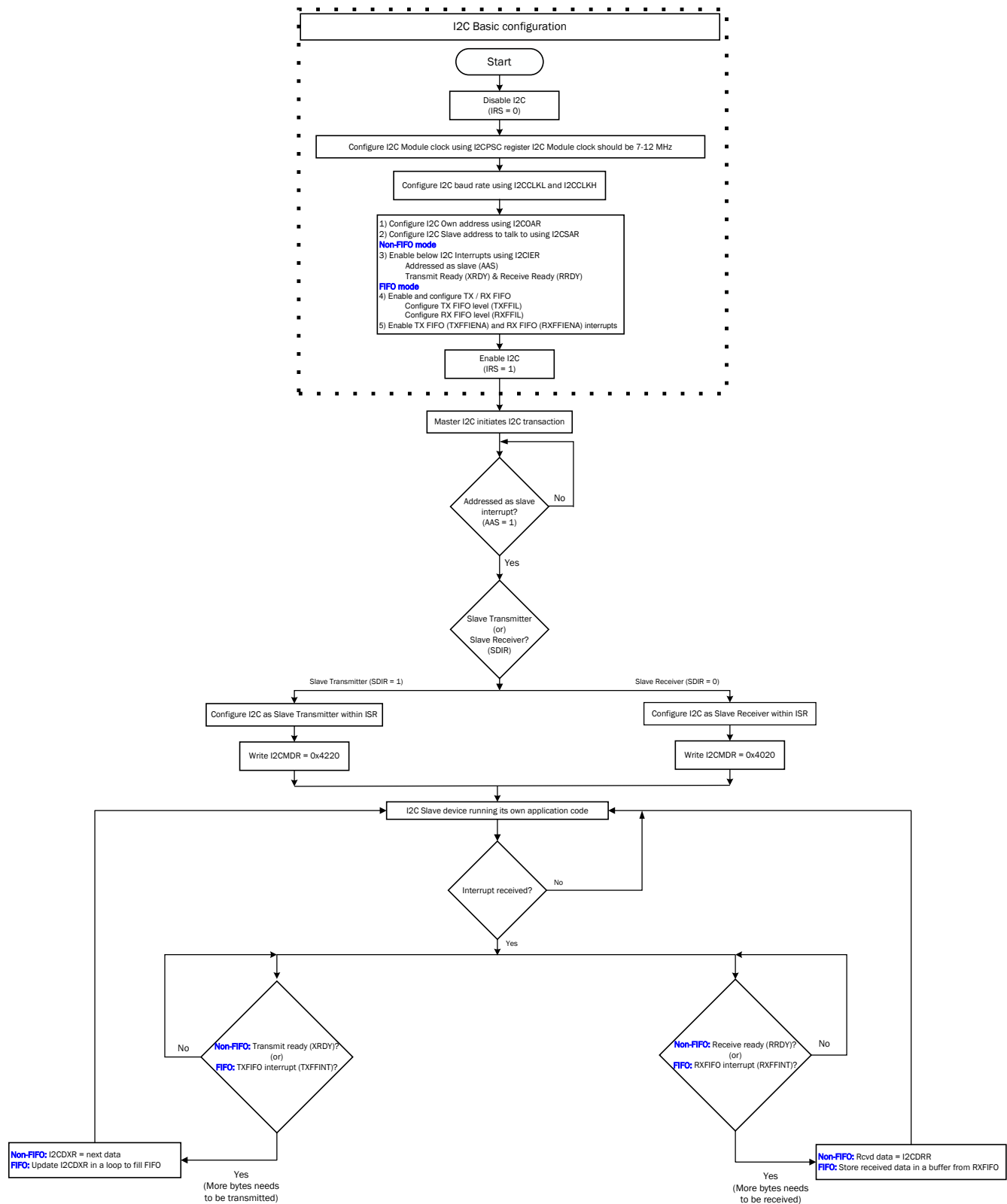
I2C slave nodes have to accept and provide data when the I2C master node requests it.

- To release SCL in slave-receiver mode, read data from I2CDRR.
- To release SCL in slave-transmitter mode, write data to I2CDXR.
- To force a release without handling the data, reset the module using the I2CMDR.IRS bit.

**Table 14-3. Master-Transmitter/Receiver Bus Activity Defined by the RM, STT, and STP Bits of I2CMDR**

| RM | STT | STP | Bus Activity <sup>(1)</sup> | Description                                                                                                                  |
|----|-----|-----|-----------------------------|------------------------------------------------------------------------------------------------------------------------------|
| 0  | 0   | 0   | None                        | No activity                                                                                                                  |
| 0  | 0   | 1   | P                           | STOP condition                                                                                                               |
| 0  | 1   | 0   | S-A-D..(n)..D.              | START condition, slave address, n data bytes (n = value in I2CCNT)                                                           |
| 0  | 1   | 1   | S-A-D..(n)..D-P             | START condition, slave address, n data bytes, STOP condition (n = value in I2CCNT)                                           |
| 1  | 0   | 0   | None                        | No activity                                                                                                                  |
| 1  | 0   | 1   | P                           | STOP condition                                                                                                               |
| 1  | 1   | 0   | S-A-D-D-D.                  | Repeat mode transfer: START condition, slave address, continuous data transfers until STOP condition or next START condition |
| 1  | 1   | 1   | None                        | Reserved bit combination (No activity)                                                                                       |

(1) S = START condition; A = Address; D = Data byte; P = STOP condition;



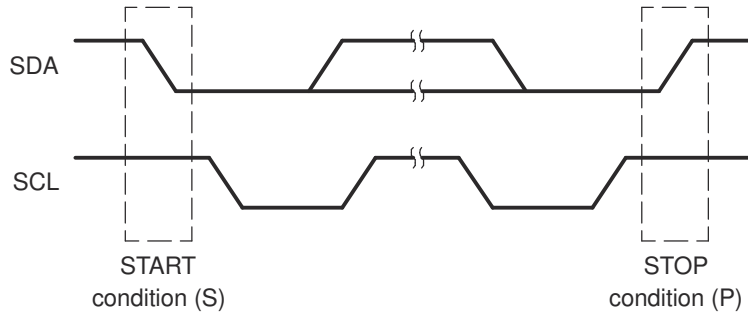
**Figure 14-6. I2C Slave TX / RX Flowchart**



#### 14.3.4 I2C Module START and STOP Conditions

START and STOP conditions can be generated by the I2C module when the module is configured to be a master on the I2C bus. As shown in [Figure 14-8](#):

- The START condition is defined as a high-to-low transition on the SDA line while SCL is high. A master drives this condition to indicate the start of a data transfer.
- The STOP condition is defined as a low-to-high transition on the SDA line while SCL is high. A master drives this condition to indicate the end of a data transfer.



**Figure 14-8. I2C Module START and STOP Conditions**

After a START condition and before a subsequent STOP condition, the I2C bus is considered busy, and the bus busy (BB) bit of I2CSTR is 1. Between a STOP condition and the next START condition, the bus is considered free, and BB is 0.

For the I2C module to start a data transfer with a START condition, the master mode bit (MST) and the START condition bit (STT) in I2CMDR must both be 1. For the I2C module to end a data transfer with a STOP condition, the STOP condition bit (STP) must be set to 1. When the BB bit is set to 1 and the STT bit is set to 1, a repeated START condition is generated. For a description of I2CMDR and its bits (including MST, STT, and STP), see [Section 14.6](#).

The I2C peripheral cannot detect a START or STOP condition while it is in reset (IRS = 0). The BB bit will remain in the cleared state (BB = 0) while the I2C peripheral is in reset (IRS = 0). When the I2C peripheral is taken out of reset (IRS set to 1) the BB bit will not correctly reflect the I2C bus status until a START or STOP condition is detected.

Follow these steps before initiating the first data transfer with I2C:

1. After taking the I2C peripheral out of reset by setting the IRS bit to 1, wait a period larger than the total time taken for the longest data transfer in the application. By waiting for a period of time after I2C comes out of reset, users can ensure that at least one START or STOP condition will have occurred on the I2C bus and been captured by the BB bit. After this period, the BB bit will correctly reflect the state of the I2C bus.
2. Check the BB bit and verify that BB = 0 (bus not busy) before proceeding.
3. Begin data transfers.

Not resetting the I2C peripheral in between transfers ensures that the BB bit reflects the actual bus status. If users must reset the I2C peripheral in between transfers, repeat steps 1 through 3 every time the I2C peripheral is taken out of reset.

### 14.3.5 Non-repeat Mode versus Repeat Mode

#### Non-repeat mode:

- When I2CMR.RM = 0, I2C module is configured in non-repeat mode.
- I2CCNT register determines the number of bytes to be transmitted (or) received.
- If STP = 0 in I2CMR, the ARDY bit is set when the internal data counter counts down to 0.
- If STP = 1, ARDY bit doesn't get set and I2C module generates a STOP condition when the internal data counter counts down to 0.

---

**Note**

In non-repeat mode (RM = 0), if I2CCNT is set to 0, I2C state machine expects to transmit (or) receive 65536 bytes and not 0 bytes.

---

#### Repeat mode:

- When I2CMR.RM = 1, I2C module is configured in repeat mode.
- I2CCNT register contents don't determine the number of bytes to be transmitted (or) received.
- Number of bytes to be transmitted (or) received can be controlled by software.
- ARDY bit gets set at end of transmission and reception of each byte.

---

**Note**

Once you start I2C transaction in non-repeat mode (or) repeat mode, you cannot switch into another mode until the I2C transaction is completed with a STOP condition.

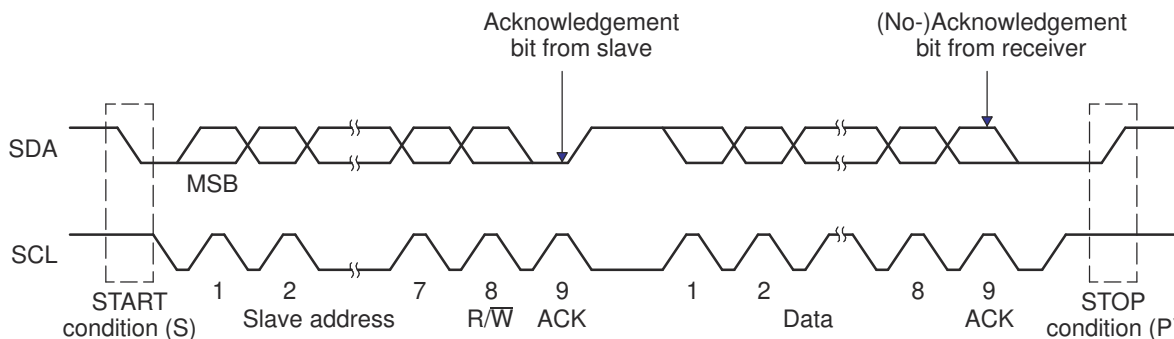
---

### 14.3.6 Serial Data Formats

Figure 14-9 shows an example of a data transfer on the I2C bus. The I2C module supports 1 to 8-bit data values. In Figure 14-9, 8-bit data is transferred. Each bit put on the SDA line equates to 1 pulse on the SCL line, and the values are always transferred with the most significant bit (MSB) first. The number of data values that can be transmitted or received is unrestricted. The serial data format used in Figure 14-9 is the 7-bit addressing format. The I2C module supports the formats shown in Figure 14-10 through Figure 14-12 and described in the paragraphs that follow the figures.

#### Note

In Figure 14-9 through Figure 14-12,  $n$  = the number of data bits (from 1 to 8) specified by the bit count (BC) field of I2CMDR.



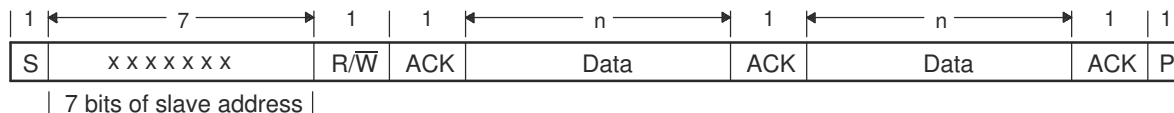
**Figure 14-9. I2C Module Data Transfer (7-Bit Addressing with 8-bit Data Configuration Shown)**

#### 14.3.6.1 7-Bit Addressing Format

The 7-bit addressing format is the default format after reset. Disabling expanded address (I2CMDR.XA = 0) and free data format (I2CMDR.FDF = 0) enables 7-bit addressing format.

In this format (see Figure 14-10), the first byte after a START condition (S) consists of a 7-bit slave address followed by a R/  $\overline{W}$  bit. R/  $\overline{W}$  determines the direction of the data:

- R/  $\overline{W}$  = 0: The I2C master writes (transmits) data to the addressed slave. This can be achieved by setting I2CMDR.TRX = 1 (Transmitter mode)
- R/  $\overline{W}$  = 1: The I2C master reads (receives) data from the slave. This can be achieved by setting I2CMDR.TRX = 0 (Receiver mode)



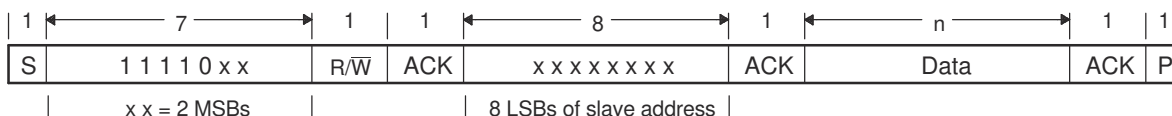
**Figure 14-10. I2C Module 7-Bit Addressing Format (FDF = 0, XA = 0 in I2CMDR)**

An extra clock cycle dedicated for acknowledgment (ACK) is inserted after each byte. If the ACK bit is inserted by the slave after the first byte from the master, it is followed by  $n$  bits of data from the transmitter (master or slave, depending on the R/  $\overline{W}$  bit).  $n$  is a number from 1 to 8 determined by the bit count (BC) field of I2CMDR. After the data bits have been transferred, the receiver inserts an ACK bit.

### 14.3.6.2 10-Bit Addressing Format

The 10-bit addressing format can be enabled by setting expanded address (I2CMDR.XA = 1) and disabling free data format (I2CMDR.FDF = 0).

The 10-bit addressing format (see Figure 14-11) is similar to the 7-bit addressing format, but the master sends the slave address in two separate byte transfers. The first byte consists of 11110b, the two MSBs of the 10-bit slave address, and R/  $\overline{W}$ . The second byte is the remaining 8 bits of the 10-bit slave address. The slave must send acknowledgment after each of the two byte transfers. Once the master has written the second byte to the slave, the master can either write data or use a repeated START condition to change the data direction. For more details about using 10-bit addressing, see the NXP Semiconductors I2C bus specification.

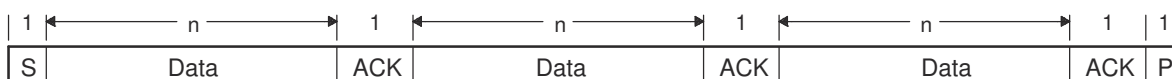


**Figure 14-11. I2C Module 10-Bit Addressing Format (FDF = 0, XA = 1 in I2CMDR)**

### 14.3.6.3 Free Data Format

The free data format can be enabled by setting I2CMDR.FDF = 1.

In this format (see Figure 14-12), the first byte after a START condition (S) is a data byte. An ACK bit is inserted after each data byte, which can be from 1 to 8 bits, depending on the BC field of I2CMDR. No address or data-direction bit is sent. Therefore, the transmitter and the receiver must both support the free data format, and the direction of the data must be constant throughout the transfer.



**Figure 14-12. I2C Module Free Data Format (FDF = 1 in I2CMDR)**

#### Note

The free data format is not supported in the digital loopback mode (I2CMDR.DLB = 1).

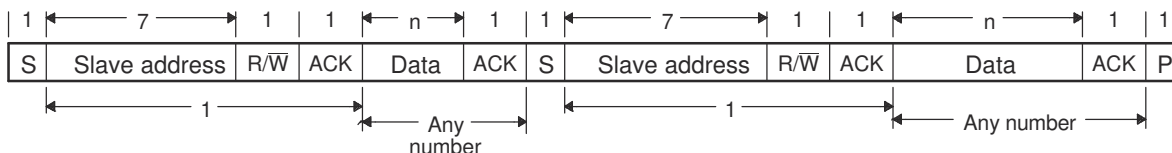
**Table 14-4. How the MST and FDF Bits of I2CMDR Affect the Role of the TRX Bit of I2CMDR**

| MST | FDF | I2C Module State                             | Function of TRX                                                                                                                                                                                                                                               |
|-----|-----|----------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 0   | 0   | In slave mode but not free data format mode  | TRX is a don't care. Depending on the command from the master, the I2C module responds as a receiver or a transmitter.                                                                                                                                        |
| 0   | 1   | In slave mode and free data format mode      | The free data format mode requires that the I2C module remains the transmitter or the receiver throughout the transfer. TRX identifies the role of the I2C module:<br><br>TRX = 1: The I2C module is a transmitter.<br>TRX = 0: The I2C module is a receiver. |
| 1   | 0   | In master mode but not free data format mode | TRX = 1: The I2C module is a transmitter.<br>TRX = 0: The I2C module is a receiver.                                                                                                                                                                           |
| 1   | 1   | In master mode and free data format mode     | TRX = 0: The I2C module is a receiver.<br>TRX = 1: The I2C module is a transmitter.                                                                                                                                                                           |



#### 14.3.6.4 Using a Repeated START Condition

I2C master can communicate with multiple slave addresses without having to give up control of the I2C bus by driving a STOP condition. This can be achieved by driving another START condition at the end of each data type. The repeated START condition can be used with the 7-bit addressing, 10-bit addressing, and free data formats. Figure 14-13 shows a repeated START condition in the 7-bit addressing format.



**Figure 14-13. Repeated START Condition (in This Case, 7-Bit Addressing Format)**

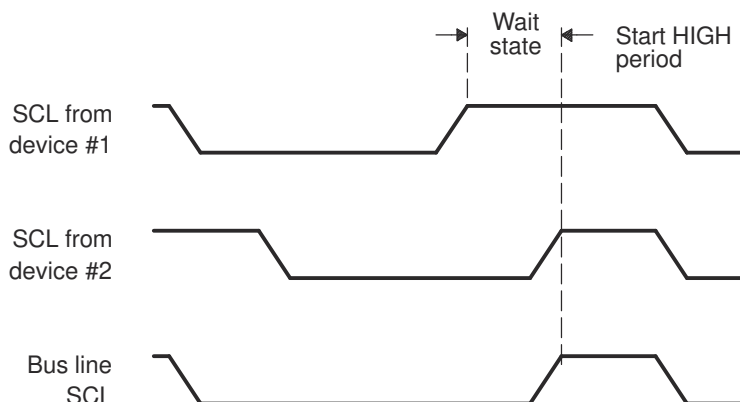
#### Note

In Figure 14-13, n = the number of data bits (from 1 to 8) specified by the bit count (BC) field of I2CMDR.

#### 14.3.7 Clock Synchronization

Under normal conditions, only one master device generates the clock signal, SCL. During the arbitration procedure, however, there are two or more masters and the clock must be synchronized so that the data output can be compared. Figure 14-14 illustrates the clock synchronization. The wired-AND property of SCL means that a device that first generates a low period on SCL overrules the other devices. At this high-to-low transition, the clock generators of the other devices are forced to start their own low period. The SCL is held low by the device with the longest low period. The other devices that finish their low periods must wait for SCL to be released, before starting their high periods. A synchronized signal on SCL is obtained, where the slowest device determines the length of the low period and the fastest device determines the length of the high period.

If a device pulls down the clock line for a longer time, the result is that all clock generators must enter the wait state. In this way, a slave slows down a fast master and the slow device creates enough time to store a received byte or to prepare a byte to be transmitted.



**Figure 14-14. Synchronization of Two I2C Clock Generators During Arbitration**

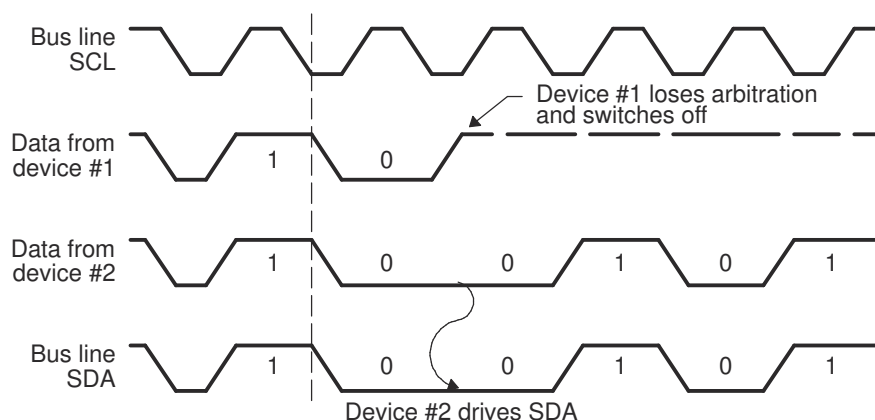
### 14.3.8 Arbitration

If two or more master-transmitters attempt to start a transmission on the same bus at approximately the same time, an arbitration procedure is invoked. The arbitration procedure uses the data presented on the serial data bus (SDA) by the competing transmitters. Figure 14-15 illustrates the arbitration procedure between two devices. The first master-transmitter that releases the SDA line high is overruled by another master-transmitter that drives the SDA low. The arbitration procedure gives priority to the device that transmits the serial data stream with the lowest binary value. Should two or more devices send identical first bytes, arbitration continues on the subsequent bytes.

If the I2C module is the losing master, it switches to the slave-receiver mode, sets the arbitration lost (ARBL) flag, and generates the arbitration-lost interrupt request.

If during a serial transfer the arbitration procedure is still in progress when a repeated START condition or a STOP condition is transmitted to SDA, the master-transmitters involved must send the repeated START condition or the STOP condition at the same position in the format frame. Arbitration is not allowed between:

- A repeated START condition and a data bit
- A STOP condition and a data bit
- A repeated START condition and a STOP condition



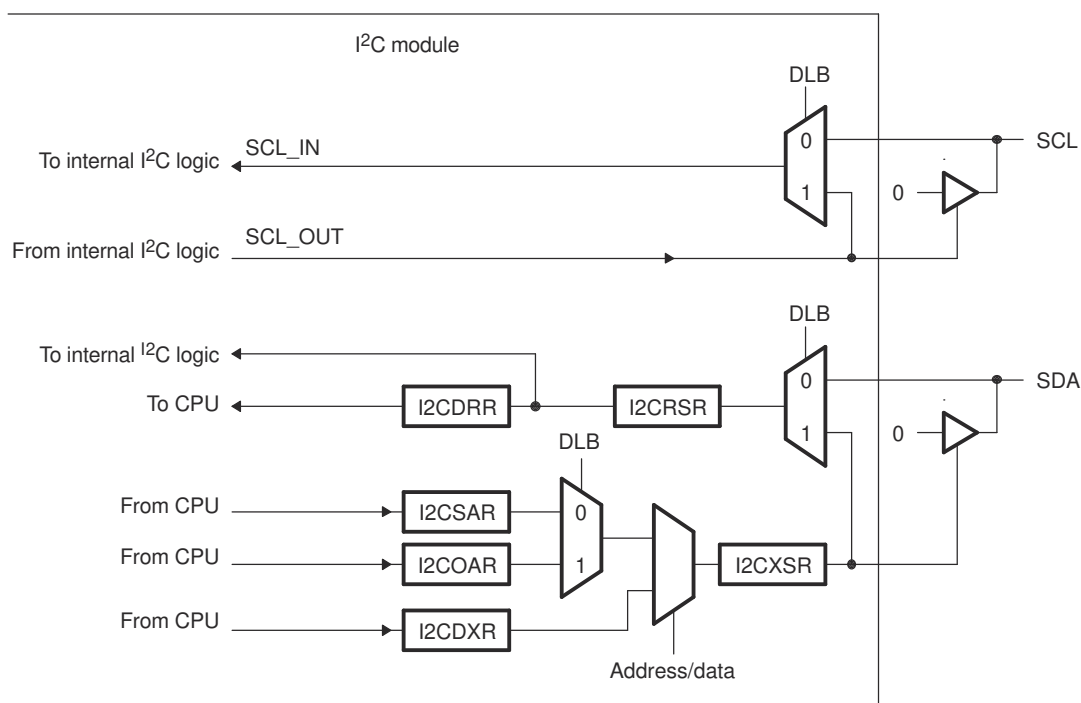
**Figure 14-15. Arbitration Procedure Between Two Master-Transmitters**

### 14.3.9 Digital Loopback Mode

The I2C module support a self-test mode called digital loopback, which is enabled by setting the DLB bit in the I2CMDR register. In this mode, data transmitted out of the I2CDXR register is received in the I2CDRR register. The data follows an internal path, and takes  $n$  cycles to reach I2CDRR, where:

$$n = 8 * (\text{SYSCLK}) / (\text{I2C module clock (Fmod)})$$

The transmit clock and the receive clock are the same. The address seen on the external SDA pin is the address in the I2COAR register. Figure 14-16 shows the signal routing in digital loopback mode.



**Figure 14-16. Pin Diagram Showing the Effects of the Digital Loopback Mode (DLB) Bit**

#### Note

The free data format (I2CMDR.FDF = 1) is not supported in digital loopback mode.

### 14.3.10 NACK Bit Generation

When the I2C module is a receiver (master or slave), it can acknowledge or ignore bits sent by the transmitter. To ignore any new bits, the I2C module must send a no-acknowledge (NACK) bit during the acknowledge cycle on the bus. [Table 14-5](#) summarizes the various ways you can allow the I2C module to send a NACK bit.

**Table 14-5. Ways to Generate a NACK Bit**

| I2C Module Condition                                       | NACK Bit Generation Options                                                                                                                                                                                                                                                                                                                    |
|------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Slave-receiver modes                                       | Allow an overrun condition (RSFULL = 1 in I2CSTR)<br>Reset the module (IRS = 0 in I2CMDR)<br>Set the NACKMOD bit of I2CMDR before the rising edge of the last data bit you intend to receive                                                                                                                                                   |
| Master-receiver mode AND Repeat mode (RM = 1 in I2CMDR)    | Generate a STOP condition (STP = 1 in I2CMDR)<br>Reset the module (IRS = 0 in I2CMDR)<br>Set the NACKMOD bit of I2CMDR before the rising edge of the last data bit you intend to receive                                                                                                                                                       |
| Master-receiver mode AND Nonrepeat mode (RM = 0 in I2CMDR) | If STP = 1 in I2CMDR, allow the internal data counter to count down to 0 and thus force a STOP condition<br>If STP = 0, make STP = 1 to generate a STOP condition<br>Reset the module (IRS = 0 in I2CMDR). = 1 to generate a STOP condition<br>Set the NACKMOD bit of I2CMDR before the rising edge of the last data bit you intend to receive |

## 14.4 Interrupt Requests Generated by the I2C Module

Each I2C module can generate two CPU interrupts.

1. Basic I2C interrupt: Possible basic I2C interrupt sources that can trigger this interrupt are described in [Section 14.4.1](#).
2. I2C FIFO interrupt: Possible I2C FIFO interrupt sources that can trigger this interrupt are described in [Section 14.4.2](#)

### 14.4.1 Basic I2C Interrupt Requests

The I2C module generates the interrupt requests described in [Table 14-6](#). As shown in [Figure 14-17](#), all requests are multiplexed through an arbiter to a single I2C interrupt request to the CPU. Each interrupt request has a flag bit in the status register (I2CSTR) and an enable bit in the interrupt enable register (I2CIER). When one of the specified events occurs, its flag bit is set. If the corresponding enable bit is 0, the interrupt request is blocked. If the enable bit is 1, the request is forwarded to the CPU as an I2C interrupt.

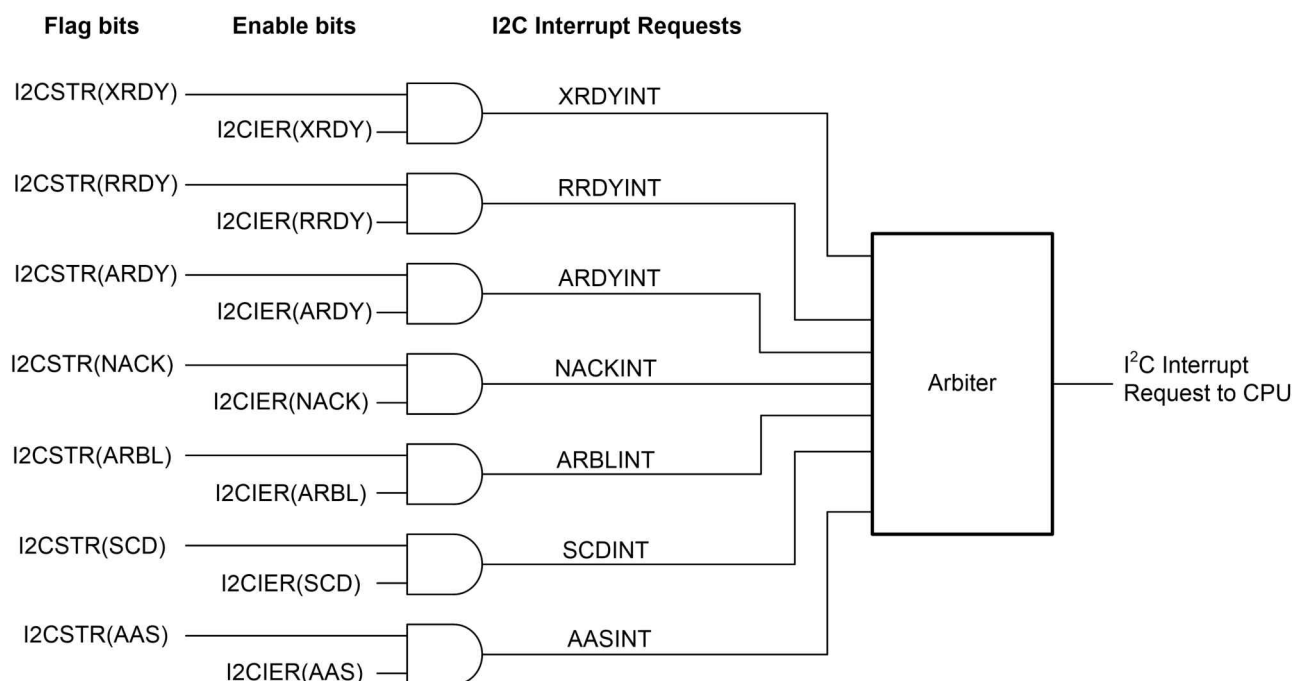
The I2C interrupt is one of the maskable interrupts of the CPU. As with any maskable interrupt request, if it is properly enabled in the CPU, the CPU executes the corresponding interrupt service routine (I2CINT1A\_ISR). The I2CINT1A\_ISR for the I2C interrupt can determine the interrupt source by reading the interrupt source register, I2CISRC. Then the I2CINT1A\_ISR can branch to the appropriate subroutine.

After the CPU reads I2CISRC, the following events occur:

1. The flag for the source interrupt is cleared in I2CSTR. Exception: The ARDY, RRDY, and XRDY bits in I2CSTR are not cleared when I2CISRC is read. To clear one of these bits, write a 1 to it.
2. The arbiter determines which of the remaining interrupt requests has the highest priority, writes the code for that interrupt to I2CISRC, and forwards the interrupt request to the CPU.

**Table 14-6. Descriptions of the Basic I2C Interrupt Requests**

| I2C Interrupt Request | Interrupt Source                                                                                                                                                                                                                                                                                                                                                             |
|-----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| XRDYINT               | Transmit ready condition: The data transmit register (I2CDXR) is ready to accept new data because the previous data has been copied from I2CDXR to the transmit shift register (I2CXHR).<br>As an alternative to using XRDYINT, the CPU can poll the XRDY bit of the status register, I2CSTR. XRDYINT should not be used when in FIFO mode. Use the FIFO interrupts instead. |
| RRDYINT               | Receive ready condition: The data receive register (I2CDRR) is ready to be read because data has been copied from the receive shift register (I2CRSR) to I2CDRR.<br>As an alternative to using RRDYINT, the CPU can poll the RRDY bit of I2CSTR. RRDYINT should not be used when in FIFO mode. Use the FIFO interrupts instead.                                              |
| ARDYINT               | Register-access ready condition: The I2C module registers are ready to be accessed because the previously programmed address, data, and command values have been used.<br>The specific events that generate ARDYINT are the same events that set the ARDY bit of I2CSTR.<br>As an alternative to using ARDYINT, the CPU can poll the ARDY bit.                               |
| NACKINT               | No-acknowledgment condition: The I2C module is configured as a master-transmitter and did not received acknowledgment from the slave-receiver.<br>As an alternative to using NACKINT, the CPU can poll the NACK bit of I2CSTR.                                                                                                                                               |
| ARBLINT               | Arbitration-lost condition: The I2C module has lost an arbitration contest with another master-transmitter.<br>As an alternative to using ARBLINT, the CPU can poll the ARBL bit of I2CSTR.                                                                                                                                                                                  |
| SCDINT                | Stop condition detected: A STOP condition was detected on the I2C bus.<br>As an alternative to using SCDINT, the CPU can poll the SCD bit of the status register, I2CSTR.                                                                                                                                                                                                    |
| AASINT                | Addressed as slave condition: The I2C has been addressed as a slave device by another master on the I2C bus.<br>As an alternative to using AASINT, the CPU can poll the AAS bit of the status register, I2CSTR.                                                                                                                                                              |



**Figure 14-17. Enable Paths of the I2C Interrupt Requests**

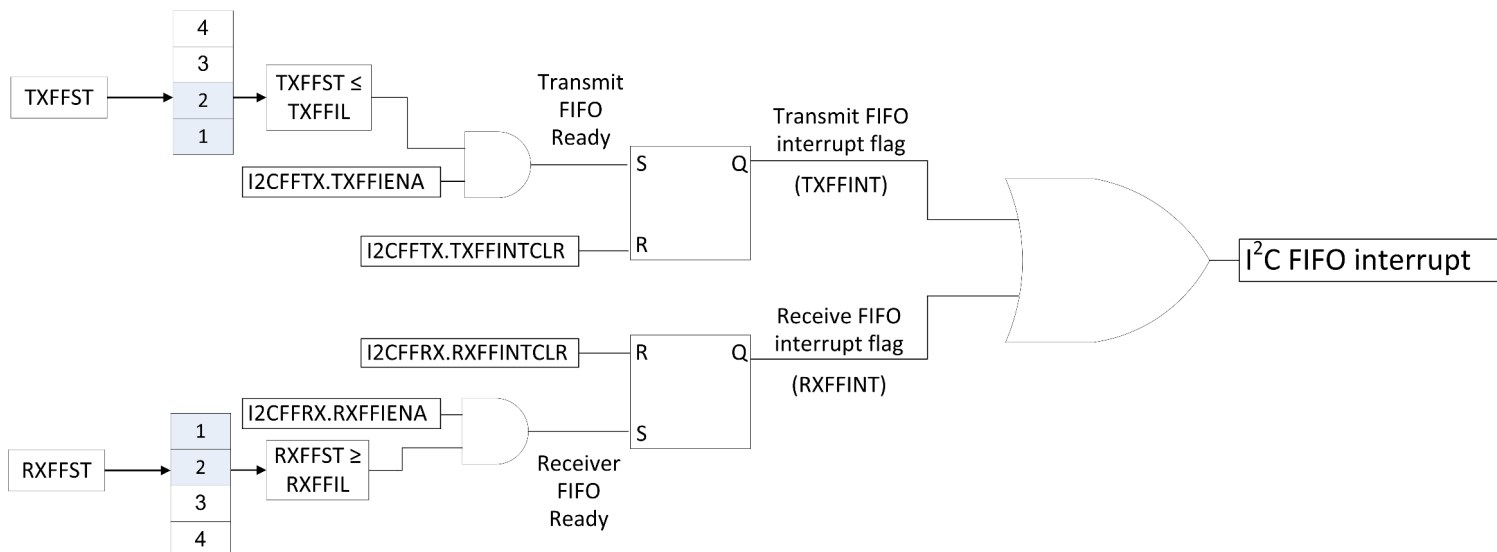
The priorities of the basic I2C interrupt requests are listed in order of highest priority to lowest priority:

1. ARBLINT
2. NACKINT
3. ARDYINT
4. RRDYINT
5. XRDYINT
6. SCDINT
7. AASINT

The I2C module has a backwards compatibility bit (BC) in the I2CEMDR register. The timing diagram in demonstrates the effect the backwards compatibility bit has on I2C module registers and interrupts when configured as a slave-transmitter.

## 14.4.2 I2C FIFO Interrupts

In addition to the seven basic I2C interrupts, the transmit and receive FIFOs each contain the ability to generate an interrupt (I2CINT2A). The transmit FIFO can be configured to generate an interrupt after transmitting a defined number of bytes, up to 4. The receive FIFO can be configured to generate an interrupt after receiving a defined number of bytes, up to 4. These two interrupt sources are ORed together into a single maskable CPU interrupt. Figure 14-18 shows the structure of I2C FIFO interrupt. The interrupt service routine can then read the FIFO interrupt status flags to determine from which source the interrupt came. See the I2C transmit FIFO register (I2CFFTX) and the I2C receive FIFO register (I2CFFRX) descriptions.



**Figure 14-18. I2C FIFO Interrupt**

## 14.5 Resetting or Disabling the I2C Module

You can reset or disable the I2C module in two ways:

- Write 0 to the I2C reset bit (IRS) in the I2C mode register (I2CMDR). All status bits (in I2CSTR) are forced to their default values, and the I2C module remains disabled until IRS is changed to 1. The SDA and SCL pins are in the high-impedance state.
- Initiate a device reset by driving the  $\overline{\text{XRS}}$  pin low. The entire device is reset and is held in the reset state until you drive the pin high. When the  $\overline{\text{XRS}}$  pin is released, all I2C module registers are reset to their default values. The IRS bit is forced to 0, which resets the I2C module. The I2C module stays in the reset state until you write 1 to IRS.

The IRS must be 0 while you configure or reconfigure the I2C module. Forcing IRS to 0 can be used to save power and to clear error conditions.

## 14.6 I2C Registers

This section describes the C28x I2C Module Registers.

### 14.6.1 I2C Base Address Table

**Table 14-7. I2C Base Address Table**

| Bit Field Name |           | Base Address |
|----------------|-----------|--------------|
| Instance       | Structure |              |
| I2caRegs       | I2C_REGS  | 0x0000_7900  |

## 14.6.2 I2C\_REGS Registers

Table 14-8 lists the I2C\_REGS registers. All register offset addresses not listed in Table 14-8 should be considered as reserved locations and the register contents should not be modified.

**Table 14-8. I2C\_REGS Registers**

| Offset | Acronym | Register Name               | Write Protection | Section                           |
|--------|---------|-----------------------------|------------------|-----------------------------------|
| 0h     | I2COAR  | I2C Own Address             |                  | <a href="#">Section 14.6.2.1</a>  |
| 1h     | I2CIER  | I2C Interrupt Enable        |                  | <a href="#">Section 14.6.2.2</a>  |
| 2h     | I2CSTR  | I2C Status                  |                  | <a href="#">Section 14.6.2.3</a>  |
| 3h     | I2CCLKL | I2C Clock Low-time Divider  |                  | <a href="#">Section 14.6.2.4</a>  |
| 4h     | I2CCLKH | I2C Clock High-time Divider |                  | <a href="#">Section 14.6.2.5</a>  |
| 5h     | I2CCNT  | I2C Data Count              |                  | <a href="#">Section 14.6.2.6</a>  |
| 6h     | I2CDRR  | I2C Data Receive            |                  | <a href="#">Section 14.6.2.7</a>  |
| 7h     | I2CSAR  | I2C Slave Address           |                  | <a href="#">Section 14.6.2.8</a>  |
| 8h     | I2CDXR  | I2C Data Transmit           |                  | <a href="#">Section 14.6.2.9</a>  |
| 9h     | I2CMDR  | I2C Mode                    |                  | <a href="#">Section 14.6.2.10</a> |
| Ah     | I2CISRC | I2C Interrupt Source        |                  | <a href="#">Section 14.6.2.11</a> |
| Bh     | I2CEMDR | I2C Extended Mode           |                  | <a href="#">Section 14.6.2.12</a> |
| Ch     | I2CPSC  | I2C Prescaler               |                  | <a href="#">Section 14.6.2.13</a> |
| 20h    | I2CFFTX | I2C FIFO Transmit           |                  | <a href="#">Section 14.6.2.14</a> |
| 21h    | I2CFFRX | I2C FIFO Receive            |                  | <a href="#">Section 14.6.2.15</a> |



Complex bit access types are encoded to fit into small table cells. [Table 14-9](#) shows the codes that are used for access types in this section.

**Table 14-9. I2C\_REGS Access Type Codes**

| Access Type              | Code | Description                                                                                                                                                                                                                                                                          |
|--------------------------|------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Read Type                |      |                                                                                                                                                                                                                                                                                      |
| R                        | R    | Read                                                                                                                                                                                                                                                                                 |
| R-0                      | R-0  | Read<br>Returns 0s                                                                                                                                                                                                                                                                   |
| Write Type               |      |                                                                                                                                                                                                                                                                                      |
| W                        | W    | Write                                                                                                                                                                                                                                                                                |
| W1C                      | W1C  | Write<br>1 to clear                                                                                                                                                                                                                                                                  |
| W1S                      | W1S  | Write<br>1 to set                                                                                                                                                                                                                                                                    |
| Reset or Default Value   |      |                                                                                                                                                                                                                                                                                      |
| -n                       |      | Value after reset or the default value                                                                                                                                                                                                                                               |
| Register Array Variables |      |                                                                                                                                                                                                                                                                                      |
| i,j,k,l,m,n              |      | When these variables are used in a register name, an offset, or an address, they refer to the value of a register array where the register is part of a group of repeating registers. The register groups form a hierarchical structure and the array is represented with a formula. |
| y                        |      | When this variable is used in a register name, an offset, or an address it refers to the value of a register array.                                                                                                                                                                  |

### 14.6.2.1 I2C Own Address Register (I2COAR) (Offset = 0h) [reset = 0h]

The I2C own address register (I2COAR) is a 16-bit register. The I2C module uses this register to specify its own slave address, which distinguishes it from other slaves connected to the I2C-bus. If the 7-bit addressing mode is selected (XA = 0 in I2CMDR), only bits 6-0 are used; write 0s to bits 9-7.

**Figure 14-19. I2C Own Address Register (I2COAR)**

|          |    |    |    |    |    |        |   |
|----------|----|----|----|----|----|--------|---|
| 15       | 14 | 13 | 12 | 11 | 10 | 9      | 8 |
| RESERVED |    |    |    |    |    | OAR    |   |
| R-0h     |    |    |    |    |    | R/W-0h |   |
| 7        | 6  | 5  | 4  | 3  | 2  | 1      | 0 |
| OAR      |    |    |    |    |    |        |   |
| R/W-0h   |    |    |    |    |    |        |   |

**Table 14-10. I2C Own Address Register (I2COAR) Field Descriptions**

| Bit   | Field    | Type | Reset | Description                                                                                                                                                                                                                                                                                   |
|-------|----------|------|-------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-10 | RESERVED | R    | 0h    | Reserved                                                                                                                                                                                                                                                                                      |
| 9-0   | OAR      | R/W  | 0h    | In 7-bit addressing mode (XA = 0 in I2CMDR):<br>00h-7Fh Bits 6-0 provide the 7-bit slave address of the I2C module.<br>Write 0s to bits 9-7.<br>In 10-bit addressing mode (XA = 1 in I2CMDR):<br>000h-3FFh Bits 9-0 provide the 10-bit slave address of the I2C module.<br>Reset type: SYSRSn |

#### 14.6.2.2 I2C Interrupt Enable Register (I2CIER) (Offset = 1h) [reset = 0h]

I2CIER is used by the CPU to individually enable or disable I2C interrupt requests.

**Figure 14-20. I2C Interrupt Enable Register (I2CIER)**

|          |        |        |        |        |        |        |        |
|----------|--------|--------|--------|--------|--------|--------|--------|
| 15       | 14     | 13     | 12     | 11     | 10     | 9      | 8      |
| RESERVED |        |        |        |        |        |        |        |
| R-0h     |        |        |        |        |        |        |        |
| 7        | 6      | 5      | 4      | 3      | 2      | 1      | 0      |
| RESERVED | AAS    | SCD    | XRDY   | RRDY   | ARDY   | NACK   | ARBL   |
| R-0h     | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h |

**Table 14-11. I2C Interrupt Enable Register (I2CIER) Field Descriptions**

| Bit  | Field    | Type | Reset | Description                                                                                                                                                                                          |
|------|----------|------|-------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-7 | RESERVED | R    | 0h    | Reserved                                                                                                                                                                                             |
| 6    | AAS      | R/W  | 0h    | Addressed as slave interrupt enable<br>Reset type: SYSRSn<br>0h (R/W) = Interrupt request disabled<br>1h (R/W) = Interrupt request enabled                                                           |
| 5    | SCD      | R/W  | 0h    | Stop condition detected interrupt enable<br>Reset type: SYSRSn<br>0h (R/W) = Interrupt request disabled<br>1h (R/W) = Interrupt request enabled                                                      |
| 4    | XRDY     | R/W  | 0h    | Transmit-data-ready interrupt enable bit.<br>This bit should not be set when using FIFO mode.<br>Reset type: SYSRSn<br>0h (R/W) = Interrupt request disabled<br>1h (R/W) = Interrupt request enabled |
| 3    | RRDY     | R/W  | 0h    | Receive-data-ready interrupt enable bit.<br>This bit should not be set when using FIFO mode.<br>Reset type: SYSRSn<br>0h (R/W) = Interrupt request disabled<br>1h (R/W) = Interrupt request enabled  |
| 2    | ARDY     | R/W  | 0h    | Register-access-ready interrupt enable<br>Reset type: SYSRSn<br>0h (R/W) = Interrupt request disabled<br>1h (R/W) = Interrupt request enabled                                                        |
| 1    | NACK     | R/W  | 0h    | No-acknowledgment interrupt enable<br>Reset type: SYSRSn<br>0h (R/W) = Interrupt request disabled<br>1h (R/W) = Interrupt request enabled                                                            |
| 0    | ARBL     | R/W  | 0h    | Arbitration-lost interrupt enable<br>Reset type: SYSRSn<br>0h (R/W) = Interrupt request disabled<br>1h (R/W) = Interrupt request enabled                                                             |

### 14.6.2.3 I2C Status Register (I2CSTR) (Offset = 2h) [reset = 410h]

The I2C status register (I2CSTR) is a 16-bit register used to determine which interrupt has occurred and to read status information.

**Figure 14-21. I2C Status Register (I2CSTR)**

|          |          |          |          |          |          |          |          |
|----------|----------|----------|----------|----------|----------|----------|----------|
| 15       | 14       | 13       | 12       | 11       | 10       | 9        | 8        |
| RESERVED | SDIR     | NACKSNT  | BB       | RSFULL   | XSMT     | AAS      | AD0      |
| R-0h     | R/W1C-0h | R/W1C-0h | R-0h     | R-0h     | R-1h     | R-0h     | R-0h     |
| 7        | 6        | 5        | 4        | 3        | 2        | 1        | 0        |
| RESERVED | SCD      | XRDY     | RRDY     | ARDY     | NACK     | ARBL     |          |
| R/W-0h   | R/W1C-0h | R-1h     | R/W1C-0h | R/W1C-0h | R/W1C-0h | R/W1C-0h | R/W1C-0h |

**Table 14-12. I2C Status Register (I2CSTR) Field Descriptions**

| Bit | Field    | Type  | Reset | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|-----|----------|-------|-------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15  | RESERVED | R     | 0h    | Reserved                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| 14  | SDIR     | R/W1C | 0h    | Slave direction bit<br>Reset type: SYSRSn<br>0h (R/W) = I2C is not addressed as a slave transmitter. SDIR is cleared by one of the following events:<br>- It is manually cleared. To clear this bit, write a 1 to it.<br>- Digital loopback mode is enabled.<br>- A START or STOP condition occurs on the I2C bus.<br>1h (R/W) = I2C is addressed as a slave transmitter.                                                                                                                                                                                                                          |
| 13  | NACKSNT  | R/W1C | 0h    | NACK sent bit.<br>This bit is used when the I2C module is in the receiver mode. One instance in which NACKSNT is affected is when the NACK mode is used (see the description for NACKMOD in<br>Reset type: SYSRSn<br>0h (R/W) = NACK not sent. NACKSNT bit is cleared by any one of the following events:<br>- It is manually cleared. To clear this bit, write a 1 to it.<br>- The I2C module is reset (either when 0 is written to the IRS bit of I2CMDR or when the whole device is reset).<br>1h (R/W) = NACK sent: A no-acknowledge bit was sent during the acknowledge cycle on the I2C-bus. |
| 12  | BB       | R     | 0h    | Bus busy bit.<br>BB indicates whether the I2C-bus is busy or is free for another data transfer. See the paragraph following the table for more information<br>Reset type: SYSRSn<br>0h (R/W) = Bus free. BB is cleared by any one of the following events:<br>- The I2C module receives or transmits a STOP bit (bus free).<br>- The I2C module is reset.<br>1h (R/W) = Bus busy: The I2C module has received or transmitted a START bit on the bus.                                                                                                                                               |

**Table 14-12. I2C Status Register (I2CSTR) Field Descriptions (continued)**

| Bit | Field    | Type  | Reset | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|-----|----------|-------|-------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 11  | RSFULL   | R     | 0h    | <p>Receive shift register full bit.</p> <p>RSFULL indicates an overrun condition during reception. Overrun occurs when new data is received into the shift register (I2CRSR) and the old data has not been read from the receive register (I2CDRR). As new bits arrive from the SDA pin, they overwrite the bits in I2CRSR. The new data will not be copied to ICDRR until the previous data is read.</p> <p>Reset type: SYSRSn</p> <p>0h (R/W) = No overrun detected. RSFULL is cleared by any one of the following events:</p> <ul style="list-style-type: none"> <li>- I2CDRR is read by the CPU. Debug probe reads of the I2CDRR do not affect this bit.</li> <li>- The I2C module is reset.</li> </ul> <p>1h (R/W) = Overrun detected</p>                         |
| 10  | XSMT     | R     | 1h    | <p>Transmit shift register empty bit.</p> <p>XSMT = 0 indicates that the transmitter has experienced underflow. Underflow occurs when the transmit shift register (I2CXSR) is empty but the data transmit register (I2CDXR) has not been loaded since the last I2CDXR-to-I2CXSR transfer. The next I2CDXR-to-I2CXSR transfer will not occur until new data is in I2CDXR. If new data is not transferred in time, the previous data may be re-transmitted on the SDA pin.</p> <p>Reset type: SYSRSn</p> <p>0h (R/W) = Underflow detected (empty)</p> <p>1h (R/W) = No underflow detected (not empty). XSMT is set by one of the following events:</p> <ul style="list-style-type: none"> <li>- Data is written to I2CDXR.</li> <li>- The I2C module is reset</li> </ul> |
| 9   | AAS      | R     | 0h    | <p>Addressed-as-slave bit</p> <p>Reset type: SYSRSn</p> <p>0h (R/W) = In the 7-bit addressing mode, the AAS bit is cleared when receiving a NACK, a STOP condition, or a repeated START condition. In the 10-bit addressing mode, the AAS bit is cleared when receiving a NACK, a STOP condition, or by a slave address different from the I2C peripheral's own slave address.</p> <p>1h (R/W) = The I2C module has recognized its own slave address or an address of all zeros (general call).</p>                                                                                                                                                                                                                                                                    |
| 8   | AD0      | R     | 0h    | <p>Address 0 bits</p> <p>Reset type: SYSRSn</p> <p>0h (R/W) = AD0 has been cleared by a START or STOP condition.</p> <p>1h (R/W) = An address of all zeros (general call) is detected.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| 7-6 | RESERVED | R/W   | 0h    | Reserved                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| 5   | SCD      | R/W1C | 0h    | <p>Stop condition detected bit.</p> <p>SCD is set when the I2C sends or receives a STOP condition. The I2C module delays clearing of the I2CMDR[STP] bit until the SCD bit is set.</p> <p>Reset type: SYSRSn</p> <p>0h (R/W) = STOP condition not detected since SCD was last cleared. SCD is cleared by any one of the following events:</p> <ul style="list-style-type: none"> <li>- I2CISRC is read by the CPU when it contains the value 110b (stop condition detected). Debug probe reads of the I2CISRC do not affect this bit.</li> <li>- SCD is manually cleared. To clear this bit, write a 1 to it.</li> <li>- The I2C module is reset.</li> </ul> <p>1h (R/W) = A STOP condition has been detected on the I2C bus.</p>                                      |

**Table 14-12. I2C Status Register (I2CSTR) Field Descriptions (continued)**

| Bit | Field | Type  | Reset | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|-----|-------|-------|-------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 4   | XRDY  | R     | 1h    | <p>Transmit-data-ready interrupt flag bit.</p> <p>When not in FIFO mode, XRDY indicates that the data transmit register (I2CDXR) is ready to accept new data because the previous data has been copied from I2CDXR to the transmit shift register (I2CXSR). The CPU can poll XRDY or use the XRDY interrupt request. When in FIFO mode, use TXFFINT instead.</p> <p>Reset type: SYSRSn</p> <p>0h (R/W) = I2CDXR not ready. XRDY is cleared when data is written to I2CDXR.</p> <p>1h (R/W) = I2CDXR ready: Data has been copied from I2CDXR to I2CXSR.</p> <p>XRDY is also forced to 1 when the I2C module is reset.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| 3   | RRDY  | R/W1C | 0h    | <p>Receive-data-ready interrupt flag bit.</p> <p>When not in FIFO mode, RRDY indicates that the data receive register (I2CDRR) is ready to be read because data has been copied from the receive shift register (I2CRSR) to I2CDRR. The CPU can poll RRDY or use the RRDY interrupt request. When in FIFO mode, use RXFFINT instead.</p> <p>Reset type: SYSRSn</p> <p>0h (R/W) = I2CDRR not ready. RRDY is cleared by any one of the following events:</p> <ul style="list-style-type: none"> <li>- I2CDRR is read by the CPU. Debug probe reads of the I2CDRR do not affect this bit.</li> <li>- RRDY is manually cleared. To clear this bit, write a 1 to it.</li> <li>- The I2C module is reset.</li> </ul> <p>1h (R/W) = I2CDRR ready: Data has been copied from I2CRSR to I2CDRR.</p>                                                                                                                                                                                                                                                                                                                                              |
| 2   | ARDY  | R/W1C | 0h    | <p>Register-access-ready interrupt flag bit (only applicable when the I2C module is in the master mode).</p> <p>ARDY indicates that the I2C module registers are ready to be accessed because the previously programmed address, data, and command values have been used. The CPU can poll ARDY or use the ARDY interrupt request.</p> <p>Reset type: SYSRSn</p> <p>0h (R/W) = The registers are not ready to be accessed. ARDY is cleared by any one of the following events:</p> <ul style="list-style-type: none"> <li>- The I2C module starts using the current register contents.</li> <li>- ARDY is manually cleared. To clear this bit, write a 1 to it.</li> <li>- The I2C module is reset.</li> </ul> <p>1h (R/W) = The registers are ready to be accessed.</p> <p>In the nonrepeat mode (RM = 0 in I2CMDR): If STP = 0 in I2CMDR, the ARDY bit is set when the internal data counter counts down to 0. If STP = 1, ARDY is not affected (instead, the I2C module generates a STOP condition when the counter reaches 0).</p> <p>In the repeat mode (RM = 1): ARDY is set at the end of each byte transmitted from I2CDXR.</p> |

**Table 14-12. I2C Status Register (I2CSTR) Field Descriptions (continued)**

| Bit | Field | Type  | Reset | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|-----|-------|-------|-------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1   | NACK  | R/W1C | 0h    | <p>No-acknowledgment interrupt flag bit.</p> <p>NACK applies when the I2C module is a master transmitter. NACK indicates whether the I2C module has detected an acknowledge bit (ACK) or a noacknowledge bit (NACK) from the slave receiver. The CPU can poll NACK or use the NACK interrupt request.</p> <p>Reset type: SYSRSn</p> <p>0h (R/W) = ACK received/NACK not received. This bit is cleared by any one of the following events:</p> <ul style="list-style-type: none"> <li>- An acknowledge bit (ACK) has been sent by the slave receiver.</li> <li>- NACK is manually cleared. To clear this bit, write a 1 to it.</li> <li>- The CPU reads the interrupt source register (I2CISRC) and the register contains the code for a NACK interrupt. Debug probe reads of the I2CISRC do not affect this bit.</li> <li>- The I2C module is reset.</li> </ul> <p>1h (R/W) = NACK bit received. The hardware detects that a no-acknowledge (NACK) bit has been received.</p> <p>Note: While the I2C module performs a general call transfer, NACK is 1, even if one or more slaves send acknowledgment.</p>                                                                                                                                                                                                |
| 0   | ARBL  | R/W1C | 0h    | <p>Arbitration-lost interrupt flag bit (only applicable when the I2C module is a master-transmitter).</p> <p>ARBL primarily indicates when the I2C module has lost an arbitration contest with another master-transmitter. The CPU can poll ARBL or use the ARBL interrupt request.</p> <p>Reset type: SYSRSn</p> <p>0h (R/W) = Arbitration not lost. AL is cleared by any one of the following events:</p> <ul style="list-style-type: none"> <li>- AL is manually cleared. To clear this bit, write a 1 to it.</li> <li>- The CPU reads the interrupt source register (I2CISRC) and the register contains the code for an</li> </ul> <p>AL interrupt. Debug probe reads of the I2CISRC do not affect this bit.</p> <ul style="list-style-type: none"> <li>- The I2C module is reset.</li> </ul> <p>1h (R/W) = Arbitration lost. AL is set by any one of the following events:</p> <ul style="list-style-type: none"> <li>- The I2C module senses that it has lost an arbitration with two or more competing transmitters that started a transmission almost simultaneously.</li> <li>- The I2C module attempts to start a transfer while the BB (bus busy) bit is set to 1.</li> </ul> <p>When AL becomes 1, the MST and STP bits of I2CMDR are cleared, and the I2C module becomes a slave-receiver.</p> |

#### 14.6.2.4 I2C Clock Low-time Divider (I2CCLKL) Register (Offset = 3h) [reset = 0h]

I2C Clock low-time divider

**Figure 14-22. I2C Clock Low-time Divider (I2CCLKL) Register**

|         |    |    |    |    |    |   |   |
|---------|----|----|----|----|----|---|---|
| 15      | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
| I2CCLKL |    |    |    |    |    |   |   |
| R/W-0h  |    |    |    |    |    |   |   |
| 7       | 6  | 5  | 4  | 3  | 2  | 1 | 0 |
| I2CCLKL |    |    |    |    |    |   |   |
| R/W-0h  |    |    |    |    |    |   |   |

**Table 14-13. I2C Clock Low-time Divider (I2CCLKL) Register Field Descriptions**

| Bit  | Field   | Type | Reset | Description                                                                                                                                                                                                                                                                                                                                                                                                     |
|------|---------|------|-------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-0 | I2CCLKL | R/W  | 0h    | <p>Clock low-time divide-down value.</p> <p>To produce the low time duration of the master clock, the period of the module clock is multiplied by (ICCL + d). d is an adjustment factor based on the prescaler. See the Clock Divider Registers section of the Introduction for details.</p> <p>Note: These bits must be set to a non-zero value for proper I2C clock generation.</p> <p>Reset type: SYSRSn</p> |

#### 14.6.2.5 I2C Clock High-time Divider (I2CCLKH) Register (Offset = 4h) [reset = 0h]

I2C Clock high-time divider

**Figure 14-23. I2C Clock High-time Divider (I2CCLKH) Register**

|         |    |    |    |    |    |   |   |
|---------|----|----|----|----|----|---|---|
| 15      | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
| I2CCLKH |    |    |    |    |    |   |   |
| R/W-0h  |    |    |    |    |    |   |   |
| 7       | 6  | 5  | 4  | 3  | 2  | 1 | 0 |
| I2CCLKH |    |    |    |    |    |   |   |
| R/W-0h  |    |    |    |    |    |   |   |

**Table 14-14. I2C Clock High-time Divider (I2CCLKH) Register Field Descriptions**

| Bit  | Field   | Type | Reset | Description                                                                                                                                                                                                                                                                                                                                                                                                       |
|------|---------|------|-------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-0 | I2CCLKH | R/W  | 0h    | <p>Clock high-time divide-down value.</p> <p>To produce the high time duration of the master clock, the period of the module clock is multiplied by (ICCL + d). d is an adjustment factor based on the prescaler. See the Clock Divider Registers section of the Introduction for details.</p> <p>Note: These bits must be set to a non-zero value for proper I2C clock generation.</p> <p>Reset type: SYSRSn</p> |



#### 14.6.2.6 I2C Data Count (I2CCNT) Register (Offset = 5h) [reset = 0h]

I2CCNT is a 16-bit register used to indicate how many data bytes to transfer when the I2C module is configured as a transmitter, or to receive when configured as a master receiver. In the repeat mode (RM = 1), I2CCNT is not used. The value written to I2CCNT is copied to an internal data counter. The internal data counter is decremented by 1 for each byte transferred (I2CCNT remains unchanged). If a STOP condition is requested in the master mode (STP = 1 in I2CMDR), the I2C module terminates the transfer with a STOP condition when the countdown is complete (that is, when the last byte has been transferred).

**Figure 14-24. I2C Data Count (I2CCNT) Register**

|        |    |    |    |    |    |   |   |
|--------|----|----|----|----|----|---|---|
| 15     | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
| I2CCNT |    |    |    |    |    |   |   |
| R/W-0h |    |    |    |    |    |   |   |
| 7      | 6  | 5  | 4  | 3  | 2  | 1 | 0 |
| I2CCNT |    |    |    |    |    |   |   |
| R/W-0h |    |    |    |    |    |   |   |

**Table 14-15. I2C Data Count (I2CCNT) Register Field Descriptions**

| Bit  | Field  | Type | Reset | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
|------|--------|------|-------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-0 | I2CCNT | R/W  | 0h    | <p>Data count value. I2CCNT indicates the number of data bytes to transfer or receive.</p> <p>If a STOP condition is specified (STP=1) then I2CCNT will decrease after each byte is sent until it reaches zero, which in turn will generate a STOP condition.</p> <p>The value in I2CCNT is a don't care when the RM bit in I2CMDR is set to 1.</p> <p>The start value loaded to the internal data counter is 65536.</p> <p>The start value loaded to internal data counter is 1-65535.</p> <p>Reset type: SYSRSn</p> <p>0h (R/W) = data count value is 65536</p> <p>1h (R/W) = data count value is 1</p> <p>2h (R/W) = data count value is 2</p> <p>FFFFh (R/W) = data count value is 65535</p> |

### 14.6.2.7 I2C Data Receive Register (I2CDRR) (Offset = 6h) [reset = 0h]

I2CDRR is a 16-bit register used by the CPU to read received data. The I2C module can receive a data byte with 1 to 8 bits. The number of bits is selected with the bit count (BC) bits in I2CMDR. One bit at a time is shifted in from the SDA pin to the receive shift register (I2CRSR). When a complete data byte has been received, the I2C module copies the data byte from I2CRSR to I2CDRR. The CPU cannot access I2CRSR directly. If a data byte with fewer than 8 bits is in I2CDRR, the data value is right-justified, and the other bits of I2CDRR(7-0) are undefined. For example, if BC = 011 (3-bit data size), the receive data is in I2CDRR(2-0), and the content of I2CDRR(7-3) is undefined. When in the receive FIFO mode, the I2CDRR register acts as the receive FIFO buffer.

**Figure 14-25. I2C Data Receive Register (I2CDRR)**

|          |    |    |    |    |    |   |   |
|----------|----|----|----|----|----|---|---|
| 15       | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
| RESERVED |    |    |    |    |    |   |   |
| R-0h     |    |    |    |    |    |   |   |
| 7        | 6  | 5  | 4  | 3  | 2  | 1 | 0 |
| DATA     |    |    |    |    |    |   |   |
| R-0h     |    |    |    |    |    |   |   |

**Table 14-16. I2C Data Receive Register (I2CDRR) Field Descriptions**

| Bit  | Field    | Type | Reset | Description                        |
|------|----------|------|-------|------------------------------------|
| 15-8 | RESERVED | R    | 0h    | Reserved                           |
| 7-0  | DATA     | R    | 0h    | Receive data<br>Reset type: SYSRSn |

#### 14.6.2.8 I2C Slave Address Register (I2CSAR) (Offset = 7h) [reset = 3FFh]

The I2C slave address register (I2CSAR) is a 16-bit register for storing the next slave address that will be transmitted by the I2C module when it is a master. The SAR field of I2CSAR contains a 7-bit or 10-bit slave address. When the I2C module is not using the free data format (FDF = 0 in I2CMDR), it uses this address to initiate data transfers with a slave, or slaves. When the address is nonzero, the address is for a particular slave. When the address is 0, the address is a general call to all slaves. If the 7-bit addressing mode is selected (XA = 0 in I2CMDR), only bits 6-0 of I2CSAR are used write 0s to bits 9-7.

**Figure 14-26. I2C Slave Address Register (I2CSAR)**

|          |    |    |    |    |    |   |          |
|----------|----|----|----|----|----|---|----------|
| 15       | 14 | 13 | 12 | 11 | 10 | 9 | 8        |
| RESERVED |    |    |    |    |    |   | SAR      |
| R-0h     |    |    |    |    |    |   | R/W-3FFh |
| 7        | 6  | 5  | 4  | 3  | 2  | 1 | 0        |
| SAR      |    |    |    |    |    |   |          |
| R/W-3FFh |    |    |    |    |    |   |          |

**Table 14-17. I2C Slave Address Register (I2CSAR) Field Descriptions**

| Bit   | Field    | Type | Reset | Description                                                                                                                                                                                                                                                                                                                                                                                            |
|-------|----------|------|-------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-10 | RESERVED | R    | 0h    | Reserved                                                                                                                                                                                                                                                                                                                                                                                               |
| 9-0   | SAR      | R/W  | 3FFh  | In 7-bit addressing mode (XA = 0 in I2CMDR):<br>00h-7Fh Bits 6-0 provide the 7-bit slave address that the I2C module transmits when it is in the master-transmitter mode. Write 0s to bits 9-7.<br>In 10-bit addressing mode (XA = 1 in I2CMDR):<br>000h-3FFh Bits 9-0 provide the 10-bit slave address that the I2C module transmits when it is in the master transmitter mode.<br>Reset type: SYSRSn |

### 14.6.2.9 I2C Data Transmit Register (I2CDXR) (Offset = 8h) [reset = 0h]

The CPU writes transmit data to I2CDXR. This 16-bit register accepts a data byte with 1 to 8 bits. Before writing to I2CDXR, specify how many bits are in a data byte by loading the appropriate value into the bit count (BC) bits of I2CMDR. When writing a data byte with fewer than 8 bits, make sure the value is right-aligned in I2CDXR. After a data byte is written to I2CDXR, the I2C module copies the data byte to the transmit shift register (I2CXSR). The CPU cannot access I2CXSR directly. From I2CXSR, the I2C module shifts the data byte out on the SDA pin, one bit at a time. When in the transmit FIFO mode, the I2CDXR register acts as the transmit FIFO buffer.

**Figure 14-27. I2C Data Transmit Register (I2CDXR)**

|          |    |    |    |    |    |   |   |
|----------|----|----|----|----|----|---|---|
| 15       | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
| RESERVED |    |    |    |    |    |   |   |
| R-0h     |    |    |    |    |    |   |   |
| 7        | 6  | 5  | 4  | 3  | 2  | 1 | 0 |
| DATA     |    |    |    |    |    |   |   |
| R/W-0h   |    |    |    |    |    |   |   |

**Table 14-18. I2C Data Transmit Register (I2CDXR) Field Descriptions**

| Bit  | Field    | Type | Reset | Description                         |
|------|----------|------|-------|-------------------------------------|
| 15-8 | RESERVED | R    | 0h    | Reserved                            |
| 7-0  | DATA     | R/W  | 0h    | Transmit data<br>Reset type: SYSRSn |

#### 14.6.2.10 I2C Mode Register (I2CMDR) (Offset = 9h) [reset = 0h]

The I2C mode register (I2CMDR) is a 16-bit register that contains the control bits of the I2C module.

**Figure 14-28. I2C Mode Register (I2CMDR)**

|         |        |        |          |        |        |        |        |
|---------|--------|--------|----------|--------|--------|--------|--------|
| 15      | 14     | 13     | 12       | 11     | 10     | 9      | 8      |
| NACKMOD | FREE   | STT    | RESERVED | STP    | MST    | TRX    | XA     |
| R/W-0h  | R/W-0h | R/W-0h | R-0h     | R/W-0h | R/W-0h | R/W-0h | R/W-0h |
| 7       | 6      | 5      | 4        | 3      | 2      | 1      | 0      |
| RM      | DLB    | IRS    | STB      | FDF    | BC     |        |        |
| R/W-0h  | R/W-0h | R/W-0h | R/W-0h   | R/W-0h | R/W-0h |        |        |

**Table 14-19. I2C Mode Register (I2CMDR) Field Descriptions**

| Bit | Field   | Type | Reset | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|-----|---------|------|-------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15  | NACKMOD | R/W  | 0h    | <p>NACK mode bit. This bit is only applicable when the I2C module is acting as a receiver.</p> <p>Reset type: SYSRSn</p> <p>0h (R/W) = In the slave-receiver mode: The I2C module sends an acknowledge (ACK) bit to the transmitter during each acknowledge cycle on the bus. The I2C module only sends a no-acknowledge (NACK) bit if you set the NACKMOD bit.</p> <p>In the master-receiver mode: The I2C module sends an ACK bit during each acknowledge cycle until the internal data counter counts down to 0. At that point, the I2C module sends a NACK bit to the transmitter. To have a NACK bit sent earlier, you must set the NACKMOD bit</p> <p>1h (R/W) = In either slave-receiver or master-receiver mode: The I2C module sends a NACK bit to the transmitter during the next acknowledge cycle on the bus. Once the NACK bit has been sent, NACKMOD is cleared.</p> <p>Important: To send a NACK bit in the next acknowledge cycle, you must set NACKMOD before the rising edge of the last data bit.</p> |
| 14  | FREE    | R/W  | 0h    | <p>This bit controls the action taken by the I2C module when a debugger breakpoint is encountered.</p> <p>Reset type: SYSRSn</p> <p>0h (R/W) = When I2C module is master:</p> <p>If SCL is low when the breakpoint occurs, the I2C module stops immediately and keeps driving SCL low, whether the I2C module is the transmitter or the receiver. If SCL is high, the I2C module waits until SCL becomes low and then stops.</p> <p>When I2C module is slave:</p> <p>A breakpoint forces the I2C module to stop when the current transmission/reception is complete.</p> <p>1h (R/W) = The I2C module runs free that is, it continues to operate when a breakpoint occurs.</p>                                                                                                                                                                                                                                                                                                                                           |
| 13  | STT     | R/W  | 0h    | <p>START condition bit (only applicable when the I2C module is a master). The RM, STT, and STP bits determine when the I2C module starts and stops data transmissions (see Table 9-6). Note that the STT and STP bits can be used to terminate the repeat mode, and that this bit is not writable when IRS = 0.</p> <p>Reset type: SYSRSn</p> <p>0h (R/W) = In the master mode, STT is automatically cleared after the START condition has been generated.</p> <p>1h (R/W) = In the master mode, setting STT to 1 causes the I2C module to generate a START condition on the I2C-bus</p>                                                                                                                                                                                                                                                                                                                                                                                                                                 |

**Table 14-19. I2C Mode Register (I2CMDR) Field Descriptions (continued)**

| Bit | Field    | Type | Reset | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|-----|----------|------|-------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 12  | RESERVED | R    | 0h    | Reserved                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| 11  | STP      | R/W  | 0h    | <p>STOP condition bit (only applicable when the I2C module is a master).</p> <p>In the master mode, the RM,STT, and STP bits determine when the I2C module starts and stops data transmissions.</p> <p>Note that the STT and STP bits can be used to terminate the repeat mode, and that this bit is not writable when IRS=0. When in non-repeat mode, at least one byte must be transferred before a stop condition can be generated. The I2C module delays clearing of this bit until after the I2CSTR[SCD] bit is set. To avoid disrupting the I2C state machine, the user must wait until this bit is clear before initiating a new message.</p> <p>Reset type: SYSRSn</p> <p>0h (R/W) = STP is automatically cleared after the STOP condition has been generated</p> <p>1h (R/W) = STP has been set by the device to generate a STOP condition when the internal data counter of the I2C module counts down to 0.</p> |
| 10  | MST      | R/W  | 0h    | <p>Master mode bit.</p> <p>MST determines whether the I2C module is in the slave mode or the master mode. MST is automatically changed from 1 to 0 when the I2C master generates a STOP condition</p> <p>Reset type: SYSRSn</p> <p>0h (R/W) = Slave mode. The I2C module is a slave and receives the serial clock from the master.</p> <p>1h (R/W) = Master mode. The I2C module is a master and generates the serial clock on the SCL pin.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| 9   | TRX      | R/W  | 0h    | <p>Transmitter mode bit.</p> <p>When relevant, TRX selects whether the I2C module is in the transmitter mode or the receiver mode.</p> <p>Reset type: SYSRSn</p> <p>0h (R/W) = Receiver mode. The I2C module is a receiver and receives data on the SDA pin.</p> <p>1h (R/W) = Transmitter mode. The I2C module is a transmitter and transmits data on the SDA pin.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| 8   | XA       | R/W  | 0h    | <p>Expanded address enable bit.</p> <p>Reset type: SYSRSn</p> <p>0h (R/W) = 7-bit addressing mode (normal address mode). The I2C module transmits 7-bit slave addresses (from bits 6-0 of I2CSAR), and its own slave address has 7 bits (bits 6-0 of I2COAR).</p> <p>1h (R/W) = 10-bit addressing mode (expanded address mode). The I2C module transmits 10-bit slave addresses (from bits 9-0 of I2CSAR), and its own slave address has 10 bits (bits 9-0 of I2COAR).</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                 |

**Table 14-19. I2C Mode Register (I2CMDR) Field Descriptions (continued)**

| Bit | Field | Type | Reset | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
|-----|-------|------|-------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 7   | RM    | R/W  | 0h    | <p>Repeat mode bit (only applicable when the I2C module is a master-transmitter).</p> <p>The RM, STT, and STP bits determine when the I2C module starts and stops data transmissions</p> <p>Reset type: SYSRSn</p> <p>0h (R/W) = Nonrepeat mode. The value in the data count register (I2CCNT) determines how many bytes are received/transmitted by the I2C module.</p> <p>1h (R/W) = Repeat mode. A data byte is transmitted each time the I2CDXR register is written to (or until the transmit FIFO is empty when in FIFO mode) until the STP bit is manually set. The value of I2CCNT is ignored. The ARDY bit/interrupt can be used to determine when the I2CDXR (or FIFO) is ready for more data, or when the data has all been sent and the CPU is allowed to write to the STP bit.</p>                                                                                                                                                                                   |
| 6   | DLB   | R/W  | 0h    | <p>Digital loopback mode bit.</p> <p>Reset type: SYSRSn</p> <p>0h (R/W) = Digital loopback mode is disabled.</p> <p>1h (R/W) = Digital loopback mode is enabled. For proper operation in this mode, the MST bit must be 1.</p> <p>In the digital loopback mode, data transmitted out of I2CDXR is received in I2CDRR after n device cycles by an internal path, where:</p> $n = ((\text{I2C input clock frequency} / \text{module clock frequency}) \times 8)$ <p>The transmit clock is also the receive clock. The address transmitted on the SDA pin is the address in I2COAR.</p> <p>Note: The free data format (FDF = 1) is not supported in the digital loopback mode.</p>                                                                                                                                                                                                                                                                                                  |
| 5   | IRS   | R/W  | 0h    | <p>I2C module reset bit.</p> <p>Reset type: SYSRSn</p> <p>0h (R/W) = The I2C module is in reset/disabled. When this bit is cleared to 0, all status bits (in I2CSTR) are set to their default values.</p> <p>1h (R/W) = The I2C module is enabled. This has the effect of releasing the I2C bus if the I2C peripheral is holding it.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| 4   | STB   | R/W  | 0h    | <p>START byte mode bit. This bit is only applicable when the I2C module is a master. As described in version 2.1 of the Philips Semiconductors I2C-bus specification, the START byte can be used to help a slave that needs extra time to detect a START condition. When the I2C module is a slave, it ignores a START byte from a master, regardless of the value of the STB bit.</p> <p>Reset type: SYSRSn</p> <p>0h (R/W) = The I2C module is not in the START byte mode.</p> <p>1h (R/W) = The I2C module is in the START byte mode. When you set the START condition bit (STT), the I2C module begins the transfer with more than just a START condition. Specifically, it generates:</p> <ol style="list-style-type: none"> <li>1. A START condition</li> <li>2. A START byte (0000 0001b)</li> <li>3. A dummy acknowledge clock pulse</li> <li>4. A repeated START condition</li> </ol> <p>Then, as normal, the I2C module sends the slave address that is in I2CSAR.</p> |

**Table 14-19. I2C Mode Register (I2CMDR) Field Descriptions (continued)**

| Bit | Field | Type | Reset | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
|-----|-------|------|-------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 3   | FDF   | R/W  | 0h    | Free data format mode bit.<br>Reset type: SYSRSn<br>0h (R/W) = Free data format mode is disabled. Transfers use the 7-/10-bit addressing format selected by the XA bit.<br>1h (R/W) = Free data format mode is enabled. Transfers have the free data (no address) format described in Section 9.2.5.<br>The free data format is not supported in the digital loopback mode (DLB=1).                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| 2-0 | BC    | R/W  | 0h    | Bit count bits.<br>BC defines the number of bits (1 to 8) in the next data byte that is to be received or transmitted by the I2C module. The number of bits selected with BC must match the data size of the other device. Notice that when BC = 000b, a data byte has 8 bits. BC does not affect address bytes, which always have 8 bits.<br>Note: If the bit count is less than 8, receive data is right-justified in I2CDRR(7-0), and the other bits of I2CDRR(7-0) are undefined. Also, transmit data written to I2CDXR must be right-justified<br>Reset type: SYSRSn<br>0h (R/W) = 8 bits per data byte<br>1h (R/W) = 1 bit per data byte<br>2h (R/W) = 2 bits per data byte<br>3h (R/W) = 3 bits per data byte<br>4h (R/W) = 4 bits per data byte<br>5h (R/W) = 5 bits per data byte<br>6h (R/W) = 6 bits per data byte<br>7h (R/W) = 7 bits per data byte |



#### 14.6.2.11 I2C Interrupt Source (I2CISRC) Register (Offset = Ah) [reset = 0h]

The I2C interrupt source register (I2CISRC) is a 16-bit register used by the CPU to determine which event generated the I2C interrupt.

**Figure 14-29. I2C Interrupt Source (I2CISRC) Register**

|          |    |    |    |             |    |   |   |
|----------|----|----|----|-------------|----|---|---|
| 15       | 14 | 13 | 12 | 11          | 10 | 9 | 8 |
| RESERVED |    |    |    | WRITE_ZEROS |    |   |   |
| R-0h     |    |    |    | R/W-0h      |    |   |   |
| 7        | 6  | 5  | 4  | 3           | 2  | 1 | 0 |
| RESERVED |    |    |    | INTCODE     |    |   |   |
| R-0h     |    |    |    | R-0h        |    |   |   |

**Table 14-20. I2C Interrupt Source (I2CISRC) Register Field Descriptions**

| Bit   | Field       | Type | Reset | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
|-------|-------------|------|-------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-12 | RESERVED    | R    | 0h    | Reserved                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| 11-8  | WRITE_ZEROS | R/W  | 0h    | TI internal testing bits<br>These reserved bit locations should always be written as zeros.<br>Reset type: SYSRSn                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| 7-3   | RESERVED    | R    | 0h    | Reserved                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| 2-0   | INTCODE     | R    | 0h    | Interrupt code bits.<br>The binary code in INTCODE indicates the event that generated an I2C interrupt.<br>A CPU read will clear this field. If another lower priority interrupt is pending and enabled, the value corresponding to that interrupt will then be loaded. Otherwise, the value will stay cleared.<br>In the case of an arbitration lost, a no-acknowledgment condition detected, or a stop condition detected, a CPU read will also clear the associated interrupt flag bit in the I2CSTR register. Inversely, in the case of a Register-access-ready (ARDY), a Receive-data-ready (RRDY), a Transmit-data-ready (XRDY), or an Addressed-as-slave (AAS) condition, a CPU read will NOT clear the associated interrupt flag bit in the I2CSTR register<br>Debug probe reads will not affect the state of this field or of the status bits in the I2CSTR register.<br>Reset type: SYSRSn<br>0h (R/W) = None<br>1h (R/W) = Arbitration lost<br>2h (R/W) = No-acknowledgment condition detected<br>3h (R/W) = Registers ready to be accessed<br>4h (R/W) = Receive data ready<br>5h (R/W) = Transmit data ready<br>6h (R/W) = Stop condition detected<br>7h (R/W) = Addressed as slave |

### 14.6.2.12 I2C Extended Mode Register (I2CEMDR) (Offset = Bh) [reset = 1h]

I2C Extended Mode

**Figure 14-30. I2C Extended Mode Register (I2CEMDR)**

|          |    |    |    |    |    |   |        |
|----------|----|----|----|----|----|---|--------|
| 15       | 14 | 13 | 12 | 11 | 10 | 9 | 8      |
| RESERVED |    |    |    |    |    |   |        |
| R-0h     |    |    |    |    |    |   |        |
| 7        | 6  | 5  | 4  | 3  | 2  | 1 | 0      |
| RESERVED |    |    |    |    |    |   | BC     |
| R-0h     |    |    |    |    |    |   | R/W-1h |

**Table 14-21. I2C Extended Mode Register (I2CEMDR) Field Descriptions**

| Bit  | Field    | Type | Reset | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
|------|----------|------|-------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-1 | RESERVED | R    | 0h    | Reserved                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| 0    | BC       | R/W  | 1h    | <p>Backwards compatibility mode.</p> <p>This bit affects the timing of the transmit status bits (XRDY and XSMT) in the I2CSTR register when in slave transmitter mode. Check Backwards Compatibility Mode Bit, Slave Transmitter diagram for more details.</p> <p>Reset type: SYSRSn</p> <p>0h (R/W) = See the "Backwards Compatibility Mode Bit, Slave Transmitter" Figure for details.</p> <p>1h (R/W) = See the "Backwards Compatibility Mode Bit, Slave Transmitter" Figure for details.</p> |

#### 14.6.2.13 I2C Prescaler (I2CPSC) Register (Offset = Ch) [reset = 0h]

The I2C prescaler register (I2CPSC) is a 16-bit register used for dividing down the I2C input clock to obtain the desired module clock for the operation of the I2C module. See the device-specific data manual for the supported range of values for the module clock frequency. IPSC must be initialized while the I2C module is in reset (IRS = 0 in I2CMDR). The prescaled frequency takes effect only when IRS is changed to 1. Changing the IPSC value while IRS = 1 has no effect.

**Figure 14-31. I2C Prescaler (I2CPSC) Register**

|          |    |    |    |    |    |   |   |
|----------|----|----|----|----|----|---|---|
| 15       | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
| RESERVED |    |    |    |    |    |   |   |
| R-0h     |    |    |    |    |    |   |   |
| 7        | 6  | 5  | 4  | 3  | 2  | 1 | 0 |
| IPSC     |    |    |    |    |    |   |   |
| R/W-0h   |    |    |    |    |    |   |   |

**Table 14-22. I2C Prescaler (I2CPSC) Register Field Descriptions**

| Bit  | Field    | Type | Reset | Description                                                                                                                                                                                                                                                                                                        |
|------|----------|------|-------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-8 | RESERVED | R    | 0h    | Reserved                                                                                                                                                                                                                                                                                                           |
| 7-0  | IPSC     | R/W  | 0h    | I2C prescaler divide-down value.<br>IPSC determines how much the CPU clock is divided to create the module clock of the I2C module:<br>module clock frequency = I2C input clock frequency/(IPSC + 1)<br>Note: IPSC must be initialized while the I2C module is in reset (IRS = 0 in I2CMDR).<br>Reset type: SYSRSn |

#### 14.6.2.14 I2C FIFO Transmit (I2CFFTX) Register (Offset = 20h) [reset = 0h]

The I2C transmit FIFO register (I2CFFTX) is a 16-bit register that contains the I2C FIFO mode enable bit as well as the control and status bits for the transmit FIFO mode of operation on the I2C peripheral.

**Figure 14-32. I2C FIFO Transmit (I2CFFTX) Register**

| 15       | 14         | 13       | 12     | 11 | 10 | 9 | 8 |
|----------|------------|----------|--------|----|----|---|---|
| RESERVED | I2CFFEN    | TXFFRST  | TXFFST |    |    |   |   |
| R-0h     | R/W-0h     | R/W-0h   | R-0h   |    |    |   |   |
| 7        | 6          | 5        | 4      | 3  | 2  | 1 | 0 |
| TXFFINT  | TXFFINTCLR | TXFFIENA | TXFFIL |    |    |   |   |
| R-0h     | R-0/W1S-0h | R/W-0h   | R/W-0h |    |    |   |   |

**Table 14-23. I2C FIFO Transmit (I2CFFTX) Register Field Descriptions**

| Bit  | Field      | Type    | Reset | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|------|------------|---------|-------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15   | RESERVED   | R       | 0h    | Reserved                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| 14   | I2CFFEN    | R/W     | 0h    | I2C FIFO mode enable bit.<br>This bit must be enabled for either the transmit or the receive FIFO to operate correctly.<br>Reset type: SYSRSn<br>0h (R/W) = Disable the I2C FIFO mode.<br>1h (R/W) = Enable the I2C FIFO mode.                                                                                                                                                                                                                                                                                                           |
| 13   | TXFFRST    | R/W     | 0h    | Transmit FIFO Reset<br>Reset type: SYSRSn<br>0h (R/W) = Reset the transmit FIFO pointer to 0000 and hold the transmit FIFO in the reset state.<br>1h (R/W) = Enable the transmit FIFO operation.                                                                                                                                                                                                                                                                                                                                         |
| 12-8 | TXFFST     | R       | 0h    | Contains the status of the transmit FIFO:<br>xxxxx Transmit FIFO contains xxxxx bytes.<br>00000 Transmit FIFO is empty.<br>Note: Since these bits are reset to zero, the transmit FIFO interrupt flag will be set when the transmit FIFO operation is enabled and the I2C is taken out of reset. This will generate a transmit FIFO interrupt if enabled. To avoid any detrimental effects from this, write a one to the TXFFINTCLR once the transmit FIFO operation is enabled and the I2C is taken out of reset.<br>Reset type: SYSRSn |
| 7    | TXFFINT    | R       | 0h    | Transmit FIFO interrupt flag.<br>This bit cleared by a CPU write of a 1 to the TXFFINTCLR bit. If the TXFFIENA bit is set, this bit will generate an interrupt when it is set.<br>Reset type: SYSRSn<br>0h (R/W) = Transmit FIFO interrupt condition has not occurred.<br>1h (R/W) = Transmit FIFO interrupt condition has occurred.                                                                                                                                                                                                     |
| 6    | TXFFINTCLR | R-0/W1S | 0h    | Transmit FIFO Interrupt Flag Clear<br>Reset type: SYSRSn<br>0h (R/W) = Writes of zeros have no effect. Reads return a 0.<br>1h (R/W) = Writing a 1 to this bit clears the TXFFINT flag.                                                                                                                                                                                                                                                                                                                                                  |
| 5    | TXFFIENA   | R/W     | 0h    | Transmit FIFO Interrupt Enable<br>Reset type: SYSRSn<br>0h (R/W) = Disabled. TXFFINT flag does not generate an interrupt when set.<br>1h (R/W) = Enabled. TXFFINT flag does generate an interrupt when set.                                                                                                                                                                                                                                                                                                                              |

**Table 14-23. I2C FIFO Transmit (I2CFFTX) Register Field Descriptions (continued)**

| Bit | Field  | Type | Reset | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|-----|--------|------|-------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 4-0 | TXFFIL | R/W  | 0h    | <p>Transmit FIFO interrupt level.</p> <p>These bits set the status level that will set the transmit interrupt flag. When the TXFFST4-0 bits reach a value equal to or less than these bits, the TXFFINT flag will be set. This will generate an interrupt if the TXFFIENA bit is set. Because the I2C on this device has a 4-level transmit FIFO, these bits cannot be configured for an interrupt of more than 4 FIFO levels.</p> <p>Reset type: SYSRSn</p> |

#### 14.6.2.15 I2C FIFO Receive (I2CFFRX) Register (Offset = 21h) [reset = 0h]

The I2C receive FIFO register (I2CFFRX) is a 16-bit register that contains the control and status bits for the receive FIFO mode of operation on the I2C peripheral.

**Figure 14-33. I2C FIFO Receive (I2CFFRX) Register**

|          |            |          |        |    |      |   |   |
|----------|------------|----------|--------|----|------|---|---|
| 15       | 14         | 13       | 12     | 11 | 10   | 9 | 8 |
| RESERVED |            | RXFFRST  | RXFFST |    |      |   |   |
| R-0h     |            | R/W-0h   |        |    | R-0h |   |   |
| 7        | 6          | 5        | 4      | 3  | 2    | 1 | 0 |
| RXFFINT  | RXFFINTCLR | RXFFIENA | RXFFIL |    |      |   |   |
| R-0h     | R-0/W1S-0h | R/W-0h   | R/W-0h |    |      |   |   |

**Table 14-24. I2C FIFO Receive (I2CFFRX) Register Field Descriptions**

| Bit   | Field      | Type    | Reset | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|-------|------------|---------|-------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-14 | RESERVED   | R       | 0h    | Reserved                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| 13    | RXFFRST    | R/W     | 0h    | I2C receive FIFO reset bit<br>Reset type: SYSRSn<br>0h (R/W) = Reset the receive FIFO pointer to 0000 and hold the receive FIFO in the reset state.<br>1h (R/W) = Enable the receive FIFO operation.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| 12-8  | RXFFST     | R       | 0h    | Contains the status of the receive FIFO:<br>xxxxx Receive FIFO contains xxxxx bytes<br>00000 Receive FIFO is empty.<br>Reset type: SYSRSn                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| 7     | RXFFINT    | R       | 0h    | Receive FIFO interrupt flag.<br>This bit cleared by a CPU write of a 1 to the RXFFINTCLR bit. If the RXFFIENA bit is set, this bit will generate an interrupt when it is set<br>Reset type: SYSRSn<br>0h (R/W) = Receive FIFO interrupt condition has not occurred.<br>1h (R/W) = Receive FIFO interrupt condition has occurred.                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| 6     | RXFFINTCLR | R-0/W1S | 0h    | Receive FIFO interrupt flag clear bit.<br>Reset type: SYSRSn<br>0h (R/W) = Writes of zeros have no effect. Reads return a zero.<br>1h (R/W) = Writing a 1 to this bit clears the RXFFINT flag.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| 5     | RXFFIENA   | R/W     | 0h    | Receive FIFO interrupt enable bit.<br>Reset type: SYSRSn<br>0h (R/W) = Disabled. RXFFINT flag does not generate an interrupt when set.<br>1h (R/W) = Enabled. RXFFINT flag does generate an interrupt when set.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| 4-0   | RXFFIL     | R/W     | 0h    | Receive FIFO interrupt level.<br>These bits set the status level that will set the receive interrupt flag. When the RXFFST4-0 bits reach a value equal to or greater than these bits, the RXFFINT flag is set. This will generate an interrupt if the RXFFIENA bit is set.<br>Note: Since these bits are reset to zero, the receive FIFO interrupt flag will be set if the receive FIFO operation is enabled and the I2C is taken out of reset. This will generate a receive FIFO interrupt if enabled. To avoid this, modify these bits on the same instruction as or prior to setting the RXFFRST bit. Because the I2C on this device has a 4-level receive FIFO, these bits cannot be configured for an interrupt of more than 4 FIFO levels.<br>Reset type: SYSRSn |

## Multichannel Buffered Serial Port (McBSP)

This chapter describes the multichannel buffered serial port (McBSP). This device provides one high-speed multichannel buffered serial port (McBSP) that allows direct interface to codecs and other devices in a system.

|                                                   |     |
|---------------------------------------------------|-----|
| 15.1 Introduction.....                            | 878 |
| 15.2 McBSP Operation.....                         | 880 |
| 15.3 McBSP Sample Rate Generator.....             | 890 |
| 15.4 McBSP Exception/Error Conditions.....        | 896 |
| 15.5 Multichannel Selection Modes.....            | 904 |
| 15.6 SPI Operation Using the Clock Stop Mode..... | 912 |
| 15.7 Receiver Configuration.....                  | 919 |
| 15.8 Transmitter Configuration.....               | 938 |
| 15.9 Emulation and Reset Considerations.....      | 954 |
| 15.10 Data Packing Examples.....                  | 956 |
| 15.11 McBSP_REGS Registers.....                   | 959 |

## 15.1 Introduction

The McBSP consists of a data-flow path and a control path connected to external devices by six pins as shown in [Figure 15-1](#).

Data is communicated to devices interfaced with the McBSP via the data transmit (DX) pin for transmission and via the data receive (DR) pin for reception. Control information in the form of clocking and frame synchronization is communicated via the following pins: CLKX (transmit clock), CLKR (receive clock), FSX (transmit frame synchronization), and FSR (receive frame synchronization).

The CPU and the DMA controller communicate with the McBSP through 16-bit-wide registers accessible via the internal peripheral bus. The CPU or the DMA controller writes the data to be transmitted to the data transmit registers (DXR1, DXR2). Data written to the DXRs is shifted out to DX via the transmit shift registers (XSR1, XSR2). Similarly, receive data on the DR pin is shifted into the receive shift registers (RSR1, RSR2) and copied into the receive buffer registers (RBR1, RBR2). The contents of the RBRs is then copied to the DRRs, which can be read by the CPU or the DMA controller. This allows simultaneous movement of internal and external data communications.

DRR2, RBR2, RSR2, DXR2, and XSR2 are not used (written, read, or shifted) if the serial word length is 8 bits, 12 bits, or 16 bits. For larger word lengths, these registers are needed to hold the most significant bits.

The frame and clock loop-back is implemented at chip level to enable CLKX and FSX to drive CLKR and FSR. If the loop-back is enabled, the CLKR and FSR get their signals from the CLKX and FSX pads; instead of the CLKR and FSR pins.

### 15.1.1 Features of the McBSP

The McBSP features:

- Full-duplex communication
- Double-buffered transmission and triple-buffered reception, allowing a continuous data stream
- Independent clocking and framing for reception and transmission
- The capability to send interrupts to the CPU and to send DMA events to the DMA controller
- 128 channels for transmission and reception
- Multichannel selection modes that enable or disable block transfers in each of the channels
- Direct interface to industry-standard codecs, analog interface chips (AICs), and other serially connected A/D and D/A devices
- Support for external generation of clock signals and frame-synchronization signals
- A programmable sample rate generator for internal generation and control of clock signals and frame-synchronization signals
- Programmable polarity for frame-synchronization pulses and clock signals
- Direct interface to:
  - T1/E1 framers
  - IOM-2 compliant devices
  - AC97-compliant devices (the necessary multiphase frame capability is provided)
  - I2S compliant devices
  - SPI devices
- A wide selection of data sizes: 8, 12, 16, 20, 24, and 32 bits

---

#### Note

A value of the chosen data size is referred to as a *serial word* or *word* throughout the McBSP documentation. Elsewhere, *word* is used to describe a 16-bit value.

- 
- $\mu$ -law and A-law companding
  - The option of transmitting/receiving 8-bit data with the LSB first
  - Status bits for flagging exception/error conditions
  - ABIS mode is not supported.



### 15.1.2 McBSP Pins/Signals

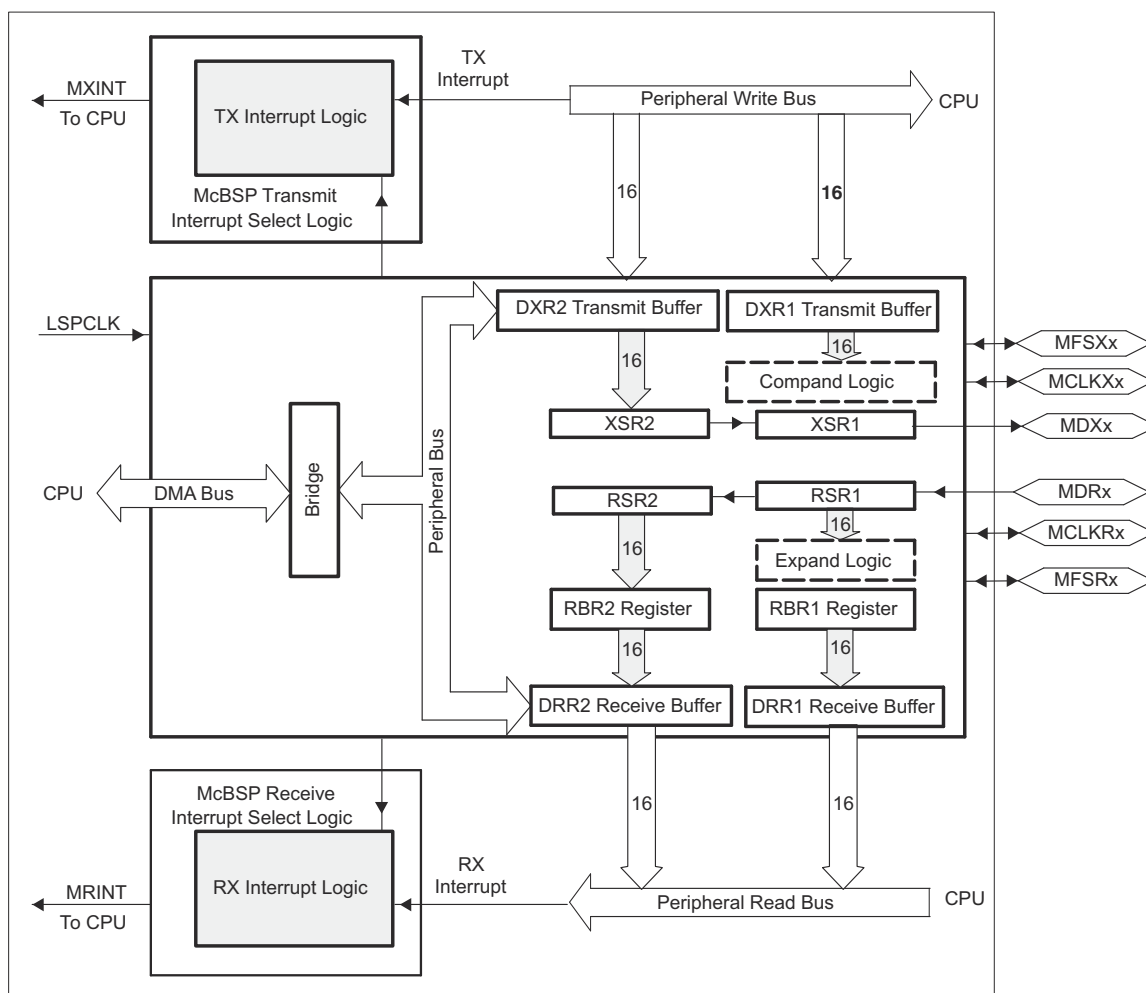
Table 15-1 describes the McBSP interface pins and some internal signals.

**Table 15-1. McBSP Interface Pins/Signals**

| McBSP-A Pin                  | Type | Description                                                                                                                                               |
|------------------------------|------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| MCLKRA                       | I/O  | Supplying or reflecting the receive clock; supplying the input clock of the sample rate generator                                                         |
| MCLKXA                       | I/O  | Supplying or reflecting the transmit clock; supplying the input clock of the sample rate generator                                                        |
| MDRA                         | I    | Serial data receive pin                                                                                                                                   |
| MDXA                         | O    | Serial data transmit pin                                                                                                                                  |
| MFSRA                        | I/O  | Supplying or reflecting the receive frame-sync signal; controlling sample rate generator synchronization for the case when GSYNC = 1 (see Section 15.3.3) |
| MFSXA                        | I/O  | Supplying or reflecting the transmit frame-sync signal                                                                                                    |
| <b>CPU Interrupt Signals</b> |      |                                                                                                                                                           |
| MRINT                        |      | Receive interrupt to CPU                                                                                                                                  |
| MXINT                        |      | Transmit interrupt to CPU                                                                                                                                 |
| <b>DMA Events</b>            |      |                                                                                                                                                           |
| REVT                         |      | Receive synchronization event to DMA                                                                                                                      |
| XEVT                         |      | Transmit synchronization event to DMA                                                                                                                     |

#### 15.1.2.1 McBSP Generic Block Diagram

The McBSP consists of a data-flow path and a control path connected to external devices by six pins as shown in Figure 15-1. The figure and the text in this section use generic pin names.



A. Not available in all devices. See the device-specific data sheet

**Figure 15-1. Conceptual Block Diagram of the McBSP**

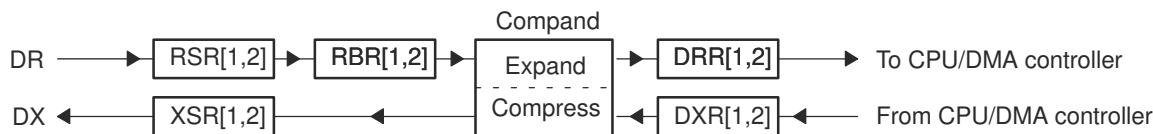
## 15.2 McBSP Operation

This section addresses the following topics:

- Data transfer process
- Companding (compressing and expanding) data
- Clocking and framing data
- Frame phases
- McBSP reception
- McBSP transmission
- Interrupts and DMA events generated by McBSPs

### 15.2.1 Data Transfer Process of McBSP

Figure 15-2 shows a diagram of the McBSP data transfer paths. The McBSP receive operation is triple-buffered, and transmit operation is double-buffered. The use of registers varies, depending on whether the defined length of each serial word is 16 bits.



**Figure 15-2. McBSP Data Transfer Paths**

#### 15.2.1.1 Data Transfer Process for Word Length of 8, 12, or 16 Bits

If the word length is 16 bits or smaller, only one 16-bit register is needed at each stage of the data transfer paths. The registers DRR2, RBR2, RSR2, DXR2, and XSR2 are not used (written, read, or shifted).

Receive data arrives on the DR pin and is shifted into receive shift register 1 (RSR1). Once a full word is received, the content of RSR1 is copied to receive buffer register 1 (RBR1) if RBR1 is not full with previous data. RBR1 is then copied to data receive register 1 (DRR1), unless the previous content of DRR1 has not been read by the CPU or the DMA controller. If the companding feature of the McBSP is implemented, the required word length is 8 bits and receive data is expanded into the appropriate format before being passed from RBR1 to DRR1. For more details about reception, see [Section 15.2.4.5](#).

Transmit data is written by the CPU or the DMA controller to data transmit register 1 (DXR1). If there is no previous data in transmit shift register (XSR1), the value in DXR1 is copied to XSR1; otherwise, DXR1 is copied to XSR1 when the last bit of the previous data is shifted out on the DX pin. If selected, the companding module compresses 16-bit data into the appropriate 8-bit format before passing it to XSR1. After transmit frame synchronization, the transmitter begins shifting bits from XSR1 to the DX pin. For more details about transmission, see [Section 15.2.4.6](#).

#### 15.2.1.2 Data Transfer Process for Word Length of 20, 24, or 32 Bits

If the word length is larger than 16 bits, two 16-bit registers are needed at each stage of the data transfer paths. The registers DRR2, RBR2, RSR2, DXR2, and XSR2 are needed to hold the most significant bits.

Receive data arrives on the DR pin and is shifted first into RSR2 and then into RSR1. Once the full word is received, the contents of RSR2 and RSR1 are copied to RBR2 and RBR1, respectively, if RBR1 is not full. Then the contents of RBR2 and RBR1 are copied to DRR2 and DRR1, respectively, unless the previous content of DRR1 has not been read by the CPU or the DMA controller. The CPU or the DMA controller must read data from DRR2 first and then from DRR1. When DRR1 is read, the next RBR-to-DRR copy occurs. For more details about reception, see [Section 15.2.4.5](#).

For transmission, the CPU or the DMA controller must write data to DXR2 first and then to DXR1. When new data arrives in DXR1, if there is no previous data in XSR1, the contents of DXR2 and DXR1 are copied to XSR2 and XSR1, respectively; otherwise, the contents of the DXRs are copied to the XSRs when the last bit of the previous data is shifted out on the DX pin. After transmit frame synchronization, the transmitter begins shifting bits from the XSRs to the DX pin. For more details about transmission, see [Section 15.2.4.6](#).

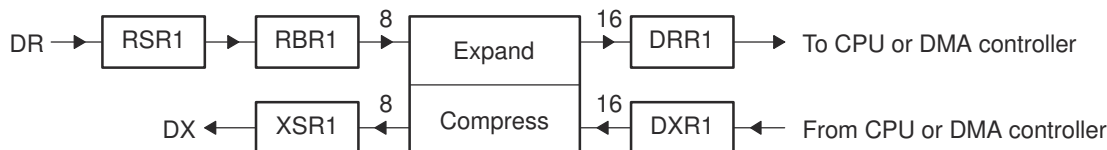
### 15.2.2 Companding (Compressing and Expanding) Data

Companding (COMpressing and exPANDING) hardware allows compression and expansion of data in either  $\mu$ -law or A-law format. The companding standard employed in the United States and Japan is  $\mu$ -law. The European companding standard is referred to as A-law. The specifications for  $\mu$ -law and A-law log PCM are part of the CCITT G.711 recommendation.

A-law and  $\mu$ -law allow 13 bits and 14 bits of dynamic range, respectively. Any values outside this range are set to the most positive or most negative value. Thus, for companding to work best, the data transferred to and from the McBSP via the CPU or DMA controller must be at least 16 bits wide.

The  $\mu$ -law and A-law formats both encode data into 8-bit code words. Companded data is always 8 bits wide; the appropriate word length bits (RWDLEN1, RWDLEN2, XWDLEN1, XWDLEN2) must therefore be set to 0, indicating an 8-bit wide serial data stream. If companding is enabled and either of the frame phases does not have an 8-bit word length, companding continues as if the word length is 8 bits.

Figure 15-3 illustrates the companding processes. When companding is chosen for the transmitter, compression occurs during the process of copying data from DXR1 to XSR1. The transmit data is encoded according to the specified companding law (A-law or  $\mu$ -law). When companding is chosen for the receiver, expansion occurs during the process of copying data from RBR1 to DRR1. The receive data is decoded to twos-complement format.

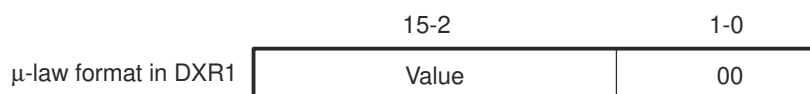


**Figure 15-3. Companding Processes**

### 15.2.2.1 Companding Formats

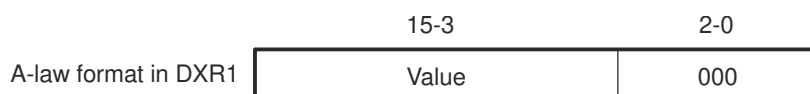
For reception, the 8-bit compressed data in RBR1 is expanded to left-justified 16-bit data in DRR1. The receive sign-extension and justification mode specified in RJUST is ignored when companding is used.

For transmission using  $\mu$ -law compression, the 14 data bits must be left-justified in DXR1 and that the remaining two low-order bits are filled with 0s as shown in Figure 15-4.



**Figure 15-4.  $\mu$ -Law Transmit Data Companding Format**

For transmission using A-law compression, the 13 data bits must be left-justified in DXR1, with the remaining three low-order bits filled with 0s as shown in Figure 15-5.



**Figure 15-5. A-Law Transmit Data Companding Format**

### 15.2.2.2 Capability to Compand Internal Data

If the McBSP is otherwise unused (the serial port transmit and receive sections are reset), the companding hardware can compand internal data. This can be used to:

- Convert linear to the appropriate  $\mu$ -law or A-law format
- Convert  $\mu$ -law or A-law to the linear format
- Observe the quantization effects in companding by transmitting linear data and compressing and re-expanding this data. This is useful only if both XCOMPAND and RCOMPAND enable the same companding format.

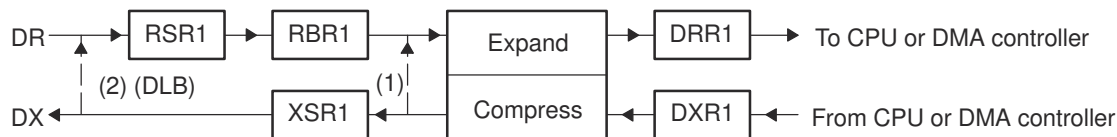
Figure 15-6 shows two methods by which the McBSP can compand internal data. Data paths for these two methods are used to indicate:

- When both the transmit and receive sections of the serial port are reset, DRR1 and DXR1 are connected internally through the companding logic. Values from DXR1 are compressed, as selected by XCOMPAND, and then expanded, as selected by RCOMPAND. RRDY and XRDY bits are not set. However, data is available in DRR1 within four CPU clocks after being written to DXR1.

The advantage of this method is its speed. The disadvantage is that there is no synchronization available to the CPU and DMA to control the flow. DRR1 and DXR1 are internally connected if the (X/R)COMPAND bits are set to 10b or 11b (compand using  $\mu$ -law or A-law).

- The McBSP is enabled in digital loopback mode with companding appropriately enabled by RCOMPAND and XCOMPAND. Receive and transmit interrupts (RINT when RINTM = 0 and XINT when XINTM = 0) or

synchronization events (REVT and XEVT) allow synchronization of the CPU or DMA to these conversions, respectively. Here, the time for this companding depends on the serial bit rate selected.



**Figure 15-6. Two Methods by Which the McBSP Can Compand Internal Data**

### 15.2.2.3 Reversing Bit Order: Option to Transfer LSB First

Generally, the McBSP transmits or receives all data with the most significant bit (MSB) first. However, certain 8-bit data protocols (that do not use companded data) require the least significant bit (LSB) to be transferred first. If you set XCOMPAND = 01b in XCR2, the bit ordering of 8-bit words is reversed (LSB first) before being sent from the serial port. If you set RCOMPAND = 01b in RCR2, the bit ordering of 8-bit words is reversed during reception. Similar to companding, this feature is enabled only if the appropriate word length bits are set to 0, indicating that 8-bit words are to be transferred serially. If either phase of the frame does not have an 8-bit word length, the McBSP assumes the word length is eight bits, and LSB-first ordering is done.

## 15.2.3 Clocking and Framing Data

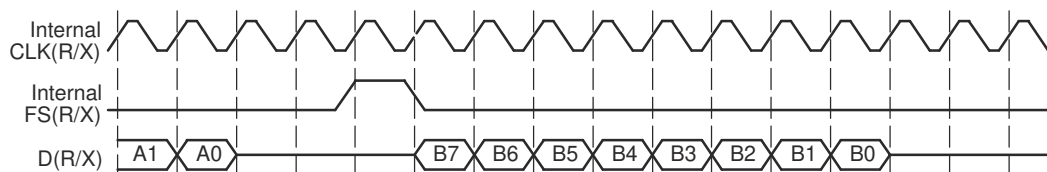
This section explains basic concepts and terminology important for understanding how McBSP data transfers are timed and delimited.

### 15.2.3.1 Clocking

Data is shifted one bit at a time from the DR pin to the RSR(s) or from the XSR(s) to the DX pin. The time for each bit transfer is controlled by the rising or falling edge of a clock signal.

The receive clock signal (CLKR) controls bit transfers from the DR pin to the RSR(s). The transmit clock signal (CLKX) controls bit transfers from the XSR(s) to the DX pin. CLKR or CLKX can be derived from a pin at the boundary of the McBSP or derived from inside the McBSP. The polarities of CLKR and CLKX are programmable.

In the example in Figure 15-7, the clock signal controls the timing of each bit transfer on the pin.



**Figure 15-7. Example - Clock Signal Control of Bit Transfer Timing**

### Note

The McBSP cannot operate at a frequency faster than  $\frac{1}{2}$  the LSPCLK frequency. When driving CLKX or CLKR at the pin, choose an appropriate input clock frequency. When using the internal sample rate generator for CLKX and/or CLKR, choose an appropriate input clock frequency and divide down value (CLKDV) (i.e., be certain that  $\text{CLKX or CLKR} \leq \text{LSPCLK}/2$ ).

### 15.2.3.2 Serial Words

Bits traveling between a shift register (RSR or XSR) and a data pin (DR or DX) are transferred in a group called a serial word. You can define how many bits are in a word.

Bits coming in on the DR pin are held in RSR until RSR holds a full serial word. Only then is the word passed to RBR (and ultimately to the DRR).

During transmission, XSR does not accept new data from DXR until a full serial word has been passed from XSR to the DX pin.

In the example in [Figure 15-7](#), an 8-bit word size was defined (see bits 7 through 0 of word B being transferred).

### 15.2.3.3 Frames and Frame Synchronization

One or more words are transferred in a group called a frame. You can define how many words are in a frame.

All of the words in a frame are sent in a continuous stream. However, there can be pauses between frame transfers. The McBSP uses frame-synchronization signals to determine when each frame is received/transmitted. When a pulse occurs on a frame-synchronization signal, the McBSP begins receiving/transmitting a frame of data. When the next pulse occurs, the McBSP receives/transmits the next frame, and so on.

Pulses on the receive frame-synchronization (FSR) signal initiate frame transfers on DR. Pulses on the transmit frame-sync (FSX) signal initiate frame transfers on DX. FSR or FSX can be derived from a pin at the boundary of the McBSP or derived from inside the McBSP.

In the example in [Figure 15-7](#), a one-word frame is transferred when a frame-synchronization pulse occurs.

In McBSP operation, the inactive-to-active transition of the frame-synchronization signal indicates the start of the next frame. For this reason, the frame-synchronization signal may be high for an arbitrary number of clock cycles. Only after the signal is recognized to have gone inactive, and then active again, does the next frame synchronization occur.

### 15.2.3.4 Generating Transmit and Receive Interrupts

The McBSP can send receive and transmit interrupts to the CPU to indicate specific events in the McBSP. To facilitate detection of frame synchronization, these interrupts can be sent in response to frame-synchronization pulses. Set the appropriate interrupt mode bits to 10b (for reception, RINTM = 10b; for transmission, XINTM = 10b).

#### 15.2.3.4.1 Detecting Frame-Synchronization Pulses, Even in Reset State

Unlike other serial port interrupt modes, this mode can operate while the associated portion of the serial port is in reset (such as activating RINT when the receiver is in reset). In this case, FSRM/FSXM and FSRP/FSXP still select the appropriate source and polarity of frame synchronization. Thus, even when the serial port is in the reset state, these signals are synchronized to the CPU clock and then sent to the CPU in the form of RINT and XINT at the point at which they feed the receiver and transmitter of the serial port. Consequently, a new frame-synchronization pulse can be detected, and after this occurs the CPU can take the serial port out of reset safely.

### 15.2.3.5 Ignoring Frame-Synchronization Pulses

The McBSP can be configured to ignore transmit and/or receive frame-synchronization pulses. To have the receiver or transmitter recognize frame-synchronization pulses, clear the appropriate frame-synchronization ignore bit (RFIG = 0 for the receiver, XFIG = 0 for the transmitter). To have the receiver or transmitter ignore frame-synchronization pulses until the desired frame length or number of words is reached, set the appropriate frame-synchronization ignore bit (RFIG = 1 for the receiver, XFIG = 1 for the transmitter). For more details on unexpected frame-synchronization pulses, see one of the following topics:

- *Unexpected Receive Frame-Synchronization Pulse* (see [Section 15.4.3](#))
- *Unexpected Transmit Frame-Synchronization Pulse* (see [Section 15.4.5](#))

You can also use the frame-synchronization ignore function for data packing (for more details, see [Section 15.10.2](#)).

### 15.2.3.6 Frame Frequency

The frame frequency is determined by the period between frame-synchronization pulses and is defined as shown by Equation 10.

$$\text{Frame Frequency} = \frac{\text{Clock Frequency}}{\text{Number of Clock Cycles Between Frame- Sync Pulses}} \quad (10)$$

The frame frequency can be increased by decreasing the time between frame-synchronization pulses (limited only by the number of bits per frame). As the frame transmit frequency increases, the inactivity period between the data packets for adjacent transfers decreases to zero.

### 15.2.3.7 Maximum Frame Frequency

The minimum number of clock cycles between frame synchronization pulses is equal to the number of bits transferred per frame. The maximum frame frequency is defined as shown by Equation 11.

$$\text{Maximum Frame Frequency} = \frac{\text{Clock Frequency}}{\text{Number of Bits Per Frame}} \quad (11)$$

Figure 15-8 shows the McBSP operating at maximum packet frequency. At maximum packet frequency, the data bits in consecutive packets are transmitted contiguously with no inactivity between bits.

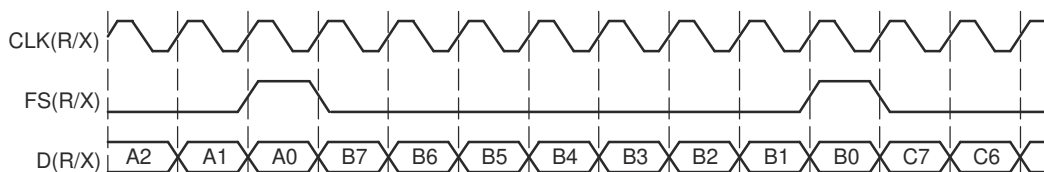


Figure 15-8. McBSP Operating at Maximum Packet Frequency

If there is a 1-bit data delay as shown in this figure, the frame-synchronization pulse overlaps the last bit transmitted in the previous frame. Effectively, this permits a continuous stream of data, making frame-synchronization pulses redundant. Theoretically, only an initial frame-synchronization pulse is required to initiate a multipacket transfer.

The McBSP supports operation of the serial port in this fashion by ignoring the successive frame-synchronization pulses. Data is clocked into the receiver or clocked out of the transmitter during every clock cycle.

#### Note

For XDATDLY = 0 (0-bit data delay), the first bit of data is transmitted asynchronously to the internal transmit clock signal (CLKX). For more details, see Section 15.8.12.

## 15.2.4 Frame Phases

The McBSP allows you to configure each frame to contain one or two phases. The number of words and the number of bits per word can be specified differently for each of the two phases of a frame, allowing greater flexibility in structuring data transfers. For example, you might define a frame as consisting of one phase containing two words of 16 bits each, followed by a second phase consisting of 10 words of 8 bits each. This configuration permits you to compose frames for custom applications or, in general, to maximize the efficiency of data transfers.

### 15.2.4.1 Number of Phases, Words, and Bits Per Frame

Table 15-2 shows which bit-fields in the receive control registers (RCR1 and RCR2) and in the transmit control registers (XCR1 and XCR2) determine the number of phases per frame, the number of words per frame, and number of bits per word for each phase, for the receiver and transmitter. The maximum number of words per



frame is 128 for a single-phase frame and 256 for a dual-phase frame. The number of bits per word can be 8, 12, 16, 20, 24, or 32 bits.

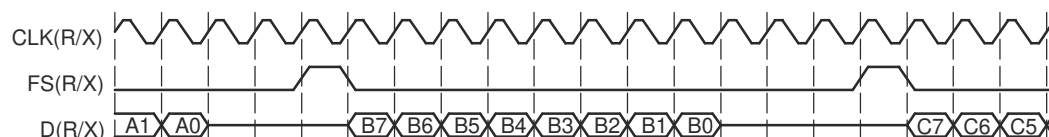
**Table 15-2. Register Bits That Determine the Number of Phases, Words, and Bits**

| Operation    | Number of Phases | Words per Frame Set With | Bits per Word Set With                     |
|--------------|------------------|--------------------------|--------------------------------------------|
| Reception    | 1 (RPHASE = 0)   | RFRLN1                   | RWDLEN1                                    |
| Reception    | 2 (RPHASE = 1)   | RFRLN1 and RFRLN2        | RWDLEN1 for phase 1<br>RWDLEN2 for phase 2 |
| Transmission | 1 (XPHASE = 0)   | XFRLN1                   | XWDLEN1                                    |
| Transmission | 2 (XPHASE = 1)   | XFRLN1 and XFRLN2        | XWDLEN1 for phase 1<br>XWDLEN2 for phase 2 |

### 15.2.4.2 Single-Phase Frame Example

Figure 15-9 shows an example of a single-phase data frame containing one 8-bit word. Because the transfer is configured for one data bit delay, the data on the DX and DR pins are available one clock cycle after FS(R/X) goes active. The figure makes the following assumptions:

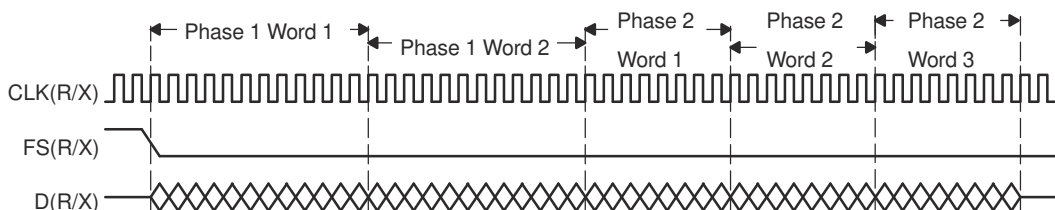
- (R/X)PHASE = 0: Single-phase frame
- (R/X)FRLN1 = 0b: 1 word per frame
- (R/X)WDLEN1 = 000b: 8-bit word length
- (R/X)FRLN2 and (R/X)WDLEN2 are ignored
- CLK(X/R)P = 0: Receive data clocked on falling edge; transmit data clocked on rising edge
- FS(R/X)P = 0: Active-high frame-synchronization signals
- (R/X)DATDLY = 01b: 1-bit data delay



**Figure 15-9. Single-Phase Frame for a McBSP Data Transfer**

### 15.2.4.3 Dual-Phase Frame Example

Figure 15-10 shows an example of a frame where the first phase consists of two words of 12 bits each, followed by a second phase of three words of 8 bits each. The entire bit stream in the frame is contiguous. There are no gaps either between words or between phases.



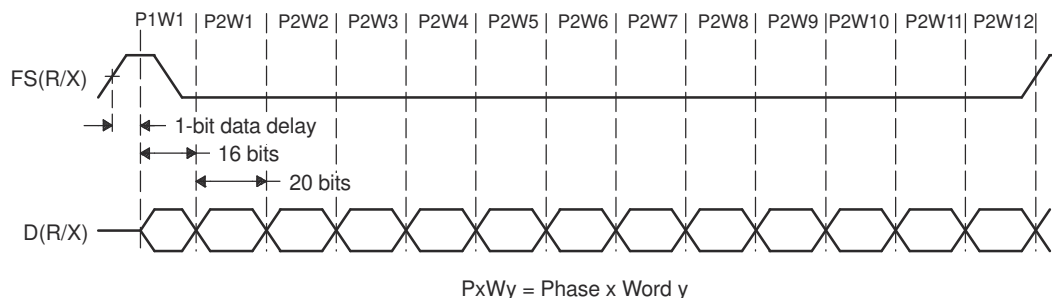
A. XRDY gets asserted once per phase. So, if there are 2 phases, XRDY gets asserted twice (once per phase).

**Figure 15-10. Dual-Phase Frame for a McBSP Data Transfer**

### 15.2.4.4 Implementing the AC97 Standard With a Dual-Phase Frame

Figure 15-11 shows an example of the Audio Codec '97 (AC97) standard, which uses the dual-phase frame feature. Notice that words, not individual bits, are shown on the D(R/X) signal. The first phase (P1) consists of a single 16-bit word. The second phase (P2) consists of twelve 20-bit words. The phase configurations are listed after the figure.

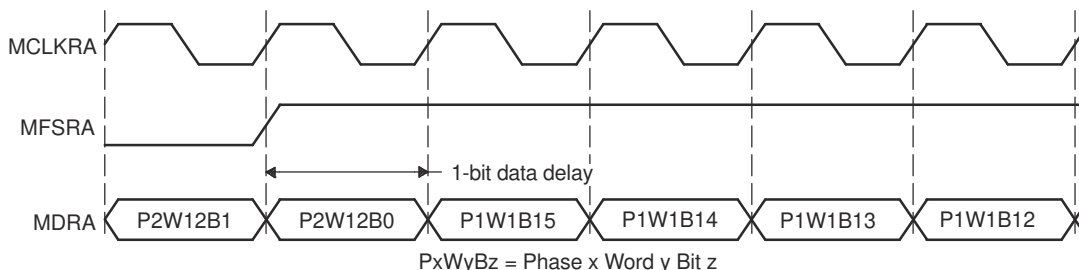




**Figure 15-11. Implementing the AC97 Standard With a Dual-Phase Frame**

- (R/X)PHASE = 1: Dual-phase frame
- (R/X)FRLN1 = 0000000b: 1 word in phase 1
- (R/X)WDLEN1 = 010b: 16 bits per word in phase 1
- (R/X)FRLN2 = 0001011b: 12 words in phase 2
- (R/X)WDLEN2 = 011b: 20 bits per word in phase 2
- CLKRP/CLKXP= 0: Receive data sampled on falling edge of internal CLKR / transmit data clocked on rising edge of internal CLKX
- FSRP/FSXP = 0: Active-high frame-sync signal
- (R/X)DATDLY = 01b: Data delay of 1 clock cycle (1-bit data delay)

Figure 15-12 shows the timing of an AC97-standard data transfer near frame synchronization. In this figure, individual bits are shown on D(R/X). Specifically, the figure shows the last two bits of phase 2 of one frame and the first four bits of phase 1 of the next frame. Regardless of the data delay, data transfers can occur without gaps. The first bit of the second frame (P1W1B15) immediately follows the last bit of the first frame (P2W12B0). Because a 1-bit data delay has been chosen, the transition on the frame-sync signal can occur when P2W12B0 is transferred.

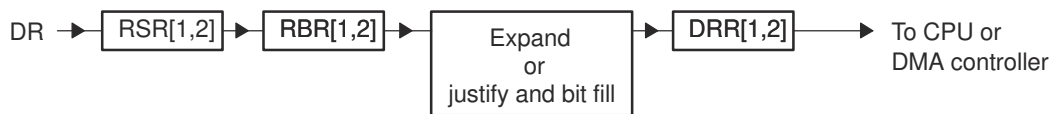


**Figure 15-12. Timing of an AC97-Standard Data Transfer Near Frame Synchronization**

#### 15.2.4.5 McBSP Reception

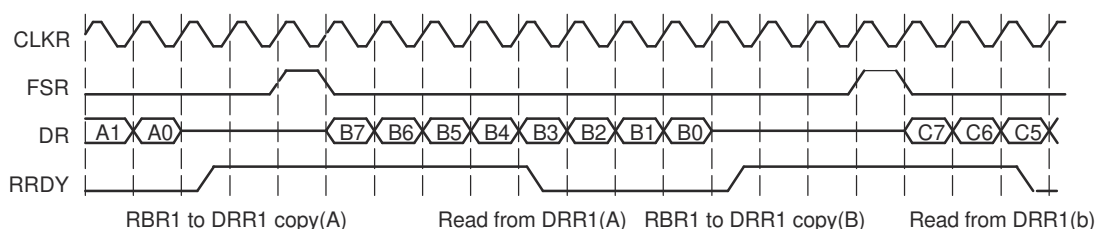
This section explains the fundamental process of reception in the McBSP. For details about how to program the McBSP receiver, see [Section 15.7](#).

Figure 15-13 and Figure 15-14 show how reception occurs in the McBSP. Figure 15-13 shows the physical path for the data. Figure 15-14 is a timing diagram showing signal activity for one possible reception scenario. A description of the process follows the figures.



- RSR[1,2]: Receive shift registers 1 and 2
- RBR[1,2]: Receive buffer registers 1 and 2
- DRR[1,2]: Data receive registers 1 and 2

**Figure 15-13. McBSP Reception Physical Data Path**



- A. CLKR: Internal receive clock  
 B. FSR: Internal receive frame-synchronization signal  
 C. DR: Data on DR pin  
 D. RRDY: Status of receiver ready bit (high is 1)

**Figure 15-14. McBSP Reception Signal Activity**

The following process describes how data travels from the DR pin to the CPU or to the DMA controller:

1. The McBSP waits for a receive frame-synchronization pulse on internal FSR.
2. When the pulse arrives, the McBSP inserts the appropriate data delay that is selected with the RDATDLY bits of RCR2.

In the preceding timing diagram (Figure 15-14), a 1-bit data delay is selected.

3. The McBSP accepts data bits on the DR pin and shifts them into the receive shift registers.

If the word length is 16 bits or smaller, only RSR1 is used. If the word length is larger than 16 bits, RSR2 and RSR1 are used and RSR2 contains the most significant bits. For details on choosing a word length, see Section 15.7.8.

4. When a full word is received, the McBSP copies the contents of the receive shift registers to the receive buffer registers, provided that RBR1 is not full with previous data.

If the word length is 16 bits or smaller, only RBR1 is used. If the word length is larger than 16 bits, RBR2 and RBR1 are used and RBR2 contains the most significant bits.

5. The McBSP copies the contents of the receive buffer registers into the data receive registers, provided that DRR1 is not full with previous data. When DRR1 receives new data, the receiver ready bit (RRDY) is set in SPCR1. This indicates that received data is ready to be read by the CPU or the DMA controller.

If the word length is 16 bits or smaller, only DRR1 is used. If the word length is larger than 16 bits, DRR2 and DRR1 are used and DRR2 contains the most significant bits.

If companding is used during the copy (RCOMPAND = 10b or 11b in RCR2), the 8-bit compressed data in RBR1 is expanded to a left-justified 16-bit value in DRR1. If companding is disabled, the data copied from RBR[1,2] to DRR[1,2] is justified and bit filled according to the RJUST bits.

6. The CPU or the DMA controller reads the data from the data receive registers. When DRR1 is read, RRDY is cleared and the next RBR-to-DRR copy is initiated.

#### Note

If both DRRs are required (word length larger than 16 bits), the CPU or the DMA controller must read from DRR2 first and then from DRR1. As soon as DRR1 is read, the next RBR-to-DRR copy is initiated. If DRR2 is not read first, the data in DRR2 is lost.

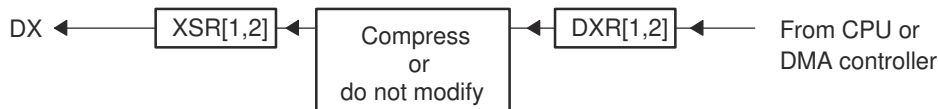
When activity is not properly timed, errors can occur. See the following topics for more details:

- *Overflow in the Receiver* (see Section 15.4.2)
- *Unexpected Receive Frame-Synchronization Pulse* (see Section 15.4.3)

#### 15.2.4.6 McBSP Transmission

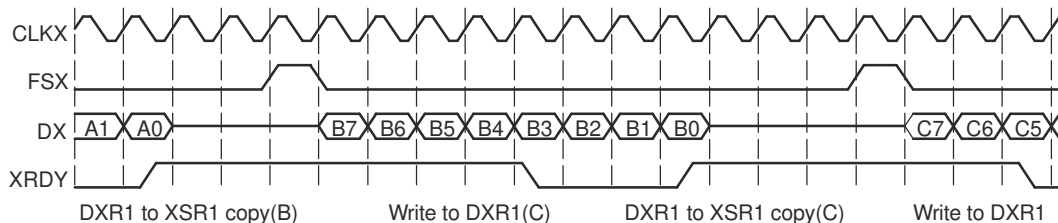
This section explains the fundamental process of transmission in the McBSP. For details about how to program the McBSP transmitter, see Section 15.8.

Figure 15-15 and Figure 15-16 show how transmission occurs in the McBSP. Figure 15-15 shows the physical path for the data. Figure 15-16 is a timing diagram showing signal activity for one possible transmission scenario. A description of the process follows the figures.



- A. XSR[1,2]: Transmit shift registers 1 and 2
- B. DXR[1,2]: Data transmit registers 1 and 2

**Figure 15-15. McBSP Transmission Physical Data Path**



- A. CLKX: Internal transmit clock
- B. FSX: Internal transmit frame-synchronization signal
- C. DX: Data on DX pin
- D. XRDY: Status of transmitter ready bit (high is 1)

**Figure 15-16. McBSP Transmission Signal Activity**

- The CPU or the DMA controller writes data to the data transmit register(s). When DXR1 is loaded, the transmitter ready bit (XRDY) is cleared in SPCR2 to indicate that the transmitter is not ready for new data.

If the word length is 16 bits or smaller, only DXR1 is used. If the word length is larger than 16 bits, DXR2 and DXR1 are used and DXR2 contains the most significant bits. For details on choosing a word length, see [Section 15.8.8](#).

#### Note

If both DXRs are needed (word length larger than 16 bits), the CPU or the DMA controller must load DXR2 first and then load DXR1. As soon as DXR1 is loaded, the contents of both DXRs are copied to the transmit shift registers (XSRs), as described in the next step. If DXR2 is not loaded first, the previous content of DXR2 is passed to the XSR2.

- When new data arrives in DXR1, the McBSP copies the content of the data transmit register(s) to the transmit shift register(s). In addition, the transmit ready bit (XRDY) is set. This indicates that the transmitter is ready to accept new data from the CPU or the DMA controller.

If the word length is 16 bits or smaller, only XSR1 is used. If the word length is larger than 16 bits, XSR2 and XSR1 are used and XSR2 contains the most significant bits.

If companding is used during the transfer (XCOMPAND = 10b or 11b in XCR2), the McBSP compresses the 16-bit data in DXR1 to 8-bit data in the  $\mu$ -law or A-law format in XSR1. If companding is disabled, the McBSP passes data from the DXR(s) to the XSR(s) without modification.

- The McBSP waits for a transmit frame-synchronization pulse on internal FSX.
- When the pulse arrives, the McBSP inserts the appropriate data delay that is selected with the XDATDLY bits of XCR2.

In the preceding timing diagram (Figure 15-16), a 1-bit data delay is selected.

- The McBSP shifts data bits from the transmit shift register(s) to the DX pin.

When activity is not properly timed, errors can occur. See the following topics for more details:

- Overwrite in the Transmitter* ([Section 15.4.4](#))
- Underflow in the Transmitter* ([Section 15.4.4.3](#))

- [Unexpected Transmit Frame-Synchronization Pulse \(Section 15.4.5\)](#)

### 15.2.4.7 Interrupts and DMA Events Generated by a McBSP

The McBSP sends notification of important events to the CPU and the DMA controller via the internal signals shown in [Table 15-3](#).

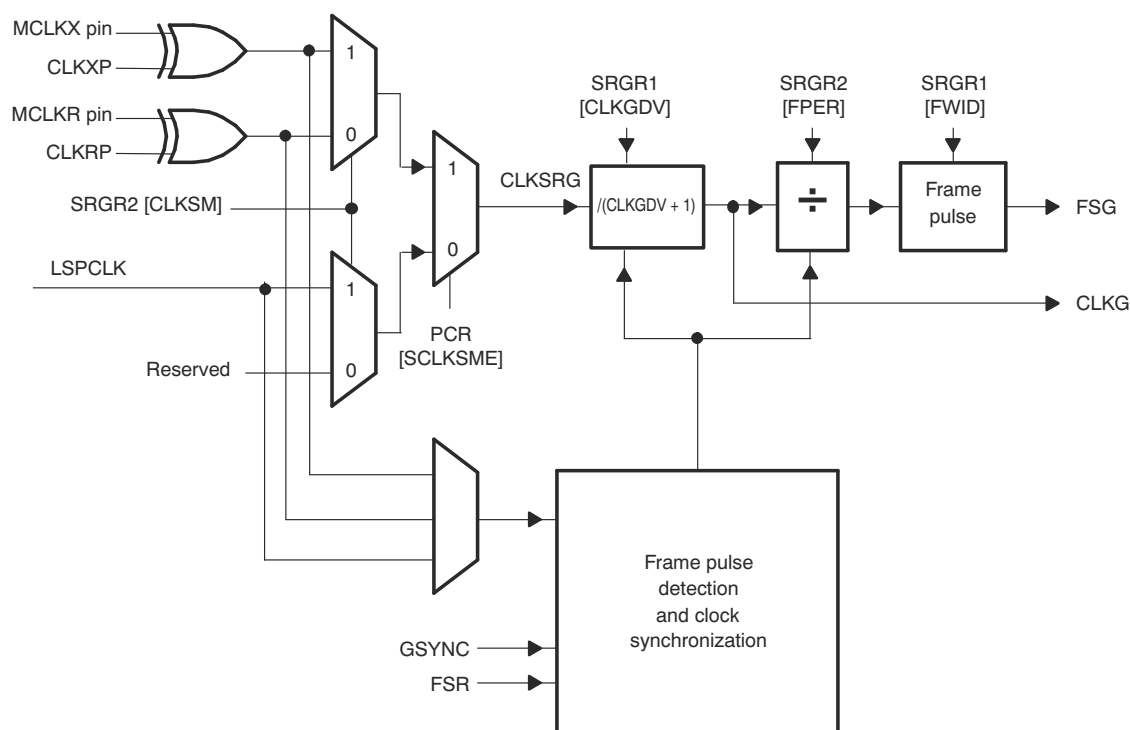
**Table 15-3. Interrupts and DMA Events Generated by a McBSP**

| Internal Signal | Description                                                                                                                                                                                         |
|-----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| RINT            | Receive interrupt<br><br>The McBSP can send a receive interrupt request to CPU based upon a selected condition in the receiver of the McBSP (a condition selected by the RINTM bits of SPCR1).      |
| XINT            | Transmit interrupt<br><br>The McBSP can send a transmit interrupt request to CPU based upon a selected condition in the transmitter of the McBSP (a condition selected by the XINTM bits of SPCR2). |
| REVT            | Receive synchronization event<br><br>An REVT signal is sent to the DMA when data has been received in the data receive registers (DRRs).                                                            |
| XEVT            | Transmit synchronization event<br><br>An XEVT signal is sent to the DMA when the data transmit registers (DXRs) are ready to accept the next serial word for transmission.                          |

## 15.3 McBSP Sample Rate Generator

Each McBSP contains a sample rate generator (SRG) that can be programmed to generate an internal data clock (CLKG) and an internal frame-synchronization signal (FSG). CLKG can be used for bit shifting on the data receive (DR) pin and/or the data transmit (DX) pin. FSG can be used to initiate frame transfers on DR and/or DX. [Figure 15-17](#) is a conceptual block diagram of the sample rate generator.

### 15.3.1 Block Diagram



**Figure 15-17. Conceptual Block Diagram of the Sample Rate Generator**

The source clock for the sample rate generator (labeled CLKSRG in the diagram) can be supplied by the LSPCLK, or by an external pin (MCLKX or MCLKR). The source is selected with the SCLKME bit of PCR and

the CLKSM bit of SRGR2. If a pin is used, the polarity of the incoming signal can be inverted with the appropriate polarity bit (CLKXP of PCR or CLKRP of PCR).

The sample rate generator has a three-stage clock divider that gives CLKG and FSG programmability. The three stages provide:

- Clock divide-down. The source clock is divided according to the CLKGDV bits of SRGR1 to produce CLKG.
- Frame period divide-down. CLKG is divided according to the FPER bits of SRGR2 to control the period from the start of a frame-pulse to the start of the next pulse.
- Frame-synchronization pulse-width countdown. CLKG cycles are counted according to the FWID bits of SRGR1 to control the width of each frame-synchronization pulse.

#### Note

The McBSP cannot operate at a frequency faster than  $\frac{1}{2}$  the source clock frequency. Choose an input clock frequency and a CLKGDV value such that CLKG is less than or equal to  $\frac{1}{2}$  the source clock frequency.

In addition to the three-stage clock divider, the sample rate generator has a frame-synchronization pulse detection and clock synchronization module that allows synchronization of the clock divide down with an incoming frame-synchronization pulse on the FSR pin. This feature is enabled or disabled with the GSYNC bit of SRGR2.

For details on getting the sample rate generator ready for operation, see [Section 15.3.4, Reset and Initialization Procedure](#).

#### 15.3.1.1 Clock Generation in the Sample Rate Generator

The sample rate generator can produce a clock signal (CLKG) for use by the receiver, the transmitter, or both. Use of the sample rate generator to drive clocking is controlled by the clock mode bits (CLKRM and CLKXM) in the pin control register (PCR). When a clock mode bit is set to 1 (CLKRM = 1 for reception, CLKXM = 1 for transmission), the corresponding data clock (CLKR for reception, CLKX for transmission) is driven by the internal sample rate generator output clock (CLKG).

The effects of CLKRM = 1 and CLKXM = 1 on the McBSP are partially affected by the use of the digital loopback mode and the clock stop (SPI) mode, respectively, as described in [Table 15-4](#). The digital loopback mode (described in [Section 15.7.4](#)) is selected with the DLB bit of SPCR1. The clock stop mode (described in [Section 15.6.2](#)) is selected with the CLKSTP bits of SPCR1.

When using the sample rate generator as a clock source, make sure the sample rate generator is enabled (GRST = 1).

**Table 15-4. Effects of DLB and CLKSTP on Clock Modes**

| Mode Bit Settings |                                                         | Effect                                                                                                                                                                                                  |
|-------------------|---------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CLKRM = 1         | DLB = 0<br>(Digital loopback mode disabled)             | CLKR is an output pin driven by the sample rate generator output clock (CLKG).                                                                                                                          |
|                   | DLB = 1<br>(Digital loopback mode enabled)              | CLKR is an output pin driven by internal CLKX. The source for CLKX depends on the CLKXM bit.                                                                                                            |
| CLKXM = 1         | CLKSTP = 00b or 01b<br>(Clock stop (SPI) mode disabled) | CLKX is an output pin driven by the sample rate generator output clock (CLKG).                                                                                                                          |
|                   | CLKSTP = 10b or 11b<br>(Clock stop (SPI) mode enabled)  | The McBSP is a master in an SPI system. Internal CLKX drives internal CLKR and the shift clocks of any SPI-compliant slave devices in the system. CLKX is driven by the internal sample rate generator. |

#### 15.3.1.2 Choosing an Input Clock

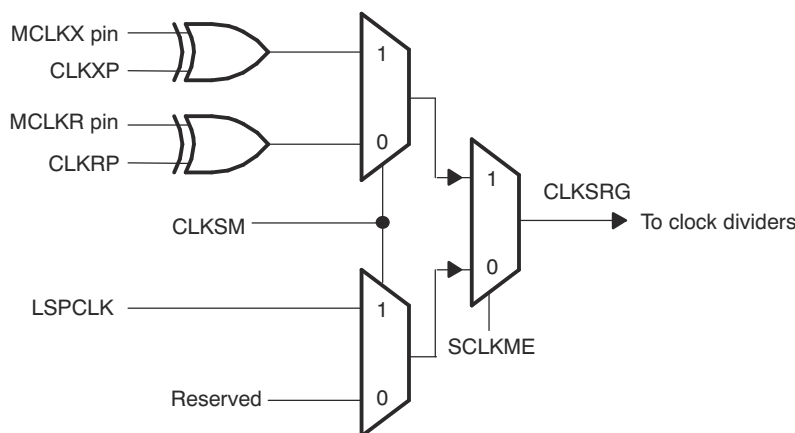
The sample rate generator must be driven by an input clock signal from one of the three sources selectable with the SCLKME bit of PCR and the CLKSM bit of SRGR2 (see [Table 15-5](#)). When CLKSM = 1, the minimum divide down value in CLKGDV bits is 1. CLKGDV is described in [Section 15.3.1.4](#).

**Table 15-5. Choosing an Input Clock for the Sample Rate Generator with the SCLKME and CLKSM Bits**

| SCLKME | CLKSM | Input Clock for Sample Rate Generator |
|--------|-------|---------------------------------------|
| 0      | 0     | Reserved                              |
| 0      | 1     | LSPCLK                                |
| 1      | 0     | Signal on MCLKR pin                   |
| 1      | 1     | Signal on MCLKX pin                   |

### 15.3.1.3 Choosing a Polarity for the Input Clock

As shown in [Figure 15-18](#), when the input clock is received from a pin, you can choose the polarity of the input clock. The rising edge of CLKSRG generates CLKG and FSG, but you can determine which edge of the input clock causes a rising edge on CLKSRG. The polarity options and their effects are described in [Table 15-6](#).


**Figure 15-18. Possible Inputs to the Sample Rate Generator and the Polarity Bits**
**Table 15-6. Polarity Options for the Input to the Sample Rate Generator**

| Input Clock         | Polarity Option          | Effect                                                           |
|---------------------|--------------------------|------------------------------------------------------------------|
| LSPCLK              | Always positive polarity | Rising edge of CPU clock generates transitions on CLKG and FSG.  |
| Signal on MCLKR pin | CLKRP = 0 in PCR         | Falling edge on MCLKR pin generates transitions on CLKG and FSG. |
|                     | CLKRP = 1 in PCR         | Rising edge on MCLKR pin generates transitions on CLKG and FSG.  |
| Signal on MCLKX pin | CLKXP = 0 in PCR         | Rising edge on MCLKX pin generates transitions on CLKG and FSG.  |
|                     | CLKXP = 1 in PCR         | Falling edge on MCLKX pin generates transitions on CLKG and FSG. |

### 15.3.1.4 Choosing a Frequency for the Output Clock (CLKG)

The input clock (LSPCLK or external clock) can be divided down by a programmable value to drive CLKG. Regardless of the source to the sample rate generator, the rising edge of CLKSRG (see [Figure 15-17](#)) generates CLKG and FSG.

The first divider stage of the sample rate generator creates the output clock from the input clock. This divider stage uses a counter that is preloaded with the divide down value in the CLKGDV bits of SRGR1. The output of this stage is the data clock (CLKG). CLKG has the frequency represented by [Equation 12](#).

$$\text{CLKG frequency} = \frac{\text{Input clock frequency}}{(\text{CLKGDV} + 1)} \quad (12)$$

Thus, the input clock frequency is divided by a value between 1 and 256. When CLKGDV is odd or equal to 0, the CLKG duty cycle is 50%. When CLKGDV is an even value, 2p, representing an odd divide down, the high-state duration is p+1 cycles and the low-state duration is p cycles.



### 15.3.1.5 Keeping CLKG Synchronized to External MCLKR

When the MCLKR pin is used to drive the sample rate generator (see [Section 15.3.1.2](#)), the GSYNC bit in SRGR2 and the FSR pin can be used to configure the timing of the output clock (CLKG) relative to the input clock. Note that this feature is available only when the MCLKR pin is used to feed the external clock.

GSYNC = 1 ensures that the McBSP and an external device are dividing down the input clock with the same phase relationship. If GSYNC = 1, an inactive-to-active transition on the FSR pin triggers a resynchronization of CLKG and generation of FSG.

For more details about synchronization, see [Section 15.3.3](#).

### 15.3.2 Frame Synchronization Generation in the Sample Rate Generator

The sample rate generator can produce a frame-synchronization signal (FSG) for use by the receiver, the transmitter, or both.

If you want the receiver to use FSG for frame synchronization, make sure FSRM = 1. (When FSRM = 0, receive frame synchronization is supplied via the FSR pin.)

If you want the transmitter to use FSG for frame synchronization, you must set:

- FSXM = 1 in PCR: This indicates that transmit frame synchronization is supplied by the McBSP itself rather than from the FSX pin.
- FSGM = 1 in SRGR2: This indicates that when FSXM = 1, transmit frame synchronization is supplied by the sample rate generator. (When FSGM = 0 and FSXM = 1, the transmitter uses frame-synchronization pulses generated every time data is transferred from DXR[1,2] to XSR[1,2].)

In either case, the sample rate generator must be enabled (GRST = 1) and the frame-synchronization logic in the sample rate generator must be enabled (FRST = 0).

#### 15.3.2.1 Choosing the Width of the Frame-Synchronization Pulse on FSG

Each pulse on FSG has a programmable width. You program the FWID bits of SRGR1, and the resulting pulse width is (FWID + 1) CLKG cycles, where CLKG is the output clock of the sample rate generator.

#### 15.3.2.2 Controlling the Period Between the Starting Edges of Frame-Synchronization Pulses on FSG

You can control the amount of time from the starting edge of one FSG pulse to the starting edge of the next FSG pulse. This period is controlled in one of two ways, depending on the configuration of the sample rate generator:

- If the sample rate generator is using an external input clock and GSYNC = 1 in SRGR2, FSG pulses in response to an inactive-to-active transition on the FSR pin. Thus, the frame-synchronization period is controlled by an external device.
- Otherwise, you program the FPER bits of SRGR2, and the resulting frame-synchronization period is (FPER + 1) CLKG cycles, where CLKG is the output clock of the sample rate generator.

#### 15.3.2.3 Keeping FSG Synchronized to an External Clock

When an external signal is selected to drive the sample rate generator (see [Section 15.3.1.2](#) on page [Section 15.3.1.2](#)), the GSYNC bit of SRGR2 and the FSR pin can be used to configure the timing of FSG pulses.

GSYNC = 1 ensures that the McBSP and an external device are dividing down the input clock with the same phase relationship. If GSYNC = 1, an inactive-to-active transition on the FSR pin triggers a resynchronization of CLKG and generation of FSG.

See [Section 15.3.3](#) for more details about synchronization.

### 15.3.3 Synchronizing Sample Rate Generator Outputs to an External Clock

The sample rate generator can produce a clock signal (CLKG) and a frame-synchronization signal (FSG) based on an input clock signal that is either the CPU clock signal or a signal at the MCLKR or MCLKX pin. When an external clock is selected to drive the sample rate generator, the GSYNC bit of SRGR2 and the FSR pin can be used to control the timing of CLKG and the pulsing of FSG relative to the chosen input clock.

Make GSYNC = 1 when you want the McBSP and an external device to divide down the input clock with the same phase relationship. If GSYNC = 1:

- An inactive-to-active transition on the FSR pin triggers a resynchronization of CLKG and a pulsing of FSG.
- CLKG always begins with a high state after synchronization.
- FSR is always detected at the same edge of the input clock signal that generates CLKG, no matter how long the FSR pulse is.
- The FPER bits of SRGR2 are ignored because the frame-synchronization period on FSG is determined by the arrival of the next frame-synchronization pulse on the FSR pin.

If GSYNC = 0, CLKG runs freely and is not resynchronized, and the frame-synchronization period on FSG is determined by FPER.

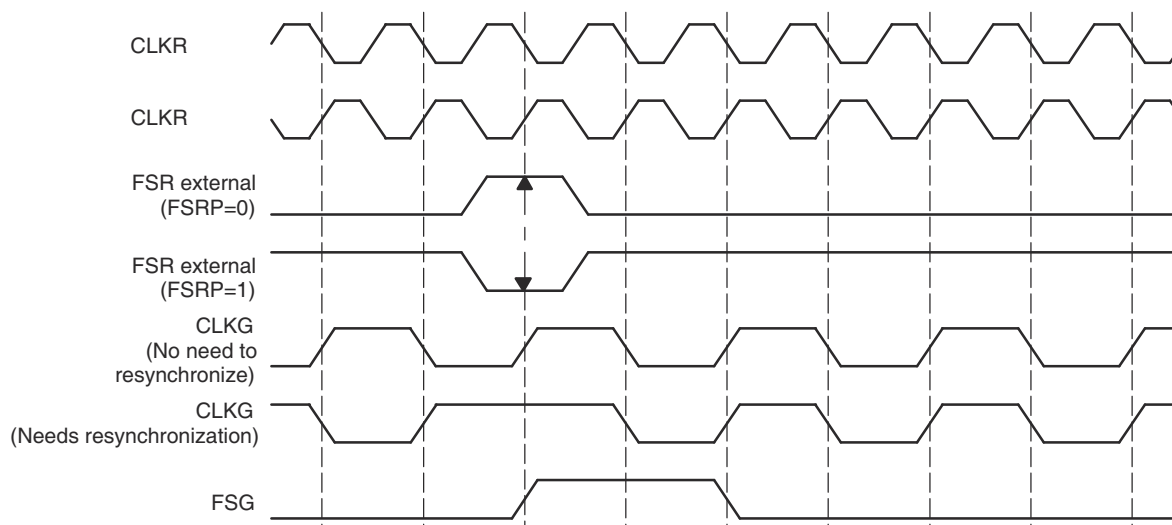
### 15.3.3.1 Operating the Transmitter Synchronously with the Receiver

When GSYNC = 1, the transmitter can operate synchronously with the receiver, provided that:

- FSX is programmed to be driven by FSG (FSGM = 1 in SRGR2 and FSXM = 1 in PCR). If the input FSR has appropriate timing so that it can be sampled by the falling edge of CLKG, it can be used, instead, by setting FSXM = 0 and connecting FSR to FSX externally.
- The sample rate generator clock drives the transmit and receive clocking (CLKRM = CLKXM = 1 in PCR).

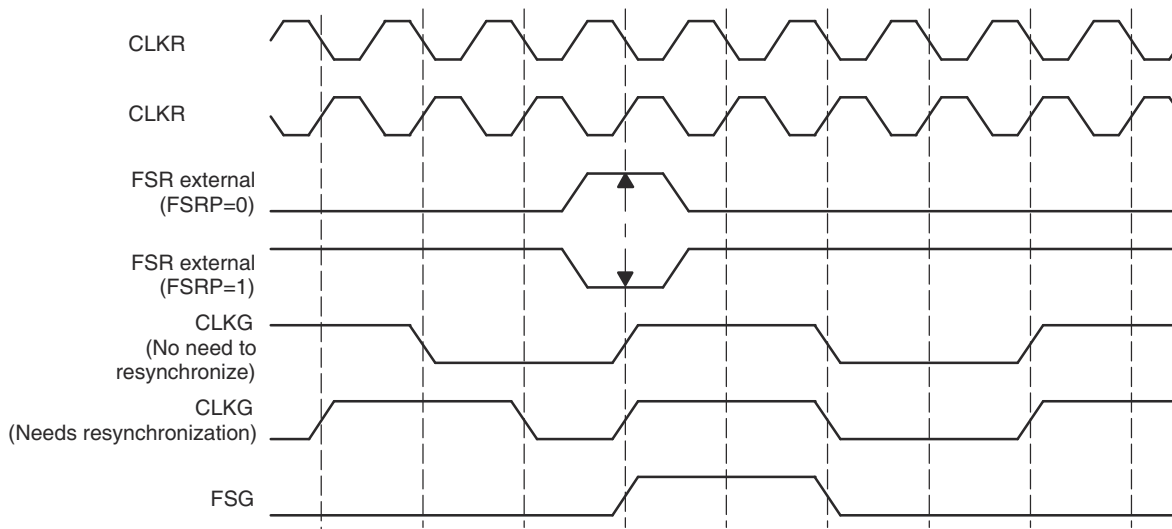
### 15.3.3.2 Synchronization Examples

Figure 15-19 and Figure 15-20 show the clock and frame-synchronization operation with various polarities of CLKR and FSR. These figures assume FWID = 0 in SRGR1, for an FSG pulse that is one CLKG cycle wide. The FPER bits of SRGR2 are not programmed; the period from the start of a frame-synchronization pulse to the start of the next pulse is determined by the arrival of the next inactive-to-active transition on the FSR pin. Each of the figures shows what happens to CLKG when it is initially synchronized and GSYNC = 1, and when it is not initially synchronized and GSYNC = 1. The second figure has a slower CLKG frequency (it has a larger divide-down value in the CLKGDV bits of SRGR1).



**Figure 15-19. CLKG Synchronization and FSG Generation When GSYNC = 1 and CLKGDV = 1**





**Figure 15-20. CLKG Synchronization and FSG Generation When GSYNC = 1 and CLKGDV = 3**

### 15.3.4 Reset and Initialization Procedure for the Sample Rate Generator

To reset and initialize the sample rate generator:

1. Place the McBSP/sample rate generator in reset.

During a DSP reset, the sample rate generator, the receiver, and the transmitter reset bits (GRST, RRST, and XRST) are automatically forced to 0. Otherwise, during normal operation, the sample rate generator can be reset by making GRST = 0 in SPCR2, provided that CLKG and/or FSG is not used by any portion of the McBSP. Depending on your system you may also want to reset the receiver (RRST = 0 in SPCR1) and reset the transmitter (XRST = 0 in SPCR2).

If GRST = 0 due to a device reset, CLKG is driven by the CPU clock divided by 2, and FSG is driven inactive-low. If GRST = 0 due to program code, CLKG and FSG are driven low (inactive).

2. Program the registers that affect the sample rate generator.

Program the sample rate generator registers (SRGR1 and SRGR2) as required for your application. If necessary, other control registers can be loaded with desired values, provided the respective portion of the McBSP (the receiver or transmitter) is in reset.

After the sample rate generator registers are programmed, wait 2 CLKSRG cycles. This ensures proper synchronization internally.

3. Enable the sample rate generator (take it out of reset).

In SPCR2, make GRST = 1 to enable the sample rate generator.

After the sample rate generator is enabled, wait two CLKG cycles for the sample rate generator logic to stabilize.

On the next rising edge of CLKSRG, CLKG transitions to 1 and starts clocking with a frequency equal to [Equation 13](#).

$$\text{CLKG frequency} = \frac{\text{Input clock frequency}}{(\text{CLKGDV} + 1)} \quad (13)$$

where the input clock is selected with the SCLKME bit of PCR and the CLKSM bit of SRGR2 in one of the configurations shown in [Table 15-7](#).

4. If necessary, enable the receiver and the transmitter.

If necessary, remove the receiver and transmitter from reset by setting RRST and/or XRST = 1.

5. If necessary, enable the frame-synchronization logic of the sample rate generator.

After the required data acquisition setup is done (DXR[1,2] is loaded with data), set GRST = 1 in SPCR2 if an internally generated frame-synchronization pulse is required. FSG is generated with an active-high edge after the programmed number of CLKG clocks (FPER + 1) have elapsed.

**Table 15-7. Input Clock Selection for Sample Rate Generator**

| SCLKME | CLKSM | Input Clock for Sample Rate Generator |
|--------|-------|---------------------------------------|
| 0      | 0     | Reserved                              |
| 0      | 1     | LSPCLK                                |
| 1      | 0     | Signal on MCLKR pin                   |
| 1      | 1     | Signal on MCLKX pin                   |

## 15.4 McBSP Exception/Error Conditions

This section describes exception/error conditions and how to handle them.

### 15.4.1 Types of Errors

There are five serial port events that can constitute a system error:

- Receiver overrun (RFULL = 1)

This occurs when DRR1 has not been read since the last RBR-to-DRR copy. Consequently, the receiver does not copy a new word from the RBRs to the DRRs and the RSRs are now full with another new word shifted in from DR. Therefore, RFULL = 1 indicates an error condition wherein any new data that can arrive at this time on DR replaces the contents of the RSRs, and the previous word is lost. The RSRs continue to be overwritten as long as new data arrives on DR and DRR1 is not read. For more details about overrun in the receiver, see [Section 15.4.2](#).

- Unexpected receive frame-synchronization pulse (RSYNCERR = 1)

This occurs during reception when RFIG = 0 and an unexpected frame-synchronization pulse occurs. An unexpected frame-synchronization pulse is one that begins the next frame transfer before all the bits of the current frame have been received. Such a pulse causes data reception to abort and restart. If new data has been copied into the RBRs from the RSRs since the last RBR-to-DRR copy, this new data in the RBRs is lost. This is because no RBR-to-DRR copy occurs; the reception has been restarted. For more details about receive frame-synchronization errors, see [Section 15.4.3](#).

- Transmitter data overwrite

This occurs when the CPU or DMA controller overwrites data in the DXRs before the data is copied to the XSRs. The overwritten data never reaches the DX pin. For more details about overwrite in the transmitter, see [Section 15.4.4](#).

- Transmitter underflow (XEMPTY = 0)

If a new frame-synchronization signal arrives before new data is loaded into DXR1, the previous data in the DXRs is sent again. This procedure continues for every new frame-synchronization pulse that arrives until DXR1 is loaded with new data. For more details about underflow in the transmitter, see [Section 15.4.4.3](#).

- Unexpected transmit frame-synchronization pulse (XSYNCERR = 1)

This occurs during transmission when XFIG = 0 and an unexpected frame-synchronization pulse occurs. An unexpected frame-synchronization pulse is one that begins the next frame transfer before all the bits of the current frame have been transferred. Such a pulse causes the current data transmission to abort and restart. If new data has been written to the DXRs since the last DXR-to-XSR copy, the current value in the XSRs is lost. For more details about transmit frame-synchronization errors, see [Section 15.4.5](#).

### 15.4.2 Overrun in the Receiver

RFULL = 1 in SPCR1 indicates that the receiver has experienced overrun and is in an error condition. RFULL is set when all of the following conditions are met:

1. DRR1 has not been read since the last RBR-to-DRR copy (RRDY = 1).
2. RBR1 is full and an RBR-to-DRR copy has not occurred.
3. RSR1 is full and an RSR1-to-RBR copy has not occurred.

As described in [Section 15.2.4.5](#), data arriving on DR is continuously shifted into RSR1 (for word length of 16 bits or smaller) or RSR2 and RSR1 (for word length larger than 16 bits). Once a complete word is shifted into the RSRs, an RSR-to-RBR copy can occur only if the previous data in RBR1 has been copied to DRR1. The RRDY bit is set when new data arrives in DRR1 and is cleared when that data is read from DRR1. Until RRDY = 0, the next RBR-to-DRR copy does not take place, and the data is held in the RSRs. New data arriving on the DR pin is shifted into RSR(s), and the previous content of the RSR(s) is lost.

You can prevent the loss of data if DRR1 is read no later than 2.5 cycles before the end of the third word is shifted into the RSR1.

#### Note

If both DRRs are needed (word length larger than 16 bits), the CPU or the DMA controller must read from DRR2 first and then from DRR1. As soon as DRR1 is read, the next RBR-to-DRR copy is initiated. If DRR2 is not read first, the data in DRR2 is lost.

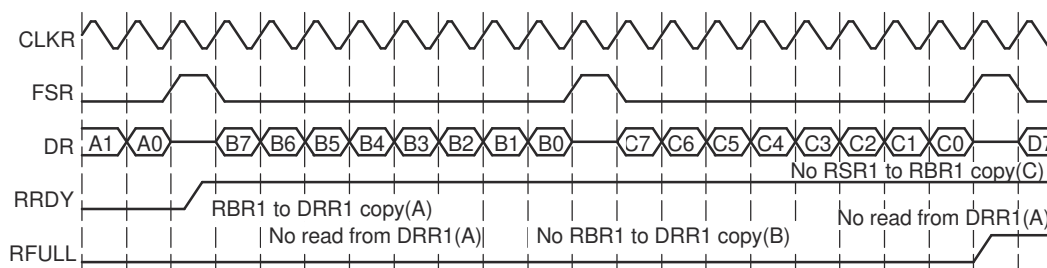
After the receiver starts running from reset, a minimum of three words must be received before RFULL is set. Either of the following events clears the RFULL bit and allows subsequent transfers to be read properly:

- The CPU or DMA controller reads DRR1.
- The receiver is reset individually (RRST = 0) or as part of a device reset.

Another frame-synchronization pulse is required to restart the receiver.

#### 15.4.2.1 Example of Overrun Condition

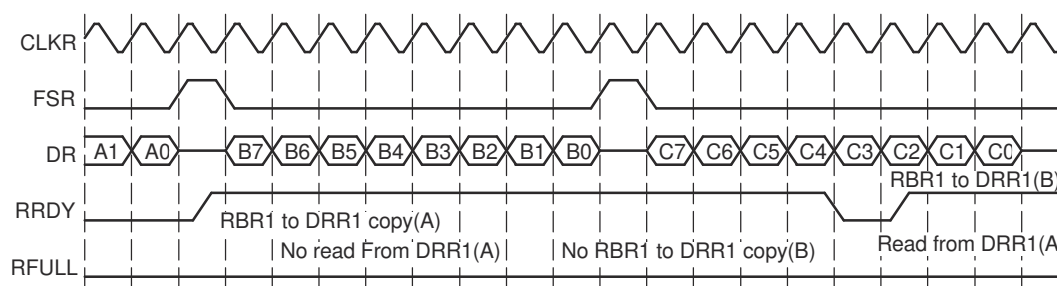
[Figure 15-21](#) shows the receive overrun condition. Because serial word A is not read from DRR1 before serial word B arrives in RBR1, B is not transferred to DRR1 yet. Another new word © arrives and RSR1 is full with this data. DRR1 is finally read, but not earlier than 2.5 cycles before the end of word C. Therefore, new data (D) overwrites word C in RSR1. If DRR1 is not read in time, the next word can overwrite D.



**Figure 15-21. Overrun in the McBSP Receiver**

#### 15.4.2.2 Example of Preventing Overrun Condition

[Figure 15-22](#) shows the case where RFULL is set, but the overrun condition is prevented by a read from DRR1 at least 2.5 cycles before the next serial word © is completely shifted into RSR1. This ensures that an RBR1-to-DRR1 copy of word B occurs before receiver attempts to transfer word C from RSR1 to RBR1.



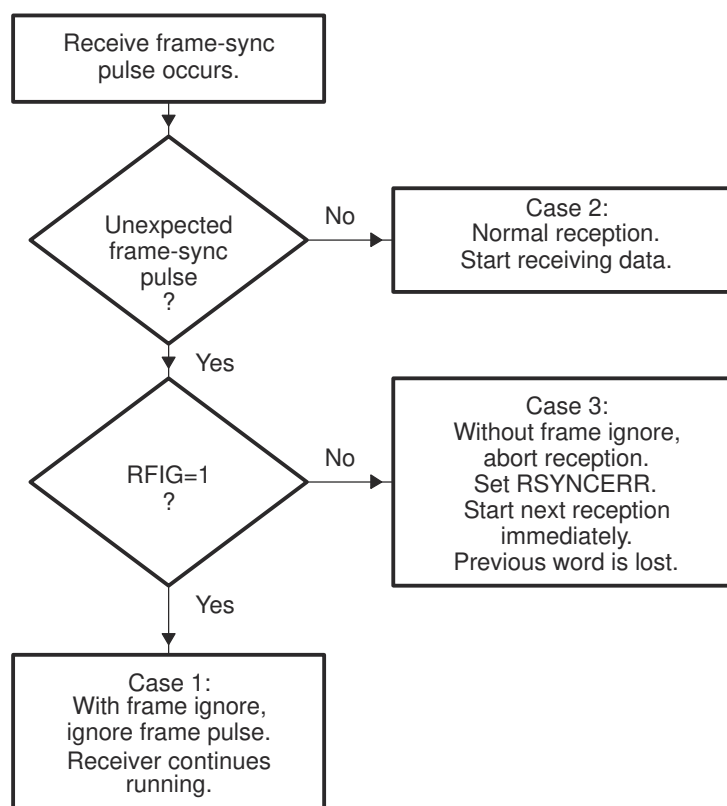
**Figure 15-22. Overrun Prevented in the McBSP Receiver**

### 15.4.3 Unexpected Receive Frame-Synchronization Pulse

Section 15.4.3.1 shows how the McBSP responds to any receive frame-synchronization pulses, including an unexpected pulse. Section 15.4.3.2 and Section 15.4.3.3 show an examples of a frame-synchronization error and an example of how to prevent such an error, respectively.

#### 15.4.3.1 Possible Responses to Receive Frame-Synchronization Pulses

Figure 15-23 shows the decision tree that the receiver uses to handle all incoming frame-synchronization pulses. The figure assumes that the receiver has been started (RRST = 1 in SPCR1). Case 3 in the figure is the case in which an error occurs.



**Figure 15-23. Possible Responses to Receive Frame-Synchronization Pulses**

Any one of three cases can occur:

- Case 1: Unexpected internal FSR pulses with RFIG = 1 in RCR2. Receive frame-synchronization pulses are ignored, and the reception continues.

- Case 2: Normal serial port reception. Reception continues normally because the frame-synchronization pulse is not unexpected. There are three possible reasons why a receive operation might *not* be in progress when the pulse occurs:
  - The FSR pulse is the first after the receiver is enabled (RRST = 1 in SPCR1).
  - The FSR pulse is the first after DRR[1,2] is read, clearing a receiver full (RFULL = 1 in SPCR1) condition.
  - The serial port is in the interpacket intervals. The programmed data delay for reception (programmed with the RDATDLY bits in RCR2) may start during these interpacket intervals for the first bit of the next word to be received. Thus, at maximum frame frequency, frame synchronization can still be received 0 to 2 clock cycles before the first bit of the synchronized frame.
- Case 3: Unexpected receive frame synchronization with RFIG = 0 (frame-synchronization pulses not ignored). Unexpected frame-synchronization pulses can originate from an external source or from the internal sample rate generator.

If a frame-synchronization pulse starts the transfer of a new frame before the current frame is fully received, this pulse is treated as an unexpected frame-synchronization pulse, and the receiver sets the receive frame-synchronization error bit (RSYNCERR) in SPCR1. RSYNCERR can be cleared only by a receiver reset or by a write of 0 to this bit.

If you want the McBSP to notify the CPU of receive frame-synchronization errors, you can set a special receive interrupt mode with the RINTM bits of SPCR1. When RINTM = 11b, the McBSP sends a receive interrupt (RINT) request to the CPU each time that RSYNCERR is set.

#### 15.4.3.2 Example of Unexpected Receive Frame-Synchronization Pulse

Figure 15-24 shows an unexpected receive frame-synchronization pulse during normal operation of the serial port, with time intervals between data packets. When the unexpected frame-synchronization pulse occurs, the RSYNCERR bit is set, the reception of data B is aborted, and the reception of data C begins. In addition, if RINTM = 11b, the McBSP sends a receive interrupt (RINT) request to the CPU.

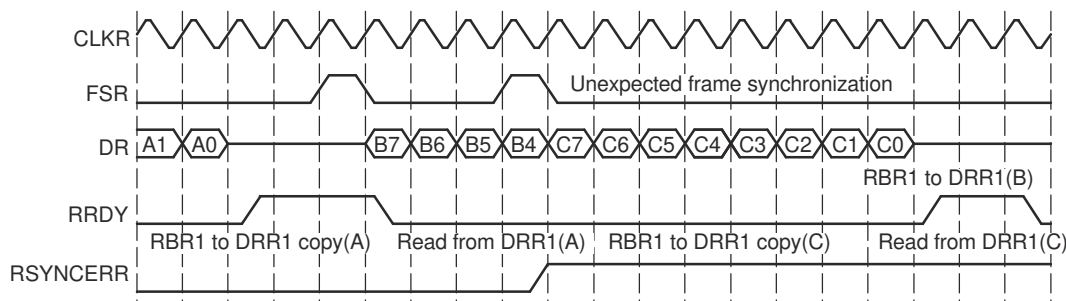
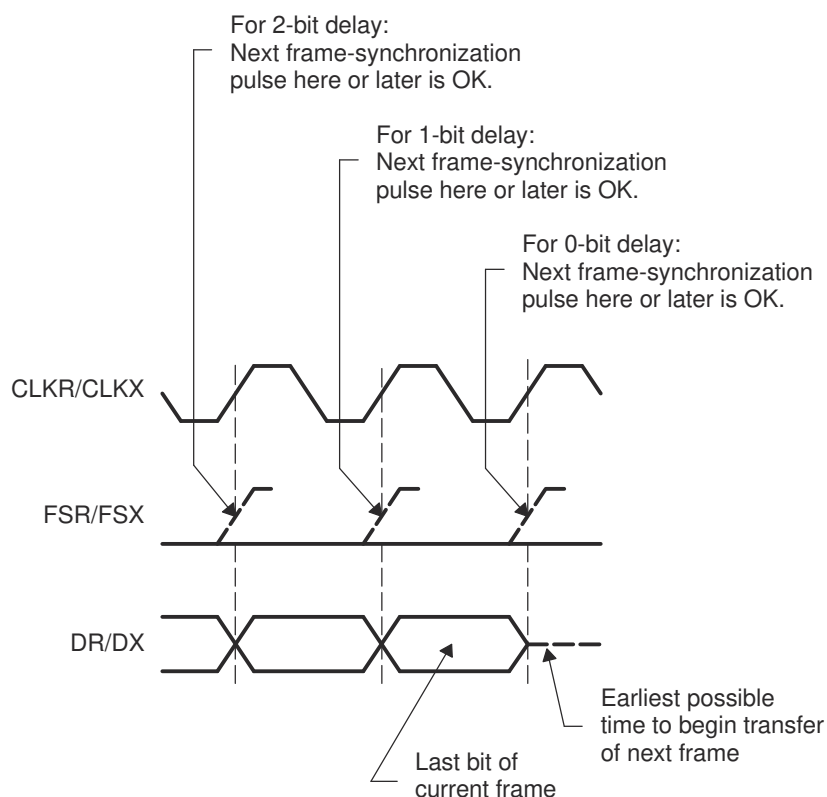


Figure 15-24. An Unexpected Frame-Synchronization Pulse During a McBSP Reception

#### 15.4.3.3 Preventing Unexpected Receive Frame-Synchronization Pulses

Each frame transfer can be delayed by 0, 1, or 2 MCLKR cycles, depending on the value in the RDATDLY bits of RCR2. For each possible data delay, Figure 15-25 shows when a new frame-synchronization pulse on FSR can safely occur relative to the last bit of the current frame.



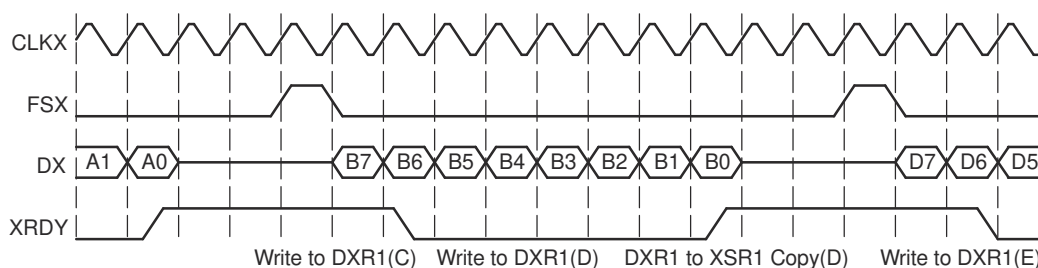
**Figure 15-25. Proper Positioning of Frame-Synchronization Pulses**

#### 15.4.4 Overwrite in the Transmitter

As described in McBSP transmission ([Section 15.2.4.6](#)), the transmitter must copy the data previously written to the DXRs by the CPU or DMA controller into the XSRs and then shift each bit from the XSRs to the DX pin. If new data is written to the DXRs before the previous data is copied to the XSRs, the previous data in the DXRs is overwritten and thus lost.

##### 15.4.4.1 Example of Overwrite Condition

[Figure 15-26](#) shows what happens if the data in DXR1 is overwritten before being transmitted. Initially, DXR1 is loaded with data C. A subsequent write to DXR1 overwrites C with D before C is copied to XSR1. Thus, C is never transmitted on DX.



**Figure 15-26. Data in the McBSP Transmitter Overwritten and Thus Not Transmitted**

##### 15.4.4.2 Preventing Overwrites

You can prevent CPU overwrites by making the CPU:

- Poll for XRDY = 1 in SPCR2 before writing to the DXR(s). XRDY is set when data is copied from DXR1 to XSR1 and is cleared when new data is written to DXR1.

- Wait for a transmit interrupt (XINT) before writing to the DXR(s). When XINTM = 00b in SPCR2, the transmitter sends XINT to the CPU each time XRDY is set.

You can prevent DMA overwrites by synchronizing DMA transfers to the transmit synchronization event XEVT. The transmitter sends an XEVT signal each time XRDY is set.

#### 15.4.4.3 Underflow in the Transmitter

The McBSP indicates a transmitter empty (or underflow) condition by clearing the  $\overline{\text{XEMPTY}}$  bit in SPCR2. Either of the following events activates  $\overline{\text{XEMPTY}}$  ( $\text{XEMPTY} = 0$ ):

- DXR1 has not been loaded since the last DXR-to-XSR copy, and all bits of the data word in the XSR(s) have been shifted out on the DX pin.
- The transmitter is reset (by forcing XRST = 0 in SPCR2, or by a device reset) and is then restarted.

In the underflow condition, the transmitter continues to transmit the old data that is in the DXR(s) for every new transmit frame-synchronization signal until a new value is loaded into DXR1 by the CPU or the DMA controller.

#### Note

If both DXRs are needed (word length larger than 16 bits), the CPU or the DMA controller must load DXR2 first and then load DXR1. As soon as DXR1 is loaded, the contents of both DXRs are copied to the transmit shift registers (XSRs). If DXR2 is not loaded first, the previous content of DXR2 is passed to the XSR2.

XEMPTY is deactivated ( $\text{XEMPTY} = 1$ ) when a new word in DXR1 is transferred to XSR1. If FSXM = 1 in PCR and FSGM = 0 in SRGR2, the transmitter generates a single internal FSX pulse in response to a DXR-to-XSR copy. Otherwise, the transmitter waits for the next frame-synchronization pulse before sending out the next frame on DX.

When the transmitter is taken out of reset ( $\text{XRST} = 1$ ), it is in a transmitter ready ( $\text{XRDY} = 1$  in SPCR2) and transmitter empty ( $\text{XEMPTY} = 0$ ) state. If DXR1 is loaded by the CPU or the DMA controller before internal FSX goes active high, a valid DXR-to-XSR transfer occurs. This allows for the first word of the first frame to be valid even before the transmit frame-synchronization pulse is generated or detected. Alternatively, if a transmit frame-synchronization pulse is detected before DXR1 is loaded, zeros are output on DX.

#### 15.4.4.3.1 Example of the Underflow Condition

Figure 15-27 shows an underflow condition. After B is transmitted, DXR1 is not reloaded before the subsequent frame-synchronization pulse. Thus, B is again transmitted on DX.

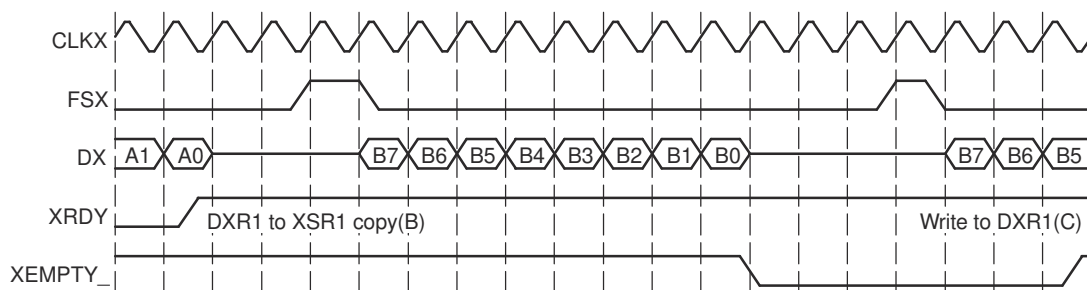
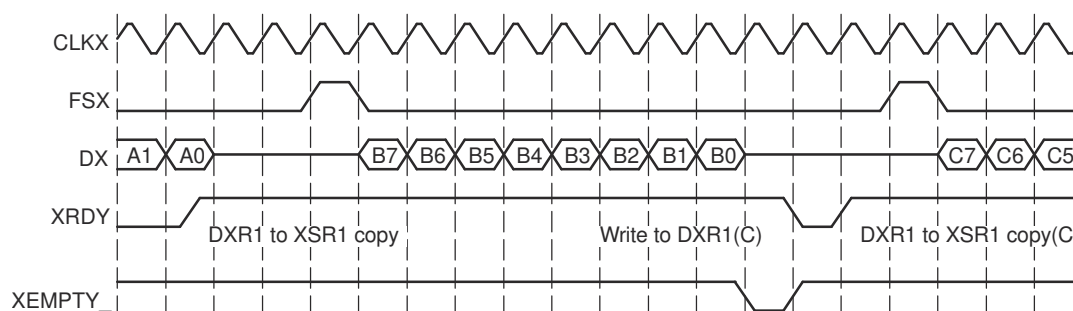


Figure 15-27. Underflow During McBSP Transmission

#### 15.4.4.3.2 Example of Preventing Underflow Condition

Figure 15-28 shows the case of writing to DXR1 just before an underflow condition would otherwise occur. After B is transmitted, C is written to DXR1 before the next frame-synchronization pulse. As a result, there is no underflow; B is not transmitted twice.





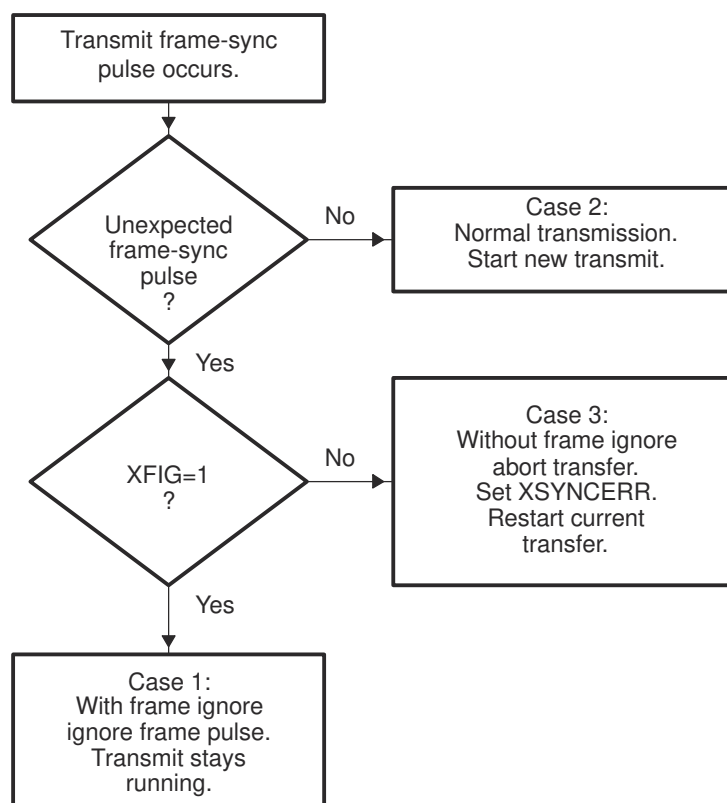
**Figure 15-28. Underflow Prevented in the McBSP Transmitter**

### 15.4.5 Unexpected Transmit Frame-Synchronization Pulse

Section 15.4.5.1 shows how the McBSP responds to any transmit frame-synchronization pulses, including an unexpected pulse. Section 15.4.5.2 and Section 15.4.5.3 show examples of a frame-synchronization error and an example of how to prevent such an error, respectively.

#### 15.4.5.1 Possible Responses to Transmit Frame-Synchronization Pulses

Figure 15-29 shows the decision tree that the transmitter uses to handle all incoming frame-synchronization pulses. The figure assumes that the transmitter has been started (XRST = 1 in SPCR2). Case 3 in the figure is the case in which an error occurs.



**Figure 15-29. Possible Responses to Transmit Frame-Synchronization Pulses**

Any one of three cases can occur:

- Case 1: Unexpected internal FSX pulses with XFIG = 1 in XCR2. Transmit frame-synchronization pulses are ignored, and the transmission continues.



- Case 2: Normal serial port transmission. Transmission continues normally because the frame-synchronization pulse is not unexpected. There are two possible reasons why a transmit operations might *not* be in progress when the pulse occurs:

This FSX pulse is the first after the transmitter is enabled (XRST = 1).

The serial port is in the interpacket intervals. The programmed data delay for transmission (programmed with the XDADLY bits of XCR2) may start during these interpacket intervals before the first bit of the previous word is transmitted. Thus, at maximum packet frequency, frame synchronization can still be received 0 to 2 clock cycles before the first bit of the synchronized frame.

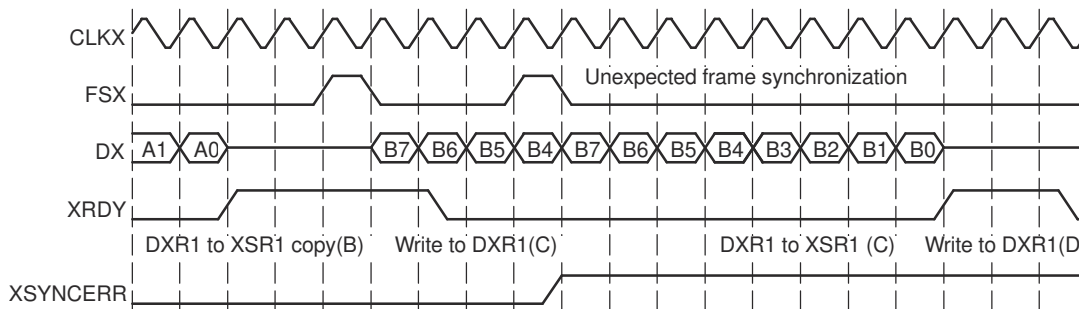
- Case 3: Unexpected transmit frame synchronization with XFIG = 0 (frame-synchronization pulses not ignored). Unexpected frame-synchronization pulses can originate from an external source or from the internal sample rate generator.

If a frame-synchronization pulse starts the transfer of a new frame before the current frame is fully transmitted, this pulse is treated as an unexpected frame-synchronization pulse, and the transmitter sets the transmit frame-synchronization error bit (XSYNCERR) in SPCR2. XSYNCERR can be cleared only by a transmitter reset or by a write of 0 to this bit.

If you want the McBSP to notify the CPU of frame-synchronization errors, you can set a special transmit interrupt mode with the XINTM bits of SPCR2. When XINTM = 11b, the McBSP sends a transmit interrupt (XINT) request to the CPU each time that XSYNCERR is set.

#### 15.4.5.2 Example of Unexpected Transmit Frame-Synchronization Pulse

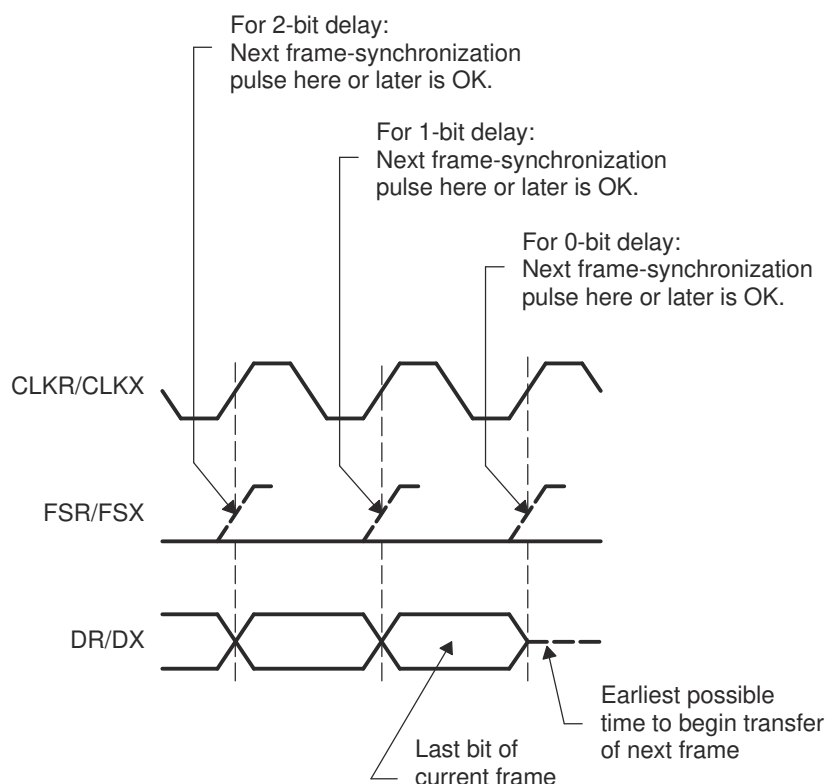
Figure 15-30 shows an unexpected transmit frame-synchronization pulse during normal operation of the serial port with intervals between the data packets. When the unexpected frame-synchronization pulse occurs, the XSYNCERR bit is set and the transmission of data B is restarted because no new data has been passed to XSR1 yet. In addition, if XINTM = 11b, the McBSP sends a transmit interrupt (XINT) request to the CPU.



**Figure 15-30. An Unexpected Frame-Synchronization Pulse During a McBSP Transmission**

#### 15.4.5.3 Preventing Unexpected Transmit Frame-Synchronization Pulses

Each frame transfer can be delayed by 0, 1, or 2 CLKX cycles, depending on the value in the XDADLY bits of XCR2. For each possible data delay, Figure 15-31 shows when a new frame-synchronization pulse on FSX can safely occur relative to the last bit of the current frame.



**Figure 15-31. Proper Positioning of Frame-Synchronization Pulses**

## 15.5 Multichannel Selection Modes

This section discusses the multichannel selection modes for the McBSP.

### 15.5.1 Channels, Blocks, and Partitions

A McBSP channel is a time slot for shifting in/out the bits of one serial word. Each McBSP supports up to 128 channels for reception and 128 channels for transmission.

In the receiver and in the transmitter, the 128 available channels are divided into eight blocks that each contain 16 contiguous channels (see [Table 15-8](#) through [Table 15-10](#)) :

- It is possible to have two receive partitions (A and B) and 8 transmit partitions (A–H).
- McBSP can transmit/receive on selected channels.
- Each channel partition has a dedicated channel-enable register. Each bit controls whether data flow is allowed or prevented in one of the channels assigned to that partition.
- There are three transmit multichannel modes and one receive multichannel mode.

**Table 15-8. Block - Channel Assignment**

| Block | Channels  |
|-------|-----------|
| 0     | 0 -15     |
| 1     | 16 - 31   |
| 2     | 32 - 47   |
| 3     | 48 - 63   |
| 4     | 64 - 79   |
| 5     | 80 - 95   |
| 6     | 96 - 111  |
| 7     | 112 - 127 |

The blocks are assigned to partitions according to the selected partition mode. In the 2-partition mode (described in [Section 15.5.4](#)), you assign one even-numbered block (0, 2, 4, or 6) to partition A and one odd-numbered block (1, 3, 5, or 7) to partition B. In the 8-partition mode (described in [Section 15.5.5](#)), blocks 0 through 7 are automatically assigned to partitions, A through H, respectively.

**Table 15-9. 2-Partition Mode**

| Partition | Blocks           |
|-----------|------------------|
| A         | 0 or 2 or 4 or 6 |
| B         | 1 or 3 or 5 or 7 |

**Table 15-10. 8-Partition Mode**

| Partition | Blocks | Channels  |
|-----------|--------|-----------|
| A         | 0      | 0 - 15    |
| B         | 1      | 16 - 31   |
| C         | 2      | 32 - 47   |
| D         | 3      | 48 - 63   |
| E         | 4      | 64 - 79   |
| F         | 5      | 80 - 95   |
| G         | 6      | 96 - 111  |
| H         | 7      | 112 - 127 |

The number of partitions for reception and the number of partitions for transmission are independent. For example, it is possible to use two receive partitions (A and B) and eight transmit partitions (A-H).

### 15.5.2 Multichannel Selection

When a McBSP uses a time-division multiplexed (TDM) data stream while communicating with other McBSPs or serial devices, the McBSP may need to receive and/or transmit on only a few channels. To save memory and bus bandwidth, you can use a multichannel selection mode to prevent data flow in some of the channels.

Each channel partition has a dedicated channel enable register. If the appropriate multichannel selection mode is on, each bit in the register controls whether data flow is allowed or prevented in one of the channels that is assigned to that partition.

The McBSP has one receive multichannel selection mode (described in [Section 15.5.6](#)) and three transmit multichannel selection modes (described in [Section 15.5.7](#)).

### 15.5.3 Configuring a Frame for Multichannel Selection

Before you enable a multichannel selection mode, make sure you properly configure the data frame:

- Select a single-phase frame (RPHASE/XPHASE = 0). Each frame represents a TDM data stream.
- Set a frame length (in RFRLN1/XFRLN1) that includes the highest-numbered channel to be used. For example, if you plan to use channels 0, 15, and 39 for reception, the receive frame length must be at least 40 (RFRLN1 = 39). If XFRLN1 = 39 in this case, the receiver creates 40 time slots per frame but only receives data during time slots 0, 15, and 39 of each frame.

### 15.5.4 Using Two Partitions

For multichannel selection operation in the receiver and/or the transmitter, you can use two partitions or eight partitions (described in [Section 15.5.5](#)). If you choose the 2-partition mode (RMCME = 0 for reception, XMCME = 0 for transmission), McBSP channels are activated using an alternating scheme. In response to a frame-synchronization pulse, the receiver or transmitter begins with the channels in partition A and then alternates between partitions B and A until the complete frame has been transferred. When the next frame-synchronization pulse occurs, the next frame is transferred beginning with the channels in partition A.

### 15.5.4.1 Assigning Blocks to Partitions A and B

For reception, any two of the eight receive-channel blocks can be assigned to receive partitions A and B, which means up to 32 receive channels can be enabled at any given point in time. Similarly, any two of the eight transmit-channel blocks (up to 32 enabled transmit channels) can be assigned to transmit partitions A and B.

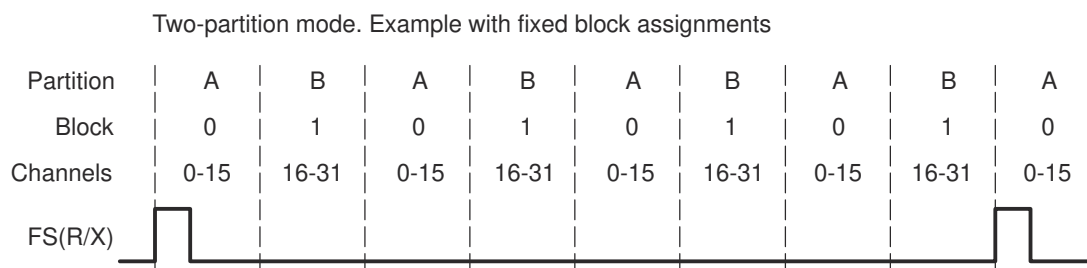
For reception:

- Assign an even-numbered channel block (0, 2, 4, or 6) to receive partition A by writing to the RPABLK bits. In the receive multichannel selection mode (described in [Section 15.5.6](#)), the channels in this partition are controlled by receive channel enable register A (RCERA).
- Assign an odd-numbered block (1, 3, 5, or 7) to receive partition B with the RPBBLK bits. In the receive multichannel selection mode, the channels in this partition are controlled by receive channel enable register B (RCERB).

For transmission:

- Assign an even-numbered channel block (0, 2, 4, or 6) to transmit partition A by writing to the XPABLK bits. In one of the transmit multichannel selection modes (described in [Section 15.5.7](#)), the channels in this partition are controlled by transmit channel enable register A (XCERA).
- Assign an odd-numbered block (1, 3, 5, or 7) to transmit partition B with the XPBBLK bits. In one of the transmit multichannel selection modes, the channels in this partition are controlled by transmit channel enable register B (XCERB).

[Figure 15-32](#) shows an example of alternating between the channels of partition A and the channels of partition B. Channels 0-15 have been assigned to partition A, and channels 16-31 have been assigned to partition B. In response to a frame-synchronization pulse, the McBSP begins a frame transfer with partition A and then alternates between partitions B and A until the complete frame is transferred.



**Figure 15-32. Alternating Between the Channels of Partition A and the Channels of Partition B**

As explained in [Section 15.5.4.2](#), you can dynamically change which blocks of channels are assigned to the partitions.

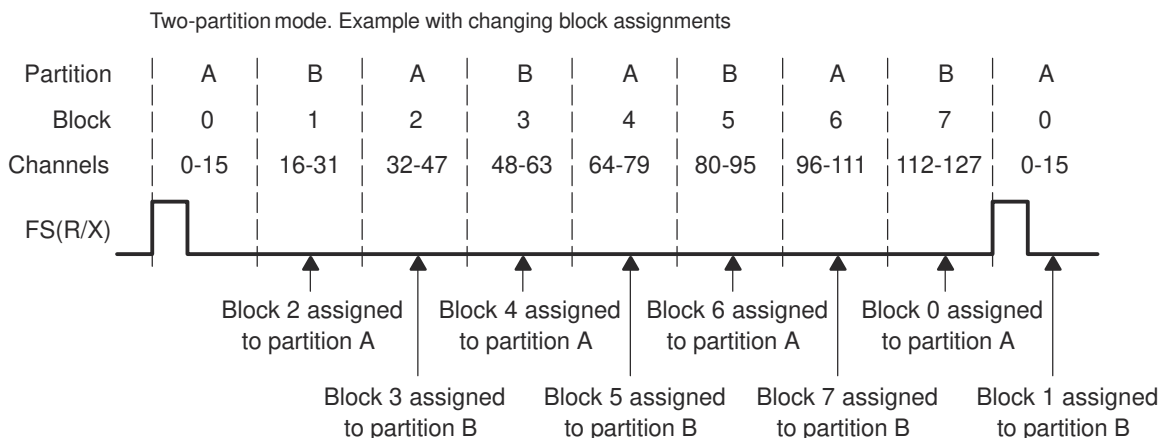
### 15.5.4.2 Reassigning Blocks During Reception/Transmission

If you want to use more than 32 channels, you can change which channel blocks are assigned to partitions A and B during the course of a data transfer. However, these changes must be carefully timed. While a partition is being transferred, its associated block assignment bits cannot be modified and its associated channel enable register cannot be modified. For example, if block 3 is being transferred and block 3 is assigned to partition A, you can modify neither (R/X)PABLK to assign different channels to partition A nor (R/X)CERA to change the channel configuration for partition A.

Several features of the McBSP help you time the reassignment:

- The block of channels currently involved in reception/transmission (the current block) is reflected in the RCBLK/XCBLK bits. Your program can poll these bits to determine which partition is active. When a partition is not active, it is safe to change its block assignment and channel configuration.
- At the end of every block (at the boundary of two partitions), an interrupt can be sent to the CPU. In response to the interrupt, the CPU can then check the RCBLK/XCBLK bits and update the inactive partition. See [Section 15.5.7.3](#).

[Figure 15-33](#) shows an example of reassigning channels throughout a data transfer. In response to a frame-synchronization pulse, the McBSP alternates between partitions A and B. Whenever partition B is active, the CPU changes the block assignment for partition A. Whenever partition A is active, the CPU changes the block assignment for partition B.



**Figure 15-33. Reassigning Channel Blocks Throughout a McBSP Data Transfer**

### 15.5.5 Using Eight Partitions

For multichannel selection operation in the receiver and/or the transmitter, you can use eight partitions or two partitions (described in [Section 15.5.4](#)). If you choose the 8-partition mode (RMCME = 1 for reception, XMCME = 1 for transmission), McBSP channels are activated in the following order: A, B, C, D, E, F, G, H. In response to a frame-synchronization pulse, the receiver or transmitter begins with the channels in partition A and then continues with the other partitions in order until the complete frame has been transferred. When the next frame-synchronization pulse occurs, the next frame is transferred, beginning with the channels in partition A.

In the 8-partition mode, the (R/X)PABLK and (R/X)PBBLK bits are ignored and the 16-channel blocks are assigned to the partitions as shown in [Table 15-11](#) and [Table 15-12](#). These assignments cannot be changed. The tables also show the registers used to control the channels in the partitions.

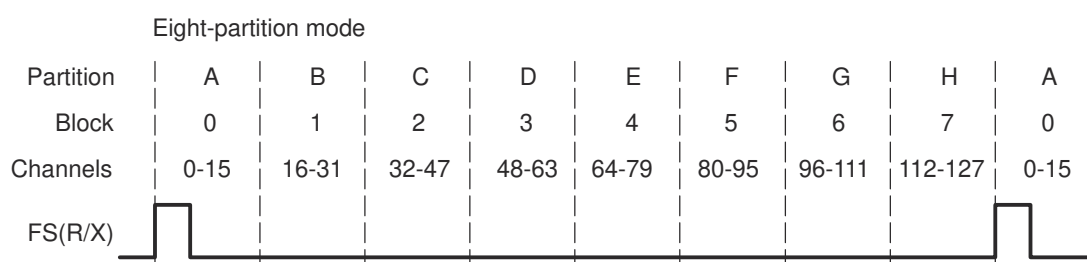
**Table 15-11. Receive Channel Assignment and Control With Eight Receive Partitions**

| Receive Partition | Assigned Block of Receive Channels | Register Used For Channel Control |
|-------------------|------------------------------------|-----------------------------------|
| A                 | Block 0: channels 0 through 15     | RCERA                             |
| B                 | Block 1: channels 16 through 31    | RCERB                             |
| C                 | Block 2: channels 32 through 47    | RCERC                             |
| D                 | Block 3: channels 48 through 63    | RCERD                             |
| E                 | Block 4: channels 64 through 79    | RCERE                             |
| F                 | Block 5: channels 80 through 95    | RCERF                             |
| G                 | Block 6: channels 96 through 111   | RCERG                             |
| H                 | Block 7: channels 112 through 127  | RCERH                             |

**Table 15-12. Transmit Channel Assignment and Control When Eight Transmit Partitions Are Used**

| Transmit Partition | Assigned Block of Transmit Channels | Register Used For Channel Control |
|--------------------|-------------------------------------|-----------------------------------|
| A                  | Block 0: channels 0 through 15      | XCERA                             |
| B                  | Block 1: channels 16 through 31     | XCERB                             |
| C                  | Block 2: channels 32 through 47     | XCERC                             |
| D                  | Block 3: channels 48 through 63     | XCERD                             |
| E                  | Block 4: channels 64 through 79     | XCERE                             |
| F                  | Block 5: channels 80 through 95     | XCERF                             |
| G                  | Block 6: channels 96 through 111    | XCERG                             |
| H                  | Block 7: channels 112 through 127   | XCERH                             |

Figure 15-34 shows an example of the McBSP using the 8-partition mode. In response to a frame-synchronization pulse, the McBSP begins a frame transfer with partition A and then activates B, C, D, E, F, G, and H to complete a 128-word frame.


**Figure 15-34. McBSP Data Transfer in the 8-Partition Mode**

### 15.5.6 Receive Multichannel Selection Mode

The RMCM bit of MCR1 determines whether all channels or only selected channels are enabled for reception. When RMCM = 0, all 128 receive channels are enabled and cannot be disabled. When RMCM = 1, the receive multichannel selection mode is enabled. In this mode:

- Channels can be individually enabled or disabled. The only channels enabled are those selected in the appropriate receive channel enable registers (RCERs). The way channels are assigned to the RCERs depends on the number of receive channel partitions (2 or 8), as defined by the RMCME bit of MCR1.
- If a receive channel is disabled, any bits received in that channel are passed only as far as the receive buffer register(s) (RBR(s)). The receiver does not copy the content of the RBR(s) to the DRR(s), and as a result, does not set the receiver ready bit (RRDY). Therefore, no DMA synchronization event (REVT) is generated and, if the receiver interrupt mode depends on RRDY (RINTM = 00b), no interrupt is generated.

As an example of how the McBSP behaves in the receive multichannel selection mode, suppose you enable only channels 0, 15, and 39 and that the frame length is 40. The McBSP:

- Accepts bits shifted in from the DR pin in channel 0
- Ignores bits received in channels 1-14
- Accepts bits shifted in from the DR pin in channel 15
- Ignores bits received in channels 16-38
- Accepts bits shifted in from the DR pin in channel 39

### 15.5.7 Transmit Multichannel Selection Modes

The XMCM bits of XCR2 determine whether all channels or only selected channels are enabled and unmasked for transmission. More details on enabling and masking are in [Section 15.5.7.1](#). The McBSP has three transmit multichannel selection modes (XMCM = 01b, XMCM = 10b, and XMCM = 11b), which are described in the following table.

**Table 15-13. Selecting a Transmit Multichannel Selection Mode With the XMCM Bits**

| <b>XMCM</b> | <b>Transmit Multichannel Selection Mode</b>                                                                                                                                                                                                                                                                                                                                                                                                               |
|-------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 00b         | No transmit multichannel selection mode is on. All channels are enabled and unmasked. No channels can be disabled or masked.                                                                                                                                                                                                                                                                                                                              |
| 01b         | All channels are disabled unless they are selected in the appropriate transmit channel enable registers (XCERs). If enabled, a channel in this mode is also unmasked.<br><br>The XMCME bit of MCR2 determines whether 32 channels or 128 channels are selectable in XCERs.                                                                                                                                                                                |
| 10b         | All channels are enabled, but they are masked unless they are selected in the appropriate transmit channel enable registers (XCERs).<br><br>The XMCME bit of MCR2 determines whether 32 channels or 128 channels are selectable in XCERs.                                                                                                                                                                                                                 |
| 11b         | This mode is used for symmetric transmission and reception.<br><br>All channels are disabled for transmission unless they are enabled for reception in the appropriate receive channel enable registers (RCERs). Once enabled, they are masked unless they are also selected in the appropriate transmit channel enable registers (XCERs).<br><br>The XMCME bit of MCR2 determines whether 32 channels or 128 channels are selectable in RCERs and XCERs. |

As an example of how the McBSP behaves in a transmit multichannel selection mode, suppose that XMCM = 01b (all channels disabled unless individually enabled) and that you have enabled only channels 0, 15, and 39. Suppose also that the frame length is 40. The McBSP:...

1. Shifts data to the DX pin in channel 0
2. Places the DX pin in the high impedance state in channels 1-14
3. Shifts data to the DX pin in channel 15
4. Places the DX pin in the high impedance state in channels 16-38
5. Shifts data to the DX pin in channel 39



### 15.5.7.1 Disabling or Enabling Versus Masking or Unmasking

For transmission, a channel can be:

- Enabled and unmasked (transmission can begin and can be completed)
- Enabled but masked (transmission can begin but cannot be completed)
- Disabled (transmission cannot occur)

The following definitions explain the channel control options:

|                         |                                                                                                                                                                                                                                                                                                                                                                                                                             |
|-------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Enabled channel</b>  | A channel that can begin transmission by passing data from the data transmit register(s) (DXR(s)) to the transmit shift registers (XSR(s)).                                                                                                                                                                                                                                                                                 |
| <b>Masked channel</b>   | <p>A channel that cannot complete transmission. The DX pin is held in the high impedance state; data cannot be shifted out on the DX pin.</p> <p>In systems where symmetric transmit and receive provides software benefits, this feature allows transmit channels to be disabled on a shared serial bus. A similar feature is not needed for reception because multiple receptions cannot cause serial bus contention.</p> |
| <b>Disabled channel</b> | <p>A channel that is not enabled. A disabled channel is also masked.</p> <p>Because no DXR-to-XSR copy occurs, the XRDY bit of SPCR2 is not set. Therefore, no DMA synchronization event (XEVT) is generated, and if the transmit interrupt mode depends on XRDY (XINTM = 00b in SPCR2), no interrupt is generated.</p> <p>The XEMPTY bit of SPCR2 is not affected.</p>                                                     |
| <b>Unmasked channel</b> | A channel that is not masked. Data in the XSR(s) is shifted out on the DX pin.                                                                                                                                                                                                                                                                                                                                              |

### 15.5.7.2 Activity on McBSP Pins for Different Values of XMCM

Figure 15-35 shows the activity on the McBSP pins for the various XMCM values. In all cases, the transmit frame is configured as follows:

- XPHASE = 0: Single-phase frame (required for multichannel selection modes)
- XFRLN1 = 0000011b: 4 words per frame
- XWDLEN1 = 000b: 8 bits per word
- XMCM = 0: 2-partition mode (only partitions A and B used)

In the case where XMCM = 11b, transmission and reception are symmetric, which means the corresponding bits for the receiver (RPHASE, RFRLN1, RWDLEN1, and RMCME) must have the same values as XPHASE, XFRLN1, and XWDLEN1, respectively.

In the figure, the arrows showing where the various events occur are only sample indications. Wherever possible, there is a time window in which these events can occur.

### 15.5.7.3 Using Interrupts Between Block Transfers

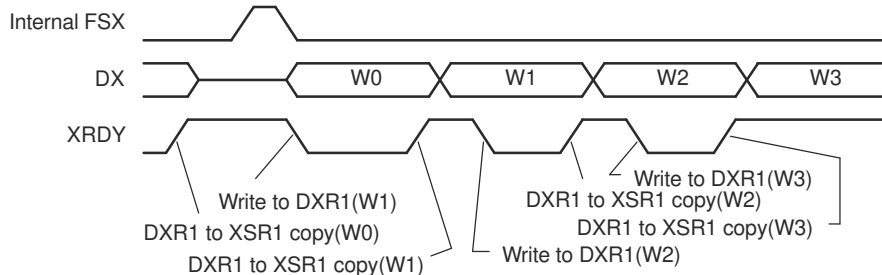
When a multichannel selection mode is used, an interrupt request can be sent to the CPU at the end of every 16-channel block (at the boundary between partitions and at the end of the frame). In the receive multichannel selection mode, a receive interrupt (RINT) request is generated at the end of each block transfer if RINTM = 01b. In any of the transmit multichannel selection modes, a transmit interrupt (XINT) request is generated at the end of each block transfer if XINTM = 01b. When RINTM/XINTM = 01b, no interrupt is generated unless a multichannel selection mode is on.

These interrupt pulses are active high and last for two CPU clock cycles.

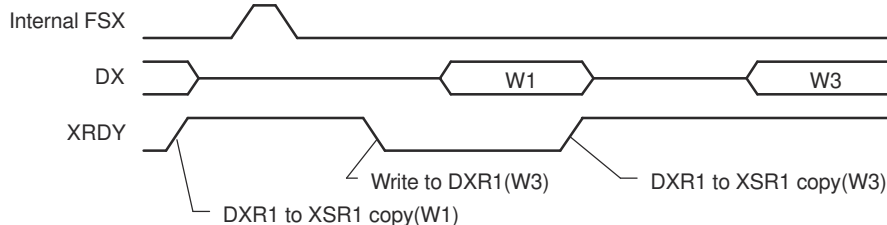
This type of interrupt is especially helpful if you are using the two-partition mode (described in [Section 15.5.4](#)) and you want to know when you can assign a different block of channels to partition A or B.



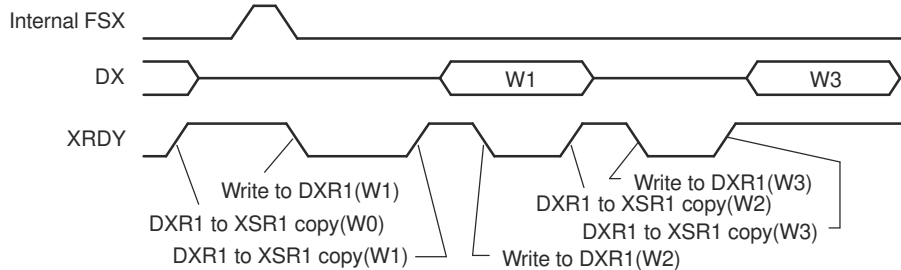
(a) XMCM = 00b: All channels enabled and unmasked



(b) XMCM = 01b, XPABLK = 00b, XCERA = 1010b: Only channels 1 and 3 enabled and unmasked

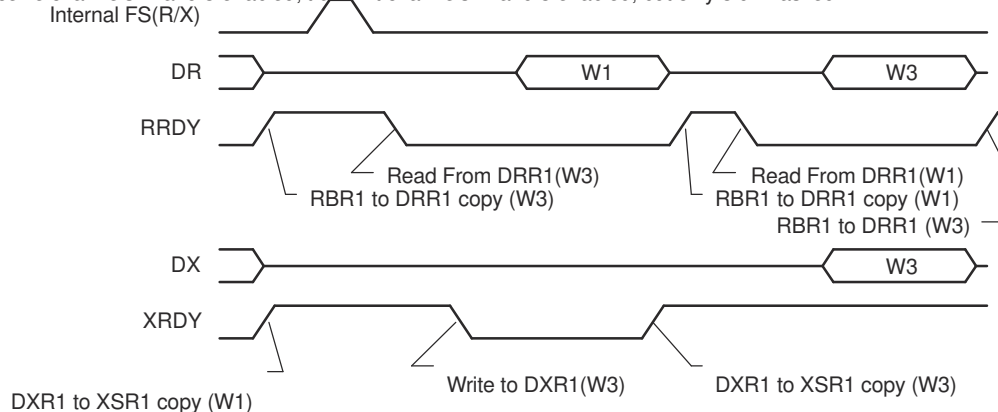


(c) XMCM = 10b, XPABLK = 00b, XCERA = 1010b: All channels enabled, only 1 and 3 unmasked



(d) XMCM = 11b, RPABLK = 00b, XPABLK = X, RCERA = 1010b, XCERA = 1000b:

Receive channels: 1 and 3 enabled; transmit channels: 1 and 3 enabled, but only 3 unmasked



**Figure 15-35. Activity on McBSP Pins for the Possible Values of XMCM**

## 15.6 SPI Operation Using the Clock Stop Mode

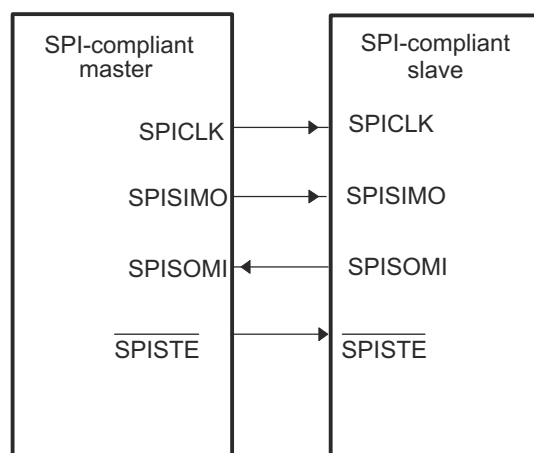
This section explains how to use the McBSP in SPI mode.

### 15.6.1 SPI Protocol

The SPI protocol is a master-slave configuration with one master device and one or more slave devices. The interface consists of the following four signals:

- Serial data input (also referred to as slave out/master in, or SOMI)
- Serial data output (also referred to as slave in/master out, or SIMO)
- Shift-clock (also referred to as SPICLK)
- Slave-enable signal (also referred to as  $\overline{\text{SPISTE}}$ )

A typical SPI interface with a single slave device is shown in [Figure 15-36](#).



**Figure 15-36. Typical SPI Interface**

The master device controls the flow of communication by providing shift-clock and slave-enable signals. The slave-enable signal is an optional active-low signal that enables the serial data input and output of the slave device (device not sending out the clock).

In the absence of a dedicated slave-enable signal, communication between the master and slave is determined by the presence or absence of an active shift-clock. When the McBSP is operating in SPI master mode and the  $\overline{\text{SPISTE}}$  signal is not used by the slave SPI port, the slave device must remain enabled at all times, and multiple slaves cannot be used.

### 15.6.2 Clock Stop Mode

The clock stop mode of the McBSP provides compatibility with the SPI protocol. When the McBSP is configured in clock stop mode, the transmitter and receiver are internally synchronized so that the McBSP functions as an SPI master or slave device. The transmit clock signal (CLKX) corresponds to the serial clock signal (SPICLK) of the SPI protocol, while the transmit frame-synchronization signal (FSX) is used as the slave-enable signal ( $\overline{\text{SPISTE}}$ ).

The receive clock signal (MCLKR) and receive frame-synchronization signal (FSR) are not used in the clock stop mode because these signals are internally connected to their transmit counterparts, CLKX and FSX.

### 15.6.3 Bits Used to Enable and Configure the Clock Stop Mode

The bits required to configure the McBSP as an SPI device are introduced in [Table 15-14](#). [Table 15-15](#) shows how the various combinations of the CLKSTP bit and the polarity bits CLKXP and CLKRP create four possible clock stop mode configurations. The timing diagrams in [Section 15.6.4](#) show the effects of CLKSTP, CLKXP, and CLKRP.

**Table 15-14. Bits Used to Enable and Configure the Clock Stop Mode**

| Bit Field            | Description                                                                                                                                                                                         |
|----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CLKSTP bits of SPCR1 | Use these bits to enable the clock stop mode and to select one of two timing variations. (See also <a href="#">Table 15-15</a> .)                                                                   |
| CLKXP bit of PCR     | This bit determines the polarity of the CLKX signal. (See also <a href="#">Table 15-15</a> .)                                                                                                       |
| CLKRP bit of PCR     | This bit determines the polarity of the MCLKR signal. (See also <a href="#">Table 15-15</a> .)                                                                                                      |
| CLKXM bit of PCR     | This bit determines whether CLKX is an input signal (McBSP as slave) or an output signal (McBSP as master).                                                                                         |
| XPHASE bit of XCR2   | You must use a single-phase transmit frame (XPHASE = 0).                                                                                                                                            |
| RPHASE bit of RCR2   | You must use a single-phase receive frame (RPHASE = 0).                                                                                                                                             |
| XFRLN1 bits of XCR1  | You must use a transmit frame length of 1 serial word (XFRLN1 = 0).                                                                                                                                 |
| RFRLN1 bits of RCR1  | You must use a receive frame length of 1 serial word (RFRLN1 = 0).                                                                                                                                  |
| XWDLEN1 bits of XCR1 | The XWDLEN1 bits determine the transmit packet length. XWDLEN1 must be equal to RWDLEN1 because in the clock stop mode. The McBSP transmit and receive circuits are synchronized to a single clock. |
| RWDLEN1 bits of RCR1 | The RWDLEN1 bits determine the receive packet length. RWDLEN1 must be equal to XWDLEN1 because in the clock stop mode. The McBSP transmit and receive circuits are synchronized to a single clock.  |

**Table 15-15. Effects of CLKSTP, CLKXP, and CLKRP on the Clock Scheme**

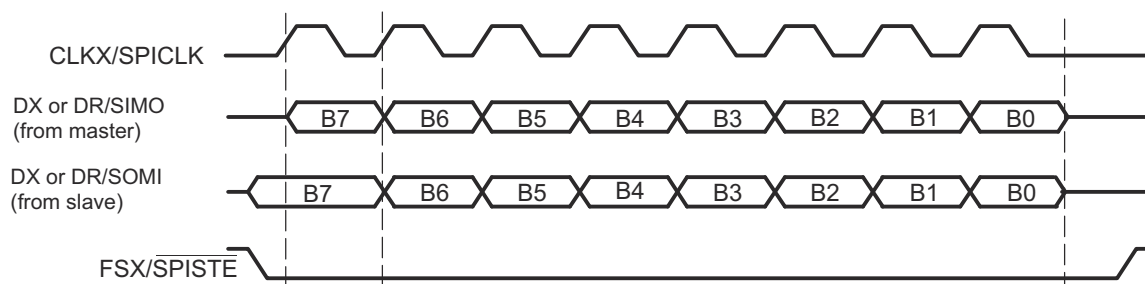
| Bit Settings                                            | Clock Scheme                                                                                                                                              |
|---------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| CLKSTP = 00b or 01b<br>CLKXP = 0 or 1<br>CLKRP = 0 or 1 | Clock stop mode disabled. Clock enabled for non-SPI mode.                                                                                                 |
| CLKSTP = 10b<br>CLKXP = 0<br>CLKRP = 0                  | Low inactive state without delay: The McBSP transmits data on the rising edge of CLKX and receives data on the falling edge of MCLKR.                     |
| CLKSTP = 11b<br>CLKXP = 0<br>CLKRP = 1                  | Low inactive state with delay: The McBSP transmits data one-half cycle ahead of the rising edge of CLKX and receives data on the rising edge of MCLKR.    |
| CLKSTP = 10b<br>CLKXP = 1<br>CLKRP = 0                  | High inactive state without delay: The McBSP transmits data on the falling edge of CLKX and receives data on the rising edge of MCLKR.                    |
| CLKSTP = 11b<br>CLKXP = 1<br>CLKRP = 1                  | High inactive state with delay: The McBSP transmits data one-half cycle ahead of the falling edge of CLKX and receives data on the falling edge of MCLKR. |

## 15.6.4 Clock Stop Mode Timing Diagrams

The timing diagrams for the four possible clock stop mode configurations are shown here. Notice that the frame-synchronization signal used in clock stop mode is active throughout the entire transmission as a slave-enable signal. Although the timing diagrams show 8-bit transfers, the packet length can be set to 8, 12, 16, 20, 24, or 32 bits per packet. The receive packet length is selected with the RWDLEN1 bits of RCR1, and the transmit packet length is selected with the XWDLEN1 bits of XCR1. For clock stop mode, the values of RWDLEN1 and XWDLEN1 must be the same because the McBSP transmit and receive circuits are synchronized to a single clock.

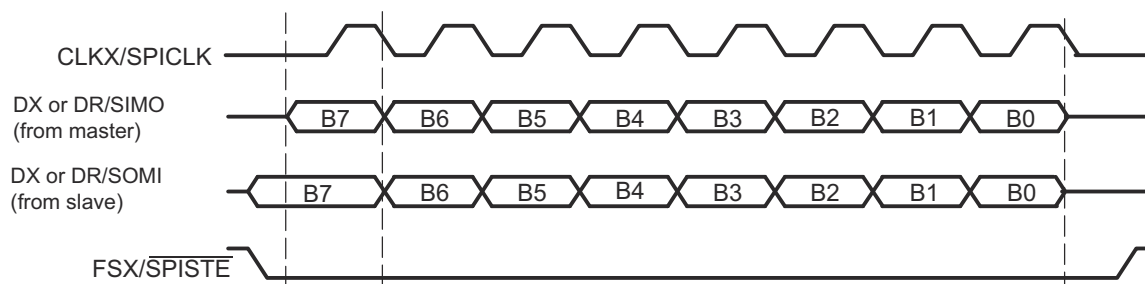
### Note

Even if multiple words are consecutively transferred, the CLKX signal is always stopped and the FSX signal returns to the inactive state after a packet transfer. When consecutive packet transfers are performed, this leads to a minimum idle time of two bit-periods between each packet transfer.



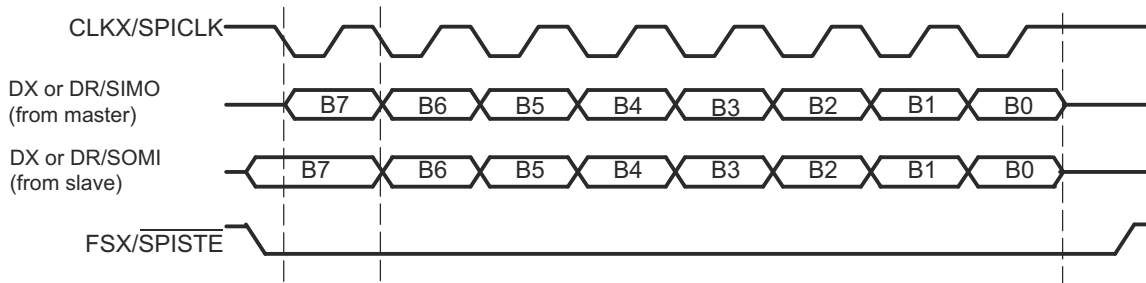
- A. If the McBSP is the SPI master (CLKXM = 1), SIMO = DX. If the McBSP is the SPI slave (CLKXM = 0), SIMO = DR.
- B. If the McBSP is the SPI master (CLKXM = 1), SOMI = DR. If the McBSP is the SPI slave (CLKXM = 0), SOMI = DX.

**Figure 15-37. SPI Transfer With CLKSTP = 10b (No Clock Delay), CLKXP = 0, and CLKRP = 0**



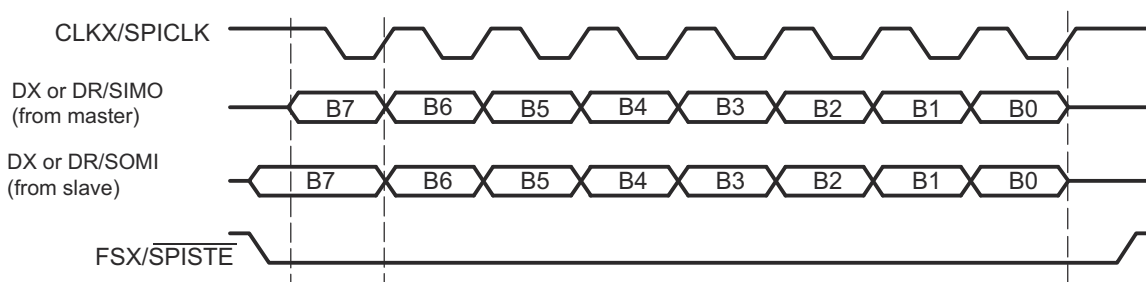
- A. If the McBSP is the SPI master (CLKXM = 1), SIMO = DX. If the McBSP is the SPI slave (CLKXM = 0), SIMO = DR.
- B. If the McBSP is the SPI master (CLKXM = 1), SOMI = DR. If the McBSP is the SPI slave (CLKXM = 0), SOMI = DX.

**Figure 15-38. SPI Transfer With CLKSTP = 11b (Clock Delay), CLKXP = 0, CLKRP = 1**



- A. If the McBSP is the SPI master (CLKXM = 1), SIMO = DX. If the McBSP is the SPI slave (CLKXM = 0), SIMO = DR.  
 B. If the McBSP is the SPI master (CLKXM = 1), SOMI = DR. If the McBSP is the SPI slave (CLKXM = 0), SOMI = DX.

**Figure 15-39. SPI Transfer With CLKSTP = 10b (No Clock Delay), CLKXP = 1, and CLKRP = 0**



- A. If the McBSP is the SPI master (CLKXM = 1), SIMO=DX. If the McBSP is the SPI slave (CLKXM = 0), SIMO = DR.  
 B. If the McBSP is the SPI master (CLKXM = 1), SOMI=DR. If the McBSP is the SPI slave (CLKXM = 0), SOMI = DX.

**Figure 15-40. SPI Transfer With CLKSTP = 11b (Clock Delay), CLKXP = 1, CLKRP = 1**

### 15.6.5 Procedure for Configuring a McBSP for SPI Operation

To configure the McBSP for SPI master or slave operation:

1. Place the transmitter and receiver in reset.

Clear the transmitter reset bit (XRST = 0) in SPCR2 to reset the transmitter. Clear the receiver reset bit (RRST = 0) in SPCR1 to reset the receiver.

2. Place the sample rate generator in reset.

Clear the sample rate generator reset bit (GRST = 0) in SPCR2 to reset the sample rate generator.

3. Program registers that affect SPI operation.

Program the appropriate McBSP registers to configure the McBSP for proper operation as an SPI master or an SPI slave. For a list of important bits settings, see one of the following topics:

- *McBSP as the SPI Master* ([Section 15.6.6](#))
- *McBSP as an SPI Slave* ([Section 15.6.7](#))

4. Enable the sample rate generator.

To release the sample rate generator from reset, set the sample rate generator reset bit (GRST = 1) in SPCR2.

Make sure that during the write to SPCR2, you only modify GRST. Otherwise, you modify the McBSP configuration you selected in the previous step.

5. Enable the transmitter and receiver.

After the sample rate generator is released from reset, wait two sample rate generator clock periods for the McBSP logic to stabilize.

If the CPU services the McBSP transmit and receive buffers, then you can immediately enable the transmitter (XRST = 1 in SPCR2) and enable the receiver (RRST = 1 in SPCR1).

If the DMA controller services the McBSP transmit and receive buffers, then you must first configure the DMA controller (this includes enabling the channels that service the McBSP buffers). When the DMA controller is ready, make XRST = 1 and RRST = 1.

In either case, make sure you only change XRST and RRST when you write to SPCR2 and SPCR1. Otherwise, you modify the bit settings you selected earlier in this procedure.

After the transmitter and receiver are released from reset, wait two sample rate generator clock periods for the McBSP logic to stabilize.

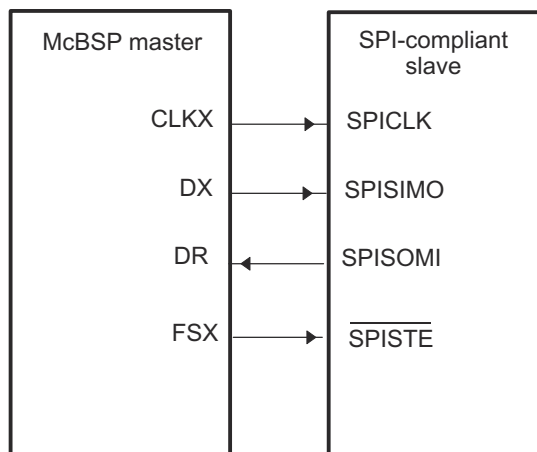
6. If necessary, enable the frame-synchronization logic of the sample rate generator.

After the required data acquisition setup is done (DXR[1,2] is loaded with data), set FRST = 1 if an internally generated frame-synchronization pulse is required (that is, if the McBSP is the SPI master).

### 15.6.6 McBSP as the SPI Master

An SPI interface with the McBSP used as the master is shown in [Figure 15-41](#). When the McBSP is configured as a master, the transmit output signal (DX) is used as the SIMO signal of the SPI protocol and the receive input signal (DR) is used as the SOMI signal.

The register bit values required to configure the McBSP as a master are listed in [Table 15-16](#). After the table are more details about the configuration requirements.



**Figure 15-41. SPI Interface with McBSP Used as Master**

**Table 15-16. Bit Values Required to Configure the McBSP as an SPI Master**

| Required Bit Setting            | Description                                                                                                                                                                                 |
|---------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CLKSTP = 10b or 11b             | The clock stop mode (without or with a clock delay) is selected.                                                                                                                            |
| CLKXP = 0 or 1                  | The polarity of CLKX as seen on the MCLKX pin is positive (CLKXP = 0) or negative (CLKXP = 1).                                                                                              |
| CLKRP = 0 or 1                  | The polarity of MCLKR as seen on the MCLKR pin is positive (CLKRP = 0) or negative (CLKRP = 1).                                                                                             |
| CLKXM = 1                       | The MCLKX pin is an output pin driven by the internal sample rate generator. Because CLKSTP is equal to 10b or 11b, MCLKR is driven internally by CLKX.                                     |
| SCLKME = 0                      | The clock generated by the sample rate generator (CLKG) is derived from the CPU clock.                                                                                                      |
| CLKSM = 1                       |                                                                                                                                                                                             |
| CLKGDV is a value from 1 to 255 | CLKGDV defines the divide down value for CLKG.                                                                                                                                              |
| FSXM = 1                        | The FSX pin is an output pin driven according to the FSGM bit. (See the <a href="#">TMS320F2833x</a> , <a href="#">TMS320F2823x Digital Signal Controllers (DSCs)</a> for more information. |
| FSGM = 0                        | The transmitter drives a frame-synchronization pulse on the FSX pin every time data is transferred from DXR1 to XSR1.                                                                       |
| FSXP = 1                        | The FSX pin is active low.                                                                                                                                                                  |
| XDATDLY = 01b                   | This setting provides the correct setup time on the FSX signal.                                                                                                                             |
| RDATDLY = 01b                   |                                                                                                                                                                                             |

When the McBSP functions as the SPI master, it controls the transmission of data by producing the serial clock signal. The clock signal on the MCLKX pin is enabled only during packet transfers. When packets are not being transferred, the MCLKX pin remains high or low depending on the polarity used.

For SPI master operation, the MCLKX pin must be configured as an output. The sample rate generator is then used to derive the CLKX signal from the CPU clock. The clock stop mode internally connects the MCLKX pin to the MCLKR signal so that no external signal connection is required on the MCLKR pin and both the transmit and receive circuits are clocked by the master clock (CLKX).

The data delay parameters of the McBSP (XDATDLY and RDATDLY) must be set to 1 for proper SPI master operation. A data delay value of 0 or 2 is undefined in the clock stop mode.

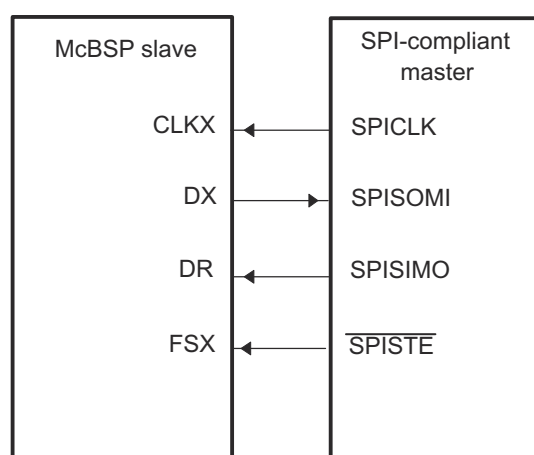
The McBSP can also provide a slave-enable signal ( $\overline{\text{SPISTE}}$ ) on the FSX pin. If a slave-enable signal is required, the FSX pin must be configured as an output and the transmitter must be configured so that a frame-synchronization pulse is generated automatically each time a packet is transmitted (FSGM = 0). The polarity of the FSX pin is programmable high or low; however, in most cases the pin must be configured active low.

When the McBSP is configured as described for SPI-master operation, the bit fields for frame-synchronization pulse width (FWID) and frame-synchronization period (FPER) are overridden, and custom frame-synchronization waveforms are not allowed. To see the resulting waveform produced on the FSX pin, see the timing diagrams in [Section 15.6.4](#). The signal becomes active before the first bit of a packet transfer, and remains active until the last bit of the packet is transferred. After the packet transfer is complete, the FSX signal returns to the inactive state.

### 15.6.7 McBSP as an SPI Slave

An SPI interface with the McBSP used as a slave is shown in [Figure 15-42](#). When the McBSP is configured as a slave, DX is used as the SOMI signal and DR is used as the SIMO signal.

The register bit values required to configure the McBSP as a slave are listed in [Table 15-17](#). Following the table are more details about configuration requirements.



**Figure 15-42. SPI Interface With McBSP Used as Slave**

**Table 15-17. Bit Values Required to Configure the McBSP as an SPI Slave**

| Required Bit Setting | Description                                                                                                                                                                                           |
|----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CLKSTP = 10b or 11b  | The clock stop mode (without or with a clock delay) is selected.                                                                                                                                      |
| CLKXP = 0 or 1       | The polarity of CLKX as seen on the MCLKX pin is positive (CLKXP = 0) or negative (CLKXP = 1).                                                                                                        |
| CLKRP = 0 or 1       | The polarity of MCLKR as seen on the MCLKR pin is positive (CLKRP = 0) or negative (CLKRP = 1).                                                                                                       |
| CLKXM = 0            | The MCLKX pin is an input pin, so that it can be driven by the SPI master. Because CLKSTP = 10b or 11b, MCLKR is driven internally by CLKX.                                                           |
| SCLKME = 0           | The clock generated by the sample rate generator (CLKG) is derived from the CPU clock. (The sample rate generator is used to synchronize the McBSP logic with the externally-generated master clock.) |
| CLKSM = 1            |                                                                                                                                                                                                       |
| CLKGDV = 1           | The sample rate generator divides the CPU clock before generating CLKG.                                                                                                                               |
| FSXM = 0             | The FSX pin is an input pin, so that it can be driven by the SPI master.                                                                                                                              |
| FSXP = 1             | The FSX pin is active low.                                                                                                                                                                            |
| XDATDLY = 00b        | These bits must be 0s for SPI slave operation.                                                                                                                                                        |
| RDATDLY = 00b        |                                                                                                                                                                                                       |

When the McBSP is used as an SPI slave, the master clock and slave-enable signals are generated externally by a master device. Accordingly, the CLKX and FSX pins must be configured as inputs. The MCLKX pin is



internally connected to the MCLKR signal, so that both the transmit and receive circuits of the McBSP are clocked by the external master clock. The FSX pin is also internally connected to the FSR signal, and no external signal connections are required on the MCLKR and FSR pins.

Although the CLKX signal is generated externally by the master and is asynchronous to the McBSP, the sample rate generator of the McBSP must be enabled for proper SPI slave operation. The sample rate generator must be programmed to its maximum rate of half the CPU clock rate. The internal sample rate clock is then used to synchronize the McBSP logic to the external master clock and slave-enable signals.

The McBSP requires an active edge of the slave-enable signal on the FSX input for each transfer. This means that the master device must assert the slave-enable signal at the beginning of each transfer, and deassert the signal after the completion of each packet transfer; the slave-enable signal cannot remain active between transfers. Unlike the standard SPI, this pin cannot be tied low all the time.

The data delay parameters of the McBSP must be set to 0 for proper SPI slave operation. A value of 1 or 2 is undefined in the clock stop mode.

## 15.7 Receiver Configuration

To configure the McBSP receiver, perform the following procedure:

1. Place the McBSP/receiver in reset (see [Section 15.7.2](#)).
2. Program McBSP registers for the desired receiver operation (see [Section 15.7.1](#)).
3. Take the receiver out of reset (see [Section 15.7.2](#)).

### 15.7.1 Programming the McBSP Registers for the Desired Receiver Operation

The following is a list of important tasks to be performed when you are configuring the McBSP receiver. Each task corresponds to one or more McBSP register bit fields.

- Global behavior:
  - Set the receiver pins to operate as McBSP pins.
  - Enable/disable the digital loopback mode.
  - Enable/disable the clock stop mode.
  - Enable/disable the receive multichannel selection mode.
- Data behavior:
  - Choose 1 or 2 phases for the receive frame.
  - Set the receive word length(s).
  - Set the receive frame length.
  - Enable/disable the receive frame-synchronization ignore function.
  - Set the receive companding mode.
  - Set the receive data delay.
  - Set the receive sign-extension and justification mode.
  - Set the receive interrupt mode.
- Frame-synchronization behavior:
  - Set the receive frame-synchronization mode.
  - Set the receive frame-synchronization polarity.
  - Set the sample rate generator (SRG) frame-synchronization period and pulse width.
- Clock behavior:
  - Set the receive clock mode.
  - Set the receive clock polarity.
  - Set the SRG clock divide-down value.
  - Set the SRG clock synchronization mode.
  - Set the SRG clock mode (choose an input clock).
  - Set the SRG input clock polarity.

## 15.7.2 Resetting and Enabling the Receiver

The first step of the receiver configuration procedure is to reset the receiver, and the last step is to enable the receiver (to take it out of reset). [Table 15-18](#) describes the bits used for both of these steps.

**Table 15-18. Register Bits Used to Reset or Enable the McBSP Receiver Field Descriptions**

| Register | Bit | Field | Value | Description                                                                                                                                                                                                                                                                         |
|----------|-----|-------|-------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| SPCR2    | 7   | FRST  | 0     | Frame-synchronization logic reset<br>Frame-synchronization logic is reset. The sample rate generator does not generate frame-synchronization signal FSG, even if GRST = 1.                                                                                                          |
|          |     |       | 1     | If GRST = 1, frame-synchronization signal FSG is generated after (FPER + 1) number of CLKG clock cycles; all frame counters are loaded with their programmed values.                                                                                                                |
| SPCR2    | 6   | GRST  | 0     | Sample rate generator reset<br>Sample rate generator is reset. If GRST = 0 due to a DSP reset, CLKG is driven by the CPU clock divided by 2, and FSG is driven low (inactive). If GRST = 0 due to program code, CLKG and FSG are both driven low (inactive).                        |
|          |     |       | 1     | Sample rate generator is enabled. CLKG is driven according to the configuration programmed in the sample rate generator registers (SRGR[1,2]). If FRST = 1, the generator also generates the frame-synchronization signal FSG as programmed in the sample rate generator registers. |
| SPCR1    | 0   | RRST  | 0     | Receiver reset<br>The serial port receiver is disabled and in the reset state.                                                                                                                                                                                                      |
|          |     |       | 1     | The serial port receiver is enabled.                                                                                                                                                                                                                                                |

### 15.7.2.1 Reset Considerations

The serial port can be reset in the following two ways:

1. The DSP reset ( $\overline{\text{XRS}}$  signal driven low) places the receiver, transmitter, and sample rate generator in reset. When the device reset is removed ( $\overline{\text{XRS}}$  signal released), GRST = FRST = RRST = XRST = 0 keep the entire serial port in the reset state, provided the McBSP clock is turned on.
2. The serial port transmitter and receiver can be reset directly using the RRST and XRST bits in the serial port control registers. The sample rate generator can be reset directly using the GRST bit in SPCR2.

[Table 15-19](#) shows the state of McBSP pins when the serial port is reset due to a device reset and a direct receiver/transmitter reset.

For more details about McBSP reset conditions and effects, see [Section 15.9.2](#).

**Table 15-19. Reset State of Each McBSP Pin**

| Pin    | Possible State(s) | State Forced By Device Reset | State Forced By Receiver Reset (RRST = 0 and GRST = 1)                                           |
|--------|-------------------|------------------------------|--------------------------------------------------------------------------------------------------|
| MDRx   | I                 | GPIO Input                   | Input                                                                                            |
| MCLKRx | I/O/Z             | GPIO Input                   | Known state if input; MCLKR running if output                                                    |
| MFSRx  | I/O/Z             | GPIO Input                   | Known state if input; FSRP inactive state if output<br>Transmitter reset (XRST = 0 and GRST = 1) |
| MDXx   | O/Z               | GPIO Input                   | Low impedance after transmit bit clock provided                                                  |
| MCLKXx | I/O/Z             | GPIO Input                   | Known state if input; CLKX running if output                                                     |
| MFSXx  | I/O/Z             | GPIO Input                   | Known state if input; FSXP inactive state if output                                              |

## 15.7.3 Set the Receiver Pins to Operate as McBSP Pins

To configure a pin for its McBSP function, you should configure the bits of the GPxMUXn register appropriately. In addition to this, bits 12 and 13 of the PCR register must be set to 0. These bits are defined as reserved.

### 15.7.4 Enable/Disable the Digital Loopback Mode

The DLB bit determines whether the digital loopback mode is on. DLB is described in [Table 15-20](#).

**Table 15-20. Register Bit Used to Enable/Disable the Digital Loopback Mode**

| Register | Bit | Name | Function              | Type | Reset Value                        |
|----------|-----|------|-----------------------|------|------------------------------------|
| SPCR1    | 15  | DLB  | Digital loopback mode | R/W  | 0                                  |
|          |     |      | DLB = 0               |      | Digital loopback mode is disabled. |
|          |     |      | DLB = 1               |      | Digital loopback mode is enabled.  |

#### 15.7.4.1 Digital Loopback Mode

In the digital loopback mode, the receive signals are connected internally through multiplexers to the corresponding transmit signals, as shown in [Table 15-21](#). This mode allows testing of serial port code with a single DSP device; the McBSP receives the data it transmits.

**Table 15-21. Receive Signals Connected to Transmit Signals in Digital Loopback Mode**

| This Receive Signal                  | Is Fed Internally by This Transmit Signal |
|--------------------------------------|-------------------------------------------|
| MDR (receive data)                   | MDX (transmit data)                       |
| MFSR (receive frame synchronization) | MFSX (transmit frame synchronization)     |
| MCLKR (receive clock)                | MCLKX (transmit clock)                    |

### 15.7.5 Enable/Disable the Clock Stop Mode

The CLKSTP bits determine whether the clock stop mode is on. CLKSTP is described in [Table 15-22](#).

**Table 15-22. Register Bits Used to Enable/Disable the Clock Stop Mode**

| Register | Bit   | Name   | Function        | Type | Reset Value                                                |
|----------|-------|--------|-----------------|------|------------------------------------------------------------|
| SPCR1    | 12-11 | CLKSTP | Clock stop mode | R/W  | 00                                                         |
|          |       |        | CLKSTP = 0Xb    |      | Clock stop mode disabled; normal clocking for non-SPI mode |
|          |       |        | CLKSTP = 10b    |      | Clock stop mode enabled, without clock delay               |
|          |       |        | CLKSTP = 11b    |      | Clock stop mode enabled, with clock delay                  |

#### 15.7.5.1 Clock Stop Mode

The clock stop mode supports the SPI master-slave protocol. If you do not plan to use the SPI protocol, you can clear CLKSTP to disable the clock stop mode.

In the clock stop mode, the clock stops at the end of each data transfer. At the beginning of each data transfer, the clock starts immediately (CLKSTP = 10b) or after a half-cycle delay (CLKSTP = 11b). The CLKXP bit determines whether the starting edge of the clock on the MCLKX pin is rising or falling. The CLKRP bit determines whether receive data is sampled on the rising or falling edge of the clock shown on the MCLKR pin.

[Table 15-23](#) summarizes the impact of CLKSTP, CLKXP, and CLKRP on serial port operation. In the clock stop mode, the receive clock is tied internally to the transmit clock, and the receive frame-synchronization signal is tied internally to the transmit frame-synchronization signal.

**Table 15-23. Effects of CLKSTP, CLKXP, and CLKRP on the Clock Scheme**

| Bit Settings                                            | Clock Scheme                                                                                                                                              |
|---------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| CLKSTP = 00b or 01b<br>CLKXP = 0 or 1<br>CLKRP = 0 or 1 | Clock stop mode disabled. Clock enabled for non-SPI mode.                                                                                                 |
| CLKSTP = 10b<br>CLKXP = 0<br>CLKRP = 0                  | Low inactive state without delay: The McBSP transmits data on the rising edge of CLKX and receives data on the falling edge of MCLKR.                     |
| CLKSTP = 11b<br>CLKXP = 0<br>CLKRP = 1                  | Low inactive state with delay: The McBSP transmits data one-half cycle ahead of the rising edge of CLKX and receives data on the rising edge of MCLKR.    |
| CLKSTP = 10b<br>CLKXP = 1<br>CLKRP = 0                  | High inactive state without delay: The McBSP transmits data on the falling edge of CLKX and receives data on the rising edge of MCLKR.                    |
| CLKSTP = 11b<br>CLKXP = 1<br>CLKRP = 1                  | High inactive state with delay: The McBSP transmits data one-half cycle ahead of the falling edge of CLKX and receives data on the falling edge of MCLKR. |

### 15.7.6 Enable/Disable the Receive Multichannel Selection Mode

The RMCM bit determines whether the receive multichannel selection mode is on. RMCM is described in [Table 15-24](#). For more details, see [Section 15.5.6](#).

**Table 15-24. Register Bit Used to Enable/Disable the Receive Multichannel Selection Mode**

| Register | Bit | Name | Function                                                                                                                                                                                                                                                                                                                                                                                                                                                              | Type | Reset Value |
|----------|-----|------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------|-------------|
| MCR1     | 0   | RMCM | Receive multichannel selection mode<br><br>RMCM = 0      The mode is disabled.<br>All 128 channels are enabled.<br><br>RMCM = 1      The mode is enabled.<br>Channels can be individually enabled or disabled.<br><br>The only channels enabled are those selected in the appropriate receive channel enable registers (RCERs). The way channels are assigned to the RCERs depends on the number of receive channel partitions (2 or 8), as defined by the RMCME bit. | R/W  | 0           |

### 15.7.7 Choose One or Two Phases for the Receive Frame

The RPHASE bit (see [Table 15-25](#)) determines whether the receive data frame has one or two phases.

**Table 15-25. Register Bit Used to Choose One or Two Phases for the Receive Frame**

| Register | Bit | Name   | Function                                                                                                                                                             | Type | Reset Value |
|----------|-----|--------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|------|-------------|
| RCR2     | 15  | RPHASE | Receive phase number<br><br>Specifies whether the receive frame has 1 or 2 phases.<br><br>RPHASE = 0      Single-phase frame<br><br>RPHASE = 1      Dual-phase frame | R/W  | 0           |

### 15.7.8 Set the Receive Word Length(s)

The RWDLEN1 and RWDLEN2 bit fields (see [Table 15-26](#)) determine how many bits are in each serial word in phase 1 and in phase 2, respectively, of the receive data frame.

**Table 15-26. Register Bits Used to Set the Receive Word Length(s)**

| Register | Bit | Name    | Function                                                                                                      | Type | Reset Value |
|----------|-----|---------|---------------------------------------------------------------------------------------------------------------|------|-------------|
| RCR1     | 7-5 | RWDLEN1 | Receive word length 1                                                                                         | R/W  | 000         |
|          |     |         | Specifies the length of every serial word in phase 1 of the receive frame.                                    |      |             |
|          |     |         | RWDLEN1 = 000                                                                                                 |      |             |
|          |     |         | 8 bits                                                                                                        |      |             |
|          |     |         | RWDLEN1 = 001                                                                                                 |      |             |
|          |     |         | 12 bits                                                                                                       |      |             |
|          |     |         | RWDLEN1 = 010                                                                                                 |      |             |
|          |     |         | 16 bits                                                                                                       |      |             |
|          |     |         | RWDLEN1 = 011                                                                                                 |      |             |
| RCR2     | 7-5 | RWDLEN2 | Receive word length 2                                                                                         | R/W  | 000         |
|          |     |         | If a dual-phase frame is selected, RWDLEN2 specifies the length of every serial word in phase 2 of the frame. |      |             |
|          |     |         | RWDLEN2 = 000                                                                                                 |      |             |
|          |     |         | 8 bits                                                                                                        |      |             |
|          |     |         | RWDLEN2 = 001                                                                                                 |      |             |
|          |     |         | 12 bits                                                                                                       |      |             |
|          |     |         | RWDLEN2 = 010                                                                                                 |      |             |
|          |     |         | 16 bits                                                                                                       |      |             |
|          |     |         | RWDLEN2 = 011                                                                                                 |      |             |
|          |     |         | 20 bits                                                                                                       |      |             |
|          |     |         | RWDLEN2 = 100                                                                                                 |      |             |
|          |     |         | 24 bits                                                                                                       |      |             |
|          |     |         | RWDLEN2 = 101                                                                                                 |      |             |
|          |     |         | 32 bits                                                                                                       |      |             |
|          |     |         | RWDLEN2 = 11X                                                                                                 |      |             |
|          |     |         | Reserved                                                                                                      |      |             |

#### 15.7.8.1 Word Length Bits

Each frame can have one or two phases, depending on the value that you load into the RPHASE bit. If a single-phase frame is selected, RWDLEN1 selects the length for every serial word received in the frame. If a dual-phase frame is selected, RWDLEN1 determines the length of the serial words in phase 1 of the frame and RWDLEN2 determines the word length in phase 2 of the frame.

### 15.7.9 Set the Receive Frame Length

The RFRLN1 and RFRLN2 bit fields (see [Table 15-27](#)) determine how many serial words are in phase 1 and in phase 2, respectively, of the receive data frame.

**Table 15-27. Register Bits Used to Set the Receive Frame Length**

| Register | Bit  | Name   | Function                                                                    | Type | Reset Value |
|----------|------|--------|-----------------------------------------------------------------------------|------|-------------|
| RCR1     | 14-8 | RFRLN1 | Receive frame length 1                                                      | R/W  | 000 0000    |
|          |      |        | (RFRLN1 + 1) is the number of serial words in phase 1 of the receive frame. |      |             |
|          |      |        | RFRLN1 = 000 0000                                                           |      |             |
|          |      |        | 1 word in phase 1                                                           |      |             |
|          |      |        | RFRLN1 = 000 0001                                                           |      |             |
|          |      |        | 2 words in phase 1                                                          |      |             |
|          |      |        |                                                                             |      |             |
|          |      |        |                                                                             |      |             |
|          |      |        |                                                                             |      |             |
|          |      |        |                                                                             |      |             |
|          |      |        | RFRLN1 = 111 1111                                                           |      |             |
|          |      |        | 128 words in phase 1                                                        |      |             |

**Table 15-27. Register Bits Used to Set the Receive Frame Length (continued)**

| Register | Bit  | Name   | Function                                                                                                                                                                                                                                                                                                                                           | Type | Reset Value |
|----------|------|--------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------|-------------|
| RCR2     | 14-8 | RFLEN2 | Receive frame length 2<br><br>If a dual-phase frame is selected, (RFLEN2 + 1) is the number of serial words in phase 2 of the receive frame.<br><br>RFLEN2 = 000 0000                      1 word in phase 2<br>RFLEN2 = 000 0001                      2 words in phase 2<br> <br> <br>RFLEN2 = 111 1111                      128 words in phase 2 | R/W  | 000 0000    |

### 15.7.9.1 Selected Frame Length

The receive frame length is the number of serial words in the receive frame. Each frame can have one or two phases, depending on value that you load into the RPHASE bit.

If a single-phase frame is selected (RPHASE = 0), the frame length is equal to the length of phase 1. If a dual-phase frame is selected (RPHASE = 1), the frame length is the length of phase 1 plus the length of phase 2.

The 7-bit RFLEN fields allow up to 128 words per phase. See [Table 15-28](#) for a summary of how to calculate the frame length. This length corresponds to the number of words or logical time slots or channels per frame-synchronization pulse.

Program the RFLEN fields with  $[w \text{ minus } 1]$ , where  $w$  represents the number of words per phase. For the example, if you want a phase length of 128 words in phase 1, load 127 into RFLEN1.

**Table 15-28. How to Calculate the Length of the Receive Frame**

| RPHASE | RFLEN1                          | RFLEN2                          | Frame Length                      |
|--------|---------------------------------|---------------------------------|-----------------------------------|
| 0      | $0 \leq \text{RFLEN1} \leq 127$ | Don't care                      | (RFLEN1 + 1) words                |
| 1      | $0 \leq \text{RFLEN1} \leq 127$ | $0 \leq \text{RFLEN2} \leq 127$ | (RFLEN1 + 1) + (RFLEN2 + 1) words |

### 15.7.10 Enable/Disable the Receive Frame-Synchronization Ignore Function

The RFIG bit (see [Table 15-29](#)) controls the receive frame-synchronization ignore function.

**Table 15-29. Register Bit Used to Enable/Disable the Receive Frame-Synchronization Ignore Function**

| Register | Bit | Name | Function                                                                                                                                                                                                                                    | Type | Reset Value |
|----------|-----|------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------|-------------|
| RCR2     | 2   | RFIG | Receive frame-synchronization ignore<br><br>RFIG = 0    An unexpected receive frame-synchronization pulse causes the McBSP to restart the frame transfer.<br>RFIG = 1    The McBSP ignores unexpected receive frame-synchronization pulses. | R/W  | 0           |

#### 15.7.10.1 Unexpected Frame-Synchronization Pulses and the Frame-Synchronization Ignore Function

If a frame-synchronization pulse starts the transfer of a new frame before the current frame is fully received, this pulse is treated as an unexpected frame-synchronization pulse.

When RFIG = 1, reception continues, ignoring the unexpected frame-synchronization pulses.

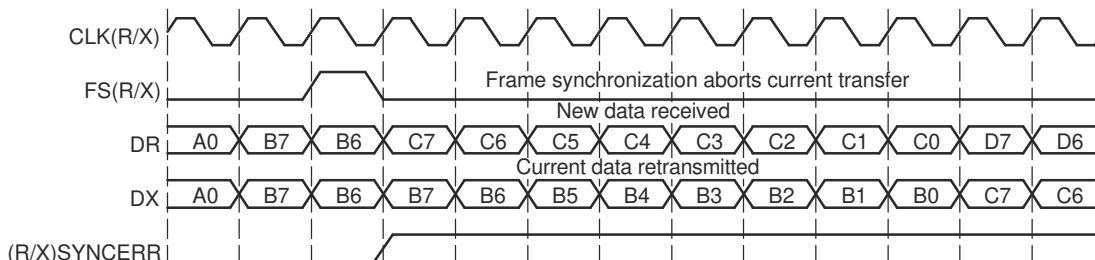
When RFIG = 0, an unexpected FSR pulse causes the McBSP to discard the contents of RSR[1,2] in favor of the new incoming data. Therefore, if RFIG = 0 and an unexpected frame-synchronization pulse occurs, the serial port:

1. Aborts the current data transfer
2. Sets RSYNCERR in SPCR1 to 1
3. Begins the transfer of a new data word

For more details about the frame-synchronization error condition, see [Section 15.4.3](#).

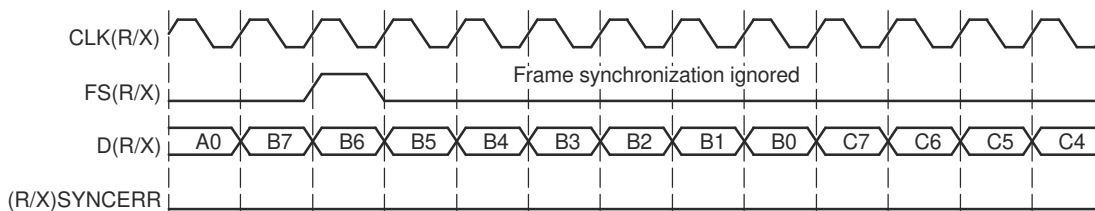
#### 15.7.10.2 Examples of Effects of RFIG

[Figure 15-43](#) shows an example in which word B is interrupted by an unexpected frame-synchronization pulse when (R/X)FIG = 0. In the case of reception, the reception of B is aborted (B is lost), and a new data word © in this example) is received after the appropriate data delay. This condition is a receive synchronization error, which sets the RSYNCERR bit.



**Figure 15-43. Unexpected Frame-Synchronization Pulse With (R/X)FIG = 0**

In contrast with [Figure 15-43](#), [Figure 15-44](#) shows McBSP operation when unexpected frame-synchronization signals are ignored (when (R/X)FIG = 1). Here, the transfer of word B is not affected by an unexpected pulse.



**Figure 15-44. Unexpected Frame-Synchronization Pulse With (R/X)FIG = 1**

#### 15.7.11 Set the Receive Companding Mode

The RCOMPAND bits (see [Table 15-30](#)) determine whether companding or another data transfer option is chosen for McBSP reception.

**Table 15-30. Register Bits Used to Set the Receive Companding Mode**

| Register | Bit | Name     | Function                                                                                                           | Type | Reset Value |
|----------|-----|----------|--------------------------------------------------------------------------------------------------------------------|------|-------------|
| RCR2     | 4-3 | RCOMPAND | Receive companding mode                                                                                            | R/W  | 00          |
|          |     |          | Modes other than 00b are enabled only when the appropriate RWDLEN is 000b, indicating 8-bit data.                  |      |             |
|          |     |          | RCOMPAND = 00 No companding, any size data, MSB received first                                                     |      |             |
|          |     |          | RCOMPAND = 01 No companding, 8-bit data, LSB received first (for details, see <a href="#">Section 15.7.11.4</a> ). |      |             |
|          |     |          | RCOMPAND = 10 $\mu$ -law companding, 8-bit data, MSB received first                                                |      |             |
|          |     |          | RCOMPAND = 11 A-law companding, 8-bit data, MSB received first                                                     |      |             |



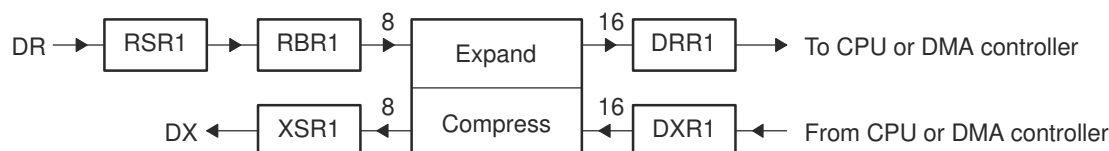
### 15.7.11.1 Companding

Companding (COMpressing and exPANDING) hardware allows compression and expansion of data in either  $\mu$ -law or A-law format. The companding standard employed in the United States and Japan is  $\mu$ -law. The European companding standard is referred to as A-law. The specifications for  $\mu$ -law and A-law log PCM are part of the CCITT G.711 recommendation.

A-law and  $\mu$ -law allow 13 bits and 14 bits of dynamic range, respectively. Any values outside this range are set to the most positive or most negative value. Thus, for companding to work best, the data transferred to and from the McBSP via the CPU or DMA controller must be at least 16 bits wide.

The  $\mu$ -law and A-law formats both encode data into 8-bit code words. Companded data is always 8 bits wide; the appropriate word length bits (RWDLEN1, RWDLEN2, XWDLEN1, XWDLEN2) must therefore be set to 0, indicating an 8-bit wide serial data stream. If companding is enabled and either of the frame phases does not have an 8-bit word length, companding continues as if the word length is 8 bits.

Figure 15-45 illustrates the companding processes. When companding is chosen for the transmitter, compression occurs during the process of copying data from DXR1 to XSR1. The transmit data is encoded according to the specified companding law (A-law or  $\mu$ -law). When companding is chosen for the receiver, expansion occurs during the process of copying data from RBR1 to DRR1. The receive data is decoded to 2's-complement format.



**Figure 15-45. Companding Processes for Reception and for Transmission**

### 15.7.11.2 Format of Expanded Data

For reception, the 8-bit compressed data in RBR1 is expanded to left-justified 16-bit data in DRR1. The RJUST bit of SPCR1 is ignored when companding is used.

### 15.7.11.3 Companding Internal Data

If the McBSP is otherwise unused (the serial port transmit and receive sections are reset), the companding hardware can compand internal data. See [Section 15.2.2.2](#).

### 15.7.11.4 Option to Receive LSB First

Normally, the McBSP transmits or receives all data with the most significant bit (MSB) first. However, certain 8-bit data protocols (that do not use companded data) require the least significant bit (LSB) to be transferred first. If you set RCOMPAND = 01b in RCR2, the bit ordering of 8-bit words is reversed during reception. Similar to companding, this feature is enabled only if the appropriate word length bits are set to 0, indicating that 8-bit words are to be transferred serially. If either phase of the frame does not have an 8-bit word length, the McBSP assumes the word length is eight bits and LSB-first ordering is done.

### 15.7.12 Set the Receive Data Delay

The RDATDLY bits (see [Table 15-31](#)) determine the length of the data delay for the receive frame.

**Table 15-31. Register Bits Used to Set the Receive Data Delay**

| Register | Bit | Name    | Function           | Type | Reset Value |
|----------|-----|---------|--------------------|------|-------------|
| RCR2     | 1-0 | RDATDLY | Receive data delay | R/W  | 00          |
|          |     |         | RDATDLY = 00       |      |             |
|          |     |         | RDATDLY = 01       |      |             |
|          |     |         | RDATDLY = 10       |      |             |



**Table 15-31. Register Bits Used to Set the Receive Data Delay (continued)**

| Register | Bit | Name | Function     | Type     | Reset Value |
|----------|-----|------|--------------|----------|-------------|
|          |     |      | RDATDLY = 11 | Reserved |             |

### 15.7.12.1 Data Delay

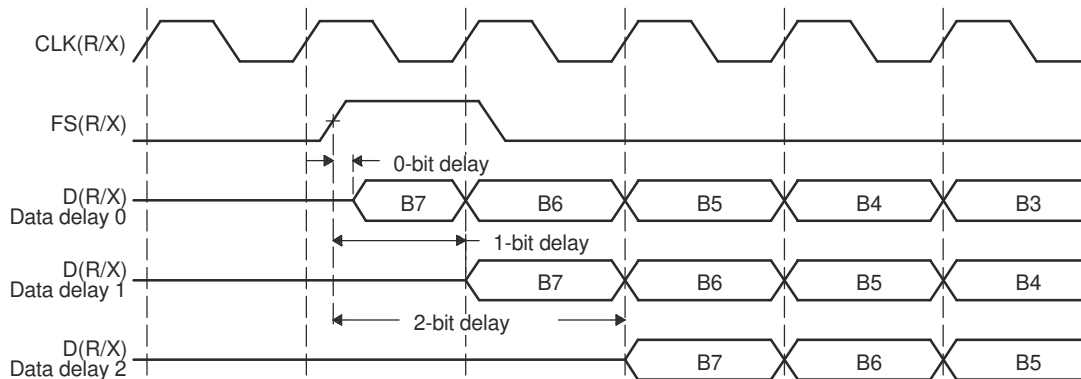
The start of a frame is defined by the first clock cycle in which frame synchronization is found to be active. The beginning of actual data reception or transmission with respect to the start of the frame can be delayed if required. This delay is called data delay.

RDATDLY specifies the data delay for reception. The range of programmable data delay is zero to two bit-clocks (RDATDLY = 00b-10b), as described in Table 15-31 and shown in Figure 15-46. In this figure, the data transferred is an 8-bit value with bits labeled B7, B6, B5, and so on. Typically a 1-bit delay is selected, because data often follows a 1-cycle active frame-synchronization pulse.

### 15.7.12.2 0-Bit Data Delay

Normally, a frame-synchronization pulse is detected or sampled with respect to an edge of internal serial clock CLK(R/X). Thus, on the following cycle or later (depending on the data delay value), data may be received or transmitted. However, in the case of 0-bit data delay, the data must be ready for reception and/or transmission on the same serial clock cycle.

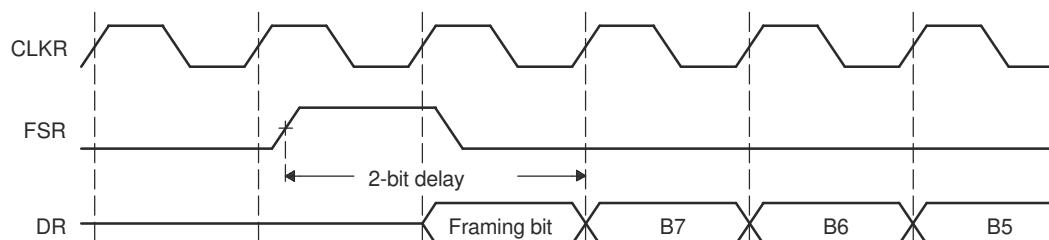
For reception, this problem is solved because receive data is sampled on the first falling edge of MCLKR where an active-high internal FSR is detected. However, data transmission must begin on the rising edge of the internal CLKX clock that generated the frame synchronization. Therefore, the first data bit is assumed to be present in XSR1, and thus on DX. The transmitter then asynchronously detects the frame-synchronization signal (FSX) going active high and immediately starts driving the first bit to be transmitted on the DX pin.



**Figure 15-46. Range of Programmable Data Delay**

### 15.7.12.3 2-Bit Data Delay

A data delay of two bit periods allows the serial port to interface to different types of T1 framing devices where the data stream is preceded by a framing bit. During reception of such a stream with data delay of two bits (framing bit appears after a 1-bit delay and data appears after a 2-bit delay), the serial port essentially discards the framing bit from the data stream, as shown in Figure 15-47. In this figure, the data transferred is an 8-bit value with bits labeled B7, B6, B5, and so on.



**Figure 15-47. 2-Bit Data Delay Used to Skip a Framing Bit**

### 15.7.13 Set the Receive Sign-Extension and Justification Mode

The RJUST bits (see [Table 15-32](#)) determine whether data received by the McBSP is sign-extended and how it is justified.

**Table 15-32. Register Bits Used to Set the Receive Sign-Extension and Justification Mode**

| Register | Bit   | Name  | Function                                                                   | Type | Reset Value |
|----------|-------|-------|----------------------------------------------------------------------------|------|-------------|
| SPCR1    | 14-13 | RJUST | Receive sign-extension and justification mode                              | R/W  | 00          |
|          |       |       | RJUST = 00 Right justify data and zero fill MSBs in DRR[1,2]               |      |             |
|          |       |       | RJUST = 01 Right justify data and sign extend it into the MSBs in DRR[1,2] |      |             |
|          |       |       | RJUST = 10 Left justify data and zero fill LSBs in DRR[1,2]                |      |             |
|          |       |       | RJUST = 11 Reserved                                                        |      |             |

#### 15.7.13.1 Sign-Extension and the Justification

RJUST in SPCR1 selects whether data in RBR[1,2] is right- or left-justified (with respect to the MSB) in DRR[1,2] and whether unused bits in DRR[1,2] are filled with zeros or with sign bits.

[Table 15-33](#) and [Table 15-34](#) show the effects of various RJUST values. The first table shows the effect on an example 12-bit receive-data value ABCCh. The second table shows the effect on an example 20-bit receive-data value ABCDEh.

**Table 15-33. Example: Use of RJUST Field With 12-Bit Data Value ABCCh**

| RJUST | Justification | Extension                  | Value in DRR2 | Value in DRR1 |
|-------|---------------|----------------------------|---------------|---------------|
| 00b   | Right         | Zero fill MSBs             | 0000h         | 0ABCCh        |
| 01b   | Right         | Sign extend data into MSBs | FFFFh         | FABCCh        |
| 10b   | Left          | Zero fill LSBs             | 0000h         | ABC0h         |
| 11b   | Reserved      | Reserved                   | Reserved      | Reserved      |

**Table 15-34. Example: Use of RJUST Field With 20-Bit Data Value ABCDEh**

| RJUST | Justification | Extension                  | Value in DRR2 | Value in DRR1 |
|-------|---------------|----------------------------|---------------|---------------|
| 00b   | Right         | Zero fill MSBs             | 000Ah         | BCDEh         |
| 01b   | Right         | Sign extend data into MSBs | FFFAh         | BCDEh         |
| 10b   | Left          | Zero fill LSBs             | ABCDh         | E000h         |
| 11b   | Reserved      | Reserved                   | Reserved      | Reserved      |

### 15.7.14 Set the Receive Interrupt Mode

The RINTM bits (see [Table 15-35](#)) determine which event generates a receive interrupt request to the CPU.

The receive interrupt (RINT) informs the CPU of changes to the serial port status. Four options exist for configuring this interrupt. The options are set by the receive interrupt mode bits, RINTM, in SPCR1.

**Table 15-35. Register Bits Used to Set the Receive Interrupt Mode**

| Register | Bit | Name  | Function                                                                                                                                                                                                                                                                                                                                                                                                                               | Type | Reset Value |
|----------|-----|-------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------|-------------|
| SPCR1    | 5-4 | RINTM | Receive interrupt mode                                                                                                                                                                                                                                                                                                                                                                                                                 | R/W  | 00          |
|          |     |       | RINTM = 00 RINT generated when RRDY changes from 0 to 1. Interrupt on every serial word by tracking the RRDY bit in SPCR1. Regardless of the value of RINTM, RRDY can be read to detect the RRDY = 1 condition.                                                                                                                                                                                                                        |      |             |
|          |     |       | RINTM = 01 RINT generated by an end-of-block or end-of-frame condition in the receive multichannel selection mode. In the multichannel selection mode, interrupt after every 16-channel block boundary has been crossed within a frame and at the end of the frame. For details, see <a href="#">Section 15.5.7.3</a> . In any other serial transfer case, this setting is not applicable and, therefore, no interrupts are generated. |      |             |
|          |     |       | RINTM = 10 RINT generated by a new receive frame-synchronization pulse. Interrupt on detection of receive frame-synchronization pulses. This generates an interrupt even when the receiver is in its reset state. This is done by synchronizing the incoming frame-synchronization pulse to the CPU clock and sending it to the CPU via RINT.                                                                                          |      |             |
|          |     |       | RINTM = 11 RINT generated when RSYNCERR is set. Interrupt on frame-synchronization error. Regardless of the value of RINTM, RSYNCERR can be read to detect this condition. For information on using RSYNCERR, see <a href="#">Section 15.4.3</a> .                                                                                                                                                                                     |      |             |

### 15.7.15 Set the Receive Frame-Synchronization Mode

The bits described in [Table 15-36](#) determine the source for receive frame synchronization and the function of the FSR pin.

**Table 15-36. Register Bits Used to Set the Receive Frame Synchronization Mode**

| Register | Bit   | Name   | Function                                                                                                                                                                                                      | Type | Reset Value |
|----------|-------|--------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------|-------------|
| PCR      | 10    | FSRM   | Receive frame-synchronization mode                                                                                                                                                                            | R/W  | 0           |
|          |       |        | FSRM = 0                                                                                                                                                                                                      |      |             |
|          |       |        | FSRM = 1                                                                                                                                                                                                      |      |             |
| SRGR2    | 15    | GSYNC  | Sample rate generator clock synchronization mode                                                                                                                                                              | R/W  | 0           |
|          |       |        | If the sample rate generator creates a frame-synchronization signal (FSG) that is derived from an external input clock, the GSYNC bit determines whether FSG is kept synchronized with pulses on the FSR pin. |      |             |
|          |       |        | GSYNC = 0                                                                                                                                                                                                     |      |             |
| SPCR1    | 15    | DLB    | Digital loopback mode                                                                                                                                                                                         | R/W  | 0           |
|          |       |        | DLB = 0                                                                                                                                                                                                       |      |             |
|          |       |        | DLB = 1                                                                                                                                                                                                       |      |             |
| SPCR1    | 12-11 | CLKSTP | Clock stop mode                                                                                                                                                                                               | R/W  | 00          |
|          |       |        | CLKSTP = 0Xb                                                                                                                                                                                                  |      |             |
|          |       |        | CLKSTP = 10b                                                                                                                                                                                                  |      |             |
|          |       |        | CLKSTP = 11b                                                                                                                                                                                                  |      |             |

### 15.7.15.1 Receive Frame-Synchronization Modes

Table 15-37 shows how you can select various sources to provide the receive frame-synchronization signal and the effect on the FSR pin. The polarity of the signal on the FSR pin is determined by the FSRP bit.

In digital loopback mode (DLB = 1), the transmit frame-synchronization signal is used as the receive frame-synchronization signal.

Also in the clock stop mode, the internal receive clock signal (MCLKR) and the internal receive frame-synchronization signal (FSR) are internally connected to their transmit counterparts, CLKX and FSX.

**Table 15-37. Select Sources to Provide the Receive Frame-Synchronization Signal and the Effect on the FSR Pin**

| DLB | FSRM   | GSYNC  | Source of Receive Frame Synchronization                                                                                                                             | FSR Pin Status                                                                                                                     |
|-----|--------|--------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|
| 0   | 0      | 0 or 1 | An external frame-synchronization signal enters the McBSP through the FSR pin. The signal is then inverted as determined by FSRP before being used as internal FSR. | Input                                                                                                                              |
| 0   | 1      | 0      | Internal FSR is driven by the sample rate generator frame-synchronization signal (FSG).                                                                             | Output. FSG is inverted as determined by FSRP before being driven out on the FSR pin.                                              |
| 0   | 1      | 1      | Internal FSR is driven by the sample rate generator frame-synchronization signal (FSG).                                                                             | Input. The external frame-synchronization input on the FSR pin is used to synchronize CLKG and generate FSG pulses.                |
| 1   | 0      | 0      | Internal FSX drives internal FSR.                                                                                                                                   | High impedance                                                                                                                     |
| 1   | 0 or 1 | 1      | Internal FSX drives internal FSR.                                                                                                                                   | Input. If the sample rate generator is running, external FSR is used to synchronize CLKG and generate FSG pulses.                  |
| 1   | 1      | 0      | Internal FSX drives internal FSR.                                                                                                                                   | Output. Receive (same as transmit) frame synchronization is inverted as determined by FSRP before being driven out on the FSR pin. |

### 15.7.16 Set the Receive Frame-Synchronization Polarity

The FSRP bit (see [Table 15-38](#)) determines whether frame-synchronization pulses are active high or active low on the FSR pin.

**Table 15-38. Register Bit Used to Set Receive Frame-Synchronization Polarity**

| Register | Bit | Name | Function                               | Type | Reset Value                                     |
|----------|-----|------|----------------------------------------|------|-------------------------------------------------|
| PCR      | 2   | FSRP | Receive frame-synchronization polarity | R/W  | 0                                               |
|          |     |      | FSRP = 0                               |      | Frame-synchronization pulse FSR is active high. |
|          |     |      | FSRP = 1                               |      | Frame-synchronization pulse FSR is active low.  |

#### 15.7.16.1 Frame-Synchronization Pulses, Clock Signals, and Their Polarities

Receive frame-synchronization pulses can be generated internally by the sample rate generator (see [Section 15.3.2](#)) or driven by an external source. The source of frame synchronization is selected by programming the mode bit, FSRM, in PCR. FSR is also affected by the GSYNC bit in SRGR2. For information about the effects of FSRM and GSYNC, see [Section 15.7.15](#). Similarly, receive clocks can be selected to be inputs or outputs by programming the mode bit, CLKRM, in the PCR (see [Section 15.7.17](#)).

When FSR and FSX are inputs (FSXM = FSRM = 0, external frame-synchronization pulses), the McBSP detects them on the internal falling edge of clock, internal MCLKR, and internal CLKX, respectively. The receive data arriving at the DR pin is also sampled on the falling edge of internal MCLKR. These internal clock signals are either derived from an external source via CLK(R/X) pins or driven by the sample rate generator clock (CLKG) internal to the McBSP.

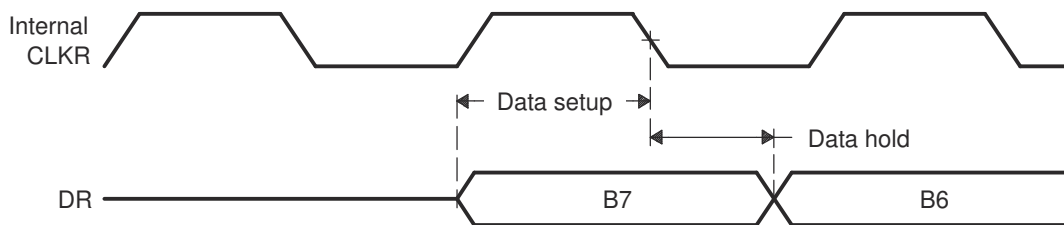
When FSR and FSX are outputs, implying that they are driven by the sample rate generator, they are generated (transition to their active state) on the rising edge of the internal clock, CLK(R/X). Similarly, data on the DX pin is output on the rising edge of internal CLKX.

FSRP, FSXP, CLKRP, and CLKXP in the pin control register (PCR) configure the polarities of the FSR, FSX, MCLKR, and CLKX signals, respectively. All frame-synchronization signals (internal FSR, internal FSX) that are internal to the serial port are active high. If the serial port is configured for external frame synchronization (FSR/FSX are inputs to McBSP), and FSRP = FSXP = 1, the external active-low frame-synchronization signals are inverted before being sent to the receiver (internal FSR) and transmitter (internal FSX). Similarly, if internal synchronization (FSR/FSX are output pins and GSYNC = 0) is selected, the internal active-high frame-synchronization signals are inverted, if the polarity bit FS(R/X)P = 1, before being sent to the FS(R/X) pin.

On the transmit side, the transmit clock polarity bit, CLKXP, sets the edge used to shift and clock out transmit data. Data is always transmitted on the rising edge of internal CLKX. If CLKXP = 1 and external clocking is selected (CLKXM = 0 and CLKX is an input), the external falling-edge triggered input clock on CLKX is inverted to a rising-edge triggered clock before being sent to the transmitter. If CLKXP = 1, and internal clocking selected (CLKXM = 1 and CLKX is an output pin), the internal (rising-edge triggered) clock, internal CLKX, is inverted before being sent out on the MCLKX pin.

Similarly, the receiver can reliably sample data that is clocked with a rising edge clock (by the transmitter). The receive clock polarity bit, CLKRP, sets the edge used to sample received data. The receive data is always sampled on the falling edge of internal MCLKR. Therefore, if CLKRP = 1 and external clocking is selected (CLKRM = 0 and MCLKR is an input pin), the external rising-edge triggered input clock on MCLKR is inverted to a falling-edge triggered clock before being sent to the receiver. If CLKRP = 1 and internal clocking is selected (CLKRM = 1), the internal falling-edge triggered clock is inverted to a rising-edge triggered clock before being sent out on the MCLKR pin.

MCLKRP = CLKXP in a system where the same clock (internal or external) is used to clock the receiver and transmitter. The receiver uses the opposite edge as the transmitter to ensure valid setup and hold of data around this edge. [Figure 15-48](#) shows how data clocked by an external serial device using a rising edge can be sampled by the McBSP receiver on the falling edge of the same clock.



**Figure 15-48. Data Clocked Externally Using a Rising Edge and Sampled by the McBSP Receiver on a Falling Edge**

Set the SRG Frame-Synchronization Period and Pulse Width.

#### 15.7.16.2 Frame-Synchronization Period and the Frame-Synchronization Pulse Width

The sample rate generator can produce a clock signal, CLKG, and a frame-synchronization signal, FSG. If the sample rate generator is supplying receive or transmit frame synchronization, you must program the bit fields FPER and FWID.

On FSG, the period from the start of a frame-synchronization pulse to the start of the next pulse is  $(FPER + 1)$  CLKG cycles. The 12 bits of FPER allow a frame-synchronization period of 1 to 4096 CLKG cycles, which allows up to 4096 data bits per frame. When GSYNC = 1, FPER is a don't care value.

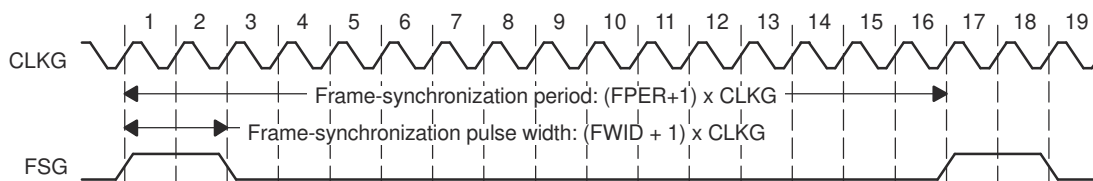
Each pulse on FSG has a width of  $(FWID + 1)$  CLKG cycles. The eight bits of FWID allow a pulse width of 1 to 256 CLKG cycles. It is recommended that FWID be programmed to a value less than the programmed word length.

The values in FPER and FWID are loaded into separate down-counters. The 12-bit FPER counter counts down the generated clock cycles from the programmed value (4095 maximum) to 0. The 8-bit FWID counter counts down from the programmed value (255 maximum) to 0. Table 15-39 shows settings for FPER and FWID.

Figure 15-49 shows a frame-synchronization period of 16 CLKG periods ( $FPER = 15$  or 00001111b) and a frame-synchronization pulse with an active width of 2 CLKG periods ( $FWID = 1$ ).

**Table 15-39. Register Bits Used to Set the SRG Frame-Synchronization Period and Pulse Width**

| Register | Bit  | Name | Function                                                                                                                                                                                                                                                                                              | Type | Reset Value    |
|----------|------|------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------|----------------|
| SRGR2    | 11-0 | FPER | Sample rate generator frame-synchronization period<br><br>For the frame-synchronization signal FSG, $(FPER + 1)$ determines the period from the start of a frame-synchronization pulse to the start of the next frame-synchronization pulse.<br><br>Range for $(FPER + 1)$ :<br>1 to 4096 CLKG cycles | R/W  | 0000 0000 0000 |
| SRGR1    | 15-8 | FWID | Sample rate generator frame-synchronization pulse width<br><br>This field plus 1 determines the width of each frame-synchronization pulse on FSG.<br><br>Range for $(FWID + 1)$ :<br>1 to 256 CLKG cycles                                                                                             | R/W  | 0000 0000      |



**Figure 15-49. Frame of Period 16 CLKG Periods and Active Width of 2 CLKG Periods**

When the sample rate generator comes out of reset, FSG is in its inactive state. Then, when GRST = 1 and FSGM = 1, a frame-synchronization pulse is generated. The frame width value  $(FWID + 1)$  is counted down on

every CLKG cycle until it reaches 0, at which time FSG goes low. At the same time, the frame period value (FPER + 1) is also counting down. When this value reaches 0, FSG goes high, indicating a new frame.

### 15.7.17 Set the Receive Clock Mode

Table 15-40 shows the settings for bits used to set receive clock mode.

**Table 15-40. Register Bits Used to Set the Receive Clock Mode**

| Register | Bit   | Name   | Function                                                                                                                                                                                                                                    | Type | Reset Value |
|----------|-------|--------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------|-------------|
| PCR      | 8     | CLKRM  | Receive clock mode                                                                                                                                                                                                                          | R/W  | 0           |
|          |       |        | Case 1: Digital loopback mode not set (DLB = 0) in SPCR1.                                                                                                                                                                                   |      |             |
|          |       |        | CLKRM = 0      The MCLKR pin is an input pin that supplies the internal receive clock (MCLKR).                                                                                                                                              |      |             |
|          |       |        | CLKRM = 1      Internal MCLKR is driven by the sample rate generator of the McBSP. The MCLKR pin is an output pin that reflects internal MCLKR.                                                                                             |      |             |
|          |       |        | Case 2: Digital loopback mode set (DLB = 1) in SPCR1.                                                                                                                                                                                       |      |             |
|          |       |        | CLKRM = 0      The MCLKR pin is in the high impedance state. The internal receive clock (MCLKR) is driven by the internal transmit clock (CLKX). Internal CLKX is derived according to the CLKXM bit of PCR.                                |      |             |
| SPCR1    | 15    | DLB    | Digital loopback mode                                                                                                                                                                                                                       | R/W  | 00          |
|          |       |        | DLB = 0      Digital loopback mode is disabled.                                                                                                                                                                                             |      |             |
|          |       |        | DLB = 1      Digital loopback mode is enabled. The receive signals, including the receive frame-synchronization signal, are connected internally through multiplexers to the corresponding transmit signals.                                |      |             |
| SPCR1    | 12-11 | CLKSTP | Clock stop mode                                                                                                                                                                                                                             | R/W  | 00          |
|          |       |        | CLKSTP = 0Xb      Clock stop mode disabled; normal clocking for non-SPI mode.                                                                                                                                                               |      |             |
|          |       |        | CLKSTP = 10b      Clock stop mode enabled without clock delay. The internal receive clock signal (MCLKR) and the internal receive frame-synchronization signal (FSR) are internally connected to their transmit counterparts, CLKX and FSX. |      |             |
|          |       |        | CLKSTP = 11b      Clock stop mode enabled with clock delay. The internal receive clock signal (MCLKR) and the internal receive frame-synchronization signal (FSR) are internally connected to their transmit counterparts, CLKX and FSX.    |      |             |

#### 15.7.17.1 Selecting a Source for the Receive Clock and a Data Direction for the MCLKR Pin

Table 15-41 shows how you can select various sources to provide the receive clock signal and affect the MCLKR pin. The polarity of the signal on the MCLKR pin is determined by the CLKRP bit.

In the digital loopback mode (DLB = 1), the transmit clock signal is used as the receive clock signal.

Also, in the clock stop mode, the internal receive clock signal (MCLKR) and the internal receive frame-synchronization signal (FSR) are internally connected to their transmit counterparts, CLKX and FSX.



**Table 15-41. Receive Clock Signal Source Selection**

| DLB in SPCR1 | CLKRM in PCR | Source of Receive Clock                                                                                                                | MCLKR Pin Status                                                                                                            |
|--------------|--------------|----------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| 0            | 0            | The MCLKR pin is an input driven by an external clock. The external clock signal is inverted as determined by CLKRP before being used. | Input                                                                                                                       |
| 0            | 1            | The sample rate generator clock (CLKG) drives internal MCLKR.                                                                          | Output. CLKG, inverted as determined by CLKRP, is driven out on the MCLKR pin.                                              |
| 1            | 0            | Internal CLKX drives internal MCLKR. To configure CLKX, see <a href="#">Section 15.8.18</a> .                                          | High impedance                                                                                                              |
| 1            | 1            | Internal CLKX drives internal MCLKR. To configure CLKX, see <a href="#">Section 15.8.18</a> .                                          | Output. Internal MCLKR (same as internal CLKX) is inverted as determined by CLKRP before being driven out on the MCLKR pin. |

### 15.7.18 Set the Receive Clock Polarity

**Table 15-42. Register Bit Used to Set Receive Clock Polarity**

| Register | Bit | Name  | Function                                                     | Type | Reset Value |
|----------|-----|-------|--------------------------------------------------------------|------|-------------|
| PCR      | 0   | CLKRP | Receive clock polarity                                       | R/W  | 0           |
|          |     |       | CLKRP = 0      Receive data sampled on falling edge of MCLKR |      |             |
|          |     |       | CLKRP = 1      Receive data sampled on rising edge of MCLKR  |      |             |

#### 15.7.18.1 Frame Synchronization Pulses, Clock Signals, and Their Polarities

Receive frame-synchronization pulses can be generated internally by the sample rate generator (see [Section 15.3.2](#)) or driven by an external source. The source of frame synchronization is selected by programming the mode bit, FSRM, in PCR. FSR is also affected by the GSYNC bit in SRGR2. For information about the effects of FSRM and GSYNC, see [Section 15.7.15](#). Similarly, receive clocks can be selected to be inputs or outputs by programming the mode bit, CLKRM, in the PCR (see [Section 15.7.17](#)).

When FSR and FSX are inputs (FSXM = FSRM = 0, external frame-synchronization pulses), the McBSP detects them on the internal falling edge of clock, internal MCLKR, and internal CLKX, respectively. The receive data arriving at the DR pin is also sampled on the falling edge of internal MCLKR. These internal clock signals are either derived from external source via CLK(R/X) pins or driven by the sample rate generator clock (CLKG) internal to the McBSP.

When FSR and FSX are outputs, implying that they are driven by the sample rate generator, they are generated (transition to their active state) on the rising edge of internal clock, CLK(R/X). Similarly, data on the DX pin is output on the rising edge of internal CLKX.

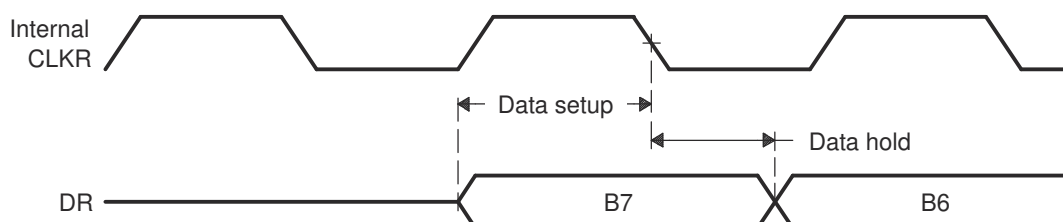
FSRP, FSXP, CLKRP, and CLKXP in the pin control register (PCR) configure the polarities of the FSR, FSX, MCLKR, and CLKX signals, respectively. All frame-synchronization signals (internal FSR, internal FSX) that are internal to the serial port are active high. If the serial port is configured for external frame synchronization (FSR/FSX are inputs to McBSP) and FSRP = FSXP = 1, the external active-low frame-synchronization signals are inverted before being sent to the receiver (internal FSR) and transmitter (internal FSX). Similarly, if internal synchronization (FSR/FSX are output pins and GSYNC = 0) is selected, the internal active-high frame-synchronization signals are inverted, if the polarity bit FS(R/X)P = 1, before being sent to the FS(R/X) pin.

On the transmit side, the transmit clock polarity bit, CLKXP, sets the edge used to shift and clock out transmit data. Data is always transmitted on the rising edge of internal CLKX. If CLKXP = 1 and external clocking is selected (CLKXM = 0 and CLKX is an input), the external falling-edge triggered input clock on CLKX is inverted to a rising-edge triggered clock before being sent to the transmitter. If CLKXP = 1 and internal clocking is selected (CLKXM = 1 and CLKX is an output pin), the internal (rising-edge triggered) clock, internal CLKX, is inverted before being sent out on the MCLKX pin.

Similarly, the receiver can reliably sample data that is clocked with a rising edge clock (by the transmitter). The receive clock polarity bit, CLKRP, sets the edge used to sample received data. The receive data is always sampled on the falling edge of internal MCLKR. Therefore, if CLKRP = 1 and external clocking is selected

(CLKRM = 0 and MCLKR is an input pin), the external rising-edge triggered input clock on MCLKR is inverted to a falling-edge triggered clock before being sent to the receiver. If CLKRP = 1 and internal clocking is selected (CLKRM = 1), the internal falling-edge triggered clock is inverted to a rising-edge triggered clock before being sent out on the MCLKR pin.

CLKRP = CLKXP in a system where the same clock (internal or external) is used to clock the receiver and transmitter. The receiver uses the opposite edge as the transmitter to ensure valid setup and hold of data around this edge. Figure 15-50 shows how data clocked by an external serial device using a rising edge can be sampled by the McBSP receiver on the falling edge of the same clock.



**Figure 15-50. Data Clocked Externally Using a Rising Edge and Sampled by the McBSP Receiver on a Falling Edge**

### 15.7.19 Set the SRG Clock Divide-Down Value

**Table 15-43. Register Bits Used to Set the Sample Rate Generator (SRG) Clock Divide-Down Value**

| Register | Bit | Name   | Function                                                                                                                                                                                                                                                | Type | Reset Value |
|----------|-----|--------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------|-------------|
| SRGR1    | 7-0 | CLKGDV | Sample rate generator clock divide-down value<br>The input clock of the sample rate generator is divided by (CLKGDV + 1) to generate the required sample rate generator clock frequency.<br>The default value of CLKGDV is 1 (divide input clock by 2). | R/W  | 0000 0001   |

#### 15.7.19.1 Sample Rate Generator Clock Divider

The first divider stage generates the serial data bit clock from the input clock. This divider stage utilizes a counter, preloaded by CLKGDV, that contains the divide ratio value.

The output of the first divider stage is the data bit clock, which is output as CLKG and which serves as the input for the second and third stages of the divider.

CLKG has a frequency equal to  $1/(\text{CLKGDV} + 1)$  of sample rate generator input clock. Thus, the sample generator input clock frequency is divided by a value between 1 and 256. When CLKGDV is odd or equal to 0, the CLKG duty cycle is 50%. When CLKGDV is an even value,  $2p$ , representing an odd divide-down, the high-state duration is  $p + 1$  cycles and the low-state duration is  $p$  cycles.

### 15.7.20 Set the SRG Clock Synchronization Mode

For more details on using the clock synchronization feature, see [Section 15.3.3](#).

**Table 15-44. Register Bit Used to Set the SRG Clock Synchronization Mode**

| Register | Bit | Name  | Function                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                | Type | Reset Value |
|----------|-----|-------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------|-------------|
| SRGR2    | 15  | GSYNC | <p>Sample rate generator clock synchronization</p> <p>GSYNC is used only when the input clock source for the sample rate generator is external—on the MCLKR or MCLKX pin.</p> <p>GSYNC = 0      The sample rate generator clock (CLKG) is free running. CLKG oscillates without adjustment, and FSG pulses every (FPER + 1) CLKG cycles.</p> <p>GSYNC = 1      Clock synchronization is performed. When a pulse is detected on the FSR pin:</p> <ul style="list-style-type: none"> <li>• CLKG is adjusted as necessary so that it is synchronized with the input clock on the MCLKR or MCLKX pin.</li> <li>• FSG pulses. FSG only pulses in response to a pulse on the FSR pin. The frame-synchronization period defined in FPER is ignored.</li> </ul> | R/W  | 0           |

### 15.7.21 Set the SRG Clock Mode (Choose an Input Clock)

**Table 15-45. Register Bits Used to Set the SRG Clock Mode (Choose an Input Clock)**

| Register | Bit | Name   | Function                                                                                                                                                                                                                                                                                                                                 | Type | Reset Value |
|----------|-----|--------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------|-------------|
| PCR      | 7   | SCLKME | Sample rate generator clock mode                                                                                                                                                                                                                                                                                                         | R/W  | 0           |
| SRGR2    | 13  | CLKSM  | <p>SCLKME = 0      Reserved</p> <p>CLKSM = 0</p> <p>SCLKME = 0      Sample rate generator clock derived from LSPCLK (default)</p> <p>CLKSM = 1</p> <p>SCLKME = 1      Sample rate generator clock derived from MCLKR pin</p> <p>CLKSM = 0</p> <p>SCLKME = 1      Sample rate generator clock derived from MCLKX pin</p> <p>CLKSM = 1</p> | R/W  | 1           |

#### 15.7.21.1 SRG Clock Mode

The sample rate generator can produce a clock signal (CLKG) for use by the receiver, the transmitter, or both, but CLKG is derived from an input clock. [Table 15-45](#) shows the four possible sources of the input clock. For more details on generating CLKG, see [Section 15.3.1.1](#).

## 15.7.22 Set the SRG Input Clock Polarity

**Table 15-46. Register Bits Used to Set the SRG Input Clock Polarity**

| Register | Bit | Name  | Function                                                                                                                                                                                                                                                                                                                 | Type | Reset Value |
|----------|-----|-------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------|-------------|
| PCR      | 1   | CLKXP | MCLKX pin polarity<br><br>CLKXP determines the input clock polarity when the MCLKX pin supplies the input clock (SCLKME = 1 and CLKSM = 1).<br><br>CLKXP = 0      Rising edge on MCLKX pin generates transitions on CLKG and FSG.<br><br>CLKXP = 1      Falling edge on MCLKX pin generates transitions on CLKG and FSG. | R/W  | 0           |
| PCR      | 0   | CLKRP | MCLKR pin polarity<br><br>CLKRP determines the input clock polarity when the MCLKR pin supplies the input clock (SCLKME = 1 and CLKSM = 0).<br><br>CLKRP = 0      Falling edge on MCLKR pin generates transitions on CLKG and FSG.<br><br>CLKRP = 1      Rising edge on MCLKR pin generates transitions on CLKG and FSG. | R/W  | 0           |

### 15.7.22.1 Using CLKXP/CLKRP to Choose an Input Clock Polarity

The sample rate generator can produce a clock signal (CLKG) and a frame-synchronization signal (FSG) for use by the receiver, the transmitter, or both. To produce CLKG and FSG, the sample rate generator must be driven by an input clock signal derived from the CPU clock or from an external clock on the CLKX or MCLKR pin. If you use a pin, choose a polarity for that pin by using the appropriate polarity bit (CLKXP for the MCLKX pin, CLKRP for the MCLKR pin). The polarity determines whether the rising or falling edge of the input clock generates transitions on CLKG and FSG.

## 15.8 Transmitter Configuration

To configure the McBSP transmitter, perform the following procedure:

1. Place the McBSP/transmitter in reset (see [Section 15.8.2](#)).
2. Program the McBSP registers for the desired transmitter operation (see [Section 15.8.1](#)).
3. Take the transmitter out of reset (see [Section 15.8.2](#)).

### 15.8.1 Programming the McBSP Registers for the Desired Transmitter Operation

The following is a list of important tasks to be performed when you are configuring the McBSP transmitter. Each task corresponds to one or more McBSP register bit fields.

- Global behavior:
  - Set the transmitter pins to operate as McBSP pins.
  - Enable/disable the digital loopback mode.
  - Enable/disable the clock stop mode.
  - Enable/disable transmit multichannel selection.
- Data behavior:
  - Choose 1 or 2 phases for the transmit frame.
  - Set the transmit word length(s).
  - Set the transmit frame length.
  - Enable/disable the transmit frame-synchronization ignore function.
  - Set the transmit companding mode.
  - Set the transmit data delay.
  - Set the transmit DXENA mode.
  - Set the transmit interrupt mode.
- Frame-synchronization behavior:
  - Set the transmit frame-synchronization mode.

- Set the transmit frame-synchronization polarity.
- Set the SRG frame-synchronization period and pulse width.
- Clock behavior:
  - Set the transmit clock mode.
  - Set the transmit clock polarity.
  - Set the SRG clock divide-down value.
  - Set the SRG clock synchronization mode.
  - Set the SRG clock mode (choose an input clock).
  - Set the SRG input clock polarity.

## 15.8.2 Resetting and Enabling the Transmitter

The first step of the transmitter configuration procedure is to reset the transmitter, and the last step is to enable the transmitter (to take it out of reset). [Table 15-47](#) describes the bits used for both of these steps.

**Table 15-47. Register Bits Used to Place Transmitter in Reset Field Descriptions**

| Register | Bit | Field | Value | Description                                                                                                                                                                                                                                                                         |
|----------|-----|-------|-------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| SPCR2    | 7   | FRST  | 0     | Frame-synchronization logic reset<br>Frame-synchronization logic is reset. The sample rate generator does not generate frame-synchronization signal FSG, even if GRST = 1.                                                                                                          |
|          |     |       | 1     | Frame-synchronization is enabled. If GRST = 1, frame-synchronization signal FSG is generated after (FPER + 1) number of CLKG clock cycles; all frame counters are loaded with their programmed values.                                                                              |
| SPCR2    | 6   | GRST  | 0     | Sample rate generator reset<br>Sample rate generator is reset. If GRST = 0 due to a device reset, CLKG is driven by the CPU clock divided by 2, and FSG is driven low (inactive). If GRST = 0 due to program code, CLKG and FSG are both driven low (inactive).                     |
|          |     |       | 1     | Sample rate generator is enabled. CLKG is driven according to the configuration programmed in the sample rate generator registers (SRGR[1,2]). If FRST = 1, the generator also generates the frame-synchronization signal FSG as programmed in the sample rate generator registers. |
| SPCR2    | 0   | XRST  | 0     | Transmitter reset<br>The serial port transmitter is disabled and in the reset state.                                                                                                                                                                                                |
|          |     |       | 1     | The serial port transmitter is enabled.                                                                                                                                                                                                                                             |

### 15.8.2.1 Reset Considerations

The serial port can be reset in the following two ways:

1. A DSP reset ( $\overline{\text{XRS}}$  signal driven low) places the receiver, transmitter, and sample rate generator in reset. When the device reset is removed, GRST = FRST = RRST = XRST = 0, keeping the entire serial port in the reset state.
2. The serial port transmitter and receiver can be reset directly using the RRST and XRST bits in the serial port control registers. The sample rate generator can be reset directly using the GRST bit in SPCR2.
3. When using the DMA, the order in which McBSP events must occur is important. DMA channel and peripheral interrupts must be configured prior to releasing the McBSP transmitter from reset.

The reason for this is that an XRDY is fired when XRST = 1. The XRDY signals the DMA to start copying data from the buffer into the transmit register. If the McBSP transmitter is released from reset before the DMA channel and peripheral interrupts are configured, the XRDY signals before the DMA channel can receive the signal; therefore, the DMA does not move the data from the buffer to the transmit register. The DMA PERINTFLG is edge-sensitive and will fail to recognize the XRDY, which is continuously high.

For more details about McBSP reset conditions and effects, see [Section 15.9.2](#).

### 15.8.3 Set the Transmitter Pins to Operate as McBSP Pins

To configure a pin for its McBSP function, you should configure the bits of the GPxMUXn register appropriately. In addition to this, bits 12 and 13 of the PCR register must be set to 0. These bits are defined as reserved.

### 15.8.4 Enable/Disable the Digital Loopback Mode

The DLB bit determines whether the digital loopback mode is on. DLB is described in [Table 15-48](#).

**Table 15-48. Register Bit Used to Enable/Disable the Digital Loopback Mode**

| Register | Bit | Name | Function              | Type | Reset Value                        |
|----------|-----|------|-----------------------|------|------------------------------------|
| SPCR1    | 15  | DLB  | Digital loopback mode | R/W  | 0                                  |
|          |     |      | DLB = 0               |      | Digital loopback mode is disabled. |
|          |     |      | DLB = 1               |      | Digital loopback mode is enabled.  |

#### 15.8.4.1 Digital Loopback Mode

In the digital loopback mode, the receive signals are connected internally through multiplexers to the corresponding transmit signals, as shown in [Table 15-49](#). This mode allows testing of serial port code with a single DSP device; the McBSP receives the data it transmits.

**Table 15-49. Receive Signals Connected to Transmit Signals in Digital Loopback Mode**

| This Receive Signal                 | Is Fed Internally by This Transmit Signal |
|-------------------------------------|-------------------------------------------|
| DR (receive data)                   | DX (transmit data)                        |
| FSR (receive frame synchronization) | FSX (transmit frame synchronization)      |
| MCLKR (receive clock)               | CLKX (transmit clock)                     |

### 15.8.5 Enable/Disable the Clock Stop Mode

The CLKSTP bits determine whether the clock stop mode is on. CLKSTP is described in [Table 15-50](#).

**Table 15-50. Register Bits Used to Enable/Disable the Clock Stop Mode**

| Register | Bit   | Name   | Function        | Type | Reset Value                                                 |
|----------|-------|--------|-----------------|------|-------------------------------------------------------------|
| SPCR1    | 12-11 | CLKSTP | Clock stop mode | R/W  | 00                                                          |
|          |       |        | CLKSTP = 0Xb    |      | Clock stop mode disabled; normal clocking for non-SPI mode. |
|          |       |        | CLKSTP = 10b    |      | Clock stop mode enabled without clock delay                 |
|          |       |        | CLKSTP = 11b    |      | Clock stop mode enabled with clock delay                    |

#### 15.8.5.1 Clock Stop Mode

The clock stop mode supports the SPI master-slave protocol. If you do not plan to use the SPI protocol, you can clear CLKSTP to disable the clock stop mode.

In the clock stop mode, the clock stops at the end of each data transfer. At the beginning of each data transfer, the clock starts immediately (CLKSTP = 10b) or after a half-cycle delay (CLKSTP = 11b). The CLKXP bit determines whether the starting edge of the clock on the MCLKX pin is rising or falling. The CLKRP bit determines whether receive data is sampled on the rising or falling edge of the clock shown on the MCLKR pin.

[Table 15-51](#) summarizes the impact of CLKSTP, CLKXP, and CLKRP on serial port operation. In the clock stop mode, the receive clock is tied internally to the transmit clock, and the receive frame-synchronization signal is tied internally to the transmit frame-synchronization signal.

**Table 15-51. Effects of CLKSTP, CLKXP, and CLKRP on the Clock Scheme**

| Bit Settings                                            | Clock Scheme                                                                                                                                              |
|---------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| CLKSTP = 00b or 01b<br>CLKXP = 0 or 1<br>CLKRP = 0 or 1 | Clock stop mode disabled. Clock enabled for non-SPI mode.                                                                                                 |
| CLKSTP = 10b<br>CLKXP = 0<br>CLKRP = 0                  | Low inactive state without delay: The McBSP transmits data on the rising edge of CLKX and receives data on the falling edge of MCLKR.                     |
| CLKSTP = 11b<br>CLKXP = 0<br>CLKRP = 1                  | Low inactive state with delay: The McBSP transmits data one-half cycle ahead of the rising edge of CLKX and receives data on the rising edge of MCLKR.    |
| CLKSTP = 10b<br>CLKXP = 1<br>CLKRP = 0                  | High inactive state without delay: The McBSP transmits data on the falling edge of CLKX and receives data on the rising edge of MCLKR.                    |
| CLKSTP = 11b<br>CLKXP = 1<br>CLKRP = 1                  | High inactive state with delay: The McBSP transmits data one-half cycle ahead of the falling edge of CLKX and receives data on the falling edge of MCLKR. |

### 15.8.6 Enable/Disable Transmit Multichannel Selection

For more details, see [Section 15.5.7](#).

**Table 15-52. Register Bits Used to Enable/Disable Transmit Multichannel Selection**

| Register | Bit | Name | Function                                                                                                                                                                                                                                                                                                                                                                                                                                                        | Type | Reset Value |
|----------|-----|------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------|-------------|
| MCR2     | 1-0 | XMCM | Transmit multichannel selection                                                                                                                                                                                                                                                                                                                                                                                                                                 | R/W  | 00          |
|          |     |      | XMCM = 00b<br>No transmit multichannel selection mode is on. All channels are enabled and unmasked. No channels can be disabled or masked.                                                                                                                                                                                                                                                                                                                      |      |             |
|          |     |      | XMCM = 01b<br>All channels are disabled unless they are selected in the appropriate transmit channel enable registers (XCERs). If enabled, a channel in this mode is also unmasked.<br><br>The XMCME bit determines whether 32 channels or 128 channels are selectable in XCERs.                                                                                                                                                                                |      |             |
|          |     |      | XMCM = 10b<br>All channels are enabled, but they are masked unless they are selected in the appropriate transmit channel enable registers (XCERs).<br><br>The XMCME bit determines whether 32 channels or 128 channels are selectable in XCERs.                                                                                                                                                                                                                 |      |             |
|          |     |      | XMCM = 11b<br>This mode is used for symmetric transmission and reception.<br><br>All channels are disabled for transmission unless they are enabled for reception in the appropriate receive channel enable registers (RCERs). Once enabled, they are masked unless they are also selected in the appropriate transmit channel enable registers (XCERs).<br><br>The XMCME bit determines whether 32 channels or 128 channels are selectable in RCERs and XCERs. |      |             |

## 15.8.7 Choose One or Two Phases for the Transmit Frame

**Table 15-53. Register Bit Used to Choose 1 or 2 Phases for the Transmit Frame**

| Register | Bit | Name   | Function                                                                                                                                                               | Type | Reset Value |
|----------|-----|--------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------|-------------|
| XCR2     | 15  | XPHASE | Transmit phase number<br>Specifies whether the transmit frame has 1 or 2 phases.<br>XPHASE = 0            Single-phase frame<br>XPHASE = 1            Dual-phase frame | R/W  | 0           |

## 15.8.8 Set the Transmit Word Length(s)

**Table 15-54. Register Bits Used to Set the Transmit Word Length(s)**

| Register | Bit | Name    | Function                                                                                                                                                                                                                                                                     | Type | Reset Value |
|----------|-----|---------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------|-------------|
| XCR1     | 7-5 | XWDLEN1 | Transmit word length of frame phase 1<br>XWDLEN1 = 000b        8 bits<br>XWDLEN1 = 001b        12 bits<br>XWDLEN1 = 010b        16 bits<br>XWDLEN1 = 011b        20 bits<br>XWDLEN1 = 100b        24 bits<br>XWDLEN1 = 101b        32 bits<br>XWDLEN1 = 11Xb        Reserved | R/W  | 000         |
| XCR2     | 7-5 | XWDLEN2 | Transmit word length of frame phase 2<br>XWDLEN2 = 000b        8 bits<br>XWDLEN2 = 001b        12 bits<br>XWDLEN2 = 010b        16 bits<br>XWDLEN2 = 011b        20 bits<br>XWDLEN2 = 100b        24 bits<br>XWDLEN2 = 101b        32 bits<br>XWDLEN2 = 11Xb        Reserved | R/W  | 000         |

### 15.8.8.1 Word Length Bits

Each frame can have one or two phases, depending on the value that you load into the RPHASE bit. If a single-phase frame is selected, XWDLEN1 selects the length for every serial word transmitted in the frame. If a dual-phase frame is selected, XWDLEN1 determines the length of the serial words in phase 1 of the frame, and XWDLEN2 determines the word length in phase 2 of the frame.



## 15.8.9 Set the Transmit Frame Length

**Table 15-55. Register Bits Used to Set the Transmit Frame Length**

| Register | Bit  | Name    | Function                                                                                                         | Type | Reset Value |
|----------|------|---------|------------------------------------------------------------------------------------------------------------------|------|-------------|
| XCR1     | 14-8 | XFRLEN1 | Transmit frame length 1                                                                                          | R/W  | 000 0000    |
|          |      |         | (XFRLEN1 + 1) is the number of serial words in phase 1 of the transmit frame.                                    |      |             |
|          |      |         | XFRLEN1 = 000 0000                                                                                               |      |             |
|          |      |         | XFRLEN1 = 000 0001                                                                                               |      |             |
|          |      |         |                                                                                                                  |      |             |
| XCR2     | 14-8 | XFRLEN2 | Transmit frame length 2                                                                                          | R/W  | 000 0000    |
|          |      |         | If a dual-phase frame is selected, (XFRLEN2 + 1) is the number of serial words in phase 2 of the transmit frame. |      |             |
|          |      |         | XFRLEN2 = 000 0000                                                                                               |      |             |
|          |      |         | XFRLEN2 = 000 0001                                                                                               |      |             |
|          |      |         |                                                                                                                  |      |             |

### 15.8.9.1 Selected Frame Length

The transmit frame length is the number of serial words in the transmit frame. Each frame can have one or two phases, depending on the value that you load into the XPHASE bit.

If a single-phase frame is selected (XPHASE = 0), the frame length is equal to the length of phase 1. If a dual-phase frame is selected (XPHASE = 1), the frame length is the length of phase 1 plus the length of phase 2.

The 7-bit XFRLEN fields allow up to 128 words per phase. See [Table 15-56](#) for a summary of how to calculate the frame length. This length corresponds to the number of words or logical time slots or channels per frame-synchronization pulse.

#### Note

Program the XFRLEN fields with  $[w \text{ minus } 1]$ , where  $w$  represents the number of words per phase. For example, if you want a phase length of 128 words in phase 1, load 127 into XFRLEN1.

**Table 15-56. How to Calculate Frame Length**

| XPHASE | XFRLEN1                          | XFRLEN2                          | Frame Length                        |
|--------|----------------------------------|----------------------------------|-------------------------------------|
| 0      | $0 \leq \text{XFRLEN1} \leq 127$ | Don't care                       | (XFRLEN1 + 1) words                 |
| 1      | $0 \leq \text{XFRLEN1} \leq 127$ | $0 \leq \text{XFRLEN2} \leq 127$ | (XFRLEN1 + 1) + (XFRLEN2 + 1) words |

## 15.8.10 Enable/Disable the Transmit Frame-Synchronization Ignore Function

**Table 15-57. Register Bit Used to Enable/Disable the Transmit Frame-Synchronization Ignore Function**

| Register | Bit | Name | Function                                                                                                       | Type | Reset Value |
|----------|-----|------|----------------------------------------------------------------------------------------------------------------|------|-------------|
| XCR2     | 2   | XFIG | Transmit frame-synchronization ignore                                                                          | R/W  | 0           |
|          |     |      | XFIG = 0<br>An unexpected transmit frame-synchronization pulse causes the McBSP to restart the frame transfer. |      |             |
|          |     |      | XFIG = 1<br>The McBSP ignores unexpected transmit frame-synchronization pulses.                                |      |             |

### 15.8.10.1 Unexpected Frame-Synchronization Pulses and Frame-Synchronization Ignore

If a frame-synchronization pulse starts the transfer of a new frame before the current frame is fully transmitted, this pulse is treated as an unexpected frame-synchronization pulse.

When XFIG = 1, normal transmission continues with unexpected frame-synchronization signals ignored.

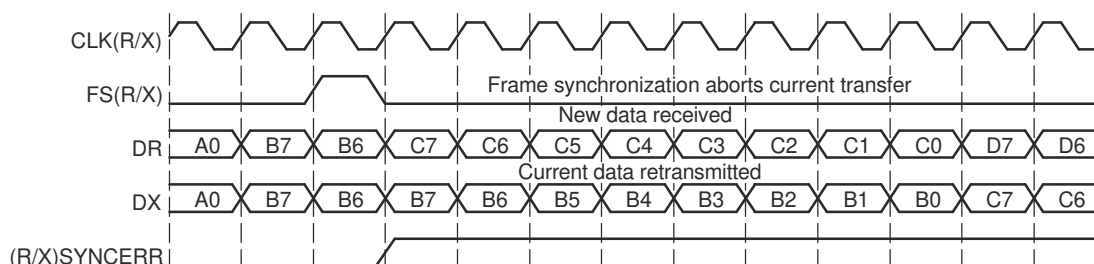
When XFIG = 0 and an unexpected frame-synchronization pulse occurs, the serial port:

1. Aborts the present transmission
2. Sets XSYNCERR to 1 in SPCR2
3. Reinitiates transmission of the current word that was aborted

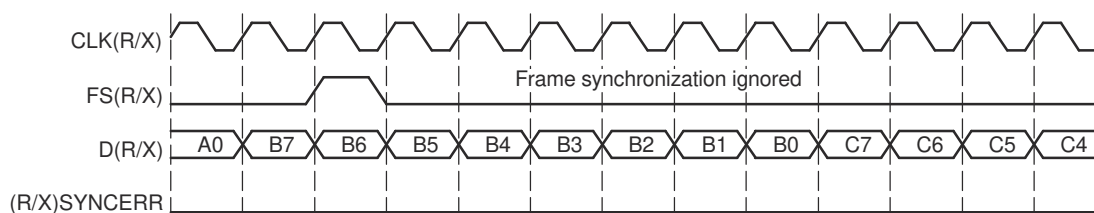
For more details about the frame-synchronization error condition, see [Section 15.4.5](#).

### 15.8.10.2 Examples Showing the Effects of XFIG

[Figure 15-51](#) shows an example in which word B is interrupted by an unexpected frame-synchronization pulse when (R/X)FIG = 0. In the case of transmission, the transmission of B is aborted (B is lost). This condition is a transmit synchronization error, which sets the XSYNCERR bit. No new data has been written to DXR[1,2]; therefore, the McBSP transmits B again.


**Figure 15-51. Unexpected Frame-Synchronization Pulse With (R/X) FIG = 0**

In contrast with [Figure 15-51](#), [Figure 15-52](#) shows McBSP operation when unexpected frame-synchronization signals are ignored (when (R/X)FIG = 1). Here, the transfer of word B is not affected by an unexpected frame-synchronization pulse.


**Figure 15-52. Unexpected Frame-Synchronization Pulse With (R/X) FIG = 1**

## 15.8.11 Set the Transmit Companding Mode

**Table 15-58. Register Bits Used to Set the Transmit Companding Mode**

| Register | Bit | Name     | Function                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         | Type | Reset Value |
|----------|-----|----------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------|-------------|
| XCR2     | 4-3 | XCOMPAND | Transmit companding mode<br><br>Modes other than 00b are enabled only when the appropriate XWDLEN is 000b, indicating 8-bit data.<br><br>XCOMPAND = 00b      No companding, any size data, MSB transmitted first<br><br>XCOMPAND = 01b      No companding, 8-bit data, LSB transmitted first (for details, see <a href="#">Section 15.7.11.4</a> )<br><br>XCOMPAND = 10b $\mu$ -law companding, 8-bit data, MSB transmitted first<br><br>XCOMPAND = 11b      A-law companding, 8-bit data, MSB transmitted first | R/W  | 00          |

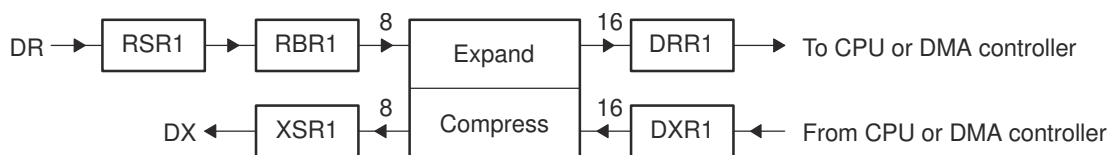
### 15.8.11.1 Companding

Companding (COMpressing and exPANDING) hardware allows compression and expansion of data in either  $\mu$ -law or A-law format. The companding standard employed in the United States and Japan is  $\mu$ -law. The European companding standard is referred to as A-law. The specifications for  $\mu$ -law and A-law log PCM are part of the CCITT G.711 recommendation.

A-law and  $\mu$ -law allow 13 bits and 14 bits of dynamic range, respectively. Any values outside this range are set to the most positive or most negative value. Thus, for companding to work best, the data transferred to and from the McBSP via the CPU or DMA controller must be at least 16 bits wide.

The  $\mu$ -law and A-law formats both encode data into 8-bit code words. Companded data is always 8 bits wide; the appropriate word length bits (RWDLEN1, RWDLEN2, XWDLEN1, XWDLEN2) must therefore be set to 0, indicating an 8-bit wide serial data stream. If companding is enabled and either of the frame phases does not have an 8-bit word length, companding continues as if the word length is 8 bits.

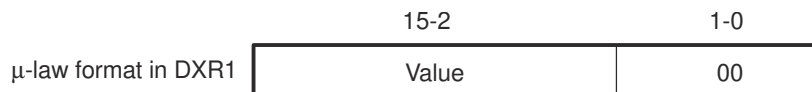
[Figure 15-53](#) illustrates the companding processes. When companding is chosen for the transmitter, compression occurs during the process of copying data from DXR1 to XSR1. The transmit data is encoded according to the specified companding law (A-law or  $\mu$ -law). When companding is chosen for the receiver, expansion occurs during the process of copying data from RBR1 to DRR1. The receive data is decoded to two's-complement format.



**Figure 15-53. Companding Processes for Reception and for Transmission**

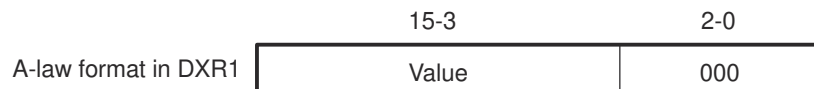
### 15.8.11.2 Format for Data To Be Compressed

For transmission using  $\mu$ -law compression, make sure the 14 data bits are left-justified in DXR1, with the remaining two low-order bits filled with 0s as shown in [Figure 15-54](#).



**Figure 15-54.  $\mu$ -Law Transmit Data Companding Format**

For transmission using A-law compression, make sure the 13 data bits are left-justified in DXR1, with the remaining three low-order bits filled with 0s as shown in [Figure 15-55](#).



**Figure 15-55. A-Law Transmit Data Companding Format**

### 15.8.11.3 Capability to Compand Internal Data

If the McBSP is otherwise unused (the serial port transmit and receive sections are reset), the companding hardware can compand internal data.

### 15.8.11.4 Option to Transmit LSB First

Normally, the McBSP transmit or receives all data with the most significant bit (MSB) first. However, certain 8-bit data protocols (that do not use companded data) require the least significant bit (LSB) to be transferred first. If you set XCOMPAND = 01b in XCR2, the bit ordering of 8-bit words is reversed (LSB first) before being sent from the serial port. Similar to companding, this feature is enabled only if the appropriate word length bits are set to 0, indicating that 8-bit words are to be transferred serially. If either phase of the frame does not have an 8-bit word length, the McBSP assumes the word length is eight bits and LSB-first ordering is done.

### 15.8.12 Set the Transmit Data Delay

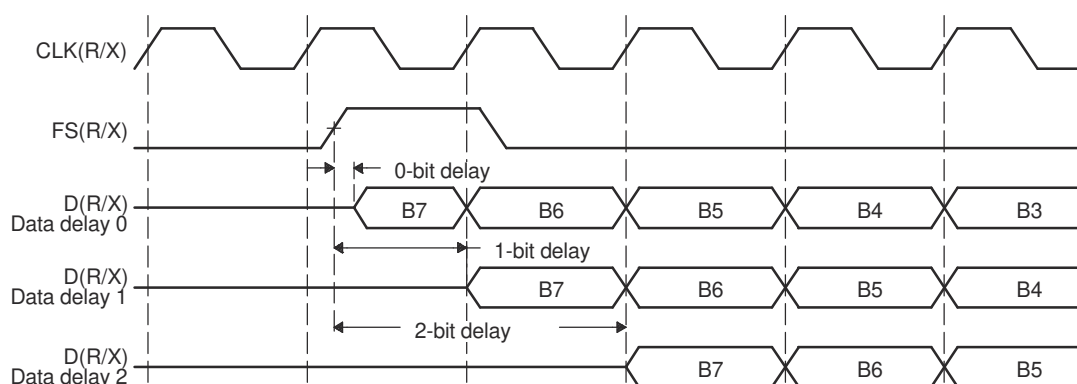
**Table 15-59. Register Bits Used to Set the Transmit Data Delay**

| Register | Bit | Name    | Function               | Type | Reset Value      |
|----------|-----|---------|------------------------|------|------------------|
| XCR2     | 1-0 | XDATDLY | Transmitter data delay | R/W  | 00               |
|          |     |         | XDATDLY = 00           |      | 0-bit data delay |
|          |     |         | XDATDLY = 01           |      | 1-bit data delay |
|          |     |         | XDATDLY = 10           |      | 2-bit data delay |
|          |     |         | XDATDLY = 11           |      | Reserved         |

#### 15.8.12.1 Data Delay

The start of a frame is defined by the first clock cycle in which frame synchronization is found to be active. The beginning of actual data reception or transmission with respect to the start of the frame can be delayed if necessary. This delay is called data delay.

XDATDLY specifies the data delay for transmission. The range of programmable data delay is zero to two bit-clocks (XDATDLY = 00b-10b), as described in [Table 15-59](#) and [Figure 15-56](#). In this figure, the data transferred is an 8-bit value with bits labeled B7, B6, B5, and so on. Typically a 1-bit delay is selected, because data often follows a 1-cycle active frame-synchronization pulse.



**Figure 15-56. Range of Programmable Data Delay**

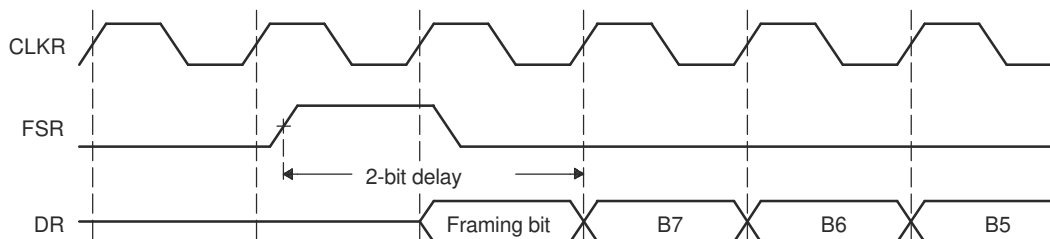
### 15.8.12.2 0-Bit Data Delay

Normally, a frame-synchronization pulse is detected or sampled with respect to an edge of serial clock internal CLK(R/X). Thus, on the following cycle or later (depending on the data delay value), data can be received or transmitted. However, in the case of 0-bit data delay, the data must be ready for reception and/or transmission on the same serial clock cycle.

For reception this problem is solved because receive data is sampled on the first falling edge of MCLKR where an active-high internal FSR is detected. However, data transmission must begin on the rising edge of the internal CLKX clock that generated the frame synchronization. Therefore, the first data bit is assumed to be present in XSR1, and thus DX. The transmitter then asynchronously detects the frame synchronization, FSX, going active high and immediately starts driving the first bit to be transmitted on the DX pin.

### 15.8.12.3 2-Bit Data Delay

A data delay of two bit-periods allows the serial port to interface to different types of T1 framing devices where the data stream is preceded by a framing bit. During reception of such a stream with data delay of two bits (framing bit appears after a 1-bit delay and data appears after a 2-bit delay), the serial port essentially discards the framing bit from the data stream, as shown in the following figure. In this figure, the data transferred is an 8-bit value with bits labeled B7, B6, B5, and so on.



**Figure 15-57. 2-Bit Data Delay Used to Skip a Framing Bit**

### 15.8.13 Set the Transmit DXENA Mode

**Table 15-60. Register Bit Used to Set the Transmit DXENA (DX Delay Enabler) Mode**

| Register | Bit | Name  | Function                                | Type | Reset Value |
|----------|-----|-------|-----------------------------------------|------|-------------|
| SPCR1    | 7   | DXENA | DX delay enabler mode                   | R/W  | 0           |
|          |     |       | DXENA = 0      DX delay enabler is off. |      |             |
|          |     |       | DXENA = 1      DX delay enabler is on.  |      |             |

#### 15.8.13.1 DXENA Mode

The DXENA bit controls the delay enabler on the DX pin. Set DXENA to enable an extra delay for turn-on time. This bit does not control the data itself, so only the first bit is delayed.

If you connect together the DX pins of multiple McBSPs, make sure DXENA = 1 to avoid having more than one McBSP transmit on the data line at one time.

### 15.8.14 Set the Transmit Interrupt Mode

The transmitter interrupt (XINT) signals the CPU of changes to the serial port status. Four options exist for configuring this interrupt. The options are set by the transmit interrupt mode bits, XINTM, in SPCR2.

**Table 15-61. Register Bits Used to Set the Transmit Interrupt Mode**

| Register | Bit | Name  | Function                                                                                                                                                                                                                                                                                                                                                                                                                                                    | Type | Reset Value |
|----------|-----|-------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------|-------------|
| SPCR2    | 5-4 | XINTM | Transmit interrupt mode                                                                                                                                                                                                                                                                                                                                                                                                                                     | R/W  | 00          |
|          |     |       | XINTM = 00      XINT generated when XRDY changes from 0 to 1.                                                                                                                                                                                                                                                                                                                                                                                               |      |             |
|          |     |       | XINTM = 01      XINT generated by an end-of-block or end-of-frame condition in a transmit multichannel selection mode. In any of the transmit multichannel selection modes, interrupt after every 16-channel block boundary has been crossed within a frame and at the end of the frame. For details, see <a href="#">Section 15.5.7.3</a> . In any other serial transfer case, this setting is not applicable and, therefore, no interrupts are generated. |      |             |
|          |     |       | XINTM = 10      XINT generated by a new transmit frame-synchronization pulse. Interrupt on detection of each transmit frame-synchronization pulse. This generates an interrupt even when the transmitter is in its reset state. This is done by synchronizing the incoming frame-synchronization pulse to the CPU clock and sending it to the CPU via XINT.                                                                                                 |      |             |
|          |     |       | XINTM = 11      XINT generated when XSYNCERR is set. Interrupt on frame-synchronization error. Regardless of the value of XINTM, XSYNCERR can be read to detect this condition. For more information on using XSYNCERR, see <a href="#">Section 15.4.5</a> .                                                                                                                                                                                                |      |             |

### 15.8.15 Set the Transmit Frame-Synchronization Mode

**Table 15-62. Register Bits Used to Set the Transmit Frame-Synchronization Mode**

| Register | Bit | Name | Function                                                                                                                                                                                                                     | Type | Reset Value |
|----------|-----|------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------|-------------|
| PCR      | 11  | FSXM | Transmit frame-synchronization mode                                                                                                                                                                                          | R/W  | 0           |
|          |     |      | FSXM = 0      Transmit frame synchronization is supplied by an external source via the FSX pin.                                                                                                                              |      |             |
|          |     |      | FSXM = 1      Transmit frame synchronization is supplied by the McBSP, as determined by the FSGM bit of SRGR2.                                                                                                               |      |             |
| SRGR2    | 12  | FSGM | Sample rate generator transmit frame-synchronization mode                                                                                                                                                                    | R/W  | 0           |
|          |     |      | Used when FSXM = 1 in PCR.                                                                                                                                                                                                   |      |             |
|          |     |      | FSGM = 0      The McBSP generates a transmit frame-synchronization pulse when the content of DXR[1,2] is copied to XSR[1,2].                                                                                                 |      |             |
|          |     |      | FSGM = 1      The transmitter uses frame-synchronization pulses generated by the sample rate generator. Program the FWID bits to set the width of each pulse. Program the FPER bits to set the frame-synchronization period. |      |             |

#### 15.8.15.1 Transmit Frame-Synchronization Modes

Table 15-63 shows how FSXM and FSGM select the source of transmit frame-synchronization pulses. The three choices are:

- External frame-synchronization input
- Sample rate generator frame-synchronization signal (FSG)
- Internal signal that indicates a DXR-to-XSR copy has been made

Table 15-63 also shows the effect of each bit setting on the FSX pin. The polarity of the signal on the FSX pin is determined by the FSXP bit.

**Table 15-63. How FSXM and FSGM Select the Source of Transmit Frame-Synchronization Pulses**

| FSXM | FSGM   | Source of Transmit Frame Synchronization                                                                                                              | FSX Pin Status                                                                                                          |
|------|--------|-------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| 0    | 0 or 1 | An external frame-synchronization signal enters the McBSP through the FSX pin. The signal is then inverted by FSXP before being used as internal FSX. | Input                                                                                                                   |
| 1    | 1      | Internal FSX is driven by the sample rate generator frame-synchronization signal (FSG).                                                               | Output. FSG is inverted by FSXP before being driven out on FSX pin.                                                     |
| 1    | 0      | A DXR-to-XSR copy causes the McBSP to generate a transmit frame-synchronization pulse that is 1 cycle wide.                                           | Output. The generated frame-synchronization pulse is inverted as determined by FSXP before being driven out on FSX pin. |

#### 15.8.15.2 Other Considerations

If the sample rate generator creates a frame-synchronization signal (FSG) that is derived from an external input clock, the GSYNC bit determines whether FSG is kept synchronized with pulses on the FSR pin. For more details, see [Section 15.3.3](#).

In the clock stop mode (CLKSTP = 10b or 11b), the McBSP can act as a master or as a slave in the SPI protocol. If the McBSP is a master and must provide a slave-enable signal (SPISTE) on the FSX pin, make sure that FSXM = 1 and FSGM = 0 so that FSX is an output and is driven active for the duration of each transmission. If the McBSP is a slave, make sure that FSXM = 0 so that the McBSP can receive the slave-enable signal on the FSX pin.

## 15.8.16 Set the Transmit Frame-Synchronization Polarity

**Table 15-64. Register Bit Used to Set Transmit Frame-Synchronization Polarity**

| Register | Bit | Name | Function                                                      | Type | Reset Value |
|----------|-----|------|---------------------------------------------------------------|------|-------------|
| PCR      | 3   | FSXP | Transmit frame-synchronization polarity                       | R/W  | 0           |
|          |     |      | FSXP = 0      Frame-synchronization pulse FSX is active high. |      |             |
|          |     |      | FSXP = 1      Frame-synchronization pulse FSX is active low.  |      |             |

### 15.8.16.1 Frame Synchronization Pulses, Clock Signals, and Their Polarities

Transmit frame-synchronization pulses can be generated internally by the sample rate generator (see [Section 15.3.2](#)) or driven by an external source. The source of frame synchronization is selected by programming the mode bit, FSXM, in PCR. FSX is also affected by the FSGM bit in SRGR2. For information about the effects of FSXM and FSGM, see [Section 15.8.15](#)). Similarly, transmit clocks can be selected to be inputs or outputs by programming the mode bit, CLKXM, in the PCR (see [Section 15.8.18](#)).

When FSR and FSX are inputs (FSXM = FSRM = 0, external frame-synchronization pulses), the McBSP detects them on the internal falling edge of clock, internal MCLKR, and internal CLKX, respectively. The receive data arriving at the DR pin is also sampled on the falling edge of internal MCLKR. These internal clock signals are either derived from external source via CLK(R/X) pins or driven by the sample rate generator clock (CLKG) internal to the McBSP.

When FSR and FSX are outputs, implying that they are driven by the sample rate generator, they are generated (transition to their active state) on the rising edge of internal clock, CLK(R/X). Similarly, data on the DX pin is output on the rising edge of internal CLKX.

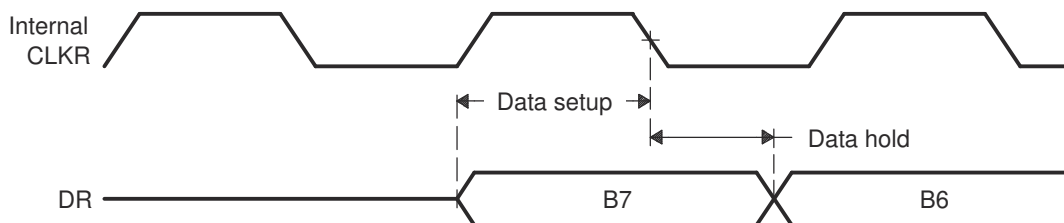
FSRP, FSXP, CLKRP, and CLKXP in the pin control register (PCR) configure the polarities of the FSR, FSX, MCLKR, and CLKX signals, respectively. All frame-synchronization signals (internal FSR, internal FSX) that are internal to the serial port are active high. If the serial port is configured for external frame synchronization (FSR/FSX are inputs to McBSP) and FSRP = FSXP = 1, the external active-low frame-synchronization signals are inverted before being sent to the receiver (internal FSR) and transmitter (internal FSX). Similarly, if internal synchronization (FSR/FSX are output pins and GSYNC = 0) is selected and the polarity bit FS(R/X)P = 1, the internal active-high frame-synchronization signals are inverted before being sent to the FS(R/X) pin.

On the transmit side, the transmit clock polarity bit, CLKXP, sets the edge used to shift and clock out transmit data. Data is always transmitted on the rising edge of internal CLKX. If CLKXP = 1 and external clocking is selected (CLKXM = 0 and CLKX is an input), the external falling-edge triggered input clock on CLKX is inverted to a rising-edge triggered clock before being sent to the transmitter. If CLKXP = 1, and internal clocking selected (CLKXM = 1 and CLKX is an output pin), the internal (rising-edge triggered) clock, internal CLKX, is inverted before being sent out on the MCLKX pin.

Similarly, the receiver can reliably sample data that is clocked with a rising edge clock (by the transmitter). The receive clock polarity bit, CLKRP, sets the edge used to sample received data. The receive data is always sampled on the falling edge of internal MCLKR. Therefore, if CLKRP = 1 and external clocking is selected (CLKRM = 0 and MCLKR is an input pin), the external rising-edge triggered input clock on MCLKR is inverted to a falling-edge triggered clock before being sent to the receiver. If CLKRP = 1 and internal clocking is selected (CLKRM = 1), the internal falling-edge triggered clock is inverted to a rising-edge triggered clock before being sent out on the MCLKR pin.

CLKRP = CLKXP in a system where the same clock (internal or external) is used to clock the receiver and transmitter. The receiver uses the opposite edge as the transmitter to ensure valid setup and hold of data around this edge. [Figure 15-58](#) shows how data clocked by an external serial device using a rising edge can be sampled by the McBSP receiver on the falling edge of the same clock.





**Figure 15-58. Data Clocked Externally Using a Rising Edge and Sampled by the McBSP Receiver on a Falling Edge**

### 15.8.17 Set the SRG Frame-Synchronization Period and Pulse Width

**Table 15-65. Register Bits Used to Set SRG Frame-Synchronization Period and Pulse Width**

| Register | Bit  | Name | Function                                                                                                                                                                                                                                                                               | Type | Reset Value    |
|----------|------|------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------|----------------|
| SRGR2    | 11-0 | FPER | Sample rate generator frame-synchronization period<br>For the frame-synchronization signal FSG, (FPER + 1) determines the period from the start of a frame-synchronization pulse to the start of the next frame-synchronization pulse.<br>Range for (FPER + 1): 1 to 4096 CLKG cycles. | R/W  | 0000 0000 0000 |
| SRGR1    | 15-8 | FWID | Sample rate generator frame-synchronization pulse width<br>This field plus 1 determines the width of each frame-synchronization pulse on FSG.<br>Range for (FWID + 1): 1 to 256 CLKG cycles.                                                                                           | R/W  | 0000 0000      |

#### 15.8.17.1 Frame-Synchronization Period and Frame-Synchronization Pulse Width

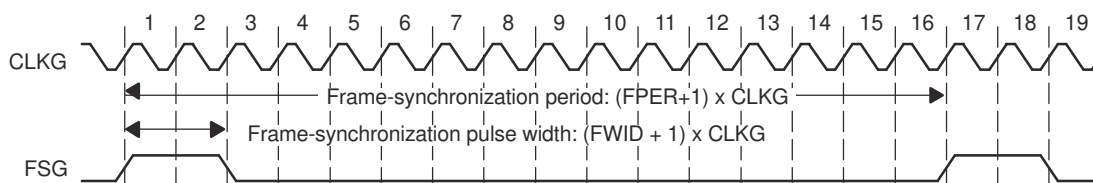
The sample rate generator can produce a clock signal, CLKG, and a frame-synchronization signal, FSG. If the sample rate generator is supplying receive or transmit frame synchronization, you must program the bit fields FPER and FWID.

On FSG, the period from the start of a frame-synchronization pulse to the start of the next pulse is (FPER + 1) CLKG cycles. The 12 bits of FPER allow a frame-synchronization period of 1 to 4096 CLKG cycles, which allows up to 4096 data bits per frame. When GSYNC = 1, FPER is a don't care value.

Each pulse on FSG has a width of (FWID + 1) CLKG cycles. The eight bits of FWID allow a pulse width of 1 to 256 CLKG cycles. It is recommended that FWID be programmed to a value less than the programmed word length.

The values in FPER and FWID are loaded into separate down-counters. The 12-bit FPER counter counts down the generated clock cycles from the programmed value (4095 maximum) to 0. The 8-bit FWID counter counts down from the programmed value (255 maximum) to 0.

Figure 15-59 shows a frame-synchronization period of 16 CLKG periods (FPER = 15 or 00001111b) and a frame-synchronization pulse with an active width of 2 CLKG periods (FWID = 1).



**Figure 15-59. Frame of Period 16 CLKG Periods and Active Width of 2 CLKG Periods**

When the sample rate generator comes out of reset, FSG is in its inactive state. Then, when GRST = 1 and FSGM = 1, a frame-synchronization pulse is generated. The frame width value (FWID + 1) is counted down on every CLKG cycle until it reaches 0, at which time FSG goes low. At the same time, the frame period value (FPER + 1) is also counting down. When this value reaches 0, FSG goes high, indicating a new frame.

## 15.8.18 Set the Transmit Clock Mode

**Table 15-66. Register Bit Used to Set the Transmit Clock Mode**

| Register | Bit | Name  | Function            | Type | Reset Value                                                                      |
|----------|-----|-------|---------------------|------|----------------------------------------------------------------------------------|
| PCR      | 9   | CLKXM | Transmit clock mode | R/W  | 0                                                                                |
|          |     |       | CLKXM = 0           |      | The transmitter gets its clock signal from an external source via the MCLKX pin. |
|          |     |       | CLKXM = 1           |      | The MCLKX pin is an output pin driven by the sample rate generator of the McBSP. |

### 15.8.18.1 Selecting a Source for the Transmit Clock and a Data Direction for the MCLKX pin

Table 15-67 shows how the CLKXM bit selects the transmit clock and the corresponding status of the MCLKX pin. The polarity of the signal on the MCLKX pin is determined by the CLKXP bit.

**Table 15-67. How the CLKXM Bit Selects the Transmit Clock and the Corresponding Status of the MCLKX pin**

| CLKXM in PCR | Source of Transmit Clock                                                                                                  | MCLKX pin Status                                                      |
|--------------|---------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------|
| 0            | Internal CLKX is driven by an external clock on the MCLKX pin. CLKX is inverted as determined by CLKXP before being used. | Input                                                                 |
| 1            | Internal CLKX is driven by the sample rate generator clock, CLKG.                                                         | Output. CLKG, inverted as determined by CLKXP, is driven out on CLKX. |

### 15.8.18.2 Other Considerations

If the sample rate generator creates a clock signal (CLKG) that is derived from an external input clock, the GSYNC bit determines whether CLKG is kept synchronized with pulses on the FSR pin. For more details, see [Section 15.3.3](#).

In the clock stop mode (CLKSTP = 10b or 11b), the McBSP can act as a master or as a slave in the SPI protocol. If the McBSP is a master, make sure that CLKXM = 1 so that CLKX is an output to supply the master clock to any slave devices. If the McBSP is a slave, make sure that CLKXM = 0 so that CLKX is an input to accept the master clock signal.

## 15.8.19 Set the Transmit Clock Polarity

**Table 15-68. Register Bit Used to Set Transmit Clock Polarity**

| Register | Bit | Name  | Function                | Type | Reset Value                                    |
|----------|-----|-------|-------------------------|------|------------------------------------------------|
| PCR      | 1   | CLKXP | Transmit clock polarity | R/W  | 0                                              |
|          |     |       | CLKXP = 0               |      | Transmit data sampled on rising edge of CLKX.  |
|          |     |       | CLKXP = 1               |      | Transmit data sampled on falling edge of CLKX. |

### 15.8.19.1 Frame Synchronization Pulses, Clock Signals, and Their Polarities

Transmit frame-synchronization pulses can be either generated internally by the sample rate generator (see [Section 15.3.2](#)) or driven by an external source. The source of frame synchronization is selected by programming the mode bit, FSXM, in PCR. FSX is also affected by the FSGM bit in SRGR2. For information about the effects of FSXM and FSGM, see [Section 15.8.15](#)). Similarly, transmit clocks can be selected to be inputs or outputs by programming the mode bit, CLKXM, in the PCR (see [Section 15.8.18](#)).

When FSR and FSX are inputs (FSXM = FSRM = 0, external frame-synchronization pulses), the McBSP detects them on the internal falling edge of clock, internal MCLKR, and internal CLKX, respectively. The receive data arriving at the DR pin is also sampled on the falling edge of internal MCLKR. These internal clock signals are either derived from external source via CLK(R/X) pins or driven by the sample rate generator clock (CLKG) internal to the McBSP.

When FSR and FSX are outputs, implying that they are driven by the sample rate generator, they are generated (transition to their active state) on the rising edge of internal clock, CLK(R/X). Similarly, data on the DX pin is output on the rising edge of internal CLKX.

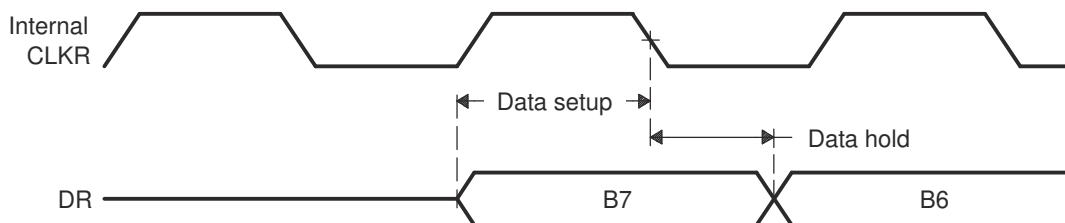
FSRP, FSXP, CLKRP, and CLKXP in the pin control register (PCR) configure the polarities of the FSR, FSX, MCLKR, and CLKX signals, respectively. All frame-synchronization signals (internal FSR, internal FSX) that are internal to the serial port are active high. If the serial port is configured for external frame synchronization (FSR/FSX are inputs to McBSP), and FSRP = FSXP = 1, the external active-low frame-synchronization signals are inverted before being sent to the receiver (internal FSR) and transmitter (internal FSX). Similarly, if internal synchronization (FSR/FSX are output pins and GSYNC = 0) is selected, the internal active-high frame-synchronization signals are inverted, if the polarity bit FS(R/X)P = 1, before being sent to the FS(R/X) pin.

On the transmit side, the transmit clock polarity bit, CLKXP, sets the edge used to shift and clock out transmit data. Data is always transmitted on the rising edge of internal CLKX. If CLKXP = 1 and external clocking is selected (CLKXM = 0 and CLKX is an input), the external falling-edge triggered input clock on CLKX is inverted to a rising-edge triggered clock before being sent to the transmitter. If CLKXP = 1 and internal clocking is selected (CLKXM = 1 and CLKX is an output pin), the internal (rising-edge triggered) clock, internal CLKX, is inverted before being sent out on the MCLKX pin.

Similarly, the receiver can reliably sample data that is clocked with a rising edge clock (by the transmitter). The receive clock polarity bit, CLKRP, sets the edge used to sample received data. The receive data is always sampled on the falling edge of internal MCLKR. Therefore, if CLKRP = 1 and external clocking is selected (CLKRM = 0 and CLKR is an input pin), the external rising-edge triggered input clock on CLKR is inverted to a falling-edge triggered clock before being sent to the receiver. If CLKRP = 1 and internal clocking is selected (CLKRM = 1), the internal falling-edge triggered clock is inverted to a rising-edge triggered clock before being sent out on the MCLKR pin.

CLKRP = CLKXP in a system where the same clock (internal or external) is used to clock the receiver and transmitter. The receiver uses the opposite edge as the transmitter to ensure valid setup and hold of data around this edge (see [Figure 15-58](#)).

[Figure 15-60](#) shows how data clocked by an external serial device using a rising edge can be sampled by the McBSP receiver on the falling edge of the same clock.



**Figure 15-60. Data Clocked Externally Using a Rising Edge and Sampled by the McBSP Receiver on a Falling Edge**

## 15.9 Emulation and Reset Considerations

This section covers the following topics:

- How to program McBSP response to a breakpoint in the high-level language debugger (see [Section 15.9.1](#))
- How to reset and initialize the various parts of the McBSP (see [Section 15.9.2](#))

### 15.9.1 McBSP Emulation Mode

FREE and SOFT are special emulation bits in SPCR2 that determine the state of the McBSP when a breakpoint is encountered in the high-level language debugger. If FREE = 1, the clock continues to run upon a software breakpoint and data is still shifted out. When FREE = 1, the SOFT bit is a *don't care*.

If FREE = 0, the SOFT bit takes effect. If SOFT = 0 when breakpoint occurs, the clock stops immediately, aborting a transmission. If SOFT = 1 and a breakpoint occurs while transmission is in progress, the transmission continues until completion of the transfer and then the clock halts. These options are listed in [Table 15-69](#).

The McBSP receiver functions in a similar fashion. If a mode other than the immediate stop mode (SOFT = FREE = 0) is chosen, the receiver continues running and an overrun error is possible.

**Table 15-69. McBSP Emulation Modes Selectable with FREE and SOFT Bits of SPCR2**

| FREE | SOFT   | McBSP Emulation Mode                                                                                                                  |
|------|--------|---------------------------------------------------------------------------------------------------------------------------------------|
| 0    | 0      | Immediate stop mode (reset condition)<br>The transmitter or receiver stops immediately in response to a breakpoint.                   |
| 0    | 1      | Soft stop mode<br>When a breakpoint occurs, the transmitter stops after completion of the current word. The receiver is not affected. |
| 1    | 0 or 1 | Free run mode<br>The transmitter and receiver continue to run when a breakpoint occurs.                                               |

### 15.9.2 Resetting and Initializing McBSP

#### 15.9.2.1 McBSP Pin States: DSP Reset Versus Receiver/Transmitter Reset

[Table 15-70](#) shows the state of McBSP pins when the serial port is reset due to direct receiver or transmitter reset on the device.

**Table 15-70. Reset State of Each McBSP Pin**

| Pin                                              | Possible State(s) <sup>(1)</sup> | State Forced by Device Reset | State Forced by Receiver/Transmitter Reset          |
|--------------------------------------------------|----------------------------------|------------------------------|-----------------------------------------------------|
| <b>Receiver reset (RRST = 0 and GRST = 1)</b>    |                                  |                              |                                                     |
| MDRx                                             | I                                | GPIO-input                   | Input                                               |
| MCLKRx                                           | I/O/Z                            | GPIO-input                   | Known state if input; MCLKR running if output       |
| MFSRx                                            | I/O/Z                            | GPIO-input                   | Known state if input; FSRP inactive state if output |
| <b>Transmitter reset (XRST = 0 and GRST = 1)</b> |                                  |                              |                                                     |
| MDXx                                             | O/Z                              | GPIO Input                   | High impedance                                      |
| MCLKXx                                           | I/O/Z                            | GPIO-input                   | Known state if input; CLKX running if output        |
| MFSXx                                            | I/O/Z                            | GPIO-input                   | Known state if input; FSXP inactive state if output |

(1) In Possible State(s) column, I = Input, O = Output, Z = High impedance. In the 28x family, at device reset, all I/Os default to GPIO function and generally as inputs.

#### 15.9.2.2 Device Reset, McBSP Reset, and Sample Rate Generator Reset

When the McBSP is reset in either of the above two ways, the machine is reset to its initial state, including reset of all counters and status bits. The receive status bits include RFULL, RRDY, and RSYNCERR. The transmit status bits include XEMPTY, XRDY, and XSYNCERR.

- Device reset. When the whole DSP is reset ( $\overline{\text{XRS}}$  signal is driven low), all McBSP pins are in GPIO mode. When the device is pulled out of reset, the clock to the McBSP modules remains disabled.

- **McBSP reset.** When the receiver and transmitter reset bits, RRST and XRST, are loaded with 0s, the respective portions of the McBSP are reset and activity in the corresponding section of the serial port stops. Input-only pins such as MDRx, and all other pins that are configured as inputs are in a known state. The MFSRx and MFSXx pins are driven to their inactive state if they are not outputs. If the MCLKR and MCLKX pins are programmed as outputs, they are driven by CLKG, provided that GRST = 1. Lastly, the MDXx pin is in the high-impedance state when the transmitter and/or the device is reset.

During normal operation, the sample rate generator is reset if the GRST bit is cleared. GRST must be 0 only when neither the transmitter nor the receiver is using the sample rate generator. In this case, the internal sample rate generator clock (CLKG) and its frame-synchronization signal (FSG) are driven inactive low.

When the sample rate generator is not in the reset state (GRST = 1), pins MFSRx and MFSXx are in an inactive state when RRST = 0 and XRST = 0, respectively, even if they are outputs driven by FSG. This ensures that when only one portion of the McBSP is in reset, the other portion can continue operation when GRST = 1 and its frame synchronization is driven by FSG.

- **Sample rate generator reset.** The sample rate generator is reset when GRST is loaded with 0.

When neither the transmitter nor the receiver is fed by CLKG and FSG, you can reset the sample rate generator by clearing GRST. In this case, CLKG and FSG are driven inactive low. If you then set GRST, CLKG starts and runs as programmed. Later, if GRST = 1, FSG pulses active high after the programmed number of CLKG cycles has elapsed.

### 15.9.2.3 McBSP Initialization Procedure

The serial port initialization procedure is as follows:

1. Make XRST = RRST = GRST = 0 in SPCR[1,2]. If coming out of a device reset, this step is not required.
2. While the serial port is in the reset state, program only the McBSP configuration registers (not the data registers) as required.
3. Wait for two clock cycles. This ensures proper internal synchronization.
4. Set up data acquisition as required (such as writing to DXR[1,2]).
5. Make XRST = RRST = 1 to enable the serial port. Make sure that as you set these reset bits, you do not modify any of the other bits in SPCR1 and SPCR2. Otherwise, you change the configuration you selected in step 2.
6. Set FRST = 1, if internally generated frame synchronization is required.
7. Wait two clock cycles for the receiver and transmitter to become active.

Alternatively, on either write (step 1 or 5), the transmitter and receiver can be placed in or taken out of reset individually by modifying the desired bit.

The above procedure for reset/initialization can be applied in general when the receiver or transmitter must be reset during its normal operation and when the sample rate generator is not used for either operation.

#### Note

1. The necessary duration of the active-low period of XRST or RRST is at least two MCLKR/CLKX cycles.
2. The appropriate bits in serial port configuration registers SPCR[1,2], PCR, RCR[1,2], XCR[1,2], and SRGR[1,2] must only be modified when the affected portion of the serial port is in its reset state.
3. In most cases, the data transmit registers (DXR[1,2]) must be loaded by the CPU or by the DMA controller only when the transmitter is enabled (XRST = 1). An exception to this rule is when these registers are used for companding internal data (see [Section 15.8.11.3](#)).
4. The bits of the channel control registers—MCR[1,2], RCER[A-H], XCER[A-H]—can be modified at any time as long as they are not being used by the current reception/transmission in a multichannel selection mode.

### 15.9.2.4 Resetting the Transmitter While the Receiver is Running

**Example 15-1** shows values in the control registers that reset and configure the transmitter while the receiver is running.

#### Example 15-1. Resetting and Configuring McBSP Transmitter While McBSP Receiver Running

```
SPCR1 = 0001h SPCR2 = 0030h
; The receiver is running with the receive interrupt (RINT) triggered by the
; receiver ready bit (RRDY). The transmitter is in its reset state
; The transmit interrupt (XINT) will be triggered by the transmit frame-sync
; error bit (XSYNCERR). PCR = 0900h
; Transmit frame synchronization is generated internally according to the
; FSGM bit of SRGR2.
; The transmit clock is driven by an external source.
; The receive clock continues to be driven by sample rate generator. The input clock
; of the sample rate generator is supplied by the CPU clock SRGR1 = 0001h SRGR2 = 2000h
; The CPU clock is the input clock for the sample rate generator. The sample
; rate generator divides the CPU clock by 2 to generate its output clock (CLKG).
; Transmit frame synchronization is tied to the automatic copying of data from
; the DXR(s) to the XSR(s). XCR1 = 0740h XCR2 = 8321h
; The transmit frame has two phases. Phase 1 has eight 16-bit words. Phase 2
; has four 12-bit words. There is 1-bit data delay between the start of a
; frame-sync pulse and the first data bit
; transmitted. SPCR2 = 0031h
; The transmitter is taken out of reset.
```

## 15.10 Data Packing Examples

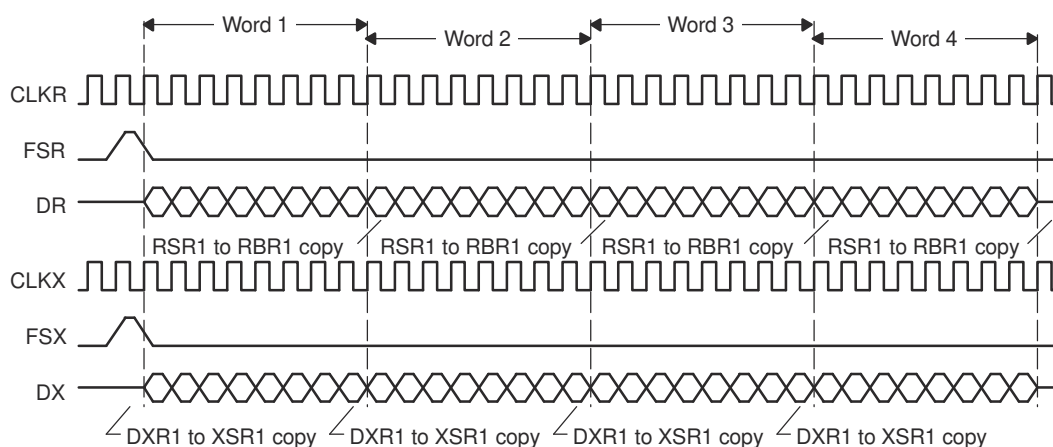
This section shows two ways to implement data packing in the McBSP.

### 15.10.1 Data Packing Using Frame Length and Word Length

Frame length and word length can be manipulated to effectively pack data. For example, consider a situation where four 8-bit words are transferred in a single-phase frame as shown in [Figure 15-61](#). In this case:

- (R/X)PHASE = 0: Single-phase frame
- (R/X)FRLN1 = 0000011b: 4-word frame
- (R/X)WDLEN1 = 000b: 8-bit words

Four 8-bit data words are transferred to and from the McBSP by the CPU or by the DMA controller. Thus, four reads from DRR1 and four writes to DXR1 are necessary for each frame.



**Figure 15-61. Four 8-Bit Data Words Transferred To/From the McBSP**

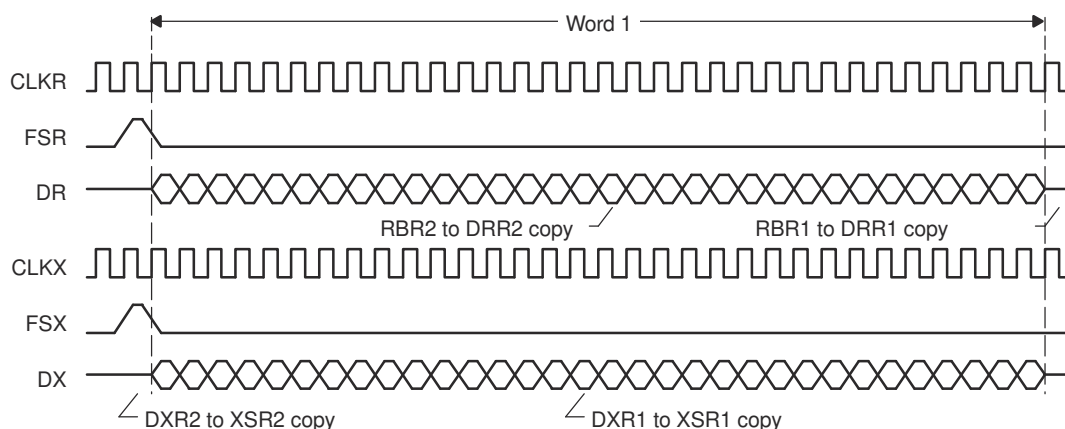
This data can also be treated as a single-phase frame consisting of one 32-bit data word, as shown in [Figure 15-62](#). In this case:

- (R/X)PHASE = 0: Single-phase frame
- (R/X)FRLLEN1 = 0000000b: 1-word frame
- (R/X)WDLEN1 = 101b: 32-bit word

Two 16-bit data words are transferred to and from the McBSP by the CPU or DMA controller. Thus, two reads, from DRR2 and DRR1, and two writes, to DXR2 and DXR1, are necessary for each frame. This results in only half the number of transfers compared to the previous case. This manipulation reduces the percentage of bus time required for serial port data movement.

#### Note

When the word length is larger than 16 bits, make sure you access DRR2/DXR2 before you access DRR1/DXR1. McBSP activity is tied to accesses of DRR1/DXR1. During the reception of 24-bit or 32-bit words, read DRR2 and then read DRR1. Otherwise, the next RBR[1,2]-to-DRR[1,2] copy occurs before DRR2 is read. Similarly, during the transmission of 24-bit or 32-bit words, write to DXR2 and then write to DXR1. Otherwise, the next DXR[1,2]-to-XSR[1,2] copy occurs before DXR2 is loaded with new data.

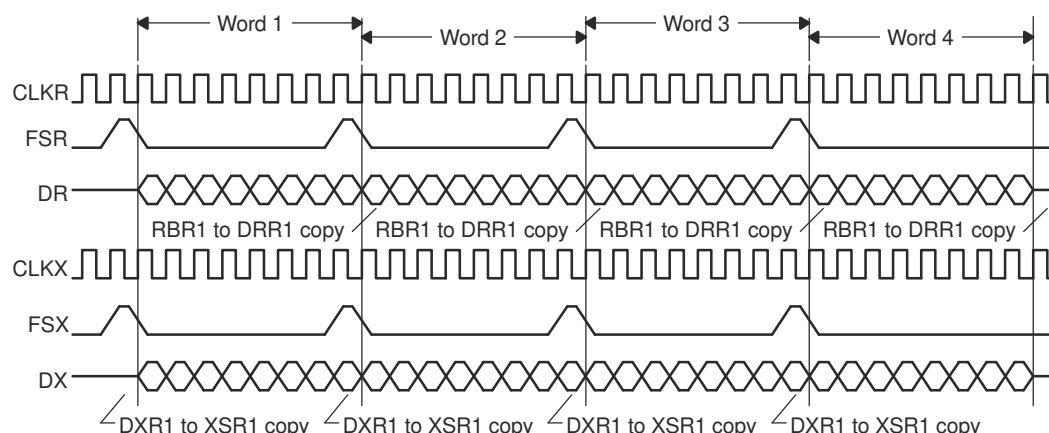


**Figure 15-62. One 32-Bit Data Word Transferred To/From the McBSP**



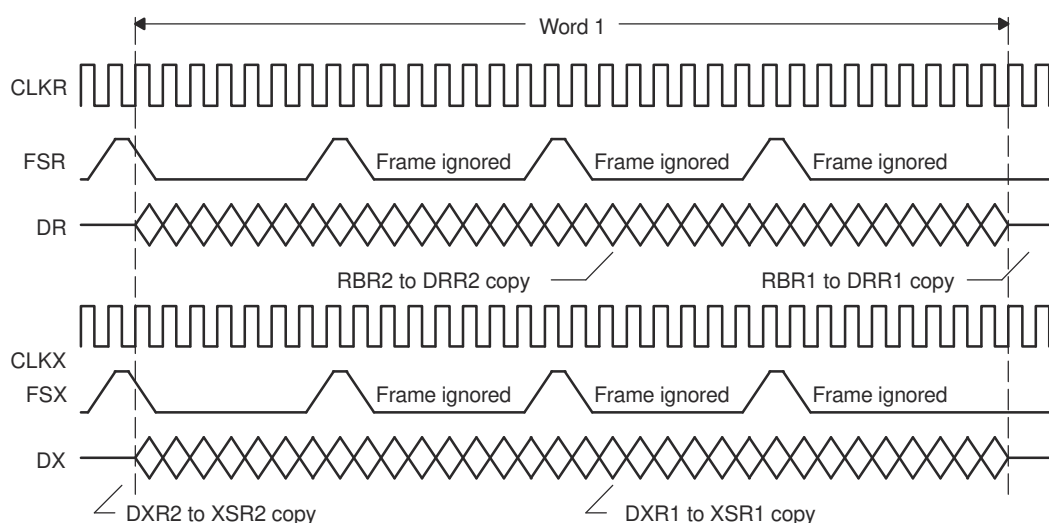
### 15.10.2 Data Packing Using Word Length and the Frame-Synchronization Ignore Function

When there are multiple words per frame, you can implement data packing by increasing the word length (defining a serial word with more bits) and by ignoring frame-synchronization pulses. First, consider [Figure 15-63](#), which shows the McBSP operating at the maximum packet frequency. Here, each frame only has a single 8-bit word. Notice the frame-synchronization pulse that initiates each frame transfer for reception and for transmission. For reception, this configuration requires one read operation for each word. For transmission, this configuration requires one write operation for each word.



**Figure 15-63. 8-Bit Data Words Transferred at Maximum Packet Frequency**

[Figure 15-64](#) shows the McBSP configured to treat this data stream as a continuous 32-bit word. In this example, the McBSP responds to an initial frame-synchronization pulse. However, (R/X)FIG = 1 so that the McBSP ignores subsequent pulses. Only two read transfers or two write transfers are needed every 32 bits. This configuration effectively reduces the required bus bandwidth to half the bandwidth needed to transfer four 8-bit words.



**Figure 15-64. Configuring the Data Stream of [Figure 15-63](#) as a Continuous 32-Bit Word**



## 15.11 McBSP\_REGS Registers

Table 15-71 lists the McBSP\_REGS registers. All register offset addresses not listed in Table 15-71 should be considered as reserved locations and the register contents should not be modified.

**Table 15-71. MCBSP\_REGS Registers**

| Offset | Acronym | Register Name                       | Write Protection | Section            |
|--------|---------|-------------------------------------|------------------|--------------------|
| 0h     | DDR2    | Data receive register bits 31-16    |                  | <a href="#">Go</a> |
| 1h     | DDR1    | Data receive register bits 15-0     |                  | <a href="#">Go</a> |
| 2h     | DXR2    | Data transmit register bits 31-16   |                  | <a href="#">Go</a> |
| 3h     | DXR1    | Data transmit register bits 15-0    |                  | <a href="#">Go</a> |
| 4h     | SPCR2   | Serial port control register 2      |                  | <a href="#">Go</a> |
| 5h     | SPCR1   | Serial port control register 1      |                  | <a href="#">Go</a> |
| 6h     | RCR2    | Receive Control register 2          |                  | <a href="#">Go</a> |
| 7h     | RCR1    | Receive Control register 1          |                  | <a href="#">Go</a> |
| 8h     | XCR2    | Transmit Control register 2         |                  | <a href="#">Go</a> |
| 9h     | XCR1    | Transmit Control register 1         |                  | <a href="#">Go</a> |
| Ah     | SRGR2   | Sample rate generator register 2    |                  | <a href="#">Go</a> |
| Bh     | SRGR1   | Sample rate generator register 1    |                  | <a href="#">Go</a> |
| Ch     | MCR2    | Multichannel control register 2     |                  | <a href="#">Go</a> |
| Dh     | MCR1    | Multichannel control register 1     |                  | <a href="#">Go</a> |
| Eh     | RCERA   | Receive channel enable partition A  |                  | <a href="#">Go</a> |
| Fh     | RCERB   | Receive channel enable partition B  |                  | <a href="#">Go</a> |
| 10h    | XCERA   | Transmit channel enable partition A |                  | <a href="#">Go</a> |
| 11h    | XCERB   | Transmit channel enable partition B |                  | <a href="#">Go</a> |
| 12h    | PCR     | Pin Control register                |                  | <a href="#">Go</a> |
| 13h    | RCERC   | Receive channel enable partition C  |                  | <a href="#">Go</a> |
| 14h    | RCERD   | Receive channel enable partition D  |                  | <a href="#">Go</a> |
| 15h    | XCERC   | Transmit channel enable partition C |                  | <a href="#">Go</a> |
| 16h    | XCERD   | Transmit channel enable partition D |                  | <a href="#">Go</a> |
| 17h    | RCERE   | Receive channel enable partition E  |                  | <a href="#">Go</a> |
| 18h    | RCERF   | Receive channel enable partition F  |                  | <a href="#">Go</a> |
| 19h    | XCERE   | Transmit channel enable partition E |                  | <a href="#">Go</a> |
| 1Ah    | XCERF   | Transmit channel enable partition F |                  | <a href="#">Go</a> |
| 1Bh    | RCERG   | Receive channel enable partition G  |                  | <a href="#">Go</a> |
| 1Ch    | RCERH   | Receive channel enable partition H  |                  | <a href="#">Go</a> |
| 1Dh    | XCERG   | Transmit channel enable partition G |                  | <a href="#">Go</a> |
| 1Eh    | XCERH   | Transmit channel enable partition H |                  | <a href="#">Go</a> |
| 23h    | MFFINT  | Interrupt enable                    |                  | <a href="#">Go</a> |

Complex bit access types are encoded to fit into small table cells. [Table 15-72](#) shows the codes that are used for access types in this section.

**Table 15-72. McBSP\_REGS Access Type Codes**

| Access Type              | Code | Description                                                                                                                                                                                                                                                                          |
|--------------------------|------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Read Type                |      |                                                                                                                                                                                                                                                                                      |
| R                        | R    | Read                                                                                                                                                                                                                                                                                 |
| Write Type               |      |                                                                                                                                                                                                                                                                                      |
| W                        | W    | Write                                                                                                                                                                                                                                                                                |
| Reset or Default Value   |      |                                                                                                                                                                                                                                                                                      |
| -n                       |      | Value after reset or the default value                                                                                                                                                                                                                                               |
| Register Array Variables |      |                                                                                                                                                                                                                                                                                      |
| i,j,k,l,m,n              |      | When these variables are used in a register name, an offset, or an address, they refer to the value of a register array where the register is part of a group of repeating registers. The register groups form a hierarchical structure and the array is represented with a formula. |
| y                        |      | When this variable is used in a register name, an offset, or an address it refers to the value of a register array.                                                                                                                                                                  |

### 15.11.1 DRR2 Register (Offset = 0h) [reset = 0h]

DRR2 is shown in [Figure 15-65](#) and described in [Table 15-73](#).

Return to the [Summary Table](#).

DRR2 contains the upper 16 bits of the received data to be read by the CPU or DMA. DRR2 is only used if the word length is greater than 16 bits.

**Figure 15-65. DRR2 Register**

|        |    |    |    |    |    |   |   |
|--------|----|----|----|----|----|---|---|
| 15     | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
| HWHB   |    |    |    |    |    |   |   |
| R/W-0h |    |    |    |    |    |   |   |
| 7      | 6  | 5  | 4  | 3  | 2  | 1 | 0 |
| HWLB   |    |    |    |    |    |   |   |
| R/W-0h |    |    |    |    |    |   |   |

**Table 15-73. DRR2 Register Field Descriptions**

| Bit  | Field | Type | Reset | Description                               |
|------|-------|------|-------|-------------------------------------------|
| 15-8 | HWHB  | R/W  | 0h    | High-word high-byte<br>Reset type: SYSRSn |
| 7-0  | HWLB  | R/W  | 0h    | High-word low-byte<br>Reset type: SYSRSn  |

### 15.11.2 DRR1 Register (Offset = 1h) [reset = 0h]

DRR1 is shown in [Figure 15-66](#) and described in [Table 15-74](#).

Return to the [Summary Table](#).

DRR1 contains the lower 16 bits of the received data to be read by either the CPU or DMA.

**Figure 15-66. DRR1 Register**

|        |    |    |    |    |    |   |   |
|--------|----|----|----|----|----|---|---|
| 15     | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
| LWHB   |    |    |    |    |    |   |   |
| R/W-0h |    |    |    |    |    |   |   |
| 7      | 6  | 5  | 4  | 3  | 2  | 1 | 0 |
| LWLB   |    |    |    |    |    |   |   |
| R/W-0h |    |    |    |    |    |   |   |

**Table 15-74. DRR1 Register Field Descriptions**

| Bit  | Field | Type | Reset | Description                              |
|------|-------|------|-------|------------------------------------------|
| 15-8 | LWHB  | R/W  | 0h    | Low-word high-byte<br>Reset type: SYSRSn |
| 7-0  | LWLB  | R/W  | 0h    | Low-word low-byte<br>Reset type: SYSRSn  |

### 15.11.3 DXR2 Register (Offset = 2h) [reset = 0h]

DXR2 is shown in [Figure 15-67](#) and described in [Table 15-75](#).

Return to the [Summary Table](#).

DXR2 contains the upper 16 bits of the data to be transmitted after being written by the CPU or DMA. DXR2 is only used if the word length is greater than 16 bits.

**Figure 15-67. DXR2 Register**

|        |    |    |    |    |    |   |   |
|--------|----|----|----|----|----|---|---|
| 15     | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
| HWHB   |    |    |    |    |    |   |   |
| R/W-0h |    |    |    |    |    |   |   |
| 7      | 6  | 5  | 4  | 3  | 2  | 1 | 0 |
| HWLB   |    |    |    |    |    |   |   |
| R/W-0h |    |    |    |    |    |   |   |

**Table 15-75. DXR2 Register Field Descriptions**

| Bit  | Field | Type | Reset | Description                              |
|------|-------|------|-------|------------------------------------------|
| 15-8 | HWHB  | R/W  | 0h    | Low-word high-byte<br>Reset type: SYSRSn |
| 7-0  | HWLB  | R/W  | 0h    | Low-word low-byte<br>Reset type: SYSRSn  |

### 15.11.4 DXR1 Register (Offset = 3h) [reset = 0h]

DXR1 is shown in [Figure 15-68](#) and described in [Table 15-76](#).

Return to the [Summary Table](#).

DXR1 contains the lower 16 bits of the data to be transmitted after being written by the CPU or DMA.

**Figure 15-68. DXR1 Register**

|        |    |    |    |    |    |   |   |
|--------|----|----|----|----|----|---|---|
| 15     | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
| LWHB   |    |    |    |    |    |   |   |
| R/W-0h |    |    |    |    |    |   |   |
| 7      | 6  | 5  | 4  | 3  | 2  | 1 | 0 |
| LWLB   |    |    |    |    |    |   |   |
| R/W-0h |    |    |    |    |    |   |   |

**Table 15-76. DXR1 Register Field Descriptions**

| Bit  | Field | Type | Reset | Description                              |
|------|-------|------|-------|------------------------------------------|
| 15-8 | LWHB  | R/W  | 0h    | Low-word high-byte<br>Reset type: SYSRSn |
| 7-0  | LWLB  | R/W  | 0h    | Low-word low-byte<br>Reset type: SYSRSn  |

### 15.11.5 SPCR2 Register (Offset = 4h) [reset = 0h]

SPCR2 is shown in [Figure 15-69](#) and described in [Table 15-77](#).

Return to the [Summary Table](#).

SPCR2 contains control and status bits for various McBSP functions such as emulation modes, transmit interrupt mode control, transmitter status bits, and transmitter and other internal reset controls.

**Figure 15-69. SPCR2 Register**

|          |        |        |    |          |        |        |        |
|----------|--------|--------|----|----------|--------|--------|--------|
| 15       | 14     | 13     | 12 | 11       | 10     | 9      | 8      |
| RESERVED |        |        |    |          |        | FREE   | SOFT   |
| R-0h     |        |        |    |          |        | R/W-0h | R/W-0h |
| 7        | 6      | 5      | 4  | 3        | 2      | 1      | 0      |
| FRST     | GRST   | XINTM  |    | XSYNCERR | XEMPTY | XRDY   | XRST   |
| R/W-0h   | R/W-0h | R/W-0h |    | R/W-0h   | R-0h   | R-0h   | R/W-0h |

**Table 15-77. SPCR2 Register Field Descriptions**

| Bit   | Field    | Type | Reset | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|-------|----------|------|-------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-10 | RESERVED | R    | 0h    | Reserved                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| 9     | FREE     | R/W  | 0h    | Free run bit.<br>When a breakpoint is encountered in the high-level language debugger, FREE determines whether the McBSP transmit and receive clocks continue to run or whether they are affected as determined by the SOFT bit. When one of the clocks stops, the corresponding data transfer (transmission or reception) stops.<br>Reset type: SYSRSn                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| 8     | SOFT     | R/W  | 0h    | Soft stop bit.<br>When FREE = 0, SOFT determines the response of the McBSP transmit and receive clocks when a breakpoint is encountered in the high-level language debugger. When one of the clocks stops, the corresponding data transfer (transmission or reception) stops.<br>Reset type: SYSRSn                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| 7     | FRST     | R/W  | 0h    | Frame-synchronization logic reset bit.<br>The sample rate generator of the McBSP includes frame-synchronization logic to generate an internal frame-synchronization signal. You can use FRST to take the frame-synchronization logic into and out of its reset state. This bit has a negative polarity<br>FRST = 0 indicates the reset state.<br>Reset type: SYSRSn<br>0h (R/W) = If you read a 0, the frame-synchronization logic is in its reset state.<br>If you write a 0, you reset the frame-synchronization logic.<br>In the reset state, the frame-synchronization logic does not generate a frame-synchronization signal (FSG).<br>1h (R/W) = If you read a 1, the frame-synchronization logic is enabled.<br>If you write a 1, you enable the frame-synchronization logic by taking it out of its reset state.<br>When the frame-synchronization logic is enabled (FRST = 1) and the sample rate generator as a whole is enabled (GRST = 1), the frame-synchronization logic generates the frame-synchronization signal FSG as programmed. |

**Table 15-77. SPCR2 Register Field Descriptions (continued)**

| Bit | Field    | Type | Reset | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|-----|----------|------|-------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 6   | GRST     | R/W  | 0h    | <p>Sample rate generator reset bit.</p> <p>You can use GRST to take the McBSP sample rate generator into and out of its reset state. This bit has a negative polarity GRST = 0 indicates the reset state.</p> <p>Reset type: SYSRSn</p> <p>0h (R/W) = If you read a 0, the sample rate generator is in its reset state.</p> <p>If you write a 0, you reset the sample rate generator.</p> <p>If GRST = 0 due to a reset, CLKG is driven by the CPU clock divided by 2, and FSG is driven low (inactive). If GRST = 0 due to program code, CLKG and FSG are both driven low (inactive).</p> <p>1h (R/W) = If you read a 1, the sample rate generator is enabled. If you write a 1, you enable the sample rate generator by taking it out of its reset state.</p> <p>When enabled, the sample rate generator generates the clock signal CLKG as programmed in the sample rate generator registers. If FRST = 1, the generator also generates the frame-synchronization signal FSG as programmed in the sample rate generator registers.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| 5-4 | XINTM    | R/W  | 0h    | <p>Transmit interrupt mode bits.</p> <p>XINTM determines which event in the McBSP transmitter generates a transmit interrupt (XINT) request. If XINT is properly enabled, the CPU services the interrupt request otherwise, the CPU ignores the request.</p> <p>Reset type: SYSRSn</p> <p>0h (R/W) = The McBSP sends a transmit interrupt (XINT) request to the CPU when the XRDY bit changes from 0 to 1, indicating that transmitter is ready to accept new data (the content of DXR[1,2] has been copied to XSR[1,2]).</p> <p>Regardless of the value of XINTM, you can check XRDY to determine whether a word transfer is complete.</p> <p>The McBSP sends an XINT request to the CPU when 16 enabled bits have been transmitted on the DX pin.</p> <p>1h (R/W) = In the multichannel selection mode, the McBSP sends an XINT request to the CPU after every 16- channel block is transmitted in a frame.</p> <p>Outside of the multichannel selection mode, no interrupt request is sent.</p> <p>2h (R/W) = The McBSP sends an XINT request to the CPU when each transmit frame-synchronization pulse is detected. The interrupt request is sent even if the transmitter is in its reset state.</p> <p>3h (R/W) = The McBSP sends an XINT request to the CPU when the XSYNCERR bit is set, indicating a transmit frame-synchronization error.</p> <p>Regardless of the value of XINTM, you can check XSYNCERR to determine whether a transmit frame-synchronization error occurred.</p> |
| 3   | XSYNCERR | R/W  | 0h    | <p>Transmit frame-synchronization error bit.</p> <p>XSYNCERR is set when a transmit frame-synchronization error is detected by the McBSP. If XINTM = 11b, the McBSP sends a transmit interrupt (XINT) request to the CPU when XSYNCERR is set. The flag remains set until you write a 0 to it or reset the transmitter.</p> <p>Reset type: SYSRSn</p> <p>0h (R/W) = No error</p> <p>1h (R/W) = Transmit frame-synchronization error</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |

**Table 15-77. SPCR2 Register Field Descriptions (continued)**

| Bit | Field  | Type | Reset | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|-----|--------|------|-------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 2   | XEMPTY | R    | 0h    | <p>Transmitter empty bit.</p> <p>XEMPTY is cleared when the transmitter is ready to send new data but no new data is available (transmitter-empty condition). This bit has a negative polarity</p> <p>a transmitter-empty condition is indicated by XEMPTY = 0.</p> <p>Reset type: SYSRSn</p> <p>0h (R/W) = Transmitter-empty condition</p> <p>Typically this indicates that all the bits of the current word have been transmitted but there is no new</p> <p>data in DXR1. XEMPTY is also cleared if the transmitter is reset and then restarted.</p> <p>1h (R/W) = No transmitter-empty condition</p>                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| 1   | XRDY   | R    | 0h    | <p>Transmitter ready bit.</p> <p>XRDY is set when the transmitter is ready to accept new data in DXR[1,2]. Specifically, XRDY is set in response to a copy from DXR1 to XSR1.</p> <p>If the transmit interrupt mode is XINTM = 00b, the McBSP sends a transmit interrupt (XINT) request to the CPU when XRDY changes from 0 to 1.</p> <p>Also, when XRDY changes from 0 to 1, the McBSP sends a transmit synchronization event (XEVT) signal to the DMA controller.</p> <p>Reset type: SYSRSn</p> <p>0h (R/W) = Transmitter not ready</p> <p>When DXR1 is loaded, XRDY is automatically cleared.</p> <p>1h (R/W) = Transmitter ready: DXR[1,2] is ready to accept new data.</p> <p>If both DXRs are needed (word length larger than 16 bits), the CPU or the DMA controller must load DXR2 first and then load DXR1. As soon as DXR1 is loaded, the contents of both DXRs are copied to the transmit shift registers (XSRs), as described in the next step. If DXR2 is not loaded first, the previous content of DXR2 is passed to the XSR2</p> |
| 0   | XRST   | R/W  | 0h    | <p>Transmitter reset bit. You can use XRST to take the McBSP transmitter into and out of its reset state. This bit has a negative polarity</p> <p>XRST = 0 indicates the reset state.</p> <p>To read about the effects of a transmitter reset, see Section 15.10.2, Resetting and Initializing a McBSP.</p> <p>Reset type: SYSRSn</p> <p>0h (R/W) = If you read a 0, the transmitter is in its reset state.</p> <p>If you write a 0, you reset the transmitter.</p> <p>1h (R/W) = If you read a 1, the transmitter is enabled.</p> <p>If you write a 1, you enable the transmitter by taking it out of its reset state.</p>                                                                                                                                                                                                                                                                                                                                                                                                                     |

### 15.11.6 SPCR1 Register (Offset = 5h) [reset = 0h]

SPCR1 is shown in [Figure 15-70](#) and described in [Table 15-78](#).

Return to the [Summary Table](#).

SPCR1 contains control and status bits for various McBSP functions such as digital loopback, receive data justification, clock stop mode, receive interrupt mode, DX pin delay enabler, receiver status bits, and receiver reset control.

**Figure 15-70. SPCR1 Register**

| 15     | 14       | 13     | 12     | 11       | 10       | 9    | 8      |
|--------|----------|--------|--------|----------|----------|------|--------|
| DLB    | RJUST    |        | CLKSTP |          | RESERVED |      |        |
| R/W-0h | R/W-0h   |        | R/W-0h |          | R-0h     |      |        |
| 7      | 6        | 5      | 4      | 3        | 2        | 1    | 0      |
| DXENA  | RESERVED | RINTM  |        | RSYNCERR | RFULL    | RRDY | RRST   |
| R/W-0h |          | R/W-0h |        | R/W-0h   | R-0h     | R-0h | R/W-0h |

**Table 15-78. SPCR1 Register Field Descriptions**

| Bit   | Field | Type | Reset | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|-------|-------|------|-------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15    | DLB   | R/W  | 0h    | Digital loopback mode bit.<br>DLB disables or enables the digital loopback mode of the McBSP:<br>Reset type: SYSRSn<br>0h (R/W) = Disabled<br>Internal DR is supplied by the MDRx pin. Internal FSR and internal MCLKR can be supplied by their respective pins or by the sample rate generator, depending on the mode bits FSRM and CLKRM.<br>Internal DX is supplied by the MDXx pin. Internal FSX and internal CLKX are supplied by their respective pins or are generated internally, depending on the mode bits FSXM and CLKXM.<br>1h (R/W) = Enabled<br>Internal receive signals are supplied by internal transmit signals:<br>MDRx connected to MDXx<br>MFSRx connected to MFSXx<br>MCLKR connected to MCLKXx<br>This mode allows you to test serial port code with a single DSP. The McBSP transmitter directly supplies data, frame synchronization, and clocking to the McBSP receiver. |
| 14-13 | RJUST | R/W  | 0h    | Receive sign-extension and justification mode bits.<br>During reception, RJUST determines how data is justified and bit filled before being passed to the data receive registers (DRR1, DRR2). RJUST is ignored if you enable a companding mode with the RCOMPAND bits. In a companding mode, the 8-bit compressed data in RBR1 is expanded to left-justified 16-bit data in DRR1.<br>Reset type: SYSRSn<br>0h (R/W) = Right justify the data and zero fill the MSBs<br>1h (R/W) = Right justify the data and sign-extend the data into the MSBs<br>2h (R/W) = Left justify the data and zero fill the LSBs<br>3h (R/W) = Reserved (do not use)                                                                                                                                                                                                                                                   |



**Table 15-78. SPCR1 Register Field Descriptions (continued)**

| Bit   | Field    | Type | Reset | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|-------|----------|------|-------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 12-11 | CLKSTP   | R/W  | 0h    | <p>Clock stop mode bits.</p> <p>CLKSTP allows you to use the clock stop mode to support the SPI master/slave protocol. If you will not be using the SPI protocol, you can clear CLKSTP to disable the clock stop mode.</p> <p>In the clock stop mode, the clock stops at the end of each data transfer. At the beginning of each data transfer, the clock starts immediately (CLKSTP = 10b) or after a half-cycle delay (CLKSTP = 11b).</p> <p>Reset type: SYSRSn</p> <p>0h (R/W) = Clock stop mode is disabled.</p> <p>1h (R/W) = Clock stop mode is disabled.</p> <p>2h (R/W) = Clock stop mode, without clock delay</p> <p>3h (R/W) = Clock stop mode, with half-cycle clock delay</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| 10-8  | RESERVED | R    | 0h    | Reserved                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| 7     | DXENA    | R/W  | 0h    | <p>DX delay enabler mode bit.</p> <p>DXENA controls the delay enabler for the DX pin. The enabler creates an extra delay for turn-on time (for the length of the delay, see the device-specific data sheet).</p> <p>Reset type: SYSRSn</p> <p>0h (R/W) = DX delay enabler off</p> <p>1h (R/W) = DX delay enabler on</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| 6     | RESERVED | R/W  | 0h    | Reserved                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| 5-4   | RINTM    | R/W  | 0h    | <p>Receive interrupt mode bits.</p> <p>RINTM determines which event in the McBSP receiver generates a receive interrupt (RINT) request. If RINT is properly enabled inside the CPU, the CPU services the interrupt request otherwise, the CPU ignores the request.</p> <p>Reset type: SYSRSn</p> <p>0h (R/W) = The McBSP sends a receive interrupt (RINT) request to the CPU when the RRDY bit changes from 0 to 1, indicating that receive data is ready to be read (the content of RBR[1,2] has been copied to DRR[1,2]):</p> <p>Regardless of the value of RINTM, you can check RRDY to determine whether a word transfer is complete.</p> <p>The McBSP sends a RINT request to the CPU when 16 enabled bits have been received on the DR pin.</p> <p>1h (R/W) = In the multichannel selection mode, the McBSP sends a RINT request to the CPU after every 16- channel block is received in a frame.</p> <p>Outside of the multichannel selection mode, no interrupt request is sent.</p> <p>2h (R/W) = The McBSP sends a RINT request to the CPU when each receive frame-synchronization pulse is detected. The interrupt request is sent even if the receiver is in its reset state.</p> <p>3h (R/W) = The McBSP sends a RINT request to the CPU when the RSYNCERR bit is set, indicating a receive frame-synchronization error.</p> <p>Regardless of the value of RINTM, you can check RSYNCERR to determine whether a receive frame-synchronization error occurred.</p> |

**Table 15-78. SPCR1 Register Field Descriptions (continued)**

| Bit | Field    | Type | Reset | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|-----|----------|------|-------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 3   | RSYNCERR | R/W  | 0h    | <p>Receive frame-sync error bit.</p> <p>RSYNCERR is set when a receive frame-sync error is detected by the McBSP. If RINTM = 11b, the McBSP sends a receive interrupt (RINT) request to the CPU when RSYNCERR is set. The flag remains set until you write a 0 to it or reset the receiver.</p> <p>Reset type: SYSRSn</p> <p>0h (R/W) = No error</p> <p>1h (R/W) = Receive frame-synchronization error.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| 2   | RFULL    | R    | 0h    | <p>Receiver full bit.</p> <p>RFULL is set when the receiver is full with new data and the previously received data has not been read (receiver-full condition). For more details about this condition,</p> <p>Reset type: SYSRSn</p> <p>0h (R/W) = No receiver-full condition</p> <p>1h (R/W) = Receiver-full condition: RSR[1,2] and RBR[1,2] are full with new data, but the previous data in DRR[1,2] has not been read</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| 1   | RRDY     | R    | 0h    | <p>Receiver ready bit.</p> <p>RRDY is set when data is ready to be read from DRR[1,2]. Specifically, RRDY is set in response to a copy from RBR1 to DRR1. If the receive interrupt mode is RINTM = 00b, the McBSP sends a receive interrupt request to the CPU when RRDY changes from 0 to 1.</p> <p>Also, when RRDY changes from 0 to 1, the McBSP sends a receive synchronization event (REVT) signal to the DMA controller.</p> <p>Reset type: SYSRSn</p> <p>0h (R/W) = Receiver not ready</p> <p>When the content of DRR1 is read, RRDY is automatically cleared.</p> <p>1h (R/W) = Receiver ready: New data can be read from DRR[1,2].</p> <p>Important: If both DRRs are required (word length larger than 16 bits), the CPU or the DMA controller must read from DRR2 first and then from DRR1. As soon as DRR1 is read, the next RBR-to-DRR copy is initiated. If DRR2 is not read first, the data in DRR2 is lost.</p> |
| 0   | RRST     | R/W  | 0h    | <p>Receiver reset bit.</p> <p>You can use RRST to take the McBSP receiver into and out of its reset state. This bit has a negative polarity</p> <p>RRST = 0 indicates the reset state.</p> <p>Reset type: SYSRSn</p> <p>0h (R/W) = If you read a 0, the receiver is in its reset state.</p> <p>If you write a 0, you reset the receiver.</p> <p>1h (R/W) = If you read a 1, the receiver is enabled.</p> <p>If you write a 1, you enable the receiver by taking it out of its reset state.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                  |

### 15.11.7 RCR2 Register (Offset = 6h) [reset = 0h]

RCR2 is shown in [Figure 15-71](#) and described in [Table 15-79](#).

Return to the [Summary Table](#).

RCR2 contains control bits for the receiver such as number of phases in each frame, the serial word length and number of words for phase 2 of dual-phase frames, receive companding mode, receive frame synchronization ignore function, and the receive data delay.

**Figure 15-71. RCR2 Register**

|         |  |         |  |    |  |          |  |        |  |    |  |        |  |         |  |        |  |  |  |
|---------|--|---------|--|----|--|----------|--|--------|--|----|--|--------|--|---------|--|--------|--|--|--|
| 15      |  | 14      |  | 13 |  | 12       |  | 11     |  | 10 |  | 9      |  | 8       |  |        |  |  |  |
| RPHASE  |  | RFRLEN2 |  |    |  |          |  |        |  |    |  |        |  |         |  |        |  |  |  |
| R/W-0h  |  |         |  |    |  |          |  | R/W-0h |  |    |  |        |  |         |  |        |  |  |  |
| 7       |  | 6       |  | 5  |  | 4        |  | 3      |  | 2  |  | 1      |  | 0       |  |        |  |  |  |
| RWDLEN2 |  |         |  |    |  | RCOMPAND |  |        |  |    |  | RFIG   |  | RDATDLY |  |        |  |  |  |
| R/W-0h  |  |         |  |    |  | R/W-0h   |  |        |  |    |  | R/W-0h |  |         |  | R/W-0h |  |  |  |

**Table 15-79. RCR2 Register Field Descriptions**

| Bit  | Field  | Type | Reset | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|------|--------|------|-------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15   | RPHASE | R/W  | 0h    | Receive phase number bit.<br>RPHASE determines whether the receive frame has one phase or two phases. For each phase you can define the serial word length and the number of serial words in the phase. To set up phase 1, program RWDLEN1 (word length) and RFRLN1 (number of words). To set up phase 2 (if there are two phases), program RWDLEN2 and RFRLN2.<br>Reset type: SYSRSn<br>0h (R/W) = Single-phase frame<br>The receive frame has only one phase, phase 1.<br>1h (R/W) = Dual-phase frame<br>The receive frame has two phases, phase 1 and phase 2.                                                                                                                                                                                                                                                                                                                                                                                 |
| 14-8 | RFRLN2 | R/W  | 0h    | Receive frame length 2 (1 to 128 words). Each frame of receive data can have one or two phases, depending on value that you load into the RPHASE bit. If a single-phase frame is selected, RFRLN1 in RCR1 selects the number of serial words (8, 12, 16, 20, 24, or 32 bits per word) in the frame. If a dual-phase frame is selected, RFRLN1 determines the number of serial words in phase 1 of the frame, and RFRLN2 in RCR2 determines the number of words in phase 2 of the frame. The 7-bit RFRLN fields allow up to 128 words per phase. See <a href="#">Section 15.7.9</a> to determine the frame length. This length corresponds to the number of words or logical time slots or channels per frame-synchronization period.<br>Program the RFRLN fields with [w minus 1], where w represents the number of words per phase. For example, if you want a phase length of 128 words in phase 2, load 127 into RFRLN2.<br>Reset type: SYSRSn |

**Table 15-79. RCR2 Register Field Descriptions (continued)**

| Bit | Field    | Type | Reset | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|-----|----------|------|-------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 7-5 | RWDLEN2  | R/W  | 0h    | <p>Receive word length 2. Each frame of receive data can have one or two phases, depending on the value that you load into the RPHASE bit. If a single-phase frame is selected, RWDLEN1 in RCR1 selects the length for every serial word received in the frame. If a dual-phase frame is selected, RWDLEN1 determines the length of the serial words in phase 1 of the frame, and RWDLEN2 in RCR2 determines the word length in phase 2 of the frame.</p> <p>Reset type: SYSRSn</p> <p>0h (R/W) = 8 bits<br/> 1h (R/W) = 12 bits<br/> 2h (R/W) = 16 bits<br/> 3h (R/W) = 20 bits<br/> 4h (R/W) = 24 bits<br/> 5h (R/W) = 32 bits<br/> 6h (R/W) = Reserved (do not use)<br/> 7h (R/W) = Reserved (do not use)</p>                                                                                                                                                                                                   |
| 4-3 | RCOMPAND | R/W  | 0h    | <p>Receive companding mode bits. Companding (COMpress and exPAND) hardware allows compression and expansion of data in either u-law or A-law format. RCOMPAND allows you to choose one of the following companding modes for the McBSP receiver:</p> <p>For more details about these companding modes, see Section 15.1.5, Companding (Compressing and Expanding) Data.</p> <p>Reset type: SYSRSn</p> <p>0h (R/W) = No companding, any size data, MSB received first<br/> 1h (R/W) = No companding, 8-bit data, LSB received first<br/> 2h (R/W) = u-law companding, 8-bit data, MSB received first<br/> 3h (R/W) = A-law companding, 8-bit data, MSB received first</p>                                                                                                                                                                                                                                           |
| 2   | RFIG     | R/W  | 0h    | <p>Receive frame-synchronization ignore bit. If a frame-synchronization pulse starts the transfer of a new frame before the current frame is fully received, this pulse is treated as an unexpected frame-synchronization pulse. Unexpected Receive Frame-Synchronization Pulse.</p> <p>Setting RFIG causes the serial port to ignore unexpected frame-synchronization signals during reception.</p> <p>Reset type: SYSRSn</p> <p>0h (R/W) = Frame-synchronization detect. An unexpected FSR pulse causes the receiver to discard the contents of RSR[1,2] in favor of the new incoming data. The receiver:</p> <ol style="list-style-type: none"> <li>1. Aborts the current data transfer</li> <li>2. Sets RSYNCERR in SPCR1</li> <li>3. Begins the transfer of a new data word</li> </ol> <p>1h (R/W) = Frame-synchronization ignore. An unexpected FSR pulse is ignored. Reception continues uninterrupted.</p> |
| 1-0 | RDATDLY  | R/W  | 0h    | <p>Receive data delay bits. RDATDLY specifies a data delay of 0, 1, or 2 receive clock cycles after frame synchronization and before the reception of the first bit of the frame.</p> <p>Reset type: SYSRSn</p> <p>0h (R/W) = 0-bit data delay<br/> 1h (R/W) = 1-bit data delay<br/> 2h (R/W) = 2-bit data delay<br/> 3h (R/W) = Reserved (do not use)</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |

### 15.11.8 RCR1 Register (Offset = 7h) [reset = 0h]

RCR1 is shown in [Figure 15-72](#) and described in [Table 15-80](#).

Return to the [Summary Table](#).

RCR1 contains control bits for the receiver such as the serial word length and number of words for single-phase transmissions, or phase 1 if dual-phase frames are used.

**Figure 15-72. RCR1 Register**

|          |        |    |    |          |    |   |   |
|----------|--------|----|----|----------|----|---|---|
| 15       | 14     | 13 | 12 | 11       | 10 | 9 | 8 |
| RESERVED | RFRLN1 |    |    |          |    |   |   |
| R-0h     | R/W-0h |    |    |          |    |   |   |
| 7        | 6      | 5  | 4  | 3        | 2  | 1 | 0 |
| RWDLEN1  |        |    |    | RESERVED |    |   |   |
| R/W-0h   |        |    |    | R-0h     |    |   |   |

**Table 15-80. RCR1 Register Field Descriptions**

| Bit  | Field    | Type | Reset | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|------|----------|------|-------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15   | RESERVED | R    | 0h    | Reserved                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| 14-8 | RFRLN1   | R/W  | 0h    | Receive frame length 1 (1 to 128 words).<br>Each frame of receive data can have one or two phases, depending on value that you load into the RPHASE bit. If a single-phase frame is selected, RFRLN1 in RCR1 selects the number of serial words (8, 12, 16, 20, 24, or 32 bits per word) in the frame. If a dual-phase frame is selected, RFRLN1 determines the number of serial words in phase 1 of the frame, and RFRLN2 in RCR2 determines the number of words in phase 2 of the frame. The 7-bit RFRLN fields allow up to 128 words per phase. See <a href="#">Section 15.7.9</a> to determine the frame length. This length corresponds to the number of words or logical time slots or channels per frame-synchronization period. Program the RFRLN fields with [w minus 1], where w represents the number of words per phase. For example, if you want a phase length of 128 words in phase 1, load 127 into RFRLN1.<br>Note: When operating in SPI mode, the frame length can only be 1 word.<br>Reset type: SYSRSn |
| 7-5  | RWDLEN1  | R/W  | 0h    | Receive word length 1.<br>Each frame of receive data can have one or two phases, depending on the value that you load into the RPHASE bit. If a single-phase frame is selected, RWDLEN1 in RCR1 selects the length for every serial word received in the frame. If a dual-phase frame is selected, RWDLEN1 determines the length of the serial words in phase 1 of the frame, and RWDLEN2 in RCR2 determines the word length in phase 2 of the frame.<br>Reset type: SYSRSn<br>0h (R/W) = 8 bits<br>1h (R/W) = 12 bits<br>2h (R/W) = 16 bits<br>3h (R/W) = 20 bits<br>4h (R/W) = 24 bits<br>5h (R/W) = 32 bits<br>6h (R/W) = Reserved (do not use)<br>7h (R/W) = Reserved (do not use)                                                                                                                                                                                                                                                                                                                                      |
| 4-0  | RESERVED | R    | 0h    | Reserved                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |

### 15.11.9 XCR2 Register (Offset = 8h) [reset = 0h]

XCR2 is shown in [Figure 15-73](#) and described in [Table 15-81](#).

Return to the [Summary Table](#).

XCR2 contains control bits for the transmitter such as number of phases in each frame, the serial word length and number of words for phase 2 of dual-phase frames, transmit companding mode, transmit frame-synchronization ignore function, and the transmit data-delay control.

**Figure 15-73. XCR2 Register**

|         |  |         |  |    |  |          |  |        |  |        |  |         |  |   |  |
|---------|--|---------|--|----|--|----------|--|--------|--|--------|--|---------|--|---|--|
| 15      |  | 14      |  | 13 |  | 12       |  | 11     |  | 10     |  | 9       |  | 8 |  |
| XPHASE  |  | XFRLEN2 |  |    |  |          |  |        |  |        |  |         |  |   |  |
| R/W-0h  |  |         |  |    |  |          |  | R/W-0h |  |        |  |         |  |   |  |
| 7       |  | 6       |  | 5  |  | 4        |  | 3      |  | 2      |  | 1       |  | 0 |  |
| XWDLEN2 |  |         |  |    |  | XCOMPAND |  |        |  | XFIG   |  | XDATDLY |  |   |  |
| R/W-0h  |  |         |  |    |  | R/W-0h   |  |        |  | R/W-0h |  | R/W-0h  |  |   |  |

**Table 15-81. XCR2 Register Field Descriptions**

| Bit  | Field   | Type | Reset | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|------|---------|------|-------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15   | XPHASE  | R/W  | 0h    | Transmit phase number bit.<br>XPHASE determines whether the transmit frame has one phase or two phases. For each phase you can define the serial word length and the number of serial words in the phase. To set up phase 1, program XWDLEN1 (word length) and XFRLEN1 (number of words). To set up phase 2 (if there are two phases), program XWDLEN2 and XFRLEN2.<br>Reset type: SYSRSn                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| 14-8 | XFRLEN2 | R/W  | 0h    | Transmit frame length 2 (1 to 128 words).<br>Each frame of transmit data can have one or two phases, depending on value that you load into the XPHASE bit. If a single-phase frame is selected, XFRLEN1 in XCR1 selects the number of serial words (8, 12, 16, 20, 24, or 32 bits per word) in the frame. If a dual-phase frame is selected, XFRLEN1 determines the number of serial words in phase 1 of the frame and XFRLEN2 in XCR2 determines the number of words in phase 2 of the frame. The 7-bit XFRLEN fields allow up to 128 words per phase. See <a href="#">Section 15.8.9</a> to determine the frame length. This length corresponds to the number of words or logical time slots or channels per frame-synchronization period. Program the XFRLEN fields with [w minus 1], where w represents the number of words per phase. For example, if you want a phase length of 128 words in phase 1, load 127 into XFRLEN1.<br>Reset type: SYSRSn |

**Table 15-81. XCR2 Register Field Descriptions (continued)**

| Bit | Field    | Type | Reset | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|-----|----------|------|-------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 7-5 | XWDLEN2  | R/W  | 0h    | <p>Transmit word length 2.</p> <p>Each frame of transmit data can have one or two phases, depending on the value that you load into the XPHASE bit. If a single-phase frame is selected, XWDLEN1 in XCR1 selects the length for every serial word transmitted in the frame. If a dual-phase frame is selected, XWDLEN1 determines the length of the serial words in phase 1 of the frame and XWDLEN2 in XCR2 determines the word length in phase 2 of the frame.</p> <p>Reset type: SYSRSn</p> <p>0h (R/W) = 8 bits</p> <p>1h (R/W) = 12 bits</p> <p>2h (R/W) = 16 bits</p> <p>3h (R/W) = 20 bits</p> <p>4h (R/W) = 24 bits</p> <p>5h (R/W) = 32 bits</p> <p>6h (R/W) = Reserved (do not use)</p> <p>7h (R/W) = Reserved (do not use)</p>                                                                                                                                                                                                                                                                            |
| 4-3 | XCOMPAND | R/W  | 0h    | <p>Transmit companding mode bits.</p> <p>Companding (COMpress and exPAND) hardware allows compression and expansion of data in either <math>\mu</math>-law or A-law format.</p> <p>Reset type: SYSRSn</p> <p>0h (R/W) = No companding, any size data, MSB received first</p> <p>1h (R/W) = No companding, 8-bit data, LSB received first</p> <p>2h (R/W) = u-law companding, 8-bit data, MSB received first</p> <p>3h (R/W) = A-law companding, 8-bit data, MSB received first</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| 2   | XFIG     | R/W  | 0h    | <p>Transmit frame-synchronization ignore bit.</p> <p>If a frame-synchronization pulse starts the transfer of a new frame before the current frame is fully transmitted, this pulse is treated as an unexpected frame-synchronization pulse.</p> <p>Setting XFIG causes the serial port to ignore unexpected frame-synchronization pulses during transmission.</p> <p>Reset type: SYSRSn</p> <p>0h (R/W) = Frame-synchronization detect. An unexpected FSX pulse causes the transmitter to discard the content of XSR[1,2]. The transmitter:</p> <ol style="list-style-type: none"> <li>1. Aborts the present transmission</li> <li>2. Sets XSYNCERR in SPCR2</li> <li>3. Begins a new transmission from DXR[1,2]. If new data was written to DXR[1,2] since the last DXR[1,2]-to-XSR[1,2] copy, the current value in XSR[1,2] is lost. Otherwise, the same data is transmitted.</li> </ol> <p>1h (R/W) = Frame-synchronization ignore. An unexpected FSX pulse is ignored. Transmission continues uninterrupted.</p> |
| 1-0 | XDATDLY  | R/W  | 0h    | <p>Transmit data delay bits. XDATDLY specifies a data delay of 0, 1, or 2 transmit clock cycles after frame synchronization and before the transmission of the first bit of the frame.</p> <p>Reset type: SYSRSn</p> <p>0h (R/W) = 0-bit data delay</p> <p>1h (R/W) = 1-bit data delay</p> <p>2h (R/W) = 2-bit data delay</p> <p>3h (R/W) = Reserved (do not use)</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |

### 15.11.10 XCR1 Register (Offset = 9h) [reset = 0h]

XCR1 is shown in [Figure 15-74](#) and described in [Table 15-82](#).

Return to the [Summary Table](#).

XCR1 contains control bits for the transmitter such as the serial word length and number of words for single-phase transmissions, or phase 1 if dual-phase frames are used.

**Figure 15-74. XCR1 Register**

|          |        |    |    |          |    |   |   |
|----------|--------|----|----|----------|----|---|---|
| 15       | 14     | 13 | 12 | 11       | 10 | 9 | 8 |
| RESERVED | XFRLN1 |    |    |          |    |   |   |
| R-0h     |        |    |    | R/W-0h   |    |   |   |
| 7        | 6      | 5  | 4  | 3        | 2  | 1 | 0 |
| XWDLEN1  |        |    |    | RESERVED |    |   |   |
| R/W-0h   |        |    |    | R-0h     |    |   |   |

**Table 15-82. XCR1 Register Field Descriptions**

| Bit  | Field    | Type | Reset | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|------|----------|------|-------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15   | RESERVED | R    | 0h    | Reserved                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| 14-8 | XFRLN1   | R/W  | 0h    | <p>Transmit frame length 1 (1 to 128 words). Each frame of transmit data can have one or two phases, depending on value that you load into the XPHASE bit. If a single-phase frame is selected, XFRLN1 in XCR1 selects the number of serial words (8, 12, 16, 20, 24, or 32 bits per word) in the frame. If a dual-phase frame is selected, XFRLN1 determines the number of serial words in phase 1 of the frame and XFRLN2 in XCR2 determines the number of words in phase 2 of the frame. The 7-bit XFRLN fields allow up to 128 words per phase. See <a href="#">Section 15.8.9</a> to determine the frame length. This length corresponds to the number of words or logical time slots or channels per frame-synchronization period. Program the XFRLN fields with [w minus 1], where w represents the number of words per phase. For example, if you want a phase length of 128 words in phase 1, load 127 into XFRLN1.</p> <p>Note: When operating in SPI mode, the frame length can only be 1 word.</p> <p>Reset type: SYSRSn</p> |
| 7-5  | XWDLEN1  | R/W  | 0h    | <p>Transmit word length 1.</p> <p>Each frame of transmit data can have one or two phases, depending on the value that you load into the XPHASE bit. If a single-phase frame is selected, XWDLEN1 in XCR1 selects the length for every serial word transmitted in the frame. If a dual-phase frame is selected, XWDLEN1 determines the length of the serial words in phase 1 of the frame and XWDLEN2 in XCR2 determines the word length in phase 2 of the frame.</p> <p>Reset type: SYSRSn</p> <p>0h (R/W) = 8 bits</p> <p>1h (R/W) = 12 bits</p> <p>2h (R/W) = 16 bits</p> <p>3h (R/W) = 20 bits</p> <p>4h (R/W) = 24 bits</p> <p>5h (R/W) = 32 bits</p> <p>6h (R/W) = Reserved (do not use)</p> <p>7h (R/W) = Reserved (do not use)</p>                                                                                                                                                                                                                                                                                                |
| 4-0  | RESERVED | R    | 0h    | Reserved                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |



### 15.11.11 SRGR2 Register (Offset = Ah) [reset = 0h]

SRGR2 is shown in [Figure 15-75](#) and described in [Table 15-83](#).

Return to the [Summary Table](#).

SRGR2 contains control bits for the sample rate generator such as input clock selection, internal transmit frame-synchronization source selection, and the period between frame-synchronization pulses.

If an external source provides the input clock source for the sample rate generator, a control bit is provided to make the CLKG synchronized to an external frame-synchronization pulse on the FSR pin so that CLKG is kept in phase with the input clock.

**Figure 15-75. SRGR2 Register**

|        |          |        |        |        |    |   |   |
|--------|----------|--------|--------|--------|----|---|---|
| 15     | 14       | 13     | 12     | 11     | 10 | 9 | 8 |
| GSYNC  | RESERVED | CLKSM  | FSGM   | FPER   |    |   |   |
| R/W-0h | R/W-0h   | R/W-0h | R/W-0h | R/W-0h |    |   |   |
| 7      | 6        | 5      | 4      | 3      | 2  | 1 | 0 |
| FPER   |          |        |        |        |    |   |   |
| R/W-0h |          |        |        |        |    |   |   |

**Table 15-83. SRGR2 Register Field Descriptions**

| Bit | Field    | Type | Reset | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|-----|----------|------|-------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15  | GSYNC    | R/W  | 0h    | <p>Clock synchronization mode bit for CLKG. GSYNC is used only when the input clock source for the sample rate generator is external?on the MCLKR pin. When GSYNC = 1, the clock signal (CLKG) and the frame-synchronization signal (FSG) generated by the sample rate generator are made dependent on pulses on the FSR pin.</p> <p>Reset type: SYSRSn</p> <p>0h (R/W) = No clock synchronization</p> <p>CLKG oscillates without adjustment, and FSG pulses every (FPER + 1) CLKG cycles.</p> <p>1h (R/W) = Clock synchronization</p> <ul style="list-style-type: none"> <li>- CLKG is adjusted as necessary so that it is synchronized with the input clock on the MCLKR pin.</li> <li>- FSG pulses. FSG only pulses in response to a pulse on the FSR pin.</li> </ul> <p>The frame-synchronization period defined in FPER is ignored.</p> |
| 14  | RESERVED | R/W  | 0h    | Reserved                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |

**Table 15-83. SRGR2 Register Field Descriptions (continued)**

| Bit  | Field | Type | Reset | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|------|-------|------|-------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 13   | CLKSM | R/W  | 0h    | <p>Sample rate generator mode</p> <p>Reset type: SYSRSn</p> <p>0h (R/W) = Sample rate generator input clock mode bit. The sample rate generator can accept an input clock signal and divide it down according to CLKGDV to produce an output clock signal, CLKG. The frequency of CLKG is:</p> $\text{CLKG frequency} = (\text{input clock frequency}) / (\text{CLKGDV} + 1)$ <p>CLKSM is used in conjunction with the SCLKME bit to determine the source for the input clock.</p> <p>A reset selects the CPU clock as the input clock and forces the CLKG frequency to half the LSPCLK frequency.</p> <p>The input clock for the sample rate generator is taken from the MCLKR pin, depending on the value of the SCLKME bit of PCR:</p> <p>SCLKME CLKSM Input Clock For<br/>Sample Rate Generator</p> <p>0 0 Reserved</p> <p>1 0 Signal on MCLKR pin</p> <p>1h (R/W) = The input clock for the sample rate generator is taken from the LSPCLK or from the MCLKX pin, depending on the value of the SCLKME bit of PCR:</p> <p>SCLKME CLKSM Input Clock For<br/>Sample Rate Generator</p> <p>0 1 LSPCLK</p> <p>1 1 Signal on MCLKX pin</p> |
| 12   | FSGM  | R/W  | 0h    | <p>Sample rate generator transmit frame-synchronization mode bit. The transmitter can get frame synchronization from the FSX pin (FSXM = 0) or from inside the McBSP (FSXM = 1). When FSXM = 1, the FSGM bit determines how the McBSP supplies frame-synchronization pulses.</p> <p>Reset type: SYSRSn</p> <p>0h (R/W) = If FSXM = 1, the McBSP generates a transmit frame-synchronization pulse when the content of DXR[1,2] is copied to XSR[1,2].</p> <p>1h (R/W) = If FSXM = 1, the transmitter uses frame-synchronization pulses generated by the sample rate generator. Program the FWID bits to set the width of each pulse. Program the FPER bits to set the period between pulses.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| 11-0 | FPER  | R/W  | 0h    | <p>Frame-synchronization period bits for FSG.</p> <p>The sample rate generator can produce a clock signal, CLKG, and a frame-synchronization signal, FSG. The period between frame-synchronization pulses on FSG is (FPER + 1) CLKG cycles. The 12 bits of FPER allow a frame-synchronization period of 1 to 4096 CLKG cycles:</p> <p>Reset type: SYSRSn</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |

### 15.11.12 SRGR1 Register (Offset = Bh) [reset = 1h]

SRGR1 is shown in [Figure 15-76](#) and described in [Table 15-84](#).

Return to the [Summary Table](#).

SRGR1 contains control bits for the sample rate generator functions such as the divide down frequency, and the width of the frame-synchronization pulses on FSG.

**Figure 15-76. SRGR1 Register**

|        |    |    |    |    |    |   |   |
|--------|----|----|----|----|----|---|---|
| 15     | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
| FWID   |    |    |    |    |    |   |   |
| R/W-0h |    |    |    |    |    |   |   |
| 7      | 6  | 5  | 4  | 3  | 2  | 1 | 0 |
| CLKGDV |    |    |    |    |    |   |   |
| R/W-1h |    |    |    |    |    |   |   |

**Table 15-84. SRGR1 Register Field Descriptions**

| Bit  | Field  | Type | Reset | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|------|--------|------|-------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-8 | FWID   | R/W  | 0h    | <p>Divide-down value for CLKG.</p> <p>The sample rate generator can accept an input clock signal and divide it down according to CLKGDV to produce an output clock signal, CLKG. The frequency of CLKG is:</p> $\text{CLKG frequency} = (\text{Input clock frequency}) / (\text{CLKGDV} + 1)$ <p>The input clock is selected by the SCLKME and CLKSM bits:</p> <p>SCLKME CLKSM Input Clock For Sample Rate Generator</p> <p>0 0 Reserved</p> <p>0 1 LSPCLK</p> <p>1 0 Signal on MCLKR pin</p> <p>1 1 Signal on MCLKX pin</p> <p>Reset type: SYSRSn</p> |
| 7-0  | CLKGDV | R/W  | 1h    | <p>Frame-synchronization pulse width bits for FSG</p> <p>The sample rate generator can produce a clock signal, CLKG, and a frame-synchronization signal, FSG. For frame-synchronization pulses on FSG, (FWID + 1) is the pulse width in CLKG cycles. The eight bits of FWID allow a pulse width of 1 to 256 CLKG cycles:</p> $0 \leq \text{FWID} \leq 255$ $1 \leq (\text{FWID} + 1) \leq 256 \text{ CLKG cycles}$ <p>The period between the frame-synchronization pulses on FSG is defined by the FPER bits.</p> <p>Reset type: SYSRSn</p>            |

### 15.11.13 MCR2 Register (Offset = Ch) [reset = 0h]

MCR2 is shown in Figure 15-77 and described in Table 15-85.

Return to the [Summary Table](#).

MCR2 contains control bits for the transmitter Multichannel functions such as channel enable mode selection, channel partition modes, channel block assignments, and active channel status bits.

**Figure 15-77. MCR2 Register**

|          |        |    |       |    |    |        |        |
|----------|--------|----|-------|----|----|--------|--------|
| 15       | 14     | 13 | 12    | 11 | 10 | 9      | 8      |
| RESERVED |        |    |       |    |    | XMCME  | XPBBLK |
| R-0h     |        |    |       |    |    | R/W-0h | R/W-0h |
| 7        | 6      | 5  | 4     | 3  | 2  | 1      | 0      |
| XPBBLK   | XPABLK |    | XCBLK |    |    | XMCM   |        |
| R/W-0h   | R/W-0h |    | R-0h  |    |    | R/W-0h |        |

**Table 15-85. MCR2 Register Field Descriptions**

| Bit   | Field    | Type | Reset | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|-------|----------|------|-------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-10 | RESERVED | R    | 0h    | Reserved                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| 9     | XMCME    | R/W  | 0h    | Transmit multichannel partition mode bit.<br>XMCME determines whether only 32 channels or all 128 channels are to be individually selectable. XMCME is only applicable if channels can be individually disabled/enabled or masked/unmasked for transmission (XMCM is nonzero).<br>Reset type: SYSRSn                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| 8-7   | XPBBLK   | R/W  | 0h    | Transmit partition B block bits<br>XPBBLK is only applicable if channels can be individually disabled/enabled and masked/unmasked (XMCM is nonzero) and the 2-partition mode is selected (XMCME = 0). Under these conditions, the McBSP transmitter can transmit or withhold data in any of the 32 channels that are assigned to partitions A and B of the transmitter.<br>The 128 transmit channels of the McBSP are divided equally among 8 blocks (0 through 7). When XPBBLK is applicable, use XPBBLK to assign one of the odd-numbered blocks (1, 3, 5, or 7) to partition B. Use the PABLK bit to assign one of the even numbered blocks (0, 2, 4, or 6) to partition A.<br>If you want to use more than 32 channels, you can change block assignments dynamically. You can assign a new block to one partition while the transmitter is handling activity in the other partition. For example, while the block in partition A is active, you can change which block is assigned to partition B. The XCBLK bits are regularly updated to indicate which block is active.<br>When XMCM = 11b (for symmetric transmission and reception), the transmitter uses the receive block bits (RPABLK and RPBBLK) rather than the transmit block bits (XPABLK and XPBBLK).<br>Reset type: SYSRSn<br>0h (R/W) = Block 1: channels 16 through 31<br>1h (R/W) = Block 3: channels 48 through 63<br>2h (R/W) = Block 5: channels 80 through 95<br>3h (R/W) = Block 7: channels 112 through 127 |

**Table 15-85. MCR2 Register Field Descriptions (continued)**

| Bit | Field  | Type | Reset | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|-----|--------|------|-------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 6-5 | XPABLK | R/W  | 0h    | <p>Transmit partition A block bits.</p> <p>XPABLK is only applicable if channels can be individually disabled/enabled and masked/unmasked (XMCME is nonzero) and the 2-partition mode is selected (XMCME = 0). Under these conditions, the McBSP transmitter can transmit or withhold data in any of the 32 channels that are assigned to partitions A and B of the transmitter. See XPBBLK (bits 8-7) for more information about assigning blocks to partitions A and B.</p> <p>Reset type: SYSRSn</p> <p>0h (R/W) = Block 0: channels 0 through 15</p> <p>1h (R/W) = Block 2: channels 32 through 47</p> <p>2h (R/W) = Block 4: channels 64 through 79</p> <p>3h (R/W) = Block 6: channels 96 through 111</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| 4-2 | XCBLK  | R    | 0h    | <p>Transmit current block indicator.</p> <p>XCBLK indicates which block of 16 channels is involved in the current McBSP transmission:</p> <p>Reset type: SYSRSn</p> <p>0h (R/W) = Block 0: channels 0 through 15</p> <p>1h (R/W) = Block 1: channels 16 through 31</p> <p>2h (R/W) = Block 2: channels 32 through 47</p> <p>3h (R/W) = Block 3: channels 48 through 63</p> <p>4h (R/W) = Block 4: channels 64 through 79</p> <p>5h (R/W) = Block 5: channels 80 through 95</p> <p>6h (R/W) = Block 6: channels 96 through 111</p> <p>7h (R/W) = Block 7: channels 112 through 127</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| 1-0 | XMCM   | R/W  | 0h    | <p>Transmit multichannel selection mode bits.</p> <p>XMCM determines whether all channels or only selected channels are enabled and unmasked for transmission.</p> <p>Reset type: SYSRSn</p> <p>0h (R/W) = No transmit multichannel selection mode is on. All channels are enabled and unmasked. No channels can be disabled or masked</p> <p>1h (R/W) = All channels are disabled unless they are selected in the appropriate transmit channel enable registers (XCERs). If enabled, a channel in this mode is also unmasked.</p> <p>The XMCME bit determines whether 32 channels or 128 channels are selectable in XCERs.</p> <p>2h (R/W) = All channels are enabled, but they are masked unless they are selected in the appropriate transmit channel enable registers (XCERs).</p> <p>The XMCME bit determines whether 32 channels or 128 channels are selectable in XCERs.</p> <p>3h (R/W) = This mode is used for symmetric transmission and reception.</p> <p>All channels are disabled for transmission unless they are enabled for reception in the appropriate receive channel enable registers (RCERs). Once enabled, they are masked unless they are also selected in the appropriate transmit channel enable registers (XCERs).</p> <p>The XMCME bit determines whether 32 channels or 128 channels are selectable in RCERs and XCERs.</p> |

### 15.11.14 MCR1 Register (Offset = Dh) [reset = 0h]

MCR1 is shown in [Figure 15-78](#) and described in [Table 15-86](#).

Return to the [Summary Table](#).

MCR1 contains control bits for the receiver Multichannel functions such as channel enable mode selection, channel partition modes, channel block assignments, and active channel status bits.

**Figure 15-78. MCR1 Register**

|          |        |    |       |    |    |          |        |
|----------|--------|----|-------|----|----|----------|--------|
| 15       | 14     | 13 | 12    | 11 | 10 | 9        | 8      |
| RESERVED |        |    |       |    |    | RMCM     | RPBBLK |
| R-0h     |        |    |       |    |    | R/W-0h   | R/W-0h |
| 7        | 6      | 5  | 4     | 3  | 2  | 1        | 0      |
| RPBBLK   | RPABLK |    | RCBLK |    |    | RESERVED | RMCM   |
| R/W-0h   | R/W-0h |    | R-0h  |    |    | R/W-0h   | R/W-0h |

**Table 15-86. MCR1 Register Field Descriptions**

| Bit   | Field    | Type | Reset | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|-------|----------|------|-------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-10 | RESERVED | R    | 0h    | Reserved                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| 9     | RMCM     | R/W  | 0h    | <p>Receive multichannel partition mode bit.</p> <p>RMCM is only applicable if channels can be individually enabled or disabled for reception (RMCM = 1).</p> <p>RMCM determines whether only 32 channels or all 128 channels are to be individually selectable.</p> <p>Reset type: SYSRSn</p> <p>0h (R/W) = 2-partition mode</p> <p>Only partitions A and B are used. You can control up to 32 channels in the receive multichannel selection mode (RMCM = 1).</p> <p>Assign 16 channels to partition A with the RPABLK bits.</p> <p>Assign 16 channels to partition B with the RPBBLK bits.</p> <p>You control the channels with the appropriate receive channel enable registers:</p> <p>RCERA: Channels in partition A</p> <p>RCERB: Channels in partition B</p> <p>1h (R/W) = 8-partition mode</p> <p>All partitions (A through H) are used. You can control up to 128 channels in the receive multichannel selection mode. You control the channels with the appropriate receive channel enable registers:</p> <p>RCERA: Channels 0 through 15</p> <p>RCERB: Channels 16 through 31</p> <p>RCERC: Channels 32 through 47</p> <p>RCERD: Channels 48 through 63</p> <p>RCERE: Channels 64 through 79</p> <p>RCERF: Channels 80 through 95</p> <p>RCERG: Channels 96 through 111</p> <p>RCERH: Channels 112 through 127</p> |

**Table 15-86. MCR1 Register Field Descriptions (continued)**

| Bit | Field    | Type | Reset | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|-----|----------|------|-------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 8-7 | RPBBLK   | R/W  | 0h    | <p>Receive partition B block bits</p> <p>RPBBLK is only applicable if channels can be individually enabled or disabled (RMCM = 1) and the 2-partition mode is selected (RMCME = 0). Under these conditions, the McBSP receiver can accept or ignore data in any of the 32 channels that are assigned to partitions A and B of the receiver.</p> <p>The 128 receive channels of the McBSP are divided equally among 8 blocks (0 through 7). When RPBBLK is applicable, use RPBBLK to assign one of the odd-numbered blocks (1, 3, 5, or 7) to partition B. Use the RPABLK bits to assign one of the even-numbered blocks (0, 2, 4, or 6) to partition A.</p> <p>If you want to use more than 32 channels, you can change block assignments dynamically. You can assign a new block to one partition while the receiver is handling activity in the other partition. For example, while the block in partition A is active, you can change which block is assigned to partition B. The RCBLK bits are regularly updated to indicate which block is active.</p> <p>When XMCM = 11b (for symmetric transmission and reception), the transmitter uses the receive block bits (RPABLK and RPBBLK) rather than the transmit block bits (XPABLK and XPBBLK).</p> <p>Reset type: SYSRSn</p> <p>0h (R/W) = Block 1: channels 16 through 31<br/> 1h (R/W) = Block 3: channels 48 through 63<br/> 2h (R/W) = Block 5: channels 80 through 95<br/> 3h (R/W) = Block 7: channels 112 through 127</p> |
| 6-5 | RPABLK   | R/W  | 0h    | <p>Receive partition A block bits</p> <p>RPABLK is only applicable if channels can be individually enabled or disabled (RMCM = 1) and the 2-partition mode is selected (RMCME = 0). Under these conditions, the McBSP receiver can accept or ignore data in any of the 32 channels that are assigned to partitions A and B of the receiver. See the RPBBLK (bits 8-7) for more information about assigning blocks to partitions A and B.</p> <p>Reset type: SYSRSn</p> <p>0h (R/W) = Block 0: channels 0 through 15<br/> 1h (R/W) = Block 2: channels 32 through 47<br/> 2h (R/W) = Block 4: channels 64 through 79<br/> 3h (R/W) = Block 6: channels 96 through 111</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| 4-2 | RCBLK    | R    | 0h    | <p>Receive current block indicator.</p> <p>RCBLK indicates which block of 16 channels is involved in the current McBSP reception</p> <p>Reset type: SYSRSn</p> <p>0h (R/W) = Block 0: channels 0 through 15<br/> 1h (R/W) = Block 1: channels 16 through 31<br/> 2h (R/W) = Block 2: channels 32 through 47<br/> 3h (R/W) = Block 3: channels 48 through 63<br/> 4h (R/W) = Block 4: channels 64 through 79<br/> 5h (R/W) = Block 5: channels 80 through 95<br/> 6h (R/W) = Block 6: channels 96 through 111<br/> 7h (R/W) = Block 7: channels 112 through 127</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| 1   | RESERVED | R/W  | 0h    | Reserved                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |

**Table 15-86. MCR1 Register Field Descriptions (continued)**

| Bit | Field | Type | Reset | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
|-----|-------|------|-------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 0   | RMCM  | R/W  | 0h    | <p>Receive multichannel selection mode bit. RMCM determines whether all channels or only selected channels are enabled for reception:<br/>Reset type: SYSRSn</p> <p>0h (R/W) = All 128 channels are enabled.<br/>1h (R/W) = Multichannel selection mode. Channels can be individually enabled or disabled.</p> <p>The only channels enabled are those selected in the appropriate receive channel enable registers (RCERs). The way channels are assigned to the RCERs depends on the number of receive channel partitions (2 or 8), as defined by the RMCME bit.</p> |



### 15.11.15 RCERA Register (Offset = Eh) [reset = 0h]

RCERA is shown in [Figure 15-79](#) and described in [Table 15-87](#).

Return to the [Summary Table](#).

RCERA contains the receive channel enable registers for the A partition. This register is only used when the receiver is configured to allow individual disabling or enabling and masking or unmasking of the channels (for example, RMCM is nonzero).

**Figure 15-79. RCERA Register**

|        |    |    |    |    |    |   |   |
|--------|----|----|----|----|----|---|---|
| 15     | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
| RCEA   |    |    |    |    |    |   |   |
| R/W-0h |    |    |    |    |    |   |   |
| 7      | 6  | 5  | 4  | 3  | 2  | 1 | 0 |
| RCEA   |    |    |    |    |    |   |   |
| R/W-0h |    |    |    |    |    |   |   |

**Table 15-87. RCERA Register Field Descriptions**

| Bit  | Field | Type | Reset | Description                                                                                                                                                                                                                 |
|------|-------|------|-------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-0 | RCEA  | R/W  | 0h    | Receive channel enable bit.<br>For receive multichannel selection mode (RMCM = 1):<br>Reset type: SYSRSn<br>0h (R/W) = Disable the channel that is mapped to RCEX.<br>1h (R/W) = Enable the channel that is mapped to RCEX. |

### 15.11.16 RCERB Register (Offset = Fh) [reset = 0h]

RCERB is shown in [Figure 15-80](#) and described in [Table 15-88](#).

Return to the [Summary Table](#).

RCERB contains the receive channel enable registers for the B partition. This register is only used when the receiver is configured to allow individual disabling or enabling and masking or unmasking of the channels (for example, RMCM is nonzero).

**Figure 15-80. RCERB Register**

|        |    |    |    |    |    |   |   |
|--------|----|----|----|----|----|---|---|
| 15     | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
| RCEB   |    |    |    |    |    |   |   |
| R/W-0h |    |    |    |    |    |   |   |
| 7      | 6  | 5  | 4  | 3  | 2  | 1 | 0 |
| RCEB   |    |    |    |    |    |   |   |
| R/W-0h |    |    |    |    |    |   |   |

**Table 15-88. RCERB Register Field Descriptions**

| Bit  | Field | Type | Reset | Description                                                                                                                                                                                                                 |
|------|-------|------|-------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-0 | RCEB  | R/W  | 0h    | Receive channel enable bit.<br>For receive multichannel selection mode (RMCM = 1):<br>Reset type: SYSRSn<br>0h (R/W) = Disable the channel that is mapped to RCEX.<br>1h (R/W) = Enable the channel that is mapped to RCEX. |

### 15.11.17 XCERA Register (Offset = 10h) [reset = 0h]

XCERA is shown in [Figure 15-81](#) and described in [Table 15-89](#).

Return to the [Summary Table](#).

XCERA contains the transmit channel enable registers for the A partition. This register is only used when the transmitter is configured to allow individual disabling or enabling and masking or unmasking of the channels (for example, XMCM is nonzero).

**Figure 15-81. XCERA Register**

|        |    |    |    |    |    |   |   |
|--------|----|----|----|----|----|---|---|
| 15     | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
| XCERA  |    |    |    |    |    |   |   |
| R/W-0h |    |    |    |    |    |   |   |
| 7      | 6  | 5  | 4  | 3  | 2  | 1 | 0 |
| XCERA  |    |    |    |    |    |   |   |
| R/W-0h |    |    |    |    |    |   |   |

**Table 15-89. XCERA Register Field Descriptions**

| Bit  | Field | Type | Reset | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|------|-------|------|-------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-0 | XCERA | R/W  | 0h    | <p>Transmit channel enable bit.</p> <p>The role of this bit depends on which transmit multichannel selection mode is selected with the XMCM bits.</p> <p>Reset type: SYSRSn</p> <p>0h (R/W) = For multichannel selection when XMCM = 01b<br/>(all channels disabled unless selected):<br/>Disable and mask the channel that is mapped to XCEX.<br/>For multichannel selection when XMCM = 10b<br/>(all channels enabled but masked unless selected):<br/>Mask the channel that is mapped to XCEX.<br/>For multichannel selection when XMCM = 11b<br/>(all channels masked unless selected):<br/>Mask the channel that is mapped to XCEX. Even if the channel is enabled by the corresponding receive channel enable bit, this channel's data cannot appear on the DX pin.</p> <p>1h (R/W) = For multichannel selection when XMCM = 01b<br/>(all channels disabled unless selected):<br/>Enable and unmask the channel that is mapped to XCEX.<br/>For multichannel selection when XMCM = 10b<br/>(all channels enabled but masked unless selected):<br/>Unmask the channel that is mapped to XCEX.<br/>For multichannel selection when XMCM = 11b<br/>(all channels masked unless selected):<br/>Unmask the channel that is mapped to XCEX. If the channel is also enabled by the corresponding receive channel enable bit, full transmission can occur.</p> |

### 15.11.18 XCERB Register (Offset = 11h) [reset = 0h]

XCERB is shown in [Figure 15-82](#) and described in [Table 15-90](#).

Return to the [Summary Table](#).

XCERB contains the transmit channel enable registers for the B partition. This register is only used when the transmitter is configured to allow individual disabling or enabling and masking or unmasking of the channels (for example, XMCM is nonzero).

**Figure 15-82. XCERB Register**

|        |    |    |    |    |    |   |   |
|--------|----|----|----|----|----|---|---|
| 15     | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
| XCERB  |    |    |    |    |    |   |   |
| R/W-0h |    |    |    |    |    |   |   |
| 7      | 6  | 5  | 4  | 3  | 2  | 1 | 0 |
| XCERB  |    |    |    |    |    |   |   |
| R/W-0h |    |    |    |    |    |   |   |

**Table 15-90. XCERB Register Field Descriptions**

| Bit  | Field | Type | Reset | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|------|-------|------|-------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-0 | XCERB | R/W  | 0h    | <p>Transmit channel enable bit.</p> <p>The role of this bit depends on which transmit multichannel selection mode is selected with the XMCM bits.</p> <p>Reset type: SYSRSn</p> <p>0h (R/W) = For multichannel selection when XMCM = 01b<br/>(all channels disabled unless selected):<br/>Disable and mask the channel that is mapped to XCEX.<br/>For multichannel selection when XMCM = 10b<br/>(all channels enabled but masked unless selected):<br/>Mask the channel that is mapped to XCEX.<br/>For multichannel selection when XMCM = 11b<br/>(all channels masked unless selected):<br/>Mask the channel that is mapped to XCEX. Even if the channel is enabled by the corresponding receive channel enable bit, this channel's data cannot appear on the DX pin.</p> <p>1h (R/W) = For multichannel selection when XMCM = 01b<br/>(all channels disabled unless selected):<br/>Enable and unmask the channel that is mapped to XCEX.<br/>For multichannel selection when XMCM = 10b<br/>(all channels enabled but masked unless selected):<br/>Unmask the channel that is mapped to XCEX.<br/>For multichannel selection when XMCM = 11b<br/>(all channels masked unless selected):<br/>Unmask the channel that is mapped to XCEX. If the channel is also enabled by the corresponding receive channel enable bit, full transmission can occur.</p> |

### 15.11.19 PCR Register (Offset = 12h) [reset = 0h]

PCR is shown in [Figure 15-83](#) and described in [Table 15-91](#).

Return to the [Summary Table](#).

PCR contains control bits for the McBSP pins such as frame synchronization modes, clock modes, input clock source selection for the sample rate generator, frame-synchronization pulse active polarity, and transmit/receive active edge selection.

**Figure 15-83. PCR Register**

| 15       | 14       | 13 | 12 | 11     | 10     | 9      | 8      |
|----------|----------|----|----|--------|--------|--------|--------|
| RESERVED |          |    |    | FSXM   | FSRM   | CLKXM  | CLKRM  |
| R-0h     |          |    |    | R/W-0h | R/W-0h | R/W-0h | R/W-0h |
| 7        | 6        | 5  | 4  | 3      | 2      | 1      | 0      |
| SCLKME   | RESERVED |    |    | FSXP   | FSRP   | CLKXP  | CLKRP  |
| R/W-0h   | R-0h     |    |    | R/W-0h | R-0h   | R-0h   | R/W-0h |

**Table 15-91. PCR Register Field Descriptions**

| Bit   | Field    | Type | Reset | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|-------|----------|------|-------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-12 | RESERVED | R    | 0h    | Reserved                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| 11    | FSXM     | R/W  | 0h    | Transmit frame-synchronization mode bit.<br>FSXM determines whether transmit frame-synchronization pulses are supplied externally or internally. The polarity of the signal on the FSX pin is determined by the FSXP bit.<br>Reset type: SYSRSn<br>0h (R/W) = Transmit frame synchronization is supplied by an external source via the FSX pin.<br>1h (R/W) = Transmit frame synchronization is generated internally by the Sample Rate generator, as determined by the FSGM bit of SRGR2                       |
| 10    | FSRM     | R/W  | 0h    | Receive frame-synchronization mode bit.<br>FSRM determines whether receive frame-synchronization pulses are supplied externally or internally. The polarity of the signal on the FSR pin is determined by the FSRP bit.<br>Reset type: SYSRSn<br>0h (R/W) = Receive frame synchronization is supplied by an external source via the FSR pin.<br>1h (R/W) = Receive frame synchronization is supplied by the sample rate generator. FSR is an output pin reflecting internal FSR, except when GSYNC = 1 in SRGR2 |

**Table 15-91. PCR Register Field Descriptions (continued)**

| Bit | Field | Type | Reset | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|-----|-------|------|-------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 9   | CLKXM | R/W  | 0h    | <p>Transmit clock mode bit.</p> <p>CLKXM determines whether the source for the transmit clock is external or internal, and whether the MCLKX pin is an input or an output. The polarity of the signal on the MCLKX pin is determined by the CLKXP bit.</p> <p>In the clock stop mode (CLKSTP = 10b or 11b), the McBSP can act as a master or as a slave in the SPI protocol. If the McBSP is a master, make sure that CLKX is an output. If the McBSP is a slave, make sure that CLKX is an input.</p> <p>Reset type: SYSRSn</p> <p>0h (R/W) = Not in clock stop mode (CLKSTP = 00b or 01b):</p> <p>The transmitter gets its clock signal from an external source via the MCLKX pin.</p> <p>In clock stop mode (CLKSTP = 10b or 11b):</p> <p>The McBSP is a slave in the SPI protocol. The internal transmit clock (CLKX) is driven by the SPI master via the MCLKX pin. The internal receive clock (MCLKR) is driven internally by CLKX, so that both the transmitter and the receiver are controlled by the external master clock.</p> <p>1h (R/W) = Not in clock stop mode (CLKSTP = 00b or 01b):</p> <p>Internal CLKX is driven by the sample rate generator of the McBSP. The MCLKX pin is an output pin that reflects internal CLKX.</p> <p>In clock stop mode (CLKSTP = 10b or 11b):</p> <p>The McBSP is a master in the SPI protocol. The sample rate generator drives the internal transmit clock (CLKX). Internal CLKX is reflected on the MCLKX pin to drive the shift clock of the SPI-compliant slaves in the system. Internal CLKX also drives the internal receive clock (MCLKR), so that both the transmitter and the receiver are controlled by the internal master clock</p> |
| 8   | CLKRM | R/W  | 0h    | <p>Receive clock mode bit.</p> <p>The role of CLKRM and the resulting effect on the MCLKR pin depend on whether the McBSP is in the digital loopback mode (DLB = 1).</p> <p>The polarity of the signal on the MCLKR pin is determined by the CLKRP bit.</p> <p>Reset type: SYSRSn</p> <p>0h (R/W) = Not in digital loopback mode (DLB = 0):</p> <p>The MCLKR pin is an input pin that supplies the internal receive clock (MCLKR).</p> <p>In digital loopback mode (DLB = 1):</p> <p>The MCLKR pin is in the high impedance state. The internal receive clock (MCLKR) is driven by the internal transmit clock (CLKX). CLKX is derived according to the CLKXM bit.</p> <p>1h (R/W) = Not in digital loopback mode (DLB = 0):</p> <p>Internal MCLKR is driven by the sample rate generator of the McBSP. The MCLKR pin is an output pin that reflects internal MCLKR.</p> <p>In digital loopback mode (DLB = 1):</p> <p>Internal MCLKR is driven by internal CLKX. The MCLKR pin is an output pin that reflects internal MCLKR. CLKX is derived according to the CLKXM bit.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |

**Table 15-91. PCR Register Field Descriptions (continued)**

| Bit | Field    | Type | Reset | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|-----|----------|------|-------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 7   | SCLKME   | R/W  | 0h    | <p>Sample rate generator input clock mode bit. The sample rate generator can produce a clock signal, CLKG. The frequency of CLKG is:</p> $\text{CLKG freq.} = (\text{Input clock frequency}) / (\text{CLKGDV} + 1)$ <p>SCLKME is used in conjunction with the CLKSM bit to select the input clock.</p> <p>SCLKME CLKSM Input Clock For<br/>Sample Rate Generator</p> <p>0 0 Reserved<br/>0 1 LSPCLK</p> <p>The input clock for the sample rate generator is taken from the MCLKR pin or from the MCLKX pin, depending on the value of the CLKSM bit of SRGR2:</p> <p>SCLKME CLKSM Input Clock For<br/>Sample Rate Generator</p> <p>1 0 Signal on MCLKR pin<br/>1 1 Signal on MCLKX pin</p> <p>Reset type: SYSRSn</p> |
| 6-4 | RESERVED | R    | 0h    | Reserved                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| 3   | FSXP     | R/W  | 0h    | <p>Transmit frame-synchronization polarity bit. FSXP determines the polarity of FSX as seen on the FSX pin.</p> <p>Reset type: SYSRSn</p> <p>0h (R/W) = Transmit frame-synchronization pulses are active high.<br/>1h (R/W) = Transmit frame-synchronization pulses are active low.</p>                                                                                                                                                                                                                                                                                                                                                                                                                              |
| 2   | FSRP     | R    | 0h    | <p>Receive frame-synchronization polarity bit. FSRP determines the polarity of FSR as seen on the FSR pin.</p> <p>Reset type: SYSRSn</p> <p>0h (R/W) = Receive frame-synchronization pulses are active high.<br/>1h (R/W) = Receive frame-synchronization pulses are active low.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| 1   | CLKXP    | R    | 0h    | <p>Transmit clock polarity bit. CLKXP determines the polarity of CLKX as seen on the MCLKX pin.</p> <p>Reset type: SYSRSn</p> <p>0h (R/W) = Transmit data is sampled on the rising edge of CLKX.<br/>1h (R/W) = Transmit data is sampled on the falling edge of CLKX.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| 0   | CLKRP    | R/W  | 0h    | <p>Receive clock polarity bit. CLKRP determines the polarity of CLKR as seen on the MCLKR pin.</p> <p>Reset type: SYSRSn</p> <p>0h (R/W) = Receive data is sampled on the falling edge of MCLKR.<br/>1h (R/W) = Receive data is sampled on the rising edge of MCLKR.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                             |

### 15.11.20 RCERC Register (Offset = 13h) [reset = 0h]

RCERC is shown in [Figure 15-84](#) and described in [Table 15-92](#).

Return to the [Summary Table](#).

RCERC contains the receive channel enable registers for the C partition. This register is only used when the receiver is configured to allow individual disabling or enabling and masking or unmasking of the channels (for example, RMCM is nonzero).

**Figure 15-84. RCERC Register**

|        |    |    |    |    |    |   |   |
|--------|----|----|----|----|----|---|---|
| 15     | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
| RCEC   |    |    |    |    |    |   |   |
| R/W-0h |    |    |    |    |    |   |   |
| 7      | 6  | 5  | 4  | 3  | 2  | 1 | 0 |
| RCEC   |    |    |    |    |    |   |   |
| R/W-0h |    |    |    |    |    |   |   |

**Table 15-92. RCERC Register Field Descriptions**

| Bit  | Field | Type | Reset | Description                                                                                                                                                                                                                 |
|------|-------|------|-------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-0 | RCEC  | R/W  | 0h    | Receive channel enable bit.<br>For receive multichannel selection mode (RMCM = 1):<br>Reset type: SYSRSn<br>0h (R/W) = Disable the channel that is mapped to RCEX.<br>1h (R/W) = Enable the channel that is mapped to RCEX. |

### 15.11.21 RCERD Register (Offset = 14h) [reset = 0h]

RCERD is shown in [Figure 15-85](#) and described in [Table 15-93](#).

Return to the [Summary Table](#).

RCERD contains the receive channel enable registers for the D partition. This register is only used when the receiver is configured to allow individual disabling or enabling and masking or unmasking of the channels (for example, RMCM is nonzero).

**Figure 15-85. RCERD Register**

|        |    |    |    |    |    |   |   |
|--------|----|----|----|----|----|---|---|
| 15     | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
| RCED   |    |    |    |    |    |   |   |
| R/W-0h |    |    |    |    |    |   |   |
| 7      | 6  | 5  | 4  | 3  | 2  | 1 | 0 |
| RCED   |    |    |    |    |    |   |   |
| R/W-0h |    |    |    |    |    |   |   |

**Table 15-93. RCERD Register Field Descriptions**

| Bit  | Field | Type | Reset | Description                                                                                                                                                                                                                 |
|------|-------|------|-------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-0 | RCED  | R/W  | 0h    | Receive channel enable bit.<br>For receive multichannel selection mode (RMCM = 1):<br>Reset type: SYSRSn<br>0h (R/W) = Disable the channel that is mapped to RCEX.<br>1h (R/W) = Enable the channel that is mapped to RCEX. |

### 15.11.22 XCERC Register (Offset = 15h) [reset = 0h]

XCERC is shown in [Figure 15-86](#) and described in [Table 15-94](#).

Return to the [Summary Table](#).

XCERC contains the transmit channel enable registers for the C partition. This register is only used when the transmitter is configured to allow individual disabling or enabling and masking or unmasking of the channels (for example, XMCM is nonzero).

**Figure 15-86. XCERC Register**

|        |    |    |    |    |    |   |   |
|--------|----|----|----|----|----|---|---|
| 15     | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
| XCERC  |    |    |    |    |    |   |   |
| R/W-0h |    |    |    |    |    |   |   |
| 7      | 6  | 5  | 4  | 3  | 2  | 1 | 0 |
| XCERC  |    |    |    |    |    |   |   |
| R/W-0h |    |    |    |    |    |   |   |

**Table 15-94. XCERC Register Field Descriptions**

| Bit  | Field | Type | Reset | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|------|-------|------|-------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-0 | XCERC | R/W  | 0h    | <p>Transmit channel enable bit.</p> <p>The role of this bit depends on which transmit multichannel selection mode is selected with the XMCM bits.</p> <p>Reset type: SYSRSn</p> <p>0h (R/W) = For multichannel selection when XMCM = 01b<br/>(all channels disabled unless selected):<br/>Disable and mask the channel that is mapped to XCEX.<br/>For multichannel selection when XMCM = 10b<br/>(all channels enabled but masked unless selected):<br/>Mask the channel that is mapped to XCEX.<br/>For multichannel selection when XMCM = 11b<br/>(all channels masked unless selected):<br/>Mask the channel that is mapped to XCEX. Even if the channel is enabled by the corresponding receive channel enable bit, this channel's data cannot appear on the DX pin.</p> <p>1h (R/W) = For multichannel selection when XMCM = 01b<br/>(all channels disabled unless selected):<br/>Enable and unmask the channel that is mapped to XCEX.<br/>For multichannel selection when XMCM = 10b<br/>(all channels enabled but masked unless selected):<br/>Unmask the channel that is mapped to XCEX.<br/>For multichannel selection when XMCM = 11b<br/>(all channels masked unless selected):<br/>Unmask the channel that is mapped to XCEX. If the channel is also enabled by the corresponding receive channel enable bit, full transmission can occur.</p> |



### 15.11.23 XCERD Register (Offset = 16h) [reset = 0h]

XCERD is shown in [Figure 15-87](#) and described in [Table 15-95](#).

Return to the [Summary Table](#).

XCERD contains the transmit channel enable registers for the D partition. This register is only used when the transmitter is configured to allow individual disabling or enabling and masking or unmasking of the channels (for example, XMCM is nonzero).

**Figure 15-87. XCERD Register**

|        |    |    |    |    |    |   |   |
|--------|----|----|----|----|----|---|---|
| 15     | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
| XCERD  |    |    |    |    |    |   |   |
| R/W-0h |    |    |    |    |    |   |   |
| 7      | 6  | 5  | 4  | 3  | 2  | 1 | 0 |
| XCERD  |    |    |    |    |    |   |   |
| R/W-0h |    |    |    |    |    |   |   |

**Table 15-95. XCERD Register Field Descriptions**

| Bit  | Field | Type | Reset | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|------|-------|------|-------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-0 | XCERD | R/W  | 0h    | <p>Transmit channel enable bit.</p> <p>The role of this bit depends on which transmit multichannel selection mode is selected with the XMCM bits.</p> <p>Reset type: SYSRSn</p> <p>0h (R/W) = For multichannel selection when XMCM = 01b<br/>(all channels disabled unless selected):<br/>Disable and mask the channel that is mapped to XCEX.<br/>For multichannel selection when XMCM = 10b<br/>(all channels enabled but masked unless selected):<br/>Mask the channel that is mapped to XCEX.<br/>For multichannel selection when XMCM = 11b<br/>(all channels masked unless selected):<br/>Mask the channel that is mapped to XCEX. Even if the channel is enabled by the corresponding receive channel enable bit, this channel's data cannot appear on the DX pin.</p> <p>1h (R/W) = For multichannel selection when XMCM = 01b<br/>(all channels disabled unless selected):<br/>Enable and unmask the channel that is mapped to XCEX.<br/>For multichannel selection when XMCM = 10b<br/>(all channels enabled but masked unless selected):<br/>Unmask the channel that is mapped to XCEX.<br/>For multichannel selection when XMCM = 11b<br/>(all channels masked unless selected):<br/>Unmask the channel that is mapped to XCEX. If the channel is also enabled by the corresponding receive channel enable bit, full transmission can occur.</p> |

### 15.11.24 RCERE Register (Offset = 17h) [reset = 0h]

RCERE is shown in [Figure 15-88](#) and described in [Table 15-96](#).

Return to the [Summary Table](#).

RCERE contains the receive channel enable registers for the E partition. This register is only used when the receiver is configured to allow individual disabling or enabling and masking or unmasking of the channels (for example, RMCM is nonzero).

**Figure 15-88. RCERE Register**

|        |    |    |    |    |    |   |   |
|--------|----|----|----|----|----|---|---|
| 15     | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
| RCEE   |    |    |    |    |    |   |   |
| R/W-0h |    |    |    |    |    |   |   |
| 7      | 6  | 5  | 4  | 3  | 2  | 1 | 0 |
| RCEE   |    |    |    |    |    |   |   |
| R/W-0h |    |    |    |    |    |   |   |

**Table 15-96. RCERE Register Field Descriptions**

| Bit  | Field | Type | Reset | Description                                                                                                                                                                                                                 |
|------|-------|------|-------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-0 | RCEE  | R/W  | 0h    | Receive channel enable bit.<br>For receive multichannel selection mode (RMCM = 1):<br>Reset type: SYSRSn<br>0h (R/W) = Disable the channel that is mapped to RCEX.<br>1h (R/W) = Enable the channel that is mapped to RCEX. |

### 15.11.25 RCERF Register (Offset = 18h) [reset = 0h]

RCERF is shown in [Figure 15-89](#) and described in [Table 15-97](#).

Return to the [Summary Table](#).

RCERF contains the receive channel enable registers for the F partition. This register is only used when the receiver is configured to allow individual disabling or enabling and masking or unmasking of the channels (for example, RMCM is nonzero).

**Figure 15-89. RCERF Register**

|        |    |    |    |    |    |   |   |
|--------|----|----|----|----|----|---|---|
| 15     | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
| RCEF   |    |    |    |    |    |   |   |
| R/W-0h |    |    |    |    |    |   |   |
| 7      | 6  | 5  | 4  | 3  | 2  | 1 | 0 |
| RCEF   |    |    |    |    |    |   |   |
| R/W-0h |    |    |    |    |    |   |   |

**Table 15-97. RCERF Register Field Descriptions**

| Bit  | Field | Type | Reset | Description                                                                                                                                                                                                                 |
|------|-------|------|-------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-0 | RCEF  | R/W  | 0h    | Receive channel enable bit.<br>For receive multichannel selection mode (RMCM = 1):<br>Reset type: SYSRSn<br>0h (R/W) = Disable the channel that is mapped to RCEX.<br>1h (R/W) = Enable the channel that is mapped to RCEX. |

### 15.11.26 XCERE Register (Offset = 19h) [reset = 0h]

XCERE is shown in [Figure 15-90](#) and described in [Table 15-98](#).

Return to the [Summary Table](#).

XCERE contains the transmit channel enable registers for the E partition. This register is only used when the transmitter is configured to allow individual disabling or enabling and masking or unmasking of the channels (for example, XMCM is nonzero).

**Figure 15-90. XCERE Register**

|        |    |    |    |    |    |   |   |
|--------|----|----|----|----|----|---|---|
| 15     | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
| XCERE  |    |    |    |    |    |   |   |
| R/W-0h |    |    |    |    |    |   |   |
| 7      | 6  | 5  | 4  | 3  | 2  | 1 | 0 |
| XCERE  |    |    |    |    |    |   |   |
| R/W-0h |    |    |    |    |    |   |   |

**Table 15-98. XCERE Register Field Descriptions**

| Bit  | Field | Type | Reset | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|------|-------|------|-------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-0 | XCERE | R/W  | 0h    | <p>Transmit channel enable bit.</p> <p>The role of this bit depends on which transmit multichannel selection mode is selected with the XMCM bits.</p> <p>Reset type: SYSRSn</p> <p>0h (R/W) = For multichannel selection when XMCM = 01b<br/>(all channels disabled unless selected):<br/>Disable and mask the channel that is mapped to XCEx.<br/>For multichannel selection when XMCM = 10b<br/>(all channels enabled but masked unless selected):<br/>Mask the channel that is mapped to XCEx.<br/>For multichannel selection when XMCM = 11b<br/>(all channels masked unless selected):<br/>Mask the channel that is mapped to XCEx. Even if the channel is enabled by the corresponding receive channel enable bit, this channel's data cannot appear on the DX pin.</p> <p>1h (R/W) = For multichannel selection when XMCM = 01b<br/>(all channels disabled unless selected):<br/>Enable and unmask the channel that is mapped to XCEx.<br/>For multichannel selection when XMCM = 10b<br/>(all channels enabled but masked unless selected):<br/>Unmask the channel that is mapped to XCEx.<br/>For multichannel selection when XMCM = 11b<br/>(all channels masked unless selected):<br/>Unmask the channel that is mapped to XCEx. If the channel is also enabled by the corresponding receive channel enable bit, full transmission can occur.</p> |

### 15.11.27 XCERF Register (Offset = 1Ah) [reset = 0h]

XCERF is shown in [Figure 15-91](#) and described in [Table 15-99](#).

Return to the [Summary Table](#).

XCERF contains the transmit channel enable registers for the F partition. This register is only used when the transmitter is configured to allow individual disabling or enabling and masking or unmasking of the channels (for example, XMCM is nonzero).

**Figure 15-91. XCERF Register**

|        |    |    |    |    |    |   |   |
|--------|----|----|----|----|----|---|---|
| 15     | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
| XCERF  |    |    |    |    |    |   |   |
| R/W-0h |    |    |    |    |    |   |   |
| 7      | 6  | 5  | 4  | 3  | 2  | 1 | 0 |
| XCERF  |    |    |    |    |    |   |   |
| R/W-0h |    |    |    |    |    |   |   |

**Table 15-99. XCERF Register Field Descriptions**

| Bit  | Field | Type | Reset | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|------|-------|------|-------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-0 | XCERF | R/W  | 0h    | <p>Transmit channel enable bit.</p> <p>The role of this bit depends on which transmit multichannel selection mode is selected with the XMCM bits.</p> <p>Reset type: SYSRSn</p> <p>0h (R/W) = For multichannel selection when XMCM = 01b<br/>(all channels disabled unless selected):<br/>Disable and mask the channel that is mapped to XCEX.<br/>For multichannel selection when XMCM = 10b<br/>(all channels enabled but masked unless selected):<br/>Mask the channel that is mapped to XCEX.<br/>For multichannel selection when XMCM = 11b<br/>(all channels masked unless selected):<br/>Mask the channel that is mapped to XCEX. Even if the channel is enabled by the corresponding receive channel enable bit, this channel's data cannot appear on the DX pin.</p> <p>1h (R/W) = For multichannel selection when XMCM = 01b<br/>(all channels disabled unless selected):<br/>Enable and unmask the channel that is mapped to XCEX.<br/>For multichannel selection when XMCM = 10b<br/>(all channels enabled but masked unless selected):<br/>Unmask the channel that is mapped to XCEX.<br/>For multichannel selection when XMCM = 11b<br/>(all channels masked unless selected):<br/>Unmask the channel that is mapped to XCEX. If the channel is also enabled by the corresponding receive channel enable bit, full transmission can occur.</p> |

### 15.11.28 RCERG Register (Offset = 1Bh) [reset = 0h]

RCERG is shown in [Figure 15-92](#) and described in [Table 15-100](#).

Return to the [Summary Table](#).

RCERG contains the receive channel enable registers for the G partition. This register is only used when the receiver is configured to allow individual disabling or enabling and masking or unmasking of the channels (for example, RMCM is nonzero).

**Figure 15-92. RCERG Register**

|        |    |    |    |    |    |   |   |
|--------|----|----|----|----|----|---|---|
| 15     | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
| RCEG   |    |    |    |    |    |   |   |
| R/W-0h |    |    |    |    |    |   |   |
| 7      | 6  | 5  | 4  | 3  | 2  | 1 | 0 |
| RCEG   |    |    |    |    |    |   |   |
| R/W-0h |    |    |    |    |    |   |   |

**Table 15-100. RCERG Register Field Descriptions**

| Bit  | Field | Type | Reset | Description                                                                                                                                                                                                                 |
|------|-------|------|-------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-0 | RCEG  | R/W  | 0h    | Receive channel enable bit.<br>For receive multichannel selection mode (RMCM = 1):<br>Reset type: SYSRSn<br>0h (R/W) = Disable the channel that is mapped to RCEX.<br>1h (R/W) = Enable the channel that is mapped to RCEX. |

### 15.11.29 RCERH Register (Offset = 1Ch) [reset = 0h]

RCERH is shown in [Figure 15-93](#) and described in [Table 15-101](#).

Return to the [Summary Table](#).

RCERH contains the receive channel enable registers for the H partition. This register is only used when the receiver is configured to allow individual disabling or enabling and masking or unmasking of the channels (for example, RMCM is nonzero).

**Figure 15-93. RCERH Register**

|        |    |    |    |    |    |   |   |
|--------|----|----|----|----|----|---|---|
| 15     | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
| RCEH   |    |    |    |    |    |   |   |
| R/W-0h |    |    |    |    |    |   |   |
| 7      | 6  | 5  | 4  | 3  | 2  | 1 | 0 |
| RCEH   |    |    |    |    |    |   |   |
| R/W-0h |    |    |    |    |    |   |   |

**Table 15-101. RCERH Register Field Descriptions**

| Bit  | Field | Type | Reset | Description                                                                                                                                                                                                                 |
|------|-------|------|-------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-0 | RCEH  | R/W  | 0h    | Receive channel enable bit.<br>For receive multichannel selection mode (RMCM = 1):<br>Reset type: SYSRSn<br>0h (R/W) = Disable the channel that is mapped to RCEX.<br>1h (R/W) = Enable the channel that is mapped to RCEX. |

### 15.11.30 XCERG Register (Offset = 1Dh) [reset = 0h]

XCERG is shown in [Figure 15-94](#) and described in [Table 15-102](#).

Return to the [Summary Table](#).

XCERG contains the transmit channel enable registers for the G partition. This register is only used when the transmitter is configured to allow individual disabling or enabling and masking or unmasking of the channels (for example, XMCM is nonzero).

**Figure 15-94. XCERG Register**

|        |    |    |    |    |    |   |   |
|--------|----|----|----|----|----|---|---|
| 15     | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
| XCERG  |    |    |    |    |    |   |   |
| R/W-0h |    |    |    |    |    |   |   |
| 7      | 6  | 5  | 4  | 3  | 2  | 1 | 0 |
| XCERG  |    |    |    |    |    |   |   |
| R/W-0h |    |    |    |    |    |   |   |

**Table 15-102. XCERG Register Field Descriptions**

| Bit  | Field | Type | Reset | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|------|-------|------|-------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-0 | XCERG | R/W  | 0h    | <p>Transmit channel enable bit.</p> <p>The role of this bit depends on which transmit multichannel selection mode is selected with the XMCM bits.</p> <p>Reset type: SYSRSn</p> <p>0h (R/W) = For multichannel selection when XMCM = 01b<br/>(all channels disabled unless selected):<br/>Disable and mask the channel that is mapped to XCEX.<br/>For multichannel selection when XMCM = 10b<br/>(all channels enabled but masked unless selected):<br/>Mask the channel that is mapped to XCEX.<br/>For multichannel selection when XMCM = 11b<br/>(all channels masked unless selected):<br/>Mask the channel that is mapped to XCEX. Even if the channel is enabled by the corresponding receive channel enable bit, this channel's data cannot appear on the DX pin.</p> <p>1h (R/W) = For multichannel selection when XMCM = 01b<br/>(all channels disabled unless selected):<br/>Enable and unmask the channel that is mapped to XCEX.<br/>For multichannel selection when XMCM = 10b<br/>(all channels enabled but masked unless selected):<br/>Unmask the channel that is mapped to XCEX.<br/>For multichannel selection when XMCM = 11b<br/>(all channels masked unless selected):<br/>Unmask the channel that is mapped to XCEX. If the channel is also enabled by the corresponding receive channel enable bit, full transmission can occur.</p> |

### 15.11.31 XCERH Register (Offset = 1Eh) [reset = 0h]

XCERH is shown in [Figure 15-95](#) and described in [Table 15-103](#).

Return to the [Summary Table](#).

XCERH contains the transmit channel enable registers for the H partition. This register is only used when the transmitter is configured to allow individual disabling or enabling and masking or unmasking of the channels (for example, XMCM is nonzero).

**Figure 15-95. XCERH Register**

|        |    |    |    |    |    |   |   |
|--------|----|----|----|----|----|---|---|
| 15     | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
| XCERH  |    |    |    |    |    |   |   |
| R/W-0h |    |    |    |    |    |   |   |
| 7      | 6  | 5  | 4  | 3  | 2  | 1 | 0 |
| XCERH  |    |    |    |    |    |   |   |
| R/W-0h |    |    |    |    |    |   |   |

**Table 15-103. XCERH Register Field Descriptions**

| Bit  | Field | Type | Reset | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|------|-------|------|-------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-0 | XCERH | R/W  | 0h    | <p>Transmit channel enable bit.</p> <p>The role of this bit depends on which transmit multichannel selection mode is selected with the XMCM bits.</p> <p>Reset type: SYSRSn</p> <p>0h (R/W) = For multichannel selection when XMCM = 01b<br/>(all channels disabled unless selected):<br/>Disable and mask the channel that is mapped to XCEX.<br/>For multichannel selection when XMCM = 10b<br/>(all channels enabled but masked unless selected):<br/>Mask the channel that is mapped to XCEX.<br/>For multichannel selection when XMCM = 11b<br/>(all channels masked unless selected):<br/>Mask the channel that is mapped to XCEX. Even if the channel is enabled by the corresponding receive channel enable bit, this channel's data cannot appear on the DX pin.</p> <p>1h (R/W) = For multichannel selection when XMCM = 01b<br/>(all channels disabled unless selected):<br/>Enable and unmask the channel that is mapped to XCEX.<br/>For multichannel selection when XMCM = 10b<br/>(all channels enabled but masked unless selected):<br/>Unmask the channel that is mapped to XCEX.<br/>For multichannel selection when XMCM = 11b<br/>(all channels masked unless selected):<br/>Unmask the channel that is mapped to XCEX. If the channel is also enabled by the corresponding receive channel enable bit, full transmission can occur.</p> |

### 15.11.32 MFFINT Register (Offset = 23h) [reset = 0h]

MFFINT is shown in [Figure 15-96](#) and described in [Table 15-104](#).

Return to the [Summary Table](#).

MFFINT contains the enable bits for both the transmitter and receiver interrupts.

**Figure 15-96. MFFINT Register**

|          |    |    |    |    |        |          |        |
|----------|----|----|----|----|--------|----------|--------|
| 15       | 14 | 13 | 12 | 11 | 10     | 9        | 8      |
| RESERVED |    |    |    |    |        |          |        |
| R-0h     |    |    |    |    |        |          |        |
| 7        | 6  | 5  | 4  | 3  | 2      | 1        | 0      |
| RESERVED |    |    |    |    | RINT   | RESERVED | XINT   |
| R-0h     |    |    |    |    | R/W-0h | R-0h     | R/W-0h |

**Table 15-104. MFFINT Register Field Descriptions**

| Bit  | Field    | Type | Reset | Description                                                                                                                                                    |
|------|----------|------|-------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-3 | RESERVED | R    | 0h    | Reserved                                                                                                                                                       |
| 2    | RINT     | R/W  | 0h    | Enable for transmit Interrupt<br>Reset type: SYSRSn<br>0h (R/W) = Transmit interrupt on XRDY is disabled.<br>1h (R/W) = Transmit interrupt on XRDY is enabled. |
| 1    | RESERVED | R    | 0h    | Reserved                                                                                                                                                       |
| 0    | XINT     | R/W  | 0h    | Enable for Receive Interrupt<br>Reset type: SYSRSn<br>0h (R/W) = Receive interrupt on RRDY is disabled.<br>1h (R/W) = Receive interrupt on RRDY is enabled.    |



The enhanced Controller Area Network (eCAN) module is a full-CAN controller and is compatible with the CAN 2.0B standard (active). It uses established protocol to communicate serially with other controllers in electrically noisy environments. With 32 fully configurable mailboxes and a time-stamping feature, the eCAN module provides a versatile and robust serial communication interface. Refer to the [C2000 Real-Time Control Peripheral Reference Guide](#) for a list of devices with the eCAN module. Some devices have a second CAN module, eCAN-B. The word eCAN is generically used to refer to the CAN modules. The specific module reference (A or B) is used where appropriate. For a given eCAN module, the same address space is used for the module registers in all applicable 28xx /28xxx devices. Refer to [Programming Examples for the TMS320x28xx eCAN](#) that provides many useful examples that illustrate how to program the eCAN module.

|                                                               |             |
|---------------------------------------------------------------|-------------|
| <b>16.1 Introduction.....</b>                                 | <b>1000</b> |
| <b>16.2 eCAN Compatibility With Other TI CAN Modules.....</b> | <b>1001</b> |
| <b>16.3 CAN Network and Module.....</b>                       | <b>1002</b> |
| <b>16.4 eCAN Controller Overview.....</b>                     | <b>1004</b> |
| <b>16.5 Message Objects.....</b>                              | <b>1007</b> |
| <b>16.6 Message Mailbox.....</b>                              | <b>1007</b> |
| <b>16.7 eCAN Configuration.....</b>                           | <b>1010</b> |
| <b>16.8 Acceptance Filter.....</b>                            | <b>1023</b> |
| <b>16.9 eCAN Registers.....</b>                               | <b>1025</b> |
| <b>16.10 Message Data Registers (CANMDL, CANMDH).....</b>     | <b>1053</b> |

## 16.1 Introduction

### 16.1.1 Features

The eCAN module has the following features:

- Fully compliant with CAN protocol, version 2.0B
- Supports data rates up to 1 Mbps
- Thirty-two mailboxes, each with the following properties:
  - Configurable as receive or transmit
  - Configurable with standard or extended identifier
  - Has a programmable acceptance filter mask
  - Supports data and remote frame
  - Supports 0 to 8 bytes of data
  - Uses a 32-bit time stamp on received and transmitted message
  - Protects against reception of new message
  - Allows dynamically programmable priority of transmit message
  - Employs a programmable interrupt scheme with two interrupt levels
  - Employs a programmable interrupt on transmission or reception time-out
- Low-power mode
- Programmable wake-up on bus activity
- Automatic reply to a remote request message
- Automatic retransmission of a frame in case of loss of arbitration or error
- 32-bit time-stamp counter synchronized by a specific message (communication in conjunction with mailbox 16)
- Self-test mode
  - Operates in a loopback mode receiving its own message. A self-generated acknowledge is provided, thereby eliminating the need for another node to provide the acknowledge bit.

### 16.1.2 ECAN Related Collateral

#### Foundational Materials

- [Automotive CAN Overview and Training](#) (Video)
- [CAN Physical layer](#) (Video)
- [CAN and CAN FD Overview](#) (Video)

#### Getting Started Materials

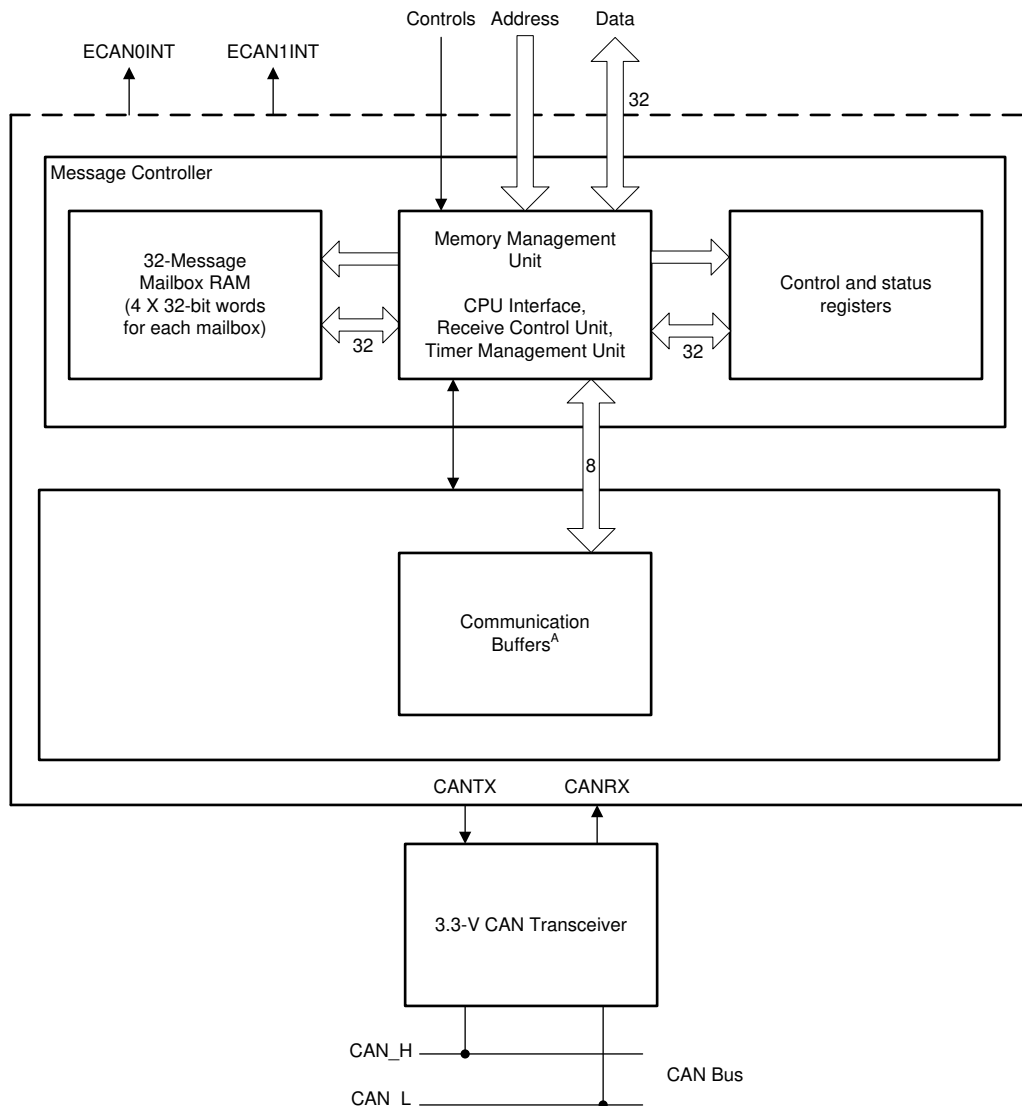
- [MCU CAN Module Operation Using the On-Chip Zero-Pin Oscillator Application Report](#)
- [Programming Examples for the TMS320x28xx eCAN Application Report](#)
- [Using the CAN Bootloader at High Temperature Application Report](#)

#### Expert Materials

- [Calculator for CAN Bit Timing Parameters Application Report](#)

### 16.1.3 Block Diagram

Figure 16-1 shows the block diagram of eCAN.



A. The communication buffers are transparent to the user and are not accessible by user code.

**Figure 16-1. eCAN Block Diagram and Interface Circuit**

### 16.2 eCAN Compatibility With Other TI CAN Modules

The eCAN module is identical to the High-End CAN Controller (HECC) used in the TMS470 series of microcontrollers from Texas Instruments with some minor changes. The eCAN module features several enhancements (such as increased number of mailboxes with individual acceptance masks, time stamping, and so on) over the CAN module featured in the LF240xA series of devices. For this reason, code written for LF240xA CAN modules cannot be directly ported to eCAN. However, eCAN follows the same register bit-layout structure and bit functionality as that of LF240xA CAN (for registers that exist in both devices) that is, many registers and bits perform exactly identical functions across these two platforms. This makes code migration a relatively easy task, more so with code written in C language.

## 16.3 CAN Network and Module

The controller area network (CAN) uses a serial multimaster communication protocol that efficiently supports distributed real-time control, with a high level of reliability, and a communication rate of up to 1 Mbps. The CAN bus is ideal for applications operating in electrically noisy environments, such as in the automotive and other industrial fields that require reliable communication.

Prioritized messages of up to eight bytes in data length can be sent on a multimaster serial bus using an arbitration protocol and an error-detection mechanism for a high level of data integrity.

### 16.3.1 CAN Protocol Overview

The CAN protocol supports four different frame types for communication:

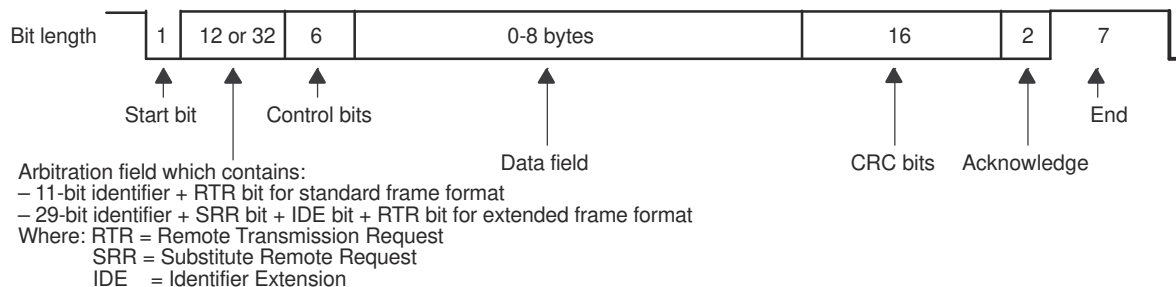
- Data frames that carry data from a transmitter node to the receiver node(s).
- Remote frames that are transmitted by a node to request the transmission of a data frame with the same identifier.
- Error frames that are transmitted by any node upon detecting an error condition.
- Overload frames that provide an extra delay between the preceding and the succeeding data frames or remote frames.

In addition, CAN specification version 2.0B defines two different formats that differ in the length of the identifier field: standard frames with an 11-bit identifier and extended frames with 29-bit identifier.

CAN standard data frames contain from 44 to 108 bits and CAN extended data frames contain 64 to 128 bits. Furthermore, up to 23 stuff bits can be inserted in a standard data frame, and up to 28 stuff bits in an extended data frame, depending on the data-stream coding. The overall maximum data frame length is then 131 bits for a standard frame and 156 bits for an extended frame.

The bit fields that make up standard or extended data frames, along with their position as shown in [Figure 16-2](#) include the following:

- Start of frame
- Arbitration field containing the identifier and the type of message being sent
- Control field indicating the number of bytes being transmitted.
- Up to 8 bytes of data
- Cyclic redundancy check (CRC)
- Acknowledgment
- End-of-frame bits

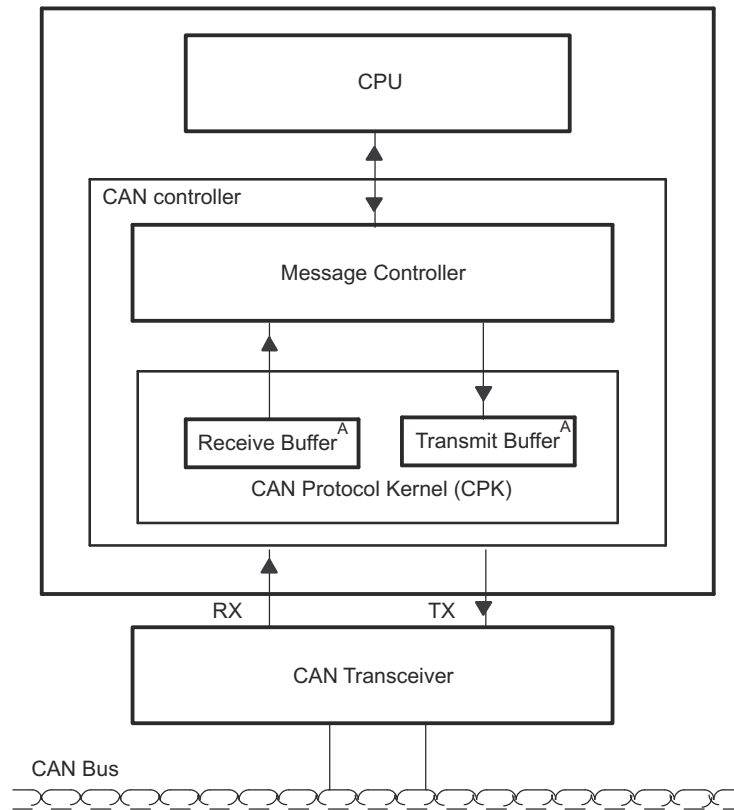


**Note:** Unless otherwise noted, numbers are amount of bits in field.

**Figure 16-2. CAN Data Frame**

The eCAN controller provides the CPU with full functionality of the CAN protocol. The CAN controller minimizes the CPU's load in communication overhead and enhances the CAN standard by providing additional features.

The architecture of eCAN module, shown in [Figure 16-3](#), is composed of a CAN protocol kernel (CPK) and a message controller.



A. The receive and transmit buffers are transparent to the user and are not accessible by user code.

**Figure 16-3. Architecture of the eCAN Module**

One function of the CPK is to decode all messages received on the CAN bus and to transfer these messages into a receive buffer. Another function is to transmit messages on the CAN bus according to the CAN protocol.

The message controller of a CAN controller is responsible for determining if any message received by the CPK must be preserved for the CPU use (that is, copied into the mailbox RAM) or be discarded. At the initialization phase, the CPU specifies to the message controller all message identifiers used by the application. The message controller is also responsible for sending the next message to transmit to the CPK according to the message's priority.

## 16.4 eCAN Controller Overview

The eCAN has an internal 32-bit architecture.

The eCAN module consists of:

- The CAN protocol kernel (CPK)
- The message controller comprising:
  - The memory management unit (MMU), including the CPU interface and the receive control unit (acceptance filtering), and the timer management unit
  - Mailbox RAM enabling the storage of 32 messages
  - Control and status registers

After the reception of a valid message by the CPK, the receive control unit of the message controller determines if the received message must be stored into one of the 32 message objects of the mailbox RAM. The receive control unit checks the state, the identifier, and the mask of all message objects to determine the appropriate mailbox location. The received message is stored into the first mailbox passing the acceptance filtering. If the receive control unit could not find any mailbox to store the received message, the message is discarded.

A message is composed of an 11- or 29-bit identifier, a control field, and up to 8 bytes of data.

When a message must be transmitted, the message controller transfers the message into the transmit buffer of the CPK in order to start the message transmission at the next bus-idle state. When more than one message must be transmitted, the message with the highest priority that is ready to be transmitted is transferred into the CPK by the message controller. If two mailboxes have the same priority, then the mailbox with the higher number is transmitted first.

The timer management unit comprises a time-stamp counter and apposes a time stamp to all messages received or transmitted. It generates an interrupt when a message has not been received or transmitted during an allowed period of time (time-out). The time-stamping feature is available in eCAN mode only.

To initiate a data transfer, the transmission request bit (TRS.n) must be set. The entire transmission procedure and possible error handling are then performed without any CPU involvement. If a mailbox has been configured to receive messages, the CPU easily reads its data registers using CPU read instructions. The mailbox may be configured to interrupt the CPU after every successful message transmission or reception.

### 16.4.1 Standard CAN Controller (SCC) Mode

The SCC Mode is a reduced functionality mode of the eCAN. Only 16 mailboxes (0 through 15) are available in this mode. The time stamping feature is not available and the number of acceptance masks available is reduced. This mode is selected by default. The SCC mode or the full featured eCAN mode is selected using the SCB bit (CANMC.13).

## 16.4.2 Memory Map

The eCAN module has two different address segments mapped in the memory. The first segment is used to access the control registers, the status registers, the acceptance masks, the time stamp, and the time-out of the message objects. The access to the control and status registers is limited to 32-bit wide accesses. The local acceptance masks, the time stamp registers, and the time-out registers can be accessed 8-bit, 16-bit and 32-bit wide. The second address segment is used to access the mailboxes. This memory range can be accessed 8-bit, 16-bit and 32-bit wide. Each of these two memory blocks, shown in [Figure 16-4](#), uses 512 bytes of address space.

The message storage is implemented by a RAM that can be addressed by the CAN controller or the CPU. The CPU controls the CAN controller by modifying the various mailboxes in the RAM or the additional registers. The contents of the various storage elements are used to perform the functions of the acceptance filtering, message transmission, and interrupt handling.

The mailbox module in the eCAN provides 32 message mailboxes of 8-byte data length, a 29-bit identifier, and several control bits. Each mailbox can be configured as either transmit or receive. In the eCAN mode, each mailbox has its individual acceptance mask.

### Note

LAMn, MOTSn and MOTOn registers and mailboxes not used in an application (disabled in the CANME register) may be used as general-purpose data memory by the CPU.

### 16.4.2.1 32-bit Access to Control and Status Registers

As indicated in [Section 16.4.2](#), only 32-bit accesses are allowed to the Control and Status registers. 16-bit access to these registers could potentially corrupt the register contents or return false data. The C2000Ware files released by TI employs a shadow register structure that aids in 32-bit access. Following are a few examples of how the shadow register structure may be employed to perform 32-bit reads and writes:

#### Example 16-1. Modifying a Bit in a Register

```
ECanaShadow.CANTIOC.all = ECanaRegs.CANTIOC.all; // Step 1
ECanaShadow.CANTIOC.bit.TXFUNC = 1; // Step 2
ECanaRegs.CANTIOC.all = ECanaShadow.CANTIOC.all; // Step 3
```

Step 1: Perform a 32-bit read to copy the entire register to its shadow

Step 2: Modify the needed bit or bits in the shadow

Step 3: Perform a 32-bit write to copy the modified shadow to the original register.

### Note

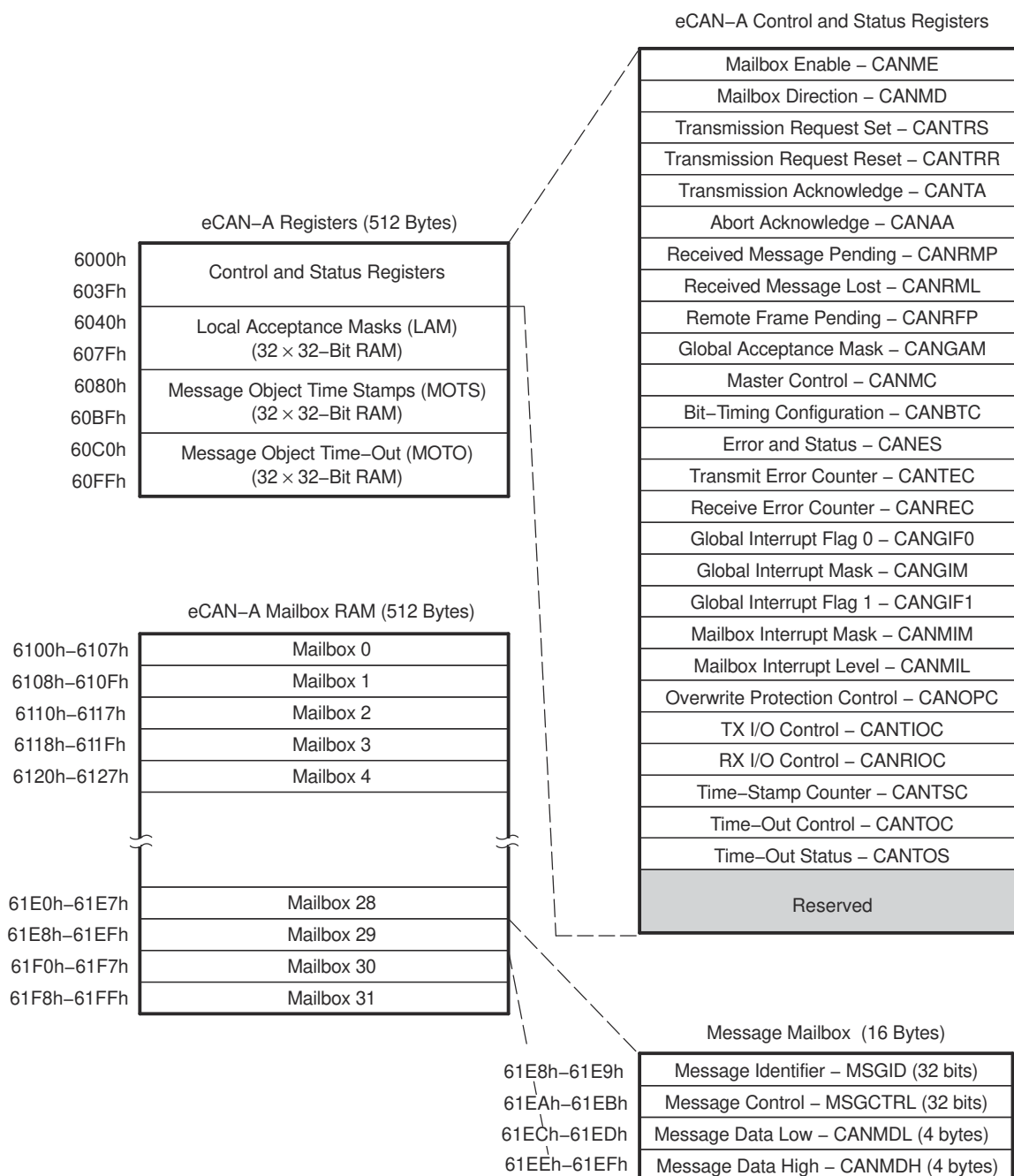
Some bits like TAn and RMPn are cleared by writing a 1 to it. Care should be taken not to clear bits inadvertently. It is good practice to initialize the shadow registers to zero before step 1.

#### Example 16-2. Checking the Value of a Bit in a Register

```
do { ECanaShadow.CANTA.all = ECanaRegs.CANTA.all;
}while(ECanaShadow.CANTA.bit.TA25 == 0); // Wait for TA5 bit to be set..
```

In [Example 16-2](#), the value of TA25 bit needs to be checked. This is done by first copying the entire CANTA register to its shadow (using a 32-bit read) and then checking the relevant bit, repeating this operation until that condition is satisfied. TA25 bit should NOT be checked with the following statement:

```
while (ECanaRegs.CANTA.bit.TA25 == 0);
```



**Figure 16-4. eCAN-A Memory Map**



## 16.5 Message Objects

The eCAN module has 32 message objects (mailboxes).

Each message object can be configured to either transmit or receive. Each message object has its own acceptance mask (in the eCAN mode; not in SCC mode).

A message object consists of a message mailbox with:

- The 29-bit message identifier
- The message control register
- 8 bytes of message data
- A 29-bit acceptance mask
- A 32-bit time stamp
- A 32-bit time-out value

Furthermore, corresponding control and status bits located in the registers allow control of the message objects.

## 16.6 Message Mailbox

The message mailboxes are the RAM area where the CAN messages are actually stored after they are received or before they are transmitted.

The CPU may use the RAM area of the message mailboxes that are not used for storing messages as normal memory.

Each mailbox contains:

- The message identifier
  - 29 bits for extended identifier
  - 11 bits for standard identifier
- The identifier extension bit, IDE (MSGID.31)
- The acceptance mask enable bit, AME (MSGID.30)
- The auto answer mode bit, AAM (MSGID.29)
- The transmit priority level, TPL (MSGCTRL.12-8)
- The remote transmission request bit, RTR (MSGCTRL.4)
- The data length code, DLC (MSGCTRL.3-0)
- Up to eight bytes for the data field

Each of the mailboxes can be configured as one of four message object types (see [Table 16-1](#)). Transmit and receive message objects are used for data exchange between one sender and multiple receivers (1 to n communication link), whereas request and reply message objects are used to set up a one-to-one communication link. [Table 16-2](#) lists the mailbox RAM layout.

**Table 16-1. Message Object Behavior Configuration**

| Message Object Behavior       | Mailbox Direction Register (CANMD) | Auto-Answer Mode Bit (AAM) | Remote Transmission Request Bit (RTR) |
|-------------------------------|------------------------------------|----------------------------|---------------------------------------|
| Transmit message object       | 0                                  | 0                          | 0                                     |
| Receive message object        | 1                                  | 0                          | 0                                     |
| Remote-Request message object | 1                                  | 0                          | 1                                     |
| Auto-Reply message object     | 0                                  | 1                          | 0                                     |

**Table 16-2. eCAN-A Mailbox RAM Layout**

| Mailbox | MSGID<br>MSGIDL-MSGIDH | MSGCTRL<br>MSGCTRL-Rsvd | CANMDL<br>CANMDL_L- CANMDL_H | CANMDH<br>CANMDH_L- CANMDH_H |
|---------|------------------------|-------------------------|------------------------------|------------------------------|
| 0       | 6100-6101h             | 6102-6103h              | 6104-6105h                   | 6106-6107h                   |
| 1       | 6108-6109h             | 610A-610Bh              | 610C-610Dh                   | 610E-610Fh                   |
| 2       | 6110 - 6111h           | 6112-6113h              | 6114-6115h                   | 6116-6117h                   |
| 3       | 6118-6119h             | 611A-611Bh              | 611C-611Dh                   | 611E-611Fh                   |
| 4       | 6120-6121h             | 6122-6123h              | 6124-6125h                   | 6126-6127h                   |
| 5       | 6128-6129h             | 612A-612Bh              | 612C-612Dh                   | 612E-612Fh                   |
| 6       | 6130-6131h             | 6132-6133h              | 6134-6135h                   | 6136-6137h                   |
| 7       | 6138-6139h             | 613A-613Bh              | 613C-613Dh                   | 613E-613Fh                   |
| 8       | 6140-6141h             | 6142-6143h              | 6144-6145h                   | 6146-6147h                   |
| 9       | 6148-6149h             | 614A-614Bh              | 614C-614Dh                   | 614E-614Fh                   |
| 10      | 6150-6151h             | 6152-6153h              | 6154-6155h                   | 6156-6157h                   |
| 11      | 6158-6159h             | 615A-615Bh              | 615C-615Dh                   | 615E-615Fh                   |
| 12      | 6160-6161h             | 6162-6163h              | 6164-6165h                   | 6166-6167h                   |
| 13      | 6168-6169h             | 616A-616Bh              | 616C-616Dh                   | 616E-616Fh                   |
| 14      | 6170-6171h             | 6172-6173h              | 6174-6175h                   | 6176-6177h                   |
| 15      | 6178-6179h             | 617A-617Bh              | 617C-617Dh                   | 617E-617Fh                   |
| 16      | 6180-6181h             | 6182-6183h              | 6184-6185h                   | 6186-6187h                   |
| 17      | 6188-6189h             | 618A-618Bh              | 618C-618Dh                   | 618E-618Fh                   |
| 18      | 6190-6191h             | 6192-6193h              | 6194-6195h                   | 6196-6197h                   |
| 19      | 6198-6199h             | 619A-619Bh              | 619C-619Dh                   | 619E-619Fh                   |
| 20      | 61A0-61A1h             | 61A2-61A3h              | 61A4-61A5h                   | 61A6-61A7h                   |
| 21      | 61A8-61A9h             | 61AA-61ABh              | 61AC-61ADh                   | 61AE-61AFh                   |
| 22      | 61B0-61B1h             | 61B2-61B3h              | 61B4-61B5h                   | 61B6-61B7h                   |
| 23      | 61B8-61B9h             | 61BA-61BBh              | 61BC-61BDh                   | 61BE-61BFh                   |
| 24      | 61C0-61C1h             | 61C2-61C3h              | 61C4-61C5h                   | 61C6-61C7h                   |
| 25      | 61C8-61C9h             | 61CA-61CBh              | 61CC-61CDh                   | 61CE-61CFh                   |
| 26      | 61D0-61D1h             | 61D2-61D3h              | 61D4-61D5h                   | 61D6-61D7h                   |
| 27      | 61D8-61D9h             | 61DA-61DBh              | 61DC-61DDh                   | 61DE-61DFh                   |
| 28      | 61E0-61E1h             | 61E2-61E3h              | 61E4-61E5h                   | 61E6-61E7h                   |
| 29      | 61E8-61E9h             | 61EA-61EBh              | 61EC-61EDh                   | 61EE-61EFh                   |
| 30      | 61F0-61F1h             | 61F2-61F3h              | 61F4-61F5h                   | 61F6-61F7h                   |
| 31      | 61F8-61F9h             | 61FA-61FBh              | 61FC-61FDh                   | 61FE-61FFh                   |

**Table 16-3. Addresses of LAM, MOTS, and MOTO Registers for Mailboxes (eCAN-A)**

| Mailbox | LAM         | MOTS        | MOTO        |
|---------|-------------|-------------|-------------|
| 0       | 6040h-6041h | 6080h-6081h | 60C0h-60C1h |
| 1       | 6042h-6043h | 6082h-6083h | 60C2h-60C3h |
| 2       | 6044h-6045h | 6084h-6085h | 60C4h-60C5h |
| 3       | 6046h-6047h | 6086h-6087h | 60C6h-60C7h |
| 4       | 6048h-6049h | 6088h-6089h | 60C8h-60C9h |
| 5       | 604Ah-604Bh | 608Ah-608Bh | 60CAh-60CBh |
| 6       | 604Ch-604Dh | 608Ch-608Dh | 60CCh-60CDh |
| 7       | 604Eh-604Fh | 608Eh-608Fh | 60CEh-60CFh |
| 8       | 6050h-6051h | 6090h-6091h | 60D0h-60D1h |
| 9       | 6052h-6053h | 6092h-6093h | 60D2h-60D3h |
| 10      | 6054h-6055h | 6094h-6095h | 60D4h-60D5h |
| 11      | 6056h-6057h | 6096h-6097h | 60D6h-60D7h |
| 12      | 6058h-6059h | 6098h-6099h | 60D8h-60D9h |
| 13      | 605Ah-605Bh | 609Ah-609Bh | 60DAh-60DBh |
| 14      | 605Ch-605Dh | 609Ch-609Dh | 60DCh-60DDh |
| 15      | 605Eh-605Fh | 609Eh-609Fh | 60DEh-60DFh |
| 16      | 6060h-6061h | 60A0h-60A1h | 60E0h-60E1h |
| 17      | 6062h-6063h | 60A2h-60A3h | 60E2h-60E3h |
| 18      | 6064h-6065h | 60A4h-60A5h | 60E4h-60E5h |
| 19      | 6066h-6067h | 60A6h-60A7h | 60E6h-60E7h |
| 20      | 6068h-6069h | 60A8h-60A9h | 60E8h-60E9h |
| 21      | 606Ah-606Bh | 60AAh-60ABh | 60EAh-60EBh |
| 22      | 606Ch-606Dh | 60ACh-60ADh | 60ECh-60EDh |
| 23      | 606Eh-606Fh | 60AEh-60AFh | 60EEh-60EFh |
| 24      | 6070h-6071h | 60B0h-60B1h | 60F0h-60F1h |
| 25      | 6072h-6073h | 60B2h-60B3h | 60F2h-60F3h |
| 26      | 6074h-6075h | 60B4h-60B5h | 60F4h-60F5h |
| 27      | 6076h-6077h | 60B6h-60B7h | 60F6h-60F7h |
| 28      | 6078h-6079h | 60B8h-60B9h | 60F8h-60F9h |
| 29      | 607Ah-607Bh | 60BAh-60BBh | 60FAh-60FBh |
| 30      | 607Ch-607Dh | 60BCh-60BDh | 60FCh-60FDh |
| 31      | 607Eh-607Fh | 60BEh-60BFh | 60FEh-60FFh |

### 16.6.1 Transmit Mailbox

The CPU stores the data to be transmitted in a mailbox configured as transmit mailbox. After writing the data and the identifier into the RAM, the message is sent if the corresponding TRS[n] bit has been set, provided the mailbox is enabled by setting the corresponding the CANME.n bit.

If more than one mailbox is configured as transmit mailbox and more than one corresponding TRS[n] is set, the messages are sent one after another in falling order beginning with the mailbox with the highest priority.

In the SCC-compatibility mode, the priority of the mailbox transmission depends on the mailbox number. The highest mailbox number (=15) comprises the highest transmit priority.

In the eCAN mode, the priority of the mailbox transmission depends on the setting of the TPL field in the message control field (MSGCTRL) register. The mailbox with the highest value in the TPL is transmitted first. Only when two mailboxes have the same value in the TPL is the higher numbered mailbox transmitted first.

If a transmission fails due to a loss of arbitration or an error, the message transmission will be reattempted. Before reattempting the transmission, the CAN module checks if other transmissions are requested. If the TRS bit of a higher-priority (determined either by the MBX number or by the associated TPL value) mailbox had been set before the message in the transmit-buffer has lost arbitration, the transmit-buffer contents will be replaced with that of the higher-priority mailbox and the higher priority mailbox will be transmitted after the arbitration loss. However, if that TRS bit was set after the message in the transmit-buffer has lost arbitration, the higher priority MBX will be transmitted only after the current message in the transmit-buffer has been transmitted.

### 16.6.2 Receive Mailbox

The identifier of each incoming message is compared to the identifiers held in the receive mailboxes using the appropriate mask. When equality is detected, the received identifier, the control bits, and the data bytes are written into the matching RAM location. At the same time, the corresponding receive-message-pending bit, RMP[n] (RMP.31-0), is set and a receive interrupt is generated if enabled. If no match is detected, the message is not stored.

When a message is received, the message controller starts looking for a matching identifier at the mailbox with the highest mailbox number. Mailbox 15 of the eCAN in SCC compatible mode has the highest receive priority; mailbox 31 has the highest receive priority of the eCAN in eCAN mode.

RMP[n] (RMP.31-0) has to be reset by the CPU after reading the data. If a second message has been received for this mailbox and the receive-message-pending bit is already set, the corresponding message-lost bit (RML[n] (RML.31-0)) is set. In this case, the stored message is overwritten with the new data if the overwrite-protection bit OPC[n] (OPC.31-0) is cleared; otherwise, the next mailboxes are checked.

If a mailbox is configured as a receive mailbox and the RTR bit is set for it, the mailbox can send a remote frame. Once the remote frame is sent, the TRS bit of the mailbox is cleared by the CAN module. In addition to the TRS bit, the RTR bit is also cleared. The mailbox needs to be disabled in order to set this bit again.

### 16.6.3 CAN Module Operation in Normal Configuration

If the CAN module is being used in normal configuration (that is, not in self-test mode), there should be at least one more CAN module on the network, configured for the same bit rate. The other CAN module need NOT be configured to actually receive messages into the mailbox RAM from the transmitting node. But, it should be configured for the same bit rate. This is because a transmitting CAN module expects at least one node in the CAN network to acknowledge the proper reception of a transmitted message. Per CAN protocol specification, any CAN node that received a message will acknowledge (unless the acknowledge mechanism has been explicitly turned off), irrespective of whether it has been configured to store the received message or not. It is not possible to turn off the acknowledge mechanism in the eCAN module.

The requirement of another node does not exist for the self-test mode (STM). In this mode, a transmitting node generates its own acknowledge signal. The only requirement is that the node be configured for any valid bit-rate. That is, the bit timing registers should not contain a value that is not permitted by the CAN protocol.

It is not possible to achieve a direct digital loopback externally by connecting the CANTX and CANRX pins together (as is possible with SCI/SPI/McBSP modules). An internal loopback is possible in the self-test mode (STM).

## 16.7 eCAN Configuration

This section explains the process of initialization and describes the procedures to configure the eCAN module.

### 16.7.1 CAN Module Initialization

The CAN module must be initialized before the utilization. Initialization is only possible if the module is in initialization mode. [Figure 16-5](#) is a flow chart showing the process.

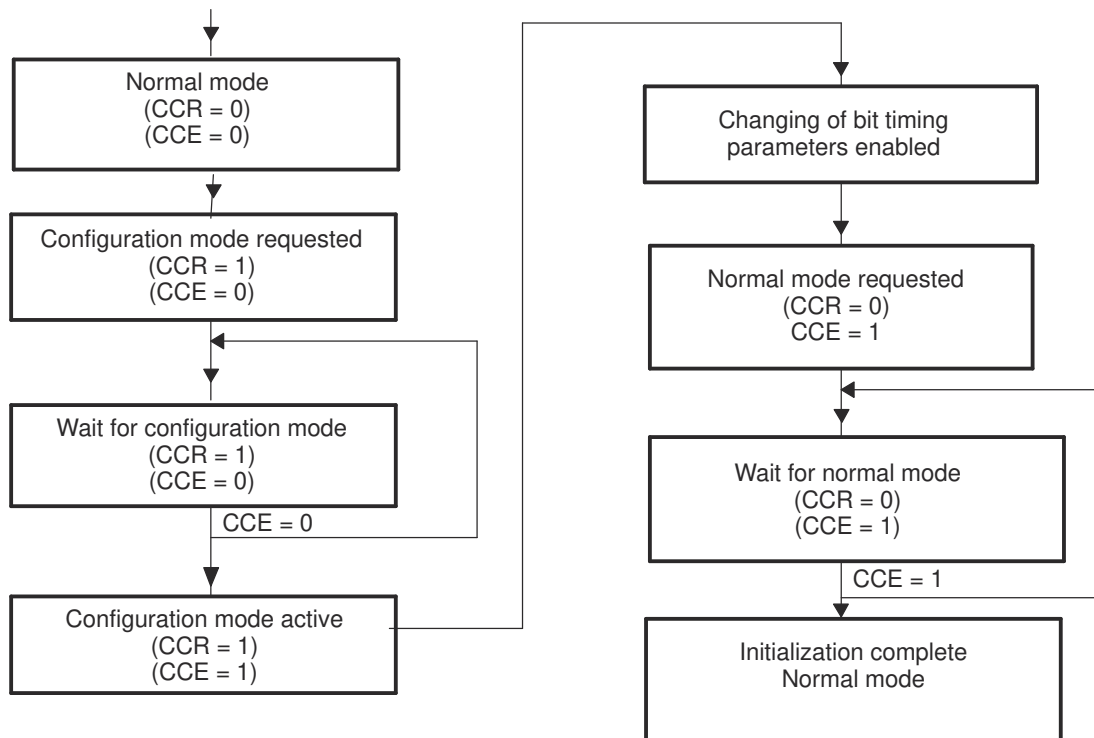
Programming CCR (CANMC.12) = 1 sets the initialization mode. The initialization can be performed only when CCE (CANES.4) = 1. Afterwards, the configuration registers can be written.

SCC mode only:

- In order to modify the global acceptance mask register (CANGAM) and the two local acceptance mask registers [LAM(0) and LAM(3)], the CAN module also must be set in the initialization mode.
- The module is activated again by programming CCR(CANMC.12) = 0.
- After hardware reset, the initialization mode is active.

#### Note

If the CANBTC register is programmed with a zero value, or left with the initial value, the CAN module never leaves the initialization mode, that is CCE (CANES.4) bit remains at 1 when clearing the CCR bit.



**Figure 16-5. Initialization Sequence**

#### Note

The transition between initialization mode and normal mode and conversely is performed in synchronization with the CAN network. That is, the CAN controller waits until it detects a bus idle sequence (= 11 recessive bits) before it changes the mode. In the event of a stuck-to-dominant bus error, the CAN controller cannot detect a bus-idle condition and therefore is unable to perform a mode transition.

### 16.7.1.1 CAN Bit-Timing Configuration

The CAN protocol specification partitions the nominal bit time into four different time segments:

**SYNC\_SEG:** This part of bit time is used to synchronize the various nodes on the bus. An edge is expected to lie within this segment. This segment is always 1 TIME QUANTUM (TQ).

**PROP\_SEG:** This part of the bit time is used to compensate for the physical delay times within the network. It is twice the sum of the signal's propagation 'time on the bus line, the input comparator delay, and the output driver delay. This segment is programmable from 1 to 8 TIME QUANTA (TQ).

**PHASE\_SEG1:** This phase is used to compensate for positive edge phase error. This segment is programmable from 1 to 8 TIME QUANTA (TQ) and can be lengthened by resynchronization.

**PHASE\_SEG2:** This phase is used to compensate for negative edge phase error. This segment is programmable from 2 to 8 TIME QUANTA (TQ) and can be shortened by resynchronization.

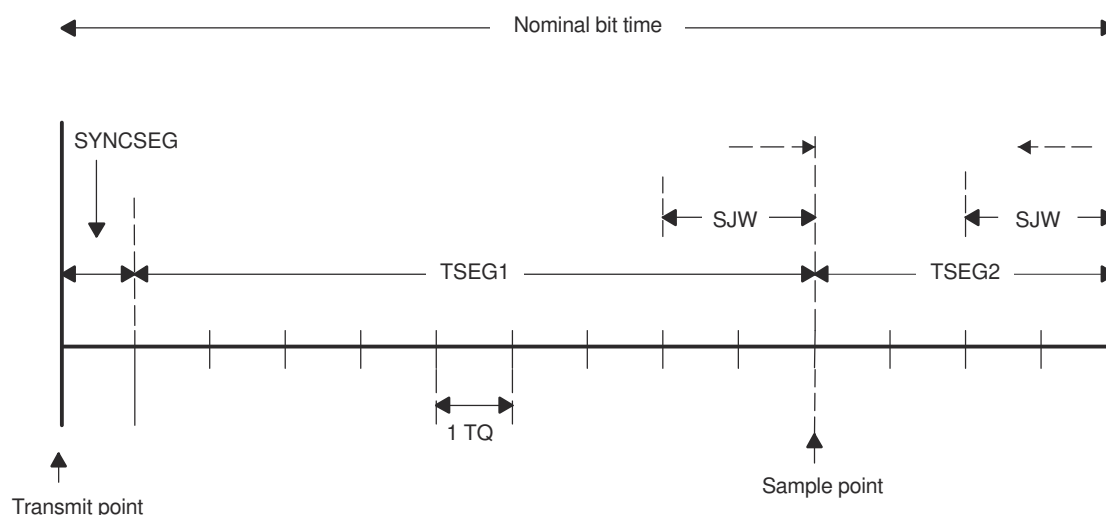
In the eCAN module, the length of a bit on the CAN bus is determined by the parameters TSEG1 (BTC.6-3), TSEG2 (BTC.2-0), and BRP (BTC.23.16).

TSEG1 combines the two time segments PROP\_SEG and PHASE\_SEG1 as defined by the CAN protocol. TSEG2 defines the length of the time segment PHASE\_SEG2.

IPT (information processing time) corresponds to the time necessary for the processing of the bit read. IPT corresponds to two units of TQ.

The following bit timing rules must be fulfilled when determining the bit segment values:

- $TSEG1(min) \geq TSEG2$
- $IPT \leq TSEG1 \leq 16 TQ$
- $IPT \leq TSEG2 \leq 8 TQ$
- $IPT = 3/BRP$  (the resulting IPT has to be rounded up to the next integer value)
- $1 TQ \leq SJW \leq \min[4 TQ, TSEG2]$  (SJW = Synchronization jump width)
- To utilize three-time sampling mode,  $BRP \geq 5$  has to be selected



A. TSEG1 can be lengthened or TSEG2 shortened by the SJW

**Figure 16-6. CAN Bit Timing**

### 16.7.1.2 CAN Bit Rate Calculation

Bit-rate is calculated in bits per second as follows:

$$\text{Bit rate} = \frac{\text{SYSCLKOUT} / 2}{\text{BRP} \times \text{Bit Time}}$$

Where bit-time is the number of time quanta (TQ) per bit. SYSCLKOUT is the device clock frequency, which is the same as the CPU clock frequency. BRP is the value of BRP<sub>reg</sub> + 1 (CANBTC.23-16).

Bit-time is defined as follows:

$$\text{Bit-time} = (\text{TSEG1}_{\text{reg}} + 1) + (\text{TSEG2}_{\text{reg}} + 1) + 1$$

In the above equation TSEG1<sub>reg</sub> and TSEG2<sub>reg</sub> represent the actual values written in the corresponding fields in the CANBTC register. The parameters TSEG1<sub>reg</sub>, TSEG2<sub>reg</sub>, SJW<sub>reg</sub>, and BRP<sub>reg</sub> are automatically enhanced by 1 when the CAN module accesses these parameters. TSEG1, TSEG2 and SJW, represent the values as applicable per Figure 16-6.

$$\text{Bit-time} = \text{TSEG1} + \text{TSEG2} + 1$$

### 16.7.1.3 Bit Configuration Parameters for 45-MHz CAN Clock

This section provides example values for the CANBTC bit fields for some bit rates and sampling points. Note that these values are for illustrative purposes only. In a real-world application, parameters such as the oscillator accuracy and the propagation delay introduced by various entities such as the network cable, transceivers/ isolators must be taken into account before choosing the timing parameters.

Table 16-4 shows how the BRP<sub>reg</sub> field may be changed to achieve different bit rates with a BT of 15 for a 80% SP.

**Table 16-4. BRP Field for Bit Rates (BT = 15, TSEG1<sub>reg</sub> = 10, TSEG2<sub>reg</sub> = 2, Sampling Point = 80%)**

| CAN Bus Speed | BRP                         | CAN Module Clock |
|---------------|-----------------------------|------------------|
| 1 Mbps        | BRP <sub>reg</sub> + 1 = 3  | 15 MHz           |
| 500 kbps      | BRP <sub>reg</sub> + 1 = 6  | 7.5 MHz          |
| 250 kbps      | BRP <sub>reg</sub> + 1 = 12 | 3.75 MHz         |
| 125 kbps      | BRP <sub>reg</sub> + 1 = 24 | 1.875 MHz        |
| 100 kbps      | BRP <sub>reg</sub> + 1 = 30 | 1.5 MHz          |
| 50 kbps       | BRP <sub>reg</sub> + 1 = 60 | 0.75 MHz         |

Table 16-5 shows how to achieve different sampling points with a BT of 15.

**Table 16-5. Achieving Different Sampling Points With a BT of 15**

| TSEG1 <sub>reg</sub> | TSEG2 <sub>reg</sub> | SP  |
|----------------------|----------------------|-----|
| 10                   | 2                    | 80% |
| 9                    | 3                    | 73% |
| 8                    | 4                    | 66% |
| 7                    | 5                    | 60% |

#### Note

For a SYSCLKOUT of 90 MHz, the smallest bit rate possible is 7.031 kbps.

#### 16.7.1.4 EALLOW Protection

To protect against inadvertent modification, some critical registers/bits of the eCAN module are EALLOW protected. These registers and bits can be changed only if the EALLOW protection has been disabled. Following are the registers and bits that are EALLOW protected in the eCAN module:

- CANMC[15:9, 7:6]
- CANBTC
- CANGIM
- CANMIM[31..0]
- CANTSC[31..0]
- CANTIOC[3]
- CANRIOC[3]

#### 16.7.2 Steps to Configure eCAN

---

**Note**

This sequence must be done with EALLOW enabled.

---

The following steps must be performed to configure the eCAN for operation:

1. Enable clock to the CAN module.
2. Set the CANTX and the CANRX pins to CAN functions:
  - a. Write CANTIOC.3:0 = 0x08
  - b. Write CANRIOC.3:0 = 0x08
3. After a reset, bit CCR (CANMC.12) and bit CCE (CANES.4) are set to 1. This allows the user to configure the bit-timing configuration register (CANBTC). If the CCE bit is set (CANES.4 = 1), proceed to next step; otherwise, set the CCR bit (CANMC.12 = 1) and wait until CCE bit is set (CANES.4 = 1).
4. Program the CANBTC register with the appropriate timing values. Make sure that the values TSEG1 and TSEG2 are not 0. If they are 0, the module does not leave the initialization mode.
5. For the SCC, program the acceptance masks now. For example:  
Write LAM(3) = 0x3C0000
6. Program the master control register (CANMC) as follows:
  - a. Clear CCR (CANMC.12) = 0
  - b. Clear PDR (CANMC.11) = 0
  - c. Clear DBO (CANMC.10) = 0
  - d. Clear WUBA (CANMC.9) = 0
  - e. Clear CDR (CANMC.8) = 0
  - f. Clear ABO (CANMC.7) = 0
  - g. Clear STM (CANMC.6) = 0
  - h. Clear SRES (CANMC.5) = 0
  - i. Clear MBNR (CANMC.4-0) = 0
7. Initialize all bits of MSGCTRLn registers to zero.
8. Verify the CCE bit is cleared (CANES.4 = 0), indicating that the CAN module has been configured.

This completes the setup for the basic functionality.



### 16.7.2.1 Configuring a Mailbox for Transmit

To transmit a message, the following steps need to be performed (in this example, for mailbox 1):

1. Clear the appropriate bit in the CANTRS register to 0:  
Clear CANTRS.1 = 0 (Writing a 0 to TRS has no effect; instead, set TRR.1 and wait until TRS.1 clears.) If the RTR bit is set, the TRS bit can send a remote frame. Once the remote frame is sent, the TRS bit of the mailbox is cleared by the CAN module. The same node can be used to request a data frame from another node.
2. Disable the mailbox by clearing the corresponding bit in the mailbox enable (CANME) register.  
Clear CANME.1 = 0
3. Load the message identifier (MSGID) register of the mailbox. Clear the AME (MSGID.30) and AAM (MSGID.29) bits for a normal send mailbox (MSGID.30 = 0 and MSGID.29 = 0). This register is usually not modified during operation. It can only be modified when the mailbox is disabled. For example:
  - a. Write MSGID(1) = 0x15AC0000
  - b. Write the data length into the DLC field of the message control field register (MSGCTRL.3:0). The RTR flag is usually cleared (MSGCTRL.4 = 0).
  - c. Set the mailbox direction by clearing the corresponding bit in the CANMD register.
  - d. Clear CANMD.1 = 0
4. Set the mailbox enable by setting the corresponding bit in the CANME register  
Set CANME.1 = 1

This configures mailbox 1 for transmit mode.

### 16.7.2.2 Transmitting a Message

To start a transmission (in this example, for mailbox1):

1. Write the message data into the mailbox data field.
2. Set the corresponding flag in the transmit request register (CANTRS.1 = 1) to start the transmission of the message. The CAN module now handles the complete transmission of the CAN message.
3. Wait until the transmit-acknowledge flag of the corresponding mailbox is set (TA.1 = 1). After a successful transmission, this flag is set by the CAN module.
4. The TRS flag is reset to 0 by the module after a successful or aborted transmission (TRS.1 = 0).
5. The transmit acknowledge must be cleared for the next transmission (from the same mailbox).
  - a. Set TA.1 = 1
  - b. Wait until read TA.1 is 0
6. To transmit another message in the same mailbox, the mailbox RAM data must be updated. Setting the TRS.1 flag starts the next transmission. Writing to the mailbox RAM can be half-word (16 bits) or full word (32 bits) but the module always returns 32-bit from even boundary. The CPU must accept all the 32 bits or part of it.

### 16.7.2.3 Configuring Mailboxes for Receive

To configure a mailbox to receive messages, the following steps must be performed (in this example, mailbox 3):

1. Disable the mailbox by clearing the corresponding bit in the mailbox enable (CANME) register.  
Clear CANME.3 = 0
2. Write the selected identifier into the corresponding MSGID register. The identifier extension bit must be configured to fit the expected identifier. If the acceptance mask is used, the acceptance mask enable (AME) bit must be set (MSGID.30 = 1). For example:  
Write MSGID(3) = 0x4F780000
3. If the AME bit is set to 1, the corresponding acceptance mask must be programmed.  
Write LAM(3) = 0x03C0000.
4. Configure the mailbox as a receive mailbox by setting the corresponding flag in the mailbox direction register (CANMD.3 = 1). Make sure no other bits in this register are affected by this operation.

5. If data in the mailbox is to be protected, the overwrite protection control register (CANOPC) should be programmed now. This protection is useful if no message must be lost. If OPC is set, the software has to make sure that an additional mailbox (buffer mailbox) is configured to store 'overflow' messages. Otherwise messages can be lost without notification.  
Write `OPC.3 = 1`
6. Enable the mailbox by setting the appropriate flag in the mailbox enable register (CANME). This should be done by reading CANME, and writing back (`CANME |= 0x0008`) to make sure no other flag has changed accidentally.

The object is now configured for the receive mode. Any incoming message for that object is handled automatically.

#### **16.7.2.4 Receiving a Message**

This example uses mailbox 3. When a message is received, the corresponding flag in the receive message pending register (CANRMP) is set to 1 and an interrupt can be initiated. The CPU can then read the message from the mailbox RAM. Before the CPU reads the message from the mailbox, it should first clear the RMP bit (`RMP.3 = 1`).

After reading the data, the CPU needs to check that the RMP bit has not been set again by the module. If the RMP bit has been set to 1, the data may have been corrupted. The CPU needs to read the data again because a new message was received while the CPU was reading the old one. The CPU should also check the receive message lost flag `RML.3 = 1`. Depending on the application, the CPU has to decide how to handle this situation.

#### **16.7.2.5 Handling of Overload Situations**

If the CPU is not able to handle important messages fast enough, it may be advisable to configure more than one mailbox for that identifier. Here is an example where the objects 3, 4, and 5 have the same identifier and share the same mask. For the SCC, the mask is `LAM(3)`. For the eCAN, each object has its own LAM: `LAM(3)`, `LAM(4)`, and `LAM(5)`, all of which need to be programmed with the same value.

To make sure that no message is lost, set the OPC flag for objects 4 and 5, which prevents unread messages from being overwritten. If the CAN module must store a received message, it first checks mailbox 5. If the mailbox is empty, the message is stored there. If the RMP flag of object 5 is set (mailbox occupied), the CAN module checks the condition of mailbox 4. If that mailbox is also busy, the module checks in mailbox 3 and stores the message there since the OPC flag is not set for mailbox 3. If mailbox 3 contents have not been previously read, it sets the RML flag of object 3, which can initiate an interrupt.

It is also advisable to have object 4 generate an interrupt signaling the CPU to read mailboxes 4 and 5 at once. This technique is also useful for messages that require more than 8 bytes of data (that is, more than one message). In this case, all data needed for the message can be collected in the mailboxes and be read at once.

### **16.7.3 Handling of Remote Frame Mailboxes**

There are two functions for remote frame handling. One is a request by the module for data from another node, the other is a request by another node for data that the module needs to answer.

#### **16.7.3.1 Requesting Data From Another Node**

In this case the mailbox is configured as receive mailbox as mentioned above. To request data from another node, the CPU needs to do the following:

1. Set the RTR bit in the message control field register (CANMSGCTRL) to 1.
2. Write the correct identifier into the message identifier register (MSGID).
3. Set the CANTRS flag for that mailbox. Since the mailbox is configured as receive, it only sends a remote request message to the other node.
4. The module stores the answer in that mailbox and sets the RMP bit when it is received. This action can initiate an interrupt. Also, make sure no other mailbox has the same ID.
5. Read the received message.

### 16.7.3.2 Answering a Remote Request

To answer a remote request, the following is needed:

1. Configure the object as a transmit mailbox.
2. Set the auto answer mode (AAM) (MSGID.29) bit in the MSGID register before the mailbox is enabled.
3. Update the data field.  
MDL, MDH(1) = xxxxxxxh
4. Enable the mailbox by setting the CANME bit to 1.  
When a remote request is received from another node, the TRS flag is set automatically and the data is transmitted to that node. The identifier of the received message and the transmitted message are the same. After transmission of the data, the TA flag is set. The CPU can then update the data.

### 16.7.3.3 Updating the Data Field

To update the data of an object that is configured in auto answer mode, the following steps need to be performed. This sequence can also be used to update the data of an object configured in normal transmission with TRS flag set.

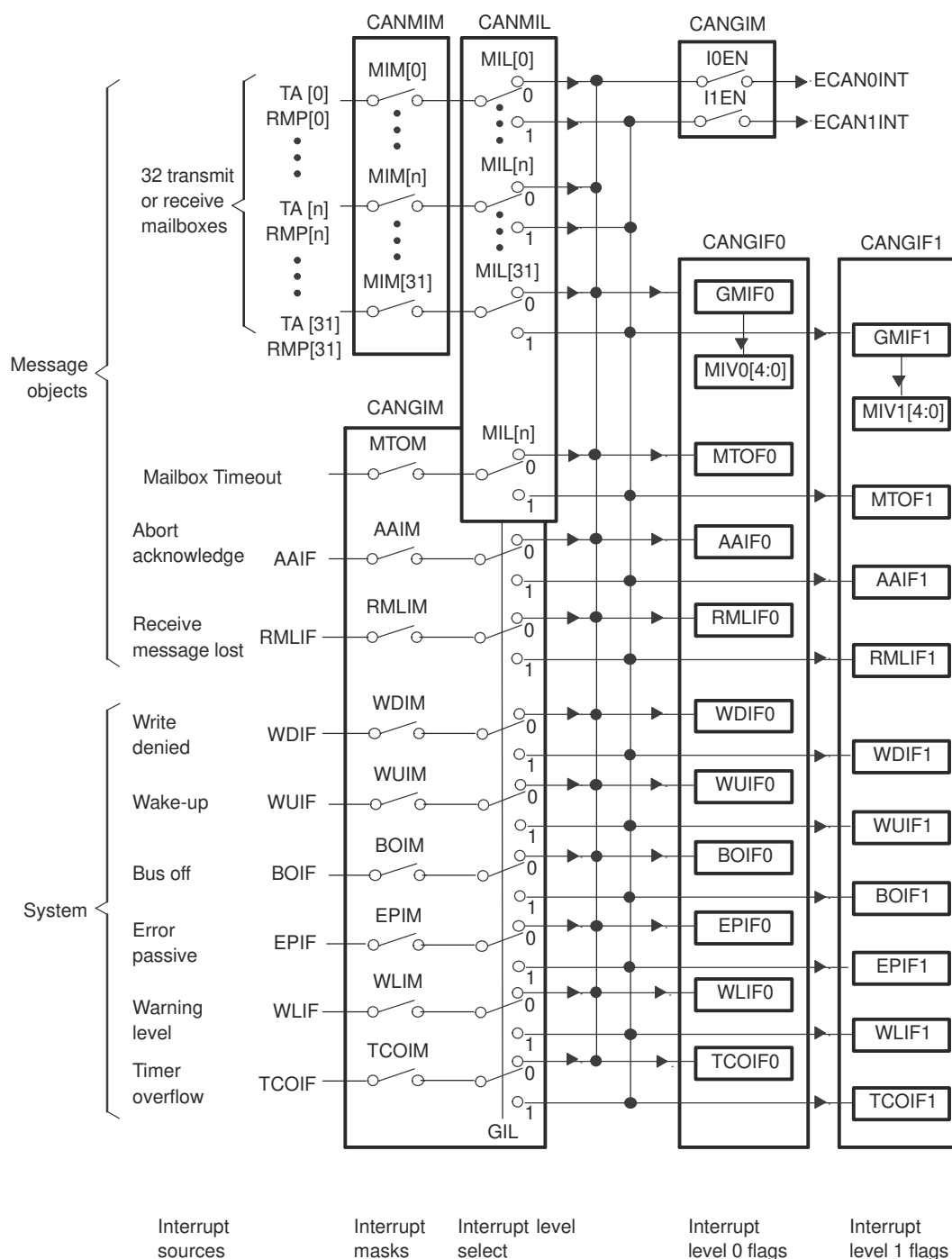
1. Set the change data request (CDR) (CANMC.8) bit and the mailbox number (MBNR) of that object in the master control register (CANMC). This signals the CAN module that the CPU wants to change the data field.  
For example, for object 1:  
Write CANMC = 0x0000101
2. Write the message data into the mailbox data register. For example:  
Write CANMDL(1) = xxxx0000h
3. Clear the CDR bit (CANMC.8) to enable the object.  
Write CANMC = 0x00000000

### 16.7.4 Interrupts

There are two different types of interrupts. One type of interrupt is a mailbox related interrupt, for example, the receive-message-pending interrupt or the abort-acknowledge interrupt. The other type of interrupt is a system interrupt that handles errors or system-related interrupt sources, for example, the error-passive interrupt or the wake-up interrupt. See [Figure 16-7](#).

The following events can initiate one of the two interrupts:

- Mailbox interrupts
  - Message reception interrupt: a message was received
  - Message transmission interrupt: a message was transmitted successfully
  - Abort-acknowledge interrupt: a pending transmission was aborted
  - Received-message-lost interrupt: an old message was overwritten by a new one (before the old message was read)
  - Mailbox timeout interrupt (eCAN mode only): one of the messages was not transmitted or received within a predefined time frame
- System interrupts
  - Write-denied interrupt: the CPU tried to write to a mailbox but was not allowed to
  - Wake-up interrupt: this interrupt is generated after a wake up
  - Bus-off interrupt: the CAN module enters the bus-off state
  - Error-passive interrupt: the CAN module enters the error-passive mode
  - Warning level interrupt: one or both error counters are greater than or equal to 96
  - Time-stamp counter overflow interrupt (eCAN only): the time-stamp counter had an overflow


**Figure 16-7. Interrupts Scheme**

#### 16.7.4.1 Interrupts Scheme

The interrupt flags are set if the corresponding interrupt condition occurred. The system interrupt flags are set depending on the setting of GIL (CANGIM.2). If set, the global interrupts set the bits in the CANGIF1 register, otherwise they set in the CANGIF0 register.

The GMIF0/GMIF1(CANGIF0.15/CANGIF1.15) bit is set depending on the setting of the MIL[n] bit that corresponds to the mailbox originating that interrupt. If the MIL[n] bit is set, the corresponding mailbox interrupt flag MIF[n] sets the GMIF1 flag in the CANGIF1 register, otherwise, it sets the GMIF0 flag.

If all interrupt flags are cleared and a new interrupt flag is set, the CAN module interrupt output line (ECAN0INT or ECAN1INT) is activated if the corresponding interrupt mask bit is set. The interrupt line stays active until the interrupt flag is cleared by the CPU by writing a 1 to the appropriate bit.

The GMIF0 (CANGIF0.15) or GMIF1 (CANGIF1.15) bit must be cleared by writing a 1 to the appropriate bit in the CANTA register or the CANRMP register (depending on mailbox configuration) and cannot be cleared in the CANGIF0/CANGIF1 register.

After clearing one or more interrupt flags, and one or more interrupt flags are still pending, a new interrupt is generated. The interrupt flags are cleared by writing a 1 to the corresponding bit location. If the GMIF0 or GMIF1 bit is set, the mailbox interrupt vector MIV0 (CANGIF0.4-0) or MIV1 (CANGIF1.4-0) indicates the mailbox number of the mailbox that caused the setting of the GMIF0/1. It always displays the highest mailbox interrupt vector assigned to that interrupt line.

#### 16.7.4.2 Mailbox Interrupt

Each of the 32 mailboxes in the eCAN or the 16 mailboxes in the SCC can initiate an interrupt on one of the two interrupt output lines 1 or 0. These interrupts can be receive or transmit interrupts depending on the mailbox configuration.

There is one interrupt mask bit (MIM[n]) and one interrupt level bit (MIL[n]) dedicated to each mailbox. To generate a mailbox interrupt upon a receive/transmit event, the MIM bit has to be set. If a CAN message is received (RMP[n]=1) in a receive mailbox or transmitted (TA[n]=1) from a transmit mailbox, an interrupt is asserted. If a mailbox is configured as remote request mailbox (CANMD[n]=1, MSGCTRL.RTR=1), an interrupt occurs upon reception of the reply frame. A remote reply mailbox generates an interrupt upon successful transmission of the reply frame (CANMD[n]=0, MSGID.AAM=1).

The setting of the RMP[n] bit or the TA[n] bit also sets the GMIF0/GMIF1 (GIF0.15/GIF1.15) flag in the GIF0/GIF1 register if the corresponding interrupt mask bit is set. The GMIF0/GMIF1 flag then generates an interrupt and the corresponding mailbox vector (= mailbox number) can be read from the bit field MIV0/MIV1 in the GIF0/GIF1 register. If more than one mailbox interrupts are pending, the actual value of MIV0/MIV1 reflects the highest priority interrupt vector. The interrupt generated depends on the setting in the mailbox interrupt level (MIL) register.

The abort acknowledge flag (AA[n]) and the abort acknowledge interrupt flag (AAIF) in the GIF0/GIF1 register are set when a transmit message is aborted by setting the TRR[n] bit. An interrupt is asserted upon transmission abortion if the mask bit AAIM in the GIM register is set. Clearing the AA[n] flag clears the AAIF0/AAIF1 flag.

A lost receive message is notified by setting the receive message lost flag RML[n] and the receive message lost interrupt flag RMLIF0/RMLIF1 in the GIF0/GIF1 register. If an interrupt shall be generated upon the lost receive message event, the receive message lost interrupt mask bit (RMLIM) in the GIM register has to be set. Clearing the RML[n] flag does not reset the RMLIF0/RMLIF1 flag. The interrupt flag has to be cleared separately.

Each mailbox of the eCAN (in eCAN mode only) is linked to a message- object, time-out register (MOTO). If a time-out event occurs (TOS[n] = 1), a mailbox timeout interrupt is asserted to one of the two interrupt lines if the mailbox timeout interrupt mask bit (MTOM) in the CANGIM register is set. The interrupt line for mailbox timeout interrupt is selected in accordance with the mailbox interrupt level (MIL[n]) of the concerned mailbox.

### 16.7.4.3 Interrupt Handling

The CPU is interrupted by asserting one of the two interrupt lines. After handling the interrupt, which should generally also clear the interrupt source, the interrupt flag must be cleared by the CPU. To do this, the interrupt flag must be cleared in the CANGIF0 or CANGIF1 register. This is generally done by writing a 1 to the interrupt flag. There are some exceptions to this as stated in Table 16-6. This also releases the interrupt line if no other interrupt is pending.

**Table 16-6. eCAN Interrupt Assertion/Clearing**

| Interrupt Flag <sup>(1)</sup> | Interrupt Condition                                                           | GIF0/GIF1 Determination | Clearing Mechanism                                                                                                                             |
|-------------------------------|-------------------------------------------------------------------------------|-------------------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| WLIFn                         | One or both error counters are >= 96                                          | GIL bit                 | Cleared by writing a 1 to it                                                                                                                   |
| EPIFn                         | CAN module has entered "error passive" mode                                   | GIL bit                 | Cleared by writing a 1 to it                                                                                                                   |
| BOIFn                         | CAN module has entered "bus-off" mode                                         | GIL bit                 | Cleared by writing a 1 to it                                                                                                                   |
| RMLIFn                        | An overflow condition has occurred in one of the receive mailboxes            | GIL bit                 | Cleared by clearing the set RMPn bit                                                                                                           |
| WUIFn                         | CAN module has left the local power-down mode                                 | GIL bit                 | Cleared by writing a 1 to it                                                                                                                   |
| WDIFn                         | A write access to a mailbox was denied                                        | GIL bit                 | Cleared by writing a 1 to it                                                                                                                   |
| AAIFn                         | A transmission request was aborted                                            | GIL bit                 | Cleared by clearing the set AAn bit.                                                                                                           |
| GMIFn                         | One of the mailboxes successfully transmitted/received a message              | MILn bit                | Cleared by appropriate handling of the interrupt causing condition. Cleared by writing a 1 to the appropriate bit in CANTA or CANRMP registers |
| TCOFn                         | The MSB of the TSC has changed from 0 to 1                                    | GIL bit                 | Cleared by writing a 1 to it                                                                                                                   |
| MTOFn                         | One of the mailboxes did not transmit/receive within the specified time frame | MILn bit                | Cleared by clearing the set TOSn bit                                                                                                           |

(1) Key to interpreting the table above:

- 1) Interrupt flag: This is the name of the interrupt flag bit as applicable to CANGIF0/CANGIF1 registers.
- 2) Interrupt condition: This column illustrates the conditions that cause the interrupt to be asserted.
- 3) GIF0/GIF1 determination: Interrupt flag bits can be set in either CANGIF0 or CANGIF1 registers. This is determined by either the GIL bit in CANGIM register or MILn bit in the CANMIL register, depending on the interrupt under consideration. This column illustrates whether a particular interrupt is dependent on GIL bit or MILn bit.
- 4) Clearing mechanism: This column explains how a flag bit can be cleared. Some bits are cleared by writing a 1 to it. Other bits are cleared by manipulating some other bit in the CAN control register.

#### 16.7.4.3.1 Configuring for Interrupt Handling

To configure for interrupt handling, the mailbox interrupt level register (CANMIL), the mailbox interrupt mask register (CANMIM), and the global interrupt mask register (CANGIM) need to be configured. The steps to do this are described below:

1. Write the CANMIL register. This defines whether a successful transmission or reception asserts interrupt line 0 or 1. For example, CANMIL = 0xFFFFFFFF sets all mailbox interrupts to level 1.
2. Configure the mailbox interrupt mask register (CANMIM) to mask out the mailboxes that should not cause an interrupt. This register could be set to 0xFFFFFFFF, which enables all mailbox interrupts. Mailboxes that are not used do not cause any interrupts anyhow.
3. Now configure the CANGIM register. The flags AAIM, WDIM, WUIM, BOIM, EPIM, and WLIM should always be set (enabling these interrupts). The GIL bit (CANGIM.2) can be cleared to have the global interrupts on another level than the mailbox interrupts. Both the I1EN (CANGIM.1) and I0EN (CANGIM.0) flags should be set to enable both interrupt lines. The flag RMLIM (CANGIM.11) can also be set depending on the load of the CPU.

This configuration puts all mailbox interrupts on line 1 and all system interrupts on line 0. Thus, the CPU can handle all system interrupts (which are always serious) with high priority, and the mailbox interrupts (on the other line) with a lower priority. All messages with a high priority can also be directed to the interrupt line 0.



#### 16.7.4.3.2 Handling Mailbox Interrupts

There are three interrupt flags for mailbox interrupts. These are listed below:

GMIF0/GMIF1: One of the objects has received or transmitted a message. The number of the mailbox is in MIV0/MIV1 (GIF0.4-0/GIF1.4-0). The normal handling routine is as follows:

1. Do a half-word read on the GIF register that caused the interrupt. If the value is negative, a mailbox caused the interrupt. Otherwise, check the AAIF0/AAIF1 (GIF0.14/GIF1.14) bit (abort-acknowledge interrupt flag) or the RMLIF0/RMLIF1 (GIF0.11/GIF1.11) bit (receive-message-lost interrupt flag). Otherwise, a system interrupt has occurred. In this case, each of the system-interrupt flags must be checked.
2. If the RMLIF (GIF0.11) flag caused the interrupt, the message in one of the mailboxes has been overwritten by a new one. This should not happen in normal operation. The CPU needs to clear that flag by writing a 1 to it. The CPU must check the receive-message-lost register (RML) to find out which mailbox caused that interrupt. Depending on the application, the CPU has to decide what to do next. This interrupt comes together with an GMIF0/GMIF1 interrupt.
3. If the AAIF (GIF.14) flag caused the interrupt, a send transmission operation was aborted by the CPU. The CPU should check the abort acknowledge register (AA.31-0) to find out which mailbox caused the interrupt and send that message again if requested. The flag must be cleared by writing a 1 to it.
4. If the GMIF0/GMIF1 (GIF0.15/GIF1.15) flag caused the interrupt, the mailbox number that caused the interrupt can be read from the MIV0/MIV1 (GIF0.4-0/GIF1.4-0) field. This vector can be used to jump to a location where that mailbox is handled. If it is a receive mailbox, the CPU should read the data as described above and clear the RMP.31-0 flag by writing a 1 to it. If it is a send mailbox, no further action is required, unless the CPU needs to send more data. In this case, the normal send procedure as described above is necessary. The CPU needs to clear the transmit acknowledge bit (TA.31-0) by writing a 1 to it.

#### 16.7.4.3.3 Interrupt Handling Sequence

In order for the CPU core to recognize and service CAN interrupts, the following must be done in any CAN ISR:

1. The flag bit in the CANGIF0/CANGIF1 register which caused the interrupt in the first place must be cleared. There are two kinds of bits in these registers:
  - a. the very same bit that needs to be cleared. The following bits fall under this category: TCOFn, WDIFn, WUIFn, BOIFn, EPIFn, WLIFn
  - b. The second group of bits are cleared by writing to the corresponding bits in the associated registers. The following bits fall under this category: MTOFn, GMIFn, AAIFn, RMLIFn
    - i. The MTOFn bit is cleared by clearing the corresponding bit in the TOS register. For example, if mailbox 27 caused a time-out condition due to which the MTOFn bit was set, the ISR (after taking appropriate actions for the timeout condition) needs to clear the TOS27 bit in order to clear the MTOFn bit.
    - ii. The GMIFn bit is cleared by clearing the appropriate bit in TA or RMP register. For example, if mailbox 19 has been configured as a transmit mailbox and has completed a transmission, TA19 is set, which in turn sets GMIFn. The ISR (after taking appropriate actions) needs to clear the TA19 bit in order to clear the GMIFn bit. If mailbox 8 has been configured as a receive mailbox and has completed a reception, RMP8 is set, which in turn sets GMIFn. The ISR (after taking appropriate actions) needs to clear the RMP8 bit in order to clear the GMIFn bit.
    - iii. The AAIFn bit is cleared by clearing the corresponding bit in the AA register. For example, if mailbox 13's transmission was aborted due to which the AAIFn bit was set, the ISR needs to clear the AA13 bit in order to clear the AAIFn bit.
    - iv. The RMLIFn bit is cleared by clearing the corresponding bit in the RMP register. For example, if mailbox 13's message was overwritten due to which the RMLIFn bit was set, the ISR needs to clear the RMP13 bit in order to clear the RMLIFn bit.

2. The PIEACK bit corresponding to the CAN module must be written with a 1, which can be accomplished with the following C language statement:

```
PieCtrlRegs.PIEACK.bit.ACK9 = 1; // Enables PIE to drive a pulse into the CPU
```

3. The interrupt line into the CPU corresponding to the CAN module must be enabled, which can be accomplished with the following C language statement:

```
IER |= 0x0100; // Enable INT9
```

4. The CPU interrupts must be enabled globally by clearing the INTM bit.

### 16.7.5 CAN Power-Down Mode

A local power-down mode has been implemented where the CAN module internal clock is deactivated by the CAN module itself.

#### 16.7.5.1 Entering and Exiting Local Power-Down Mode

During local power-down mode, the clock of the CAN module is turned off (by the CAN module itself) and only the wake-up logic is still active. The other peripherals continue to operate normally.

The local power-down mode is requested by writing a 1 to the PDR (CANMC.11) bit, allowing transmission of any packet in progress to complete. After the transmission is completed, the status bit PDA (CANES.3) is set. This confirms that the CAN module has entered the power-down mode.

The value read on the CANES register is 0x08 (PDA bit is set). All other register read accesses deliver the value 0x00.

The module leaves the local power-down mode when the PDR bit is cleared or if any bus activity is detected on the CAN bus line (if the wake-up-on bus activity is enabled).

The automatic wake-up-on bus activity can be enabled or disabled with the configuration bit WUBA of CANMC register. If there is any activity on the CAN bus line, the module begins its power-up sequence. The module waits until it detects 11 consecutive recessive bits on the CANRX pin and then it goes bus-active.

---

#### Note

The first CAN message, which initiates the bus activity, cannot be received. This means that the first message received in power-down and automatic wake-up mode is lost.

---

After leaving the sleep mode, the PDR and PDA bits are cleared. The CAN error counters remain unchanged.

If the module is transmitting a message when the PDR bit is set, the transmission is continued until a successful transmission, a lost arbitration, or an error condition on the CAN bus line occurs. Then, the PDA bit is activated so the module causes no error condition on the CAN bus line.

To implement the local power-down mode, two separate clocks are used within the CAN module. One clock stays active all the time to ensure power-down operation (that is, the wake-up logic and the write and read access to the PDA (CANES.3) bit). The other clock is enabled depending on the setting of the PDR bit.



### 16.7.5.2 Precautions for Entering and Exiting Device Low-Power Modes (LPM)

The 28x device features two low-power modes, STANDBY and HALT, in which the peripheral clocks are turned off. Since the CAN module is connected to multiple nodes across a network, you must take care before entering and exiting device low-power modes such as STANDBY and HALT. A CAN packet must be received in full by all the nodes; therefore, if transmission is aborted half-way through the process, the aborted packet would violate the CAN protocol resulting in all the nodes generating error frames. The node exiting LPM should do so unobtrusively. For example, if a node exits LPM when there is traffic on the CAN bus it could “see” a truncated packet and disturb the bus with error frames.

The following points must be considered before entering a device low-power mode:

1. The CAN module has completed the transmission of the last packet requested.
2. The CAN module has signaled to the CPU that it is ready to enter LPM.

In other words, device low-power modes should be entered into only after putting the CAN module in local power-down mode.

### 16.7.5.3 Enabling or Disabling Clock to the CAN Module

The CAN module cannot be used unless the clock to the module is enabled. It is enabled or disabled by using bit 14 of the PCLKCR0 register for eCAN-A module. This bit is useful in applications that do not use the CAN module at all. In such applications, the CAN module clock can be permanently turned off, resulting in some power saving. This bit is not intended to put the CAN module in low-power mode and should not be used for that purpose. Like all other peripherals, clock to the CAN module is disabled upon reset.

### 16.7.5.4 Possible Failure Modes External to the CAN Controller Module

This section lists some potential failure modes in a CAN based system. The failure modes listed are external to the CAN controller and hence, need to be evaluated at the system level.

- CAN\_H and CAN\_L shorted together
- CAN\_H and/or CAN\_L shorted to ground
- CAN\_H and/or CAN\_L shorted to supply
- Failed CAN transceiver
- Electrical disturbance on CAN bus

## 16.8 Acceptance Filter

The identifier of the incoming message is first compared to the message identifier of the mailbox (which is stored in the mailbox). Then, the appropriate acceptance mask is used to mask out the bits of the identifier that should not be compared.

In the SCC-compatible mode, the global acceptance mask (GAM) is used for the mailboxes 6 to 15. An incoming message is stored in the highest numbered mailbox with a matching identifier. If there is no matching identifier in mailboxes 15 to 6, the incoming message is compared to the identifier stored in mailboxes 5 to 3 and then 2 to 0.

The mailboxes 5 to 3 use the local-acceptance mask LAM(3) of the SCC registers. The mailboxes 2 to 0 use the local-acceptance mask LAM(0) of the SCC registers. For specific uses, see [Section 16.8.1](#).

To modify the global acceptance mask register (CANGAM) and the two local-acceptance mask registers of the SCC, the CAN module must be set in the initialization mode (see [Section 16.7.1](#)).

In eCAN mode, each of the 32 mailboxes has its own local-acceptance mask LAM(0) to LAM(31). There is no global-acceptance mask in the eCAN mode.

The selection of the mask to be used for the comparison depends on which mode (SCC or eCAN) is used.

### 16.8.1 Local-Acceptance Masks (CANLAM)

The local-acceptance filtering allows the user to locally mask (don't care) any identifier bits of the incoming message.

In the SCC, the local-acceptance-mask register LAM(0) is used for mailboxes 2 to 0. The local-acceptance-mask register LAM(3) is used for mailboxes 5 to 3. For the mailboxes 6 to 15, the global-acceptance-mask (CANGAM) register is used.

After a hardware or a software reset of the SCC module, CANGAM is reset to zero. After a reset of the eCAN, the LAM registers are not modified.

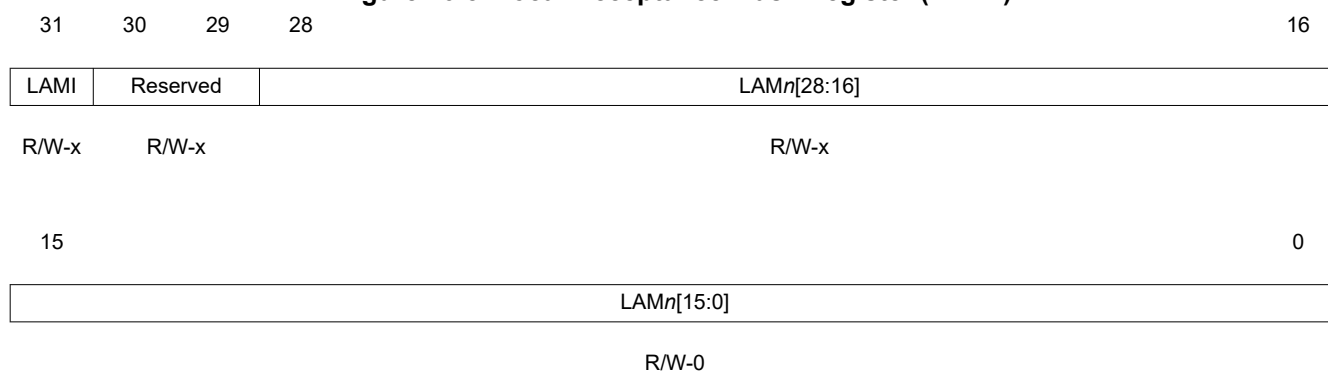
In eCAN mode, each mailbox (0 to 31) has its own mask register, LAM(0) to LAM(31). An incoming message is stored in the highest numbered mailbox with a matching identifier.

You can locally mask any identifier bits of the incoming message. A 1 value means "don't care" or accept either a 0 or 1 for that bit position. A 0 value means that the incoming bit value must match the corresponding bit in the message identifier.

If the local-acceptance mask identifier extension bit is set (LAMI = 1 => don't care) standard and extended frames can be received. An extended frame uses all 29 bits of the identifier stored in the mailbox and all 29 bits of local-acceptance mask register for the filter. For a standard frame only the first eleven bits (bit 28 to 18) of the identifier and the local-acceptance mask are used.

If the local-acceptance mask identifier extension bit is reset (LAMI = 0), the identifier extension bit stored in the mailbox determines the messages that are received.

**Figure 16-8. Local-Acceptance-Mask Register (LAM<sub>n</sub>)**



LEGEND: R/W = Read/Write; -n = value after reset (x:Undefined)

**Table 16-7. Local-Acceptance-Mask Register (LAM<sub>n</sub>) Field Descriptions**

| Bit   | Field     | Value | Description                                                                                                                                                                                                                                                                                                                                          |
|-------|-----------|-------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 31    | LAMI      | 1     | Local-acceptance-mask identifier extension bit                                                                                                                                                                                                                                                                                                       |
|       |           | 0     | Standard and extended frames can be received. In case of an extended frame, all 29 bits of the identifier are stored in the mailbox and all 29 bits of the local-acceptance mask register are used for the filter. In case of a standard frame, only the first eleven bits (bits 28 to 18) of the identifier and the local-acceptance mask are used. |
| 30-29 | Reserved  |       | The identifier extension bit stored in the mailbox determines which messages shall be received.                                                                                                                                                                                                                                                      |
| 28-0  | LAM[28:0] | 1     | Reads are undefined and writes have no effect.                                                                                                                                                                                                                                                                                                       |
|       |           | 0     | These bits enable the masking of any identifier bit of an incoming message.                                                                                                                                                                                                                                                                          |
|       |           | 1     | Accept a 0 or a 1 (don't care) for the corresponding bit of the received identifier.                                                                                                                                                                                                                                                                 |
|       |           | 0     | Received identifier bit value must match the corresponding identifier bit of the MSGID register.                                                                                                                                                                                                                                                     |

## 16.9 eCAN Registers

This chapter describes the Enhanced Controller Area Network (eCAN) registers.

### 16.9.1 Mailbox Enable Register (CANME)

This register is used to enable/disable individual mailboxes.

**Figure 16-9. Mailbox-Enable Register (CANME)**

31

0

|             |
|-------------|
| CANME[31:0] |
|-------------|

R/W-0

LEGEND: R/W = Read/Write; -n = value after reset

**Table 16-8. Mailbox-Enable Register (CANME) Field Descriptions**

| Bit  | Field       | Value | Description                                                                                                                                                                                                                                                                                                                                                                                                                             |
|------|-------------|-------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 31:0 | CANME[31:0] | 1     | Mailbox enable bits. After power-up, all bits in CANME are cleared. Disabled mailboxes can be used as additional memory for the CPU.<br>The corresponding mailbox is enabled. The mailbox must be disabled before writing to the contents of any identifier field. If the corresponding bit in CANME is set, the write access to the identifier of a mailbox is denied and an interrupt (write-denied interrupt) generated, if enabled. |
|      |             | 0     | The corresponding mailbox RAM area is disabled for the eCAN; however, it is accessible to the CPU as normal RAM.                                                                                                                                                                                                                                                                                                                        |

### 16.9.2 Mailbox-Direction Register (CANMD)

This register is used to configure a mailbox for transmit or receive operation.

**Figure 16-10. Mailbox-Direction Register (CANMD)**

31

0

|             |
|-------------|
| CANMD[31:0] |
|-------------|

R/W-0

LEGEND: R/W = Read/Write; -n = value after reset

**Table 16-9. Mailbox-Direction Register (CANMD) Field Descriptions**

| Bit  | Field       | Value | Description                                                                                                                    |
|------|-------------|-------|--------------------------------------------------------------------------------------------------------------------------------|
| 31:0 | CANMD[31:0] | 1     | Mailbox direction bits. After power-up, all bits are cleared.<br>The corresponding mailbox is configured as a receive mailbox. |
|      |             | 0     | The corresponding mailbox is configured as a transmit mailbox.                                                                 |

### 16.9.3 Transmission-Request Set Register (CANTRS)

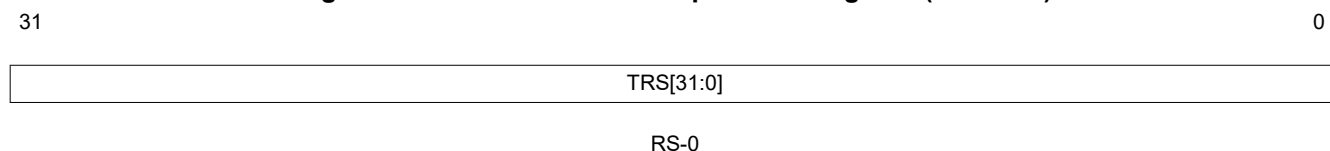
When mailbox  $n$  is ready to be transmitted, the CPU should set the TRS[ $n$ ] bit to 1 to start the transmission.

These bits are normally set by the CPU and cleared by the CAN module logic. The CAN module can set these bits for a remote frame request. These bits are reset when a transmission is successful or aborted. If a mailbox is configured as a receive mailbox, the corresponding bit in CANTRS is ignored unless the receive mailbox is configured to handle remote frames. The TRS[ $n$ ] bit of a receive mailbox is not ignored if the RTR bit is set. Therefore, a receive mailbox (whose RTR is set) can send a remote frame if its TRS bit is set. Once the remote frame is sent, the TRS[ $n$ ] bit is cleared by the CAN module. Therefore, the same mailbox can be used to request a data frame from another mode. If the CPU tries to set a bit while the eCAN module tries to clear it, the bit is set.

Setting CANTRS[ $n$ ] causes the particular message  $n$  to be transmitted. Several bits can be set simultaneously. Therefore, all messages with the TRS bit set are transmitted in turn, starting with the mailbox having the highest mailbox number (= highest priority), unless TPL bits dictate otherwise.

The bits in CANTRS are set by writing a 1 from the CPU. Writing a 0 has no effect. After power up, all bits are cleared.

**Figure 16-11. Transmission-Request Set Register (CANTRS)**



LEGEND: RS = Read/Set; - $n$  = value after reset

**Table 16-10. Transmission-Request Set Register (CANTRS) Field Descriptions**

| Bit  | Field     | Value | Description                                                                                                                                                                           |
|------|-----------|-------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 31:0 | TRS[31:0] | 1     | Transmit-request-set bits<br>Setting TRS $n$ commences the transmission of the message in that mailbox. Several bits can be set simultaneously with all messages transmitted in turn. |
|      |           | 0     | No operation                                                                                                                                                                          |

#### 16.9.4 Transmission-Request-Reset Register (CANTRR)

These bits can only be set by the CPU and reset by the internal logic. These bits are reset when a transmission is successful or is aborted. If the CPU tries to set a bit while the CAN tries to clear it, the bit is set.

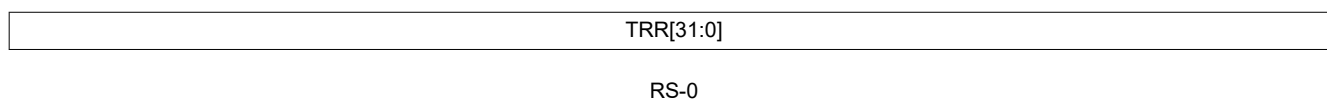
Setting the TRR[*n*] bit of the message object *n* cancels a transmission request if it was initiated by the corresponding bit (TRS[*n*]) and is not currently being processed. If the corresponding message is currently being processed, the bit is reset when a transmission is successful (normal operation) or when an aborted transmission due to a lost arbitration or an error condition is detected on the CAN bus line. When a transmission is aborted, the corresponding status bit (AA.31-0) is set. When a transmission is successful, the status bit (TA.31-0) is set. The status of the transmission request reset can be read from the TRS.31-0 bit.

The bits in CANTRR are set by writing a 1 from the CPU.

**Figure 16-12. Transmission-Request-Reset Register (CANTRR)**

31

0



LEGEND: RS = Read/Set; -*n* = value after reset

**Table 16-11. Transmission-Request-Reset Register (CANTRR) Field Descriptions**

| Bit  | Field     | Value | Description                                                    |
|------|-----------|-------|----------------------------------------------------------------|
| 31:0 | TRR[31:0] | 1     | Transmit-request-reset bits                                    |
|      |           | 1     | Setting TRR <sub><i>n</i></sub> cancels a transmission request |
|      |           | 0     | No operation                                                   |

#### 16.9.5 Transmission-Acknowledge Register (CANTA)

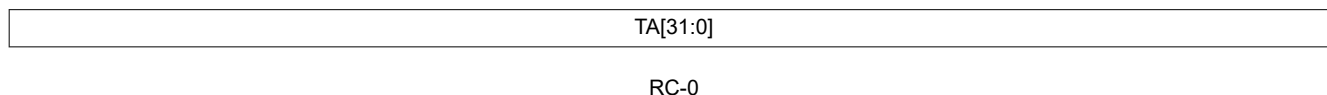
If the message of mailbox *n* was sent successfully, the bit TA[*n*] is set. This also sets the GMIF0/GMIF1 (CANGIF0.15/CANGIF1.15) bit if the corresponding interrupt mask bit in the CANMIM register is set. The GMIF0/GMIF1 bit initiates an interrupt.

The CPU resets the bits in CANTA by writing a 1. This also clears the interrupt if an interrupt has been generated. Writing a 0 has no effect. If the CPU tries to reset the bit while the CAN tries to set it, the bit is set. After power-up, all bits are cleared.

**Figure 16-13. Transmission-Acknowledge Register (CANTA)**

31

0



LEGEND: RC = Read/Clear; -*n* = value after reset

**Table 16-12. Transmission-Acknowledge Register (CANTA) Field Descriptions**

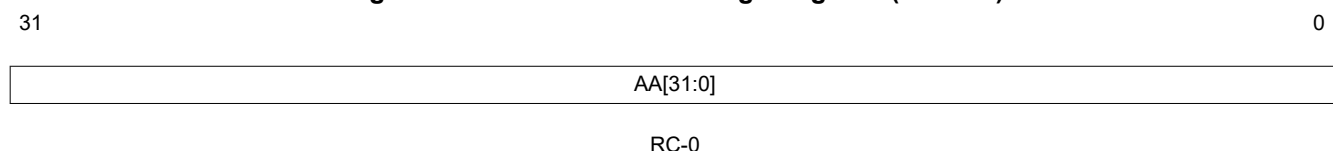
| Bit  | Field    | Value | Description                                                                                        |
|------|----------|-------|----------------------------------------------------------------------------------------------------|
| 31:0 | TA[31:0] | 1     | Transmit-acknowledge bits                                                                          |
|      |          | 1     | If the message of mailbox <i>n</i> is sent successfully, the bit <i>n</i> of this register is set. |
|      |          | 0     | The message is not sent.                                                                           |

### 16.9.6 Abort-Acknowledge Register (CANAA)

If the transmission of the message in mailbox  $n$  was aborted, the bit AA[ $n$ ] is set and the AAIF (CANGIF.14) bit is set, which may generate an interrupt if enabled.

The bits in CANAA are reset by writing a 1 from the CPU. Writing a 0 has no effect. If the CPU tries to reset a bit and the CAN tries to set the bit at the same time, the bit is set. After power-up all bits are cleared.

**Figure 16-14. Abort-Acknowledge Register (CANAA)**



LEGEND: RC = Read/Clear; - $n$  = value after reset

**Table 16-13. Abort-Acknowledge Register (CANAA) Field Descriptions**

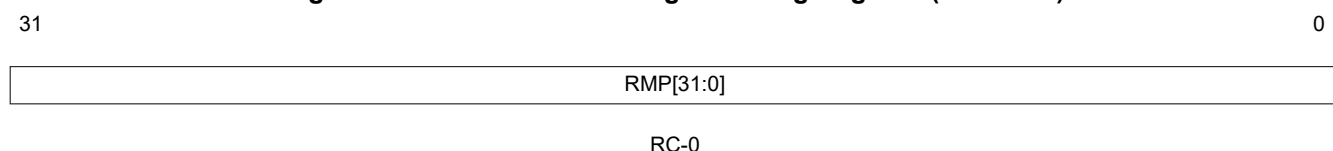
| Bit  | Field    | Value | Description                                                                                        |
|------|----------|-------|----------------------------------------------------------------------------------------------------|
| 31:0 | AA[31:0] |       | Abort-acknowledge bits                                                                             |
|      |          | 1     | If the transmission of the message in mailbox $n$ is aborted, the bit $n$ of this register is set. |
|      |          | 0     | The transmission is not aborted.                                                                   |

### 16.9.7 Received-Message-Pending Register (CANRMP)

If mailbox  $n$  contains a received message, the bit RMP[ $n$ ] of this register is set. These bits can be reset only by the CPU and set by the internal logic. A new incoming message overwrites the stored one if the OPC[ $n$ ](OPC.31-0) bit is cleared, otherwise the next mailboxes are checked for a matching ID. If a mailbox is overwritten, the corresponding status bit RML[ $n$ ] is set. The bits in the CANRMP and the CANRML registers are cleared by a write to register CANRMP, with a 1 at the corresponding bit location. If the CPU tries to reset a bit and the CAN tries to set the bit at the same time, the bit is set.

The bits in the CANRMP register can set GMIF0/GMIF1 (GIF0.15/GIF1.15) if the corresponding interrupt mask bit in the CANMIM register is set. The GMIF0/GMIF1 bit initiates an interrupt.

**Figure 16-15. Received-Message-Pending Register (CANRMP)**



LEGEND: RC = Read/Clear; - $n$  = value after reset

**Table 16-14. Received-Message-Pending Register (CANRMP) Field Descriptions**

| Bit  | Field     | Value | Description                                                                         |
|------|-----------|-------|-------------------------------------------------------------------------------------|
| 31:0 | RMP[31:0] |       | Received-message-pending bits                                                       |
|      |           | 1     | If mailbox $n$ contains a received message, bit RMP[ $n$ ] of this register is set. |
|      |           | 0     | The mailbox does not contain a message.                                             |

### 16.9.8 Received-Message-Lost Register (CANRML)

An RML[*n*] bit is set if an old message has been overwritten by a new one in mailbox *n*. These bits can only be reset by the CPU, and set by the internal logic. The bits can be cleared by a write access to the CANRMP register with a 1 at the corresponding bit location. If the CPU tries to reset a bit and the CAN tries to set the bit at the same time, the bit is set. The CANRML register is not changed if the OPC[*n*] (OPC.31-0) bit is set.

If one or more of the bits in the CANRML register are set, the RMLIF (GIF0.11/ GIF1.11) bit is also set. This can initiate an interrupt if the RMLIM (GIM.11) bit is set.

**Figure 16-16. Received-Message-Lost Register (CANRML)**

31 0

|           |
|-----------|
| RML[31:0] |
|-----------|

R-0

LEGEND: R = Read; -*n* = value after reset

**Table 16-15. Received-Message-Lost Register (CANRML) Field Descriptions**

| Bit  | Field     | Value | Description                                                                                       |
|------|-----------|-------|---------------------------------------------------------------------------------------------------|
| 31:0 | RML[31:0] | 1     | Received-message-lost bits                                                                        |
|      |           | 0     | An old unread message has been overwritten by a new one in that mailbox.                          |
|      |           |       | No message was lost.                                                                              |
|      |           |       | Note: The RML <sub><i>n</i></sub> bit is cleared by clearing the set RMP <sub><i>n</i></sub> bit. |

### 16.9.9 Remote-Frame-Pending Register (CANRFP)

Whenever a remote frame request is received by the CAN module, the corresponding bit RFP[*n*] in the remote frame pending register is set. If a remote frame is stored in a receive mailbox (AAM=0, CANMD=1), the RFP<sub>*n*</sub> bit will not be set.

To prevent an auto-answer mailbox from replying to a remote frame request, the CPU has to clear the RFP[*n*] flag and the TRS[*n*] bit by setting the corresponding transmission request reset bit TRR[*n*]. The AAM bit can also be cleared by the CPU to stop the module from sending the message.

If the CPU tries to reset a bit and the CAN module tries to set the bit at the same time, the bit is not set. The CPU cannot interrupt an ongoing transfer.

**Figure 16-17. Remote-Frame-Pending Register (CANRFP)**

31 0

|          |
|----------|
| RFP.31:0 |
|----------|

RC-0

LEGEND: RC = Read/Clear; -*n* = value after reset

**Table 16-16. Remote-Frame-Pending Register (CANRFP) Field Descriptions**

| Bit  | Field    | Value | Description                                                                                                                                                                                                                                                                                                       |
|------|----------|-------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 31:0 | RFP.31:0 |       | Remote-frame-pending register.<br>For a receive mailbox, $RFP_n$ is not set if a remote frame is received and $TRSn$ is not affected.<br>For a transmit mailbox, $RFP_n$ is set if a remote frame is received and $TRSn$ is set if AAM of the mailbox is 1. The ID of the mailbox must match the remote frame ID. |
|      |          | 1     | A remote-frame request was received by the module.                                                                                                                                                                                                                                                                |
|      |          | 0     | No remote-frame request was received. The register is cleared by the CPU.                                                                                                                                                                                                                                         |

### 16.9.9.1 Handling of Remote Frames

If a remote frame is received (the incoming message has the RTR bit set), the CAN module compares the identifier to all identifiers of the mailboxes using the appropriate masks starting at the highest mailbox number in descending order.

In the case of a matching identifier (with the message object configured as send mailbox and AAM (MSGID.29) in this message object set) this message object is marked as to be sent ( $TRSn$  is set).

In case of a matching identifier with the mailbox configured as a send mailbox and bit AAM in this mailbox is not set, this message is not received in that mailbox.

After finding a matching identifier in a mailbox no further compare is done.

With a matching identifier and the message object configured as receive mailbox, this message is handled like a data frame and the corresponding bit in the receive message pending (CANRMP) register is set. The CPU then has to decide how to handle this situation. For information about the CANRMP register, see [Section 16.9.7](#).

For the CPU to change the data in a mailbox that is configured as a remote frame mailbox (AAM set) it has to set the mailbox number and the change data request (CDR) bit [CANMC.8] first. The CPU can then write the new data and clear the CDR bit to indicate to the eCAN that the access is finished. Until the CDR bit is cleared, the transmission of this mailbox is not permitted. Therefore, the newest data is sent.

To change the identifier in that mailbox, the mailbox must be disabled first ( $CANME_n = 0$ ).

For the CPU to request data from another node it configures the mailbox as a receive mailbox and sets the TRS bit. In this case the module sends a remote frame request and receives the data frame in the same mailbox that sent the request. Therefore, only one mailbox is necessary to do a remote request. Note that the CPU must set RTR (MSGCTRL.4) to enable a remote frame transmission. Once the remote frame is sent, the TRS bit of the mailbox is cleared by CAN. In this case, bit  $TAn$  will not be set for that mailbox.

The behavior of the message object  $n$  is configured with  $CANMD[n]$  ( $CANMD.31-0$ ), the AAM (MSGID.29), and RTR (MSGCTRL.4). It shows how to configure a message object according to the desired behavior.

To summarize, a message object can be configured with four different behaviors:

1. A transmit message object is only able to transmit messages.
2. A receive message object is only able to receive messages.
3. A remote-request message object is able to transmit a remote request frame and to wait for the corresponding data frame.
4. A auto-reply message object is able to transmit a data frame whenever a remote request frame is received for the corresponding identifier.

#### Note

When a remote transmission request is successfully transmitted with a message object configured in request mode, the CANTA register is not set and no interrupt is generated. When the remote reply message is received, the behavior of the message object is the same as a message object configured in receive mode.

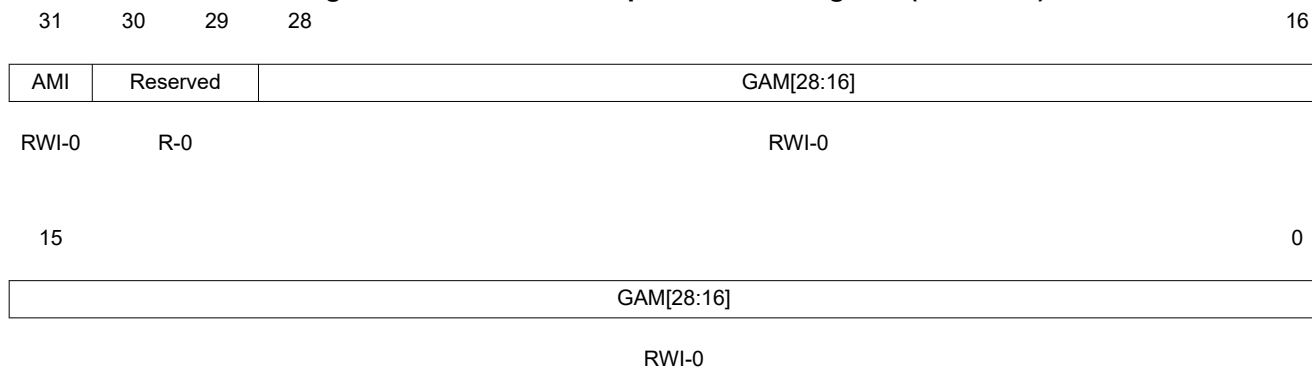


### 16.9.10 Global Acceptance Mask Register (CANGAM)

The global-acceptance mask is used by the eCAN in SCC mode. The global-acceptance mask is used for the mailboxes 6 to 15 if the AME bit (MSGID.30) of the corresponding mailbox is set. A received message is only stored in the first mailbox with a matching identifier.

The global-acceptance mask is used for the mailboxes 6 to 15 of the SCC.

### Figure 16-18. Global Acceptance Mask Register (CANGAM)



LEGEND: RWI = Read at any time, write during initialization mode only; -n = value after reset

### Table 16-17. Global Acceptance Mask Register (CANGAM) Field Descriptions

| Bit   | Field    | Value | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|-------|----------|-------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 31    | AMI      | 1     | Global Acceptance-mask identifier extension bit<br><br>Standard and extended frames can be received. In case of an extended frame, all 29 bits of the identifier are stored in the mailbox and all 29 bits of global acceptance mask register are used for the filter. In case of a standard frame, only the first eleven bits (bit 28 to 18) of the identifier and the global acceptance mask are used.                                                                                    |
|       |          | 0     | The IDE bit of the receive mailbox is a "don't care" and is overwritten by the IDE bit of the transmitted message. The filtering criterion must be satisfied in order to receive a message. The number of bits to be compared is a function of the value of the IDE bit of the transmitted message.<br><br>The identifier extension bit stored in the mailbox determines which messages shall be received. The IDE bit of the receive mailbox determines the number of bits to be compared. |
| 30:29 | Reserved |       | Reads are undefined and writes have no effect.                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| 28:0  | GAM 28:0 |       | Global-acceptance mask. These bits allow any identifier bits of an incoming message to be masked. A value of "0" for a bit position means received identifier bit value must match the corresponding identifier bit of the MSGID register. A value of "1" means "don't care".                                                                                                                                                                                                               |

### 16.9.11 Master Control Register (CANMC)

This register is used to control the settings of the CAN module. Some bits of the CANMC register are EALLOW protected. For read/write operations, only 32-bit access is supported.

**Figure 16-19. Master Control Register (CANMC)**

|          |        |        |        |        |        |        |        |    |       |
|----------|--------|--------|--------|--------|--------|--------|--------|----|-------|
| 31       |        |        |        |        |        |        |        | 17 | 16    |
| Reserved |        |        |        |        |        |        |        |    | SUSP  |
| R-0      |        |        |        |        |        |        |        |    | R/W-0 |
| 15       | 14     | 13     | 12     | 11     | 10     | 9      | 8      |    |       |
| MBCC     | TCC    | SCB    | CCR    | PDR    | DBO    | WUBA   | CDR    |    |       |
| R/WP-0   | SP-x   | R/WP-0 | R/WP-1 | R/WP-0 | R/WP-0 | R/WP-0 | R/WP-0 |    |       |
| 7        | 6      | 5      | 4      |        |        |        |        |    | 0     |
| ABO      | STM    | SRES   | MBNR   |        |        |        |        |    |       |
| R/WP-0   | R/WP-0 | R/S-0  |        |        |        |        |        |    | R/W-0 |

LEGEND: R = Read, WP = Write in EALLOW mode only, S = Set in EALLOW mode only; -n = value after reset; x = Indeterminate  
Note: eCAN only, reserved in the SCC

**Table 16-18. Master Control Register (CANMC) Field Descriptions**

| Bit   | Field    | Value | Description                                                                                                                                                                                                   |
|-------|----------|-------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 31:17 | Reserved |       | Reads are undefined and writes have no effect.                                                                                                                                                                |
| 16    | SUSP     | 1     | SUSPEND. This bit determines the action of the CAN module in SUSPEND (emulation stop such as breakpoint or single stepping).                                                                                  |
|       |          | 0     | FREE mode. The peripheral continues to run in SUSPEND. The node would participate in CAN communication normally (sending acknowledge, generating error frames, transmitting/receiving data) while in SUSPEND. |
|       |          | 0     | SOFT mode. The peripheral shuts down during SUSPEND after the current transmission is complete.                                                                                                               |
| 15    | MBCC     | 1     | Mailbox timestamp counter clear bit. This bit is reserved in SCC mode and it is EALLOW protected.                                                                                                             |
|       |          | 0     | The time stamp counter is reset to 0 after a successful transmission or reception of mailbox 16.                                                                                                              |
|       |          | 0     | The time stamp counter is not reset.                                                                                                                                                                          |
| 14    | TCC      | 1     | Time stamp counter MSB clear bit. This bit is reserved in SCC mode and it is EALLOW protected.                                                                                                                |
|       |          | 0     | The MSB of the time stamp counter is reset to 0. The TCC bit is reset after one clock cycle by the internal logic.                                                                                            |
|       |          | 0     | The time stamp counter is not changed.                                                                                                                                                                        |
| 13    | SCB      | 1     | SCC compatibility bit. This bit is EALLOW protected.                                                                                                                                                          |
|       |          | 0     | Select eCAN mode.                                                                                                                                                                                             |
|       |          | 0     | The eCAN is in SCC mode. Only mailboxes 15 to 0 can be used. Timestamping feature is not available.                                                                                                           |

**Table 16-18. Master Control Register (CANMC) Field Descriptions (continued)**

| Bit | Field | Value | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|-----|-------|-------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 12  | CCR   | 1     | Change-configuration request. This bit is EALLOW protected.<br><br>The CPU requests write access to the configuration register CANBTC and the acceptance mask registers (CANGAM, LAM[0], and LAM[3]) of the SCC. After setting this bit, the CPU must wait until the CCE flag of CANES register is at 1 before proceeding to configure the CANBTC register.<br><br>The CCR bit will also be set upon a bus-off condition, if the ABO bit is not set. The BO condition can be exited by clearing this bit (after 128 * 11 consecutive recessive bits on the bus).                                                                                                                                                                                                                                   |
|     |       | 0     | The CPU requests normal operation. This can be done only after the configuration register CANBTC was set to the allowed values. It also exits the bus-off state after the obligatory bus-off recovery sequence.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| 11  | PDR   | 1     | Power down mode request. This bit is automatically cleared by the eCAN module upon wakeup from low-power mode. This bit is EALLOW protected.<br><br>The local power-down mode is requested.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|     |       | 0     | The local power-down mode is not requested (normal operation).<br><br><b>Note:</b> If an application sets the TRSn bit for a mailbox and then immediately sets the PDR bit, the CAN module goes into LPM without transmitting the data frame. This is because it takes about 80 CPU cycles for the data to be transferred from the mailbox RAM to the transmit buffer. Therefore, the application has to ensure that any pending transmission has been completed before writing to the PDR bit. The TAn bit could be polled to ensure completion of transmission.                                                                                                                                                                                                                                  |
| 10  | DBO   | 1     | Data byte order. This bit selects the byte order of the message data field. This bit is EALLOW protected.<br><br>The data is received or transmitted least significant byte first.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|     |       | 0     | The data is received or transmitted most significant byte first.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| 9   | WUBA  | 1     | Wake up on bus activity. This bit is EALLOW protected.<br><br>The module leaves the power-down mode after detecting any bus activity.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|     |       | 0     | The module leaves the power-down mode only after writing a 0 to the PDR bit.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| 8   | CDR   | 1     | Change data field request. This bit allows fast data message update.<br><br>The CPU requests write access to the data field of the mailbox specified by the MBNR.4:0 field (CANMC.4-0). The CPU must clear the CDR bit after accessing the mailbox. The module does not transmit that mailbox content while the CDR is set. This is checked by the state machine before and after it reads the data from the mailbox to store it in the transmit buffer.<br><br><b>Note:</b> Once the TRS bit is set for a mailbox and then data is changed in the mailbox using the CDR bit, the CAN module fails to transmit the new data and transmits the old data instead. To avoid this, reset transmission in that mailbox using the TRRn bit and set the TRSn bit again. The new data is then transmitted. |
|     |       | 0     | The CPU requests normal operation.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| 7   | ABO   | 1     | Auto bus on. This bit is EALLOW protected.<br><br>After the bus-off state, the module goes back automatically into bus-on state after 128 * 11 recessive bits have been monitored.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|     |       | 0     | The bus-off state may only be exited after 128 * 11 consecutive recessive bits on the bus and after having cleared the CCR bit.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| 6   | STM   | 1     | Self test mode. This bit is EALLOW protected.<br><br>The module is in self-test mode. In this mode, the CAN module generates its own acknowledge (ACK) signal, thus enabling operation without a bus connected to the module. The message is not sent, but read back and stored in the appropriate mailbox. The MSGID of the received frame is not stored in the MBR in STM.                                                                                                                                                                                                                                                                                                                                                                                                                       |
|     |       | 0     | The module is in normal mode.<br><br><b>Note:</b> In STM, if no MBX has been configured to receive a transmitted frame, then that frame will be stored in MBX0, even if MBX0 has not been configured for receive operations. If LAMs are configured such that some mailboxes can receive and store data frames, then a data frame that does not satisfy the acceptance mask filtering criterion for any receive mailbox will be lost.                                                                                                                                                                                                                                                                                                                                                              |

**Table 16-18. Master Control Register (CANMC) Field Descriptions (continued)**

| Bit | Field    | Value  | Description                                                                                                                                                                                                                                                                                              |
|-----|----------|--------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 5   | SRES     |        | This bit can only be written and is always read as zero.                                                                                                                                                                                                                                                 |
|     |          | 1      | A write access to this bit causes a software reset of the module (all parameters, except the protected registers, are reset to their default values). The mailbox contents and the error counters are not modified. Pending and ongoing transmissions are canceled without perturbing the communication. |
|     |          | 0      | 0 No effect                                                                                                                                                                                                                                                                                              |
| 4:0 | MBNR 4:0 | 0 - 31 | Mailbox for which the CPU requests a write access to the data field. This field is used in conjunction with the CDR bit. The bit MBNR.4 is for eCAN only, and is reserved in the SCC mode.                                                                                                               |

### 16.9.11.1 CAN Module Action in SUSPEND

The following points describe the behavior of the module in SUSPEND mode.

1. If there is no traffic on the CAN bus and SUSPEND mode is requested, the node goes into SUSPEND mode.
2. If there is traffic on the CAN bus and SUSPEND mode is requested, the node goes into SUSPEND mode when the ongoing frame is over.
3. If the node was transmitting, when SUSPEND is requested, it goes to SUSPEND state after it gets the acknowledgment. If it does not get an acknowledgment or if there are some other errors, it transmits an error frame and then goes to SUSPEND state. The TEC is modified accordingly. In the second case, that is, it is suspended after transmitting an error frame, the node re-transmits the original frame after coming out of suspended state. The TEC is modified after transmission of the frame accordingly.
4. If the node was receiving, when SUSPEND is requested, it goes to SUSPEND state after transmitting the acknowledgment bit. If there is any error, the node sends an error frame and go to SUSPEND state. The REC is modified accordingly before going to SUSPEND state.
5. If there is no traffic on the CAN bus and SUSPEND removal is requested, the node comes out of SUSPEND state.
6. If there is traffic on the CAN bus and SUSPEND removal is requested, the node comes out after the bus goes to idle. Therefore, a node does not receive any "partial" frame, which could lead to generation of error frames.
7. When the node is suspended, it does not participate in transmitting or receiving any data. Thus, neither acknowledgment bit nor any error frame is sent. TEC and REC are not modified during SUSPEND state.

### 16.9.12 Bit-Timing Configuration Register (CANBTC)

The CANBTC register is used to configure the CAN node with the appropriate network-timing parameters. This register must be programmed before using the CAN module.

This register is write-protected in user mode and can only be written in initialization mode (see [Section 16.7.1](#)).

## Note

To avoid unpredictable behavior of the CAN module, the CANBTC register should never be programmed with values not allowed by the CAN protocol specification and by the bit timing rules listed in [Section 16.7.1.1](#).

### Figure 16-20. Bit-Timing Configuration Register (CANBTC)

[illegible]

LEGEND: RWPI = Read in all modes, write in EALLOW mode during initialization mode only; -n = value after reset

### Table 16-19. Bit-Timing Configuration Register (CANBTC) Field Descriptions

| Bit   | Field                   | Value | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|-------|-------------------------|-------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 31:24 | Reserved                |       | Reads are undefined and writes have no effect.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| 23:16 | BRP <sub>reg</sub> :7:0 |       | <p>Baud rate prescaler. This register sets the prescaler for the baud rate settings. The length of one TQ is defined by:</p> $TQ = \frac{1}{SYSCLKOUT / 2} \times (BRP_{reg} + 1)$ <p>where SYSCLKOUT / 2 is the frequency of the CAN module clock.</p> <p>BRP<sub>reg</sub> denotes the "register value" of the prescaler; that is, value written into bits 23:16 of the CANBTC register. This value is automatically enhanced by 1 when the CAN module accesses it. The enhanced value is denoted by the symbol BRP (BRP = BRP<sub>reg</sub> + 1). BRP is programmable from 1 to 256.</p> <p><b>Note:</b> For the special case of BRP = 1, the Information Processing Time (IPT) is equal to 3 time quanta (TQ). This is not compliant to the ISO 11898 Standard, where the IPT is defined to be less than or equal to 2 TQ. Thus the usage of this mode (BRP<sub>reg</sub> = 0) is not allowed.</p> |
| 15:10 | Reserved                |       | Reads are undefined. Must be written with all zeroes only.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| 9:8   | SJW <sub>reg</sub> :1:0 |       | <p>Synchronization jump width. The parameter SJW indicates, by how many units of TQ a bit is allowed to be lengthened or shortened when resynchronizing.</p> <p>SJW<sub>reg</sub> denotes the "register value" of the "resynchronization jump width;" that is, the value written into bits 9:8 of the CANBTC register. This value is automatically enhanced by 1 when the CAN module accesses it. This enhanced value is denoted by the symbol SJW.</p> $SJW = SJW_{reg} + 1$ <p>SJW is programmable from 1 to 4 TQ. The maximum value of SJW is determined by the minimum value of TSEG2 and 4 TQ.</p> $SJW_{(max)} = \min [4 \text{ TQ}, TSEG2]$                                                                                                                                                                                                                                                     |

**Table 16-19. Bit-Timing Configuration Register (CANBTC) Field Descriptions (continued)**

| Bit | Field                | Value | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|-----|----------------------|-------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 7   | SAM                  |       | <p>This parameter sets the number of samples used by the CAN module to determine the actual level of the CAN bus. When the SAM bit is set, the level determined by the CAN bus corresponds to the result from the majority decision of the last three values. The sample points are at the sample point and twice before with a distance of <math>\frac{1}{2}</math> TQ.</p> <p>1 The CAN module samples three times and make a majority decision. The triple sample mode shall be selected only for bit rate prescale values greater than 4 (BRP &gt; 4).</p> <p>0 The CAN module samples only once at the sampling point.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| 6:3 | TSEG1 <sub>reg</sub> |       | <p>Time segment 1. The length of a bit on the CAN bus is determined by the parameters TSEG1, TSEG2, and BRP. All controllers on the CAN bus must have the same baud rate and bit length. For different clock frequencies of the individual controllers, the baud rate has to be adjusted by the said parameters.</p> <p>This parameter specifies the length of the TSEG1 segment in TQ units. TSEG1 combines PROP_SEG and PHASE_SEG1 segments:</p> $\text{TSEG1} = \text{PROP\_SEG} + \text{PHASE\_SEG1}$ <p>where PROP_SEG and PHASE_SEG1 are the length of these two segments in TQ units.</p> <p>TSEG1<sub>reg</sub> denotes the "register value" of "time segment 1;" that is, the value written into bits 6:3 of the CANBTC register. This value is automatically enhanced by 1 when the CAN module accesses it. This enhanced value is denoted by the symbol TSEG1.</p> $\text{TSEG1} = \text{TSEG1}_{\text{reg}} + 1$ <p>TSEG1 value should be chosen such that TSEG1 is greater than or equal to TSEG2 and IPT. For more information on IPT, see <a href="#">Section 16.7.1.1</a>.</p> |
| 2:0 | TSEG2 <sub>reg</sub> |       | <p>Time Segment 2. TSEG2 defines the length of PHASE_SEG2 segment in TQ units:</p> <p>TSEG2 is programmable in the range of 1 TQ to 8 TQ and has to fulfill the following timing rule:</p> <p>TSEG2 must be smaller than or equal to TSEG1 and must be greater than or equal to IPT.</p> <p>TSEG2<sub>reg</sub> denotes the "register value" of "time segment 2;" that is, the value written into bits 2:0 of the CANBTC register. This value is automatically enhanced by 1 when the CAN module accesses it. This enhanced value is denoted by the symbol TSEG2.</p> $\text{TSEG2} = \text{TSEG2}_{\text{reg}} + 1$                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |

### 16.9.13 Error and Status Register (CANES)

The status of the CAN module is shown by the Error and Status Register (CANES) and the error counter registers, which are described in this section.

The error and status register comprises information about the actual status of the CAN module and displays bus error flags as well as error status flags. If one of these error flags is set, then the current state of all other error flags is frozen, that is, only the first error is stored. In order to update the CANES register subsequently, the error flag which is set has to be acknowledged by writing a 1 to it. This action also clears the flag bit.

**Figure 16-21. Error and Status Register (CANES)**

|          |  |    |      |      |     |      |      |      |      |      |      |
|----------|--|----|------|------|-----|------|------|------|------|------|------|
| 31       |  | 25 | 24   | 23   | 22  | 21   | 20   | 19   | 18   | 17   | 16   |
| Reserved |  |    | FE   | BE   | SA1 | CRCE | SE   | ACKE | BO   | EP   | EW   |
| R-0      |  |    | RC-0 | RC-0 | R-1 | RC-0 | RC-0 | RC-0 | RC-0 | RC-0 | RC-0 |
| 15       |  |    |      |      | 6   | 5    | 4    | 3    | 2    | 1    | 0    |
| Reserved |  |    |      |      |     | SMA  | CCE  | PDA  | Rsvd | RM   | TM   |
| R-0      |  |    |      |      |     | R-0  | R-1  | R-0  | R-0  | R-0  | R-0  |

LEGEND: R = Read; C = Clear; -n = value after reset

**Table 16-20. Error and Status Register (CANES) Field Descriptions**

| Bit   | Field    | Value  | Description                                                                                                                                                                                                                                                                          |
|-------|----------|--------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 31:25 | Reserved |        | Reads are undefined and writes have no effect.                                                                                                                                                                                                                                       |
| 24    | FE       | 1<br>0 | Form error flag<br>A form error occurred on the bus. This means that one or more of the fixed-form bit fields had the wrong level on the bus.<br>No form error detected; the CAN module was able to send and receive correctly.                                                      |
| 23    | BE       | 1<br>0 | Bit error flag<br>The received bit does not match the transmitted bit outside of the arbitration field or during transmission of the arbitration field, a dominant bit was sent but a recessive bit was received.<br>No bit error detected.                                          |
| 22    | SA1      | 1<br>0 | Stuck at dominant error. The SA1 bit is always at 1 after a hardware reset, a software reset, or a Bus-Off condition. This bit is cleared when a recessive bit is detected on the bus.<br>The CAN module never detected a recessive bit.<br>The CAN module detected a recessive bit. |
| 21    | CRCE     | 1<br>0 | CRC error.<br>The CAN module received a wrong CRC.<br>The CAN module never received a wrong CRC.                                                                                                                                                                                     |
| 20    | SE       | 1<br>0 | Stuff error.<br>A stuff bit error occurred.<br>No stuff bit error occurred.                                                                                                                                                                                                          |
| 19    | ACKE     | 1<br>0 | Acknowledge error.<br>The CAN module received no acknowledge.<br>All messages have been correctly acknowledged.                                                                                                                                                                      |

**Table 16-20. Error and Status Register (CANES) Field Descriptions (continued)**

| Bit  | Field    | Value | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|------|----------|-------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 18   | BO       | 1     | Bus-off status. The CAN module is in bus-off state.<br><br>There is an abnormal rate of errors on the CAN bus. This condition occurs when the transmit error counter (CANTEC) has reached the limit of 256. During Bus Off, no messages can be received or transmitted. The bus-off state can be exited by clearing the CCR bit in CANMC register or if the Auto Bus On (ABO) (CANMC.7) bit is set, after 128 * 11 receive bits have been received. After leaving Bus Off, the error counters are cleared.                                                      |
|      |          | 0     | Normal operation                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| 17   | EP       |       | Error-passive state                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|      |          | 1     | The CAN module is in error-passive mode. CANTEC has reached 128.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|      |          | 0     | The CAN module is in error-active mode.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| 16   | EW       |       | Warning status                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|      |          | 1     | One of the two error counters (CANREC or CANTEC) has reached the warning level of 96.                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
|      |          | 0     | Values of both error counters (CANREC and CANTEC) are less than 96.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| 15:6 | Reserved |       | Reads are undefined and writes have no effect.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| 5    | SMA      |       | Suspend mode acknowledge. This bit is set after a latency of one clock cycle—up to the length of one frame—after the suspend mode was activated. The suspend mode is activated with the debugger tool when the circuit is not in run mode. During the suspend mode, the CAN module is frozen and cannot receive or transmit any frame. However, if the CAN module is transmitting or receiving a frame when the suspend mode is activated, the module enters suspend mode only at the end of the frame. Run mode is when SOFT mode is activated (CANMC.16 = 1). |
|      |          | 1     | The module has entered suspend mode.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|      |          | 0     | The module is not in suspend mode.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| 4    | CCE      |       | Change configuration enable. This bit displays the configuration access right. This bit is set after a latency of one clock cycle.                                                                                                                                                                                                                                                                                                                                                                                                                              |
|      |          | 1     | The CPU has write access to the configuration registers.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|      |          | 0     | The CPU is denied write access to the configuration registers.<br><br><b>Note:</b> The reset state of the CCE bit is 1. That is, upon reset, you can write to the bit timing registers. However, once the CCE bit is cleared (as part of the module initialization), the CANRX pin must be sensed high before you can set the CCE bit to 1 again.                                                                                                                                                                                                               |
| 3    | PDA      |       | Power-down mode acknowledge                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|      |          | 1     | The CAN module has entered the power-down mode.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|      |          | 0     | Normal operation                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| 2    | Reserved |       | Reads are undefined and writes have no effect.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| 1    | RM       |       | Receive mode. The CAN module is in receive mode. This bit reflects what the CAN module is actually doing regardless of mailbox configuration.                                                                                                                                                                                                                                                                                                                                                                                                                   |
|      |          | 1     | The CAN module is receiving a message.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|      |          | 0     | The CAN module is not receiving a message.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| 0    | TM       |       | Transmit mode. The CAN module is in transmit mode. This bit reflects what the CAN module is actually doing regardless of mailbox configuration.                                                                                                                                                                                                                                                                                                                                                                                                                 |
|      |          | 1     | The CAN module is transmitting a message.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|      |          | 0     | The CAN module is not transmitting a message.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |



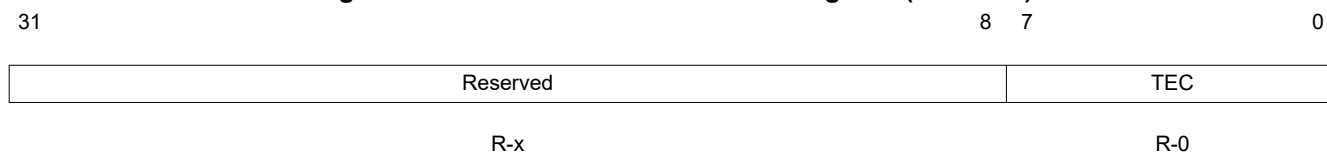
#### 16.9.14 CAN Error Counter Registers (CANTEC/CANREC)

The CAN module contains two error counters: the receive error counter (CANREC) and the transmit error counter (CANTEC). The values of both counters can be read via the CPU interface. These counters are incremented or decremented according to the CAN protocol specification version 2.0.

After reaching or exceeding the error passive limit (128), the receive error counter will not be increased anymore. When a message was received correctly, the counter is set again to a value between 119 and 127 (compare with CAN specification).

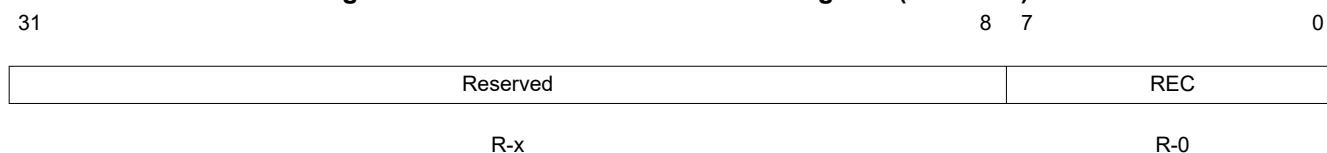
After reaching the bus-off state, the transmit error counter is undefined while the receive error counter changes its function. After reaching the bus-off state, the receive error counter is cleared. It is then incremented after every 11 consecutive recessive bits on the bus. These 11 bits correspond to the gap between two frames on the bus. If the counter reaches 128, the module automatically changes back to the bus-on status if this feature is enabled (Auto Bus On bit (ABO) (CANMC.7) set). All internal flags are reset and the error counters are cleared. After leaving initialization mode, the error counters are cleared.

### Figure 16-22. Transmit-Error-Counter Register (CANTEC)



LEGEND: R = Read only; -n = value after reset

### Figure 16-23. Receive-Error-Counter Register (CANREC)



LEGEND: R = Read only; -n = value after reset

### 16.9.15 Interrupt Registers

Interrupts are controlled by the interrupt flag registers, interrupt mask registers and mailbox interrupt level registers. These registers are described in the following subsections.

#### 16.9.15.1 Global Interrupt Flag Registers (CANGIF0/CANGIF1)

These registers allow the CPU to identify the interrupt source.

The interrupt flag bits are set if the corresponding interrupt condition did occur. The global interrupt flags are set depending on the setting of the GIL bit in the CANGIM register. If that bit is set, the global interrupts set the bits in the CANGIF1 register; otherwise, in the CANGIF0 register. This also applies to the Interrupt Flags AAIF and RMLIF. These bits are set according to the setting of the appropriate GIL bit in the CANGIM register.

The following bits are set regardless of the corresponding interrupt mask bits in the CANGIM register: MTOFn, WDIFn, BOIFn, TCOFn, WUIFn, EPIFn, AAIFn, RMLIFn, and WLIFn.

For any mailbox, the GMIFn bit is set only when the corresponding mailbox interrupt mask bit (in the CANMIM register) is set.

If all interrupt flags are cleared and a new interrupt flag is set the interrupt output line is activated when the corresponding interrupt mask bit is set. The interrupt line stays active until the interrupt flag is cleared by the CPU by writing a 1 to the appropriate bit or by clearing the interrupt-causing condition.

The GMIFx flags must be cleared by writing a 1 to the appropriate bit in the CANTA register or the CANRMP register (depending on mailbox configuration) and cannot be cleared in the CANGIFx register. After clearing one or more interrupt flags and one or more interrupt flags still set, a new interrupt is generated. The interrupt flags are cleared by writing a 1 to the corresponding bit location. If the GMIFx is set the Mailbox Interrupt Vector MIVx indicates the mailbox number of the mailbox that caused the setting of the GMIFx. In case more than one mailbox interrupt is pending, it always displays the highest mailbox interrupt vector assigned to that interrupt line.

---

#### Note

The bit descriptions are applicable to both the CANGIF0 and CANGIF1 registers. For the following interrupt flags, whether they are set in the CANGIF0 or the CANGIF1 register is determined by the value of the GIL bit in the CANGIM register: TCOFn, AAIFn, WDIFn, WUIFn, RMLIFn, BOIFn, EPIFn, and WLIFn.

If GIL = 0, these flags are set in the CANGIF0 register; if GIL = 1, they are set in the CANGIF1 register.

Similarly, the choice of the CANGIF0 and CANGIF1 register for the MTOFn and GMIFn bits is determined by the MILn bit in the CANMIL register.

---

**Figure 16-24. Global Interrupt Flag 0 Register (CANGIF0)**

|          |       |        |       |        |        |        |        |       |
|----------|-------|--------|-------|--------|--------|--------|--------|-------|
| 31       |       |        |       |        |        | 18     | 17     | 16    |
| Reserved |       |        |       |        |        | MTOF0  |        | TCOF0 |
| R-x      |       |        |       |        |        | R-0    |        | RC-0  |
| 15       | 14    | 13     | 12    | 11     | 10     | 9      | 8      |       |
| GMIF0    | AAIF0 | WDIF0  | WUIF0 | RMLIF0 | BOIF0  | EPIF0  | WLIF0  |       |
| R/W-0    | R-0   | RC-0   | RC-0  | R-0    | RC-0   | RC-0   | RC-0   |       |
| 7        |       | 5      | 4     | 3      | 2      | 1      | 0      |       |
| Reserved |       | MIV0.4 |       | MIV0.3 | MIV0.2 | MIV0.1 | MIV0.0 |       |
| R/W-0    |       | R-0    |       | R-0    | R-0    | R-0    | R-0    |       |

LEGEND: R/W = Read/Write; R = Read; C = Clear; -n = value after reset

**Figure 16-25. Global Interrupt Flag 1 Register (CANGIF1)**

|          |       |        |       |        |        |        |        |       |
|----------|-------|--------|-------|--------|--------|--------|--------|-------|
| 31       |       |        |       |        |        | 18     | 17     | 16    |
| Reserved |       |        |       |        |        | MTOF1  |        | TCOF1 |
| R-x      |       |        |       |        |        | R-0    |        | RC-0  |
| 15       | 14    | 13     | 12    | 11     | 10     | 9      | 8      |       |
| GMIF1    | AAIF1 | WDIF1  | WUIF1 | RMLIF1 | BOIF1  | EPIF1  | WLIF1  |       |
| R/W-0    | R-0   | RC-0   | RC-0  | R-0    | RC-0   | RC-0   | RC-0   |       |
| 7        |       | 5      | 4     | 3      | 2      | 1      | 0      |       |
| Reserved |       | MIV0.4 |       | MIV0.3 | MIV0.2 | MIV0.1 | MIV0.0 |       |
| R/W-0    |       | R-0    |       | R-0    | R-0    | R-0    | R-0    |       |

LEGEND: R/W = Read/Write; R = Read; C = Clear; -n = value after reset

Note: eCAN only, reserved in the SCC

**Table 16-21. Global Interrupt Flag Registers (CANGIF0/CANGIF1) Field Descriptions**

| Bit   | Field             | Value  | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|-------|-------------------|--------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 31-18 | Reserved          |        | Reserved. Reads are undefined and writes have no effect.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| 17    | MTOF0/1           | 1<br>0 | Mailbox time-out flag. This bit is not available in the SCC mode.<br>One of the mailboxes did not transmit or receive a message within the specified time frame.<br>No time out for the mailboxes occurred.<br><b>Note:</b> Whether the MTOFn bit gets set in CANGIF0 or CANGIF1 depends on the value of MILn. MTOFn gets cleared when TOSn is cleared. The TOSn bit will be cleared upon (eventual) successful transmission/reception.                                                                                                                                                |
| 16    | TCOF0/1           | 1<br>0 | Time stamp counter overflow flag.<br>The MSB of the time stamp counter has changed from 0 to 1.<br>The MSB of the time stamp counter is 0. That is, it has not changed from 0 to 1.                                                                                                                                                                                                                                                                                                                                                                                                    |
| 15    | GMIF0/1           | 1<br>0 | Global mailbox interrupt flag. This bit is set only when the corresponding mailbox interrupt mask bit in the CANMIM register is set.<br>One of the mailboxes transmitted or received a message successfully.<br>No message has been transmitted or received.                                                                                                                                                                                                                                                                                                                           |
| 14    | AAIF0/1           | 1<br>0 | Abort-acknowledge interrupt flag<br>A send transmission request has been aborted.<br>No transmission has been aborted.<br><b>Note:</b> The AAIFn bit is cleared by clearing the set AAn bit.                                                                                                                                                                                                                                                                                                                                                                                           |
| 13    | WDIF0/WDIF1       | 1<br>0 | Write-denied interrupt flag<br>The CPU write access to a mailbox was not successful. The WDIF interrupt is asserted when the identifier field of a mailbox is written to, while it is enabled. Before writing to the MSGID field of a MBX, it should be disabled. If you try this operation when the MBX is still enabled, the WDIF bit will be set and a CAN interrupt asserted.<br>The CPU write access to the mailbox was successful.                                                                                                                                               |
| 12    | WUIF0/WUIF1       | 1<br>0 | Wake-up interrupt flag<br>During local power down, this flag indicates that the module has left sleep mode.<br>The module is still in sleep mode or normal operation                                                                                                                                                                                                                                                                                                                                                                                                                   |
| 11    | RMLIF0/1          | 1<br>0 | Receive-message-lost interrupt flag<br>At least for one of the receive mailboxes, an overflow condition has occurred and the corresponding bit in the MILn register is cleared.<br>No message has been lost.<br><b>Note:</b> The RMLIFn bit is cleared by clearing the set RMPn bit.                                                                                                                                                                                                                                                                                                   |
| 10    | BOIF0/BOIF1       | 1<br>0 | Bus off interrupt flag<br>The CAN module has entered bus-off mode.<br>The CAN module is still in bus-on mode.                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| 9     | EPIF0/EPIF1       | 1<br>0 | Error passive interrupt flag<br>The CAN module has entered error-passive mode.<br>The CAN module is not in error-passive mode.                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| 8     | WLIF0/WLIF1       | 1<br>0 | Warning level interrupt flag<br>At least one of the error counters has reached the warning level.<br>None of the error counters has reached the warning level.                                                                                                                                                                                                                                                                                                                                                                                                                         |
| 7-5   | Reserved          |        | Reads are undefined and writes have no effect.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| 4-0   | MIV0.4:0/MIV1.4:0 |        | Mailbox interrupt vector. Only bits 3:0 are available in SCC mode.<br>This vector indicates the number of the mailbox that set the global mailbox interrupt flag. It keeps that vector until the appropriate MIFn bit is cleared or when a higher priority mailbox interrupt occurred. Then the highest interrupt vector is displayed, with mailbox 31 having the highest priority. In the SCC mode, mailbox 15 has the highest priority. Mailboxes 16 to 31 are not recognized.<br>If no flag is set in the TA/RMP register and GMIF1 or GMIF0 also cleared, this value is undefined. |

### 16.9.15.2 Global Interrupt Mask Register (CANGIM)

The set up for the interrupt mask register is the same as for the interrupt flag register. If a bit is set, the corresponding interrupt is enabled. This register is EALLOW protected.

The GMIF has no corresponding bit in the CANGIM because the mailboxes have individual mask bits in the CANMIM register.

**Figure 16-26. Global Interrupt Mask Register (CANGIM)**

|          |        |        |        |        |        |        |        |        |        |    |
|----------|--------|--------|--------|--------|--------|--------|--------|--------|--------|----|
| 31       |        |        |        |        |        |        |        | 18     | 17     | 16 |
| Reserved |        |        |        |        |        |        |        | MTOM   | TCOM   |    |
| R-0      |        |        |        |        |        |        |        | R/WP-0 | R/WP-0 |    |
| 15       | 14     | 13     | 12     | 11     | 10     | 9      | 8      |        |        |    |
| Reserved | AAIM   | WDIM   | WUIM   | RMLIM  | BOIM   | EPIM   | WLIM   |        |        |    |
| R-0      | R/WP-0 | R/WP-0 | R/WP-0 | R/WP-0 | R/WP-0 | R/WP-0 | R/WP-0 |        |        |    |
| 7        |        |        |        | 3      | 2      | 1      | 0      |        |        |    |
| Reserved |        |        |        |        | GIL    | I1EN   | I0EN   |        |        |    |
| R-0      |        |        |        |        | R/WP-0 | R/WP-0 | R/WP-0 |        |        |    |

LEGEND: R = Read; W = Write; WP = Write in EALLOW mode only; -n = value after reset

**Table 16-22. Global Interrupt Mask Register (CANGIM) Field Descriptions**

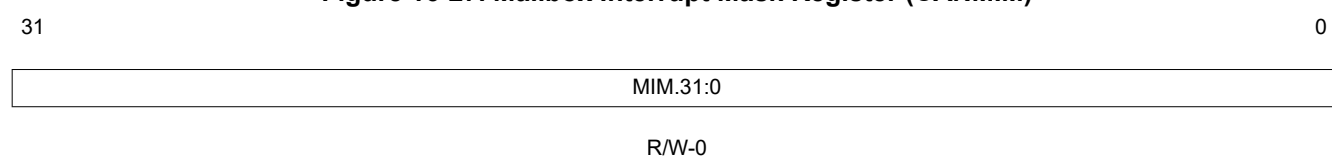
| Bit   | Field    | Value  | Description                                                 |
|-------|----------|--------|-------------------------------------------------------------|
| 31:18 | Reserved |        | Reads are undefined and writes have no effect.              |
| 17    | MTOM     | 1<br>0 | Mailbox time-out interrupt mask<br>Enabled<br>Disabled      |
| 16    | TCOM     | 1<br>0 | Time stamp counter overflow mask<br>Enabled<br>Disabled     |
| 15    | Reserved |        | Reads are undefined and writes have no effect.              |
| 14    | AAIM     | 1<br>0 | Abort Acknowledge Interrupt Mask.<br>Enabled<br>Disabled    |
| 13    | WDIM     | 1<br>0 | Write denied interrupt mask<br>Enabled<br>Disabled          |
| 12    | WUIM     | 1<br>0 | Wake-up interrupt mask<br>Enabled<br>Disabled               |
| 11    | RMLIM    | 1<br>0 | Received-message-lost interrupt mask<br>Enabled<br>Disabled |

**Table 16-22. Global Interrupt Mask Register (CANGIM) Field Descriptions (continued)**

| Bit | Field    | Value  | Description                                                                                                                                                                                                                           |
|-----|----------|--------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 10  | BOIM     | 1<br>0 | Bus-off interrupt mask<br>Enabled<br>Disabled                                                                                                                                                                                         |
| 9   | EPIM     | 1<br>0 | Error-passive interrupt mask<br>Enabled<br>Disabled                                                                                                                                                                                   |
| 8   | WLIM     | 1<br>0 | Warning level interrupt mask<br>Enabled<br>Disabled                                                                                                                                                                                   |
| 7:3 | Reserved |        | Reads are undefined and writes have no effect.                                                                                                                                                                                        |
| 2   | GIL      | 1<br>0 | Global interrupt level for the interrupts TCOF, WDIF, WUIF, BOIF, EPIF, RMLIF, AAIF and WLIF.<br>All global interrupts are mapped to the ECAN1INT interrupt line.<br>All global interrupts are mapped to the ECAN0INT interrupt line. |
| 1   | I1EN     | 1<br>0 | Interrupt 1 enable<br>This bit globally enables all interrupts for the ECAN1INT line if the corresponding masks are set.<br>The ECAN1INT interrupt line is disabled.                                                                  |
| 0   | I0EN     | 1<br>0 | Interrupt 0 enable<br>This bit globally enables all interrupts for the ECAN0INT line if the corresponding masks are set.<br>The ECAN0INT interrupt line is disabled.                                                                  |

### 16.9.15.3 Mailbox Interrupt Mask Register (CANMIM)

There is one interrupt flag available for each mailbox. This can be a receive or a transmit interrupt depending on the configuration of the mailbox. This register is EALLOW protected.

**Figure 16-27. Mailbox Interrupt Mask Register (CANMIM)**


LEGEND: R/W = Read/Write; -n = value after reset

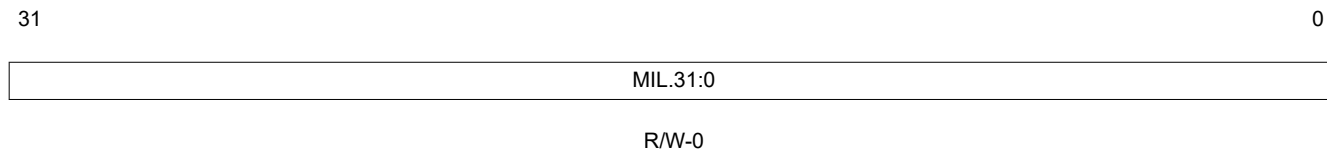
**Table 16-23. Mailbox Interrupt Mask Register (CANMIM) Field Descriptions**

| Bit  | Field    | Value  | Description                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|------|----------|--------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 31:0 | MIM.31:0 | 1<br>0 | Mailbox interrupt mask. After power up all interrupt mask bits are cleared and the interrupts are disabled. These bits allow any mailbox interrupt to be masked individually.<br>Mailbox interrupt is enabled. An interrupt is generated if a message has been transmitted successfully (in case of a transmit mailbox) or if a message has been received without any error (in case of a receive mailbox).<br>Mailbox interrupt is disabled. |

### 16.9.15.4 Mailbox Interrupt Level Register (CANMIL)

Each of the 32 mailboxes may initiate an interrupt on one of the two interrupt lines. Depending on the setting in the mailbox interrupt level register (CANMIL), the interrupt is generated on ECAN0INT (MIL $n$  = 0) or on line ECAN1INT (MIL $n$  = 1).

**Figure 16-28. Mailbox Interrupt Level Register (CANMIL)**



LEGEND: R/W = Read/Write; - $n$  = value after reset

**Table 16-24. Mailbox Interrupt Level Register (CANMIL) Field Descriptions**

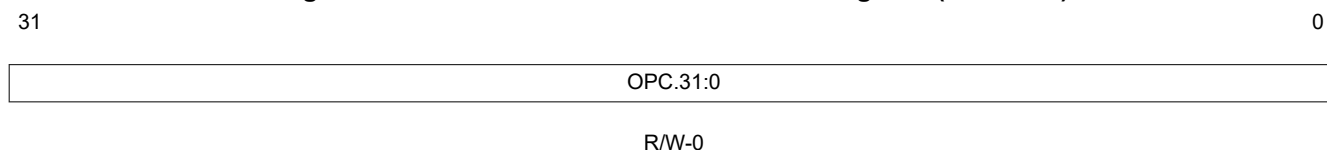
| Bit  | Field    | Value | Description                                                                                                                                                   |
|------|----------|-------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 31:0 | MIL.31:0 | 1     | Mailbox interrupt level. These bits allow any mailbox interrupt level to be selected individually.<br>The mailbox interrupt is generated on interrupt line 1. |
|      |          | 0     | The mailbox interrupt is generated on interrupt line 0.                                                                                                       |

### 16.9.16 Overwrite Protection Control Register (CANOPC)

If there is an overflow condition for mailbox  $n$  (RMP $[n]$  is set to 1 and a new receive message would fit for mailbox  $n$ ), the new message is stored depending on the settings in the CANOPC register. If the corresponding bit OPC $[n]$  is set to 1, the old message is protected against being overwritten by the new message; thus, the next mailboxes are checked for a matching ID. If no other mailbox is found, the message is lost without further notification. If the bit OPC $[n]$  is cleared to 0, the old message is overwritten by the new one. This is notified by setting the receive message lost bit RML $[n]$ .

For read/write operations, only 32-bit access is supported.

**Figure 16-29. Overwrite Protection Control Register (CANOPC)**



LEGEND: R/W = Read/Write; - $n$  = value after reset

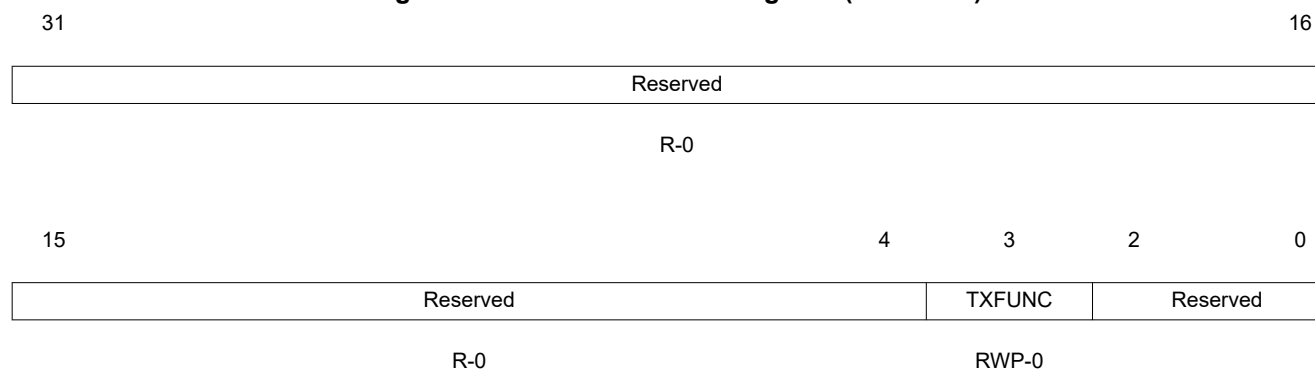
**Table 16-25. Overwrite Protection Control Register (CANOPC) Field Descriptions**

| Bit  | Field    | Value | Description                                                                                                                                                               |
|------|----------|-------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 31:0 | OPC.31:0 | 1     | Overwrite protection control bits<br>1 If the bit OPC $[n]$ is set to 1, an old message stored in that mailbox is protected against being overwritten by the new message. |
|      |          | 0     | 0 If the bit OPC $[n]$ is not set, the old message can be overwritten by a new one.                                                                                       |

### 16.9.17 eCAN I/O Control Registers (CANTIOC, CANRIOC)

The CANTX and CANRX pins should be configured for CAN use. This is done using the CANTIOC and CANRIOC registers.

**Figure 16-30. TX I/O Control Register (CANTIOC)**

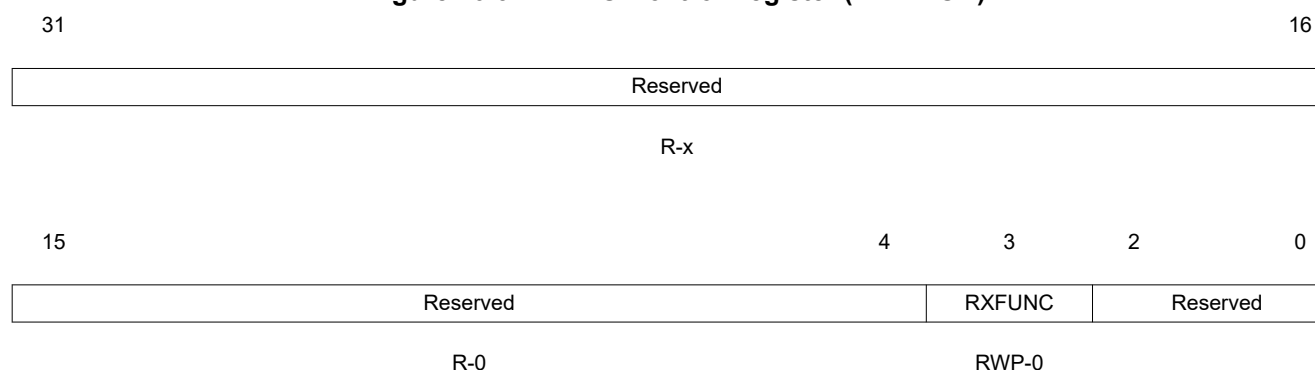


LEGEND: RWP = Read in all modes, write in EALLOW-mode only; R = Read only; -n = value after reset

**Table 16-26. TX I/O Control Register (CANTIOC) Field Descriptions**

| Bit  | Field    | Value  | Description                                                                                                        |
|------|----------|--------|--------------------------------------------------------------------------------------------------------------------|
| 31:4 | Reserved |        | Reads are undefined and writes have no effect.                                                                     |
| 3    | TXFUNC   | 1<br>0 | This bit must be set for CAN module function.<br>The CANTX pin is used for the CAN transmit functions.<br>Reserved |
| 2:0  | Reserved |        | Reserved                                                                                                           |

**Figure 16-31. RX I/O Control Register (CANRIOC)**



LEGEND: RWP = Read in all modes, write in EALLOW-mode only; R = Read only; -n = value after reset; x = indeterminate

**Table 16-27. RX I/O Control Register (CANRIOC) Field Descriptions**

| Bit  | Field    | Value  | Description                                                                                                       |
|------|----------|--------|-------------------------------------------------------------------------------------------------------------------|
| 31:4 | Reserved |        | Reads are undefined and writes have no effect.                                                                    |
| 3    | RXFUNC   | 1<br>0 | This bit must be set for CAN module function.<br>The CANRX pin is used for the CAN receive functions.<br>Reserved |
| 2:0  | Reserved |        | Reserved                                                                                                          |



## 16.9.18 Timer Management Unit

Several functions are implemented in the eCAN to monitor the time when messages are transmitted/received. A separate state machine is included in the eCAN to handle the time-control functions. This state machine has lower priority when accessing the registers than the CAN state machine has. Therefore, the time-control functions may be delayed by other ongoing actions.

### 16.9.18.1 Time Stamp Functions

To get an indication of the time of reception or transmission of a message, a free-running 32-bit timer (TSC) is implemented in the module. Its content is written into the time stamp register of the corresponding mailbox (Message Object Time Stamp [MOTS]) when a received message is stored or a message has been transmitted.

The counter is driven from the bit clock of the CAN bus line. The timer is stopped during the initialization mode or if the module is in sleep or suspend mode. After power-up reset, the free-running counter is cleared.

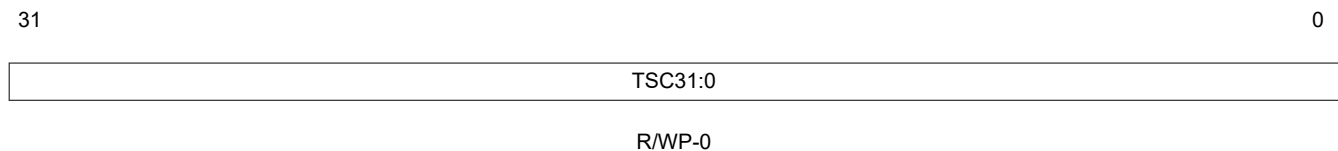
The most significant bit of the TSC register is cleared by writing a 1 to TCC (CANMC.14). The TSC register can also be cleared when mailbox 16 transmitted or received (depending on the setting of CANMD.16 bit) a message successfully. This is enabled by setting the MBCC bit (CANMC.15). Therefore, it is possible to use mailbox 16 for global time synchronization of the network. The CPU can read and write the counter.

Overflow of the counter is detected by the TSC-counter-overflow-interrupt flag (TCOF $n$ -CANGIF $n$ .16). An overflow occurs when the highest bit of the TSC counter changes to 1. Therefore, the CPU has enough time to handle this situation.

#### 16.9.18.1.1 Time-Stamp Counter Register (CANTSC)

This register holds the time-stamp counter value at any instant of time. This is a free-running 32-bit timer which is clocked by the bit clock of the CAN bus. For example, at a bit rate of 1 Mbps, CANTSC would increment every 1  $\mu$ s.

**Figure 16-32. Time-Stamp Counter Register (CANTSC)**



LEGEND: R = Read; WP = Write in EALLOW enabled mode only; - $n$  = value after reset  
Note: eCAN mode only, reserved in the SCC

**Table 16-28. Time-Stamp Counter Register (CANTSC) Field Descriptions**

| Bit  | Field   | Value | Description                                                                                                          |
|------|---------|-------|----------------------------------------------------------------------------------------------------------------------|
| 31:0 | TSC31:0 |       | Time-stamp counter register. Value of the local network time counter used for the time-stamp and time-out functions. |

### 16.9.18.2 Time-Out Functions

To ensure that all messages are sent or received within a predefined period, each mailbox has its own time-out register. If a message has not been sent or received by the time indicated in the time-out register and the corresponding bit TOC[n] is set in the TOC register, a flag is set in the time-out status register (TOS).

For transmit mailboxes the TOS[n] flag is cleared when the TOC[n] bit is cleared or when the corresponding TRS[n] bit is cleared, no matter whether due to successful transmission or abortion of the transmit request. For receive mailboxes, the TOS[n] flag is cleared when the corresponding TOC[n] bit is cleared.

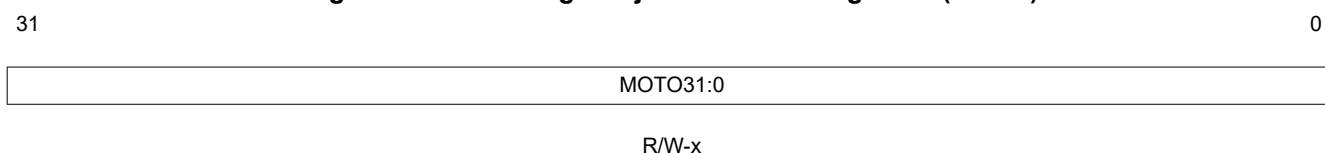
The CPU can also clear the time-out status register flags by writing a 1 into the time-out status register.

The message object time-out registers (MOTO) are implemented as a RAM. The state machine scans all the MOTO registers and compares them to the TSC counter value. If the value in the TSC register is equal to or greater than the value in the time-out register, and the corresponding TRS bit (applies to transmit mailboxes only) is set, and the TOC[n] bit is set, the appropriate bit TOS[n] is set. Since all the time-out registers are scanned sequentially, there can be a delay before the TOS[n] bit is set.

#### 16.9.18.2.1 Message-Object Time-Out Registers (MOTO)

This register holds the time-out value of the TSC by which the corresponding mailbox data should be successfully transmitted or received. Each mailbox has its own MOTO register.

**Figure 16-33. Message-Object Time-Out Registers (MOTO)**



LEGEND: R/W = Read/Write; -n = value after reset; x = indeterminate

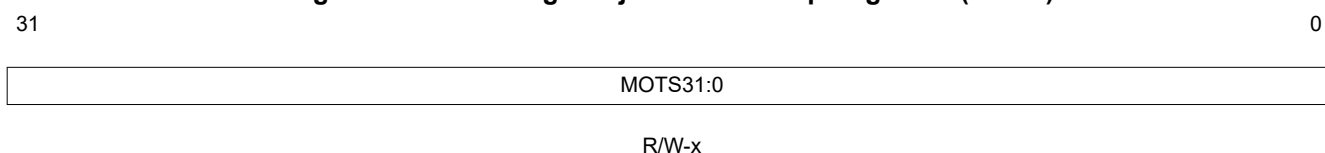
**Table 16-29. Message-Object Time-Out Registers (MOTO) Field Descriptions**

| Bit  | Field    | Value | Description                                                                                                                |
|------|----------|-------|----------------------------------------------------------------------------------------------------------------------------|
| 31:0 | MOTO31:0 |       | Message object time-out register. Limit-value of the time-stamp counter (TSC) to actually transmit or receive the message. |

#### 16.9.18.2.2 Message-Object Time Stamp Registers (MOTS)

This register holds the value of the TSC when the corresponding mailbox data was successfully transmitted or received. Each mailbox has its own MOTS register.

**Figure 16-34. Message-Object Time Stamp Registers (MOTS)**



LEGEND: R/W = Read/Write; -n = value after reset; x = indeterminate

**Table 16-30. Message-Object Time Stamp Registers (MOTS) Field Descriptions**

| Bit  | Field    | Value | Description                                                                                       |
|------|----------|-------|---------------------------------------------------------------------------------------------------|
| 31:0 | MOTS31:0 |       | Value of the time stamp counter (TSC) when the message has been actually received or transmitted. |

### 16.9.18.2.3 Time-Out Control Register (CANTOC)

This register controls whether or not time-out functionality is enabled for a given mailbox.

**Figure 16-35. Time-Out Control Register (CANTOC)**

31

0

|         |  |
|---------|--|
| TOC31:0 |  |
| R/W-0   |  |

LEGEND: R/W = Read/Write; -n = value after reset

**Table 16-31. Time-Out Control Register (CANTOC) Field Descriptions**

| Bit  | Field   | Value | Description                                                                                                                                                                                                           |
|------|---------|-------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 31:0 | TOC31:0 | 1     | Time-out control register                                                                                                                                                                                             |
|      |         | 0     | The TOC[n] bit must be set by the CPU to enable the time-out function for mailbox <i>n</i> . Before setting the TOC[n] bit, the corresponding MOTO register should be loaded with the time-out value relative to TSC. |
|      |         | 0     | The time-out function is disabled. The TOS[n] flag is never set.                                                                                                                                                      |

### 16.9.18.2.4 Time-Out Status Register (CANTOS)

This register holds the status information of mailboxes that have timed out.

The TOS<sub>*n*</sub> bit is set when all three of the following conditions are met:

1. The TSC value is greater than or equal to the value in the time-out register (MOTO<sub>*n*</sub>).
2. The TOC<sub>*n*</sub> bit is set.
3. The TRS<sub>*n*</sub> bit is set (in the case of a transmit mailbox).

The time-out registers are implemented as a RAM. The state machine scans all the time-out registers and compares them to the time stamp counter value. Since all the time out registers are scanned sequentially, it is possible that even though a transmit mailbox has timed out, the TOS<sub>*n*</sub> bit is not set. This can happen when the mailbox succeeded in transmitting and clearing the TRS<sub>*n*</sub> bit before the state machine scans the time-out register of that mailbox. This is true for the receive mailbox as well. In this case, the RMP<sub>*n*</sub> bit can be set to 1 by the time the state machine scans the time-out register of that mailbox. However, the receive mailbox probably did not receive the message before the time specified in the time-out register.

**Figure 16-36. Time-Out Status Register (CANTOS)**

31

0

|         |  |
|---------|--|
| TOS31:0 |  |
| R/C-0   |  |

LEGEND: R/C = Read/Clear; -n = value after reset

**Table 16-32. Time-Out Status Register (CANTOS) Field Descriptions**

| Bit  | Field    | Value | Description                                                                                                                                                                      |
|------|----------|-------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 31:0 | TOS 31:0 | 1     | Time-out status register                                                                                                                                                         |
|      |          | 0     | Mailbox[n] has timed out. The value in the TSC register is larger or equal to the value in the time-out register that corresponds to mailbox <i>n</i> and the TOC[n] bit is set. |
|      |          | 0     | No time-out occurred or it is disabled for that mailbox.                                                                                                                         |

### 16.9.18.3 Behavior/Usage of MTOF0/1 Bit in User Applications

The MTOF0/1 bit is automatically cleared by the CPK (along with the TOS $n$  bit) upon transmission/reception by the mailbox, which asserted this flag in the first place. It can also be cleared by the user (via the CPU). On a time-out condition, the MTOF0/1 bit (and the TOS. $n$  bit) is set. On an (eventual) successful communication, these bits are automatically cleared by the CPK. Following are the possible behaviors/usage for the MTOF0/1 bit:

1. Time-out condition occurs. Both MTOF. $n$  bit and TOS. $n$  bits are set. Communication is never successful; that is, the frame was never transmitted (or received). An interrupt is asserted. Application should handle this issue as desired and clear TOC. $n$  bit which clears TOS. $n$  bit which in turn clears the MTOF. $n$  bit.
2. Time-out condition occurs. Both MTOF. $n$  bit and TOS. $n$  bits are set. However, communication is eventually successful; that is, the frame gets transmitted (or received). Both MTOF. $n$  bit and TOS. $n$  bits are cleared automatically by the CPK. An interrupt is still asserted because the interrupt occurrence was recorded in the PIE module. When the ISR scans the GIF register, it doesn't see the MTOF0/1 bit set. This is the phantom interrupt scenario. This is handled per the application requirements.
3. Time-out condition occurs. Both MTOF0/1 bit and TOS. $n$  bits are set. While executing the ISR pertaining to time-out, communication is successful. This situation must be handled carefully. The application should not re-transmit a mailbox if the mailbox is sent between the time the interrupt is asserted and the time the ISR is attempting to take corrective action. One way of doing this is to poll the TM/RM bits in the CANES register. These bits indicate if the CPK is currently transmitting/receiving. If that is the case, the application should wait till the communication is over and then check the TOS. $n$  bit again. If the communication is still not successful, then the application should take the corrective action.

### 16.9.19 Mailbox Layout

The following four 32-bit registers comprise each mailbox:

- MSGID – Stores the message ID
- MSGCTRL – Defines number of bytes, transmission priority and remote frames
- CANMDL – 4 bytes of data
- CANMDH – 4 bytes of data

#### 16.9.19.1 Message Identifier Register (MSGID)

This register contains the message ID and other control bits for a given mailbox.

#### Note

This register can be written only when mailbox  $n$  is disabled (CANME[ $n$ ] (CANME.31-0) = 0). The reset-state of IDE, AME, and AAM bits are undefined. As part of module initialization, these bits must be initialized as appropriate. Otherwise, they may assume random values and lead to improper operation of the mailboxes.

**Figure 16-37. Message Identifier Register (MSGID) Register**

|       |       |       |          |   |
|-------|-------|-------|----------|---|
| 31    | 30    | 29    | 28       | 0 |
| IDE   | AME   | AAM   | ID[28:0] |   |
| R/W-x | R/W-x | R/W-x | R/W-x    |   |

LEGEND: R/W = Read/Write; R = Read only; - $n$  = value after reset; x = indeterminate

**Table 16-33. Message Identifier Register (MSGID) Field Descriptions**

| Bit  | Field    | Value | Description                                                                                                                                                                                                                                                      |
|------|----------|-------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 31   | IDE      | 1     | Identifier extension bit.<br>Receive Mailbox -> The received message has an extended identifier (29 bits). Transmit Mailbox-> The message to be sent has an extended identifier (29 bits).                                                                       |
|      |          | 0     | Receive Mailbox -> The received message has a standard identifier (11 bits). Transmit Mailbox-> The message to be sent has a standard identifier (11 bits).                                                                                                      |
| 30   | AME      |       | Acceptance mask enable bit. AME is only used for receive mailboxes. This bit is not modified by a message reception.                                                                                                                                             |
|      |          | 1     | The corresponding acceptance mask is used.                                                                                                                                                                                                                       |
|      |          | 0     | No acceptance mask is used, all identifier bits must match to receive the message                                                                                                                                                                                |
| 29   | AAM      |       | Auto answer mode bit. This bit is only valid for message mailboxes configured as transmit. For receive mailboxes, this bit has no effect: the mailbox is always configured for normal receive operation.<br><br>This bit is not modified by a message reception. |
|      |          | 1     | Auto answer mode. If a matching remote request is received, the CAN module answers to the remote request by sending the contents of the mailbox.                                                                                                                 |
|      |          | 0     | Normal transmit mode. The mailbox does not reply to remote requests. The reception of a remote request frame has no effect on the message mailbox.                                                                                                               |
| 28:0 | ID[28:0] |       | Message identifier                                                                                                                                                                                                                                               |
|      |          | 1     | In standard identifier mode, if the IDE bit (MSGID.31) = 0, the message identifier is stored in bits ID.28:18. In this case, bits ID.17:0 have no meaning.                                                                                                       |
|      |          | 0     | In extended identifier mode, if the IDE bit (MSGID.31) = 1, the message identifier is stored in bits ID.28:0.                                                                                                                                                    |

#### 16.9.19.2 CPU Mailbox Access

Write accesses to the identifier can only be accomplished when the mailbox is disabled (CANME[n] (CANME.31-0) = 0). During access to the data field, it is critical that the data does not change while the CAN module is reading it. Hence, a write access to the data field is disabled for a receive mailbox.

For send mailboxes, an access is usually denied if the TRS (TRS.31-0) or the TRR (TRR.31-0) flag is set. In these cases, an interrupt can be asserted. A way to access those mailboxes is to set CDR (CANMC.8) before accessing the mailbox data.

After the CPU access is finished, the CPU must clear the CDR flag by writing a 0 to it. The CAN module checks for that flag before and after reading the mailbox. If the CDR flag is set during those checks, the CAN module does not transmit the message but continues to look for other transmit requests. The setting of the CDR flag also stops the write-denied interrupt (WDI) from being asserted.

### 16.9.19.3 Message-Control Register (MSGCTRL)

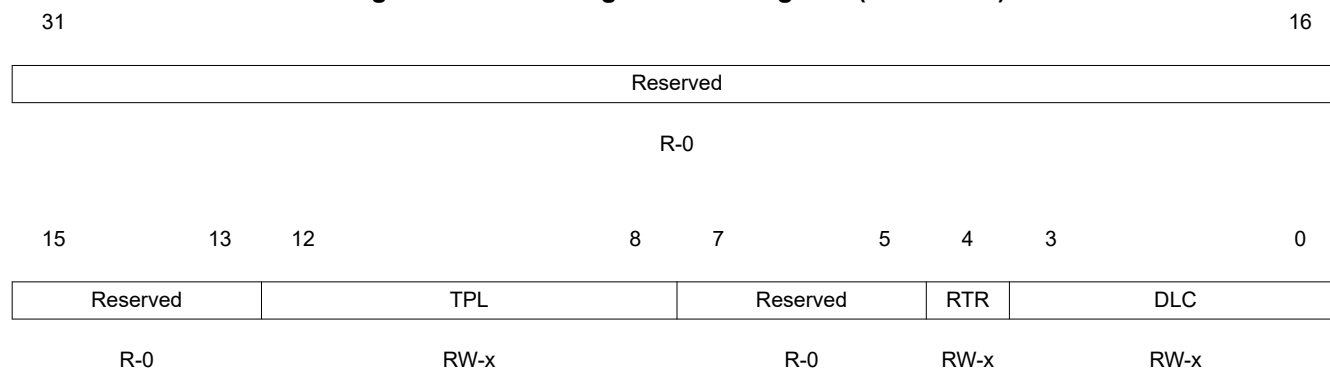
For a transmit mailbox, this register specifies the number of bytes to be transmitted and the transmission priority. It also specifies the remote-frame operation.

#### Note

As part of the CAN module initialization process, all the bits of the MSGCTRL $n$  registers must first be initialized to zero before proceeding to initialize the various bit fields to the desired values.

The register MSGCTRL( $n$ ) can only be written if the mailbox  $n$  is configured for transmission (CANMD[ $n$ ] (CANMD.31-0)=0) or if the mailbox is disabled (CANME[ $n$ ] (CANME.31-0) =0).

**Figure 16-38. Message-Control Register (MSGCTRL)**



LEGEND: RW = Read any time, write when mailbox is disabled or configured for transmission; - $n$  = value after reset; x = indeterminate

**Table 16-34. Message-Control Register (MSGCTRL) Field Descriptions**

| Bit   | Field    | Value  | Description                                                                                                                                                                                                                                                                                                                                                                                                                                           |
|-------|----------|--------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 31:13 | Reserved |        | Reserved                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| 12:8  | TPL 4:0  |        | Transmit-priority level. This 5-bit field defines the priority of this mailbox as compared to the other 31 mailboxes. The highest number has the highest priority. When two mailboxes have the same priority, the one with the higher mailbox number is transmitted. TPL applies only for transmit mailboxes. TPL is not used in SCC-mode.                                                                                                            |
| 7:5   | Reserved |        | Reserved                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| 4     | RTR      | 1<br>0 | Remote-transmission-request bit<br>For receive mailbox: If the TRS flag is set, a remote frame is transmitted and the corresponding data frame is received in the same mailbox. Once the remote frame is sent, the TRS bit of the mailbox is cleared by CAN.<br>For transmit mailbox: If the TRS flag is set, a remote frame is transmitted, but the corresponding data frame has to be received in another mailbox.<br>No remote frame is requested. |
| 3:0   | DLC 3:0  |        | Data-length code. The number in these bits determines how many data bytes are sent or received. Valid value range is from 0 to 8. Values from 9 to 15 are not allowed.                                                                                                                                                                                                                                                                                |

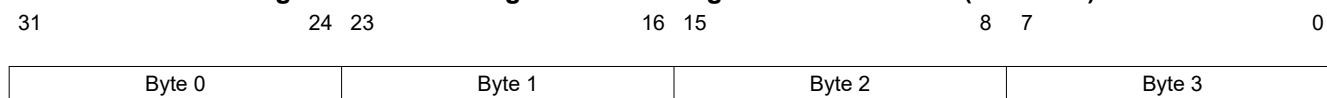
## 16.10 Message Data Registers (CANMDL, CANMDH)

Eight bytes of the mailbox are used to store the data field of a CAN message. The setting of DBO (CANMC.10) determines the ordering of stored data. The data is transmitted or received from the CAN bus, starting with byte 0.

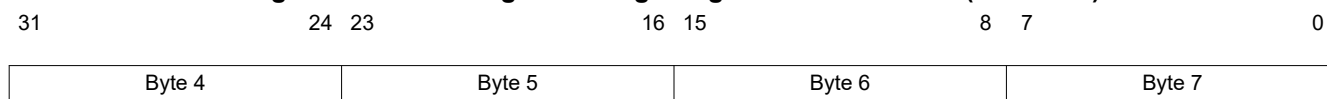
- When DBO (CANMC.10) = 1, the data is stored or read starting with the least significant byte of the CANMDL register and ending with the most significant byte of the CANMDH register.
- When DBO (CANMC.10) = 0, the data is stored or read starting with the most significant byte of the CANMDL register and ending with the least significant byte of the CANMDH register.

The registers CANMDL(*n*) and CANMDH(*n*) can be written only if the mailbox *n* is configured for transmission (CANMD[*n*] (CANMD.31-0)=0) or the mailbox is disabled (CANME[*n*] (CANME.31-0)=0). If TRS[*n*] (TRS.31-0)=1, the registers CANMDL(*n*) and CANMDH(*n*) cannot be written, unless CDR (CANMC.8)=1, with MBNR (CANMC.4-0) set to *n*. These settings also apply for a message object configured in reply mode (AAM (MSGID.29)=1).

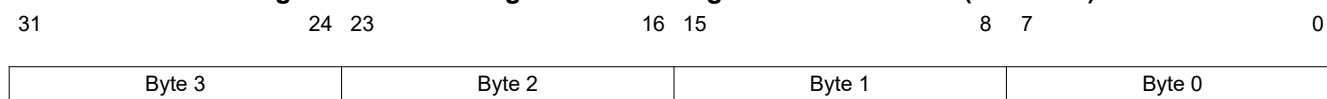
**Figure 16-39. Message-Data-Low Register With DBO = 0 (CANMDL)**



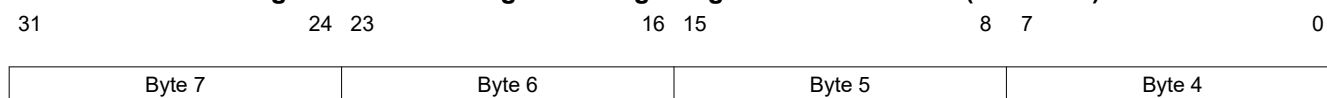
**Figure 16-40. Message-Data-High Register With DBO = 0 (CANMDH)**



**Figure 16-41. Message-Data-Low Register With DBO = 1 (CANMDL)**



**Figure 16-42. Message-Data-High Register With DBO = 1 (CANMDH)**



### Note

The data field beyond the valid received data is modified by any message reception and is indeterminate.

This page intentionally left blank.



This chapter discusses the features and functions of the universal serial bus (USB) controller.

|                                                   |             |
|---------------------------------------------------|-------------|
| <b>17.1 Introduction.....</b>                     | <b>1056</b> |
| <b>17.2 Functional Description.....</b>           | <b>1058</b> |
| <b>17.3 Initialization and Configuration.....</b> | <b>1068</b> |
| <b>17.4 USB Global Interrupts.....</b>            | <b>1069</b> |
| <b>17.5 USB Registers.....</b>                    | <b>1069</b> |

## 17.1 Introduction

The USB controller operates as a full-speed function controller during point-to-point communications with the USB host. The controller complies with the USB 2.0 standard, which includes SUSPEND and RESUME signaling. It has eight endpoints, one-half of them being for IN transactions and one-half of them being for OUT transactions. One IN and one OUT endpoint are fixed-function endpoints used for control transfers; the others are defined by firmware. A dynamically sizeable FIFO supports queuing multiple packets. Software-controlled connect and disconnect allow flexibility during USB device startup.

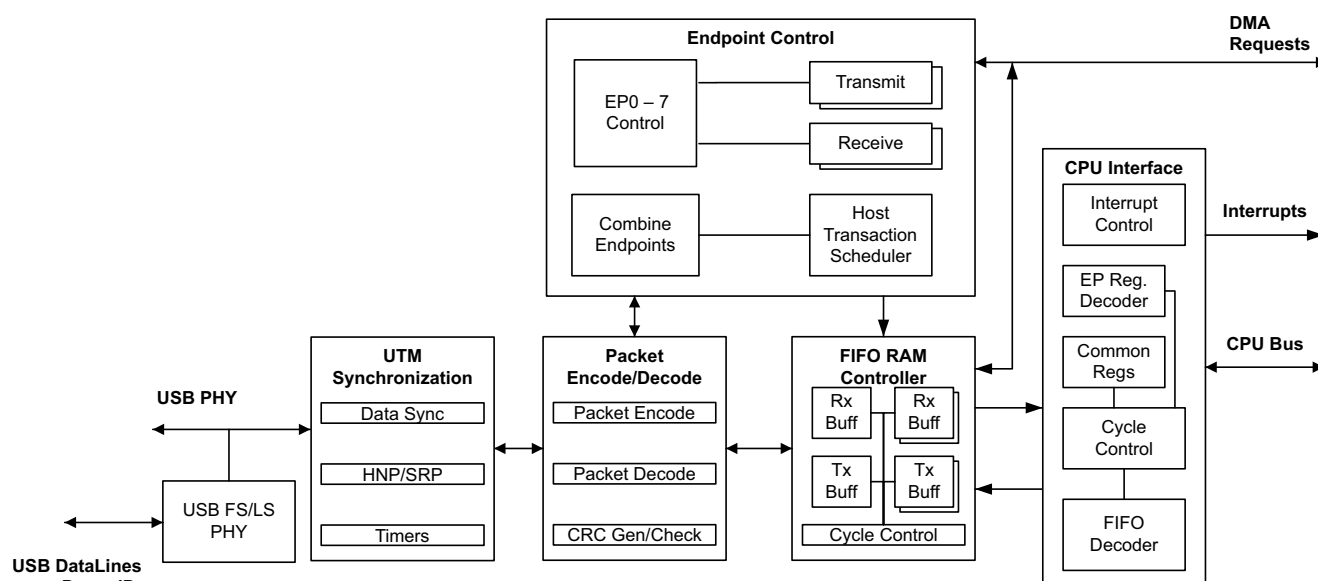
### 17.1.1 Features

The USB module has the following features:

- Complies with USB-IF certification standards
- USB 2.0 full-speed (12 Mbps) operation in host and device modes as well as low-speed (1.5 Mbps) operation in host mode
- Integrated PHY
- Three transfer types: Control, Interrupt, and Bulk
- Eight endpoints
  - One dedicated control IN endpoint and one dedicated control OUT endpoint
  - Three configurable IN endpoints and three configurable OUT endpoints
- Four KB dedicated endpoint memory

#### 17.1.1.1 Block Diagram

The USB block diagram is shown in [Figure 17-1](#).



**Figure 17-1. USB Block Diagram**

#### 17.1.1.2 Signal Description

The USB controller requires a total of three signals (D+, D-, and  $V_{BUS}$ ) to operate in device mode and two signals (D+, D-) to operate in embedded host mode. Because of the differential signaling needed for USB, the pins D+ and D- have special buffers to support USB. As such, their position on the chip is not user-selectable. These pins at reset are, by default, GPIOs. They must be configured before being used as USB function pins. The USBIOEN bit in the GPIOA Control 2 (GPACTRL2) register should be set to choose the USB function. The signals USB bus voltage ( $V_{BUS}$ ), external power enable (EPEN), and power fault (PFLT) are not hardwired to any

pin and some applications will require they be implemented in software via a GPIO. Software that implements these signals is available in the USB software library.

### 17.1.1.3 VBus Recommendations

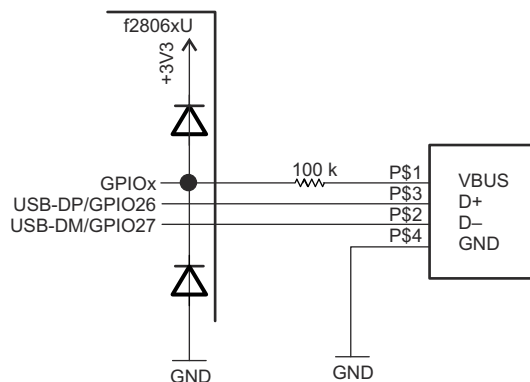
Most applications do not need to monitor  $V_{BUS}$ . Because of this, a dedicated  $V_{BUS}$  monitoring pin was not included on this microcontroller. If you are designing a bus-powered device application or an embedded host application, you do NOT need to monitor  $V_{BUS}$ . If you are designing a self-powered device, you will need to actively monitor the state of the  $V_{BUS}$  pin in order to ensure compliance with the USB specification. In Section 7.1.5 and Section 7.2.1 of the USB Specification Revision 2.0™ it is stated respectively that:

- "The voltage source on the [speed identification] pull-up resistor must be derived from or controlled by the power supplied on the USB cable such that when  $V_{BUS}$  is removed, the pull-up resistor does not supply current on the data line to which it is attached.
- When  $V_{BUS}$  is removed, the device must remove power from the D+/D- pull-up resistor within 10 seconds.
- Later in the timing tables (Section 7.3.2) of the USB Specification 2.0 it is also stated that the D+/D- pull-up resistor should be applied within 100ms of  $V_{BUS}$  reaching a valid level."

Meeting the above specification is easy because of the slow timing requirements. In this chapter we will discuss the hardware part of the  $V_{BUS}$  monitoring solution. The corresponding software will be discussed briefly, but for examples and an explanation, please consult the USB software guide.

The pins of this microcontroller are not 5V tolerant, and because of this, the  $V_{BUS}$  signal cannot be directly connected to a GPIO pin. Directly connecting 5V to a pin of the microcontroller will destroy the I/O buffer of the pin and possibly more of the chip. The most cost-effective way of making any pin capable of reading a 5V input is to use a series resistance in conjunction with the ESD diode clamps already present inside the device on every pin. It is recommended to use a 100kΩ series resistor between the  $V_{BUS}$  signal and the pin chosen to monitor it. A diagram of this setup is shown in Figure 17-2.

In the below diagram, if  $V_{BUS}$  is above or below 3.3V and 0V, respectively, one of the ESD clamp diodes will be forward-biased, allowing current to flow through the 100kΩ resistor. The purpose of the diode clamps is to protect the pins of the microcontroller from very short overvoltage spikes of a high magnitude. They do this by clamping the voltage excursion to one of the supply rails. We are effectively requiring the ESD clamps to do the same thing they were designed to do, but instead of a short high magnitude pulse, we are giving them a long low magnitude static value via the 100kΩ resistor.



**Figure 17-2. USB Scheme**

Any pin that has digital input/output functionality could potentially be used to monitor  $V_{BUS}$ , but the use of an interrupt-capable GPIO is recommended. A pin that does not have external interrupt capability may also be used, but the input state of the pin must be polled periodically by the application software to ensure appropriate action is taken whenever  $V_{BUS}$  is applied or removed. If an interrupt-capable GPIO is chosen, it should be configured to generate an interrupt on both the rising and falling edge. More information on external interrupts can be found in the *System Control and Interrupts* chapter. Example code that implements  $V_{BUS}$  monitoring using external interrupts and takes the appropriate actions is documented in the USB Software Guide and can be found in the associated USB software package.

## 17.1.2 USB Related Collateral

### Foundational Materials

- [USB Precision Labs](#) (Video)

### Getting Started Materials

- [USB system design in Sitara devices using Linux](#) (Video)
  - Concepts of modes and technology applicable to C2000

### Expert Materials

- [High-Speed Interface Layout Guidelines Application Report](#)
- [USB Flash Programming of C2000 Microcontrollers Application Report](#)

## 17.2 Functional Description

The USB controller can be configured to act as either a dedicated host or device. However, when the USB controller is acting as a self-powered device, a GPIO input or analog comparator input must be connected to  $V_{BUS}$  and configured to generate an interrupt when the  $V_{BUS}$  level drops. This interrupt is used to disable the pullup resistor on the USB0DP signal.

---

### Note

When USB is used in the system, the minimum system frequency is 20 MHz.

---

### 17.2.1 Operation as a Device

This section describes how the USB controller performs when it is being used as a USB device. IN endpoints, OUT endpoints, entry into and exit from SUSPEND mode, and recognition of start of frame (SOF) are all described.

When in device mode, IN transactions are controlled by the endpoint transmit interface and uses the transmit endpoint registers for the given endpoint. OUT transactions are handled with the endpoints receive interface and use the receive endpoint registers for the given endpoint. When configuring the size of the FIFOs for endpoints, take into account the maximum packet size for an endpoint. Note the following:

- Bulk endpoints should be the size of the maximum packet (up to 64 bytes) or twice the maximum packet size if double buffering is used (described further in the following section).
- Interrupt endpoints should be the size of the maximum packet (up to 64 bytes) or twice the maximum packet size if double buffering is used.
- It is also possible to specify a separate control endpoint for a USB device. However, in most cases the USB device should use the dedicated control endpoint on the USB controller's endpoint 0.

### 17.2.1.1 Control and Configurable Endpoints

When operating as a device, the USB controller provides two dedicated control endpoints (IN and OUT). The remaining available configurable endpoints (one-half IN and one-half OUT) can be used for communications with a host controller. The endpoint number and direction associated with an endpoint is directly related to its register designation. For example, when the Host is transmitting to endpoint 1, all configuration and data is in the endpoint 1 transmit register interface. Endpoint 0 is a dedicated control endpoint used for all control transactions to endpoint 0 during enumeration or when any other control requests are made to endpoint 0. Endpoint 0 uses the first 64 bytes of the USB controller's FIFO RAM as a shared memory for both IN and OUT transactions. The remaining six endpoints can be configured as control, bulk, or interrupt endpoints. They should be treated as three configurable IN and three configurable OUT endpoints. The endpoint pairs are not required to have the same type for their IN and OUT endpoint configuration. For example, the OUT portion of an endpoint pair could be a bulk endpoint, while the IN portion of that endpoint pair could be an interrupt endpoint. The address and size of the FIFOs attached to each endpoint can be modified to fit the application's needs.

#### 17.2.1.1.1 IN Transactions as a Device

When operating as a USB device, data for IN transactions is handled through the FIFOs attached to the transmit endpoints. The sizes of the FIFOs for the configurable IN endpoints are determined by the USB Transmit FIFO Start Address (USBTXFIFOADD) register. The maximum size of a data packet that may be placed in a transmit endpoint's FIFO for transmission is programmable and is determined by the value written to the USB Maximum Transmit Data Endpoint n (USBTXMAXPn) register for that endpoint. The endpoint's FIFO can also be configured to use double-packet or single-packet buffering. When double-packet buffering is enabled, two data packets can be buffered in the FIFO, which also requires that the FIFO is at least two packets in size. When double-packet buffering is disabled, only one packet can be buffered, even if the packet size is less than half the FIFO size.

**Note:** The maximum packet size set for any endpoint must not exceed the FIFO size. The USBTXMAXPn register should not be written to while data is in the FIFO as unexpected results may occur.

#### Single-Packet Buffering

If the size of the transmit endpoint's FIFO is less than twice the maximum packet size for this endpoint (as set in the USB Transmit Dynamic FIFO Sizing (USBTXFIFOSZ) register), only one packet can be buffered in the FIFO and single-packet buffering is required. When each packet is completely loaded into the transmit FIFO, the TXRDY bit in the USB Transmit Control and Status Endpoint n Low (USBTXCSRLn) register must be set. If the AUTOSET bit in the USB Transmit Control and Status Endpoint n High (USBTXCSRHn) register is set, the TXRDY bit is automatically set when a maximum-sized packet is loaded into the FIFO. For packet sizes less than the maximum, the TXRDY bit must be set manually. When the TXRDY bit is set, either manually or automatically, the packet is ready to be sent. When the packet has been successfully sent, both TXRDY and FIFONE are cleared, and the appropriate transmit endpoint interrupt signaled. At this point, the next packet can be loaded into the FIFO.

#### Double-Packet Buffering

If the size of the transmit endpoint's FIFO is at least twice the maximum packet size for this endpoint, two packets can be buffered in the FIFO and double-packet buffering is allowed. As each packet is loaded into the transmit FIFO, the TXRDY bit in the USBTXCSRLn register must be set. If the AUTOSET bit in the USBTXCSRHn register is set, the TXRDY bit is automatically set when a maximum-sized packet is loaded into the FIFO. For packet sizes less than the maximum, TXRDY must be set manually. When the TXRDY bit is set, either manually or automatically, the packet is ready to be sent. After the first packet is loaded, TXRDY is immediately cleared and an interrupt is generated. A second packet can now be loaded into the transmit FIFO and TXRDY set again (either manually or automatically if the packet is the maximum size). At this point, both packets are ready to be sent. After each packet has been successfully sent, TXRDY is automatically cleared and the appropriate transmit endpoint interrupt signaled to indicate that another packet can now be loaded into the transmit FIFO. The state of the FIFONE bit in the USBTXCSRLn register at this point indicates how many packets may be loaded. If the FIFONE bit is set, then another packet is in the FIFO and only one more packet can be loaded. If the FIFONE bit is clear, then no packets are in the FIFO and two more packets can be loaded.

**Note:** Double-packet buffering is disabled if an endpoint's corresponding EPn bit is set in the USB Transmit Double Packet Buffer Disable (USBTXDPKTBUDIS) register. This bit is set by default, so it must be cleared to enable double-packet buffering.

#### 17.2.1.1.2 Out Transactions as a Device

When in device mode, OUT transactions are handled through the USB controller receive FIFOs. The sizes of the receive FIFOs for the configurable OUT endpoints are determined by the USB Receive FIFO Start Address (USBRXFIFOADD) register. The maximum amount of data received by an endpoint in any packet is determined by the value written to the USB Maximum Receive Data Endpoint n (USBRXMAXPn) register for that endpoint. When double-packet buffering is enabled, two data packets can be buffered in the FIFO. When double-packet buffering is disabled, only one packet can be buffered even if the packet is less than half the FIFO size.

**Note:** In all cases, the maximum packet size must not exceed the FIFO size.

#### Single-Packet Buffering

If the size of the receive endpoint FIFO is less than twice the maximum packet size for an endpoint, only one data packet can be buffered in the FIFO and single-packet buffering is required. When a packet is received and placed in the receive FIFO, the RXRDY and FULL bits in the USB Receive Control and Status Endpoint n Low (USBRXCSSL[n]) register are set and the appropriate receive endpoint is signaled, indicating that a packet can now be unloaded from the FIFO. After the packet has been unloaded, the RXRDY bit must be cleared in order to allow further packets to be received. This action also generates the acknowledge signaling to the Host controller. If the AUTOCL bit in the USB Receive Control and Status Endpoint n High (USBRXCSSH[n]) register is set and a maximum-sized packet is unloaded from the FIFO, the RXRDY and FULL bits are cleared automatically. For packet sizes less than the maximum, RXRDY must be cleared manually.

#### Double-Packet Buffering

If the size of the receive endpoint FIFO is at least twice the maximum packet size for the endpoint, two data packets can be buffered and double-packet buffering can be used. When the first packet is received and loaded into the receive FIFO, the RXRDY bit in the USBXCSSL[n] register is set and the appropriate receive endpoint interrupt is signaled to indicate that a packet can now be unloaded from the FIFO.

**Note:** The FULL bit in USBXCSSL[n] is not set when the first packet is received. It is only set if a second packet is received and loaded into the receive FIFO.

After each packet has been unloaded, the RXRDY bit must be cleared to allow further packets to be received. If the AUTOCL bit in the USBXCSSH[n] register is set and a maximum-sized packet is unloaded from the FIFO, the RXRDY bit is cleared automatically. For packet sizes less than the maximum, RXRDY must be cleared manually. If the FULL bit is set when RXRDY is cleared, the USB controller first clears the FULL bit, then sets RXRDY again to indicate that there is another packet waiting in the FIFO to be unloaded.

**Note:** Double-packet buffering is disabled if an endpoint's corresponding EPn bit is set in the USB Receive Double Packet Buffer Disable (USBRXDPKTBUDIS) register. This bit is set by default, so it must be cleared to enable double-packet buffering.

#### 17.2.1.1.3 Scheduling

The device has no control over the scheduling of transactions as scheduling is determined by the Host controller. The USB controller can set up a transaction at any time. The USB controller waits for the request from the Host controller and generates an interrupt when the transaction is complete or if it was terminated due to some error. If the Host controller makes a request and the device controller is not ready, the USB controller sends a busy response (NAK) to all requests until it is ready.

#### 17.2.1.1.4 Additional Actions

The USB controller responds automatically to certain conditions on the USB bus or actions by the Host controller such as when the USB controller automatically stalls a control transfer or unexpected zero length OUT data packets.

#### Stalled Control Transfer

The USB controller automatically issues a STALL handshake to a control transfer under the following conditions:

1. The Host sends more data during an OUT data phase of a control transfer than was specified in the device request during the SETUP phase. This condition is detected by the USB controller when the Host sends an OUT token (instead of an IN token) after the last OUT packet has been unloaded and the DATAEND bit in the USB Control and Status Endpoint 0 Low (USBCSRL0) register has been set.
2. The Host requests more data during an IN data phase of a control transfer than was specified in the device request during the SETUP phase. This condition is detected by the USB controller when the Host sends an IN token (instead of an OUT token) after the CPU has cleared TXRDY and set DATAEND in response to the ACK issued by the Host to what should have been the last packet.
3. The Host sends more than USBRXMAXPn bytes of data with an OUT data token.
4. The Host sends more than a zero length data packet for the OUT STATUS phase.

#### Zero Length OUT Data Packets

A zero-length OUT data packet is used to indicate the end of a control transfer. In normal operation, such packets should only be received after the entire length of the device request has been transferred. However, if the Host sends a zero-length OUT data packet before the entire length of device request has been transferred, it is signaling the premature end of the transfer. In this case, the USB controller automatically flushes any IN token ready for the data phase from the FIFO and sets the DATAEND bit in the USBCSRL0 register.

#### Setting the Device Address

When a Host is attempting to enumerate the USB device, it requests that the device change its address from zero to some other value. The address is changed by writing the value that the Host requested to the USB Device Functional Address (USBFADDR) register. However, care should be taken when writing to USBFADDR to avoid changing the address before the transaction is complete. This register should only be set after the SET\_ADDRESS command is complete. Like all control transactions, the transaction is only complete after the device has left the STATUS phase. In the case of a SET\_ADDRESS command, the transaction is completed by responding to the IN request from the Host with a zero-byte packet. Once the device has responded to the IN request, the USBFADDR register should be programmed to the new value as soon as possible to avoid missing any new commands sent to the new address.

**Note:** If the USBFADDR register is set to the new value as soon as the device receives the OUT transaction with the SET\_ADDRESS command in the packet, it changes the address during the control transfer. In this case, the device does not receive the IN request that allows the USB transaction to exit the STATUS phase of the control transfer because it is sent to the old address. As a result, the Host does not get a response to the IN request, and the Host fails to enumerate the device.



#### 17.2.1.1.5 Device Mode Suspend

When no activity has occurred on the USB bus for 3 ms, the USB controller automatically enters SUSPEND mode. If the SUSPEND interrupt has been enabled in the USB Interrupt Enable (USBIE) register, an interrupt is generated at this time. When in SUSPEND mode, the PHY also goes into SUSPEND mode. When RESUME signaling is detected, the USB controller exits SUSPEND mode and takes the PHY out of SUSPEND. If the RESUME interrupt is enabled, an interrupt is generated. The USB controller can also be forced to exit SUSPEND mode by setting the RESUME bit in the USB Power (USBPOWER) register. When this bit is set, the USB controller exits SUSPEND mode and drives RESUME signaling onto the bus. The RESUME bit must be cleared after 10 ms (a maximum of 15 ms) to end RESUME signaling. To meet USB power requirements, the controller can be put into Deep Sleep mode which keeps the controller in a static state.

#### 17.2.1.1.6 Start of Frame

When the USB controller is operating in device mode, it receives a Start-Of-Frame (SOF) packet from the Host once every millisecond. When the SOF packet is received, the 11-bit frame number contained in the packet is written into the USB Frame Value (USBFRAME) register, and an SOF interrupt is also signaled and can be handled by the application. Once the USB controller has started to receive SOF packets, it expects one every millisecond. If no SOF packet is received after 1.00358 ms, the packet is assumed to have been lost, and the USBFRAME register is not updated. The USB controller continues and resynchronizes these pulses to the received SOF packets when these packets are successfully received again.

#### 17.2.1.1.7 USB Reset

When the USB controller is in device mode and a RESET condition is detected on the USB bus, the USB controller automatically performs the following actions:

- Clears the USBFADDR register
- Clears the USB Endpoint Index (USBEPIDX) register
- Flushes all endpoint FIFOs
- Clears all control/status registers
- Enables all endpoint interrupts
- Generates a RESET interrupt

#### 17.2.1.1.8 Connect/Disconnect

The USB controller connection to the USB bus is handled by software. The USB PHY can be switched between normal mode and non-driving mode by setting or clearing the SOFTCONN bit of the USBPOWER register. When the SOFTCONN bit is set, the PHY is placed in its normal mode, and the USB0DP/USB0DM lines of the USB bus are enabled. At the same time, the USB controller is placed into a state, in which it does not respond to any USB signaling except a USB RESET. When the SOFTCONN bit is cleared, the PHY is put into non-driving mode, USB0DP and USB0DM are tristated, and the USB controller appears to other devices on the USB bus as if it has been disconnected. The non-driving mode is the default so the USB controller appears disconnected until the SOFTCONN bit has been set. The application software can then choose when to set the PHY into its normal mode. Systems with a lengthy initialization procedure may use this to ensure that initialization is complete, and the system is ready to perform enumeration before connecting to the USB bus. Once the SOFTCONN bit has been set, the USB controller can be disconnected by clearing this bit.

**Note:** The USB controller does not generate an interrupt when the device is connected to the Host. However, an interrupt is generated when the Host terminates a session.

### 17.2.2 Operation as a Host

When the USB controller is operating in Host mode, it can either be used for point-to-point communications with another USB device or, when attached to a hub, for communication with multiple devices. Full-speed and low-speed USB devices are supported, both for point-to-point communication and for operation through a hub. The USB controller automatically carries out the necessary transaction translation needed to allow a low-speed or full-speed device to be used with a USB 2.0 hub. Control, bulk, and interrupt transactions are supported. This section describes the USB controller's actions when it is being used as a USB Host. Configuration of IN endpoints, OUT endpoints, entry into and exit from SUSPEND mode, and RESET are all described.



When in Host mode, IN transactions are controlled by an endpoint's receive interface. All IN transactions use the receive endpoint registers and all OUT endpoints use the transmit endpoint registers for a given endpoint. As in device mode, the FIFOs for endpoints should take into account the maximum packet size for an endpoint.

- Bulk endpoints should be the size of the maximum packet (up to 64 bytes) or twice the maximum packet size if double buffering is used (described further in the following section).
- Interrupt endpoints should be the size of the maximum packet (up to 64 bytes) or twice the maximum packet size if double buffering is used.
- It is also possible to specify a separate control endpoint to communicate with a device. However, in most cases the USB controller should use the dedicated control endpoint to communicate with a device's endpoint 0.

### 17.2.2.1 Endpoint Registers

The endpoint registers are used to control the USB endpoint interfaces which communicate with device(s) that are connected. The endpoints consist of a dedicated control IN endpoint and a dedicated control OUT endpoint. The remaining available endpoints are configurable, with one-half of them being OUT endpoints, and one-half of them being IN endpoints. See [Section 17.1.1](#) for the number of available endpoints on this device.

The dedicated control interface can only be used for control transactions to endpoint 0 of devices. These control transactions are used during enumeration or other control functions that communicate using endpoint 0 of devices. This control endpoint shares the first 64 bytes of the USB controller's FIFO RAM for IN and OUT transactions. The remaining IN and OUT interfaces can be configured to communicate with control, bulk, or interrupt endpoints.

These USB interfaces can be used to simultaneously schedule as many as three independent OUT and three independent IN transactions to any endpoints on any device. The IN and OUT controls are paired together in the same set of registers for the respective endpoints. However, they can be configured to communicate with different types of endpoints and different endpoints on devices. For example, the first pair of endpoint controls can be split so that the OUT portion is communicating with a device's bulk OUT endpoint 1, while the IN portion is communicating with a device's interrupt IN endpoint 2.

Before accessing any device, whether for point-to-point communications or for communications via a hub, the relevant USB Receive Functional Address Endpoint *n* (USB<sub>RXF</sub>ADDRn) or USB Transmit Functional Address Endpoint *n* (USB<sub>TXF</sub>ADDRn) registers must be set for each receive or transmit endpoint to record the address of the device being accessed.

The USB controller also supports connections to devices through a USB hub by providing a register that specifies the hub address and port of each USB transfer. The FIFO address and size are customizable and can be specified for each USB IN and OUT transfer. Customization includes allowing one FIFO per transaction, sharing a FIFO across transactions, and allowing for double-buffered FIFOs.

### 17.2.2.2 IN Transactions as a Host

IN transactions are handled in a similar manner to the way in which OUT transactions are handled when the USB controller is in device mode except that the transaction first must be initiated by setting the REQPKT bit in the USB<sub>CS</sub>RL0 register, indicating to the transaction scheduler that there is an active transaction on this endpoint. The transaction scheduler then sends an IN token to the target device. When the packet is received and placed in the receive FIFO, the RXRDY bit in the USB<sub>CS</sub>RL0 register is set, and the appropriate receive endpoint interrupt is signaled to indicate that a packet can now be unloaded from the FIFO.

When the packet has been unloaded, RXRDY must be cleared. The AUTOCL bit in the USB<sub>RX</sub>CSRn register can be used to have RXRDY automatically cleared when a maximum-sized packet has been unloaded from the FIFO. The AUTORQ bit in USB<sub>RX</sub>CSRn causes the REQPKT bit to be automatically set when the RXRDY bit is cleared. When the RXRDY bit is cleared, the controller sends an acknowledge to the device. When there is a known number of packets to be transferred, the USB Request Packet Count in Block Transfer Endpoint *n* (USB<sub>RQPKT</sub>COUNTn) register associated with the endpoint should be configured to the number of packets to be transferred. The USB controller decrements the value in the USB<sub>RQPKT</sub>COUNTn register following each request. When the USB<sub>RQPKT</sub>COUNTn value decrements to 0, the AUTORQ bit is cleared to prevent any further transactions being attempted. For cases where the size of the transfer is unknown, USB<sub>RQPKT</sub>COUNTn

should be cleared. AUTORQ then remains set until cleared by the reception of a short packet (that is, less than the MAXLOAD value in the USBRXMAXPn register) such as may occur at the end of a bulk transfer.

If the device responds to a bulk or interrupt IN token with a NAK, the USB Host controller keeps retrying the transaction until any NAK Limit that has been set has been reached. If the target device responds with a STALL, however, the USB Host controller does not retry the transaction but sets the STALLED bit in the USBCSRLO register. If the target device does not respond to the IN token within the required time, or the packet contained a CRC or bit-stuff error, the USB Host controller retries the transaction. If after three attempts the target device has still not responded, the USB Host controller clears the REQPKT bit and sets the ERROR bit in the USBCSRLO register.

### 17.2.2.3 OUT Transactions as a Host

OUT transactions are handled in a similar manner to the way in which IN transactions are handled when the USB controller is in device mode. The TXRDY bit in the USBTXCSRLn register must be set as each packet is loaded into the transmit FIFO. Again, setting the AUTOSET bit in the USBTXCSRHn register automatically sets TXRDY when a maximum-sized packet has been loaded into the FIFO.

If the target device responds to the OUT token with a NAK, the USB Host controller keeps retrying the transaction until the NAK Limit that has been set has been reached. However, if the target device responds with a STALL, the USB controller does not retry the transaction but interrupts the main processor by setting the STALLED bit in the USBTXCSRLn register. If the target device does not respond to the OUT token within the required time, or the packet contained a CRC or bit-stuff error, the USB Host controller retries the transaction. If after three attempts the target device has still not responded, the USB controller flushes the FIFO and sets the ERROR bit in the USBTXCSRLn register.

### 17.2.2.4 Transaction Scheduling

Scheduling of transactions is handled automatically by the USB Host controller. The Host controller allows configuration of the endpoint communication scheduling based on the type of endpoint transaction. Interrupt transactions can be scheduled to occur in the range of every frame to every 255 frames in 1 frame increments. Bulk endpoints do not allow scheduling parameters, but do allow for a NAK timeout in the event an endpoint on a device is not responding.

The USB controller maintains a frame counter. If the target device is a full-speed device, the USB controller automatically sends an SOF packet at the start of each frame and increments the frame counter. If the target device is a low-speed device, a K state is transmitted on the bus to act as a keep-alive to stop the low-speed device from going into SUSPEND mode.

After the SOF packet has been transmitted, the USB Host controller cycles through all the configured endpoints looking for active transactions. An active transaction is defined as a receive endpoint for which the REQPKT bit is set or a transmit endpoint for which the TXRDY bit and/or the FIFONE bit is set.

An interrupt transaction is started if the transaction is found on the first scheduler cycle of a frame and if the interval counter for that endpoint has counted down to zero. As a result, only one interrupt transaction occurs per endpoint every n frames, where n is the interval set via the USB Host Transmit Interval Endpoint n (USBTXINTERVAL[n]) or USB Host Receive Interval Endpoint n (USBRXINTERVAL[n]) register for that endpoint.

An active bulk transaction starts immediately, provided sufficient time is left in the frame to complete the transaction before the next SOF packet is due. If the transaction must be retried (for example, because a NAK was received or the target device did not respond), then the transaction is not retried until the transaction scheduler has first checked all the other endpoints for active transactions. This process ensures that an endpoint that is sending a lot of NAKs does not block other transactions on the bus. The controller also allows the user to specify a limit to the length of time for NAKs to be received from a target device before the endpoint times out.

### 17.2.2.5 USB Hubs

The following setup requirements apply to the USB Host controller only if it is used with a USB hub. When a full- or low-speed device is connected to the USB controller via a USB 2.0 hub, details of the hub address and the hub port also must be recorded in the corresponding USB Receive Hub Address Endpoint

n (USBRXHUBADDRn) and USB Receive Hub Port Endpoint n (USBRXHUBPORTn) or the USB Transmit Hub Address Endpoint n (USBTXHUBADDRn) and USB Transmit Hub Port Endpoint n (USBTXHUBPORTn) registers. In addition, the speed at which the device operates (full or low) must be recorded in the USB Type Endpoint 0 (USBTYPE0) (endpoint 0), USB Host Configure Transmit Type Endpoint n (USBTXTYPEn), or USB Host Configure Receive Type Endpoint n (USBRXTYPEn) registers for each endpoint that is accessed by the device.

For hub communications, the settings in these registers record the current allocation of the endpoints to the attached USB devices. To maximize the number of devices supported, the USB Host controller allows this allocation to be changed dynamically by simply updating the address and speed information recorded in these registers. Any changes in the allocation of endpoints to device functions must be made following the completion of any on-going transactions on the endpoints affected.

#### **17.2.2.6 Babble**

The USB Host controller does not start a transaction until the bus has been inactive for at least the minimum inter-packet delay. The controller also does not start a transaction unless it can be finished before the end of the frame. If the bus is still active at the end of a frame, then the USB Host controller assumes that the target device to which it is connected has malfunctioned, and the USB controller suspends all transactions and generates a babble interrupt.

#### **17.2.2.7 Host SUSPEND**

If the SUSPEND bit in the USBPOWER register is set, the USB Host controller completes the current transaction then stops the transaction scheduler and frame counter. No further transactions are started and no SOF packets are generated.

To exit SUSPEND mode, set the RESUME bit and clear the SUSPEND bit. While the RESUME bit is set, the USB Host controller generates RESUME signaling on the bus. After 20 ms, the RESUME bit must be cleared, at which point the frame counter and transaction scheduler start. The Host supports the detection of a remote wake-up.

#### **17.2.2.8 USB RESET**

If the RESET bit in the USBPOWER register is set, the USB Host controller generates USB RESET signaling on the bus. The RESET bit must be set for at least 20 ms to ensure correct resetting of the target device. After the CPU has cleared the bit, the USB Host controller starts its frame counter and transaction scheduler.

#### **17.2.2.9 Connect/Disconnect**

A session is started by setting the SESSION bit in the USB device Control (USBDEVCTL) register, enabling the USB controller to wait for a device to be connected. When a device is detected, a connect interrupt is generated. The speed of the device that has been connected can be determined by reading the USBDEVCTL register where the FSDEV bit is set for a full-speed device, and the LSDEV bit is set for a low-speed device. The USB controller must generate a RESET to the device, and then the USB Host controller can begin device enumeration. If the device is disconnected while a session is in progress, a disconnect interrupt is generated.

### 17.2.3 DMA Operation

The USB module's DMA trigger signals are not supported on this device. The DMA controller may be used to read and write the USB FIFOs via software triggering. see the Direct Memory Access (DMA) chapter for more details about programming the DMA controller. See the *USB DMA Event Trigger* advisory in the device errata for more information.

### 17.2.4 Address/Data Bus Bridge

The USB controller on this device is the same controller that is on the Stellaris devices. This controller was originally designed to connect to an ARM AHB bus, but has been modified in order to function with the C28x device's bus architecture. The modifications made are largely invisible to the user application, but there are some things to note.

- The USB memory space is 8 bits wide, while the C28x memory space is 16 bits wide.
- 32- and 16-bit accesses (r/w) are completely transparent to the user application code, no changes need be made.
- The C28x core only supports 8 bit accesses through a byte intrinsic type. This can be used to perform 8 bit reads or writes to the USB controller.
  - `int &__byte(int *array, unsigned int byte_index);`
  - `*array = ptr to address to access, byte_index = always 0 (for USB)`  
See [Table 17-1](#) for example.
  - See the [TMS320C28x Optimizing C/C++ Compiler User's Guide](#) and the [TMS320C28x Assembly Language Tools User's Guide](#)
- Because of the bridge, the memory view of the USB controller memory space in CCS isn't a 1:1 representation of what is in the controller
  - When the view mode is
    - 32 bit or 16 bit, even address are effectively duplicated, ignore odd addresses.
    - 8 bit, Even addresses from within the controller are duplicated into odd address in the view window; odd addresses from within the controller are not displayed.  
See [Table 17-2](#) for example.

**Table 17-1. USB Memory Access From Software**

| USB Controller Memory |               |      | C28x 8 Bit              |            |
|-----------------------|---------------|------|-------------------------|------------|
| Address               | Register Name | Data | Access                  | Data       |
| 0x00                  | FADDR         | 0x00 | __byte__((int *)0x00,0) | 0x0000     |
| 0x01                  | POWER         | 0x11 | __byte__((int *)0x01,0) | 0x0011     |
| 0x02                  | TXIS (LSB)    | 0x22 | __byte__((int *)0x02,0) | 0x0022     |
| 0x03                  | TXIS (MSB)    | 0x33 | __byte__((int *)0x03,0) | 0x0033     |
| 0x04                  | RXIS (LSB)    | 0x44 | __byte__((int *)0x04,0) | 0x0044     |
| 0x05                  | RXIS (MSB)    | 0x55 | __byte__((int *)0x05,0) | 0x0055     |
| 0x06                  | TXIE (LSB)    | 0x66 | __byte__((int *)0x06,0) | 0x0066     |
| 0x07                  | TXIE (MSB)    | 0x77 | __byte__((int *)0x07,0) | 0x0077     |
| 0x08                  | RXIE (LSB)    | 0x88 | __byte__((int *)0x08,0) | 0x0088     |
| 0x09                  | RXIE (MSB)    | 0x99 | __byte__((int *)0x09,0) | 0x0099     |
| 0x0A                  | USBIS         | 0xAA | __byte__((int *)0x0A,0) | 0x00AA     |
| 0x0B                  | USBIE         | 0xBB | __byte__((int *)0x0B,0) | 0x00BB     |
| 0x0C                  | FRAME (LSB)   | 0xCC | __byte__((int *)0x0C,0) | 0x00CC     |
| 0x0D                  | FRAME (MSB)   | 0xDD | __byte__((int *)0x0D,0) | 0x00DD     |
| 0x0E                  | EPIDX         | 0xEE | __byte__((int *)0x0E,0) | 0x00EE     |
| 0x0F                  | TEST          | 0xFF | __byte__((int *)0x0F,0) | 0x00FF     |
| C28x 16 Bit           |               |      | C28x 32 Bit             |            |
| Access                | Data          |      | Access                  | Data       |
| ((short *)0x00))      | 0x1100        |      | ((long *)0x00))         | 0x33221100 |
| ((short *)0x01))      | 0x1100        |      | ((long *)0x01))         | 0x33221100 |
| ((short *)0x02))      | 0x3322        |      | ((long *)0x02))         | 0x33221100 |
| ((short *)0x03))      | 0x3322        |      | ((long *)0x03))         | 0x33221100 |
| ((short *)0x04))      | 0x5544        |      | ((long *)0x04))         | 0x77665544 |
| ((short *)0x05))      | 0x5544        |      | ((long *)0x05))         | 0x77665544 |
| ((short *)0x06))      | 0x7766        |      | ((long *)0x06))         | 0x77665544 |
| ((short *)0x07))      | 0x7766        |      | ((long *)0x07))         | 0x77665544 |
| ((short *)0x08))      | 0x9988        |      | ((long *)0x08))         | 0xBBAA9988 |
| ((short *)0x09))      | 0x9988        |      | ((long *)0x09))         | 0xBBAA9988 |
| ((short *)0x0A))      | 0xBBAA        |      | ((long *)0x0A))         | 0xBBAA9988 |
| ((short *)0x0B))      | 0xBBAA        |      | ((long *)0x0B))         | 0xBBAA9988 |
| ((short *)0x0C))      | 0xDDCC        |      | ((long *)0x0C))         | 0xFFEEDDCC |
| ((short *)0x0D))      | 0xDDCC        |      | ((long *)0x0D))         | 0xFFEEDDCC |
| ((short *)0x0E))      | 0xFFEE        |      | ((long *)0x0E))         | 0xFFEEDDCC |
| ((short *)0x0F))      | 0xFFEE        |      | ((long *)0x0F))         | 0xFFEEDDCC |

**Table 17-2. USB Memory Access From CCS**

| CCS 8 Bit |                | CCS 16 Bit |                | CCS 32 Bit |                |
|-----------|----------------|------------|----------------|------------|----------------|
| Address   | Displayed Data | Address    | Displayed Data | Address    | Displayed Data |
| 0x00      | 0x00           | 0x00       | 0x1100         | 0x00       | 0x11001100     |
| 0x01      | 0x00           | 0x01       | 0x1100         | 0x02       | 0x33223322     |
| 0x02      | 0x22           | 0x02       | 0x3322         | 0x04       | 0x55445544     |
| 0x03      | 0x22           | 0x03       | 0x3322         | 0x06       | 0x77667766     |
| 0x04      | 0x44           | 0x04       | 0x5544         | 0x08       | 0x99889988     |
| 0x05      | 0x44           | 0x05       | 0x5544         | 0x0A       | 0xBBAABBAA     |
| 0x06      | 0x66           | 0x06       | 0x7766         | 0x0C       | 0xDDCCDDCC     |
| 0x07      | 0x66           | 0x07       | 0x7766         | 0x0E       | 0xFFEEFFEE     |
| 0x08      | 0x88           | 0x08       | 0x9988         |            |                |
| 0x09      | 0x88           | 0x09       | 0x9988         |            |                |
| 0x0A      | 0xAA           | 0x0A       | 0xBBAA         |            |                |
| 0x0B      | 0xAA           | 0x0B       | 0xBBAA         |            |                |
| 0x0C      | 0xCC           | 0x0C       | 0xDDCC         |            |                |
| 0x0D      | 0xCC           | 0x0D       | 0xDDCC         |            |                |
| 0x0E      | 0xEE           | 0x0E       | 0xFFEE         |            |                |
| 0x0F      | 0xEE           | 0x0F       | 0xFFEE         |            |                |

## 17.3 Initialization and Configuration

To use the USB Controller, the clock for the USB controller must first be configured. USBCLK is driven via the second PLL within the chip. The PLL should be configured to operate at 60MHz via the PLL2CTL, PLL2MULT, PLL2STS registers. After the PLL is enabled and locked, the controller can be clocked by enabling the peripheral in the PCLKCR3 register. In addition to the clock, the USB PHY must be enabled. Configure the USBIOEN field in the GPMCTRL2 register to enable USB functionality on the designated pins. After the clock has been enabled and the USB PHY turned on the USB peripheral is ready for operation and the associated software initialization routines may be called.

### 17.3.1 Pin Configuration

In order to give the user more flexibility, the signals External Power Enable (EPEN) and Power Fault (PFLT) were not implemented in hardware. Instead, it is left up to the user to implement these signals in software. Examples of how to implement these signals in software can be found in the F2806x USB Software Guide.

When using the device controller portion of the USB controller in a system that also provides host functionality, the power to  $V_{BUS}$  must be disabled to allow the external host controller to supply power. Usually, the EPEN signal is used to control the external regulator and should be negated to avoid having two devices driving the  $V_{BUS}$  power pin on the USB connector.

When the USB controller is acting as a host, it is in control of two signals that are attached to an external voltage supply that provides power to  $V_{BUS}$ . The Host controller uses the EPEN signal to enable or disable power to the  $V_{BUS}$  pin on the USB connector. An input pin, PFLT, provides feedback when there has been a power fault on  $V_{BUS}$ . The PFLT signal can be configured to either automatically negate the EPEN signal to disable power, and/or it can generate an interrupt to the interrupt controller to allow software to handle the power fault condition. The polarity and actions related to both EPEN and PFLT are fully configurable in the USB controller. The controller also provides interrupts on device insertion and removal to allow the Host controller code to respond to these external events.



### 17.3.2 Endpoint Configuration

To start communication in Host or device mode, the endpoint registers must first be configured. In Host mode, this configuration establishes a connection between an endpoint register and an endpoint on a device. In device mode, an endpoint must be configured before enumerating to the Host controller.

In both cases, the endpoint 0 configuration is limited because it is a fixed-function, fixed-FIFO-size endpoint. In device and Host modes, the endpoint requires little setup but does require a software-based state machine to progress through the setup, data, and status phases of a standard control transaction. In device mode, the configuration of the remaining endpoints is done once before enumerating and then only changed if an alternate configuration is selected by the Host controller. In Host mode, the endpoints must be configured to operate as control, bulk, or interrupt mode. Once the type of endpoint is configured, a FIFO area must be assigned to each endpoint. In the case of bulk, control and interrupt endpoints, each has a maximum of 64 bytes per transaction. The maximum packet size for the given endpoint must be set prior to sending or receiving data.

Configuring each endpoint's FIFO involves reserving a portion of the overall USB FIFO RAM to each endpoint. The total FIFO RAM available is 4 Kbytes with the first 64 bytes reserved for endpoint 0. The endpoint's FIFO must be at least as large as the maximum packet size. The FIFO can also be configured as a double-buffered FIFO so that interrupts occur at the end of each packet and allow filling the other half of the FIFO.

If operating as a device, the USB device controller's soft connect must be enabled when the device is ready to start communications, indicating to the host controller that the device is ready to start the enumeration process. If operating as a Host controller, the device soft connect must be disabled and power must be provided to  $V_{BUS}$  via the USB0EPEN signal.

### 17.4 USB Global Interrupts

Global interrupt enable, flag, and clear registers have been added to ensure that no interrupt is missed. The USB interrupt can be enabled or blocked using the INTEN bit. The INTFLG bit indicates whether an interrupt has occurred or not. Finally the INTFLGCLR bit will clear the INTFLG when the value '1' is written to the field.

### 17.5 USB Registers

[Table 17-3](#) lists the registers. All addresses given are relative to the USB base address of 0x4000. Note that the USB controller clock must be enabled before the registers can be programmed (see [Chapter 1](#)).

**Table 17-3. Universal Serial Bus (USB) Controller Register Map**

| Offset | Name                           | Type | Reset       | Description                      | Section                         |
|--------|--------------------------------|------|-------------|----------------------------------|---------------------------------|
| 0x000  | USBFADDR <sup>(1)</sup>        | R/W  | 0x00        | USB Device Functional Address    | <a href="#">Section 17.5.1</a>  |
| 0x001  | USBPOWER <sup>(1) (2)</sup>    | R/W  | 0x20        | USB Power                        | <a href="#">Section 17.5.2</a>  |
| 0x002  | USBTXIS <sup>(1) (2)</sup>     | RO   | 0x0000      | USB Transmit Interrupt Status    | <a href="#">Section 17.5.3</a>  |
| 0x004  | USBRXIS <sup>(1) (2)</sup>     | RO   | 0x0000      | USB Receive Interrupt Status     | <a href="#">Section 17.5.4</a>  |
| 0x006  | USBTXIE <sup>(1) (2)</sup>     | R/W  | 0xFFFF      | USB Transmit Interrupt Enable    | <a href="#">Section 17.5.5</a>  |
| 0x008  | USBRXIE <sup>(1) (2)</sup>     | R/W  | 0xFFFE      | USB Receive Interrupt Enable     | <a href="#">Section 17.5.6</a>  |
| 0x00A  | USBIS <sup>(1) (2)</sup>       | RO   | 0x00        | USB General Interrupt Status     | <a href="#">Section 17.5.7</a>  |
| 0x00B  | USBIE <sup>(1) (2)</sup>       | R/W  | 0x06        | USB Interrupt Enable             | <a href="#">Section 17.5.8</a>  |
| 0x00C  | USBFRAME <sup>(1) (2)</sup>    | RO   | 0x0000      | USB Frame Value                  | <a href="#">Section 17.5.9</a>  |
| 0x00E  | USBEPIDX <sup>(1) (2)</sup>    | R/W  | 0x00        | USB Endpoint Index               | <a href="#">Section 17.5.10</a> |
| 0x00F  | USBTEST <sup>(1) (2)</sup>     | R/W  | 0x00        | USB Test Mode                    | <a href="#">Section 17.5.11</a> |
| 0x020  | USBFIFO0 <sup>(1) (2)</sup>    | R/W  | 0x0000.0000 | USB FIFO Endpoint 0              | <a href="#">Section 17.5.12</a> |
| 0x024  | USBFIFO1 <sup>(1) (2)</sup>    | R/W  | 0x0000.0000 | USB FIFO Endpoint 1              | <a href="#">Section 17.5.12</a> |
| 0x028  | USBFIFO2 <sup>(1) (2)</sup>    | R/W  | 0x0000.0000 | USB FIFO Endpoint 2              | <a href="#">Section 17.5.12</a> |
| 0x02C  | USBFIFO3 <sup>(1) (2)</sup>    | R/W  | 0x0000.0000 | USB FIFO Endpoint 3              | <a href="#">Section 17.5.12</a> |
| 0x060  | USBDEVCTL <sup>(2)</sup>       | R/W  | 0x80        | USB Device Control               | <a href="#">Section 17.5.13</a> |
| 0x062  | USBTXFIFOSZ <sup>(1) (2)</sup> | R/W  | 0x00        | USB Transmit Dynamic FIFO Sizing | <a href="#">Section 17.5.14</a> |
| 0x063  | USBRXFIFOSZ <sup>(1) (2)</sup> | R/W  | 0x00        | USB Receive Dynamic FIFO Sizing  | <a href="#">Section 17.5.15</a> |

**Table 17-3. Universal Serial Bus (USB) Controller Register Map (continued)**

| Offset | Name                            | Type | Reset  | Description                                            | Section                         |
|--------|---------------------------------|------|--------|--------------------------------------------------------|---------------------------------|
| 0x064  | USBTXFIFOADD <sup>(1) (2)</sup> | R/W  | 0x0000 | USB Transmit FIFO Start Address                        | <a href="#">Section 17.5.16</a> |
| 0x066  | USBRXFIFOADD <sup>(1) (2)</sup> | R/W  | 0x0000 | USB Receive FIFO Start Address                         | <a href="#">Section 17.5.17</a> |
| 0x07A  | USBCONTIM <sup>(1) (2)</sup>    | R/W  | 0x5C   | USB Connect Timing                                     | <a href="#">Section 17.5.18</a> |
| 0x07D  | USBFSEOF <sup>(1) (2)</sup>     | R/W  | 0x77   | USB Full-Speed Last Transaction to End of Frame Timing | <a href="#">Section 17.5.19</a> |
| 0x07E  | USBLSEOF <sup>(1) (2)</sup>     | R/W  | 0x72   | USB Low-Speed Last Transaction to End of Frame Timing  | <a href="#">Section 17.5.20</a> |
| 0x080  | USBTXFUNCADDR0 <sup>(2)</sup>   | R/W  | 0x00   | USB Transmit Functional Address Endpoint 0             | <a href="#">Section 17.5.21</a> |
| 0x082  | USBTXHUBADDR0 <sup>(2)</sup>    | R/W  | 0x00   | USB Transmit Hub Address Endpoint 0                    | <a href="#">Section 17.5.22</a> |
| 0x083  | USBTXHUBPORT0 <sup>(2)</sup>    | R/W  | 0x00   | USB Transmit Hub Port Endpoint 0                       | <a href="#">Section 17.5.23</a> |
| 0x088  | USBTXFUNCADDR1 <sup>(2)</sup>   | R/W  | 0x00   | USB Transmit Functional Address Endpoint 1             | <a href="#">Section 17.5.21</a> |
| 0x08A  | USBTXHUBADDR1 <sup>(2)</sup>    | R/W  | 0x00   | USB Transmit Hub Address Endpoint 1                    | <a href="#">Section 17.5.22</a> |
| 0x08B  | USBTXHUBPORT1 <sup>(2)</sup>    | R/W  | 0x00   | USB Transmit Hub Port Endpoint 1                       | <a href="#">Section 17.5.23</a> |
| 0x08C  | USBRXFUNCADDR1 <sup>(2)</sup>   | R/W  | 0x00   | USB Receive Functional Address Endpoint 1              | <a href="#">Section 17.5.24</a> |
| 0x08E  | USBRXHUBADDR1 <sup>(2)</sup>    | R/W  | 0x00   | USB Receive Hub Address Endpoint 1                     | <a href="#">Section 17.5.25</a> |
| 0x08F  | USBRXHUBPORT1 <sup>(2)</sup>    | R/W  | 0x00   | USB Receive Hub Port Endpoint 1                        | <a href="#">Section 17.5.26</a> |
| 0x090  | USBTXFUNCADDR2 <sup>(2)</sup>   | R/W  | 0x00   | USB Transmit Functional Address Endpoint 2             | <a href="#">Section 17.5.21</a> |
| 0x092  | USBTXHUBADDR2 <sup>(2)</sup>    | R/W  | 0x00   | USB Transmit Hub Address Endpoint 2                    | <a href="#">Section 17.5.22</a> |
| 0x093  | USBTXHUBPORT2 <sup>(2)</sup>    | R/W  | 0x00   | USB Transmit Hub Port Endpoint 2                       | <a href="#">Section 17.5.23</a> |
| 0x094  | USBRXFUNCADDR2 <sup>(2)</sup>   | R/W  | 0x00   | USB Receive Functional Address Endpoint 2              | <a href="#">Section 17.5.24</a> |
| 0x096  | USBRXHUBADDR2 <sup>(2)</sup>    | R/W  | 0x00   | USB Receive Hub Address Endpoint 2                     | <a href="#">Section 17.5.25</a> |
| 0x097  | USBRXHUBPORT2 <sup>(2)</sup>    | R/W  | 0x00   | USB Receive Hub Port Endpoint 2                        | <a href="#">Section 17.5.26</a> |
| 0x098  | USBTXFUNCADDR3 <sup>(2)</sup>   | R/W  | 0x00   | USB Transmit Functional Address Endpoint 3             | <a href="#">Section 17.5.21</a> |
| 0x09A  | USBTXHUBADDR3 <sup>(2)</sup>    | R/W  | 0x00   | USB Transmit Hub Address Endpoint 3                    | <a href="#">Section 17.5.22</a> |
| 0x09B  | USBTXHUBPORT3 <sup>(2)</sup>    | R/W  | 0x00   | USB Transmit Hub Port Endpoint 3                       | <a href="#">Section 17.5.23</a> |
| 0x09C  | USBRXFUNCADDR3 <sup>(2)</sup>   | R/W  | 0x00   | USB Receive Functional Address Endpoint 3              | <a href="#">Section 17.5.24</a> |
| 0x09E  | USBRXHUBADDR3 <sup>(2)</sup>    | R/W  | 0x00   | USB Receive Hub Address Endpoint 3                     | <a href="#">Section 17.5.25</a> |
| 0x09F  | USBRXHUBPORT3 <sup>(2)</sup>    | R/W  | 0x00   | USB Receive Hub Port Endpoint 3                        | <a href="#">Section 17.5.26</a> |
| 0x102  | USBCSRL0 <sup>(1) (2)</sup>     | W1C  | 0x00   | USB Control and Status Endpoint 0 Low                  | <a href="#">Section 17.5.28</a> |
| 0x103  | USBCSRH0 <sup>(1) (2)</sup>     | W1C  | 0x00   | USB Control and Status Endpoint 0 High                 | <a href="#">Section 17.5.29</a> |
| 0x108  | USBCOUNT0 <sup>(1) (2)</sup>    | R/o  | 0x00   | USB Receive Byte Count Endpoint 0                      | <a href="#">Section 17.5.30</a> |
| 0x10A  | USBTTYPE0 <sup>(2)</sup>        | R/W  | 0x00   | USB Type Endpoint 0                                    | <a href="#">Section 17.5.31</a> |
| 0x10B  | USBNAKLMT <sup>(2)</sup>        | R/W  | 0x00   | USB NAK Limit                                          | <a href="#">Section 17.5.32</a> |
| 0x110  | USBTXMAXP1 <sup>(1) (2)</sup>   | R/W  | 0x0000 | USB Maximum Transmit Data Endpoint 1                   | <a href="#">Section 17.5.27</a> |
| 0x112  | USBTXCSRL1 <sup>(1) (2)</sup>   | R/W  | 0x00   | USB Transmit Control and Status Endpoint 1 Low         | <a href="#">Section 17.5.33</a> |
| 0x113  | USBTXCSRH1 <sup>(1) (2)</sup>   | R/W  | 0x00   | USB Transmit Control and Status Endpoint 1 High        | <a href="#">Section 17.5.34</a> |
| 0x114  | USBRXMAXP1 <sup>(1) (2)</sup>   | R/W  | 0x0000 | USB Maximum Receive Data Endpoint 1                    | <a href="#">Section 17.5.35</a> |
| 0x116  | USBRXCSRL1 <sup>(1) (2)</sup>   | R/W  | 0x00   | USB Receive Control and Status Endpoint 1 Low          | <a href="#">Section 17.5.36</a> |
| 0x117  | USBRXCSRH1 <sup>(1) (2)</sup>   | R/W  | 0x00   | USB Receive Control and Status Endpoint 1 High         | <a href="#">Section 17.5.37</a> |
| 0x118  | USBRXCOUNT1 <sup>(1) (2)</sup>  | RO   | 0x0000 | USB Receive Byte Count Endpoint 1                      | <a href="#">Section 17.5.38</a> |
| 0x11A  | USBTXTYPE1 <sup>(2)</sup>       | R/W  | 0x00   | USB Host Transmit Configure Type Endpoint 1            | <a href="#">Section 17.5.39</a> |
| 0x11B  | USBTXINTERVAL1 <sup>(2)</sup>   | R/W  | 0x00   | USB Host Transmit Interval Endpoint 1                  | <a href="#">Section 17.5.40</a> |
| 0x11C  | USBRXTYPE1 <sup>(2)</sup>       | R/W  | 0x00   | USB Host Configure Receive Type Endpoint 1             | <a href="#">Section 17.5.41</a> |
| 0x11D  | USBRXINTERVAL1 <sup>(2)</sup>   | R/W  | 0x00   | USB Host Receive Polling Interval Endpoint 1           | <a href="#">Section 17.5.42</a> |
| 0x120  | USBTXMAXP2 <sup>(1) (2)</sup>   | R/W  | 0x0000 | USB Maximum Transmit Data Endpoint 2                   | <a href="#">Section 17.5.27</a> |



**Table 17-3. Universal Serial Bus (USB) Controller Register Map (continued)**

| Offset | Name                                | Type | Reset       | Description                                           | Section                         |
|--------|-------------------------------------|------|-------------|-------------------------------------------------------|---------------------------------|
| 0x122  | USBTXCSRL2 <sup>(1) (2)</sup>       | R/W  | 0x00        | USB Transmit Control and Status Endpoint 2 Low        | <a href="#">Section 17.5.33</a> |
| 0x123  | USBTXCSRH2 <sup>(1) (2)</sup>       | R/W  | 0x00        | USB Transmit Control and Status Endpoint 2 High       | <a href="#">Section 17.5.34</a> |
| 0x124  | USBRXMAXP2 <sup>(1) (2)</sup>       | R/W  | 0x0000      | USB Maximum Receive Data Endpoint 2                   | <a href="#">Section 17.5.35</a> |
| 0x126  | USBRXCSRL2 <sup>(1) (2)</sup>       | R/W  | 0x00        | USB Receive Control and Status Endpoint 2 Low         | <a href="#">Section 17.5.36</a> |
| 0x127  | USBRXCSRH2 <sup>(1) (2)</sup>       | R/W  | 0x00        | USB Receive Control and Status Endpoint 2 High        | <a href="#">Section 17.5.37</a> |
| 0x128  | USBRXCOUNT2 <sup>(1) (2)</sup>      | RO   | 0x0000      | USB Receive Byte Count Endpoint 2                     | <a href="#">Section 17.5.38</a> |
| 0x12A  | USBTXTYPE2 <sup>(2)</sup>           | R/W  | 0x00        | USB Host Transmit Configure Type Endpoint 2           | <a href="#">Section 17.5.39</a> |
| 0x12B  | USBTXINTERVAL2 <sup>(2)</sup>       | R/W  | 0x00        | USB Host Transmit Interval Endpoint 2                 | <a href="#">Section 17.5.40</a> |
| 0x12C  | USBRXTYPE2 <sup>(2)</sup>           | R/W  | 0x00        | USB Host Configure Receive Type Endpoint 2            | <a href="#">Section 17.5.41</a> |
| 0x12D  | USBRXINTERVAL2 <sup>(2)</sup>       | R/W  | 0x00        | USB Host Receive Polling Interval Endpoint 2          | <a href="#">Section 17.5.42</a> |
| 0x130  | USBTXMAXP3 <sup>(1) (2)</sup>       | R/W  | 0x0000      | USB Maximum Transmit Data Endpoint 3                  | <a href="#">Section 17.5.27</a> |
| 0x132  | USBTXCSRL3 <sup>(1) (2)</sup>       | R/W  | 0x00        | USB Transmit Control and Status Endpoint 3 Low        | <a href="#">Section 17.5.33</a> |
| 0x133  | USBTXCSRH3 <sup>(1) (2)</sup>       | R/W  | 0x00        | USB Transmit Control and Status Endpoint 3 High       | <a href="#">Section 17.5.34</a> |
| 0x134  | USBRXMAXP3 <sup>(1) (2)</sup>       | R/W  | 0x0000      | USB Maximum Receive Data Endpoint 3                   | <a href="#">Section 17.5.35</a> |
| 0x136  | USBRXCSRL3 <sup>(1) (2)</sup>       | R/W  | 0x00        | USB Receive Control and Status Endpoint 3 Low         | <a href="#">Section 17.5.33</a> |
| 0x137  | USBRXCSRH3 <sup>(1) (2)</sup>       | R/W  | 0x00        | USB Receive Control and Status Endpoint 3 High        | <a href="#">Section 17.5.36</a> |
| 0x138  | USBRXCOUNT3 <sup>(1) (2)</sup>      | RO   | 0x0000      | USB Receive Byte Count Endpoint 3                     | <a href="#">Section 17.5.38</a> |
| 0x13A  | USBTXTYPE3 <sup>(2)</sup>           | R/W  | 0x00        | USB Host Transmit Configure Type Endpoint 3           | <a href="#">Section 17.5.39</a> |
| 0x13B  | USBTXINTERVAL3 <sup>(2)</sup>       | R/W  | 0x00        | USB Host Transmit Interval Endpoint 3                 | <a href="#">Section 17.5.40</a> |
| 0x13C  | USBRXTYPE3 <sup>(2)</sup>           | R/W  | 0x00        | USB Host Configure Receive Type Endpoint 3            | <a href="#">Section 17.5.41</a> |
| 0x13D  | USBRXINTERVAL3 <sup>(2)</sup>       | R/W  | 0x00        | USB Host Receive Polling Interval Endpoint 3          | <a href="#">Section 17.5.42</a> |
| 0x304  | USBRQPKTCOUNT1 <sup>(2)</sup>       | R/W  | 0x0000 1    | USB Request Packet Count in Block Transfer Endpoint 1 | <a href="#">Section 17.5.43</a> |
| 0x308  | USBRQPKTCOUNT2 <sup>(2)</sup>       | R/W  | 0x0000      | USB Request Packet Count in Block Transfer Endpoint 2 | <a href="#">Section 17.5.43</a> |
| 0x30C  | USBRQPKTCOUNT3 <sup>(2)</sup>       | R/W  | 0x0000      | USB Request Packet Count in Block Transfer Endpoint 3 | <a href="#">Section 17.5.43</a> |
| 0x340  | USBRXDPKTBUFFDIS <sup>(1) (2)</sup> | R/W  | 0x0000      | USB Receive Double Packet Buffer Disable              | <a href="#">Section 17.5.44</a> |
| 0x342  | USBTXDPKTBUFFDIS <sup>(1) (2)</sup> | R/W  | 0x0000      | USB Transmit Double Packet Buffer Disable             | <a href="#">Section 17.5.45</a> |
| 0x400  | USBEPIC <sup>(1) (2)</sup>          | R/W  | 0x0000.0000 | USB External Power Control                            | <a href="#">Section 17.5.46</a> |
| 0x404  | USBEPICRIS <sup>(1) (2)</sup>       | RO   | 0x0000.0000 | USB External Power Control Raw Interrupt Status       | <a href="#">Section 17.5.47</a> |
| 0x408  | USBEPICIM <sup>(1) (2)</sup>        | R/W  | 0x0000.0000 | USB External Power Control Interrupt Mask             | <a href="#">Section 17.5.48</a> |
| 0x40C  | USBEPICISC <sup>(1) (2)</sup>       | R/W  | 0x0000.0000 | USB External Power Control Interrupt Status and Clear | <a href="#">Section 17.5.49</a> |
| 0x410  | USBDRRIS <sup>(1) (2)</sup>         | RO   | 0x0000.0000 | USB Device RESUME Raw Interrupt Status                | <a href="#">Section 17.5.50</a> |
| 0x414  | USBDRIM <sup>(1) (2)</sup>          | R/W  | 0x0000.0000 | USB Device RESUME Interrupt Mask                      | <a href="#">Section 17.5.51</a> |
| 0x418  | USBDRISC <sup>(1) (2)</sup>         | W1C  | 0x0000.0000 | USB Device RESUME Interrupt Status and Clear          | <a href="#">Section 17.5.52</a> |
| 0x41C  | USBGPCS <sup>(1) (2)</sup>          | R/W  | 0x0000.0000 | USB General-Purpose Control and Status                | <a href="#">Section 17.5.53</a> |
| 0x450  | USBDMASEL <sup>(1) (2)</sup>        | R/W  | 0x0033.2211 | USB DMA Select                                        | <a href="#">Section 17.5.54</a> |

(1) This register is used in Device mode. Some registers are used for both Host and Device mode and may have different bit definitions depending on the mode.

(2) This register is used in Host mode. Some registers are used for both Host and Device mode and may have different bit definitions depending on the mode. The USB controller is in Device mode upon reset, so the reset values shown for these registers apply to the Device mode definition.

### 17.5.1 USB Device Functional Address Register (USBFADDR), offset 0x000

The USB function address 8-bit register (USBFADDR) contains the 7-bit address of the device part of the transaction.

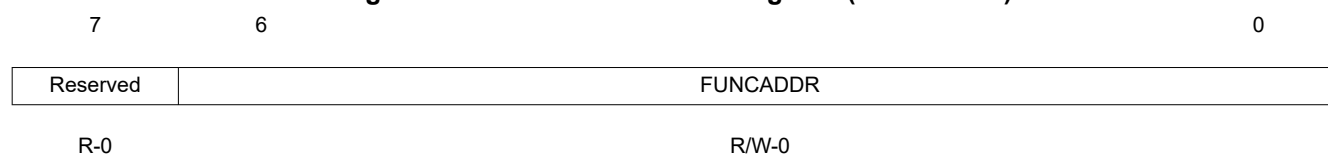
When the USB controller is being used in device mode (the HOST bit in the USBDEVCTL register is clear), this register must be written with the address received through a SET\_ADDRESS command, which is then used for decoding the function address in subsequent token packets.

**Mode(s):** Device

For special considerations when writing this register, see the *Setting the Device Address* in [Section 17.2.1.1.4](#).

USBFADDR is shown in [Figure 17-3](#) and described in [Table 17-4](#).

**Figure 17-3. Function Address Register (USBFADDR)**



LEGEND: R/W = Read/Write; -n = value after reset

**Table 17-4. Function Address Register (USBFADDR) Field Descriptions**

| Bit | Field    | Value | Description                                                 |
|-----|----------|-------|-------------------------------------------------------------|
| 7   | Reserved | 0     | Reserved                                                    |
| 6-0 | FUNCADDR | 0-7Fh | Function Address of Device as received through SET_ADDRESS. |

## 17.5.2 USB Power Management Register (USBPOWER), offset 0x001

The power management 8-bit register (USBPOWER) is used for controlling SUSPEND and RESUME signaling, and some basic operational aspects of the USB controller.

**Mode(s):** Host Device

USBPOWER in Host Mode is shown in [Figure 17-4](#) and described in [Table 17-5](#).

**Figure 17-4. Power Management Register (USBPOWER) in Host Mode**

|          |   |       |        |         |          |
|----------|---|-------|--------|---------|----------|
| 7        | 4 | 3     | 2      | 1       | 0        |
| Reserved |   | RESET | RESUME | SUSPEND | PWRDNPHY |
| R-0      |   | R/W-0 | R/W-0  | R/W-1S  | R/W-0    |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 17-5. Power Management Register (USBPOWER) in Host Mode Field Descriptions**

| Bit | Field    | Value | Description                                                                    |
|-----|----------|-------|--------------------------------------------------------------------------------|
| 7-4 | Reserved | 0     | Reserved                                                                       |
| 3   | RESET    | 0     | RESET signaling.                                                               |
|     |          | 0     | Ends RESET signaling on the bus.                                               |
|     |          | 1     | Enables RESET signaling on the bus.                                            |
| 2   | RESUME   |       | RESUME signaling. The bit should be cleared by software 20 ms after being set. |
|     |          | 0     | Ends RESUME signaling on the bus.                                              |
|     |          | 1     | Enables RESUME signaling when the Device is in SUSPEND mode.                   |
| 1   | SUSPEND  |       | SUSPEND mode                                                                   |
|     |          | 0     | No effect                                                                      |
|     |          | 1     | Enables SUSPEND mode.                                                          |
| 0   | PWRDNPHY |       | Power Down PHY                                                                 |
|     |          | 0     | No effect                                                                      |
|     |          | 1     | Powers down the internal USB PHY.                                              |

USBPOWER in Device Mode is shown in [Figure 17-5](#) and described in [Table 17-6](#).

**Figure 17-5. Power Management Register (USBPOWER) in Device Mode**

|           |          |          |       |        |         |          |   |
|-----------|----------|----------|-------|--------|---------|----------|---|
| 7         | 6        | 5        | 4     | 3      | 2       | 1        | 0 |
| ISOUPDATE | SOFTCONN | Reserved | RESET | RESUME | SUSPEND | PWRDNPHY |   |
| R/W-0     | R/W-0    | R-0      | R/W-0 | R/W-0  | R-0     | R/W-0    |   |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 17-6. Power Management Register (USBPOWER) in Device Mode Field Descriptions**

| Bit | Field    | Value  | Description                                                                                                                                                                                              |
|-----|----------|--------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 7   | Reserved |        | Reserved                                                                                                                                                                                                 |
| 6   | SOFTCONN | 0<br>1 | Soft Connect/Disconnect<br>The USB D+/D- lines are tri-stated.<br>The USB D+/D- lines are enabled.                                                                                                       |
| 5-4 | Reserved | 0      | Reserved                                                                                                                                                                                                 |
| 3   | RESET    | 0<br>1 | RESET signaling<br>Ends RESET signaling on the bus.<br>Enables RESET signaling on the bus.                                                                                                               |
| 2   | RESUME   | 0<br>1 | RESUME signaling. The bit should be cleared by software 10 ms (a maximum of 15 ms) after being set.<br>Ends RESUME signaling on the bus.<br>Enables RESUME signaling when the Device is in SUSPEND mode. |
| 1   | SUSPEND  | 0<br>1 | SUSPEND mode.<br>This bit is cleared when software reads the interrupt register or sets the RESUME bit above.<br>The USB controller is in SUSPEND mode.                                                  |
| 0   | PWRDNPHY | 0<br>1 | Power Down PHY<br>No effect<br>Powers down the internal USB PHY.                                                                                                                                         |

### 17.5.3 USB Transmit Interrupt Status Register (USBTXIS), offset 0x002

#### Note

Use caution when reading this register. Performing a read may change bit status.

The USB transmit interrupt status 16-bit read-only register (USBTXIS) indicates which interrupts are currently active for control endpoint 0 and the transmit endpoints 1–3. The meaning of the EPn bits in this register is based on the mode of the device. The EP1 through EP3 bits always indicate that the USB controller is sending data; however, in Host mode, the bits refer to OUT endpoints; while in Device mode, the bits refer to IN endpoints.

**Note:** The EP0 bit is special in Host and Device modes and indicates that either a control IN or control OUT endpoint has generated an interrupt. Both the control IN and control OUT endpoints are captured in the EP0 bit of the USBTXIS register.

**Mode(s):**      Host                                  Device

USBTXIS is shown in [Figure 17-6](#) and described in [Table 17-7](#).

**Figure 17-6. USB Transmit Interrupt Status Register (USBTXIS)**

|          |  |  |  |  |  |   |     |     |     |     |
|----------|--|--|--|--|--|---|-----|-----|-----|-----|
| 15       |  |  |  |  |  | 4 | 3   | 2   | 1   | 0   |
| Reserved |  |  |  |  |  |   | EP3 | EP2 | EP1 | EP0 |
| R-0      |  |  |  |  |  |   | R-0 | R-0 | R-0 | R-0 |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset sho

**Table 17-7. USB Transmit Interrupt Status Register (USBTXIS) Field Descriptions**

| Bit  | Field    | Value  | Description                                                                                                 |
|------|----------|--------|-------------------------------------------------------------------------------------------------------------|
| 15-4 | Reserved |        | Reserved                                                                                                    |
| 3    | EP3      | 0<br>1 | TX Endpoint 3 Interrupt<br>No interrupt<br>The Endpoint 3 transmit interrupt is asserted.                   |
| 2    | EP2      | 0<br>1 | TX Endpoint 2 Interrupt<br>No interrupt<br>The Endpoint 2 transmit interrupt is asserted.                   |
| 1    | EP1      | 0<br>1 | TX Endpoint 1 Interrupt<br>No interrupt<br>The Endpoint 1 transmit interrupt is asserted.                   |
| 0    | EP0      | 0<br>1 | TX and RX Endpoint 0 Interrupt<br>No interrupt<br>The Endpoint 0 transmit or receive interrupt is asserted. |



### 17.5.5 USB Transmit Interrupt Enable Register (USBTXIE), offset 0x006

The USB transmit interrupt enable 16-bit register (USBTXIE) provides interrupt enable bits for the interrupts in the USBTXIS register. When a bit is set, the USB interrupt is asserted to the interrupt controller when the corresponding interrupt bit in the USBTXIS register is set. When a bit is cleared, the interrupt in the USBTXIS register is still set but the USB interrupt to the interrupt controller is not asserted. On reset, all interrupts are enabled.

**Note:**The EP0 bit is special in Host and Device modes. Both the control IN and control OUT endpoints are captured in the EP0 bit of the USBTXIE register.

| Mode(s): | Host | Device |
|----------|------|--------|
|----------|------|--------|

USBTXIS is shown in [Figure 17-8](#) and described in [Table 17-9](#).

**Figure 17-8. USB Transmit Interrupt Status Enable Register (USBTXIE)**

|          |  |  |  |  |  |  |  |  |  |  |   |       |       |       |       |
|----------|--|--|--|--|--|--|--|--|--|--|---|-------|-------|-------|-------|
| 15       |  |  |  |  |  |  |  |  |  |  | 4 | 3     | 2     | 1     | 0     |
| Reserved |  |  |  |  |  |  |  |  |  |  |   | EP3   | EP2   | EP1   | EP0   |
| R-0      |  |  |  |  |  |  |  |  |  |  |   | R/W-1 | R/W-1 | R/W-1 | R/W-1 |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

### Table 17-9. USB Transmit Interrupt Status Register (USBTXIE) Field Descriptions

| Bit  | Field    | Value | Description                                                                                                                             |
|------|----------|-------|-----------------------------------------------------------------------------------------------------------------------------------------|
| 15-4 | Reserved |       | Reserved                                                                                                                                |
| 2    | EP2      | 0     | TX Endpoint 2 Interrupt Enable<br>The EP2 transmit interrupt is suppressed and not sent to the interrupt controller.                    |
|      |          | 1     | An interrupt is sent to the interrupt controller when the EP2 bit in the USBTXIS register is set.                                       |
| 1    | EP1      | 0     | TX Endpoint 1 Interrupt Enable<br>The EP1 transmit interrupt is suppressed and not sent to the interrupt controller.                    |
|      |          | 1     | An interrupt is sent to the interrupt controller when the EP1 bit in the USBTXIS register is set.                                       |
| 0    | EP0      | 0     | TX and RX Endpoint 0 Interrupt Enable<br>The EP0 transmit and receive interrupt is suppressed and not sent to the interrupt controller. |
|      |          | 1     | An interrupt is sent to the interrupt controller when the EP0 bit in the USBTXIS register is set.                                       |





### 17.5.7 USB General Interrupt Status Register (USBIS), offset 0x00A

#### Note

Use caution when reading this register. Performing a read may change bit status.

The USB general interrupt status 8-bit read-only register (USBIS) indicates which USB interrupts are currently active. All active interrupts are cleared when this register is read.

**Mode(s):**      Host                                  Device

USBIS in Host Mode is shown in [Figure 17-10](#) and described in [Table 17-11](#).

**Figure 17-10. USB General Interrupt Status Register (USBIS) in Host Mode**

|         |        |        |      |     |        |        |          |
|---------|--------|--------|------|-----|--------|--------|----------|
| 7       | 6      | 5      | 4    | 3   | 2      | 1      | 0        |
| VBUSERR | SESREQ | DISCON | CONN | SOF | BABBLE | RESUME | Reserved |
| R-0     | R-0    | R-0    | R-0  | R-0 | R-0    | R-0    | R-0      |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 17-11. USB General Interrupt Status Register (USBIS) in Host Mode Field Descriptions**

| Bit | Field    | Value  | Description                                                                                                                                                                                                                                                                                                                      |
|-----|----------|--------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 7   | VBUSERR  | 0<br>1 | VBUS Error<br>No interrupt<br>VBUS has dropped below the VBUS Valid threshold during a session.                                                                                                                                                                                                                                  |
| 6   | SESREQ   | 0<br>1 | Session Request<br>No interrupt<br>SESSION REQUEST signaling has been detected.                                                                                                                                                                                                                                                  |
| 5   | DISCON   | 0<br>1 | Session Disconnect<br>No interrupt<br>A Device disconnect has been detected.                                                                                                                                                                                                                                                     |
| 4   | CONN     | 0<br>1 | Session Connect<br>No interrupt<br>A Device connection has been detected.                                                                                                                                                                                                                                                        |
| 3   | SOF      | 0<br>1 | Start of Frame<br>No interrupt<br>A new frame has started.                                                                                                                                                                                                                                                                       |
| 2   | BABBLE   | 0<br>1 | Babble Detected<br>No interrupt<br>Babble has been detected. This interrupt is active only after the first SOF has been sent.                                                                                                                                                                                                    |
| 1   | RESUME   | 0<br>1 | RESUME Signaling Detected. This interrupt can only be used if the USB controller's system clock is enabled. If the user disables the clock programming, the USBDRRIS, USBDRIM, and USBDRISC registers should be used.<br>No effect<br>RESUME signaling has been detected on the bus while the USB controller is in SUSPEND mode. |
| 0   | Reserved | 0      | Reserved                                                                                                                                                                                                                                                                                                                         |

USBIS in Device Mode is shown in [Figure 17-11](#) and described in [Table 17-12](#).

**Figure 17-11. USB General Interrupt Status Register (USBIS) in Device Mode**

|          |        |          |     |       |        |         |     |
|----------|--------|----------|-----|-------|--------|---------|-----|
| 7        | 6      | 5        | 4   | 3     | 2      | 1       | 0   |
| Reserved | DISCON | Reserved | SOF | RESET | RESUME | SUSPEND |     |
| R-0      | R-0    | R-0      | R-0 | R-0   | R-0    | R-0     | R-0 |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 17-12. USB General Interrupt Status Register (USBIS) in Device Mode Field Descriptions**

| Bit | Field    | Value | Description                                                                                                                                                                                                           |
|-----|----------|-------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 7-6 | Reserved | 0     | Reserved                                                                                                                                                                                                              |
| 5   | DISCON   | 0     | Session Disconnect                                                                                                                                                                                                    |
|     |          | 0     | No interrupt                                                                                                                                                                                                          |
|     |          | 1     | The device has been disconnected from the host.                                                                                                                                                                       |
| 4   | Reserved | 0     | Reserved                                                                                                                                                                                                              |
| 3   | SOF      | 0     | Start of frame                                                                                                                                                                                                        |
|     |          | 0     | No interrupt                                                                                                                                                                                                          |
|     |          | 1     | A new frame has started.                                                                                                                                                                                              |
| 2   | RESET    | 0     | RESET Signaling Detected                                                                                                                                                                                              |
|     |          | 0     | No interrupt                                                                                                                                                                                                          |
|     |          | 1     | RESET signaling has been detected on the bus.                                                                                                                                                                         |
| 1   | RESUME   | 0     | RESUME Signaling Detected. This interrupt can only be used if the USB controller's system clock is enabled. If the user disables the clock programming, the USBDRRIS, USBDRIM, and USBDRISC registers should be used. |
|     |          | 0     | No interrupt                                                                                                                                                                                                          |
|     |          | 1     | RESUME signaling has been detected on the bus while the USB controller is in SUSPEND mode.                                                                                                                            |
| 0   | SUSPEND  | 0     | SUSPEND Signaling Detected                                                                                                                                                                                            |
|     |          | 0     | No interrupt                                                                                                                                                                                                          |
|     |          | 1     | SUSPEND signaling has been detected on the bus.                                                                                                                                                                       |

## 17.5.8 USB Interrupt Enable Register (USBIE), offset 0x00B

### Note

Use caution when reading this register. Performing a read may change bit status.

The USB interrupt enable 8-bit register (USBIE) provides interrupt enable bits for each of the interrupts in USBIS. At reset interrupts 1 and 2 are enabled in device mode.

**Mode(s):**      Host                                  Device

USBIE in Host Mode is shown in [Figure 17-12](#) and described in [Table 17-13](#).

**Figure 17-12. USB Interrupt Enable Register (USBIE) in Host Mode**

|         |        |        |      |     |        |        |          |
|---------|--------|--------|------|-----|--------|--------|----------|
| 7       | 6      | 5      | 4    | 3   | 2      | 1      | 0        |
| VBUSERR | SESREQ | DISCON | CONN | SOF | BABBLE | RESUME | Reserved |
| R-W     | R-W    | R-W    | R-W  | R-W | R-W    | R-W    | R-0      |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 17-13. USB Interrupt Enable Register (USBIE) in Host Mode Field Descriptions**

| Bit | Field    | Value  | Description                                                                                                                                                                                                                                                                                                                                                                                                 |
|-----|----------|--------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 7   | VBUSERR  | 0<br>1 | Enable VBUS Error Interrupt<br>The VBUSERR interrupt is suppressed and not sent to the interrupt controller.<br>An interrupt is sent to the interrupt controller when the VBUSERR bit in the USBIS register is set.                                                                                                                                                                                         |
| 6   | SESREQ   | 0<br>1 | Enable Session Request<br>The SESREQ interrupt is suppressed and not sent to the interrupt controller.<br>An interrupt is sent to the interrupt controller when the SESREQ bit in the USBIS register is set.                                                                                                                                                                                                |
| 5   | DISCON   | 0<br>1 | Enable Disconnect Interrupt<br>The DISCON interrupt is suppressed and not sent to the interrupt controller.<br>An interrupt is sent to the interrupt controller when the DISCON bit in the USBIS register is set.                                                                                                                                                                                           |
| 4   | CONN     | 0<br>1 | Enable Connect Interrupt<br>The CONN interrupt is suppressed and not sent to the interrupt controller.<br>An interrupt is sent to the interrupt controller when the CONN bit in the USBIS register is set.                                                                                                                                                                                                  |
| 3   | SOF      | 0<br>1 | Start of Frame<br>The SOF interrupt is suppressed and not sent to the interrupt controller.<br>An interrupt is sent to the interrupt controller when the SOF bit in the USBIS register is set.                                                                                                                                                                                                              |
| 2   | BABBLE   | 0<br>1 | Babble Detected<br>The BABBLE interrupt is suppressed and not sent to the interrupt controller.<br>An interrupt is sent to the interrupt controller when the BABBLE bit in the USBIS register is set.                                                                                                                                                                                                       |
| 1   | RESUME   | 0<br>1 | RESUME Signaling Detected. This interrupt can only be used if the USB controller's system clock is enabled. If the user disables the clock programming, the USBDRRIS, USBDRIM, and USBDRISC registers should be used.<br>The RESUME interrupt is suppressed and not sent to the interrupt controller.<br>An interrupt is sent to the interrupt controller when the RESUME bit in the USBIS register is set. |
| 0   | Reserved | 0      | Reserved                                                                                                                                                                                                                                                                                                                                                                                                    |

USBIE in Device Mode is shown in [Figure 17-11](#) and described in [Table 17-12](#).

**Figure 17-13. USB Interrupt Enable Register (USBIE) in Device Mode**

|          |        |          |       |       |        |         |   |
|----------|--------|----------|-------|-------|--------|---------|---|
| 7        | 6      | 5        | 4     | 3     | 2      | 1       | 0 |
| Reserved | DISCON | Reserved | SOF   | RESET | RESUME | SUSPEND |   |
| R-0      | R/W-0  | R-0      | R/W-0 | R/W-1 | RW-1   | R/W-0   |   |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 17-14. USB Interrupt Enable Register (USBIE) in Device Mode Field Descriptions**

| Bit | Field    | Value | Description                                                                                                                                                                                                                                                                                           |
|-----|----------|-------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 7-6 | Reserved | 0     | Reserved                                                                                                                                                                                                                                                                                              |
| 5   | DISCON   | 0     | Enable Disconnect Interrupt<br>The DISCON interrupt is suppressed and not sent to the interrupt controller.                                                                                                                                                                                           |
|     |          | 1     | An interrupt is sent to the interrupt controller when the DISCON bit in the USBIS register is set.                                                                                                                                                                                                    |
| 4   | Reserved | 0     | Reserved                                                                                                                                                                                                                                                                                              |
| 3   | SOF      | 0     | Start of frame<br>The SOF interrupt is suppressed and not sent to the interrupt controller.                                                                                                                                                                                                           |
|     |          | 1     | An interrupt is sent to the interrupt controller when the SOF bit in the USBIS register is set.                                                                                                                                                                                                       |
| 2   | RESET    | 0     | RESET Signaling Detected<br>The RESET interrupt is suppressed and not sent to the interrupt controller.                                                                                                                                                                                               |
|     |          | 1     | An interrupt is sent to the interrupt controller when the RESET bit in the USBIS register is set.                                                                                                                                                                                                     |
| 1   | RESUME   | 0     | RESUME Signaling Detected. This interrupt can only be used if the USB controller's system clock is enabled. If the user disables the clock programming, the USBDRRIS, USBDRIM, and USBDRISC registers should be used.<br>The RESUME interrupt is suppressed and not sent to the interrupt controller. |
|     |          | 1     | An interrupt is sent to the interrupt controller when the RESUME bit in the USBIS register is set.                                                                                                                                                                                                    |
| 0   | SUSPEND  | 0     | SUSPEND Signaling Detected<br>The SUSPEND interrupt is suppressed and not sent to the interrupt controller.                                                                                                                                                                                           |
|     |          | 1     | An interrupt is sent to the interrupt controller when the DISCON bit in the USBIS register is set.                                                                                                                                                                                                    |

### 17.5.9 USB Frame Value Register (USBFRAME), offset 0x00C

The frame number 16-bit read-only register (USBFRAME) holds the last received frame number.

**Mode(s):** Host Device

USBFRAME is shown in [Figure 17-14](#) and described in [Table 17-15](#).

**Figure 17-14. Frame Number Register (FRAME)**

|          |    |       |   |
|----------|----|-------|---|
| 15       | 11 | 10    | 0 |
|          |    |       |   |
| Reserved |    | FRAME |   |
| R-0      |    | R-0   |   |

LEGEND: R = Read only; -n = value after reset

**Table 17-15. Frame Number Register (FRAME) Field Descriptions**

| Bit   | Field    | Value  | Description                |
|-------|----------|--------|----------------------------|
| 15-11 | Reserved | 0      | Reserved                   |
| 10-0  | FRAME    | 0-7FFh | Last received frame number |

### 17.5.10 USB Endpoint Index Register (USBEPIDX), offset 0x00E

Each endpoint buffer can be accessed by configuring a FIFO size and starting address. The endpoint index 16-bit register (USBEPIDX) is used with the USBTXFIFOSZ, USBRXFIFOSZ, USBTXFIFOADD, and USBRXFIFOADD registers.

**Mode(s):** Host Device

USBEPIDX is shown in [Figure 17-15](#) and described in [Table 17-16](#).

**Figure 17-15. USB Endpoint Index Register (USBEPIDX)**

|          |   |       |   |
|----------|---|-------|---|
| 7        | 4 | 3     | 0 |
|          |   |       |   |
| Reserved |   | EPIDX |   |
| R-0      |   | R/W-0 |   |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 17-16. USB Endpoint Index Register (USBEPIDX) Field Descriptions**

| Bit | Field    | Value | Description                                                                                                                                                                                                                            |
|-----|----------|-------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 7-4 | Reserved | 0     | Reserved                                                                                                                                                                                                                               |
| 3-0 | EPIDX    | 0-4h  | Endpoint Index. This bit field configures which endpoint is accessed when reading or writing to one of the USB controller's indexed registers. A value of 0x0 corresponds to Endpoint 0 and a value of 0xF corresponds to Endpoint 15. |



USBTEST in Device Mode is shown in [Figure 17-17](#) and described in [Table 17-18](#).

**Figure 17-17. USB Test Mode Register (USBTEST) in Device Mode**

|          |         |         |          |   |
|----------|---------|---------|----------|---|
| 7        | 6       | 5       | 4        | 0 |
| Reserved | FIFOACC | FORCEFS | Reserved |   |
| R-0      | R/W1S-0 | R/W-0   | R-0      |   |

LEGEND: R/W = Read/Write; W = Write only; -n = value after reset

**Table 17-18. USB Test Mode Register (USBTEST) in Device Mode Field Descriptions**

| Bit | Field    | Value  | Description                                                                                                                                                                        |
|-----|----------|--------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 7   | Reserved |        | Force Host Mode. While in this mode, status of the bus connection may be read using the DEV bit of the USBDEVCTL register. The operating speed is determined from the FORCEFS bit. |
| 6   | FIFOACC  | 0<br>1 | FIFO Access<br>No effect<br>Transfers the packet in the endpoint 0 transmit FIFO to the endpoint 0 receive FIFO.                                                                   |
| 5   | FORCEFS  | 0<br>1 | Force Full-Speed Mode<br>The USB controller operates at Low Speed.<br>Forces the USB controller into Full-Speed mode upon receiving a USB RESET.                                   |
| 4-0 | Reserved | 0      | Reserved                                                                                                                                                                           |

### 17.5.12 USB FIFO Endpoint n Register (USBFIFO[0]-USBFIFO[3])

### Note

Use caution when reading these registers. Performing a read may change bit status.

The USB FIFO endpoint  $n$  32-bit registers (USBFIFO[ $n$ ]) provide an address for CPU access to the FIFOs for each endpoint. Writing to these addresses loads data into the Transmit FIFO for the corresponding endpoint. Reading from these addresses unloads data from the Receive FIFO for the corresponding endpoint.

Transfers to and from FIFOs can be 8-bit, 16-bit or 32-bit as required, and any combination of accesses is allowed provided the data accessed is contiguous. All transfers associated with one packet must be of the same width so that the data is consistently byte-, halfword- or word-aligned. However, the last transfer may contain fewer bytes than the previous transfers in order to complete an odd-byte or odd-word transfer.

Depending on the size of the FIFO and the expected maximum packet size, the FIFOs support either single-packet or double-packet buffering (see *Single-Packet Buffering* in [Section 17.2.1.1.2](#)). Burst writing of multiple packets is not supported as flags must be set after each packet is written.

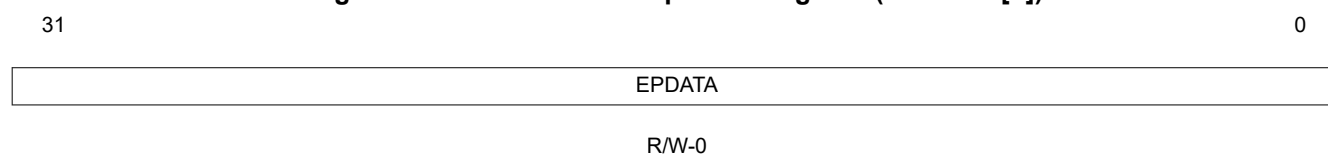
Following a STALL response or a transmit error on endpoint 1–3, the associated FIFO is completely flushed.

For the specific offset for each FIFO register, see [Table 17-3](#).

|                 |      |        |
|-----------------|------|--------|
| <b>Mode(s):</b> | Host | Device |
|-----------------|------|--------|

USBFIFO0-3 are shown in [Figure 17-18](#) and described in [Table 17-19](#).

**Figure 17-18. USB FIFO Endpoint *n* Register (USBFIFO[*n*])**



LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

### Table 17-19. USB FIFO Endpoint *n* Register (USBFIFO[*n*]) Field Descriptions

| Bit  | Field  | Reset       | Description                                                                                                                   |
|------|--------|-------------|-------------------------------------------------------------------------------------------------------------------------------|
| 31-0 | EPDATA | 0x0000.0000 | Endpoint Data. Writing to this register loads the data into the Transmit FIFO and reading unloads data from the Receive FIFO. |



### 17.5.13 USB Device Control Register (USBDEVCTL), offset 0x060

The USB device control 8-bit register (USBDEVCTL) is used for controlling and monitoring the USB VBUS line. If the PHY is suspended, no PHY clock is received and the VBUS is not sampled. In addition, in Host mode, USBDEVCTL provides the status information for the current operating mode (Host or Device) of the USB controller. If the USB controller is in Host mode, this register also indicates if a full- or low-speed Device has been connected.

|                 |      |        |
|-----------------|------|--------|
| <b>Mode(s):</b> | Host | Device |
|-----------------|------|--------|

USBDEVCTL is shown in [Figure 17-19](#) and described in [Table 17-20](#).

### Figure 17-19. USB Device Control Register (USBDEVCTL)

|     |       |       |      |   |          |         |         |
|-----|-------|-------|------|---|----------|---------|---------|
| 7   | 6     | 5     | 4    | 3 | 2        | 1       | 0       |
| DEV | FSDEV | LSDEV | VBUS |   | HOSTMODE | HOSTREQ | SESSION |
| R-1 | R-0   | R-0   | R-0  |   | R-0      | R/W-0   | R/W-0   |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

### Table 17-20. USB Device Control Register (USBDEVCTL) Field Descriptions

| Bit | Field    | Value                       | Description                                                                                                                                                                                                                                                                                                                                          |
|-----|----------|-----------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 7   | DEV      | 0<br>1                      | Device mode<br>The USB controller is operating on the OTG A side of the cable.<br>The USB controller is operating on the OTG B side of the cable.<br>Only valid while a session is in progress.                                                                                                                                                      |
| 6   | FSDEV    | 0<br>1                      | Full-Speed Device Detected<br>A full-speed Device has not been detected on the port.<br>A full-speed Device has been detected on the port.                                                                                                                                                                                                           |
| 5   | LSDEV    | 0<br>1                      | Low-Speed Device Detected<br>A low-speed Device has not been detected on the port.<br>A low-speed Device has been detected on the port.                                                                                                                                                                                                              |
| 4-3 | VBUS     | 0-3h<br>0<br>1h<br>2h<br>3h | These read-only bits encode the current VBus level as follows:<br>Below Session End. VBUS is detected as under 0.5 V.<br>Above Session End, below AValid. VBUS is detected as above 0.5 V and under 1.5 V.<br>Above AValid, below VBusValid. VBUS is detected as above 1.5 V and below 4.75 V.<br>Above VBusValid. VBUS is detected as above 4.75 V. |
| 2   | HOSTMODE | 0<br>1                      | This read-only bit is set when the USB controller is acting as a Host.<br>The USB controller is acting as a Device.<br>The USB controller is acting as a Host.<br>Only valid while a session is in progress.                                                                                                                                         |
| 1   | HOSTREQ  | 0<br>1                      | When set, the USB controller will initiate the Host Negotiation when Suspend mode is entered. It is cleared when Host Negotiation is completed.<br>No effect<br>Initiates the Host Negotiation when SUSPENDmode is entered.                                                                                                                          |

**Table 17-20. USB Device Control Register (USBDEVCTL) Field Descriptions (continued)**

| Bit | Field   | Value | Description                                                                                                                                               |
|-----|---------|-------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| 0   | SESSION |       | Session Start/End<br>When operating as a Host:                                                                                                            |
|     |         | 0     | When cleared by software, this bit ends a session.                                                                                                        |
|     |         | 1     | When set by software, this bit starts a session.                                                                                                          |
|     |         |       | When operating as a Device:                                                                                                                               |
|     |         | 0     | The USB controller has ended a session. When the USB controller is in SUSPEND mode, this bit may be cleared by software to perform a software disconnect. |
|     |         | 1     | The USB controller has started a session. When set by software, the Session Request Protocol is initiated.                                                |
|     |         |       | Clearing this bit when the USB controller is not suspended results in undefined behavior.                                                                 |



### 17.5.15 USB Receive Dynamic FIFO Sizing Register (USBXFIFOSZ), offset 0x063

The USB receive dynamic FIFO sizing 8-bit register (USBXFIFOSZ) allows the selected RX endpoint FIFOs to be dynamically sized.

**Mode(s):** Host Device

USBXFIFOSZ is shown in [Figure 17-21](#) and described in [Table 17-22](#).

**Figure 17-21. USB Receive Dynamic FIFO Sizing Register (USBXFIFOSZ)**

|          |   |       |     |   |
|----------|---|-------|-----|---|
| 7        | 5 | 4     | 3   | 0 |
| Reserved |   | DPB   | SZ  |   |
| R-0      |   | R/W-0 | R-0 |   |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 17-22. USB Receive Dynamic FIFO Sizing Register (USBXFIFOSZ) Field Descriptions**

| Bit | Field    | Value | Description                                                                                                                               |
|-----|----------|-------|-------------------------------------------------------------------------------------------------------------------------------------------|
| 7-5 | Reserved | 0     | Reserved                                                                                                                                  |
| 4   | DPB      | 0     | Double Packet Buffering Support<br>Single packet buffering is supported.                                                                  |
|     |          | 1     | Double packet buffering is enabled.                                                                                                       |
| 3-0 | SZ       |       | Maximum packet size to be allowed. If DPB = 0, the FIFO also is this size; if DPB = 1, the FIFO is twice this size. Packet size in bytes: |
|     |          | 0h    | 8                                                                                                                                         |
|     |          | 1h    | 16                                                                                                                                        |
|     |          | 2h    | 32                                                                                                                                        |
|     |          | 3h    | 64                                                                                                                                        |
|     |          | 4h    | 128                                                                                                                                       |
|     |          | 5h    | 256                                                                                                                                       |
|     |          | 6h    | 512                                                                                                                                       |
|     |          | 7h    | 1024                                                                                                                                      |
|     |          | 8h    | 2048                                                                                                                                      |
|     |          | 9-Fh  | Reserved                                                                                                                                  |

### 17.5.16 USB Transmit FIFO Start Address Register (USBTXFIFOADD), offset 0x064

The USB transmit FIFO start address 16-bit register (USBTXFIFOADD) controls the start address of the selected transmit endpoint FIFOs.

**Mode(s):** Host Device

USBTXFIFOADDR is shown in [Figure 17-22](#) and described in [Table 17-23](#).

**Figure 17-22. USB Transmit FIFO Start Address Register (USBTXFIFOADDR)**

|          |   |       |   |
|----------|---|-------|---|
| 15       | 9 | 8     | 0 |
| Reserved |   | ADDR  |   |
| R-0      |   | R/W-0 |   |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 17-23. USB Transmit FIFO Start Address Register (USBTXFIFOADDR) Field Descriptions**

| Bit  | Field    | Value                                                            | Description                                                                                                               |
|------|----------|------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------|
| 15-9 | Reserved | 0                                                                | Reserved                                                                                                                  |
| 8-0  | ADDR     | 0h<br>1h<br>2h<br>3h<br>4h<br>5h<br>6h<br>7h<br>8h<br>..<br>1FFh | Start Address of the endpoint FIFO in units of 8 bytes.<br>0<br>8<br>16<br>24<br>32<br>40<br>48<br>56<br>64<br>..<br>4095 |

### 17.5.17 USB Receive FIFO Start Address Register (USBXFIFOADD), offset 0x066

The USB receive FIFO start address 16-bit register (USBRXFIFOADD) controls the start address of the selected receive endpoint FIFOs.

|                 |      |        |
|-----------------|------|--------|
| <b>Mode(s):</b> | Host | Device |
|-----------------|------|--------|

USBRXFIFOADDR is shown in [Figure 17-23](#) and described in [Table 17-24](#).

**Figure 17-23. USB Receive FIFO Start Address Register (USBRXFIFOADDR)**

|          |   |       |   |
|----------|---|-------|---|
| 15       | 9 | 8     | 0 |
| Reserved |   | ADDR  |   |
| R-0      |   | R/W-0 |   |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 17-24. USB Receive FIFO Start Address Register (USBRXFIFOADDR) Field Descriptions**

| Bit  | Field    | Value | Description                                             |
|------|----------|-------|---------------------------------------------------------|
| 15-9 | Reserved | 0     | Reserved                                                |
| 8-0  | ADDR     |       | Start Address of the endpoint FIFO in units of 8 bytes. |
|      |          | 0h    | 0                                                       |
|      |          | 1h    | 8                                                       |
|      |          | 2h    | 16                                                      |
|      |          | 3h    | 24                                                      |
|      |          | 4h    | 32                                                      |
|      |          | 5h    | 40                                                      |
|      |          | 6h    | 48                                                      |
|      |          | 7h    | 56                                                      |
|      |          | 8h    | 64                                                      |
|      |          | ..    | ..                                                      |
|      |          | 1FFh  | 4095                                                    |

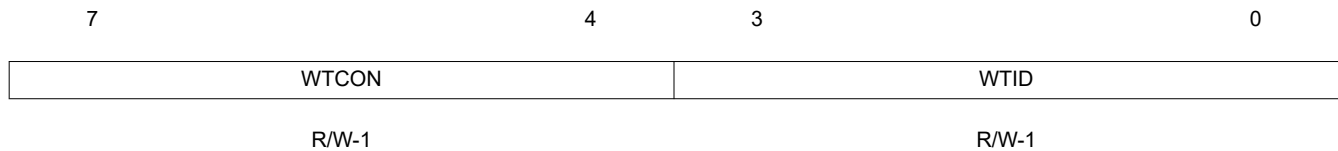
### 17.5.18 USB Connect Timing Register (USBCONTIM), offset 0x07A

The USB connect timing 8-bit configuration register (USBCONTIM) specifies connection and negotiation delays.

| Mode(s): | Host | Device |
|----------|------|--------|
|----------|------|--------|

USBCONTIM is shown in [Figure 17-24](#) and described in [Table 17-25](#).

### Figure 17-24. USB Connect Timing Register (USBCONTIM)



LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

### Table 17-25. USB Connect Timing Register (USBCONTIM) Field Descriptions

| Bit | Field | Value | Description                                                                                                                                                                   |
|-----|-------|-------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 7-4 | WTCON | 5h    | The connect wait field configures the wait required to allow for the user's connect/disconnect filter, in units of 533.3 ns. The default corresponds to 2.667 $\mu$ s.        |
| 3-0 | WTID  | Ch    | The wait ID field configures the delay required from the enable of the ID detection to when the ID value is valid, in units of 4.369 ms. The default corresponds to 52.43 ms. |

### 17.5.19 USB Full-Speed Last Transaction to End of Frame Timing Register (USBFSEOF), offset 0x07D

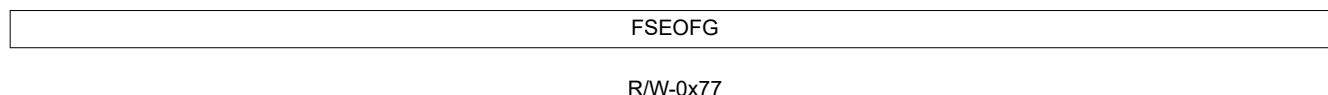
USB full-speed last transaction to end of frame timing 8-bit configuration register (USBFSEOF) specifies the minimum time gap allowed between the start of the last transaction and the EOF for full-speed transactions.

**Mode(s):** Host Device

USBFSEOF is shown in [Figure 17-25](#) and described in [Table 17-26](#).

**Figure 17-25. USB Full-Speed Last Transaction to End of Frame Timing Register (USBFSEOF)**

7 0



LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 17-26. USB Full-Speed Last Transaction to End of Frame Timing Register (USBFSEOF) Field Descriptions**

| Bit | Field  | Reset | Description                                                                                                                                                                                                                |
|-----|--------|-------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 7-0 | FSEOFG | 77h   | The full-speed end-of-frame gap field is used during full-speed transactions to configure the gap between the last transaction and the End-of-Frame (EOF), in units of 533.3 ns. The default corresponds to 63.46 $\mu$ s. |

### 17.5.20 USB Low-Speed Last Transaction to End of Frame Timing Register (USBLSEOF), offset 0x07E

The USB low-speed last transaction to end of frame timing 8-bit configuration register (USBLSEOF) specifies the minimum time gap that is to be allowed between the start of the last transaction and the EOF for low-speed transactions.

**Mode(s):** Host Device

USBLSEOF is shown in [Figure 17-26](#) and described in [Table 17-27](#).

**Figure 17-26. USB Low-Speed Last Transaction to End of Frame Timing Register (USBLSEOF)**

7 0



LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 17-27. USB Low-Speed Last Transaction to End of Frame Timing Register (USBLSEOF) Field Descriptions**

| Bit | Field  | Reset | Description                                                                                                                                                                                                             |
|-----|--------|-------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 7-0 | LSEOFG | 72h   | The low-speed end-of-frame gap field is used during low-speed transactions to set the gap between the last transaction and the End-of-Frame (EOF), in units of 1.067 $\mu$ s. The default corresponds to 121.6 $\mu$ s. |



### 17.5.21 USB Transmit Functional Address Endpoint $n$ Registers (USBTXFUNCADDR[0]-USBTXFUNCADDR[3])

The transmit functional address endpoint  $n$  8-bit registers (USBTXFUNCADDR[ $n$ ]) record the address of the target function to be accessed through the associated endpoint (EP $n$ ). USBTXFUNCADDR $x$  must be defined for each transmit endpoint that is used.

**Note:** USBTXFUNCADDR0 is used for both receive and transmit for endpoint 0.

For the specific offset for each register, see [Table 17-3](#).

**Mode(s):** Host

The USBTXFUNCADDR[ $n$ ] registers are shown in [Figure 17-27](#) and described in [Table 17-28](#).

**Figure 17-27. USB Transmit Functional Address Endpoint  $n$  Registers (USBTXFUNCADDR[ $n$ ])**

7 6 0

|          |       |
|----------|-------|
| Reserved | ADDR  |
| R-0      | R/W-0 |

LEGEND: R/W = Read/Write; - $n$  = value after reset

**Table 17-28. USB Transmit Functional Address Endpoint  $n$  Registers (USBTXFUNCADDR[ $n$ ]) Field Descriptions**

| Bit | Field    | Value | Description                                                         |
|-----|----------|-------|---------------------------------------------------------------------|
| 7   | Reserved | 0     | Reserved                                                            |
| 6-0 | ADDR     | 0     | Device Address specifies the USB bus address for the target Device. |

## 17.5.22 USB Transmit Hub Address Endpoint $n$ Registers (USBTXHUBADDR[0]-USBTXHUBADDR[3])

The transmit hub address endpoint  $n$  8-bit read/write registers (USBTXHUBADDR[ $n$ ]), like USBTXHUBPORT[ $n$ ], must be written only when a USB device is connected to transmit endpoint EP $n$  via a USB 2.0 hub. This register records the address of the USB 2.0 hub through which the target associated with the endpoint is accessed.

**Note:** USBTXHUBADDR0 is used for both receive and transmit for endpoint 0.

For the specific offset for each register, see [Table 17-3](#).

**Mode(s):** Host

The USBTXHUBADDR[ $n$ ] registers are shown in [Figure 17-27](#) and described in [Table 17-28](#).

**Figure 17-28. USB Transmit Hub Address Endpoint  $n$  Registers (USBTXHUBADDR[ $n$ ])**

|          |       |   |
|----------|-------|---|
| 7        | 6     | 0 |
| Reserved | ADDR  |   |
| R-0      | R/W-0 |   |

LEGEND: R/W = Read/Write; - $n$  = value after reset

**Table 17-29. USB Transmit Hub Address Endpoint  $n$  Registers(USBTXHUBADDR[ $n$ ]) Field Descriptions**

| Bit | Field    | Value | Description                                                         |
|-----|----------|-------|---------------------------------------------------------------------|
| 7   | Reserved | 0     | Reserved                                                            |
| 6-0 | ADDR     | 0     | Device Address specifies the USB bus address for the target Device. |

### 17.5.23 USB Transmit Hub Port Endpoint $n$ Registers (USBTXHUBPORT[0]-USBTXHUBPORT[3])

The transmit hub port endpoint  $n$  8-bit read/write registers (USBTXHUBPORT[ $n$ ]), like USBTXHUBADDR[ $n$ ], must be written only when a full- or low-speed Device is connected to transmit endpoint EP $n$  with a USB 2.0 hub. This register records the port of the USB 2.0 hub through which the target associated with the endpoint is accessed.

**Note:** USBTXHUBPORT0 is used for both receive and transmit for endpoint 0.

For the specific offset for each register, see [Table 17-3](#).

**Mode(s):** Host

The USBTXHUBPORT $n$  registers are shown in [Figure 17-29](#) and described in [Table 17-30](#).

**Figure 17-29. USB Transmit Hub Port Endpoint  $n$  Registers (USBTXHUBPORT[ $n$ ])**

|          |       |   |
|----------|-------|---|
| 7        | 6     | 0 |
| Reserved | PORT  |   |
| R-0      | R/W-0 |   |

LEGEND: R/W = Read/Write; - $n$  = value after reset

**Table 17-30. USB Transmit Hub Port Endpoint  $n$  Registers(USBTXHUBPORT[ $n$ ]) Field Descriptions**

| Bit | Field    | Value | Description                                 |
|-----|----------|-------|---------------------------------------------|
| 7   | Reserved | 0     | Reserved                                    |
| 6-0 | PORT     | 0     | Hub Port specifies the USB hub port number. |

### 17.5.24 USB Receive Functional Address Endpoint $n$ Registers (USBXFUNCADDR[1]-USBXFUNCADDR[3])

The receive functional address endpoint  $n$  8-bit read/write registers (USBXFUNCADDR[ $n$ ]) record the address of the target function to be accessed through the associated endpoint (EP $n$ ). USBXFUNCADDR $x$  must be defined for each receive endpoint that is used.

**Note:** USBTXFUNCADDR0 is used for both receive and transmit for endpoint 0.

For the specific offset for each register, see [Table 17-3](#).

**Mode(s):** Host

The USBXFUNCADDR[ $n$ ] registers are shown in [Figure 17-30](#) and described in [Table 17-31](#).

**Figure 17-30. USB Receive Functional Address Endpoint  $n$  Registers (USBFIFO[ $n$ ])**

|          |       |   |
|----------|-------|---|
| 7        | 6     | 0 |
| Reserved | ADDR  |   |
| R-0      | R/W-0 |   |

LEGEND: R/W = Read/Write; - $n$  = value after reset

**Table 17-31. USB Receive Functional Address Endpoint  $n$  Registers(USBFIFO[ $n$ ]) Field Descriptions**

| Bit | Field    | Value | Description                                                         |
|-----|----------|-------|---------------------------------------------------------------------|
| 7   | Reserved | 0     | Reserved                                                            |
| 6-0 | ADDR     | 0     | Device Address specifies the USB bus address for the target Device. |

### 17.5.25 USB Receive Hub Address Endpoint $n$ Registers (USBXHUBADDR[1]-USBXHUBADDR[3])

The receive hub address endpoint  $n$  8-bit read/write registers (USBXHUBADDR[ $n$ ]), like [ $n$ ], must be written only when a full- or low-speed Device is connected to receive endpoint EP $n$  with a USB 2.0 hub. Each register records the address of the USB 2.0 hub through which the target associated with the endpoint is accessed.

**Note:** USBTXHUBADDR0 is used for both receive and transmit for endpoint 0.

For the specific offset for each register, see [Table 17-3](#).

**Mode(s):** Host

The USBXHUBADDR[ $n$ ] registers are shown in [Figure 17-31](#) and described in [Table 17-32](#).

**Figure 17-31. USB Receive Hub Address Endpoint  $n$  Registers (USBXHUBADDR[ $n$ ])**

7 6 0

|          |      |
|----------|------|
| MULTTRAN | ADDR |
|----------|------|

R/w-0

R/W-0

LEGEND: R/W = Read/Write; - $n$  = value after reset

**Table 17-32. USB Receive Hub Address Endpoint  $n$  Registers(USBXHUBADDR[ $n$ ]) Field Descriptions**

| Bit | Field    | Value | Description                                                                                 |
|-----|----------|-------|---------------------------------------------------------------------------------------------|
| 7   | MULTTRAN | 0     | Multiple Translators<br>Clear to indicate that the hub has a single transaction translator. |
|     |          | 1     | Set to indicate that the hub has multiple transaction translators.                          |
| 6-0 | ADDR     | 0     | Device Address specifies the USB bus address for the target Device.                         |

### 17.5.26 USB Receive Hub Port Endpoint $n$ Registers (USBXHUBPORT[1]-USBXHUBPORT[3])

The receive hub port endpoint  $n$  8-bit read/write registers (USBXHUBPORT[ $n$ ]), like USBXHUBADDR[ $n$ ], must be written only when a full- or low-speed device is connected to receive endpoint EP $n$  with a USB 2.0 hub. Each register records the port of the USB 2.0 hub through which the target associated with the endpoint is accessed.

**Note:** USBTXHUBPORT0 is used for both receive and transmit for endpoint 0.

For the specific offset for each register, see [Table 17-3](#).

**Mode(s):** Host

The USBXHUBPORT $n$  registers are shown in [Figure 17-32](#) and described in [Table 17-33](#).

**Figure 17-32. USB Transmit Hub Port Endpoint  $n$  Registers (USBXHUBPORT[ $n$ ])**

|          |       |   |
|----------|-------|---|
| 7        | 6     | 0 |
| Reserved | PORT  |   |
| R-0      | R/W-0 |   |

LEGEND: R/W = Read/Write; - $n$  = value after reset

**Table 17-33. USB Transmit Hub Port Endpoint  $n$  Registers(USBXHUBPORT[ $n$ ]) Field Descriptions**

| Bit | Field    | Value | Description                                 |
|-----|----------|-------|---------------------------------------------|
| 7   | Reserved | 0     | Reserved                                    |
| 6-0 | PORT     | 0     | Hub Port specifies the USB hub port number. |



### 17.5.28 USB Control and Status Endpoint 0 Low Register (USBCSRL0), offset 0x102

The USB control and status endpoint 0 low 8-bit register (USBCSRL0) provides control and status bits for endpoint 0.

**Mode(s):** Host Device

USBCSRL0 in Host mode is shown in [Figure 17-34](#) and described in [Table 17-35](#).

**Figure 17-34. USB Control and Status Endpoint 0 Low Register (USBCSRL0) in Host Mode**

|       |        |        |       |       |         |       |       |
|-------|--------|--------|-------|-------|---------|-------|-------|
| 7     | 6      | 5      | 4     | 3     | 2       | 1     | 0     |
| NAKTO | STATUS | REQPKT | ERROR | SETUP | STALLED | TXRDY | RXRDY |
| R/W-0 | R/W-0  | R/W-0  | R/W-0 | R/W-0 | R/W-0   | R/W-0 | R/W-0 |

LEGEND: R/W = Read/Write; -n = value after reset

**Table 17-35. USB Control and Status Endpoint 0 Low Register(USBCSRL0) in Host Mode Field Descriptions**

| Bit | Field   | Value  | Description                                                                                                                                                                                                                                                                                                                                                      |
|-----|---------|--------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 7   | NAKTO   | 0<br>1 | NAK Timeout. Software must clear this bit to allow the endpoint to continue.<br>No timeout<br>Indicates that endpoint 0 is halted following the receipt of NAK responses for longer than the time set by the USBNAKLMT register.                                                                                                                                 |
| 6   | STATUS  | 0<br>1 | Status Packet. Setting this bit ensures that the DT bit is set in the USBCSRH0 register so that a DATA1 packet is used for the STATUS stage transaction.<br>No transaction<br>Initiates a STATUS stage transaction. This bit must be set at the same time as the TXRDY or REQPKT bit is set.<br>This bit is automatically cleared when the STATUS stage is over. |
| 5   | REQPKT  | 0<br>1 | Request Packet. This bit is cleared when the RXRDY bit is set.<br>No request<br>Requests an IN transaction.                                                                                                                                                                                                                                                      |
| 4   | ERROR   | 0<br>1 | Error. Software must clear this bit.<br>No error<br>Three attempts have been made to perform a transaction with no response from the peripheral. The EP0 bit in the USBTXIS register is also set in this situation.                                                                                                                                              |
| 3   | SETUP   | 0<br>1 | Setup Packet. Setting this bit always clears the DT bit in the USBCSRH0 register to send a DATA0 packet.<br>Sends an OUT token.<br>Sends a SETUP token instead of an OUT token for the transaction. This bit should be set at the same time as the TXRDY bit is set.                                                                                             |
| 2   | STALLED | 0<br>1 | Endpoint Stalled. Software must clear this bit.<br>No handshake has been received.<br>A STALL handshake has been received.                                                                                                                                                                                                                                       |
| 1   | TXRDY   | 0<br>1 | Transmit Packet Ready. If both the TXRDY and SETUP bits are set, a setup packet is sent. If just TXRDY is set, an OUT packet is sent.<br>No transmit packet is ready.<br>Software sets this bit after loading a data packet into the TX FIFO. The EP0 bit in the USBTXIS register is also set in this situation.                                                 |



**Table 17-35. USB Control and Status Endpoint 0 Low Register(USBCSRL0) in Host Mode Field Descriptions (continued)**

| Bit | Field | Value | Description                                                                                                                                               |
|-----|-------|-------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| 0   | RXRDY |       | Receive Packet Ready. Software must clear this bit after the packet has been read from the FIFO to acknowledge that the data has been read from the FIFO. |
|     |       | 0     | No receive packet has been received.                                                                                                                      |
|     |       | 1     | Indicates that a data packet has been received in the RX FIFO. The EP0 bit in the USBTXIS register is also set in this situation.                         |

USBCSRL0 in Device mode is shown in [Figure 17-35](#) and described in [Table 17-36](#).

**Figure 17-35. USB Control and Status Endpoint 0 Low Register (USBCSRL0) in Device Mode**

|         |        |       |        |         |         |       |       |
|---------|--------|-------|--------|---------|---------|-------|-------|
| 7       | 6      | 5     | 4      | 3       | 2       | 1     | 0     |
| SETENDC | RXRDYC | STALL | SETEND | DATAEND | STALLED | TXRDY | RXRDY |
| W1C-0   | W1C-0  | R/W-0 | R-0    | R/W-0   | R/W-0   | R/W-0 | R-0   |

LEGEND: R/W = Read/Write; -n = value after reset

**Table 17-36. USB Control and Status Endpoint 0 Low Register (USBCSRL0) in Device Mode Field Descriptions**

| Bit | Field   | Value | Description                                                                                                                                                                                                                                                                                            |
|-----|---------|-------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 7   | SETENDC |       | Setup End Clear                                                                                                                                                                                                                                                                                        |
|     |         | 0     | No effect                                                                                                                                                                                                                                                                                              |
|     |         | 1     | Writing a 1 to this bit clears the SETEND bit.                                                                                                                                                                                                                                                         |
| 6   | RXRDYC  |       | RXRDY Clear                                                                                                                                                                                                                                                                                            |
|     |         | 0     | No effect                                                                                                                                                                                                                                                                                              |
|     |         | 1     | Writing a 1 to this bit clears the RXRDY bit.                                                                                                                                                                                                                                                          |
| 5   | STALL   |       | Send Stall.                                                                                                                                                                                                                                                                                            |
|     |         | 0     | No effect                                                                                                                                                                                                                                                                                              |
|     |         | 1     | Terminates the current transaction and transmits the STALL handshake. This bit is cleared automatically after the STALL handshake is transmitted.                                                                                                                                                      |
| 4   | SETEND  |       | Setup end.                                                                                                                                                                                                                                                                                             |
|     |         | 0     | A control transaction has not ended or ended after the DATAEND bit was set.                                                                                                                                                                                                                            |
|     |         | 1     | A control transaction has ended before the DATAEND bit has been set. The EP0 bit in the USBTXIS register is also set in this situation. This bit is cleared by writing a 1 to the SETENDC bit.                                                                                                         |
| 3   | DATAEND |       | Data end.                                                                                                                                                                                                                                                                                              |
|     |         | 0     | No effect                                                                                                                                                                                                                                                                                              |
|     |         | 1     | Set this bit in the following situations: <ul style="list-style-type: none"> <li>When setting TXRDY for the last data packet</li> <li>When clearing RXRDY after unloading the last data packet</li> <li>When setting TXRDY for a zero-length data packet</li> </ul> This bit is cleared automatically. |
| 2   | STALLED |       | Endpoint Stalled. Software must clear this bit.                                                                                                                                                                                                                                                        |
|     |         | 0     | A STALL handshake has not been transmitted.                                                                                                                                                                                                                                                            |
|     |         | 1     | A STALL handshake has been transmitted.                                                                                                                                                                                                                                                                |

**Table 17-36. USB Control and Status Endpoint 0 Low Register (USBCSRL0) in Device Mode Field Descriptions (continued)**

| Bit | Field | Value | Description                                                                                                                                                           |
|-----|-------|-------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1   | TXRDY | 0     | Transmit Packet Ready. If both the TXRDY and SETUP bits are set, a setup packet is sent. If just TXRDY is set, an OUT packet is sent.<br>No transmit packet is ready. |
|     |       | 1     | Software sets this bit after loading an IN data packet into the TX FIFO. The EP0 bit in the USBTXIS register is also set in this situation.                           |
| 0   | RXRDY | 0     | Receive Packet Ready.<br>No receive packet has been received.                                                                                                         |
|     |       | 1     | A data packet has been received. The EP0 bit in the USBTXIS register is also set in this situation.<br>This bit is cleared by writing a 1 to the RXRDYC bit.          |

### 17.5.29 USB Control and Status Endpoint 0 High Register (USBCSRH0), offset 0x103

The USB control and status endpoint 0 high 8-bit register (USBCSRH0) provides control and status bits for endpoint 0.

**Mode(s):** Host Device

USBCSRH0 in Host mode is shown in [Figure 17-36](#) and described in [Table 17-37](#).

**Figure 17-36. USB Control and Status Endpoint 0 High Register (USBCSRH0) in Host Mode**

|          |   |       |       |       |
|----------|---|-------|-------|-------|
| 7        | 3 | 2     | 1     | 0     |
| Reserved |   | DTWE  | DT    | FLUSH |
| R-0      |   | R/W-0 | R/W-0 | R/W-0 |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 17-37. USB Control and Status Endpoint 0 High Register (USBCSRH0) in Host Mode Field Descriptions**

| Bit | Field    | Value | Description                                                                                                                                                                                                                                                                                                                          |
|-----|----------|-------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 7-3 | Reserved | 0     | Reserved                                                                                                                                                                                                                                                                                                                             |
| 2   | DTWE     | 0     | Data Toggle Write Enable. This bit is automatically cleared once the new value is written.<br>The DT bit cannot be written.                                                                                                                                                                                                          |
|     |          | 1     | Enables the current state of the endpoint 0 data toggle to be written (see DT bit).                                                                                                                                                                                                                                                  |
| 1   | DT       |       | Data Toggle. When read, this bit indicates the current state of the endpoint 0 data toggle.<br>If DTWE is set, this bit may be written with the required setting of the data toggle. If DTWE is Low, this bit cannot be written. Care should be taken when writing to this bit as it should only be changed to RESET USB endpoint 0. |
| 0   | FLUSH    | 0     | Flush FIFO. This bit is automatically cleared after the flush is performed.<br>No effect                                                                                                                                                                                                                                             |
|     |          | 1     | Flushes the next packet to be transmitted/read from the endpoint 0 FIFO. The FIFO pointer is reset and the TXRDY/RXRDY bit is cleared.<br><b>Note:</b> This bit should only be set when TXRDY/RXRDY is set. At other times, it may cause data to be corrupted.                                                                       |

USBCSRH0 in Device mode is shown in [Figure 17-37](#) and described in [Table 17-38](#).

**Figure 17-37. USB Control and Status Endpoint 0 High Register (USBCSRH0) in Device Mode**

7 1 0

|          |       |
|----------|-------|
| Reserved | FLUSH |
| R-0      | R/W-0 |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 17-38. USB Control and Status Endpoint 0 High Register (USBCSRH0) in Device Mode Field Descriptions**

| Bit | Field    | Value | Description                                                                                                                            |
|-----|----------|-------|----------------------------------------------------------------------------------------------------------------------------------------|
| 7-1 | Reserved | 0     | Reserved                                                                                                                               |
| 0   | FLUSH    | 0     | Flush FIFO. This bit is automatically cleared after the flush is performed.                                                            |
|     |          | 0     | No effect                                                                                                                              |
|     |          | 1     | Flushes the next packet to be transmitted/read from the endpoint 0 FIFO. The FIFO pointer is reset and the TXRDY/RXRDY bit is cleared. |
|     |          |       | <b>Note:</b> This bit should only be set when TXRDY/RXRDY is set. At other times, it may cause data to be corrupted.                   |

### 17.5.30 USB Receive Byte Count Endpoint 0 Register (USBCOUNT0), offset 0x108

The USB receive byte count endpoint 0 8-bit read-only register (USBCOUNT0) indicates the number of received data bytes in the endpoint 0 FIFO. The value returned changes as the contents of the FIFO change and is only valid while the RXRDY bit is set.

**Mode(s):** Host Device

USBCOUNT0 is shown in [Figure 17-38](#) and described in [Table 17-39](#).

**Figure 17-38. USB Receive Byte Count Endpoint 0 Register (USBCOUNT0)**

|          |       |   |
|----------|-------|---|
| 7        | 6     | 0 |
| Reserved | COUNT |   |
| R-0      | R/W-0 |   |

LEGEND: R/W = Read/Write; -n = value after reset

**Table 17-39. USB Receive Byte Count Endpoint 0 Register (USBCOUNT0) Field Descriptions**

| Bit | Field    | Value | Description                                                                                                     |
|-----|----------|-------|-----------------------------------------------------------------------------------------------------------------|
| 7   | Reserved | 0     | Reserved                                                                                                        |
| 6-0 | COUNT    | 0     | FIFO Count. COUNT is a read-only value that indicates the number of received data bytes in the endpoint 0 FIFO. |

### 17.5.31 USB Type Endpoint 0 Register (USBTYPE0), offset 0x10A

The USB type endpoint 0 8-bit register (USBTYPE0) must be written with the operating speed of the targeted Device being communicated with using endpoint 0.

**Mode(s):** Host

USBTYPE0 is shown in [Figure 17-39](#) and described in [Table 17-40](#).

**Figure 17-39. USB Type Endpoint 0 Register (USBTYPE0)**

|       |   |          |   |
|-------|---|----------|---|
| 7     | 6 | 5        | 0 |
| SPEED |   | Reserved |   |
| R/W-0 |   | R-0      |   |

LEGEND: R/W = Read/Write; -n = value after reset

**Table 17-40. USB Type Endpoint 0 Register (USBTYPE0) Field Descriptions**

| Bit | Field    | Value | Description                                                                                                                                                     |
|-----|----------|-------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 7-6 | SPEED    | 0     | Operating Speed specifies the operating speed of the target Device. If selected, the target is assumed to have the same connection speed as the USB controller. |
|     |          | 0-1h  | Reserved                                                                                                                                                        |
|     |          | 2h    | Full                                                                                                                                                            |
|     |          | 3h    | Low                                                                                                                                                             |
| 5-0 | Reserved | 0     | Reserved                                                                                                                                                        |

### 17.5.32 USB NAK Limit Register (USBNAKLMT), offset 0x10B

The USB NAK limit 8-bit read-only register (USBNAKLMT) sets the number of frames after which endpoint 0 should time out on receiving a stream of NAK responses. (Equivalent settings for other endpoints can be made through their USBTXINTERVAL[n] and USBRXINTERVAL[n] registers.)

The number of frames selected is  $2^{(m-1)}$  (where m is the value set in the register, with valid values of 2–16). If the Host receives NAK responses from the target for more frames than the number represented by the limit set in this register, the endpoint is halted.

**Note:** A value of 0 or 1 disables the NAK timeout function.

**Mode(s):** Host

USBNAKLMT is shown in [Figure 17-40](#) and described in [Table 17-41](#).

**Figure 17-40. USB NAK Limit Register (USBNAKLMT)**

|          |   |        |   |
|----------|---|--------|---|
| 7        | 5 | 4      | 0 |
| Reserved |   | NAKLMT |   |
| R-0      |   | R/W-0  |   |

LEGEND: R/W = Read/Write; -n = value after reset

**Table 17-41. USB NAK Limit Register (USBNAKLMT) Field Descriptions**

| Bit | Field    | Value | Description                                                                             |
|-----|----------|-------|-----------------------------------------------------------------------------------------|
| 7-5 | Reserved | 0     | Reserved                                                                                |
| 4-0 | NAKLMT   | 0     | EP0 NAK Limit specifies the number of frames after receiving a stream of NAK responses. |

### 17.5.33 USB Transmit Control and Status Endpoint n Low Register (USBTXCSRL[1]-USBTXCSRL[3])

The USB transmit control and status endpoint  $n$  low 8-bit registers (USBTXCSRL[ $n$ ]) provide control and status bits for transfers through the currently selected transmit endpoint.

For the specific offset for each register, see [Table 17-3](#).

| Mode(s): | Host | Device |
|----------|------|--------|
|----------|------|--------|

The USBTXCSRL[n] registers in Host Mode are shown in [Figure 17-41](#) and described in [Table 17-42](#).

**Figure 17-41. USB Transmit Control and Status Endpoint n Low Register (USBTXCSRL[n]) in Host Mode**

[illegible]

LEGEND: R/W = Read/Write; -n = value after reset

**Table 17-42. USB Transmit Control and Status Endpoint n Low Register (USBTXCSRL[n]) in Host Mode**

| Bit | Field   | Value  | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|-----|---------|--------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 7   | NAKTO   | 0<br>1 | NAK Timeout. Software must clear this bit to allow the endpoint to continue.<br>No timeout<br>Bulk endpoints only: Indicates that the transmit endpoint is halted following the receipt of NAK responses for longer than the time set by the NAKLMT field in the USBTXINTERVAL[n] register.                                                                                                                                                                                                                                                                                |
| 6   | CLRDT   | 0<br>1 | Clear DataToggle<br>No effect<br>Writing a 1 to this bit clears the DT bit in the USBTXCSRH[n] register.                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| 5   | STALLED | 0<br>1 | Endpoint Stalled. Software must clear this bit.<br>A STALL handshake has not been received<br>Indicates that a STALL handshake has been received. When this bit is set, any DMA request that is in progress is stopped, the FIFO is completely flushed, and the TXRDY bit is cleared.                                                                                                                                                                                                                                                                                      |
| 4   | SETUP   | 0<br>1 | Setup Packet.<br>No SETUP token is sent.<br>Sends a SETUP token instead of an OUT token for the transaction. This bit should be set at the same time as the TXRDY bit is set.<br><b>Note:</b> Setting this bit also clears the DT bit in the USBTXCSRH[n] register.                                                                                                                                                                                                                                                                                                        |
| 3   | FLUSH   | 0<br>1 | Flush FIFO. This bit can be set simultaneously with the TXRDY bit to abort the packet that is currently being loaded into the FIFO. Note that if the FIFO is double-buffered, FLUSH may have to be set twice to completely clear the FIFO.<br>No effect<br>Flushes the latest packet from the endpoint transmit FIFO. The FIFO pointer is reset and the TXRDY bit is cleared. The EPn bit in the USBTXIS register is also set in this situation.<br><b>Note:</b> This bit should only be set when the TXRDY bit is set. At other times, it may cause data to be corrupted. |
| 2   | ERROR   | 0<br>1 | Error. Software must clear this bit.<br>No error<br>Three attempts have been made to send a packet and no handshake packet has been received. The TXRDY bit is cleared, the EPn bit in the USBTXIS register is set, and the FIFO is completely flushed in this situation.<br><b>Note:</b> This bit is valid only when the endpoint is operating in Bulk or Interrupt mode.                                                                                                                                                                                                 |

**Table 17-42. USB Transmit Control and Status Endpoint n Low Register (USBTXCSRL[n]) in Host Mode**  
**Field Descriptions (continued)**

| Bit | Field  | Value  | Description                                                                                                                                                                                                                                                                                                                                                                   |
|-----|--------|--------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1   | FIFONE | 0<br>1 | FIFO Not Empty<br>The FIFO is empty<br>At least one packet is in the transmit FIFO.                                                                                                                                                                                                                                                                                           |
| 0   | TXRDY  | 0<br>1 | Transmit Packet Ready.<br>This bit is cleared automatically when a data packet has been transmitted. The EPn bit in the USBTXIS register is also set at this point. TXRDY is also automatically cleared prior to loading a second packet into a double-buffered FIFO.<br>No transmit packet is ready.<br>Software sets this bit after loading a data packet into the TX FIFO. |

The USBTXCSRL[n] registers in Device Mode are shown in [Figure 17-42](#) and described in [Table 17-43](#).

**Figure 17-42. USB Transmit Control and Status Endpoint n Low Register (USBTXCSRL[n]) in Device Mode**

|          |       |         |       |       |       |        |       |
|----------|-------|---------|-------|-------|-------|--------|-------|
| 7        | 6     | 5       | 4     | 3     | 2     | 1      | 0     |
| Reserved | CLRDT | STALLED | STALL | FLUSH | UNDRN | FIFONE | TXRDY |
| R-0      | R/W-0 | R/W-0   | R/W-0 | R/W-0 | R/W-0 | R/W-0  | R/W-0 |

LEGEND: R/W = Read/Write; -n = value after reset

**Table 17-43. USB Transmit Control and Status Endpoint n Low Register (USBTXCSRL[n]) in Device Mode**  
**Field Descriptions**

| Bit | Field    | Value  | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
|-----|----------|--------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 7   | Reserved | 0      | Reserved                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| 6   | CLRDT    | 0<br>1 | Clear Data Toggle<br>No effect<br>Writing a 1 to this bit clears the DT bit in the USBTXCSRH[n] register.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| 5   | STALLED  | 0<br>1 | Endpoint Stalled. Software must clear this bit.<br>A STALL handshake has not been transmitted.<br>A STALL handshake has been transmitted. The FIFO is flushed and the TXRDY bit is cleared.                                                                                                                                                                                                                                                                                                                                                                                                                      |
| 4   | STALL    | 0<br>1 | Send Stall. Software clears this bit to terminate the STALL condition.<br>No effect<br>Issues a STALL handshake to an IN token.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| 3   | FLUSH    | 0<br>1 | Flush FIFO. This bit may be set simultaneously with the TXRDY bit to abort the packet that is currently being loaded into the FIFO. Note that if the FIFO is double-buffered, FLUSH may have to be set twice to completely clear the FIFO.<br><b>Note:</b> This bit should only be set when the TXRDY bit is set. At other times, it may cause data to be corrupted.<br>No effect<br>Flushes the latest packet from the endpoint transmit FIFO. The FIFO pointer is reset and the TXRDY bit is cleared. The EPn bit in the USBTXIS register is also set in this situation.<br>This bit is cleared automatically. |
| 2   | UNDRN    | 0<br>1 | Underrun. Software must clear this bit.<br>No underrun<br>An IN token has been received when TXRDY is not set.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |



**Table 17-43. USB Transmit Control and Status Endpoint n Low Register (USBTXCSRL[n]) in Device Mode**  
**Field Descriptions (continued)**

| Bit | Field  | Value  | Description                                                                                                                                                                                                                                                                                                                                                                                                                            |
|-----|--------|--------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1   | FIFONE | 0<br>1 | FIFO Not Empty<br>The FIFO is empty.<br>At least one packet is in the transmit FIFO.                                                                                                                                                                                                                                                                                                                                                   |
| 0   | TXRDY  | 0<br>1 | Transmit Packet Ready.<br>This bit is cleared automatically when a data packet has been transmitted. The EPn bit in the USBTXIS register is also set at this point. TXRDY is also automatically cleared prior to loading a second packet into a double-buffered FIFO.<br>No transmit packet is ready.<br>Software sets this bit after loading a data packet into the TX FIFO.<br>This bit is cleared by writing a 1 to the RXRDYC bit. |



**Table 17-44. USB Transmit Control and Status Endpoint n High Register (USBTXCSRH[n]) in Host Mode**  
**Field Descriptions (continued)**

| Bit | Field | Value | Description                                                                                                                                                                                                                                                                                                                                                           |
|-----|-------|-------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 0   | DT    |       | Data Toggle. When read, this bit indicates the current state of the transmit endpoint data toggle.<br><br>If DTWE is High, this bit can be written with the required setting of the data toggle. If DTWE is Low, any value written to this bit is ignored. Care should be taken when writing to this bit as it should only be changed to RESET the transmit endpoint. |

The USBTXCSRH[n] registers in Device Mode are shown in [Figure 17-44](#) and described in [Table 17-45](#).

**Figure 17-44. USB Transmit Control and Status Endpoint n High Register (USBTXCSRH[n]) in Device Mode**

|         |       |       |       |       |        |          |   |
|---------|-------|-------|-------|-------|--------|----------|---|
| 7       | 6     | 5     | 4     | 3     | 2      | 1        | 0 |
| AUTOSET | ISO   | MODE  | DMAEN | FDT   | DMAMOD | Reserved |   |
| R/W-0   | R/W-0 | R/W-0 | R/W-0 | R/W-0 | R/W-0  | R-0      |   |

LEGEND: R/W = Read/Write; -n = value after reset

**Table 17-45. USB Transmit Control and Status Endpoint n High Register (USBTXCSRH[n]) in Device Mode Field Descriptions**

| Bit | Field    | Value  | Description                                                                                                                                                                                                                                                                                                                                                                           |
|-----|----------|--------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 7   | AUTOSET  | 0<br>1 | Auto Set<br>The TXRDY bit must be set manually.<br>Enables the TXRDY bit to be automatically set when data of the maximum packet size (value in USBTXMAXP[n]) is loaded into the transmit FIFO. If a packet of less than the maximum packet size is loaded, then the TXRDY bit must be set manually.                                                                                  |
| 6   | Reserved |        | Reserved. Should always have a value of 0.                                                                                                                                                                                                                                                                                                                                            |
| 5   | MODE     | 0<br>1 | Mode<br><b>Note:</b> This bit only has an effect when the same endpoint FIFO is used for both transmit and receive transactions.<br>Enables the endpoint direction as RX.<br>Enables the endpoint direction as TX.                                                                                                                                                                    |
| 4   | DMAEN    | 0<br>1 | DMA Request Enable<br><b>Note:</b> Three TX and three /RX endpoints can be connected to the DMA module. If this bit is set for a particular endpoint, the DMAATX, DMABTX, or DMACTX field in the USB DMA Select (USBDMASEL) register must be programmed correspondingly.<br>Disables the DMA request for the transmit endpoint.<br>Enables the DMA request for the transmit endpoint. |
| 3   | FDT      | 0<br>1 | Force Data Toggle<br>No effect<br>Forces the endpoint DT bit to switch and the data packet to be cleared from the FIFO, regardless of whether an ACK was received.                                                                                                                                                                                                                    |
| 2   | DMAMOD   | 0<br>1 | DMA Request Mode<br><b>Note:</b> This bit must not be cleared either before or in the same cycle as the above DMAEN bit is cleared.<br>An interrupt is generated after every DMA packet transfer.<br>An interrupt is generated only after the entire DMA transfer is complete.                                                                                                        |
| 0   | Reserved | 0      | Reserved                                                                                                                                                                                                                                                                                                                                                                              |

### 17.5.35 USB Maximum Receive Data Endpoint $n$ Registers (USBXMAXP[1]-USBXMAXP[3])

The USB maximum receive data endpoint  $n$  16-bit registers (USBXMAXP[ $n$ ]) define the maximum amount of data that can be transferred through the selected receive endpoint in a single operation.

Bits 10:0 define (in bytes) the maximum payload transmitted in a single transaction. The value set can be up to 1024 bytes but is subject to the constraints placed by the USB Specification on packet sizes for bulk and interrupt transfers in full-speed operation.

The total amount of data represented by the value written to this register must not exceed the FIFO size for the transmit endpoint, and must not exceed half the FIFO size if double-buffering is required.

**Note:** USBXMAXP[ $n$ ] must be set to an even number of bytes for proper interrupt generation in DMA Basic Mode.

For the specific offset for each register, see [Table 17-3](#).

**Mode(s):**      Host                                  Device

The USBXMAXP[ $n$ ] registers are shown in [Figure 17-45](#) and described in [Table 17-46](#).

**Figure 17-45. USB Maximum Receive Data Endpoint  $n$  Registers (USBXMAXP[ $n$ ])**

|          |    |         |   |
|----------|----|---------|---|
| 15       | 11 | 10      | 0 |
| Reserved |    | MAXLOAD |   |
| R-0      |    | R/W-000 |   |

LEGEND: R/W = Read/Write; R = Read only; - $n$  = value after reset

**Table 17-46. USB Maximum Receive Data Endpoint  $n$  Registers (USBXMAXP[ $n$ ]) Field Descriptions**

| Bit   | Field    | Value | Description                                                             |
|-------|----------|-------|-------------------------------------------------------------------------|
| 15-11 | Reserved | 0     | Reserved                                                                |
| 10-0  | MAXLOAD  |       | Maximum Payload specifies the maximum payload in bytes per transaction. |

### 17.5.36 USB Receive Control and Status Endpoint $n$ Low Register (USBXCSRL[1]-USBXCSRL[3])

The USB receive control and status endpoint  $n$  low 8-bit register (USBCSRL[ $n$ ]) provides control and status bits for transfers through the currently selected receive endpoint.

For the specific offset for each register, see [Table 17-3](#).

**Mode(s):** Host Device

The USBCSRL[ $n$ ] registers in Host mode are shown in [Figure 17-46](#) and described in [Table 17-47](#).

**Figure 17-46. USB Receive Control and Status Endpoint  $n$  Low Register (USBCSRL[ $n$ ]) in Host Mode**

| 7     | 6       | 5      | 4     | 3               | 2     | 1     | 0     |
|-------|---------|--------|-------|-----------------|-------|-------|-------|
| CLRDT | STALLED | REQPKT | FLUSH | DATAERR / NAKTO | ERROR | FULL  | RXRDY |
| W1C-0 | R/W-0   | R/W-0  | R/W-0 | R/W-0           | R/W-0 | R/W-0 | R/W-0 |

LEGEND: R/W = Read/Write; - $n$  = value after reset

**Table 17-47. USB Control and Status Endpoint  $n$  Low Register(USBCSRL[ $n$ ]) in Host Mode Field Descriptions**

| Bit | Field           | Value  | Description                                                                                                                                                                                                                                                                                                                                                                   |
|-----|-----------------|--------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 7   | NAKTO           | 0<br>1 | Clear Data Toggle.<br>No effect<br>Writing a 1 to this bit clears the DT bit in the USBXCSRH[ $n$ ] register.                                                                                                                                                                                                                                                                 |
| 6   | STALLED         | 0<br>1 | Endpoint Stalled. Software must clear this bit.<br>No handshake has been received.<br>A STALL handshake has been received. The EP $n$ bit in the USBRXIS register is also set.                                                                                                                                                                                                |
| 5   | REQPKT          | 0<br>1 | Request Packet. This bit is cleared when the RXRDY bit is set.<br>No request<br>Requests an IN transaction.                                                                                                                                                                                                                                                                   |
| 4   | FLUSH           | 0<br>1 | Flush FIFO. If the FIFO is double-buffered, FLUSH may have to be set twice to completely clear the FIFO.<br><b>Note:</b> This bit should only be set when the RXRDY bit is set. At other times, it may cause data to be corrupted.<br>No effect<br>Flushes the next packet to be read from the endpoint receive FIFO. The FIFO pointer is reset and the RXRDY bit is cleared. |
| 3   | DATAERR / NAKTO | 0<br>1 | Data Error / NAK Timeout<br>Normal operation<br>Bulk endpoints only: Indicates that the receive endpoint is halted following the receipt of NAK responses for longer than the time set by the NAKLMT field in the USBRXINTERVAL[ $n$ ] register. Software must clear this bit to allow the endpoint to continue.                                                              |
| 2   | ERROR           | 0<br>1 | Error. Software must clear this bit.<br><b>Note:</b> This bit is only valid when the receive endpoint is operating in Bulk or Interrupt mode.<br>No error<br>Three attempts have been made to receive a packet and no data packet has been received. The EP $n$ bit in the USBRXIS register is set in this situation.                                                         |
| 1   | FULL            | 0<br>1 | FIFO Full<br>The receive FIFO is not full.<br>No more packets can be loaded into the receive FIFO.                                                                                                                                                                                                                                                                            |

**Table 17-47. USB Control and Status Endpoint  $n$  Low Register(USBCSRL[ $n$ ]) in Host Mode Field Descriptions (continued)**

| Bit | Field | Value | Description                                                                                                                                                                                                                                                                                                                                                                                                        |
|-----|-------|-------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 0   | RXRDY |       | Receive Packet Ready.<br>If the AUTOCLR bit in the USBRXCSRH[ $n$ ] register is set, then the this bit is automatically cleared when a packet of USBRXMAXP[ $n$ ] bytes has been unloaded from the receive FIFO. If the AUTOCLR bit is clear, or if packets of less than the maximum packet size are unloaded, then software must clear this bit manually when the packet has been unloaded from the receive FIFO. |
|     |       | 0     | No data packet has been received.                                                                                                                                                                                                                                                                                                                                                                                  |
|     |       | 1     | Indicates that a data packet has been received. The EP $n$ bit in the USBTXIS register is also set in this situation.                                                                                                                                                                                                                                                                                              |

USBCSRL0 in Device mode is shown in [Figure 17-47](#) and described in [Table 17-48](#).

**Figure 17-47. USB Control and Status Endpoint  $n$  Low Register (USBCSRL[ $n$ ]) in Device Mode**

|       |         |       |       |         |       |       |       |
|-------|---------|-------|-------|---------|-------|-------|-------|
| 7     | 6       | 5     | 4     | 3       | 2     | 1     | 0     |
| CLRDT | STALLED | STALL | FLUSH | DATAERR | OVER  | FULL  | RXRDY |
| W1C-0 | W1C-0   | R/W-0 | R-0   | R/W-0   | R/W-0 | R/W-0 | R-0   |

LEGEND: R/W = Read/Write; - $n$  = value after reset

**Table 17-48. USB Control and Status Endpoint 0 Low Register(USBCSRL[ $n$ ]) in Device Mode Field Descriptions**

| Bit | Field    | Value | Description                                                                                                                                                                                                                                                                   |
|-----|----------|-------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 7   | CLRDT    |       | Clear Data Toggle                                                                                                                                                                                                                                                             |
|     |          | 0     | No effect                                                                                                                                                                                                                                                                     |
|     |          | 1     | Writing a 1 to this bit clears the DT bit in the USBRXCSRH[ $n$ ] register.                                                                                                                                                                                                   |
| 6   | STALLED  |       | Endpoint Stalled. Software must clear this bit.                                                                                                                                                                                                                               |
|     |          | 0     | A STALL handshake has been transmitted.                                                                                                                                                                                                                                       |
|     |          | 1     | A STALL handshake has been transmitted.                                                                                                                                                                                                                                       |
| 5   | STALL    |       | Send Stall. Software must clear this bit to terminate the STALL condition.                                                                                                                                                                                                    |
|     |          | 0     | No effect                                                                                                                                                                                                                                                                     |
|     |          | 1     | Issues a STALL handshake.                                                                                                                                                                                                                                                     |
| 4   | FLUSH    |       | Flush FIFO. The CPU writes a 1 to this bit to flush the next packet to be read from the endpoint receive FIFO. The FIFO pointer is reset and the RXRDY bit is cleared. Note that if the FIFO is double-buffered, FLUSH may have to be set twice to completely clear the FIFO. |
|     |          | 0     | No effect                                                                                                                                                                                                                                                                     |
|     |          | 1     | Flushes the next packet from the endpoint receive FIFO. The FIFO pointer is reset and the RXRDY bit is cleared.                                                                                                                                                               |
|     |          |       | <b>Note:</b> This bit should only be set when the RXRDY bit is set. At other times, it may cause data to be corrupted.                                                                                                                                                        |
| 3   | Reserved |       | Reserved                                                                                                                                                                                                                                                                      |
| 2   | Reserved |       | Reserved                                                                                                                                                                                                                                                                      |
| 1   | FULL     |       | FIFO Full                                                                                                                                                                                                                                                                     |
|     |          | 0     | The receive FIFO is not full.                                                                                                                                                                                                                                                 |
|     |          | 1     | No more packets can be loaded into the receive FIFO.                                                                                                                                                                                                                          |

**Table 17-48. USB Control and Status Endpoint 0 Low Register(USBCSRL[n]) in Device Mode Field Descriptions (continued)**

| Bit | Field | Value | Description                                                                                                                                                                                                                                                                                                                                                                                                |
|-----|-------|-------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 0   | RXRDY |       | Receive Packet Ready.<br>If the AUTOCLR bit in the USBRXCSRH[n] register is set, then the this bit is automatically cleared when a packet of USBRXMAXP[n] bytes has been unloaded from the receive FIFO. If the AUTOCLR bit is clear, or if packets of less than the maximum packet size are unloaded, then software must clear this bit manually when the packet has been unloaded from the receive FIFO. |
|     |       | 0     | No data packet has been received.                                                                                                                                                                                                                                                                                                                                                                          |
|     |       | 1     | A data packet has been received. The EPn bit in the USBTXIS register is also set in this situation.                                                                                                                                                                                                                                                                                                        |
|     |       |       | This bit is cleared by writing a 1 to the RXRDYC bit.                                                                                                                                                                                                                                                                                                                                                      |





The USBCSRH[n] registers in Device mode are shown in [Figure 17-49](#) and described in [Table 17-50](#).

**Figure 17-49. USB Control and Status Endpoint *n* High Register (USBCSRH[n]) in Device Mode**

|        |       |       |                     |        |   |          |   |
|--------|-------|-------|---------------------|--------|---|----------|---|
| 7      | 6     | 5     | 4                   | 3      | 2 | 1        | 0 |
| AUTOCL | ISO   | DMAEN | DISNYET /<br>PIDERR | DMAMOD |   | Reserved |   |
| R/W-0  | R/W-0 | R/W-0 | R/W-0               | R/W-0  |   | R-0      |   |

LEGEND: R/W = Read/Write; -*n* = value after reset

**Table 17-50. USB Control and Status Endpoint 0 High Register(USBCSRH[n]) in Device Mode Field Descriptions**

| Bit | Field              | Value  | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|-----|--------------------|--------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 7   | AUTOCL             | 0<br>1 | Auto Clear<br>No effect<br>Enables the RXRDY bit to be automatically cleared when a packet of USBRXMAXP[n] bytes has been unloaded from the receive FIFO. When packets of less than the maximum packet size are unloaded, RXRDY must be cleared manually. Care must be taken when using DMA to unload the receive FIFO as data is read from the receive FIFO in 4-byte chunks regardless of the value of the MAXLOAD field in the USBRXMAXP[n] register, see <a href="#">Section 17.2.3</a> . |
| 6   | Reserved           |        | Reserved                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| 5   | DMAEN              | 0<br>1 | DMA Request Enable<br><b>Note:</b> Three TX and three RX endpoints can be connected to the DMA module. If this bit is set for a particular endpoint, the DMAARX, DMABRX, or DMACRX field in the USB DMA Select (USBDMASEL) register must be programmed correspondingly.<br>Disables the DMA request for the receive endpoint.<br>Enables the DMA request for the receive endpoint.                                                                                                            |
| 4   | DISNYET/<br>PIDERR | 0<br>1 | Disable NYET / PID Error<br>No effect<br>For bulk or interrupt transactions: Disables the sending of NYET handshakes. When this bit is set, all successfully received packets are acknowledged, including at the point at which the FIFO becomes full.                                                                                                                                                                                                                                        |
| 3   | DMAMOD             | 0<br>1 | DMA Request Mode<br><b>Note:</b> This bit must not be cleared either before or in the same cycle as the above DMAEN bit is cleared.<br>An interrupt is generated after every DMA packet transfer.<br>An interrupt is generated only after the entire DMA transfer is complete.                                                                                                                                                                                                                |
| 0   | Reserved           | 0      | Reserved                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |

### 17.5.38 USB Receive Byte Count Endpoint $n$ Registers (USBRXCOUNT[1]-USBRXCOUNT[3])

The USB receive byte count endpoint  $n$  16-bit read-only registers hold the number of data bytes in the packet currently in line to be read from the receive FIFO. If the packet is transmitted as multiple bulk packets, the number given is for the combined packet.

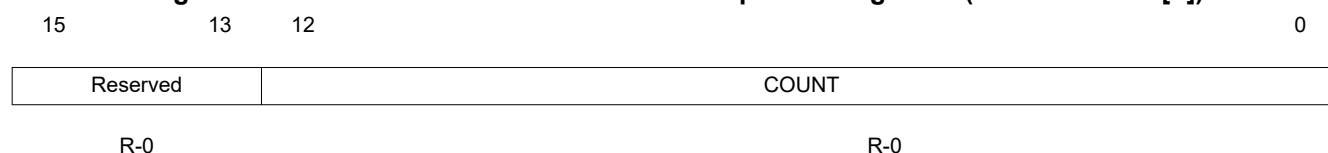
**Note:** The value returned changes as the FIFO is unloaded and is only valid while the RXRDY bit in the USBRXCSRL $n$  register is set.

For the specific offset for each register, see [Table 17-3](#).

**Mode(s):**      Host                                  Device

The USBRXCOUNT[ $n$ ] registers are shown in [Figure 17-50](#) and described in [Table 17-51](#).

**Figure 17-50. USB Maximum Receive Data Endpoint  $n$  Registers (USBRXCOUNT[ $n$ ])**



LEGEND: R/W = Read/Write; R = Read only; - $n$  = value after reset

**Table 17-51. USB Maximum Receive Data Endpoint  $n$  Registers (USBRXCOUNT[ $n$ ]) Field Descriptions**

| Bit   | Field    | Value | Description                                                               |
|-------|----------|-------|---------------------------------------------------------------------------|
| 15-13 | Reserved | 0     | Reserved                                                                  |
| 12-0  | COUNT    |       | Receive Packet Count indicates the number of bytes in the receive packet. |

### 17.5.39 USB Host Transmit Configure Type Endpoint n Register (USBTXTYPE[1]-USBTXTYPE[3])

The USB host transmit configure type endpoint  $n$  8-bit registers (USBTXTYPE[ $n$ ]) must be written with the endpoint number to be targeted by the endpoint, the transaction protocol to use for the currently selected transmit endpoint, and its operating speed.

For the specific offset for each register, see [Table 17-3](#).

**Mode(s):** Host

The USBTXTYPE[ $n$ ] registers are shown in [Figure 17-51](#) and described in [Table 17-52](#).

**Figure 17-51. USB Host Transmit Configure Type Endpoint n Register (USBTXTYPE[ $n$ ])**

|       |   |       |   |       |   |
|-------|---|-------|---|-------|---|
| 7     | 6 | 5     | 4 | 3     | 0 |
| SPEED |   | PROTO |   | TEP   |   |
| R/W-0 |   | R/W-0 |   | R/W-0 |   |

LEGEND: R/W = Read/Write; - $n$  = value after reset

**Table 17-52. USB Host Transmit Configure Type Endpoint n Register(USBTXTYPE[ $n$ ]) Field Descriptions**

| Bit | Field | Value | Description                                                                                                                                                                               |
|-----|-------|-------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 7-6 | SPEED |       | Operating Speed. This bit field specifies the operating speed of the target Device:                                                                                                       |
|     |       | 0h    | Default. The target is assumed to be using the same connection speed as the USB controller.                                                                                               |
|     |       | 1h    | Reserved                                                                                                                                                                                  |
|     |       | 2h    | Full                                                                                                                                                                                      |
|     |       | 3h    | Low                                                                                                                                                                                       |
| 5-4 | PROTO |       | Protocol. Software must configure this bit field to select the required protocol for the transmit endpoint:                                                                               |
|     |       | 0h    | Control                                                                                                                                                                                   |
|     |       | 1h    | Reserved                                                                                                                                                                                  |
|     |       | 2h    | Bulk                                                                                                                                                                                      |
|     |       | 3h    | Interrupt                                                                                                                                                                                 |
| 3-0 | TEP   | 0     | Target Endpoint Number. Software must configure this value to the endpoint number contained in the transmit endpoint descriptor returned to the USB controller during Device enumeration. |

### 17.5.40 USB Host Transmit Interval Endpoint n Register (USBTXINTERVAL[1]USBTXINTERVAL[3])

The USB host transmit interval endpoint  $n$  8-bit registers (USBTXINTERVAL[ $n$ ]), for interrupt transfers, define the polling interval for the currently selected transmit endpoint. For bulk endpoints, this register defines the number of frames after which the endpoint should time out on receiving a stream of NAK responses.

The USBTXINTERVAL[ $n$ ] registers values define a number of frames, as follows:

**Table 17-53. USBTXINTERVAL[ $n$ ] Frame Numbers**

| Transfer Type | Speed                   | Valid Values (m) | Interpretation                                                                            |
|---------------|-------------------------|------------------|-------------------------------------------------------------------------------------------|
| Interrupt     | Low-speed or Full-speed | 0x01-0xFF        | The polling interval is $m$ frames.                                                       |
| Bulk          | Full-speed              | 0x02-0x10        | The NAK Limit is $2^{(m-1)}$ frames. A value of 0 or 1 disables the NAK timeout function. |

For the specific offset for each register, see [Table 17-3](#).

**Mode(s):** Host

The USBTXINTERVAL[ $n$ ] registers are shown in [Figure 17-51](#) and described in [Table 17-52](#).

**Figure 17-52. USB Host Transmit Interval Endpoint n Register (USBTXINTERVAL[ $n$ ])**

7

0

|                 |
|-----------------|
| TXPOLL / NAKLMT |
|-----------------|

R/W-0

LEGEND: R/W = Read/Write; - $n$  = value after reset

**Table 17-54. USB Host Transmit Interval Endpoint n Register(USBTXINTERVAL[ $n$ ]) Field Descriptions**

| Bit | Field           | Value | Description                                                                                                                                                                          |
|-----|-----------------|-------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 7-0 | TXPOLL / NAKLMT | 0     | TX Polling / NAK Limit The polling interval for interrupt transfers; the NAK limit for bulk transfers. See <a href="#">Table 17-53</a> for valid entries; other values are reserved. |

### 17.5.41 USB Host Configure Receive Type Endpoint n Register (USBRTYPE[1]-USBRTYPE[3])

The USB host configure receive type endpoint  $n$  8-bit registers (USBRTYPE[ $n$ ]) must be written with the endpoint number to be targeted by the endpoint, the transaction protocol to use for the currently selected receive endpoint, and its operating speed.

For the specific offset for each register, see [Table 17-3](#).

**Mode(s):** Host

The USBRTYPE[ $n$ ] registers are shown in [Figure 17-53](#) and described in [Table 17-55](#).

**Figure 17-53. USB Host Configure Receive Type Endpoint n Register (USBRTYPE[ $n$ ])**

|       |   |       |   |       |   |
|-------|---|-------|---|-------|---|
| 7     | 6 | 5     | 4 | 3     | 0 |
| SPEED |   | PROTO |   | TEP   |   |
| R/W-0 |   | R/W-0 |   | R/W-0 |   |

LEGEND: R/W = Read/Write; - $n$  = value after reset

**Table 17-55. USB Host Configure Receive Type Endpoint n Register (USBRTYPE[ $n$ ]) Field Descriptions**

| Bit | Field | Value | Description                                                                                                                                                                               |
|-----|-------|-------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 7-6 | SPEED | 0h    | Operating Speed. This bit field specifies the operating speed of the target Device:<br>Default. The target is assumed to be using the same connection speed as the USB controller.        |
|     |       | 1h    | Reserved                                                                                                                                                                                  |
|     |       | 2h    | Full                                                                                                                                                                                      |
|     |       | 3h    | Low                                                                                                                                                                                       |
| 5-4 | PROTO | 0h    | Protocol. Software must configure this bit field to select the required protocol for the receive endpoint:<br>Control                                                                     |
|     |       | 1h    | Reserved                                                                                                                                                                                  |
|     |       | 2h    | Bulk                                                                                                                                                                                      |
|     |       | 3h    | Interrupt                                                                                                                                                                                 |
| 3-0 | TEP   | 0     | Target Endpoint Number. Software must configure this value to the endpoint number contained in the transmit endpoint descriptor returned to the USB controller during Device enumeration. |

### 17.5.42 USB Host Receive Polling Interval Endpoint $n$ Register (USBXINTERVAL[1]-USBXINTERVAL[3])

The USB host receive polling interval endpoint  $n$  8-bit registers (USBXINTERVAL[ $n$ ]), for interrupt transfers, define the polling interval for the currently selected transmit endpoint. For bulk endpoints, this register defines the number of frames after which the endpoint should time out on receiving a stream of NAK responses.

The USBXINTERVAL[ $n$ ] registers values define a number of frames, as follows:

**Table 17-56. USBXINTERVAL[ $n$ ] Frame Numbers**

| Transfer Type | Speed                   | Valid Values ( $m$ ) | Interpretation                                                                            |
|---------------|-------------------------|----------------------|-------------------------------------------------------------------------------------------|
| Interrupt     | Low-speed or Full-speed | 0x01-0xFF            | The polling interval is $m$ frames.                                                       |
| Bulk          | Full-speed              | 0x02-0x10            | The NAK Limit is $2^{(m-1)}$ frames. A value of 0 or 1 disables the NAK timeout function. |

For the specific offset for each register, see [Table 17-3](#).

**Mode(s):** Host

The USBXINTERVAL[ $n$ ] registers are shown in [Figure 17-51](#) and described in [Table 17-52](#).

**Figure 17-54. USB Host Receive Polling Interval Endpoint  $n$  Register (USBXINTERVAL[ $n$ ])**

7 0

|                 |
|-----------------|
| TXPOLL / NAKLMT |
| R/W-0           |

LEGEND: R/W = Read/Write; - $n$  = value after reset

**Table 17-57. USB Host Receive Polling Interval Endpoint  $n$  Register (USBXINTERVAL[ $n$ ]) Field Descriptions**

| Bit | Field           | Value | Description                                                                                                                                                                          |
|-----|-----------------|-------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 7-0 | TXPOLL / NAKLMT | 0     | TX Polling / NAK Limit The polling interval for interrupt transfers; the NAK limit for bulk transfers. See <a href="#">Table 17-56</a> for valid entries; other values are reserved. |

### 17.5.43 USB Request Packet Count in Block Transfer Endpoint n Registers (USBQPKTCOUNT[1]-USBQPKTCOUNT[3])

The USB receive packet count in block transfer endpoint  $n$  16-bit read/writer registers are used in Host mode to specify the number of packets that are to be transferred in a block transfer of one or more bulk packets to receive endpoint  $n$ . The USB controller uses the value recorded in this register to determine the number of requests to issue where the AUTORQ bit in the USBRCSR[n] register has been set. For more information about IN transactions as a host, see [Section 17.2.2.2](#).

**Note:** Multiple packets combined into a single bulk packet within the FIFO count as one packet.

For the specific offset for each register, see [Table 17-3](#).

**Mode(s):** Host

The USBRQPKTCOUNT[*n*] registers are shown in [Figure 17-55](#) and described in [Table 17-58](#).

**Figure 17-55. USB Request Packet Count in Block Transfer Endpoint *n* Registers (USBRQPKTCOUNT[*n*])**

[illegible]

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 17-58. USB Request Packet Count in Block Transfer Endpoint *n* Registers (USB\_RQPKTCOUNT[n])**

| Bit   | Field    | Value | Description                                                                                                                                                                                                                                                                            |
|-------|----------|-------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-13 | Reserved | 0     | Reserved                                                                                                                                                                                                                                                                               |
| 12-0  | COUNT    |       | Block Transfer Packet Count sets the number of packets of the size defined by the MAXLOAD bit field that are to be transferred in a block transfer.<br><b>Note:</b> This is only used in Host mode when AUTORQ is set. The bit has no effect in Device mode or when AUTORQ is not set. |







### 17.5.46 USB External Power Control Register (USBEPIC), offset 0x400

The USB external power control 32-bit register (USBEPIC) specifies the function of the two-pin external power interface (USB0EPEN and USB0PFLT). The assertion of the power fault input may generate an automatic action, as controlled by the hardware configuration registers. The automatic action is necessary because the fault condition may require a response faster than one provided by firmware.

**Mode(s):** Host Device

USBEPIC is shown in [Figure 17-58](#) and described in [Table 17-61](#).

**Figure 17-58. USB External Power Control Register (USBEPIC)**

|          |         |         |        |          |        |       |   |         |   |   |
|----------|---------|---------|--------|----------|--------|-------|---|---------|---|---|
| 31       |         |         |        |          |        |       |   | 10      | 9 | 8 |
| Reserved |         |         |        |          |        |       |   | PFLTACT |   |   |
| R-0      |         |         |        |          |        |       |   | R/W-0   |   |   |
| 7        | 6       | 5       | 4      | 3        | 2      | 1     | 0 |         |   |   |
| Reserved | PFLTAEN | PFLTSEN | PFLTEN | Reserved | EPENDE | EPEN  |   |         |   |   |
| R-0      | R/W-0   | R/W-0   | R/W-0  | R-0      | R/W-0  | R/W-0 |   |         |   |   |

LEGEND: R/W = Read/Write; -n = value after reset

**Table 17-61. USB External Power Control Register (USBEPIC) Field Descriptions**

| Bit   | Field    | Value                | Description                                                                                                                                                                                                                                                                                                                                                                                              |
|-------|----------|----------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 31-10 | Reserved | 0                    | Reserved                                                                                                                                                                                                                                                                                                                                                                                                 |
| 9-8   | PFLTACT  | 0h<br>1h<br>2h<br>3h | Power Fault Action. This bit field specifies how the USB0EPEN signal is changed when detecting a USB power fault.<br>Unchanged. USB0EPEN is controlled by the combination of the EPEN and EPENDE bits.<br>Tristate. USB0EPEN is undriven (tristate).<br>Low. USB0EPEN is driven Low.<br>High. USB0EPEN is driven High.                                                                                   |
| 7     | Reserved | 0                    | Reserved                                                                                                                                                                                                                                                                                                                                                                                                 |
| 6     | PFLTAEN  | 0<br>1               | Power Fault Action Enable. This bit specifies whether a USB power fault triggers any automatic corrective action regarding the driven state of the USB0EPEN signal.<br>Disabled. USB0EPEN is controlled by the combination of the EPEN and EPENDE bits.<br>Enabled. The USB0EPEN output is automatically changed to the state specified by the PFLTACT field.                                            |
| 5     | PFLTSEN  | 0<br>1               | Power Fault Sense. This bit specifies the logical sense of the USB0PFLT input signal that indicates an error condition. The complementary state is the inactive state.<br>Low Fault. If USB0PFLT is driven Low, the power fault is signaled internally (if enabled by the PFLTEN bit).<br>High Fault. If USB0PFLT is driven High, the power fault is signaled internally (if enabled by the PFLTEN bit). |
| 4     | PFLTEN   | 0<br>1               | Power Fault Input Enable. This bit specifies whether the USB0PFLT input signal is used in internal logic.<br>Not Used. The USB0PFLT signal is ignored.<br>Used. The USB0PFLT signal is used internally                                                                                                                                                                                                   |
| 3     | Reserved | 0                    | Reserved                                                                                                                                                                                                                                                                                                                                                                                                 |

**Table 17-61. USB External Power Control Register (USBEPD) Field Descriptions (continued)**

| Bit | Field  | Value | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|-----|--------|-------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 2   | EPENDE |       | <p>EPEN Drive Enable. This bit specifies whether the USB0EPEN signal is driven or undriven (tristate). When driven, the signal value is specified by the EPEN field. When not driven, the EPEN field is ignored and the USB0EPEN signal is placed in a high-impedance state.</p> <p>The USB0EPEN signal is undriven at reset because the sense of the external power supply enable is unknown. By adding the high-impedance state, system designers can bias the power supply enable to the disabled state using a large resistor (100 kΩ) and later configure and drive the output signal to enable the power supply.</p> |
|     |        | 0     | Not Driven. The USB0EPEN signal is high impedance.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|     |        | 1     | Driven. The USB0EPEN signal is driven to the logical value specified by the value of the EPEN field.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| 1-0 | EPEN   |       | <p>External Power Supply Enable Configuration. This bit field specifies and controls the logical value driven on the USB0EPEN signal.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|     |        | 0h    | Power Enable Active Low. The USB0EPEN signal is driven Low if the EPENDE bit is set.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|     |        | 1h    | Power Enable Active High. The USB0EPEN signal is driven High if the EPENDE bit is set.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|     |        | 2h    | Power Enable High if VBUS Low. The USB0EPEN signal is driven High when the A device is not recognized.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|     |        | 3h    | Power Enable High if VBUS High. The USB0EPEN signal is driven High when the A device is recognized.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |

### 17.5.47 USB External Power Control Raw Interrupt Status Register (USBEPKRIS), offset 0x404

The USB external power control raw interrupt status 32-bit register (USBEPKRIS) specifies the unmasked interrupt status of the two-pin external power interface.

**Mode(s):** Host Device

USBEPKRIS is shown in [Figure 17-59](#) and described in [Table 17-62](#).

**Figure 17-59. USB External Power Control Raw Interrupt Status Register (USBEPKRIS)**

|          |  |   |     |
|----------|--|---|-----|
| 31       |  | 1 | 0   |
| Reserved |  |   | PF  |
| R-0      |  |   | R-0 |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 17-62. USB External Power Control Raw Interrupt Status Register (USBEPKRIS) Field Descriptions**

| Bit  | Field    | Value | Description                                                                                                       |
|------|----------|-------|-------------------------------------------------------------------------------------------------------------------|
| 31-1 | Reserved | 0     | Reserved                                                                                                          |
| 0    | PF       | 0     | USB Power Fault Interrupt Status.<br>This bit is cleared by writing a 1 to the PF bit in the USBEPKRISC register. |
|      |          | 0     | A Power Fault status has been detected.                                                                           |
|      |          | 1     | An interrupt has not occurred.                                                                                    |

### 17.5.48 USB External Power Control Interrupt Mask Register (USBEPICM), offset 0x408

The USB external power control interrupt mask 32-bit register (USBEPICM) specifies the interrupt mask of the two-pin external power interface.

**Mode(s):** Host Device

USBEPICM is shown in [Figure 17-60](#) and described in [Table 17-63](#).

**Figure 17-60. USB External Power Control Interrupt Mask Register (USBEPICM)**

|          |  |   |     |
|----------|--|---|-----|
| 31       |  | 1 | 0   |
| Reserved |  |   | PF  |
| R-0      |  |   | R-0 |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 17-63. USB External Power Control Interrupt Mask Register (USBEPICM) Field Descriptions**

| Bit  | Field    | Value | Description                                                                                                                  |
|------|----------|-------|------------------------------------------------------------------------------------------------------------------------------|
| 31-1 | Reserved | 0     | Reserved                                                                                                                     |
| 0    | PF       | 0     | USB Power Fault Interrupt Mask.<br>The raw interrupt signal from a detected power fault is sent to the interrupt controller. |
|      |          | 1     | A detected power fault does not affect the interrupt status.                                                                 |

### 17.5.49 USB External Power Control Interrupt Status and Clear Register (USBEPICISC), offset 0x40C

The USB external power control interrupt status and clear 32-bit register (USBEPICISC) specifies the unmasked interrupt status of the two-pin external power interface.

**Mode(s):** Host Device

USBEPICISC is shown in [Figure 17-61](#) and described in [Table 17-64](#).

**Figure 17-61. USB External Power Control Interrupt Status and Clear Register (USBEPICISC)**

|          |   |     |
|----------|---|-----|
| 31       | 1 | 0   |
| Reserved |   | PF  |
| R-0      |   | R-0 |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 17-64. USB External Power Control Interrupt Status and Clear Register (USBEPICISC) Field Descriptions**

| Bit  | Field    | Value | Description                                                                                                                                                                                                                                                                    |
|------|----------|-------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 31-1 | Reserved | 0     | Reserved. Reset is 0x0000.000.                                                                                                                                                                                                                                                 |
| 0    | PF       | 0     | USB Power Fault Interrupt Status and Clear.<br>This bit is cleared by writing a 1. Clearing this bit also clears the PF bit in the USBEPICISC register.<br><br>The PF bits in the USBEPCIS and USBEPCIM registers are set, providing an interrupt to the interrupt controller. |
|      |          | 1     | No interrupt has occurred or the interrupt is masked.                                                                                                                                                                                                                          |



### 17.5.51 USB Device RESUME Raw Interrupt Mask Register (USBDRIM), offset 0x414

The USB device RESUME raw interrupt status register (USBDRIM) is the masked interrupt status register. On a read, this register gives the current masked status value of the corresponding interrupt. A write has no effect.

**Mode(s):**      Host                                  Device

USBDRIM is shown in [Figure 17-63](#) and described in [Table 17-66](#).

**Figure 17-63. USB Device RESUME Raw Interrupt Status Register (USBDRIS)**

|          |  |   |        |
|----------|--|---|--------|
| 31       |  | 1 | 0      |
| Reserved |  |   | RESUME |
| R-0      |  |   | R-0    |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 17-66. USB Device RESUME Raw Interrupt Status Register (USBDRIS) Field Descriptions**

| Bit  | Field    | Value | Description                                                                                                                                                                                                                |
|------|----------|-------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 31-1 | Reserved | 0     | Reserved. Reset is 0x0000.000.                                                                                                                                                                                             |
| 0    | PF       | 0     | RESUME Interrupt Mask<br>The raw interrupt signal from a detected RESUME is sent to the interrupt controller. This bit should only be set when a SUSPEND has been detected (the SUSPEND bit in the USBIS register is set). |
|      |          | 1     | A detected RESUME does not affect the interrupt status.                                                                                                                                                                    |



### 17.5.52 USB Device RESUME Interrupt Status and Clear Register (USBDRISC), offset 0x418

The USB device RESUME interrupt status and clear register (USBDRRIS) is the raw interrupt clear register. On a write of 1, the corresponding interrupt is cleared. A write of 0 has no effect.

|                 |      |        |
|-----------------|------|--------|
| <b>Mode(s):</b> | Host | Device |
|-----------------|------|--------|

USBDRISC is shown in [Figure 17-64](#) and described in [Table 17-67](#).

### Figure 17-64. USB Device RESUME Interrupt Status and Clear Register (USBDRISC)

|          |        |   |
|----------|--------|---|
| 31       | 1      | 0 |
| Reserved | RESUME |   |
| R-0      | R/W1C  |   |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 17-67. USB Device RESUME Interrupt Status and Clear Register (USBDRISC) Field Descriptions**

| Bit  | Field    | Value | Description                                                                                                                                       |
|------|----------|-------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| 31-1 | Reserved | 0     | Reserved. Reset is 0x0000.000.                                                                                                                    |
| 0    | RESUME   |       | RESUME Interrupt Status and Clear.<br>This bit is cleared by writing a 1. Clearing this bit also clears the RESUME bit in the USBDRCRIS register. |
|      |          | 0     | The RESUME bits in the USBDRRIS and USBDRCIM registers are set, providing an interrupt to the interrupt controller.                               |
|      |          | 1     | No interrupt has occurred or the interrupt is masked.                                                                                             |

### 17.5.53 USB General-Purpose Control and Status Register (USBGPCS), offset 0x41C

The USB general-purpose control and status register (USBGPCS) provides the state of the internal ID signal.

When the USB controller is used as either a dedicated Host or Device, the DEVMODOTG and DEVMOD bits in the USB General-Purpose Control and Status (USBGPCS) register should be used to connect the ID inputs to fixed levels internally. For proper self-powered Device operation, the VBUS value must be monitored to assure that if the Host removes VBUS, the self-powered Device disables the D+/D- pull-up resistors. This function can be accomplished by connecting a standard GPIO to VBUS.

**Mode(s):**      Host                                      Device

USBGPCS is shown in [Figure 17-65](#) and described in [Table 17-68](#).

**Figure 17-65. USB General-Purpose Control and Status Register (USBGPCS)**

|          |  |   |           |        |
|----------|--|---|-----------|--------|
| 31       |  | 2 | 1         | 0      |
| Reserved |  |   | DEVMODOTG | DEVMOD |
| R-0      |  |   | R/W-0     | R/W-0  |

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

**Table 17-68. USB General-Purpose Control and Status Register (USBGPCS) Field Descriptions**

| Bit  | Field     | Value  | Description                                                                                                                                                                                                                                            |
|------|-----------|--------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 31-2 | Reserved  | 0      | Reserved. Reset is 0x0000.000.                                                                                                                                                                                                                         |
| 1    | DEVMODOTG | 0<br>1 | Enable Device Mode. This bit enables the DEVMOD bit to control the state of the internal ID signal in OTG mode.<br>The mode is specified by the state of the internal ID signal.<br>This bit enables the DEVMOD bit to control the internal ID signal. |
| 0    | DEVMOD    | 0<br>1 | Device Mode This bit specifies the state of the internal ID signal in Host mode and in OTG mode when the DEVMODOTG bit is set.<br>In Device mode this bit is ignored (assumed set).<br>Host mode<br>Device mode                                        |



**Table 17-69. USB DMA Select Register (USBDMASEL) Field Descriptions (continued)**

| Bit | Field  | Value | Description                                                                                               |
|-----|--------|-------|-----------------------------------------------------------------------------------------------------------|
| 7-4 | DMAATX |       | DMA A TX Select specifies the TX mapping of the first USB endpoint on DMA channel 1 (primary assignment). |
|     |        | 0h    | Reserved                                                                                                  |
|     |        | 1h    | Endpoint 1 TX                                                                                             |
|     |        | 2h    | Endpoint 2 TX                                                                                             |
| 3-0 | DMAARX | 3h    | Endpoint 3 TX                                                                                             |
|     |        |       | DMA A RX Select specifies the RX mapping of the first USB endpoint on DMA channel 0 (primary assignment). |
|     |        | 0h    | Reserved                                                                                                  |
|     |        | 1h    | Endpoint 1 RX                                                                                             |
|     |        | 2h    | Endpoint 2 RX                                                                                             |
|     |        | 3h    | Endpoint 3 RX                                                                                             |

NOTE: Page numbers for previous revisions may differ from page numbers in the current version.

## Changes from November 4, 2019 to June 10, 2022 (from Revision H (November 2019) to Revision I (June 2022))

|                                                                         | Page |
|-------------------------------------------------------------------------|------|
| • <b>Chapter 1 System Control and Interrupts:</b> .....                 | 39   |
| • Added <a href="#">Section 1.2.6</a> .....                             | 61   |
| • Added <a href="#">Section 1.3.2.7.1</a> .....                         | 89   |
| • <b>Chapter 2 Boot ROM:</b> .....                                      | 195  |
| • <b>Chapter 3 Enhanced Pulse Width Modulator (ePWM):</b> .....         | 241  |
| • Changed <a href="#">Section 3.2.8.1</a> .....                         | 292  |
| • Added <a href="#">Section 3.2.8.2</a> .....                           | 294  |
| • Added <a href="#">Section 3.2.8.3</a> .....                           | 294  |
| • Changed <a href="#">Section 3.4.1</a> .....                           | 328  |
| • Changed <a href="#">Section 3.4.2</a> .....                           | 336  |
| • Changed <a href="#">Section 3.4.3</a> .....                           | 342  |
| • Changed <a href="#">Section 3.4.4</a> .....                           | 347  |
| • Changed <a href="#">Section 3.4.5</a> .....                           | 350  |
| • Changed <a href="#">Section 3.4.6</a> .....                           | 359  |
| • Changed <a href="#">Section 3.4.7</a> .....                           | 365  |
| • Changed <a href="#">Section 3.4.8</a> .....                           | 367  |
| • <b>Chapter 4 High-Resolution Pulse Width Modulator (HRPWM):</b> ..... | 377  |
| • Added <a href="#">Section 4.4.1</a> .....                             | 401  |
| • Added <a href="#">Section 4.4.2</a> .....                             | 402  |
| • Added <a href="#">Section 4.4.3</a> .....                             | 404  |
| • <b>Chapter 5 High Resolution Capture (HRCAP):</b> .....               | 405  |
| • Added <a href="#">Section 5.1.1</a> .....                             | 406  |
| • <b>Chapter 6 Enhanced Capture (eCAP):</b> .....                       | 429  |
| • <b>Chapter 7 Enhanced Quadrature Encoder Pulse (eQEP):</b> .....      | 461  |
| • Changed <a href="#">Table 7-2</a> .....                               | 469  |
| • <b>Chapter 8 Analog-to-Digital Converter (ADC):</b> .....             | 509  |
| • Changed <a href="#">Section 8.13.3</a> .....                          | 538  |
| • Changed <a href="#">Section 8.13.5</a> .....                          | 547  |
| • Changed <a href="#">Section 8.13.6</a> .....                          | 556  |
| • <b>Chapter 9 Comparator (COMP):</b> .....                             | 559  |
| • <b>Chapter 10 Control Law Accelerator (CLA):</b> .....                | 569  |
| • Deleted General Instructions table. Subsequent tables renumbered..... | 591  |
| • Changed order of sections in <a href="#">Section 10.7.3</a> .....     | 711  |
| • <b>Chapter 11 Direct Memory Access (DMA):</b> .....                   | 727  |
| • <b>Chapter 12 Serial Peripheral Interface (SPI):</b> .....            | 759  |
| • <b>Chapter 13 Serial Communications Interface (SCI):</b> .....        | 799  |
| • Changed <a href="#">Section 13.1</a> .....                            | 800  |
| • Added <a href="#">Section 13.1.1</a> .....                            | 800  |
| • Added <a href="#">Section 13.1.3</a> .....                            | 801  |
| • Changed <a href="#">Figure 13-10</a> .....                            | 811  |
| • <b>Chapter 14 Inter-Integrated Circuit Module (I2C):</b> .....        | 833  |

---

|                                                                                |      |
|--------------------------------------------------------------------------------|------|
| • Added <a href="#">Figure 14-6</a> .....                                      | 839  |
| • Added <a href="#">Figure 14-7</a> . Subsequent figures renumbered.....       | 839  |
| • Moved <a href="#">Section 14.3.10</a> .....                                  | 850  |
| • <a href="#">Chapter 15 Multichannel Buffered Serial Port (McBSP)</a> : ..... | 877  |
| • <a href="#">Chapter 16 Controller Area Network (CAN)</a> : .....             | 999  |
| • Changed last paragraph in <a href="#">Section 16.6.2</a> .....               | 1010 |
| • <a href="#">Chapter 17 Universal Serial Bus (USB) Controller</a> : .....     | 1055 |

---

## IMPORTANT NOTICE AND DISCLAIMER

TI PROVIDES TECHNICAL AND RELIABILITY DATA (INCLUDING DATA SHEETS), DESIGN RESOURCES (INCLUDING REFERENCE DESIGNS), APPLICATION OR OTHER DESIGN ADVICE, WEB TOOLS, SAFETY INFORMATION, AND OTHER RESOURCES "AS IS" AND WITH ALL FAULTS, AND DISCLAIMS ALL WARRANTIES, EXPRESS AND IMPLIED, INCLUDING WITHOUT LIMITATION ANY IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE OR NON-INFRINGEMENT OF THIRD PARTY INTELLECTUAL PROPERTY RIGHTS.

These resources are intended for skilled developers designing with TI products. You are solely responsible for (1) selecting the appropriate TI products for your application, (2) designing, validating and testing your application, and (3) ensuring your application meets applicable standards, and any other safety, security, regulatory or other requirements.

These resources are subject to change without notice. TI grants you permission to use these resources only for development of an application that uses the TI products described in the resource. Other reproduction and display of these resources is prohibited. No license is granted to any other TI intellectual property right or to any third party intellectual property right. TI disclaims responsibility for, and you will fully indemnify TI and its representatives against, any claims, damages, costs, losses, and liabilities arising out of your use of these resources.

TI's products are provided subject to [TI's Terms of Sale](#) or other applicable terms available either on [ti.com](#) or provided in conjunction with such TI products. TI's provision of these resources does not expand or otherwise alter TI's applicable warranties or warranty disclaimers for TI products.

TI objects to and rejects any additional or different terms you may have proposed.

Mailing Address: Texas Instruments, Post Office Box 655303, Dallas, Texas 75265  
Copyright © 2022, Texas Instruments Incorporated